



ns-3 Model Library

Release ns-3.48

ns-3 project

Jun 02, 2026

CONTENTS

1	Organization	3
2	Animation	5
3	Antenna	13
4	Ad Hoc On-Demand Distance Vector (AODV)	19
5	Applications	21
6	Bridge NetDevice	49
7	BRITE Integration	51
8	Buildings Module	55
9	Click Modular Router Integration	69
10	CSMA NetDevice	75
11	DSDV Routing	81
12	Dynamic Source Routing (DSR)	83
13	Emulation Overview	89
14	Energy Framework	113
15	Flow Monitor	125
16	Internet Models (IP, TCP, Routing, UDP)	131
17	Internet Applications Module Documentation	209
18	IEEE 802.15.4: Low-Rate Wireless Personal Area Network (LR-WPAN)	221
19	LTE Module	237
20	Wi-Fi Mesh Module Documentation	445
21	MPI for Distributed Simulation	451
22	Mobility	457

23 Network Module	481
24 Nix-Vector Routing Documentation	507
25 Optimized Link State Routing (OLSR)	513
26 OpenFlow switch support	517
27 PointToPoint NetDevice	521
28 Propagation	525
29 Spectrum Module	549
30 6LoWPAN: IPv6 over Low-Power Wireless Personal Area Networks	575
31 Topology Input Readers	585
32 Traffic Control Layer	587
33 UAN Framework	621
34 VirtualNetDevice	633
35 Wi-Fi Module	635
36 Zigbee	713
Bibliography	727
Index	735

This is the *ns-3 Model Library* documentation. Primary documentation for the ns-3 project is organized as follows:

- Several guides that are version controlled for each release (the [latest release](#)) and [development tree](#):
 - Tutorial
 - Installation Guide
 - Manual
 - Model Library (*this document*)
 - Contributing Guide
- [ns-3 Doxygen](#): Documentation of the public APIs of the simulator
- [ns-3 wiki](#)

This document is written in [reStructuredText](#) for [Sphinx](#) and is maintained in the `doc/models` directory of ns-3's source code (and much of the source content is also pulled from the `doc/` directory of each module. Source file column width is 100 columns.

ORGANIZATION

This manual compiles documentation for *ns-3* models and supporting software that enable users to construct network simulations. It is important to distinguish between **modules** and **models**:

- *ns-3* software is organized into separate *modules* that are each built as a separate software library. Individual *ns-3* programs can link the modules (libraries) they need to conduct their simulation.
- *ns-3 models* are abstract representations of real-world objects, protocols, devices, etc.

An *ns-3* module may consist of more than one model (for instance, the `internet` module contains models for both TCP and UDP). In general, *ns-3* models do not span multiple software modules, however.

This manual provides documentation about the models of *ns-3*. It complements two other sources of documentation concerning models:

- the model APIs are documented, from a programming perspective, using [Doxygen](#). Doxygen for *ns-3* models is available [on the project web server](#).
- the *ns-3* core is documented in the developer's manual. *ns-3* models make use of the facilities of the core, such as attributes, default values, random numbers, test frameworks, etc. Consult the [main web site](#) to find copies of the manual.

Finally, additional documentation about various aspects of *ns-3* may exist on the [project wiki](#).

A sample outline of how to write model library documentation can be found by executing the `create-module.py` program and looking at the template created in the file `new-module/doc/new-module.rst`.

```
$ cd src
$ ./create-module.py new-module
```

The remainder of this document is organized alphabetically by module name.

If you are new to *ns-3*, you might first want to read below about the network module, which contains some fundamental models for the simulator. The packet model, models for different address formats, and abstract base classes for objects such as nodes, net devices, channels, sockets, and applications are discussed there.

ANIMATION

Animation is an important tool for network simulation. While *ns-3* does not contain a default graphical animation tool, we currently have two ways to provide animation, namely using the PyViz method or the NetAnim method. The PyViz method is described in <http://www.nsnam.org/wiki/PyViz>.

We will describe the NetAnim method briefly here.

2.1 NetAnim

NetAnim is a standalone, Qt5-based software executable that uses a trace file generated during an *ns-3* simulation to display the topology and animate the packet flow between nodes.

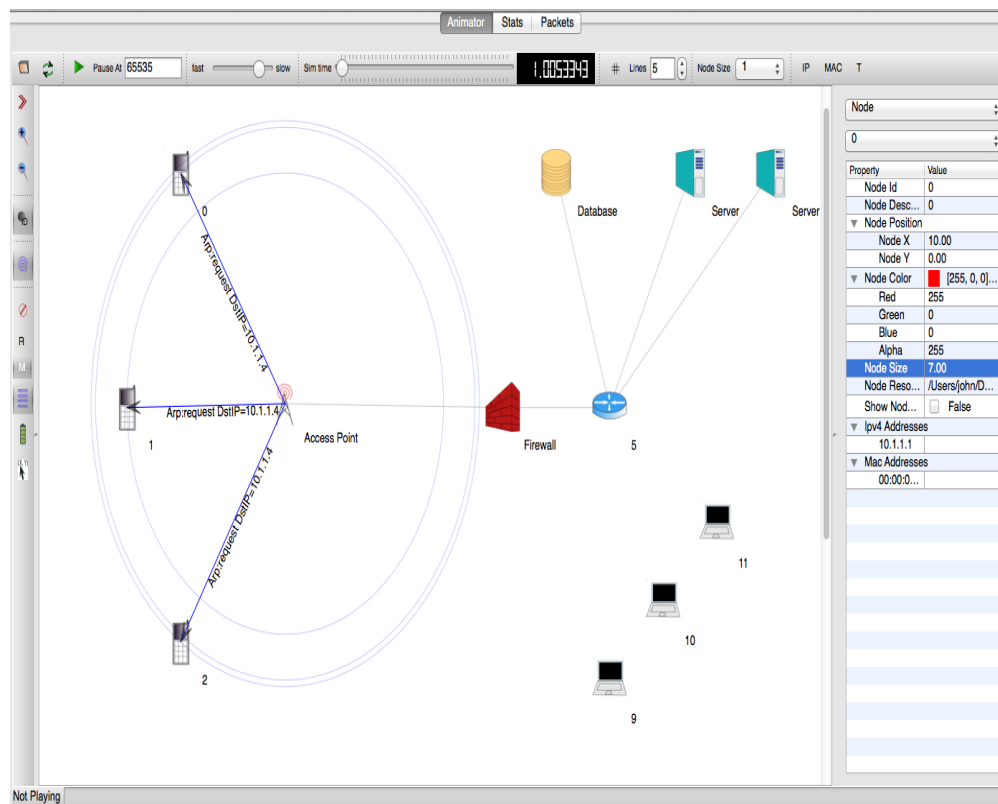


Fig. 2.1: An example of packet animation on wired-links

In addition, NetAnim also provides useful features such as tables to display meta-data of packets like the image below

Packet count	11208				
From Node Id	1				
To Node Id	0				
Transmission time >=	0				
	Apply filter				
	Select All				
	Deselect All				
☐ All Packets					
☐ Ethernet					
☐ Ppp					
☐ Wifi					
☐ Arp					
☐ Ipv4					
☐ Icmpv4					
☐ Udp					
☒ Tcp					
☐ Aodv					
☐ Olsr					
☐ Dsdv					

	Tx Time	From Node Id	To Node Id	Meta Info
1	0.0301008	1	0	TCP 50000 > 49153 SYN ACK Seq=0 Ack=1 Win=65535
2	0.0711264	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=537 Win=65535
3	0.112581	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=1609 Win=65535
4	0.154035	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=2681 Win=65535
5	0.195027	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=3753 Win=65535
6	0.195952	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=4825 Win=65535
7	0.236482	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=5897 Win=65535
8	0.237406	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=6969 Win=65535
9	0.238331	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=8041 Win=65535
10	0.277936	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=9113 Win=65535
11	0.278861	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=10185 Win=65535
12	0.279786	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=11257 Win=65535
13	0.28071	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=12329 Win=65535
14	0.318928	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=13401 Win=65535
15	0.319853	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=14473 Win=65535
16	0.320778	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=15545 Win=65535
17	0.321702	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=16617 Win=65535
18	0.322627	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=17689 Win=65535
19	0.323552	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=18761 Win=65535
20	0.35992	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=19833 Win=65535
21	0.360845	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=20905 Win=65535
22	0.36177	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=21977 Win=65535
23	0.362694	1	0	TCP 50000 > 49153 ACK Seq=1 Ack=23049 Win=65535

Fig. 2.2: An example of tables for packet meta-data with protocol filters

A way to visualize the trajectory of a mobile node

A way to display the routing-tables of multiple nodes at various points in time

A way to display counters associated with multiple nodes as a chart or a table

A way to view the timeline of packet transmit and receive events

2.1.1 Methodology

The class `ns3::AnimationInterface` is responsible for the creation the trace XML file. `AnimationInterface` uses the tracing infrastructure to track packet flows between nodes. `AnimationInterface` registers itself as a trace hook for tx and rx events before the simulation begins. When a packet is scheduled for transmission or reception, the corresponding tx and rx trace hooks in `AnimationInterface` are called. When the rx hooks are called, `AnimationInterface` will be aware of the two endpoints between which a packet has flowed, and adds this information to the trace file, in XML format along with the corresponding tx and rx timestamps. The XML format will be discussed in a later section. It is important to note that `AnimationInterface` records a packet only if the rx trace hooks are called. Every tx event must be matched by an rx event.

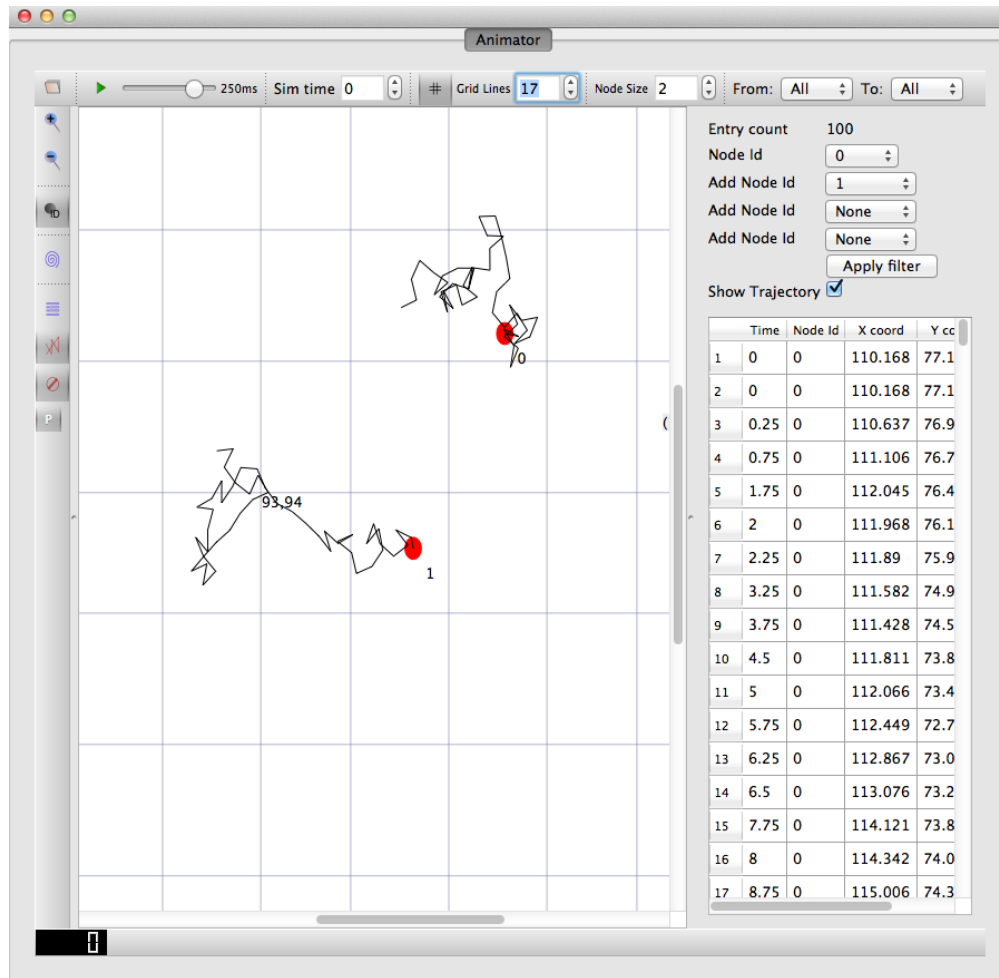
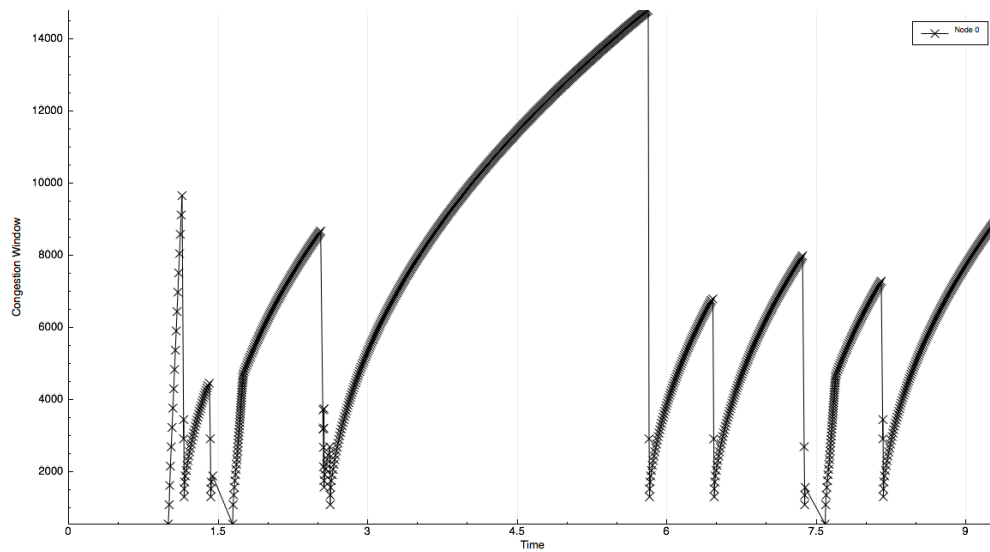


Fig. 2.3: An example of the trajectory of a mobile node

Node: 0 Time: 0s Ipv4ListRouting table									
Priority: 0	Protocol: ns3:Ipv4StaticRouting	Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface
127.0.0.0	0.0.0.0	255.255.255.0	U	0	-	-	0	-	-
10.1.3.0	0.0.0.0	255.255.255.0	U	0	-	-	1	-	-
Priority: -10 Protocol: ns3:Ipv4GlobalRouting									
Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface		
10.1.1.1	10.1.3.21	255.255.255.255	UH	-	-	-	1	-	-
10.1.1.2	10.1.3.21	255.255.255.255	UH	-	-	-	1	-	-
10.1.1.3	0.0.0.0	255.255.255.0	U	-	-	-	1	-	-
10.1.2.0	10.1.3.21	255.255.255.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.2	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.3	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.4	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.5	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.6	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.7	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.8	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.9	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.10	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.11	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.12	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.13	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.14	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.15	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.16	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.17	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.18	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.19	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.20	255.0.0.0	UG	-	-	-	1	-	-
10.1.1.0	10.1.3.21	255.255.255.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.21	255.0.0.0	UG	-	-	-	1	-	-
10.1.1.0	10.1.3.21	255.255.255.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.21	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.21	255.0.0.0	UG	-	-	-	1	-	-

Node: 1 Time: 0s Ipv4ListRouting table									
Priority: 0	Protocol: ns3:Ipv4StaticRouting	Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface
127.0.0.0	0.0.0.0	255.0.0.0	U	0	-	-	0	-	-
10.1.3.0	0.0.0.0	255.255.255.0	U	0	-	-	1	-	-
Priority: -10 Protocol: ns3:Ipv4GlobalRouting									
Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface		
10.1.1.1	10.1.3.21	255.255.255.255	UH	-	-	-	1	-	-
10.1.1.2	10.1.3.21	255.255.255.255	UH	-	-	-	1	-	-
10.1.1.3	0.0.0.0	255.255.255.0	U	-	-	-	1	-	-
10.1.2.0	10.1.3.21	255.255.255.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.1	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.2	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.3	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.4	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.5	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.6	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.7	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.8	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.9	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.10	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.11	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.12	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.13	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.14	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.15	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.16	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.17	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.18	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.19	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.20	255.0.0.0	UG	-	-	-	1	-	-
10.1.1.0	10.1.3.21	255.255.255.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.21	255.0.0.0	UG	-	-	-	1	-	-
10.1.1.0	10.1.3.21	255.255.255.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.21	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.21	255.0.0.0	UG	-	-	-	1	-	-

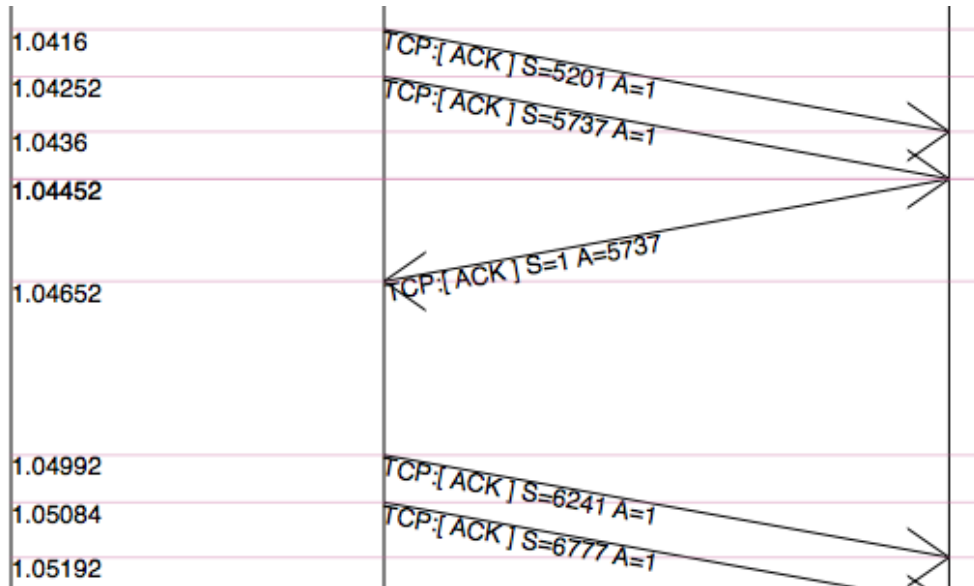
Node: 2 Time: 0s Ipv4ListRouting table									
Priority: 0	Protocol: ns3:Ipv4StaticRouting	Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface
127.0.0.0	0.0.0.0	255.0.0.0	U	0	-	-	0	-	-
10.1.3.0	0.0.0.0	255.255.255.0	U	0	-	-	1	-	-
Priority: -10 Protocol: ns3:Ipv4GlobalRouting									
Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface		
10.1.1.1	10.1.3.21	255.255.255.255	UH	-	-	-	1	-	-
10.1.1.2	10.1.3.21	255.255.255.255	UH	-	-	-	1	-	-
10.1.1.3	0.0.0.0	255.255.255.0	U	-	-	-	1	-	-
10.1.2.0	10.1.3.21	255.255.255.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.1	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.2	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.3	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.4	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.5	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.6	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.7	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.8	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.9	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.10	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.11	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.12	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.13	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.14	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.15	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.16	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.17	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.18	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.19	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.20	255.0.0.0	UG	-	-	-	1	-	-
10.1.1.0	10.1.3.21	255.255.255.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.21	255.0.0.0	UG	-	-	-	1	-	-
10.1.1.0	10.1.3.21	255.255.255.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.21	255.0.0.0	UG	-	-	-	1	-	-
127.0.0.0	10.1.3.21	255.0.0.0	UG	-	-	-	1	-	-



Counter Tables Sim Time Font Size 10 FlowMon file Double Counter 1

Export Table

	Time	0	1	2	3	4	5	6	7	8	9	10	11
1	0.1	134.532											
2	0.1					101.387							
3	0.2	164.6											
4	0.2					168.151							
5	0.3	154.444											
6	0.3					115.763							
7	0.4	179.264											
8	0.4					126.946							
9	0.5	144.204											
10	0.5					136.543							
11	0.6	116.857											
12	0.6					159.699							
13	0.7	174.138											
14	0.7					138.387							
15	0.8	101.934											
16	0.8					149.555							
17	0.9	195.25											
18	0.9					192.968							
19	1	132.337											
20	1					114.404							
21	1.1	142.992											
22	1.1					190.218							
23	1.2	122.185											



2.1.2 Downloading NetAnim

If NetAnim is not already available in the *ns-3* package you downloaded, you can do the following:

The latest version of NetAnim can be downloaded using git with the following command:

```
$ git clone https://gitlab.com/nsnam/netanim.git
```

2.1.3 Building NetAnim

Prerequisites

Qt5 (5.4 and over) and CMake are required to build NetAnim. The ns-3 Installation Guide lists some packages to install for some [Linux](#) systems, for *macOS* <<https://www.nsnam.org/docs/installation/html/macos.html#optional>>, and for [Windows](#).

The [Qt](#) site also provides download options.

Build steps

To build NetAnim use the following commands:

```
$ git clone https://gitlab.com/nsnam/netanim.git
$ cd netanim
$ mkdir build && cd build
$ cmake .. && cmake --build .
```

This should create an executable named “netanim” in the *build/bin* directory. If you prefer, you can move or copy this binary to another directory on your file system.

2.1.4 Usage

Using NetAnim is a two-step process

Step 1: Generate the animation XML trace file during simulation using “ns3::AnimationInterface” in the *ns-3* code base.

Step 2: Load the XML trace file generated in Step 1 with NetAnim.

Step 1: Generate XML animation trace file

The class “AnimationInterface” under “src/netanim” uses underlying *ns-3* trace sources to construct a timestamped ASCII file in XML format.

Examples are found under src/netanim/examples Example:

```
$ ./ns3 configure -d debug --enable-examples
$ ./ns3 run "dumbbell-animation"
```

The above will create an XML file dumbbell-animation.xml

Mandatory

1. Ensure that your program’s CMakeLists.txt includes the “netanim” module. An example of such a CMakeLists.txt is at src/netanim/examples/CMakeLists.txt.
2. Include the header [include “ns3/netanim-module.h”] in your test program
3. Add the statement

```
AnimationInterface anim("animation.xml"); // where "animation.xml" is any arbitrary
↪ filename
```

[for versions before ns-3.13 you also have to use the line “anim.SetXMLOutput() to set the XML mode and also use anim.StartAnimation();]

Optional

The following are optional but useful steps:

```
// Step 1
anim.SetMobilityPollInterval(Seconds(1));
```

AnimationInterface records the position of all nodes every 250 ms by default. The statement above sets the periodic interval at which AnimationInterface records the position of all nodes. If the nodes are expected to move very little, it is useful to set a high mobility poll interval to avoid large XML files.

```
// Step 2
anim.SetConstantPosition(Ptr< Node > n, double x, double y);
```

AnimationInterface requires that the position of all nodes be set. In *ns-3* this is done by setting an associated MobilityModel. “SetConstantPosition” is a quick way to set the x-y coordinates of a node which is stationary.

```
// Step 3
anim.SetStartTime(Seconds(150)); and anim.SetStopTime(Seconds(150));
```

AnimationInterface can generate large XML files. The above statements restricts the window between which AnimationInterface does tracing. Restricting the window serves to focus only on relevant portions of the simulation and creating manageably small XML files

```
// Step 4
AnimationInterface anim("animation.xml", 50000);
```

Using the above constructor ensures that each animation XML trace file has only 50000 packets. For example, if AnimationInterface captures 150000 packets, using the above constructor splits the capture into 3 files

- animation.xml - containing the packet range 1-50000
- animation.xml-1 - containing the packet range 50001-100000
- animation.xml-2 - containing the packet range 100001-150000

```
// Step 5
anim.EnablePacketMetadata(true);
```

With the above statement, AnimationInterface records the meta-data of each packet in the xml trace file. Metadata can be used by NetAnim to provide better statistics and filter, along with providing some brief information about the packet such as TCP sequence number or source & destination IP address during packet animation.

CAUTION: Enabling this feature will result in larger XML trace files.

```
// Step 6
anim.UpdateNodeDescription(5, "Access-point");
```

With the above statement, AnimationInterface assigns the text “Access-point” to node 5.

```
// Step 7
anim.UpdateNodeSize(6, 1.5, 1.5);
```

With the above statement, AnimationInterface sets the node size to scale by 1.5. NetAnim automatically scales the graphics view to fit the oboundaries of the topology. This means that NetAnim, can abnormally scale a node’s size too high or too low. Using AnimationInterface::UpdateNodeSize allows you to overwrite the default scaling in NetAnim and use your own custom scale.

```
// Step 8
anim.UpdateNodeCounter(89, 7, 3.4);
```

With the above statement, AnimationInterface sets the counter with Id == 89, associated with Node 7 with the value 3.4. The counter with Id 89 is obtained using AnimationInterface::AddNodeCounter. An example usage for this is in `src/netanim/examples/resource-counters.cc`.

Step 2: Loading the XML in NetAnim

1. Assuming NetAnim was built, use the command “./netanim” to launch NetAnim. Please review the section “Building NetAnim” if NetAnim is not available.
2. When NetAnim is opened, click on the File open button at the top-left corner, select the XML file generated during Step 1.
3. Hit the green play button to begin animation.

Here is a video illustrating this http://www.youtube.com/watch?v=tz_hUuNwFDs

2.1.5 Wiki

For more documentation on “NetAnim”, F.A.Qs and loading the XML trace file (mentioned earlier) using NetAnim please refer: <http://www.nsnam.org/wiki/NetAnim>

Please note that the wiki may refer to stale installation instructions.

ANTENNA

The Antenna module provides:

- A class `Angles` and utility functions to deal with angles.
- A base class `AntennaModel` that provides an interface for the modeling of the radiation pattern of an antenna with a set of classes derived from this base class, each models the radiation pattern of different types of antennas.
- A base class `PhasedArrayModel` that provides a flexible interface for modeling a number of Phase Antenna Array (PAA) models.
- A class `UniformPlanarArray` derived from this base class, implementing a Uniform Planar Array (UPA) supporting both rectangular and linear lattices.

The antenna model can be used with all the wireless technologies and physical layer models that support it. Currently, this includes the physical layer models based on the `SpectrumPhy` class. Please refer to the documentation of each of these models for details.

3.1 Scope and Limitations

Not present.

3.2 Angles

The `Angles` class holds information about an angle in 3D space using spherical coordinates in radian units. Specifically, it uses the azimuth-inclination convention, where

- Inclination is the angle between the zenith direction (positive z-axis) and the desired direction. It is included in the range $[0, \pi]$ radians.
- Azimuth is the signed angle measured from the positive x-axis, where a positive direction goes towards the positive y-axis. It is included in the range $[-\pi, \pi]$ radians.

Multiple constructors are present, supporting the most common ways to encode information on a direction. A static boolean variable allows the user to decide whether angles should be printed in radian or degree units.

A number of angle-related utilities are offered, such as radians/degree conversions, for both scalars and vectors, and angle wrapping.

3.3 AntennaModel

The AntennaModel uses the coordinate system adopted in [1] and depicted in Figure *Coordinate system of the AntennaModel*. This system is obtained by translating the Cartesian coordinate system used by the ns-3 MobilityModel into the new origin o which is the location of the antenna, and then transforming the coordinates of every generic point p of the space from Cartesian coordinates (x, y, z) into spherical coordinates (r, θ, ϕ) .

The antenna model neglects the radial component r , and only considers the angle components (θ, ϕ) . An antenna radiation pattern is then expressed as a mathematical function $g(\theta, \phi) \rightarrow \mathcal{R}$ that returns the gain (in dB) for each possible direction of transmission/reception. All angles are expressed in radians.

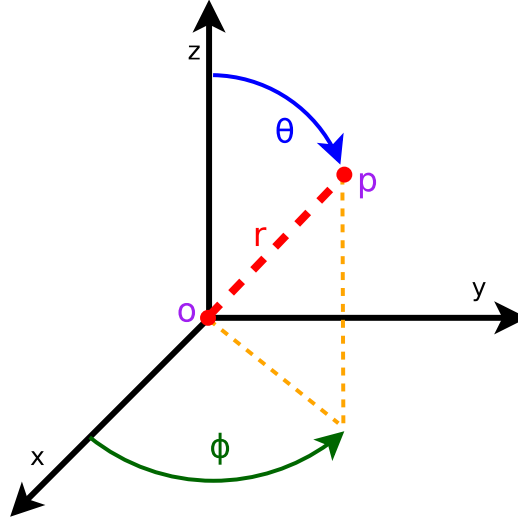


Fig. 3.1: Coordinate system of the AntennaModel

The AntennaModel is used to implement a subset of derived classes that represents different radiation patterns of a single antenna.

The current models supported are:

- Isotropic Antenna Model
- Cosine Antenna Model
- Parabolic Antenna Model
- Three Gpp Antenna Model

Isotropic Antenna Model

This is the simplest antenna model. This antenna radiation pattern model provides the same gain (0 dB) for all directions. This is implemented in the IsotropicAntennaModel class.

Cosine Antenna Model

This is the cosine model described in [2]: the antenna gain is determined as:

$$g(\phi, \theta) = \cos^n \left(\frac{\phi - \phi_0}{2} \right)$$

where ϕ_0 is the azimuthal orientation of the antenna (i.e., its direction of maximum gain) and the exponential

$$n = -\frac{3}{20 \log_{10} \left(\cos \frac{\phi_{3dB}}{4} \right)}$$

determines the desired 3dB beamwidth ϕ_{3dB} . Note that this radiation pattern is independent of the inclination angle θ .

A major difference between the model of [2] and the one implemented in the class `CosineAntennaModel` is that only the element factor (i.e., what described by the above formulas) is considered. In fact, [2] also considered an additional antenna array factor. The reason why the latter is excluded is that we expect that the average user would desire to specify a given beamwidth exactly, without adding an array factor at a latter stage which would in practice alter the effective beamwidth of the resulting radiation pattern.

This is implemented in the `CosineAntennaModel` class.

Parabolic Antenna Model

This model is based on the parabolic approximation of the main lobe radiation pattern. It is often used in the context of cellular system to model the radiation pattern of a cell sector, see for instance [3] and [4]. The antenna gain in dB is determined as:

$$g_{dB}(\phi, \theta) = -\min \left(12 \left(\frac{\phi - \phi_0}{\phi_{3dB}} \right)^2, A_{max} \right)$$

where ϕ_0 is the azimuthal orientation of the antenna (i.e., its direction of maximum gain), ϕ_{3dB} is its 3 dB beamwidth, and A_{max} is the maximum attenuation in dB of the antenna. Note that this radiation pattern is independent of the inclination angle θ .

This is implemented in the `ParabolicAntennaModel` class.

Three Gpp Antenna Model

This model implements the antenna element described in [5]. Parameters are fixed from the technical report, thus no attributes nor setters are provided. The model is largely based on the [Parabolic Antenna Model](#).

3.4 Phased Array Model

The class `PhasedArrayModel` has been created with flexibility in mind. It abstracts the basic idea of a Phased Antenna Array (PAA) by removing any constraint on the position of each element, and instead generalizes the concept of steering and beamforming vectors, solely based on the generalized location of the antenna elements. For details on Phased Array Antennas see for instance [6].

Derived classes must implement the following functions:

- `GetNumElements`: returns the number of antenna elements
- `GetElementLocation`: returns the location of the antenna element with the specified index, normalized with respect to the wavelength
- `GetElementFieldPattern`: returns the horizontal and vertical components of the antenna element field pattern at the specified direction. Same polarization (configurable) for all antenna elements of the array is considered.

The class `PhasedArrayModel` also assumes that all antenna elements are equal, a typical key assumption which allows to model the PAA field pattern as the sum of the array factor, given by the geometry of the location of the antenna elements, and the element field pattern. Any class derived from `AntennaModel` is a valid antenna element for the `PhasedArrayModel`, allowing for a great flexibility of the framework.

3.4.1 Uniform Planar Array (UPA)

The class `UniformPlanarArray` is a generic implementation of Uniform Planar Arrays (UPAs), supporting rectangular and linear regular lattices. It closely follows the implementation described in the 3GPP TR 38.901 [7], considering only a single panel, i.e., $N_g = M_g = 1$.

By default, the antenna array is orthogonal to the x-axis, pointing towards the positive direction, but the orientation can be changed through the attributes `BearingAngle`, which adjusts the azimuth angle, and `DowntiltAngle`, which adjusts the elevation angle. The slant angle is instead fixed and assumed to be 0.

The number of antenna elements in the vertical and horizontal directions can be configured through the attributes `NumRows` and `NumColumns`, while the spacing between the horizontal and vertical elements can be configured through the attributes `AntennaHorizontalSpacing` and `AntennaVerticalSpacing`.

`UniformPlanarArray` supports the concept of antenna ports following the sub-array partition model for TXRU virtualization, as described in Section 5.2.2 of 3GPP TR 36.897 [8]. The number of antenna ports in vertical and horizontal directions can be configured through the attributes `NumVerticalPorts` and `NumHorizontalPorts`, respectively. For example, if `NumRows` and `NumColumns` are configured to 2 and 4, and the number of `NumVerticalPorts` and `NumHorizontalPorts` to 1 and 2, then the antenna elements belonging to the first two columns of the antenna array will belong to the first antenna port, and the third and the fourth columns will belong to the second antenna port. Note that `NumRows` and `NumColumns` must be a multiple of `NumVerticalPorts` and `NumHorizontalPorts`, respectively.

Whether the antenna is dual-polarized or not is configured through the attribute `IsDualPolarized`. In case the antenna array is dual polarized, the total number of antenna elements is doubled and the two polarizations are overlapped in space. The polarization slant angle of the antenna elements belonging to the first polarization are configured through the attribute `PolSlantAngle`; while the antenna elements of the second polarization have the polarization slant angle minus 90 degrees, as described in [7] (i.e., ζ).

3.4.2 Circular Aperture Antenna Model

The class `CircularApertureAntennaModel` implements the radiation pattern described in [7]. Specifically, the latter represents parabolic antennas, i.e., antennas which are typically used for achieving long range communications such as earth-to-satellite links. The default boresight orientation is parallel to the positive z-axis, and it can be tuned by using the `AntennaInclination` and `AntennaAzimuth` parameters. This implementation provides an exact characterization of the antenna field pattern, by leveraging the standard library Bessel functions implementation introduced with C++17. Accordingly, the antenna gain G at an angle θ from the boresight main beam is evaluated as:

$$G \cdot 4 \left| \frac{J_1(k \cdot a \cdot \sin\theta)}{k \cdot a \cdot \sin\theta} \right|^2 \quad \text{for } 0 < |\theta| \leq 90^\circ$$
$$G \cdot 1 \quad \text{for } \theta = 0$$

where $J_1()$ is the Bessel function of the first kind and first order, and a is the radius of the antenna's circular aperture. The parameter k is equal to $k = \frac{2\pi f}{c}$, where f is the carrier frequency, and c is the speed of light in vacuum. The parameters G (in logarithmic scale), a and f can be configured by using the attributes `AntennaMaxGainDb`, `AntennaCircularApertureRadius` and `OperatingFrequency`, respectively. This type of antennas features a symmetric radiation pattern, meaning that a single angle, measured from the boresight direction, is sufficient to characterize the radiation strength along a given direction.

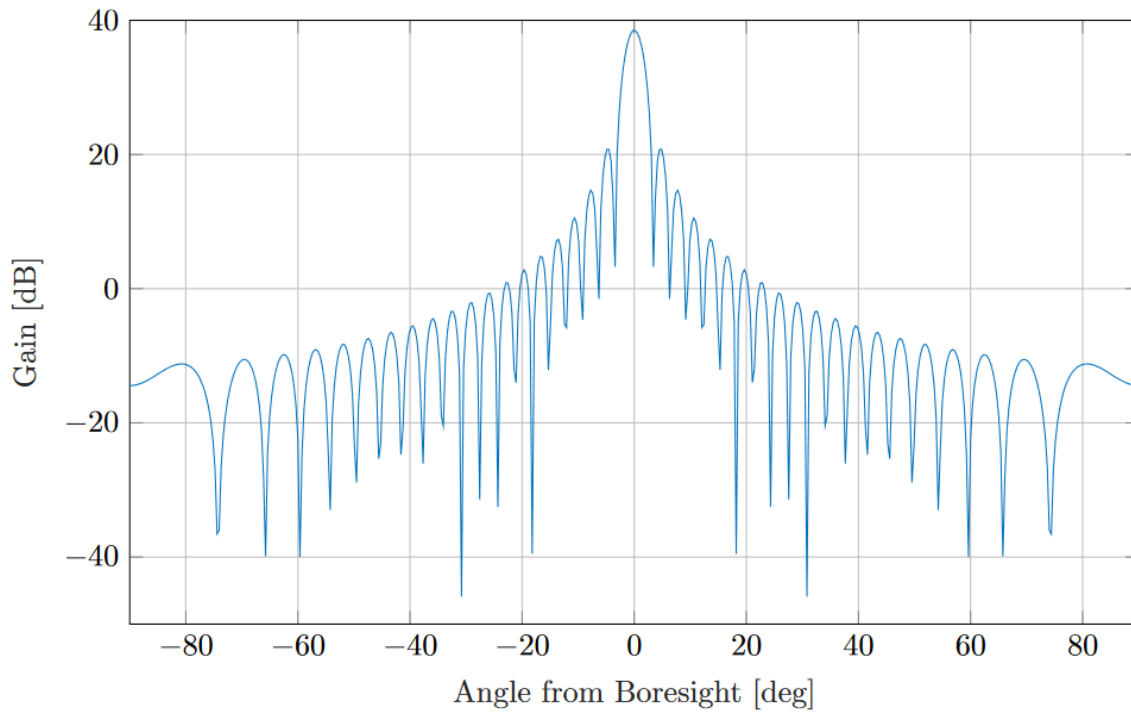


Fig. 3.2: Circular aperture antenna radiation pattern with $G = 38.5$ dB and $a = 10 \frac{c}{f}$.

3.5 Usage

Not present.

3.5.1 Helpers

Not present

3.5.2 Attributes

Not present.

3.5.3 Traces

Not present.

3.6 Examples and Tests

In this section we describe the test suites included with the antenna module that verify its correct functionality.

- `test-angles.cc`: The unit test suite `angles` verifies that the `Angles` class is constructed properly by correct conversion from 3D Cartesian coordinates according to the available methods (construction from a single vector and from a pair of vectors). For each method, several test cases are provided that compare the values (ϕ, θ) determined by the constructor to known reference values. The test passes if for each case the values are equal to the reference up to a tolerance of 10^{-10} which accounts for numerical errors.
- `test-degree-radians.cc`: The unit test suite `degrees-radians` verifies that the methods `DegreesToRadians` and `RadiansToDegrees` work properly by comparing with known reference values in a number of test cases. Each test case passes if the comparison is equal up to a tolerance of 10^{-10} which accounts for numerical errors.
- `test-isotropic-antenna.cc`: The unit test suite `isotropic-antenna-model` checks that the `IsotropicAntennaModel` class works properly, i.e., returns always a 0dB gain regardless of the direction.
- `test-cosine-antenna.cc`: The unit test suite `cosine-antenna-model` checks that the `CosineAntennaModel` class works properly. Several test cases are provided that check for the antenna gain value calculated at different directions and for different values of the orientation, the reference gain and the beamwidth. The reference gain is calculated by hand. Each test case passes if the reference gain in dB is equal to the value returned by `CosineAntennaModel` within a tolerance of 0.001, which accounts for the approximation done for the calculation of the reference values.
- `test-parabolic-antenna.cc`: The unit test suite `parabolic-antenna-model` checks that the `ParabolicAntennaModel` class works properly. Several test cases are provided that check for the antenna gain value calculated at different directions and for different values of the orientation, the maximum attenuation and the beamwidth. The reference gain is calculated by hand. Each test case passes if the reference gain in dB is equal to the value returned by `ParabolicAntennaModel` within a tolerance of 0.001, which accounts for the approximation done for the calculation of the reference values.

3.7 Validation

Validation has been performed as described in each antenna test.

3.8 References

- [1] C.A. Balanis, “Antenna Theory - Analysis and Design”, Wiley, 2nd Ed.
- [2] Li Chunjian, “Efficient Antenna Patterns for Three-Sector WCDMA Systems”, Master of Science Thesis, Chalmers University of Technology, Göteborg, Sweden, 2003.
- [3] 3GPP TSG RAN WG4 (Radio) Meeting #51, R4-092042, Simulation assumptions and parameters for FDD HeNB RF requirements.
- [4] George Calcev and Matt Dillon, “Antenna Tilt Control in CDMA Networks”, in Proc. of the 2nd Annual International Wireless Internet Conference (WICON), 2006.
- [5] 3GPP. 2018. TR 38.901, Study on channel model for frequencies from 0.5 to 100 GHz, V15.0.0. (2018-06).
- [6] Robert J. Mailloux, “Phased Array Antenna Handbook”, Artech House, 2nd Ed.
- [7] 3GPP. 2018. TR 38.811, Study on New Radio (NR) to support non-terrestrial networks, V15.4.0. (2020-09).
- [8] 3GPP. 2015. TR 36.897. Study on elevation beamforming / Full-Dimension (FD) Multiple Input Multiple Output (MIMO) for LTE. V13.0.0. (2015-06).

AD HOC ON-DEMAND DISTANCE VECTOR (AODV)

This model implements the base specification of the Ad Hoc On-Demand Distance Vector (AODV) protocol. The implementation is based on [RFC 3561](#).

The model was written by Elena Buchatskaia and Pavel Boyko of ITTP RAS, and is based on the ns-2 AODV model developed by the CMU/MONARCH group and optimized and tuned by Samir Das and Mahesh Marina, University of Cincinnati, and also on the AODV-UU implementation by Erik Nordström of Uppsala University.

4.1 Model Description

The source code for the AODV model lives in the directory *src/aodv*.

4.1.1 Design

Class `ns3::aodv::RoutingProtocol` implements all functionality of service packet exchange and inherits from `ns3::Ipv4RoutingProtocol`. The base class defines two virtual functions for packet routing and forwarding. The first one, `ns3::aodv::RouteOutput`, is used for locally originated packets, and the second one, `ns3::aodv::RouteInput`, is used for forwarding and/or delivering received packets.

Protocol operation depends on many adjustable parameters. Parameters for this functionality are attributes of `ns3::aodv::RoutingProtocol`. Parameter default values are drawn from the RFC and allow the enabling/disabling protocol features, such as broadcasting HELLO messages, broadcasting data packets and so on.

AODV discovers routes on demand. Therefore, the AODV model buffers all packets while a route request packet (RREQ) is disseminated. A packet queue is implemented in `aodv-rqueue.cc`. A smart pointer to the packet, `ns3::Ipv4RoutingProtocol::ErrorCallback`, `ns3::Ipv4RoutingProtocol::UnicastForwardCallback`, and the IP header are stored in this queue. The packet queue implements garbage collection of old packets and a queue size limit.

The routing table implementation supports garbage collection of old entries and state machine, defined in the standard. It is implemented as a STL map container. The key is a destination IP address.

Some elements of protocol operation aren't described in the RFC. These elements generally concern cooperation of different OSI model layers. The model uses the following heuristics:

- This AODV implementation can detect the presence of unidirectional links and avoid them if necessary. If the node the model receives an RREQ for is a neighbor, the cause may be a unidirectional link. This heuristic is taken from AODV-UU implementation and can be disabled.
- Protocol operation strongly depends on broken link detection mechanism. The model implements two such heuristics. First, this implementation support HELLO messages. However HELLO messages are not a good way to perform neighbor sensing in a wireless environment (at least not over 802.11). Therefore, one may experience

bad performance when running over wireless. There are several reasons for this: 1) HELLO messages are broadcasted. In 802.11, broadcasting is often done at a lower bit rate than unicasting, thus HELLO messages can travel further than unicast data. 2) HELLO messages are small, thus less prone to bit errors than data transmissions, and 3) Broadcast transmissions are not guaranteed to be bidirectional, unlike unicast transmissions. Second, we use layer 2 feedback when possible. Link are considered to be broken if frame transmission results in a transmission failure for all retries. This mechanism is meant for active links and works faster than the first method.

The layer 2 feedback implementation relies on the `TxErrHeader` trace source, currently supported in `AdhocWifiMac` only.

4.1.2 Scope and Limitations

The model is for IPv4 only. The following optional protocol optimizations are not implemented:

1. Local link repair.
2. RREP, RREQ and HELLO message extensions.

These techniques require direct access to IP header, which contradicts the assertion from the AODV RFC that AODV works over UDP. This model uses UDP for simplicity, hindering the ability to implement certain protocol optimizations. The model doesn't use low layer raw sockets because they are not portable.

4.1.3 Future Work

No announced plans.

APPLICATIONS

5.1 3GPP HTTP applications

The model is a part of the applications library. The HTTP model is based on a commonly used 3GPP model in standardization [4].

5.1.1 Design

This traffic generator simulates web browsing traffic using the Hypertext Transfer Protocol (HTTP). It consists of one or more `ThreeGppHttpClient` applications which connect to a `ThreeGppHttpServer` application. The client models a web browser which requests web pages to the server. The server is then responsible to serve the web pages as requested. Please refer to `ThreeGppHttpClientHelper` and `ThreeGppHttpServerHelper` for usage instructions.

Technically speaking, the client transmits *request objects* to demand a service from the server. Depending on the type of request received, the server transmits either:

- a *main object*, i.e., the HTML file of the web page; or
- an *embedded object*, e.g., an image referenced by the HTML file.

The main and embedded object sizes are illustrated in figures *3GPP HTTP main object size histogram* and *3GPP HTTP embedded object size histogram*.

A major portion of the traffic pattern is *reading time*, which does not generate any traffic. Because of this, one may need to simulate a good number of clients and/or sufficiently long simulation duration in order to generate any significant traffic in the system. Reading time is illustrated in *3GPP HTTP reading time histogram*.

3GPP HTTP server description

3GPP HTTP server is a model application which simulates the traffic of a web server. This application works in conjunction with `ThreeGppHttpClient` applications.

The application works by responding to requests. Each request is a small packet of data which contains `ThreeGppHttpHeader`. The value of the *content type* field of the header determines the type of object that the client is requesting. The possible type is either a *main object* or an *embedded object*.

The application is responsible to generate the right type of object and send it back to the client. The size of each object to be sent is randomly determined (see `ThreeGppHttpVariables`). Each object may be sent as multiple packets due to limited socket buffer space.

To assist with the transmission, the application maintains several instances of `ThreeGppHttpServerTxBuffer`. Each instance keeps track of the object type to be served and the number of bytes left to be sent.

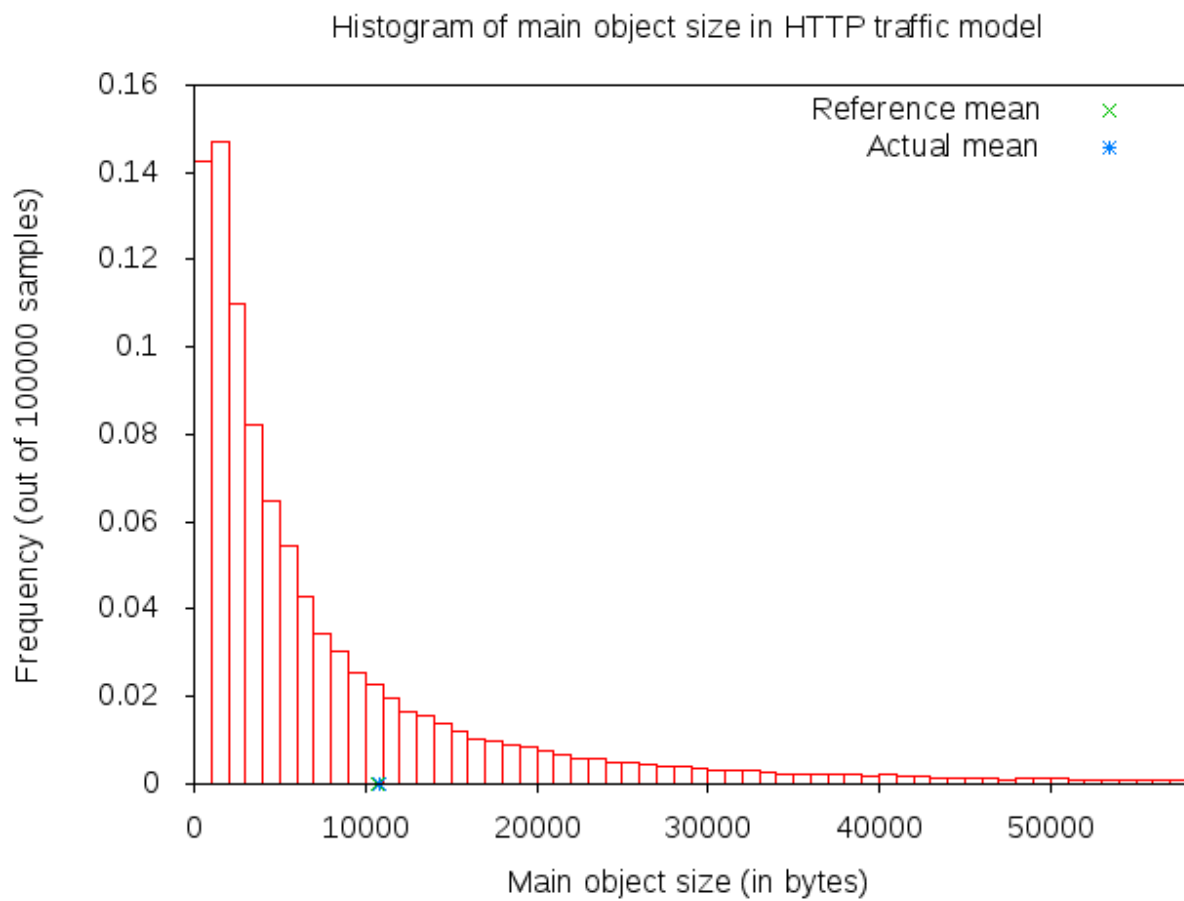


Fig. 5.1: 3GPP HTTP main object size histogram

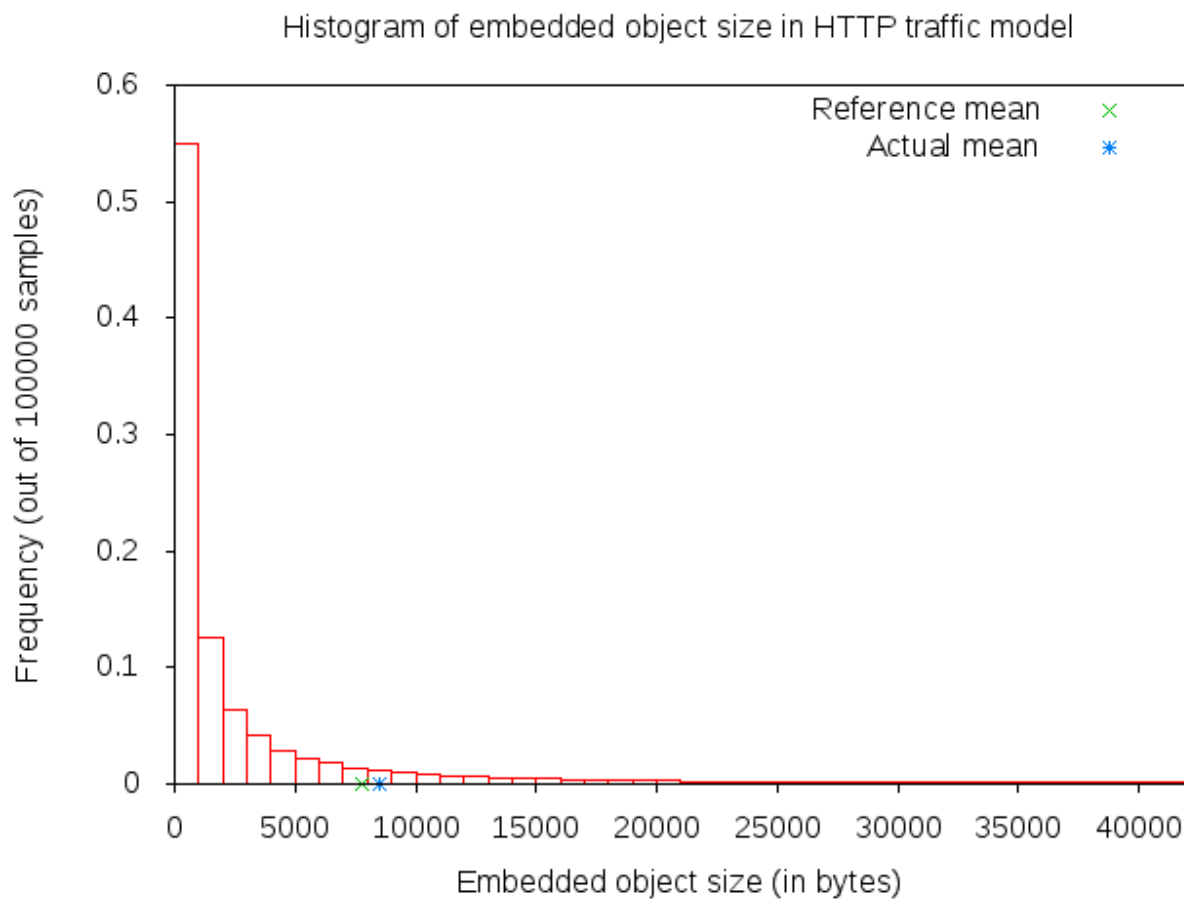


Fig. 5.2: 3GPP HTTP embedded object size histogram

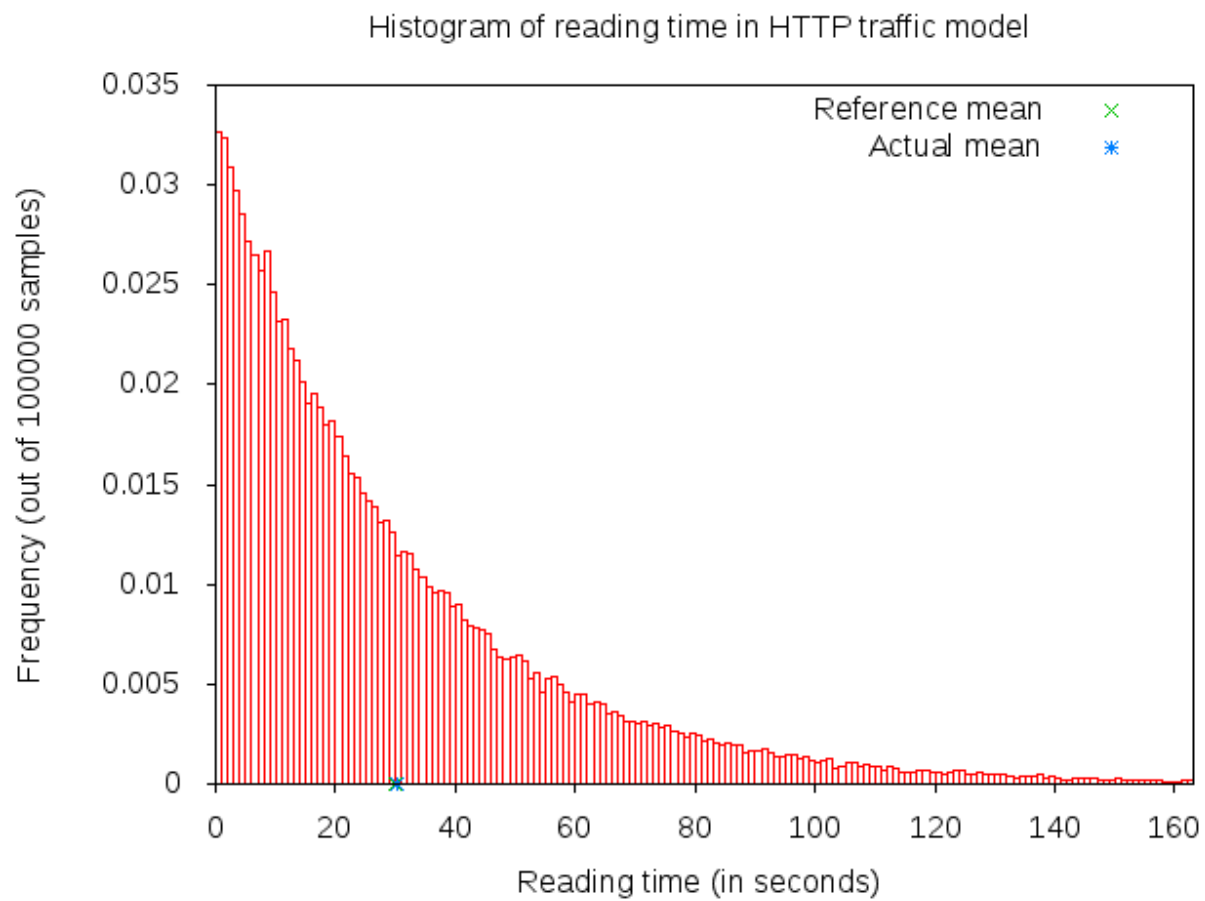


Fig. 5.3: 3GPP HTTP reading time histogram

The application accepts connection request from clients. Every connection is kept open until the client disconnects.

Maximum transmission unit (MTU) size is configurable in `ThreeGppHttpServer` or in `ThreeGppHttpVariables`. By default, the low variant is 536 bytes and high variant is 1460 bytes. The default values are set with the intention of having a TCP header (size of which is 40 bytes) added in the packet in such way that lower layers can avoid splitting packets. The change of MTU sizes affects all TCP sockets after the server application has started. It is mainly visible in sizes of packets received by `ThreeGppHttpClient` applications.

3GPP HTTP client description

3GPP HTTP client is a model application which simulates the traffic of a web browser. This application works in conjunction with an `ThreeGppHttpServer` application.

In summary, the application works as follows.

1. Upon start, it opens a connection to the destination web server (`ThreeGppHttpServer`).
2. After the connection is established, the application immediately requests a *main object* from the server by sending a request packet.
3. After receiving a main object (which can take some time if it consists of several packets), the application “parses” the main object. Parsing time is illustrated in figure *3GPP HTTP parsing time histogram*.
4. The parsing takes a short time (randomly determined) to determine the number of *embedded objects* (also randomly determined) in the web page. Number of embedded object is illustrated in *3GPP HTTP number of embedded objects histogram*.
 - **If at least one embedded object is determined, the application requests** the first embedded object from the server. The request for the next embedded object follows after the previous embedded object has been completely received.
 - **If there is no more embedded object to request, the application enters** the *reading time*.
5. Reading time is a long delay (again, randomly determined) where the application does not induce any network traffic, thus simulating the user reading the downloaded web page.
6. After the reading time is finished, the process repeats to step #2.

The client models HTTP *persistent connection*, i.e., HTTP 1.1, where the connection to the server is maintained and used for transmitting and receiving all objects.

Each request by default has a constant size of 350 bytes. A `ThreeGppHttpHeader` is attached to each request packet. The header contains information such as the content type requested (either main object or embedded object) and the timestamp when the packet is transmitted (which will be used to compute the delay and RTT of the packet).

5.1.2 References

Many aspects of the traffic are randomly determined by `ThreeGppHttpVariables`. A separate instance of this object is used by the HTTP server and client applications. These characteristics are based on a legacy 3GPP specification. The description can be found in the following references:

[1] 3GPP TR 25.892, “Feasibility Study for Orthogonal Frequency Division Multiplexing (OFDM) for UTRAN enhancement”

[2] IEEE 802.16m, “Evaluation Methodology Document (EMD)”, IEEE 802.16m-08/004r5, July 2008.

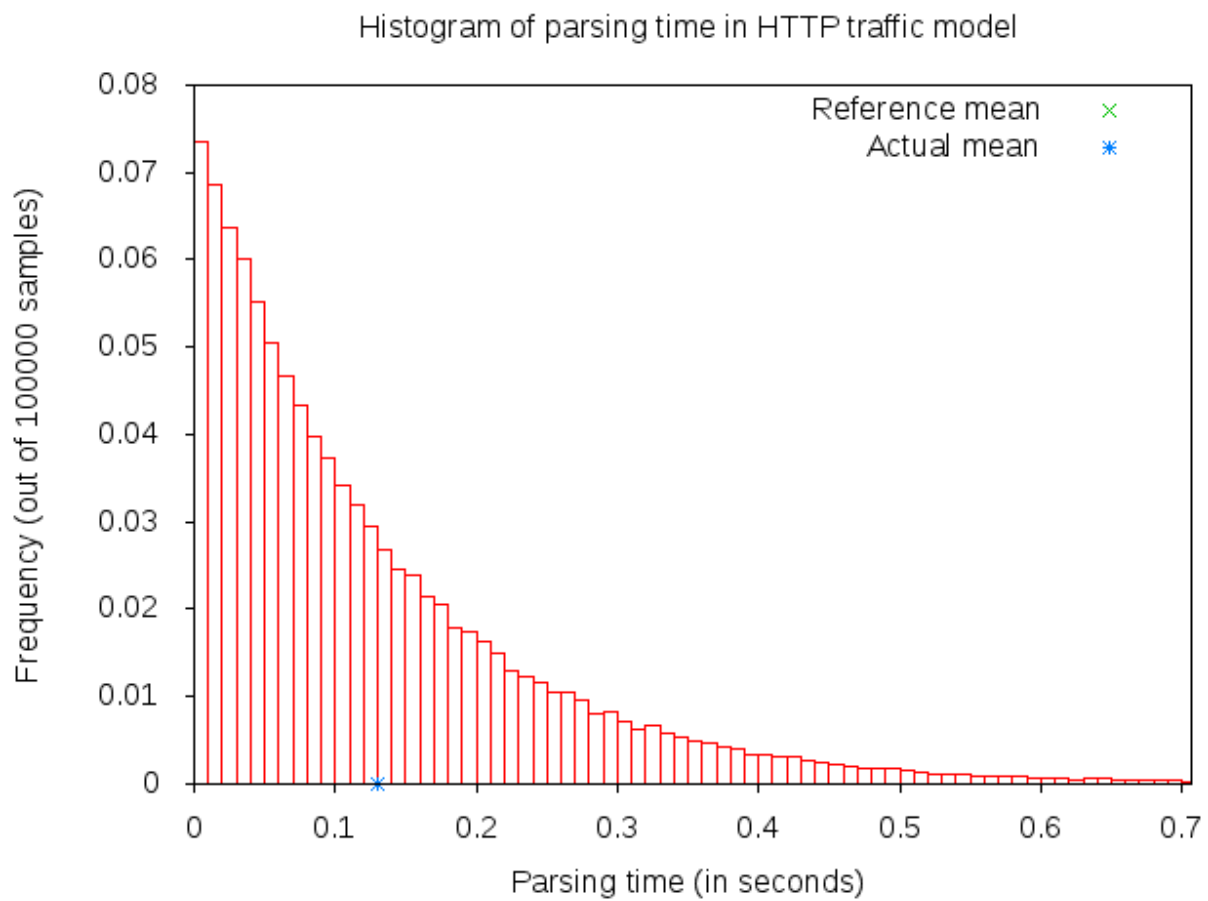


Fig. 5.4: 3GPP HTTP parsing time histogram

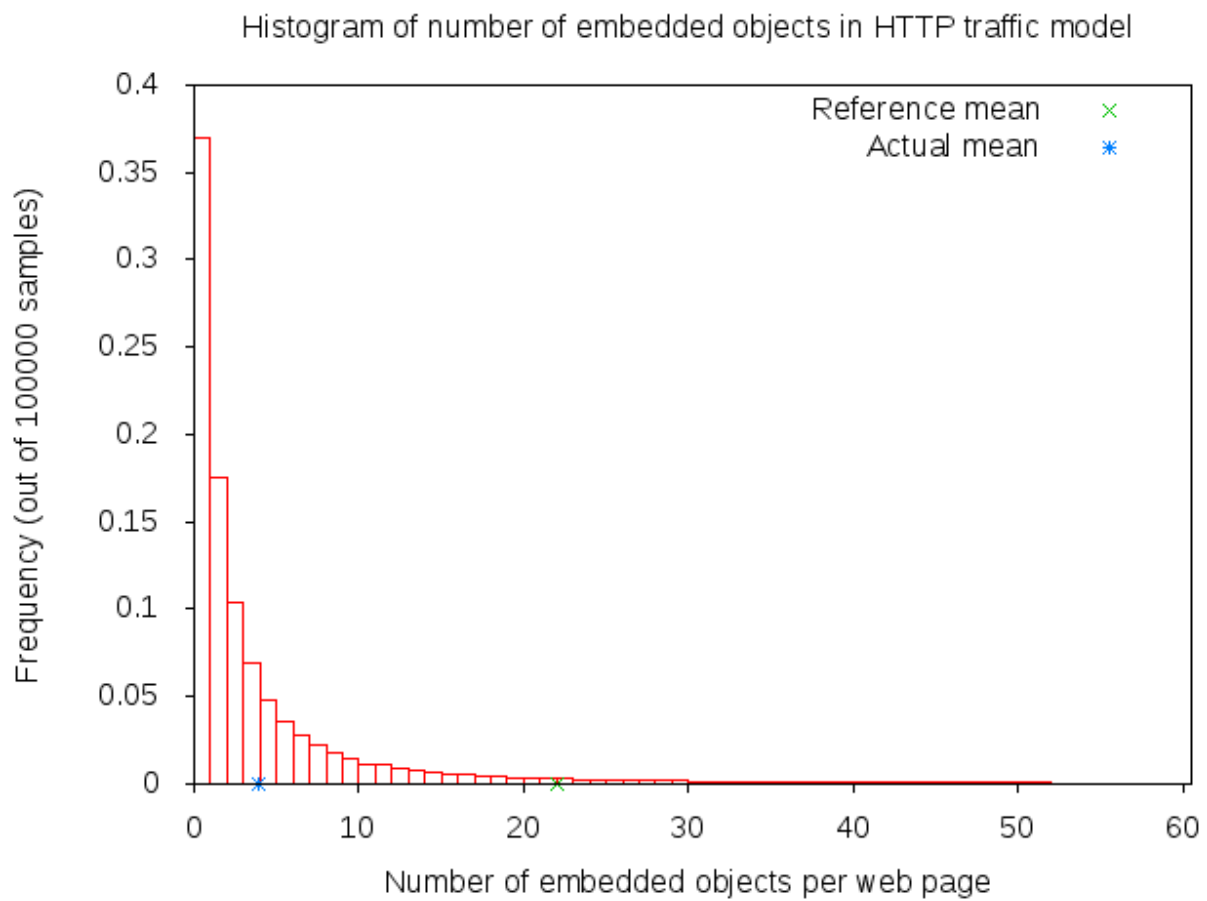


Fig. 5.5: 3GPP HTTP number of embedded objects histogram

[3] NGMN Alliance, “NGMN Radio Access Performance Evaluation Methodology”, v1.0, January 2008.

[4] 3GPP2-TSGC5, “HTTP, FTP and TCP models for 1xEV-DV simulations”, 2001.

5.1.3 Usage

The `three-gpp-http-example` can be referenced to see basic usage of the HTTP applications. In summary, using the `ThreeGppHttpServerHelper` and `ThreeGppHttpClientHelper` allow the user to easily install `ThreeGppHttpServer` and `ThreeGppHttpClient` applications to nodes. The helper objects can be used to configure attribute values for the client and server objects, but not for the `ThreeGppHttpVariables` object. Configuration of variables is done by modifying attributes of `ThreeGppHttpVariables`, which should be done prior to helpers installing applications to nodes.

The client and server provide a number of ns-3 trace sources such as “Tx”, “Rx”, “RxDelay”, and “State-Transition” on the server side, and a large number on the client side (“ConnectionEstablished”, “Connection-Closed”, “TxMainObjectRequest”, “TxEmbeddedObjectRequest”, “RxMainObjectPacket”, “RxMainObject”, “RxEmbeddedObjectPacket”, “RxEmbeddedObject”, “Rx”, “RxDelay”, “RxRtt”, “StateTransition”).

5.1.4 Examples

For an example demonstrating HTTP applications run:

```
$ ./ns3 run 'three-gpp-http-example'
```

By default, the example will print out the web page requests of the client and responses of the server and client receiving content packets by using `LOG_INFO` of `ThreeGppHttpServer` and `ThreeGppHttpClient`.

5.1.5 Tests

For testing HTTP applications, `three-gpp-http-client-server-test` is provided. Run:

```
$ ./test.py -s three-gpp-http-client-server-test
```

The test consists of simple Internet nodes having HTTP server and client applications installed. Multiple variant scenarios are tested: delay is 3ms, 30ms or 300ms, bit error rate 0 or $5.0 \cdot 10^{-6}$, MTU size 536 or 1460 bytes and either IPV4 or IPV6 is used. A simulation with each combination of these parameters is run multiple times to verify functionality with different random variables.

Test cases themselves are rather simple: test verifies that HTTP object packet bytes sent match total bytes received by the client, and that `ThreeGppHttpHeader` matches the expected packet.

5.2 VoIP Traffic Model

This section describes the implementation of the *ns-3* VoIP traffic generator model, which is based on the IEEE 802.11-14/0571r12 TGAX evaluation methodology [TGAX_EVAL]. The model generates realistic voice-over-IP (VoIP) traffic patterns suitable for evaluating IEEE 802.11 networks and other wireless systems.

The VoIP traffic model is implemented in the `src/applications/` folder.

The VoIP traffic generator models the behavior of a real VoIP source, producing traffic that alternates between periods of active talking and silence. This model is useful for simulating realistic VoIP scenarios in network simulations, particularly for evaluating the performance of wireless access technologies such as IEEE 802.11 (Wi-Fi).

The model implements a two-state Markov chain to represent voice activity, where transitions occur between active (talking) and inactive (silence) states. During active states, the source generates packets at regular intervals (20 ms) with one payload size, and during silence states, packets are generated at longer intervals (160 ms) with a smaller payload size. Both states can last for exponentially distributed durations, giving the model stochastic properties that match real voice traffic characteristics.

The VoIP model can be used with various socket types including IPv4/UDP, IPv6/UDP, and PacketSocket, making it flexible for different network configurations and simulation scenarios.

5.2.1 Scope and Limitations

The VoIP model implements the core traffic generation aspects of the IEEE 802.11-14/0571r12 specification. The following describes what is covered and what is not:

Supported Features:

- Two-state Markov model for voice activity (active/inactive states)
- Configurable state transition probabilities
- Exponential distribution for state durations
- Variable packet payload sizes for active and inactive states
- Laplacian distribution for downlink packet delay jitter
- Integration with UDP, TCP, and PacketSocket protocols
- Comprehensive trace sources for monitoring packet transmission and state changes

Known Limitations:

- Protocol header overhead (3 bytes for IPv4, 5 bytes for IPv6— both values assuming IP header compression (not implemented in ns-3) is used) must be added to payload sizes
- Jitter is applied uniformly to all packets (asymmetric uplink/downlink jitter not differentiated)
- TCP transmission does not validate buffer availability; packets may be dropped if the TX buffer is full
- The model does not enforce the implicit 12.2 kbps bit rate constraint in the IEEE document; users must configure appropriate parameters
- Directional traffic (unidirectional source only, no bidirectional conversation modeling)

5.2.2 Model Description

The `TgaxVoipTraffic` application is a source application that generates VoIP traffic patterns according to the IEEE 802.11-14/0571r12 evaluation methodology. The application uses a simple but effective two-state Markov model to generate realistic voice activity patterns.

State Machine:

The model maintains two states:

- **ACTIVE_TALKING**: The source is actively transmitting voice data. In this state, packets are generated at the voice encoder interval (default 20 ms) with a payload size configured for active voice packets (default 33 bytes).
- **INACTIVE_SILENCE**: The source is in a silent period. In this state, packets are generated at the silence encoder interval (default 160 ms) with a smaller payload size (default 7 bytes). Silence packets typically carry comfort noise or silence indication in real VoIP systems.

State Transitions:

The transition from one state to another follows a two-state Markov chain with configurable transition probabilities:

- **VoiceToSilenceProbability** (default 0.016): Probability per state update interval to transition from ACTIVE_TALKING to INACTIVE_SILENCE.
- **SilenceToVoiceProbability** (default 0.016): Probability per state update interval to transition from INACTIVE_SILENCE to ACTIVE_TALKING.

With default parameters (both probabilities at 0.016), the model produces a voice activity factor of approximately 50%, meaning the source is in the active state about half the time.

State Duration:

Once a state is entered, the source remains in that state for an exponentially distributed duration:

- **MeanActiveStateDuration** (default 1250 ms): Mean duration of active talking periods.
- **MeanInactiveStateDuration** (default 1250 ms): Mean duration of silence periods.

These exponential distributions produce realistic variability in conversation patterns, where some active periods are very short and others are quite long.

Packet Generation and Jitter:

Packets are generated at regular intervals determined by the current state (20 ms for active, 160 ms for silence). Each generated packet receives a random delay jitter before transmission, drawn from a Laplacian distribution:

- **MeanDelayJitter** (default 0 us): Mean of the Laplacian distribution.
- **ScaleDelayJitter** (default 5110 us): Scale parameter of the Laplacian distribution.
- **BoundDelayJitter** (default 80 ms): Bound on the jitter magnitude to ensure stability.

The jitter model is applied only in the downlink direction, as specified in the IEEE 802.11-14/0571r12 methodology.

Payload Sizes and Protocol Headers:

The model specifies default payload sizes based on the IEEE specification:

- **ActivePacketPayloadSize** (default 33 bytes): Size of packets during active talking.
- **SilencePacketPayloadSize** (default 7 bytes): Size of packets during silence periods.

When using UDP over IPv4 or IPv6, users should account for compressed protocol headers: - IPv4 with compression: add 3 bytes to each payload size - IPv6 with compression: add 5 bytes to each payload size

These header sizes are NOT automatically included; the user must adjust the payload size attributes accordingly if protocol header compression is desired in the simulation.

Configuration and Usage

The VoIP traffic generator is instantiated as an application through the `ApplicationHelper`. Here is a basic example using IPv4/UDP:

```
// Create nodes
NodeContainer nodes;
nodes.Create(2);

// Install Internet stack and assign IP addresses
InternetStackHelper internet;
internet.Install(nodes);

Ipv4AddressHelper ipv4;
ipv4.SetBase("10.0.0.0", "255.255.255.0");
ipv4.Assign(devices);

// Configure VoIP application
ApplicationHelper sourceHelper(TgaxVoipTraffic::GetTypeId());
sourceHelper.SetAttribute("Protocol", StringValue("ns3::UdpSocketFactory"));
sourceHelper.SetAttribute("Remote", AddressValue(InetSocketAddress(Ipv4Address("10.0.0.2", 5000)));
sourceHelper.SetAttribute("ActivePacketPayloadSize", UIntegerValue(33 + 3)); // +3 for IPv4 header
sourceHelper.SetAttribute("SilencePacketPayloadSize", UIntegerValue(7 + 3));

auto sourceApps = sourceHelper.Install(nodes.Get(0));
sourceApps.Start(Seconds(1.0));
sourceApps.Stop(Seconds(11.0));

// Create receiver
PacketSinkHelper sinkHelper("ns3::UdpSocketFactory", InetSocketAddress(Ipv4Address::GetAny(), 5000));
auto sinkApps = sinkHelper.Install(nodes.Get(1));
sinkApps.Start(Seconds(0.0));
sinkApps.Stop(Seconds(12.0));
```

5.2.3 Attributes

The VoIP traffic generator supports multiple socket types through the `Protocol` attribute:

- **ns3::UdpSocketFactory**: UDP socket (recommended for most scenarios)
- **ns3::TcpSocketFactory**: TCP socket (may experience packet drops if TX buffer is full)
- **ns3::PacketSocketFactory**: Raw packet socket (useful for link-layer only simulations)

For each socket type, the `Remote` address must be configured appropriately:

- For UDP/TCP: Use `InetSocketAddress` for IPv4 or `Inet6SocketAddress` for IPv6
- For PacketSocket: Use `PacketSocketAddress` with physical layer addressing

The following additional attributes control the behavior of the VoIP traffic generator:

Protocol Configuration:

- **Protocol:** The socket factory type to use. Default: `ns3::PacketSocketFactory`
- **Remote:** The remote address and port for the VoIP destination

Packet Sizes:

- **ActivePacketPayloadSize:** Payload size (in bytes) for packets during active talking states. Default: 33
- **SilencePacketPayloadSize:** Payload size (in bytes) for packets during silence states. Default: 7

State Machine Parameters:

- **MeanActiveStateDuration:** Mean duration (Time) of active talking states. Default: 1250 ms
- **MeanInactiveStateDuration:** Mean duration (Time) of silence states. Default: 1250 ms
- **VoiceToSilenceProbability:** Transition probability from active to silence state per state update. Default: 0.016
- **SilenceToVoiceProbability:** Transition probability from silence to active state per state update. Default: 0.016

Encoder Parameters:

- **VoiceEncoderInterval:** Time interval between generation of active voice packets. Default: 20 ms
- **SilenceEncoderInterval:** Time interval between generation of silence packets. Default: 160 ms

Jitter Model:

- **MeanDelayJitter:** Mean of the Laplacian distribution for packet delay jitter. Default: 0 us
- **ScaleDelayJitter:** Scale parameter of the Laplacian distribution for jitter. Default: 5110 us
- **BoundDelayJitter:** Upper bound on the magnitude of applied jitter. Default: 80 ms

5.2.4 Trace Sources

The VoIP traffic generator provides two main trace sources for monitoring simulation behavior:

TxWithJitter:

```
typedef void (*TxTracedCallback)(Ptr<const Packet> packet, Time jitter);
```

This trace source is fired when a packet is transmitted. It provides both the packet being transmitted and the jitter value that was applied to that packet. This is useful for analyzing packet transmission patterns and the distribution of applied jitter.

StateUpdate:

```
typedef void (*StateUpdatedCallback)(VoiceActivityState state, Time duration);
```

This trace source is fired when the voice activity state changes (from active to silence or vice versa). It provides the new state and the expected duration of that state. This is useful for analyzing voice activity patterns and verifying that the Markov model is producing the expected statistics.

5.2.5 Examples

The `voip-example.cc` program demonstrates basic usage of the VoIP traffic generator. This example sets up a simple two-node Wi-Fi network (AP and STA) with VoIP traffic flowing from the STA to the AP. The example supports three socket type configurations: IPv4/UDP, IPv6/UDP, and PacketSocket.

5.2.6 Tests

Unit tests for the VoIP traffic model are provided in `src/applications/test/tgax-voip-traffic-test-suite.cc`. These tests verify:

- Correct packet sizes for active and silence states
- Proper encoder frame intervals (20 ms for active, 160 ms for silence)
- State duration statistics matching exponential distribution expectations
- State transition correctness
- Proper application of jitter to packets
- Voice activity factor statistics matching theoretical expectations

To run the VoIP traffic tests:

```
$ ./test.py -s tgax-voip-traffic
```

5.2.7 Verification

The VoIP traffic model has been checked against the IEEE 802.11-14/0571r12 specification by comparing:

- Default parameter values against those specified in the standard
- Observed state durations against exponential distribution properties
- Voice activity factor (ratio of active to total time) against theoretical expectations
- Packet sizes and intervals against specification

The test suite demonstrates that the model correctly implements the two-state Markov behavior, produces appropriate state durations, and generates traffic patterns consistent with real VoIP systems.

5.3 TGax Video Traffic Model

This section describes the implementation of the *ns-3* TGax video traffic generator model, which is based on the IEEE 802.11-14/0571r12 TGAX evaluation methodology [TGAX_EVAL]. The model generates buffered video streaming traffic patterns (notionally generated by services such as YouTube or Netflix) suitable for evaluating IEEE 802.11 networks and other wireless systems.

The TGax video traffic model is implemented in the `src/applications/` folder.

The video traffic generator models the behavior of a video streaming source, producing traffic that simulates video frames being generated at the application layer, fragmented at the transport layer, and experiencing network latency as they traverse the Internet before reaching the access point for transmission to clients.

The model implements a Weibull distribution for video frame size generation and a Gamma distribution for network latency modeling, as specified in the IEEE 802.11-14/0571r12 evaluation methodology. The latter is disabled by default since the model is primarily intended for use with a link simulator that does not model queues and TCP behavior.

5.3.1 Scope and Limitations

The TGax video model implements the buffered video streaming and multicast video streaming traffic models from the IEEE 802.11-14/0571r12 specification. The following describes what is covered and what is not:

Supported Features:

- Eight predefined traffic model profiles (BV1-BV6 for buffered video, MC1-MC2 for multicast)
- Custom traffic model configuration with user-defined parameters
- Weibull distribution for video frame size generation
- Gamma distribution for network latency modeling (mean ~14.8ms)
- TCP and UDP protocol support
- Multicast video modes
- Automatic TCP segment size handling for large video frames
- Comprehensive trace sources for monitoring frame generation and packet transmission

Known Limitations:

- The Wireless Display (WD1-WD4) traffic models from the specification are not implemented
- The Video Conferencing (VC) traffic model is not explicitly implemented (users can simulate this by creating bidirectional BV1 streams)
- TCP acknowledgment traffic follows ns-3's built-in TCP behavior rather than the explicit 40-byte/1ms delay model in the specification
- Network latency is not regenerated if it would cause packets to arrive after simulation end

5.3.2 Model Description

The `TgaxVideoTraffic` application is a source application that generates video streaming traffic patterns according to the IEEE 802.11-14/0571r12 evaluation methodology. The application models video streaming services such as YouTube and Netflix.

Traffic Generation Process:

The model follows a three-step process:

1. **Frame Size Generation:** At the application layer, video frame sizes (in bytes) are generated according to a Weibull distribution with parameters that vary by target bit rate.
2. **Frame Transmission:** Video frames are sent to the transport layer. For TCP, frames larger than the segment size (1500 bytes) are automatically fragmented.
3. **Network Latency:** Each packet experiences network latency drawn from a Gamma distribution with mean approximately 14.8ms, simulating the delay encountered as video traffic traverses multiple hops in the Internet before reaching the access point.

(note, if code review comment accepted, step 3 should be described as optional)

Predefined Traffic Models:

The implementation provides eight predefined traffic model profiles, matching those in the IEEE specification:

Buffered Video (BV) Models:

Table 5.1: Buffered Video Traffic Models

Model ID	Bit Rate	Weibull Scale (lambda)	Weibull Shape (k)
BV1	2 Mbps	6950	0.8099
BV2	4 Mbps	13900	0.8099
BV3	6 Mbps	20850	0.8099
BV4	8 Mbps	27800	0.8099
BV5	10 Mbps	34750	0.8099
BV6	15.6 Mbps	54210	0.8099

Multicast Video (MC) Models:

Table 5.2: Multicast Video Traffic Models

Model ID	Bit Rate	Weibull Scale (lambda)	Weibull Shape (k)
MC1	3 Mbps	10425	0.8099
MC2	6 Mbps	20850	0.8099

Custom model:

The default CUSTOM model has parameter defaults corresponding to BV1 model, but is intended to be used and reconfigured when users may want to use their own model parameters.

Network Latency Model:

The network latency experienced by each packet is modeled using a Gamma distribution:

- **Shape parameter (k):** 0.2463
- **Scale parameter (theta):** 60.227
- **Mean latency:** $k * \text{theta} = 14.834 \text{ ms}$

This models the delay variation experienced as video packets traverse multiple network hops between the video server and the access point.

Configuration and Usage

The video traffic generator is instantiated as an application through the `ApplicationHelper`. Here is a basic example using UDP:

```
// Create nodes
NodeContainer nodes;
nodes.Create(2);

// Install Internet stack and assign IP addresses
InternetStackHelper internet;
internet.Install(nodes);

Ipv4AddressHelper ipv4;
ipv4.SetBase("10.0.0.0", "255.255.255.0");
ipv4.Assign(devices);

// Configure video application with BV3 (6 Mbps) profile
ApplicationHelper sourceHelper(TgaxVideoTraffic::GetTypeId());
sourceHelper.SetAttribute("Protocol", TypeIdValue(UdpSocketFactory::GetTypeId()));
```

(continues on next page)

(continued from previous page)

```

sourceHelper.SetAttribute("Remote", AddressValue(InetSocketAddress(Ipv4Address("10.0.0.2
↪"), 5000)));
sourceHelper.SetAttribute("TrafficModelClassIdentifier", StringValue("BV3"));

auto sourceApps = sourceHelper.Install(nodes.Get(0));
sourceApps.Start(Seconds(1.0));
sourceApps.Stop(Seconds(11.0));

// Create receiver
PacketSinkHelper sinkHelper("ns3::UdpSocketFactory", ↪
↪InetSocketAddress(Ipv4Address::GetAny(), 5000));
auto sinkApps = sinkHelper.Install(nodes.Get(1));
sinkApps.Start(Seconds(0.0));
sinkApps.Stop(Seconds(12.0));

```

For custom traffic parameters:

```

// Configure custom video parameters
ApplicationHelper sourceHelper(TgaxVideoTraffic::GetTypeId());
sourceHelper.SetAttribute("Protocol", TypeIdValue(UdpSocketFactory::GetTypeId()));
sourceHelper.SetAttribute("Remote", AddressValue(InetSocketAddress(Ipv4Address("10.0.0.2
↪"), 5000)));
sourceHelper.SetAttribute("TrafficModelClassIdentifier", StringValue("Custom"));
sourceHelper.SetAttribute("CustomVideoBitRate", DataRateValue(DataRate("5Mbps")));
sourceHelper.SetAttribute("CustomWeibullScale", DoubleValue(17375));
sourceHelper.SetAttribute("CustomWeibullShape", DoubleValue(0.8099));

```

5.3.3 Attributes

The video traffic generator supports UDP and TCP protocols through the Protocol attribute:

- **ns3::UdpSocketFactory**: UDP socket (recommended for video streaming simulation)
- **ns3::TcpSocketFactory**: TCP socket (realistic for actual video streaming behavior)

The Remote address must be configured appropriately:

- For UDP/TCP: Use InetSocketAddress for IPv4 or Inet6SocketAddress for IPv6

The following additional attributes control the behavior of the video traffic generator:

Protocol Configuration:

- Protocol: The socket factory type to use. Default: ns3::UdpSocketFactory
- Remote: The remote address and port for the video destination

Traffic Model Selection:

- TrafficModelClassIdentifier: The traffic model to use. Options: BV1, BV2, BV3, BV4, BV5, BV6, MC1, MC2, or Custom. Default: Custom

The BV1-BV6 and MC1-MC2 options correspond to the predefined traffic models from the IEEE 802.11-14/0571r12 specification. The Custom option is an implementation extension (not part of the IEEE specification) that allows users to specify their own Weibull distribution parameters for experimentation or interpolation between defined bit rates.

Custom Model Parameters (only used when TrafficModelClassIdentifier is Custom):

- CustomVideoBitRate: The target video bit rate. Default: 2 Mbps
- CustomWeibullScale: Scale parameter (lambda) for the Weibull distribution. Default: 6950
- CustomWeibullShape: Shape parameter (k) for the Weibull distribution. Default: 0.8099

Note: The default values for the Custom model match the BV1 parameters from the specification.

Network Latency Parameters:

- GammaShape: Shape parameter (k) for the Gamma distribution for network latency. Default: 0.2463
- GammaScale: Scale parameter (theta) for the Gamma distribution. Default: 60.227

5.3.4 Trace Sources

The video traffic generator provides two main trace sources for monitoring simulation behavior:

TxWithLatency:

```
typedef void (*TxTracedCallback)(Ptr<const Packet> packet, Time latency);
```

This trace source is fired when a packet is transmitted. It provides both the packet being transmitted and the network latency value that was applied to that packet. This is useful for analyzing packet transmission patterns and the distribution of network latency.

VideoFrameGenerated:

```
typedef void (*FrameGeneratedCallback)(uint32_t frameSize);
```

This trace source is fired when a video frame is generated at the application layer. It provides the size of the generated frame in bytes. This is useful for analyzing the frame size distribution and verifying that it matches the expected Weibull distribution.

5.3.5 Examples

The `tgax-video-example.cc` program demonstrates basic usage of the video traffic generator. This example sets up a simple Wi-Fi network (AP and STAs) with video streaming traffic flowing from the AP to the station(s). The example supports:

- All predefined traffic models (BV1-BV6, MC1-MC2)
- Both UDP and TCP protocols
- Multicast delivery to multiple receivers

To run with default settings (BV1, 2 Mbps over UDP):

```
$ ./ns3 run tgax-video-example
```

To run with a higher bit rate profile:

```
$ ./ns3 run "tgax-video-example --model=BV3"
```

To run with TCP:

```
$ ./ns3 run "tgax-video-example --protocol=tcp"
```

To run a multicast scenario:

```
$ ./ns3 run "tgax-video-example --model=MC1"
```

5.3.6 Tests

Unit tests for the video traffic model are provided in `src/applications/test/tgax-video-traffic-test-suite.cc`. These tests verify:

- All predefined traffic models (BV1-BV6) work correctly with both TCP and UDP
- Multicast models (MC1, MC2) deliver traffic to multiple receivers
- Custom traffic model parameters are applied correctly
- Measured bit rate matches expected bit rate (within 5% tolerance)
- Inter-frame duration matches expected value
- Average network latency matches specification (14.834ms)
- All transmitted bytes are received

To run the video traffic tests:

```
$ ./test.py -s applications-tgax-video-traffic
```

5.3.7 Verification

The video traffic model has been checked against the IEEE 802.11-14/0571r12 specification by comparing:

- All predefined model parameters (Weibull scale and shape) against Table 5 in the specification
- Network latency Gamma distribution parameters against the specification
- Measured bit rates against expected bit rates for each traffic model
- Frame size distribution against expected Weibull properties
- Network latency distribution against expected Gamma properties (mean ~14.8ms)

The test suite demonstrates that the model correctly implements the buffered video streaming traffic model, produces appropriate frame sizes, and generates traffic patterns consistent with real video streaming applications.

5.3.8 Video Conferencing Simulation

Although the Video Conferencing (VC) traffic model is not explicitly implemented as a separate class, users can simulate video conferencing scenarios by creating bidirectional BV1 streams. According to the specification:

- Traffic from AP to station: Use the BV1 model (with network latency)
- Traffic from station to AP: Use the BV1 model (without network latency, or set GammaScale to 0)

5.4 TGax Virtual Desktop Infrastructure Traffic Model

This section describes the implementation of the *ns-3* TGax Virtual Desktop Infrastructure (VDI) traffic generator model, which is based on the IEEE 802.11-14/0571r12 TGAX evaluation methodology [TGAX_EVAL]. The TGax VDI traffic model is implemented in the `src/applications/` folder.

Virtual Desktop Infrastructure (VDI) traffic is generated from a server and, in practice, may travel over multiple hops before arriving at an access point (AP) for transmission to stations (STAs). VDI traffic statistics are asymmetric (between downlink (server to client display) and uplink (client display to server)).

The model uses an exponential distribution for packet inter-arrival times and a normal distribution (including a bimodal distribution) for packet sizes, as specified in the IEEE 802.11-14/0571r12 evaluation methodology.

5.4.1 Scope and Limitations

The VDI model implements the traffic generation aspects of the IEEE 802.11-14/0571r12 specification. The following describes what is covered and what is not:

Supported Features:

- Exponential distribution for packet inter-arrival times
- Uniform distribution for initial packet arrival (0-20ms)
- Normal distribution for packet sizes
- Bimodal normal distribution support for downlink packet sizes
- TCP protocol (default, matching specification)
- Configurable parameters for both downlink and uplink scenarios
- Trace source for monitoring packet transmission

Known Limitations:

- (code review comment to resolve) Bimodal distribution mode selection uses equal probability instead of weighted selection (70.6%/29.4%). future versions.
- (code review comment to resolve) Each application instance is unidirectional; bidirectional VDI simulation requires two separate application instances and the uplink's parameters must be manually configured.

5.4.2 Model Description

The `TgaxVirtualDesktop` application is a source application that generates VDI traffic patterns according to the IEEE 802.11-14/0571r12 evaluation methodology.

Traffic Generation Process:

1. **Initial Packet Arrival:** The first packet is scheduled after a uniformly distributed delay in the range [0, 20ms], simulating random timing between client packet arrival and frame boundaries.
2. **Subsequent Packet Arrivals:** After the initial packet, inter-arrival times follow an exponential distribution with direction-specific mean values.
3. **Packet Size Generation:** Packet sizes are drawn from a normal distribution. For downlink traffic, a bimodal normal distribution is used to model the mix of small control packets and large display data packets.

Direction-Specific Parameters:

The specification defines different parameters for downlink and uplink traffic:

Downlink (AP to STA - Display Data):

Table 5.3: Downlink VDI Parameters

Parameter	Distribution	Value
Initial packet arrival	Uniform	[0, 20] ms
Packet inter-arrival	Exponential	Mean = 60.2269 ms
Packet size	Bimodal Normal	Mode 1: $\mu=41.0$, $\sigma=3.2$ (weight 53.7/76.1)
		Mode 2: $\mu=1478.3$, $\sigma=11.6$ (weight 22.4/76.1)

Uplink (STA to AP - Navigation/Feedback):

Table 5.4: Uplink VDI Parameters

Parameter	Distribution	Value
Initial packet arrival	Uniform	[0, 20] ms
Packet inter-arrival	Exponential	Mean = 48.2870 ms
Packet size	Normal	$\mu=50.598$, $\sigma=5.0753$

Configuration and Usage

The VDI traffic generator can be instantiated through the ApplicationHelper. The default configuration matches the downlink specification.

Downlink Configuration (Default):

```
// Create nodes
NodeContainer nodes;
nodes.Create(2);

// Install Internet stack and assign IP addresses
InternetStackHelper internet;
internet.Install(nodes);

Ipv4AddressHelper ipv4;
ipv4.SetBase("10.0.0.0", "255.255.255.0");
ipv4.Assign(devices);

// Configure VDI application (downlink - default parameters)
ApplicationHelper sourceHelper(TgaxVirtualDesktop::GetTypeId());
sourceHelper.SetAttribute("Remote", AddressValue(InetSocketAddress(Ipv4Address("10.0.0.2
↵"), 5000)));

auto sourceApps = sourceHelper.Install(nodes.Get(0));
sourceApps.Start(Seconds(1.0));
sourceApps.Stop(Seconds(11.0));

// Create receiver
PacketSinkHelper sinkHelper("ns3::TcpSocketFactory", ↵
↵ InetSocketAddress(Ipv4Address::GetAny(), 5000));
```

(continues on next page)

(continued from previous page)

```

auto sinkApps = sinkHelper.Install(nodes.Get(1));
sinkApps.Start(Seconds(0.0));
sinkApps.Stop(Seconds(12.0));

```

Uplink Configuration:

```

// Configure VDI application for uplink
ApplicationHelper sourceHelper(TgaxVirtualDesktop::GetTypeId());
sourceHelper.SetAttribute("Remote", AddressValue(InetSocketAddress(Ipv4Address("10.0.0.1
↔"), 5001)));

// Set uplink inter-arrival time (mean 48.287 ms)
auto ulInterArrival = CreateObjectWithAttributes<ExponentialRandomVariable>(
    "Mean", DoubleValue(48287000.0)); // nanoseconds
sourceHelper.SetAttribute("InterPacketArrivals", PointerValue(ulInterArrival));

// Set uplink packet size (normal distribution, mu=50.598, sigma=5.0753)
sourceHelper.SetAttribute("ParametersPacketSize", StringValue("50.598 5.0753"));

auto sourceApps = sourceHelper.Install(nodes.Get(1)); // STA node
sourceApps.Start(Seconds(1.0));
sourceApps.Stop(Seconds(11.0));

```

5.4.3 Attributes

The VDI traffic generator uses TCP by default.

The following attributes control the behavior of the VDI traffic generator:

Protocol Configuration:

- Protocol: The socket factory type to use. Default: ns3::TcpSocketFactory
- Remote: The remote address and port for the VDI destination

Timing Parameters:

- InitialPacketArrival: A uniform random variable for the initial packet arrival time. Default: Uniform ranging from 0 to 20 ms
- InterPacketArrivals: An exponential random variable for inter-packet arrival times. Default: Exponential with mean 60.2269ms (downlink specification)

Packet Size Parameters:

- ParametersPacketSize: The mean and standard deviation for each mode of the (potentially multimodal) normal distribution used to generate packet sizes. Format: “mean1 std1;mean2 std2” for bimodal, or “mean std” for unimodal. Default: “41.0 3.2;1478.3 11.6” (downlink bimodal)

5.4.4 Trace Sources

The VDI traffic generator provides a trace source for monitoring simulation behavior:

Tx:

```
typedef void (*TxTracedCallback)(Ptr<const Packet> packet);
```

This trace source is fired when a packet is transmitted. It provides the packet being transmitted. This is useful for analyzing packet transmission patterns and sizes.

5.4.5 Examples

The `tgax-virtual-desktop-example.cc` program demonstrates basic usage of the VDI traffic generator. This example sets up a simple Wi-Fi network (AP and STA) with VDI traffic. The example supports downlink, uplink, and bidirectional traffic configurations.

To run with downlink traffic (default):

```
$ ./ns3 run tgax-virtual-desktop-example
```

To run with uplink traffic:

```
$ ./ns3 run "tgax-virtual-desktop-example --direction=uplink"
```

To run with bidirectional traffic:

```
$ ./ns3 run "tgax-virtual-desktop-example --direction=bidirectional"
```

5.4.6 Tests

Unit tests for the VDI traffic model are provided in `src/applications/test/tgax-virtual-desktop-test-suite.cc`. These tests verify:

- Downlink VDI traffic with default parameters
- Uplink VDI traffic with custom parameters
- Initial packet arrival within expected bounds (0-20ms)
- Average packet inter-arrival time matches expectation
- Average packet size matches expectation
- All transmitted bytes are received

To run the VDI traffic tests:

```
$ ./test.py -s applications-tgax-virtual-desktop
```

5.4.7 Verification

The VDI traffic model has been checked against the IEEE 802.11-14/0571r12 specification by comparing:

- Initial packet arrival distribution (Uniform [0, 20ms])
- Packet inter-arrival time distribution (Exponential with direction-specific means)
- Packet size distribution parameters against specification

The test suite demonstrates that the model correctly implements the exponential inter-arrival behavior and normal packet size distributions.

5.5 RTA TIG Mobile Gaming Traffic Model

This section describes the implementation of the *ns-3* Real-Time Applications (RTA) Task Interest Group (TIG) mobile gaming traffic generator model, which is based on the IEEE 802.11-18/2009r6 RTA TIG Report [\[RTA-TIG\]](#).

The RTA TIG mobile gaming traffic model is implemented in the `src/applications/` folder.

Real-time mobile gaming is characterized by small packets (30-500 bytes) transmitted frequently (every 30-60ms) with strict latency requirements. The traffic is asymmetric and bidirectional, with different characteristics for uplink (player commands) and downlink (game state updates).

The model uses the Largest Extreme Value (LEV) distribution for packet inter-arrival times and packet sizes during the gaming stage, with uniform distributions for initial and ending packets.

5.5.1 Scope and Limitations

The mobile gaming model implements the traffic generation aspects of the IEEE 802.11-18/2009r6 specification (Section 4.1.4).

Supported Features:

- Two synchronization mechanisms: Status Sync and Frame Lockstep Sync
- Three-stage traffic model: Initial, Gaming, and Ending
- Largest Extreme Value distribution for packet arrivals and sizes
- Uniform distribution for initial and end packet sizes
- UDP protocol (default, matching specification)
- Configurable parameters for all four scenarios (DL/UL x Status-Sync/Lockstep)
- Trace source with stage information

Known Limitations:

- Each application instance is unidirectional; bidirectional gaming simulation requires two separate application instances
- No preset configuration enum; users must manually configure parameters for non-default scenarios

5.5.2 Model Description

The `RtaTigMobileGaming` application is a source application that generates real-time mobile gaming traffic patterns according to the IEEE 802.11-18/2009r6 RTA TIG Report.

Traffic Stages:

The model implements a three-stage traffic pattern:

1. **INITIAL Stage:** A single packet is sent with size drawn from a uniform distribution. This represents the initial game setup/synchronization packet.
2. **GAMING Stage:** Continuous packet transmission during active gameplay. Packet sizes and inter-arrival times are drawn from Largest Extreme Value distributions.
3. **ENDING Stage:** A single packet is sent when the application stops, with size drawn from a uniform distribution. This represents the game termination packet.

Synchronization Mechanisms:

The specification defines two traffic models for different game synchronization approaches:

Status Sync Model:

Used by games that synchronize game state between clients and server. Characterized by smaller, more frequent packets.

Table 5.5: Status Sync Traffic Parameters

Component	Distribution	DL (a, b)	UL (a, b)
Initial packet size	Uniform	[0, 20] bytes	[0, 20] bytes
Packet arrival time	Largest Extreme Value	a=13ms, b=3.7ms	a=15ms, b=5.7ms
Packet size	Largest Extreme Value	a=50, b=11 bytes	a=38, b=3.7 bytes
End packet size	Uniform	[500, 600] bytes	[400, 550] bytes

Frame Lockstep Sync Model:

Used by games that require frame-by-frame synchronization. Characterized by larger packets with slightly longer intervals.

Table 5.6: Frame Lockstep Sync Traffic Parameters

Component	Distribution	DL (a, b)	UL (a, b)
Initial packet size	Uniform	[0, 80] bytes	[0, 80] bytes
Packet arrival time	Largest Extreme Value	a=28ms, b=4.2ms	a=22ms, b=3.4ms
Packet size	Largest Extreme Value	a=210, b=35 bytes	a=92, b=38 bytes
End packet size	Uniform	[1400, 1500] bytes	[500, 600] bytes

In the Largest Extreme Value distribution, **a** is the location parameter and **b** is the scale parameter.

Configuration and Usage

The mobile gaming traffic generator can be instantiated through the ApplicationHelper. The default configuration matches the Status-Sync Downlink parameters.

Default Configuration (Status-Sync DL):

```
// Create nodes
NodeContainer nodes;
nodes.Create(2);

// Install Internet stack and assign IP addresses
InternetStackHelper internet;
internet.Install(nodes);

Ipv4AddressHelper ipv4;
ipv4.SetBase("10.0.0.0", "255.255.255.0");
ipv4.Assign(devices);

// Configure gaming application (defaults to status-sync DL)
ApplicationHelper sourceHelper(RtaTigMobileGaming::GetTypeId());
sourceHelper.SetAttribute("Remote", AddressValue(InetSocketAddress(Ipv4Address("10.0.0.2
↪"), 5000)));

auto sourceApps = sourceHelper.Install(nodes.Get(0));
sourceApps.Start(Seconds(1.0));
sourceApps.Stop(Seconds(11.0));

// Create receiver
PacketSinkHelper sinkHelper("ns3::UdpSocketFactory", ↪
↪InetSocketAddress(Ipv4Address::GetAny(), 5000));
auto sinkApps = sinkHelper.Install(nodes.Get(1));
sinkApps.Start(Seconds(0.0));
sinkApps.Stop(Seconds(12.0));
```

Lockstep DL Configuration:

Note: The below would be better configured via a mode attribute such as LOCKSTEP_DL.

```
ApplicationHelper sourceHelper(RtaTigMobileGaming::GetTypeId());
sourceHelper.SetAttribute("Remote", AddressValue(InetSocketAddress(Ipv4Address("10.0.0.2
↪"), 5000)));

// Configure lockstep DL parameters
auto ips = CreateObjectWithAttributes<UniformRandomVariable>(
    "Min", DoubleValue(0), "Max", DoubleValue(80));
sourceHelper.SetAttribute("InitialPacketSize", PointerValue(ips));

auto eps = CreateObjectWithAttributes<UniformRandomVariable>(
    "Min", DoubleValue(1400), "Max", DoubleValue(1500));
sourceHelper.SetAttribute("EndPacketSize", PointerValue(eps));

auto pal = CreateObjectWithAttributes<LargestExtremeValueRandomVariable>(
    "Location", DoubleValue(28000), // 28ms in microseconds
    "Scale", DoubleValue(4200)); // 4.2ms in microseconds
```

(continues on next page)

(continued from previous page)

```
sourceHelper.SetAttribute("PacketArrivalLev", PointerValue(pal));

auto psl = CreateObjectWithAttributes<LargestExtremeValueRandomVariable>(
    "Location", DoubleValue(210),
    "Scale", DoubleValue(35));
sourceHelper.SetAttribute("PacketSizeLev", PointerValue(psl));
```

5.5.3 Attributes

The mobile gaming traffic generator uses UDP by default, as specified for in-game data in the IEEE specification.

The following attributes control the behavior of the traffic generator:

Protocol Configuration:

- Protocol: The socket factory type to use. Default: ns3::UdpSocketFactory
- Remote: The remote address and port for the gaming traffic destination

Initial and End Packet Parameters:

- InitialPacketSize: A uniform random variable for initial packet size in bytes. Default: Uniform[0, 20]
- EndPacketSize: A uniform random variable for end packet size in bytes. Default: Uniform[500, 600]

Gaming Stage Parameters:

- PacketArrivalLev: A Largest Extreme Value random variable for packet inter-arrival times in microseconds. Default: LEV(Location=13000, Scale=3700) (13ms, 3.7ms)
- PacketSizeLev: A Largest Extreme Value random variable for gaming packet sizes in bytes. Default: LEV(Location=50, Scale=11)

5.5.4 Trace Sources

The mobile gaming traffic generator provides a trace source for monitoring simulation behavior:

TxWithStage:

```
typedef void (*TxTracedCallback)(Ptr<const Packet> packet, TrafficModelStage stage);
```

This trace source is fired when a packet is transmitted. It provides the packet and the current traffic model stage (INITIAL, GAMING, or ENDING). This is useful for analyzing traffic patterns and verifying stage transitions.

5.5.5 Examples

The rta-tig-mobile-gaming-example.cc program demonstrates usage of the mobile gaming traffic generator. This example sets up a simple Wi-Fi network (AP and STA) with gaming traffic. The example supports all four traffic model presets.

To run with Status-Sync Downlink (default):

```
$ ./ns3 run rta-tig-mobile-gaming-example
```

To run with Status-Sync Uplink:

```
$ ./ns3 run "rta-tig-mobile-gaming-example --model=status-sync-ul"
```

To run with Frame Lockstep Downlink:

```
$ ./ns3 run "rta-tig-mobile-gaming-example --model=lockstep-dl"
```

To run with Frame Lockstep Uplink:

```
$ ./ns3 run "rta-tig-mobile-gaming-example --model=lockstep-ul"
```

5.5.6 Tests

Unit tests for the mobile gaming traffic model are provided in `src/applications/test/rta-tig-mobile-gaming-test-suite.cc`. These tests verify:

- Correct stage transitions (INITIAL -> GAMING -> ENDING)
- Initial packet size within expected bounds
- End packet size within expected bounds
- Average gaming packet size matches LEV distribution mean
- Average packet inter-arrival time matches LEV distribution mean
- All transmitted bytes are received

To run the mobile gaming traffic tests:

```
$ ./test.py -s applications-rta-tig-mobile-gaming
```

5.5.7 Verification

The mobile gaming traffic model has been checked against the IEEE 802.11-18/2009r6 specification by comparing:

- Default parameter values against Table 4-3 (Status Sync) in the specification
- Largest Extreme Value distribution behavior for packet sizes and arrivals
- Stage transition logic

The test suite demonstrates that the model correctly implements the three-stage traffic pattern and produces traffic characteristics consistent with real-time mobile gaming applications.

5.5.8 References

BRIDGE NETDEVICE

Placeholder chapter

Some examples of the use of Bridge NetDevice can be found in `examples/csma/` directory.

BRITE INTEGRATION

This model implements an interface to BRITE, the Boston university Representative Internet Topology generator¹. BRITE is a standard tool for generating realistic internet topologies. The ns-3 model, described herein, provides a helper class to facilitate generating ns-3 specific topologies using BRITE configuration files. BRITE builds the original graph which is stored as nodes and edges in the ns-3 BriteTopologyHelper class. In the ns-3 integration of BRITE, the generator generates a topology and then provides access to leaf nodes for each AS generated. ns-3 users can then attach custom topologies to these leaf nodes either by creating them manually or using topology generators provided in ns-3.

There are three major types of topologies available in BRITE: Router, AS, and Hierarchical which is a combination of AS and Router. For the purposes of ns-3 simulation, the most useful are likely to be Router and Hierarchical. Router level topologies be generated using either the Waxman model or the Barabasi-Albert model. Each model has different parameters that effect topology creation. For flat router topologies, all nodes are considered to be in the same AS.

BRITE Hierarchical topologies contain two levels. The first is the AS level. This level can be also be created by using either the Waxman model or the Barabasi-Albert model. Then for each node in the AS topology, a router level topology is constructed. These router level topologies can again either use the Waxman model or the Barabasi-Albert model. BRITE interconnects these separate router topologies as specified by the AS level topology. Once the hierarchical topology is constructed, it is flattened into a large router level topology.

Further information can be found in the BRITE user manual: <http://www.cs.bu.edu/brite/publications/usermanual.pdf>

7.1 Model Description

The model relies on building an external BRITE library, and then building some ns-3 helpers that call out to the library. The source code for the ns-3 helpers lives in the directory `src/brite/helper`.

7.1.1 Design

To generate the BRITE topology, ns-3 helpers call out to the external BRITE library, and using a standard BRITE configuration file, the BRITE code builds a graph with nodes and edges according to this configuration file. Please see the BRITE documentation or the example configuration files in `src/brite/examples/conf_files` to get a better grasp of BRITE configuration options. The graph built by BRITE is returned to ns-3, and a ns-3 implementation of the graph is built. Leaf nodes for each AS are available for the user to either attach custom topologies or install ns-3 applications directly.

¹ Alberto Medina, Anukool Lakhina, Ibrahim Matta, and John Byers. BRITE: An Approach to Universal Topology Generation. In Proceedings of the International Workshop on Modeling, Analysis and Simulation of Computer and Telecommunications Systems- MASCOTS '01, Cincinnati, Ohio, August 2001.

7.1.2 References

7.2 Usage

The brite-generic-example can be referenced to see basic usage of the BRITE interface. In summary, the BriteTopologyHelper is used as the interface point by passing in a BRITE configuration file. Along with the configuration file a BRITE formatted random seed file can also be passed in. If a seed file is not passed in, the helper will create a seed file using ns-3's UniformRandomVariable. Once the topology has been generated by BRITE, BuildBriteTopology() is called to create the ns-3 representation. Next IP Address can be assigned to the topology using either AssignIpv4Addresses() or AssignIpv6Addresses(). It should be noted that each point-to-point link in the topology will be treated as a new network therefore for IPV4 a /30 subnet should be used to avoid wasting a large amount of the available address space.

Example BRITE configuration files can be found in /src/brite/examples/conf_files/. ASBarabasi and ASWaxman are examples of AS only topologies. The RTBarabasi and RTWaxman files are examples of router only topologies. Finally the TD_ASBarabasi_RTWaxman configuration file is an example of a Hierarchical topology that uses the Barabasi-Albert model for the AS level and the Waxman model for each of the router level topologies. Information on the BRITE parameters used in these files can be found in the BRITE user manual.

7.2.1 Building BRITE Integration

The first step is to download and build the ns-3 specific BRITE repository:

```
$ git clone https://gitlab.com/nsnam/BRITE.git
$ cd BRITE
$ mkdir build && cd build
$ cmake ..
$ make
```

This will build a library, libbrite.so, within the BRITE/build subdirectory.

Once BRITE has been built successfully, we proceed to configure ns-3 with BRITE support. Change to your ns-3 directory:

```
$ ./ns3 configure --with-brite=/your/path/to/BRITE --enable-examples
```

Make sure it says 'enabled' beside 'BRITE Integration'. If it does not, then something has gone wrong. Either you have forgotten to build BRITE first following the steps above, or ns-3 could not find your BRITE directory.

Next, build ns-3:

```
$ ./ns3
```

7.2.2 Examples

For an example demonstrating BRITE integration run:

```
$ ./ns3 run 'brite-generic-example'
```

By enabling the verbose parameter, the example will print out the node and edge information in a similar format to standard BRITE output. There are many other command-line parameters including confFile, tracing, and nix, described below:

confFile

A BRITE configuration file. Many different BRITE configuration file examples exist in the `src/brite/examples/conf_files` directory, for example, `RTBarabasi20.conf` and `RTWaxman.conf`. Please refer to the `conf_files` directory for more examples.

tracing

Enables ascii tracing.

nix

Enables nix-vector routing. Global routing is used by default.

The generic BRITE example also support visualization using pyviz, assuming python bindings in ns-3 are enabled:

```
$ ./ns3 run brite-generic-example --vis
```

Simulations involving BRITE can also be used with MPI. The total number of MPI instances is passed to the BRITE topology helper where a modulo divide is used to assign the nodes for each AS to a MPI instance. An example can be found in `src/brite/examples`:

```
$ mpirun -np 2 ./ns3 run brite-MPI-example
```

Please see the ns-3 MPI documentation for information on setting up MPI with ns-3.

BUILDINGS MODULE

```
cd .. include:: replace.txt
```

8.1 Design documentation

8.1.1 Overview

The Buildings module provides:

1. a new class (`Building`) that models the presence of a building in a simulation scenario;
2. a new class (`MobilityBuildingInfo`) that allows to specify the location, size and characteristics of buildings present in the simulated area, and allows the placement of nodes inside those buildings;
3. a container class with the definition of the most useful pathloss models and the correspondent variables called `BuildingsPropagationLossModel`.
4. a new propagation model (`HybridBuildingsPropagationLossModel`) working with the mobility model just introduced, that allows to model the phenomenon of indoor/outdoor propagation in the presence of buildings.
5. a simplified model working only with Okumura Hata (`OhBuildingsPropagationLossModel`) considering the phenomenon of indoor/outdoor propagation in the presence of buildings.
6. a channel condition model (`BuildingsChannelConditionModel`) which determined the LOS/NLOS channel condition based on the `Building` objects deployed in the scenario.
7. hybrid channel condition models (`ThreeGppV2vUrbanChannelConditionModel` and `ThreeGppV2vHighwayChannelConditionModel`) specifically designed to model vehicular environments (more information can be found in the [documentation of the propagation module](#))

The models have been designed with LTE in mind, though their implementation is in fact independent from any LTE-specific code, and can be used with other ns-3 wireless technologies as well (e.g., wifi).

The `HybridBuildingsPropagationLossModel` pathloss model included is obtained through a combination of several well known pathloss models in order to mimic different environmental scenarios such as urban, suburban and open areas. Moreover, the model considers both outdoor and indoor indoor and outdoor communication has to be included since HeNB might be installed either within building and either outside. In case of indoor communication, the model has to consider also the type of building in outdoor <-> indoor communication according to some general criteria such as the wall penetration losses of the common materials; moreover it includes some general configuration for the internal walls in indoor communications.

The `OhBuildingsPropagationLossModel` pathloss model has been created for simplifying the previous one removing the thresholds for switching from one model to other. For doing this it has been used only one propagation model from the one available (i.e., the Okumura Hata). The presence of building is still considered in the model; therefore all

the considerations of above regarding the building type are still valid. The same consideration can be done for what concern the environmental scenario and frequency since both of them are parameters of the model considered.

8.1.2 The Building class

The model includes a specific class called **Building** which contains a ns3 **Box** class for defining the dimension of the building. In order to implements the characteristics of the pathloss models included, the **Building** class supports the following attributes:

- building type:
 - Residential (default value)
 - Office
 - Commercial
- external walls type
 - Wood
 - ConcreteWithWindows (default value)
 - ConcreteWithoutWindows
 - StoneBlocks
- number of floors (default value 1, which means only ground-floor)
- number of rooms in x-axis (default value 1)
- number of rooms in y-axis (default value 1)

The **Building** class is based on the following assumptions:

- a buildings is represented as a rectangular parallelepiped (i.e., a box)
- the walls are parallel to the x, y, and z axis
- a building is divided into a grid of rooms, identified by the following parameters:
 - number of floors
 - number of rooms along the x-axis
 - number of rooms along the y-axis
- the z axis is the vertical axis, i.e., floor numbers increase for increasing z axis values
- the x and y room indices start from 1 and increase along the x and y axis respectively
- all rooms in a building have equal size

8.1.3 The MobilityBuildingInfo class

The **MobilityBuildingInfo** class, which inherits from the ns3 class **Object**, is in charge of maintaining information about the position of a node with respect to building. The information managed by **MobilityBuildingInfo** is:

- whether the node is indoor or outdoor
- if indoor:
 - in which building the node is
 - in which room the node is positioned (x, y and floor room indices)

The class `MobilityBuildingInfo` is used by `BuildingsPropagationLossModel` class, which inherits from the ns3 class `PropagationLossModel` and manages the pathloss computation of the single components and their composition according to the nodes' positions. Moreover, it implements also the shadowing, that is the loss due to obstacles in the main path (i.e., vegetation, buildings, etc.).

It is to be noted that, `MobilityBuildingInfo` can be used by any other propagation model. However, based on the information at the time of this writing, only the ones defined in the building module are designed for considering the constraints introduced by the buildings.

8.1.4 `ItuR1238PropagationLossModel`

This class implements a building-dependent indoor propagation loss model based on the ITU P.1238 model, which includes losses due to type of building (i.e., residential, office and commercial). The analytical expression is given in the following.

$$L_{\text{total}} = 20 \log f + N \log d + L_f(n) - 28[\text{dB}]$$

where:

$$N = \begin{cases} 28 & \text{residential} \\ 30 & \text{office} \\ 22 & \text{commercial} \end{cases} : \text{power loss coefficient [dB]}$$

$$L_f = \begin{cases} 4n & \text{residential} \\ 15 + 4(n - 1) & \text{office} \\ 6 + 3(n - 1) & \text{commercial} \end{cases}$$

n : number of floors between base station and mobile ($n \geq 1$)

f : frequency [MHz]

d : distance (where $d > 1$) [m]

8.1.5 `BuildingsPropagationLossModel`

The `BuildingsPropagationLossModel` provides an additional set of building-dependent pathloss model elements that are used to implement different pathloss logics. These pathloss model elements are described in the following subsections.

External Wall Loss (EWL)

This component models the penetration loss through walls for indoor to outdoor communications and vice-versa. The values are taken from the COST231 model.

- Wood ~ 4 dB
- Concrete with windows (not metallized) ~ 7 dB
- Concrete without windows ~ 15 dB (spans between 10 and 20 in COST231)
- Stone blocks ~ 12 dB

Internal Walls Loss (IWL)

This component models the penetration loss occurring in indoor-to-indoor communications within the same building. The total loss is calculated assuming that each single internal wall has a constant penetration loss L_{siw} , and approximating the number of walls that are penetrated with the manhattan distance (in number of rooms) between the transmitter and the receiver. In detail, let x_1, y_1, x_2, y_2 denote the room number along the x and y axis respectively for user 1 and 2; the total loss L_{IWL} is calculated as

$$L_{IWL} = L_{siw}(|x_1 - x_2| + |y_1 - y_2|)$$

Height Gain Model (HG)

This component model the gain due to the fact that the transmitting device is on a floor above the ground. In the literature [turkmani] this gain has been evaluated as about 2 dB per floor. This gain can be applied to all the indoor to outdoor communications and vice-versa.

Shadowing Model

The shadowing is modeled according to a log-normal distribution with variable standard deviation as function of the relative position (indoor or outdoor) of the MobilityModel instances involved. One random value is drawn for each pair of MobilityModels, and stays constant for that pair during the whole simulation. Thus, the model is appropriate for static nodes only.

The model considers that the mean of the shadowing loss in dB is always 0. For the variance, the model considers three possible values of standard deviation, in detail:

- outdoor (`m_shadowingSigmaOutdoor`, default value of 7 dB) $\rightarrow X_O \sim N(\mu_O, \sigma_O^2)$.
- indoor (`m_shadowingSigmaIndoor`, default value of 10 dB) $\rightarrow X_I \sim N(\mu_I, \sigma_I^2)$.
- external walls penetration (`m_shadowingSigmaExtWalls`, default value 5 dB) $\rightarrow X_W \sim N(\mu_W, \sigma_W^2)$

The simulator generates a shadowing value per each active link according to nodes' position the first time the link is used for transmitting. In case of transmissions from outdoor nodes to indoor ones, and vice-versa, the standard deviation (σ_{IO}) has to be calculated as the square root of the sum of the quadratic values of the standard deviation in case of outdoor nodes and the one for the external walls penetration. This is due to the fact that the components producing the shadowing are independent of each other; therefore, the variance of a distribution resulting from the sum of two independent normal ones is the sum of the variances.

$$\begin{aligned} X &\sim N(\mu, \sigma^2) \text{ and } Y \sim N(\nu, \tau^2) \\ Z = X + Y &\sim N(\mu + \nu, \sigma^2 + \tau^2) \\ \Rightarrow \sigma_{IO} &= \sqrt{\sigma_O^2 + \sigma_W^2} \end{aligned}$$

8.1.6 Pathloss logics

In the following we describe the different pathloss logic that are implemented by inheriting from `BuildingsPropagationLossModel`.

HybridBuildingsPropagationLossModel

The HybridBuildingsPropagationLossModel pathloss model included is obtained through a combination of several well known pathloss models in order to mimic different outdoor and indoor scenarios, as well as indoor-to-outdoor and outdoor-to-indoor scenarios. In detail, the class HybridBuildingsPropagationLossModel integrates the following pathloss models:

- OkumuraHataPropagationLossModel (OH) (at frequencies > 2.3 GHz substituted by Kun2600MhzPropagationLossModel)
- ItuR1411LosPropagationLossModel and ItuR1411NlosOverRooftopPropagationLossModel (I1411)
- ItuR1238PropagationLossModel (I1238)
- the pathloss elements of the BuildingsPropagationLossModel (EWL, HG, IWL)

The following pseudo-code illustrates how the different pathloss model elements described above are integrated in HybridBuildingsPropagationLossModel:

```

if (txNode is outdoor)
  then
    if (rxNode is outdoor)
      then
        if (distance > 1 km)
          then
            if (rxNode or txNode is below the rooftop)
              then
                L = I1411
            else
                L = OH
          else
            L = I1411
        else (rxNode is indoor)
          if (distance > 1 km)
            then
              if (rxNode or txNode is below the rooftop)
                L = I1411 + EWL + HG
              else
                L = OH + EWL + HG
            else
              L = I1411 + EWL + HG
      else (txNode is indoor)
        if (rxNode is indoor)
          then
            if (same building)
              then
                L = I1238 + IWL
            else
                L = I1411 + 2*EWL
          else (rxNode is outdoor)
            if (distance > 1 km)
              then
                if (rxNode or txNode is below the rooftop)
                  then
                    L = I1411 + EWL + HG
                else

```

(continues on next page)

(continued from previous page)

```

                L = OH + EWL + HG
else
    L = I1411 + EWL

```

We note that, for the case of communication between two nodes below rooftop level with distance is greater than 1 km, we still consider the I1411 model, since OH is specifically designed for macro cells and therefore for antennas above the roof-top level.

For the ITU-R P.1411 model we consider both the LOS and NLoS versions. In particular, we consider the LoS propagation for distances that are shorter than a tunable threshold (`m_itu1411NlosThreshold`). In case on NLoS propagation, the over the roof-top model is taken in consideration for modeling both macro BS and SC. In case on NLoS several parameters scenario dependent have been included, such as average street width, orientation, etc. The values of such parameters have to be properly set according to the scenario implemented, the model does not calculate natively their values. In case any values is provided, the standard ones are used, apart for the height of the mobile and BS, which instead their integrity is tested directly in the code (i.e., they have to be greater than zero). In the following we give the expressions of the components of the model.

We also note that the use of different propagation models (OH, I1411, I1238 with their variants) in `HybridBuildingsPropagationLossModel` can result in discontinuities of the pathloss with respect to distance. A proper tuning of the attributes (especially the distance threshold attributes) can avoid these discontinuities. However, since the behavior of each model depends on several other parameters (frequency, node height, etc), there is no default value of these thresholds that can avoid the discontinuities in all possible configurations. Hence, an appropriate tuning of these parameters is left to the user.

OhBuildingsPropagationLossModel

The `OhBuildingsPropagationLossModel` class has been created as a simple means to solve the discontinuity problems of `HybridBuildingsPropagationLossModel` without doing scenario-specific parameter tuning. The solution is to use only one propagation loss model (i.e., Okumura Hata), while retaining the structure of the pathloss logic for the calculation of other path loss components (such as wall penetration losses). The result is a model that is free of discontinuities (except those due to walls), but that is less realistic overall for a generic scenario with buildings and outdoor/indoor users, e.g., because Okumura Hata is not suitable neither for indoor communications nor for outdoor communications below rooftop level.

In detail, the class `OhBuildingsPropagationLossModel` integrates the following pathloss models:

- `OkumuraHataPropagationLossModel` (OH)
- the pathloss elements of the `BuildingsPropagationLossModel` (EWL, HG, IWL)

The following pseudo-code illustrates how the different pathloss model elements described above are integrated in `OhBuildingsPropagationLossModel`:

```

if (txNode is outdoor)
then
    if (rxNode is outdoor)
    then
        L = OH
    else (rxNode is indoor)
        L = OH + EWL
else (txNode is indoor)
    if (rxNode is indoor)
    then
        if (same building)

```

(continues on next page)

(continued from previous page)

```

    then
        L = OH + IWL
    else
        L = OH + 2*EWL
    else (rxNode is outdoor)
        L = OH + EWL

```

We note that OhBuildingsPropagationLossModel is a significant simplification with respect to HybridBuildingsPropagationLossModel, due to the fact that OH is used always. While this gives a less accurate model in some scenarios (especially below rooftop and indoor), it effectively avoids the issue of pathloss discontinuities that affects HybridBuildingsPropagationLossModel.

8.2 User Documentation

8.2.1 How to use buildings in a simulation

In this section we explain the basic usage of the buildings model within a simulation program.

Include the headers

Add this at the beginning of your simulation program:

```
#include "ns3/buildings-module.h"
```

Create a building

As an example, let's create a residential 10 x 20 x 10 building:

```

double x_min = 0.0;
double x_max = 10.0;
double y_min = 0.0;
double y_max = 20.0;
double z_min = 0.0;
double z_max = 10.0;
Ptr<Building> b = CreateObject<Building>();
b->SetBoundaries(Box(x_min, x_max, y_min, y_max, z_min, z_max));
b->SetBuildingType(Building::Residential);
b->SetExtWallsType(Building::ConcreteWithWindows);
b->SetNFloors(3);
b->SetNRoomsX(3);
b->SetNRoomsY(2);

```

This building has three floors and an internal 3 x 2 grid of rooms of equal size.

The helper class GridBuildingAllocator is also available to easily create a set of buildings with identical characteristics placed on a rectangular grid. Here's an example of how to use it:

```

Ptr<GridBuildingAllocator> gridBuildingAllocator;
gridBuildingAllocator = CreateObject<GridBuildingAllocator>();

```

(continues on next page)

(continued from previous page)

```

gridBuildingAllocator->SetAttribute("GridWidth", IntegerValue(3));
gridBuildingAllocator->SetAttribute("LengthX", DoubleValue(7));
gridBuildingAllocator->SetAttribute("LengthY", DoubleValue(13));
gridBuildingAllocator->SetAttribute("DeltaX", DoubleValue(3));
gridBuildingAllocator->SetAttribute("DeltaY", DoubleValue(3));
gridBuildingAllocator->SetAttribute("Height", DoubleValue(6));
gridBuildingAllocator->SetBuildingAttribute("NRoomsX", IntegerValue(2));
gridBuildingAllocator->SetBuildingAttribute("NRoomsY", IntegerValue(4));
gridBuildingAllocator->SetBuildingAttribute("NFloors", IntegerValue(2));
gridBuildingAllocator->SetAttribute("MinX", DoubleValue(0));
gridBuildingAllocator->SetAttribute("MinY", DoubleValue(0));
gridBuildingAllocator->Create(6);

```

This will create a 3x2 grid of 6 buildings, each 7 x 13 x 6 m with 2 x 4 rooms inside and 2 floors; the buildings are spaced by 3 m on both the x and the y axis.

Setup nodes and mobility models

Nodes and mobility models are configured as usual, however in order to use them with the buildings model you need an additional call to `BuildingsHelper::Install()`, so as to let the mobility model include the information on their position w.r.t. the buildings. Here is an example:

```

MobilityHelper mobility;
mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
ueNodes.Create(2);
mobility.Install(ueNodes);
BuildingsHelper::Install(ueNodes);

```

It is to be noted that any mobility model can be used. However, the user is advised to make sure that the behavior of the mobility model being used is consistent with the presence of Buildings. For example, using a simple random mobility over the whole simulation area in presence of buildings might easily results in node moving in and out of buildings, regardless of the presence of walls.

One dedicated buildings-aware mobility model is the `RandomWalk2dOutdoorMobilityModel`. This class is similar to the `RandomWalk2dMobilityModel` but avoids placing the trajectory on a path that would intersect a building wall. If a boundary is encountered (either the bounding box or a building wall), the model rebounds with a random direction and speed that ensures that the trajectory stays outside the buildings. An example program that demonstrates the use of this model is the `src/buildings/examples/outdoor-random-walk-example.cc` which has an associated shell script to plot the traces generated. Another example program demonstrates how this outdoor mobility model can be used as the basis of a group mobility model, with the outdoor buildings-aware model serving as the parent or reference mobility model, and with additional nodes defining a child mobility model providing the offset from the reference mobility model. This example, `src/buildings/example/outdoor-group-mobility-example.cc`, also has an associated shell script (`outdoor-group-mobility-animate.sh`) that can be used to generate an animated GIF of the group's movement.

Place some nodes

You can place nodes in your simulation using several methods, which are described in the following.

Legacy positioning methods

Any legacy ns-3 positioning method can be used to place node in the simulation. The important additional step is to For example, you can place nodes manually like this:

```
Ptr<ConstantPositionMobilityModel> mm0 = enbNodes.Get(0)->GetObject
<<ConstantPositionMobilityModel>();
Ptr<ConstantPositionMobilityModel> mm1 = enbNodes.Get(1)->GetObject
<<ConstantPositionMobilityModel>();
mm0->SetPosition(Vector(5.0, 5.0, 1.5));
mm1->SetPosition(Vector(30.0, 40.0, 1.5));

MobilityHelper mobility;
mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
ueNodes.Create(2);
mobility.Install(ueNodes);
BuildingsHelper::Install(ueNodes);
mm0->SetPosition(Vector(5.0, 5.0, 1.5));
mm1->SetPosition(Vector(30.0, 40.0, 1.5));
```

Alternatively, you could use any existing `PositionAllocator` class. The coordinates of the node will determine whether it is placed outdoor or indoor and, if indoor, in which building and room it is placed.

Building-specific positioning methods

The following position allocator classes are available to place node in special positions with respect to buildings:

- `RandomBuildingPositionAllocator`: Allocate each position by randomly choosing a building from the list of all buildings, and then randomly choosing a position inside the building.
- `RandomRoomPositionAllocator`: Allocate each position by randomly choosing a room from the list of rooms in all buildings, and then randomly choosing a position inside the room.
- `SameRoomPositionAllocator`: Walks a given `NodeContainer` sequentially, and for each node allocate a new position randomly in the same room of that node.
- `FixedRoomPositionAllocator`: Generate a random position uniformly distributed in the volume of a chosen room inside a chosen building.
- `OutdoorPositionAllocator`: Allocate positions outside of existing buildings using rejection sampling.

Making the Mobility Model Consistent for a node

Initially, a mobility model of a node is made consistent when a node is initialized, which eventually triggers a call to the `DoInitialize` method of the *MobilityBuildingInfo* class. In particular, it calls the `MakeMobilityModelConsistent` method, which goes through the lists of all buildings, determine if the node is indoor or outdoor, and if indoor it also determines the building in which the node is located and the corresponding floor number inside the building. Moreover, this method also caches the position of the node, which is used to make the mobility model consistent for a moving node whenever the `IsInside` method of *MobilityBuildingInfo* class is called.

Building-aware pathloss model

After you placed buildings and nodes in a simulation, you can use a building-aware pathloss model in a simulation exactly in the same way you would use any regular path loss model. How to do this is specific for the wireless module that you are considering (lte, wifi, etc.), so please refer to the documentation of that model for specific instructions.

Building-aware channel condition models

The class *BuildingsChannelConditionModel* implements a [channel condition model](#) which determines the LOS/NLOS channel state based on the buildings deployed in the scenario. In addition, based on the wall material of the building, low/high building penetration losses are considered, as defined in 3GPP TS 38.901 7.4.3.1. In particular, for O2I condition, in case of Wood or ConcreteWithWindows material, low losses are considered in the pathloss calculation. In case the material has been set to ConcreteWithoutWindows or StoneBlocks, high losses are considered. Notice that in certain corner cases, such as the I2O2I interference, the model underestimates losses by applying either low or high losses based on the wall material of the involved nodes. For a more accurate estimation the model can be further extended.

The classes *ThreeGppV2vUrbanChannelConditionModel* and *ThreeGppV2vHighwayChannelConditionModel* implement hybrid channel condition models, specifically designed to model vehicular environments. More information can be found in the [documentation of the propagation module](#).

8.2.2 Main configurable attributes

The *Building* class has the following configurable parameters:

- building type: Residential, Office and Commercial.
- external walls type: Wood, ConcreteWithWindows, ConcreteWithoutWindows and StoneBlocks.
- building bounds: a *Box* class with the building bounds.
- number of floors.
- number of rooms in x-axis and y-axis (rooms can be placed only in a grid way).

The *BuildingMobilityLossModel* parameter configurable with the ns3 attribute system is represented by the bound (string Bounds) of the simulation area by providing a *Box* class with the area bounds. Moreover, by means of its methods the following parameters can be configured:

- the number of floor the node is placed (default 0).
- the position in the rooms grid.

The *BuildingPropagationLossModel* class has the following configurable parameters configurable with the attribute system:

- Frequency: reference frequency (default 2160 MHz), note that by setting the frequency the wavelength is set accordingly automatically and vice-versa).

- **Lambda**: the wavelength (0.139 meters, considering the above frequency).
- **ShadowSigmaOutdoor**: the standard deviation of the shadowing for outdoor nodes (default 7.0).
- **ShadowSigmaIndoor**: the standard deviation of the shadowing for indoor nodes (default 8.0).
- **ShadowSigmaExtWalls**: the standard deviation of the shadowing due to external walls penetration for outdoor to indoor communications (default 5.0).
- **RooftopLevel**: the level of the rooftop of the building in meters (default 20 meters).
- **Los2NlosThr**: the value of distance of the switching point between line-of-sight and non-line-of-sight propagation model in meters (default 200 meters).
- **ITU1411DistanceThr**: the value of distance of the switching point between short range (ITU 1211) communications and long range (Okumura Hata) in meters (default 200 meters).
- **MinDistance**: the minimum distance in meters between two nodes for evaluating the pathloss (considered negligible before this threshold) (default 0.5 meters).
- **Environment**: the environment scenario among Urban, SubUrban and OpenAreas (default Urban).
- **CitySize**: the dimension of the city among Small, Medium, Large (default Large).

In order to use the hybrid mode, the class to be used is the `HybridBuildingMobilityLossModel`, which allows the selection of the proper pathloss model according to the pathloss logic presented in the design chapter. However, this solution has the problem that the pathloss model switching points might present discontinuities due to the different characteristics of the model. This implies that according to the specific scenario, the threshold used for switching have to be properly tuned. The simple `OhBuildingMobilityLossModel` overcome this problem by using only the Okumura Hata model and the wall penetration losses.

8.3 Testing Documentation

8.3.1 Overview

To test and validate the ns-3 Building Pathloss module, some test suites is provided which are integrated with the ns-3 test framework. To run them, you need to have configured the build of the simulator in this way:

```
$ ./ns3 configure --enable-tests --enable-modules=buildings
$ ./test.py
```

The above will run not only the test suites belonging to the buildings module, but also those belonging to all the other ns-3 modules on which the buildings module depends. See the ns-3 manual for generic information on the testing framework.

You can get a more detailed report in HTML format in this way:

```
$ ./test.py -w results.html
```

After the above command has run, you can view the detailed result for each test by opening the file `results.html` with a web browser.

You can run each test suite separately using this command:

```
$ ./test.py -s test-suite-name
```

For more details about `test.py` and the ns-3 testing framework, please refer to the ns-3 manual.

8.3.2 Description of the test suites

BuildingsHelper test

The test suite `buildings-helper` checks that the method `BuildingsHelper::MakeAllInstancesConsistent ()` works properly, i.e., that the `BuildingsHelper` is successful in locating if nodes are outdoor or indoor, and if indoor that they are located in the correct building, room and floor. Several test cases are provided with different buildings (having different size, position, rooms and floors) and different node positions. The test passes if each every node is located correctly.

BuildingPositionAllocator test

The test suite `building-position-allocator` feature two test cases that check that respectively `RandomRoomPositionAllocator` and `SameRoomPositionAllocator` work properly. Each test cases involves a single 2x3x2 room building (total 12 rooms) at known coordinates and respectively 24 and 48 nodes. Both tests check that the number of nodes allocated in each room is the expected one and that the position of the nodes is also correct.

Buildings Pathloss tests

The test suite `buildings-pathloss-model` provides different unit tests that compare the expected results of the buildings pathloss module in specific scenarios with pre calculated values obtained offline with an Octave script (`test/reference/buildings-pathloss.m`). The tests are considered passed if the two values are equal up to a tolerance of 0.1, which is deemed appropriate for the typical usage of pathloss values (which are in dB).

In the following we detailed the scenarios considered, their selection has been done for covering the wide set of possible pathloss logic combinations. The pathloss logic results therefore implicitly tested.

Test #1 Okumura Hata

In this test we test the standard Okumura Hata model; therefore both eNB and UE are placed outside at a distance of 2000 m. The frequency used is the E-UTRA band #5, which correspond to 869 MHz (see table 5.5-1 of 36.101). The test includes also the validation of the areas extensions (i.e., urban, suburban and open-areas) and of the city size (small, medium and large).

Test #2 COST231 Model

This test is aimed at validating the COST231 model. The test is similar to the Okumura Hata one, except that the frequency used is the EUTRA band #1 (2140 MHz) and that the test can be performed only for large and small cities in urban scenarios due to model limitations.

Test #3 2.6 GHz model

This test validates the 2.6 GHz Kun model. The test is similar to Okumura Hata one except that the frequency is the EUTRA band #7 (2620 MHz) and the test can be performed only in urban scenario.

Test #4 ITU1411 LoS model

This test is aimed at validating the ITU1411 model in case of line of sight within street canyons transmissions. In this case the UE is placed at 100 meters far from the eNB, since the threshold for switching between LoS and NLoS is left to default one (i.e., 200 m.).

Test #5 ITU1411 NLoS model

This test is aimed at validating the ITU1411 model in case of non line of sight over the rooftop transmissions. In this case the UE is placed at 900 meters far from the eNB, in order to be above the threshold for switching between LoS and NLoS is left to default one (i.e., 200 m.).

Test #6 ITUP1238 model

This test is aimed at validating the ITUP1238 model in case of indoor transmissions. In this case both the UE and the eNB are placed in a residential building with walls made of concrete with windows. Ue is placed at the second floor and distances 30 meters far from the eNB, which is placed at the first floor.

Test #7 Outdoor -> Indoor with Okumura Hata model

This test validates the outdoor to indoor transmissions for large distances. In this case the UE is placed in a residential building with wall made of concrete with windows and distances 2000 meters from the outdoor eNB.

Test #8 Outdoor -> Indoor with ITU1411 model

This test validates the outdoor to indoor transmissions for short distances. In this case the UE is placed in a residential building with walls made of concrete with windows and distances 100 meters from the outdoor eNB.

Test #9 Indoor -> Outdoor with ITU1411 model

This test validates the outdoor to indoor transmissions for very short distances. In this case the eNB is placed in the second floor of a residential building with walls made of concrete with windows and distances 100 meters from the outdoor UE (i.e., LoS communication). Therefore the height gain has to be included in the pathloss evaluation.

Test #10 Indoor -> Outdoor with ITU1411 model

This test validates the outdoor to indoor transmissions for short distances. In this case the eNB is placed in the second floor of a residential building with walls made of concrete with windows and distances 500 meters from the outdoor UE (i.e., NLoS communication). Therefore the height gain has to be included in the pathloss evaluation.

Buildings Shadowing Test

The test suite `buildings-shadowing-test` is a unit test intended to verify the statistical distribution of the shadowing model implemented by `BuildingsPathlossModel`. The shadowing is modeled according to a normal distribution with mean $\mu = 0$ and variable standard deviation σ , according to models commonly used in literature. Three test cases are provided, which cover the cases of indoor, outdoor and indoor-to-outdoor communications. Each test case generates 1000 different samples of shadowing for different pairs of `MobilityModel` instances in a given scenario. Shadowing values are obtained by subtracting from the total loss value returned by `HybridBuildingsPathlossModel` the path loss component which is constant and pre-determined for each test case. The test verifies that the sample mean and sample variance of the shadowing values fall within the 99% confidence interval of the sample mean and sample variance. The test also verifies that the shadowing values returned at successive times for the same pair of `MobilityModel` instances is constant.

Buildings Channel Condition Model Test

The `BuildingsChannelConditionModelTestSuite` tests the class `BuildingsChannelConditionModel`. It checks if the channel condition between two nodes is correctly determined when a building is deployed.

8.4 References

CLICK MODULAR ROUTER INTEGRATION

Click is a software architecture for building configurable routers. By using different combinations of packet processing units called elements, a Click router can be made to perform a specific kind of functionality. This flexibility provides a good platform for testing and experimenting with different protocols.

9.1 Model Description

The source code for the Click model lives in the directory `src/click`.

9.1.1 Design

ns-3's design is well suited for an integration with Click due to the following reasons:

- Packets in ns-3 are serialised/deserialised as they move up/down the stack. This allows ns-3 packets to be passed to and from Click as they are.
- This also means that any kind of ns-3 traffic generator and transport should work easily on top of Click.
- By striving to implement click as an `Ipv4RoutingProtocol` instance, we can avoid significant changes to the LL and MAC layer of the ns-3 code.

The design goal was to make the ns-3-click public API simple enough such that the user needs to merely add an `Ipv4ClickRouting` instance to the node, and inform each Click node of the Click configuration file (.click file) that it is to use.

This model implements the interface to the Click Modular Router and provides the `Ipv4ClickRouting` class to allow a node to use Click for external routing. Unlike normal `Ipv4RoutingProtocol` sub types, `Ipv4ClickRouting` doesn't use a `RouteInput()` method, but instead, receives a packet on the appropriate interface and processes it accordingly. Note that you need to have a routing table type element in your Click graph to use Click for external routing. This is needed by the `RouteOutput()` function inherited from `Ipv4RoutingProtocol`. Furthermore, a Click based node uses a different kind of L3 in the form of `Ipv4L3ClickProtocol`, which is a trimmed down version of `Ipv4L3Protocol`. `Ipv4L3ClickProtocol` passes on packets passing through the stack to `Ipv4ClickRouting` for processing.

Developing a Simulator API to allow ns-3 to interact with Click

Much of the API is already well defined, which allows Click to probe for information from the simulator (like a Node's ID, an Interface ID and so forth). By retaining most of the methods, it should be possible to write new implementations specific to ns-3 for the same functionality.

Hence, for the Click integration with ns-3, a class named `Ipv4ClickRouting` will handle the interaction with Click. The code for the same can be found in `src/click/model/ipv4-click-routing.{cc,h}`.

Packet hand off between ns-3 and Click

There are four kinds of packet hand-offs that can occur between ns-3 and Click.

- L4 to L3
- L3 to L4
- L3 to L2
- L2 to L3

To overcome this, we implement `Ipv4L3ClickProtocol`, a stripped down version of `Ipv4L3Protocol`. `Ipv4L3ClickProtocol` passes packets to and from `Ipv4ClickRouting` appropriately to perform routing.

9.1.2 Scope and Limitations

- In its current state, the NS-3 Click Integration is limited to use only with L3, leaving NS-3 to handle L2. We are currently working on adding Click MAC support as well. See the usage section to make sure that you design your Click graphs accordingly.
- Furthermore, ns-3-click will work only with userlevel elements. The complete list of elements are available at <https://web.archive.org/web/20171003052722/http://read.cs.ucla.edu/click/elements>. Elements that have 'all', 'userlevel' or 'ns' mentioned beside them may be used.
- As of now, the ns-3 interface to Click is Ipv4 only. We will be adding Ipv6 support in the future.

9.1.3 References

- Eddie Kohler, Robert Morris, Benjie Chen, John Jannotti, and M. Frans Kaashoek. The click modular router. *ACM Transactions on Computer Systems* 18(3), August 2000, pages 263-297.
- Lalith Suresh P., and Ruben Merz. Ns-3-click: click modular router integration for ns-3. In *Proc. of 3rd International ICST Workshop on NS-3 (WNS3)*, Barcelona, Spain. March, 2011.
- Michael Neufeld, Ashish Jain, and Dirk Grunwald. Nsclick: bridging network simulation and deployment. *MSWiM '02: Proceedings of the 5th ACM international workshop on Modeling analysis and simulation of wireless and mobile systems*, 2002, Atlanta, Georgia, USA. <http://doi.acm.org/10.1145/570758.570772>

9.2 Usage

9.2.1 Building Click

The first step is to clone Click from the github repository and build it:

```
$ git clone https://github.com/kohler/click
$ cd click/
$ ./configure --disable-linuxmodule --enable-nsclick --enable-wifi
$ make
```

The `--enable-wifi` flag may be skipped if you don't intend on using Click with Wifi. * Note: You don't need to do a 'make install'.

Once Click has been built successfully, change into the ns-3 directory and configure ns-3 with Click Integration support:

```
$ ./ns3 configure --enable-examples --enable-tests --with-nsclick=/path/to/click/source
```

Hint: If you have click installed one directory above ns-3 (such as in the ns-3-allinone directory), and the name of the directory is 'click' (or a symbolic link to the directory is named 'click'), then the `--with-nsclick` specifier is not necessary; the ns-3 build system will successfully find the directory.

If it says 'enabled' beside 'NS-3 Click Integration Support', then you're good to go. Note: If running modular ns-3, the minimum set of modules required to run all ns-3-click examples is wifi, csma and config-store.

Next, try running one of the examples:

```
$ ./ns3 run nsclick-simple-lan
```

You may then view the resulting .pcap traces, which are named nsclick-simple-lan-0-0.pcap and nsclick-simple-lan-0-1.pcap.

9.2.2 Click Graph Instructions

The following should be kept in mind when making your Click graph:

- Only userlevel elements can be used.
- You will need to replace FromDevice and ToDevice elements with FromSimDevice and ToSimDevice elements.
- Packets to the kernel are sent up using ToSimDevice(tap0,IP).
- For any node, the device which sends/receives packets to/from the kernel, is named 'tap0'. The remaining interfaces should be named eth0, eth1 and so forth (even if you're using wifi). Please note that the device numbering should begin from 0. In future, this will be made flexible so that users can name devices in their Click file as they wish.
- A routing table element is a mandatory. The OUTports of the routing table element should correspond to the interface number of the device through which the packet will ultimately be sent out. Violating this rule will lead to really weird packet traces. This routing table element's name should then be passed to the Ipv4ClickRouting protocol object as a simulation parameter. See the Click examples for details.
- The current implementation leaves Click with mainly L3 functionality, with ns-3 handling L2. We will soon begin working to support the use of MAC protocols on Click as well. This means that as of now, Click's Wifi specific elements cannot be used with ns-3.

9.2.3 Debugging Packet Flows from Click

From any point within a Click graph, you may use the Print (<https://web.archive.org/web/20171003052722/http://read.cs.ucla.edu/click/elements/print>) element and its variants for pretty printing of packet contents. Furthermore, you may generate pcap traces of packets flowing through a Click graph by using the ToDump (<https://web.archive.org/web/20171003052722/http://read.cs.ucla.edu/click/elements/todump>) element as well. For instance:

```
myarpquerier
-> Print(fromarpquery,64)
-> ToDump(out_arpquery,PER_NODE 1)
-> ethout;
```

and ... will print the contents of packets that flow out of the ArpQuerier, then generate a pcap trace file which will have a suffix 'out_arpquery', for each node using the Click file, before pushing packets onto 'ethout'.

9.2.4 Helper

To have a node run Click, the easiest way would be to use the ClickInternetStackHelper class in your simulation script. For instance:

```
ClickInternetStackHelper click;
click.SetClickFile(myNodeContainer, "nsclick-simple-lan.click");
click.SetRoutingTableElement(myNodeContainer, "u/r");
click.Install(myNodeContainer);
```

The example scripts inside `src/click/examples/` demonstrate the use of Click based nodes in different scenarios. The helper source can be found inside `src/click/helper/click-internet-stack-helper.{h,cc}`

9.2.5 Examples

The following examples have been written, which can be found in `src/click/examples/`:

- `nsclick-simple-lan.cc` and `nsclick-raw-wlan.cc`: A Click based node communicating with a normal ns-3 node without Click, using Csma and Wifi respectively. It also demonstrates the use of TCP on top of Click, something which the original nsclick implementation for NS-2 couldn't achieve.
- `nsclick-udp-client-server-csma.cc` and `nsclick-udp-client-server-wifi.cc`: A 3 node LAN (Csma and Wifi respectively) wherein 2 Click based nodes run a UDP client, that sends packets to a third Click based node running a UDP server.
- `nsclick-routing.cc`: One Click based node communicates to another via a third node that acts as an IP router (using the IP router Click configuration). This demonstrates routing using Click.

Scripts are available within `<click-dir>/conf/` that allow you to generate Click files for some common scenarios. The IP Router used in `nsclick-routing.cc` was generated from the `make-ip-conf.pl` file and slightly adapted to work with ns-3-click.

9.3 Validation

This model has been tested as follows:

- Unit tests have been written to verify the internals of Ipv4ClickRouting. This can be found in `src/click/ipv4-click-routing-test.cc`. These tests verify whether the methods inside Ipv4ClickRouting which deal with Device name to ID, IP Address from device name and Mac Address from device name bindings work as expected.
- The examples have been used to test Click with actual simulation scenarios. These can be found in `src/click/examples/`. These tests cover the following: the use of different kinds of transports on top of Click, TCP/UDP, whether Click nodes can communicate with non-Click based nodes, whether Click nodes can communicate with each other, using Click to route packets using static routing.
- Click has been tested with Csma, Wifi and Point-to-Point devices. Usage instructions are available in the preceding section.

CSMA NETDEVICE

This is the introduction to CSMA NetDevice chapter, to complement the CSMA model doxygen.

10.1 Overview of the CSMA model

The *ns-3* CSMA device models a simple bus network in the spirit of Ethernet. Although it does not model any real physical network you could ever build or buy, it does provide some very useful functionality.

Typically when one thinks of a bus network Ethernet or IEEE 802.3 comes to mind. Ethernet uses CSMA/CD (Carrier Sense Multiple Access with Collision Detection with exponentially increasing backoff to contend for the shared transmission medium. The *ns-3* CSMA device models only a portion of this process, using the nature of the globally available channel to provide instantaneous (faster than light) carrier sense and priority-based collision “avoidance.” Collisions in the sense of Ethernet never happen and so the *ns-3* CSMA device does not model collision detection, nor will any transmission in progress be “jammed.”

10.1.1 CSMA Layer Model

There are a number of conventions in use for describing layered communications architectures in the literature and in textbooks. The most common layering model is the ISO seven layer reference model. In this view the *CsmaNetDevice* and *CsmaChannel* pair occupies the lowest two layers – at the physical (layer one), and data link (layer two) positions. Another important reference model is that specified by RFC 1122, “Requirements for Internet Hosts – Communication Layers.” In this view the *CsmaNetDevice* and *CsmaChannel* pair occupies the lowest layer – the link layer. There is also a seemingly endless litany of alternative descriptions found in textbooks and in the literature. We adopt the naming conventions used in the IEEE 802 standards which speak of LLC, MAC, MII and PHY layering. These acronyms are defined as:

- LLC: Logical Link Control;
- MAC: Media Access Control;
- MII: Media Independent Interface;
- PHY: Physical Layer.

In this case the *LLC* and *MAC* are sublayers of the OSI data link layer and the *MI* and *PHY* are sublayers of the OSI physical layer.

The “top” of the CSMA device defines the transition from the network layer to the data link layer. This transition is performed by higher layers by calling either *CsmaNetDevice::Send* or *CsmaNetDevice::SendFrom*.

In contrast to the IEEE 802.3 standards, there is no precisely specified PHY in the CSMA model in the sense of wire types, signals or pinouts. The “bottom” interface of the *CsmaNetDevice* can be thought of as as a kind of Media Independent Interface (MII) as seen in the “Fast Ethernet” (IEEE 802.3u) specifications. This MII interface fits into a

corresponding media independent interface on the `CsmaChannel`. You will not find the equivalent of a 10BASE-T or a 1000BASE-LX PHY.

The `CsmaNetDevice` calls the `CsmaChannel` through a media independent interface. There is a method defined to tell the channel when to start “wiggling the wires” using the method `CsmaChannel::TransmitStart`, and a method to tell the channel when the transmission process is done and the channel should begin propagating the last bit across the “wire”: `CsmaChannel::TransmitEnd`.

When the `TransmitEnd` method is executed, the channel will model a single uniform signal propagation delay in the medium and deliver copies of the packet to each of the devices attached to the packet via the `CsmaNetDevice::Receive` method.

There is a “pin” in the device media independent interface corresponding to “COL” (collision). The state of the channel may be sensed by calling `CsmaChannel::GetState`. Each device will look at this “pin” before starting a send and will perform appropriate backoff operations if required.

Properly received packets are forwarded up to higher levels from the `CsmaNetDevice` via a callback mechanism. The callback function is initialized by the higher layer (when the net device is attached) using `CsmaNetDevice::SetReceiveCallback` and is invoked upon “proper” reception of a packet by the net device in order to forward the packet up the protocol stack.

10.2 CSMA Channel Model

The class `CsmaChannel` models the actual transmission medium. There is no fixed limit for the number of devices connected to the channel. The `CsmaChannel` models a data rate and a speed-of-light delay which can be accessed via the attributes “DataRate” and “Delay” respectively. The data rate provided to the channel is used to set the data rates used by the transmitter sections of the CSMA devices connected to the channel. There is no way to independently set data rates in the devices. Since the data rate is only used to calculate a delay time, there is no limitation (other than by the data type holding the value) on the speed at which CSMA channels and devices can operate; and no restriction based on any kind of PHY characteristics.

The `CsmaChannel` has three states, `IDLE`, `TRANSMITTING` and `PROPAGATING`. These three states are “seen” instantaneously by all devices on the channel. By this we mean that if one device begins or ends a simulated transmission, all devices on the channel are *immediately* aware of the change in state. There is no time during which one device may see an `IDLE` channel while another device physically further away in the collision domain may have begun transmitting with the associated signals not propagated down the channel to other devices. Thus there is no need for collision detection in the `CsmaChannel` model and it is not implemented in any way.

We do, as the name indicates, have a Carrier Sense aspect to the model. Since the simulator is single threaded, access to the common channel will be serialized by the simulator. This provides a deterministic mechanism for contending for the channel. The channel is allocated (transitioned from state `IDLE` to state `TRANSMITTING`) on a first-come first-served basis. The channel always goes through a three state process:

<code>IDLE -> TRANSMITTING -> PROPAGATING -> IDLE</code>

The `TRANSMITTING` state models the time during which the source net device is actually wiggling the signals on the wire. The `PROPAGATING` state models the time after the last bit was sent, when the signal is propagating down the wire to the “far end.”

The transition to the `TRANSMITTING` state is driven by a call to `CsmaChannel::TransmitStart` which is called by the net device that transmits the packet. It is the responsibility of that device to end the transmission with a call to `CsmaChannel::TransmitEnd` at the appropriate simulation time that reflects the time elapsed to put all of the packet bits on the wire. When `TransmitEnd` is called, the channel schedules an event corresponding to a single speed-of-light delay. This delay applies to all net devices on the channel identically. You can think of a symmetrical hub in which the packet bits propagate to a central location and then back out equal length cables to the other devices on the channel. The single “speed of light” delay then corresponds to the time it takes for: 1) a signal to propagate from one `CsmaNetDevice`

through its cable to the hub; plus 2) the time it takes for the hub to forward the packet out a port; plus 3) the time it takes for the signal in question to propagate to the destination net device.

The CsmChannel models a broadcast medium so the packet is delivered to all of the devices on the channel (including the source) at the end of the propagation time. It is the responsibility of the sending device to determine whether or not it receives a packet broadcast over the channel.

The CsmChannel provides following Attributes:

- DataRate: The bitrate for packet transmission on connected devices;
- Delay: The speed of light transmission delay for the channel.

10.3 CSMA Net Device Model

The CSMA network device appears somewhat like an Ethernet device. The CsmNetDevice provides following Attributes:

- Address: The Mac48Address of the device;
- SendEnable: Enable packet transmission if true;
- ReceiveEnable: Enable packet reception if true;
- EncapsulationMode: Type of link layer encapsulation to use;
- RxErrorModel: The receive error model;
- TxQueue: The transmit queue used by the device;
- InterframeGap: The optional time to wait between “frames”;
- Rx: A trace source for received packets;
- Drop: A trace source for dropped packets.

In addition to these attributes, CsmNetDevice exposes a number of trace sources. The most commonly used ones are:

- MacTx: Trace source for packets handed down from upper layers, and fires before the packet is enqueued into the device transmit queue;
- MacTxDrop: Trace source for packets dropped by the device before transmission;
- MacRx: Trace source for packets successfully received in non-promiscuous mode;
- MacPromiscRx: Trace source for packets received in promiscuous mode and fired only when a promiscuous receive callback is installed via SetPromiscReceiveCallback;
- Sniffer / PromiscSniffer: Trace sources simulating non-promiscuous and promiscuous packet sniffers. On the receive path they follow the same packet selection as MacRx / MacPromiscRx and on the transmit path they fire after dequeuing from the device transmit queue.

The CsmNetDevice supports the assignment of a “receive error model.” This is an ErrorModel object that is used to simulate data corruption on the link.

Packets sent over the CsmNetDevice are always routed through the transmit queue to provide a trace hook for packets sent out over the network. This transmit queue can be set (via attribute) to model different queuing strategies.

Also configurable by attribute is the encapsulation method used by the device. Every packet gets an EthernetHeader that includes the destination and source MAC addresses, and a length/type field. Every packet also gets an EthernetTrailer which includes the FCS. Data in the packet may be encapsulated in different ways.

By default, or by setting the “EncapsulationMode” attribute to “Dix”, the encapsulation is according to the DEC, Intel, Xerox standard. This is sometimes called EthernetII framing and is the familiar destination MAC, source MAC, EtherType, Data, CRC format.

If the “EncapsulationMode” attribute is set to “Llc”, the encapsulation is by LLC SNAP. In this case, a SNAP header is added that contains the EtherType (IP or ARP).

The other implemented encapsulation modes are IP_ARP (set “EncapsulationMode” to “IpArp”) in which the length type of the Ethernet header receives the protocol number of the packet; or ETHERNET_V1 (set “EncapsulationMode” to “EthernetV1”) in which the length type of the Ethernet header receives the length of the packet. A “Raw” encapsulation mode is defined but not implemented – use of the RAW mode results in an assertion.

Note that all net devices on a channel must be set to the same encapsulation mode for correct results. The encapsulation mode is not sensed at the receiver.

The CsmaNetDevice implements a random exponential backoff algorithm that is executed if the channel is determined to be busy (TRANSMITTING or PROPAGATING) when the device wants to start propagating. This results in a random delay of up to $2^{\text{retries}} - 1$ microseconds before a retry is attempted. The default maximum number of retries is 1000.

10.4 Using the CsmaNetDevice

The CSMA net devices and channels are typically created and configured using the associated CsmaHelper object. The various *ns-3* device helpers generally work in a similar way, and their use is seen in many of our example programs.

The conceptual model of interest is that of a bare computer “husk” into which you plug net devices. The bare computers are created using a NodeContainer helper. You just ask this helper to create as many computers (we call them Nodes) as you need on your network:

```
NodeContainer csmaNodes;  
csmaNodes.Create(nCsmaNodes);
```

Once you have your nodes, you need to instantiate a CsmaHelper and set any attributes you may want to change.:

```
CsmaHelper csma;  
csma.SetChannelAttribute("DataRate", StringValue("100Mbps"));  
csma.SetChannelAttribute("Delay", TimeValue(NanoSeconds(6560)));  
  
csma.SetDeviceAttribute("EncapsulationMode", StringValue("Dix"));  
csma.SetDeviceAttribute("FrameSize", UIntegerValue(2000));
```

Once the attributes are set, all that remains is to create the devices and install them on the required nodes, and to connect the devices together using a CSMA channel. When we create the net devices, we add them to a container to allow you to use them in the future. This all takes just one line of code.:

```
NetDeviceContainer csmaDevices = csma.Install(csmaNodes);
```

We recommend thinking carefully about changing these Attributes, since it can result in behavior that surprises users. We allow this because we believe flexibility is important. As an example of a possibly surprising effect of changing Attributes, consider the following:

The Mtu Attribute indicates the Maximum Transmission Unit to the device. This is the size of the largest Protocol Data Unit (PDU) that the device can send. This Attribute defaults to 1500 bytes and corresponds to a number found in RFC 894, “A Standard for the Transmission of IP Datagrams over Ethernet Networks.” The number is actually derived from the maximum packet size for 10Base5 (full-spec Ethernet) networks – 1518 bytes. If you subtract DIX encapsulation overhead for Ethernet packets (18 bytes) you will end up with a maximum possible data size (MTU) of 1500 bytes. One can also find that the MTU for IEEE 802.3 networks is 1492 bytes. This is because LLC/SNAP encapsulation adds an

extra eight bytes of overhead to the packet. In both cases, the underlying network hardware is limited to 1518 bytes, but the MTU is different because the encapsulation is different.

If one leaves the Mtu Attribute at 1500 bytes and changes the encapsulation mode Attribute to Llc, the result will be a network that encapsulates 1500 byte PDUs with LLC/SNAP framing resulting in packets of 1526 bytes. This would be illegal in many networks, but we allow you to do this. This results in a simulation that quite subtly does not reflect what you might be expecting since a real device would balk at sending a 1526 byte packet.

There also exist jumbo frames ($1500 < \text{MTU} \leq 9000$ bytes) and super-jumbo ($\text{MTU} > 9000$ bytes) frames that are not officially sanctioned by IEEE but are available in some high-speed (Gigabit) networks and NICs. In the CSMA model, one could leave the encapsulation mode set to Dix, and set the Mtu to 64000 bytes – even though an associated CsmaChannel DataRate was left at 10 megabits per second (certainly not Gigabit Ethernet). This would essentially model an Ethernet switch made out of vampire-tapped 1980s-style 10Base5 networks that support super-jumbo datagrams, which is certainly not something that was ever made, nor is likely to ever be made; however it is quite easy for you to configure.

Be careful about assumptions regarding what CSMA is actually modelling and how configuration (Attributes) may allow you to swerve considerably away from reality.

10.5 CSMA Tracing

Like all *ns-3* devices, the CSMA Model provides a number of trace sources. These trace sources can be hooked using your own custom trace code, or you can use our helper functions to arrange for tracing to be enabled on devices you specify.

10.5.1 Upper-Level (MAC) Hooks

From the point of view of tracing in the net device, there are several interesting points to insert trace hooks. A convention inherited from other simulators is that packets destined for transmission onto attached networks pass through a single “transmit queue” in the net device. We provide trace hooks at this point in packet flow, which corresponds (abstractly) only to a transition from the network to data link layer, and call them collectively the device MAC hooks.

When a packet is sent to the CSMA net device for transmission it always passes through the transmit queue. The transmit queue in the CsmaNetDevice inherits from Queue, and therefore inherits three trace sources:

- An Enqueue operation source (see Queue::m_traceEnqueue);
- A Dequeue operation source (see Queue::m_traceDequeue);
- A Drop operation source (see Queue::m_traceDrop).

The upper-level (MAC) trace hooks for the CsmaNetDevice are, in fact, exactly these three trace sources on the single transmit queue of the device.

The m_traceEnqueue event is triggered when a packet is placed on the transmit queue. This happens at the time that CsmaNetDevice::Send or CsmaNetDevice::SendFrom is called by a higher layer to queue a packet for transmission.

The m_traceDequeue event is triggered when a packet is removed from the transmit queue. Dequeues from the transmit queue can happen in three situations: 1) If the underlying channel is idle when the CsmaNetDevice::Send or CsmaNetDevice::SendFrom is called, a packet is dequeued from the transmit queue and immediately transmitted; 2) If the underlying channel is idle, a packet may be dequeued and immediately transmitted in an internal TransmitCompleteEvent that functions much like a transmit complete interrupt service routine; or 3) from the random exponential backoff handler if a timeout is detected.

Case (3) implies that a packet is dequeued from the transmit queue if it is unable to be transmitted according to the backoff rules. It is important to understand that this will appear as a Dequeued packet and it is easy to incorrectly assume that the packet was transmitted since it passed through the transmit queue. In fact, a packet is actually dropped

by the net device in this case. The reason for this behavior is due to the definition of the Queue Drop event. The `m_traceDrop` event is, by definition, fired when a packet cannot be enqueued on the transmit queue because it is full. This event only fires if the queue is full and we do not overload this event to indicate that the `CsmaChannel` is “full.”

10.5.2 Lower-Level (PHY) Hooks

Similar to the upper level trace hooks, there are trace hooks available at the lower levels of the net device. We call these the PHY hooks. These events fire from the device methods that talk directly to the `CsmaChannel`.

The trace source `m_dropTrace` is called to indicate a packet that is dropped by the device. This happens in two cases: First, if the receive side of the net device is not enabled (see `CsmaNetDevice::m_receiveEnable` and the associated attribute “`ReceiveEnable`”).

The `m_dropTrace` is also used to indicate that a packet was discarded as corrupt if a receive error model is used (see `CsmaNetDevice::m_receiveErrorModel` and the associated attribute “`ReceiveErrorModel`”).

The other low-level trace source fires on reception of an accepted packet (see `CsmaNetDevice::m_rxTrace`). A packet is accepted if it is destined for the broadcast address, a multicast address, or to the MAC address assigned to the net device.

10.6 Summary

The ns3 CSMA model is a simplistic model of an Ethernet-like network. It supports a Carrier-Sense function and allows for Multiple Access to a shared medium. It is not physical in the sense that the state of the medium is instantaneously shared among all devices. This means that there is no collision detection required in this model and none is implemented. There will never be a “jam” of a packet already on the medium. Access to the shared channel is on a first-come first-served basis as determined by the simulator scheduler. If the channel is determined to be busy by looking at the global state, a random exponential backoff is performed and a retry is attempted.

Ns-3 Attributes provide a mechanism for setting various parameters in the device and channel such as addresses, encapsulation modes and error model selection. Trace hooks are provided in the usual manner with a set of upper level hooks corresponding to a transmit queue and used in ASCII tracing; and also a set of lower level hooks used in pcap tracing.

Although the ns-3 `CsmaChannel` and `CsmaNetDevice` does not model any kind of network you could build or buy, it does provide us with some useful functionality. You should, however, understand that it is explicitly not Ethernet or any flavor of IEEE 802.3 but an interesting subset.

DSDV ROUTING

Destination-Sequenced Distance Vector (DSDV) routing protocol is a pro-active, table-driven routing protocol for MANETs developed by Charles E. Perkins and Pravin Bhagwat in 1994. It uses the hop count as metric in route selection.

This model was developed by [the ResiliNets research group](#) at the University of Kansas. A paper on this model exists at [this URL](#).

11.1 DSDV Routing Overview

DSDV Routing Table: Every node will maintain a table listing all the other nodes it has known either directly or through some neighbors. Every node has a single entry in the routing table. The entry will have information about the node's IP address, last known sequence number and the hop count to reach that node. Along with these details the table also keeps track of the nexthop neighbor to reach the destination node, the timestamp of the last update received for that node.

The DSDV update message consists of three fields, Destination Address, Sequence Number and Hop Count.

Each node uses 2 mechanisms to send out the DSDV updates. They are,

1. **Periodic Updates**

Periodic updates are sent out after every `m_periodicUpdateInterval`(default:15s). In this update the node broadcasts out its entire routing table.

2. **Trigger Updates**

Trigger Updates are small updates in-between the periodic updates. These updates are sent out whenever a node receives a DSDV packet that caused a change in its routing table. The original paper did not clearly mention when for what change in the table should a DSDV update be sent out. The current implementation sends out an update irrespective of the change in the routing table.

The updates are accepted based on the metric for a particular node. The first factor determining the acceptance of an update is the sequence number. It has to accept the update if the sequence number of the update message is higher irrespective of the metric. If the update with same sequence number is received, then the update with least metric (hopCount) is given precedence.

In highly mobile scenarios, there is a high chance of route fluctuations, thus we have the concept of weighted settling time where an update with change in metric will not be advertised to neighbors. The node waits for the settling time to make sure that it did not receive the update from its old neighbor before sending out that update.

The current implementation covers all the above features of DSDV. The current implementation also has a request queue to buffer packets that have no routes to destination. The default is set to buffer up to 5 packets per destination.

11.2 References

Link to the Paper: <http://portal.acm.org/citation.cfm?doid=190314.190336>

DYNAMIC SOURCE ROUTING (DSR)

Dynamic Source Routing (DSR) protocol is a reactive routing protocol designed specifically for use in multi-hop wireless ad hoc networks of mobile nodes. The current model implementation is based on the RFC 4728 [2], with some extensions and modifications. The initial implementation was developed by the [ResiliNets research group](#) at the University of Kansas with additions to the link cache mechanism done by Song Luan <lsuper@mail.ustc.edu.cn>.

The source code for this module lives in the directory `src/dsr`.

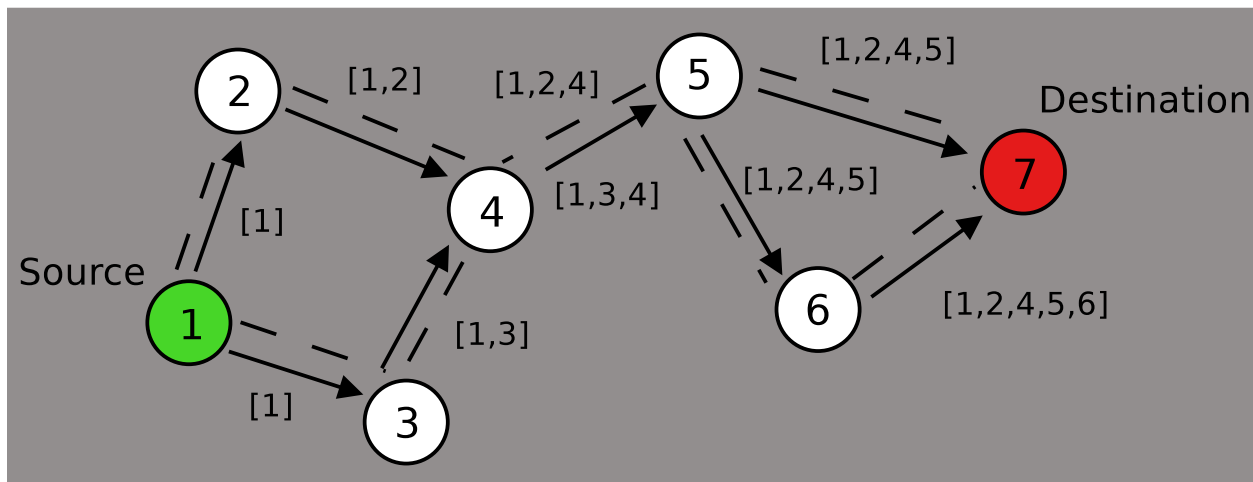


Fig. 12.1: DSR route discovery

DSR creates routes towards a destination by adding the traversed route into the packet header during the discovery phase. Different packets might have different routes even if they have the same source and the same destination. The recorded information during the route discovery phase is used by intermediate nodes to determine whom the packet should be forward. Hence, the term dynamic source routing.

12.1 Scope and Limitations

- The model is not fully compliant with RFC 4728 [2]. As an example, DSR fixed size header has been extended and it is four octets longer than the RFC specification. As a consequence, the DSR headers can not be correctly decoded by Wireshark.
- At the moment, the model does not handle ICMP packets, and assumes that traffic is either TCP or UDP.
- No flow state available.
- No First Hop External (F), Last Hop External (L) flags.

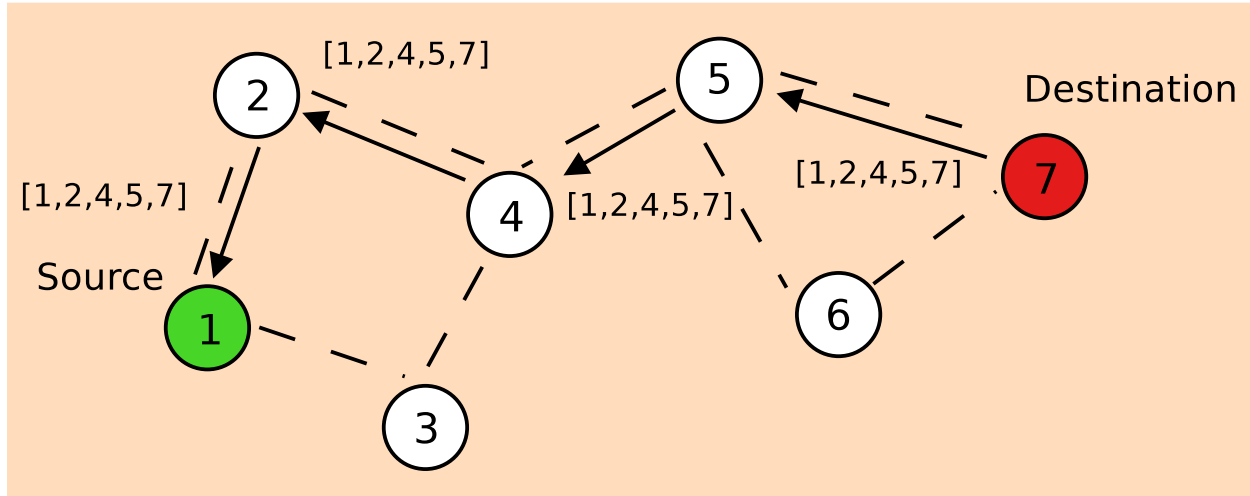


Fig. 12.2: DSR route reply using route caching

- There are no ways of handling unknown DSR options.
- DSR option flow state not supported is not handled.
- DSR option unsupported option is not handled.
- Route Reply header is not aligned to the DSR RFC descriptions.

12.2 Protocol operational details

DSR operates on a on-demand behavior. Therefore, our DSR model buffers all packets while a route request packet (RREQ) is disseminated. The packet buffer is implemented in `dsr-rsendbuff.cc`.

The packet queue implements garbage collection of old packets and a queue size limit. When the packet is sent out from the send buffer, it will be queued in maintenance buffer for next hop acknowledgment.

The maintenance buffer then buffers the already sent out packets and waits for the notification of packet delivery. Protocol operation strongly depends on broken link detection mechanism. Three heuristics detection mechanisms are implemented as recommended by the RFC as follows:

1. A link layer feedback is used when possible, which is also the fastest mechanism of these three to detect link errors. A link is considered to be broken if frame transmission results in a transmission failure for all retries. This mechanism is meant for active links and works much faster than in its absence. DSR is able to detect the link layer transmission failure and notify that as broken. Recalculation of routes will be triggered when needed. If user does not want to use link layer acknowledgment, it can be tuned by setting the `LinkAcknowledgment` attribute to false in `dsr-routing.cc`.
2. Passive acknowledgment should be used whenever possible. The node turns on “promiscuous” receive mode, in which it can receive packets not destined for itself, and when the node assures the delivery of that data packet to its destination, it cancels the passive acknowledgment timer.
3. A network layer acknowledgment scheme is used to notify the receipt of a packet. Route request packet are not be acknowledged or retransmitted.

12.2.1 Route Cache implementation

The Route Cache implementation support garbage collection of old entries and state machine, as defined in the standard. It is implemented as a STL map container. The destination IP address is used as a key in the map.

DSR operates with direct access to IP header, and operates between network and transport layer. When packet is sent out from transport layer, it passes itself to DSR and DSR header is appended.

Two caching mechanisms exists in the implementation: Path cache and Link cache.

- **Path Cache:** As the name implies, this cache saves the whole path and ensures loop-free paths. The paths are sorted based on the hop count, and whenever one path becomes unusable a different path is chosen. Some of its characteristics are as follows:
 - The cache has automatic expire policy.
 - The cache saves multiple route entries for a certain destination and sort the entries based on hop counts.
 - The `MaxEntriesEachDst` can be tuned to change the maximum entries saved for a single destination.
 - When adding multiple routes for one destination, the route is compared based on hop-count and expire time, the one with less hop count or relatively new route is favored.
- **Link Cache:** This is an improvement over the patch cache in the sense that it uses different subpaths and make use of the Dijkstra algorithm.

12.2.2 Modifications

- The `DsrFsHeader` has added 3 fields: message type, source id, destination id, and these changes only for post-processing.
 1. Message type is used to identify the data packet from control packet.
 2. Source id is used to identify the real source of the data packet since the packet is delivered hop-by-hop and the `Ipv4Header` is not carrying the real source and destination ip address as needed.
 3. Destination id is for same reason of above.
- DSR works as a shim header between transport and network protocol, it needs its own forwarding mechanism, we are changing the packet transmission to hop-by-hop delivery, so we added two fields in `dsr` fixed header to notify packet delivery.

12.2.3 Other considerations

The following should be kept in mind when running DSR as routing protocol:

- `NodeTraversalTime` is the time it takes to traverse two neighboring nodes and should be chosen to fit the transmission range
- `PassiveAckTimeout` is the time a packet in maintenance buffer wait for passive acknowledgment, normally set as two times of `NodeTraversalTime`
- `RouteCacheTimeout` should be set smaller value when the nodes' velocity become higher. The default value is 300s.

12.3 Usage

In *ns-3*, DSR is used on top of WiFi nodes. The simplest way to use it requires the use of its helper as explained in the following subsections.

12.3.1 Helpers

To have a node run DSR, the easiest way would be to use the `DsrHelper` and `DsrMainHelpers` in your simulation script. For instance:

```
NodeContainer adhocNodes;  
nodes.Create(5); // Add 5 nodes to the node container  
DsrHelper dsr;  
DsrMainHelper dsrMain;  
dsrMain.Install(dsr, adhocNodes); // Install DSR on each node
```

12.3.2 Attributes

The complete list of available attributes with descriptions for this model can be found on [this link](#).

12.3.3 Traces

The complete list of available trace sources with descriptions for this model can be found on [this link](#).

12.4 Examples and Tests

The example can be found in `src/dsr/examples/`:

- `dsr.cc`: Use DSR as routing protocol within a traditional MANETs environment.

DSR is also built in the routing comparison case in `examples/routing/`:

- `manet-routing-compare.cc`: A comparison case with built in MANET routing protocols and can generate its own results.

12.5 Validation

This model has been tested as follows:

- Unit tests have been written to verify the internals of DSR. This can be found in `src/dsr/test/dsr-test-suite.cc`. These tests verify whether the methods inside DSR module which deal with packet buffer, headers work correctly.
- Simulation cases similar to [3] have been tested and have comparable results.
- `Manet-routing-compare.cc` has been used to compare DSR with three of other routing protocols.
- A paper was presented on these results at the Workshop on ns-3 in 2011 [4].

12.6 References

- [1] Johnson, David B., David A. Maltz, and Josh Broch. “DSR: The dynamic source routing protocol for multi-hop wireless ad hoc networks.” *Ad hoc networking* 5.1 (2001): 139-172.
- [2] RFC 4728
- [3] Josh Broch, David A. Maltz, David B. Johnson, Yih-Chun Hu, and Jorjeta Jetcheva. 1998. A performance comparison of multi-hop wireless ad hoc network routing protocols. In *Proceedings of the 4th annual ACM/IEEE international conference on Mobile computing and networking (MobiCom '98)*. Association for Computing Machinery, New York, NY, USA, 85–97.
- [4] Yufei Cheng, Egemen K. Çetinkaya, and James P. G. Sterbenz. 2012. Dynamic source routing (DSR) protocol implementation in ns-3. In *Proceedings of the 5th International ICST Conference on Simulation Tools and Techniques (SIMUTOOLS '12)*. ICST (Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering), Brussels, BEL, 367–374.

EMULATION OVERVIEW

ns-3 has been designed for integration into testbed and virtual machine environments. We have addressed this need by providing two kinds of net devices. The first kind of device is a file descriptor net device (`FdNetDevice`), which is a generic device type that can read and write from a file descriptor. By associating this file descriptor with different things on the host system, different capabilities can be provided. For instance, the `FdNetDevice` can be associated with an underlying packet socket to provide emulation capabilities. This allows *ns-3* simulations to send data on a “real” network. The second kind, called a `TapBridge NetDevice` allows a “real” host to participate in an *ns-3* simulation as if it were one of the simulated nodes. An *ns-3* simulation may be constructed with any combination of simulated or emulated devices.

Note: Prior to *ns-3.17*, the emulation capability was provided by a special device called an `Emu NetDevice`; the `Emu NetDevice` has been replaced by the `FdNetDevice`.

One of the use-cases we want to support is that of a testbed. A concrete example of an environment of this kind is the ORBIT testbed. ORBIT is a laboratory emulator/field trial network arranged as a two dimensional grid of 400 802.11 radio nodes. We integrate with ORBIT by using their “imaging” process to load and run *ns-3* simulations on the ORBIT array. We can use our `EmuFdNetDevice` to drive the hardware in the testbed and we can accumulate results either using the *ns-3* tracing and logging functions, or the native ORBIT data gathering techniques. See <http://www.orbit-lab.org/> for details on the ORBIT testbed.

A simulation of this kind is shown in the following figure:

You can see that there are separate hosts, each running a subset of a “global” simulation. Instead of an *ns-3* channel connecting the hosts, we use real hardware provided by the testbed. This allows *ns-3* applications and protocol stacks attached to a simulation node to communicate over real hardware.

We expect the primary use for this configuration will be to generate repeatable experimental results in a real-world network environment that includes all of the *ns-3* tracing, logging, visualization and statistics gathering tools.

In what can be viewed as essentially an inverse configuration, we allow “real” machines running native applications and protocol stacks to integrate with an *ns-3* simulation. This allows for the simulation of large networks connected to a real machine, and also enables virtualization. A simulation of this kind is shown in the following figure:

Here, you will see that there is a single host with a number of virtual machines running on it. An *ns-3* simulation is shown running in the virtual machine shown in the center of the figure. This simulation has a number of nodes with associated *ns-3* applications and protocol stacks that are talking to an *ns-3* channel through native simulated *ns-3* net devices.

There are also two virtual machines shown at the far left and far right of the figure. These VMs are running native (Linux) applications and protocol stacks. The VM is connected into the simulation by a Linux Tap net device. The user-mode handler for the Tap device is instantiated in the simulation and attached to a proxy node that represents the native VM in the simulation. These handlers allow the Tap devices on the native VMs to behave as if they were *ns-3* net devices in the simulation VM. This, in turn, allows the native software and protocol suites in the native VMs to believe that they are connected to the simulated *ns-3* channel.

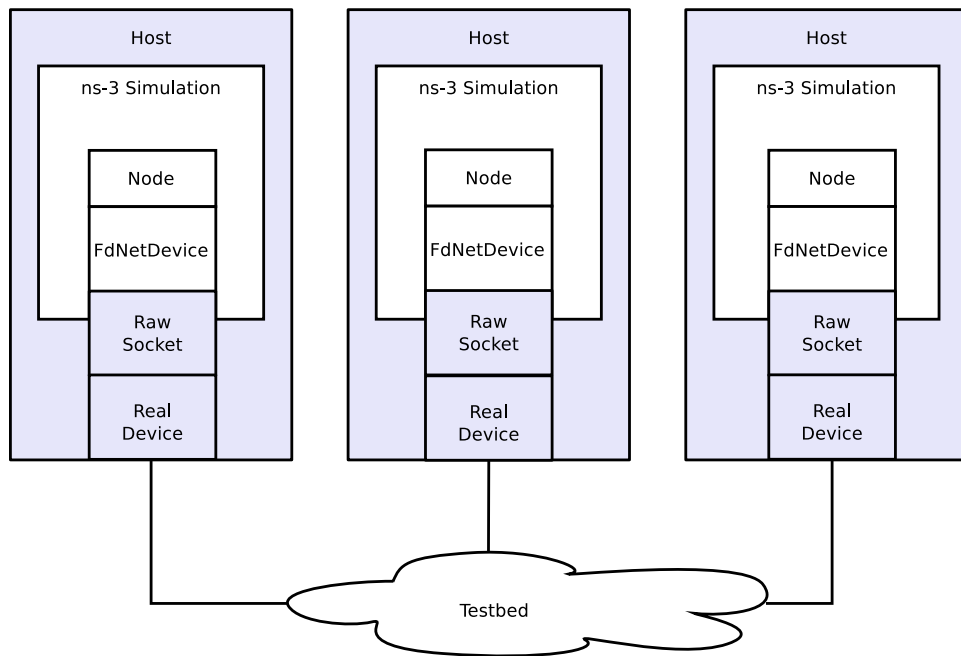


Fig. 13.1: Example Implementation of Testbed Emulation.

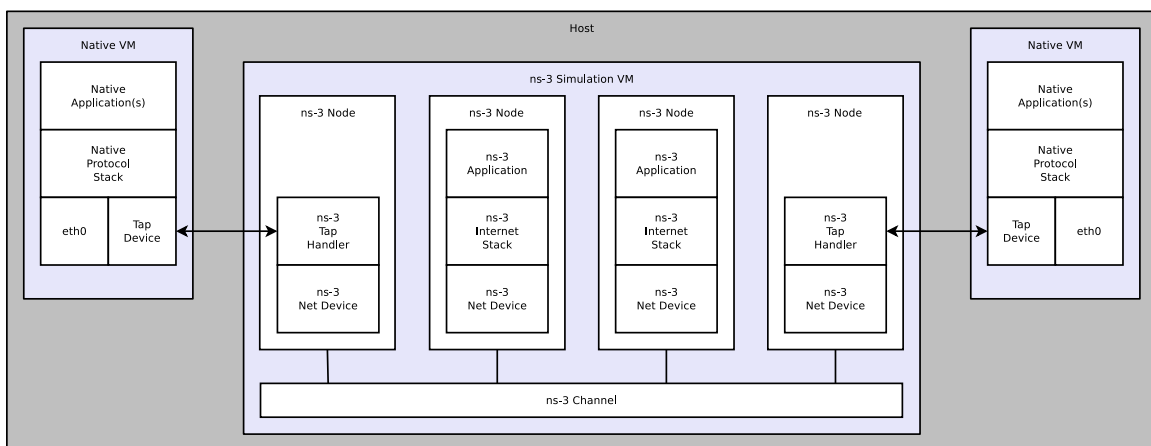


Fig. 13.2: Implementation overview of emulated channel.

We expect the typical use case for this environment will be to analyze the behavior of native applications and protocol suites in the presence of large simulated *ns-3* networks.

The basic testbed mode of emulation uses raw sockets. Two other variants (netmap-based and DPDK-based emulation) have been recently added; these make use of more recent network interface cards that make use of directly-mapped memory capabilities to improve packet processing efficiency.

For more details:

13.1 File Descriptor NetDevice

The `src/fd-net-device` module provides the `FdNetDevice` class, which is able to read and write traffic using a file descriptor provided by the user. This file descriptor can be associated to a TAP device, to a raw socket, to a user space process generating/consuming traffic, etc. The user has full freedom to define how external traffic is generated and *ns-3* traffic is consumed.

Different mechanisms to associate a simulation to external traffic can be provided through helper classes. Two specific helpers are provided:

- `EmuFdNetDeviceHelper` (to associate the *ns-3* device with a physical device in the host machine)
- `TapFdNetDeviceHelper` (to associate the *ns-3* device with the file descriptor from a tap device in the host machine)

13.1.1 Model Description

The source code for this module lives in the directory `src/fd-net-device`.

The `FdNetDevice` is a special type of *ns-3* `NetDevice` that reads traffic to and from a file descriptor. That is, unlike pure simulation `NetDevice` objects that write frames to and from a simulated channel, this `FdNetDevice` directs frames out of the simulation to a file descriptor. The file descriptor may be associated to a Linux TUN/TAP device, to a socket, or to a user-space process.

It is up to the user of this device to provide a file descriptor. The type of file descriptor being provided determines what is being modelled. For instance, if the file descriptor provides a raw socket to a WiFi card on the host machine, the device being modelled is a WiFi device.

From the conceptual “top” of the device looking down, it looks to the simulated node like a device supporting a 48-bit IEEE MAC address that can be bridged, supports broadcast, and uses IPv4 ARP or IPv6 Neighbor Discovery, although these attributes can be tuned on a per-use-case basis.

Design

The `FdNetDevice` implementation makes use of a reader object, extended from the `FdReader` class in the *ns-3* `src/core` module, which manages a separate thread from the main *ns-3* execution thread, in order to read traffic from the file descriptor.

Upon invocation of the `StartDevice` method, the reader object is initialized and starts the reading thread. Before device start, a file descriptor must be previously associated to the `FdNetDevice` with the `SetFileDescriptor` invocation.

The creation and configuration of the file descriptor can be left to a number of helpers, described in more detail below. When this is done, the invocation of `SetFileDescriptor` is responsibility of the helper and must not be directly invoked by the user.

Upon reading an incoming frame from the file descriptor, the reader will pass the frame to the `ReceiveCallback` method, whose task it is to schedule the reception of the frame by the device as a *ns-3* simulation event. Since the new frame

is passed from the reader thread to the main *ns-3* simulation thread, thread-safety issues are avoided by using the `ScheduleWithContext` call instead of the regular `Schedule` call.

In order to avoid overwhelming the scheduler when the incoming data rate is too high, a counter is kept with the number of frames that are currently scheduled to be received by the device. If this counter reaches the value given by the `RxQueueSize` attribute in the device, then the new frame will be dropped silently.

The actual reception of the new frame by the device occurs when the scheduled `ForwardUp` method is invoked by the simulator. This method acts as if a new frame had arrived from a channel attached to the device. The device then decapsulates the frame, removing any layer 2 headers, and forwards it to upper network stack layers of the node. The `ForwardUp` method will remove the frame headers, according to the frame encapsulation type defined by the `EncapsulationMode` attribute, and invoke the receive callback passing an IP packet.

An extra header, the PI header, can be present when the file descriptor is associated to a TAP device that was created without setting the `IFF_NO_PI` flag. This extra header is removed if `EncapsulationMode` is set to `DIXPI` value.

In the opposite direction, packets generated inside the simulation that are sent out through the device, will be passed to the `Send` method, which will in turn invoke the `SendFrom` method. The latter method will add the necessary layer 2 headers, and simply write the newly created frame to the file descriptor.

Scope and Limitations

Users of this device are cautioned that there is no flow control across the file descriptor boundary, when using in emulation mode. That is, in a Linux system, if the speed of writing network packets exceeds the ability of the underlying physical device to buffer the packets, backpressure up to the writing application will be applied to avoid local packet loss. No such flow control is provided across the file descriptor interface, so users must be aware of this limitation.

As explained before, the `RxQueueSize` attribute limits the number of packets that can be pending to be received by the device. Frames read from the file descriptor while the number of pending packets is in its maximum will be silently dropped.

The `mtu` of the device defaults to the Ethernet II MTU value. However, helpers are supposed to set the `mtu` to the right value to reflect the characteristics of the network interface associated to the file descriptor. If no helper is used, then the responsibility of setting the correct `mtu` value for the device falls back to the user. The size of the read buffer on the file descriptor reader is set to the `mtu` value in the `StartDevice` method.

The `FdNetDevice` class currently supports three encapsulation modes, `DIX` for Ethernet II frames, `LLC` for 802.2 LLC/SNAP frames, and `DIXPI` for Ethernet II frames with an additional TAP PI header. This means that traffic traversing the file descriptor is expected to be Ethernet II compatible. IEEE 802.1q (VLAN) tagging is not supported. Attaching an `FdNetDevice` to a wireless interface is possible as long as the driver provides Ethernet II frames to the socket API. Note that to associate a `FdNetDevice` to a wireless card in ad-hoc mode, the MAC address of the device must be set to the real card MAC address, else any incoming traffic a fake MAC address will be discarded by the driver.

As mentioned before, three helpers are provided with the `fd-net-device` module. Each individual helper (file descriptor type) may have platform limitations. For instance, threading, real-time simulation mode, and the ability to create TUN/TAP devices are prerequisites to using the provided helpers. Support for these modes can be found in the output of the `ns3 configure` step, e.g.:

Threading Primitives	: enabled
Real Time Simulator	: enabled
Emulated Net Device	: enabled
Tap Bridge	: enabled

It is important to mention that while testing the `FdNetDevice` we have found an upper bound limit for TCP throughput when using 1Gb Ethernet links of 60Mbps. This limit is most likely due to the processing power of the computers involved in the tests.

13.1.2 Usage

The usage pattern for this type of device is similar to other net devices with helpers that install to node pointers or node containers. When using the base `FdNetDeviceHelper` the user is responsible for creating and setting the file descriptor by himself.

```
FdNetDeviceHelper fd;
NetDeviceContainer devices = fd.Install(nodes);

// file descriptor generation
...

device->SetFileDescriptor(fd);
```

Most commonly a `FdNetDevice` will be used to interact with the host system. In these cases it is almost certain that the user will want to run in real-time emulation mode, and to enable checksum computations. The typical program statements are as follows:

```
GlobalValue::Bind("SimulatorImplementationType", StringValue("ns3::RealtimeSimulatorImpl
→"));
GlobalValue::Bind("ChecksumEnabled", BooleanValue(true));
```

The easiest way to set up an experiment that interacts with a Linux host system is to use the `Emu` and `Tap` helpers. Perhaps the most unusual part of these helper implementations relates to the requirement for executing some of the code with super-user permissions. Rather than force the user to execute the entire simulation as root, we provide a small “creator” program that runs as root and does any required high-permission sockets work. The easiest way to set the right privileges for the “creator” programs, is by enabling the `--enable-sudo` flag when performing `ns3 configure`.

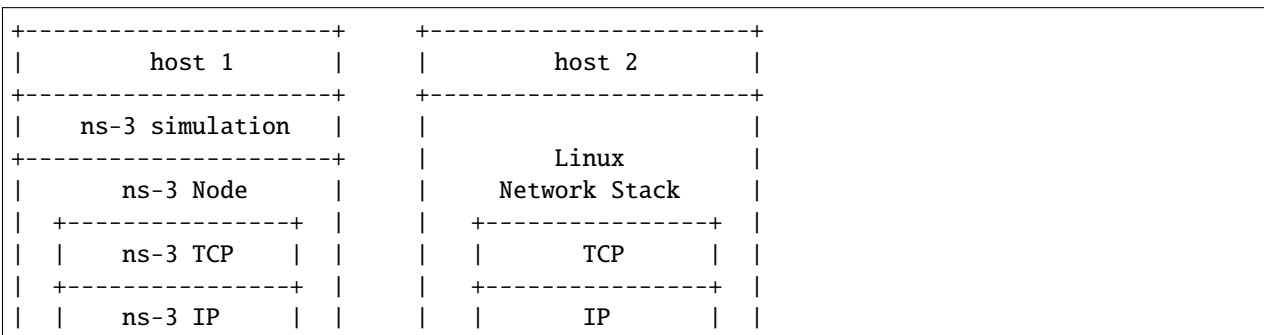
We do a similar thing for both the `Emu` and the `Tap` devices. The high-level view is that the `CreateFileDescriptor` method creates a local interprocess (Unix) socket, forks, and executes the small creation program. The small program, which runs as `suid root`, creates a raw socket and sends back the raw socket file descriptor over the Unix socket that is passed to it as a parameter. The raw socket is passed as a control message (sometimes called ancillary data) of type `SCM_RIGHTS`.

Helpers

EmuFdNetDeviceHelper

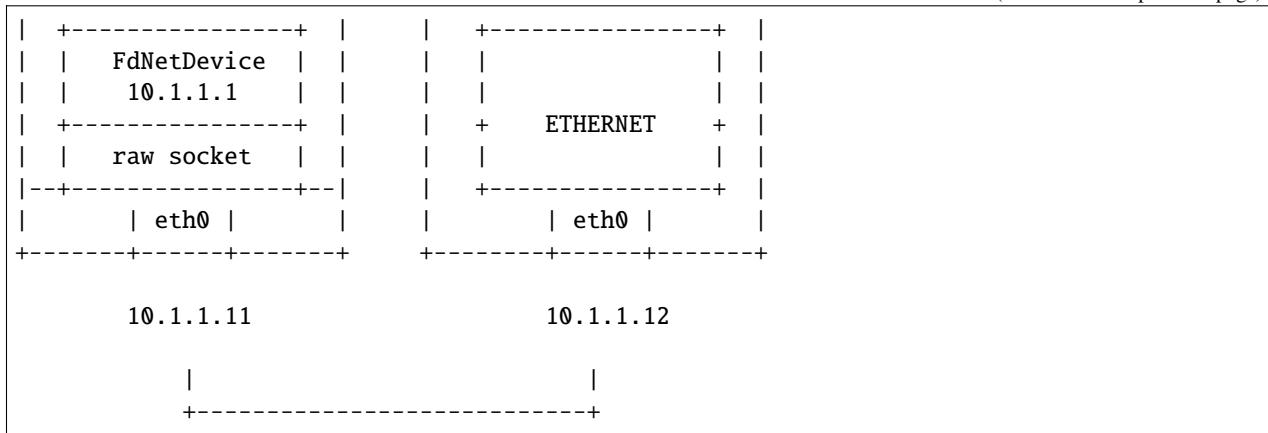
The `EmuFdNetDeviceHelper` creates a raw socket to an underlying physical device, and provides the socket descriptor to the `FdNetDevice`. This allows the *ns-3* simulation to read frames from and write frames to a network device on the host.

The emulation helper permits to transparently integrate a simulated *ns-3* node into a network composed of real nodes.



(continues on next page)

(continued from previous page)



This helper replaces the functionality of the `EmuNetDevice` found in *ns-3* prior to ns-3.17, by bringing this type of device into the common framework of the `FdNetDevice`. The `EmuNetDevice` was deprecated in favor of this new helper.

The device is configured to perform MAC spoofing to separate simulation network traffic from other network traffic that may be flowing to and from the host.

One can use this helper in a testbed situation where the host on which the simulation is running has a specific interface of interest which drives the testbed hardware. You would also need to set this specific interface into promiscuous mode and provide an appropriate device name to the *ns-3* simulation. Additionally, hardware offloading of segmentation and checksums should be disabled.

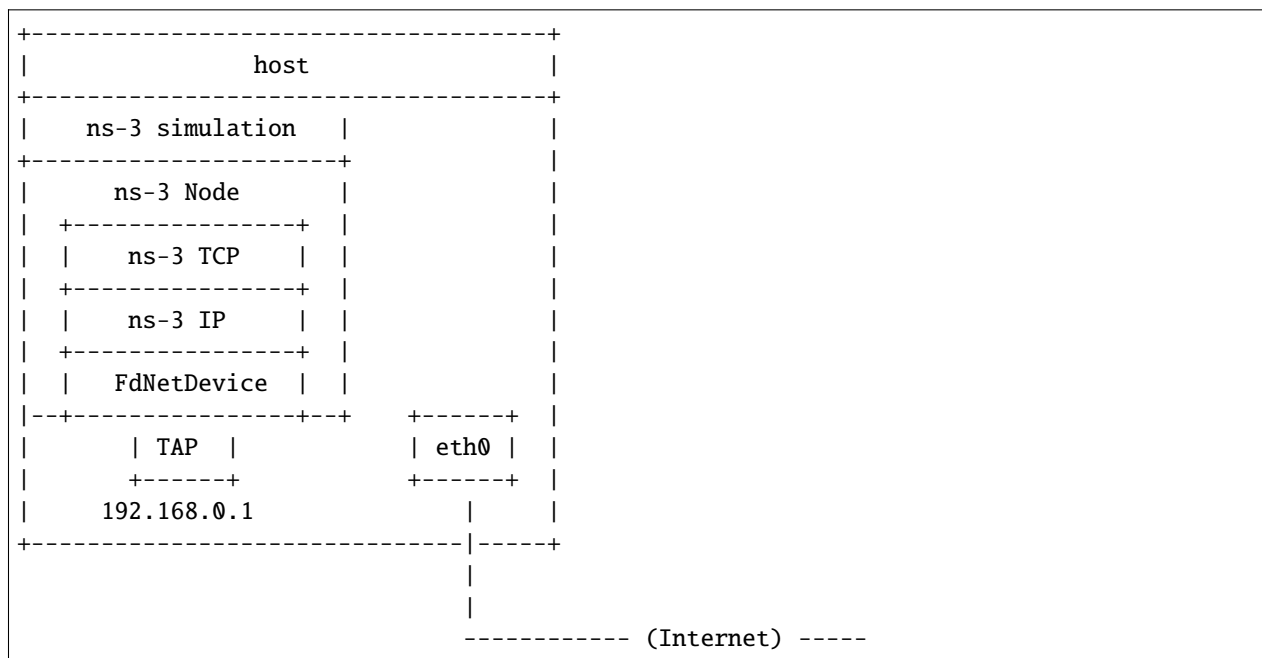
The helper only works if the underlying interface is up and in promiscuous mode. Packets will be sent out over the device, but we use MAC spoofing. The MAC addresses will be generated (by default) using the Organizationally Unique Identifier (OUI) 00:00:00 as a base. This vendor code is not assigned to any organization and so should not conflict with any real hardware.

It is always up to the user to determine that using these MAC addresses is okay on your network and won't conflict with anything else (including another simulation using such devices) on your network. If you are using the emulated `FdNetDevice` configuration in separate simulations, you must consider global MAC address assignment issues and ensure that MAC addresses are unique across all simulations. The emulated net device respects the MAC address provided in the `Address` attribute so you can do this manually. For larger simulations, you may want to set the OUI in the MAC address allocation function.

Before invoking the `Install` method, the correct device name must be configured on the helper using the `SetDeviceName` method. The device name is required to identify which physical device should be used to open the raw socket.

```
EmuFdNetDeviceHelper emu;
emu.SetDeviceName(deviceName);
NetDeviceContainer devices = emu.Install(node);
Ptr<NetDevice> device = devices.Get(0);
device->SetAttribute("Address", Mac48AddressValue(Mac48Address::Allocate()));
```

Users should note that this usage of TAP devices is different than that provided by the TapBridge NetDevice found in `src/tap-bridge`. The model in this helper is as follows:



```

+-----+
| Linux |
| host  |
+-----+
| apps  |
+-----+
| stack |
+-----+
| TAP   |
| device| <----- IPC ----->
+-----+

+-----+
| ghost |
| node  |
+-----+
| IP    |
| stack |
+=====+
| tap   |
| bridge|
+-----+
| ns-3  |
| net   |
| device|
+-----+

||

+-----+
| node  |
+-----+
| IP    |
| stack |
+-----+
| ns-3  |
| net   |
| device|
+-----+

||

+-----+
| ns-3 channel |
+-----+

```

In the above, packets instead traverse *ns-3* NetDevices and Channels.

The usage pattern for this example is that the user sets the MAC address and either (or both) the IPv4 and IPv6 addresses and masks on the device, and the PI header if needed. For example:

```
TapFdNetDeviceHelper helper;
helper.SetDeviceName(deviceName);
helper.SetModePi(modePi);
helper.SetTapIpv4Address(tapIp);
helper.SetTapIpv4Mask(tapMask);
...
helper.Install(node);
```

Attributes

The `FdNetDevice` provides a number of attributes:

- **Address:** The MAC address of the device
- **Start:** The simulation start time to spin up the device thread
- **Stop:** The simulation start time to stop the device thread
- **EncapsulationMode:** Link-layer encapsulation format
- **RxQueueSize:** **The buffer size of the read queue on the file descriptor** thread (default of 1000 packets)

Start and **Stop** do not normally need to be specified unless the user wants to limit the time during which this device is active. **Address** needs to be set to some kind of unique MAC address if the simulation will be interacting with other real devices somehow using real MAC addresses. Typical code:

```
device->SetAttribute("Address", Mac48AddressValue(Mac48Address::Allocate()));
```

Output

Ascii and PCAP tracing is provided similar to the other *ns-3* NetDevice types, through the helpers, such as (e.g.):

```
::
    EmuFdNetDeviceHelper emu; NetDeviceContainer devices = emu.Install(node); ... emu.EnablePcap("emu-
    ping", device, true);
```

The standard set of Mac-level NetDevice trace sources is provided.

- **MacTx:** Trace source triggered when *ns-3* provides the device with a new frame to send
- **MacTxDrop:** Trace source indicating a packet has been dropped by the device before transmission
- **MacPromiscRx:** Whenever any valid Mac frame is received
- **MacRx:** Whenever a valid Mac frame is received for this device
- **Sniffer:** Non-promiscuous packet sniffer
- **PromiscSniffer:** Promiscuous packet sniffer (for tcpdump-like traces)

Examples

Several examples are provided:

- `dummy-network.cc`: This simple example creates two nodes and interconnects them with a Unix pipe by passing the file descriptors from the socketpair into the `FdNetDevice` objects of the respective nodes.
- `realtime-dummy-network.cc`: Same as `dummy-network.cc` but uses the real time simulator implementation instead of the default one.
- `fd2fd-onoff.cc`: This example is aimed at measuring the throughput of the `FdNetDevice` in a pure simulation. For this purpose two `FdNetDevices`, attached to different nodes but in a same simulation, are connected using a socket pair. TCP traffic is sent at a saturating data rate.
- `fd-emu-onoff.cc`: This example is aimed at measuring the throughput of the `FdNetDevice` when using the `EmuFdNetDeviceHelper` to attach the simulated device to a real device in the host machine. This is achieved by saturating the channel with TCP traffic.
- `fd-emu-ping.cc`: This example uses the `EmuFdNetDeviceHelper` to send ICMP traffic over a real channel.
- `fd-emu-udp-echo.cc`: This example uses the `EmuFdNetDeviceHelper` to send UDP traffic over a real channel.
- `fd-tap-ping.cc`: This example uses the `TapFdNetDeviceHelper` to send ICMP traffic over a real channel.

13.2 Netmap NetDevice

The `fd-net-device` module provides the `NetmapNetDevice` class, a class derived from the `FdNetDevice` which is able to read and write traffic using a netmap file descriptor. This netmap file descriptor must be associated to a real ethernet device in the host machine. The `NetmapNetDeviceHelper` class supports the configuration of a `NetmapNetDevice`.

netmap is a fast packet processing capability that bypasses the host networking stack and gains direct access to network device. netmap was developed by Luigi Rizzo [Rizzo2012] and is maintained as an open source project on GitHub at <https://github.com/luigirizzo/netmap>.

The `NetmapNetDevice` for *ns-3* [Imputato2019] was developed by Pasquale Imputato in the 2017-19 timeframe. The use of `NetmapNetDevice` requires that the host system has netmap support (and for best performance, the drivers must support netmap and must be using a netmap-enabled device driver). Users can expect that emulation support using Netmap will support higher packets per second than emulation using `FdNetDevice` with raw sockets (which pass through the Linux networking kernel).

13.2.1 Model Description

Design

Because netmap uses file descriptor based communication to interact with the real device, the straightforward approach to design a new `NetDevice` around netmap is to have it inherit from the existing `FdNetDevice` and implement a specialized version of the operations specific to netmap. The operations that require a specialized implementation are the initialization, because the NIC has to be put in netmap mode, and the read/write methods, which have to make use of the netmap API to coordinate the exchange of packets with the netmap rings.

In the initialization stage, the network device is switched to netmap mode, so that *ns-3* is able to send/receive packets to/from the real network device by writing/reading them to/from the netmap rings. Following the design of the `FdNetDevice`, a separate reading thread is started during the initialization. The task of the reading thread is to wait for new incoming packets in the netmap receiver rings, in order to schedule the events of packet reception. In the initialization of the `NetmapNetDevice`, an additional thread, the sync thread, is started. The sync thread is required because, in order to reduce the cost of the system calls, netmap does not automatically transfer a packet written to a slot

of the netmap ring to the transmission ring or to the installed qdisc. It is up to the user process to periodically request a synchronization of the netmap ring. Therefore, the purpose of the sync thread is to periodically make a TXSYNC ioctl request, so that pending packets in the netmap ring are transferred to the transmission ring, if in native mode, or to the installed qdisc, if in generic mode. Also, as described further below, the sync thread is exploited to perform flow control and notify the BQL library about the amount of bytes that have been transferred to the network device.

The read method is called by the reading thread to retrieve new incoming packets stored in the netmap receiver ring and pass them to the appropriate *ns-3* protocol handler for further processing within the simulator's network stack. After retrieving packets, the reading thread also synchronizes the netmap receiver ring, so that the retrieved packets can be removed from the netmap receiver ring.

The NetmapNetDevice also specializes the write method, i.e., the method used to transmit a packet received from the upper layer (the *ns-3* traffic control layer). The write method uses the netmap API to write the packet to a free slot in the netmap transmission ring. After writing a packet, the write method checks whether there is enough room in the netmap transmission ring for another packet. If not, the NetmapNetDevice stops its queue so that the *ns-3* traffic control layer does not attempt to send a packet that could not be stored in the netmap transmission ring.

A stopped NetmapNetDevice queue needs to be restarted as soon as some room is made in the netmap transmission ring. The sync thread can be exploited for this purpose, given that it periodically synchronizes the netmap transmission ring. In particular, the sync thread also checks the number of free slots in the netmap transmission ring in case the NetmapNetDevice queue is stopped. If the number of free slots exceeds a configurable value, the sync thread restarts the NetmapNetDevice queue and wakes the associated *ns-3* qdisc. The NetmapNetDevice also supports BQL: the write method notifies the BQL library of the amount of bytes that have been written to the netmap transmission ring, while the sync thread notifies the BQL library of the amount of bytes that have been removed from the netmap transmission ring and transferred to the NIC since the previous notification.

Scope and Limitations

The main scope of NetmapNetDevice is to support the flow-control between the physical device and the upper layer and using at best the computational resources to process packets. However, the (Linux) system and network device must support netmap to make use of this feature.

13.2.2 Usage

The installation of netmap itself on a host machine is out of scope for this document. Refer to the [netmap GitHub README](#) for instructions.

The *ns-3* netmap code has only been tested on Linux; it is not clear whether other operating systems can be supported.

If *ns-3* is able to detect the presence of netmap on the system, it will report that:

```
Netmap emulation FdNetDevice : not enabled
```

If not, it will report:

```
Netmap emulation FdNetDevice : not enabled (needs net/netmap_user.h)
```

To run FdNetDevice-enabled simulations, one must pass the `--enable-sudo` option to `./ns3 configure`, or else run the simulations with root privileges.

Helpers

ns-3 netmap support uses a `NetMapNetDeviceHelper` helper object to install the `NetmapNetDevice`. In other respects, the API and use is similar to that of the `EmuFdNetDeviceHelper`.

Attributes

There is one attribute specialized to `NetmapNetDevice`, named `SyncAndNotifyQueuePeriod`. This value takes an integer number of microseconds, and is used as the period of time after which the device syncs the netmap ring and notifies queue status. The value should be close to the interrupt coalescence period of the real device. Users may want to tune this parameter for their own system; it should be a compromise between CPU usage and accuracy in the ring sync (if it is too high, the device goes into starvation and lower throughput occurs).

Output

The `NetmapNetDevice` does not provide any specialized output, but supports the `FdNetDevice` output and traces (such as a promiscuous sniffer trace).

Examples

Several examples are provided:

- `fd-emu-onoff.cc`: This example is aimed at measuring the throughput of the `NetmapNetDevice` when using the `NetmapNetDeviceHelper` to attach the simulated device to a real device in the host machine. This is achieved by saturating the channel with TCP or UDP traffic.
- `fd-emu-ping.cc`: This example uses the `NetmapNetDevice` to send ICMP traffic over a real device.
- **`fd-emu-tc.cc`: This example configures a router on a machine with two** interfaces in emulated mode through netmap. The aim is to explore different qdiscs behaviours on the backlog of a device emulated bottleneck side.
- `fd-emu-send.cc`: This example builds a node with a device in emulation mode through netmap. The aim is to measure the maximum transmit rate in packets per second (pps) achievable with `NetmapNetDevice` on a specific machine.

Note that all the examples run in emulation mode through netmap (with `NetmapNetDevice`) and raw socket (with `FdNetDevice`).

13.3 DPDK NetDevice

Data Plane Development Kit (DPDK) is a library hosted by The Linux Foundation to accelerate packet processing workloads (<https://www.dpdk.org/>).

The `DpdkNetDevice` class provides the implementation of a network device which uses DPDK's fast packet processing abilities and bypasses the kernel. This class is included in the `src/fd-net-device` model. The `DpdkNetDevice` class inherits the `FdNetDevice` class and overrides the functions which are required by *ns-3* to interact with DPDK environment.

The `DpdkNetDevice` for *ns-3* [Patel2019] was developed by Harsh Patel, Hrishikesh Hiraskar and Mohit P. Tahiliani. They were supported by Intel Technology India Pvt. Ltd., Bangalore for this work.

13.3.1 Model Description

DpdkNetDevice is a network device which provides network emulation capabilities i.e. to allow simulated nodes to interact with real hosts and vice versa. The main feature of the DpdkNetDevice is that it uses the Environment Abstraction Layer (EAL) provided by DPDK to perform fast packet processing. EAL hides the device specific attributes from the applications and provides an interface via which the applications can interact directly with the Network Interface Card (NIC). This allows *ns-3* to send/receive packets directly to/from the NIC without the kernel involvement.

Design

DpdkNetDevice is designed to act as an interface between *ns-3* and DPDK environment. There are 3 main phases in the life cycle of DpdkNetDevice:

- Initialization
- Packet Transfer - Read and Write
- Termination

Initialization

DpdkNetDeviceHelper model is responsible for the initialization of DpdkNetDevice. After this, the EAL is initialized, a memory pool is allocated, access to the Ethernet port is obtained and it is initialized, reception (Rx) and transmission (Tx) queues are set up on the port, Rx and Tx buffers are set up and LaunchCore method is called which will launch the HandleRx method to handle reading of packets in burst.

Packet Transfer

DPDK interacts with packet in the form of mbuf, a data structure provided by it, while *ns-3* interacts with packets in the form of raw buffer. The packet transfer functions take care of converting DPDK mbufs to *ns-3* buffers. The functions are read and write.

- Read: HandleRx method takes care of reading the packets from NIC and transferring them to *ns-3* Internet Stack. This function is called by LaunchCore method which is launched during initialization. It continuously polls the NIC using DPDK API for packets to read. It reads the mbuf packets in burst from NIC Rx ring, which are placed into Rx buffer upon read. For each mbuf packet in Rx buffer, it then converts it to *ns-3* raw buffer and then forwards the packet to *ns-3* Internet Stack.
- Write: Write method handles transmission of packets. *ns-3* provides this packet in the form of a buffer, which is converted to packet mbuf and then placed in the Tx buffer. These packets are then transferred to NIC Tx ring when the Tx buffer is full, from where they will be transmitted by the NIC. However, there might be a scenario where there are not enough packets to fill the Tx buffer. This will lead to stale packet mbufs in buffer. In such cases, the Write function schedules a manual flush of these stale packet mbufs to NIC Tx ring, which will occur upon a certain timeout period. The default value of this timeout is set to 2 ms.

Termination

When *ns-3* is done using `DpdkNetDevice`, the `DpdkNetDevice` will stop polling for Rx, free the allocated mbuf packets and then the mbuf pool. Lastly, it will stop the Ethernet device and close the port.

Scope and Limitations

The current implementation supports only one NIC to be bound to DPDK with single Rx and Tx on the NIC. This can be extended to support multiple NICs and multiple Rx/Tx queues simultaneously. Currently there is no support for Jumbo frames, which can be added. Offloading, scheduling features can also be added. Flow control and support for qdisc can be added to provide a more extensive model for network testing.

13.3.2 DPDK Installation

This section contains information on downloading DPDK source code and setting up DPDK for `DpdkNetDevice` to work.

Is my NIC supported by DPDK?

Check [Supported Devices](#).

Not supported? Use Virtual Machine instead

Install [Oracle VM VirtualBox](#). Create a new VM and install Ubuntu on it. Open settings, create a network adapter with following configuration:

- Attached to: Bridged Adapter
- Name: The host network device you want to use
- **In Advanced**
 - Adapter Type: Intel PRO/1000 MT Server (82545EM) or any other DPDK supported NIC
 - Promiscuous Mode: Allow All
 - Select Cable Connected

Then rest of the steps are same as follows.

DPDK can be installed in 2 ways:

- Install DPDK on Ubuntu
- Compile DPDK from source

Install DPDK on Ubuntu

To install DPDK on Ubuntu, run the following command:

```
apt-get install dpdk dpdk-dev libdpdk-dev dpdk-igb-uio-dkms
```

Ubuntu 20.04 has packaged DPDK v19.11 LTS which is tested with this module and DpdkNetDevice will only be enabled if this version is available.

Compile from Source

To compile DPDK from source, you need to perform the following 4 steps:

1. Download the source

Visit the [DPDK Downloads](#) page to download the latest stable source. (This module has been tested with version 19.11 LTS and DpdkNetDevice will only be enabled if this version is available.)

2. Configure DPDK as a shared library

In the DPDK directory, edit the config/common_base file to change the following line to compile DPDK as a shared library:

```
# Compile to share library  
CONFIG_RTE_BUILD_SHARED_LIB=y
```

3. Install the source

Refer to [Installation](#) for detailed instructions.

For a 64 bit linux machine with gcc, run:

```
make install T=x86_64-native-linuxapp-gcc DESTDIR=install
```

4. Export DPDK Environment variables

Export the following environment variables:

- RTE_SDK as the your DPDK source folder.
- RTE_TARGET as the build target directory.

For example:

```
export RTE_SDK=/home/username/dpdk/dpdk-stable-19.11.1  
export RTE_TARGET=x86_64-native-linuxapp-gcc
```

(Note: In case DPDK is moved, ns-3 needs to be reconfigured using `./ns3 configure [options]`)

It is advisable that you export these variables in `.bashrc` or similar for reusability.

Load DPDK Drivers to kernel

Execute the following:

```
sudo modprobe uio_pci_generic
sudo modprobe uio
sudo modprobe vfio-pci

sudo modprobe igb_uio # for ubuntu package
# OR
sudo insmod $RTE_SDK/$RTE_TARGET/kmod/igb_uio.ko # for dpdk source
```

These should be done every time you reboot your system.

Configure hugepages

Refer [System Requirements](#) for detailed instructions.

To allocate hugepages at runtime, write a value such as '256' to the following:

```
echo 256 > /sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages
```

To allocate hugepages at boot time, edit /etc/default/grub, and following to GRUB_CMDLINE_LINUX_DEFAULT:

```
hugepages=256
```

We suggest minimum of number of 256 to run our applications. (This is to test an application run at 1 Gbps on a 1 Gbps NIC.) You can use any number of hugepages based on your system capacity and application requirements.

Then update the grub configurations using:

```
sudo update-grub
```

OR

```
sudo update-grub2
```

You will need to reboot your system in order to see these changes.

To check allocation of hugepages, run:

```
cat /proc/meminfo | grep HugePages
```

You will see the number of hugepages allocated, they should be equal to the number you used above.

Once the hugepage memory is reserved (at either runtime or boot time), to make the memory available for DPDK use, perform the following steps:

```
sudo mkdir /mnt/huge
sudo mount -t hugetlbfs nodev /mnt/huge
```

The mount point can be made permanent across reboots, by adding the following line to the /etc/fstab file:

```
nodev /mnt/huge hugetlbfs defaults 0 0
```

13.3.3 Usage

The status of DPDK support is shown in the output of `./ns3 configure`. If it is found, a user should see:

DPDK NetDevice	: enabled
----------------	-----------

`DpdkNetDeviceHelper` class supports the configuration of `DpdkNetDevice`.

```

+-----+
|      host 1      |
+-----+
| ns-3 simulation  |
+-----+
|      ns-3 Node   |
| +-----+       |
| | ns-3 TCP      | |
| +-----+       |
| | ns-3 IP       | |
| +-----+       |
| | DpdkNetDevice | |
| | 10.1.1.1      | |
| +-----+       |
| | raw socket    | |
| +-----+       |
| | eth0          | |
| +-----+       |
|
| 10.1.1.11
|
| +----- ( Internet ) -----

```

Initialization of DPDK driver requires initialization of EAL. EAL requires PMD (Poll Mode Driver) Library for using NIC. DPDK supports multiple Poll Mode Drivers and you can use one that works for your NIC. PMD Library can be set via `DpdkNetDeviceHelper::SetPmdLibrary`, as follows:

```

DpdkNetDeviceHelper* dpdk = new DpdkNetDeviceHelper();
dpdk->SetPmdLibrary("librte_pmd_e1000.so");

```

Also, NIC should be bound to DPDK Driver in order to be used with EAL. The default driver used is `uio_pci_generic` which supports most of the NICs. You can change it using `DpdkNetDeviceHelper::SetDpdkDriver`, as follows:

```

DpdkNetDeviceHelper* dpdk = new DpdkNetDeviceHelper();
dpdk->SetDpdkDriver("igb_uio");

```


Attributes

The `DpdkNetDevice` provides a number of attributes:

- `TxTimeout` - The time to wait before transmitting burst from Tx Buffer (in us). (default - 2000) This attribute is only used to flush out buffer in case it is not filled. This attribute can be decrease for low data rate traffic. For high data rate traffic, this attribute needs no change.
- `MaxRxBurst` - Size of Rx Burst. (default - 64) This attribute can be increased for higher data rates.
- `MaxTxBurst` - Size of Tx Burst. (default - 64) This attribute can be increased for higher data rates.
- `MempoolCacheSize` - Size of mempool cache. (default - 256) This attribute can be increased for higher data rates.
- `NbRxDesc` - Number of Rx descriptors. (default - 1024) This attribute can be increased for higher data rates.
- `NbTxDesc` - Number of Tx descriptors. (default - 1024) This attribute can be increased for higher data rates.

Note: Default values work well with 1Gbps traffic.

Output

As `DpdkNetDevice` is inherited from `FdNetDevice`, all the output methods provided by `FdNetDevice` can be used directly.

Examples

The following examples are provided:

- `fd-emu-ping.cc`: This example can be configured to use the `DpdkNetDevice` to send ICMP traffic bypassing the kernel over a real channel.
- `fd-emu-onoff.cc`: This example can be configured to measure the throughput of the `DpdkNetDevice` by sending traffic from the simulated node to a real device using the `ns3::OnOffApplication` while leveraging DPDK's fast packet processing abilities. This is achieved by saturating the channel with TCP/UDP traffic.

13.4 Tap NetDevice

The Tap NetDevice can be used to allow a host system or virtual machines to interact with a simulation.

13.4.1 TapBridge Model Overview

The Tap Bridge is designed to integrate “real” internet hosts (or more precisely, hosts that support Tun/Tap devices) into ns-3 simulations. The goal is to make it appear to a “real” host node in that it has an ns-3 net device as a local device. The concept of a “real host” is a bit slippery since the “real host” may actually be virtualized using readily available technologies such as VMware, VirtualBox or OpenVZ.

Since we are, in essence, connecting the inputs and outputs of an ns-3 net device to the inputs and outputs of a Linux Tap net device, we call this arrangement a Tap Bridge.

There are three basic operating modes of this device available to users. Basic functionality is essentially identical, but the modes are different in details regarding how the arrangement is created and configured; and what devices can live on which side of the bridge.

We call these three modes the `ConfigureLocal`, `UseLocal` and `UseBridge` modes. The first “word” in the camel case mode identifier indicates who has the responsibility for creating and configuring the taps. For example, the “Configure”

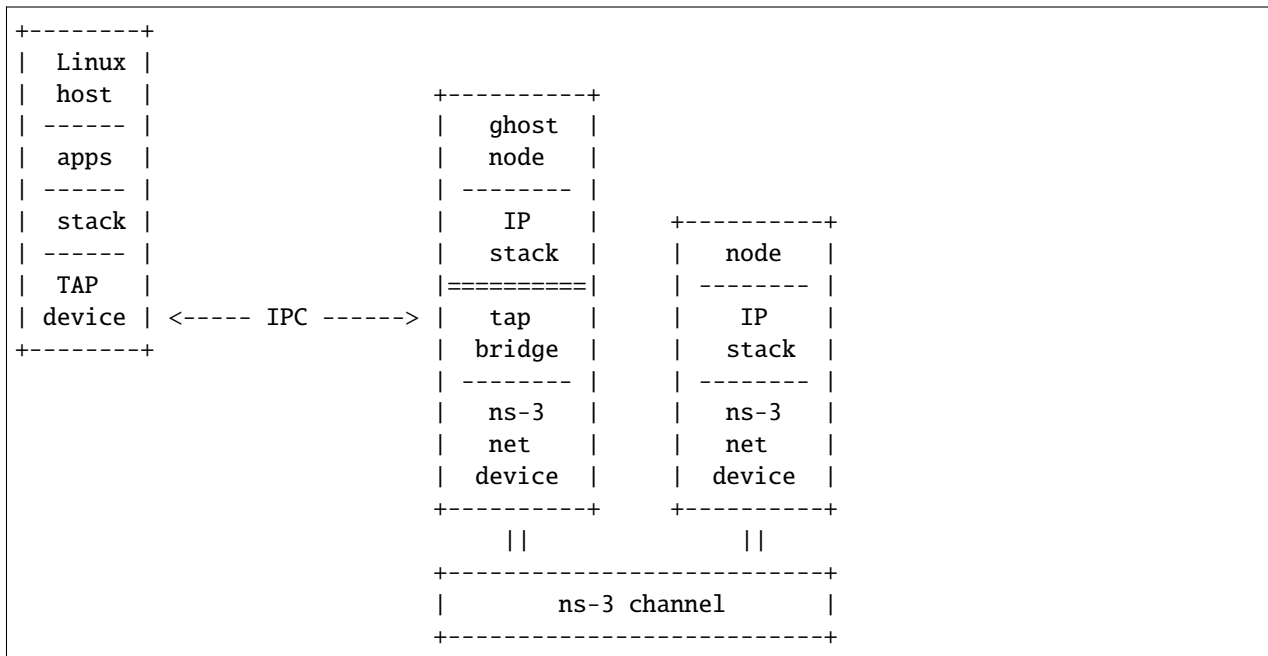
in ConfigureLocal mode indicates that it is the TapBridge that has responsibility for configuring the tap. In UseLocal mode and UseBridge modes, the “Use” prefix indicates that the TapBridge is asked to “Use” an existing configuration.

In other words, in ConfigureLocal mode, the TapBridge has the responsibility for creating and configuring the TAP devices. In UseBridge or UseLocal modes, the user provides a configuration and the TapBridge adapts to that configuration.

TapBridge ConfigureLocal Mode

In the ConfigureLocal mode, the configuration of the tap device is ns-3 configuration-centric. Configuration information is taken from a device in the ns-3 simulation and a tap device matching the ns-3 attributes is automatically created. In this case, a Linux computer is made to appear as if it was directly connected to a simulated ns-3 network.

This is illustrated below:



In this case, the “ns-3 net device” in the “ghost node” appears as if it were actually replacing the TAP device in the Linux host. The ns-3 simulation creates the TAP device on the underlying Linux OS and configures the IP and MAC addresses of the TAP device to match the values assigned to the simulated ns-3 net device. The “IPC” link shown above is the network tap mechanism in the underlying OS. The whole arrangement acts as a conventional bridge; but a bridge between devices that happen to have the same shared MAC and IP addresses.

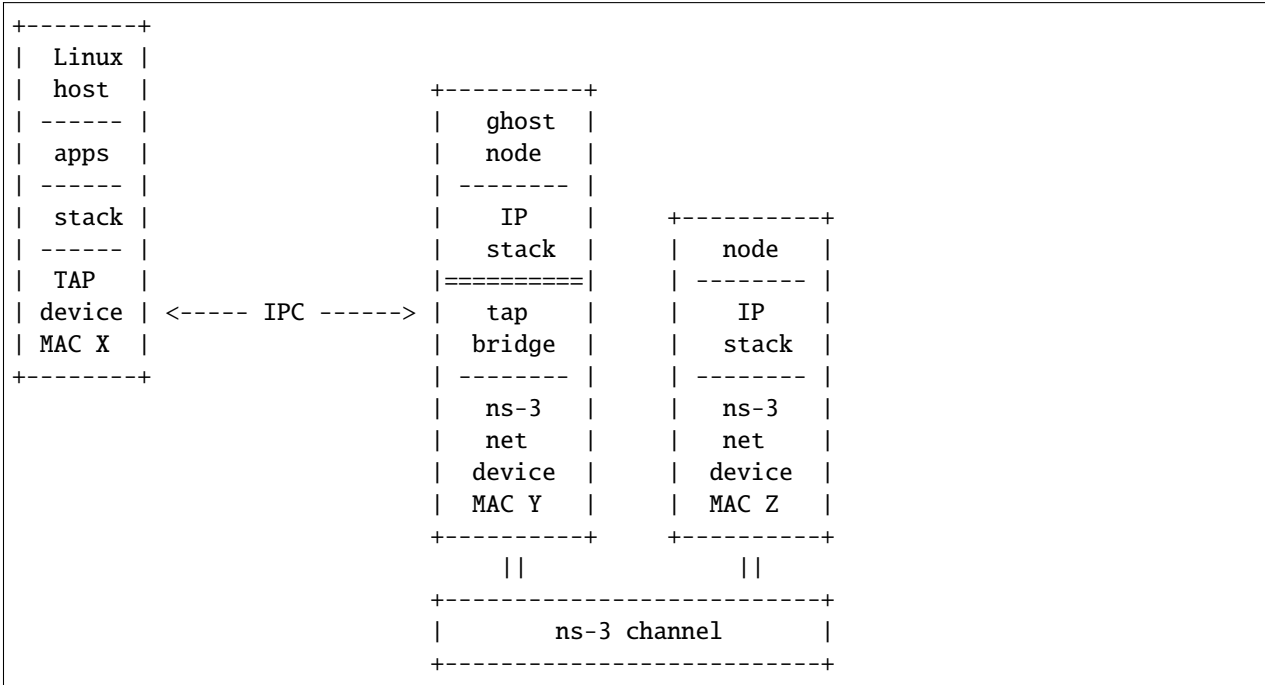
Here, the user is not required to provide any configuration information specific to the tap. A tap device will be created and configured by ns-3 according to its defaults, and the tap device will have its name assigned by the underlying operating system according to its defaults.

If the user has a requirement to access the created tap device, he or she may optionally provide a “DeviceName” attribute. In this case, the created OS tap device will be named accordingly.

The ConfigureLocal mode is the default operating mode of the Tap Bridge.

TapBridge UseLocal Mode

The UseLocal mode is quite similar to the ConfigureLocal mode. The significant difference is, as the mode name implies, the TapBridge is going to “Use” an existing tap device previously created and configured by the user. This mode is particularly useful when a virtualization scheme automatically creates tap devices and ns-3 is used to provide simulated networks for those devices.



In this case, the pre-configured MAC address of the “Tap device” (MAC X) will not be the same as that of the bridged “ns-3 net device” (MAC Y) shown in the illustration above. In order to bridge to ns-3 net devices which do not support SendFrom() (especially wireless STA nodes) we impose a requirement that only one Linux device (with one unique MAC address – here X) generates traffic that flows across the IPC link. This is because the MAC addresses of traffic across the IPC link will be “spoofed” or changed to make it appear to Linux and ns-3 that they have the same address. That is, traffic moving from the Linux host to the ns-3 ghost node will have its MAC address changed from X to Y and traffic from the ghost node to the Linux host will have its MAC address changed from Y to X. Since there is a one-to-one correspondence between devices, there may only be one MAC source flowing from the Linux side. This means that Linux bridges with more than one net device added are incompatible with UseLocal mode.

In UseLocal mode, the user is expected to create and configure a tap device completely outside the scope of the ns-3 simulation using something like:

```

$ sudo ip tuntap add mode tap tap0
$ sudo ip address add 10.1.1.1/24 dev tap0
$ sudo ip link set dev tap0 address 08:00:2e:00:00:01 up

```

To tell the TapBridge what is going on, the user will set either directly into the TapBridge or via the TapBridgeHelper, the “DeviceName” attribute. In the case of the configuration above, the “DeviceName” attribute would be set to “tap0” and the “Mode” attribute would be set to “UseLocal”.

One particular use case for this mode is in the OpenVZ environment. There it is possible to create a Tap device on the “Hardware Node” and move it into a Virtual Private Server. If the TapBridge is able to use an existing tap device it is then possible to avoid the overhead of an OS bridge in that environment.

or “vmnetx” devices that appear local to virtual hosts. In order to connect to such systems, one would need to manually create TAP devices on the host system and bridge these TAP devices to the existing (VM) virtual devices. The job of the Tap Bridge in this case is to extend the bridge to join a ns-3 net device.

TapBridge ConfigureLocal Operation

In ConfigureLocal mode, the TapBridge and therefore its associated ns-3 net device appears to the Linux host computer as a network device just like any arbitrary “eth0” or “ath0” might appear. The creation and configuration of the TAP device is done by the ns-3 simulation and no manual configuration is required by the user. The IP addresses, MAC addresses, gateways, etc., for created TAP devices are extracted from the simulation itself by querying the configuration of the ns-3 device and the TapBridge Attributes.

Since the MAC addresses are identical on the Linux side and the ns-3 side, we can use Send() on the ns-3 device which is available on all ns-3 net devices. Since the MAC addresses are identical there is no requirement to hook the promiscuous callback on the receive side. Therefore there are no restrictions on the kinds of net device that are usable in ConfigureLocal mode.

The TapBridge appears to an ns-3 simulation as a channel-less net device. This device must not have an IP address associated with it, but the bridged (ns-3) net device must have an IP address. Be aware that this is the inverse of an ns-3 BridgeNetDevice (or a conventional bridge in general) which demands that its bridge ports not have IP addresses, but allows the bridge device itself to have an IP address.

The host computer will appear in a simulation as a “ghost” node that contains one TapBridge for each NetDevice that is being bridged. From the perspective of a simulation, the only difference between a ghost node and any other node will be the presence of the TapBridge devices. Note however, that the presence of the TapBridge does affect the connectivity of the net device to the IP stack of the ghost node.

Configuration of address information and the ns-3 devices is not changed in any way if a TapBridge is present. A TapBridge will pick up the addressing information from the ns-3 net device to which it is connected (its “bridged” net device) and use that information to create and configure the TAP device on the real host.

The end result of this is a situation where one can, for example, use the standard ping utility on a real host to ping a simulated ns-3 node. If correct routes are added to the internet host (this is expected to be done automatically in future ns-3 releases), the routing systems in ns-3 will enable correct routing of the packets across simulated ns-3 networks. For an example of this, see the example program, tap-wifi-dumbbell.cc in the ns-3 distribution.

The Tap Bridge lives in a kind of a gray world somewhere between a Linux host and an ns-3 bridge device. From the Linux perspective, this code appears as the user mode handler for a TAP net device. In ConfigureLocal mode, this Tap device is automatically created by the ns-3 simulation. When the Linux host writes to one of these automatically created /dev/tap devices, the write is redirected into the TapBridge that lives in the ns-3 world; and from this perspective, the packet write on Linux becomes a packet read in the Tap Bridge. In other words, a Linux process writes a packet to a tap device and this packet is redirected by the network tap mechanism to an ns-3 process where it is received by the TapBridge as a result of a read operation there. The TapBridge then writes the packet to the ns-3 net device to which it is bridged; and therefore it appears as if the Linux host sent a packet directly through an ns-3 net device onto an ns-3 network.

In the other direction, a packet received by the ns-3 net device connected to the Tap Bridge is sent via a receive callback to the TapBridge. The TapBridge then takes that packet and writes it back to the host using the network tap mechanism. This write to the device will appear to the Linux host as if a packet has arrived on a net device; and therefore as if a packet received by the ns-3 net device during a simulation has appeared on a real Linux net device.

The upshot is that the Tap Bridge appears to bridge a tap device on a Linux host in the “real world” to an ns-3 net device in the simulation. Because the TAP device and the bridged ns-3 net device have the same MAC address and the network tap IPC link is not externalized, this particular kind of bridge makes it appear that a ns-3 net device is actually installed in the Linux host.

In order to implement this on the ns-3 side, we need a “ghost node” in the simulation to hold the bridged ns-3 net device and the TapBridge. This node should not actually do anything else in the simulation since its job is simply to make the

net device appear in Linux. This is not just arbitrary policy, it is because:

- Bits sent to the TapBridge from higher layers in the ghost node (using the TapBridge Send method) are completely ignored. The TapBridge is not, itself, connected to any network, neither in Linux nor in ns-3. You can never send nor receive data over a TapBridge from the ghost node.
- The bridged ns-3 net device has its receive callback disconnected from the ns-3 node and reconnected to the Tap Bridge. All data received by a bridged device will then be sent to the Linux host and will not be received by the node. From the perspective of the ghost node, you can send over this device but you cannot ever receive.

Of course, if you understand all of the issues you can take control of your own destiny and do whatever you want – we do not actively prevent you from using the ghost node for anything you decide. You will be able to perform typical ns-3 operations on the ghost node if you so desire. The internet stack, for example, must be there and functional on that node in order to participate in IP address assignment and global routing. However, as mentioned above, interfaces talking to any TapBridge or associated bridged net devices will not work completely. If you understand exactly what you are doing, you can set up other interfaces and devices on the ghost node and use them; or take advantage of the operational send side of the bridged devices to create traffic generators. We generally recommend that you treat this node as a ghost of the Linux host and leave it to itself, though.

TapBridge UseLocal Mode Operation

As described in above, the TapBridge acts like a bridge from the “real” world into the simulated ns-3 world. In the case of the ConfigureLocal mode, life is easy since the IP address of the Tap device matches the IP address of the ns-3 device and the MAC address of the Tap device matches the MAC address of the ns-3 device; and there is a one-to-one relationship between the devices.

Things are slightly complicated when a Tap device is externally configured with a different MAC address than the ns-3 net device. The conventional way to deal with this kind of difference is to use promiscuous mode in the bridged device to receive packets destined for the different MAC address and forward them off to Linux. In order to move packets the other way, the conventional solution is SendFrom() which allows a caller to “spoof” or change the source MAC address to match the different Linux MAC address.

We do have a specific requirement to be able to bridge Linux Virtual Machines onto wireless STA nodes. Unfortunately, the 802.11 spec doesn’t provide a good way to implement SendFrom(), so we have to work around that problem.

To this end, we provided the UseLocal mode of the Tap Bridge. This mode allows you approach the problem as if you were creating a bridge with a single net device. A single allowed address on the Linux side is remembered in the TapBridge, and all packets coming from the Linux side are repeated out the ns-3 side using the ns-3 device MAC source address. All packets coming in from the ns-3 side are repeated out the Linux side using the remembered MAC address. This allows us to use Send() on the ns-3 device side which is available on all ns-3 net devices.

UseLocal mode is identical to the ConfigureLocal mode except for the creation and configuration of the tap device and the MAC address spoofing.

TapBridge UseBridge Operation

As described in the ConfigureLocal mode section, when the Linux host writes to one of the /dev/tap devices, the write is redirected into the TapBridge that lives in the ns-3 world. In the case of the UseBridge mode, these packets will need to be sent out on the ns-3 network as if they were sent on a device participating in the Linux bridge. This means calling the SendFrom() method on the bridged device and providing the source MAC address found in the packet.

In the other direction, a packet received by an ns-3 net device is hooked via callback to the TapBridge. This must be done in promiscuous mode since the goal is to bridge the ns-3 net device onto the OS bridge of which the TAP device is a part.

For these reasons, only ns-3 net devices that support SendFrom() and have a hookable promiscuous receive callback are allowed to participate in UseBridge mode TapBridge configurations.

13.4.2 Tap Bridge Channel Model

There is no channel model associated with the Tap Bridge. In fact, the intention is make it appear that the real internet host is connected to the channel of the bridged net device.

13.4.3 Tap Bridge Tracing Model

Unlike most ns-3 devices, the TapBridge does not provide any standard trace sources. This is because the bridge is an intermediary that is essentially one function call away from the bridged device. We expect that the trace hooks in the bridged device will be sufficient for most users,

13.4.4 Using the TapBridge

We expect that most users will interact with the TapBridge device through the TapBridgeHelper. Users of other helper classes, such as CSMA or Wifi, should be comfortable with the idioms used there.

ENERGY FRAMEWORK

Energy is a key issue for wireless devices, and network researchers often need to investigate the energy consumption at a node or in the overall network while running network simulations in *ns-3*. This requires *ns-3* to support an energy framework. Further, as concepts such as fuel cells and energy scavenging are becoming viable for low power wireless devices, incorporating the effect of these emerging technologies into simulations requires support for modeling diverse energy models in *ns-3*. The *ns-3* energy framework provides the basis for energy storing consumption and harvesting.

The framework is implemented into the `src/energy/` folder.

The *ns-3* energy framework is composed of 3 essential parts:

- **Energy source models.** Represent storing energy sources such as batteries or capacitors.
- **Energy consumption models.** Represent a portion of a device that draws energy from energy sources. Examples of this include sensors, radio transceivers, vehicles, UAV, etc.
- **Energy harvesting models.** Represent devices that provide energy to energy sources. For example, solar panels and chargers.

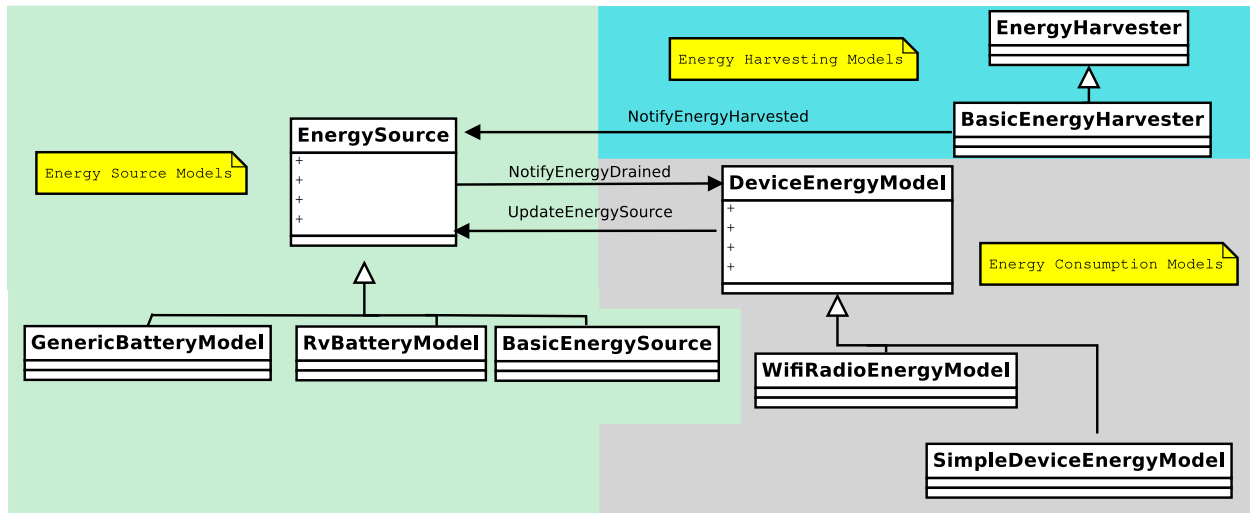


Fig. 14.1: *ns-3* energy framework

14.1 Scope and Limitations

- In the `GenericBatteryModel` charging behavior (voltage as a function of SoC) is included but is not been thoroughly tested. Testing requires the implementation of a harvesting device (A charger) capable of providing a CCCV charging method (typical of rechargeable batteries).
- In the `GenericBatteryModel` impedance (battery resistance) is constant, battery aging or temperature effects are not considered.
- The `Rv` battery model has some reported issues (See: [issue #164](#)).
- The harvesting model can only be used with a `BasicEnergySource` because it does not consider the current capacity or voltage of the battery.
- There is no energy sources or energy models others than the ones mentioned in this document. Support for other communication devices (Ir-wpan, WiMax, etc) and other pieces of hardware (UAV, sensors, CPU) is required.

14.2 Energy Source Models

An energy source represents a power supply. In *ns-3*, nodes can have one or more energy sources. Likewise, energy sources can be connected to multiple energy consumption models (Device energy models). Connecting an energy source to a device energy model implies that the corresponding device draws power from the source. When energy is completely drained from the energy source, it notifies to the device energy models on the node such that each device energy model can react to this event. Further, each node can access the energy source objects for information such as remaining capacity, voltage or state of charge (SoC). This enables the implementation of energy aware protocols in *ns-3*.

In order to model a wide range of power supplies such as batteries, the energy source must be able to capture characteristics of these supplies. There are 2 important characteristics or effects related to practical batteries:

- **Rate Capacity Effect.** Decrease of battery lifetime when the current draw is higher than the rated value of the battery.
- **Recovery Effect.** Increase of battery lifetime when the battery is alternating between discharge and idle states.

In order to incorporate the Rate Capacity Effect, the Energy Source uses current draw from all the devices on the same node to calculate energy consumption. Moreover, multiple Energy Harvesters can be connected to the Energy Source in order to replenish its energy. The Energy Source periodically polls all the devices and energy harvesters on the same node to calculate the total current drain and hence the energy consumption. When a device changes state, its corresponding Device Energy Model will notify the Energy Source of this change and new total current draw will be calculated. Similarly, every Energy Harvester update triggers an update to the connected Energy Source.

The `EnergySource` base class keeps a list of devices (`DeviceEnergyModel` objects) and energy harvesters (`EnergyHarvester` objects) that are using the particular Energy Source as power supply. When energy is completely drained, the Energy Source will notify all devices on this list. Each device can then handle this event independently, based on the desired behavior that should be followed in case of power outage.

14.2.1 Generic Battery Model

The Generic battery model is able to represent 4 basic types of batteries chemistries: Lithium-Ion (LiIon) or Lithium Polymer (LiPo), Nickel Cadmium (NiCd), Lead Acid, and Nickel-metal hydride (NiMH). The main difference between these batteries is the shape of the discharge curves when using constant discharge current and that NiCd and NiMH batteries hysteresis phenomenon is also modeled. Peurket effect, aging, temperature and variable battery impedance is not considered for all batteries. batteries with similar discharge behavior might be also represented but one of the 4 basic arquetipos must be chosen.

The Generic Battery Model is directly based by the works of Trembley et al. Tremblay's model on itself is based on a popular battery model created by Shepherd. Tremblay's model consist in visually identify a set of points from batteries manufacturers' discharge curves datasheets.

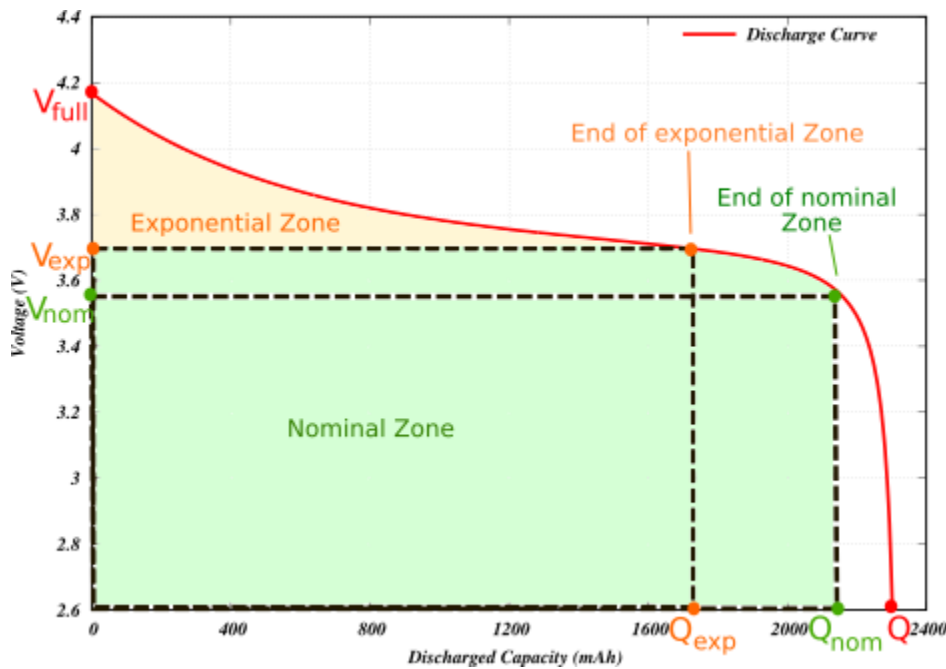


Fig. 14.2: ns-3 Generic Battery Model Points in battery discharge curve.

The 3 basic set of points that require identification in a datasheet are:

- V_{full} : The full battery voltage
- Q : The maximum battery capacity
- V_{exp} : The voltage at the end of the exponential zone
- Q_{exp} : The capacity at the end of the exponential zone
- V_{nom} : The voltage at the end of the exponential zone
- Q_{nom} : The capacity at the end of the exponential zone

Additionally, it is also necessary to set the values of:

- R : The battery impedance (The battery internal resistance)
- $i_{typical}$: The typical current value used to discharge the battery, this value is used to calculate some of the constants used in the model.
- $cutoffVoltage$: Required if we desired to inform connected energy consumption models that the battery has reached its discharged point.

- i : The discharge current used to discharge the battery. This value is provided by the energy consumption model attached to the battery.

The value of R is typically included in the datasheets, however, because R variability is not modeled in ns-3 (The resistance is fixed), it is necessary to discretely adjust its value to obtain the desired discharge curves. The value $i_{typical}$ can be obtained by inferring its value from the discharged curves shown in datasheets. When modeling the behavior of a new battery, it is important to choose values that satisfies more than one curve, trial an error adjustments might be necessary to obtain the desired results.

The process described above can be simplified by installing batteries presents of previously tested batteries using helpers. Details on helpers usage are detailed in the following sections.

14.2.2 Rv Energy Source

The Rakhmatov (Rv) energy source model is the first implementation of a non-linear Li-Ion energy source model for ns-3. Originally created for ns-2, it was later on ported to ns-3 by Wu et al. The Rv model requires some real batteries experiments apriori in order to obtain the attribute values required by the model. For more information read the original author's paper.

14.2.3 Basic Energy Source

The Basic energy source represents an “ideal” linear energy source. Its straightforward design makes it useful for testing and designing energy models. However, users should consider utilizing different energy sources for testing realistic energy consumption scenarios, as the basic energy source is not reflective of most existing energy sources, such as the non-linear nature of energy sources such as batteries.

14.3 Energy Consumption Models

A `DeviceEnergyModel` is the energy consumption model of a device installed on the node. It is designed to be a state based model where each device is assumed to have a number of states, and each state is associated with a power consumption value. Whenever the state of the device changes, the corresponding `DeviceEnergyModel` will notify the associated `EnergySourceModel` of the new current draw of the device. The `EnergySourceModel` will then calculate the new total current draw and update the remaining energy. A `DeviceEnergyModel` can also be used for devices that do not have finite number of states. For example, in an electric vehicle, the current draw of the motor is determined by its speed. Since the vehicle's speed can take continuous values within a certain range, it is infeasible to define a set of discrete states of operation. However, by converting the speed value into current draw directly, the same set of `DeviceEnergyModel` APIs can still be used.

14.3.1 WiFi Radio Energy Model

The WiFi Radio Energy Model is the energy consumption model of a Wifi net device. It provides a state for each of the available states of the PHY layer: Idle, CcaBusy, Tx, Rx, ChannelSwitch, Sleep, Off. Each of such states is associated with a value (in Ampere) of the current draw (see below for the corresponding attribute names). A Wifi Radio Energy Model PHY Listener is registered to the Wifi PHY in order to be notified of every Wifi PHY state transition. At every transition, the energy consumed in the previous state is computed and the energy source is notified in order to update its remaining energy.

The Wifi Tx Current Model gives the possibility to compute the current draw in the transmit state as a function of the nominal tx power (in dBm), as observed in several experimental measurements. To this purpose, the Wifi Radio Energy Model PHY Listener is notified of the nominal tx power used to transmit the current frame and passes such a value to the Wifi Tx Current Model which takes care of updating the current draw in the Tx state. Hence, the energy consumption

is correctly computed even if the Wifi Remote Station Manager performs per-frame power control. Currently, a Linear Wifi Tx Current Model is implemented which computes the tx current as a linear function (according to parameters that can be specified by the user) of the nominal tx power in dBm.

The Wifi Radio Energy Model offers the possibility to specify a callback that is invoked when the energy source is depleted. If such a callback is not specified when the Wifi Radio Energy Model Helper is used to install the model on a device, a callback is implicitly made so that the Wifi PHY is put in the OFF mode (hence no frame is transmitted nor received afterwards) when the energy source is depleted. Likewise, it is possible to specify a callback that is invoked when the energy source is recharged (which might occur in case an energy harvester is connected to the energy source). If such a callback is not specified when the Wifi Radio Energy Model Helper is used to install the model on a device, a callback is implicitly made so that the Wifi PHY is resumed from the OFF mode when the energy source is recharged.

14.4 Energy Harvesting Models

The energy harvester represents the elements that supply energy from the environment and recharge an energy source to which it is connected. The energy harvester includes the complete implementation of the actual energy harvesting device (e.g., a solar panel) and the environment (e.g., the solar radiation). This means that in implementing an energy harvester, the energy contribution of the environment and the additional energy requirements of the energy harvesting device such as the conversion efficiency and the internal power consumption of the device needs to be jointly modeled.

14.5 Usage

The main way that ns-3 users will typically interact with the Energy Framework is through the helper API and through the publicly visible attributes of the framework. The helper API is defined in `src/energy/helper/*.h`.

In order to use the energy framework, the user must install an Energy Source for the node of interest, the corresponding Device Energy Model for the network devices and, if necessary, the one or more Energy Harvester. Energy Source (objects) are aggregated onto each node by the Energy Source Helper. In order to allow multiple energy sources per node, we aggregate an Energy Source Container rather than directly aggregating a source object.

The Energy Source object keeps a list of Device Energy Model and Energy Harvester objects using the source as power supply. Device Energy Model objects are installed onto the Energy Source by the Device Energy Model Helper, while Energy Harvester object are installed by the Energy Harvester Helper. User can access the Device Energy Model objects through the Energy Source object to obtain energy consumption information of individual devices. Moreover, the user can access to the Energy Harvester objects in order to gather information regarding the current harvestable power and the total energy harvested by the harvester.

14.5.1 Helpers

Energy Source Helper:

Base helper class for Energy Source objects, this helper Aggregates Energy Source object onto a node. Child implementation of this class creates the actual Energy Source object.

Device Energy Model Helper:

Base helper class for Device Energy Model objects, this helper attaches Device Energy Model objects onto Energy Source objects. Child implementation of this class creates the actual Device Energy Model object.

Energy Harvesting Helper:

Base helper class for Energy Harvester objects, this helper attaches Energy Harvester objects onto Energy Source objects. Child implementation of this class creates the actual Energy Harvester object.

Generic Battery Model Helper:

The `GenericBatteryModelHelper` can be used to easily install an energy source into a node or node container of one of four types of chemistries (Li-Ion, Lead Acid, NiCd, NiMH). Users must use one of the available presets that represent an specific battery.

```
GenericBatteryModelHelper batteryHelper;
EnergySourceContainer
energySourceContainer = batteryHelper.Install(nodeContainer,
                                             PANASONIC_CGR18650DA_LION);
batteryHelper.SetCellPack(energySourceContainer, 2, 2);
```

In the previous example, the `GenericBatteryModelHelper` was used to install a Panasonic CGR18650DA Li-Ion battery. Likewise, the helper is used to define a cell pack of 4 batteries. 2 batteries connected in series and 2 connected in parallel (2S,2P).

Another option is to manually configure the values that makes the preset:

```
auto node = CreateObject<Node>();
auto devicesEnergyModel = CreateObject<SimpleDeviceEnergyModel>();
batteryModel = CreateObject<GenericBatteryModel>();
batteryModel->SetAttribute("FullVoltage", DoubleValue(1.39));           // Qfull
batteryModel->SetAttribute("MaxCapacity", DoubleValue(7.0));           // Q
batteryModel->SetAttribute("NominalVoltage", DoubleValue(1.18));       // Vnom
batteryModel->SetAttribute("NominalCapacity", DoubleValue(6.25));     // QNom
batteryModel->SetAttribute("ExponentialVoltage", DoubleValue(1.28));  // Vexp
batteryModel->SetAttribute("ExponentialCapacity", DoubleValue(1.3));  // Qexp
batteryModel->SetAttribute("InternalResistance", DoubleValue(0.0046)); // R
batteryModel->SetAttribute("TypicalDischargeCurrent", DoubleValue(1.3)); // i typical
batteryModel->SetAttribute("CutoffVoltage", DoubleValue(1.0));        // End of
↳ charge.
batteryModel->SetAttribute("BatteryType", EnumValue(NIMH_NICD));       // General
↳ battery type
batteryModel = DynamicCast<GenericBatteryModel>
    (batteryHelper.Install(node, PANASONIC_HHR650D_NIMH));
devicesEnergyModel->SetEnergySource(batteryModel);
batteryModel->AppendDeviceEnergyModel(devicesEnergyModel);
devicesEnergyModel->SetNode(node);
```

Usage of both of these type of configurations are shown in `generic-battery-discharge-example.cc`. The following table is a list of the available presents in `ns-3`:

Preset Name	Description
PANASONIC_CGR18650DA_LION	Panasonic Li-Ion (3.6V, 2450Ah, Size A)
PANASONIC_HHR650D_NIMH	Panasonic NiMh HHR550D (1.2V 6.5Ah, Size D)
CSB_GP1272_LEADACID	CSB Lead Acid GP1272 (12V, 7.2Ah)
PANASONIC_N700AAC_NICD	Panasonic NiCd N-700AAC (1.2V 700mAh, Size: AA)
RSPRO_LGP12100_LEADACID	Rs Pro Lead Acid LGP12100 (12V, 100Ah)

All the `ns-3` available presets have been created by carefully choosing the batteries attribute values based on the information found on each battery datasheet.

14.5.2 Attributes

Besides using established presets, users can create their own energy sources by using the **Generic Battery Model** attributes to create their own energy sources (i.e. batteries):

- FullVoltage: Represents the V_{full} value.
- MaxCapacity: Represents the Q value.
- ExponentialVoltage: Represents the V_{exp} value.
- ExponentialCapacity: Represents the Q_{exp} value.
- NominalVoltage: Represents the V_{nom} value.
- NominalCapacity: Represents the Q_{nom} value.
- InternalResistance: Represents the R value.
- TypicalDischargeCurrent: Represents the $i_{typical}$ value.
- CutoffVoltage: The voltage where the battery is considered depleted.
- BatteryType: Indicates the battery type used.
- PeriodicEnergyUpdateInterval: Indicates how often the update values are obtained.
- LowBatteryThreshold: Additional voltage threshold to indicate when the battery has low energy.

Rv Energy source attributes:

- RvBatteryModelPeriodicEnergyUpdateInterval: RV battery model sampling interval.
- RvBatteryModelOpenCircuitVoltage: RV battery model open circuit voltage.
- RvBatteryModelCutoffVoltage: RV battery model cutoff voltage.
- RvBatteryModelAlphaValue: RV battery model alpha value.
- RvBatteryModelBetaValue: RV battery model beta value.
- RvBatteryModelNumOfTerms: The number of terms of the infinite sum for estimating battery level.

Basic Energy source attributes:

- BasicEnergySourceInitialEnergyJ: Initial energy stored in basic energy source.
- BasicEnergySupplyVoltageV: Initial supply voltage for basic energy source.
- PeriodicEnergyUpdateInterval: Time between two consecutive periodic energy updates.

Wifi Radio Energy model attributes:

- IdleCurrentA: The default radio Idle current in Ampere.
- CcaBusyCurrentA: The default radio CCA Busy State current in Ampere.
- TxCurrentA: The radio Tx current in Ampere.
- RxCurrentA: The radio Rx current in Ampere.
- SwitchingCurrentA: The default radio Channel Switch current in Ampere.
- SleepCurrentA: The radio Sleep current in Ampere.
- TxCurrentModel: A pointer to the attached tx current model.

14.5.3 Traces

Traced values differ between Energy Sources, Devices Energy Models and Energy Harvesters implementations, please look at the specific child class for details.

Basic Energy Source

- `RemainingEnergy`: Remaining energy at `BasicEnergySource`.

RV Battery Model

- `RvBatteryModelBatteryLevel`: RV battery model battery level.
- `RvBatteryModelBatteryLifetime`: RV battery model battery lifetime.

WiFi Radio Energy Model

- `TotalEnergyConsumption`: Total energy consumption of the radio device.

Basic Energy Harvester

- `HarvestedPower`: Current power provided by the `BasicEnergyHarvester`.
- `TotalEnergyHarvested`: Total energy harvested by the `BasicEnergyHarvester`.

14.6 Examples and Tests

The following examples have been written.

Examples in `src/energy/examples`:

- `basic-energy-model-test.cc`: Demonstrates the use of a Basic energy source with a Wifi radio model.
- `generic-battery-discharge-example.cc`: Demonstrates the installation of battery energy sources. The output of this example shows the discharge curve of 5 different batteries.
- `generic-battery-discharge-example.py`: A simplified version of the previous example but using python bindings.
- `generic-battery-wifiradio-example.cc`: Demonstrates the use and installation of the Generic Battery Model with the `WifiRadio` model.
- `rv-battery-model-test.cc`: Discharge example of a RV energy source model.

Examples in `examples/energy`:

- `energy-model-example.cc`
- `energy-model-with-harvesting-example.cc`: Shows the harvesting model usage. Only usable with `BasicEnergySource`.

The following tests have been written, which can be found in `src/energy/tests/`:

- `basic-energy-harvester-test.cc`: Test energy harvester procedure
- `li-ion-energy-source-test.cc`: Test the procedure of the li-ion energy source (Deprecated). Use `generic-battery-model` instead.

14.7 Validation

The RV battery model is validated by comparing results with what was presented in the original RV battery model paper. The generic battery model is validated by superimposing the obtained discharge curves with manufacturer's datasheets plots. The following shows the results of the generic-battery-discharge-example.cc superimposed to manufacturer's datasheets charts:

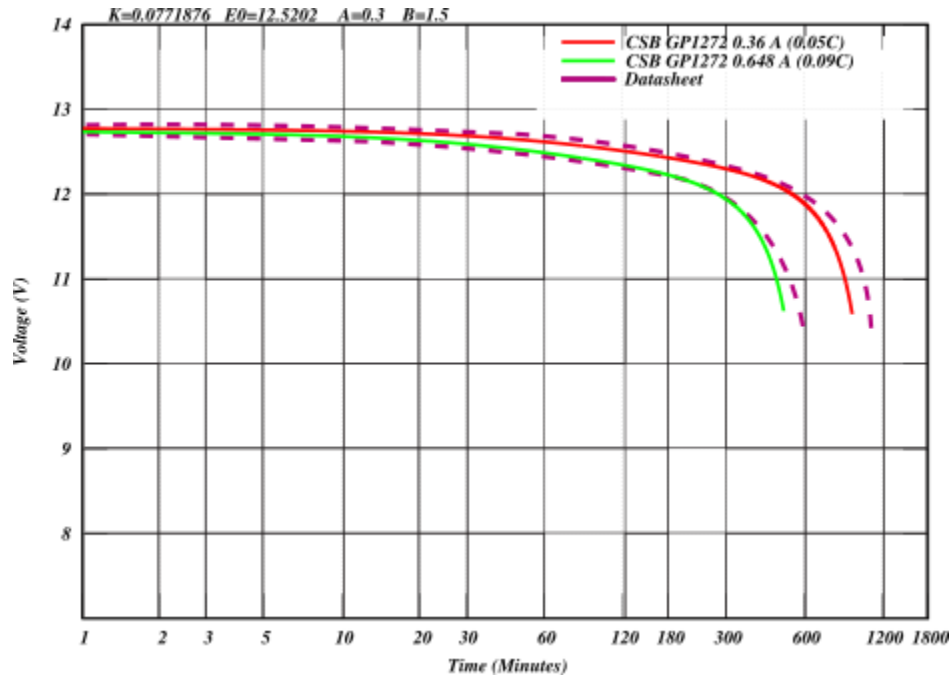


Fig. 14.3: Lead acid battery discharge curve (CSB GP1272)

14.8 References

Energy source models and energy consumption models:

- [1] H. Wu, S. Nabar and R. Poovendran. An Energy Framework for the Network Simulator 3 (ns-3).
- [2] M. Handy and D. Timmermann. Simulation of mobile wireless networks with accurate modelling of non-linear battery effects. In Proc. of Applied simulation and Modeling (ASM), 2003.
- [3] D. N. Rakhmatov and S. B. Vrudhula. An analytical high-level battery model for use in energy management of portable electronic systems. In Proc. of IEEE/ACM International Conference on Computer Aided Design (ICCAD'01), pages 488-493, November 2001.
- [4] D. N. Rakhmatov, S. B. Vrudhula, and D. A. Wallach. Battery lifetime prediction for energy-aware computing. In Proc. of the 2002 International Symposium on Low Power Electronics and Design (ISLPED'02), pages 154-159, 2002.
- [5] Olivier Tremblay and Louis-A. Dessaint. 2009. Experimental Validation of a Battery Dynamic Model for EV Applications. World Electric Vehicle Journal 3, 2 (2009), 289-298. <https://doi.org/10.3390/wevj3020289>
- [6] Olivier Tremblay, Louis-A. Dessaint, and Abdel-Ilah Dekkiche. 2007. A Generic Battery Model for the Dynamic Simulation of Hybrid Electric Vehicles. In 2007 IEEE Vehicle Power and Propulsion Conference. 284-289. <https://doi.org/10.1109/VPPC.2007.4544139>
- [7] MatWorks SimuLink Generic Battery Model

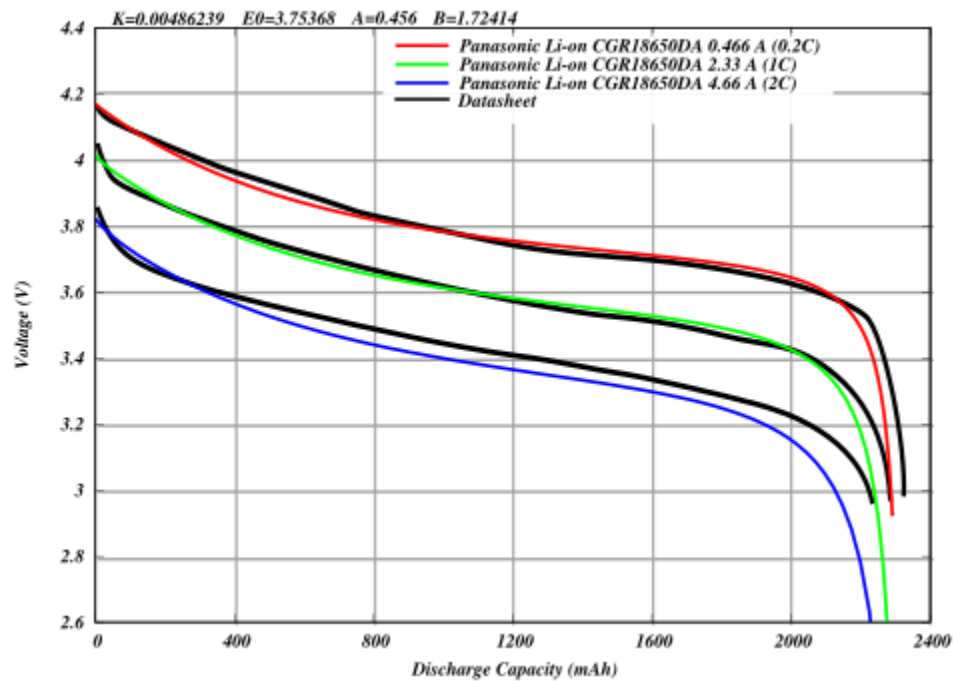


Fig. 14.4: Li-Ion battery discharge curve (Panasonic CGR18650DA)

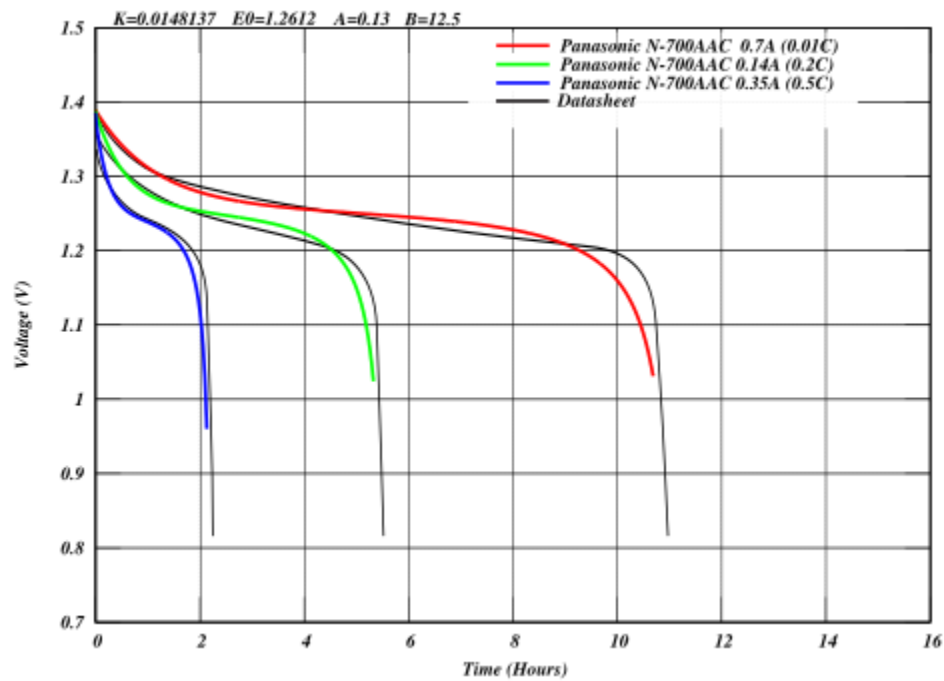


Fig. 14.5: NiCd battery discharge curve (Panasonic N-700AAC)

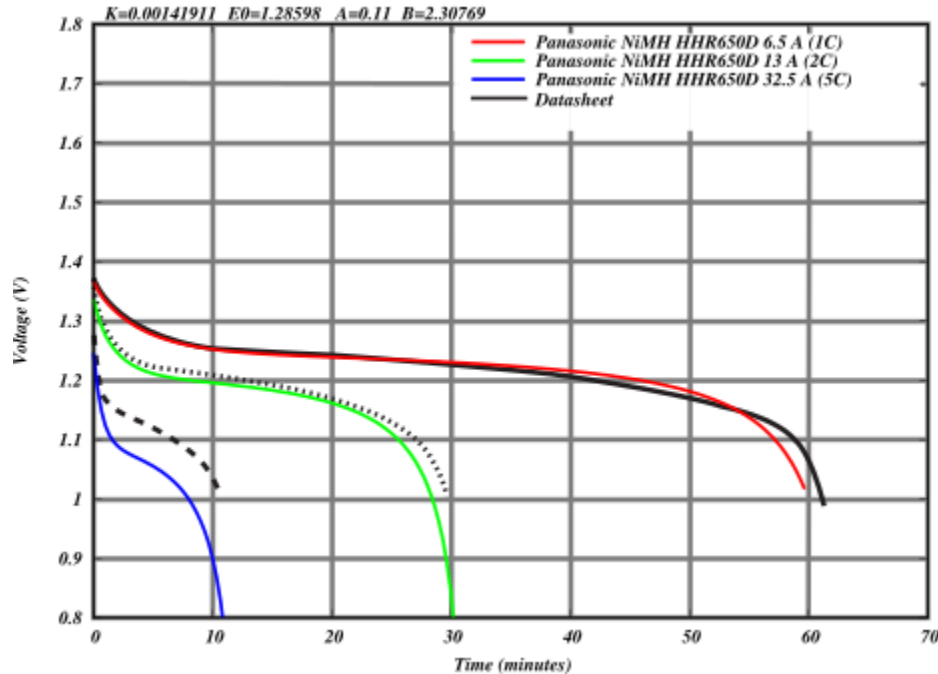


Fig. 14.6: NiMH battery discharge curve (Panasonic HHR650D)

- [8] C. M. Shepherd. 1965. Design of Primary and Secondary Cells: II . An Equation Describing Battery Discharge. *Journal of The Electrochemical Society* 112, 7 (jul 1965), 657. <https://doi.org/10.1149/1.2423659>
- [9] Alberto Gallegos Ramonet, Alexander Guzman Urbina, and Kazuhiko Kinoshita. 2023. Evaluation and Extension of ns-3 Battery Framework. In *Proceedings of the 2023 Workshop on ns-3 (WNS3 '23)*. Association for Computing Machinery, New York, NY, USA, 102–108. <https://doi.org/10.1145/3592149.3592156>

Energy harvesting models:

- [10] C. Tapparello, H. Ayatollahi and W. Heinzelman. Extending the Energy Framework for Network Simulator 3 (ns-3). *Workshop on ns-3 (WNS3)*, Poster Session, Atlanta, GA, USA. May, 2014.
- [11] C. Tapparello, H. Ayatollahi and W. Heinzelman. Energy Harvesting Framework for Network Simulator 3 (ns-3). *2nd International Workshop on Energy Neutral Sensing Systems (ENSsys)*, Memphis, TN, USA. November 6, 2014.

FLOW MONITOR

The Flow Monitor module goal is to provide a flexible system to measure the performance of network protocols. The module uses probes, installed in network nodes, to track the packets exchanged by the nodes, and it will measure a number of parameters. Packets are divided according to the flow they belong to, where each flow is defined according to the probe's characteristics (e.g., for IP, a flow is defined as the packets with the same {protocol, source (IP, port), destination (IP, port)} tuple.

The statistics are collected for each flow can be exported in XML format. Moreover, the user can access the probes directly to request specific stats about each flow.

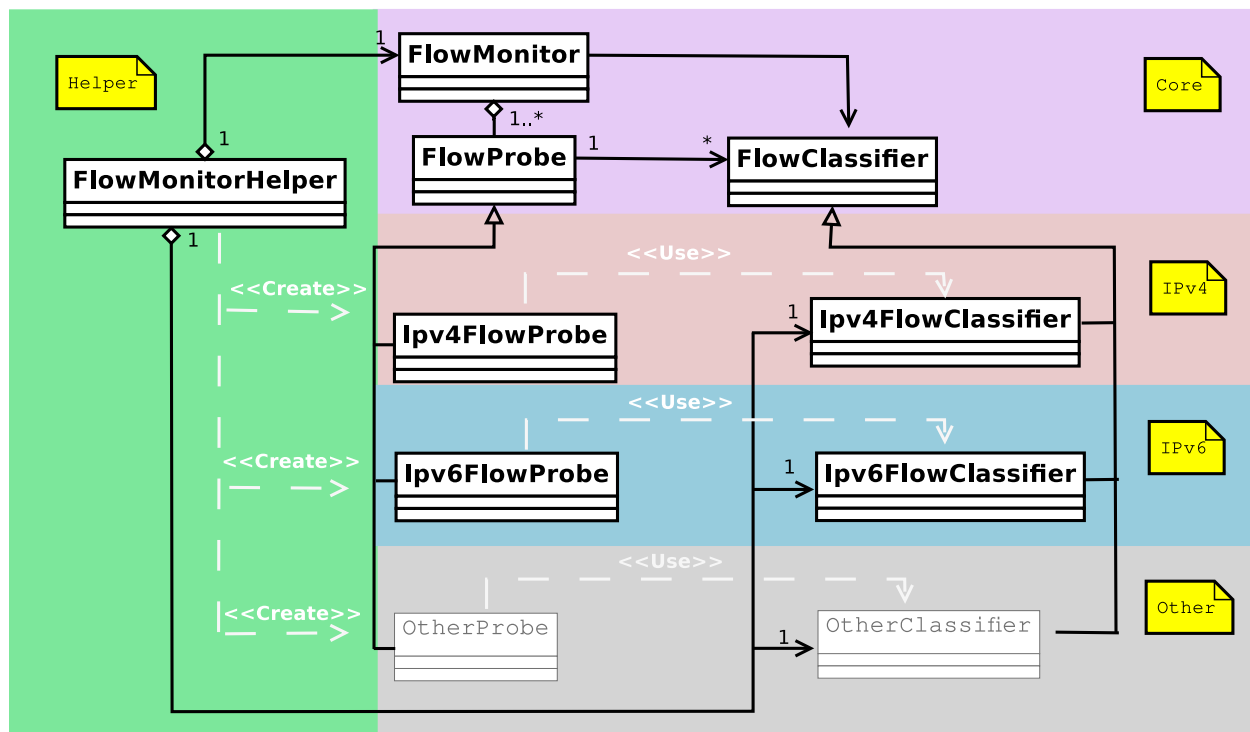


Fig. 15.1: High-level Flow Monitor Architecture

Flow Monitor module is designed in a modular way. It can be extended by subclassing `ns3::FlowProbe` and `ns3::FlowClassifier`. Typically, a subclass of `ns3::FlowProbe` works by listening to the appropriate class Traces, and then uses its own `ns3::FlowClassifier` subclass to classify the packets passing through each node.

Each Probe can try to listen to other classes traces (e.g., `ns3::Ipv4FlowProbe` will try to use any `ns3::NetDevice` trace named `TxQueue/Drop`) but this is something that the user should not rely into blindly, because the trace is not

guaranteed to be in every type of `ns3::NetDevice`. As an example, `CsmaNetDevice` and `PointToPointNetDevice` have a `TxQueue/Drop` trace, while `WiFiNetDevice` does not.

The source code for the module lives in the directory `src/flow-monitor`.

15.1 Scope and Limitations

At the moment, probes and classifiers are available only for IPv4 and IPv6.

IPv4 and IPv6 probes will classify packets in four points:

- When a packet is sent (`SendOutgoing IPv[4,6]` traces)
- When a packet is forwarded (`UnicastForward IPv[4,6]` traces)
- When a packet is received (`LocalDeliver IPv[4,6]` traces)
- When a packet is dropped (`Drop IPv[4,6]` traces)

Since the packets are tracked at IP level, any retransmission caused by L4 protocols (e.g., TCP) will be seen by the probe as a new packet.

A Tag will be added to the packet (`ns3::Ipv[4,6]FlowProbeTag`). The tag will carry basic packet's data, useful for the packet's classification.

It must be underlined that only L4 (TCP, UDP) packets are, so far, classified. Moreover, only unicast packets will be classified. These limitations may be removed in the future.

The data collected for each flow are:

- `timeFirstTxPacket`: when the first packet in the flow was transmitted;
- `timeLastTxPacket`: when the last packet in the flow was transmitted;
- `timeFirstRxPacket`: when the first packet in the flow was received by an end node;
- `timeLastRxPacket`: when the last packet in the flow was received;
- `delaySum`: the sum of all end-to-end delays for all received packets of the flow;
- `jitterSum`: the sum of all end-to-end delay jitter (delay variation) values for all received packets of the flow, as defined in [RFC 3393](#);
- `txBytes`, `txPackets`: total number of transmitted bytes / packets for the flow;
- `rxBytes`, `rxPackets`: total number of received bytes / packets for the flow;
- `lostPackets`: total number of packets that are assumed to be lost (not reported over 10 seconds);
- `timesForwarded`: the number of times a packet has been reportedly forwarded;
- `delayHistogram`, `jitterHistogram`, `packetSizeHistogram`: histogram versions for the delay, jitter, and packet sizes, respectively;
- `packetsDropped`, `bytesDropped`: the number of lost packets and bytes, divided according to the loss reason code (defined in the probe).

It is worth pointing out that the probes measure the packet bytes including IP headers. The L2 headers are not included in the measure.

These stats will be written in XML form upon request (see the Usage section).

Due to the above design, `FlowMonitor` can not generate statistics when used with DSR routing protocol (because DSR forwards packets using broadcast addresses)

The “lost” packets problem

At the end of a simulation, Flow Monitor could report about “lost” packets, i.e., packets that Flow Monitor have lost track of.

It is important to keep in mind that Flow Monitor records the packets statistics by intercepting them at a given network level - let’s say at IP level. When the simulation ends, any packet queued for transmission below the IP level will be considered as lost.

It is strongly suggested to consider this point when using Flow Monitor. The user can choose to:

- Ignore the lost packets (if their number is a statistically irrelevant quantity), or
- Stop the Applications before the actual Simulation End time, leaving enough time between the two for the queued packets to be processed.

The second method is the suggested one. Usually a few seconds are enough (the exact value depends on the network type).

It is important to stress that “lost” packets could be anywhere in the network, and could count toward the received packets or the dropped ones. Ideally, their number should be zero or a minimal fraction of the other ones, i.e., they should be “statistically irrelevant”.

15.2 Usage

The module usage is extremely simple. The helper will take care of about everything. There are however, a few things that must be considered when using Flow Monitor:

15.2.1 Helpers

The helper API follows the pattern usage of normal helpers. Through the helper you can install the monitor in the nodes, set the monitor attributes, and print the statistics.

Important: When using the `ns3::FlowMonitorHelper` must be instantiated only once in the main function.

The typical use is:

```
// Flow monitor
Ptr<FlowMonitor> flowMonitor;
FlowMonitorHelper flowHelper;
flowMonitor = flowHelper.InstallAll();

-yourApplicationsContainer-.Stop(Seconds(stop_time));;
Simulator::Stop(Seconds(stop_time+cleanup_time));
Simulator::Run();

flowMonitor->SerializeToFile("NameOfFile.xml", true, true);
```

the `SerializeToFile()` function 2nd and 3rd parameters are used respectively to activate/deactivate the histograms and the per-probe detailed stats. Other possible alternatives can be found in the Doxygen documentation, while `cleanup_time` is the time needed by in-flight packets to reach their destinations.

XML file output

The main model output is an XML formatted report about flow statistics. An example is:

```

<?xml version="1.0" ?>
<FlowMonitor>
  <FlowStats>
    <Flow flowId="1" timeFirstTxPacket="+0.0ns" timeFirstRxPacket="+20067198.0ns"
    ↳ timeLastTxPacket="+2235764408.0ns" timeLastRxPacket="+2255831606.0ns" delaySum=
    ↳ "+138731526300.0ns" jitterSum="+1849692150.0ns" lastDelay="+20067198.0ns" txBytes=
    ↳ "2149400" rxBytes="2149400" txPackets="3735" rxPackets="3735" lostPackets="0"
    ↳ timesForwarded="7466">
    </Flow>
  </FlowStats>
  <Ipv4FlowClassifier>
    <Flow flowId="1" sourceAddress="10.1.3.1" destinationAddress="10.1.2.2" protocol="6"
    ↳ sourcePort="49153" destinationPort="50000" />
  </Ipv4FlowClassifier>
  <Ipv6FlowClassifier>
  </Ipv6FlowClassifier>
  <FlowProbes>
    <FlowProbe index="0">
      <FlowStats flowId="1" packets="3735" bytes="2149400" delayFromFirstProbeSum="+0.0ns
      ↳ ">
    </FlowStats>
  </FlowProbe>
    <FlowProbe index="2">
      <FlowStats flowId="1" packets="7466" bytes="2224020" delayFromFirstProbeSum=
      ↳ "+199415389258.0ns" >
    </FlowStats>
  </FlowProbe>
    <FlowProbe index="4">
      <FlowStats flowId="1" packets="3735" bytes="2149400" delayFromFirstProbeSum=
      ↳ "+138731526300.0ns" >
    </FlowStats>
  </FlowProbe>
</FlowProbes>
</FlowMonitor>

```

The output was generated by a TCP flow from 10.1.3.1 to 10.1.2.2.

It is worth noticing that the index 2 probe is reporting more packets and more bytes than the other probes. That's a perfectly normal behaviour, as packets are fragmented at IP level in that node.

It should also be observed that the receiving node's probe (index 4) doesn't count the fragments, as the reassembly is done before the probing point.

15.2.2 Attributes

The module provides the following attributes in `ns3::FlowMonitor`:

- `MaxPerHopDelay` (Time, default 10s): The maximum per-hop delay that should be considered;
- `StartTime` (Time, default 0s): The time when the monitoring starts;
- `DelayBinWidth` (double, default 0.001): The width used in the delay histogram;
- `JitterBinWidth` (double, default 0.001): The width used in the jitter histogram;
- `PacketSizeBinWidth` (double, default 20.0): The width used in the packetSize histogram;

- `FlowInterruptionsBinWidth` (double, default 0.25): The width used in the `flowInterruptions` histogram;
- `FlowInterruptionsMinTime` (double, default 0.5): The minimum inter-arrival time that is considered a flow interruption.

15.2.3 Traces

No traces are provided by this module as it is the module itself which provides a simple monitoring functionality to other networks.

15.3 Examples and Tests

The examples are located in *src/flow-monitor/examples*.

Moreover, the following examples use the `flow-monitor` module:

- `examples/matrix-topology/matrix-topology.cc`
- `examples/routing/manet-routing-compare.cc`
- `examples/routing/simple-global-routing.cc`
- `examples/tcp/tcp-variants-comparison.cc`
- `examples/wireless/wifi-multirate.cc`
- `examples/wireless/wifi-hidden-terminal.cc`

Tests are provided to ensure the histogram correct functionality.

15.4 Validation

The paper in the references contains a full description of the module validation against a test network.

15.5 References

[1] G. Carneiro, P. Fortuna, and M. Ricardo. 2009. FlowMonitor: a network monitoring framework for the network simulator 3 (NS-3). In Proceedings of the Fourth International ICST Conference on Performance Evaluation Methodologies and Tools (VALUETOOLS '09). <http://dx.doi.org/10.4108/ICST.VALUETOOLS2009.7493>.

INTERNET MODELS (IP, TCP, ROUTING, UDP)

16.1 Internet Stack

16.1.1 Internet stack aggregation

A bare class `Node` is not very useful as-is; other objects must be aggregated to it to provide useful node functionality.

The *ns-3* source code directory `src/internet` provides implementation of TCP/IPv4- and IPv6-related components. These include IPv4, ARP, UDP, TCP, IPv6, Neighbor Discovery, and other related protocols.

Internet Nodes are not subclasses of class `Node`; they are simply Nodes that have had a bunch of IP-related objects aggregated to them. They can be put together by hand, or via a helper function `InternetStackHelper::Install ()` which does the following to all nodes passed in as arguments:

```
void
InternetStackHelper::Install(Ptr<Node> node) const
{
    if (m_ipv4Enabled)
    {
        /* IPv4 stack */
        if (node->GetObject<Ipv4>() != 0)
        {
            NS_FATAL_ERROR("InternetStackHelper::Install(): Aggregating "
                           "an InternetStack to a node with an existing Ipv4 object");
            return;
        }

        CreateAndAggregateObjectFromTypeId(node, "ns3::ArpL3Protocol");
        CreateAndAggregateObjectFromTypeId(node, "ns3::Ipv4L3Protocol");
        CreateAndAggregateObjectFromTypeId(node, "ns3::Icmpv4L4Protocol");
        // Set routing
        Ptr<Ipv4> ipv4 = node->GetObject<Ipv4>();
        Ptr<Ipv4RoutingProtocol> ipv4Routing = m_routing->Create(node);
        ipv4->SetRoutingProtocol(ipv4Routing);
    }

    if (m_ipv6Enabled)
    {
        /* IPv6 stack */
        if (node->GetObject<Ipv6>() != 0)
        {
```

(continues on next page)

(continued from previous page)

```

        NS_FATAL_ERROR("InternetStackHelper::Install(): Aggregating "
                        "an InternetStack to a node with an existing Ipv6 object");
    return;
}

CreateAndAggregateObjectFromTypeId(node, "ns3::Ipv6L3Protocol");
CreateAndAggregateObjectFromTypeId(node, "ns3::Icmpv6L4Protocol");
// Set routing
Ptr<Ipv6> ipv6 = node->GetObject<Ipv6>();
Ptr<Ipv6RoutingProtocol> ipv6Routing = m_routingv6->Create(node);
ipv6->SetRoutingProtocol(ipv6Routing);

/* register IPv6 extensions and options */
ipv6->RegisterExtensions();
ipv6->RegisterOptions();
}

if (m_ipv4Enabled || m_ipv6Enabled)
{
    /* UDP and TCP stacks */
    CreateAndAggregateObjectFromTypeId(node, "ns3::UdpL4Protocol");
    node->AggregateObject(m_tcpFactory.Create<Object>());
    Ptr<PacketSocketFactory> factory = CreateObject<PacketSocketFactory>();
    node->AggregateObject(factory);
}
}

```

Where multiple implementations exist in *ns-3* (TCP, IP routing), these objects are added by a factory object (TCP) or by a routing helper (*m_routing*).

Note that the routing protocol is configured and set outside this function. By default, the following protocols are added:

```

void InternetStackHelper::Initialize()
{
    SetTcp("ns3::TcpL4Protocol");
    Ipv4StaticRoutingHelper staticRouting;
    Ipv4GlobalRoutingHelper globalRouting;
    Ipv4ListRoutingHelper listRouting;
    Ipv6ListRoutingHelper listRoutingv6;
    Ipv6StaticRoutingHelper staticRoutingv6;
    listRouting.Add(staticRouting, 0);
    listRouting.Add(globalRouting, -10);
    listRoutingv6.Add(staticRoutingv6, 0);
    SetRoutingHelper(listRouting);
    SetRoutingHelper(listRoutingv6);
}

```

By default, IPv4 and IPv6 are enabled.

Internet Node structure

An IP-capable Node (an *ns-3* Node augmented by aggregation to have one or more IP stacks) has the following internal structure.

Layer-3 protocols

At the lowest layer, sitting above the NetDevices, are the “layer 3” protocols, including IPv4, IPv6, ARP and so on. The class `Ipv4L3Protocol` is an implementation class whose public interface is typically class `Ipv4`, but the `Ipv4L3Protocol` public API is also used internally at present.

In class `Ipv4L3Protocol`, one method described below is `Receive ()`:

```
/**
 * Lower layer calls this method after calling L3Demux::Lookup
 * The ARP subclass needs to know from which NetDevice this
 * packet is coming to:
 * - implement a per-NetDevice ARP cache
 * - send back arp replies on the right device
 */
void Receive( Ptr<NetDevice> device, Ptr<const Packet> p, uint16_t protocol,
const Address &from, const Address &to, NetDevice::PacketType packetType);
```

First, note that the `Receive ()` function has a matching signature to the `ReceiveCallback` in the class `Node`. This function pointer is inserted into the Node’s protocol handler when `AddInterface ()` is called. The actual registration is done with a statement such as follows:

```
RegisterProtocolHandler( MakeCallback(&Ipv4Protocol::Receive, ipv4),
                        Ipv4L3Protocol::PROT_NUMBER, 0);
```

The `Ipv4L3Protocol` object is aggregated to the `Node`; there is only one such `Ipv4L3Protocol` object. Higher-layer protocols that have a packet to send down to the `Ipv4L3Protocol` object can call `GetObject<Ipv4L3Protocol>()` to obtain a pointer, as follows:

```
Ptr<Ipv4L3Protocol> ipv4 = m_node->GetObject<Ipv4L3Protocol>();
if (ipv4 != 0)
{
    ipv4->Send(packet, saddr, daddr, PROT_NUMBER);
}
```

This class nicely demonstrates two techniques we exploit in *ns-3* to bind objects together: callbacks, and object aggregation.

Once IPv4 routing has determined that a packet is for the local node, it forwards it up the stack. This is done with the following function:

```
void
Ipv4L3Protocol::LocalDeliver(Ptr<const Packet> packet, Ipv4Header const&ip, uint32_t iif)
```

The first step is to find the right `Ipv4L4Protocol` object, based on IP protocol number. For instance, TCP is registered in the demux as protocol number 6. Finally, the `Receive()` function on the `Ipv4L4Protocol` (such as `TcpL4Protocol::Receive` is called.

We have not yet introduced the class `Ipv4Interface`. Basically, each `NetDevice` is paired with an IPv4 representation of such device. In Linux, this class `Ipv4Interface` roughly corresponds to the struct `in_device`; the main purpose is

to provide address-family specific information (addresses) about an interface.

All the classes have appropriate traces in order to track sent, received and lost packets. The users is encouraged to use them so to find out if (and where) a packet is dropped. A common mistake is to forget the effects of local queues when sending packets, e.g., the ARP queue. This can be particularly puzzling when sending jumbo packets or packet bursts using UDP. The ARP cache pending queue is limited (3 datagrams) and IP packets might be fragmented, easily overfilling the ARP cache queue size. In those cases it is useful to increase the ARP cache pending size to a proper value, e.g.:

```
Config::SetDefault("ns3::ArpCache::PendingQueueSize", UIntegerValue(MAX_BURST_SIZE/  
↪L2MTU*3));
```

The IPv6 implementation follows a similar architecture. Dual-stacked nodes (one with support for both IPv4 and IPv6) will allow an IPv6 socket to receive IPv4 connections as a standard dual-stacked system does. A socket bound and listening to an IPv6 endpoint can receive an IPv4 connection and will return the remote address as an IPv4-mapped address. Support for the IPV6_V6ONLY socket option does not currently exist.

Layer-4 protocols and sockets

We next describe how the transport protocols, sockets, and applications tie together. In summary, each transport protocol implementation is a socket factory. An application that needs a new socket

For instance, to create a UDP socket, an application would use a code snippet such as the following:

```
Ptr<Udp> udpSocketFactory = GetNode()->GetObject<Udp>();  
Ptr<Socket> m_socket = socketFactory->CreateSocket();  
m_socket->Bind(m_local_address);  
...
```

The above will query the node to get a pointer to its UDP socket factory, will create one such socket, and will use the socket with an API similar to the C-based sockets API, such as `Connect()` and `Send()`. The address passed to the `Bind()`, `Connect()`, or `Send()` functions may be a `Ipv4Address`, `Ipv6Address`, or `Address`. If a `Address` is passed in and contains anything other than a `Ipv4Address` or `Ipv6Address`, these functions will return an error. The `Bind()` and `Bind6()` functions bind to "0.0.0.0" and "::" respectively.

The socket can also be bound to a specific `NetDevice` though the `BindToNetDevice(Ptr<NetDevice> netdevice)` function. `BindToNetDevice(Ptr<NetDevice> netdevice)` will bind the socket to "0.0.0.0" and "::" (equivalent to calling `Bind()` and `Bind6()`), unless the socket has been already bound to a specific address. Summarizing, the correct sequence is:

```
Ptr<Udp> udpSocketFactory = GetNode()->GetObject<Udp>();  
Ptr<Socket> m_socket = socketFactory->CreateSocket();  
m_socket->BindToNetDevice(n_netDevice);  
...
```

or:

```
Ptr<Udp> udpSocketFactory = GetNode()->GetObject<Udp>();  
Ptr<Socket> m_socket = socketFactory->CreateSocket();  
m_socket->Bind(m_local_address);  
m_socket->BindToNetDevice(n_netDevice);  
...
```

The following raises an error:

```
Ptr<Udp> udpSocketFactory = GetNode()->GetObject<Udp>();
Ptr<Socket> m_socket = socketFactory->CreateSocket();
m_socket->BindToNetDevice(n_netDevice);
m_socket->Bind(m_local_address);
...
```

See the chapter on *ns-3* sockets for more information.

We have described so far a socket factory (e.g. class `Udp`) and a socket, which may be specialized (e.g., class `UdpSocket`). There are a few more key objects that relate to the specialized task of demultiplexing a packet to one or more receiving sockets. The key object in this task is class `Ipv4EndPointDemux`. This demultiplexer stores objects of class `Ipv4EndPoint`. This class holds the addressing/port tuple (local port, local address, destination port, destination address) associated with the socket, and a receive callback. This receive callback has a receive function registered by the socket. The `Lookup()` function to `Ipv4EndPointDemux` returns a list of `Ipv4EndPoint` objects (there may be a list since more than one socket may match the packet). The layer-4 protocol copies the packet to each `Ipv4EndPoint` and calls its `ForwardUp()` method, which then calls the `Receive()` function registered by the socket.

An issue that arises when working with the sockets API on real systems is the need to manage the reading from a socket, using some type of I/O (e.g., blocking, non-blocking, asynchronous, ...). *ns-3* implements an asynchronous model for socket I/O; the application sets a callback to be notified of received data ready to be read, and the callback is invoked by the transport protocol when data is available. This callback is specified as follows:

```
void Socket::SetRecvCallback(Callback<void>, Ptr<Socket>,
                             Ptr<Packet>,
                             const Address&> receivedData);
```

The data being received is conveyed in the `Packet` data buffer. An example usage is in class `PacketSink`:

```
m_socket->SetRecvCallback(MakeCallback(&PacketSink::HandleRead, this));
```

To summarize, internally, the UDP implementation is organized as follows:

- a `UdpImpl` class that implements the UDP socket factory functionality
- a `UdpL4Protocol` class that implements the protocol logic that is socket-independent
- a `UdpSocketImpl` class that implements socket-specific aspects of UDP
- a class called `Ipv4EndPoint` that stores the addressing tuple (local port, local address, destination port, destination address) associated with the socket, and a receive callback for the socket.

IP-capable node interfaces

Many of the implementation details, or internal objects themselves, of IP-capable `Node` objects are not exposed at the simulator public API. This allows for different implementations; for instance, replacing the native *ns-3* models with ported TCP/IP stack code.

The C++ public APIs of all of these objects is found in the `src/network` directory, including principally:

- `address.h`
- `socket.h`
- `node.h`
- `packet.h`

These are typically base class objects that implement the default values used in the implementation, implement access methods to get/set state variables, host attributes, and implement publicly-available methods exposed to clients such as `CreateSocket`.

Example path of a packet

These two figures show an example stack trace of how packets flow through the Internet Node objects.

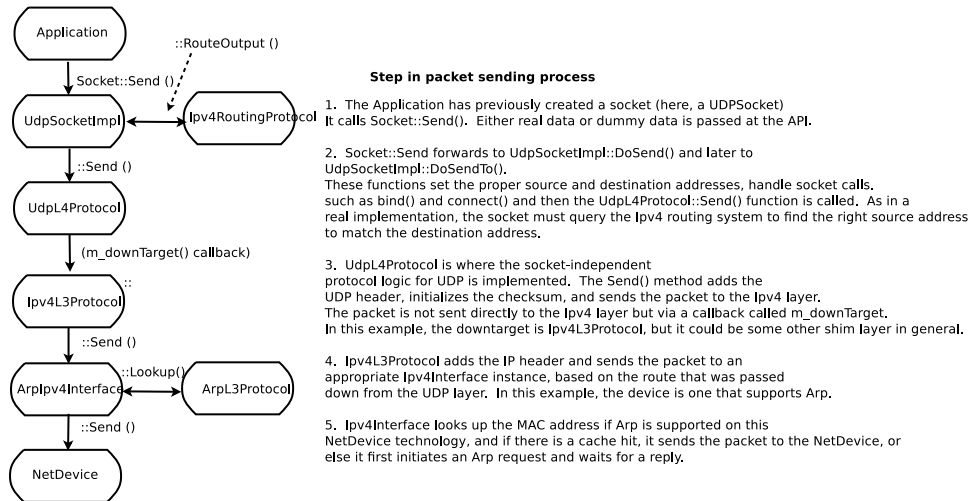


Fig. 16.1: Send path of a packet.

16.2 IPv4

This chapter describes the *ns-3* IPv4 address assignment and basic components tracking.

16.2.1 IPv4 addresses assignment

In order to use IPv4 on a network, the first thing to do is assigning IPv4 addresses.

Any IPv4-enabled *ns-3* node will have at least one `NetDevice`: the `ns3::LoopbackNetDevice`. The loopback device address is `127.0.0.1`. All the other `NetDevices` will have one (or more) IPv4 addresses.

Note that, as today, *ns-3* does not have a NAT module, and it does not follow the rules about filtering private addresses ([RFC 1918](#)): `10.0.0.0/8`, `172.16.0.0/12`, and `192.168.0.0/16`. These addresses are routed as any other address. This behaviour could change in the future.

IPv4 global addresses can be:

- manually assigned
- assigned through DHCP

ns-3 can use both methods, and it's quite important to understand the implications of both.

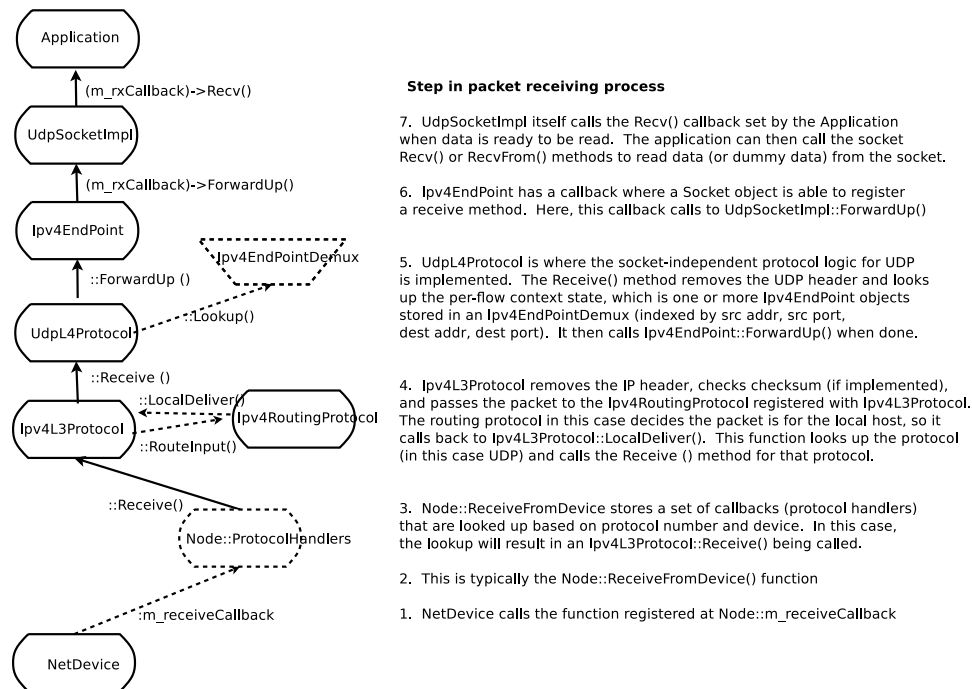


Fig. 16.2: Receive path of a packet.

Manually assigned IPv4 addresses

This is probably the easiest and most used method. As an example:

```
Ptr<Node> n0 = CreateObject<Node>();
Ptr<Node> n1 = CreateObject<Node>();
NodeContainer net(n0, n1);
CsmaHelper csma;
NetDeviceContainer ndc = csma.Install(net);

NS_LOG_INFO("Assign IPv4 Addresses.");
Ipv4AddressHelper ipv4;
ipv4.SetBase(Ipv4Address("192.168.1.0"), NetMask("/24"));
Ipv4InterfaceContainer ic = ipv4.Assign(ndc);
```

This method will add two global IPv4 addresses to the nodes.

Note that the addresses are assigned in sequence. As a consequence, the first Node / NetDevice will have "192.168.1.1", the second "192.168.1.2" and so on.

It is possible to repeat the above to assign more than one address to a node. However, due to the Ipv4AddressHelper singleton nature, one should first assign all the addresses of a network, then change the network base (SetBase), then do a new assignment.

Alternatively, it is possible to assign a specific address to a node:

```
Ptr<Node> n0 = CreateObject<Node>();
NodeContainer net(n0);
CsmaHelper csma;
NetDeviceContainer ndc = csma.Install(net);
```

(continues on next page)

(continued from previous page)

```

NS_LOG_INFO("Specifically Assign an IPv4 Address.");
Ipv4AddressHelper ipv4;
Ptr<NetDevice> device = ndc.Get(0);
Ptr<Node> node = device->GetNode();
Ptr<Ipv4> ipv4proto = node->GetObject<Ipv4>();
int32_t ifIndex = 0;
ifIndex = ipv4proto->GetInterfaceForDevice(device);
Ipv4InterfaceAddress ipv4Addr = Ipv4InterfaceAddress(Ipv4Address("192.168.1.42"),
↳ NetMask("/24"));
ipv4proto->AddAddress(ifIndex, ipv4Addr);

```

DHCP assigned IPv4 addresses

DHCP is available in the internet-apps module. In order to use DHCP you have to have a `DhcpServer` application in a node (the DHCP server node) and a `DhcpClient` application in each of the nodes. Note that it not necessary that all the nodes in a subnet use DHCP. Some nodes can have static addresses.

All the DHCP setup is performed though the `DhcpHelper` class. A complete example is in `src/internet-apps/examples/dhcp-example.cc`.

Further info about the DHCP functionalities can be found in the `internet-apps` model documentation.

16.2.2 Tracing in the IPv4 Stack

The internet stack provides a number of trace sources in its various protocol implementations. These trace sources can be hooked using your own custom trace code, or you can use our helper functions in some cases to arrange for tracing to be enabled.

Tracing in ARP

ARP provides two trace hooks, one in the cache, and one in the layer three protocol. The trace accessor in the cache is given the name “Drop.” When a packet is transmitted over an interface that requires ARP, it is first queued for transmission in the ARP cache until the required MAC address is resolved. There are a number of retries that may be done trying to get the address, and if the maximum retry count is exceeded the packet in question is dropped by ARP. The single trace hook in the ARP cache is called,

- If an outbound packet is placed in the ARP cache pending address resolution and no resolution can be made within the maximum retry count, the outbound packet is dropped and this trace is fired;

A second trace hook lives in the ARP L3 protocol (also named “Drop”) and may be called for a number of reasons.

- If an ARP reply is received for an entry that is not waiting for a reply, the ARP reply packet is dropped and this trace is fired;
- If an ARP reply is received for a non-existent entry, the ARP reply packet is dropped and this trace is fired;
- If an ARP cache entry is in the DEAD state (has timed out) and an ARP reply packet is received, the reply packet is dropped and this trace is fired.
- Each ARP cache entry has a queue of pending packets. If the size of the queue is exceeded, the outbound packet is dropped and this trace is fired.

Tracing in IPv4

The IPv4 layer three protocol provides three trace hooks. These are the “Tx” (`ns3::Ipv4L3Protocol::m_txTrace`), “Rx” (`ns3::Ipv4L3Protocol::m_rxTrace`) and “Drop” (`ns3::Ipv4L3Protocol::m_dropTrace`) trace sources.

The “Tx” trace is fired in a number of situations, all of which indicate that a given packet is about to be sent down to a given `ns3::Ipv4Interface`.

- In the case of a packet destined for the broadcast address, the `Ipv4InterfaceList` is iterated and for every interface that is up and can fragment the packet or has a large enough MTU to transmit the packet, the trace is hit. See `ns3::Ipv4L3Protocol::Send`.
- In the case of a packet that needs routing, the “Tx” trace may be fired just before a packet is sent to the interface appropriate to the default gateway. See `ns3::Ipv4L3Protocol::SendRealOut`.
- Also in the case of a packet that needs routing, the “Tx” trace may be fired just before a packet is sent to the outgoing interface appropriate to the discovered route. See `ns3::Ipv4L3Protocol::SendRealOut`.

The “Rx” trace is fired when a packet is passed from the device up to the `ns3::Ipv4L3Protocol::Receive` function.

- In the receive function, the `Ipv4InterfaceList` is iterated, and if the `Ipv4Interface` corresponding to the receiving device is found to be in the UP state, the trace is fired.

The “Drop” trace is fired in any case where the packet is dropped (in both the transmit and receive paths).

- In the `ns3::Ipv4Interface::Receive` function, the packet is dropped and the drop trace is hit if the interface corresponding to the receiving device is in the DOWN state.
- Also in the `ns3::Ipv4Interface::Receive` function, the packet is dropped and the drop trace is hit if the checksum is found to be bad.
- In `ns3::Ipv4L3Protocol::Send`, an outgoing packet bound for the broadcast address is dropped and the “Drop” trace is fired if the “don’t fragment” bit is set and fragmentation is available and required.
- Also in `ns3::Ipv4L3Protocol::Send`, an outgoing packet destined for the broadcast address is dropped and the “Drop” trace is hit if fragmentation is not available and is required ($MTU < \text{packet size}$).
- In the case of a broadcast address, an outgoing packet is cloned for each outgoing interface. If any of the interfaces is in the DOWN state, the “Drop” trace event fires with a reference to the copied packet.
- In the case of a packet requiring a route, an outgoing packet is dropped and the “Drop” trace event fires if no route to the remote host is found.
- In `ns3::Ipv4L3Protocol::SendRealOut`, an outgoing packet being routed is dropped and the “Drop” trace is fired if the “don’t fragment” bit is set and fragmentation is available and required.
- Also in `ns3::Ipv4L3Protocol::SendRealOut`, an outgoing packet being routed is dropped and the “Drop” trace is hit if fragmentation is not available and is required ($MTU < \text{packet size}$).
- An outgoing packet being routed is dropped and the “Drop” trace event fires if the required `Ipv4Interface` is in the DOWN state.
- If a packet is being forwarded, and the TTL is exceeded (see `ns3::Ipv4L3Protocol::DoForward`), the packet is dropped and the “Drop” trace event is fired.

16.2.3 Explicit Congestion Notification (ECN) bits

- In IPv4, ECN bits are the last 2 bits in TOS field and occupy 14th and 15th bits in the header.
- The IPv4 header class defines an `EcnType` enum with all four ECN codepoints (`ECN_NotECT`, `ECN_ECT1`, `ECN_ECT0`, `ECN_CE`) mentioned in RFC 3168, and also a setter and getter method to handle ECN values in the TOS field.

16.2.4 Ipv4QueueDiscItem

The traffic control sublayer in *ns-3* handles objects of class `QueueDiscItem` which are used to hold an `ns3::Packet` and an `ns3::Header`. This is done to facilitate the marking of packets for Explicit Congestion Notification. The `Mark ()` method is implemented in `Ipv4QueueDiscItem`. It returns true if marking the packet is successful, i.e., it successfully sets the CE bit in the IPv4 header. The `Mark ()` method will return false, however, if the IPv4 header indicates the `ECN_NotECT` codepoint.

16.2.5 RFC 6621 duplicate packet detection

To support mesh network protocols over broadcast-capable networks (e.g. Wi-Fi), it is useful to have support for duplicate packet detection and filtering, since nodes in a network may receive multiple copies of flooded multicast packets arriving on different paths. The `Ipv4L3Protocol` model in *ns-3* has a model for hash-based duplicate packet detection (DPD) based on Section 6.2.2 of ([RFC 6621](#)). The model, disabled by default, must be enabled by setting `EnableRFC6621` to true. A second attribute, `DuplicateExpire`, sets the expiration delay for erasing the cache entry of a packet in the duplicate cache; the delay value defaults to 1ms.

16.2.6 NeighborCache

`NeighborCacheHelper` provides a way to generate ARP cache automatically. It generates needed ARP cache before simulation start to avoid the delay and message overhead of address resolution in simulations that are focused on other performance aspects. The state of entries which are generated by `NeighborCacheHelper` is `STATIC_AUTOGENERATED`, which is similar to `PERMANENT`, but they are not manually added or removed by user, they will be managed by `NeighborCacheHelper` when user need pre-generate cache. When user is generating neighbor caches globally, neighbor caches will update dynamically when IPv4 addresses are removed or added; when user is generating neighbor caches partially, `NeighborCacheHelper` will take care of address removal, for adding address user may rerun a reduced-scope `PopulateNeighbor()` again to pick up the new IP address or manually add an entry to keep the neighbor cache up-to-date, the reason is that: when `PopulateNeighborCache()` has previously been run with a scope less than global, the code does not know whether it was previously run with a scope of `Channel`, `NetDeviceContainer`, or `Ip` interface container. The source code for `NeighborCache` is located in `src/internet/helper/neighbor-cache-helper`. A complete example is in `src/internet/examples/neighbor-cache-example.cc`.

Usage

The typical usages are:

```
* Populate neighbor ARP caches for all devices:
```

```
NeighborCacheHelper neighborCache;  
neighborCache.PopulateNeighborCache();
```

- Populate neighbor ARP caches for a given channel:

```
NeighborCacheHelper neighborCache;
neighborCache.PopulateNeighborCache(channel);    // channel is the Ptr<Channel> want to
↳generate ARP caches
```

- Populate neighbor ARP caches for devices in a given NetDeviceContainer:

```
NeighborCacheHelper neighborCache;
neighborCache.PopulateNeighborCache(netDevices); // netDevices is the
↳NetDeviceContainer want to generate ARP caches
```

- Populate neighbor ARP caches for a given Ipv4InterfaceContainer:

```
NeighborCacheHelper neighborCache;
neighborCache.PopulateNeighborCache(interfaces); // interfaces is the
↳Ipv4InterfaceContainer want to generate ARP caches
```

Neighbor cache entries are not removed by calls to `ArpCache::Flush()`, which may be called during the process of bringing IPv4 interfaces up or down. Instead, to remove them, users must call `ArpCache::RemoveAutoGenerated()`.

16.3 IPv6

This chapter describes the *ns-3* IPv6 model capabilities and limitations along with its usage and examples.

16.3.1 IPv6 model description

The IPv6 model is loosely patterned after the Linux implementation; the implementation is not complete as some features of IPv6 are not of much interest to simulation studies, and some features of IPv6 are simply not modeled yet in *ns-3*.

The base class `Ipv6` defines a generic API, while the class `Ipv6L3Protocol` is the actual class implementing the protocol. The actual classes used by the IPv6 stack are located mainly in the directory `src/internet`.

The implementation of IPv6 is contained in the following files:

```
src/internet/model/icmpv6-header.{cc,h}
src/internet/model/icmpv6-l4-protocol.{cc,h}
src/internet/model/ipv6.{cc,h}
src/internet/model/ipv6-address-generator.{cc,h}
src/internet/model/ipv6-autoconfigured-prefix.{cc,h}
src/internet/model/ipv6-end-point.{cc,h}
src/internet/model/ipv6-end-point-demux.{cc,h}
src/internet/model/ipv6-extension.{cc,h}
src/internet/model/ipv6-extension-demux.{cc,h}
src/internet/model/ipv6-extension-header.{cc,h}
src/internet/model/ipv6-header.{cc,h}
src/internet/model/ipv6-interface.{cc,h}
src/internet/model/ipv6-interface-address.{cc,h}
src/internet/model/ipv6-l3-protocol.{cc,h}
src/internet/model/ipv6-list-routing.{cc,h}
src/internet/model/ipv6-option.{cc,h}
src/internet/model/ipv6-option-demux.{cc,h}
src/internet/model/ipv6-option-header.{cc,h}
```

(continues on next page)

(continued from previous page)

```
src/internet/model/ipv6-packet-info-tag.{cc,h}
src/internet/model/ipv6-pmtu-cache.{cc,h}
src/internet/model/ipv6-raw-socket-factory.{cc,h}
src/internet/model/ipv6-raw-socket-factory-impl.{cc,h}
src/internet/model/ipv6-raw-socket-impl.{cc,h}
src/internet/model/ipv6-route.{cc,h}
src/internet/model/ipv6-routing-protocol.{cc,h}
src/internet/model/ipv6-routing-table-entry.{cc,h}
src/internet/model/ipv6-static-routing.{cc,h}
src/internet/model/ndisc-cache.{cc,h}
src/network/utils/inet6-socket-address.{cc,h}
src/network/utils/ipv6-address.{cc,h}
```

Also some helpers are involved with IPv6:

```
src/internet/helper/internet-stack-helper.{cc,h}
src/internet/helper/ipv6-address-helper.{cc,h}
src/internet/helper/ipv6-interface-container.{cc,h}
src/internet/helper/ipv6-list-routing-helper.{cc,h}
src/internet/helper/ipv6-routing-helper.{cc,h}
src/internet/helper/ipv6-static-routing-helper.{cc,h}
```

The model files can be roughly divided into:

- protocol models (e.g., ipv6, ipv6-l3-protocol, icmpv6-l4-protocol, etc.)
- routing models (i.e., anything with ‘routing’ in its name)
- sockets and interfaces (e.g., ipv6-raw-socket, ipv6-interface, ipv6-end-point, etc.)
- address-related things
- headers, option headers, extension headers, etc.
- accessory classes (e.g., ndisc-cache)

16.3.2 Usage

The following description is based on using the typical helpers found in the example code.

IPv6 does not need to be activated in a node. it is automatically added to the available protocols once the Internet Stack is installed.

In order to *not* install IPv6 along with IPv4, it is possible to use `ns3::InternetStackHelper` method `SetIpv6StackInstall` (*bool enable*) before installing the InternetStack in the nodes.

Note that to have an IPv6-only network (i.e., to not install the IPv4 stack in a node) one should use `ns3::InternetStackHelper` method `SetIpv4StackInstall` (*bool enable*) before the stack installation.

As an example, in the following code node 0 will have both IPv4 and IPv6, node 1 only only IPv6 and node 2 only IPv4:

```
NodeContainer n;
n.Create(3);

InternetStackHelper internet;
```

(continues on next page)

(continued from previous page)

```

InternetStackHelper internetV4only;
InternetStackHelper internetV6only;

internetV4only.SetIpv6StackInstall(false);
internetV6only.SetIpv4StackInstall(false);

internet.Install(n.Get(0));
internetV6only.Install(n.Get(1));
internetV4only.Install(n.Get(2));

```

IPv6 addresses assignment

In order to use IPv6 on a network, the first thing to do is assigning IPv6 addresses.

Any IPv6-enabled *ns-3* node will have at least one *NetDevice*: the *ns3::LoopbackNetDevice*. The loopback device address is `::1`. All the other *NetDevices* will have one or more IPv6 addresses:

- One link-local address: `fe80::interface ID`, where *interface ID* is derived from the *NetDevice* MAC address.
- Zero or more global addresses, e.g., `2001:db8::1`.

Typically the first address on an interface will be the link-local one, with the global address(es) being the following ones.

IPv6 global addresses might be:

- manually assigned
- auto-generated

ns-3 can use both methods, and it's quite important to understand the implications of both.

Manually assigned IPv6 addresses

This is probably the easiest and most used method. As an example:

```

Ptr<Node> n0 = CreateObject<Node>();
Ptr<Node> n1 = CreateObject<Node>();
NodeContainer net(n0, n1);
CsmaHelper csma;
NetDeviceContainer ndc = csma.Install(net);

NS_LOG_INFO("Assign IPv6 Addresses.");
Ipv6AddressHelper ipv6;
ipv6.SetBase(Ipv6Address("2001:db8::"), Ipv6Prefix(64));
Ipv6InterfaceContainer ic = ipv6.Assign(ndc);

```

This method will add two global IPv6 addresses to the nodes. Note that, as usual for IPv6, all the nodes will also have a link-local address. Typically the first address on an interface will be the link-local one, with the global address(es) being the following ones.

Note that the global addresses will be derived from the MAC address. As a consequence, expect to have addresses similar to `2001:db8::200:ff:fe00:1`.

It is possible to repeat the above to assign more than one global address to a node. However, due to the `Ipv6AddressHelper` singleton nature, one should first assign all the addresses of a network, then change the network base (`SetBase`), then do a new assignment.

Alternatively, it is possible to assign a specific address to a node:

```
Ptr<Node> n0 = CreateObject<Node>();
NodeContainer net(n0);
CsmaHelper csma;
NetDeviceContainer ndc = csma.Install(net);

NS_LOG_INFO("Specifically Assign an IPv6 Address.");
Ipv6AddressHelper ipv6;
Ptr<NetDevice> device = ndc.Get(0);
Ptr<Node> node = device->GetNode();
Ptr<Ipv6> ipv6proto = node->GetObject<Ipv6>();
int32_t ifIndex = 0;
ifIndex = ipv6proto->GetInterfaceForDevice(device);
Ipv6InterfaceAddress ipv6Addr = Ipv6InterfaceAddress(Ipv6Address("2001:db8:f00d:cafe::42
↪"), Ipv6Prefix(64));
ipv6proto->AddAddress(ifIndex, ipv6Addr);
```

Auto-generated IPv6 addresses

This is accomplished by relying on the RADVD protocol, implemented by the class `Radvd`. A helper class is available, which can be used to ease the most common tasks, e.g., setting up a prefix on an interface, if it is announced periodically, and if the router is the default router for that interface.

A fine grain configuration is possible through the `RadvdInterface` class, which allows to setup every parameter of the announced router advertisement on a given interface.

It is worth mentioning that the configurations must be set up before installing the application in the node.

Upon using this method, the nodes will acquire dynamically (i.e., during the simulation) one (or more) global address(es) according to the RADVD configuration. These addresses will be based on the RADVD announced prefix and the node's EUI-64.

Examples of RADVD use are shown in `examples/ipv6/radvd.cc` and `examples/ipv6/radvd-two-prefix.cc`.

Note that the router (i.e., the node with `Radvd`) will have to have a global address, while the nodes using the auto-generated addresses (SLAAC) will have to have a link-local address. This is accomplished using `Ipv6AddressHelper::AssignWithoutAddress`, e.g.:

```
Ipv6AddressHelper ipv6;
NetDeviceContainer tmp;
tmp.Add(d1.Get(0)); /* n0 */
Ipv6InterfaceContainer iic1 = ipv6.AssignWithoutAddress(tmp); /* n0 interface */
```


Random-generated IPv6 addresses

While IPv6 real nodes will use randomly generated addresses to protect privacy, *ns-3* does NOT have this capability. This feature haven't been so far considered as interesting for simulation.

Networks with and without the onlink property

IPv6 adds the concept of “on-link” for addresses and prefixes. Simplifying the concept, a network with the on-link property behaves roughly as IPv4: a node will assume that any address belonging to the on-link prefix is reachable on the link, so it uses Neighbor Discovery Protocol (NDP) to find the MAC address corresponding to the IPv6 address. If the prefix is not marked as “on-link”, then any packet is sent to the default router.

Radvd can announce prefixes that have the on-link flag not set. Moreover, it is possible to add an address to a node without setting the on-link property for the prefix used in the address. The function to use is `Ipv6AddressHelper::AssignWithoutOnLink`.

Duplicate Address Detection (DAD)

Nodes will perform DAD (it can be disabled using an `Icmpv6L4Protocol` attribute). Upon receiving a DAD, however, nodes will not react to it. As is: DAD reaction is incomplete so far. The main reason relies on the missing random-generated address capability. Moreover, since *ns-3* nodes will usually be well-behaving, there shouldn't be any Duplicate Address. This might be changed in the future, so as to avoid issues with real-world integrated simulations.

Explicit Congestion Notification (ECN) bits in IPv6

- In IPv6, ECN bits are the last 2 bits of the Traffic class and occupy 10th and 11th bit in the header.
- The IPv6 header class defines an `EcnType` enum with all four ECN codepoints (`ECN_NotECT`, `ECN_ECT1`, `ECN_ECT0`, `ECN_CE`) mentioned in RFC 3168, and also a setter and getter method to handle ECN values in the Traffic Class field.

16.3.3 Ipv6QueueDiscItem

The traffic control sublayer in *ns-3* handles objects of class `QueueDiscItem` which are used to hold an `ns3::Packet` and an `ns3::Header`. This is done to facilitate the marking of packets for Explicit Congestion Notification. The `Mark()` method is implemented in `Ipv6QueueDiscItem`. It returns true if marking the packet is successful, i.e., it successfully sets the CE bit in the IPv6 header. The `Mark()` method will return false, however, if the IPv6 header indicates the `ECN_NotECT` codepoint.

Host and Router behaviour in IPv6 and ns-3

In IPv6 there is a clear distinction between *routers* and *hosts*. As one might expect, routers can forward packets from an interface to another interface, while hosts drop packets not directed to them.

Unfortunately, forwarding is not the only thing affected by this distinction, and forwarding itself might be fine-tuned, e.g., to forward packets incoming from an interface and drop packets from another interface.

In *ns-3* a node is configured to be an *host* by default. There are two main ways to change this behaviour:

- Using `ns3::Ipv6InterfaceContainer::SetForwarding(uint32_t i, bool router)` where *i* is the interface index in the container.

- Changing the `ns3::Ipv6` attribute `IpForward`.

Either one can be used during the simulation.

A fine-grained setup can be accomplished by using `ns3::Ipv6Interface` *SetForwarding* (*bool forward*); which allows to change the behaviour on a per-interface-basis.

Note that the node-wide configuration only serves as a convenient method to enable/disable the `ns3::Ipv6Interface` specific setting. An `Ipv6Interface` added to a node with forwarding enabled will be set to be forwarding as well. This is really important when a node has interfaces added during the simulation.

According to the `ns3::Ipv6Interface` forwarding state, the following happens:

- Forwarding OFF
 - The node will NOT reply to Router Solicitation
 - The node will react to Router Advertisement
 - The node will periodically send Router Solicitation
 - Routing protocols MUST DROP packets not directed to the node
- Forwarding ON
 - The node will reply to Router Solicitation
 - The node will NOT react to Router Advertisement
 - The node will NOT send Router Solicitation
 - Routing protocols MUST forward packets

The behaviour is matching `ip-sysctl.txt` (<http://www.kernel.org/doc/Documentation/networking/ip-sysctl.txt>) with the difference that it's not possible to override the behaviour using esoteric settings (e.g., forwarding but accept router advertisements, `accept_ra=2`, or forwarding but send router solicitations `forwarding=2`).

Consider carefully the implications of packet forwarding. As an example, a node will NOT send ICMPv6 `PACKET_TOO_BIG` messages from an interface with forwarding off. This is completely normal, as the Routing protocol will drop the packet before attempting to forward it.

Helpers

Typically the helpers used in IPv6 setup are:

- `ns3::InternetStackHelper`
- `ns3::Ipv6AddressHelper`
- `ns3::Ipv6InterfaceContainer`

The use is almost identical to the corresponding IPv4 case, e.g.:

```
NodeContainer n;  
n.Create(4);  
  
NS_LOG_INFO("Create IPv6 Internet Stack");  
InternetStackHelper internetv6;  
internetv6.Install(n);  
  
NS_LOG_INFO("Create channels.");  
CsmaHelper csma;  
NetDeviceContainer d = csma.Install(n);
```

(continues on next page)

(continued from previous page)

```
NS_LOG_INFO("Create networks and assign IPv6 Addresses.");
Ipv6AddressHelper ipv6;
ipv6.SetBase(Ipv6Address("2001:db8::"), Ipv6Prefix(64));
Ipv6InterfaceContainer iic = ipv6.Assign(d);
```

Additionally, a common task is to enable forwarding on one of the nodes and to setup a default route toward it in the other nodes, e.g.:

```
iic.SetForwarding(0, true);
iic.SetDefaultRouteInAllNodes(0);
```

This will enable forwarding on the node 0 and will setup a default route in `ns3::Ipv6StaticRouting` on all the other nodes. Note that this requires that `Ipv6StaticRouting` is present in the nodes.

The IPv6 routing helpers enable the user to perform specific tasks on the particular routing algorithm and to print the routing tables.

Attributes

Many classes in the *ns-3* IPv6 implementation contain attributes. The most useful ones are:

- `ns3::Ipv6`
- *IpForward*, boolean, default false. Globally enable or disable IP forwarding for all current and future IPv6 devices.
- *MtuDiscover*, boolean, default true. If disabled, every interface will have its MTU set to 1280 bytes.
- `ns3::Ipv6L3Protocol`
- *DefaultTtl*, `uint8_t`, default 64. The TTL value set by default on all outgoing packets generated on this node.
- *SendIcmpv6Redirect*, boolean, default true. Send the ICMPv6 Redirect when appropriate.
- `ns3::Icmpv6L4Protocol`
- *DAD*, boolean, default true. Always do DAD (Duplicate Address Detection) check.
- `ns3::NdiscCache`
- *UnresolvedQueueSize*, `uint32_t`, default 3. Size of the queue for packets pending an NA reply.

Output

The IPv6 stack provides some useful trace sources:

- `ns3::Ipv6L3Protocol`
- *Tx*, Send IPv6 packet to outgoing interface.
- *Rx*, Receive IPv6 packet from incoming interface.
- *Drop*, Drop IPv6 packet.
- `ns3::Ipv6Extension`
- *Drop*, Drop IPv6 packet.

The latest trace source is generated when a packet contains an unknown option blocking its processing.

Mind that `ns3::NdiscCache` could drop packets as well, and they are not logged in a trace source (yet). This might generate some confusion in the sent/received packets counters.

Advanced Usage

IPv6 maximum transmission unit (MTU) and fragmentation

ns-3 NetDevices define the MTU according to the L2 simulated Device. IPv6 requires that the minimum MTU is 1280 bytes, so all NetDevices are required to support at least this MTU. This is the link-MTU.

In order to support different MTUs in a source-destination path, *ns-3* IPv6 model can perform fragmentation. This can be either triggered by receiving a packet bigger than the link-MTU from the L4 protocols (UDP, TCP, etc.), or by receiving an ICMPv6 `PACKET_TOO_BIG` message. The model mimics RFC 1981, with the following notable exceptions:

- L4 protocols are not informed of the Path MTU change
- TCP can not change its Segment Size according to the Path-MTU.

Both limitations are going to be removed in due time.

The Path-MTU cache is currently based on the source-destination IPv6 addresses. Further classifications (e.g., flow label) are possible but not yet implemented.

The Path-MTU default validity time is 10 minutes. After the cache entry expiration, the Path-MTU information is removed and the next packet will (eventually) trigger a new ICMPv6 `PACKET_TOO_BIG` message. Note that 1) this is consistent with the RFC specification and 2) L4 protocols are responsible for retransmitting the packets.

Examples

The examples for IPv6 are in the directory `examples/ipv6`. These examples focus on the most interesting IPv6 peculiarities, such as fragmentation, redirect and so on.

Moreover, most TCP and UDP examples located in `examples/udp`, `examples/tcp`, etc. have a command-line option to use IPv6 instead of IPv4.

Troubleshooting

There are just a few pitfalls to avoid while using *ns-3* IPv6.

Routing loops

Since the only (so far) routing scheme available for IPv6 is `ns3::Ipv6StaticRouting`, default router have to be setup manually. When there are two or more routers in a network (e.g., node A and node B), avoid using the helper function `SetDefaultRouteInAllNodes` for more than one router.

The consequence would be to install a default route to B in A and a default route pointing to A in B, generating a loop.

Global address leakage

Remember that addresses in IPv6 are *global* by definition. When using IPv6 with an emulation *ns-3* capability, avoid at all costs address leakage toward the global Internet. It is advisable to setup an external firewall to prevent leakage.

2001:DB8::/32 addresses

IPv6 standard (RFC 3849) defines the 2001:DB8::/32 address class for the *documentation*. This manual uses this convention. The addresses in this class are, however, only usable in a document, and routers should discard them.

16.3.4 Validation

The IPv6 protocols has not yet been extensively validated against real implementations. The actual tests involve mainly performing checks of the .pcap trace files with Wireshark, and the results are positive.

16.3.5 NeighborCache

NeighborCacheHelper provides a way to generate NDISC cache automatically. It generates needed NDISC cache before simulation start to avoid the delay and message overhead of neighbor discovery in simulations that are focused on other performance aspects. The state of entries generate by NeighborCacheHelper is `STATIC_AUTOGENERATED`, which is similar to `PERMANENT`, but they are not manually added or removed by user, they will be managed by NeighborCacheHelper when user need pre-generate cache. When user is generating neighbor caches globally, neighbor caches will update dynamically when IPv6 addresses are removed or added; when user is generating neighbor caches partially, NeighborCacheHelper will take care of address removal, for adding address user may rerun a reduced-scope `PopulateNeighbor()` again to pick up the new IP address or manually add an entry to keep the neighbor cache up-to-date, the reason is that: when `PopulateNeighborCache()` has previously been run with a scope less than global, the code does not know whether it was previously run with a scope of Channel, NetDeviceContainer, or Ip interface container. The source code for NeighborCache is located in `src/internet/helper/neighbor-cache-helper`. A complete example is in `src/internet/examples/neighbor-cache-example.cc`.

Usage

The usages for generating NDISC cache is almost the same as generating ARP cache, see `src/internet/doc/ipv4.rst`

- Populate neighbor ARP caches for a given `Ipv6InterfaceContainer`:

```
NeighborCacheHelper neighborCache;
neighborCache.PopulateNeighborCache(interfaces);    // interfaces is the
↪ Ipv6InterfaceContainer want to generate ARP caches
```

Neighbor cache entries are not removed by calls to `NdiscCache::Flush()`, which may be called during the process of bringing IPv6 interfaces up or down. Instead, to remove them, users must call `NdiscCache::RemoveAutoGenerated()`.

16.4 Routing overview

ns-3 is intended to support traditional routing approaches and protocols, support ports of open source routing implementations, and facilitate research into unorthodox routing techniques. The overall routing architecture is described below in *Routing architecture*. Users who wish to just read about how to configure global routing for wired topologies can read *Global Centralized Routing*. Unicast routing protocols are described in *Unicast routing*. Multicast routing is documented in *Multicast routing*.

16.4.1 Routing architecture

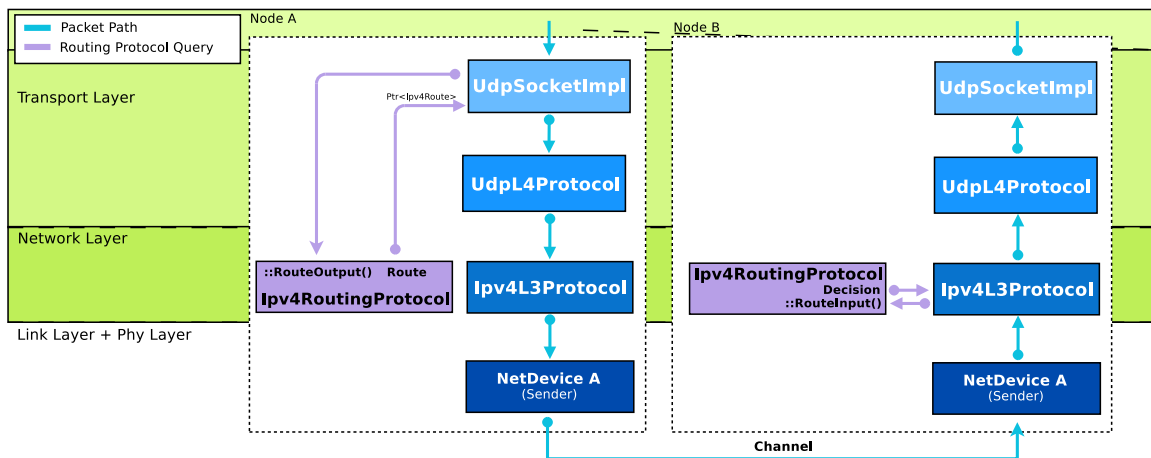


Fig. 16.3: Overview of routing

Overview of routing shows the overall routing architecture for Ipv4. The key objects are `Ipv4L3Protocol`, `Ipv4RoutingProtocol(s)` (a class to which all routing/forwarding has been delegated from `Ipv4L3Protocol`), and `Ipv4Route(s)`.

`Ipv4L3Protocol` must have at least one `Ipv4RoutingProtocol` added to it at simulation setup time. This is done explicitly by calling `Ipv4::SetRoutingProtocol()`.

The abstract base class `Ipv4RoutingProtocol()` declares a minimal interface, consisting of two methods: `RouteOutput()` and `RouteInput()`. For packets traveling outbound from a host, the transport protocol will query Ipv4 for the `Ipv4RoutingProtocol` object interface, and will request a route via `Ipv4RoutingProtocol::RouteOutput()`. A `Ptr` to `Ipv4Route` object is returned. This is analogous to a `dst_cache` entry in Linux. The `Ipv4Route` is carried down to the `Ipv4L3Protocol` to avoid a second lookup there. However, some cases (e.g. Ipv4 raw sockets) will require a call to `RouteOutput()` directly from `Ipv4L3Protocol`.

For packets received inbound for forwarding or delivery, the following steps occur. `Ipv4L3Protocol::Receive()` calls `Ipv4RoutingProtocol::RouteInput()`. This passes the packet ownership to the `Ipv4RoutingProtocol` object. There are four callbacks associated with this call:

- `LocalDeliver`
- `UnicastForward`
- `MulticastForward`
- `Error`

The `Ipv4RoutingProtocol` must eventually call one of these callbacks for each packet that it takes responsibility for. This is basically how the input routing process works in Linux.

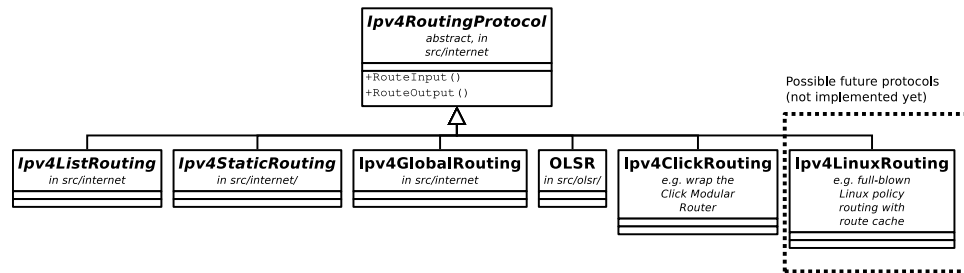


Fig. 16.4: Ipv4Routing specialization.

This overall architecture is designed to support different routing approaches, including (in the future) a Linux-like policy-based routing implementation, proactive and on-demand routing protocols, and simple routing protocols for when the simulation user does not really care about routing.

Ipv4Routing specialization. illustrates how multiple routing protocols derive from this base class. A class `Ipv4ListRouting` (implementation class `Ipv4ListRoutingImpl`) provides the existing list routing approach in *ns-3*. Its API is the same as base class `Ipv4Routing` except for the ability to add multiple prioritized routing protocols (`Ipv4ListRouting::AddRoutingProtocol()`, `Ipv4ListRouting::GetRoutingProtocol()`).

The details of these routing protocols are described below in *Unicast routing*. For now, we will first start with a basic unicast routing capability that is intended to globally build routing tables at simulation time $t=0$ for simulation users who do not care about dynamic routing.

16.4.2 Unicast routing

The following unicast routing protocols are defined for IPv4 and IPv6:

- classes `Ipv4ListRouting` and `Ipv6ListRouting` (used to store a prioritized list of routing protocols)
- classes `Ipv4StaticRouting` and `Ipv6StaticRouting` (covering both unicast and multicast)
- class `Ipv[4,6]GlobalRouting` (used to store routes computed by the global route manager, if that is used)
- class `Ipv4NixVectorRouting` (a more efficient version of global routing that stores source routes in a packet header field)
- class `Rip` - the IPv4 RIPv2 protocol ([RFC 2453](#))
- class `RipNg` - the IPv6 RIPng protocol ([RFC 2080](#))
- IPv4 Optimized Link State Routing (OLSR) (a MANET protocol defined in [RFC 3626](#))
- IPv4 Ad Hoc On Demand Distance Vector (AODV) (a MANET protocol defined in [RFC 3561](#))
- IPv4 Destination Sequenced Distance Vector (DSDV) (a MANET protocol)
- IPv4 Dynamic Source Routing (DSR) (a MANET protocol)

In the future, this architecture should also allow someone to implement a Linux-like implementation with routing cache, or a Click modular router, but those are out of scope for now.

Ipv[4,6]ListRouting

This section describes the current default *ns-3* Ipv[4,6]RoutingProtocol. Typically, multiple routing protocols are supported in user space and coordinate to write a single forwarding table in the kernel. Presently in *ns-3*, the implementation instead allows for multiple routing protocols to build/keep their own routing state, and the IP implementation will query each one of these routing protocols (in some order determined by the simulation author) until a route is found.

We chose this approach because it may better facilitate the integration of disparate routing approaches that may be difficult to coordinate the writing to a single table, approaches where more information than destination IP address (e.g., source routing) is used to determine the next hop, and on-demand routing approaches where packets must be cached.

Ipv[4,6]ListRouting::AddRoutingProtocol

Classes Ipv4ListRouting and Ipv6ListRouting provides a pure virtual function declaration for the method that allows one to add a routing protocol:

```
void AddRoutingProtocol(Ptr<Ipv4RoutingProtocol> routingProtocol,  
                        int16_t priority);  
  
void AddRoutingProtocol(Ptr<Ipv6RoutingProtocol> routingProtocol,  
                        int16_t priority);
```

These methods are implemented respectively by class Ipv4ListRoutingImpl and by class Ipv6ListRoutingImpl in the internet module.

The priority variable above governs the priority in which the routing protocols are inserted. Notice that it is a signed int. By default in *ns-3*, the helper classes will instantiate a Ipv[4,6]ListRoutingImpl object, and add to it an Ipv[4,6]StaticRoutingImpl object at priority zero. Internally, a list of Ipv[4,6]RoutingProtocols is stored, and the routing protocols are each consulted in decreasing order of priority to see whether a match is found. Therefore, if you want your Ipv4RoutingProtocol to have priority lower than the static routing, insert it with priority less than 0; e.g.:

```
Ptr<MyRoutingProtocol> myRoutingProto = CreateObject<MyRoutingProtocol>();  
listRoutingPtr->AddRoutingProtocol(myRoutingProto, -10);
```

Upon calls to RouteOutput() or RouteInput(), the list routing object will search the list of routing protocols, in priority order, until a route is found. Such routing protocol will invoke the appropriate callback and no further routing protocols will be searched.

Global Centralized Routing

Global centralized routing is an offline computation of routing tables that builds and installs routing tables without any protocol overhead on the simulated links. It is described in detail in the *Global Routing* chapter.

RIP and RIPng

The RIPv2 protocol for IPv4 is described in the [RFC 2453](#), and it consolidates a number of improvements over the base protocol defined in [RFC 1058](#).

This IPv6 routing protocol ([RFC 2080](#)) is the evolution of the well-known RIPv1 (see [RFC 1058](#) and [RFC 1723](#)) routing protocol for IPv4.

The protocols are very simple, and are normally suitable for flat, simple network topologies.

RIPv1, RIPv2, and RIPng have the very same goals and limitations. In particular, RIP considers any route with a metric equal or greater than 16 as unreachable. As a consequence, the maximum number of hops the network must be less than 15 (the number of routers is not set). Users are encouraged to read [RFC 2080](#) and [RFC 1058](#) to fully understand RIP behaviour and limitations.

Routing convergence

RIP uses a Distance-Vector algorithm, and routes are updated according to the Bellman-Ford algorithm (sometimes known as Ford-Fulkerson algorithm). The algorithm has a convergence time of $O(|V|*|E|)$ where $|V|$ and $|E|$ are the number of vertices (routers) and edges (links) respectively. It should be stressed that the convergence time is the number of steps in the algorithm, and each step is triggered by a message. Since Triggered Updates (i.e., when a route is changed) have a 1-5 seconds cooldown, the topology can require some time to be stabilized.

Users should be aware that, during routing tables construction, the routers might drop packets. Data traffic should be sent only after a time long enough to allow RIP to build the network topology. Usually 80 seconds should be enough to have a suboptimal (but working) routing setup. This includes the time needed to propagate the routes to the most distant router (16 hops) with Triggered Updates.

If the network topology is changed (e.g., a link is broken), the recovery time might be quite high, and it might be even higher than the initial setup time. Moreover, the network topology recovery is affected by the Split Horizing strategy.

The examples `examples/routing/ripng-simple-network.cc` and `examples/routing/rip-simple-network.cc` shows both the network setup and network recovery phases.

Split Horizing

Split Horizon is a strategy to prevent routing instability. Three options are possible:

- No Split Horizon
- Split Horizon
- Poison Reverse

In the first case, routes are advertised on all the router's interfaces. In the second case, routers will not advertise a route on the interface from which it was learned. Poison Reverse will advertise the route on the interface from which it was learned, but with a metric of 16 (infinity). For a full analysis of the three techniques, see [RFC 1058](#), section 2.2.

The examples are based on the network topology described in the RFC, but it does not show the effect described there.

The reason are the Triggered Updates, together with the fact that when a router invalidates a route, it will immediately propagate the route unreachability, thus preventing most of the issues described in the RFC.

However, with complex topologies, it is still possible to have route instability phenomena similar to the one described in the RFC after a link failure. As a consequence, all the considerations about Split Horizon remains valid.

Default routes

RIP protocol should be installed *only* on routers. As a consequence, nodes will not know what is the default router.

To overcome this limitation, users should either install the default route manually (e.g., by resorting to `Ipv4StaticRouting` or `Ipv6StaticRouting`), or by using RADVD (in case of IPv6). RADVD is available in *ns-3* in the Applications module, and it is strongly suggested.

Protocol parameters and options

The RIP *ns-3* implementations allow to change all the timers associated with route updates and routes lifetime.

Moreover, users can change the interface metrics on a per-node basis.

The type of Split Horizioning (to avoid routes back-propagation) can be selected on a per-node basis, with the choices being “no split horizon”, “split horizon” and “poison reverse”. See [RFC 2080](#) for further details, and [RFC 1058](#) for a complete discussion on the split horizioning strategies.

Moreover, it is possible to use a non-standard value for Link Down Value (i.e., the value after which a link is considered down). The default is value is 16.

Limitations

There is no support for the Next Hop option ([RFC 2080](#), Section 2.1.1). The Next Hop option is useful when RIP is not being run on all of the routers on a network. Support for this option may be considered in the future.

There is no support for CIDR prefix aggregation. As a result, both routing tables and route advertisements may be larger than necessary. Prefix aggregation may be added in the future.

Other routing protocols

Other routing protocols documentation can be found under the respective modules sections, e.g.:

- AODV
- Click
- DSDV
- DSR
- NixVectorRouting
- OLSR
- etc.

16.4.3 Multicast routing

The following function is used to add a static multicast route to a node:

```
void
Ipv4StaticRouting::AddMulticastRoute(Ipv4Address origin,
                                     Ipv4Address group,
                                     uint32_t inputInterface,
                                     std::vector<uint32_t> outputInterfaces);
```

A multicast route must specify an origin IP address, a multicast group and an input network interface index as conditions and provide a vector of output network interface indices over which packets matching the conditions are sent.

Typically there are two main types of multicast routes:

- Routes used during forwarding, and
- Routes used in the originator node.

In the first case all the conditions must be explicitly provided.

In the second case, the route is equivalent to a unicast route, and must be added through *Ipv4StaticRouting::AddHostRouteTo*.

Another command sets the default multicast route:

```
void
Ipv4StaticRouting::SetDefaultMulticastRoute(uint32_t outputInterface);
```

This is the multicast equivalent of the unicast version *SetDefaultRoute*. We tell the routing system what to do in the case where a specific route to a destination multicast group is not found. The system forwards packets out the specified interface in the hope that “something out there” knows better how to route the packet. This method is only used in initially sending packets off of a host. The default multicast route is not consulted during forwarding – exact routes must be specified using *AddMulticastRoute* for that case.

Since we’re basically sending packets to some entity we think may know better what to do, we don’t pay attention to “subtleties” like origin address, nor do we worry about forwarding out multiple interfaces. If the default multicast route is set, it is returned as the selected route from *LookupStatic* irrespective of origin or multicast group if another specific route is not found.

Finally, a number of additional functions are provided to fetch and remove multicast routes:

```
uint32_t GetNMulticastRoutes() const;

Ipv4MulticastRoute *GetMulticastRoute(uint32_t i) const;

Ipv4MulticastRoute *GetDefaultMulticastRoute() const;

bool RemoveMulticastRoute(Ipv4Address origin,
                          Ipv4Address group,
                          uint32_t inputInterface);

void RemoveMulticastRoute(uint32_t index);
```

16.5 Global Routing

This chapter defines the implementation of the GlobalRouting API present in the internet module. Global routing is not a dynamic routing protocol that sends packets over the simulated network. Instead, it is a simulation process that walks the simulation topology (typically just before the simulation starts), runs a shortest path algorithm on the discovered topology, and populates each node's routing tables accordingly. No actual protocol overhead (on the simulated links) is incurred with this approach.

The quagga (<https://www.nongnu.org/quagga/>) OSPF implementation was used as the basis for the routing computation logic. The Link State information exported by each global router in this API conforms to the OSPFv2 specs (**RFC 2328**) as used by the quagga implementation.

Presently, global centralized IPv[4,6] unicast routing over both point-to-point and shared (CSMA) links is supported.

The following is a high level overview of the classes that are used to implement the Global Routing API:

- **User Interface:** Ipv4GlobalRoutingHelper and GlobalRouteManager
- **Route Computation:** SPFVertex, GlobalRouteManagerLSDB and GlobalRouteManagerImpl
- **Link State Advertisements:** GlobalRoutingLSA, GlobalRoutingLinkRecord and GlobalRouter
- **RouteLookup and Forwarding:** GlobalRouting

16.5.1 Scope and Limitations

The API has a few constraints:

- **Wired only:** It is not intended for use in wireless networks. For wireless, OLSR dynamic routing is recommended, as described below.
- **Unicast only:** It does not support multicast.
- **Scalability:** Some users of this API in large topologies (e.g., 1000 nodes) have observed that the current implementation is not very scalable. The global centralized routing is planned to be modified in the future to reduce computations and improve runtime performance.

The following are the known limitations of the GlobalRouting implementation:

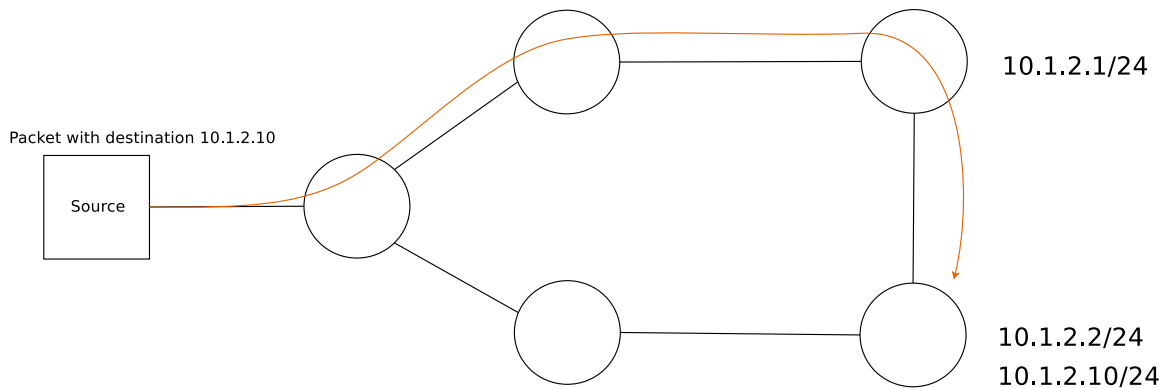
1. When calculating routes for networks using GlobalRouting, if multiple exit interfaces are present from the root node to a network, packets may travel an extra hop to reach their destination within that network. To clarify, the following are the generalized conditions:
 1. It is assumed that there is a network (e.g., 10.1.2.0/24) that has 2 (or more) routers (2 egress points, or more).
 2. It is assumed that a node has 2 (or more) equal-cost paths toward the aforementioned network.
 3. It is assumed that multiple IP addresses with that network prefix are present on one or more interfaces.

This will result in packets reaching the internal interfaces of the “other” router(s) via one unnecessary hop, passing through the designated router, and discarding the (shorter) path that directly leads to the destination.

#. Although adding multiple IP addresses on the same interface and Equal Cost Multi Path are both supported individually, the combination of both is not fully supported and may lead to unexpected results and, in some specific cases, routing loops. This limitation is tracked in issue #1242 of the issue tracker.

The Following are the known limitations of the GlobalRouting implementation for Ipv6 ONLY:

1. The GlobalRouting implementation for Ipv6 does not support non-Onlink Ipv6Addresses. The API will assert if it encounters such an address.



2. The GlobalRouting implementation for Ipv6 does not support StrongEndSystem Models. Calls to GlobalRouting via the Ipv6GlobalRoutingHelper will automatically disable StrongEndSystem model for all nodes in the system. When using the GlobalRouteManager directly, make sure to disable StrongEndSystem model for all nodes in the system.

16.5.2 Usage

To use GlobalRouting as the routing protocol, an instance of Ipv[4,6]GlobalRoutingHelper needs to be set using InternetStackHelper::SetRoutingProtocol(). Example code of this is:

```
InternetStackHelper internet;
Ipv4GlobalRoutingHelper glbrouting;
internet.SetRoutingHelper(glbrouting);
```

Further usage is typically based on the following helper code.

Helpers

Users must include the ns3/ipv4-global-routing-helper.h or ns3/ipv6-global-routing-helper.h header file for IPv4 or IPv6, respectively. After IP addresses are configured, the following function call will cause all of the nodes that have an Ipv4 interface to receive forwarding tables entered automatically by the GlobalRouteManager:

```
Ipv4GlobalRoutingHelper::PopulateRoutingTables();
```

Likewise, if IPv6 is used:

```
Ipv6GlobalRoutingHelper::PopulateRoutingTables();
```

Note: A reminder that the wifi NetDevice will work, but does not take any wireless effects into account. For wireless, we recommend OLSR dynamic routing.

Note: For IPv6, calls to GlobalRouting via the Ipv6GlobalRoutingHelper will enable forwarding for all nodes in the simulation with GlobalRouting installed. If more control is desired, it is recommended to use the GlobalRouteManager directly.

It is possible to call this function again in the midst of a simulation using the following additional public function:

```
Ipv4GlobalRoutingHelper::RecomputeRoutingTables();  
Ipv6GlobalRoutingHelper::RecomputeRoutingTables();
```

which flushes the old tables, queries the nodes for new interface information, and rebuilds the routes.

For instance, this scheduling call will cause the tables to be rebuilt at time 5 seconds:

```
Simulator::Schedule(Seconds(5),  
                    &Ipv4GlobalRoutingHelper::RecomputeRoutingTables);
```

GlobalRouteManager

GlobalRouteManager class can also be used to run the Global Routing API.

Users must include ns3/global-route-manager.h header file. After the IPv4 topology has been built and addresses assigned, users call ns3::GlobalRouteManager::PopulateRoutingTables(), prior to the ns3::Simulator::Run() call.

It has the following functions:

- DeleteGlobalRoutes() - Deletes all the routes in the routing table for each node in the simulation.
- BuildGlobalRoutingDatabase() - Queries each of the globalRouter in the simulation and collects the Link State Advertisements exported by them into a Link State Database with key as the LinkstateID.
- InitializeRoutes() -For each node that is a global router (which is determined by the presence of an aggregated GlobalRouter interface), run the Dijkstra SPF calculation on the database rooted at that router, and populate the node forwarding tables.

Note: Calls to GlobalRouteManager will in turn call the Simulation Singleton Object GlobalRouteManagerImpl to run the actual routing logic.

Default Usage

By default, when using the ns-3 helper API and the default InternetStackHelper, global routing capability will be added to the node, and global routing will be inserted as a routing protocol with lower priority than the static routes (i.e., users can insert routes via Ipv[4,6]StaticRouting API and they will take precedence over routes found by global routing).

The public API is very minimal. User scripts include the following:

```
#include "ns3/internet-module.h"
```

If the default InternetStackHelper is used, then an instance of GlobalRouting and globalRouter will be aggregated to each node.

Attributes

There are two attributes that govern the behavior of global routing.

1. Ipv4GlobalRouting::RandomEcmpRouting. If set to true, packets are randomly routed across equal-cost multi-path routes. If set to false (default), only one route is consistently used.
2. Ipv4GlobalRouting::RespondToInterfaceEvents. If set to true, dynamically recompute the global routes upon Interface notification events (up/down, or add/remove address). If set to false (default), routing may break unless the user manually calls RecomputeRoutingTables() after such events. The default is set to false to preserve legacy ns-3 program behavior.

The following code shows how to configure these attributes:

```
Config::SetDefault("ns3::Ipv4GlobalRouting::RespondToInterfaceEvents",
↳ BooleanValue(true));
Config::SetDefault("ns3::Ipv4GlobalRouting::RandomEcmpRouting", BooleanValue(true));
```

The helper also provides a `PrintRoute()` method that allows users to print the routing path calculated between two nodes; see the example program `src/internet/examples/ipv4-global-routing-print-route.cc` for a number of example statements showing how this method can be called.

Traces

Not Applicable

16.5.3 Implementation Details

This section is for those readers who care about how this is implemented. A singleton object (`GlobalRouteManager`) is responsible for populating the static routes on each node, using the public `Ipv4` API of that node. It queries each node in the topology for a “globalRouter” interface. If found, it uses the API of that interface to obtain a “link state advertisement (LSA)” for the router. Link State Advertisements are used in OSPF routing, and the formatting is followed.

It is important to note that all of these computations are done before packets are flowing in the network. In particular, there are no overhead or control packets being exchanged when using this implementation. Instead, this global route manager just walks the list of nodes to build the necessary information and configure each node’s routing table.

The `GlobalRouteManager` populates a link state database with LSAs gathered from the entire topology. Then, for each router in the topology, the `GlobalRouteManager` executes the OSPF shortest path first (SPF) computation on the database and populates the routing tables on each node.

The quagga (<https://www.nongnu.org/quagga/>) OSPF implementation was used as the basis for the routing computation logic. One benefit of following an existing OSPF SPF implementation is that OSPF already has defined link state advertisements for all common types of network links: - Point-to-point (serial links). - Point-to-multipoint (Frame Relay, ad hoc wireless). - Non-broadcast multiple access (ATM). - Broadcast (Ethernet).

Therefore, it is believed that enabling these other link types will be more straightforward now that the underlying OSPF SPF framework is in place. Currently, the implementation can handle `IPv4` point-to-point, numbered links, as well as shared broadcast (CSMA) links. Equal-cost multipath is also supported. Although wireless link types are supported by the implementation, it is noted that due to the nature of this implementation, any channel effects will not be considered, and the routing tables will assume that every node on the same shared channel is reachable from every other node (i.e., it will be treated like a broadcast CSMA link).

The `GlobalRouteManager` first walks the list of nodes and aggregates a `GlobalRouter` interface to each one as follows:

```
typedef std::vector<Ptr<Node>>::iterator Iterator;
for (Iterator i = NodeList::Begin(); i != NodeList::End(); i++)
{
    Ptr<Node> node = *i;
    Ptr<GlobalRouter> globalRouter = CreateObject<GlobalRouter>(node);
    node->AggregateObject(globalRouter);
}
```

This interface is later queried and used to generate a Link State Advertisement for each router, and this link state database is fed into the OSPF shortest path computation logic. The `Ipv4` API is finally used to populate the routes themselves.

16.5.4 Examples and Tests

Currently no examples are available for the Global Routing API.

Two TestSuites are available for the Global Routing API in `internet/test`:

- `ipv4-global-routing-test-suite`
- `global-route-manager-impl-test-suite`

The TestSuites are located in the `ns3/src/internet/test` directory. The Following is a brief description of the TestCases Contained within:

global-route-manager-impl-test-suite

- `LinkRoutesTestCase` : Checks Default Routes for stub nodes and routes for p2p links
- `LanRoutesTestCase` : Checks Network Routes for LAN Topologies. Checks that the Network LSA is handled by `GlobalRouteManagerImpl`
- `RandomEcmpTestCase` : Checks the `RandomEcmp` Attribute. i.e if equal cost multipath routes are possible build them and use them at random
- `GlobalRoutingProtocolTestCase` : Checks the normal operation of `GlobalRouting`, by looking at the output of the `RouteInput()` and `RouteOutput()` for several different paths. This mimics how normal calls to `GlobalRouting` are made.

ipv4-global-routing-test-suite

- `Link test`: Point-to-point link between two nodes, verifying default stub-link routes.
- `LAN test`: Two nodes on a broadcast network, verifying direct LAN subnet routes.
- `Two link test`: Three nodes on two point-to-point links, checking default and subnet routes.
- `Two LANs test`: Three nodes on two broadcast networks, testing inter-LAN routing.
- `Bridge test`: Two Links connected through a bridge, verifying routing across the bridged segment.
- `Two Bridge test`: links connected by two bridges in series, testing multi-hop bridging.

16.5.5 Validation

No formal validation (comparison of the routing tables generated by this implementation against the results of a real routing protocol) has been performed.

16.5.6 References

[1] The quagga routing suite was used as the basis for the routing computation logic.

[2] [RFC 2328](#) was used as the basis for the information exported by the global routers for a single area network configuration.

16.6 TCP models in ns-3

This chapter describes the TCP models available in *ns-3*.

16.6.1 Overview of support for TCP

ns-3 was written to support multiple TCP implementations. The implementations inherit from a few common header classes in the `src/network` directory, so that user code can swap out implementations with minimal changes to the scripts.

There are three important abstract base classes:

- class `TcpSocket`: This is defined in `src/internet/model/tcp-socket.{cc,h}`. This class exists for hosting `TcpSocket` attributes that can be reused across different implementations. For instance, the attribute `InitialCwnd` can be used for any of the implementations that derive from class `TcpSocket`.
- class `TcpSocketFactory`: This is used by the layer-4 protocol instance to create TCP sockets of the right type.
- class `TcpCongestionOps`: This supports different variants of congestion control— a key topic of simulation-based TCP research.

There are presently two active implementations of TCP available for *ns-3*.

- a natively implemented TCP for ns-3
- support for kernel implementations via [Direct Code Execution \(DCE\)](#)

Direct Code Execution is limited in its support for newer kernels; at present, only Linux kernel 4.4 is supported. However, the TCP implementations in kernel 4.4 can still be used for *ns-3* validation or for specialized simulation use cases.

It should also be mentioned that various ways of combining virtual machines with *ns-3* makes available also some additional TCP implementations, but those are out of scope for this chapter.

16.6.2 ns-3 TCP

In brief, the native *ns-3* TCP model supports a full bidirectional TCP with connection setup and close logic. Several congestion control algorithms are supported, with CUBIC the default, and NewReno, Westwood, Hybla, HighSpeed, Vegas, Scalable, Veno, Binary Increase Congestion Control (BIC), Yet Another HighSpeed TCP (YeAH), Illinois, H-TCP, Low Extra Delay Background Transport (LEDBAT), TCP Low Priority (TCP-LP), Data Center TCP (DCTCP) and Bottleneck Bandwidth and RTT (BBR) also supported. The model also supports Selective Acknowledgements (SACK), Forward Acknowledgement (FACK), Proportional Rate Reduction (PRR) and Explicit Congestion Notification (ECN). Multipath-TCP is not yet supported in the *ns-3* releases.

Model history

Until the ns-3.10 release, *ns-3* contained a port of the TCP model from [GTNetS](#), developed initially by George Riley and ported to *ns-3* by Raj Bhattacharjea. This implementation was substantially rewritten by Adriam Tam for ns-3.10. In 2015, the TCP module was redesigned in order to create a better environment for creating and carrying out automated tests. One of the main changes involves congestion control algorithms, and how they are implemented.

Before the ns-3.25 release, a congestion control was considered as a stand-alone TCP through an inheritance relation: each congestion control (e.g. `TcpNewReno`) was a subclass of `TcpSocketBase`, reimplementing some inherited methods. The architecture was redone to avoid this inheritance, by making each congestion control a separate class, and defining an interface to exchange important data between `TcpSocketBase` and the congestion modules. The Linux `tcp_congestion_ops` interface was used as the design reference.

Along with congestion control, Fast Retransmit and Fast Recovery algorithms have been modified; in previous releases, these algorithms were delegated to `TcpSocketBase` subclasses. Starting from ns-3.25, they have been merged inside `TcpSocketBase`. In future releases, they can be extracted as separate modules, following the congestion control design.

As of the ns-3.31 release, the default initial window was set to 10 segments (in previous releases, it was set to 1 segment). This aligns with current Linux default, and is discussed further in [RFC 6928](#).

In the ns-3.32 release, the default recovery algorithm was set to Proportional Rate Reduction (PRR) from the classic ack-clocked Fast Recovery algorithm.

In the ns-3.34 release, the default congestion control algorithm was set to CUBIC from NewReno.

CUBIC was extended to support Reno-friendliness (see RFC 9438 Section 4.3) in the ns-3.41 release. This feature is called ‘TCP friendliness’ in earlier versions of the CUBIC RFCs, and in the Linux and ns-3 implementations.

Acknowledgments

As mentioned above, *ns-3* TCP has had multiple authors and maintainers over the years. Several publications exist on aspects of *ns-3* TCP, and users of *ns-3* TCP are requested to cite one of the applicable papers when publishing new work.

A general reference on the current architecture is found in the following paper:

- Maurizio Casoni, Natale Patriciello, Next-generation TCP for ns-3 simulator, *Simulation Modelling Practice and Theory*, Volume 66, 2016, Pages 81-93. (<http://www.sciencedirect.com/science/article/pii/S1569190X15300939>)

For an academic peer-reviewed paper on the SACK implementation in ns-3, please refer to:

- Natale Patriciello. 2017. A SACK-based Conservative Loss Recovery Algorithm for ns-3 TCP: a Linux-inspired Proposal. In *Proceedings of the Workshop on ns-3 (WNS3 '17)*. ACM, New York, NY, USA, 1-8. (<https://dl.acm.org/citation.cfm?id=3067666>)

Usage

In many cases, usage of TCP is set at the application layer by telling the *ns-3* application which kind of socket factory to use.

Using the helper functions defined in `src/applications/helper` and `src/network/helper`, here is how one would create a TCP receiver:

```
// Create a packet sink on the star "hub" to receive these packets
uint16_t port = 50000;
Address sinkLocalAddress(InetSocketAddress(Ipv4Address::GetAny(), port));
PacketSinkHelper sinkHelper("ns3::TcpSocketFactory", sinkLocalAddress);
ApplicationContainer sinkApp = sinkHelper.Install(serverNode);
sinkApp.Start(Seconds(1));
sinkApp.Stop(Seconds(10));
```

Similarly, the below snippet configures OnOffApplication traffic source to use TCP:

```
// Create the OnOff applications to send TCP to the server
OnOffHelper clientHelper("ns3::TcpSocketFactory", Address());
```

The careful reader will note above that we have specified the `TypeId` of an abstract base class `TcpSocketFactory`. How does the script tell *ns-3* that it wants the native *ns-3* TCP vs. some other one? Well, when internet stacks are added to

the node, the default TCP implementation that is aggregated to the node is the *ns-3* TCP. So, by default, when using the *ns-3* helper API, the TCP that is aggregated to nodes with an Internet stack is the native *ns-3* TCP.

To configure behavior of TCP, a number of parameters are exported through the *ns-3* attribute system. These are documented in the [Doxygen](#) for class `TcpSocket`. For example, the maximum segment size is a settable attribute.

To set the default socket type before any internet stack-related objects are created, one may put the following statement at the top of the simulation program:

```
Config::SetDefault("ns3::TcpL4Protocol::SocketType", StringValue("ns3::TcpNewReno"));
```

For users who wish to have a pointer to the actual socket (so that socket operations like `Bind()`, setting socket options, etc. can be done on a per-socket basis), `Tcp` sockets can be created by using the `Socket::CreateSocket()` method. The `TypeId` passed to `CreateSocket()` must be of type `ns3::SocketFactory`, so configuring the underlying socket type must be done by twiddling the attribute associated with the underlying `TcpL4Protocol` object. The easiest way to get at this would be through the attribute configuration system. In the below example, the `Node` container “`n0n1`” is accessed to get the zeroth element, and a socket is created on this node:

```
// Create and bind the socket...
TypeId tid = TypeId::LookupByName("ns3::TcpNewReno");
Config::Set("/NodeList/*/ns3::TcpL4Protocol/SocketType", TypeIdValue(tid));
Ptr<Socket> localSocket =
    Socket::CreateSocket(n0n1.Get(0), TcpSocketFactory::GetTypeId());
```

Above, the “`*`” wild card for node number is passed to the attribute configuration system, so that all future sockets on all nodes are set to `NewReno`, not just on node ‘`n0n1.Get(0)`’. If one wants to limit it to just the specified node, one would have to do something like:

```
// Create and bind the socket...
TypeId tid = TypeId::LookupByName("ns3::TcpNewReno");
std::stringstream nodeId;
nodeId << n0n1.Get(0)->GetId();
std::string specificNode = "/NodeList/" + nodeId.str() + "/ns3::TcpL4Protocol/SocketType";
Config::Set(specificNode, TypeIdValue(tid));
Ptr<Socket> localSocket =
    Socket::CreateSocket(n0n1.Get(0), TcpSocketFactory::GetTypeId());
```

Once a TCP socket is created, one will want to follow conventional socket logic and either `connect()` and `send()` (for a TCP client) or `bind()`, `listen()`, and `accept()` (for a TCP server). Please note that applications usually create the sockets they use automatically, and so is not straightforward to connect directly to them using pointers. Please refer to the source code of your preferred application to discover how and when it creates the socket.

TCP Socket interaction and interface with Application layer

In the following there is an analysis on the public interface of the TCP socket, and how it can be used to interact with the socket itself. An analysis of the callback fired by the socket is also carried out. Please note that, for the sake of clarity, we will use the terminology “Sender” and “Receiver” to clearly divide the functionality of the socket. However, in TCP these two roles can be applied at the same time (i.e. a socket could be a sender and a receiver at the same time): our distinction does not lose generality, since the following definition can be applied to both sockets in case of full-duplex mode.

TCP state machine (for commodity use)

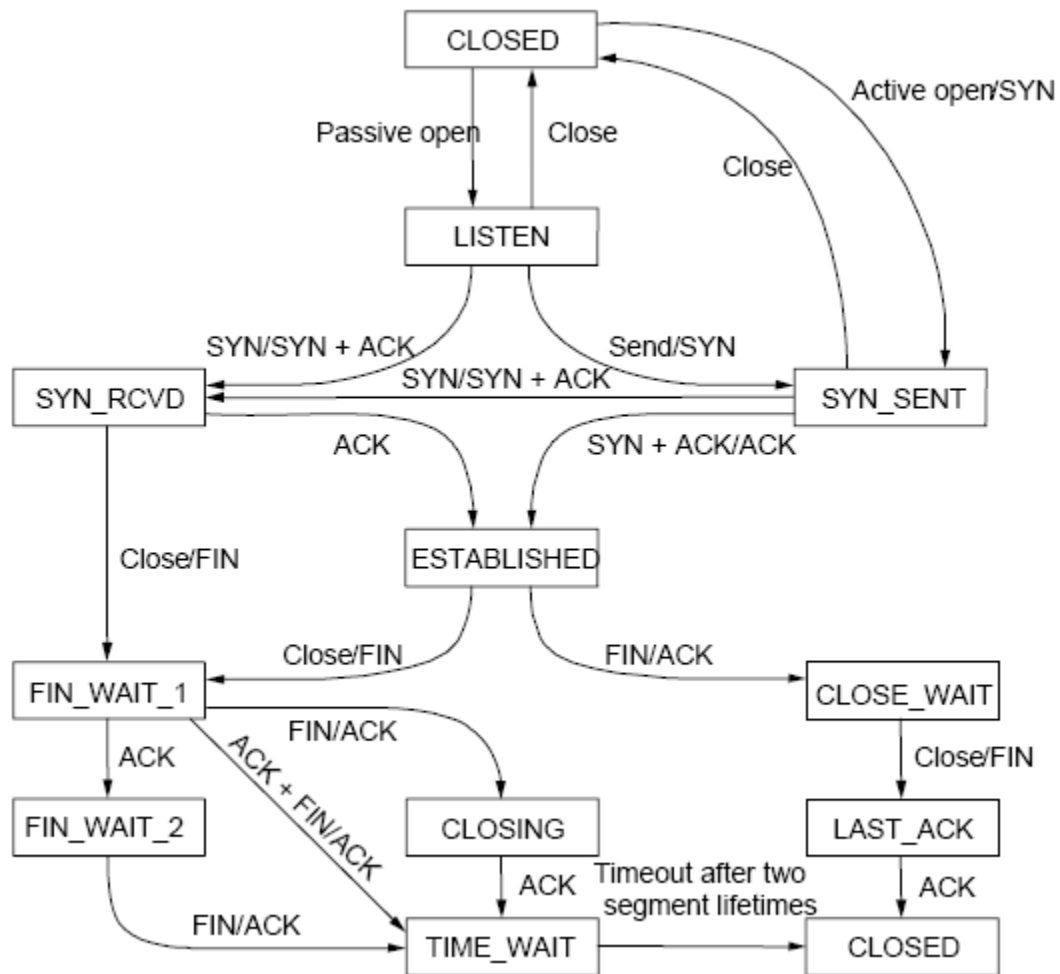


Fig. 16.5: TCP State machine

In ns-3 we are fully compliant with the state machine depicted in Figure *TCP State machine*.

Public interface for receivers (e.g. servers receiving data)

Bind()

Bind the socket to an address, or to a general endpoint. A general endpoint is an endpoint with an ephemeral port allocation (that is, a random port allocation) on the 0.0.0.0 IP address. For instance, in current applications, data senders usually binds automatically after a *Connect()* over a random port. Consequently, the connection will start from this random port towards the well-defined port of the receiver. The IP 0.0.0.0 is then translated by lower layers into the real IP of the device.

Bind6()

Same as *Bind()*, but for IPv6.

BindToNetDevice()

Bind the socket to the specified NetDevice, creating a general endpoint.

Listen()

Listen on the endpoint for an incoming connection. Please note that this function can be called only in the TCP CLOSED state, and transit in the LISTEN state. When an incoming request for connection is detected (i.e. the other peer invoked *Connect()*) the application will be signaled with the callback *NotifyConnectionRequest* (set in *SetAcceptCallback()* beforehand). If the connection is accepted (the default behavior, when the associated callback is a null one) the Socket will fork itself, i.e. a new socket is created to handle the incoming data/connection, in the state SYN_RCVD. Please note that this newly created socket is not connected anymore to the callbacks on the “father” socket (e.g. *DataSent*, *Recv*); the pointer of the newly created socket is provided in the Callback *NotifyNewConnectionCreated* (set beforehand in *SetAcceptCallback*), and should be used to connect new callbacks to interesting events (e.g. *Recv* callback). After receiving the ACK of the SYN-ACK, the socket will set the congestion control, move into ESTABLISHED state, and then notify the application with *NotifyNewConnectionCreated*.

ShutdownSend()

Signal a termination of send, or in other words prevents data from being added to the buffer. After this call, if buffer is already empty, the socket will send a FIN, otherwise FIN will go when buffer empties. Please note that this is useful only for modeling “Sink” applications. If you have data to transmit, please refer to the *Send()* / *Close()* combination of API.

GetRxAvailable()

Get the amount of data that could be returned by the Socket in one or multiple call to *Recv* or *RecvFrom*. Please use the Attribute system to configure the maximum available space on the receiver buffer (property “RcvBuf-Size”).

Recv()

Grab data from the TCP socket. Please remember that TCP is a stream socket, and it is allowed to concatenate multiple packets into bigger ones. If no data is present (i.e. *GetRxAvailable* returns 0) an empty packet is returned. Set the callback *RecvCallback* through *SetRecvCallback()* in order to have the application automatically notified when some data is ready to be read. It’s important to connect that callback to the newly created socket in case of forks.

RecvFrom()

Same as *Recv*, but with the source address as parameter.

Public interface for senders (e.g. clients uploading data)

Connect()

Set the remote endpoint, and try to connect to it. The local endpoint should be set before this call, or otherwise an ephemeral one will be created. The TCP then will be in the SYN_SENT state. If a SYN-ACK is received, the TCP will setup the congestion control, and then call the callback *ConnectionSucceeded*.

GetTxAvailable()

Return the amount of data that can be stored in the TCP Tx buffer. Set this property through the Attribute system ("SndBufSize").

Send()

Send the data into the TCP Tx buffer. From there, the TCP rules will decide if, and when, this data will be transmitted. Please note that, if the tx buffer has enough data to fill the congestion (or the receiver) window, dynamically varying the rate at which data is injected in the TCP buffer does not have any noticeable effect on the amount of data transmitted on the wire, that will continue to be decided by the TCP rules.

SendTo()

Same as *Send()*.

Close()

Terminate the local side of the connection, by sending a FIN (after all data in the tx buffer has been transmitted). This does not prevent the socket in receiving data, and employing retransmit mechanism if losses are detected. If the application calls *Close()* with unread data in its rx buffer, the socket will send a reset. If the socket is in the state SYN_SENT, CLOSING, LISTEN, FIN_WAIT_2, or LAST_ACK, after that call the application will be notified with *NotifyNormalClose()*. In other cases, the notification is delayed (see *NotifyNormalClose()*).

Public callbacks

These callbacks are called by the TCP socket to notify the application of interesting events. We will refer to these with the protected name used in socket.h, but we will provide the API function to set the pointers to these callback as well.

NotifyConnectionSucceeded: SetConnectCallback, 1st argument

Called in the SYN_SENT state, before moving to ESTABLISHED. In other words, we have sent the SYN, and we received the SYN-ACK: the socket prepares the sequence numbers, sends the ACK for the SYN-ACK, tries to send out more data (in another segment) and then invokes this callback. After this callback, it invokes the *NotifySend* callback.

NotifyConnectionFailed: SetConnectCallback, 2nd argument

Called after the SYN retransmission count goes to 0. SYN packet is lost multiple times, and the socket gives up.

NotifyNormalClose: SetCloseCallbacks, 1st argument

A normal close is invoked. A rare case is when we receive an RST segment (or a segment with bad flags) in normal states. All other cases are: - The application tries to *Connect()* over an already connected socket - Received an ACK for the FIN sent, with or without the FIN bit set (we are in LAST_ACK) - The socket reaches the maximum amount of retries in retransmitting the SYN (*) - We receive a timeout in the LAST_ACK state - Upon entering the TIME_WAIT state, before waiting the 2*Maximum Segment Lifetime seconds to finally deallocate the socket.

NotifyErrorClose: SetCloseCallbacks, 2nd argument

Invoked when we send an RST segment (for whatever reason) or we reached the maximum amount of data retries.

NotifyConnectionRequest: SetAcceptCallback, 1st argument

Invoked in the LISTEN state, when we receive a SYN. The return value indicates if the socket should accept the connection (return true) or should ignore it (return false).

NotifyNewConnectionCreated: SetAcceptCallback, 2nd argument

Invoked when from SYN_RCVD the socket passes to ESTABLISHED, and after setting up the congestion control, the sequence numbers, and processing the incoming ACK. If there is some space in the buffer, *NotifySend* is called shortly after this callback. The Socket pointer, passed with this callback, is the newly created socket, after a *Fork()*.

NotifyDataSent: SetDataSentCallback

The Socket notifies the application that some bytes have been transmitted on the IP level. These bytes could still be lost in the node (traffic control layer) or in the network.

NotifySend: SetSendCallback

Invoked if there is some space in the tx buffer when entering the ESTABLISHED state (e.g. after the ACK for SYN-ACK is received), after the connection succeeds (e.g. after the SYN-ACK is received) and after each new ACK (i.e. that advances SND.UNA).

NotifyDataRecv: SetRecvCallback

Called when in the receiver buffer there are in-order bytes, and when in FIN_WAIT_1 or FIN_WAIT_2 the socket receive a in-sequence FIN (that can carry data).

Congestion Control Algorithms

Here follows a list of supported TCP congestion control algorithms. For an academic paper on many of these congestion control algorithms, see <http://dl.acm.org/citation.cfm?id=2756518>.

NewReno

NewReno algorithm introduces partial ACKs inside the well-established Reno algorithm. This and other modifications are described in RFC 6582. We have two possible congestion window increment strategy: slow start and congestion avoidance. Taken from RFC 5681:

During slow start, a TCP increments cwnd by at most SMSS bytes for each ACK received that cumulatively acknowledges new data. Slow start ends when cwnd exceeds ssthresh (or, optionally, when it reaches it, as noted above) or when congestion is observed. While traditionally TCP implementations have increased cwnd by precisely SMSS bytes upon receipt of an ACK covering new data, we RECOMMEND that TCP implementations increase cwnd, per Equation (16.1), where N is the number of previously unacknowledged bytes acknowledged in the incoming ACK.

$$cwnd += \min(N, SMSS) \quad (16.1)$$

During congestion avoidance, cwnd is incremented by roughly 1 full-sized segment per round-trip time (RTT), and for each congestion event, the slow start threshold is halved.

CUBIC

CUBIC (class `TcpCubic`) is the default TCP congestion control in Linux, macOS (since 2014), and Microsoft Windows (since 2017). CUBIC has two main differences with respect to a more classic TCP congestion control such as NewReno. First, during the congestion avoidance phase, the window size grows according to a cubic function (concave, then convex) with the latter convex portion designed to allow for bandwidth probing. Second, a hybrid slow start (HyStart) algorithm uses observations of delay increases in the slow start phase of window growth to try to exit slow start before window growth causes queue overflow.

CUBIC is documented in [RFC 9438](#), and the ns-3 implementation is patterned partly on the Linux implementation and partly on the RFC, although the Linux 4.4 kernel implementation (through the Direct Code Execution environment) has been used to validate the behavior.

Linux Reno

TCP Linux Reno (class `TcpLinuxReno`) is designed to provide a Linux-like implementation of TCP NewReno. The implementation of class `TcpNewReno` in ns-3 follows RFC standards, and increases `cwnd` more conservatively than does Linux Reno. Linux Reno modifies slow start and congestion avoidance algorithms to increase `cwnd` based on the number of bytes being acknowledged by each arriving ACK, rather than by the number of ACKs that arrive. Another major difference in implementation is that Linux maintains the congestion window in units of segments, while the RFCs define the congestion window in units of bytes.

In slow start phase, on each incoming ACK at the TCP sender side `cwnd` is increased by the number of previously unacknowledged bytes ACKed by the incoming acknowledgment. In contrast, in ns-3 NewReno, `cwnd` is increased by one segment per acknowledgment. In standards terminology, this difference is referred to as Appropriate Byte Counting (RFC 3465); Linux follows Appropriate Byte Counting while ns-3 NewReno does not.

$$cwnd+ = segAked * segmentSize \quad (16.2)$$

$$cwnd+ = segmentSize \quad (16.3)$$

In congestion avoidance phase, the number of bytes that have been ACKed at the TCP sender side are stored in a 'bytes_acked' variable in the TCP control block. When 'bytes_acked' becomes greater than or equal to the value of the `cwnd`, 'bytes_acked' is reduced by the value of `cwnd`. Next, `cwnd` is incremented by a full-sized segment (SMSS). In contrast, in ns-3 NewReno, `cwnd` is increased by $(1/cwnd)$ with a rounding off due to type casting into int.

Listing 16.1: Linux Reno `cwnd` update

```
if (m_cWndCnt >= w)
{
    uint32_t delta = m_cWndCnt / w;

    m_cWndCnt -= delta * w;
    tcb->m_cWnd += delta * tcb->m_segmentSize;
    NS_LOG_DEBUG("Subtracting delta * w from m_cWndCnt " << delta * w);
}
```

Listing 16.2: New Reno `cwnd` update

```
if (segmentsAked > 0)
{
    double adder = static_cast<double>(tcb->m_segmentSize * tcb->m_segmentSize) / tcb->m_
    ↪ cWnd.Get();
    adder = std::max(1.0, adder);
    tcb->m_cWnd += static_cast<uint32_t>(adder);
    NS_LOG_INFO("In CongAvoid, updated to cwnd " << tcb->m_cWnd <<
        " ssthresh " << tcb->m_ssThresh);
}
```

So, there are two main difference between the TCP Linux Reno and TCP NewReno in ns-3: 1) In TCP Linux Reno, delayed acknowledgement configuration does not affect congestion window growth, while in TCP NewReno, delayed acknowledgments cause a slower congestion window growth. 2) In congestion avoidance phase, the arithmetic for counting the number of segments acked and deciding when to increment the `cwnd` is different for TCP Linux Reno and TCP NewReno.

Following graphs shows the behavior of window growth in TCP Linux Reno and TCP NewReno with delayed acknowledgement of 2 segments:

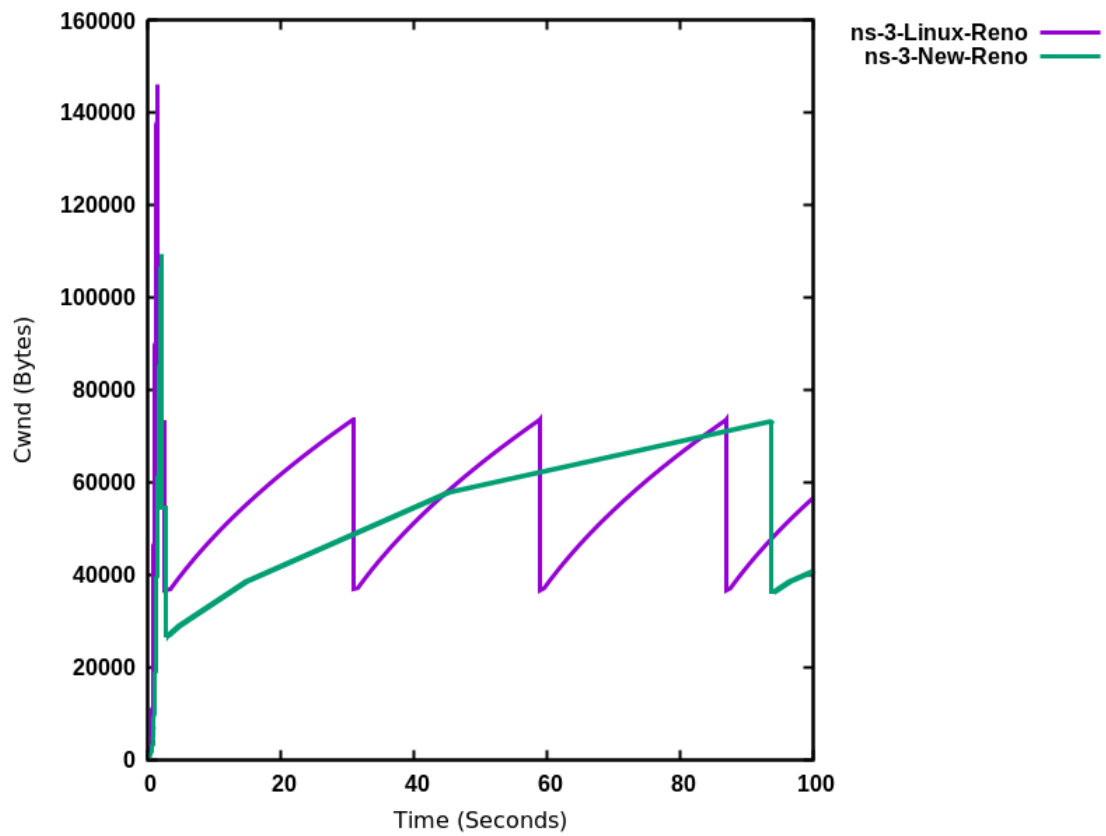


Fig. 16.6: ns-3 TCP NewReno vs. ns-3 TCP Linux Reno

HighSpeed

TCP HighSpeed is designed for high-capacity channels or, in general, for TCP connections with large congestion windows. Conceptually, with respect to the standard TCP, HighSpeed makes the cW_{nd} grow faster during the probing phases and accelerates the cW_{nd} recovery from losses. This behavior is executed only when the window grows beyond a certain threshold, which allows TCP HighSpeed to be friendly with standard TCP in environments with heavy congestion, without introducing new dangers of congestion collapse.

Mathematically:

$$cW_{nd} = cW_{nd} + \frac{a(cW_{nd})}{cW_{nd}} \quad (16.4)$$

The function $a()$ is calculated using a fixed RTT the value 100 ms (the lookup table for this function is taken from RFC 3649). For each congestion event, the slow start threshold is decreased by a value that depends on the size of the slow start threshold itself. Then, the congestion window is set to such value.

$$cW_{nd} = (1 - b(cW_{nd})) \cdot cW_{nd} \quad (16.5)$$

The lookup table for the function $b()$ is taken from the same RFC. More information at: <http://dl.acm.org/citation.cfm?id=2756518>

Hybla

The key idea behind TCP Hybla is to obtain for long RTT connections the same instantaneous transmission rate of a reference TCP connection with lower RTT. With analytical steps, it is shown that this goal can be achieved by modifying the time scale, in order for the throughput to be independent from the RTT. This independence is obtained through the use of a coefficient ρ .

This coefficient is used to calculate both the slow start threshold and the congestion window when in slow start and in congestion avoidance, respectively.

More information at: <http://dl.acm.org/citation.cfm?id=2756518>

Westwood

Westwood and Westwood+ employ the AIAD (Additive Increase/Adaptive Decrease) congestion control paradigm. When a congestion episode happens, instead of halving the $cwnd$, these protocols try to estimate the network's bandwidth and use the estimated value to adjust the $cwnd$. While Westwood performs the bandwidth sampling every ACK reception, Westwood+ samples the bandwidth every RTT.

The TCP Westwood model has been removed in ns-3.38 due to bugs that are impossible to fix without modifying the original Westwood model as presented in the published papers. For further info refer to <https://gitlab.com/nsnam/ns-3-dev/-/issues/579>

The Westwood+ model does not have such issues, and is still available.

WARNING: this TCP model lacks validation and regression tests; use with caution.

More information at: <http://dl.acm.org/citation.cfm?id=381704> and <http://dl.acm.org/citation.cfm?id=2512757>

Vegas

TCP Vegas is a pure delay-based congestion control algorithm implementing a proactive scheme that tries to prevent packet drops by maintaining a small backlog at the bottleneck queue. Vegas continuously samples the RTT and computes the actual throughput a connection achieves using Equation (16.6) and compares it with the expected throughput calculated in Equation (16.7). The difference between these 2 sending rates in Equation (16.8) reflects the amount of extra packets being queued at the bottleneck.

$$actual = \frac{cWnd}{RTT} \quad (16.6)$$

$$expected = \frac{cWnd}{BaseRTT} \quad (16.7)$$

$$diff = expected - actual \quad (16.8)$$

To avoid congestion, Vegas linearly increases/decreases its congestion window to ensure the diff value falls between the two predefined thresholds, alpha and beta. diff and another threshold, gamma, are used to determine when Vegas should change from its slow-start mode to linear increase/decrease mode. Following the implementation of Vegas in Linux, we use 2, 4, and 1 as the default values of alpha, beta, and gamma, respectively, but they can be modified through the Attribute system.

More information at: <http://dx.doi.org/10.1109/49.464716>

Scalable

Scalable improves TCP performance to better utilize the available bandwidth of a highspeed wide area network by altering NewReno congestion window adjustment algorithm. When congestion has not been detected, for each ACK received in an RTT, Scalable increases its cwnd per:

$$cwnd = cwnd + 0.01 \quad (16.9)$$

Following Linux implementation of Scalable, we use 50 instead of 100 to account for delayed ACK.

On the first detection of congestion in a given RTT, cwnd is reduced based on the following equation:

$$cwnd = cwnd - \text{ceil}(0.125 \cdot cwnd) \quad (16.10)$$

More information at: <http://dl.acm.org/citation.cfm?id=956989>

Veno

TCP Veno enhances Reno algorithm for more effectively dealing with random packet loss in wireless access networks by employing Vegas's method in estimating the backlog at the bottleneck queue to distinguish between congestive and non-congestive states.

The backlog (the number of packets accumulated at the bottleneck queue) is calculated using Equation (16.11):

$$\begin{aligned} N &= Actual \cdot (RTT - BaseRTT) \\ &= Diff \cdot BaseRTT \end{aligned} \quad (16.11)$$

where:

$$\begin{aligned} Diff &= Expected - Actual \\ &= \frac{cWnd}{BaseRTT} - \frac{cWnd}{RTT} \end{aligned} \quad (16.12)$$

Veno makes decision on cwnd modification based on the calculated N and its predefined threshold beta.

Specifically, it refines the additive increase algorithm of Reno so that the connection can stay longer in the stable state by incrementing cwnd by $1/cwnd$ for every other new ACK received after the available bandwidth has been fully utilized, i.e. when N exceeds beta. Otherwise, Veno increases its cwnd by $1/cwnd$ upon every new ACK receipt as in Reno.

In the multiplicative decrease algorithm, when Veno is in the non-congestive state, i.e. when N is less than beta, Veno decrements its cwnd by only $1/5$ because the loss encountered is more likely a corruption-based loss than a congestion-based. Only when N is greater than beta, Veno halves its sending rate as in Reno.

More information at: <http://dx.doi.org/10.1109/JSAC.2002.807336>

BIC

BIC (class TcpBic) is a predecessor of TCP CUBIC. In TCP BIC the congestion control problem is viewed as a search problem. Taking as a starting point the current window value and as a target point the last maximum window value (i.e. the cWnd value just before the loss event) a binary search technique can be used to update the cWnd value at the midpoint between the two, directly or using an additive increase strategy if the distance from the current window is too large.

This way, assuming a no-loss period, the congestion window logarithmically approaches the maximum value of cWnd until the difference between it and cWnd falls below a preset threshold. After reaching such a value (or the maximum window is unknown, i.e. the binary search does not start at all) the algorithm switches to probing the new maximum window with a ‘slow start’ strategy.

If a loss occur in either these phases, the current window (before the loss) can be treated as the new maximum, and the reduced (with a multiplicative decrease factor Beta) window size can be used as the new minimum.

More information at: <http://ieeexplore.ieee.org/xpl/articleDetails.jsp?arnumber=1354672>

YeAH

YeAH-TCP (Yet Another HighSpeed TCP) is a heuristic designed to balance various requirements of a state-of-the-art congestion control algorithm:

1. fully exploit the link capacity of high BDP networks while inducing a small number of congestion events
2. compete friendly with Reno flows
3. achieve intra and RTT fairness
4. robust to random losses
5. achieve high performance regardless of buffer size

YeAH operates between 2 modes: Fast and Slow mode. In the Fast mode when the queue occupancy is small and the network congestion level is low, YeAH increments its congestion window according to the aggressive HSTCP rule. When the number of packets in the queue grows beyond a threshold and the network congestion level is high, YeAH enters its Slow mode, acting as Reno with a decongestion algorithm. YeAH employs Vegas’ mechanism for calculating the backlog as in Equation (16.13). The estimation of the network congestion level is shown in Equation (16.14).

$$Q = (RTT - BaseRTT) \cdot \frac{cWnd}{RTT} \quad (16.13)$$

$$L = \frac{RTT - BaseRTT}{BaseRTT} \quad (16.14)$$

To ensure TCP friendliness, YeAH also implements an algorithm to detect the presence of legacy Reno flows. Upon the receipt of 3 duplicate ACKs, YeAH decreases its slow start threshold according to Equation (16.15) if it’s not competing

with Reno flows. Otherwise, the ssthresh is halved as in Reno:

$$ssthresh = \min(\max(\frac{cWnd}{8}, Q), \frac{cWnd}{2}) \quad (16.15)$$

More information: http://www.csc.lsu.edu/~sjpark/cs7601/4-YeAH_TCP.pdf

Illinois

TCP Illinois is a hybrid congestion control algorithm designed for high-speed networks. Illinois implements a Concave-AIMD (or C-AIMD) algorithm that uses packet loss as the primary congestion signal to determine the direction of window update and queueing delay as the secondary congestion signal to determine the amount of change.

The additive increase and multiplicative decrease factors (denoted as alpha and beta, respectively) are functions of the current average queueing delay da as shown in Equations (16.16) and (16.17). To improve the protocol robustness against sudden fluctuations in its delay sampling, Illinois allows the increment of alpha to alphaMax only if da stays below $d1$ for a some (theta) amount of time.

$$\alpha = \begin{cases} \alpha_{Max} & \text{if } da \leq d1 \\ k1/(k2 + da) & \text{otherwise} \end{cases} \quad (16.16)$$

$$\beta = \begin{cases} \beta_{Min} & \text{if } da \leq d2 \\ k3 + k4 da & \text{if } d2 < da < d3 \\ \beta_{Max} & \text{otherwise} \end{cases} \quad (16.17)$$

where the calculations of $k1$, $k2$, $k3$, and $k4$ are shown in the following:

$$k1 = \frac{(dm - d1) \cdot \alpha_{Min} \cdot \alpha_{Max}}{\alpha_{Max} - \alpha_{Min}} \quad (16.18)$$

$$k2 = \frac{(dm - d1) \cdot \alpha_{Min}}{\alpha_{Max} - \alpha_{Min}} - d1 \quad (16.19)$$

$$k3 = \frac{\alpha_{Min} \cdot d3 - \alpha_{Max} \cdot d2}{d3 - d2} \quad (16.20)$$

$$k4 = \frac{\alpha_{Max} - \alpha_{Min}}{d3 - d2} \quad (16.21)$$

Other parameters include da (the current average queueing delay), and Ta (the average RTT, calculated as $\text{sumRtt} / \text{cntRtt}$ in the implementation) and $Tmin$ (baseRtt in the implementation) which is the minimum RTT ever seen. dm is the maximum (average) queueing delay, and $Tmax$ (maxRtt in the implementation) is the maximum RTT ever seen.

$$da = Ta - Tmin \quad (16.22)$$

$$dm = Tmax - Tmin \quad (16.23)$$

$$d_i = \eta_i \cdot dm \quad (16.24)$$

Illinois only executes its adaptation of alpha and beta when $cwnd$ exceeds a threshold called winThresh . Otherwise, it sets alpha and beta to the base values of 1 and 0.5, respectively.

Following the implementation of Illinois in the Linux kernel, we use the following default parameter settings:

- $\alpha_{Min} = 0.3$ (0.1 in the Illinois paper)
- $\alpha_{Max} = 10.0$
- $\beta_{Min} = 0.125$

- betaMax = 0.5
- winThresh = 15 (10 in the Illinois paper)
- theta = 5
- eta1 = 0.01
- eta2 = 0.1
- eta3 = 0.8

More information: <http://www.doi.org/10.1145/1190095.1190166>

H-TCP

H-TCP has been designed for high BDP (Bandwidth-Delay Product) paths. It is a dual mode protocol. In normal conditions, it works like traditional TCP with the same rate of increment and decrement for the congestion window. However, in high BDP networks, when it finds no congestion on the path after `delta1` seconds, it increases the window size based on the alpha function in the following:

$$\alpha(\text{delta}) = 1 + 10(\text{delta} - \text{delta1}) + 0.5(\text{delta} - \text{delta1})^2 \quad (16.25)$$

where `delta1` is a threshold in seconds for switching between the modes and `delta` is the elapsed time from the last congestion. During congestion, it reduces the window size by multiplying by beta function provided in the reference paper. The calculated throughput between the last two consecutive congestion events is considered for beta calculation.

The transport `TcpHtcp` can be selected in the program `examples/tcp/tcp-variants-comparison.cc` to perform an experiment with H-TCP, although it is useful to increase the bandwidth in this example (e.g. to 20 Mb/s) to create a higher BDP link, such as:

```
./ns3 run "tcp-variants-comparison --transport_prot=TcpHtcp --bandwidth=20Mbps --  
↪duration=10"
```

More information (paper): <http://www.hamilton.ie/net/htcp3.pdf>

More information (Internet Draft): <https://tools.ietf.org/html/draft-leith-tcp-htcp-06>

LEDBAT

Low Extra Delay Background Transport (LEDBAT) is an experimental delay-based congestion control algorithm that seeks to utilize the available bandwidth on an end-to-end path while limiting the consequent increase in queueing delay on that path. LEDBAT uses changes in one-way delay measurements to limit congestion that the flow itself induces in the network.

As a first approximation, the LEDBAT sender operates as shown below:

On receipt of an ACK:

```
currentdelay = acknowledgement.delay;  
basedelay = min(basedelay, currentdelay);  
queuingdelay = currentdelay - basedelay;  
offtarget = (TARGET - queuingdelay) / TARGET;  
cwnd += GAIN * offtarget * bytesnewlyacked * MSS / cwnd;
```

TARGET is the maximum queueing delay that LEDBAT itself may introduce in the network, and GAIN determines the rate at which the cwnd responds to changes in queueing delay; offtarget is a normalized value representing the difference

between the measured current queueing delay and the predetermined TARGET delay. `offtarget` can be positive or negative; consequently, `cwnd` increases or decreases in proportion to `offtarget`.

Following the recommendation of RFC 6817, the default values of the parameters are:

- `TargetDelay = 100`
- `baseHistoryLen = 10`
- `noiseFilterLen = 4`
- `Gain = 1`

To enable LEDBAT on all TCP sockets, the following configuration can be used:

```
Config::SetDefault("ns3::TcpL4Protocol::SocketType",  
↳TypeIdValue(TcpLedbat::GetTypeId()));
```

To enable LEDBAT on a chosen TCP socket, the following configuration can be used:

```
Config::Set("$ns3::NodeListPriv/NodeList/1/$ns3::TcpL4Protocol/SocketType",  
↳TypeIdValue(TcpLedbat::GetTypeId()));
```

The following unit tests have been written to validate the implementation of LEDBAT:

- LEDBAT should operate same as NewReno during slow start
- LEDBAT should operate same as NewReno if timestamps are disabled
- Test to validate `cwnd` increment in LEDBAT

In comparison to RFC 6817, the scope and limitations of the current LEDBAT implementation are:

- It assumes that the clocks on the sender side and receiver side are synchronised
- In line with Linux implementation, the one-way delay is calculated at the sender side by using the timestamps option in TCP header
- Only the MIN function is used for noise filtering

More information about LEDBAT is available in RFC 6817: <https://tools.ietf.org/html/rfc6817>

TCP-LP

TCP-Low Priority (TCP-LP) is a delay based congestion control protocol in which the low priority data utilizes only the excess bandwidth available on an end-to-end path. TCP-LP uses one way delay measurements as an indicator of congestion as it does not influence cross-traffic in the reverse direction.

On receipt of an ACK:

$$\begin{aligned}\text{One way delay} &= \text{Receiver timestamp} - \text{Receiver timestamp echo reply} \\ \text{Smoothed one way delay} &= \frac{7}{8} \times \text{Old Smoothed one way delay} + \frac{1}{8} \times \text{one way delay} \\ \text{If smoothed one way delay} &> \text{owdMin} + \frac{15 \times (\text{owdMax} - \text{owdMin})}{100} \\ &\quad \text{if LP_WITHIN_INF} \\ &\quad \quad \text{cwnd} = 1 \\ &\quad \text{else} \\ &\quad \quad \text{cwnd} = \frac{\text{cwnd}}{2} \\ &\quad \text{Inference timer is set}\end{aligned}$$

where owdMin and owdMax are the minimum and maximum one way delays experienced throughout the connection, LP_WITHIN_INF indicates if TCP-LP is in inference phase or not

More information (paper): <http://cs.northwestern.edu/~akuzma/rice/doc/TCP-LP.pdf>

Data Center TCP (DCTCP)

DCTCP, specified in RFC 8257 and implemented in Linux, is a TCP congestion control algorithm for data center networks. It leverages Explicit Congestion Notification (ECN) to provide more fine-grained congestion feedback to the end hosts, and is intended to work with routers that implement a shallow congestion marking threshold (on the order of a few milliseconds) to achieve high throughput and low latency in the datacenter. However, because DCTCP does not react in the same way to notification of congestion experienced, there are coexistence (fairness) issues between it and legacy TCP congestion controllers, which is why it is recommended to only be used in controlled networking environments such as within data centers.

DCTCP extends the Explicit Congestion Notification signal to estimate the fraction of bytes that encounter congestion, rather than simply detecting that the congestion has occurred. DCTCP then scales the congestion window based on this estimate. This approach achieves high burst tolerance, low latency, and high throughput with shallow-buffered switches.

- *Receiver functionality:* If CE is observed in the IP header of an incoming packet at the TCP receiver, the receiver sends congestion notification to the sender by setting ECE in TCP header. This processing is different from standard receiver ECN processing which sets and holds the ECE bit for every ACK until it observes a CWR signal from the TCP sender.
- *Sender functionality:* The sender makes use of the modified receiver ECE semantics to maintain an estimate of the fraction of packets marked (α) by using the exponential weighted moving average (EWMA) as shown below:

$$\alpha = (1 - g) * \alpha + g * F$$

In the above EWMA:

- g is the estimation gain (between 0 and 1)
- F is the fraction of packets marked in current RTT.

For send windows in which at least one ACK was received with ECE set, the sender should respond by reducing the congestion window as follows, once for every window of data:

$$\text{cwnd} = \text{cwnd} * (1 - \alpha/2)$$

Following the recommendation of RFC 8257, the default values of the parameters are:

$$g = 0.0625(i.e., 1/16)$$

$$initialalpha(\alpha) = 1$$

To enable DCTCP on all TCP sockets, the following configuration can be used:

```
Config::SetDefault("ns3::TcpL4Protocol::SocketType", TypeIdValue(TcpDctcp::GetTypeId()));
```

To enable DCTCP on a selected node, one can set the “SocketType” attribute on the TcpL4Protocol object of that node to the TcpDctcp TypeId.

The ECN is enabled automatically when DCTCP is used, even if the user has not explicitly enabled it.

DCTCP depends on a simple queue management algorithm in routers / switches to mark packets. The current implementation of DCTCP in ns-3 can use RED with a simple configuration to achieve the behavior of desired queue management algorithm.

To configure RED router for DCTCP:

```
Config::SetDefault("ns3::RedQueueDisc::UseEcn", BooleanValue(true));
Config::SetDefault("ns3::RedQueueDisc::QW", DoubleValue(1.0));
Config::SetDefault("ns3::RedQueueDisc::MinTh", DoubleValue(16));
Config::SetDefault("ns3::RedQueueDisc::MaxTh", DoubleValue(16));
```

There is also the option, when running CoDel or FqCoDel, to enable ECN on the queue and to set the “CeThreshold” value to a low value such as 1ms. The following example uses CoDel:

```
Config::SetDefault("ns3::CoDelQueueDisc::UseEcn", BooleanValue(true));
Config::SetDefault("ns3::CoDelQueueDisc::CeThreshold", TimeValue(MilliSeconds(1)));
```

The following unit tests have been written to validate the implementation of DCTCP:

- ECT flags should be set for SYN, SYN+ACK, ACK and data packets for DCTCP traffic
- ECT flags should not be set for SYN, SYN+ACK and pure ACK packets, but should be set on data packets for ECN enabled traditional TCP flows
- ECE should be set only when CE flags are received at receiver and even if sender doesn’t send CWR, receiver should not send ECE if it doesn’t receive packets with CE flags

An example program, `examples/tcp/tcp-validation.cc`, can be used to experiment with DCTCP for long-running flows with different bottleneck link bandwidth, base RTTs, and queuing disciplines. A variant of this program has also been run using the ns-3 Direct Code Execution environment using DCTCP from Linux kernel 4.4, and the results were compared against ns-3 results.

An example program based on an experimental topology found in the original DCTCP SIGCOMM paper is provided in `examples/tcp/dctcp-example.cc`. This example uses a simple topology consisting of forty DCTCP senders and receivers and two ECN-enabled switches to examine throughput, fairness, and queue delay properties of the network.

This implementation was tested extensively against a version of DCTCP in the Linux kernel version 4.4 using the ns-3 direct code execution (DCE) environment. Some differences were noted:

- Linux maintains its congestion window in segments and not bytes, and the arithmetic is not floating point, so small differences in the evolution of congestion window have been observed.
- Linux uses pacing, where packets to be sent are paced out at regular intervals. However, if at any instant the number of segments that can be sent are less than two, Linux does not pace them and instead sends them back-to-back. Currently, ns-3 paces out all packets eligible to be sent in the same manner.

It is important to also note that the current model does not implement Section 3.5 of RFC 8257 regarding the handling of packet loss. This requirement states that DCTCP must react to lost packets in the same way as does a conventional TCP (as specified in RFC 5681). The current DCTCP model does not implement this, and should therefore only be used in simulations that do not involve any packet loss on the DCTCP flows.

More information about DCTCP is available in the RFC 8257: <https://tools.ietf.org/html/rfc8257>

BBR

BBR (class `TcpBbr`) is a congestion control algorithm that regulates the sending rate by deriving an estimate of the bottleneck's available bandwidth and RTT of the path. It seeks to operate at an optimal point where sender experiences maximum delivery rate with minimum RTT. It creates a network model comprising maximum delivery rate with minimum RTT observed so far, and then estimates BDP (maximum bandwidth * minimum RTT) to control the maximum amount of inflight data. BBR controls congestion by limiting the rate at which packets are sent. It caps the `cwnd` to one BDP and paces out packets at a rate which is adjusted based on the latest estimate of delivery rate. BBR algorithm is agnostic to packet losses and ECN marks.

`pacing_gain` controls the rate of sending data and `cwnd_gain` controls the amount of data to send.

The following is a high level overview of BBR congestion control algorithm:

On receiving an ACK:

```
rtt = now - packet.sent_time;
update_minimum_rtt(rtt);
delivery_rate = estimate_delivery_rate(packet);
update_maximum_bandwidth(delivery_rate);
```

After transmitting a data packet:

```
bdp = max_bandwidth * min_rtt;
if (cwnd * bdp < inflight)
{
    return;
}
if (now > nextSendTime)
{
    transmit(packet);
    nextSendTime = now + packet.size / (pacing_gain * max_bandwidth);
}
else
{
    return;
}
Schedule(nextSendTime, Send);
```

To enable BBR on all TCP sockets, the following configuration can be used:

```
Config::SetDefault("ns3::TcpL4Protocol::SocketType", TypeIdValue(TcpBbr::GetTypeId()));
```

To enable BBR on a chosen TCP socket, the following configuration can be used (note that an appropriate Node ID must be used instead of 1):

```
Config::Set("$ns3::NodeListPriv/NodeList/1/$ns3::TcpL4Protocol/SocketType",
↳ TypeIdValue(TcpBbr::GetTypeId()));
```

The ns-3 implementation of BBR is based on its Linux implementation. Linux 5.4 kernel implementation has been used to validate the behavior of ns-3 implementation of BBR (See below section on Validation).

In addition, the following unit tests have been written to validate the implementation of BBR in ns-3:

- BBR should enable (if not already done) TCP pacing feature.
- Test to validate the values of `pacing_gain` and `cwnd_gain` in different phases of BBR.

An example program, `examples/tcp/tcp-bbr-example.cc`, is provided to experiment with BBR for one long running flow. This example uses a simple topology consisting of one sender, one receiver and two routers to examine congestion window, throughput and queue control. A program similar to this has been run using the Network Stack Tester (NeST) using BBR from Linux kernel 5.4, and the results were compared against ns-3 results.

More information about BBR is available in the following Internet Draft: <https://tools.ietf.org/html/draft-cardwell-iccr-g-bbr-congestion-control-00>

More information about Delivery Rate Estimation is in the following draft: <https://tools.ietf.org/html/draft-cheng-iccr-g-delivery-rate-estimation-00>

For an academic peer-reviewed paper on the BBR implementation in ns-3, please refer to:

- Vivek Jain, Viyom Mittal and Mohit P. Tahiliani. “Design and Implementation of TCP BBR in ns-3.” In Proceedings of the 10th Workshop on ns-3, pp. 16-22. 2018. (<https://dl.acm.org/doi/abs/10.1145/3199902.3199911>)

Integration with TcpRateOps

In the older version of the TCP BBR algorithm, the `appLimited` variable was handled directly inside the `TcpBbr` class. To make the code more modular and similar to how things are done in Linux TCP, a pointer to the `TcpRateOps` class called `Ptr<TcpRateOps> m_rateOps` was introduced. This change helps manage the `appLimited` state through the rate operations interface instead of directly in `TcpBbr`.

Now, the `appLimited` value can be accessed with:

```
m_rateOps->GetConnectionRate().m_appLimited;
```

And it can be updated using:

```
m_rateOps->SetAppLimited(tcb->m_bytesInFlight.Get());
```

The `SetAppLimited` function, which is part of the `TcpRateLinux` class, takes care of updating the `appLimited` value based on the current amount of data in flight.

Here’s an example of the `SetRateOps` method in `TcpBbr`:

```
void TcpBbr::SetRateOps(Ptr<TcpRateOps> rateOps)
{
    m_rateOps = rateOps;
}
```

This setup makes the code more organized and reflects how Linux TCP handles the `appLimited` state, where it’s managed within the TCP socket and updated by rate operations.

Support for Explicit Congestion Notification (ECN)

ECN provides end-to-end notification of network congestion without dropping packets. It uses two bits in the IP header: ECN Capable Transport (ECT bit) and Congestion Experienced (CE bit), and two bits in the TCP header: Congestion Window Reduced (CWR) and ECN Echo (ECE).

More information is available in RFC 3168: <https://tools.ietf.org/html/rfc3168>

The following ECN states are declared in `src/internet/model/tcp-socket-state.h`

```
enum EcnStates_t
{
    ECN_DISABLED = 0, ///< ECN disabled traffic
    ECN_IDLE, ///< ECN is enabled but currently there is no action pertaining to
    ↪ECE or CWR to be taken
    ECN_CE_RCVD, ///< Last packet received had CE bit set in IP header
    ECN_SENDING_ECE, ///< Receiver sends an ACK with ECE bit set in TCP header
    ECN_ECE_RCVD, ///< Last ACK received had ECE bit set in TCP header
    ECN_CWR_SENT ///< Sender has reduced the congestion window, and sent a packet
    ↪with CWR bit set in TCP header. This is used for tracing.
};
```

Current implementation of ECN is based on RFC 3168 and is referred as Classic ECN.

The following enum represents the mode of ECN:

```
enum EcnMode_t
{
    ClassicEcn, ///< ECN functionality as described in RFC 3168.
    DctcpEcn, ///< ECN functionality as described in RFC 8257. Note: this mode is
    ↪specific to DCTCP.
};
```

The following are some important ECN parameters:

```
// ECN parameters
EcnMode_t          m_ecnMode {ClassicEcn}; ///< ECN mode
UseEcn_t           m_useEcn {Off}; ///< Socket ECN capability
```

Enabling ECN

By default, support for ECN is disabled in TCP sockets. To enable, change the value of the attribute `ns3::TcpSocketBase::UseEcn` to `On`. Following are supported values for the same, this functionality is aligned with Linux: <https://www.kernel.org/doc/Documentation/networking/ip-sysctl.txt>

```
enum UseEcn_t
{
    Off          = 0, ///< Disable
    On           = 1, ///< Enable
    AcceptOnly = 2, ///< Enable only when the peer endpoint is ECN capable
};
```

For example:

```
Config::SetDefault("ns3::TcpSocketBase::UseEcn", StringValue("On"))
```

ECN negotiation

ECN capability is negotiated during the three-way TCP handshake:

1. Sender sends SYN + CWR + ECE

```
if (m_useEcn == UseEcn_t::On)
{
    SendEmptyPacket(TcpHeader::SYN | TcpHeader::ECE | TcpHeader::CWR);
}
else
{
    SendEmptyPacket(TcpHeader::SYN);
}
m_ecnState = ECN_DISABLED;
```

2. Receiver sends SYN + ACK + ECE

```
if (m_useEcn != UseEcn_t::Off && (tcpHeader.GetFlags() & (TcpHeader::CWR |
↪TcpHeader::ECE)) == (TcpHeader::CWR | TcpHeader::ECE))
{
    SendEmptyPacket(TcpHeader::SYN | TcpHeader::ACK | TcpHeader::ECE);
    m_ecnState = ECN_IDLE;
}
else
{
    SendEmptyPacket(TcpHeader::SYN | TcpHeader::ACK);
    m_ecnState = ECN_DISABLED;
}
```

3. Sender sends ACK

```
if (m_useEcn != UseEcn_t::Off && (tcpHeader.GetFlags() & (TcpHeader::CWR |
↪TcpHeader::ECE)) == (TcpHeader::ECE))
{
    m_ecnState = ECN_IDLE;
}
else
{
    m_ecnState = ECN_DISABLED;
}
```

Once the ECN-negotiation is successful, the sender sends data packets with ECT bits set in the IP header.

Note: As mentioned in [Section 6.1.1 of RFC 3168](#), ECT bits should not be set during ECN negotiation. The ECN negotiation implemented in *ns-3* follows this guideline.

ECN State Transitions

1. Initially both sender and receiver have their `m_ecnState` set as `ECN_DISABLED`
2. Once the ECN negotiation is successful, their states are set to `ECN_IDLE`
3. The receiver's state changes to `ECN_CE_RCVD` when it receives a packet with CE bit set. The state then moves to `ECN_SENDING_ECE` when the receiver sends an ACK with ECE set. This state is retained until a CWR is received, following which, the state changes to `ECN_IDLE`.
4. When the sender receives an ACK with ECE bit set from receiver, its state is set as `ECN_ECE_RCVD`
5. The sender's state changes to `ECN_CWR_SENT` when it sends a packet with CWR bit set. It remains in this state until an ACK with valid ECE is received (i.e., ECE is received for a packet that belongs to a new window), following which, its state changes to `ECN_ECE_RCVD`.

RFC 3168 compliance

Based on the suggestions provided in RFC 3168, the following behavior has been implemented:

1. Pure ACK packets should not have the ECT bit set ([Section 6.1.4](#)).
2. In the current implementation, the sender only sends ECT(0) in the IP header.
3. The sender should reduce the congestion window only once in each window ([Section 6.1.2](#)).
4. The receiver should ignore the CE bits set in a packet arriving out of window ([Section 6.1.5](#)).
5. The sender should ignore the ECE bits set in the packet arriving out of window ([Section 6.1.2](#)).

Open issues

The following issues are yet to be addressed:

1. Retransmitted packets should not have the CWR bit set ([Section 6.1.5](#)).
2. Despite the congestion window size being 1 MSS, the sender should reduce its congestion window by half when it receives a packet with the ECE bit set. The sender must reset the retransmit timer on receiving the ECN-Echo packet when the congestion window is one. The sending TCP will then be able to send a new packet only when the retransmit timer expires ([Section 6.1.2](#)).
3. Support for separately handling the enabling of ECN on the incoming and outgoing TCP sessions (e.g. a TCP may perform ECN echoing but not set the ECT codepoints on its outbound data segments).

Support for Dynamic Pacing

TCP pacing refers to the sender-side practice of scheduling the transmission of a burst of eligible TCP segments across a time interval such as a TCP RTT, to avoid or reduce bursts. Historically, TCP used the natural ACK clocking mechanism to pace segments, but some network paths introduce aggregation (bursts of ACKs arriving) or ACK thinning, either of which disrupts ACK clocking. Some latency-sensitive congestion controls under development (Prague, BBR) require pacing to operate effectively.

Until recently, the state of the art in Linux was to support pacing in one of two ways:

- 1) fq/pacing with `sch_fq`
- 2) TCP internal pacing

The presentation by Dumazet and Cheng at IETF 88 summarizes: <https://www.ietf.org/proceedings/88/slides/slides-88-tcpm-9.pdf>

The first option was most often used when offloading (TSO) was enabled and when the sch_fq scheduler was used at the traffic control (qdisc) sublayer. In this case, TCP was responsible for setting the socket pacing rate, but the qdisc sublayer would enforce it. When TSO was enabled, the kernel would break a large burst into smaller chunks, with dynamic sizing based on the pacing rate, and hand off the segments to the fq qdisc for pacing.

The second option was used if sch_fq was not enabled; TCP would be responsible for internally pacing.

In 2018, Linux switched to an Early Departure Model (EDM): <https://lwn.net/Articles/766564/>.

TCP pacing in Linux was added in kernel 3.12, and authors chose to allow a pacing rate of 200% against the current rate, to allow probing for optimal throughput even during slow start phase. Some refinements were added in <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=43e122b014c9>, in which Google reported that it was better to apply a different ratio (120%) in Congestion Avoidance phase. Furthermore, authors found that after cwnd reduction, it was helpful to become more conservative and switch to the conservative ratio (120%) as soon as $cwnd \geq ssthresh/2$, as the initial ramp up (when ssthresh is infinite) still allows doubling cwnd every other RTT. Linux also does not pace the initial window (IW), typically 10 segments in practice.

Linux has also been observed to not pace if the number of eligible segments to be sent is exactly two; they will be sent back to back. If three or more, the first two are sent immediately, and additional segments are paced at the current pacing rate.

In ns-3, the model is as follows. There is no TSO/sch_fq model; only internal pacing according to current Linux policy.

Pacing may be enabled for any TCP congestion control, and a maximum pacing rate can be set. Furthermore, dynamic pacing is enabled for all TCP variants, according to the following guidelines.

- Pacing of the initial window (IW) is not done by default but can be separately enabled.
- Pacing of the initial slow start, after IW, is done according to the pacing rate of 200% of the current rate, to allow for window growth. This pacing rate can be configured to a different value than 200%.
- Pacing of congestion avoidance phase is done at a pacing rate of 120% of current rate. This can be configured to a different value than 120%.
- Pacing of subsequent slow start is done according to the following heuristic. If $cwnd < ssthresh/2$, such as after a timeout or idle period, pace at the slow start rate (200%). Otherwise, pace at the congestion avoidance rate.

Dynamic pacing is demonstrated by the example program `examples/tcp/tcp-pacing.cc`.

Validation

The following tests are found in the `src/internet/test` directory. In general, TCP tests inherit from a class called `TcpGeneralTest`, which provides common operations to set up test scenarios involving TCP objects. For more information on how to write new tests, see the section below on *Writing TCP tests*.

- **tcp:** Basic transmission of string of data from client to server
- **tcp-bytes-in-flight-test:** TCP correctly estimates bytes in flight under loss conditions
- **tcp-cong-avoid-test:** TCP congestion avoidance for different packet sizes
- **tcp-datasentcb:** Check TCP's 'data sent' callback
- **tcp-endpoint-bug2211-test:** A test for an issue that was causing stack overflow
- **tcp-fack-test:** Unit tests on FACK
- **tcp-fast-retr-test:** Fast Retransmit testing
- **tcp-header:** Unit tests on the TCP header

- **tcp-highspeed-test:** Unit tests on the HighSpeed congestion control
- **tcp-htcp-test:** Unit tests on the H-TCP congestion control
- **tcp-hybla-test:** Unit tests on the Hybla congestion control
- **tcp-vegas-test:** Unit tests on the Vegas congestion control
- **tcp-veno-test:** Unit tests on the Veno congestion control
- **tcp-scalable-test:** Unit tests on the Scalable congestion control
- **tcp-bic-test:** Unit tests on the BIC congestion control
- **tcp-yeah-test:** Unit tests on the YeAH congestion control
- **tcp-illinois-test:** Unit tests on the Illinois congestion control
- **tcp-ledbat-test:** Unit tests on the LEDBAT congestion control
- **tcp-lp-test:** Unit tests on the TCP-LP congestion control
- **tcp-dctcp-test:** Unit tests on the DCTCP congestion control
- **tcp-bbr-test:** Unit tests on the BBR congestion control
- **tcp-option:** Unit tests on TCP options
- **tcp-pkts-acked-test:** Unit test the number of time that PktsAcked is called
- **tcp-rto-test:** Unit test behavior after a RTO occurs
- **tcp-rtt-estimation-test:** Check RTT calculations, including retransmission cases
- **tcp-slow-start-test:** Check behavior of slow start
- **tcp-timestamp:** Unit test on the timestamp option
- **tcp-wscaling:** Unit test on the window scaling option
- **tcp-zero-window-test:** Unit test persist behavior for zero window conditions
- **tcp-close-test:** Unit test on the socket closing: both receiver and sender have to close their socket when all bytes are transferred
- **tcp-ecn-test:** Unit tests on Explicit Congestion Notification
- **tcp-pacing-test:** Unit tests on dynamic TCP pacing rate

Several tests have dependencies outside of the `internet` module, so they are located in a system test directory called `src/test/ns3tcp`.

- **ns3-tcp-loss:** Check behavior of ns-3 TCP upon packet losses
- **ns3-tcp-no-delay:** Check that ns-3 TCP Nagle's algorithm works correctly and that it can be disabled
- **ns3-tcp-socket:** Check that ns-3 TCP successfully transfers an application data write of various sizes
- **ns3-tcp-state:** Check the operation of the TCP state machine for several cases

Several TCP validation test results can also be found in the [wiki page](#) describing this implementation.

The ns-3 implementation of TCP Linux Reno was validated against the NewReno implementation of Linux kernel 4.4.0 using ns-3 Direct Code Execution (DCE). DCE is a framework which allows the users to run kernel space protocol inside ns-3 without changing the source code.

In this validation, cwnd traces of DCE Linux reno were compared to those of ns-3 Linux Reno and NewReno for a delayed acknowledgement configuration of 1 segment (in the ns-3 implementation; Linux does not allow direct configuration of this setting). It can be observed that cwnd traces for ns-3 Linux Reno are closely overlapping with DCE reno, while for ns-3 NewReno there was deviation in the congestion avoidance phase.

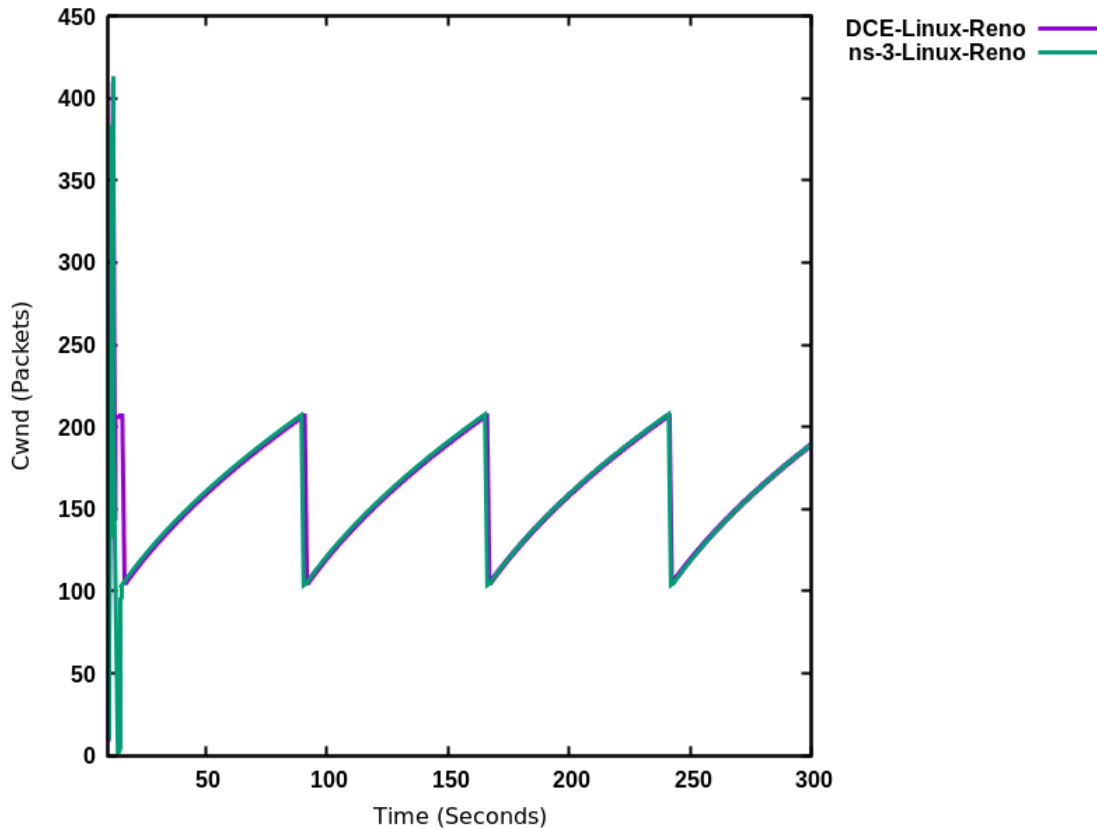


Fig. 16.7: DCE Linux Reno vs. ns-3 Linux Reno

The difference in the cwnd in the early stage of this flow is because of the way cwnd is plotted. As ns-3 provides a trace source for cwnd, an ns-3 Linux Reno cwnd sample is obtained every time the cwnd value changes, whereas for DCE Linux Reno, the kernel does not have a corresponding trace source. Instead, we use the “ss” command of the Linux kernel to obtain cwnd values. The “ss” samples cwnd at an interval of 0.5 seconds.

Figure *DCTCP throughput for 10ms/50Mbps bottleneck, 1ms CE threshold* shows a long-running file transfer using DCTCP over a 50 Mbps bottleneck (running CoDel queue disc with a 1ms CE threshold setting) with a 10 ms base RTT. The figure shows that DCTCP reaches link capacity very quickly and stays there for the duration with minimal change in throughput. In contrast, Figure *DCTCP throughput for 80ms/50Mbps bottleneck, 1ms CE threshold* plots the throughput for the same configuration except with an 80 ms base RTT. In this case, the DCTCP exits slow start early and takes a long time to build the flow throughput to the bottleneck link capacity. DCTCP is not intended to be used at such a large base RTT, but this figure highlights the sensitivity to RTT (and can be reproduced using the Linux implementation).

Similar to DCTCP, TCP CUBIC has been tested against the Linux kernel version 4.4 implementation. Figure *CUBIC cwnd evolution for 50ms/50Mbps bottleneck, no ECN* compares the congestion window evolution between ns-3 and Linux for a single flow operating over a 50 Mbps link with 50 ms base RTT and the CoDel AQM. Some differences can be observed between the peak of slow start window growth (ns-3 exits slow start earlier due to its HyStart implementation), and the window growth is a bit out-of-sync (likely due to different implementations of the algorithm), but the cubic concave/convex window pattern, and the signs of TCP CUBIC fast convergence algorithm (alternating patterns of cubic and concave window growth) can be observed. The ns-3 congestion window is maintained in bytes (unlike

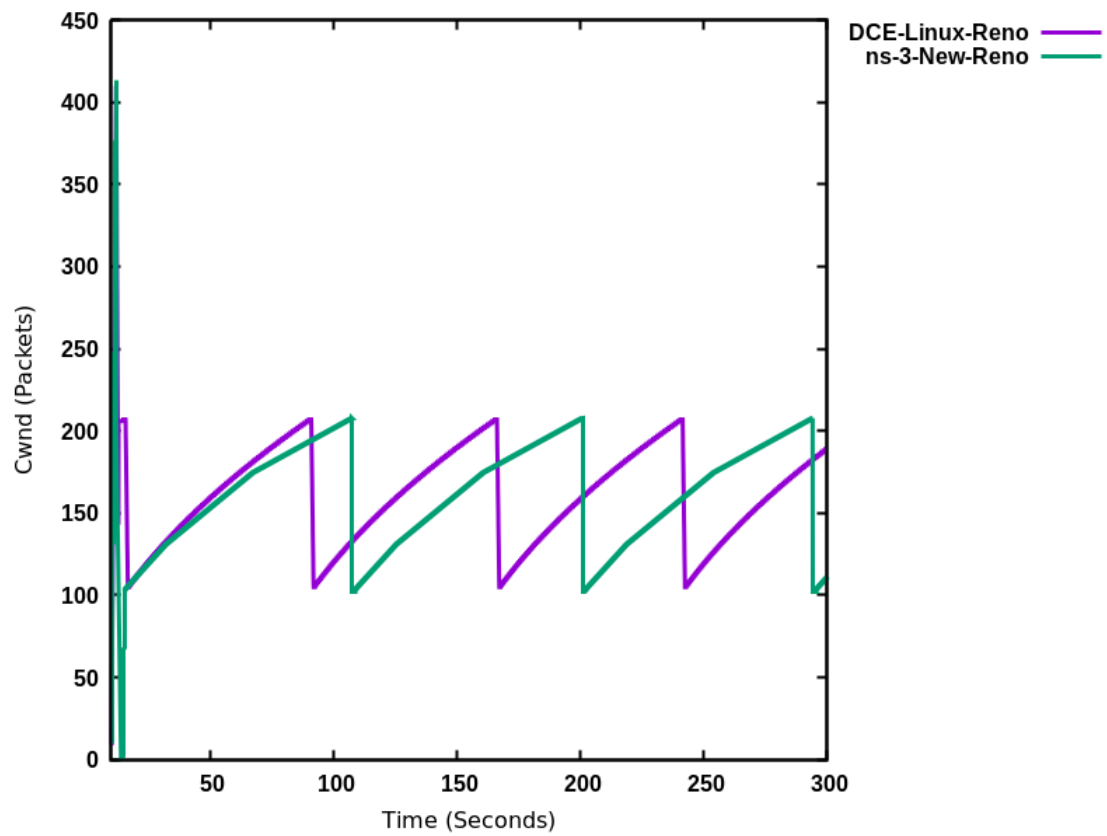


Fig. 16.8: DCE Linux Reno vs. ns-3 NewReno

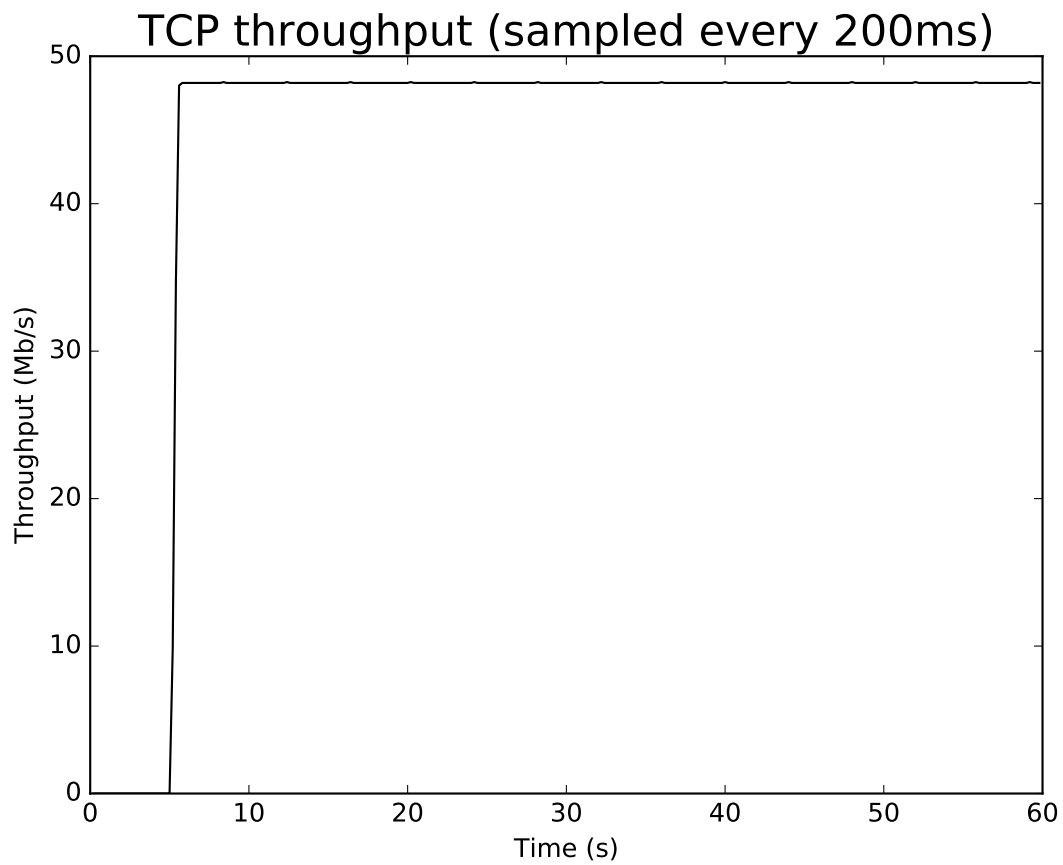


Fig. 16.9: DCTCP throughput for 10ms/50Mbps bottleneck, 1ms CE threshold

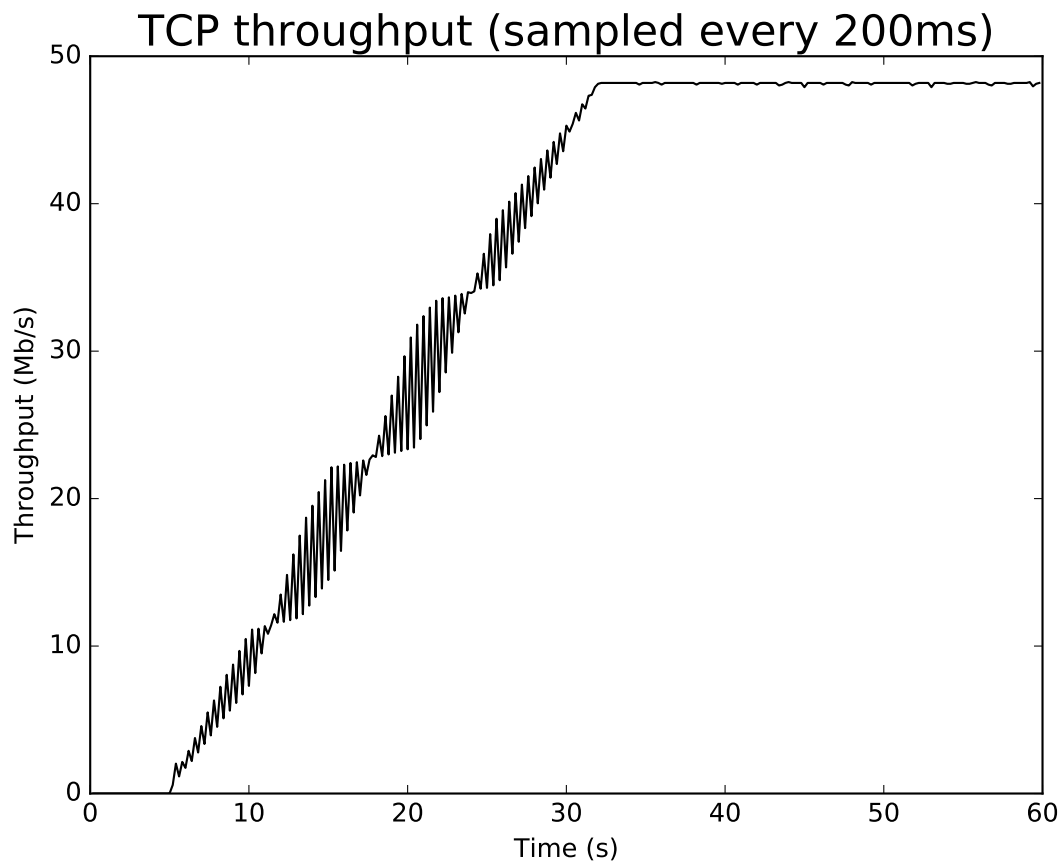


Fig. 16.10: DCTCP throughput for 80ms/50Mbps bottleneck, 1ms CE threshold

Linux which uses segments) but has been normalized to segments for these plots. Figure *CUBIC cwnd evolution for 50ms/50Mbps bottleneck, with ECN* displays the outcome of a similar scenario but with ECN enabled throughout.

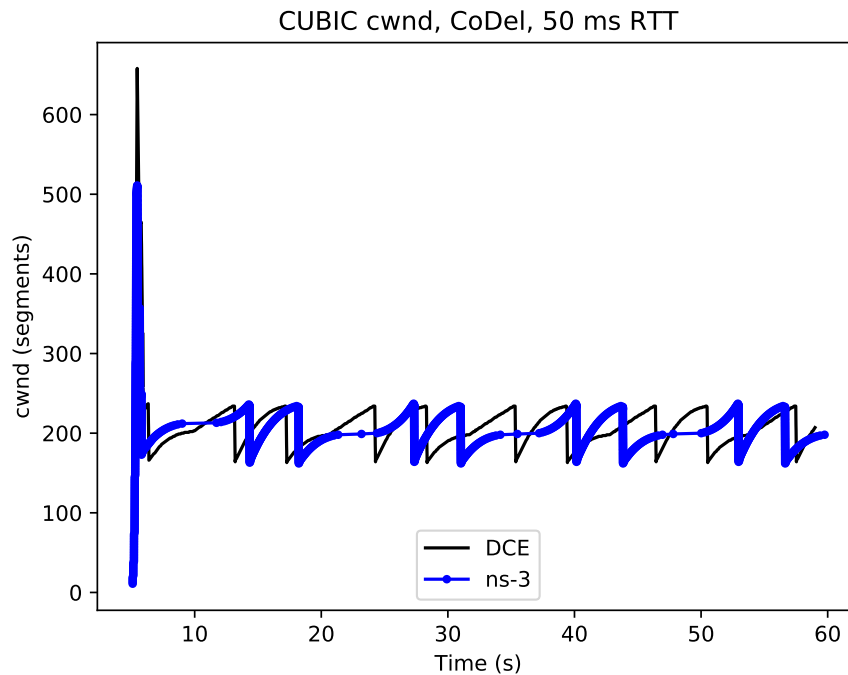


Fig. 16.11: CUBIC cwnd evolution for 50ms/50Mbps bottleneck, no ECN

TCP ECN operation is tested in the ARED and RED tests that are documented in the traffic-control module documentation.

Like DCTCP and TCP CUBIC, the ns-3 implementation of TCP BBR was validated against the BBR implementation of Linux kernel 5.4 using Network Stack Tester (NeST). NeST is a python package which allows the users to emulate kernel space protocols using Linux network namespaces. Figure *Congestion window evolution: ns-3 BBR vs. Linux BBR (using NeST)* compares the congestion window evolution between ns-3 and Linux for a single flow operating over a 10 Mbps link with 10 ms base RTT and FIFO queue discipline.

It can be observed that the congestion window traces for ns-3 BBR closely overlap with Linux BBR. The periodic drops in congestion window every 10 seconds depict the PROBE_RTT phase of the BBR algorithm. In this phase, BBR algorithm keeps the congestion window fixed to 4 segments.

The example program, `examples/tcp-bbr-example.cc` has been used to obtain the congestion window curve shown in Figure *Congestion window evolution: ns-3 BBR vs. Linux BBR (using NeST)*. The detailed instructions to reproduce ns-3 plot and NeST plot can be found at: <https://github.com/mohittahiliani/BBR-Validation>

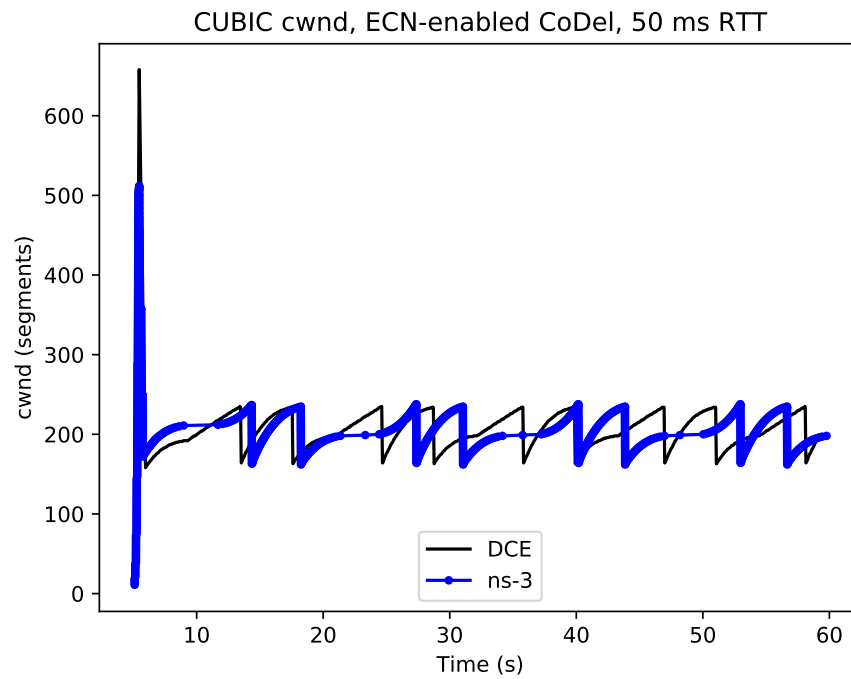


Fig. 16.12: CUBIC cwnd evolution for 50ms/50Mbps bottleneck, with ECN

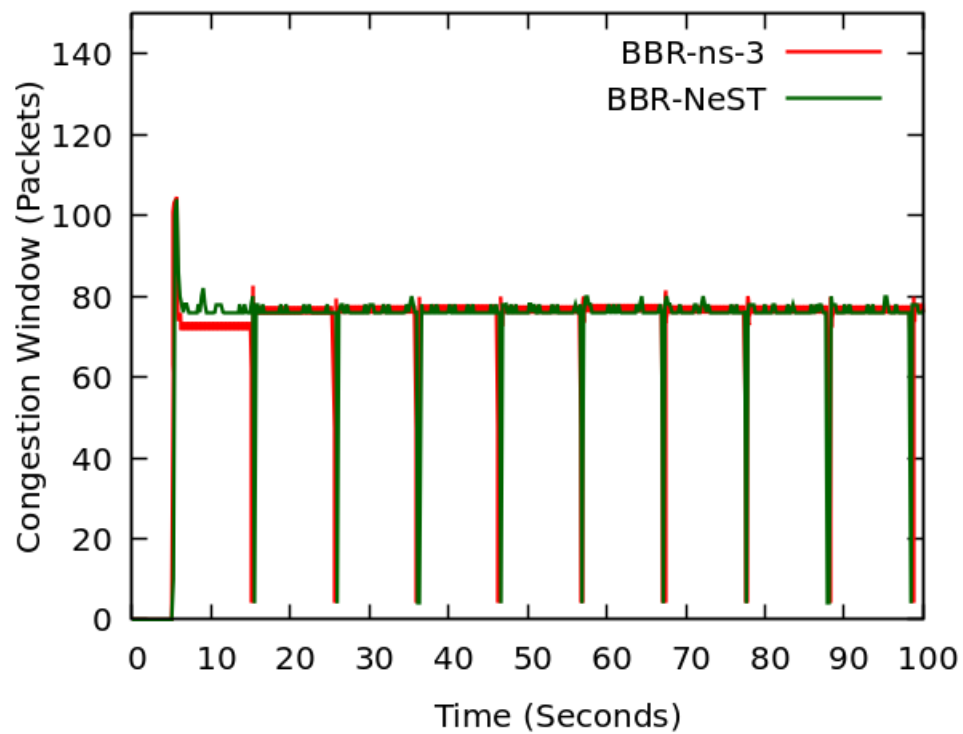


Fig. 16.13: Congestion window evolution: ns-3 BBR vs. Linux BBR (using NeST)

Writing a new congestion control algorithm

Writing (or porting) a congestion control algorithms from scratch (or from other systems) is a process completely separated from the internals of TcpSocketBase.

All operations that are delegated to a congestion control are contained in the class TcpCongestionOps. It mimics the structure tcp_congestion_ops of Linux, and the following operations are defined:

```
virtual std::string GetName() const;
virtual uint32_t GetSsThresh(Ptr<const TcpSocketState> tcb, uint32_t bytesInFlight);
virtual void IncreaseWindow(Ptr<TcpSocketState> tcb, uint32_t segmentsAacked);
virtual void PktsAacked(Ptr<TcpSocketState> tcb, uint32_t segmentsAacked, const Time& rtt);
virtual Ptr<TcpCongestionOps> Fork();
virtual void CwndEvent(Ptr<TcpSocketState> tcb, const TcpSocketState::TcpCaEvent_t_
    ↪ event);
```

The most interesting methods to write are GetSsThresh and IncreaseWindow. The latter is called when TcpSocketBase decides that it is time to increase the congestion window. Much information is available in the Transmission Control Block, and the method should increase cWnd and/or ssThresh based on the number of segments acked.

GetSsThresh is called whenever the socket needs an updated value of the slow start threshold. This happens after a loss; congestion control algorithms are then asked to lower such value, and to return it.

PktsAacked is used in case the algorithm needs timing information (such as RTT), and it is called each time an ACK is received.

CwndEvent is used in case the algorithm needs the state of socket during different congestion window event.

TCP SACK and non-SACK

To avoid code duplication and the effort of maintaining two different versions of the TCP core, namely RFC 6675 (TCP-SACK) and RFC 5681 (TCP congestion control), we have merged RFC 6675 in the current code base. If the receiver supports the option, the sender bases its retransmissions over the received SACK information. However, in the absence of that option, the best it can do is to follow the RFC 5681 specification (on Fast Retransmit/Recovery) and employing NewReno modifications in case of partial ACKs.

A similar concept is used in Linux with the function tcp_add_reno_sack. Our implementation resides in the TcpTxBuffer class that implements a scoreboard through two different lists of segments. TcpSocketBase actively uses the API provided by TcpTxBuffer to query the scoreboard; please refer to the Doxygen documentation (and to in-code comments) if you want to learn more about this implementation.

For an academic peer-reviewed paper on the SACK implementation in ns-3, please refer to <https://dl.acm.org/citation.cfm?id=3067666>.

Forward Acknowledgement (FACK)

FACK is designed to be used with the TCP SACK option. It keeps count of the total number of bytes of outstanding data in the network. It achieves this by using the additional information provided by TCP SACK.

FACK maintains two state variables: sndFack and retranData.

sndFack is updated to reflect the highest sequence number that has been selectively acknowledged. In non-recovery state, the sndFack variable is updated from the acknowledgment number in the TCP header whereas during the recovery state, the sender utilizes information contained in TCP SACK options to update sndFack.

retranData is the amount of outstanding retransmitted data in the network. Each time a segment is retransmitted, retranData is increased by the segment's size; when a retransmitted segment is determined to have left the network, retranData is decreased by the segment's size.

awnd variable is defined to be the data sender's estimate of the actual quantity of data outstanding in the network.

Assuming that all the unacknowledged data has left the network:

$$awnd = sndNxt - sndFack$$

sndNxt holds the sequence number of the first byte of unsent data.

In recovery state, data which is retransmitted must also be included in the calculation of awnd.

$$awnd = sndNxt - sndFack + retranData$$

Using this measure of outstanding data, the FACK algorithm can regulate the amount of data outstanding in the network to be within one MSS of the current value of cwnd.

In the FACK version, the cwnd adjustment and retransmission are also triggered when the receiver reports that the reassembly queue is longer than 3 segments:

```
if ((m_fackEnabled && fackDiff > m_tcb->m_segmentSize * 3) ||
    ((m_dupAckCount == m_retxThresh) &&
     (m_highRxAckMark >= m_recover || (!m_recoverActive))))
{
    EnterRecovery(currentDelivered);
    NS_ASSERT(m_tcb->m_congState == TcpSocketState::CA_RECOVERY);
}
```

By default the FACK option is disabled. To enable FACK, the following configuration can be used:

```
Config::SetDefault ("ns3::TcpSocketBase::Fack", BooleanValue (true));
```

Note that FACK requires SACK to be enabled as well:

```
Config::SetDefault ("ns3::TcpSocketBase::Sack", BooleanValue (true));
```

The following unit tests have been written to verify the implementation of FACK:

- This unit test creates a short packet flow and forces four consecutive lost segments, and verifies that the `snd.fack` variable is updated to the highest sequence number present in the incoming SACK blocks, and that its external calculation of `awnd` matches the internal state variable.

The test monitors the sender's ACK processing and performs two key checks:

1. `snd.fack`: It parses incoming SACK blocks to determine the highest acknowledged sequence number and verifies that the implementation updates `snd.fack` correctly.
2. `awnd`: It recomputes the in-flight data window using the FACK formula ($awnd = snd.nxt - snd.fack + retransmitted_data$) and confirms that this matches the implementation's internal value.

A successful test run ensures that the TCP FACK implementation accurately tracks the forward acknowledgment point and the amount of data in flight during recovery.

Example and Performance Verification

The example `examples/tcp/fack-example.cc` can be used to observe the FACK algorithm's influence on congestion control, specifically regarding the preservation of the "Self-Clock" and the "Overdamping" mechanism described in the Mathis & Mahdavi paper.

The example simulates a bottleneck link where an initial Slow Start phase results in a significant packet overshoot (buffer overflow).

Reproducing the Data:

To generate the comparison data, the simulation should be run twice:

1. **With FACK (Red Line):** Ensure `bool fack = true;` in the example code. Run the simulation and plot the `fackAwnd.dat` trace file using gnuplot. This file tracks the FACK-specific *awnd* variable.
2. **Without FACK (Blue Line):** Modify the code to set `bool fack = false;`. Run the simulation and plot the `bytesInFlight.dat` trace file using gnuplot. This file tracks the standard TCP *BytesInFlight* (SND.NXT - SND.UNA).

Plotting:

To generate the overlapping plot shown in **Figure 1**, you can use the following Gnuplot script. Save this as `plot_inflight.gp` and run it using `gnuplot plot_inflight.gp`.

```
set terminal pngcairo enhanced color lw 1.5 font 'Times Roman'
set xrange [0:10]
set yrange [0:60]
set output "inflight.png"

set xlabel "Time (sec)"
set ylabel "Inflight (Packets)"
set key top right vertical

plot \
    "bytesInFlight.dat" using 1:2 title "Without FACK" with lines lw 1.5 lc rgb "blue", \
    "fackAwnd.dat" using 1:2 title "With FACK" with lines lw 1.5 lc rgb "red"
```

Note: To generate the Congestion Window plot (**Figure 2**), use the same script logic but change the output filename to `cwnd.png`, update the y-label to "Cwnd (Packets)", and replace the input files in the plot command with the saved CWND traces (e.g., `cwnd_nofack.dat` and `cwnd_fack.dat`).

Interpretation of Results:

To fully appreciate the FACK algorithm's behavior, it is essential to analyze the relationship between the data outstanding in the network (*awnd*) and the sender's target window (*cwnd*). The following figures, generated from this example, illustrate the distinct phases of **Overshoot**, **Stall**, and **Overdamping**.

As shown in **Figure 1** and **Figure 2**, distinct behaviors are observed during the recovery phase (1.0s - 3.0s):

- **Blue Line (Without FACK):** The sharp drop in **Figure 1** indicates a loss of the TCP "Self-Clock." When losses occur, the standard algorithm cannot infer that packets have left the network, causing transmission to stall until the pipe drains.
- **Red Line (With FACK):** The *awnd* in **Figure 1** remains high (~50 packets), significantly exceeding the Congestion Window (*cwnd*, approx. 10 packets, visible in **Figure 2**) during this interval. This demonstrates the "Overdamping" mechanism (Section 4.4 of the paper):

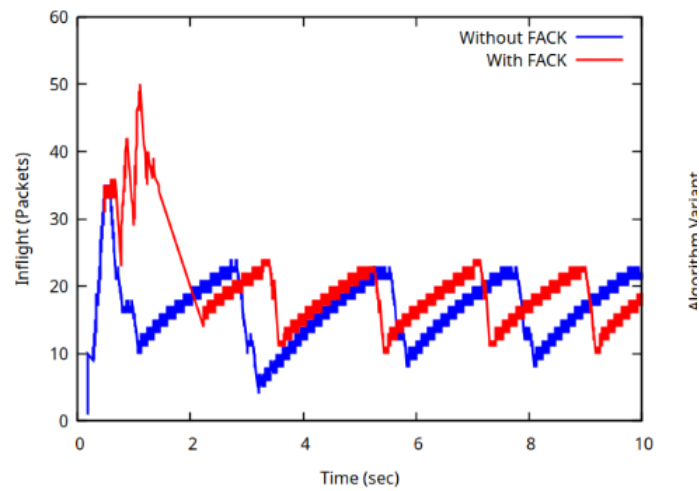


Fig. 16.14: **Figure 1:** Comparison of Inflight Data. The Blue line represents the standard TCP *BytesInFlight*. The Red line represents the FACK-specific *awnd* (Active Window), which accurately tracks the ~50 packet backlog caused by the Slow-Start overshoot.

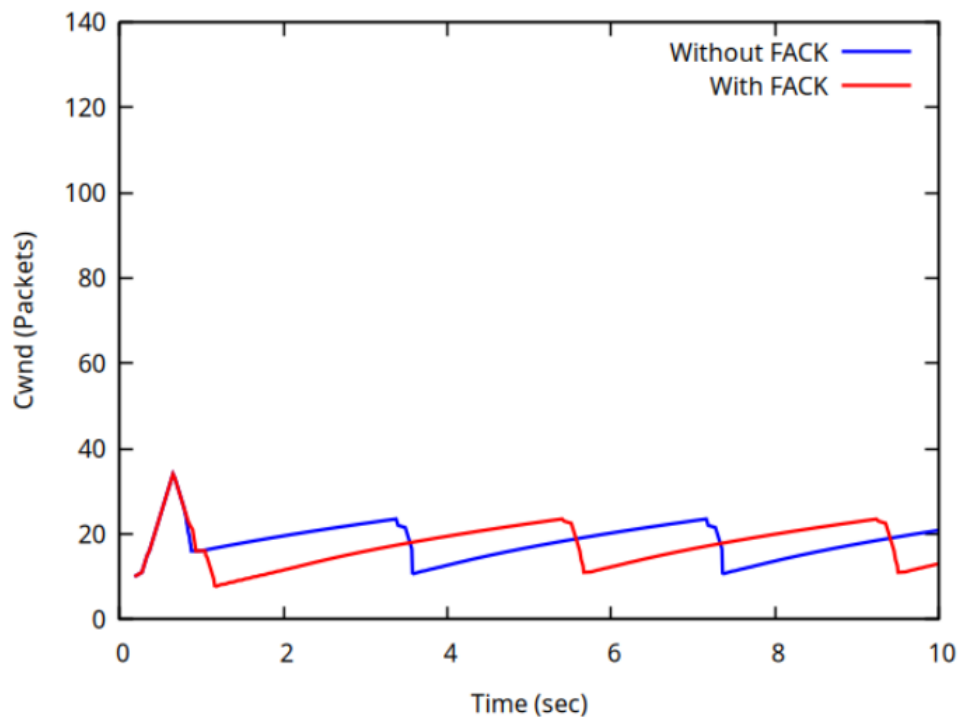


Fig. 16.15: **Figure 2:** Congestion Window (*cwnd*). Note the sharp reduction to ~10 packets at $t=1.0s$. This low target *cwnd*, combined with the high *awnd* in Figure 1, triggers the Overdamping mechanism.

1. **The Overshoot:** The initial Slow Start phase ramped up aggressively, pushing ~50 packets into the network before losses were detected (visible as the peak in the Red line).
2. **The Detection:** FACK's *awnd* correctly measured this ~50-packet backlog. Simultaneously, the congestion control algorithm cut the *cwnd* to ~10 packets in response to the losses. This behavior matches the FACK paper's statement:
"In the case when cwnd has been halved immediately following a lost segment, awnd will be significantly larger than cwnd".
3. **The Damping Action:** Because *awnd* was significantly larger than *cwnd*, the FACK algorithm inhibited new transmissions. The gradual downward slope of the Red line represents the queue draining as packets left the network. This behavior is governed by the core FACK transmission rule (*while awnd < cwnd*) defined in Section 3, which forces the sender to stop transmitting whenever the *awnd* exceeds the target window (*cwnd*).

This confirms that FACK successfully decouples data recovery from congestion control, maintaining an accurate picture of the network state even during heavy loss.

More information (paper): <https://dl.acm.org/citation.cfm?id=248181>

Loss Recovery Algorithms

The following loss recovery algorithms are supported in ns-3 TCP. The current default (as of ns-3.32 release) is Proportional Rate Reduction (PRR), while the default for ns-3.31 and earlier was Classic Recovery.

Classic Recovery

Classic Recovery refers to the combination of NewReno algorithm described in RFC 6582 along with SACK based loss recovery algorithm mentioned in RFC 6675. SACK based loss recovery is used when sender and receiver support SACK options. In the case when SACK options are disabled, the NewReno modification handles the recovery.

At the start of recovery phase the congestion window is reduced differently for NewReno and SACK based recovery. For NewReno the reduction is done as given below:

$$cWnd = ssThresh$$

For SACK based recovery, this is done as follows:

$$cWnd = ssThresh + (dupAckCount * segmentSize)$$

While in the recovery phase, the congestion window is inflated by *segmentSize* on arrival of every ACK when NewReno is used. The congestion window is kept same when SACK based loss recovery is used.

Proportional Rate Reduction

Proportional Rate Reduction (PRR) is a loss recovery algorithm described in RFC 6937 and currently used in Linux. The design of PRR helps in avoiding excess window adjustments and aims to keep the congestion window as close as possible to *ssThresh*.

PRR updates the congestion window by comparing the values of *bytesInFlight* and *ssThresh*. If the value of *bytesInFlight* is greater than *ssThresh*, congestion window is updated as shown below:

$$sndcnt = CEIL(prrDelivered * ssThresh / RecoverFS) - prrOut$$

$$cWnd = pipe + sndcnt$$

where `RecoverFS` is the value of `bytesInFlight` at the start of recovery phase, `prpDelivered` is the total bytes delivered during recovery phase, `prpOut` is the total bytes sent during recovery phase and `sndcnt` represents the number of bytes to be sent in response to each ACK.

Otherwise, the congestion window is updated by either using Conservative Reduction Bound (CRB) or Slow Start Reduction Bound (SSRB) with SSRB being the default Reduction Bound. Each Reduction Bound calculates a maximum data sending limit. For CRB, the limit is calculated as shown below:

$$limit = prpDelivered - prpOut$$

For SSRB, it is calculated as:

$$limit = MAX(prpDelivered - prpOut, DeliveredData) + MSS$$

where `DeliveredData` represents the total number of bytes delivered to the receiver as indicated by the current ACK and `MSS` is the maximum segment size.

After limit calculation, the `cWnd` is updated as given below:

$$sndcnt = MIN(ssThresh - pipe, limit)$$

$$cWnd = pipe + sndcnt$$

Thanks to Neal Cardwell for providing the following documentation of *ns-3* PRR model: The *ns-3* implementation of PRR has something like this (note that `m_isRetransDataAked` is true exactly when retransmitted data is cumulatively ACKed, and there is no RACK-TLP or similar mechanism to detect lost retransmits as of now):

```
// PRR-CRB by default
int limit = std::max(m_prrDelivered - m_prrOut, deliveredBytes);
// safeACK should be true iff ACK advances SND.UNA with no further loss indicated.
// We approximate that here (given the current lack of RACK-TLP in ns-3):
bool safeACK = tcb->m_isRetransDataAked; // retransmit cumulatively ACKed?
if (safeACK) // PRR-SSRB when recovery makes good progress
    limit += tcb->m_segmentSize;
// Attempt to catch up, as permitted
sendCount = std::min(limit, static_cast<int>(tcb->m_ssThresh - tcb->m_bytesInFlight));
}
// Force a fast retransmit upon entering fast recovery:
if (m_prrOut == 0 && sendCount == 0)
    sendCount = tcb->m_segmentSize;
```

which is close to the Linux implementation (which matches the RFC):

```
int delta = tp->snd_ssthresh - tcp_packets_in_flight(tp);
...
sndcnt = max_t(int, tp->prp_delivered - tp->prp_out,
               newly_acked_sacked);
if (flag & FLAG_SND_UNA_ADVANCED && !newly_lost)
    sndcnt++;
sndcnt = min(delta, sndcnt);
}
/* Force a fast retransmit upon entering fast recovery */
sndcnt = max(sndcnt, (tp->prp_out ? 0 : 1));
```

For reference, the Linux TCP PRR implementation is entirely contained in `tcp_init_cwnd_reduction()`, `tcp_cwnd_reduction()`, `tcp_end_cwnd_reduction()` and the updates elsewhere to `prp_out`.

More information (paper): <https://dl.acm.org/citation.cfm?id=2068832>

More information (RFC): <https://tools.ietf.org/html/rfc6937>

Alternative Backoff with ECN (ABE)

RFC 8511 recommends using an alternate `m_betaEcn` for calculation of the `ssthresh` when the congestion control algorithm detects congestion using ECN. There is an attribute `ns3::TcpSocketBase::UseAbe` which can be set to true to enable this feature. When enabled, the congestion control algorithm (NewReno or CUBIC) will use the `m_betaEcn` value to calculate the `ssthresh` when congestion is detected using ECN. The default value of `m_betaEcn` is set to 0.85 for CUBIC and 0.7 for NewReno as recommended by the RFC, but can be changed using the attribute `ns3::TcpNewReno::BetaEcn` for NewReno or using `ns3::TcpCubic::BetaEcn` for CUBIC.

For example, to enable ABE for NewReno, you can set the attribute as follows:

```
Config::SetDefault("ns3::TcpSocketBase::UseAbe", BooleanValue(true));
Config::SetDefault("ns3::TcpNewReno::BetaEcn", DoubleValue(0.7));
```

More information (RFC): <https://tools.ietf.org/html/rfc8511>

Adding a new loss recovery algorithm in ns-3

Writing (or porting) a loss recovery algorithms from scratch (or from other systems) is a process completely separated from the internals of `TcpSocketBase`.

All operations that are delegated to a loss recovery are contained in the class `TcpRecoveryOps` and are given below:

```
virtual std::string GetName() const;
virtual void EnterRecovery(Ptr<const TcpSocketState> tcb, uint32_t unAckDataCount,
                          bool isSackEnabled, uint32_t dupAckCount,
                          uint32_t bytesInFlight, uint32_t lastDeliveredBytes);
virtual void DoRecovery(Ptr<const TcpSocketState> tcb, uint32_t unAckDataCount,
                       bool isSackEnabled, uint32_t dupAckCount,
                       uint32_t bytesInFlight, uint32_t lastDeliveredBytes);
virtual void ExitRecovery(Ptr<TcpSocketState> tcb, uint32_t bytesInFlight);
virtual void UpdateBytesSent(uint32_t bytesSent);
virtual Ptr<TcpRecoveryOps> Fork();
```

`EnterRecovery` is called when packet loss is detected and recovery is triggered. While in recovery phase, each time when an ACK arrives, `DoRecovery` is called which performs the necessary congestion window changes as per the recovery algorithm. `ExitRecovery` is called just prior to exiting recovery phase in order to perform the required congestion window adjustments. `UpdateBytesSent` is used to keep track of bytes sent and is called whenever a data packet is sent during recovery phase.

Delivery Rate Estimation

Current TCP implementation measures the approximate value of the delivery rate of inflight data based on Delivery Rate Estimation.

As high level idea, keep in mind that the algorithm keeps track of 2 variables:

1. *delivered*: Total amount of data delivered so far.
2. *deliveredStamp*: Last time *delivered* was updated.

When a packet is transmitted, the value of *delivered* ($d0$) and *deliveredStamp* ($t0$) is stored in its respective `TcpTxItem`.

When an acknowledgement comes for this packet, the value of *delivered* and *deliveredStamp* is updated to $d1$ and $t1$ in the same `TcpTxItem`.

After processing the acknowledgement, the rate sample is calculated and then passed to a congestion avoidance algorithm:

$$delivery_{rate} = (d1 - d0)/(t1 - t0)$$

The implementation to estimate delivery rate is a joint work between `TcpTxBuffer` and `TcpRateOps`. For more information, please take a look at their Doxygen documentation.

The implementation follows the Internet draft (Delivery Rate Estimation): <https://tools.ietf.org/html/draft-cheng-iccr-g-delivery-rate-estimation-00>

Current limitations

- `TcpCongestionOps` interface does not contain every possible Linux operation

Writing TCP tests

The TCP subsystem supports automated test cases on both socket functions and congestion control algorithms. To show how to write tests for TCP, here we explain the process of creating a test case that reproduces the [Bug #1571](#).

The bug concerns the zero window situation, which happens when the receiver cannot handle more data. In this case, it advertises a zero window, which causes the sender to pause transmission and wait for the receiver to increase the window.

The sender has a timer to periodically check the receiver's window: however, in modern TCP implementations, when the receiver has freed a "significant" amount of data, the receiver itself sends an "active" window update, meaning that the transmission could be resumed. Nevertheless, the sender timer is still necessary because window updates can be lost.

Note: During the text, we will assume some knowledge about the general design of the TCP test infrastructure, which is explained in detail into the Doxygen documentation. As a brief summary, the strategy is to have a class that sets up a TCP connection, and that calls protected members of itself. In this way, subclasses can implement the necessary members, which will be called by the main `TcpGeneralTest` class when events occur. For example, after processing an ACK, the method `ProcessedAck` will be invoked. Subclasses interested in checking some particular things which must have happened during an ACK processing, should implement the `ProcessedAck` method and check the interesting values inside the method. To get a list of available methods, please check the Doxygen documentation.

We describe the writing of two test cases, covering both situations: the sender's zero-window probing and the receiver "active" window update. Our focus will be on dealing with the reported problems, which are:

- an ns-3 receiver does not send "active" window update when its receive buffer is being freed;
- even if the window update is artificially crafted, the transmission does not resume.

However, other things should be checked in the test:

- Persistent timer setup
- Persistent timer teardown if `rWnd` increases

To construct the test case, one first derives from the `TcpGeneralTest` class:

The code is the following:

```
TcpZeroWindowTest::TcpZeroWindowTest(const std::string &desc)
: TcpGeneralTest(desc)
{
}
```

Then, one should define the general parameters for the TCP connection, which will be one-sided (one node is acting as SENDER, while the other is acting as RECEIVER):

- Application packet size set to 500, and 20 packets in total (meaning a stream of 10k bytes)
- Segment size for both SENDER and RECEIVER set to 500 bytes
- Initial slow start threshold set to UINT32_MAX
- Initial congestion window for the SENDER set to 10 segments (5000 bytes)
- Congestion control: NewReno

We have also to define the link properties, because the above definition does not work for every combination of propagation delay and sender application behavior.

- Link one-way propagation delay: 50 ms
- Application packet generation interval: 10 ms
- Application starting time: 20 s after the starting point

To define the properties of the environment (e.g. properties which should be set before the object creation, such as propagation delay) one next implements the method ConfigureEnvironment:

```
void
TcpZeroWindowTest::ConfigureEnvironment()
{
    TcpGeneralTest::ConfigureEnvironment();
    SetAppPktCount(20);
    SetMTU(500);
    SetTransmitStart(Seconds(2));
    SetPropagationDelay(MilliSeconds(50));
}
```

For other properties, set after the object creation, one can use ConfigureProperties(). The difference is that some values, such as initial congestion window or initial slow start threshold, are applicable only to a single instance, not to every instance we have. Usually, methods that requires an id and a value are meant to be called inside ConfigureProperties(). Please see the Doxygen documentation for an exhaustive list of the tunable properties.

```
void
TcpZeroWindowTest::ConfigureProperties()
{
    TcpGeneralTest::ConfigureProperties();
    SetInitialCwnd(SENDER, 10);
}
```

To see the default value for the experiment, please see the implementation of both methods inside TcpGeneralTest class.

Note: If some configuration parameters are missing, add a method called “SetSomeValue” which takes as input the value only (if it is meant to be called inside ConfigureEnvironment) or the socket and the value (if it is meant to be called inside ConfigureProperties).

To define a zero-window situation, we choose (by design) to initiate the connection with a 0-byte rx buffer. This implies that the RECEIVER, in its first SYN-ACK, advertises a zero window. This can be accomplished by implementing the method `CreateReceiverSocket`, setting an Rx buffer value of 0 bytes (at line 6 of the following code):

```
1 Ptr<TcpSocketMsgBase>
2 TcpZeroWindowTest::CreateReceiverSocket(Ptr<Node> node)
3 {
4     Ptr<TcpSocketMsgBase> socket = TcpGeneralTest::CreateReceiverSocket(node);
5
6     socket->SetAttribute("RcvBufSize", UintegerValue(0));
7     Simulator::Schedule(Seconds(10),
8                         &TcpZeroWindowTest::IncreaseBufSize, this);
9
10    return socket;
11 }
```

Even so, to check the active window update, we should schedule an increase of the buffer size. We do this at line 7 and 8, scheduling the function `IncreaseBufSize`.

```
void
TcpZeroWindowTest::IncreaseBufSize()
{
    SetRcvBufSize(RECEIVER, 2500);
}
```

Which utilizes the `SetRcvBufSize` method to edit the `RxBuffer` object of the RECEIVER. As said before, check the Doxygen documentation for class `TcpGeneralTest` to be aware of the various possibilities that it offers.

Note: By design, we choose to maintain a close relationship between `TcpSocketBase` and `TcpGeneralTest`: they are connected by a friendship relation. Since friendship is not passed through inheritance, if one discovers that one needs to access or to modify a private (or protected) member of `TcpSocketBase`, one can do so by adding a method in the class `TcpGeneralSocket`. An example of such method is `SetRcvBufSize`, which allows `TcpGeneralSocket` subclasses to forcefully set the `RxBuffer` size.

```
void
TcpGeneralTest::SetRcvBufSize(SocketWho who, uint32_t size)
{
    if (who == SENDER)
    {
        m_senderSocket->SetRcvBufSize(size);
    }
    else if (who == RECEIVER)
    {
        m_receiverSocket->SetRcvBufSize(size);
    }
    else
    {
        NS_FATAL_ERROR("Not defined");
    }
}
```

Next, we can start to follow the TCP connection:

1. At time 0.0 s the connection is opened sender side, with a SYN packet sent from SENDER to RECEIVER

2. At time 0.05 s the RECEIVER gets the SYN and replies with a SYN-ACK
3. At time 0.10 s the SENDER gets the SYN-ACK and replies with a SYN.

While the general structure is defined, and the connection is started, we need to define a way to check the `rWnd` field on the segments. To this aim, we can implement the methods `Rx` and `Tx` in the `TcpGeneralTest` subclass, checking each time the actions of the RECEIVER and the SENDER. These methods are defined in `TcpGeneralTest`, and they are attached to the `Rx` and `Tx` traces in the `TcpSocketBase`. One should write small tests for every detail that one wants to ensure during the connection (it will prevent the test from changing over the time, and it ensures that the behavior will stay consistent through releases). We start by ensuring that the first SYN-ACK has 0 as advertised window size:

```
void
TcpZeroWindowTest::Tx(const Ptr<const Packet> p, const TcpHeader &h, SocketWho who)
{
    ...
    else if (who == RECEIVER)
    {
        NS_LOG_INFO("\tRECEIVER TX " << h << " size " << p->GetSize());

        if (h.GetFlags() & TcpHeader::SYN)
        {
            NS_TEST_ASSERT_MSG_EQ(h.GetWindowSize(),
                                   0,
                                   "RECEIVER window size is not 0 in the SYN-ACK");
        }
    }
    ...
}
```

Practically, we are checking that every SYN packet sent by the RECEIVER has the advertised window set to 0. The same thing is done also by checking, in the `Rx` method, that each SYN received by SENDER has the advertised window set to 0. Thanks to the log subsystem, we can print what is happening through messages. If we run the experiment, enabling the logging, we can see the following:

```
./ns3 shell
gdb --args ./build/utils/ns3-dev-test-runner-debug --test-name=tcp-zero-window-test --
→stop-on-failure --fullness=QUICK --assert-on-failure --verbose
(gdb) run

0.00s TcpZeroWindowTestSuite:Tx(): 0.00 SENDER TX 49153 > 4477 [SYN] Seq=0 Ack=0
→Win=32768 ns3::TcpOptionWinScale(2) ns3::TcpOptionTS(0;0) size 36
0.05s TcpZeroWindowTestSuite:Rx(): 0.05 RECEIVER RX 49153 > 4477 [SYN] Seq=0 Ack=0
→Win=32768 ns3::TcpOptionWinScale(2) ns3::TcpOptionTS(0;0) ns3::TcpOptionEnd(EOL) size 0
0.05s TcpZeroWindowTestSuite:Tx(): 0.05 RECEIVER TX 4477 > 49153 [SYN|ACK] Seq=0 Ack=1
→Win=0 ns3::TcpOptionWinScale(0) ns3::TcpOptionTS(50;0) size 36
0.10s TcpZeroWindowTestSuite:Rx(): 0.10 SENDER RX 4477 > 49153 [SYN|ACK] Seq=0 Ack=1
→Win=0 ns3::TcpOptionWinScale(0) ns3::TcpOptionTS(50;0) ns3::TcpOptionEnd(EOL) size 0
0.10s TcpZeroWindowTestSuite:Tx(): 0.10 SENDER TX 49153 > 4477 [ACK] Seq=1 Ack=1
→Win=32768 ns3::TcpOptionTS(100;50) size 32
0.15s TcpZeroWindowTestSuite:Rx(): 0.15 RECEIVER RX 49153 > 4477 [ACK] Seq=1 Ack=1
→Win=32768 ns3::TcpOptionTS(100;50) ns3::TcpOptionEnd(EOL) size 0
(...)
```

The output is cut to show the threeway handshake. As we can see from the headers, the `rWnd` of RECEIVER is set to 0, and thankfully our tests are not failing. Now we need to test for the persistent timer, which should be started by the

SENDER after it receives the SYN-ACK. Since the Rx method is called before any computation on the received packet, we should utilize another method, namely ProcessedAck, which is the method called after each processed ACK. In the following, we show how to check if the persistent event is running after the processing of the SYN-ACK:

```
void
TcpZeroWindowTest::ProcessedAck(const Ptr<const TcpSocketState> tcb,
                                const TcpHeader& h,
                                SocketWho who)
{
    if (who == SENDER)
    {
        if (h.GetFlags() & TcpHeader::SYN)
        {
            EventId persistentEvent = GetPersistentEvent(SENDER);
            NS_TEST_ASSERT_MSG_EQ(persistentEvent.IsPending(),
                                   true,
                                   "Persistent event not started");
        }
    }
}
```

Since we programmed the increase of the buffer size after 10 simulated seconds, we expect the persistent timer to fire before any rWnd changes. When it fires, the SENDER should send a window probe, and the receiver should reply reporting again a zero window situation. At first, we investigate on what the sender sends:

```
1  if (Simulator::Now().GetSeconds() <= 6.0)
2  {
3      NS_TEST_ASSERT_MSG_EQ(p->GetSize() - h.GetSerializedSize(),
4                             0,
5                             "Data packet sent anyway");
6  }
7  else if (Simulator::Now().GetSeconds() > 6.0 &&
8           Simulator::Now().GetSeconds() <= 7.0)
9  {
10     NS_TEST_ASSERT_MSG_EQ(m_zeroWindowProbe, false, "Sent another probe");
11
12     if (!m_zeroWindowProbe)
13     {
14         NS_TEST_ASSERT_MSG_EQ(p->GetSize() - h.GetSerializedSize(),
15                                1,
16                                "Data packet sent instead of window probe");
17         NS_TEST_ASSERT_MSG_EQ(h.GetSequenceNumber(),
18                                SequenceNumber32(1),
19                                "Data packet sent instead of window probe");
20         m_zeroWindowProbe = true;
21     }
22 }
```

We divide the events by simulated time. At line 1, we check everything that happens before the 6.0 seconds mark; for instance, that no data packets are sent, and that the state remains OPEN for both sender and receiver.

Since the persist timeout is initialized at 6 seconds (exercise left for the reader: edit the test, getting this value from the Attribute system), we need to check (line 6) between 6.0 and 7.0 simulated seconds that the probe is sent. Only one probe is allowed, and this is the reason for the check at line 11.

```

1  if (Simulator::Now().GetSeconds() > 6.0 &&
2      Simulator::Now().GetSeconds() <= 7.0)
3  {
4      NS_TEST_ASSERT_MSG_EQ(h.GetSequenceNumber(),
5                             SequenceNumber32(1),
6                             "Data packet sent instead of window probe");
7      NS_TEST_ASSERT_MSG_EQ(h.GetWindowSize(),
8                             0,
9                             "No zero window advertised by RECEIVER");
10 }

```

For the RECEIVER, the interval between 6 and 7 seconds is when the zero-window segment is sent.

Other checks are redundant; the safest approach is to deny any other packet exchange between the 7 and 10 seconds mark.

```

else if (Simulator::Now().GetSeconds() > 7.0 &&
         Simulator::Now().GetSeconds() < 10.0)
{
    NS_FATAL_ERROR("No packets should be sent before the window update");
}

```

The state checks are performed at the end of the methods, since they are valid in every condition:

```

NS_TEST_ASSERT_MSG_EQ(GetCongStateFrom(GetTcb(SENDER)),
                       TcpSocketState::CA_OPEN,
                       "Sender State is not OPEN");
NS_TEST_ASSERT_MSG_EQ(GetCongStateFrom(GetTcb(RECEIVER)),
                       TcpSocketState::CA_OPEN,
                       "Receiver State is not OPEN");

```

Now, the interesting part in the Tx method is to check that after the 10.0 seconds mark (when the RECEIVER sends the active window update) the value of the window should be greater than zero (and precisely, set to 2500):

```

else if (Simulator::Now().GetSeconds() >= 10.0)
{
    NS_TEST_ASSERT_MSG_EQ(h.GetWindowSize(),
                           2500,
                           "Receiver window not updated");
}

```

To be sure that the sender receives the window update, we can use the Rx method:

```

1  if (Simulator::Now().GetSeconds() >= 10.0)
2  {
3      NS_TEST_ASSERT_MSG_EQ(h.GetWindowSize(),
4                             2500,
5                             "Receiver window not updated");
6      m_windowUpdated = true;
7  }

```

We check every packet after the 10 seconds mark to see if it has the window updated. At line 5, we also set to true a boolean variable, to check that we effectively reach this test.

Last but not least, we implement also the NormalClose() method, to check that the connection ends with a success:

```
void
TcpZeroWindowTest::NormalClose(SocketWho who)
{
    if (who == SENDER)
    {
        m_senderFinished = true;
    }
    else if (who == RECEIVER)
    {
        m_receiverFinished = true;
    }
}
```

The method is called only if all bytes are transmitted successfully. Then, in the method `FinalChecks()`, we check all variables, which should be true (which indicates that we have perfectly closed the connection).

```
void
TcpZeroWindowTest::FinalChecks()
{
    NS_TEST_ASSERT_MSG_EQ(m_zeroWindowProbe,
                          true,
                          "Zero window probe not sent");
    NS_TEST_ASSERT_MSG_EQ(m_windowUpdated,
                          true,
                          "Window has not updated during the connection");
    NS_TEST_ASSERT_MSG_EQ(m_senderFinished,
                          true,
                          "Connection not closed successfully(SENDER)");
    NS_TEST_ASSERT_MSG_EQ(m_receiverFinished,
                          true,
                          "Connection not closed successfully(RECEIVER)");
}
```

To run the test, the usual way is

```
./test.py -s tcp-zero-window-test

PASS: TestSuite tcp-zero-window-test
1 of 1 tests passed (1 passed, 0 skipped, 0 failed, 0 crashed, 0 valgrind errors)
```

To see INFO messages, use a combination of `./ns3 shell` and `gdb` (really useful):

```
./ns3 shell && gdb --args ./build/utils/ns3-dev-test-runner-debug --test-name=tcp-zero-
window-test --stop-on-failure --fullness=QUICK --assert-on-failure --verbose
```

and then, hit “Run”.

Note: This code magically runs without any reported errors; however, in real cases, when you discover a bug you should expect the existing test to fail (this could indicate a well-written test and a bad-written model, or a bad-written test; hopefully the first situation). Correcting bugs is an iterative process. For instance, commits created to make this test case running without errors are 11633:6b74df04cf44, (others to be merged).

16.7 UDP model in ns-3

This chapter describes the UDP model available in *ns-3*.

16.7.1 Generic support for UDP

ns-3 supports a native implementation of UDP. It provides a connectionless, unreliable datagram packet service. Packets may be reordered or duplicated before they arrive. UDP calculates and checks checksums to catch transmission errors.

This implementation inherits from a few common header classes in the `src/network` directory, so that user code can swap out implementations with minimal changes to the scripts.

Here are the important abstract base classes:

- `class UdpSocket`: This is defined in: `src/internet/model/udp-socket.{cc,h}` This is an abstract base class of all UDP sockets. This class exists solely for hosting `UdpSocket` attributes that can be reused across different implementations, and for declaring UDP-specific multicast API.
- `class UdpSocketImpl`: This class subclasses `UdpSocket`, and provides a socket interface to *ns-3*'s implementation of UDP.
- `class UdpSocketFactory`: This is used by the layer-4 protocol instance to create UDP sockets.
- `class UdpSocketFactoryImpl`: This class is derived from `SocketFactory` and implements the API for creating UDP sockets.
- `class UdpHeader`: This class contains fields corresponding to those in a network UDP header (port numbers, payload size, checksum) as well as methods for serialization to and deserialization from a byte buffer.
- `class UdpL4Protocol`: This is a subclass of `IpL4Protocol` and provides an implementation of the UDP protocol.

16.7.2 ns-3 UDP

This is an implementation of the User Datagram Protocol described in RFC 768. UDP uses a simple connectionless communication model with a minimum of protocol mechanism. The implementation provides checksums for data integrity, and port numbers for addressing different functions at the source and destination of the datagram. It has no handshaking dialogues, and thus exposes the user's data to any unreliability of the underlying network. There is no guarantee of data delivery, ordering, or duplicate protection.

Usage

In many cases, usage of UDP is set at the application layer by telling the *ns-3* application which kind of socket factory to use.

Using the helper functions defined in `src/applications/helper`, here is how one would create a UDP receiver:

```
// Create a packet sink on the receiver
uint16_t port = 50000;
Address sinkLocalAddress(InetSocketAddress(Ipv4Address::GetAny(), port));
PacketSinkHelper sinkHelper("ns3::UdpSocketFactory", sinkLocalAddress);
ApplicationContainer sinkApp = sinkHelper.Install(serverNode);
sinkApp.Start(Seconds(1));
sinkApp.Stop(Seconds(10));
```

Similarly, the below snippet configures `OnOffApplication` traffic source to use UDP:

```
// Create the OnOff applications to send data to the UDP receiver
OnOffHelper clientHelper("ns3::UdpSocketFactory", Address());
clientHelper.SetAttribute("Remote", remoteAddress);
ApplicationContainer clientApps =(clientHelper.Install(clientNode);
clientApps.Start(Seconds(2));
clientApps.Stop(Seconds(9));
```

For users who wish to have a pointer to the actual socket(so that socket operations like Bind(), setting socket options, etc. can be done on a per-socket basis), UDP sockets can be created by using the Socket::CreateSocket() method as given below:

```
Ptr<Node> node = CreateObject<Node>();
InternetStackHelper internet;
internet.Install(node);

Ptr<SocketFactory> socketFactory = node->GetObject<UdpSocketFactory>();
Ptr<Socket> socket = socketFactory->CreateSocket();
socket->Bind(InetSocketAddress(Ipv4Address::GetAny(), 80));
```

Once a UDP socket is created, we do not need an explicit connection setup before sending and receiving data. Being a connectionless protocol, all we need to do is to create a socket and bind it to a known port. For a client, simply create a socket and start sending data. The Bind() call allows an application to specify a port number and an address on the local machine. It allocates a local IPv4 endpoint for this socket.

At the end of data transmission, the socket is closed using the Socket::Close(). It returns a 0 on success and -1 on failure.

Please note that applications usually create the sockets automatically. Please refer to the source code of your preferred application to discover how and when it creates the socket.

UDP Socket interaction and interface with Application layer

The following is the description of the public interface of the UDP socket, and how the interface is used to interact with the socket itself.

Socket APIs for UDP connections:

Connect()

This is called when Send() is used instead of SendTo() by the user. It sets the address of the remote endpoint which is used by Send(). If the remote address is valid, this method makes a callback to *ConnectionSucceeded*.

Bind()

Bind the socket to an address, or to a general endpoint. A general endpoint is an endpoint with an ephemeral port allocation (that is, a random port allocation) on the 0.0.0.0 IP address. For instance, in current applications, data senders usually bind automatically after a Connect() over a random port. Consequently, the connection will start from this random port towards the well-defined port of the receiver. The IP 0.0.0.0 is then translated by lower layers into the real IP of the device.

Bind6()

Same as Bind(), but for IPv6.

BindToNetDevice()

Bind the socket to the specified NetDevice. If set on a socket, this option will force packets to leave the bound device regardless of the device that IP routing would naturally choose. In the receive direction, only packets received from the bound interface will be delivered.

ShutdownSend()

Signals the termination of send, or in other words, prevents data from being added to the buffer.

Recv()

Grabs data from the UDP socket and forwards it to the application layer. If no data is present (i.e. `m_deliveryQueue.empty()` returns 0), an empty packet is returned.

RecvFrom()

Same as `Recv()`, but with the source address as parameter.

SendTo()

The `SendTo()` API is the UDP counterpart of the TCP API `Send()`. It additionally specifies the address to which the message is to be sent because no prior connection is established in UDP communication. It returns the number of bytes sent or -1 in case of failure.

Close()

The close API closes a socket and terminates the connection. This results in freeing all the data structures previously allocated.

Public callbacks

These callbacks are called by the UDP socket to notify the application of interesting events. We will refer to these with the protected name used in `socket.h`, but we will provide the API function to set the pointers to these callback as well.

NotifyConnectionSucceeded: SetConnectCallback, 1st argument

Called when the `Connect()` succeeds and the remote address is validated.

NotifyConnectionFailed: SetConnectCallback, 2nd argument

Called in `Connect()` when the the remote address validation fails.

NotifyDataSent: SetDataSentCallback

The socket notifies the application that some bytes have been transmitted at the IP layer. These bytes could still be lost in the node (traffic control layer) or in the network.

NotifySend: SetSendCallback

Invoked to get the space available in the tx buffer when a packet (that carries data) is sent.

NotifyDataRecv: SetRecvCallback

Called when the socket receives a packet (that carries data) in the receiver buffer.

Validation

The following test cases have been provided for UDP implementation in the `src/internet/test/udp-test.cc` file.

- **UdpSocketImplTest:** Checks data received via UDP Socket over IPv4.
- **UdpSocketLoopbackTest:** Checks data received via UDP Socket Loopback over IPv4.
- **Udp6SocketImplTest :** Checks data received via UDP Socket over IPv6.
- **Udp6SocketLoopbackTest :** Checks data received via UDP Socket Loopback over IPv6 Test.

Limitations

- UDP_CORK is presently not the part of this implementation.
- `NotifyNormalClose`, `NotifyErrorClose`, `NotifyConnectionRequest` and `NotifyNewConnectionCreated` socket API callbacks are not supported.

INTERNET APPLICATIONS MODULE DOCUMENTATION

The goal of this module is to hold all the Internet-specific applications, and most notably some very specific applications (e.g., ping) or daemons (e.g., radvd). Other non-Internet-specific applications such as packet generators are contained in other modules.

The source code for the new module lives in the directory `src/internet-apps`.

Each application has its own goals, limitations and scope, which are briefly explained in the following.

All the applications are extensively used in the top-level `examples` directories. The users are encouraged to check the scripts therein to have a clear overview of the various options and usage tricks.

17.1 Ping

The Ping application supports both IPv4 and IPv6 and replaces earlier *ns-3* implementations called `v4Ping` and `Ping6` that were address family dependent. Ping was introduced in the *ns-3.38* release cycle.

17.1.1 Model Description

This application behaves similarly to the Unix `ping` application, although with fewer options supported. Ping sends ICMP Echo Request messages to a remote address, and collects statistics and reports on the ICMP Echo Reply responses that are received. The application can be used to send ICMP echo requests to unicast, broadcast, and multicast IPv4 and IPv6 addresses. The application can produce a verbose output similar to the real application, and can also export statistics and results via trace sources. The following can be controlled via attributes of this class:

- Destination address
- Local address (sender address)
- Packet size (default 56 bytes)
- Packet interval (default 1 second)
- Timeout value (default 1 second)
- The count, or maximum number of packets to send
- Verbose mode

In practice, the real-world `ping` application behavior varies slightly depending on the operating system (Linux, macOS, Windows, etc.). Most implementations also support a very large number of options. The *ns-3* model is intended to handle the most common use cases of testing for reachability.

Design

The aim of *ns-3* Ping application is to mimic the built-in application found in most operating systems. In practice, ping is usually used to check reachability of a destination, but additional options have been added over time and the tool can be used in different ways to gather statistics about reachability and round trip times (RTT). Since *ns-3* is mainly used for performance studies and not for operational forensics, some options of real ping implementations may not be useful for simulations. However, the *ns-3* application can deliver output and RTT samples similar to how the real application operates.

Ping is usually installed on a source node and does not require any *ns-3* application installation on the destination node. Ping is an Application that can be started and stopped using the base class Application APIs.

Behavior

The behavior of real ping applications varies across operating systems. For example, on Linux, the first ICMP sequence number sent is one, while on macOS, the first sequence number is zero. The behavior when pinging non-existent hosts also can differ (Linux is quiet while macOS is verbose). Windows and other operating systems like Cisco routers also can behave slightly differently.

This implementation tries to generally follow the Linux behavior, except that it will print out a verbose ‘request timed out’ message when an echo request is sent and no reply arrives in a timely manner. The timeout value (time that ping waits for a response to return) defaults to one second, but once there are RTT samples available, the timeout is set to twice the observed RTT. In contrast to Linux (but aligned with macOS), the first sequence number sent is zero.

Scope and Limitations

ping implementations have a lot of command-line options. The *ns-3* implementation only supports a few of the most commonly-used options; patches to add additional options would be welcome.

At the present time, fragmentation (sending an ICMP Echo Request larger than the path MTU) is not handled correctly during Echo Response reassembly.

17.1.2 Usage

Users may create and install Ping applications on nodes on a one-by-one basis using `CreateObject` or by using the `PingHelper`. For `CreateObject`, the following can be used:

```
Ptr<Node> n = ...;
Ptr<Ping> ping = CreateObject<Ping> ();
// Configure ping as needed...
n->AddApplication (ping);
```

Users should be aware of how this application stops. For most *ns-3* applications, `StopApplication()` should be called before the simulation is stopped. If the `Count` attribute of this application is set to a positive integer, the application will stop (and a report will be printed) either when `Count` responses have been received or when `StopApplication()` is called, whichever comes first. If `Count` is zero, meaning infinite pings, then `StopApplication()` should be used to eventually stop the application and generate the report. If `StopApplication()` is called while a packet (echo request) is in-flight, the response cannot be received and the packet will be treated as lost in the report— real ping applications work this way as well. To avoid this, it is recommended to call `StopApplication()` at a time when an Echo Request or Echo Response packet is not expected to be in flight.

Helpers

The PingHelper supports the typical Install usage pattern in ns-3. The following sample code is from the program examples/tcp/tcp-validation.cc.

```
PingHelper pingHelper(Ipv4Address("192.168.1.2"));
pingHelper.SetAttribute("Interval", TimeValue(pingInterval));
pingHelper.SetAttribute("Size", UIntegerValue(pingSize));
pingHelper.SetAttribute("VerboseMode", EnumValue(Ping::VerboseMode::SILENT));
ApplicationContainer pingContainer = pingHelper.Install(pingServer);
Ptr<Ping> ping = pingContainer.Get(0)->GetObject<Ping>();
ping->TraceConnectWithoutContext("Rtt", MakeBoundCallback(&TracePingRtt, &pingOfStream));
pingContainer.Start(Seconds(1));
pingContainer.Stop(stopTime - Seconds(1));
```

The first statement sets the remote address (destination) for all application instances created with this helper. The second and third statements perform further configuration. The fourth statement configures the verbosity to be totally silent. The fifth statement is a typical Install() method that returns an ApplicationContainer (in this case, of size 1). The sixth and seventh statements fetch the application instance created and configure a trace sink (TracePingRtt) for the Rtt trace source. The eighth and ninth statements configure the start and stop time, respectively.

The helper is most useful when there are many similarly configured applications to install on a collection of nodes (a NodeContainer). When there is only one Ping application to configure in a program, or when the configuration between different instances is different, it may be more straightforward to directly create the Ping applications without the PingHelper.

Attributes

The following attributes can be configured:

- Destination: The IPv4 or IPv6 address of the machine we want to ping
- VerboseMode: Configure verbose, quiet, or silent output
- Interval: Time interval between sending each packet
- Size: The number of data bytes to be sent, before ICMP and IP headers are added
- Count: The maximum number of packets the application will send
- InterfaceAddress: Local address of the sender
- Timeout: Time to wait for response if no RTT samples are available

Output

If VerboseMode mode is set to VERBOSE, ping will output the results of ICMP Echo Reply responses to std::cout output stream. If the mode is set to QUIET, only the initial statement and summary are printed. If the mode is set to SILENT, no output will be printed to std::cout. These behavioral differences can be seen with the ping-example.cc as follows:

```
$ ./ns3 run --no-build 'ping-example --ns3::Ping::VerboseMode=Verbose'
$ ./ns3 run --no-build 'ping-example --ns3::Ping::VerboseMode=Quiet'
$ ./ns3 run --no-build 'ping-example --ns3::Ping::VerboseMode=Silent'
```

Additional output can be gathered by using the four trace sources provided by Ping:

- **Tx:** This trace executes when a new packet is sent, and returns the sequence number and full packet (including ICMP header).
- **Rtt:** Each time an ICMP echo reply is received, this trace is called and reports the sequence number and RTT.
- **Drop:** If an ICMP error is returned instead of an echo reply, the sequence number and reason for reported drop are returned.
- **Report:** When ping completes and exits, it prints output statistics to the terminal. These values are copied to a struct `PingReport` and returned in this trace source.

Example

A basic `ping-example.cc` program is provided to highlight the following usage. The topology has three nodes interconnected by two point-to-point links. Each link has 5 ms one-way delay, for a round-trip propagation delay of 20 ms. The transmission rate on each link is 100 Mbps. The routing between links is enabled by ns-3's NixVector routing.

By default, this program will send 5 pings from node A to node C. When using the default IPv6, the output will look like this:

The example program will also produce four pcap traces (one for each NetDevice in the scenario) that can be viewed using tcpdump or Wireshark.

Other program options include options to change the destination and source addresses, number of packets (count), packet size, interval, and whether to enable logging (if logging is enabled in the build). These program options will override any corresponding attribute settings.

Finally, the program has some code that can be enabled to selectively force packet drops to check such behavior.

17.1.3 Validation

The following test cases have been added for regression testing:

1. Unlimited pings, no losses, `StopApplication ()` with no packets in flight
2. Unlimited pings, no losses, `StopApplication ()` with one packet in flight
3. Test for operation of count attribute and exit time after all pings are received, for IPv4"
4. Test the operation of interval attribute, for IPv4
5. Test for behavior of pinging an unreachable host when the network does not send an ICMP unreachable message
6. Test pinging to IPv4 broadcast address and IPv6 all nodes multicast address
7. Test behavior of first reply lost in a count-limited configuration
8. Test behavior of second reply lost in a count-limited configuration
9. Test behavior of last reply lost in a count-limited configuration.

17.2 Radvd

This app mimics a “RADVD” daemon. I.e., the daemon responsible for IPv6 routers advertisements. All the IPv6 routers should have a RADVD daemon installed.

The configuration of the Radvd application mimics the one of the radvd Linux program.

17.3 DHCPv4

The *ns-3* implementation of Dynamic Host Configuration Protocol (DHCP) follows the specifications of [RFC 2131](#) and [RFC 2132](#).

The source code for DHCP is located in `src/internet-apps/model` and consists of the following 6 files:

- `dhcp-server.h`,
- `dhcp-server.cc`,
- `dhcp-client.h`,
- `dhcp-client.cc`,
- `dhcp-header.h` and
- `dhcp-header.cc`

17.3.1 Helpers

The following two files have been added to `src/internet-apps/helper` for DHCP:

- `dhcp-helper.h` and
- `dhcp-helper.cc`

17.3.2 Tests

The tests for DHCP can be found at `src/internet-apps/test/dhcp-test.cc`

17.3.3 Examples

The examples for DHCP can be found at `src/internet-apps/examples/dhcp-example.cc`

17.3.4 Scope and Limitations

The server should be provided with a network address, mask and a range of address for the pool. One client application can be installed on only one netdevice in a node, and can configure address for only that netdevice.

The following five basic DHCP messages are supported:

- DHCP DISCOVER
- DHCP OFFER
- DHCP REQUEST
- DHCP ACK

- DHCP NACK

Also, the following eight options of BootP are supported:

- 1 (Mask)
- 50 (Requested Address)
- 51 (Address Lease Time)
- 53 (DHCP message type)
- 54 (DHCP server identifier)
- 58 (Address renew time)
- 59 (Address rebind time)
- 255 (end)

The client identifier option (61) can be implemented in near future.

In the current implementation, a DHCP client can obtain IPv4 address dynamically from the DHCP server, and can renew it within a lease time period.

Multiple DHCP servers can be configured, but the implementation does not support the use of a DHCP Relay yet.

17.4 V4TraceRoute

Documentation is missing for this application.

17.5 DHCPv6

The *ns-3* implementation of Dynamic Host Configuration Protocol for IPv6 (DHCPv6) follows the specifications of [RFC 8415](#).

17.5.1 Model Description

The aim of the DHCPv6 application is to provide a method for dynamically assigning IPv6 addresses to nodes. This application functions similarly to the `dhcpcd` daemon in Linux, although it supports fewer options. The DHCPv6 client installed on a node sends Solicit messages to a server configured on the link, which then responds with an IPv6 address offer.

The source code for DHCPv6 is located in `src/internet-apps/model`.

There are two major operational models for DHCPv6: stateful and stateless. Stateless DHCP is used when the client only needs to request configuration parameters from the server.

Typically, the DHCPv6 server is used to assign both addresses and other options like DNS information. The server maintains information about the Identity Associations (IAs), which are collections of leases assigned to a client.

There are three types of IAs: IA_NA (non-temporary addresses), IA_TA (temporary addresses), and IA_PD (prefix delegation). Temporary addresses were introduced in RFC 4941 to address privacy concerns. In DHCPv6, the IA_TA (Identity Association for Temporary Addresses) is used for addresses that are only used for a short time, and their life-times are generally not extended. Non-temporary addresses, on the other hand, are used for relatively longer durations and are usually renewed by the clients. Note that non-temporary addresses are not permanent leases - the term is merely used to differentiate between the IA_TA and IA_NA types.

Currently, the application uses only IA_NA options to lease addresses to clients. The state of the lease changes over time, which is why this is known as stateful DHCPv6.

The DHCPv6 client application is installed on each NetDevice (interface) of the client node. Therefore, if there are two NetDevices installed on a single node, two client applications must be installed. The DHCPv6 server application is installed on a single server node. However, the user can specify multiple interfaces for the server to listen on.

Notes:

- Stateless DHCPv6 has not been implemented, as it is used to request DNS configuration information that is not currently used in *ns-3*.
- All client and server nodes must be on the same link. The application does not support DHCPv6 relays and hence cannot work when the client and server are in entirely different subnets.

Stateful DHCPv6

The server listens on port 547 for any incoming messages on the link. Upon receiving a **Solicit** message from a client, the server sends an **Advertise** message with the available address pool. The server is designed to advertise one available address from each address pool that is configured on it. When the client sends a **Request** message, the server updates lease bindings with the new expiry time, and sends a **Reply** message.

The client attempts to accept all addresses present in the server's **Reply**. However, before using the offered address, the client checks whether any other node on the link is using the same address (Duplicate Address Detection). If the address is already in use, the client sends a **Decline** message to the server, which updates the lease bindings accordingly.

At the end of the preferred lifetime, the client sends a **Renew** message to the server. The server should update the lease bindings with the new expiry time, and send a **Reply** with the corresponding status code. In case the client does not receive a **Reply** from the server, it sends a **Rebind** message at the end of the valid lifetime. The server is expected to update the lease bindings and send a **Reply** message back to the client.

The order of the message exchange between the client and the server is as follows:

The formats of the messages between a client and server are as follows:

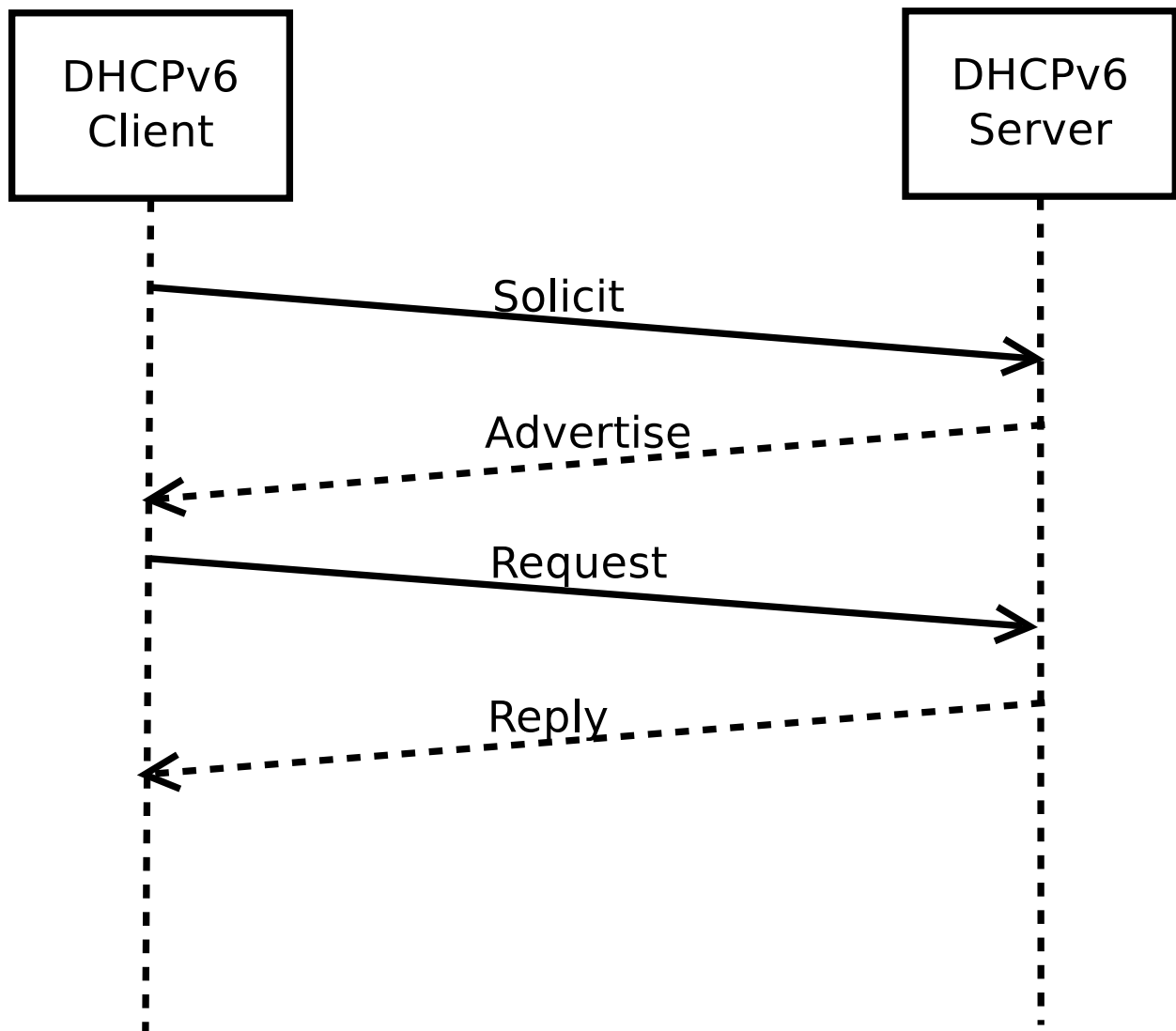
Identifiers

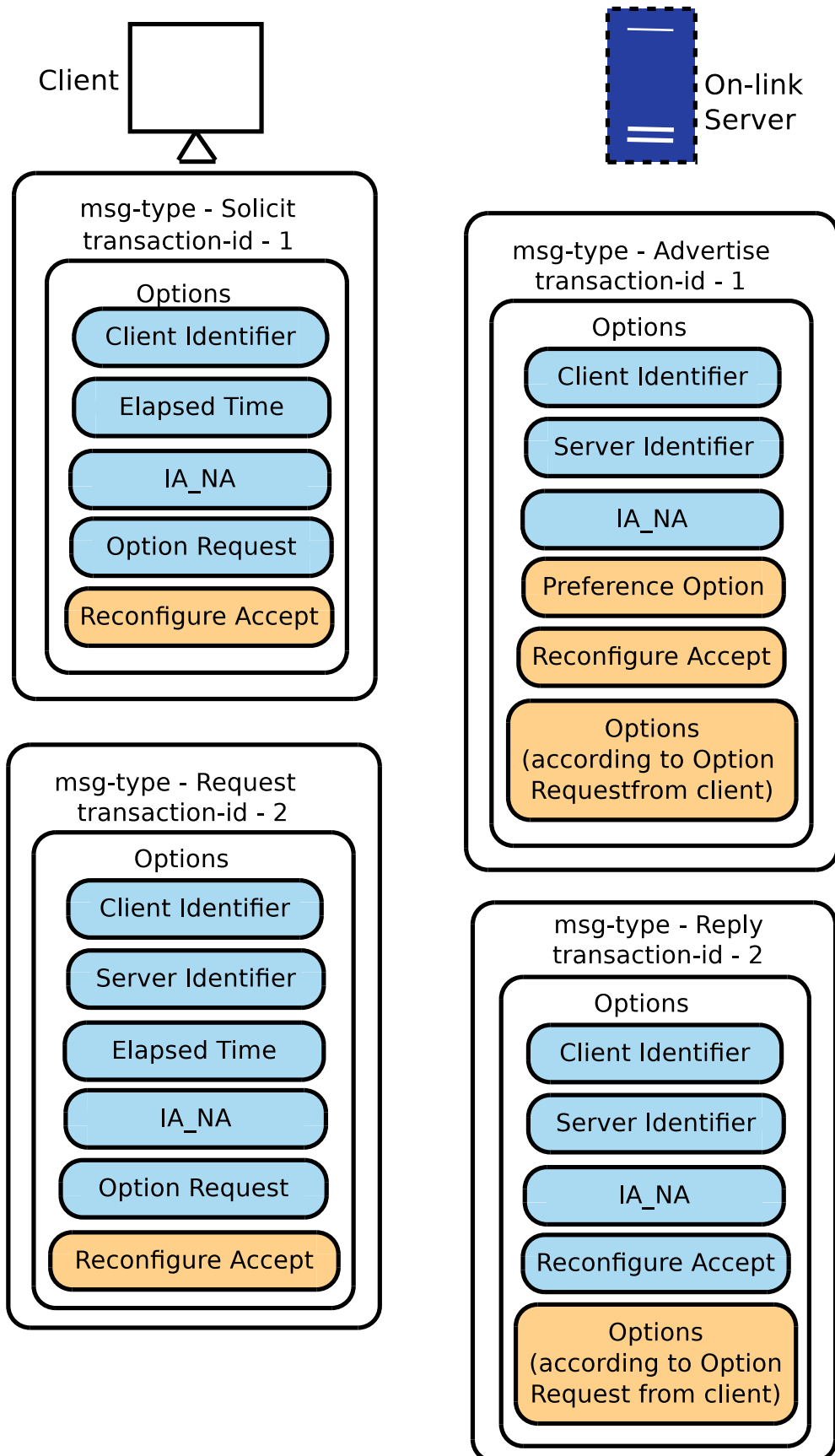
Each client and server has a unique identifier (DHCP Unique Identifier), which is used to identify the client and server in the message exchange. There are four types of DUIDs that can be used in DHCPv6:

1. **DUID-LLT (Link-layer Address plus Time)**: The DUID-LLT consists of a 32-bit timestamp and a variable-length link layer address.
2. **DUID-EN (Vendor-assigned unique ID based on Enterprise Number)**: This consists of the vendor's enterprise number and an identifier whose source is defined by the vendor.
3. **DUID-LL (Link-layer address)**: DUID-LL consists of a variable-length link-layer address as the identifier.
4. **DUID-UUID (Universally unique identifier)**: This should be an identifier that is persistent across system restarts and reconfigurations.

In the current implementation, the DUID is created as follows:

- DUID-LLT and DUID-LL: the identifier is created by selecting the NetDevices with the longest MAC address length, and among them, the one with the smallest index.
- DUID-LLT: the creation time is taken from the simulation time.
- DUID-EN: the Enterprise number is 0xf00dcafe (arbitrary number), and the identifier is based on the Node ID.
- DUID-UUID: the UUID is based on the Node ID.





17.5.2 Usage

The main way for *ns-3* users to use the DHCPv6 application is through the helper API and the publicly visible attributes of the client and server applications.

To use the DHCPv6 application, the user must install the client application on the node by passing a `Ptr<Node>` to the `InstallDhcp6Client` helper.

The server application is installed on the node of interest. Users select the interfaces on which the server should listen for incoming messages and add them to a `NetDeviceContainer`. This is then passed to the `InstallDhcp6Server` helper method.

Helpers

The `Dhcp6Helper` supports the typical `Install` usage pattern in *ns-3*. It can be used to easily install DHCPv6 server and client applications on a set of interfaces.

```
NetDeviceContainer serverNetDevices;  
serverNetDevices.Add(netdevice);  
ApplicationContainer dhcpServerApp = dhcp6Helper.InstallDhcp6Server(serverNetDevices);  
  
ApplicationContainer dhcpClientApp = dhcp6Helper.InstallDhcp6Client(clientNode);
```

Address pools can be configured on the DHCPv6 server using the following API:

```
Ptr<Dhcp6Server> server = DynamicCast<Dhcp6Server>(dhcpServerApp.Get(0));  
server->AddSubnet(Ipv6Address("2001:db8::"), Ipv6Prefix(64), Ipv6Address("2001:db8::1"),  
↳ Ipv6Address("2001:db8::ff"));
```

In the line above, the `AddSubnet()` method has the following parameters:

1. `Ipv6Address("2001:db8::")` - The address pool that is managed by the server.
2. `Ipv6Prefix(64)` - The prefix of the address pool
3. `Ipv6Address("2001:db8::1")` - The minimum address that can be assigned to a client.
4. `Ipv6Address("2001:db8::ff")` - The maximum address that can be assigned to a client.

Essentially, parameters 1 and 2 define the subnet(s) managed by the server, while parameters 3 and 4 define the range of addresses that can be assigned. If this method is not called, the server will not have any subnet configured and will not be able to assign addresses to clients. While it does not throw an error, a user who does not configure any subnets on the server will not see any addresses leased to the client.

Attributes

The following attributes can be configured on the client:

- `Transactions`: A random variable used to set the transaction numbers.
- `SolicitJitter`: The jitter in milliseconds that a node waits before sending a `Solicit` to the server.
- `IdaiValue`: The identifier of a new Identity Association that is created by a client.

The following values can be initially set on the client interface before stateful DHCPv6 begins. However, they are overridden with values received from the server during the message exchange: * `RenewTime`: The time after which client should renew its lease. * `RebindTime`: The time after which client should rebind its leased addresses. *

PreferredLifetime: The preferred lifetime of the leased address. * **ValidLifetime:** Time after which client should release the address.

The following attributes can be configured on the server: * **RenewTime:** The time after which client should renew its lease. * **RebindTime:** The time after which client should rebind its leased addresses. * **PreferredLifetime:** The preferred lifetime of the leased address. * **ValidLifetime:** The time after which client should release the address.

Example

The following example has been written in `src/internet-apps/examples/`:

- `dhcp6-example.cc`: Demonstrates the working of stateful DHCPv6 with a single server and 2 client nodes.

17.5.3 Test

The following example has been written in `src/internet-apps/test/`:

- `dhcp6-test.cc`: Tests the working of DHCPv6 with a single server and 2 client nodes that have 1 CSMA interface and 1 Wifi interface each.

17.5.4 Scope and Limitations

- Limited options have been included in the DHCPv6 implementation, namely:
 - Server Identifier
 - Client Identifier
 - Identity Association for Non-temporary Addresses (IA_NA)
 - Option Request (for Solicit MAX_RT)
 - Status Code
- The implementation does not support the use of a DHCP Relay agent. Hence, all the server and client nodes should be on the same link.
- The application does not yet support prefix delegation. Each client currently receives only one IPv6 address.
- If a DUID-LLT (Link-Layer Address plus Time) is used as the client identifier, there is a possibility that the timestamps of two clients may be the same. This could lead to a conflict in the lease bindings on the server if the link-layer addresses are not unique.
- If a node (such as a Wifi device) leaves the network and rejoins at a later time, the Solicit messages are not automatically restarted.
- The client application does not retransmit lost messages. Its use in wireless scenarios might lead to inconsistencies.

17.5.5 Future Work

- The Rapid Commit option may be implemented to allow a Solicit / Reply message exchange between the client and server.
- Addition of DHCPv6 relays to allow the client and server to be in different subnets.
- Implementation of stateless DHCPv6 to allow the client to request only configuration information from the server.
- Implement the ability to configure DHCPv6 through a configuration file, similar to how it is done in Linux systems.
- Allow users to manually set the DUIDs for the client(s) and server(s).
- Implement support for host reservations, allowing specific IPv6 addresses to be assigned to particular clients based on their DUIDs or other identifying information.
- Include the Preference Option in the Advertise message sent by the server. This option allows the client to identify and choose a single server based on the preference value, instead of obtaining leases from each server that responds to the Solicit message

17.5.6 References

- **RFC 8415** - Dynamic Host Configuration Protocol for IPv6 (DHCPv6)
- Infoblox Blog <<https://blogs.infoblox.com/ipv6-coe/slaac-to-basics-part-2-of-2-configuring-slaac/> > to understand how SLAAC and DHCPv6 operate at the same time.

IEEE 802.15.4: LOW-RATE WIRELESS PERSONAL AREA NETWORK (LR-WPAN)

This chapter describes the implementation of *ns-3* models for the low-rate, wireless personal area network (LR-WPAN) as specified by IEEE standard 802.15.4 (2003,2006,2011 and a few features of 2015). The current emphasis is on direct transmissions running on both, slotted and unslotted mode (CSMA/CA) of IEEE 802.15.4 operation for use in Zigbee (TM) and 6LoWPAN networks.

Both beacon and non-beacon modes are supported as well as the bootstrap mechanism (scan and association).

In general, this document describes what is modeled and how it is modeled; the following section Scope and Limitations provides more details about what is not covered or what is missing in the model.

The model is implemented into the `src/lrwpan/` folder.

The model design closely follows the standard from an architectural standpoint.

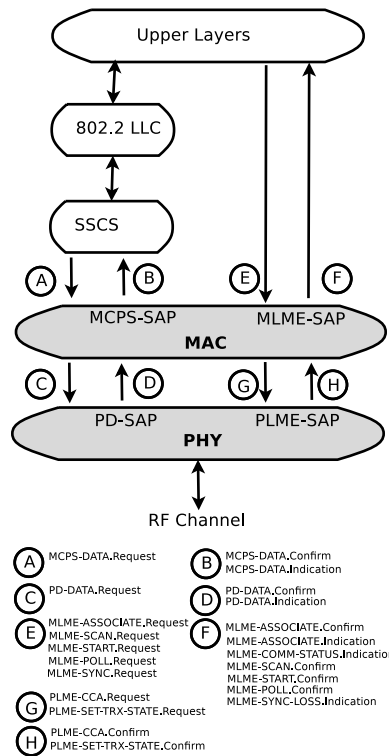


Fig. 18.1: Architecture and scope of lr-wpan models

The grey areas in the figure (adapted from Fig 3. of IEEE Std. 802.15.4-2006) show the scope of the model.

The Spectrum NetDevice from Nicola Baldo is the basis for the implementation.

The implementation also borrows some ideas from the ns-2 models developed by Zheng and Lee.

The APIs closely follow the standard, adapted for *ns-3* naming conventions and idioms. The APIs are organized around the concept of service primitives as shown in the following figure adapted from Figure 14 of IEEE Std. 802.15.4-2006.

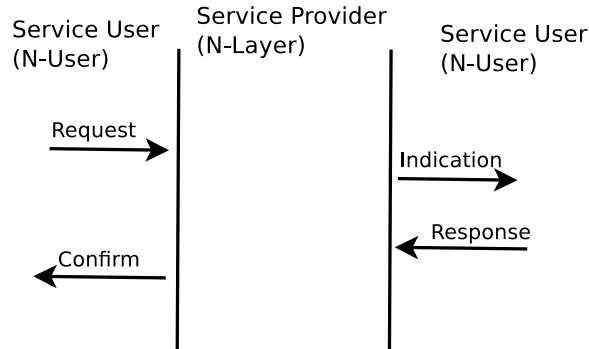


Fig. 18.2: Service primitives

The APIs primitives are organized around four conceptual services and service access points (SAP):

- MAC data service (MCPS)
- MAC management service (MLME)
- PHY data service (PD)
- PHY management service (PLME)

In general, primitives are standardized as follows (e.g. Sec 7.1.1.1.1 of IEEE 802.15.4-2006)::

```
MCPS-DATA.request(
```

```

    SrcAddrMode,
    DstAddrMode,
    DstPANId,
    DstAddr,
    msduLength,
    msdu,
    msduHandle,
    TxOptions,
    SecurityLevel,
    KeyIdMode,
    KeySource,
    KeyIndex
)
```

In *ns-3* this maps to classes, structs and methods such as::

```

struct McpsDataRequestParameters
{
    uint8_t m_srcAddrMode;
    uint8_t m_dstAddrMode;
    ...
};
```

(continues on next page)

(continued from previous page)

```

void
LrWpanMac::McpsDataRequest(McpsDataRequestParameters params)
{
    ...
}

```

The MAC primitives currently supported by the *ns-3* model are:

- MCPS-DATA (Request, Confirm, Indication)
- MLME-START (Request, Confirm)
- MLME-SCAN (Request, Confirm)
- MLME-BEACON-NOFIFY (Indication)
- MLME-ASSOCIATE.Request (Request, Confirm, Response, Indication)
- MLME-POLL (Confirm)
- MLME-COMM-STATUS (Indication)
- MLME-SYNC (Request)
- MLME-SYNC-LOSS (Indication)
- MLME-SET (Request, Confirm)
- MLME-GET (Request, Confirm)

The PHY primitives currently supported by the *ns-3* model are:

- PLME-CCA (Request, Confirm)
- PD-DATA (Request, Confirm, Indication)
- PLME-SET-TRX-STATE (Request, Confirm)
- PLME-SET (Request, Confirm)
- PLME-GET (Request, Confirm)

For more information on primitives, See IEEE 802.15.4-2011, Table 8.

In addition, this module provides a configuration for connecting various components. The current design of the `LrWpanNetDevice` allows for the configuration of different instances of `LrWpanMac`, which can only be set before the initialization of the `NetDevice`. This design can be extended in the future to support other subclasses of `LrWpanMacBase`, enabling the configuration of different types of MAC layers using the same `LrWpanNetDevice`.

The `LrWpanNetDevice` presented in this module is intended to be used with other technology profiles, such as the 6LoWPAN and ZigBee stack.

18.1 Scope and Limitations

- Indirect data transmissions are not supported but planned for a future update.
- Interference is modeled as AWGN but this is currently not thoroughly tested.
- The standard describes the support of multiple PHY band-modulations but currently, only 250kbps O-QPSK (channel page 0) is supported.
- Active and passive MAC scans are able to obtain a LQI value from a beacon frame, however, the scan primitives assumes LQI is correctly implemented and does not check the validity of its value.
- Configuration of the ED thresholds are currently not supported.
- Coordinator realignment command is only supported in orphan scans.
- Disassociation primitives are not supported.
- Security is not supported.
- Guaranteed Time Slots (GTS) are not supported.
- Not all attributes are supported by the MLME-SET and MLME-GET primitives.
- Indirect transmissions are only supported during the association process.
- PHY and MAC are currently not supported by the attribute system. To change the behavior of the PHY and MAC the standard SET primitives (e.g. MLME-SET.request) must be used.
- No radio energy model is supported.
- The PHY does not include sleep state.

18.2 The PHY layer

The physical layer components consist of a PHY model, an error rate model, and a loss model. The PHY state transitions are roughly model after ATMEL's AT86RF233.

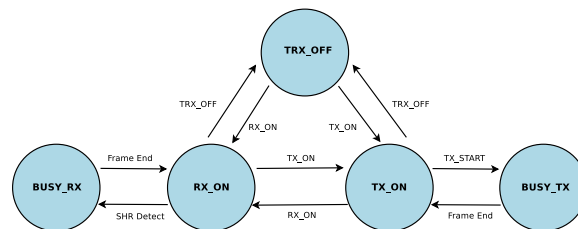


Fig. 18.3: Ns-3 Ir-wpan PHY basic operating mode state diagram

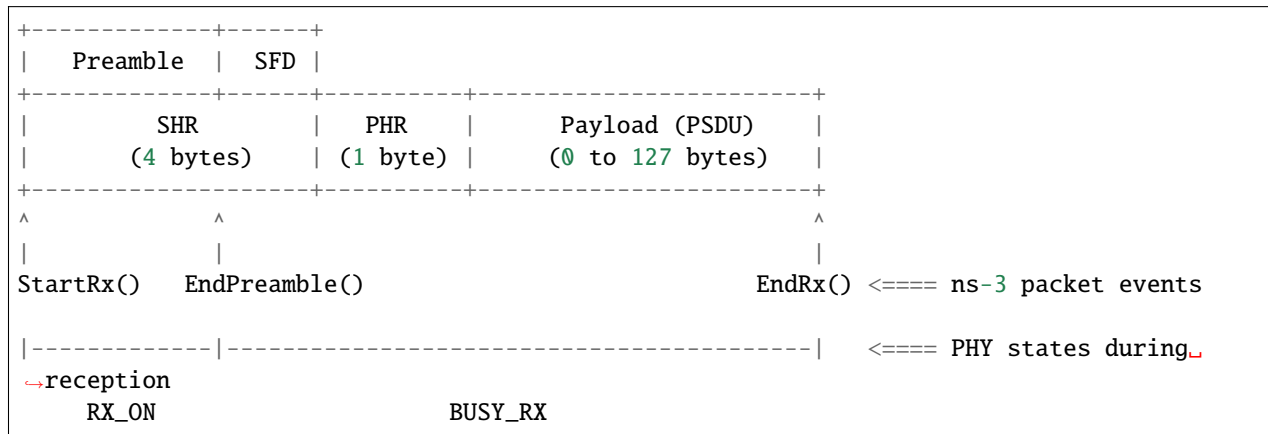
The error rate model presently models the error rate for IEEE 802.15.4 2.4 GHz AWGN channel for OQPSK; the model description can be found in IEEE Std 802.15.4-2006, section E.4.1.7. The Phy model is based on SpectrumPhy and it follows specification described in section 6 of IEEE Std 802.15.4-2006. It models PHY service specifications, PPDU formats, PHY constants and PIB attributes. It currently only supports the transmit power spectral density mask specified in 2.4 GHz per section 6.5.3.1. The noise power density assumes uniformly distributed thermal noise across the frequency bands. The loss model can fully utilize all existing simple (non-spectrum phy) loss models. The Phy model uses the existing single spectrum channel model. The physical layer is modeled at packet level, meaning there are no I/Q samples, chips, correlators or real bit decoding. Reception is modeled as follows: The physical layer is modeled at packet level, meaning that no I/Q samples, chips, correlators, or explicit bit decoding are simulated. Reception is

modeled as follows. The signal-to-interference-plus-noise ratio (SINR) is computed from the received signal power, the thermal noise floor, and the aggregate interference from other ongoing transmissions in the channel.

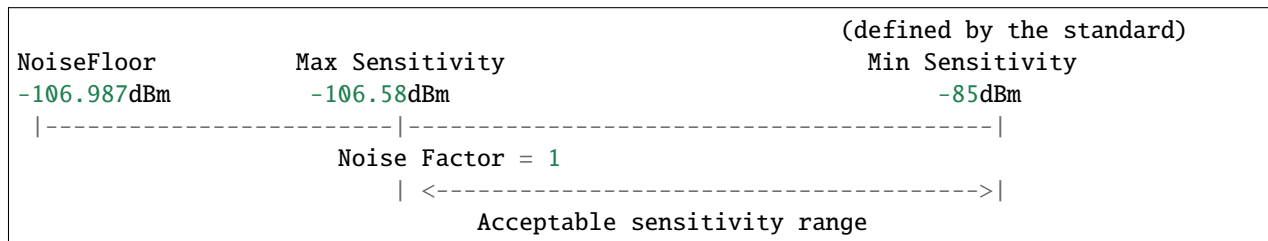
At the end of the preamble, a preliminary SINR check is performed to determine whether the packet is decodable. If the SINR is greater than -5 dB (IEEE Std 802.15.4-2006, Appendix E, Figure E.2), the receiver locks onto the packet and continues reception. Otherwise, the packet is considered undecodable, dropped, and treated as interference for other ongoing receptions.

For packets that pass the preamble check, interference is tracked over the duration of the packet. At the end of reception, the error rate model is applied using the accumulated SINR and packet parameters to probabilistically determine whether the packet is successfully received or corrupted. If the SNR is below the sensitivity threshold, the reception is labeled as failed (i.e., cannot be heard by the radio).

The following figure shows the PPDU format and the relevant events and states during packet reception.



Rx sensitivity is defined as the weakest possible signal point at which a receiver can receive and decode a packet with a high success rate. According to the standard (IEEE Std 802.15.4-2006, section 6.1.7), this corresponds to the point where the packet error rate is under 1% for 20 bytes PSDU reference packets (11 bytes MAC header + 7 bytes payload (MSDU) + FCS 2 bytes). Setting low Rx sensitivity values (increasing the radio hearing capabilities) have the effect to receive more packets (and at a greater distance) but it raises the probability to have dropped packets at the MAC layer or the probability of corrupted packets. By default, the receiver sensitivity is set to the maximum theoretical possible value of -106.58 dBm for the supported IEEE 802.15.4 O-QPSK 250kps. This rx sensitivity is set for the “perfect radio” which only considers the floor noise, in essence, this does not include the noise factor (noise introduced by imperfections in the demodulator chip or external factors). The receiver sensitivity can be changed to different values using SetRxSensitivity function in the PHY to simulate the hearing capabilities of different compliant radio transceivers (the standard minimum compliant Rx sensitivity is -85 dBm):



The example `1r-wpan-per-plot.cc` shows that at given Rx sensitivity, packets are dropped regardless of their theoretical error probability. This program outputs a file named `802.15.4-per-vs-rxSignal.plt`. Loading this file into gnuplot yields a file `802.15.4-per-vs-rsSignal.eps`, which can be converted to pdf or other formats. Packet payload size, Tx power and Rx sensitivity can be configured. The point where the blue line crosses with the PER indicates the Rx sensitivity. The default output is shown below.

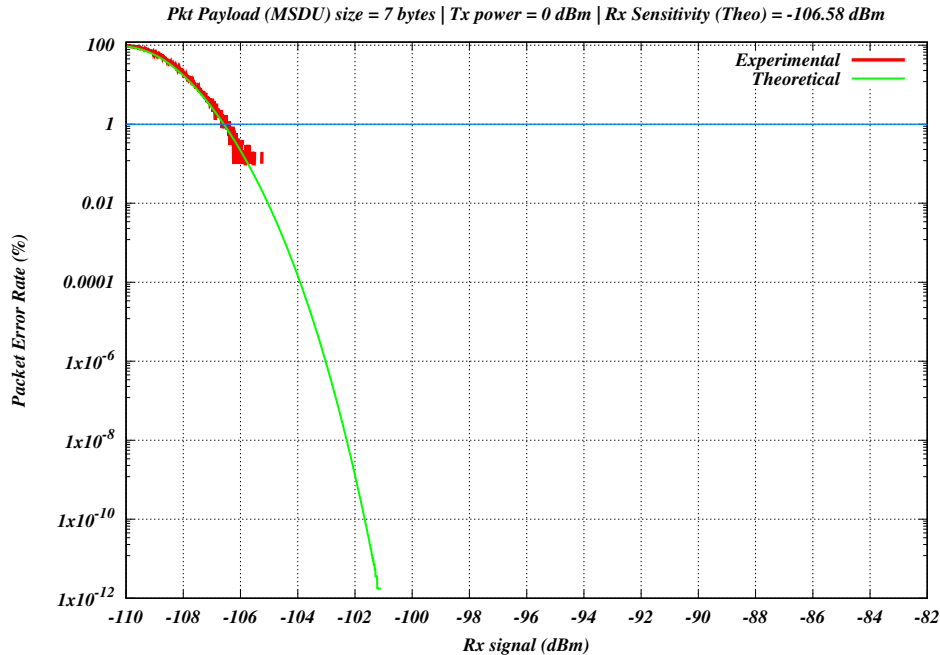


Fig. 18.4: Default output of the program `lr-wpan-per-plot.cc`

18.3 The MAC layer

The MAC at present implements both, the unslotted CSMA/CA (non-beacon mode) and the slotted CSMA/CA (beacon-enabled mode). The beacon-enabled mode supports only direct transmissions. Indirect transmissions and Guaranteed Time Slots (GTS) are currently not supported.

The present implementation supports a single PAN coordinator, support for additional coordinators is under consideration for future releases.

The implemented MAC is similar to Contiki's NullMAC, i.e., a MAC without sleep features. The radio is assumed to be always active (receiving or transmitting), or completely shut down. Frame reception is not disabled while performing the CCA.

The main API supported is the data transfer API (McpDataRequest/Indication/Confirm). CSMA/CA according to Std 802.15.4-2006, section 7.5.1.4 is supported. Frame reception and rejection according to Std 802.15.4-2006, section 7.5.6.2 is supported, including acknowledgements. Both short and extended addressing are supported. Various trace sources are supported, and trace sources can be hooked to sinks.

18.3.1 MAC layer modes

The MAC layer can operate in two different modes: beacon-enabled mode and non-beacon mode. In beacon-enabled mode, the PAN coordinator periodically transmits beacons to synchronize the devices in the PAN. The beacons define superframe structures that organize the channel time into active and inactive periods. During the active period, the channel time is further divided into a contention access period (CAP) and an optional contention free period (CFP). In beacon mode, devices use slotted CSMA/CA to access the channel and take decisions on when to transmit or receive data based on the time slots of the superframe structure. For further details see [2] and [4]. The following code snippet shows how to configure a PAN coordinator in beacon-enabled mode.

```

NodeContainer coordinators;
coordinators.Create(1);

LrWpanHelper lrWpanHelper;
NetDeviceContainer lrwpanDevices = lrWpanHelper.Install(coordinators);
Ptr<LrWpanNetDevice> dev = lrwpanDevices.Get(0)->GetObject<LrWpanNetDevice>();

MlmeStartRequestParams params;
params.m_panCoor = true;
params.m_PanId = 5;
params.m_bcnOrd = 14; // Setting the beacon interval in time slots (2^14 time slots)
params.m_sfrmOrd = 6; // Setting the size of the active and inactive periods between
↳ beacons
                        // (2^6 = 64 time slots)

dev->GetMac()->MlmeStartRequest(params); // Start the PAN coordinator with the given
↳ parameters

```

In the example above, a PAN coordinator is created and configured to operate in beacon-enabled mode by setting the beacon order (bcnOrd) attribute to a value less than 15. The superframe order (sfrmOrd) attribute is set to 6, which means that the active period of the superframe will occupy $2^6 = 64$ time slots out of the total $2^{14} = 16384$ slots. A single time slot is equivalent to 960 symbols (aBaseSuperframeDuration) or 15.36 ms (where each symbol is 16 us long at 62.5 ksymbols/s). The remaining slots ($16384 - 64 = 16320$ slots) will be inactive period where devices can enter a low power state to save energy.

Devices maintain synchronization with the PAN coordinator by receiving the periodic beacons. If a device fails to receive a certain number of consecutive beacons, it considers that it has lost synchronization with the PAN coordinator and notifies the higher layers using the MLME-SYNC-LOSS.indication primitive. Decisions about how to recover from this situation are left to the higher layers (e.g., re-association).

Finally, devices can also operate in non-beacon mode. Non-beacon mode is the default mode and implies that there is no limitation on when devices can receive or transmit data other than the constraints imposed by the unslotted CSMA/CA algorithm. This also implies that there are no inactive times.

When both the beacon order and superframe order attribute is set to 15 (default), the coordinator as well as any devices associated to the coordinator operate in non-beacon mode.

The following shows a table with some time equivalents for different beacon order or superframe order values running on the 2.4 GHz band (symbol rate of 62.5 ksymbols/s):

BcnOrd SfrmOrd exponents	Time Interval
0	15.36 ms
1	30.72 ms
2	61.44 ms
3	122.88 ms
4	245.76 ms
5	491.52 ms
6	983.04 ms
7	1.966 s
8	3.932 s
9	7.864 s
10	15.728 s
11	31.456 s
12	62.912 s
13	125.824 s
14	251.648 s
15	Non-beacon mode

Later revisions of the standard (IEEE 802.15.4-2015) introduced a new modes of operation known as “behaviors”. For example, the TSCH (Time Slotted Channel Hopping) behavior and the DSME (Deterministic and Synchronous Multi-Channel Extension) behavior. However, these new modes of operation are not currently supported by *ns-3*.

18.3.2 Scan and Association

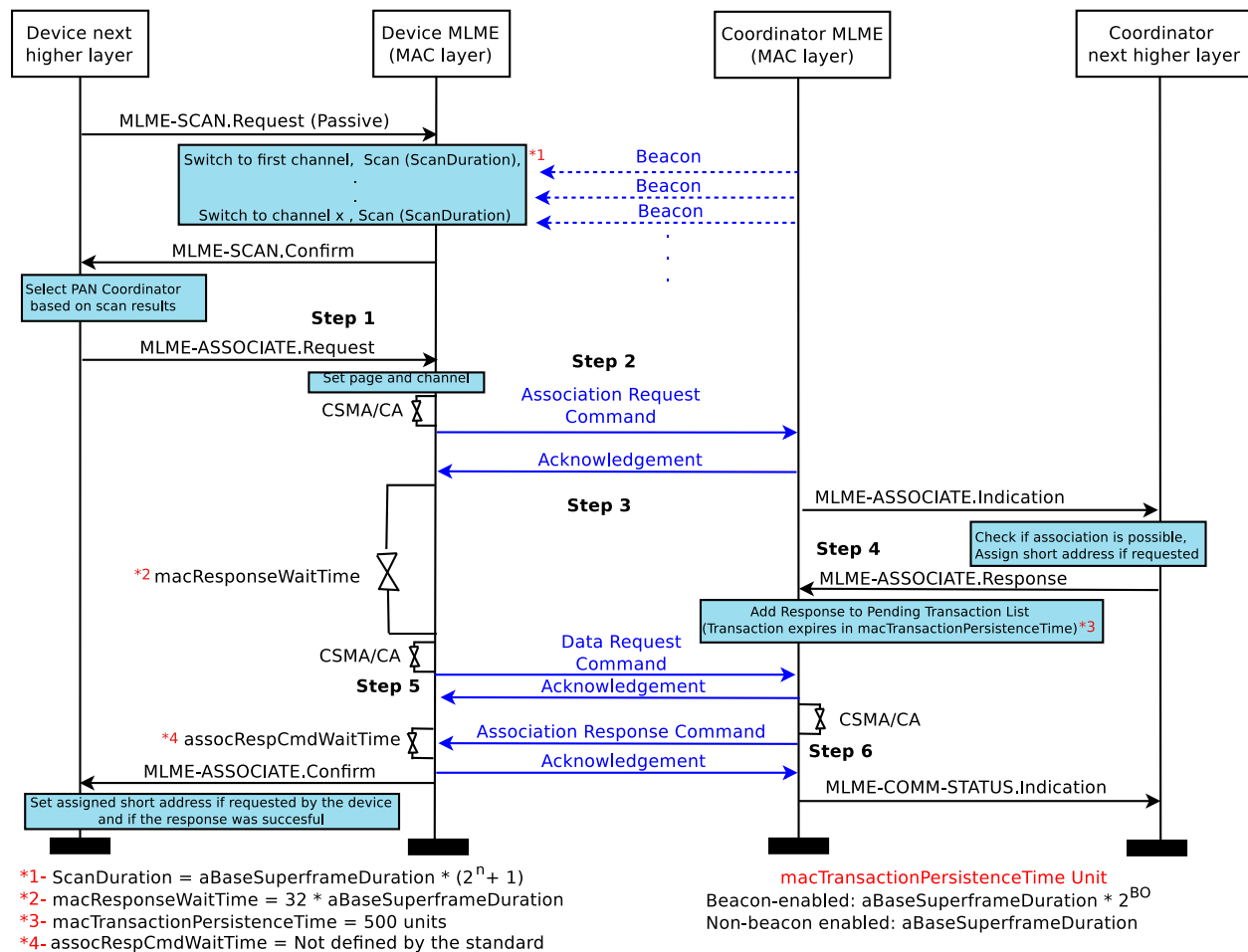
The implemented *ns-3* MAC layer supports scanning. Typically, a scanning request is preceded by an association request but these can be used independently. *ns-3* IEEE 802.15.4 MAC layer supports 4 types of scanning:

- **Energy Detection (ED) Scan:** In an energy scan, a device or a coordinator scan a set number of channels looking for traces of energy. The maximum energy registered during a given amount of time is stored. Energy scan is typically used to measure the quality of a channel at any given time. For this reason, coordinators often use this scan before initiating a PAN on a channel.
- **Active Scan:** A device sends `beacon request` commands on a set number of channels looking for a PAN coordinator. The receiving coordinator must be configured on non-beacon mode. Coordinators on beacon-mode ignore these requests. The coordinators who accept the request, respond with a beacon. After an active scan take place, during the association process devices extract the information in the PAN descriptors from the collected beacons and based on this information (e.g. channel, LQI level), choose a coordinator to associate with.
- **Passive Scan:** In a passive scan, no `beacon requests` commands are sent. Devices scan a set number of channels looking for beacons currently being transmitted (coordinators in beacon-mode). Like in the active scan, the information from beacons is stored in PAN descriptors and used by the device to choose a coordinator to associate with.
- **Orphan Scan:** Orphan scan is used typically by device as a result of repeated communication failure attempts with a coordinator. In other words, an orphan scan represents the intent of a device to relocate its coordinator. In some situations, it can be used by devices higher layers to not only rejoin a network but also join a network for the first time. In an orphan scan, a device send a `orphan notification` command to a given list of channels. If a coordinator receives this notification, it responds to the device with a `coordinator realignment` command.

In active and passive scans, the link quality indicator (LQI) is the main parameter used to determine the optimal coordinator. LQI values range from 0 to 255. Where 255 is the highest quality link value and 0 the lowest. Typically, a link lower than 127 is considered a link with poor quality.

In LR-WPAN, association is used to join PANs. All devices in LR-WPAN must belong to a PAN to communicate. *ns-3* uses a classic association procedure described in the standard. The standard also covers a more effective association procedure known as fast association (See IEEE 802.15.4-2015, fastA) but this association is currently not supported by *ns-3*. Alternatively, *ns-3* can do a “quick and dirty” association using either `LrWpanHelper::AssociateToPan`` or `LrWpanHelper::AssociateToBeaconPan``. These functions are used when a preset association can be done. For example, when the relationships between existing nodes and coordinators are known and can be set before the beginning of the simulation. In other situations, like in many networks in real deployments or in large networks, it is desirable that devices “associate themselves” with the best possible available coordinator candidates. This is a process known as bootstrap, and simulating this process makes it possible to demonstrate the kind of situations a node would face in which large networks to associate in real environment.

Bootstrap (a.k.a. network initialization) [2] [5] is possible with a combination of scan and association MAC primitives. Details on the general process for this network initialization is described in the standard. Bootstrap is a complex process that not only requires the scanning networks, but also the exchange of command frames and the use of a pending transaction list (indirect transmissions) in the coordinator to store command frames. The following summarizes the whole process:



Bootstrap as whole depends on procedures that also take place on higher layers of devices and coordinators. These procedures are briefly described in the standard but out of its scope (See IEEE 802.15.4-2011 Section 5.1.3.1.). However, these procedures are necessary for a “complete bootstrap” process. In the examples in *ns-3*, these high layer procedures are only briefly implemented to demonstrate a complete example that shows the use of scan and association. A full high layer (e.g. such as those found in Zigbee and Thread protocol stacks) should complete these procedures more robustly. Examples such as `lr-wpan-bootstrap.cc` demonstrate the whole bootstrap process (including scanning and association) while doing some place holder of procedures that take place in higher layers which are not part of standard

focus.

A key element to remember is that bootstrap have 2 objectives:

1. Enable devices to join a new formed network (associate).
2. Assign short addresses and PAN ID.

Note: Devices that have the short address FF:FF are not associated and cannot participate in any unicast communication in the network. Devices that have the short address FF:FE and have a valid PAN ID can communicate with other devices in the network using the extended address mode. In this mode, devices will use its 64 bit address (A.K.A. extended address) to communicate in the network.

Before ns-3.44, the MAC association struggled to handle scenarios where the association response command arrived before the acknowledgment (ACK) for a data request command. This issue could arise in saturated networks and is caused by a delayed data request command ACK (which do not use CSMA/CA but can be retried several times if necessary). This situation highlighted a problem and a design flaw in the original standard, where association responses are sent immediately after sending a data request acknowledgments. As a result, their arrival could become inverted. However, the current MAC implementation is now capable of correctly reacting to this delayed ACK issue.

Finally, a fixed association is possible in *ns-3* without the use of the bootstrap process. For this purpose, the `LrWpanHelper::CreateAssociatedPan` is used. See the Helpers subsection for more details.

18.3.3 MAC transmission Queues

By default, Tx queue and Ind Tx queue (the pending transaction list) are not limited but they can configure to drop packets after they reach a limit of elements (transaction overflow). Additionally, the Ind Tx queue drop packets when the packet has been longer than `macTransactionPersistenceTime` (transaction expiration). Expiration of packets in the Tx queue is not supported. Finally, packets in the Tx queue may be dropped due to excessive transmission retries or channel access failure.

18.3.4 MAC addresses

Contrary to other technologies, a IEEE 802.15.4 has 2 different kind of addresses:

- Long addresses (64 bits)
- Short addresses (16 bits)

The 64-bit addresses are unique worldwide, and set by the device vendor (in a real device). The 16-bit addresses are not guaranteed to be unique, and they are typically either assigned during the devices deployment, or assigned dynamically during the device bootstrap.

The other relevant “address” to consider is the `PanId` (16 bits), which represents the PAN the device is attached to.

Due to the limited number of available bytes in a packet, IEEE 802.15.4 tries to use short addresses instead of long addresses, even though the two might be used at the same time.

For the sake of communicating with the upper layers, and in particular to generate auto-configured IPv6 addresses, each `NetDevice` must identify itself with a MAC address. The MAC addresses are also used during packet reception, so it is important to use them consistently.

Focusing on IPv6 Stateless address autoconfiguration (SLAAC), there are two relevant RFCs to consider: RFC 4944 and RFC 6282, and the two differ on how to build the IPv6 address given the `NetDevice` address.

RFC 4944 mandates that the IID part of the IPv6 address is calculated as `YYYY:00ff:fe00:XXXX`, while RFC 6282 mandates that the IID part of the IPv6 address is calculated as `0000:00ff:fe00:XXXX` where `XXXX` is the device short

address, and YYY is the PanId. In both cases the U/L bit must be set to local, so in the RFC 4944 the PanId might have one bit flipped.

In order to facilitate interoperability, and to avoid unwanted module dependencies, the *ns-3* implementation moves the IID calculation in the `LrWpanNetDevice::GetAddress()`, which will return an Address formatted properly, i.e.:

- The Long address (a `Mac64Address`) if the Short address has not been set, or
- A properly formatted 48-bit pseudo-address (a `Mac48Address`) if the short address has been set.

The 48-bit pseudo-address is generated according to either RFC 4944 or RFC 6282 depending on the configuration of an Attribute (`PseudoMacAddressMode`).

The default is to use RFC 6282 style addresses.

Note: On reception, a packet might contain either a short or a long address. This is reflected in the upper-layer notification callback, which can contain either the pseudo-address (48 bits) or the long address (64 bit) of the sender. Also RFC 4944 or RFC 6282 are the RFCs defining the IPv6 address compression formats (HC1 and IPHC respectively). It is definitely not a good idea to either mix devices using different pseudo-address format or compression types in the same network. This point is further discussed in the *sixlowpan* module documentation.

18.4 Usage

18.4.1 Helpers

The helper is patterned after other device helpers. In particular, tracing (`ascii` and `pcap`) is enabled similarly, and enabling of all *lr-wpan* log components is performed similarly. Use of the helper is exemplified in many of the examples found in `src\lrwpan\examples`.

When the helper is used, the default propagation loss model added to the channel is the `LogDistancePropagationLossModel` with default parameters. The default delay model used is `ConstantSpeedPropagationDelayMode`. Alternatively, users can specify the models used by the helper with the `SetPropagationDelayModel` and the `AddPropagationLossModel` functions. The helper can also be used to form a fully associated PANs:

```
NodeContainer nodes;
nodes.Create(4);
LrWpanHelper lrWpanHelper;
// The propagation and Delay Model are optional, a default is used if not provided.
lrWpanHelper.SetPropagationDelayModel("ns3::ConstantSpeedPropagationDelayModel");
lrWpanHelper.AddPropagationLossModel("ns3::LogDistancePropagationLossModel");
// Create and associate a PAN with addresses, The first node is used as coordinator
lrWpanHelper.CreateAssociatedPan(lrWpanDevices, 0xCAFE);
NetDeviceContainer lrWpanDevices = lrWpanHelper.Install(nodes);
```

In the example shown above, the function `CreateAssociatedPan` forms a star topology PAN with devices sharing the same PAN ID (In the example `0xCAFE`). Devices are assigned both 64 bit addresses (extended addresses) and 16 bit addresses (short addresses) and can use either to communicate when the configuration is completed. The first node in the `NodeContainer` is assigned the role of PAN coordinator. The number of nodes provided to the helper are limited by the number of possible assignable short addresses (`[00:01]-[FF:FD]`).

It is also possible to manually assign the PAN ID, the extended address and the short address without the use of a helper. To know how to do this, refer to the `lr-wpan-data.cc` example.

For more complex topologies an auto-configured address assignation scheme is recommended, the bootstrap process (scan and association) described by the standard can help with this.

Note: As mentioned in previous sections, devices that does not have short addresses assigned (Any address other than FF:FF) and with a valid PAN IDs cannot communicate in lr-wpan network. Make sure both are present in your simulated devices using any of the schemes described.

Finally, the LrWpanHelper can be used to generate pcap or ascii traces. These pcap files can be later on visualized in Wireshark.

```
// Tracing
LrWpanHelper.EnablePcapAll(std::string("lr-wpan-data"), true);
AsciiTraceHelper ascii;
Ptr<OutputStreamWrapper> stream = ascii.CreateFileStream("lr-wpan-data.tr");
LrWpanHelper.EnableAsciiAll(stream);
```

18.4.2 Attributes

The following is a list of attributes used to configure the behavior of the PHY layer:

- **PostReceptionErrorModel:** An optional packet error model can be added to the receive packet process after any propagation-based (SNR-based) error models have been applied. Typically this is used to force specific packet drops, for testing purposes.

The following is a list of attributes that can be set directly to configure the general behavior of the **MAC layer**. Most of this attributes have a counter part with the PIB attributes and constants described by the standard.

- **PanId:** Set the 16-bit identifier of the associated PAN.

18.4.3 Traces

The following is a list of the trace source that can be used to monitor the behavior of the **PHY layer**:

- **TrxStateValue:** The state of the finit state machine of the PHY.
- **TrxState:** The state of the transceiver.
- **PhyTxBegin:** Indicates that a packet has begun transmitting over the channel medium.
- **PhyTxEnd:** Indicates that a packet has been completely transmitted over the channel.
- **PhyTxDrop:** Indicates that a packet has been dropped by the device during transmission.
- **PhyRxBegin:** Indicates that a packet has begun being received from the channel medium by the receiver.
- **PhyRxEnd:** Indicates that a packet has been completely received from the channel medium.
- **PhyRxDrop:** Indicates that a packet has been dropped by the device during reception.

The following is a list of the trace sources that can be used to monitor the behavior of the **MAC layer**:

- **MacTxEnqueue:** Trace source indicating a packet has been enqueued in the transaction queue.
- **MacTxDequeue:** Trace source indicating a packet has was dequeued from the transaction queue.
- **MacIndTxEnqueue:** Trace source indicating a packet has been enqueued in the indirect transaction queue.
- **MacIndTxDequeue:** Trace source indicating a packet has was dequeued from the indirect transaction queue.

- **MacTx:** Trace source indicating a packet has arrived for transmission by this device.
- **MacTxOk:** Trace source indicating a packet has been successfully sent.
- **MacTxDrop:** Trace source indicating a packet has been dropped during transmission.
- **MacIndTxDrop:** Trace source indicating a packet has been dropped from the indirect transaction queue (The pending transaction list).
- **MacPromiscRx:** A packet has been received by this device, has been passed up from the physical layer and is being forwarded up the local protocol stack.
- **MacRx:** A packet has been received by this device, has been passed up from the physical layer and is being forwarded up the local protocol stack.
- **MacRxDrop:** Trace source indicating a packet was received, but dropped before being forwarded up the stack.
- **Sniffer:** Trace source simulating a non-promiscuous packet sniffer attached to the device.
- **PromiscSniffer:** Trace source simulating a promiscuous packet sniffer attached to the device.
- **MacStateValue:** Shows the current state of LrWpan MAC layer.
- **MacIncSuperframeStatus:** When using the beacon mode, shows the period status of the incoming superframe.
- **MacOutSuperframeStatus:** When using the beacon mode, shows the period status of the outgoing superframe.
- **MacState:** The state of LrWpan Mac.
- **MacSentPkt:** Trace source reporting some information about the sent packet.
- **IfsEnd:** Reports the end of an Interframe space period (IFS).

18.5 Examples and Tests

The following examples have been written, which can be found in `src/lr-wpan/examples/`:

- `lr-wpan-data.cc`: A simple example showing end-to-end data transfer.
- `lr-wpan-error-distance-plot.cc`: An example to plot variations of the packet success ratio as a function of distance.
- `lr-wpan-per-plot.cc`: An example to plot the theoretical and experimental packet error rate (PER) as a function of receive signal.
- `lr-wpan-error-model-plot.cc`: An example to test the phy.
- `lr-wpan-packet-print.cc`: An example to print out the MAC header fields.
- `lr-wpan-phy-test.cc`: An example to test the phy.
- `lr-wpan-ed-scan.cc`: Simple example showing the use of energy detection (ED) scan in the MAC.
- `lr-wpan-active-scan.cc`: A simple example showing the use of an active scan in the MAC.
- `lr-wpan-beacon-mode.cc`: Demonstrates the use of lr-wpan beacon mode. Nodes use a manual association (i.e. No bootstrap) in this example.
- `lr-wpan-bootstrap.cc`: Demonstrates the use of scanning and association working together to initiate a PAN.
- `lr-wpan-orphan-scan.cc`: Demonstrates the use of an orphan scanning in a simple network joining procedure.

In particular, the module enables a very simplified end-to-end data transfer scenario, implemented in `lr-wpan-data.cc`. The figure shows a sequence of events that are triggered when the MAC receives a `DataRequest` from the higher

layer. It invokes a Clear Channel Assessment (CCA) from the PHY, and if successful, sends the frame down to the PHY where it is transmitted over the channel and results in a DataIndication on the peer node.

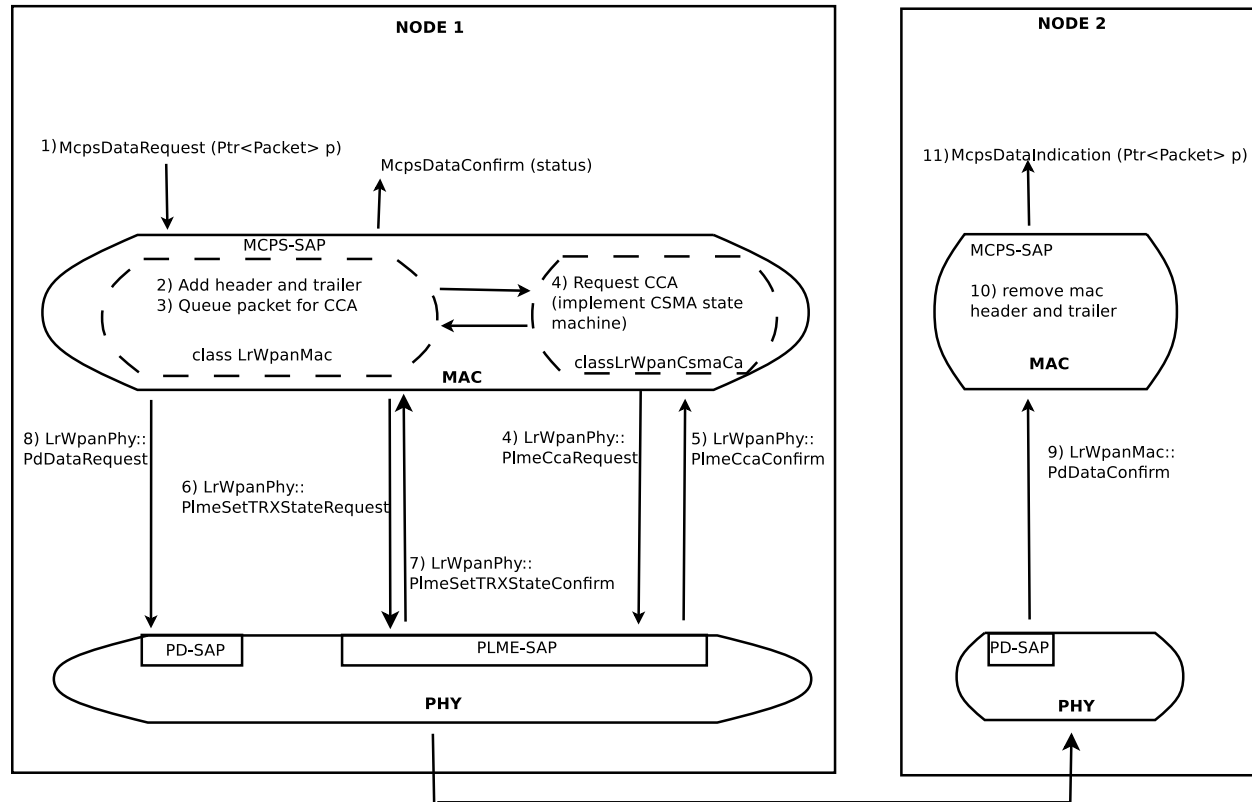


Fig. 18.5: Data example for simple LR-WPAN data transfer end-to-end

The example `lr-wpan-error-distance-plot.cc` plots the packet success ratio (PSR) as a function of distance, using the default LogDistance propagation loss model and the 802.15.4 error model. The channel (default 11), packet size (default PSDU 20 bytes = 11 bytes MAC header + data payload), transmit power (default 0 dBm) and Rx sensitivity (default -106.58 dBm) can be varied by command line arguments. The program outputs a file named `802.15.4-psr-distance.plt`. Loading this file into gnuplot yields a file `802.15.4-psr-distance.eps`, which can be converted to pdf or other formats. The following image shows the output of multiple runs using different Rx sensitivity values. A higher Rx sensitivity (lower dBm) results in a increased communication distance but also makes the radio susceptible to more interference from surrounding devices.

The following tests have been written, which can be found in `src/lr-wpan/tests/`:

- `lr-wpan-ack-test.cc`: Check that acknowledgments are being used and issued in the correct order.
- `lr-wpan-cca-test.cc`: Test the behavior of CCA under specific circumstances (Hidden terminal and CCA vulnerable windows)
- `lr-wpan-collision-test.cc`: Test correct reception of packets with interference and collisions.
- `lr-wpan-ed-test.cc`: Test the energy detection (ED) capabilities of the Lr-Wpan implementation.
- `lr-wpan-error-model-test.cc`: Check that the error model gives predictable values.
- `lr-wpan-ifs-test.cc`: Check that the Intraframe Spaces (IFS) are being used and issued in the correct order.
- `lr-wpan-mac-test.cc`: Test various MAC capabilities such different types of scanning and `RX_ON_WHEN_IDLE`.
- `lr-wpan-packet-test.cc`: Test the 802.15.4 MAC header/trailer classes

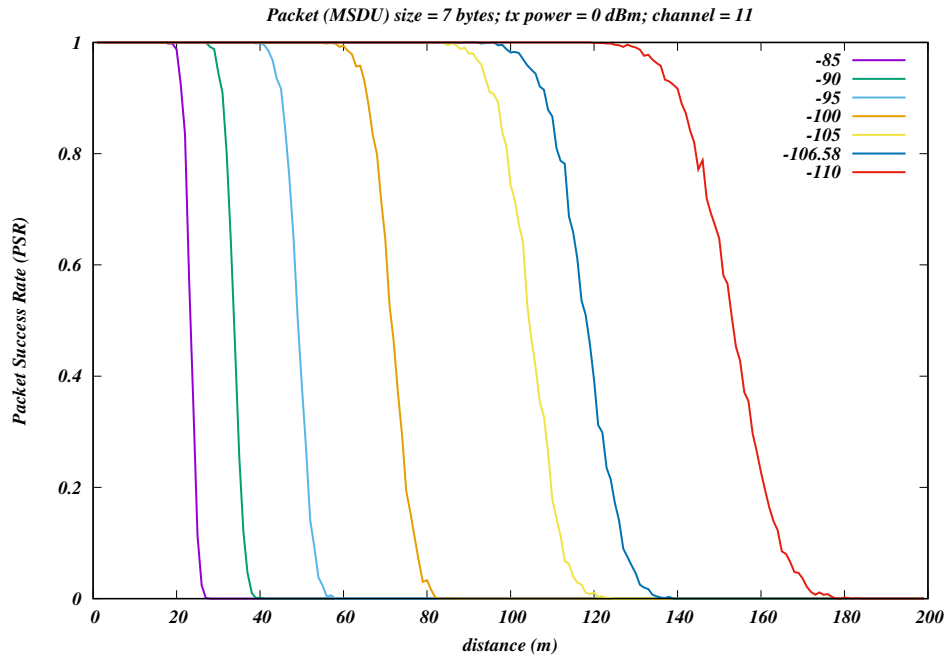


Fig. 18.6: Default output of the program `lr-wpan-error-distance-plot.cc`

- `lr-wpan-pd-plme-sap-test.cc`: Test the PLME and PD primitives from Lr-wpan's PHY.
- `lr-wpan-spectrum-value-helper-test.cc`: Test that the conversion between power (expressed as a scalar quantity) and spectral power, and back again, falls within a 25% tolerance across the range of possible channels and input powers.
- `lr-wpan-slotted-csmaca-test.cc`: Test the transmission and deferring of data packets in the Contention Access Period (CAP) for the slotted CSMA/CA (beacon-enabled mode).

The test suite `lr-wpan-cca-test.cc` demonstrate some known CCA behaviors. The test suite includes a test that demonstrates the well known hidden terminal problem. The second test included in the test suite shows a known vulnerability window (192us) in CCA that can cause a false positive identification of channel as IDLE caused by the turnAround delay between the CCA and the actual transmission of the frame.

18.6 Validation

The model has not been validated against real hardware. The error model has been validated against the data in IEEE Std 802.15.4-2006, section E.4.1.7 (Figure E.2). The MAC behavior (CSMA backoff) has been validated by hand against expected behavior. The below plot is an example of the error model validation and can be reproduced by running `lr-wpan-error-model-plot.cc`:

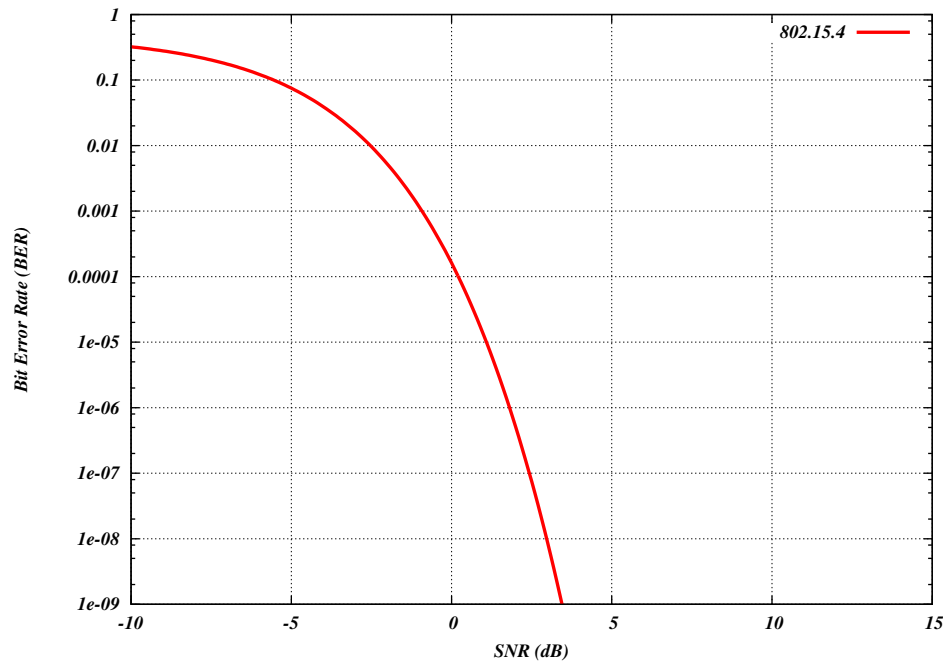


Fig. 18.7: Default output of the program `lr-wpan-error-model-plot.cc`

18.7 References

- [1] Wireless Medium Access Control (MAC) and Physical Layer (PHY) Specifications for Low-Rate Wireless Personal Area Networks (WPANs), IEEE Computer Society, IEEE Std 802.15.4-2006, 8 September 2006.
- [2] IEEE Standard for Local and metropolitan area networks—Part 15.4: Low-Rate Wireless Personal Area Networks (LR-WPANs),” in IEEE Std 802.15.4-2011 (Revision of IEEE Std 802.15.4-2006) , vol., no., pp.1-314, 5 Sept. 2011, doi: 10.1109/IEEESTD.2011.6012487.
- [3] J. Zheng and Myung J. Lee, “A comprehensive performance study of IEEE 802.15.4,” Sensor Network Operations, IEEE Press, Wiley Interscience, Chapter 4, pp. 218-237, 2006.
- [4] Alberto Gallegos Ramonet and Taku Noguchi. 2020. LR-WPAN: Beacon Enabled Direct Transmissions on Ns-3. In 2020 the 6th International Conference on Communication and Information Processing (ICCIP 2020). Association for Computing Machinery, New York, NY, USA, 115–122. <https://doi.org/10.1145/3442555.3442574>.
- [5] Gallegos Ramonet, A.; Noguchi, T. Performance Analysis of IEEE 802.15.4 Bootstrap Process. Electronics 2022, 11, 4090. <https://doi.org/10.3390/electronics11244090>.

LTE MODULE

19.1 Design Documentation

19.1.1 Overview

An overview of the LTE-EPC simulation model is depicted in the figure *Overview of the LTE-EPC simulation model*. There are two main components:

- the LTE Model. This model includes the LTE Radio Protocol stack (RRC, PDCP, RLC, MAC, PHY). These entities reside entirely within the UE and the eNB nodes.
- the EPC Model. This model includes core network interfaces, protocols and entities. These entities and protocols reside within the SGW, PGW and MME nodes, and partially within the eNB nodes.

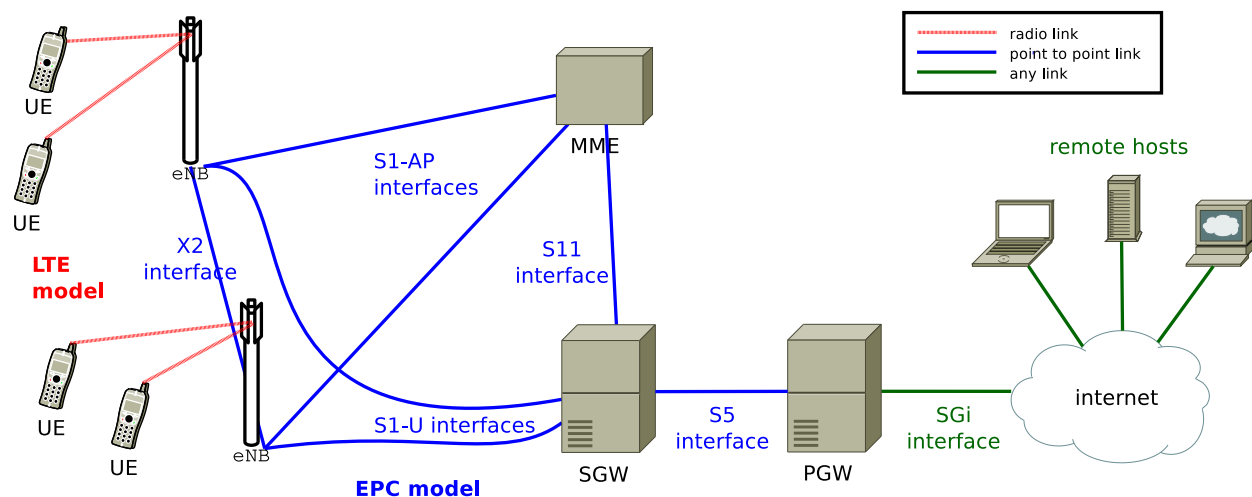


Fig. 19.1: Overview of the LTE-EPC simulation model

19.1.2 Design Criteria

LTE Model

The LTE model has been designed to support the evaluation of the following aspects of LTE systems:

- Radio Resource Management
- QoS-aware Packet Scheduling
- Inter-cell Interference Coordination
- Dynamic Spectrum Access

In order to model LTE systems to a level of detail that is sufficient to allow a correct evaluation of the above mentioned aspects, the following requirements have been considered:

1. At the radio level, the granularity of the model should be at least that of the Resource Block (RB). In fact, this is the fundamental unit being used for resource allocation. Without this minimum level of granularity, it is not possible to model accurately packet scheduling and inter-cell-interference. The reason is that, since packet scheduling is done on a per-RB basis, an eNB might transmit on a subset only of all the available RBs, hence interfering with other eNBs only on those RBs where it is transmitting. Note that this requirement rules out the adoption of a system level simulation approach, which evaluates resource allocation only at the granularity of call/bearer establishment.
2. The simulator should scale up to tens of eNBs and hundreds of User Equipment (UEs). This rules out the use of a link level simulator, i.e., a simulator whose radio interface is modeled with a granularity up to the symbol level. This is because to have a symbol level model it is necessary to implement all the PHY layer signal processing, whose huge computational complexity severely limits simulation. In fact, link-level simulators are normally limited to a single eNB and one or a few UEs.
3. It should be possible within the simulation to configure different cells so that they use different carrier frequencies and system bandwidths. The bandwidth used by different cells should be allowed to overlap, in order to support dynamic spectrum licensing solutions such as those described in [Ofcom2600MHz] and [RealWireless]. The calculation of interference should handle appropriately this case.
4. To be more representative of the LTE standard, as well as to be as close as possible to real-world implementations, the simulator should support the MAC Scheduler API published by the FemtoForum [FFAPI]. This interface is expected to be used by femtocell manufacturers for the implementation of scheduling and Radio Resource Management (RRM) algorithms. By introducing support for this interface in the simulator, we make it possible for LTE equipment vendors and operators to test in a simulative environment exactly the same algorithms that would be deployed in a real system.
5. The LTE simulation model should contain its own implementation of the API defined in [FFAPI]. Neither binary nor data structure compatibility with vendor-specific implementations of the same interface are expected; hence, a compatibility layer should be interposed whenever a vendor-specific MAC scheduler is to be used with the simulator. This requirement is necessary to allow the simulator to be independent from vendor-specific implementations of this interface specification. We note that [FFAPI] is a logical specification only, and its implementation (e.g., translation to some specific programming language) is left to the vendors.
6. The model is to be used to simulate the transmission of IP packets by the upper layers. With this respect, it shall be considered that in LTE the Scheduling and Radio Resource Management do not work with IP packets directly, but rather with RLC PDUs, which are obtained by segmentation and concatenation of IP packets done by the RLC entities. Hence, these functionalities of the RLC layer should be modeled accurately.

EPC Model

The main objective of the EPC model is to provide means for the simulation of end-to-end IP connectivity over the LTE model. To this aim, it supports for the interconnection of multiple UEs to the Internet, via a radio access network of multiple eNBs connected to the core network, as shown in Figure *Overview of the LTE-EPC simulation model*.

The following design choices have been made for the EPC model:

1. The Packet Data Network (PDN) type supported is both IPv4 and IPv6. In other words, the end-to-end connections between the UEs and the remote hosts can be IPv4 and IPv6. However, the networks between the core network elements (MME, SGWs and PGWs) are IPv4-only.
2. The SGW and PGW functional entities are implemented in different nodes, which are hence referred to as the SGW node and PGW node, respectively.
3. The MME functional entities is implemented as a network node, which is hence referred to as the MME node.
4. The scenarios with inter-SGW mobility are not of interest. But several SGW nodes may be present in simulations scenarios.
5. A requirement for the EPC model is that it can be used to simulate the end-to-end performance of realistic applications. Hence, it should be possible to use with the EPC model any regular ns-3 application working on top of TCP or UDP.
6. Another requirement is the possibility of simulating network topologies with the presence of multiple eNBs, some of which might be equipped with a backhaul connection with limited capabilities. In order to simulate such scenarios, the user data plane protocols being used between the eNBs and the SGW should be modeled accurately.
7. It should be possible for a single UE to use different applications with different QoS profiles. Hence, multiple EPS bearers should be supported for each UE. This includes the necessary classification of TCP/UDP traffic over IP done at the UE in the uplink and at the PGW in the downlink.
8. The initial focus of the EPC model is mainly on the EPC data plane. The accurate modeling of the EPC control plane is, for the time being, not a requirement; however, the necessary control plane interactions among the different network nodes of the core network are realized by implementing control protocols/messages among them. Direct interaction among the different simulation objects via the provided helper objects should be avoided as much as possible.
9. The focus of the EPC model is on simulations of active users in ECM connected mode. Hence, all the functionality that is only relevant for ECM idle mode (in particular, tracking area update and paging) are not modeled at all.
10. The model should allow the possibility to perform an X2-based handover between two eNBs.

19.1.3 Architecture

LTE Model

UE architecture

The architecture of the LTE radio protocol stack model of the UE is represented in the figures *LTE radio protocol stack architecture for the UE on the data plane* and *LTE radio protocol stack architecture for the UE on the control plane* which highlight respectively the data plane and the control plane.

The architecture of the PHY/channel model of the UE is represented in figure *PHY and channel model architecture for the UE*.

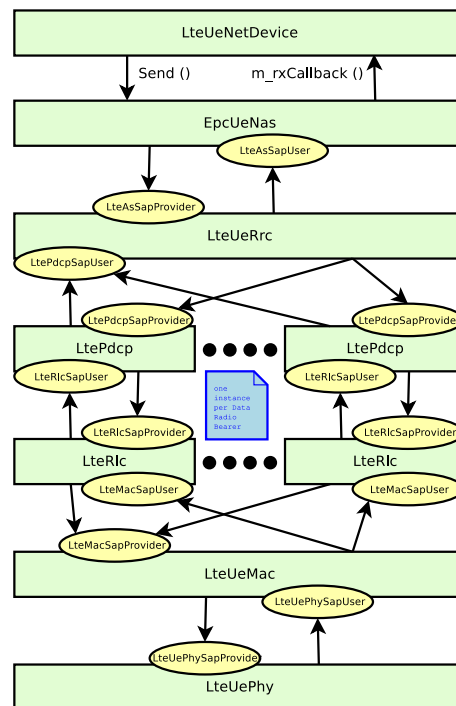


Fig. 19.2: LTE radio protocol stack architecture for the UE on the data plane

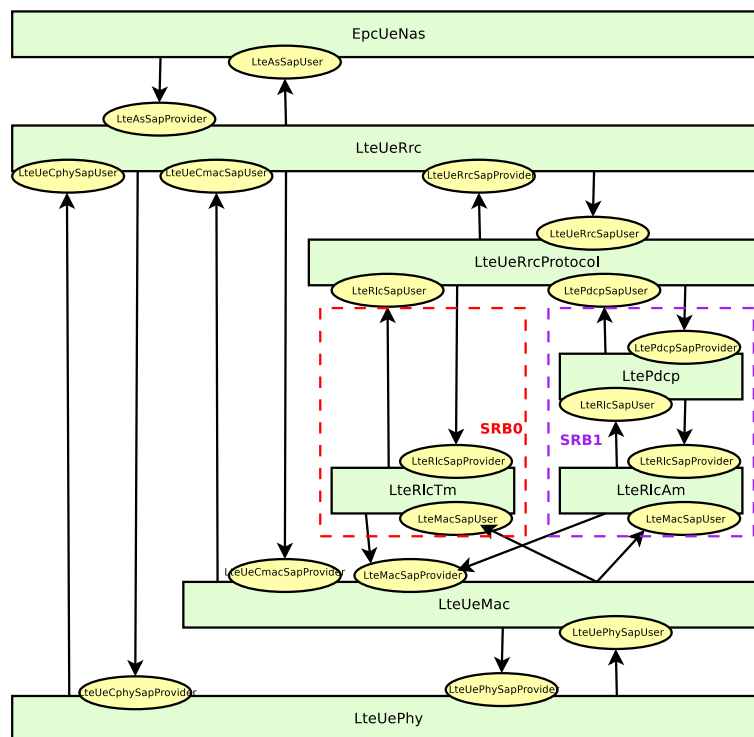


Fig. 19.3: LTE radio protocol stack architecture for the UE on the control plane

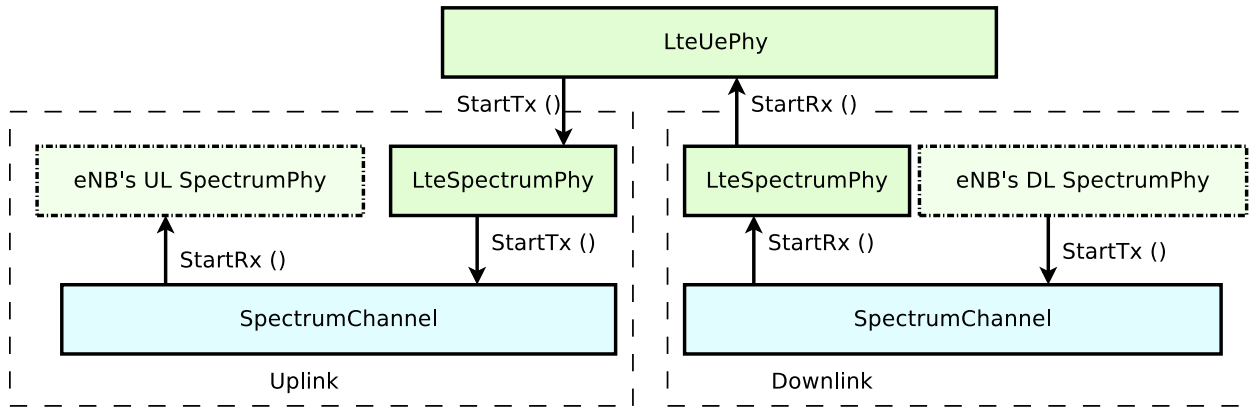


Fig. 19.4: PHY and channel model architecture for the UE

eNB architecture

The architecture of the LTE radio protocol stack model of the eNB is represented in the figures *LTE radio protocol stack architecture for the eNB on the data plane* and *LTE radio protocol stack architecture for the eNB on the control plane* which highlight respectively the data plane and the control plane.

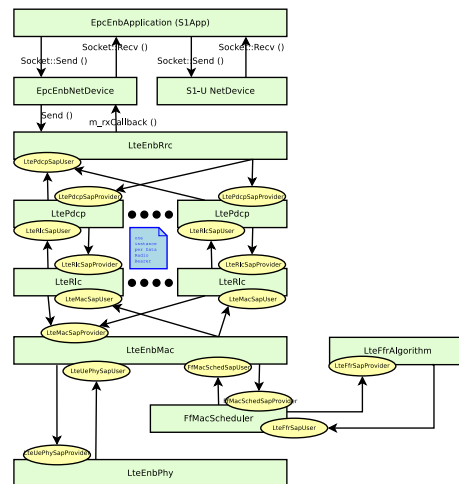


Fig. 19.5: LTE radio protocol stack architecture for the eNB on the data plane

The architecture of the PHY/channel model of the eNB is represented in figure *PHY and channel model architecture for the eNB*.

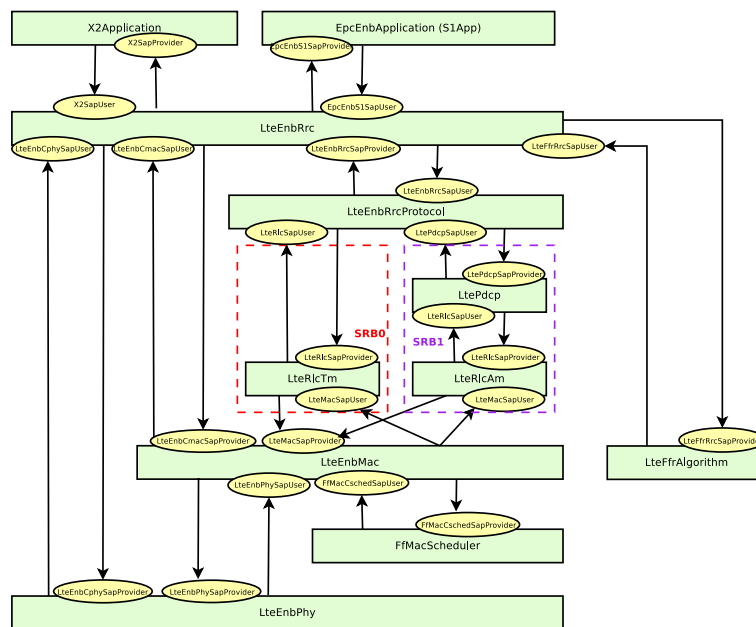


Fig. 19.6: LTE radio protocol stack architecture for the eNB on the control plane

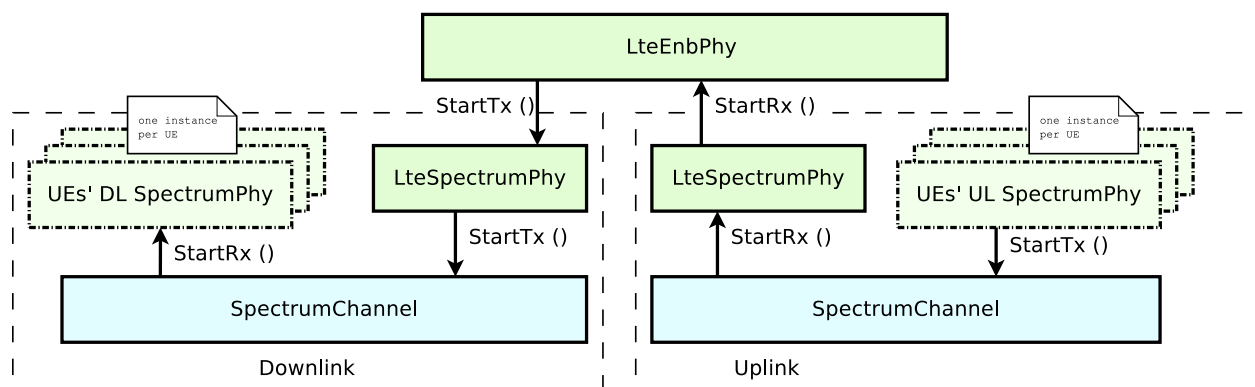


Fig. 19.7: PHY and channel model architecture for the eNB

EPC Model

EPC data plane

In Figure *LTE-EPC data plane protocol stack*, we represent the end-to-end LTE-EPC data plane protocol stack as it is modeled in the simulator. The figure shows all nodes in the data path, i.e. UE, eNB, SGW, PGW and a remote host in the Internet. All protocol stacks (S5 protocol stack, S1-U protocol stack and the LTE radio protocol stack) specified by 3GPP are present.

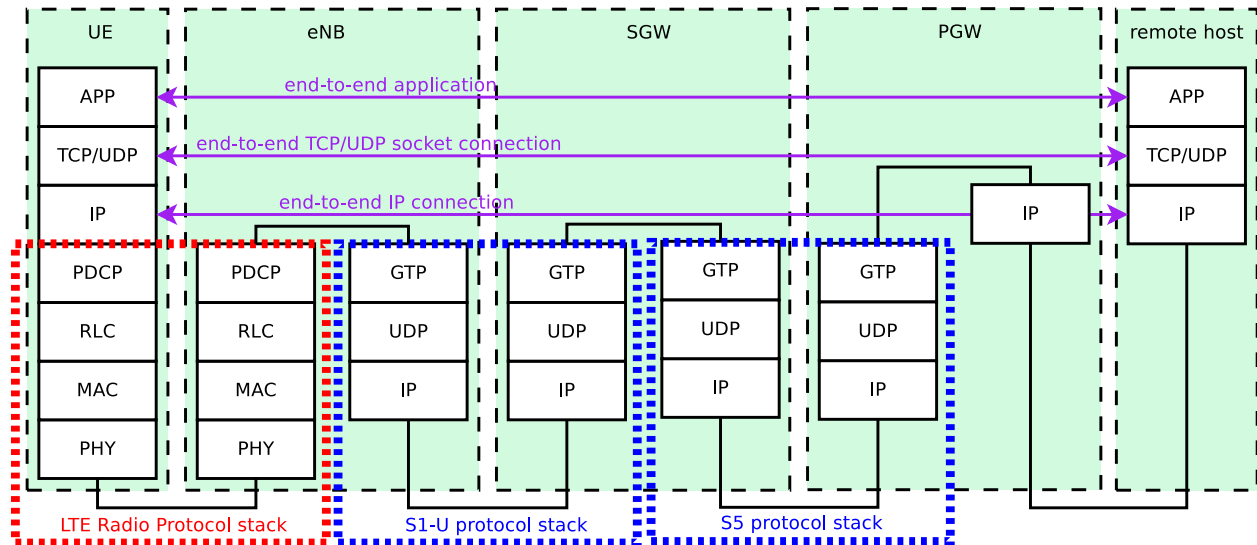


Fig. 19.8: LTE-EPC data plane protocol stack

EPC control plane

The architecture of the implementation of the control plane model is shown in figure *LTE-EPC control plane protocol stack*. The control interfaces that are modeled explicitly are the S1-MME, the S11, and the S5 interfaces. The X2 interface is also modeled explicitly and it is described in more detail in section *X2*

The S1-MME, the S11 and the S5 interfaces are modeled using protocol data units sent over its respective links. These interfaces use the SCTP protocol as transport protocol but currently, the SCTP protocol is not modeled in the ns-3 simulator, so the UDP protocol is used instead of the SCTP protocol.

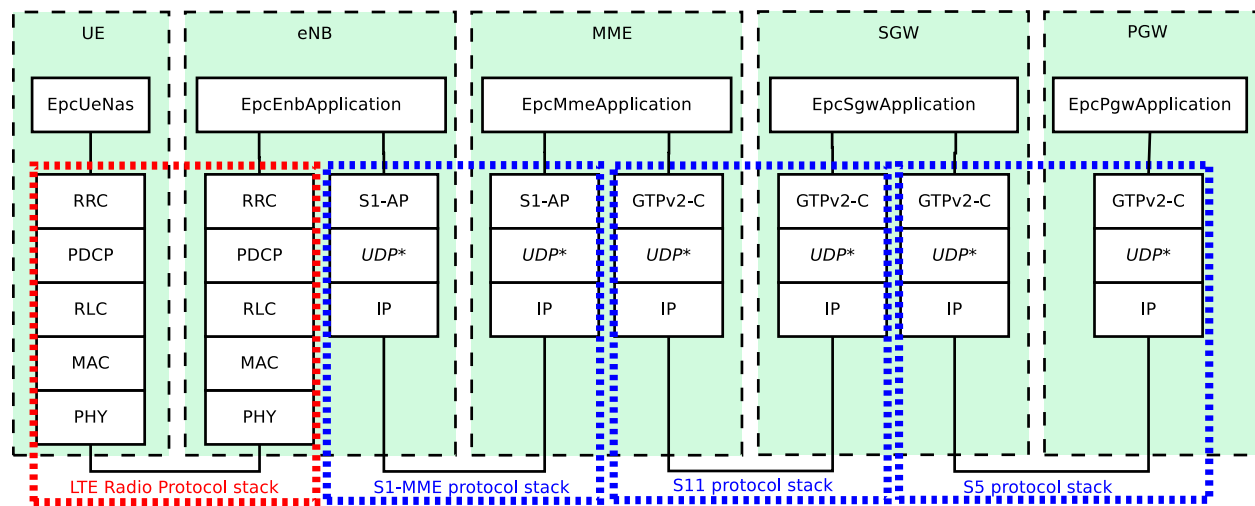


Fig. 19.9: LTE-EPC control plane protocol stack

19.1.4 Channel and Propagation

For channel modeling purposes, the LTE module uses the `SpectrumChannel` interface provided by the spectrum module. At the time of this writing, two implementations of such interface are available: `SingleModelSpectrumChannel` and `MultiModelSpectrumChannel`, and the LTE module requires the use of the `MultiModelSpectrumChannel` in order to work properly. This is because of the need to support different frequency and bandwidth configurations. All the propagation models supported by `MultiModelSpectrumChannel` can be used within the LTE module.

Use of the Buildings model with LTE

The recommended propagation model to be used with the LTE module is the one provided by the Buildings module, which was in fact designed specifically with LTE (though it can be used with other wireless technologies as well). Please refer to the documentation of the Buildings module for generic information on the propagation model it provides.

In this section we will highlight some considerations that specifically apply when the Buildings module is used together with the LTE module.

The naming convention used in the following will be:

- User equipment: UE
- Macro Base Station: MBS
- Small cell Base Station (e.g., pico/femtocell): SC

The LTE module considers FDD only, and implements downlink and uplink propagation separately. As a consequence, the following pathloss computations are performed

- MBS <-> UE (indoor and outdoor)
- SC (indoor and outdoor) <-> UE (indoor and outdoor)

The LTE model does not provide the following pathloss computations:

- UE <-> UE
- MBS <-> MBS
- MBS <-> SC
- SC <-> SC

The Buildings model does not know the actual type of the node; i.e., it is not aware of whether a transmitter node is a UE, a MBS, or a SC. Rather, the Buildings model only cares about the position of the node: whether it is indoor and outdoor, and what is its z-axis respect to the rooftop level. As a consequence, for an eNB node that is placed outdoor and at a z-coordinate above the rooftop level, the propagation models typical of MBS will be used by the Buildings module. Conversely, for an eNB that is placed outdoor but below the rooftop, or indoor, the propagation models typical of pico and femtocells will be used.

For communications involving at least one indoor node, the corresponding wall penetration losses will be calculated by the Buildings model. This covers the following use cases:

- MBS <-> indoor UE
- outdoor SC <-> indoor UE
- indoor SC <-> indoor UE
- indoor SC <-> outdoor UE

Please refer to the documentation of the Buildings module for details on the actual models used in each case.

Fading Model

The LTE module includes a trace-based fading model derived from the one developed during the GSoC 2010 [Piro2011]. The main characteristic of this model is the fact that the fading evaluation during simulation run-time is based on pre-calculated traces. This is done to limit the computational complexity of the simulator. On the other hand, it needs huge structures for storing the traces; therefore, a trade-off between the number of possible parameters and the memory occupancy has to be found. The most important ones are:

- users' speed: relative speed between users (affects the Doppler frequency, which in turns affects the time-variance property of the fading)
- number of taps (and relative power): number of multiple paths considered, which affects the frequency property of the fading.
- time granularity of the trace: sampling time of the trace.
- frequency granularity of the trace: number of values in frequency to be evaluated.
- length of trace: ideally large as the simulation time, might be reduced by windowing mechanism.
- number of users: number of independent traces to be used (ideally one trace per user).

With respect to the mathematical channel propagation model, we suggest the one provided by the `rayleighchan` function of Matlab, since it provides a well accepted channel modelization both in time and frequency domain. For more information, the reader is referred to [mathworks].

The simulator provides a matlab script (`src/lte/model/fading-traces/fading-trace-generator.m`) for generating traces based on the format used by the simulator. In detail, the channel object created with the `rayleighchan` function is used for filtering a discrete-time impulse signal in order to obtain the channel impulse response. The filtering is repeated for different TTI, thus yielding subsequent time-correlated channel responses (one per TTI). The channel response is then processed with the `pwelch` function for obtaining its power spectral density values, which are then saved in a file with the proper format compatible with the simulator model.

Since the number of variable it is pretty high, generate traces considering all of them might produce a high number of traces of huge size. On this matter, we considered the following assumptions of the parameters based on the 3GPP fading propagation conditions (see Annex B.2 of [TS36104]):

- users' speed: typically only a few discrete values are considered, i.e.:
 - 0 and 3 kmph for pedestrian scenarios
 - 30 and 60 kmph for vehicular scenarios
 - 0, 3, 30 and 60 for urban scenarios
- channel taps: only a limited number of sets of channel taps are normally considered, for example three models are mentioned in Annex B.2 of [TS36104].
- time granularity: we need one fading value per TTI, i.e., every 1 ms (as this is the granularity in time of the ns-3 LTE PHY model).
- frequency granularity: we need one fading value per RB (which is the frequency granularity of the spectrum model used by the ns-3 LTE model).
- length of the trace: the simulator includes the windowing mechanism implemented during the GSoC 2011, which consists of picking up a window of the trace each window length in a random fashion.
- per-user fading process: users share the same fading trace, but for each user a different starting point in the trace is randomly picked up. This choice was made to avoid the need to provide one fading trace per user.

According to the parameters we considered, the following formula express in detail the total size S_{traces} of the fading traces:

$$S_{traces} = S_{sample} \times N_{RB} \times \frac{T_{trace}}{T_{sample}} \times N_{scenarios} \text{ [bytes]}$$

where S_{sample} is the size in bytes of the sample (e.g., 8 in case of double precision, 4 in case of float precision), N_{RB} is the number of RB or set of RBs to be considered, T_{trace} is the total length of the trace, T_{sample} is the time resolution of the trace (1 ms), and $N_{scenarios}$ is the number of fading scenarios that are desired (i.e., combinations of different sets of channel taps and user speed values). We provide traces for 3 different scenarios one for each taps configuration defined in Annex B.2 of [TS36104]:

- Pedestrian: with nodes' speed of 3 kmph.
- Vehicular: with nodes' speed of 60 kmph.
- Urban: with nodes' speed of 3 kmph.

hence $N_{scenarios} = 3$. All traces have $T_{trace} = 10$ s and $RB_{NUM} = 100$. This results in a total 24 MB bytes of traces.

Antennas

Being based on the SpectrumPhy, the LTE PHY model supports antenna modeling via the ns-3 `AntennaModel` class. Hence, any model based on this class can be associated with any eNB or UE instance. For instance, the use of the `CosineAntennaModel` associated with an eNB device allows to model one sector of a macro base station. By default, the `IsotropicAntennaModel` is used for both eNBs and UEs.

19.1.5 PHY

Overview

The physical layer model provided in this LTE simulator is based on the one described in [Piro2011], with the following modifications. The model now includes the inter cell interference calculation and the simulation of uplink traffic, including both packet transmission and CQI generation.

Subframe Structure

The subframe is divided into control and data part as described in Figure *LTE subframe division*.

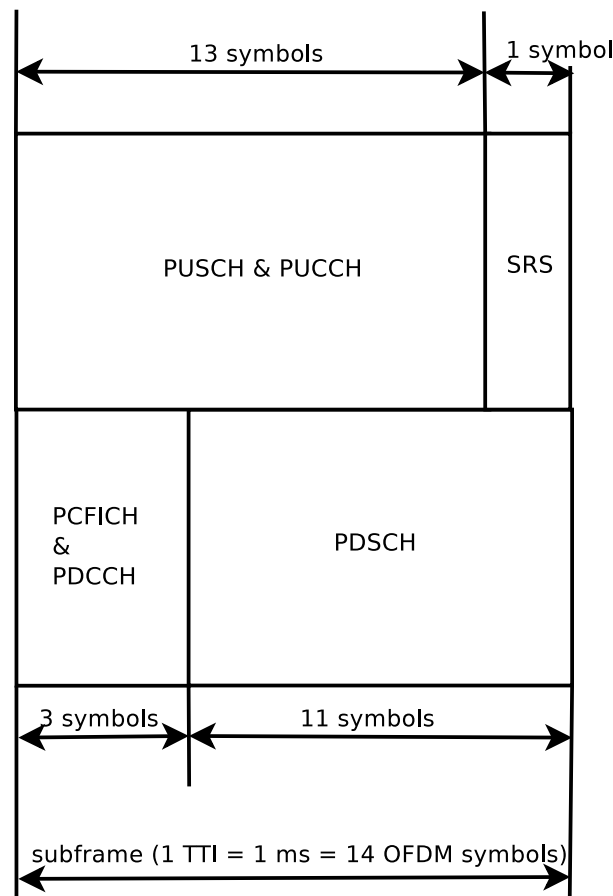


Fig. 19.10: LTE subframe division.

Considering the granularity of the simulator based on RB, the control and the reference signaling have to be consequently modeled considering this constraint. According to the standard [TS36211], the downlink control frame starts at the beginning of each subframe and lasts up to three symbols across the whole system bandwidth, where the actual duration is provided by the Physical Control Format Indicator Channel (PCFICH). The information on the allocation are then mapped in the remaining resource up to the duration defined by the PCFICH, in the so called Physical Downlink Control Channel (PDCCH). A PDCCH transports a single message called Downlink Control Information (DCI) coming from the MAC layer, where the scheduler indicates the resource allocation for a specific user. The PCFICH and PDCCH are modeled with the transmission of the control frame of a fixed duration of 3/14 of milliseconds spanning in the whole available bandwidth, since the scheduler does not estimate the size of the control region. This implies that a single transmission block models the entire control frame with a fixed power (i.e., the one used for the PDSCH)

across all the available RBs. According to this feature, this transmission represents also a valuable support for the Reference Signal (RS). This allows of having every TTI an evaluation of the interference scenario since all the eNB are transmitting (simultaneously) the control frame over the respective available bandwidths. We note that, the model does not include the power boosting since it does not reflect any improvement in the implemented model of the channel estimation.

The Sounding Reference Signal (SRS) is modeled similar to the downlink control frame. The SRS is periodically placed in the last symbol of the subframe in the whole system bandwidth. The RRC module already includes an algorithm for dynamically assigning the periodicity as function of the actual number of UEs attached to a eNB according to the UE-specific procedure (see Section 8.2 of [TS36213]).

MAC to Channel delay

To model the latency of real MAC and PHY implementations, the PHY model simulates a MAC-to-channel delay in multiples of TTIs (1ms). The transmission of both data and control packets are delayed by this amount.

CQI feedback

The generation of CQI feedback is done accordingly to what specified in [FFAPI]. In detail, we considered the generation of periodic wideband CQI (i.e., a single value of channel state that is deemed representative of all RBs in use) and inband CQIs (i.e., a set of value representing the channel state for each RB).

The CQI index to be reported is obtained by first obtaining a SINR measurement and then passing this SINR measurement to the *Adaptive Modulation and Coding* module which will map it to the CQI index.

In downlink, the SINR used to generate CQI feedback can be calculated in two different ways:

1. *Ctrl* method: SINR is calculated combining the signal power from the reference signals (which in the simulation is equivalent to the PDCCH) and the interference power from the PDCCH. This approach results in considering any neighboring eNB as an interferer, regardless of whether this eNB is actually performing any PDSCH transmission, and regardless of the power and RBs used for eventual interfering PDSCH transmissions.
2. *Mixed* method: SINR is calculated combining the signal power from the reference signals (which in the simulation is equivalent to the PDCCH) and the interference power from the PDSCH. This approach results in considering as interferers only those neighboring eNBs that are actively transmitting data on the PDSCH, and allows to generate inband CQIs that account for different amounts of interference on different RBs according to the actual interference level. In the case that no PDSCH transmission is performed by any eNB, this method consider that interference is zero, i.e., the SINR will be calculated as the ratio of signal to noise only.

To switch between this two CQI generation approaches, `LteHelper::UsePdschForCqiGeneration` needs to be configured: false for first approach and true for second approach (true is default value):

```
Config::SetDefault("ns3::LteHelper::UsePdschForCqiGeneration", BooleanValue(true));
```

In uplink, two types of CQIs are implemented:

- SRS based, periodically sent by the UEs.
- PUSCH based, calculated from the actual transmitted data.

The scheduler interface include an attribute system called `ULCqiFilter` for managing the filtering of the CQIs according to their nature, in detail:

- `SRS_UL_CQI` for storing only SRS based CQIs.
- `PUSCH_UL_CQI` for storing only PUSCH based CQIs.

It has to be noted that, the `FfMacScheduler` provides only the interface and it is matter of the actual scheduler implementation to include the code for managing these attributes (see scheduler related section for more information on this matter).

Interference Model

The PHY model is based on the well-known Gaussian interference models, according to which the powers of interfering signals (in linear units) are summed up together to determine the overall interference power.

The sequence diagram of Figure *Sequence diagram of the PHY interference calculation procedure* shows how interfering signals are processed to calculate the SINR, and how SINR is then used for the generation of CQI feedback.

LTE Spectrum Model

The usage of the radio spectrum by eNBs and UEs in LTE is described in [TS36101]. In the simulator, radio spectrum usage is modeled as follows. Let f_c denote the LTE Absolute Radio Frequency Channel Number, which identifies the carrier frequency on a 100 kHz raster; furthermore, let B be the Transmission Bandwidth Configuration in number of Resource Blocks. For every pair (f_c, B) used in the simulation we define a corresponding `SpectrumModel` using the functionality provided by the *Spectrum Module*. model using the Spectrum framework described in [Baldo2009]. f_c and B can be configured for every eNB instantiated in the simulation; hence, each eNB can use a different spectrum model. Every UE will automatically use the spectrum model of the eNB it is attached to. Using the `MultiModelSpectrumChannel` described in [Baldo2009], the interference among eNBs that use different spectrum models is properly accounted for. This allows to simulate dynamic spectrum access policies, such as for example the spectrum licensing policies that are discussed in [Ofcom2600MHz].

Data PHY Error Model

The simulator includes an error model of the data plane (i.e., PDSCH and PUSCH) according to the standard link-to-system mapping (LSM) techniques. The choice is aligned with the standard system simulation methodology of OFDMA radio transmission technology. Thanks to LSM we are able to maintain a good level of accuracy and at the same time limiting the computational complexity increase. It is based on the mapping of single link layer performance obtained by means of link level simulators to system (in our case network) simulators. In particular link the layer simulator is used for generating the performance of a single link from a PHY layer perspective, usually in terms of code block error rate (BLER), under specific static conditions. LSM allows the usage of these parameters in more complex scenarios, typical of system/network simulators, where we have more links, interference and “colored” channel propagation phenomena (e.g., frequency selective fading).

To do this the Vienna LTE Simulator [ViennaLteSim] has been used for what concerns the extraction of link layer performance and the Mutual Information Based Effective SINR (MIESM) as LSM mapping function using part of the work recently published by the Signet Group of University of Padua [PaduaPEM].

MIESM

The specific LSM method adopted is the one based on the usage of a mutual information metric, commonly referred to as the mutual information per per coded bit (MIB or MMIB when a mean of multiples MIBs is involved). Another option would be represented by the Exponential ESM (EESM); however, recent studies demonstrate that MIESM outperforms EESM in terms of accuracy [LozanoCost].

The mutual information (MI) is dependent on the constellation mapping and can be calculated per transport block (TB) basis, by evaluating the MI over the symbols and the subcarrier. However, this would be too complex for a network simulator. Hence, in our implementation a flat channel response within the RB has been considered; therefore the overall MI of a TB is calculated averaging the MI evaluated per each RB used in the TB. In detail, the implemented scheme is

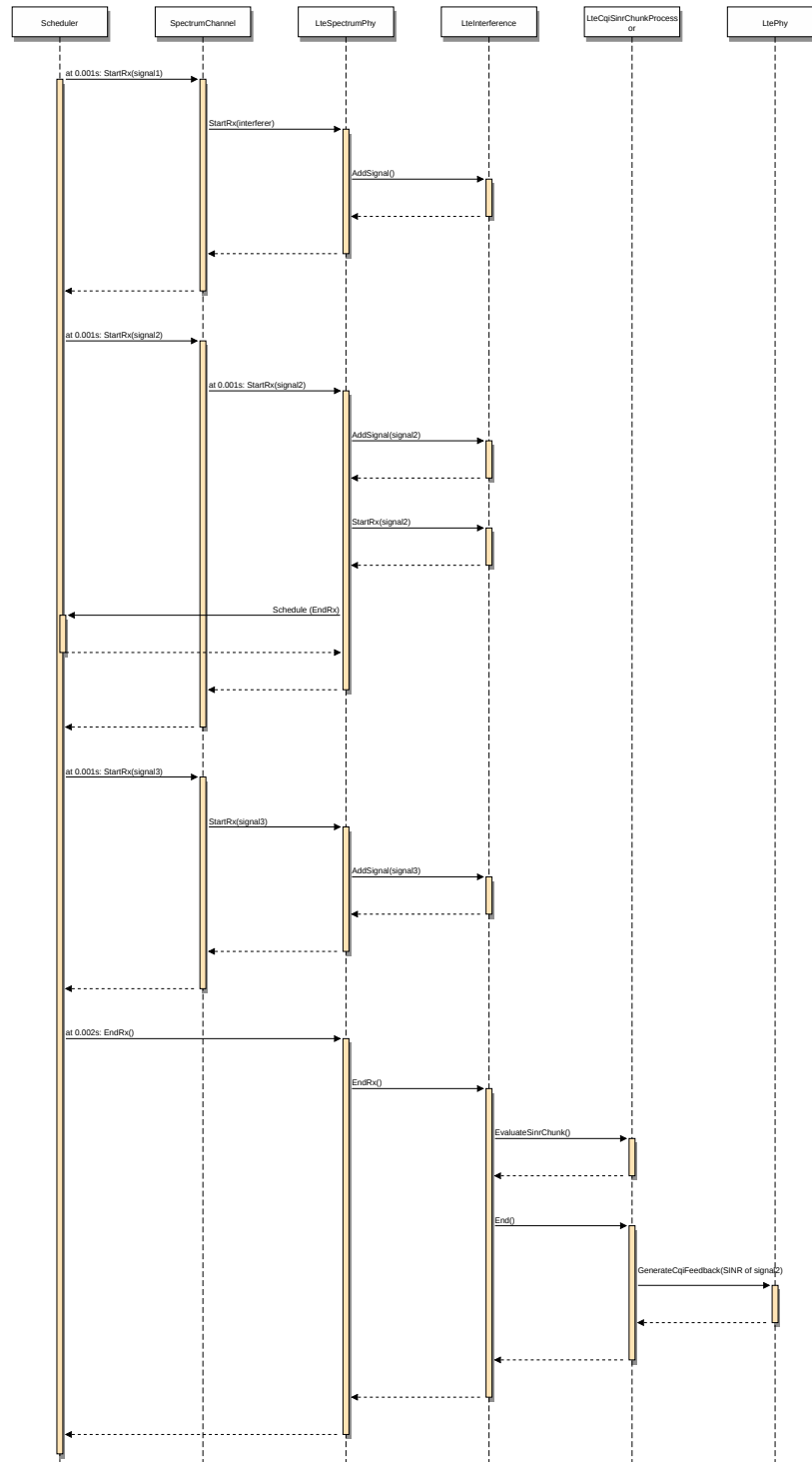


Fig. 19.11: Sequence diagram of the PHY interference calculation procedure

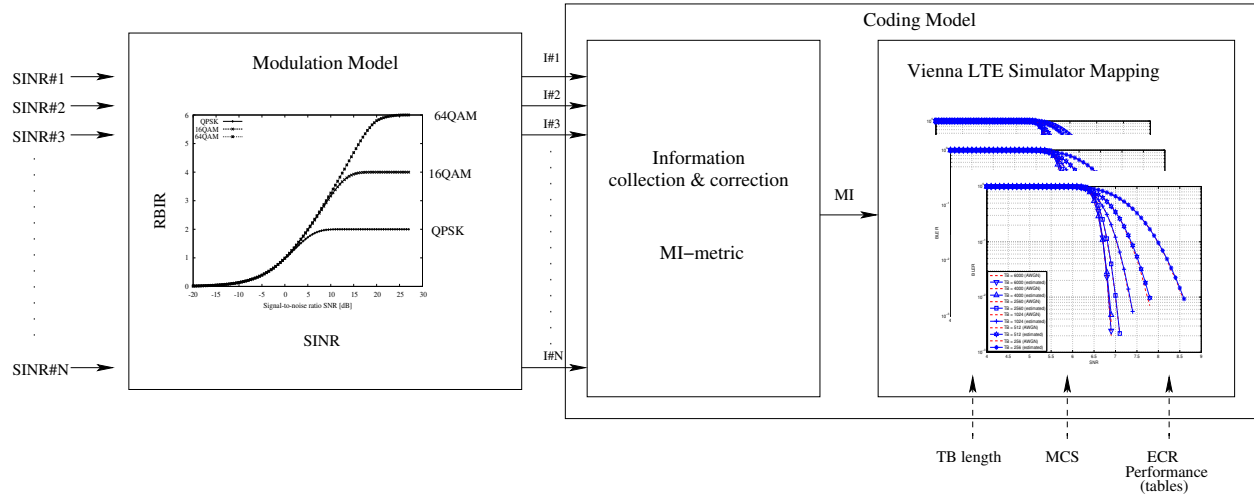


Fig. 19.12: MIESM computational procedure diagram

depicted in Figure *MIESM computational procedure diagram*, where we see that the model starts by evaluating the MI value for each RB, represented in the figure by the SINR samples. Then the equivalent MI is evaluated per TB basis by averaging the MI values. Finally, a further step has to be done since the link level simulator returns the performance of the link in terms of block error rate (BLER) in a additive white gaussian noise (AWGN) channel, where the blocks are the code blocks (CBs) independently encoded/decoded by the turbo encoder. On this matter the standard 3GPP segmentation scheme has been used for estimating the actual CB size (described in section 5.1.2 of [TS36212]). This scheme divides the TB in N_{K-} blocks of size K_- and N_{K+} blocks of size K_+ . Therefore the overall TB BLER (TBBLER) can be expressed as

$$TBBLER = 1 - \prod_{i=1}^C (1 - CBLER_i)$$

where the $CBLER_i$ is the BLER of the CB i obtained according to the link level simulator CB BLER curves. For estimating the $CBLER_i$, the MI evaluation has been implemented according to its numerical approximation defined in [wimaxEmd]. Moreover, for reducing the complexity of the computation, the approximation has been converted into lookup tables. In detail, Gaussian cumulative model has been used for approximating the AWGN BLER curves with three parameters which provides a close fit to the standard AWGN performances, in formula:

$$CBLER_i = \frac{1}{2} \left[1 - \operatorname{erf} \left(\frac{x - b_{ECR}}{\sqrt{2}c_{ECR}} \right) \right]$$

where x is the MI of the TB, b_{ECR} represents the “transition center” and c_{ECR} is related to the “transition width” of the Gaussian cumulative distribution for each Effective Code Rate (ECR) which is the actual transmission rate according to the channel coding and MCS. For limiting the computational complexity of the model we considered only a subset of the possible ECRs in fact we would have potentially 5076 possible ECRs (i.e., 27 MCSs and 188 CB sizes). On this respect, we will limit the CB sizes to some representative values (i.e., 40, 140, 160, 256, 512, 1024, 2048, 4032, 6144), while for the others the worst one approximating the real one will be used (i.e., the smaller CB size value available respect to the real one). This choice is aligned to the typical performance of turbo codes, where the CB size is not strongly impacting on the BLER. However, it is to be notes that for CB sizes lower than 1000 bits the effect might be relevant (i.e., till 2 dB); therefore, we adopt this unbalanced sampling interval for having more precision where it is necessary. This behaviour is confirmed by the figures presented in the Annes Section.

BLER Curves

On this respect, we reused part of the curves obtained within [PaduaPEM]. In detail, we introduced the CB size dependency to the CB BLER curves with the support of the developers of [PaduaPEM] and of the LTE Vienna Simulator. In fact, the module released provides the link layer performance only for what concerns the MCSs (i.e., with a given fixed ECR). In detail the new error rate curves for each has been evaluated with a simulation campaign with the link layer simulator for a single link with AWGN noise and for CB size of 104, 140, 256, 512, 1024, 2048, 4032 and 6144. These curves has been mapped with the Gaussian cumulative model formula presented above for obtaining the correspondents b_{ECR} and c_{ECR} parameters.

The BLER performance of all MCS obtained with the link level simulator are plotted in the following figures (blue lines) together with their correspondent mapping to the Gaussian cumulative distribution (red dashed lines).

Integration of the BLER curves in the ns-3 LTE module

The model implemented uses the curves for the LSM of the recently LTE PHY Error Model released in the ns3 community by the Signet Group [PaduaPEM] and the new ones generated for different CB sizes. The `LteSpectrumPhy` class is in charge of evaluating the TB BLER thanks to the methods provided by the `LteMiErrorModel` class, which is in charge of evaluating the TB BLER according to the vector of the perceived SINR per RB, the MCS and the size in order to proper model the segmentation of the TB in CBs. In order to obtain the vector of the perceived SINRs for data and control signals, two instances of `LteChunkProcessor` (dedicated to evaluate the SINR for obtaining physical error performance) have been attached to UE downlink and eNB uplink `LteSpectrumPhy` modules for evaluating the error model distribution of PDSCH (UE side) and ULSCH (eNB side).

The model can be disabled for working with a zero-losses channel by setting the `DataErrorModelEnabled` attribute of the `LteSpectrumPhy` class (by default is active). This can be done according to the standard ns3 attribute system procedure, that is:

```
Config::SetDefault("ns3::LteSpectrumPhy::DataErrorModelEnabled", BooleanValue(false));
```

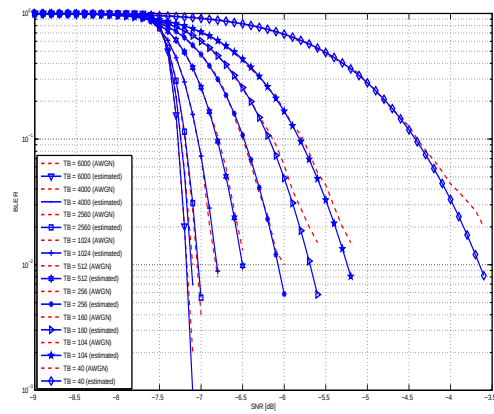
Control Channels PHY Error Model

The simulator includes the error model for downlink control channels (PCFICH and PDCCH), while in uplink it is assumed and ideal error-free channel. The model is based on the MIESM approach presented before for considering the effects of the frequency selective channel since most of the control channels span the whole available bandwidth.

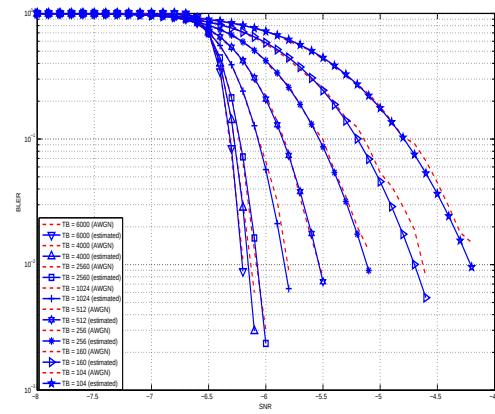
PCFICH + PDCCH Error Model

The model adopted for the error distribution of these channels is based on an evaluation study carried out in the RAN4 of 3GPP, where different vendors investigated the demodulation performance of the PCFICH jointly with PDCCH. This is due to the fact that the PCFICH is the channel in charge of communicating to the UEs the actual dimension of the PDCCH (which spans between 1 and 3 symbols); therefore the correct decodification of the DCIs depends on the correct interpretation of both ones. In 3GPP this problem have been evaluated for improving the cell-edge performance [FujitsuWhitePaper], where the interference among neighboring cells can be relatively high due to signal degradation. A similar problem has been notices in femto-cell scenario and, more in general, in HetNet scenarios the bottleneck has been detected mainly as the PCFICH channel [Bharucha2011], where in case of many eNBs are deployed in the same service area, this channel may collide in frequency, making impossible the correct detection of the PDCCH channel, too.

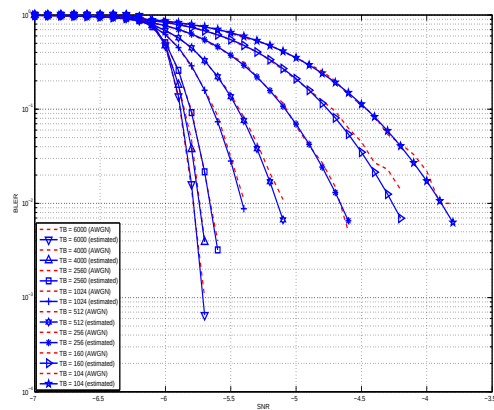
In the simulator, the SINR perceived during the reception has been estimated according to the MIESM model presented above in order to evaluate the error distribution of PCFICH and PDCCH. In detail, the SINR samples of all the RBs are included in the evaluation of the MI associated to the control frame and, according to this values, the effective SINR



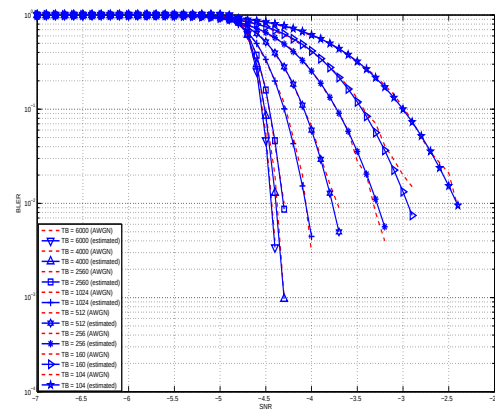
a) MCS1



b) MCS2

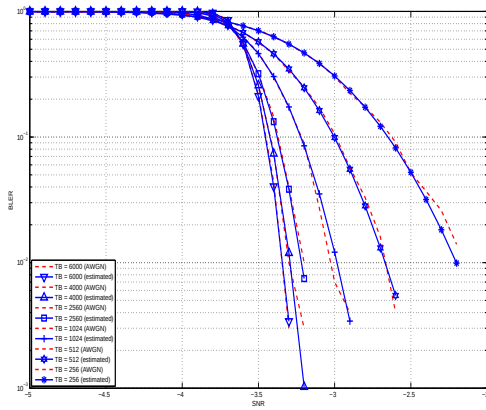


c) MCS3

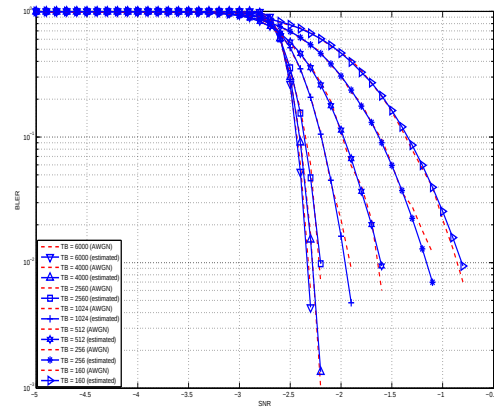


d) MCS4

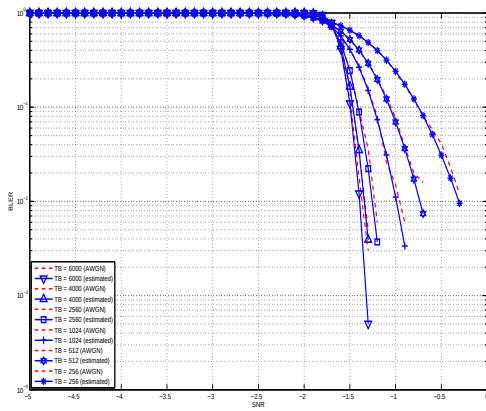
Fig. 19.13: BLER for MCS 1, 2, 3 and 4.



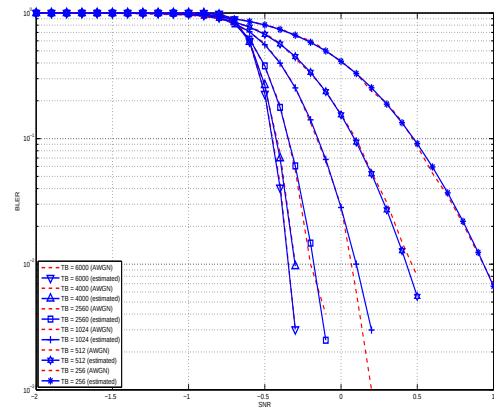
a) MCS5



b) MCS6

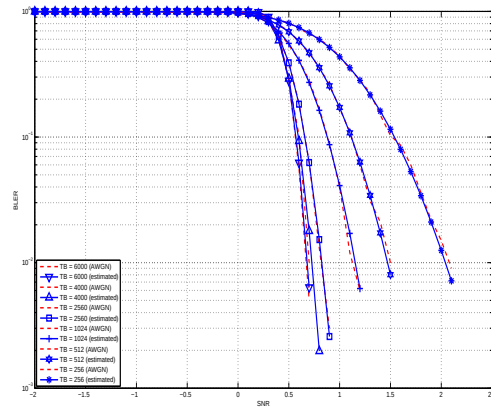


c) MCS7

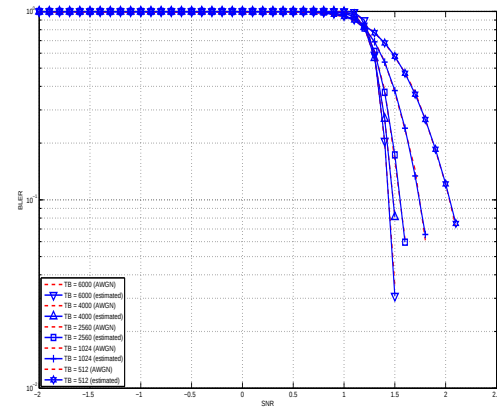


d) MCS8

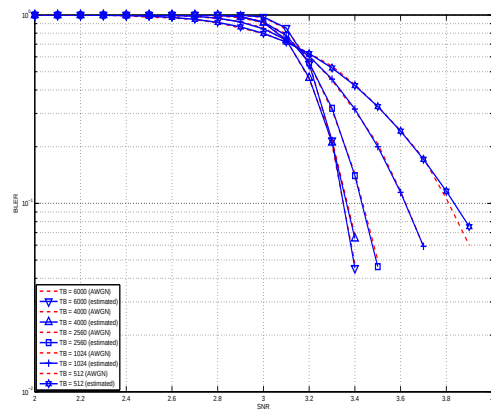
Fig. 19.14: BLER for MCS 5, 6, 7 and 8.



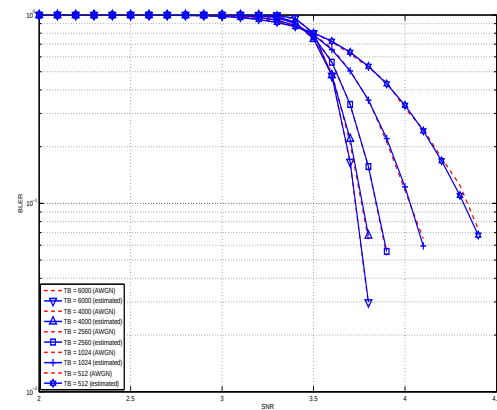
a) MCS9



b) MCS10

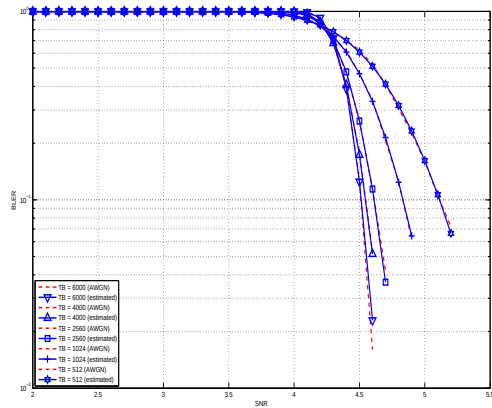


c) MCS11

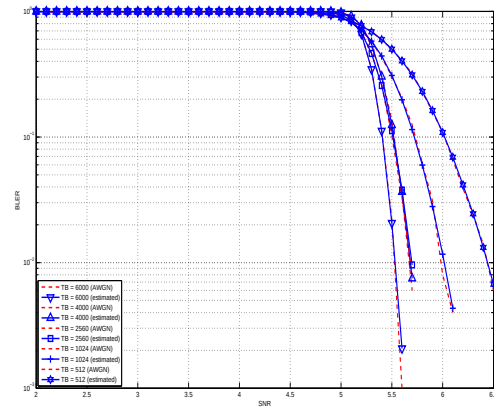


d) MCS12

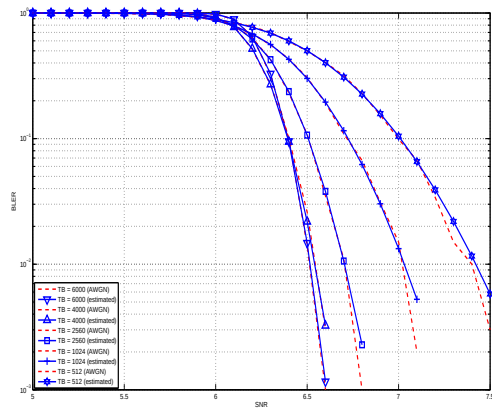
Fig. 19.15: BLER for MCS 9, 10, 11 and 12.



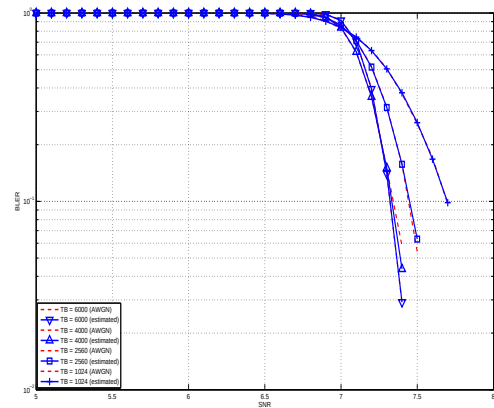
a) MCS13



b) MCS14

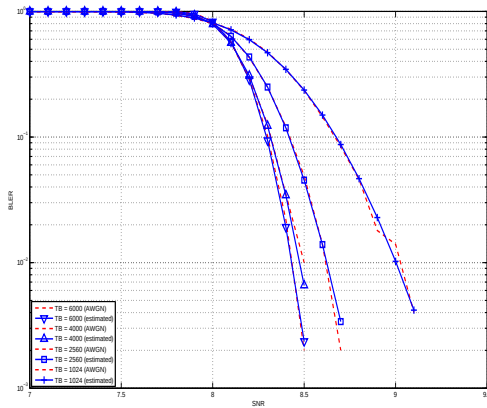


c) MCS15

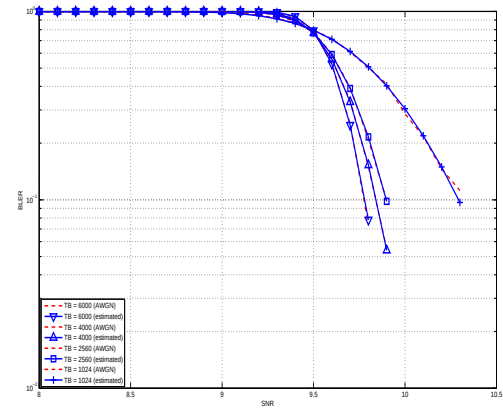


d) MCS16

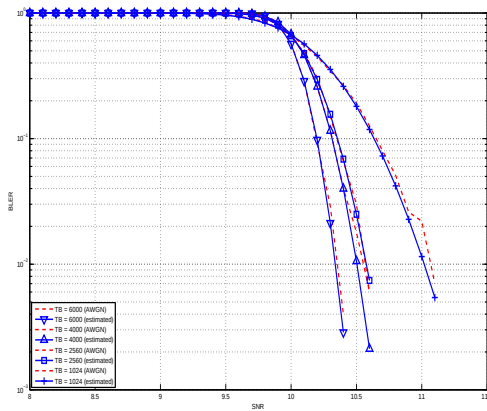
Fig. 19.16: BLER for MCS 13, 14, 15 and 16.



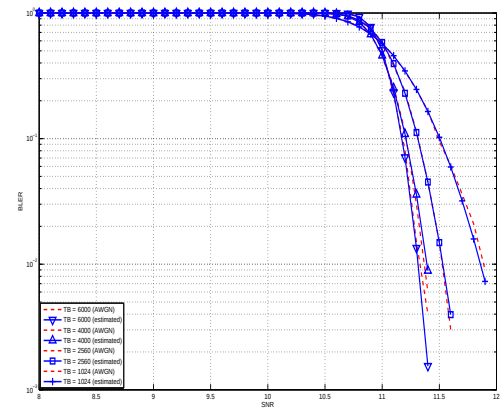
a) MCS17



b) MCS18

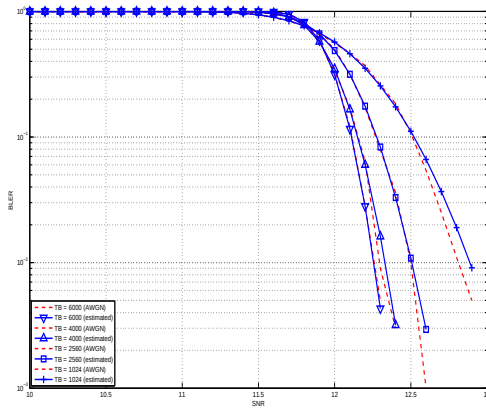


c) MCS19

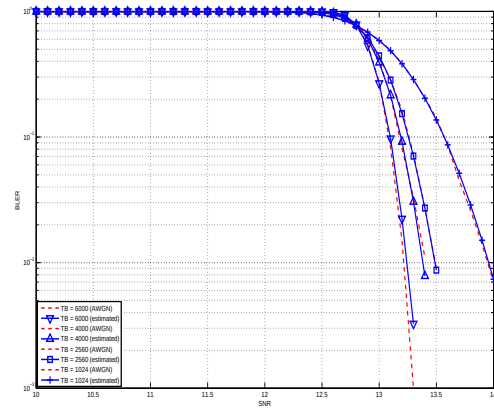


d) MCS20

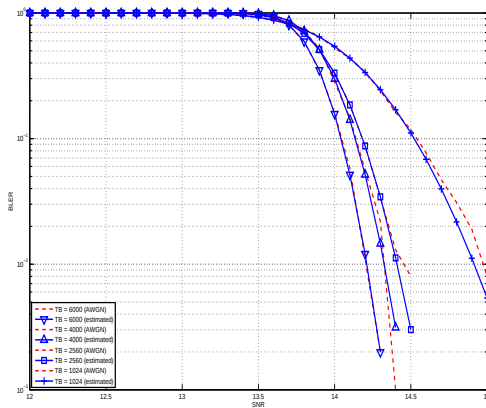
Fig. 19.17: BLER for MCS 17, 17, 19 and 20.



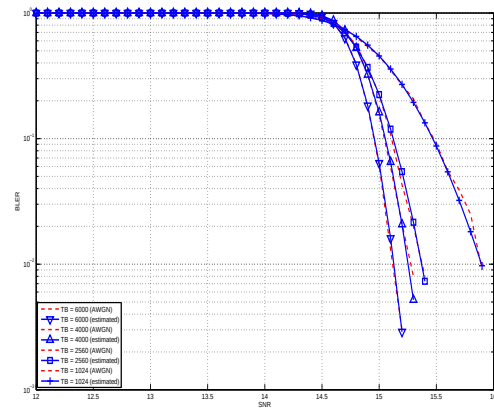
a) MCS21



b) MCS22

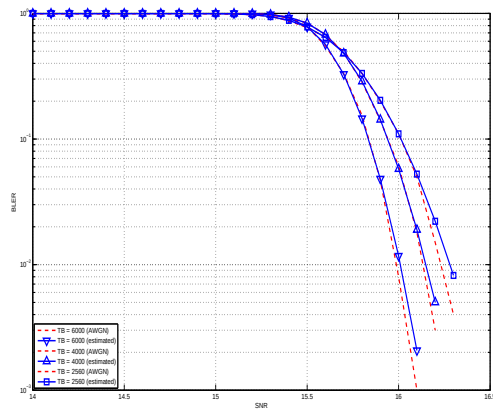


c) MCS23

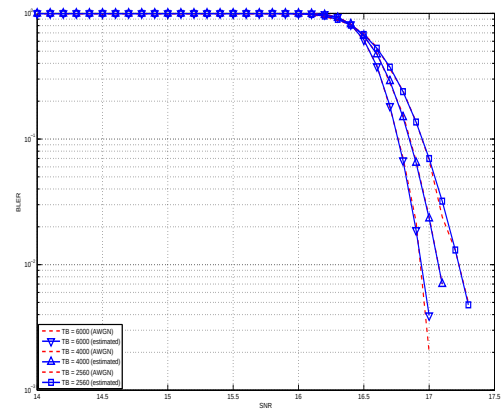


d) MCS24

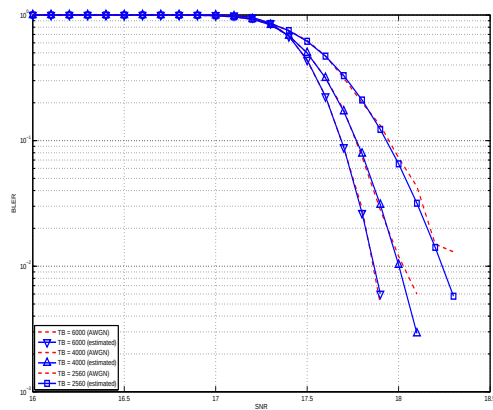
Fig. 19.18: BLER for MCS 21, 22, 23 and 24.



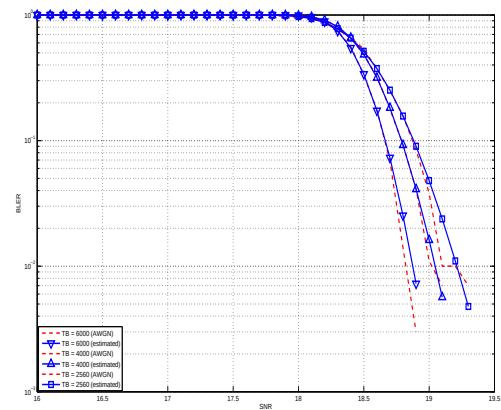
a) MCS25



b) MCS26

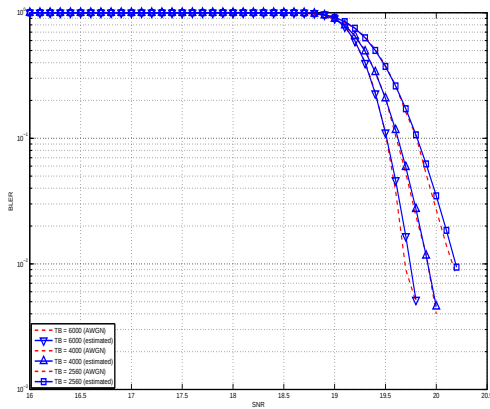


c) MCS27



d) MCS28

Fig. 19.19: BLER for MCS 25, 26, 27 and 28.



a) MCS29

Fig. 19.20: BLER for MCS 29.

(eSINR) is obtained by inverting the MI evaluation process. It has to be noted that, in case of MIMO transmission, both PCFICH and the PDCCH use always the transmit diversity mode as defined by the standard. According to the eSINR perceived the decodification error probability can be estimated as function of the results presented in [R4-081920]. In case an error occur, the DCIs discarded and therefore the UE will be not able to receive the correspondent Tbs, therefore resulting lost.

MIMO Model

The use of multiple antennas both at transmitter and receiver side, known as multiple-input and multiple-output (MIMO), is a problem well studied in literature during the past years. Most of the work concentrate on evaluating analytically the gain that the different MIMO schemes might have in term of capacity; however someones provide also information of the gain in terms of received power [CatreuxMIMO].

According to the considerations above, a model more flexible can be obtained considering the gain that MIMO schemes bring in the system from a statistical point of view. As highlighted before, [CatreuxMIMO] presents the statistical gain of several MIMO solutions respect to the SISO one in case of no correlation between the antennas. In the work the gain is presented as the cumulative distribution function (CDF) of the output SINR for what concern SISO, MIMO-Alamouti, MIMO-MMSE, MIMO-OSIC-MMSE and MIMO-ZF schemes. Elaborating the results, the output SINR distribution can be approximated with a log-normal one with different mean and variance as function of the scheme considered. However, the variances are not so different and they are approximately equal to the one of the SISO mode already included in the shadowing component of the BuildingsPropagationLossModel, in detail:

- SISO: $\mu = 13.5$ and $\sigma = 20$ [dB].
- MIMO-Alamouti: $\mu = 17.7$ and $\sigma = 11.1$ [dB].
- MIMO-MMSE: $\mu = 10.7$ and $\sigma = 16.6$ [dB].
- MIMO-OSIC-MMSE: $\mu = 12.6$ and $\sigma = 15.5$ [dB].
- MIMO-ZF: $\mu = 10.3$ and $\sigma = 12.6$ [dB].

Therefore the PHY layer implements the MIMO model as the gain perceived by the receiver when using a MIMO scheme respect to the one obtained using SISO one. We note that, these gains referred to a case where there is no correlation between the antennas in MIMO scheme; therefore do not model degradation due to paths correlation.

UE PHY Measurements Model

According to [TS36214], the UE has to report a set of measurements of the eNBs that the device is able to perceive: the reference signal received power (RSRP) and the reference signal received quality (RSRQ). The former is a measure of the received power of a specific eNB, while the latter includes also channel interference and thermal noise. The UE has to report the measurements jointly with the physical cell identity (PCI) of the cell. Both the RSRP and RSRQ measurements are performed during the reception of the RS, while the PCI is obtained with the Primary Synchronization Signal (PSS). The PSS is sent by the eNB each 5 subframes and in detail in the subframes 1 and 6. In real systems, only 504 distinct PCIs are available, and hence it could occur that two nearby eNBs use the same PCI; however, in the simulator we model PCIs using simulation metadata, and we allow up to 65535 distinct PCIs, thereby avoiding PCI collisions provided that less than 65535 eNBs are simulated in the same scenario.

According to [TS36133] sections 9.1.4 and 9.1.7, RSRP is reported by PHY layer in dBm while RSRQ in dB. The values of RSRP and RSRQ are provided to higher layers through the C-PHY SAP (by means of UeMeasurementsParameters struct) every 200 ms as defined in [TS36331]. Layer 1 filtering is performed by averaging the all the measurements collected during the last window slot. The periodicity of reporting can be adjusted for research purposes by means of the LteUePhy::UeMeasurementsFilterPeriod attribute.

The formulas of the RSRP and RSRQ can be simplified considering the assumption of the PHY layer that the channel is flat within the RB, the finest level of accuracy. In fact, this implies that all the REs within a RB have the same power,

therefore:

$$RSRP = \frac{\sum_{k=0}^{K-1} \frac{\sum_{m=0}^{M-1} (P(k, m))}{M}}{K} = \frac{\sum_{k=0}^{K-1} \frac{(M \times P(k))}{M}}{K} = \frac{\sum_{k=0}^{K-1} (P(k))}{K}$$

where $P(k, m)$ represents the signal power of the RE m within the RB k , which, as observed before, is constant within the same RB and equal to $P(k)$, M is the number of REs carrying the RS in a RB and K is the number of RBs. It is to be noted that $P(k)$, and in general all the powers defined in this section, is obtained in the simulator from the PSD of the RB (which is provided by the `LteInterferencePowerChunkProcessor`), in detail:

$$P(k) = PSD_{RB}(k) * 180000/12$$

where $PSD_{RB}(k)$ is the power spectral density of the RB k , 180000 is the bandwidth in Hz of the RB and 12 is the number of REs per RB in an OFDM symbol. Similarly, for RSSI we have

$$RSSI = \frac{\sum_{k=0}^{K-1} \frac{\sum_{s=0}^{S-1} \sum_{r=0}^{R-1} (P(k, s, r) + I(k, s, r) + N(k, s, r))}{S}}{S}$$

where S is the number of OFDM symbols carrying RS in a RB and R is the number of REs carrying a RS in a OFDM symbol (which is fixed to 2) while $P(k, s, r)$, $I(k, s, r)$ and $N(k, s, r)$ represent respectively the perceived power of the serving cell, the interference power and the noise power of the RE r in symbol s . As for RSRP, the measurements within a RB are always equals among each others according to the PHY model; therefore $P(k, s, r) = P(k)$, $I(k, s, r) = I(k)$ and $N(k, s, r) = N(k)$, which implies that the RSSI can be calculated as:

$$RSSI = \frac{\sum_{k=0}^{K-1} \frac{S \times 2 \times (P(k) + I(k) + N(k))}{S}}{S} = \frac{\sum_{k=0}^{K-1} 2 \times (P(k) + I(k) + N(k))}{S}$$

Considering the constraints of the PHY reception chain implementation, and in order to maintain the level of computational complexity low, only RSRP can be directly obtained for all the cells. This is due to the fact that `LteSpectrumPhy` is designed for evaluating the interference only respect to the signal of the serving eNB. This implies that the PHY layer is optimized for managing the power signals information with the serving eNB as a reference. However, RSRP and RSRQ of neighbor cell i can be extracted by the current information available of the serving cell j as detailed in the following:

$$\begin{aligned} RSRP_i &= \frac{\sum_{k=0}^{K-1} (P_i(k))}{K} \\ RSSI_i &= RSSI_j = \sum_{k=0}^{K-1} 2 \times (I_j(k) + P_j(k) + N_j(k)) \\ RSRQ_i^j &= K \times RSRP_i / RSSI_j \end{aligned}$$

where $RSRP_i$ is the RSRP of the neighbor cell i , $P_i(k)$ is the power perceived at any RE within the RB k , K is the total number of RBs, $RSSI_i$ is the RSSI of the neighbor cell i when the UE is attached to cell j (which, since it is the sum of all the received powers, coincides with $RSSI_j$), $I_j(k)$ is the total interference perceived by UE in any RE of RB k when attached to cell i (obtained by the `LteInterferencePowerChunkProcessor`), $P_j(k)$ is the power perceived of cell j in any RE of the RB k and N is the power noise spectral density in any RE. The sample is considered as valid in case of the RSRQ evaluated is above the `LteUePhy::RsrqUeMeasThreshold` attribute.

19.1.6 HARQ

The HARQ scheme implemented is based on an incremental redundancy (IR) solution combined with multiple stop-and-wait processes for enabling a continuous data flow. In detail, the solution adopted is the *soft combining hybrid IR Full incremental redundancy* (also called IR Type II), which implies that the retransmissions contain only new information respect to the previous ones. The resource allocation algorithm of the HARQ has been implemented within the respective scheduler classes (i.e., `RrFfMacScheduler` and `PfFfMacScheduler`, refer to their correspondent sections for more info), while the decodification part of the HARQ has been implemented in the `LteSpectrumPhy` and `LteHarqPhy` classes which will be detailed in this section.

According to the standard, the UL retransmissions are synchronous and therefore are allocated 7 ms after the original transmission. On the other hand, for the DL, they are asynchronous and therefore can be allocated in a more flexible way starting from 7 ms and it is a matter of the specific scheduler implementation. The HARQ processes behavior is depicted in Figure:ref:fig-harq-processes-scheme.

At the MAC layer, the HARQ entity residing in the scheduler is in charge of controlling the 8 HARQ processes for generating new packets and managing the retransmissions both for the DL and the UL. The scheduler collects the HARQ feedback from eNB and UE PHY layers (respectively for UL and DL connection) by means of the FF API primitives `SchedULTriggerReq` and `SchedDLTriggerReq`. According to the HARQ feedback and the RLC buffers status, the scheduler generates a set of DCIs including both retransmissions of HARQ blocks received erroneous and new transmissions, in general, giving priority to the former. On this matter, the scheduler has to take into consideration one constraint when allocating the resource for HARQ retransmissions, it must use the same modulation order of the first transmission attempt (i.e., QPSK for $MCS \in [0..9]$, 16QAM for $MCS \in [10..16]$ and 64QAM for $MCS \in [17..28]$). This restriction comes from the specification of the rate matcher in the 3GPP standard [TS36212], where the algorithm fixes the modulation order for generating the different blocks of the redundancy versions.

The PHY Error Model model (i.e., the `LteMiErrorModel` class already presented before) has been extended for considering IR HARQ according to [wimaxEmd], where the parameters for the AWGN curves mapping for MIESM mapping in case of retransmissions are given by:

$$R_{eff} = \frac{X}{\sum_{i=1}^q C_i}$$

$$M_{Ieff} = \frac{\sum_{i=1}^q C_i M_i}{\sum_{i=1}^q C_i}$$

where X is the number of original information bits, C_i are number of coded bits, M_i are the mutual information per HARQ block received on the total number of q retransmissions. Therefore, in order to be able to return the error probability with the error model implemented in the simulator evaluates the R_{eff} and the M_{Ieff} and return the value of error probability of the ECR of the same modulation with closest lower rate respect to the R_{eff} . In order to consider the effect of HARQ retransmissions a new sets of curves have been integrated respect to the standard one used for the original MCS. The new curves are intended for covering the cases when the most conservative MCS of a modulation is used which implies the generation of R_{eff} lower respect to the one of standard MCSs. On this matter the curves for 1, 2 and 3 retransmissions have been evaluated for 10 and 17. For MCS 0 we considered only the first retransmission since the produced code rate is already very conservative (i.e., 0.04) and returns an error rate enough robust for the reception (i.e., the downturn of the BLER is centered around -18 dB). It is to be noted that, the size of first TB transmission has been assumed as containing all the information bits to be coded; therefore X is equal to the size of the first TB sent of a HARQ process. The model assumes that the eventual presence of parity bits in the codewords is already considered in the link level curves. This implies that as soon as the minimum R_{eff} is reached the model is not including the gain due to the transmission of further parity bits.

The part of HARQ devoted to manage the decodification of the HARQ blocks has been implemented in the `LteHarqPhy` and `LteSpectrumPhy` classes. The former is in charge of maintaining the HARQ information for each active process. The latter interacts with `LteMiErrorModel` class for evaluating the correctness of the blocks received and includes

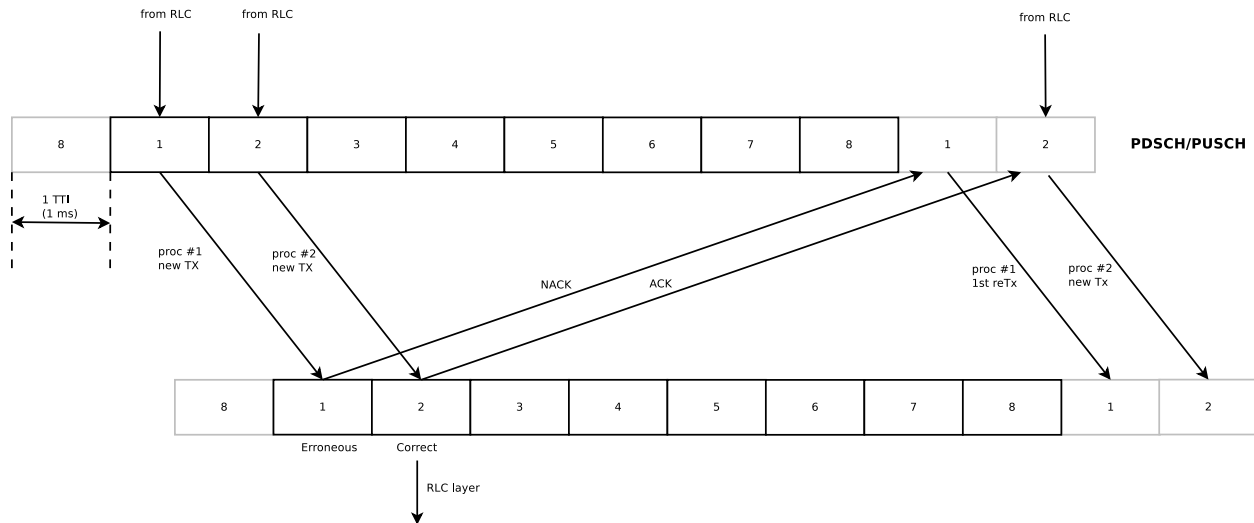


Fig. 19.21: HARQ processes behavior in LTE

the messaging algorithm in charge of communicating to the HARQ entity in the scheduler the result of the decodings. These messages are encapsulated in the `dlInfoListElement` for DL and `ulInfoListElement` for UL and sent through the PUCCH and the PHICH respectively with an ideal error free model according to the assumptions in their implementation. A sketch of the iteration between HARQ and LTE protocol stack is represented in Figure:ref:fig-harq-architecture.

Finally, the HARQ engine is always active both at MAC and PHY layer; however, in case of the scheduler does not support HARQ the system will continue to work with the HARQ functions inhibited (i.e., buffers are filled but not used). This implementation characteristic gives backward compatibility with schedulers implemented before HARQ integration.

19.1.7 MAC

Resource Allocation Model

We now briefly describe how resource allocation is handled in LTE, clarifying how it is modeled in the simulator. The scheduler is in charge of generating specific structures called *Data Control Indication* (DCI) which are then transmitted by the PHY of the eNB to the connected UEs, in order to inform them of the resource allocation on a per subframe basis. In doing this in the downlink direction, the scheduler has to fill some specific fields of the DCI structure with all the information, such as: the Modulation and Coding Scheme (MCS) to be used, the MAC Transport Block (TB) size, and the allocation bitmap which identifies which RBs will contain the data transmitted by the eNB to each user.

For the mapping of resources to physical RBs, we adopt a *localized mapping* approach (see [Sesia2009], Section 9.2.2.1); hence in a given subframe each RB is always allocated to the same user in both slots. The allocation bitmap can be coded in different formats; in this implementation, we considered the *Allocation Type 0* defined in [TS36213], according to which the RBs are grouped in Resource Block Groups (RBG) of different size determined as a function of the Transmission Bandwidth Configuration in use.

For certain bandwidth values not all the RBs are usable, since the group size is not a common divisor of the group. This is for instance the case when the bandwidth is equal to 25 RBs, which results in a RBG size of 2 RBs, and therefore 1 RB will result not addressable. In uplink the format of the DCIs is different, since only adjacent RBs can be used because of the SC-FDMA modulation. As a consequence, all RBs can be allocated by the eNB regardless of the bandwidth configuration.

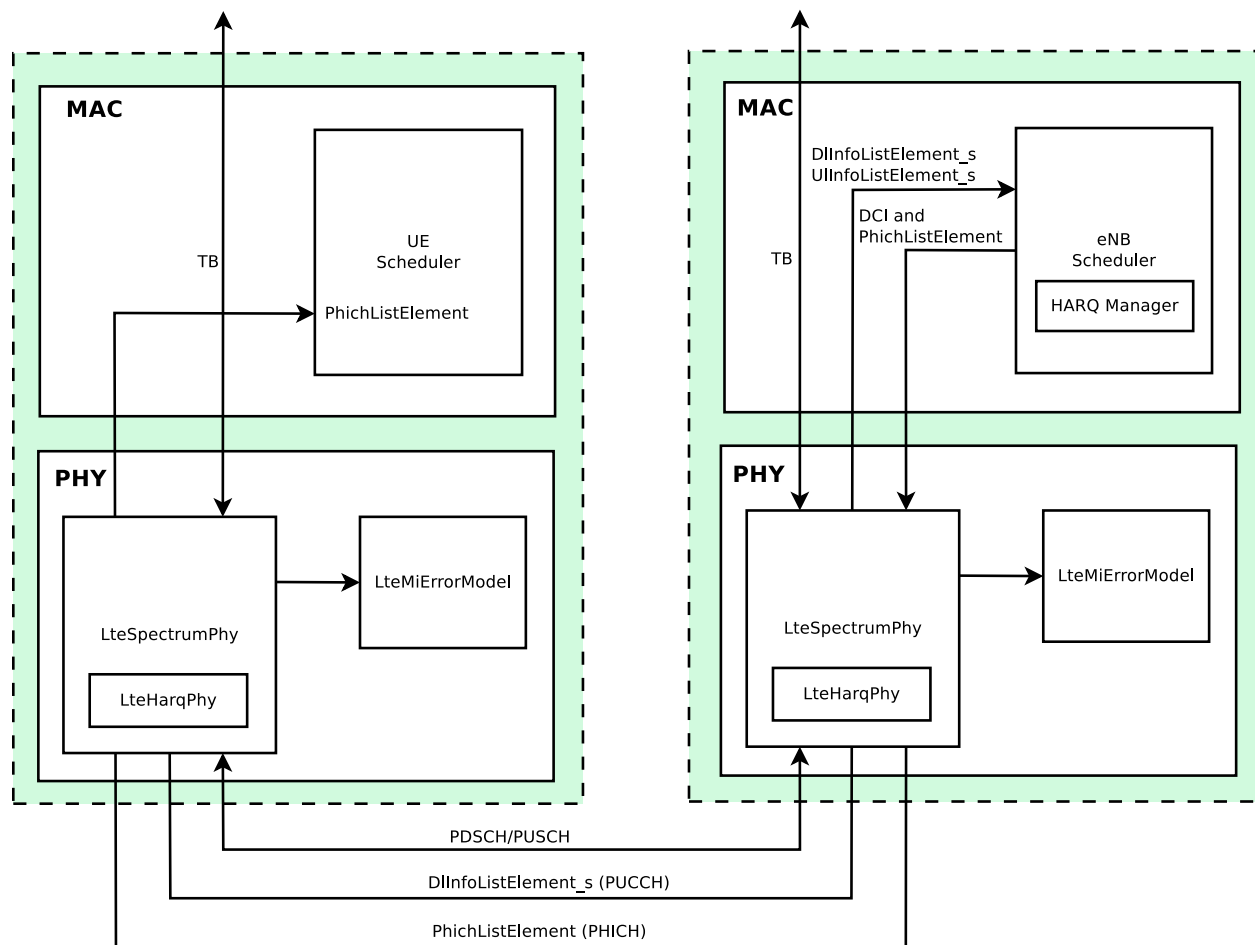


Fig. 19.22: Interaction between HARQ and LTE protocol stack

Adaptive Modulation and Coding

The simulator provides two Adaptive Modulation and Coding (AMC) models: one based on the GSoC model [Piro2011] and one based on the physical error model (described in the following sections).

The former model is a modified version of the model described in [Piro2011], which in turn is inspired from [Seo2004]. Our version is described in the following. Let i denote the generic user, and let γ_i be its SINR. We get the spectral efficiency η_i of user i using the following equations:

$$\begin{aligned} \text{BER} &= 0.00005 \\ \Gamma &= \frac{-\ln(5 * \text{BER})}{1.5} \\ \eta_i &= \log_2 \left(1 + \frac{\gamma_i}{\Gamma} \right) \end{aligned}$$

The procedure described in [R1-081483] is used to get the corresponding MCS scheme. The spectral efficiency is quantized based on the channel quality indicator (CQI), rounding to the lowest value, and is mapped to the corresponding MCS scheme.

Finally, we note that there are some discrepancies between the MCS index in [R1-081483] and that indicated by the standard: [TS36213] Table 7.1.7.1-1 says that the MCS index goes from 0 to 31, and 0 appears to be a valid MCS scheme (TB size is not 0) but in [R1-081483] the first useful MCS index is 1. Hence to get the value as intended by the standard we need to subtract 1 from the index reported in [R1-081483].

The alternative model is based on the physical error model developed for this simulator and explained in the following subsections. This scheme is able to adapt the MCS selection to the actual PHY layer performance according to the specific CQI report. According to their definition, a CQI index is assigned when a single PDSCH TB with the modulation coding scheme and code rate correspondent to that CQI index in table 7.2.3-1 of [TS36213] can be received with an error probability less than 0.1. In case of wideband CQIs, the reference TB includes all the RBGs available in order to have a reference based on the whole available resources; while, for subband CQIs, the reference TB is sized as the RBGs.

Transport Block model

The model of the MAC Transport Blocks (TBs) provided by the simulator is simplified with respect to the 3GPP specifications. In particular, a simulator-specific class (PacketBurst) is used to aggregate MAC SDUs in order to achieve the simulator's equivalent of a TB, without the corresponding implementation complexity. The multiplexing of different logical channels to and from the RLC layer is performed using a dedicated packet tag (LteRadioBearerTag), which performs a functionality which is partially equivalent to that of the MAC headers specified by 3GPP.

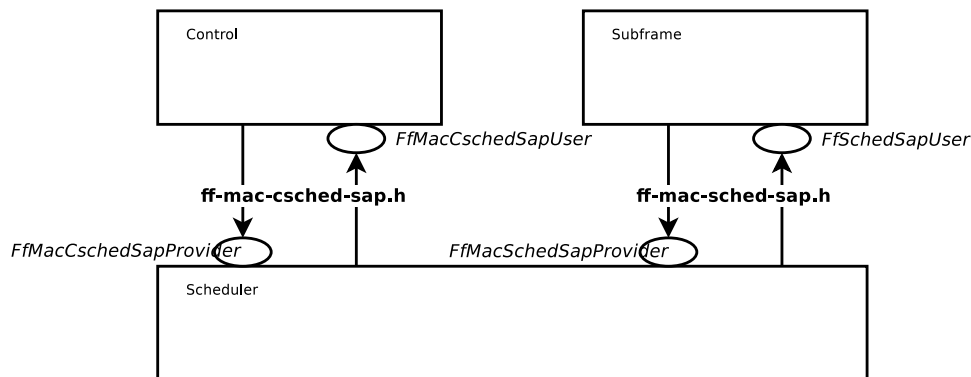
The FemtoForum MAC Scheduler Interface

This section describes the ns-3 specific version of the LTE MAC Scheduler Interface Specification published by the FemtoForum [FFAPI].

We implemented the ns-3 specific version of the FemtoForum MAC Scheduler Interface [FFAPI] as a set of C++ abstract classes; in particular, each primitive is translated to a C++ method of a given class. The term *implemented* here is used with the same meaning adopted in [FFAPI], and hence refers to the process of translating the logical interface specification to a particular programming language. The primitives in [FFAPI] are grouped in two groups: the CSCHED primitives, which deal with scheduler configuration, and the SCHED primitives, which deal with the execution of the scheduler. Furthermore, [FFAPI] defines primitives of two different kinds: those of type REQ go from the MAC to the Scheduler, and those of type IND/CNF go from the scheduler to the MAC. To translate these characteristics into C++, we define the following abstract classes that implement Service Access Points (SAPs) to be used to issue the primitives:

- the `FfMacSchedSapProvider` class defines all the C++ methods that correspond to SCHED primitives of type REQ;
- the `FfMacSchedSapUser` class defines all the C++ methods that correspond to SCHED primitives of type CNF/IND;
- the `FfMacCschedSapProvider` class defines all the C++ methods that correspond to CSCHED primitives of type REQ;
- the `FfMacCschedSapUser` class defines all the C++ methods that correspond to CSCHED primitives of type CNF/IND;

There are 3 blocks involved in the MAC Scheduler interface: Control block, Subframe block and Scheduler block. Each of these blocks provide one part of the MAC Scheduler interface. The figure below shows the relationship between the blocks and the SAPs defined in our implementation of the MAC Scheduler Interface.

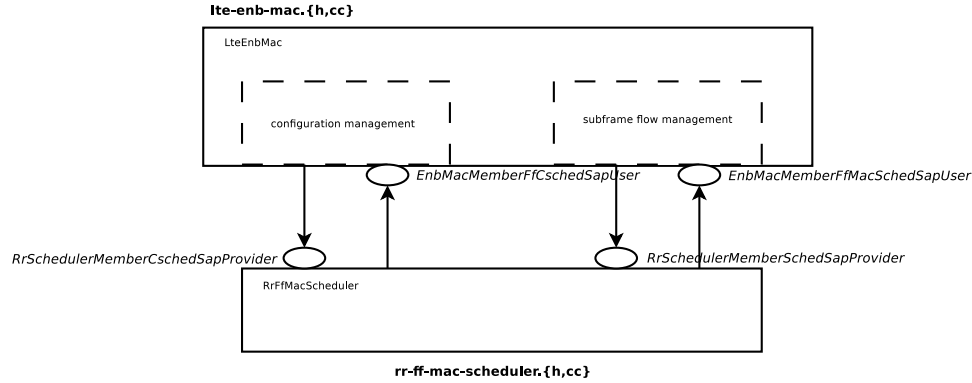


In addition to the above principles, the following design choices have been taken:

- The definition of the MAC Scheduler interface classes follows the naming conventions of the *ns-3* Coding Style. In particular, we follow the CamelCase convention for the primitive names. For example, the primitive `CSCHED_CELL_CONFIG_REQ` is translated to `CschedCellConfigReq` in the *ns-3* code.
- The same naming conventions are followed for the primitive parameters. As the primitive parameters are member variables of classes, they are also prefixed with a `m_`.
- regarding the use of vectors and lists in data structures, we note that [FFAPI] is a pretty much C-oriented API. However, considered that C++ is used in *ns-3*, and that the use of C arrays is discouraged, we used STL vectors (`std::vector`) for the implementation of the MAC Scheduler Interface, instead of using C arrays as implicitly suggested by the way [FFAPI] is written.
- In C++, members with constructors and destructors are not allowed in unions. Hence all those data structures that are said to be unions in [FFAPI] have been defined as structs in our code.

The figure below shows how the MAC Scheduler Interface is used within the eNB.

The User side of both the CSCHED SAP and the SCHED SAP are implemented within the eNB MAC, i.e., in the file `lte-enb-mac.cc`. The eNB MAC can be used with different scheduler implementations without modifications. The same figure also shows, as an example, how the Round Robin Scheduler is implemented: to interact with the MAC of the eNB, the Round Robin scheduler implements the Provider side of the SCHED SAP and CSCHED SAP interfaces. A similar approach can be used to implement other schedulers as well. A description of each of the scheduler implementations that we provide as part of our LTE simulation module is provided in the following subsections.



Round Robin (RR) Scheduler

The Round Robin (RR) scheduler is probably the simplest scheduler found in the literature. It works by dividing the available resources among the active flows, i.e., those logical channels which have a non-empty RLC queue. If the number of RBGs is greater than the number of active flows, all the flows can be allocated in the same subframe. Otherwise, if the number of active flows is greater than the number of RBGs, not all the flows can be scheduled in a given subframe; then, in the next subframe the allocation will start from the last flow that was not allocated. The MCS to be adopted for each user is done according to the received wideband CQIs.

For what concern the HARQ, RR implements the non adaptive version, which implies that in allocating the retransmission attempts RR uses the same allocation configuration of the original block, which means maintaining the same RBGs and MCS. UEs that are allocated for HARQ retransmissions are not considered for the transmission of new data in case they have a transmission opportunity available in the same TTI. Finally, HARQ can be disabled with ns3 attribute system for maintaining backward compatibility with old test cases and code, in detail:

```
Config::SetDefault("ns3::RrFfMacScheduler::HarqEnabled", BooleanValue(false));
```

The scheduler implements the filtering of the uplink CQIs according to their nature with **ULCqiFilter** attribute, in detail:

- **SRS_UL_CQI**: only SRS based CQI are stored in the internal attributes.
- **PUSCH_UL_CQI**: only PUSCH based CQI are stored in the internal attributes.

Proportional Fair (PF) Scheduler

The Proportional Fair (PF) scheduler [Sesia2009] works by scheduling a user when its instantaneous channel quality is high relative to its own average channel condition over time. Let i, j denote generic users; let t be the subframe index, and k be the resource block index; let $M_{i,k}(t)$ be MCS usable by user i on resource block k according to what reported by the AMC model (see *Adaptive Modulation and Coding*); finally, let $S(M, B)$ be the TB size in bits as defined in [TS36213] for the case where a number B of resource blocks is used. The achievable rate $R_i(k, t)$ in bit/s for user i on resource block group k at subframe t is defined as

$$R_i(k, t) = \frac{S(M_{i,k}(t), 1)}{\tau}$$

where τ is the TTI duration. At the start of each subframe t , each RBG is assigned to a certain user. In detail, the index $\hat{i}_k(t)$ to which RBG k is assigned at time t is determined as

$$\hat{i}_k(t) = \underset{j=1, \dots, N}{\operatorname{argmax}} \left(\frac{R_j(k, t)}{T_j(t)} \right)$$

where $T_j(t)$ is the past throughput performance perceived by the user j . According to the above scheduling algorithm, a user can be allocated to different RBGs, which can be either adjacent or not, depending on the current condition of the channel and the past throughput performance $T_j(t)$. The latter is determined at the end of the subframe t using the following exponential moving average approach:

$$T_j(t) = (1 - \frac{1}{\alpha})T_j(t-1) + \frac{1}{\alpha}\hat{T}_j(t)$$

where α is the time constant (in number of subframes) of the exponential moving average, and $\hat{T}_j(t)$ is the actual throughput achieved by the user i in the subframe t . $\hat{T}_j(t)$ is measured according to the following procedure. First we determine the MCS $\hat{M}_j(t)$ actually used by user j :

$$\hat{M}_j(t) = \min_{k:\hat{i}_k(t)=j} M_{j,k}(t)$$

then we determine the total number $\hat{B}_j(t)$ of RBGs allocated to user j :

$$\hat{B}_j(t) = |\{k : \hat{i}_k(t) = j\}|$$

where $|\cdot|$ indicates the cardinality of the set; finally,

$$\hat{T}_j(t) = \frac{S(\hat{M}_j(t), \hat{B}_j(t))}{\tau}$$

For what concern the HARQ, PF implements the non adaptive version, which implies that in allocating the retransmission attempts the scheduler uses the same allocation configuration of the original block, which means maintaining the same RBGs and MCS. UEs that are allocated for HARQ retransmissions are not considered for the transmission of new data in case they have a transmission opportunity available in the same TTI. Finally, HARQ can be disabled with ns3 attribute system for maintaining backward compatibility with old test cases and code, in detail:

```
Config::SetDefault("ns3::PffMacScheduler::HarqEnabled", BooleanValue(false));
```

Maximum Throughput (MT) Scheduler

The Maximum Throughput (MT) scheduler [FCapo2012] aims to maximize the overall throughput of eNB. It allocates each RB to the user that can achieve the maximum achievable rate in the current TTI. Currently, MT scheduler in NS-3 has two versions: frequency domain (FDMT) and time domain (TDMT). In FDMT, every TTI, MAC scheduler allocates RBGs to the UE who has highest achievable rate calculated by subband CQI. In TDMT, every TTI, MAC scheduler selects one UE which has highest achievable rate calculated by wideband CQI. Then MAC scheduler allocates all RBGs to this UE in current TTI. The calculation of achievable rate in FDMT and TDMT is as same as the one in PF. Let i, j denote generic users; let t be the subframe index, and k be the resource block index; let $M_{i,k}(t)$ be MCS usable by user i on resource block k according to what reported by the AMC model (see *Adaptive Modulation and Coding*); finally, let $S(M, B)$ be the TB size in bits as defined in [TS36213] for the case where a number B of resource blocks is used. The achievable rate $R_i(k, t)$ in bit/s for user i on resource block k at subframe t is defined as

$$R_i(k, t) = \frac{S(M_{i,k}(t), 1)}{\tau}$$

where τ is the TTI duration. At the start of each subframe t , each RB is assigned to a certain user. In detail, the index $\hat{i}_k(t)$ to which RB k is assigned at time t is determined as

$$\hat{i}_k(t) = \operatorname{argmax}_{j=1, \dots, N} (R_j(k, t))$$

When there are several UEs having the same achievable rate, current implementation always selects the first UE created in script. Although MT can maximize cell throughput, it cannot provide fairness to UEs in poor channel condition.

Throughput to Average (TTA) Scheduler

The Throughput to Average (TTA) scheduler [FCapo2012] can be considered as an intermediate between MT and PF. The metric used in TTA is calculated as follows:

$$\hat{i}_k(t) = \operatorname{argmax}_{j=1,\dots,N} \left(\frac{R_j(k,t)}{R_j(t)} \right)$$

Here, $R_i(k,t)$ in bit/s represents the achievable rate for user i on resource block k at subframe t . The calculation method already is shown in MT and PF. Meanwhile, $R_i(t)$ in bit/s stands for the achievable rate for i at subframe t . The difference between those two achievable rates is how to get MCS. For $R_i(k,t)$, MCS is calculated by subband CQI while $R_i(t)$ is calculated by wideband CQI. TTA scheduler can only be implemented in frequency domain (FD) because the achievable rate of particular RBG is only related to FD scheduling.

Blind Average Throughput Scheduler

The Blind Average Throughput scheduler [FCapo2012] aims to provide equal throughput to all UEs under eNB. The metric used in TTA is calculated as follows:

$$\hat{i}_k(t) = \operatorname{argmax}_{j=1,\dots,N} \left(\frac{1}{T_j(t)} \right)$$

where $T_j(t)$ is the past throughput performance perceived by the user j and can be calculated by the same method in PF scheduler. In the time domain blind average throughput (TD-BET), the scheduler selects the UE with largest priority metric and allocates all RBGs to this UE. On the other hand, in the frequency domain blind average throughput (FD-BET), every TTI, the scheduler first selects one UE with lowest pastAverageThroughput (largest priority metric). Then scheduler assigns one RBG to this UE, it calculates expected throughput of this UE and uses it to compare with past average throughput $T_j(t)$ of other UEs. The scheduler continues to allocate RBG to this UE until its expected throughput is not the smallest one among past average throughput $T_j(t)$ of all UE. Then the scheduler will use the same way to allocate RBG for a new UE which has the lowest past average throughput $T_j(t)$ until all RBGs are allocated to UEs. The principle behind this is that, in every TTI, the scheduler tries the best to achieve the equal throughput among all UEs.

Token Bank Fair Queue Scheduler

Token Bank Fair Queue (TBFQ) is a QoS aware scheduler which derives from the leaky-bucket mechanism. In TBFQ, a traffic flow of user i is characterized by following parameters:

- t_i : packet arrival rate (byte/sec)
- r_i : token generation rate (byte/sec)
- p_i : token pool size (byte)
- E_i : counter that records the number of token borrowed from or given to the token bank by flow i ; E_i can be smaller than zero

Each K bytes data consumes k tokens. Also, TBFQ maintains a shared token bank (B) so as to balance the traffic between different flows. If token generation rate r_i is bigger than packet arrival rate t_i , then tokens overflowing from token pool are added to the token bank, and E_i is increased by the same amount. Otherwise, flow i needs to withdraw tokens from token bank based on a priority metric $\frac{r_i}{E_i}$, and E_i is decreased. Obviously, the user contributes more on token bank has higher priority to borrow tokens; on the other hand, the user borrows more tokens from bank has lower priority to continue to withdraw tokens. Therefore, in case of several users having the same token generation rate, traffic rate and token pool size, user suffers from higher interference has more opportunity to borrow tokens from bank. In addition, TBFQ can police the traffic by setting the token generation rate to limit the throughput. Additionally, TBFQ also maintains following three parameters for each flow:

- Debt limit d_i : if E_i is below this threshold, user i cannot further borrow tokens from bank. This is for preventing malicious UE to borrow too much tokens.
- Credit limit c_i : the maximum number of tokens UE i can borrow from the bank in one time.
- Credit threshold C : once E_i reaches debt limit, UE i must store C tokens to bank in order to further borrow token from bank.

LTE in NS-3 has two versions of TBFQ scheduler: frequency domain TBFQ (FD-TBFQ) and time domain TBFQ (TD-TBFQ). In FD-TBFQ, the scheduler always select UE with highest metric and allocates RBG with highest subband CQI until there are no packets within UE's RLC buffer or all RBGs are allocated [FABokhari2009]. In TD-TBFQ, after selecting UE with maximum metric, it allocates all RBGs to this UE by using wideband CQI [WKWong2004].

Priority Set Scheduler

Priority set scheduler (PSS) is a QoS aware scheduler which combines time domain (TD) and frequency domain (FD) packet scheduling operations into one scheduler [GMonghal2008]. It controls the fairness among UEs by a specified Target Bit Rate (TBR).

In TD scheduler part, PSS first selects UEs with non-empty RLC buffer and then divide them into two sets based on the TBR:

- set 1: UE whose past average throughput is smaller than TBR; TD scheduler calculates their priority metric in Blind Equal Throughput (BET) style:

$$\hat{i}_k(t) = \operatorname{argmax}_{j=1,\dots,N} \left(\frac{1}{T_j(t)} \right)$$

- set 2: UE whose past average throughput is larger (or equal) than TBR; TD scheduler calculates their priority metric in Proportional Fair (PF) style:

$$\hat{i}_k(t) = \operatorname{argmax}_{j=1,\dots,N} \left(\frac{R_j(k,t)}{T_j(t)} \right)$$

UEs belonged to set 1 have higher priority than ones in set 2. Then PSS will select N_{mux} UEs with highest metric in two sets and forward those UE to FD scheduler. In PSS, FD scheduler allocates RBG k to UE n that maximums the chosen metric. Two PF schedulers are used in PF scheduler:

- Proportional Fair scheduled (PFsch)

$$\widehat{Msch}_k(t) = \operatorname{argmax}_{j=1,\dots,N} \left(\frac{R_j(k,t)}{Tsch_j(t)} \right)$$

- Carrier over Interference to Average (CoIta)

$$\widehat{Mcoi}_k(t) = \operatorname{argmax}_{j=1,\dots,N} \left(\frac{CoI[j,k]}{\sum_{k=0}^{N_{RBG}} CoI[j,k]} \right)$$

where $Tsch_j(t)$ is similar past throughput performance perceived by the user j , with the difference that it is updated only when the i -th user is actually served. $CoI[j,k]$ is an estimation of the SINR on the RBG k of UE j . Both PFsch and CoIta is for decoupling FD metric from TD scheduler. In addition, PSS FD scheduler also provide a weight metric $W[n]$ for helping controlling fairness in case of low number of UEs.

$$W[n] = \max\left(1, \frac{TBR}{T_j(t)}\right)$$

where $T_j(t)$ is the past throughput performance perceived by the user j . Therefore, on RBG k , the FD scheduler selects the UE j that maximizes the product of the frequency domain metric ($Msch$, $MCoI$) by weight $W[n]$. This strategy will guarantee the throughput of lower quality UE tend towards the TBR.


```
Config::SetDefault("ns3::PffMacScheduler::HarqEnabled", BooleanValue(false));
```

The scheduler implements the filtering of the uplink CQIs according to their nature with `ULCqiFilter` attribute, in detail:

- SRS_UL_CQI: only SRS based CQI are stored in the internal attributes.
- PUSCH_UL_CQI: only PUSCH based CQI are stored in the internal attributes.

Channel and QoS Aware Scheduler

The Channel and QoS Aware (CQA) Scheduler [Bbojovic2014] is an LTE MAC downlink scheduling algorithm that considers the head of line (HOL) delay, the GBR parameters and channel quality over different subbands. The CQA scheduler is based on joint TD and FD scheduling.

In the TD (at each TTI) the CQA scheduler groups users by priority. The purpose of grouping is to enforce the FD scheduling to consider first the flows with highest HOL delay. The grouping metric m_{td} for user $j = 1, \dots, N$ is defined in the following way:

$$m_{td}^j(t) = \lceil \frac{d_{hol}^j(t)}{g} \rceil ,$$

where $d_{hol}^j(t)$ is the current value of HOL delay of flow j , and g is a grouping parameter that determines granularity of the groups, i.e. the number of the flows that will be considered in the FD scheduling iteration.

The groups of flows selected in the TD iteration are forwarded to the FD scheduling starting from the flows with the highest value of the m_{td} metric until all RBGs are assigned in the corresponding TTI. In the FD, for each RBG $k = 1, \dots, K$, the CQA scheduler assigns the current RBG to the user j that has the maximum value of the FD metric which we define in the following way:

$$m_{fd}^{(k,j)}(t) = d_{HOL}^j(t) \cdot m_{GBR}^j(t) \cdot m_{ca}^{k,j}(t) ,$$

where $m_{GBR}^j(t)$ is calculated as follows:

$$m_{GBR}^j(t) = \frac{GBR^j}{\bar{R}^j(t)} = \frac{GBR^j}{(1 - \alpha) \cdot \bar{R}^j(t - 1) + \alpha \cdot r^j(t)} ,$$

where GBR^j is the bit rate specified in EPS bearer of the flow j , $\bar{R}^j(t)$ is the past averaged throughput that is calculated with a moving average, $r^j(t)$ is the throughput achieved at the time t , and α is a coefficient such that $0 \leq \alpha \leq 1$.

For $m_{ca}^{(k,j)}(t)$ we consider two different metrics: $m_{pf}^{(k,j)}(t)$ and $m_{ff}^{(k,j)}(t)$. m_{pf} is the Proportional Fair metric which is defined as follows:

$$m_{pf}^{(k,j)}(t) = \frac{R_e^{(k,j)}}{\bar{R}^j(t)} ,$$

where $R_e^{(k,j)}(t)$ is the estimated achievable throughput of user j over RBG k calculated by the Adaptive Modulation and Coding (AMC) scheme that maps the channel quality indicator (CQI) value to the transport block size in bits.

The other channel awareness metric that we consider is m_{ff} which is proposed in [GMonghal2008] and it represents the frequency selective fading gains over RBG k for user j and is calculated in the following way:

$$m_{ff}^{(k,j)}(t) = \frac{CQI^{(k,j)}(t)}{\sum_{k=1}^K CQI(t)^{(k,j)}},$$

where $CQI^{(k,j)}(t)$ is the last reported CQI value from user j for the k -th RBG.

The user can select whether m_{pf} or m_{ff} is used by setting the attribute `ns3::CqaFfMacScheduler::CqaMetric` respectively to "CqaPf" or "CqaFf".

Random Access

The LTE model includes a model of the Random Access procedure based on some simplifying assumptions, which are detailed in the following for each of the messages and signals described in the specs [TS36321].

- **Random Access (RA) preamble:** in real LTE systems this corresponds to a Zadoff-Chu (ZC) sequence using one of several formats available and sent in the PRACH slots which could in principle overlap with PUSCH. PRACH Configuration Index 14 is assumed, i.e., preambles can be sent on any system frame number and sub-frame number. The RA preamble is modeled using the `LteControlMessage` class, i.e., as an ideal message that does not consume any radio resources. The collision of preamble transmission by multiple UEs in the same cell are modeled using a protocol interference model, i.e., whenever two or more identical preambles are transmitted in same cell at the same TTI, no one of these identical preambles will be received by the eNB. Other than this collision model, no error model is associated with the reception of a RA preamble.
- **Random Access Response (RAR):** in real LTE systems, this is a special MAC PDU sent on the DL-SCH. Since MAC control elements are not accurately modeled in the simulator (only RLC and above PDUs are), the RAR is modeled as an `LteControlMessage` that does not consume any radio resources. Still, during the RA procedure, the `LteEnbMac` will request to the scheduler the allocation of resources for the RAR using the FF MAC Scheduler primitive `SCHED_DL_RACH_INFO_REQ`. Hence, an enhanced scheduler implementation (not available at the moment) could allocate radio resources for the RAR, thus modeling the consumption of Radio Resources for the transmission of the RAR.
- **Message 3:** in real LTE systems, this is an RLC TM SDU sent over resources specified in the UL Grant in the RAR. In the simulator, this is modeled as a real RLC TM RLC PDU whose UL resources are allocated by the scheduler upon call to `SCHED_DL_RACH_INFO_REQ`.
- **Contention Resolution (CR):** in real LTE system, the CR phase is needed to address the case where two or more UE sent the same RA preamble in the same TTI, and the eNB was able to detect this preamble in spite of the collision. Since this event does not occur due to the protocol interference model used for the reception of RA preambles, the CR phase is not modeled in the simulator, i.e., the CR MAC CE is never sent by the eNB and the UEs consider the RA to be successful upon reception of the RAR. As a consequence, the radio resources consumed for the transmission of the CR MAC CE are not modeled.

Figure *Sequence diagram of the Contention-based MAC Random Access procedure* and *Sequence diagram of the Non-contention-based MAC Random Access procedure* shows the sequence diagrams of respectively the contention-based and non-contention-based MAC random access procedure, highlighting the interactions between the MAC and the other entities.

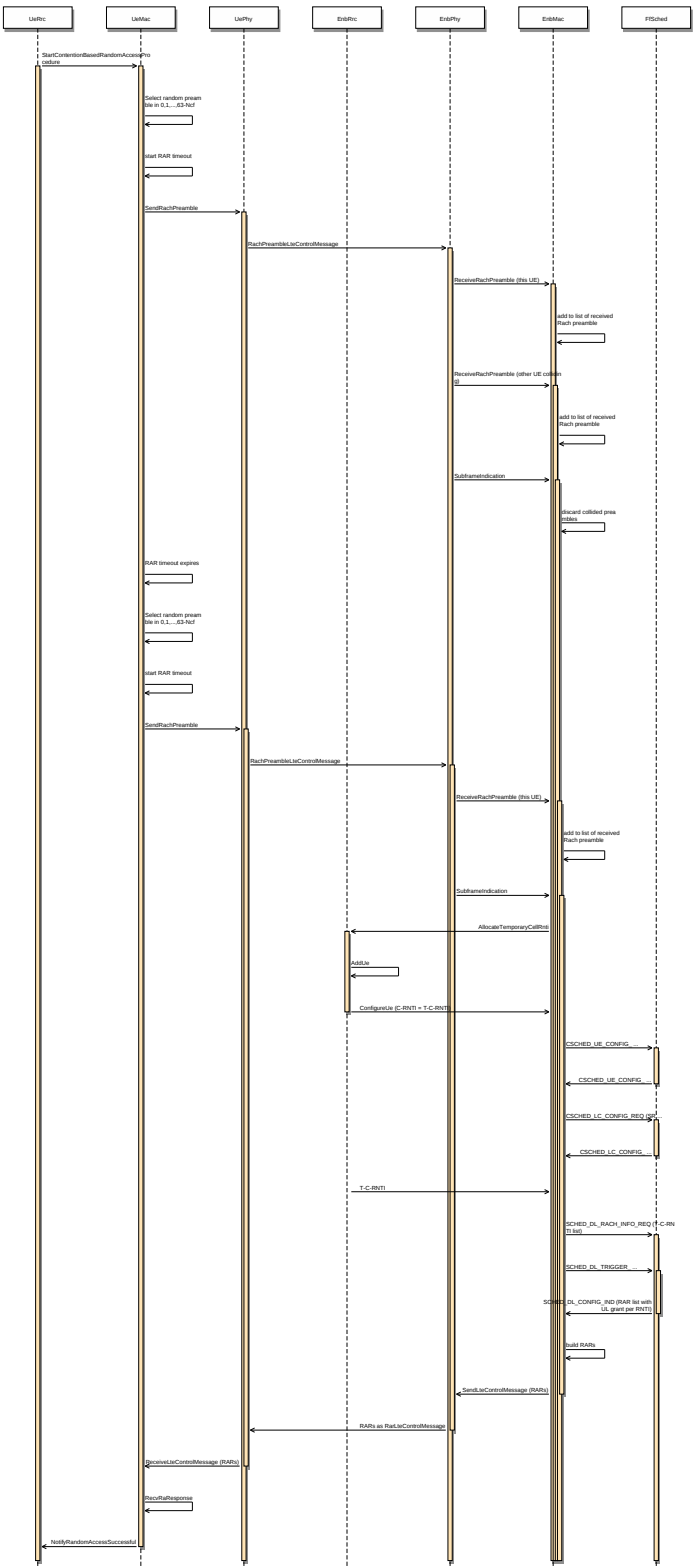


Fig. 19.23: Sequence diagram of the Contention-based MAC Random Access procedure

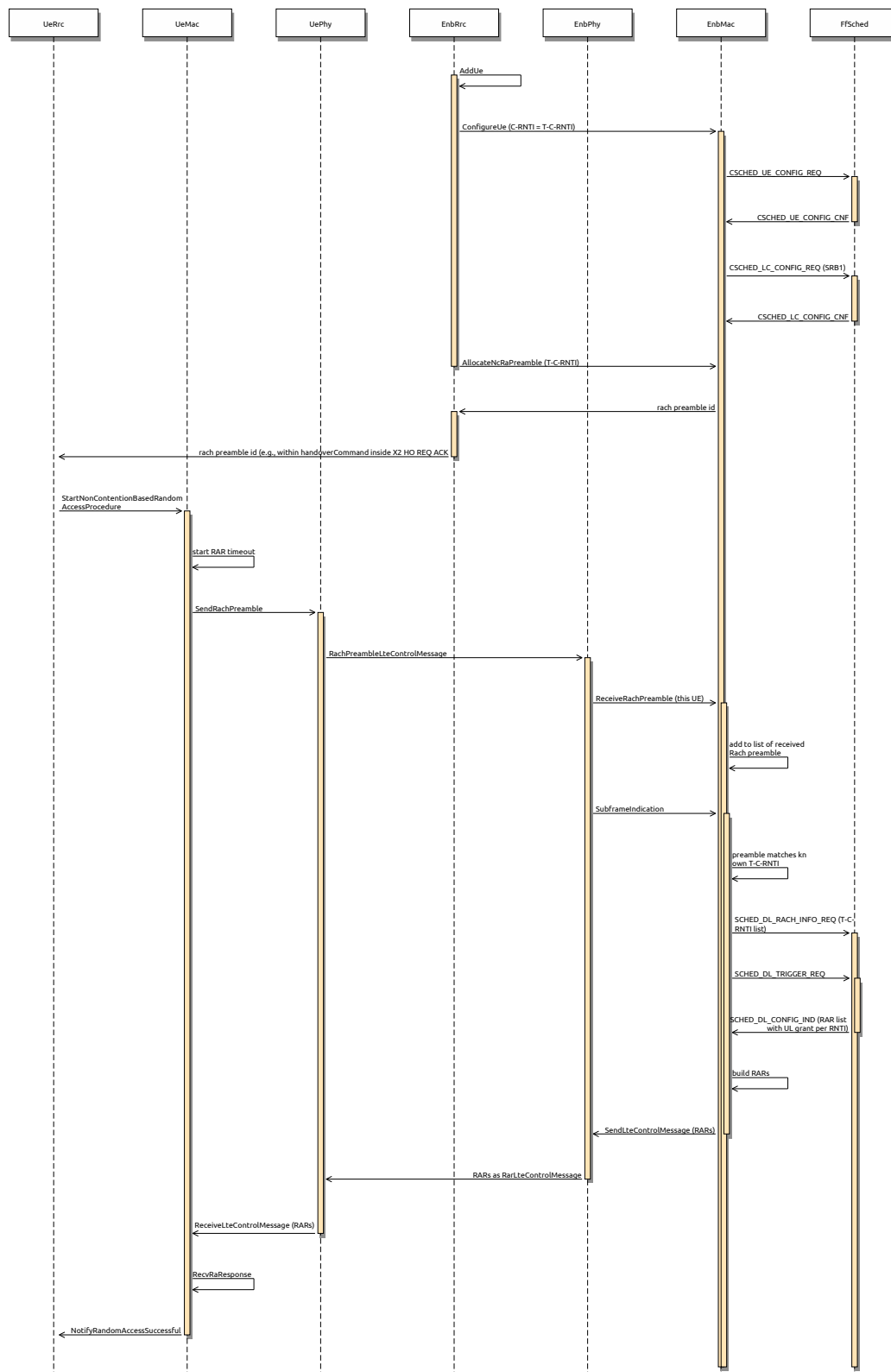


Fig. 19.24: Sequence diagram of the Non-contention-based MAC Random Access procedure

19.1.8 RLC

Overview

The RLC entity is specified in the 3GPP technical specification [TS36322], and comprises three different types of RLC: Transparent Mode (TM), Unacknowledged Mode (UM) and Acknowledged Mode (AM). The simulator includes one model for each of these entities

The RLC entities provide the RLC service interface to the upper PDCP layer and the MAC service interface to the lower MAC layer. The RLC entities use the PDCP service interface from the upper PDCP layer and the MAC service interface from the lower MAC layer.

Figure *Implementation Model of PDCP, RLC and MAC entities and SAPs* shows the implementation model of the RLC entities and its relationship with all the other entities and services in the protocol stack.

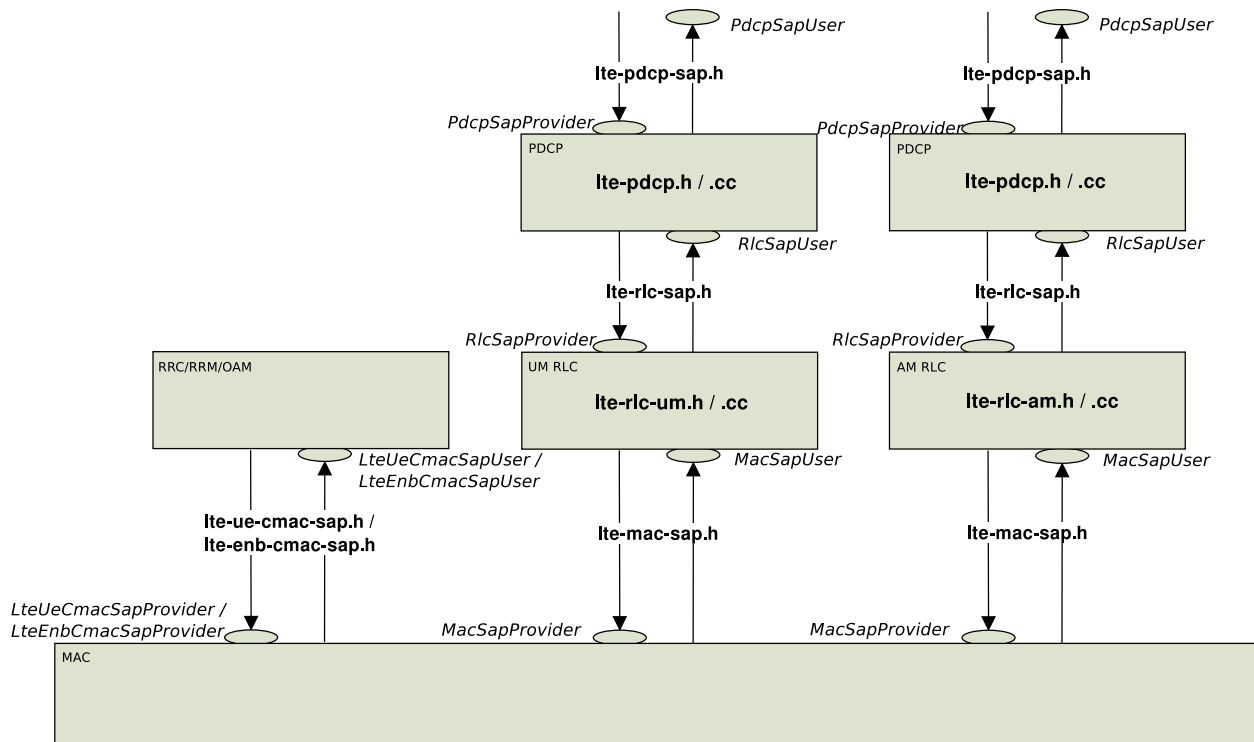


Fig. 19.25: Implementation Model of PDCP, RLC and MAC entities and SAPs

Service Interfaces

RLC Service Interface

The RLC service interface is divided into two parts:

- the `RlcSapProvider` part is provided by the RLC layer and used by the upper PDCP layer and
- the `RlcSapUser` part is provided by the upper PDCP layer and used by the RLC layer.

Both the UM and the AM RLC entities provide the same RLC service interface to the upper PDCP layer.

RLC Service Primitives

The following list specifies which service primitives are provided by the RLC service interfaces:

- `RlcSapProvider::TransmitPdcPdu`
 - The PDCP entity uses this primitive to send a PDCP PDU to the lower RLC entity in the transmitter peer
- `RlcSapUser::ReceivePdcPdu`
 - The RLC entity uses this primitive to send a PDCP PDU to the upper PDCP entity in the receiver peer

MAC Service Interface

The MAC service interface is divided into two parts:

- the `MacSapProvider` part is provided by the MAC layer and used by the upper RLC layer and
- the `MacSapUser` part is provided by the upper RLC layer and used by the MAC layer.

MAC Service Primitives

The following list specifies which service primitives are provided by the MAC service interfaces:

- `MacSapProvider::TransmitPdu`
 - The RLC entity uses this primitive to send a RLC PDU to the lower MAC entity in the transmitter peer
- `MacSapProvider::ReportBufferStatus`
 - The RLC entity uses this primitive to report the MAC entity the size of pending buffers in the transmitter peer
- `MacSapUser::NotifyTxOpportunity`
 - The MAC entity uses this primitive to notify the RLC entity a transmission opportunity
- `MacSapUser::ReceivePdu`
 - The MAC entity uses this primitive to send an RLC PDU to the upper RLC entity in the receiver peer

AM RLC

The processing of the data transfer in the Acknowledge Mode (AM) RLC entity is explained in section 5.1.3 of [TS36322]. In this section we describe some details of the implementation of the RLC entity.

Buffers for the transmit operations

Our implementation of the AM RLC entity maintains 3 buffers for the transmit operations:

- **Transmission Buffer:** it is the RLC SDU queue. When the AM RLC entity receives a SDU in the `TransmitPdcPdu` service primitive from the upper PDCP entity, it enqueues it in the Transmission Buffer. We put a limit on the RLC buffer size and the `LteRlc TxDrop` trace source is called when a drop due to a full buffer occurs.
- **Transmitted PDUs Buffer:** it is the queue of transmitted RLC PDUs for which an ACK/NACK has not been received yet. When the AM RLC entity sends a PDU to the MAC entity, it also puts a copy of the transmitted PDU in the Transmitted PDUs Buffer.

- **Retransmission Buffer:** it is the queue of RLC PDUs which are considered for retransmission (i.e., they have been NACKed). The AM RLC entity moves this PDU to the Retransmission Buffer, when it retransmits a PDU from the Transmitted Buffer.

Transmit operations in downlink

The following sequence diagram shows the interactions between the different entities (RRC, PDCP, AM RLC, MAC and MAC scheduler) of the eNB in the downlink to perform data communications.

Figure *Sequence diagram of data PDU transmission in downlink* shows how the upper layers send data PDUs and how the data flow is processed by the different entities/services of the LTE protocol stack.

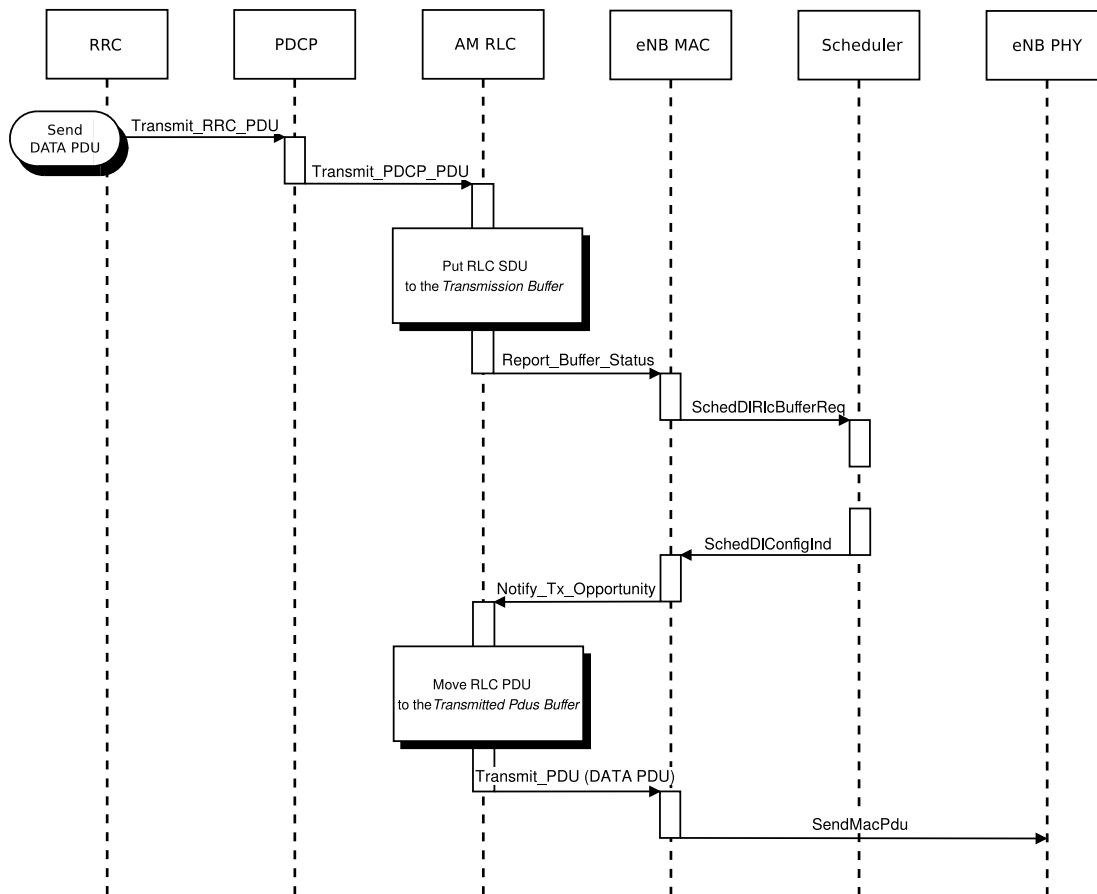


Fig. 19.26: Sequence diagram of data PDU transmission in downlink

The PDCP entity calls the `Transmit_PDCP_PDU` service primitive in order to send a data PDU. The AM RLC entity processes this service primitive according to the AM data transfer procedures defined in section 5.1.3 of [TS36322].

When the `Transmit_PDCP_PDU` service primitive is called, the AM RLC entity performs the following operations:

- Put the data SDU in the Transmission Buffer.
- Compute the size of the buffers (how the size of buffers is computed will be explained afterwards).
- Call the `Report_Buffer_Status` service primitive of the eNB MAC entity in order to notify to the eNB MAC entity the sizes of the buffers of the AM RLC entity. Then, the eNB MAC entity updates the buffer status in the MAC scheduler using the `SchedDIRlcBufferReq` service primitive of the FF MAC Scheduler API.

Afterwards, when the MAC scheduler decides that some data can be sent, the MAC entity notifies it to the RLC entity, i.e. it calls the `Notify_Tx_Opportunity` service primitive, then the AM RLC entity does the following:

- Create a single data PDU by segmenting and/or concatenating the SDUs in the Transmission Buffer.
- Move the data PDU from the Transmission Buffer to the Transmitted PDUs Buffer.
- Update state variables according section 5.1.3.1.1 of [TS36322].
- Call the `Transmit_PDU` primitive in order to send the data PDU to the MAC entity.

Retransmission in downlink

The sequence diagram of Figure *Sequence diagram of data PDU retransmission in downlink* shows the interactions between the different entities (AM RLC, MAC and MAC scheduler) of the eNB in downlink when data PDUs must be retransmitted by the AM RLC entity.

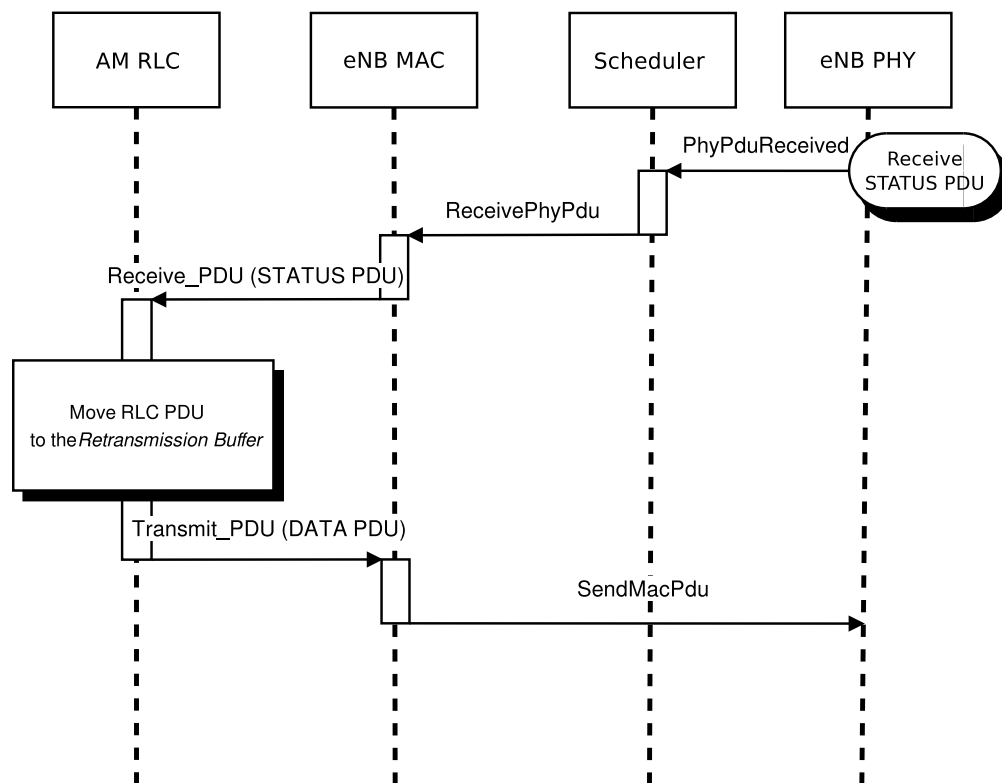


Fig. 19.27: Sequence diagram of data PDU retransmission in downlink

The transmitting AM RLC entity can receive STATUS PDUs from the peer AM RLC entity. STATUS PDUs are sent according section 5.3.2 of [TS36322] and the processing of reception is made according section 5.2.1 of [TS36322].

When a data PDU is retransmitted from the Transmitted PDUs Buffer, it is also moved to the Retransmission Buffer.

Transmit operations in uplink

The sequence diagram of Figure *Sequence diagram of data PDU transmission in uplink* shows the interactions between the different entities of the UE (RRC, PDCP, RLC and MAC) and the eNB (MAC and Scheduler) in uplink when data PDUs are sent by the upper layers.

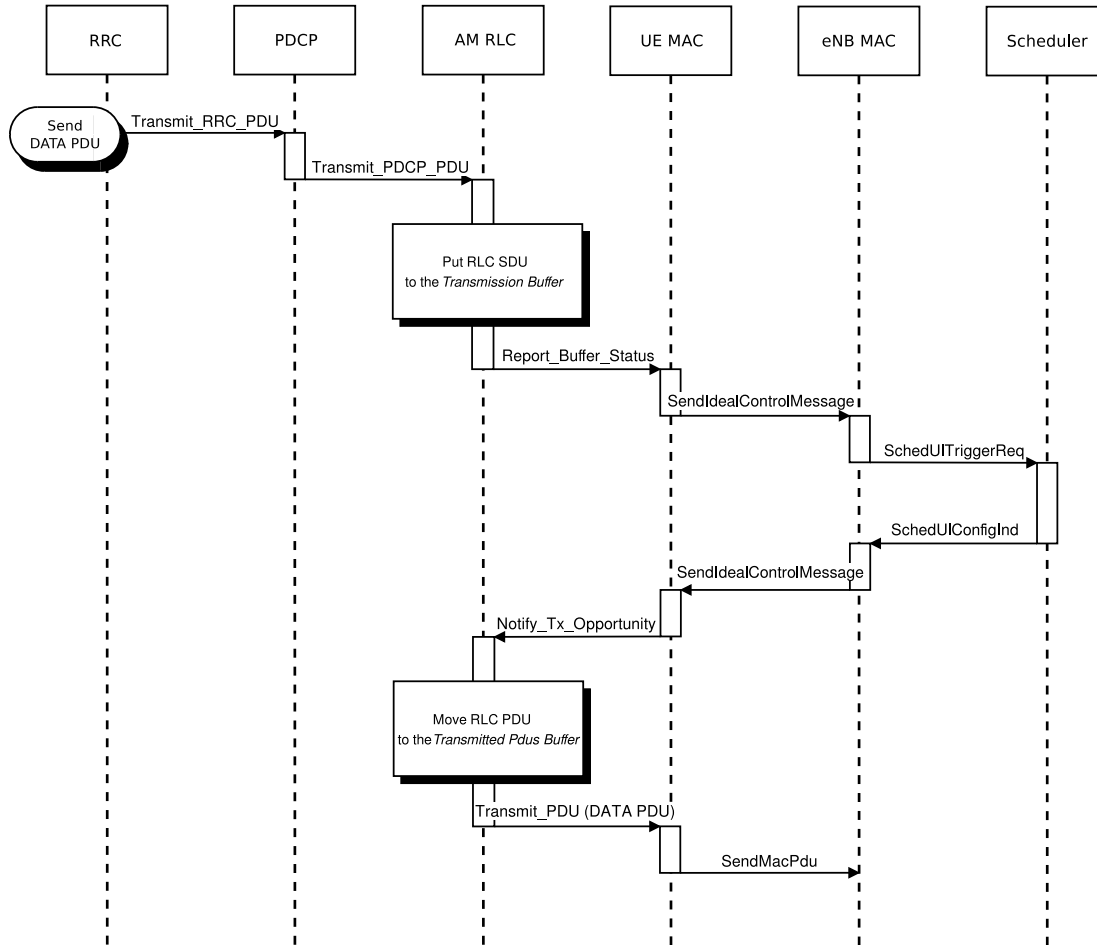


Fig. 19.28: Sequence diagram of data PDU transmission in uplink

It is similar to the sequence diagram in downlink; the main difference is that in this case the `Report_Buffer_Status` is sent from the UE MAC to the MAC Scheduler in the eNB over the air using the control channel.

Retransmission in uplink

The sequence diagram of Figure *Sequence diagram of data PDU retransmission in uplink* shows the interactions between the different entities of the UE (AM RLC and MAC) and the eNB (MAC) in uplink when data PDUs must be retransmitted by the AM RLC entity.

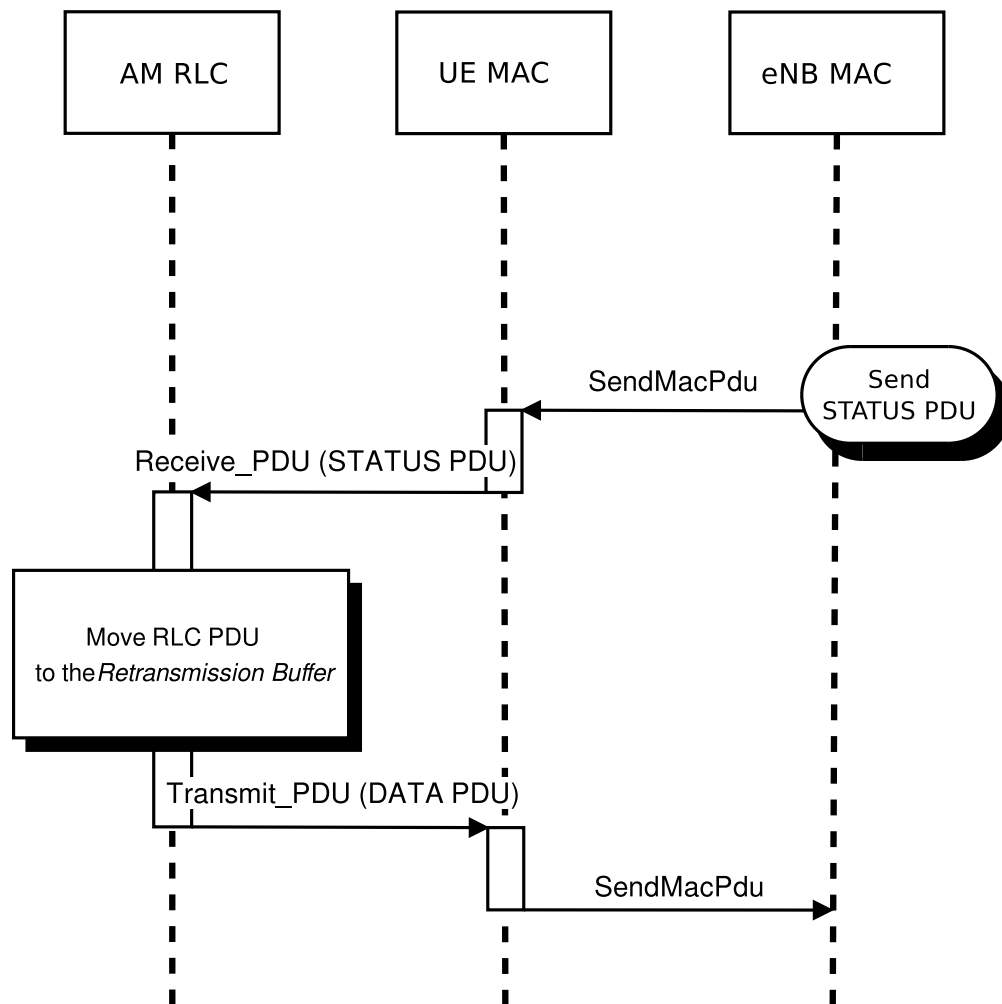


Fig. 19.29: Sequence diagram of data PDU retransmission in uplink

Calculation of the buffer size

The Transmission Buffer contains RLC SDUs. A RLC PDU is one or more SDU segments plus an RLC header. The size of the RLC header of one RLC PDU depends on the number of SDU segments the PDU contains.

The 3GPP standard (section 6.1.3.1 of [TS36321]) says clearly that, for the uplink, the RLC and MAC headers are not considered in the buffer size that is to be report as part of the Buffer Status Report. For the downlink, the behavior is not specified. Neither [FFAPI] specifies how to do it. Our RLC model works by assuming that the calculation of the buffer size in the downlink is done exactly as in the uplink, i.e., not considering the RLC and MAC header size.

We note that this choice affects the interoperation with the MAC scheduler, since, in response to the `Notify_Tx_Opportunity` service primitive, the RLC is expected to create a PDU of no more than the size requested by the MAC, including RLC overhead. Hence, unneeded fragmentation can occur if (for example) the MAC notifies a transmission exactly equal to the buffer size previously reported by the RLC. We assume that it is left to the Scheduler to implement smart strategies for the selection of the size of the transmission opportunity, in order to eventually avoid the inefficiency of unneeded fragmentation.

Concatenation and Segmentation

The AM RLC entity generates and sends exactly one RLC PDU for each transmission opportunity even if it is smaller than the size reported by the transmission opportunity. So for instance, if a STATUS PDU is to be sent, then only this PDU will be sent in that transmission opportunity.

The segmentation and concatenation for the SDU queue of the AM RLC entity follows the same philosophy as the same procedures of the UM RLC entity but there are new state variables (see [TS36322] section 7.1) only present in the AM RLC entity.

It is noted that, according to the 3GPP specs, there is no concatenation for the Retransmission Buffer.

Re-segmentation

The current model of the AM RLC entity does not support the re-segmentation of the retransmission buffer. Rather, the AM RLC entity just waits to receive a big enough transmission opportunity.

Unsupported features

We do not support the following procedures of [TS36322] :

- “Send an indication of successful delivery of RLC SDU” (See section 5.1.3.1.1)
- “Indicate to upper layers that max retransmission has been reached” (See section 5.2.1)
- “SDU discard procedures” (See section 5.3)
- “Re-establishment procedure” (See section 5.4)

We do not support any of the additional primitives of RLC SAP for AM RLC entity. In particular:

- no SDU discard notified by PDCP
- no notification of successful / failed delivery by AM RLC entity to PDCP entity

UM RLC

In this section we describe the implementation of the Unacknowledged Mode (UM) RLC entity.

Transmit operations in downlink

The transmit operations of the UM RLC are similar to those of the AM RLC previously described in Section *Transmit operations in downlink*, with the difference that, following the specifications of [TS36322], retransmission are not performed, and there are no STATUS PDUs.

Transmit operations in uplink

The transmit operations in the uplink are similar to those of the downlink, with the main difference that the `Report_Buffer_Status` is sent from the UE MAC to the MAC Scheduler in the eNB over the air using the control channel.

Calculation of the buffer size

The calculation of the buffer size for the UM RLC is done using the same approach of the AM RLC, please refer to section *Calculation of the buffer size* for the corresponding description.

TM RLC

In this section we describe the implementation of the Transparent Mode (TM) RLC entity.

Transmit operations in downlink

In the simulator, the TM RLC still provides to the upper layers the same service interface provided by the AM and UM RLC entities to the PDCP layer; in practice, this interface is used by an RRC entity (not a PDCP entity) for the transmission of RLC SDUs. This choice is motivated by the fact that the services provided by the TM RLC to the upper layers, according to [TS36322], is a subset of those provided by the UM and AM RLC entities to the PDCP layer; hence, we reused the same interface for simplicity.

The transmit operations in the downlink are performed as follows. When the `Transmit_PDCP_PDU` service primitive is called by the upper layers, the TM RLC does the following:

- put the SDU in the Transmission Buffer
- compute the size of the Transmission Buffer
- call the `Report_Buffer_Status` service primitive of the eNB MAC entity

Afterwards, when the MAC scheduler decides that some data can be sent by the logical channel to which the TM RLC entity belongs, the MAC entity notifies it to the TM RLC entity by calling the `Notify_Tx_Opportunity` service primitive. Upon reception of this primitive, the TM RLC entity does the following:

- if the TX opportunity has a size that is greater than or equal to the size of the head-of-line SDU in the Transmission Buffer
 - dequeue the head-of-line SDU from the Transmission Buffer
 - create one RLC PDU that contains entirely that SDU, without any RLC header
 - Call the `Transmit_PDU` primitive in order to send the RLC PDU to the MAC entity.

Transmit operations in uplink

The transmit operations in the uplink are similar to those of the downlink, with the main difference that a transmission opportunity can also arise from the assignment of the UL GRANT as part of the Random Access procedure, without an explicit Buffer Status Report issued by the TM RLC entity.

Calculation of the buffer size

As per the specifications [TS36322], the TM RLC does not add any RLC header to the PDUs being transmitted. Because of this, the buffer size reported to the MAC layer is calculated simply by summing the size of all packets in the transmission buffer, thus notifying to the MAC the exact buffer size.

SM RLC

In addition to the AM, UM and TM implementations that are modeled after the 3GPP specifications, a simplified RLC model is provided, which is called Saturation Mode (SM) RLC. This RLC model does not accept PDUs from any above layer (such as PDCP); rather, the SM RLC takes care of the generation of RLC PDUs in response to the notification of transmission opportunities notified by the MAC. In other words, the SM RLC simulates saturation conditions, i.e., it assumes that the RLC buffer is always full and can generate a new PDU whenever notified by the scheduler.

The SM RLC is used for simplified simulation scenarios in which only the LTE Radio model is used, without the EPC and hence without any IP networking support. We note that, although the SM RLC is an unrealistic traffic model, it still allows for the correct simulation of scenarios with multiple flows belonging to different (non real-time) QoS classes, in order to test the QoS performance obtained by different schedulers. This can be done since it is the task of the Scheduler to assign transmission resources based on the characteristics (e.g., Guaranteed Bit Rate) of each Radio Bearer, which are specified upon the definition of each Bearer within the simulation program.

As for schedulers designed to work with real-time QoS traffic that has delay constraints, the SM RLC is probably not an appropriate choice. This is because the absence of actual RLC SDUs (replaced by the artificial generation of Buffer Status Reports) makes it not possible to provide the Scheduler with meaningful head-of-line-delay information, which is often the metric of choice for the implementation of scheduling policies for real-time traffic flows. For the simulation and testing of such schedulers, it is advisable to use either the UM or the AM RLC models instead.

19.1.9 PDCP

PDCP Model Overview

The reference document for the specification of the PDCP entity is [TS36323]. With respect to this specification, the PDCP model implemented in the simulator supports only the following features:

- transfer of data (user plane or control plane);
- maintenance of PDCP SNs;
- transfer of SN status (for use upon handover);

The following features are currently not supported:

- header compression and decompression of IP data flows using the ROHC protocol;
- in-sequence delivery of upper layer PDUs at re-establishment of lower layers;
- duplicate elimination of lower layer SDUs at re-establishment of lower layers for radio bearers mapped on RLC AM;
- ciphering and deciphering of user plane data and control plane data;

- integrity protection and integrity verification of control plane data;
- timer based discard;
- duplicate discarding.

PDCP Service Interface

The PDCP service interface is divided into two parts:

- the `PdcpSapProvider` part is provided by the PDCP layer and used by the upper layer and
- the `PdcpSapUser` part is provided by the upper layer and used by the PDCP layer.

PDCP Service Primitives

The following list specifies which service primitives are provided by the PDCP service interfaces:

- `PdcpSapProvider::TransmitPdcpsdu`
 - The RRC entity uses this primitive to send an RRC PDU to the lower PDCP entity in the transmitter peer
- `PdcpSapUser::ReceivePdcpsdu`
 - The PDCP entity uses this primitive to send an RRC PDU to the upper RRC entity in the receiver peer

19.1.10 RRC

Features

The RRC model implemented in the simulator provides the following functionality:

- generation (at the eNB) and interpretation (at the UE) of System Information (in particular the Master Information Block and, at the time of this writing, only System Information Block Type 1 and 2)
- initial cell selection
- RRC connection establishment procedure
- RRC reconfiguration procedure, supporting the following use cases: + reconfiguration of the SRS configuration index + reconfiguration of the PHY TX mode (MIMO) + reconfiguration of UE measurements + data radio bearer setup + handover
- RRC connection re-establishment, supporting the following use cases: + handover

Architecture

The RRC model is divided into the following components:

- the RRC entities *LteUeRrc* and *LteEnbRrc*, which implement the state machines of the RRC entities respectively at the UE and the eNB;
- the RRC SAPs *LteUeRrcSapProvider*, *LteUeRrcSapUser*, *LteEnbRrcSapProvider*, *LteEnbRrcSapUser*, which allow the RRC entities to send and receive RRC messages and information elements;
- the RRC protocol classes *LteUeRrcProtocolIdeal*, *LteEnbRrcProtocolIdeal*, *LteUeRrcProtocolReal*, *LteEnbRrcProtocolReal*, which implement two different models for the transmission of RRC messages.

Additionally, the RRC components use various other SAPs in order to interact with the rest of the protocol stack. A representation of all the SAPs that are used is provided in the figures *LTE radio protocol stack architecture for the UE on the data plane*, *LTE radio protocol stack architecture for the UE on the control plane*, *LTE radio protocol stack architecture for the eNB on the data plane* and *LTE radio protocol stack architecture for the eNB on the control plane*.

UE RRC State Machine

In Figure *UE RRC State Machine* we represent the state machine as implemented in the RRC UE entity.

All the states are transient, however, the UE in “CONNECTED_NORMALLY” state will only switch to the IDLE state if the downlink SINR is below a defined threshold, which would lead to radio link failure *Radio Link Failure*. On the other hand, the UE would not be able to switch to IDLE mode due to a handover failure, as mentioned in *X2*.

ENB RRC State Machine

The eNB RRC maintains the state for each UE that is attached to the cell. From an implementation point of view, the state of each UE is contained in an instance of the *UeManager* class. The state machine is represented in Figure *ENB RRC State Machine for each UE*.

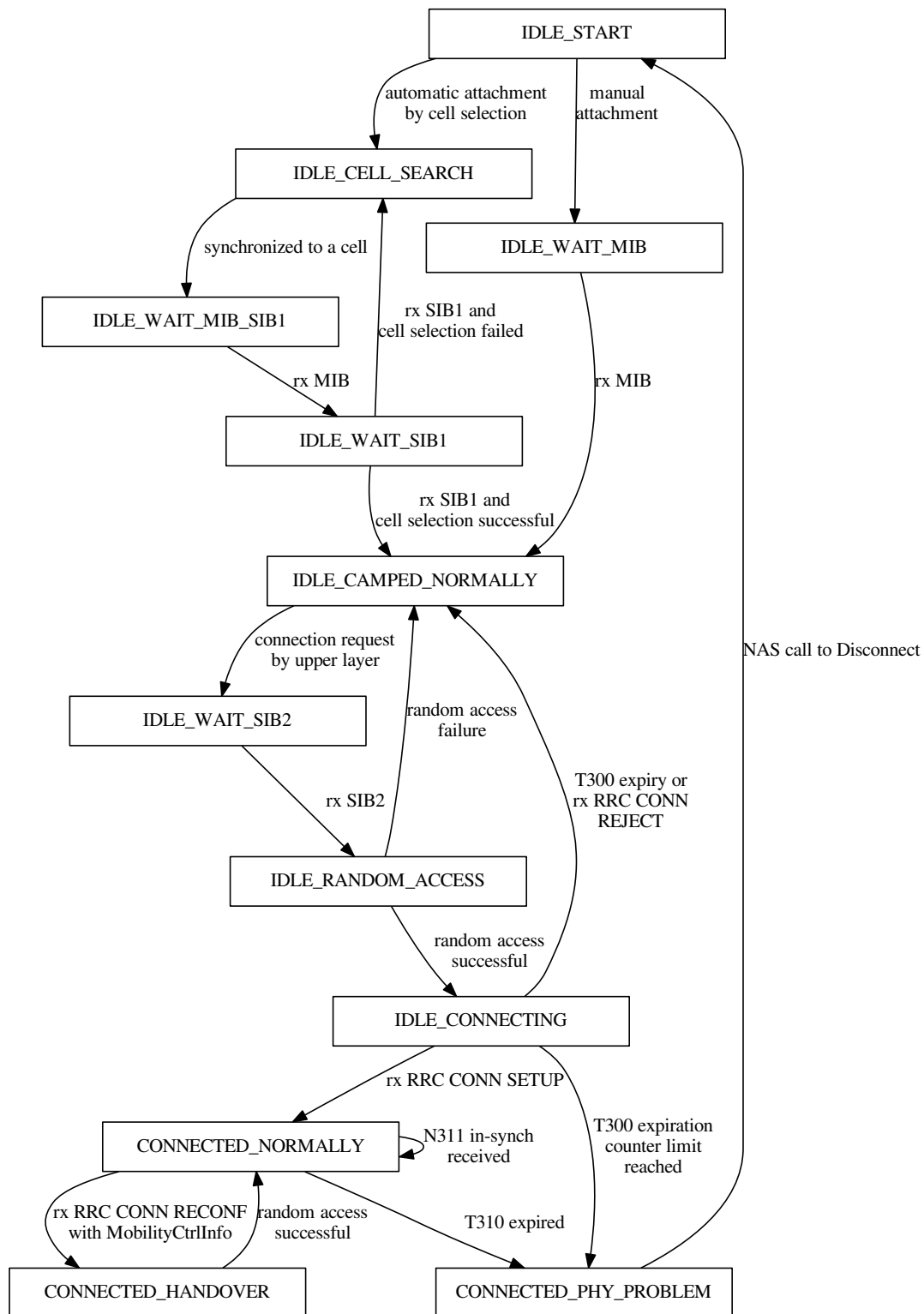


Fig. 19.30: UE RRC State Machine

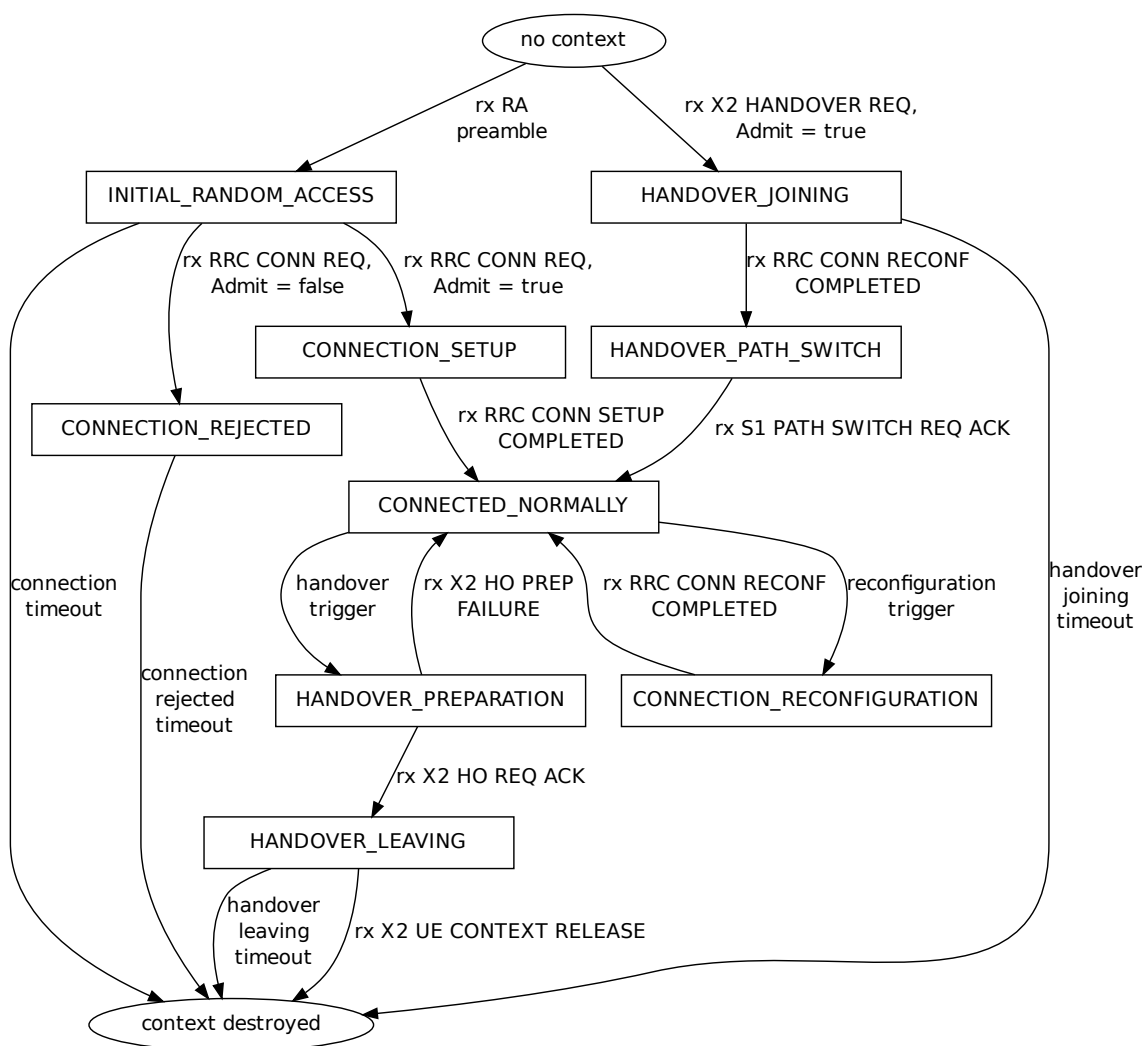


Fig. 19.31: ENB RRC State Machine for each UE

Initial Cell Selection

Initial cell selection is an IDLE mode procedure, performed by UE when it has not yet camped or attached to an eNodeB. The objective of the procedure is to find a suitable cell and attach to it to gain access to the cellular network.

It is typically done at the beginning of simulation, as depicted in Figure [Sample runs of initial cell selection in UE and timing of related events](#) below. The time diagram on the left side is illustrating the case where initial cell selection succeed on first try, while the diagram on the right side is for the case where it fails on the first try and succeed on the second try. The timing assumes the use of real RRC protocol model (see [RRC protocol models](#)) and no transmission error.

The functionality is based on 3GPP IDLE mode specifications, such as in [TS36300], [TS36304], and [TS36331]. However, a proper implementation of IDLE mode is still missing in the simulator, so we reserve several simplifying assumptions:

- multiple carrier frequency is not supported;
- multiple Public Land Mobile Network (PLMN) identities (i.e. multiple network operators) is not supported;
- RSRQ measurements are not utilized;
- stored information cell selection is not supported;
- “Any Cell Selection” state and camping to an acceptable cell is not supported;
- marking a cell as barred or reserved is not supported;
- Idle cell reselection is not supported, hence it is not possible for UE to camp to a different cell after the initial camp has been placed; and
- UE’s Closed Subscriber Group (CSG) white list contains only one CSG identity.

Also note that initial cell selection is only available for EPC-enabled simulations. LTE-only simulations must use the manual attachment method. See section [Network Attachment](#) of the User Documentation for more information on their differences in usage.

The next subsections cover different parts of initial cell selection, namely *cell search*, *broadcast of system information*, and *cell selection evaluation*.

Cell Search

Cell search aims to detect surrounding cells and measure the strength of received signal from each of these cells. One of these cells will become the UE’s entry point to join the cellular network.

The measurements are based on the RSRP of the received PSS, averaged by Layer 1 filtering, and performed by the PHY layer, as previously described in more detail in section [UE PHY Measurements Model](#). PSS is transmitted by eNodeB over the central 72 sub-carriers of the DL channel (Section 5.1.7.3 [TS36300]), hence we model cell search to operate using a DL bandwidth of 6 RBs. Note that measurements of RSRQ are not available at this point of time in simulation. As a consequence, the `LteUEPhy::RsrqUeMeasThreshold` attribute does not apply during cell search.

By using the measured RSRP, the PHY entity is able to generate a list of detected cells, each with its corresponding cell ID and averaged RSRP. This list is periodically pushed via CPHY SAP to the RRC entity as a measurement report.

The RRC entity inspects the report and simply choose the cell with the strongest RSRP, as also indicated in Section 5.2.3.1 of [TS36304]. Then it instructs back the PHY entity to synchronize to this particular cell. The actual operating bandwidth of the cell is still unknown at this time, so the PHY entity listens only to the minimum bandwidth of 6 RBs. Nevertheless, the PHY entity will be able to receive system broadcast message from this particular eNodeB, which is the topic of the next subsection.

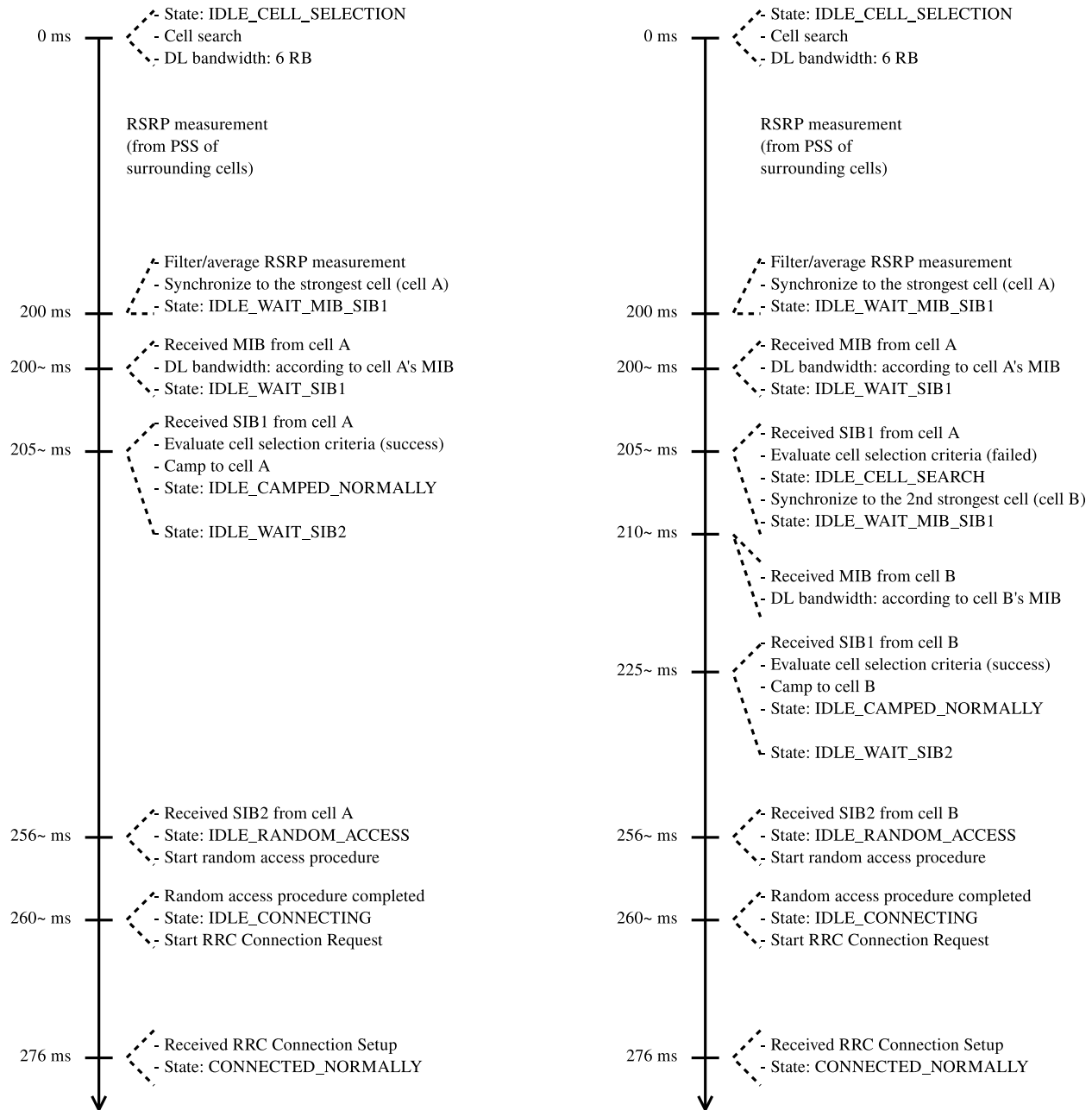


Fig. 19.32: Sample runs of initial cell selection in UE and timing of related events

Broadcast of System Information

System information blocks are broadcasted by eNodeB to UEs at predefined time intervals, adapted from Section 5.2.1.2 of [TS36331]. The supported system information blocks are:

- **Master Information Block (MIB)**
Contains parameters related to the PHY layer, generated during cell configuration and broadcasted every 10 ms at the beginning of radio frame as a control message.
- **System Information Block Type 1 (SIB1)**
Contains information regarding network access, broadcasted every 20 ms at the middle of radio frame as a control message. Not used in manual attachment method. UE must have decoded MIB before it can receive SIB1.
- **System Information Block Type 2 (SIB2)**
Contains UL- and RACH-related settings, scheduled to transmit via RRC protocol at 16 ms after cell configuration, and then repeats every 80 ms (configurable through *LteEnbRrc::SystemInformationPeriodicity* attribute). UE must be camped to a cell in order to be able to receive its SIB2.

Reception of system information is fundamental for UE to advance in its lifecycle. MIB enables the UE to increase the initial DL bandwidth of 6 RBs to the actual operating bandwidth of the network. SIB1 provides information necessary for cell selection evaluation (explained in the next section). And finally SIB2 is required before the UE is allowed to switch to CONNECTED state.

Cell Selection Evaluation

UE RRC reviews the measurement report produced in *Cell Search* and the cell access information provided by SIB1. Once both information is available for a specific cell, the UE triggers the evaluation process. The purpose of this process is to determine whether the cell is a suitable cell to camp to.

The evaluation process is a slightly simplified version of Section 5.2.3.2 of [TS36304]. It consists of the following criteria:

- Rx level criterion; and
- closed subscriber group (CSG) criterion.

The first criterion, Rx level, is based on the cell's measured RSRP $Q_{rxlevmeas}$, which has to be higher than a required minimum $Q_{rxlevmin}$ in order to pass the criterion:

$$Q_{rxlevmeas} - Q_{rxlevmin} > 0$$

where $Q_{rxlevmin}$ is determined by each eNodeB and is obtainable by UE from SIB1.

The last criterion, CSG, is a combination of a true-or-false parameter called *CSG indication* and a simple number *CSG identity*. The basic rule is that UE shall not camp to eNodeB with a different CSG identity. But this rule is only enforced when CSG indication is valued as true. More details are provided in Section *Network Attachment* of the User Documentation.

When the cell passes all the above criteria, the cell is deemed as *suitable*. Then UE camps to it (*IDLE_CAMPED_NORMALLY* state).

After this, upper layer may request UE to enter CONNECTED mode. Please refer to section *RRC connection establishment* for details on this.

On the other hand, when the cell does not pass the CSG criterion, then the cell is labeled as *acceptable* (Section 10.1.1.1 [TS36300]). In this case, the RRC entity will tell the PHY entity to synchronize to the second strongest cell and repeat the initial cell selection procedure using that cell. As long as no suitable cell is found, the UE will repeat these steps while avoiding cells that have been identified as acceptable.

Radio Admission Control

Radio Admission Control is supported by having the eNB RRC reply to an RRC CONNECTION REQUEST message sent by the UE with either an RRC CONNECTION SETUP message or an RRC CONNECTION REJECT message, depending on whether the new UE is to be admitted or not. In the current implementation, the behavior is determined by the boolean attribute `ns3::LteEnbRrc::AdmitRrcConnectionRequest`. There is currently no Radio Admission Control algorithm that dynamically decides whether a new connection shall be admitted or not.

Radio Bearer Configuration

Some implementation choices have been made in the RRC regarding the setup of radio bearers:

- three Logical Channel Groups (out of four available) are configured for uplink buffer status report purposes, according to the following policy:
 - LCG 0 is for signaling radio bearers
 - LCG 1 is for GBR data radio bearers
 - LCG 2 is for Non-GBR data radio bearers

Radio Link Failure

In real LTE networks, Radio link failure (RLF) can happen due to several reasons. It can be triggered if a UE is unable to decode PDCCH due to poor signal quality, upon maximum RLC retransmissions, RACH problems and other reasons. 3GPP only specifies guidelines to detect RLF at the UE side, in [TS36331] and [TS36133]. On the other hand, the eNB implementation is expected to be vendor specific. To implement the RLF functionality in ns-3, we have assumed the following simplifications:

- The RLF detection procedure at eNodeB is not implemented. **Instead, a direct function call by using the SAP between UE and eNB RRC (for both ideal and real RRC) is used to notify the eNB about the RLF.**
- No RRC connection re-establishment procedure is implemented, thus, the UE directly goes to the IDLE state upon RLF. This is in fact as per the standard [TS36331] sec 5.3.11.3, since, at this stage the LTE module does not support the Access Stratum (AS) security.

The above mentioned RLF specifications can be divided into the following two categories:

1. RLF detection
2. Actions upon RLF detection

In the following, we will explain the RLF implementation in context of these two categories.

RLF detection implementation

The RLF detection at the UE is implemented as per [TS36133], i.e., by monitoring the radio link quality based on the reference signals (which in the simulation is equivalent to the PDCCH) in the downlink. Thus, it is independent of the method used for the downlink CQI computation, i.e., *Ctrl* method and *Mixed method*. Moreover, when using FFR, especially for hard-FFR, and CQIs based on *Mixed method*, UEs might experience relatively good performance and RLF simultaneously. This is due to the fact that the interference in PDSCH is affected by the actual data transmissions on the specific RBs and the power control. Therefore, UEs might experience good SINR in PDSCH, while bad SINR in PDCCH channel. For more details about these methods please refer to *CQI feedback*. Also, it does not matter if the DL control error model is disabled, a UE can still detect the RLF since the SINR based on the control channel is reported to the `LteUePhy` class, using a callback hooked in `LteHelper` while installing a UE device.

The RLF detection starts once the RRC connection is established between UE and eNodeB, i.e., UE is in “CONNECTED_NORMALLY” state; upon which the RLF parameters are configured (see `LteUePhy::DoConfigureRadioLinkFailureDetection`). In real networks, these parameters are transmitted by the eNB using IE UE-TimersAndConstants or RLF-TimersAndConstants. However, for the sake of simplification, in the simulator they are presented as the attributes of the `LteUePhy` and `LteUeRrc` classes. Moreover, what concerns the carrier aggregation, i.e., when a UE is configured with multiple component carriers, the RLF detection is only performed by the primary component carrier, i.e. component carrier id 0 (see `LteUePhy::DoNotifyConnectionSuccessful`). In `LteUePhy` class, CQI calculation is triggered for every downlink subframe received, and the average SINR value is measured across all resource blocks. For the RLF detection, these SINR values are averaged over a downlink frame and if the result is less than a defined threshold Q_{out} (default: -5dB), the frame cannot be decoded (see `LteUePhy::RadioLinkFailureDetection`). The Q_{out} threshold corresponds to 10% block error rate (BLER) of a hypothetical PDCCH transmission taking into account the PCFICH errors [R4-081920] (also refer to *Control Channels PHY Error Model*). Once, the UE is unable to decode 20 consecutive frames, i.e., the Q_{out} evaluation period (200ms) is reached, an out-of-sync indication is sent to the UE RRC layer (see `LteUeRrc::DoNotifyOutOfSync`). Else, the counter for the unsuccessfully decoded frames is reset to zero. At the `LteUeRrc`, when the number of consecutive out-of-sync indications matches with the value of N310 parameter, the T310 timer is started and `LteUePhy` is notified to start measuring for in-sync indications (see `LteUePhy::DoStartInSyncDetection`). We note that, the UE RRC state is not changed till the expiration of T310 timer. If the resultant SINR values averaged over a downlink frame is greater than a defined threshold Q_{in} (default: -3.8dB), the frame is considered to be successfully decoded. Q_{in} corresponds to 2% BLER [R4-081920] of a hypothetical PDCCH transmission taking into account the PCFICH errors. Once the UE is able to decode 10 consecutive frames, an in-sync indication is sent to the UE RRC layer (see `LteUeRrc::DoNotifyInSync`). Else, the counter for the successfully decoded frames is reset to zero. If prior to the T310 timer expiry, the number of consecutive in-sync indications matches with N311 parameter of `LteUeRRC`, the UE is considered back in-sync. At this stage, the related parameters are reset to initiate the radio link failure detection from the beginning (see `LteUePhy::DoConfigureRadioLinkFailureDetection`). On the other hand, If the T310 timer expires, the UE considers that a RLF has occurred (see `LteUeRrc::RadioLinkFailureDetected`).

Actions upon RLF

Once the T310 timer is expired, a UE is considered to be in RLF; upon which the UE RRC:

- Sends a request to the eNB RRC to remove the UE context
- Moves to “CONNECTED_PHY_PROBLEM” state
- Notifies the UE NAS layer about the release of RRC connection.

Then, after getting the notification from the UE RRC the NAS does the following:

- Delete all the TFTs
- Reset the bearer counter
- Restore the bearer list, which is used to activate the bearers for the next RRC connection. This restoration of the bearers is achieved by maintaining an additional list, i.e., `m_bearersToBeActivatedListForReconnection` in `EpcUeNas` class
- Switch the NAS state to OFF by calling `EpcUeNas::Disconnect`
- Tells the UE RRC to disconnect

The UE RRC, upon receiving the call to disconnect from the `EpcUeNas` class, performs the action as specified by [TS36331] 5.3.11.3, and finally leaves the connected state, i.e., its RRC state is changed from “CONNECTED_PHY_PROBLEM” to “IDLE_START” to perform cell selection as shown in figures *UE RRC State Machine* and *UE procedures after radio link failure*.

At this stage, the LTE module does not support the paging functionality, therefore, to allow a UE to read SIB2 message after camping on a suitable cell after RLF, a work around is used in `LteUeRrc::EvaluateCellForSelection` method. As

per this workaround, the UE RRC invokes the call to `LteUeRrc::DoConnect` method, which enables the UE to switch its state from “IDLE_CAMPED_NORMALLY” to “IDLE_WAIT_SIB2”, thus, allowing it to perform the random access.

The eNB RRC, after receiving the notification from the UE RRC starts the procedure of UE context deletion, which also involves the deletion of the UE context removal from the EPC *UE context removal from EPC* and the eNB stack *UE context removal from eNB stack*. We note that, the UE context at the MME is not removed since, bearers are only added at the start of a simulation in MME, and cannot be added again unless scheduled for addition during a simulation.

UE RRC Measurements Model

UE RRC measurements support

The UE RRC entity provides support for UE measurements; in particular, it implements the procedures described in Section 5.5 of [TS36331], with the following simplifying assumptions:

- only E-UTRA intra-frequency measurements are supported, which implies:
 - only one measurement object is used during the simulation;
 - measurement gaps are not needed to perform the measurements;
 - Event B1 and B2 are not implemented;
- only *reportStrongestCells* purpose is supported, while *reportCGI* and *reportStrongestCellsForSON* purposes are not supported;
- *s-Measure* is not supported;
- carrier aggregation is now supported in the LTE module - Event A6 is not implemented;
- speed dependent scaling of time-to-trigger (Section 5.5.6.2 of [TS36331]) is not supported.

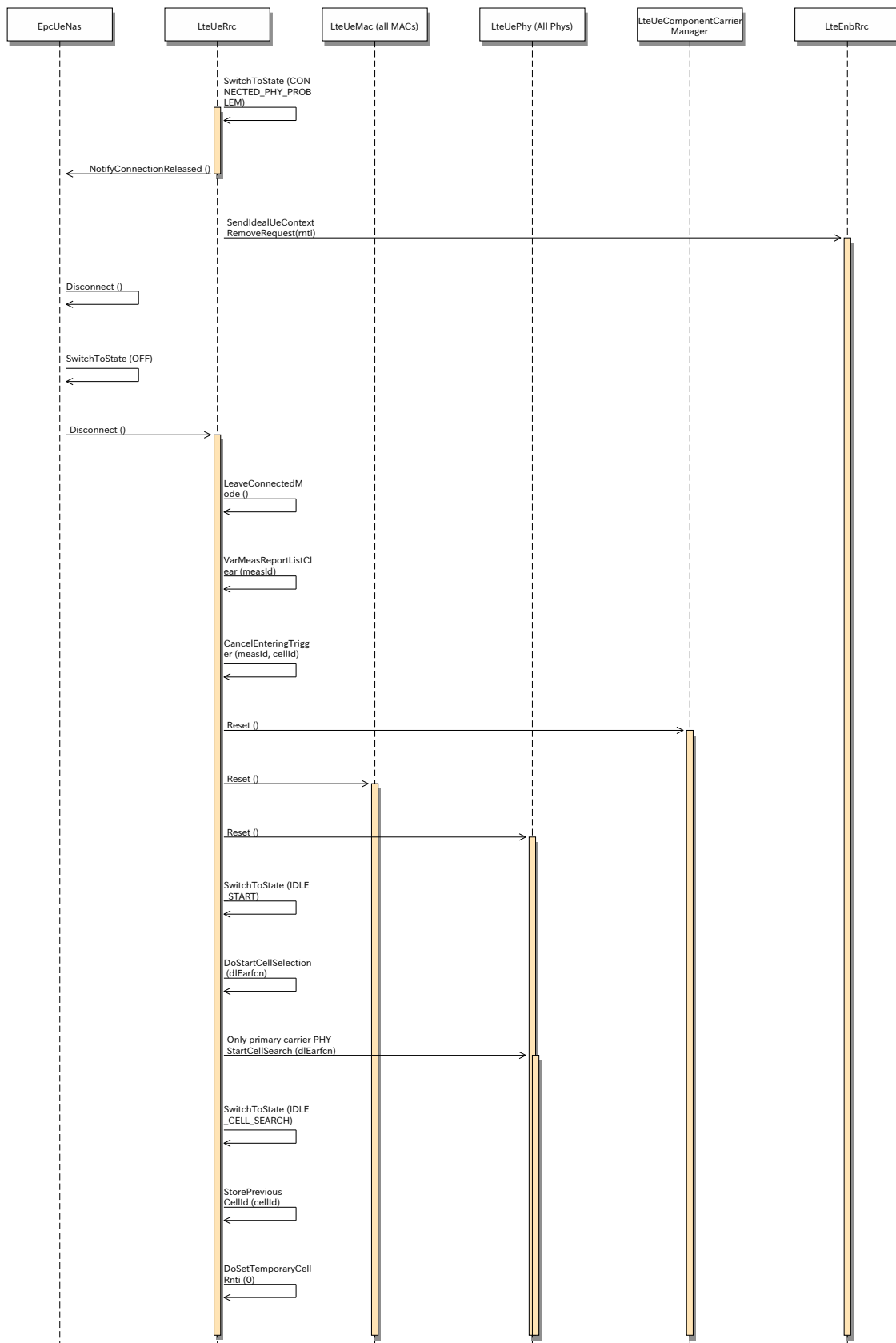
Overall design

The model is based on the concept of *UE measurements consumer*, which is an entity that may request an eNodeB RRC entity to provide UE measurement reports. Consumers are, for example, *Handover algorithm*, which compute handover decision based on UE measurement reports. Test cases and user's programs may also become consumers. Figure *Relationship between UE measurements and its consumers* depicts the relationship between these entities.

The whole UE measurements function at the RRC level is divided into 4 major parts:

1. Measurement configuration (handled by `LteUeRrc::ApplyMeasConfig`)
2. Performing measurements (handled by `LteUeRrc::DoReportUeMeasurements`)
3. Measurement report triggering (handled by `LteUeRrc::MeasurementReportTriggering`)
4. Measurement reporting (handled by `LteUeRrc::SendMeasurementReport`)

The following sections will describe each of the parts above.



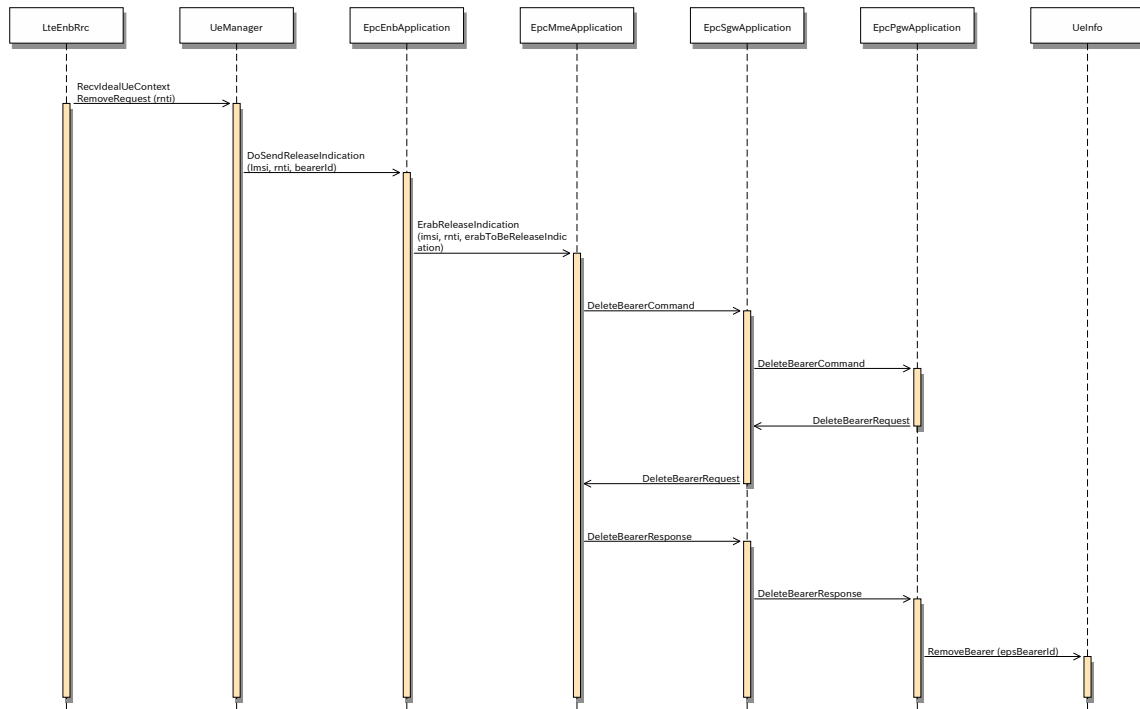


Fig. 19.34: UE context removal from EPC

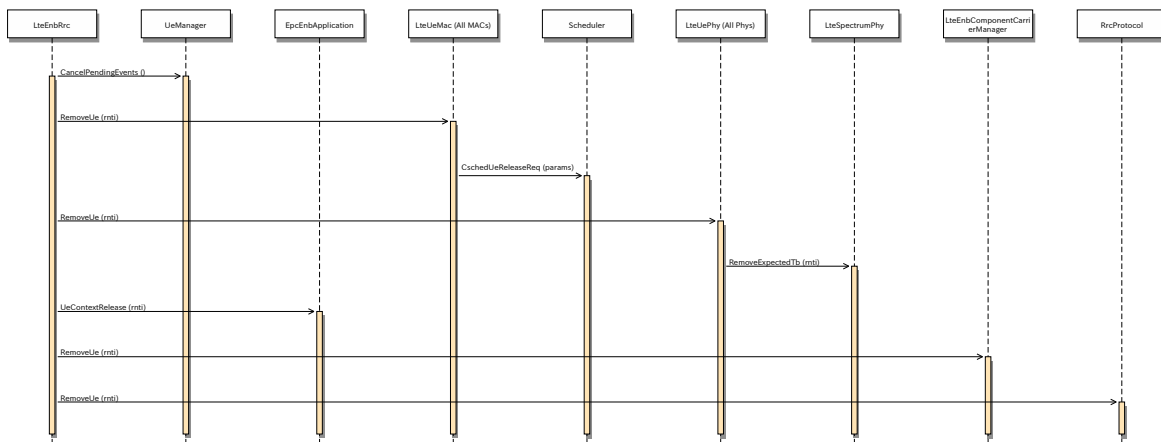


Fig. 19.35: UE context removal from eNB stack

UE Measurements Consumers

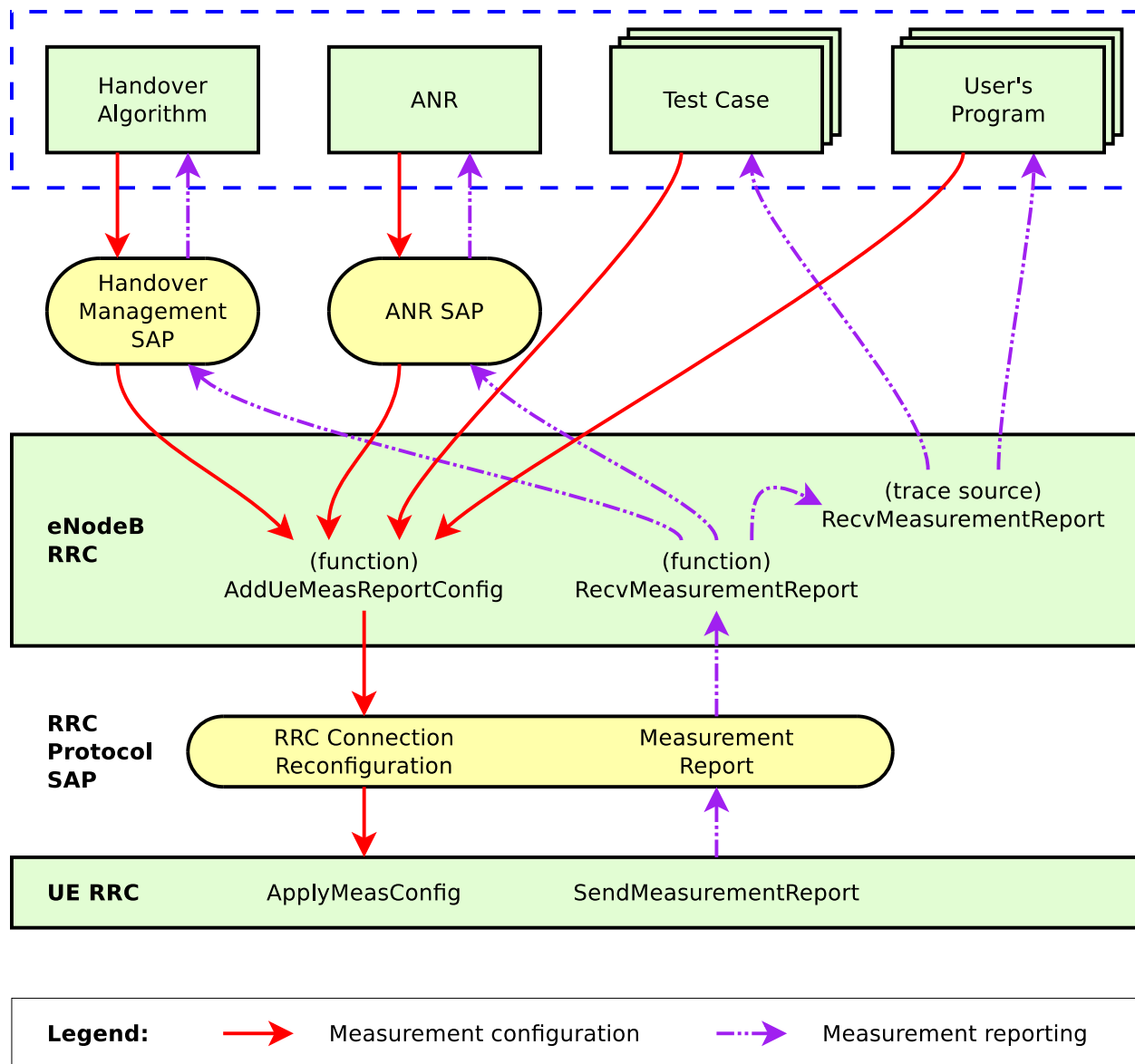


Fig. 19.36: Relationship between UE measurements and its consumers

Measurement configuration

An eNodeB RRC entity configures UE measurements by sending the configuration parameters to the UE RRC entity. This set of parameters are defined within the MeasConfig Information Element (IE) of the RRC Connection Reconfiguration message (*RRC connection reconfiguration*).

The eNodeB RRC entity implements the configuration parameters and procedures described in Section 5.5.2 of [TS36331], with the following simplifying assumption:

- configuration (i.e. addition, modification, and removal) can only be done before the simulation begins;
- all UEs attached to the eNodeB will be configured the same way, i.e. there is no support for configuring specific measurement for specific UE; and
- it is assumed that there is a one-to-one mapping between the PCI and the E-UTRAN Global Cell Identifier (EGCI). This is consistent with the PCI modeling assumptions described in *UE PHY Measurements Model*.

The eNodeB RRC instance here acts as an intermediary between the consumers and the attached UEs. At the beginning of simulation, each consumer provides the eNodeB RRC instance with the UE measurements configuration that it requires. After that, the eNodeB RRC distributes the configuration to attached UEs.

Users may customize the measurement configuration using several methods. Please refer to Section *Configure UE measurements* of the User Documentation for the description of these methods.

Performing measurements

UE RRC receives both RSRP and RSRQ measurements on periodical basis from UE PHY, as described in *UE PHY Measurements Model*. Layer 3 filtering will be applied to these received measurements. The implementation of the filtering follows Section 5.5.3.2 of [TS36331]:

$$F_n = (1 - a) \times F_{n-1} + a \times M_n$$

where:

- M_n is the latest received measurement result from the physical layer;
- F_n is the updated filtered measurement result;
- F_{n-1} is the old filtered measurement result, where $F_0 = M_1$ (i.e. the first measurement is not filtered); and
- $a = (\frac{1}{2})^{\frac{k}{4}}$, where k is the configurable *filterCoefficient* provided by the QuantityConfig;

$k = 4$ is the default value, but can be configured by setting the *RsrpFilterCoefficient* and *RsrqFilterCoefficient* attributes in *LteEnbRrc*.

Therefore $k = 0$ will disable Layer 3 filtering. On the other hand, past measurements can be granted more influence on the filtering results by using larger value of k .

Measurement reporting triggering

In this part, UE RRC will go through the list of active measurement configuration and check whether the triggering condition is fulfilled in accordance with Section 5.5.4 of [TS36331]. When at least one triggering condition from all the active measurement configuration is fulfilled, the measurement reporting procedure (described in the next subsection) will be initiated.

3GPP defines two kinds of *triggerType*: *periodical* and *event-based*. At the moment, only event-based criterion is supported. There are various events that can be selected, which are briefly described in the table below:

Table 19.1: List of supported event-based triggering criteria

Name	Description
Event A1	Serving cell becomes better than <i>threshold</i>
Event A2	Serving cell becomes worse than <i>threshold</i>
Event A3	Neighbour becomes <i>offset</i> dB better than serving cell
Event A4	Neighbour becomes better than <i>threshold</i>
Event A5	Serving becomes worse than <i>threshold1</i> AND neighbour becomes better than <i>threshold2</i>

Two main conditions to be checked in an event-based trigger are the *entering condition* and the *leaving condition*. More details on these two can be found in Section 5.5.4 of [TS36331].

An event-based trigger can be further configured by introducing hysteresis and time-to-trigger. *Hysteresis* (*Hys*) defines the distance between the entering and leaving conditions in dB. Similarly, *time-to-trigger* introduces delay to both entering and leaving conditions, but as a unit of time.

The *periodical* type of reporting trigger is not supported, but its behavior can be easily obtained by using an event-based trigger. This can be done by configuring the measurement in such a way that the entering condition is always fulfilled, for example, by setting the threshold of Event A1 to zero (the minimum level). As a result, the measurement reports will always be triggered at every certain interval, as determined by the *reportInterval* field within `LteRrcSap::ReportConfigEutra`, therefore producing the same behaviour as periodical reporting.

As a limitation with respect to 3GPP specifications, the current model does not support any cell-specific configuration. These configuration parameters are defined in measurement object. As a consequence, incorporating a list of black cells into the triggering process is not supported. Moreover, cell-specific offset (i.e., O_{cn} and O_{cp} in Event A3, A4, and A5) are not supported as well. The value equal to zero is always assumed in place of them.

Measurement reporting

This part handles the submission of measurement report from the UE RRC entity to the serving eNodeB entity via RRC protocol. Several simplifying assumptions have been adopted:

- *reportAmount* is *not* applicable (i.e. always assumed to be infinite);
- in measurement reports, the *reportQuantity* is always assumed to be *BOTH*, i.e., both RSRP and RSRQ are always reported, regardless of the *triggerQuantity*.

Handover

The RRC model supports UE mobility in CONNECTED mode by invoking the X2-based handover procedure. The model is intra-EUTRAN and intra-frequency, as based on Section 10.1.2.1 of [TS36300].

This section focuses on the process of triggering a handover. The handover execution procedure itself is covered in Section X2.

There are two ways to trigger the handover procedure:

- *explicitly* (or manually) triggered by the simulation program by scheduling an execution of the method `LteEnbRrc::SendHandoverRequest`; or
- *automatically* triggered by the eNodeB RRC entity based on UE measurements and according to the selected handover algorithm.

Section *X2-based handover* of the User Documentation provides some examples on using both explicit and automatic handover triggers in simulation. The next subsection will take a closer look on the automatic method, by describing the design aspects of the handover algorithm interface and the available handover algorithms.

Handover algorithm

Handover in 3GPP LTE has the following properties:

- **UE-assisted**
The UE provides input to the network in the form of measurement reports. This is handled by the *UE RRC Measurements Model*.
- **Network-controlled**
The network (i.e. the source eNodeB and the target eNodeB) decides when to trigger the handover and oversees its execution.

The *handover algorithm* operates at the source eNodeB and is responsible in making handover decisions in an “automatic” manner. It interacts with an eNodeB RRC instance via the *Handover Management SAP* interface. These relationships are illustrated in Figure *Relationship between UE measurements and its consumers* from the previous section.

The handover algorithm interface consists of the following methods:

- **AddUeMeasReportConfigForHandover**
(Handover Algorithm -> eNodeB RRC) Used by the handover algorithm to request measurement reports from the eNodeB RRC entity, by passing the desired reporting configuration. The configuration will be applied to all future attached UEs.
- **ReportUeMeas**
(eNodeB RRC -> Handover Algorithm) Based on the UE measurements configured earlier in AddUeMeasReportConfigForHandover, UE may submit measurement reports to the eNodeB. The eNodeB RRC entity uses the ReportUeMeas interface to forward these measurement reports to the handover algorithm.
- **TriggerHandover**
(Handover Algorithm -> eNodeB RRC) After examining the measurement reports (but not necessarily), the handover algorithm may declare a handover. This method is used to notify the eNodeB RRC entity about this decision, which will then proceed to commence the handover procedure.

One note for the AddUeMeasReportConfigForHandover. The method will return the measId (measurement identity) of the newly created measurement configuration. Typically a handover algorithm would store this unique number. It may be useful in the ReportUeMeas method, for example when more than one configuration has been requested and the handover algorithm needs to differentiate incoming reports based on the configuration that triggered them.

A handover algorithm is implemented by writing a subclass of the LteHandoverAlgorithm abstract superclass and implementing each of the above mentioned SAP interface methods. Users may develop their own handover algorithm this way, and then use it in any simulation by following the steps outlined in Section *X2-based handover* of the User Documentation.

Alternatively, users may choose to use one of the 3 built-in handover algorithms provided by the LTE module: no-op, A2-A4-RSRQ, and strongest cell handover algorithm. They are ready to be used in simulations or can be taken as an example of implementing a handover algorithm. Each of these built-in algorithms is covered in each of the following subsections.

No-op handover algorithm

The *no-op handover algorithm* (`NoOpHandoverAlgorithm` class) is the simplest possible implementation of handover algorithm. It basically does nothing, i.e., does not call any of the Handover Management SAP interface methods. Users may choose this handover algorithm if they wish to disable automatic handover trigger in their simulation.

A2-A4-RSRQ handover algorithm

The *A2-A4-RSRQ handover algorithm* provides the functionality of the default handover algorithm originally included in LENA M6 (ns-3.18), ported to the Handover Management SAP interface as the `A2A4RsrqHandoverAlgorithm` class.

As the name implies, the algorithm utilizes the Reference Signal Received Quality (RSRQ) measurements acquired from Event A2 and Event A4. Thus, the algorithm will add 2 measurement configuration to the corresponding eNodeB RRC instance. Their intended use are described as follows:

- *Event A2* (serving cell's RSRQ becomes worse than *threshold*) is leveraged to indicate that the UE is experiencing poor signal quality and may benefit from a handover.
- *Event A4* (neighbour cell's RSRQ becomes better than *threshold*) is used to detect neighbouring cells and acquire their corresponding RSRQ from every attached UE, which are then stored internally by the algorithm. By default, the algorithm configures Event A4 with a very low threshold, so that the trigger criteria are always true.

Figure *A2-A4-RSRQ handover algorithm* below summarizes this procedure.

Two attributes can be set to tune the algorithm behaviour:

- **ServingCellThreshold**
The *threshold* for Event A2, i.e. a UE must have an RSRQ lower than this threshold to be considered for a handover.
- **NeighbourCellOffset**
The *offset* that aims to ensure that the UE would receive better signal quality after the handover. A neighbouring cell is considered as a target cell for the handover only if its RSRQ is higher than the serving cell's RSRQ by the amount of this *offset*.

The value of both attributes are expressed as RSRQ range (Section 9.1.7 of [TS36133]), which is an integer between 0 and 34, with 0 as the lowest RSRQ.

Strongest cell handover algorithm

The *strongest cell handover algorithm*, or also sometimes known as the *traditional power budget (PBGT) algorithm*, is developed using [Dimou2009] as reference. The idea is to provide each UE with the best possible Reference Signal Received Power (RSRP). This is done by performing a handover as soon as a better cell (i.e. with stronger RSRP) is detected.

Event A3 (neighbour cell's RSRP becomes better than serving cell's RSRP) is chosen to realize this concept. The `A3RsrpHandoverAlgorithm` class is the result of the implementation. Handover is triggered for the UE to the best cell in the measurement report.

A simulation which uses this algorithm is usually more vulnerable to ping-pong handover (consecutive handover to the previous source eNodeB within short period of time), especially when the *Fading Model* is enabled. This problem is typically tackled by introducing a certain delay to the handover. The algorithm does this by including hysteresis and time-to-trigger parameters (Section 6.3.5 of [TS36331]) to the UE measurements configuration.

Hysteresis (a.k.a. handover margin) delays the handover in regard of RSRP. The value is expressed in dB, ranges between 0 to 15 dB, and have a 0.5 dB accuracy, e.g., an input value of 2.7 dB is rounded to 2.5 dB.

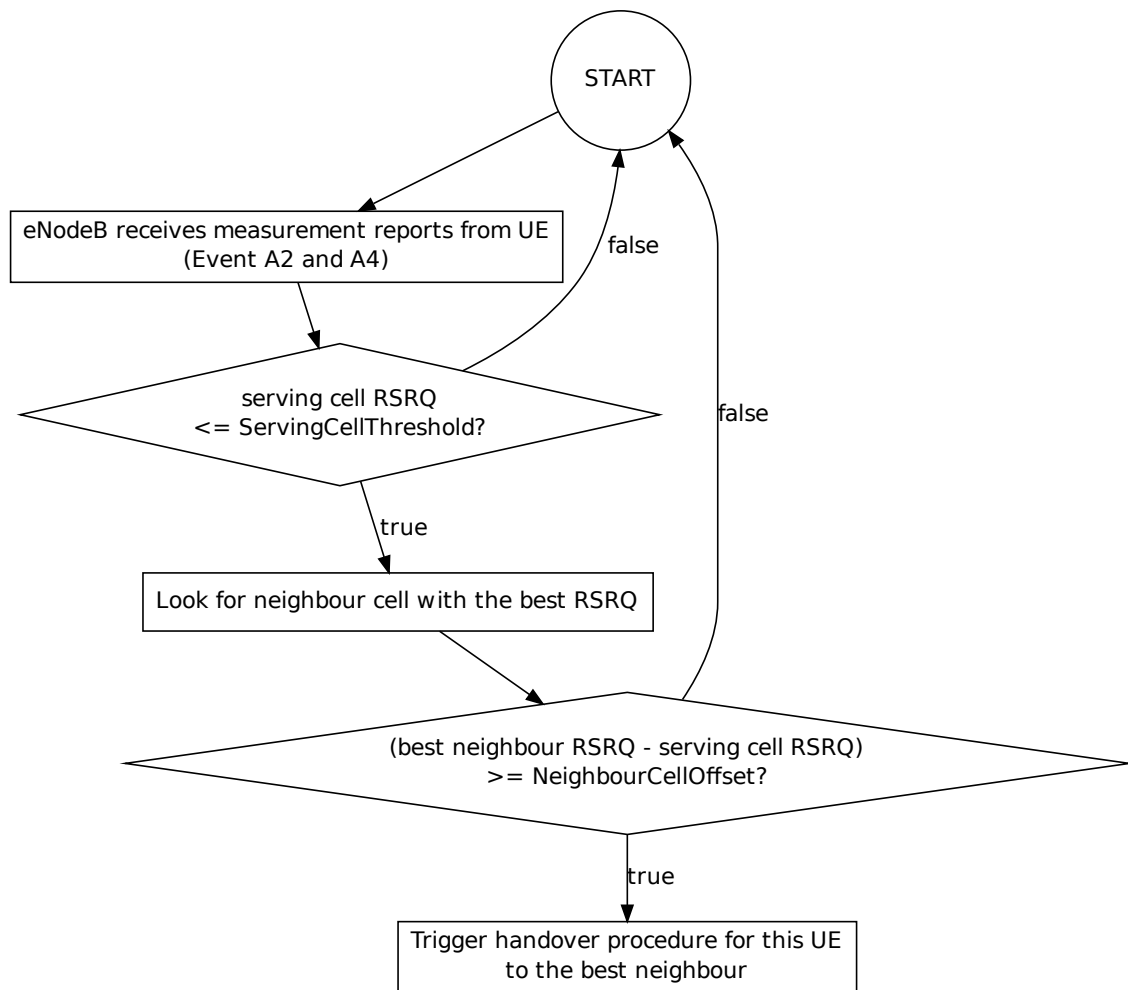


Fig. 19.37: A2-A4-RSRQ handover algorithm

On the other hand, *time-to-trigger* delays the handover in regard of time. 3GPP defines 16 valid values for time-to-trigger (all in milliseconds): 0, 40, 64, 80, 100, 128, 160, 256, 320, 480, 512, 640, 1024, 1280, 2560, and 5120.

The difference between hysteresis and time-to-trigger is illustrated in Figure *Effect of hysteresis and time-to-trigger in strongest cell handover algorithm* below, which is taken from the *lena-x2-handover-measures* example. It depicts the perceived RSRP of serving cell and a neighbouring cell by a UE which moves pass the border of the cells.

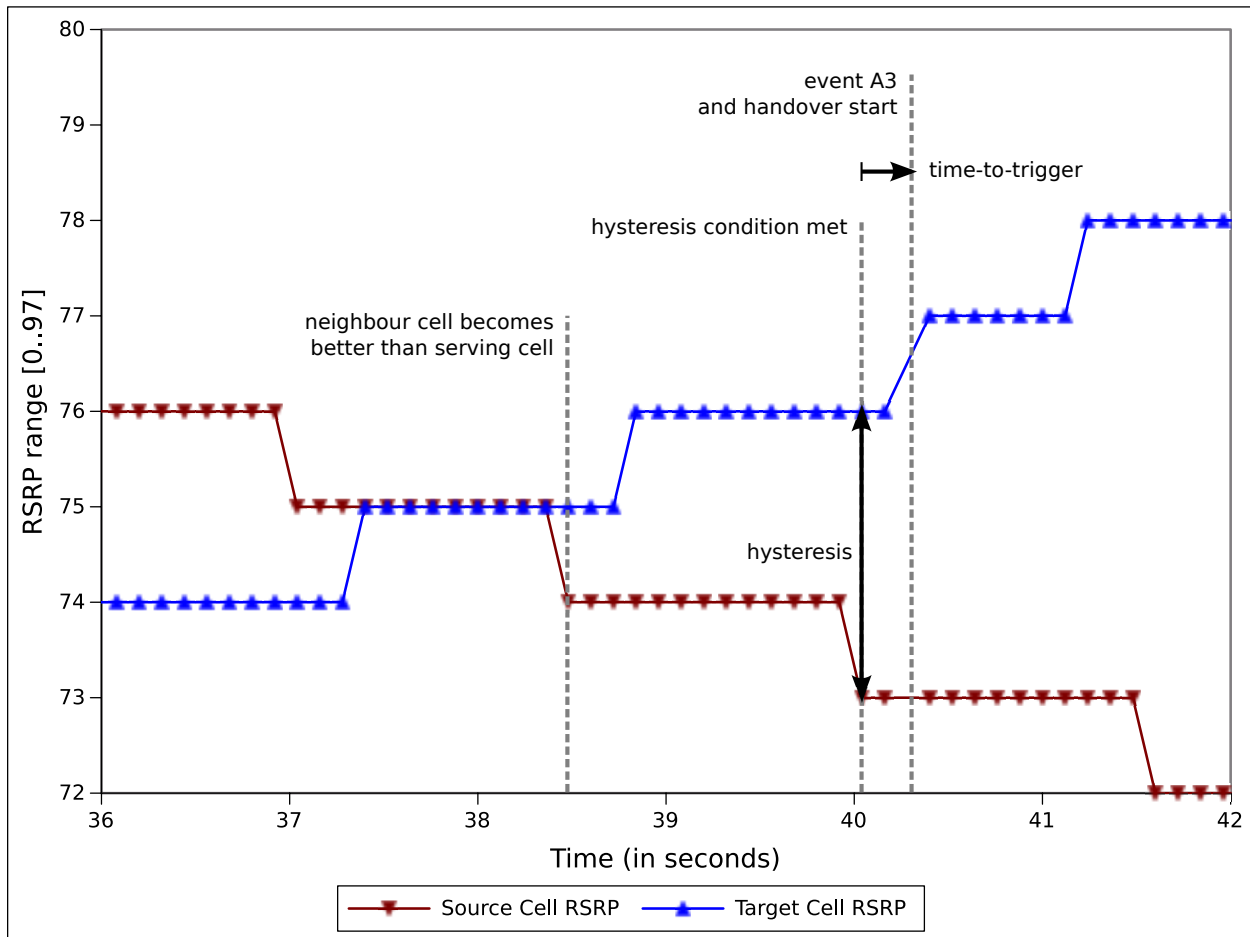


Fig. 19.38: Effect of hysteresis and time-to-trigger in strongest cell handover algorithm

By default, the algorithm uses a hysteresis of 3.0 dB and time-to-trigger of 256 ms. These values can be tuned through the `Hysteresis` and `TimeToTrigger` attributes of the `A3RsrpHandoverAlgorithm` class.

Neighbour Relation

LTE module supports a simplified *Automatic Neighbour Relation* (ANR) function. This is handled by the `LteAnr` class, which interacts with an eNodeB RRC instance through the ANR SAP interface.

Neighbour Relation Table

The ANR holds a *Neighbour Relation Table* (NRT), similar to the description in Section 22.3.2a of [TS36300]. Each entry in the table is called a *Neighbour Relation* (NR) and represents a detected neighbouring cell, which contains the following boolean fields:

- **No Remove**
Indicates that the NR shall *not* be removed from the NRT. This is *true* by default for user-provided NR and *false* otherwise.
- **No X2**
Indicates that the NR shall *not* use an X2 interface in order to initiate procedures towards the eNodeB parenting the target cell. This is *false* by default for user-provided NR, and *true* otherwise.
- **No HO**
Indicates that the NR shall *not* be used by the eNodeB for handover reasons. This is *true* in most cases, except when the NR is both user-provided and network-detected.

Each NR entry may have at least one of the following properties:

- **User-provided**
This type of NR is created as instructed by the simulation user. For example, a NR is created automatically upon a user-initiated establishment of X2 connection between 2 eNodeBs, e.g. as described in Section [X2-based handover](#). Another way to create a user-provided NR is to call the `AddNeighbourRelation` function explicitly.
- **Network-detected**
This type of NR is automatically created during the simulation as a result of the discovery of a nearby cell.

In order to automatically create network-detected NR, ANR utilizes UE measurements. In other words, ANR is a consumer of UE measurements, as depicted in Figure [Relationship between UE measurements and its consumers](#). RSRQ and Event A4 (neighbour becomes better than *threshold*) are used for the reporting configuration. The default Event A4 *threshold* is set to the lowest possible, i.e., maximum detection capability, but can be changed by setting the `Threshold` attribute of `LteAnr` class. Note that the A2-A4-RSRQ handover algorithm also utilizes a similar reporting configuration. Despite the similarity, when both ANR and this handover algorithm are active in the eNodeB, they use separate reporting configuration.

Also note that automatic setup of X2 interface is not supported. This is the reason why the *No X2* and *No HO* fields are true in a network-detected but not user-detected NR.

Role of ANR in Simulation

The ANR SAP interface provides the means of communication between ANR and eNodeB RRC. Some interface functions are used by eNodeB RRC to interact with the NRT, as shown below:

- **AddNeighbourRelation**
(eNodeB RRC -> ANR) Add a new user-provided NR entry into the NRT.
- **GetNoRemove**
(eNodeB RRC -> ANR) Get the value of *No Remove* field of an NR entry of the given cell ID.

- **GetNoHo**
(eNodeB RRC -> ANR) Get the value of *No HO* field of an NR entry of the given cell ID.
- **GetNoX2**
(eNodeB RRC -> ANR) Get the value of *No X2* field of an NR entry of the given cell ID.

Other interface functions exist to support the role of ANR as a UE measurements consumer, as listed below:

- **AddUeMeasReportConfigForAnr**
(ANR -> eNodeB RRC) Used by the ANR to request measurement reports from the eNodeB RRC entity, by passing the desired reporting configuration. The configuration will be applied to all future attached UEs.
- **ReportUeMeas**
(eNodeB RRC -> ANR) Based on the UE measurements configured earlier in `AddUeMeasReportConfigForAnr`, UE may submit measurement reports to the eNodeB. The eNodeB RRC entity uses the `ReportUeMeas` interface to forward these measurement reports to the ANR.

Please refer to the corresponding API documentation for `LteAnrSap` class for more details on the usage and the required parameters.

The ANR is utilized by the eNodeB RRC instance as a data structure to keep track of the situation of nearby neighbouring cells. The ANR also helps the eNodeB RRC instance to determine whether it is possible to execute a handover procedure to a neighbouring cell. This is realized by the fact that eNodeB RRC will only allow a handover procedure to happen if the NR entry of the target cell has both *No HO* and *No X2* fields set to *false*.

ANR is enabled by default in every eNodeB instance in the simulation. It can be disabled by setting the `AnrEnabled` attribute in `LteHelper` class to *false*.

RRC sequence diagrams

In this section we provide some sequence diagrams that explain the most important RRC procedures being modeled.

RRC connection establishment

Figure *Sequence diagram of the RRC Connection Establishment procedure* shows how the RRC Connection Establishment procedure is modeled, highlighting the role of the RRC layer at both the UE and the eNB, as well as the interaction with the other layers.

There are several timeouts related to this procedure, which are listed in the following Table *Timers in RRC connection establishment procedure*. If any of these timers expired, the RRC connection establishment procedure is terminated in failure. At the UE side, if T300 timer has expired a consecutive *connEstFailCount* times on the same cell it performs the cell selection again [TS36331]. Else, the upper layer (UE NAS) will immediately attempt to retry the procedure.

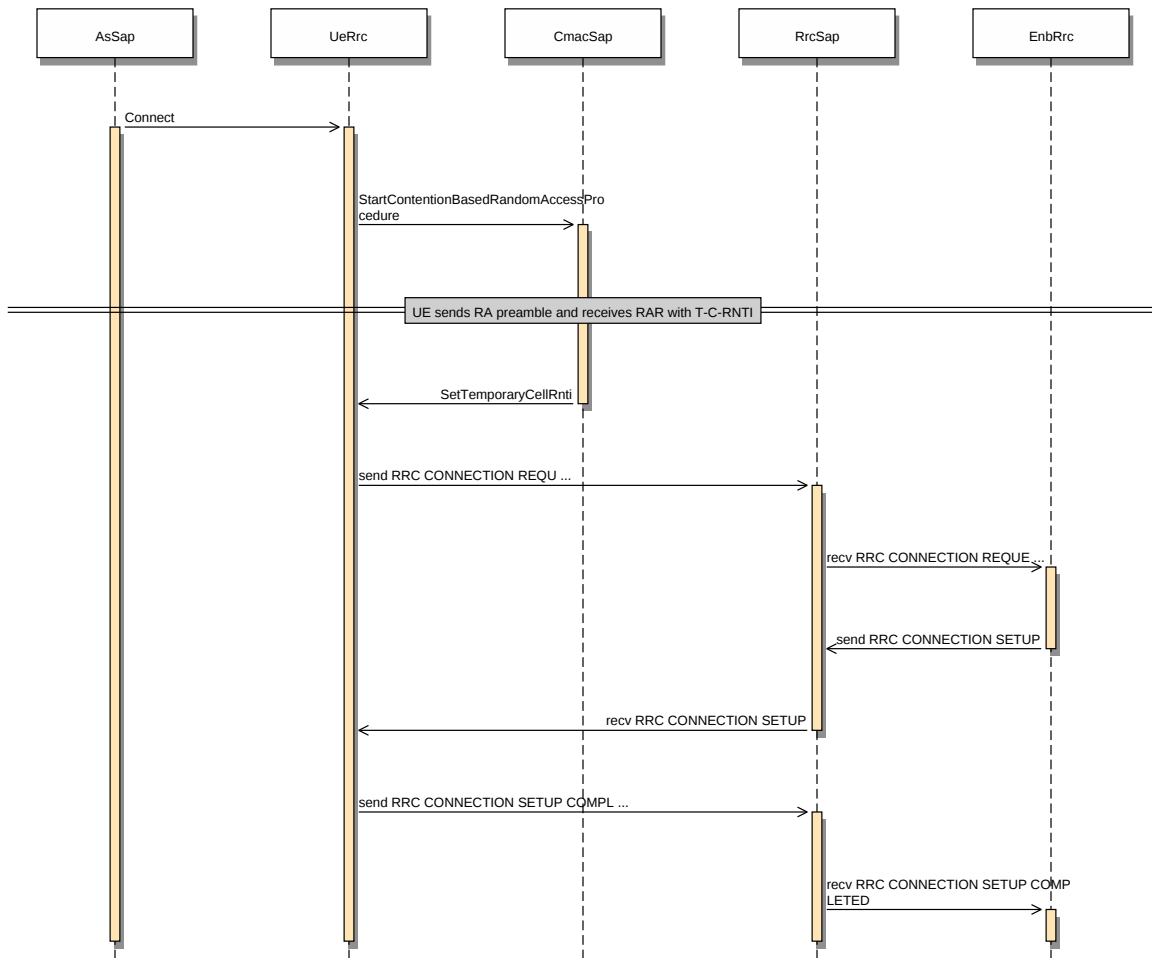


Fig. 19.39: Sequence diagram of the RRC Connection Establishment procedure

Table 19.2: Timers in RRC connection establishment procedure

Name	Location	Timer starts	Timer stops	Default duration	When timer expired
Connection request timeout	eNodeB RRC	New UE context added	Receive RRC CONNECTION REQUEST	15 ms (Max)	Remove UE context
Connection timeout (T300 timer)	UE RRC	Send RRC CONNECTION REQUEST	Receive RRC CONNECTION SETUP or REJECT	100 ms	Reset UE MAC
Connection setup timeout	eNodeB RRC	Send RRC CONNECTION SETUP	Receive RRC CONNECTION SETUP COMPLETE	100 ms	Remove UE context
Connection rejected timeout	eNodeB RRC	Send RRC CONNECTION REJECT	Never	30 ms	Remove UE context

Note: The value of connection request timeout timer at the eNB RRC should not be higher than the T300 timer at UE RRC. It is to make sure that the UE context is already removed at the eNB, once the UE will perform cell selection upon reaching the *connEstFailCount* count. Moreover, at the time of writing this document the *Cell Selection Evaluation* does not include the *Qoffset_{temp}* parameter, thus, it is not applied while selecting the same cell again.

Table 19.3: Counters in RRC connection establishment procedure

Name	Location	Msg	Monitored by	Default value	Limit not reached	Limit reached
ConnEst-Fail-Count	eNB MAC	RachConfigCommon in SIB2, HO REQ and HO Ack	UE RRC	1	Increment the local counter. Invalidated the prev SIB2 msg, and try random access with the same cell.	Reset the local counter and perform cell selection.

RRC connection reconfiguration

Figure *Sequence diagram of the RRC Connection Reconfiguration procedure* shows how the RRC Connection Reconfiguration procedure is modeled for the case where MobilityControlInfo is not provided, i.e., handover is not performed.

Figure *Sequence diagram of the RRC Connection Reconfiguration procedure for the handover case* shows how the RRC Connection Reconfiguration procedure is modeled for the case where MobilityControlInfo is provided, i.e., handover is to be performed. As specified in [TS36331], *After receiving the handover message, the UE attempts to access the target cell at the first available RACH occasion according to Random Access resource selection defined in [TS36321], i.e. the handover is asynchronous. Consequently, when allocating a dedicated preamble for the random access in the target cell, E-UTRA shall ensure it is available from the first RACH occasion the UE may use. Upon successful completion of the handover, the UE sends a message used to confirm the handover.* Note that the random access procedure in this case is non-contention based, hence in a real LTE system it differs slightly from the one used in RRC connection established. Also note that the RA Preamble ID is signaled via the Handover Command included in the X2 Handover Request ACK message sent from the target eNB to the source eNB; in particular, the preamble is included in the RACH-ConfigDedicated IE which is part of MobilityControlInfo.

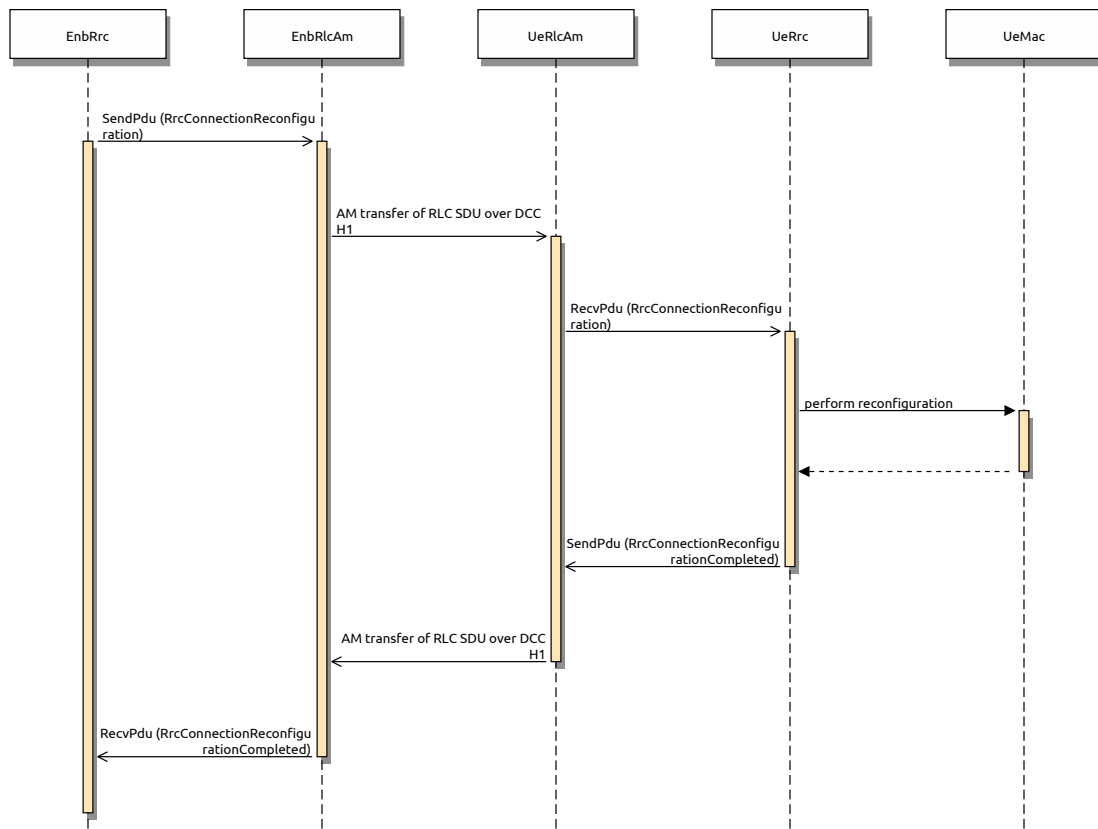


Fig. 19.40: Sequence diagram of the RRC Connection Reconfiguration procedure

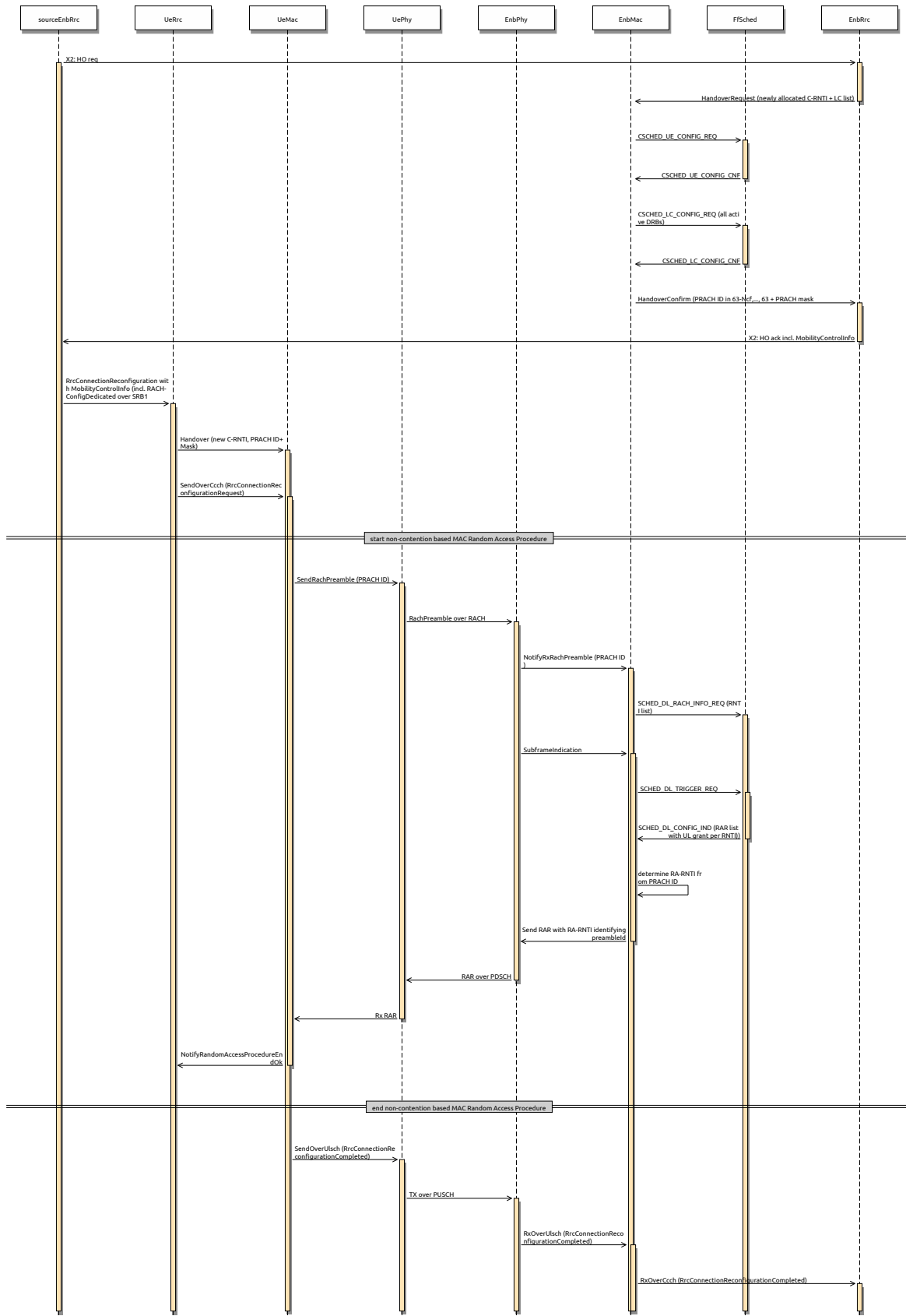


Fig. 19.41: Sequence diagram of the RRC Connection Reconfiguration procedure for the handover case

RRC protocol models

As previously anticipated, we provide two different models for the transmission and reception of RRC messages: *Ideal* and *Real*. Each of them is described in one of the following subsections.

Ideal RRC protocol model

According to this model, implemented in the classes *LteUeRrcProtocolIdeal* and *LteEnbRrcProtocolIdeal*, all RRC messages and information elements are transmitted between the eNB and the UE in an ideal fashion, without consuming radio resources and without errors. From an implementation point of view, this is achieved by passing the RRC data structure directly between the UE and eNB RRC entities, without involving the lower layers (PDCP, RLC, MAC, scheduler).

Real RRC protocol model

This model is implemented in the classes *LteUeRrcProtocolReal* and *LteEnbRrcProtocolReal* and aims at modeling the transmission of RRC PDUs as commonly performed in real LTE systems. In particular:

- for every RRC message being sent, a real RRC PDUs is created following the ASN.1 encoding of RRC PDUs and information elements (IEs) specified in [TS36331]. Some simplification are made with respect to the IEs included in the PDU, i.e., only those IEs that are useful for simulation purposes are included. For a detailed list, please see the IEs defined in *lte-rrc-sap.h* and compare with [TS36331].
- the encoded RRC PDUs are sent on Signaling Radio Bearers and are subject to the same transmission modeling used for data communications, thus including scheduling, radio resource consumption, channel errors, delays, retransmissions, etc.

Signaling Radio Bearer model

We now describe the Signaling Radio Bearer model that is used for the *Real* RRC protocol model.

- **SRB0** messages (over CCCH):
 - **RrcConnectionRequest**: in real LTE systems, this is an RLC TM SDU sent over resources specified in the UL Grant in the RAR (not in UL DCIs); the reason is that C-RNTI is not known yet at this stage. In the simulator, this is modeled as a real RLC TM RLC PDU whose UL resources are allocated by the scheduler upon call to `SCHED_DL_RACH_INFO_REQ`.
 - **RrcConnectionSetup**: in the simulator this is implemented as in real LTE systems, i.e., with an RLC TM SDU sent over resources indicated by a regular UL DCI, allocated with `SCHED_DL_RLC_BUFFER_REQ` triggered by the RLC TM instance that is mapped to LCID 0 (the CCCH).
- **SRB1** messages (over DCCH):
 - All the SRB1 messages modeled in the simulator (e.g., **RrcConnectionCompleted**) are implemented as in real LTE systems, i.e., with a real RLC SDU sent over RLC AM using DL resources allocated via Buffer Status Reports. See the RLC model documentation for details.
- **SRB2** messages (over DCCH):
 - According to [TS36331], “SRB1 is for RRC messages (which may include a piggybacked NAS message) as well as for NAS messages prior to the establishment of SRB2, all using DCCH logical channel”, whereas “SRB2 is for NAS messages, using DCCH logical channel” and “SRB2 has a lower-priority than SRB1 and is always configured by E-UTRAN after security activation”. Modeling security-related aspects is not a requirement of the LTE simulation model, hence we always use SRB1 and never activate SRB2.

ASN.1 encoding of RRC IE's

The messages defined in RRC SAP, common to all Ue/Enb SAP Users/Providers, are transported in a transparent container to/from a Ue/Enb. The encoding format for the different Information Elements are specified in [TS36331], using ASN.1 rules in the unaligned variant. The implementation in Ns3/Lte has been divided in the following classes:

- **Asn1Header** : Contains the encoding / decoding of basic ASN types
- **RrcAsn1Header** : Inherits **Asn1Header** and contains the encoding / decoding of common IE's defined in [TS36331]
- **Rrc specific messages/IEs classes** : A class for each of the messages defined in RRC SAP header

Asn1Header class - Implementation of base ASN.1 types

This class implements the methods to Serialize / Deserialize the ASN.1 types being used in [TS36331], according to the packed encoding rules in ITU-T X.691. The types considered are:

- **Boolean** : a boolean value uses a single bit (1=true, 0=false).
- **Integer** : a constrained integer (with min and max values defined) uses the minimum amount of bits to encode its range (max-min+1).
- **Bitstring** : a bistring will be copied bit by bit to the serialization buffer.
- **Octetstring** : not being currently used.
- **Sequence** : the sequence generates a preamble indicating the presence of optional and default fields. It also adds a bit indicating the presence of extension marker.
- **Sequence...Of** : the sequence...of type encodes the number of elements of the sequence as an integer (the subsequent elements will need to be encoded afterwards).
- **Choice** : indicates which element among the ones in the choice set is being encoded.
- **Enumeration** : is serialized as an integer indicating which value is used, among the ones in the enumeration, with the number of elements in the enumeration as upper bound.
- **Null** : the null value is not encoded, although its serialization function is defined to provide a clearer map between specification and implementation.

The class inherits from ns-3 Header, but **Deserialize()** function is declared pure virtual, thus inherited classes having to implement it. The reason is that deserialization will retrieve the elements in RRC messages, each of them containing different information elements.

Additionally, it has to be noted that the resulting byte length of a specific type/message can vary, according to the presence of optional fields, and due to the optimized encoding. Hence, the serialized bits will be processed using **PreSerialize()** function, saving the result in **m_serializationResult** Buffer. As the methods to read/write in a ns3 buffer are defined in a byte basis, the serialization bits are stored into **m_serializationPendingBits** attribute, until the 8 bits are set and can be written to buffer iterator. Finally, when invoking **Serialize()**, the contents of the **m_serializationResult** attribute will be copied to **Buffer::Iterator** parameter

RrcAsn1Header : Common IEs

As some Information Elements are being used for several RRC messages, this class implements the following common IE's:

- SrbToAddModList
- DrbToAddModList
- LogicalChannelConfig
- RadioResourceConfigDedicated
- PhysicalConfigDedicated
- SystemInformationBlockType1
- SystemInformationBlockType2
- RadioResourceConfigCommonSIB

Rrc specific messages/IEs classes

The following RRC SAP have been implemented:

- RrcConnectionRequest
- RrcConnectionSetup
- RrcConnectionSetupCompleted
- RrcConnectionReconfiguration
- RrcConnectionReconfigurationCompleted
- HandoverPreparationInfo
- RrcConnectionReestablishmentRequest
- RrcConnectionReestablishment
- RrcConnectionReestablishmentComplete
- RrcConnectionReestablishmentReject
- RrcConnectionRelease

19.1.11 NAS

The focus of the LTE-EPC model is on the NAS Active state, which corresponds to EMM Registered, ECM connected, and RRC connected. Because of this, the following simplifications are made:

- EMM and ECM are not modeled explicitly; instead, the NAS entity at the UE will interact directly with the MME to perform actions that are equivalent (with gross simplifications) to taking the UE to the states EMM Connected and ECM Connected;
- the NAS also takes care of multiplexing uplink data packets coming from the upper layers into the appropriate EPS bearer by using the Traffic Flow Template classifier (TftClassifier).
- the NAS does not support PLMN and CSG selection
- the NAS does not support any location update/paging procedure in idle mode

Figure *Sequence diagram of the attach procedure* shows how the simplified NAS model implements the attach procedure. Note that both the default and eventual dedicated EPS bearers are activated as part of this procedure.

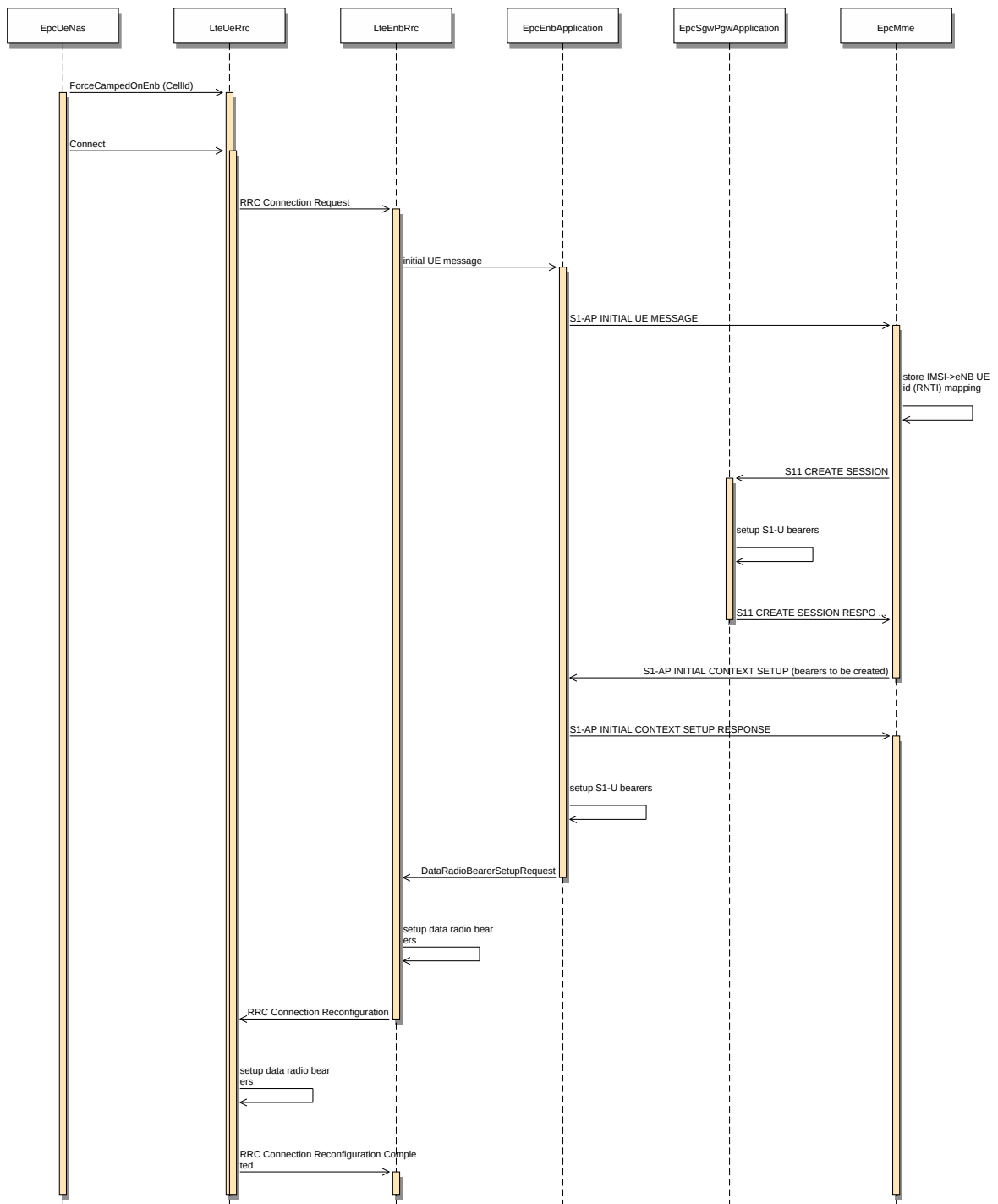


Fig. 19.42: Sequence diagram of the attach procedure

19.1.12 S1, S5 and S11

S1-U and S5 (user plane)

The S1-U and S5 interfaces are modeled in a realistic way by encapsulating data packets over GTP/UDP/IP, as done in real LTE-EPC systems. The corresponding protocol stack is shown in Figure *LTE-EPC data plane protocol stack*. As shown in the figure, there are two different layers of IP networking. The first one is the end-to-end layer, which provides end-to-end connectivity to the users; this layer involves the UEs, the PGW and the remote host (including eventual internet routers and hosts in between), but does not involve the eNB and the SGW. In this version of LTE, the EPC supports both IPv4 and IPv6 type users. The 3GPP unique 64 bit IPv6 prefix allocation process for each UE and PGW is followed here. Each EPC is assigned a unique 16 bit IPv4 and a 48 bit IPv6 network address from the pool of 7.0.0.0/8 and 7777:f00d::/32 respectively. In the end-to-end IP connection between UE and PGW, all addresses are configured using these prefixes. The PGW's address is used by all UEs as the gateway to reach the internet.

The second layer of IP networking is the EPC local area network. This involves all eNB nodes, SGW nodes and PGW nodes. This network is implemented as a set of point-to-point links which connect each eNB with its corresponding SGW node and a point-to-point link which connect each SGW node with its corresponding PGW node; thus, each SGW has a set of point-to-point devices, each providing connectivity to a different eNB. By default, a 10.x.y.z/30 subnet is assigned to each point-to-point link (a /30 subnet is the smallest subnet that allows for two distinct host addresses).

As specified by 3GPP, the end-to-end IP communications is tunneled over the local EPC IP network using GTP/UDP/IP. In the following, we explain how this tunneling is implemented in the EPC model. The explanation is done by discussing the end-to-end flow of data packets.

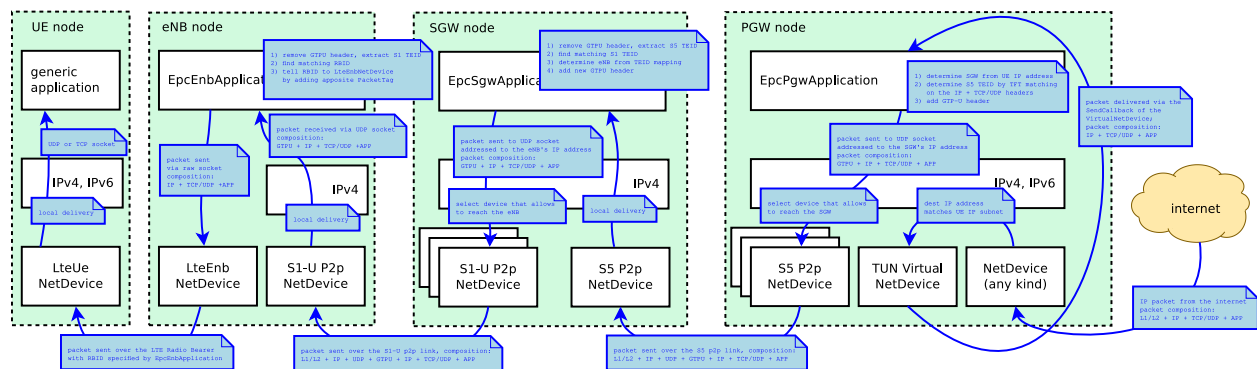


Fig. 19.43: Data flow in the downlink between the internet and the UE

To begin with, we consider the case of the downlink, which is depicted in Figure *Data flow in the downlink between the internet and the UE*. Downlink IPv4/IPv6 packets are generated from a generic remote host, and addressed to one of the UE device. Internet routing will take care of forwarding the packet to the generic NetDevice of the PGW node which is connected to the internet (this is the Gi interface according to 3GPP terminology). The PGW has a VirtualNetDevice which is assigned the base IPv4 address of the EPC network; hence, static routing rules will cause the incoming packet from the internet to be routed through this VirtualNetDevice. In case of IPv6 address as destination, a manual route towards the VirtualNetDevice is inserted in the routing table, containing the 48 bit IPv6 prefix from which all the IPv6 addresses of the UEs and PGW are configured. Such device starts the GTP/UDP/IP tunneling procedure, by forwarding the packet to a dedicated application in the PGW node which is called *EpcPgwApplication*. This application does the following operations:

1. it determines the SGW node to which it must route the traffic for this UE, by looking at the IP destination address (which is the address of the UE);
2. it classifies the packet using Traffic Flow Templates (TFTs) to identify to which EPS Bearer it belongs. EPS bearers have a one-to-one mapping to S5 Bearers, so this operation returns the GTP-U Tunnel Endpoint Identifier (TEID) to which the packet belongs;

3. it adds the corresponding GTP-U protocol header to the packet;
4. finally, it sends the packet over a UDP socket to the S5 point-to-point NetDevice, addressed to the appropriate SGW.

As a consequence, the end-to-end IP packet with newly added IP, UDP and GTP headers is sent through one of the S5 links to the SGW, where it is received and delivered locally (as the destination address of the outermost IP header matches the SGW IP address). The local delivery process will forward the packet, via an UDP socket, to a dedicated application called EpcSgwApplication. This application then performs the following operations:

1. it determines the eNB node to which the UE is attached, by looking at the S5 TEID;
2. it maps the S5 TEID to get the S1 TEID. EPS bearers have a one-to-one mapping to S1-U Bearers, so this operation returns the S1 GTP-U Tunnel Endpoint Identifier (TEID) to which the packet belongs;
3. it adds a new GTP-U protocol header to the packet;
4. finally, it sends the packet over a UDP socket to the S1-U point-to-point NetDevice, addressed to the eNB to which the UE is attached.

Finally, the end-to-end IP packet with newly added IP, UDP and GTP headers is sent through one of the S1 links to the eNB, where it is received and delivered locally (as the destination address of the outermost IP header matches the eNB IP address). The local delivery process will forward the packet, via an UDP socket, to a dedicated application called EpcEnbApplication. This application then performs the following operations:

1. it removes the GTP header and retrieves the S1 TEID which is contained in it;
2. leveraging on the one-to-one mapping between S1-U bearers and Radio Bearers (which is a 3GPP requirement), it determines the Bearer ID (BID) to which the packet belongs;
3. it records the BID in a dedicated tag called EpsBearerTag, which is added to the packet;
4. it forwards the packet to the LteEnbNetDevice of the eNB node via a raw packet socket

Note that, at this point, the outmost header of the packet is the end-to-end IP header, since the IP/UDP/GTP headers of the S1 protocol stack have already been stripped. Upon reception of the packet from the EpcEnbApplication, the LteEnbNetDevice will retrieve the BID from the EpsBearerTag, and based on the BID will determine the Radio Bearer instance (and the corresponding PDCP and RLC protocol instances) which are then used to forward the packet to the UE over the LTE radio interface. Finally, the LteUeNetDevice of the UE will receive the packet, and delivery it locally to the IP protocol stack, which will in turn delivery it to the application of the UE, which is the end point of the downlink communication.

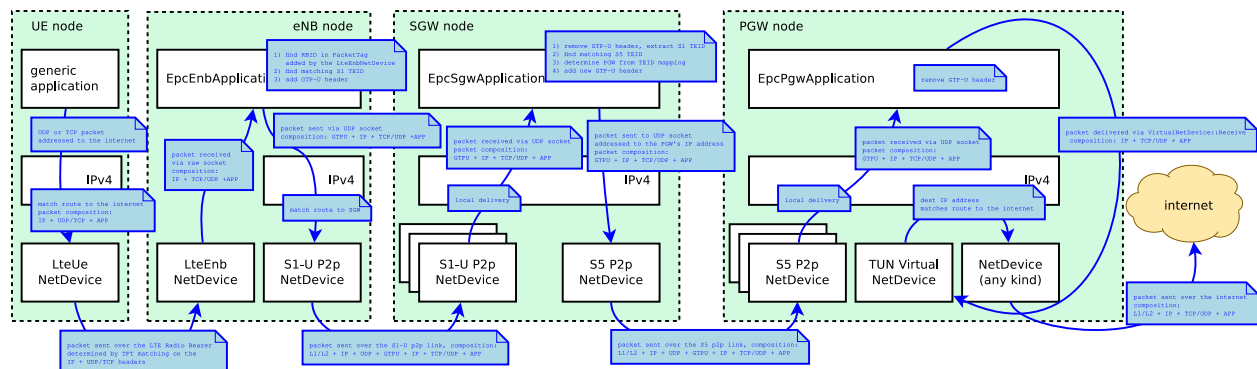


Fig. 19.44: Data flow in the uplink between the UE and the internet

The case of the uplink is depicted in Figure *Data flow in the uplink between the UE and the internet*. Uplink IP packets are generated by a generic application inside the UE, and forwarded by the local TCP/IP stack to the LteUeNetDevice of the UE. The LteUeNetDevice then performs the following operations:

1. it classifies the packet using TFTs and determines the Radio Bearer to which the packet belongs (and the corresponding RBID);
2. it identifies the corresponding PDCP protocol instance, which is the entry point of the LTE Radio Protocol stack for this packet;
3. it sends the packet to the eNB over the LTE Radio Protocol stack.

The eNB receives the packet via its `LteEnbNetDevice`. Since there is a single PDCP and RLC protocol instance for each Radio Bearer, the `LteEnbNetDevice` is able to determine the BID of the packet. This BID is then recorded onto an `EpsBearerTag`, which is added to the packet. The `LteEnbNetDevice` then forwards the packet to the `EpcEnbApplication` via a raw packet socket.

Upon receiving the packet, the `EpcEnbApplication` performs the following operations:

1. it retrieves the BID from the `EpsBearerTag` in the packet;
2. it determines the corresponding EPS Bearer instance and GTP-U TEID by leveraging on the one-to-one mapping between S1-U bearers and Radio Bearers;
3. it adds a GTP-U header on the packet, including the TEID determined previously;
4. it sends the packet to the SGW node via the UDP socket connected to the S1-U point-to-point net device.

At this point, the packet contains the S1-U IP, UDP and GTP headers in addition to the original end-to-end IP header. When the packet is received by the corresponding S1-U point-to-point `NetDevice` of the SGW node, it is delivered locally (as the destination address of the outmost IP header matches the address of the point-to-point net device). The local delivery process will forward the packet to the `EpcSgwApplication` via the corresponding UDP socket. The `EpcSgwApplication` then performs the following operations:

1. it removes the GTP header and retrieves the S1-U TEID;
2. it maps the S1-U TEID to get the S5 TEID to which the packet belongs;
3. it determines the PGW to which it must send the packet from the TEID mapping;
4. it add a new GTP-U protocol header to the packet;
5. finally, it sends the packet over a UDP socket to the S5 point-to-point `NetDevice`, addressed to the corresponding PGW.

At this point, the packet contains the S5 IP, UDP and GTP headers in addition to the original end-to-end IP header. When the packet is received by the corresponding S5 point-to-point `NetDevice` of the PGW node, it is delivered locally (as the destination address of the outmost IP header matches the address of the point-to-point net device). The local delivery process will forward the packet to the `EpcPgwApplication` via the corresponding UDP socket. The `EpcPgwApplication` then removes the GTP header and forwards the packet to the `VirtualNetDevice`. At this point, the outmost header of the packet is the end-to-end IP header. Hence, if the destination address within this header is a remote host on the internet, the packet is sent to the internet via the corresponding `NetDevice` of the PGW. In the event that the packet is addressed to another UE, the IP stack of the PGW will redirect the packet again to the `VirtualNetDevice`, and the packet will go through the downlink delivery process in order to reach its destination UE.

Note that the EPS Bearer QoS is not enforced on the S1-U and S5 links, it is assumed that the overprovisioning of the link bandwidth is sufficient to meet the QoS requirements of all bearers.

S1AP

The S1-AP interface provides control plane interaction between the eNB and the MME. In the simulator, this interface is modeled in a realistic fashion transmitting the encoded S1AP messages and information elements specified in [TS36413] on the S1-MME link.

The S1-AP primitives that are modeled are:

- INITIAL UE MESSAGE
- INITIAL CONTEXT SETUP REQUEST
- INITIAL CONTEXT SETUP RESPONSE
- PATH SWITCH REQUEST
- PATH SWITCH REQUEST ACKNOWLEDGE

S5 and S11

The S5 interface provides control plane interaction between the SGW and the PGW. The S11 interface provides control plane interaction between the SGW and the MME. Both interfaces use the GPRS Tunneling Protocol (GTPv2-C) to tunnel signalling messages [TS29274] and use UDP as transport protocol. In the simulator, these interfaces and protocol are modeled in a realistic fashion transmitting the encoded GTP-C messages.

The GTPv2-C primitives that are modeled are:

- CREATE SESSION REQUEST
- CREATE SESSION RESPONSE
- MODIFY BEARER REQUEST
- MODIFY BEARER RESPONSE
- DELETE SESSION REQUEST
- DELETE SESSION RESPONSE
- DELETE BEARER COMMAND
- DELETE BEARER REQUEST
- DELETE BEARER RESPONSE

Of these primitives, the first two are used upon initial UE attachment for the establishment of the S1-U and S5 bearers. Section [NAS](#) shows the implementation of the attach procedure. The other primitives are used during the handover to switch the S1-U bearers from the source eNB to the target eNB as a consequence of the reception by the MME of a PATH SWITCH REQUEST S1-AP message.

19.1.13 X2

The X2 interface interconnects two eNBs [TS36420]. From a logical point of view, the X2 interface is a point-to-point interface between the two eNBs. In a real E-UTRAN, the logical point-to-point interface should be feasible even in the absence of a physical direct connection between the two eNBs. In the X2 model implemented in the simulator, the X2 interface is a point-to-point link between the two eNBs. A point-to-point device is created in both eNBs and the two point-to-point devices are attached to the point-to-point link.

For a representation of how the X2 interface fits in the overall architecture of the LENA simulation model, the reader is referred to the figure *Overview of the LTE-EPC simulation model*.

The X2 interface implemented in the simulator provides detailed implementation of the following elementary procedures of the Mobility Management functionality [TS36423]:

- Handover Request procedure
- Handover Request Acknowledgement procedure
- SN Status Transfer procedure
- UE Context Release procedure

These procedures are involved in the X2-based handover. You can find the detailed description of the handover in section 10.1.2.1 of [TS36300]. We note that the simulator model currently supports only the *seamless handover* as defined in Section 2.6.3.1 of [Sesia2009]; in particular, *lossless handover* as described in Section 2.6.3.2 of [Sesia2009] is not supported at the time of this writing.

Figure *Sequence diagram of the X2-based handover* below shows the interaction of the entities of the X2 model in the simulator. The shaded labels indicate the moments when the UE or eNodeB transition to another RRC state.

The figure also shows two timers within the handover procedure: the *handover leaving timer* is maintained by the source eNodeB, while the *handover joining timer* by the target eNodeB. The duration of the timers can be configured in the `HandoverLeavingTimeoutDuration` and `HandoverJoiningTimeoutDuration` attributes of the respective `LteEnbRrc` instances. When one of these timers expire, the handover procedure is considered as failed.

However, there is no proper handling of handover failure in the current version of LTE module. Users should tune the simulation properly in order to avoid handover failure, otherwise unexpected behaviour may occur. Please refer to Section *Tuning simulation with handover* of the User Documentation for some tips regarding this matter.

The X2 model is an entity that uses services from:

- the X2 interfaces,
 - They are implemented as Sockets on top of the point-to-point devices.
 - They are used to send/receive X2 messages through the X2-C and X2-U interfaces (i.e. the point-to-point device attached to the point-to-point link) towards the peer eNB.
- the S1 application.
 - Currently, it is the `EpcEnbApplication`.
 - It is used to get some information needed for the Elementary Procedures of the X2 messages.

and it provides services to:

- the RRC entity (X2 SAP)
 - to send/receive RRC messages. The X2 entity sends the RRC message as a transparent container in the X2 message. This RRC message is sent to the UE.

Figure *Implementation Model of X2 entity and SAPs* shows the implementation model of the X2 entity and its relationship with all the other entities and services in the protocol stack.

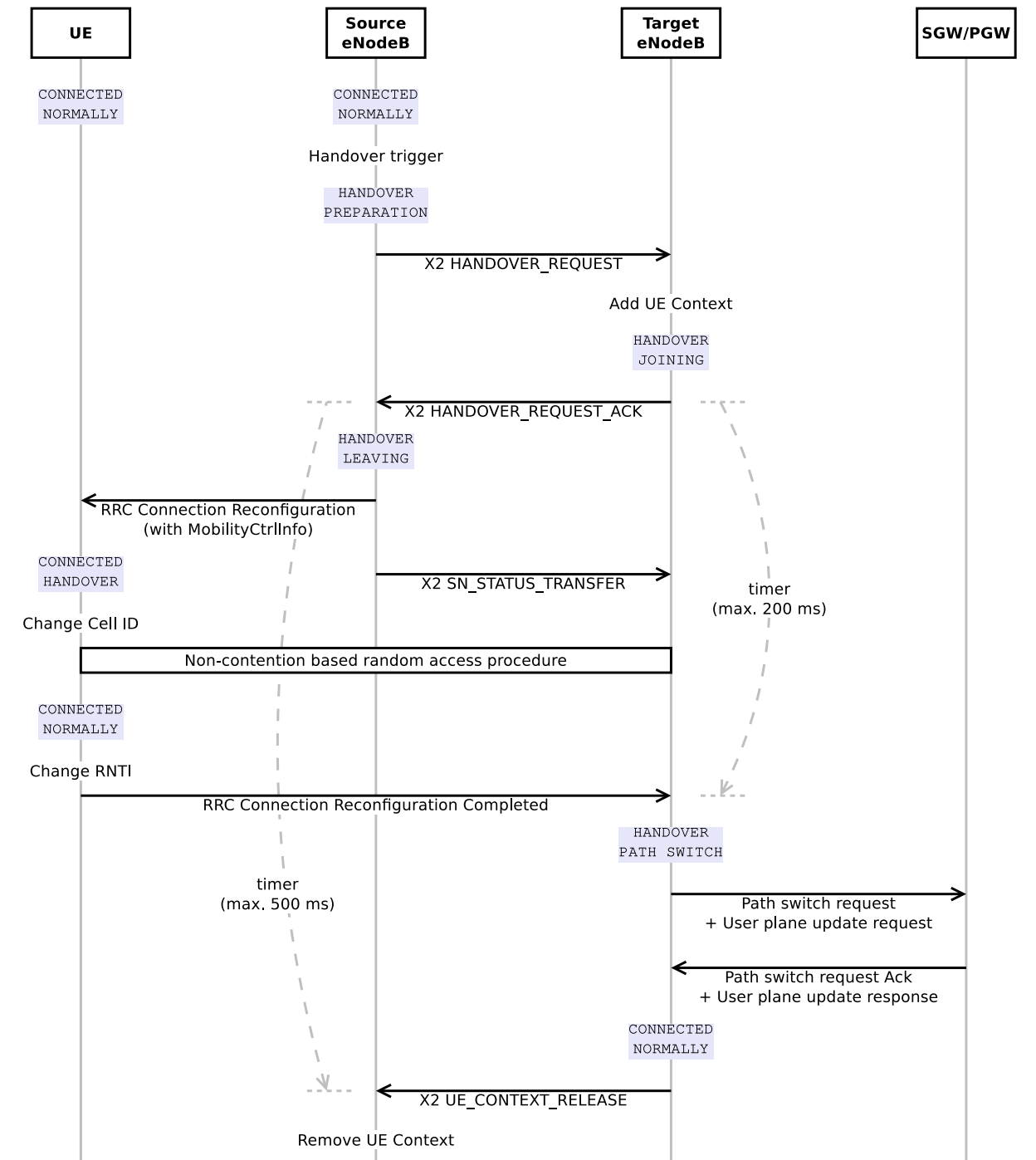


Fig. 19.45: Sequence diagram of the X2-based handover

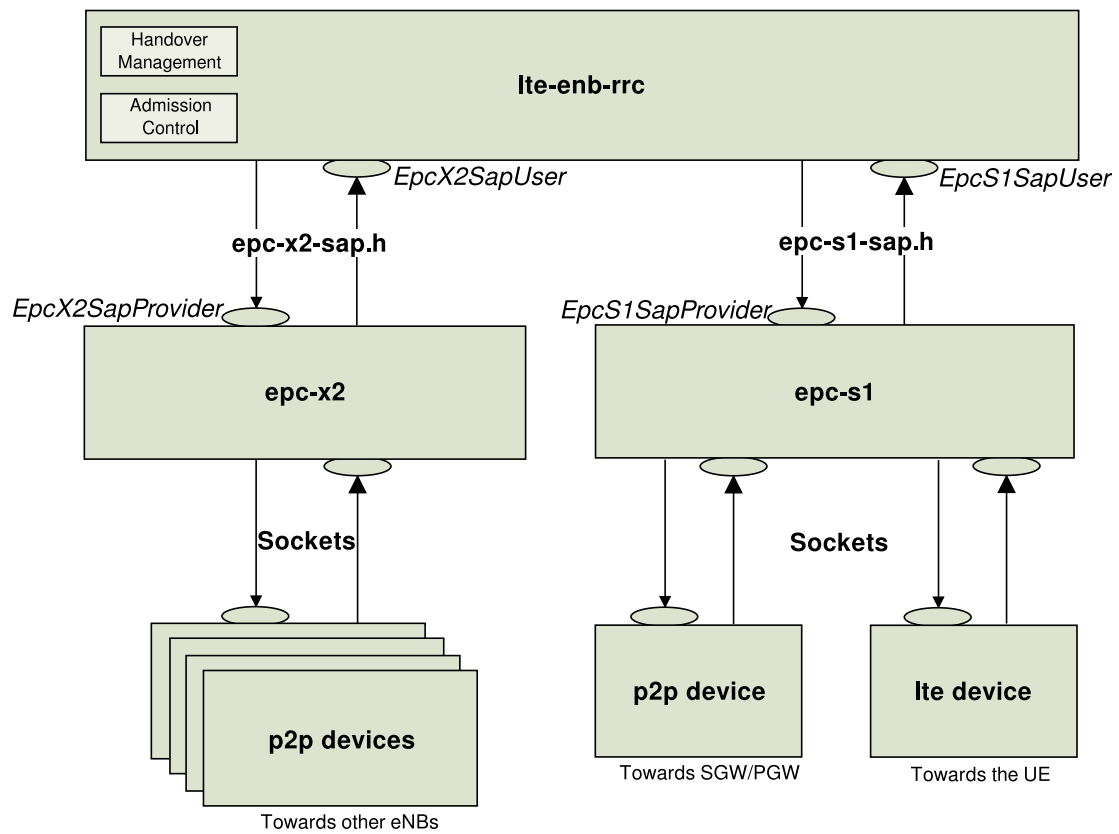


Fig. 19.46: Implementation Model of X2 entity and SAPs

The RRC entity manages the initiation of the handover procedure. This is done in the Handover Management submodule of the eNB RRC entity. The target eNB may perform some Admission Control procedures. This is done in the Admission Control submodule. Initially, this submodule will accept any handover request.

X2 interfaces

The X2 model contains two interfaces:

- the X2-C interface. It is the control interface and it is used to send the X2-AP PDUs (i.e. the elementary procedures).
- the X2-U interface. It is used to send the bearer data when there is *DL forwarding*.

Figure *X2 interface protocol stacks* shows the protocol stacks of the X2-U interface and X2-C interface modeled in the simulator.

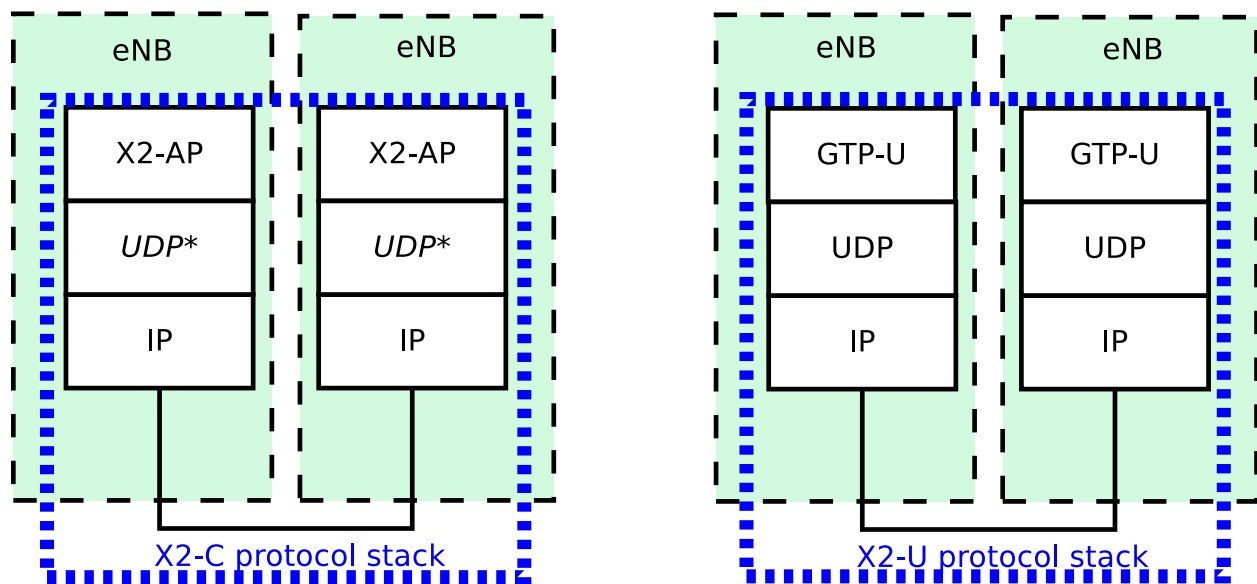


Fig. 19.47: X2 interface protocol stacks

X2-C

The X2-C interface is the control part of the X2 interface and it is used to send the X2-AP PDUs (i.e. the elementary procedures).

In the original X2 interface control plane protocol stack, SCTP is used as the transport protocol but currently, the SCTP protocol is not modeled in the ns-3 simulator and its implementation is out-of-scope of the project. The UDP protocol is used as the datagram oriented protocol instead of the SCTP protocol.

X2-U

The X2-U interface is used to send the bearer data when there is *DL forwarding* during the execution of the X2-based handover procedure. Similarly to what done for the S1-U interface, data packets are encapsulated over GTP/UDP/IP when being sent over this interface. Note that the EPS Bearer QoS is not enforced on the X2-U links, it is assumed that the overprovisioning of the link bandwidth is sufficient to meet the QoS requirements of all bearers.

X2 Service Interface

The X2 service interface is used by the RRC entity to send and receive messages of the X2 procedures. It is divided into two parts:

- the EpcX2SapProvider part is provided by the X2 entity and used by the RRC entity and
- the EpcX2SapUser part is provided by the RRC entity and used by the RRC entity.

The primitives that are supported in our X2-C model are described in the following subsections.

X2-C primitives for handover execution

The following primitives are used for the X2-based handover:

- HANDOVER REQUEST
- HANDOVER REQUEST ACK
- HANDOVER PREPARATION FAILURE
- SN STATUS STRANSFER
- UE CONTEXT RELEASE

all the above primitives are used by the currently implemented RRC model during the preparation and execution of the handover procedure. Their usage interacts with the RRC state machine; therefore, they are not meant to be used for code customization, at least unless it is desired to modify the RRC state machine.

X2-C SON primitives

The following primitives can be used to implement Self-Organized Network (SON) functionalities:

- LOAD INFORMATION
- RESOURCE STATUS UPDATE

note that the current RRC model does not actually use these primitives, they are included in the model just to make it possible to develop SON algorithms included in the RRC logic that make use of them.

As a first example, we show here how the load information primitive can be used. We assume that the LteEnbRrc has been modified to include the following new member variables:

```
std::vector<EpcX2Sap::UlInterferenceOverloadIndicationItem>
    m_currentUlInterferenceOverloadIndicationList;
std::vector<EpcX2Sap::UlHighInterferenceInformationItem>
    m_currentUlHighInterferenceInformationList;
EpcX2Sap::RelativeNarrowbandTxBand m_currentRelativeNarrowbandTxBand;
```

for a detailed description of the type of these variables, we suggest to consult the file `epc-x2-sap.h`, the corresponding doxygen documentation, and the references therein to the relevant sections of 3GPP TS 36.423. Now, assume that at run time these variables have been set to meaningful values following the specifications just mentioned. Then, you can add the following code in the `LteEnbRrc` class implementation in order to send a load information primitive:

```
EpcX2Sap::CellInformationItem cii;
cii.sourceCellId = m_cellId;
cii.ulInterferenceOverloadIndicationList = m_currentUlInterferenceOverloadIndicationList;
cii.ulHighInterferenceInformationList = m_currentUlHighInterferenceInformationList;
cii.relativeNarrowbandTxBand = m_currentRelativeNarrowbandTxBand;

EpcX2Sap::LoadInformationParams params;
params.targetCellId = cellId;
params.cellInformationList.push_back(cii);
m_x2SapProvider->SendLoadInformation(params);
```

The above code allows the source eNB to send the message. The method `LteEnbRrc::DoRecvLoadInformation` will be called when the target eNB receives the message. The desired processing of the load information should therefore be implemented within that method.

In the following second example we show how the resource status update primitive is used. We assume that the `LteEnbRrc` has been modified to include the following new member variable:

```
EpcX2Sap::CellMeasurementResultItem m_cmri;
```

similarly to before, we refer to `epc-x2-sap.h` and the references therein for detailed information about this variable type. Again, we assume that the variable has been already set to a meaningful value. Then, you can add the following code in order to send a resource status update:

```
EpcX2Sap::ResourceStatusUpdateParams params;
params.targetCellId = cellId;
params.cellMeasurementResultList.push_back(m_cmri);
m_x2SapProvider->SendResourceStatusUpdate(params);
```

The method `eEnbRrc::DoRecvResourceStatusUpdate` will be called when the target eNB receives the resource status update message. The desired processing of this message should therefore be implemented within that method.

Finally, we note that the setting and processing of the appropriate values for the variable passed to the above described primitives is deemed to be specific of the SON algorithm being implemented, and hence is not covered by this documentation.

Unsupported primitives

Mobility Robustness Optimization primitives such as Radio Link Failure indication and Handover Report are not supported at this stage.

19.1.14 S11

The S11 interface provides control plane interaction between the SGW and the MME using the GTPv2-C protocol specified in [TS29274]. In the simulator, this interface is modeled in an ideal fashion, with direct interaction between the SGW and the MME objects, without actually implementing the encoding of the messages and without actually transmitting any PDU on any link.

The S11 primitives that are modeled are:

- CREATE SESSION REQUEST
- CREATE SESSION RESPONSE
- MODIFY BEARER REQUEST
- MODIFY BEARER RESPONSE

Of these primitives, the first two are used upon initial UE attachment for the establishment of the S1-U bearers; the other two are used during handover to switch the S1-U bearers from the source eNB to the target eNB as a consequence of the reception by the MME of a PATH SWITCH REQUEST S1-AP message.

19.1.15 Power Control

This section describes the ns-3 implementation of Downlink and Uplink Power Control.

Downlink Power Control

Since some of Frequency Reuse Algorithms require Downlink Power Control, this feature was also implemented in ns-3.

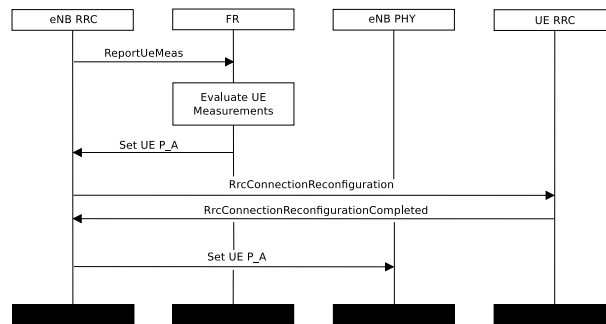


Fig. 19.48: Sequence diagram of Downlink Power Control

Figure *Sequence diagram of Downlink Power Control* shows the sequence diagram of setting downlink P_A value for UE, highlighting the interactions between the RRC and the other entities. FR algorithm triggers RRC to change P_A values for UE. Then RRC starts RrcConnectionReconfiguration function to inform UE about new configuration. After successful RrcConnectionReconfiguration, RRC can set P_A value for UE by calling function SetPa from CphySap, value is saved in new map m_paMap which contain P_A values for each UE served by eNb.

When LteEnbPhy starts new subframe, DCI control messages are processed to get vector of used RBs. Now also GeneratePowerAllocationMap(uint16_t rnti, int rbId) function is also called. This function check P_A value for UE, generate power for each RB and store it in m_dlPowerAllocationMap. Then this map is used by CreateTxPowerSpectralDensityWithPowerAllocation function to create Ptr<SpectrumValue> txPsd.

PdschConfigDedicated (TS 36.331, 6.3.2 PDSCH-Config) was added in LteRrcSap::PhysicalConfigDedicated struct, which is used in RrcConnectionReconfiguration process.

Uplink Power Control

Uplink power control controls the transmit power of the different uplink physical channels. This functionality is described in 3GPP TS 36.213 section 5.

Uplink Power Control is enabled by default, and can be disabled by attribute system:

```
Config::SetDefault("ns3::LteUePhy::EnableUplinkPowerControl", BooleanValue(false));
```

Two Uplink Power Control mechanisms are implemented:

- Open Loop Uplink Power Control: the UE transmission power depends on estimation of the downlink path-loss and channel configuration
- Closed Loop Uplink Power Control: as in Open Loop, in addition eNB can control the UE transmission power by means of explicit Transmit Power Control TPC commands transmitted in the downlink.

To switch between these two mechanism types, one should change parameter:

```
Config::SetDefault("ns3::LteUePowerControl::ClosedLoop", BooleanValue(true));
```

By default, Closed Loop Power Control is enabled.

Two modes of Closed Loop Uplink Power Control are available:

- Absolute mode: TxPower is computed with absolute TPC values
- Accumulative mode: TxPower is computed with accumulated TPC values

To switch between these two modes, one should change parameter:

```
Config::SetDefault("ns3::LteUePowerControl::AccumulationEnabled", BooleanValue(true));
```

By default, Accumulation Mode is enabled and TPC commands in DL-DCI are set by all schedulers to 1, what is mapped to value of 0 in Accumulation Mode.

Uplink Power Control for PUSCH

The setting of the UE Transmit power for a Physical Uplink Shared Channel (PUSCH) transmission is defined as follows:

- If the UE transmits PUSCH without a simultaneous PUCCH for the serving cell c , then the UE transmit power $P_{PUSCH,c}(i)$ for PUSCH transmission in subframe i for the serving cell c is given by:

$$P_{PUSCH,c}(i) = \min \left\{ 10 \log_{10}(M_{PUSCH,c}(i)) + P_{O_PUSCH,c}(j) + \alpha_c(j) * PL_c + \Delta_{TF,c}(i) + f_c(i) \right\} [dBm]$$

- If the UE transmits PUSCH simultaneous with PUCCH for the serving cell c , then the UE transmit power $P_{PUSCH,c}(i)$ for the PUSCH transmission in subframe i for the serving cell c is given by:

$$P_{PUSCH,c}(i) = \min \left\{ 10 \log_{10}(\hat{P}_{CMAX,c}(i) - \hat{P}_{PUCCH}(i)) + 10 \log_{10}(M_{PUSCH,c}(i)) + P_{O_PUSCH,c}(j) + \alpha_c(j) * PL_c + \Delta_{TF,c}(i) + f_c(i) \right\} [dBm]$$

Since Uplink Power Control for PUCCH is not implemented, this case is not implemented as well.

- If the UE is not transmitting PUSCH for the serving cell c , for the accumulation of TPC command received with DCI format 3/3A for PUSCH, the UE shall assume that the UE transmit power $P_{PUSCH,c}(i)$ for the PUSCH transmission in subframe i for the serving cell c is computed by

$$P_{PUSCH,c}(i) = \min \left\{ P_{CMAX,c}(i), P_{O_PUSCH,c}(1) + \alpha_c(1) * PL_c + f_c(i) \right\} [dBm]$$

where:

- $P_{CMAX,c}(i)$ is the configured UE transmit power defined in 3GPP 36.101. Table 6.2.2-1 in subframe i for serving cell c and $\hat{P}_{CMAX,c}(i)$ is the linear value of $P_{CMAX,c}(i)$. Default value for $P_{CMAX,c}(i)$ is 23 dBm
- $M_{PUSCH,c}(i)$ is the bandwidth of the PUSCH resource assignment expressed in number of resource blocks valid for subframe i and serving cell c .
- $P_{O_PUSCH,c}(j)$ is a parameter composed of the sum of a component $P_{O_NOMINAL_PUSCH,c}(j)$ provided from higher layers for $j = 0, 1$ and a component $P_{O_UE_PUSCH,c}(j)$ provided by higher layers for $j = 0, 1$ for serving cell c . SIB2 message needs to be extended to carry these two components, but currently they can be set via attribute system:


```
Config::SetDefault("ns3::LteUePowerControl::PoNominalPusch", IntegerValue(-90));
Config::SetDefault("ns3::LteUePowerControl::PoUePusch", IntegerValue(7));
```

- $\alpha_c(j)$ is a 3-bit parameter provided by higher layers for serving cell c . For $j = 0, 1$, $\alpha_c \in \{0, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1\}$ For $j = 2$, $\alpha_c(j) = 1$. This parameter is configurable by attribute system:

```
Config::SetDefault("ns3::LteUePowerControl::Alpha", DoubleValue(0.8));
```

- PL_c is the downlink pathloss estimate calculated in the UE for serving cell c in dB and $PL_c = referenceSignalPower \sim higherlayerFilteredRSRP$, where $referenceSignalPower$ is provided by higher layers and RSRP. $referenceSignalPower$ is provided in SIB2 message
- $\Delta_{TF,c}(i) = 10 \log_{10}((2^{BPRE \cdot K_s} - 1) \cdot \beta_{offset}^{PUSCH})$ for $K_s = 1.25$ and $\Delta_{TF,c}(i) = 0$ for $K_s = 0$. Only second case is implemented.
- $f_c(i)$ is component of Closed Loop Power Control. It is the current PUSCH power control adjustment state for serving cell c .

If Accumulation Mode is enabled $f_c(i)$ is given by:

$$f_c(i) = f_c(i-1) + \delta_{PUSCH,c}(i - K_{PUSCH})$$

where: $\delta_{PUSCH,c}$ is a correction value, also referred to as a TPC command and is included in PDCCH with DCI; $\delta_{PUSCH,c}(i - K_{PUSCH})$ was signalled on PDCCH/EPDCCH with DCI for serving cell c on subframe $(i - K_{PUSCH})$; $K_{PUSCH} = 4$ for FDD.

If UE has reached $P_{CMAX,c}(i)$ for serving cell c , positive TPC commands for serving cell c are not be accumulated. If UE has reached minimum power, negative TPC commands are not be accumulated. Minimum UE power is defined in TS36.101 section 6.2.3. Default value is -40 dBm.

If Accumulation Mode is not enabled $f_c(i)$ is given by:

$$f_c(i) = \delta_{PUSCH,c}(i - K_{PUSCH})$$

where: $\delta_{PUSCH,c}$ is a correction value, also referred to as a TPC command and is included in PDCCH with DCI; $\delta_{PUSCH,c}(i - K_{PUSCH})$ was signalled on PDCCH/EPDCCH with DCI for serving cell c on subframe $(i - K_{PUSCH})$; $K_{PUSCH} = 4$ for FDD.

Mapping of TPC Command Field in DCI format 0/3/4 to absolute and accumulated $\delta_{PUSCH,c}$ values is defined in TS36.231 section 5.1.1.1 Table 5.1.1.1-2

Uplink Power Control for PUCCH

Since all uplink control messages are an ideal messages and do not consume any radio resources, Uplink Power Control for PUCCH is not needed and it is not implemented.

Uplink Power Control for SRS

The setting of the UE Transmit power P_{SRS} for the SRS transmitted on subframe i for serving cell c is defined by

$$P_{PUSCH,c}(i) = \min \left\{ P_{SRS_OFFSET,c}(m) + 10 \log_{10}(M_{SRS,c}) + P_{O_PUSCH,c}(j) + \alpha_c(j) * PL_c + f_c(i) \right\} [dBm]$$

where:

- $P_{CMAX,c}(i)$ is the configured UE transmit power defined in 3GPP 36.101. Table 6.2.2-1. Default value for $P_{CMAX,c}(i)$ is 23 dBm
- $P_{SRS_OFFSET,c}(m)$ is semi-statically configured by higher layers for $m = 0, 1$ for serving cell c . For SRS transmission given trigger type 0 then $m = 0, 1$ and for SRS transmission given trigger type 1 then $m = 1$. For $K_s = 0$ $P_{Srs_Offset_Value}$ is computed with equation:

$$P_{SRS_OFFSET,c}(m)_{value} = -10.5 + P_{SRS_OFFSET,c}(m) * 1.5 [dBm]$$

This parameter is configurable by attribute system:

```
Config::SetDefault("ns3::LteUePowerControl::PsrsOffset", IntegerValue(7));
```

- $M_{SRS,c}$ is the bandwidth of the SRS transmission in subframe i for serving cell c expressed in number of resource blocks. In current implementation SRS is sent over entire UL bandwidth.
- $f_c(i)$ is the current PUSCH power control adjustment state for serving cell c , as defined in *Uplink Power Control for PUSCH*
- $P_{O_PUSCH,c}(j)$ and $\alpha_c(j)$ are parameters as defined in *Uplink Power Control for PUSCH*, where $j = 1$.

19.1.16 Fractional Frequency Reuse

Overview

This section describes the ns-3 support for Fractional Frequency Reuse algorithms. All implemented algorithms are described in [ASHamza2013]. Currently 7 FR algorithms are implemented:

- ns3::LteFrNoOpAlgorithm
- ns3::LteFrHardAlgorithm
- ns3::LteFrStrictAlgorithm
- ns3::LteFrSoftAlgorithm
- ns3::LteFfrSoftAlgorithm
- ns3::LteFfrEnhancedAlgorithm
- ns3::LteFfrDistributedAlgorithm

New LteFfrAlgorithm class was created and it is a abstract class for Frequency Reuse algorithms implementation. Also, two new SAPs between FR-Scheduler and FR-RRC were added.

Figure *Sequence diagram of Scheduling with FR algorithm* shows the sequence diagram of scheduling process with FR algorithm. In the beginning of scheduling process, scheduler asks FR entity for available RBGs. According to implementation FR returns all RBGs available in cell or filter them based on its policy. Then when trying to assign some RBG to UE, scheduler asks FR entity if this RBG is allowed for this UE. When FR returns true, scheduler can assign this RBG to this UE, if not scheduler is checking another RBG for this UE. Again, FR response depends on implementation and policy applied to UE.

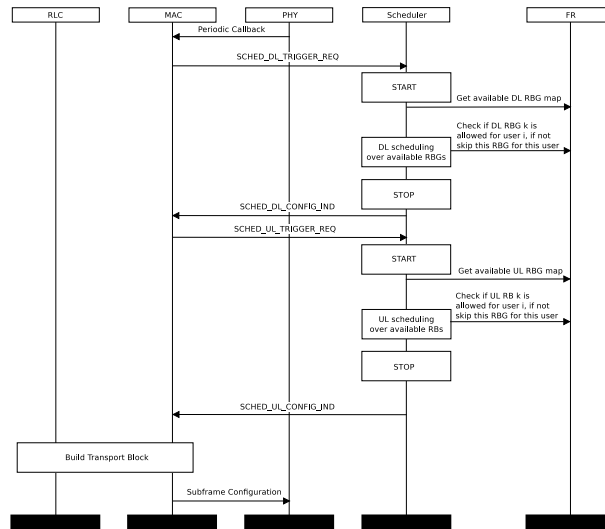


Fig. 19.49: Sequence diagram of Scheduling with FR algorithm

Supported FR algorithms

No Frequency Reuse

The NoOp FR algorithm (LteFrNoOpAlgorithm class) is implementation of Full Frequency Reuse scheme, that means no frequency partitioning is performed between eNBs of the same network (frequency reuse factor, FRF equals 1). eNBs use entire system bandwidth and transmit with uniform power over all RBGs. It is the simplest scheme and is the basic way of operating an LTE network. This scheme allows for achieving the high peak data rate. But from the other hand, due to heavy interference levels from neighbouring cells, cell-edge users performance is greatly limited.

Figure *Full Frequency Reuse scheme* below presents frequency and power plan for Full Frequency Reuse scheme.

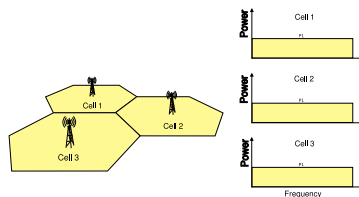


Fig. 19.50: Full Frequency Reuse scheme

In ns-3, the NoOp FR algorithm always allows scheduler to use full bandwidth and allows all UEs to use any RBG. It simply does nothing new (i.e. it does not limit eNB bandwidth, FR algorithm is disabled), it is the simplest implementation of FrAlgorithm class and is installed in eNb by default.

Hard Frequency Reuse

The Hard Frequency Reuse algorithm provides the simplest scheme which allows to reduce inter-cell interference level. In this scheme whole frequency bandwidth is divided into few (typically 3, 4, or 7) disjoint sub-bands. Adjacent eNBs are allocated with different sub-band. Frequency reuse factor equals the number of sub-bands. This scheme allows to significantly reduce ICI at the cell edge, so the performance of cell-users is improved. But due to the fact, that each eNB uses only one part of whole bandwidth, peak data rate level is also reduced by the factor equal to the reuse factor.

Figure *Hard Frequency Reuse scheme* below presents frequency and power plan for Hard Frequency Reuse scheme.

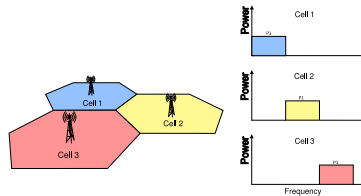


Fig. 19.51: Hard Frequency Reuse scheme

In our implementation, the Hard FR algorithm has only vector of RBGs available for eNB and pass it to MAC Scheduler during scheduling functions. When scheduler ask, if RBG is allowed for specific UE it always return true.

Strict Frequency Reuse

Strict Frequency Reuse scheme is combination of Full and Hard Frequency Reuse schemes. It consists of dividing the system bandwidth into two parts which will have different frequency reuse. One common sub-band of the system bandwidth is used in each cell interior (frequency reuse-1), while the other part of the bandwidth is divided among the neighboring eNBs as in hard frequency reuse (frequency reuse- N , $N > 1$), in order to create one sub-band with a low inter-cell interference level in each sector. Center UEs will be granted with the fully-reused frequency chunks, while cell-edge UEs with orthogonal chunks. It means that interior UEs from one cell do not share any spectrum with edge UEs from second cell, which reduces interference for both. As can be noticed, Strict FR requires a total of $N + 1$ sub-bands, and allows to achieve RFR in the middle between 1 and 3.

Figure *Strict Frequency Reuse scheme* below presents frequency and power plan for Strict Frequency Reuse scheme with a cell-edge reuse factor of $N = 3$.

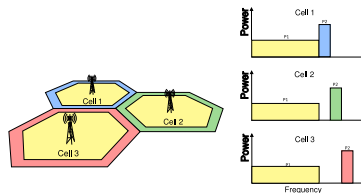


Fig. 19.52: Strict Frequency Reuse scheme

In our implementation, Strict FR algorithm has two maps, one for each sub-band. If UE can be served within private sub-band, its RNTI is added to `m_privateSubBandUe` map. If UE can be served within common sub-band, its RNTI is added to `m_commonSubBandUe` map. Strict FR algorithm needs to decide within which sub-band UE should be served. It uses UE measurements provided by RRB and compare them with signal quality threshold (this parameter can be easily tuned by attribute mechanism). Threshold has influence on interior to cell radius ratio.

Soft Frequency Reuse

In Soft Frequency Reuse (SFR) scheme each eNb transmits over the entire system bandwidth, but there are two sub-bands, within UEs are served with different power level. Since cell-center UEs share the bandwidth with neighboring cells, they usually transmit at lower power level than the cell-edge UEs. SFR is more bandwidth efficient than Strict FR, because it uses entire system bandwidth, but it also results in more interference to both cell interior and edge users.

There are two possible versions of SFR scheme:

- In first version, the sub-band dedicated for the cell-edge UEs may also be used by the cell-center UEs but with reduced power level and only if it is not occupied by the cell-edge UEs. Cell-center sub-band is available to the centre UEs only. Figure *Soft Frequency Reuse scheme version 1* below presents frequency and power plan for this version of Soft Frequency Reuse scheme.

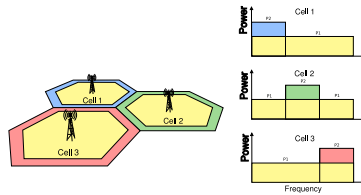


Fig. 19.53: Soft Frequency Reuse scheme version 1

- In second version, cell-center UEs do not have access to cell-edge sub-band. In this way, each cell can use the whole system bandwidth while reducing the interference to the neighbors cells. From the other hand, lower ICI level at the cell-edge is achieved at the expense of lower spectrum utilization. Figure *Soft Frequency Reuse scheme version 2* below presents frequency and power plan for this version of Soft Frequency Reuse scheme.

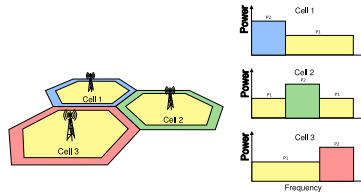


Fig. 19.54: Soft Frequency Reuse scheme version 2

SFR algorithm maintain two maps. If UE should be served with lower power level, its RNTI is added to `m_lowPowerSubBandUe` map. If UE should be served with higher power level, its RNTI is added to `m_highPowerSubBandUe` map. To decide with which power level UE should be served SFR algorithm utilize UE measurements, and compares them to threshold. Signal quality threshold and `PdschConfigDedicated` (i.e. `P_A` value) for inner and outer area can be configured by attributes system. SFR utilizes Downlink Power Control described here.

Soft Fractional Frequency Reuse

Soft Fractional Frequency Reuse (SFFR) is an combination of Strict and Soft Frequency Reuse schemes. While Strict FR do not use the subbands allocated for outer region in the adjacent cells, soft FFR uses these subbands for the inner UEs with low transmit power. As a result, the SFFR, like SFR, use the subband with high transmit power level and with low transmit power level. Unlike the Soft FR and like Strict FR, the Soft FFR uses the common sub-band which can enhance the throughput of the inner users.

Figure *Soft Fractional Fractional Frequency Reuse scheme* below presents frequency and power plan for Soft Fractional Frequency Reuse.

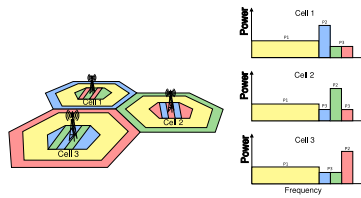


Fig. 19.55: Soft Fractional Frequency Reuse scheme

Enhanced Fractional Frequency Reuse

Enhanced Fractional Frequency Reuse (EFFF) described in [ZXie2009] defines 3 cell-types for directly neighboring cells in a cellular system, and reserves for each cell-type a part of the whole frequency band named *Primary Segment*, which among different type cells should be orthogonal. The remaining subchannels constitute the *Secondary Segment*. The *Primary Segment* of a cell-type is at the same time a part of the *Secondary Segments* belonging to the other two cell-types. Each cell can occupy all subchannels of its *Primary Segment* at will, whereas only a part of subchannels in the *Secondary Segment* can be used by this cell in an interference-aware manner. The *Primary Segment* of each cell is divided into a reuse-3 part and reuse-1 part. The reuse-1 part can be reused by all types of cells in the system, whereas reuse-3 part can only be exclusively reused by other same type cells(i.e. the reuse-3 subchannels cannot be reused by directly neighboring cells). On the *Secondary Segment* cell acts as a guest, and occupying secondary subchannels is actually reuse the primary subchannels belonging to the directly neighboring cells, thus reuse on the *Secondary Segment* by each cell should conform to two rules:

- monitor before use
- resource reuse based on SINR estimation

Each cell listens on every secondary subchannel all the time. And before occupation, it makes SINR evaluation according to the gathered channel quality information (CQI) and chooses resources with best estimation values for reuse. If CQI value for RBG is above configured threshold for some user, transmission for this user can be performed using this RBG.

In [ZXie2009] scheduling process is described, it consist of three steps and two scheduling polices. Since none of currently implemented schedulers allow for this behaviour, some simplification were applied. In our implementation reuse-1 subchannels can be used only by cell center users. Reuse-3 subchannels can be used by edge users, and only if there is no edge user, transmission for cell center users can be served in reuse-3 subchannels.

Figure *Enhanced Fractional Frequency Reuse scheme* below presents frequency and power plan for Enhanced Fractional Frequency Reuse.

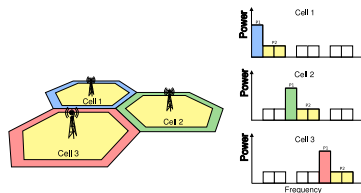


Fig. 19.56: Enhanced Fractional Frequency Reuse scheme

Distributed Fractional Frequency Reuse

This Distributed Fractional Frequency Reuse Algorithm was presented in [DKimura2012]. It automatically optimizes cell-edge sub-bands by focusing on user distribution (in particular, receive-power distribution). This algorithm adaptively selects RBs for cell-edge sub-band on basis of coordination information from adjacent cells and notifies the base stations of the adjacent cells, which RBs it selected to use in edge sub-band. The base station of each cell uses the received information and the following equation to compute cell-edge-band metric A_k for each RB.

$$A_k = \sum_{j \in J} w_j X_{j,k}$$

where J is a set of neighbor cells, $X_{j,k} = \{0, 1\}$ is the RNTP from the j -th neighbor cell. It takes a value of 1 when the k -th RB in the j -th neighbor cell is used as a cell-edge sub-band and 0 otherwise. The symbol w_j denotes weight with respect to adjacent cell j , that is, the number of users for which the difference between the power of the signal received from the serving cell i and the power of the signal received from the adjacent cell j is less than a threshold value (i.e., the number of users near the cell edge in the service cell). A large received power difference means that cell-edge users in the i -th cell suffer strong interference from the j -th cell.

The RB for which metric A_k is smallest is considered to be least affected by interference from another cell. Serving cell selects a configured number of RBs as cell-edge sub-band in ascending order of A_k . As a result, the RBs in which a small number of cell-edge users receive high interference from adjacent base stations are selected.

The updated RNTP is then sent to all the neighbor cells. In order to avoid the meaningless oscillation of cell-edge-band selection, a base station ignores an RNTP from another base station that has larger cell ID than the base station.

Repeating this process across all cells enables the allocation of RBs to cell-edge areas to be optimized over the system and to be adjusted with changes in user distribution.

Figure *Sequence diagram of Distributed Frequency Reuse Scheme* below presents sequence diagram of Distributed Fractional Frequency Reuse Scheme.

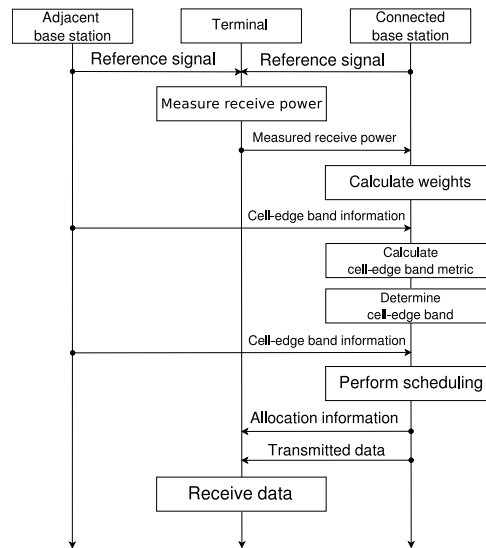


Fig. 19.57: Sequence diagram of Distributed Frequency Reuse Scheme

19.1.17 Carrier Aggregation

Overview

This section describes the ns-3 support for Carrier Aggregation. The references in the standard are [TS36211], [TS36213] and [TS36331].

Note: Carrier Aggregation was introduced in release 3.27 and currently, only works in downlink.

3GPP standardizes, in release R10, the Carrier Aggregation (CA) technology.

This technology consists of the possibility, to aggregate radio resources belonging to different carriers, in order to have more bandwidth available, and to achieve a higher throughput. Carrier Aggregation as defined by 3GPP can be used with both TDD and FDD. Since ns-3 only supports FDD LTE implementation, we will consider only this case in this section. Each aggregated carrier is referred to as a component carrier, CC. The component carrier can have a bandwidth of 1.4, 3, 5, 10, 15 or 20 MHz and a maximum of five component carriers can be aggregated, hence the maximum aggregated bandwidth is 100 MHz. In FDD the number of aggregated carriers can be different in DL and UL. However, the number of UL component carriers is always equal to or lower than the number of DL component carriers. The individual component carriers can also be of different bandwidths. When carrier aggregation is used there are a number of serving cells, one for each component carrier. The coverage of the serving cells may differ, for example due to that CCs on different frequency bands will experience different pathloss. The RRC connection is only handled by one cell, the Primary serving cell, served by the Primary component carrier (DL and UL PCC). It is also on the DL PCC that the UE receives NAS information, such as security parameters.

3GPP defines three different CA bandwidth classes in releases 10 and 11 (where ATBC is Aggregated Transmission Bandwidth Configuration):

Class A: $ATBC \leq 100$, maximum number of CC = 1

Class B: $ATBC \leq 100$, maximum number of CC = 2

Class C: $100 \leq ATBC \leq 200$, maximum number of CC = 2

Figure *CA impact on different layers of LTE protocol stack (from 3gpp.org)* (from 3gpp.org) shows the main impact of CA technology on the different layers of the LTE protocol stack. Introduction of carrier aggregation influences mainly the MAC and new RRC messages are introduced. In order to keep R8/R9 compatibility the protocol changes will be kept to a minimum. Basically each component carrier is treated as an R8 carrier. However some changes are required, such as new RRC messages in order to handle the secondary component carrier (SCC), and MAC must be able to handle scheduling on a number of CCs. In the following we describe the impact of the carrier aggregation implementation on the different layers of the LTE protocol stack in ns-3.

Impact on RRC layer

The main impacts on the RRC layer are related to secondary carrier configuration and measurements reporting. To enable these features we have enhanced the already existing procedures for the RRC Connection Reconfiguration and UE RRC Measurements Model.

The carrier aggregation enabling procedure is shown in figure *A schematic overview of the secondary carrier enabling procedure*. As per 3GPP definition, the secondary cell is a cell, operating on a secondary frequency, which may be configured once an RRC connection is established and which may be used to provide additional radio resources. Hence, the procedure starts when the UE is in the CONNECTED_NORMALLY state (see the RRC state machine description). This part of the procedure is the same as in the previous architecture. In order to simplify the implementation, the *UE Capability Inquiry* and *UE Capability Information* are not implemented. This implies to assume that each UE can support the carrier aggregation, and any specific configuration provided by the eNB to which is attached. The eNB RRC sends to the UE the secondary carrier configuration parameters through the RRC Connection Reconfiguration procedure. This procedure may be used for various purposes related to modifications of the RRC connection, e.g. to establish, modify or release RBs, to perform handover, to setup, modify or

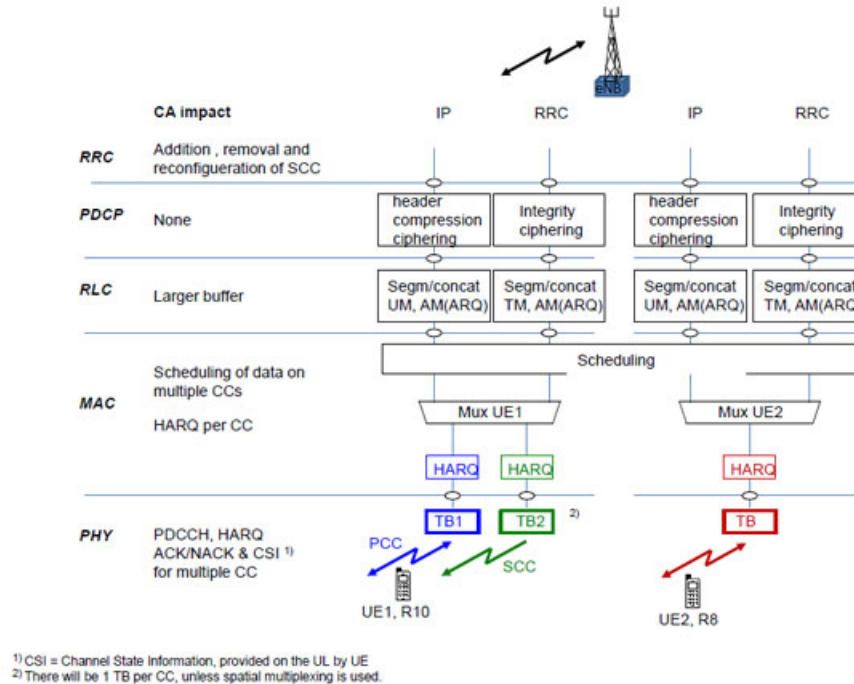


Fig. 19.58: CA impact on different layers of LTE protocol stack (from 3gpp.org)

release measurements, to add, modify and release secondary cells (SCells). At UE side, the RRC is extended to configure the lower layers, in such a way that the SCell(s) are considered. Once the carriers are configured, the *Reconfiguration Completed* message is sent back to the eNB RRC, informing the eNB RRC and CCM that the secondary carriers have been properly configured. The RRC layer at both the UE and the eNB sides is extended to allow measurement reporting for the secondary carriers. Finally, in order to allow the procedures for configuration and measurement reporting, the RRC is enhanced to support serialization and deserialization of RRC message structures that carry information related to the secondary carriers, e.g., if the *RRCConnectionReconfiguration* message includes *sCellToAddModList* structure, SCell addition or modification will be performed, or, if it contains *measConfig* the measurement reporting will be configured. To allow transmission of this information the following structures are implemented for the sCell: *RadioResourceConfigCommonSCell*, *RadioResourceConfigDedicatedSCell* and *PhysicalConfigDedicatedSCell* and *NonCriticalExtensionConfiguration*. *RadioResourceConfigCommonSCell* and *RadioResourceConfigDedicatedSCell* are used for SCell addition and modification (see TS 36.331, 5.3.10.3b). *PhysicalConfigDedicatedSCell* is used for physical channel reconfiguration (see TS 36.331, 5.3.10.6). Finally, *NonCriticalExtensionConfiguration* is used to carry information of *sCellToAddModList* and *sCellToReleaseList*, which is a modified structure comparing to TS 36.331, 6.6.2, according to which these are directly in the root of *RRC-ConnectionReconfiguration* message. Measurement reporting is extended with *measResultSCell* structure to include RSRP and RSRQ measurements for each configured SCell. However, the measurement report triggering event A6 (neighbour becomes offset better than SCell) is not implemented yet.

Handover is possible between different component carriers on the same eNB (i.e., intra-eNB handover from one frequency to another) and between carriers on different eNB (i.e., inter-eNB). One constraint for inter-eNB handovers is that both eNB must have the same number of component carriers.

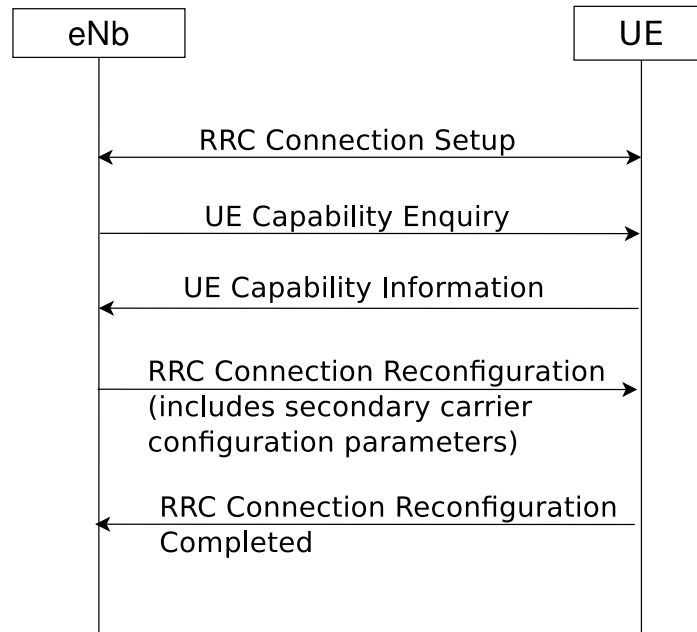


Fig. 19.59: A schematic overview of the secondary carrier enabling procedure

Impact on PDCP layer

There is no impact on PDCP layer.

Impact on RLC layer

The impact on the RLC layer is relatively small. There is some impact on configuration of the buffer and the usage of SAP interfaces between RLC and MAC. Since the capacity of the lower layers increases with the carrier aggregation it is necessary to accordingly adjust the size of the RLC buffer. The impact on the implementation of the RLC layer is very small thanks to the design choice that allows the CCM manager to serve the different RLC instances through the `LteMacSapProvider` interface. Thanks to this design choice, the RLC is using the same interface as in the earlier LTE module architecture, the `LteMacSapProvider`, but the actual SAP provider in the new architecture is the CCM (some class that inherits `LteEnbComponentCarrierManager`). The CCM acts as a proxy, it receives function calls that are meant for the MAC, and forwards them to the MAC of the different component carriers. Additionally, it uses the information of the UEs and the logical channels for its own functionalities.

Impact on MAC layer

The impact on the MAC layer depends on the CA scheduling scheme in use. Two different scheduling schemes are proposed in R10 and are shown in figure [CA scheduling schemes \(from 3gpp.org\)](#).

The CIF (Carrier Indicator Field) on PDCCH (represented by the red area) indicates on which carrier the scheduled resource is located. In the following we describe both the schemes:

- a) scheduling grant and resources on the same carrier. One PDCCH is supported per carrier.
- b) cross-carrier scheduling: it is used to schedule resources on the secondary carrier without PDCCH.

Current implementation covers only option 1, so there is no cross-carrier scheduling. The MAC layer of the eNodeB has suffered minor changes and they are mainly related to addition of component carrier information in message exchange

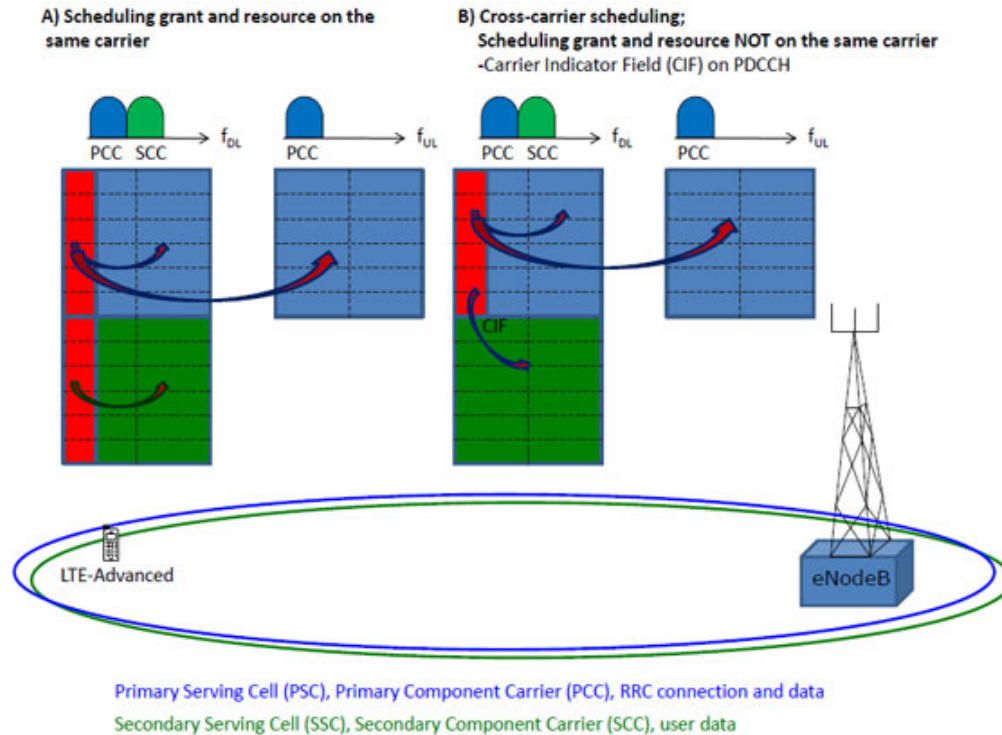


Fig. 19.60: CA scheduling schemes (from 3gpp.org)

between layers.

Impact on PHY layer

The impact on PHY layer is minor. There is an instance of PHY layer per each component carrier and the SAP interface functions remain unchanged. As shown in *CA scheduling schemes (from 3gpp.org)* the difference is that since there are multiple PHY instances, there are also multiple instances of PDCCH, HARQ, ACK/NACK and CSI per carrier. So, at the eNB PHY, the changes are related to the addition of the component carrier id information, while at the UE PHY the information of the Component Carrier is used for some functionalities that depend on the Component Carrier to which the PHY instance belongs. For example, the UE PHY is extended to allow disabling of the sounding reference signal (SRS) at the secondary carriers. This is necessary because there is one UE PHY instance per component carrier, but according to *CA scheduling schemes (from 3gpp.org)*, only a single carrier is used and the uplink traffic is transmitted only over the primary carrier.

Code Structure Design

This section briefly introduces the software design and implementation of the carrier aggregation functionality.

Both `LteEnbNetDevice` and `LteUeNetDevice` are created by the `LteHelper` using the method `InstallSingleEnbDevice` and `InstallSingleUeDevice`. These functions are now extended to allow the carrier aggregation configuration. In the following we explain the main differences comparing to the previous architecture.

Figure *Changes in LteEnbNetDevice to support CA* shows the attributes and associations of the `LteEnbNetDevice` that are affected by the implementation, or are created in order to support the carrier aggregation functionality. Since `LteEnbNetDevice` may have several component carriers, the attributes that were formerly part of the `LteEnbNetDevice` and are carrier specific are migrated to the `ComponentCarrier` class, e.g. physical layer configuration parameters.

The attributes that are specific for the eNB component carrier are migrated to `ComponentCarrierEnb`, e.g. pointers to MAC, PHY, scheduler, fractional frequency reuse instances. `LteEnbNetDevice` can contain pointers to several `ComponentCarrierEnb` instances. This architecture allows that each CC may have its own configuration for PHY, MAC, scheduling algorithm and fractional frequency reuse algorithm. These attributes are currently maintained also in the `LteEnbNetDevice` for backward compatibility purpose. By default the `LteEnbNetDevice` attributes are the same as the primary carrier attributes.

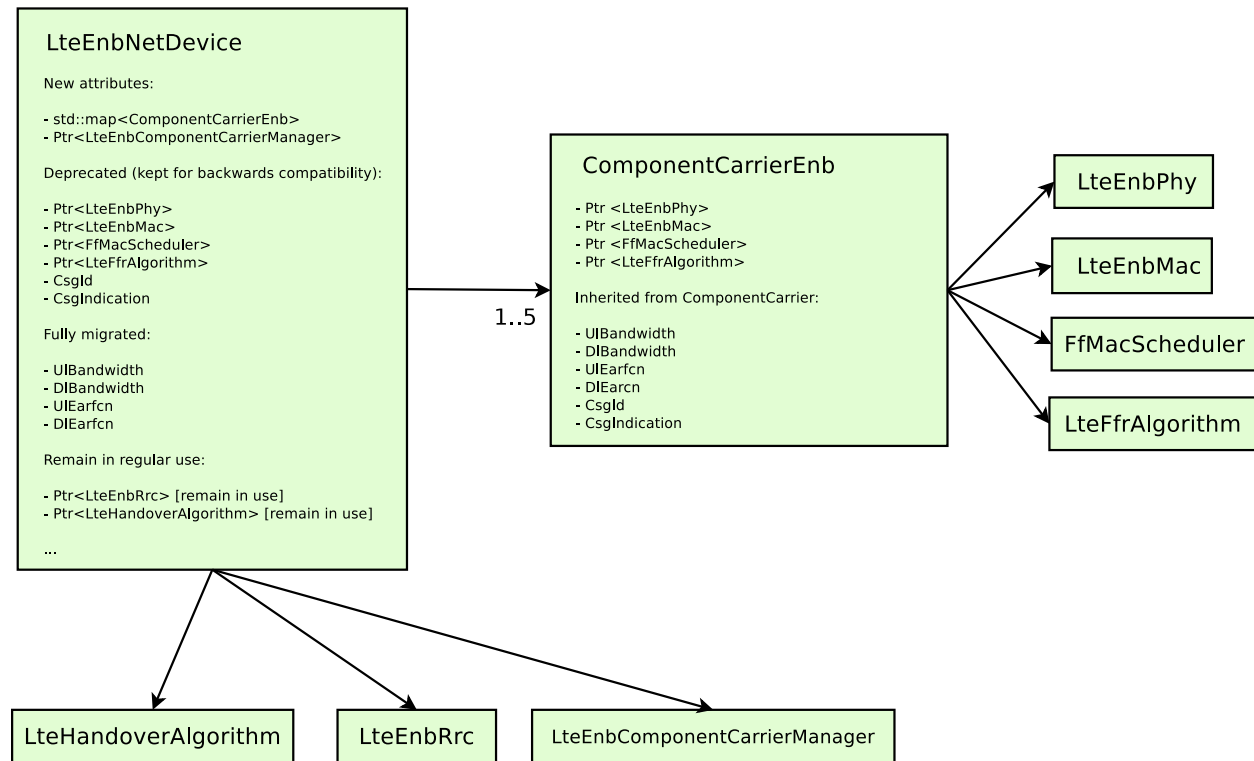


Fig. 19.61: Changes in `LteEnbNetDevice` to support CA

Figure *Changes in `LteUeNetDevice` to support CA* shows the attributes and associations of `LteUeNetDevice` that are affected by the carrier aggregation implementation. Similarly, to the changes in `LteEnbNetDevice`, pointers that are specific to UE component carrier are migrated to the `ComponentCarrierUe` class. `LteUeNetDevice` has maintained `m_dIEarfcn` for initial cell selection purposes.

CA impact on data plane of eNodeB

Figure *eNB Data Plane Architecture* shows the class diagram of the data plane at the eNB.

The main impact is the insertion of the `LteEnbComponentCarrierManager` class in the middle of the LTE protocol stack. During the design phase it was decided to keep the same SAP interfaces design that existed between MAC and RLC in order to avoid unnecessary changes in these parts of protocol stack. To achieve this the `LteEnbComponentCarrierManager` implements all functions that were previously exposed by RLC to MAC through `LteMacSapUser` interface. It also implements functions that were previously exposed by MAC to RLC through the `LteMacSapProvider` interface. In this way, the carrier aggregation is transparent to upper and lower layers. The only difference is that the MAC instance sees now only one `LteMacSapUser`, whereas formerly it was seeing only one `LteMacSapUser` per RLC instance.

The `LteEnbComponentCarrierManager` is responsible for the forwarding messages in both directions. In the current implementation, a PDCP and a RLC instances are activated each time a new data radio bearer is configured. The

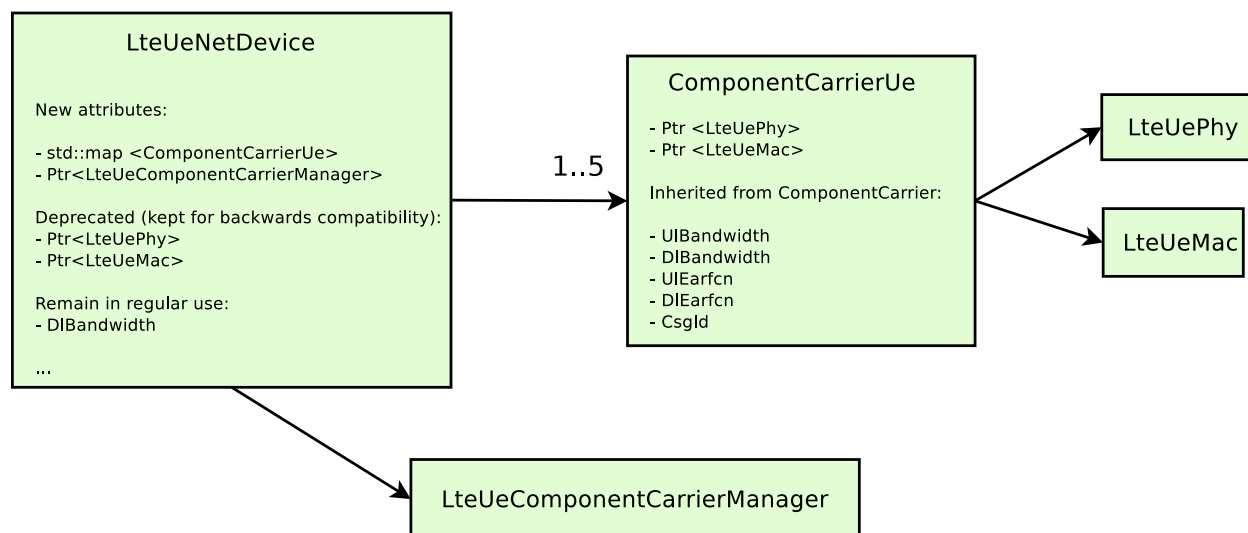


Fig. 19.62: Changes in LteUeNetDevice to support CA

correspondence between a new data radio bearer and a RLC instance is one to one. In order to maintain the same behavior, when a new logical channel is activated, the logical channel configurations are propagated to each MAC layer object in “as is” fashion.

Figure *Sequence Diagram of downlink buffer status reporting (BSR) with CA* shows a sequence diagram of downlink buffer status reporting with a carrier aggregation implementation of only one secondary carrier. Each time that an RLC instance sends a buffer status report (BSR), the LteEnbComponentCarrierManager propagates the BSR to the MAC instances. The LteEnbComponentCarrierManager may modify a BSR before sending it to the MAC instances. This modification depends on the traffic split algorithm implemented in CCM class that inherits LteEnbComponentCarrierManager.

CA impact on control plane of eNodeB

Figure *eNB Control Plane Architecture* shows the class diagram of the control plane at the eNB. During the design phase it was decided to maintain the same hooks as in the former architecture. To do so, at each component carrier the PHY and the MAC are directly associated to the RRC instance. However, the RRC instance is additionally connected to the LteEnbComponentCarrierManager, which is responsible for enabling and disabling the component carriers. When the simulation starts, the number of component carrier is fixed, but only the primary carrier component is enabled. Depending on the LteEnbComponentCarrierManager algorithm the other carrier components could be activated or not.

Figure *Sequence Diagram of Data Radio Bearer Setup* shows how the Radio Bearer are configured.

CA impact on data plane of UE

Figure *UE Data Plane Architecture* shows the relation between the different classes related to the UE data plane. The UE data plane architecture is similar to the eNB data plane implementation. The LteUeComponentCarrierManager is responsible to (re)map each MacSapUserProvider to the corresponding RLC instance or to the proper MAC instance. The channel remapping depends on algorithm used as LteUeComponentCarrierManager. A particular case is represented by the UE buffer status report (BSR) to eNB. Since, i) the standard does not specify how the BSR has to be reported on each component carrier and ii) it is decided to map one-to-one the logical channel to each MAC layer, the only way to send BSRs to the eNB is through the primary carrier. Figure *Uplink buffer status reporting with CA* shows the sequence diagram. Each time a BSR is generated, the LteUeComponentCarrierManager sends it through the primary carrier component. When the primary component carrier at the eNB receives the BSR, it sends it to

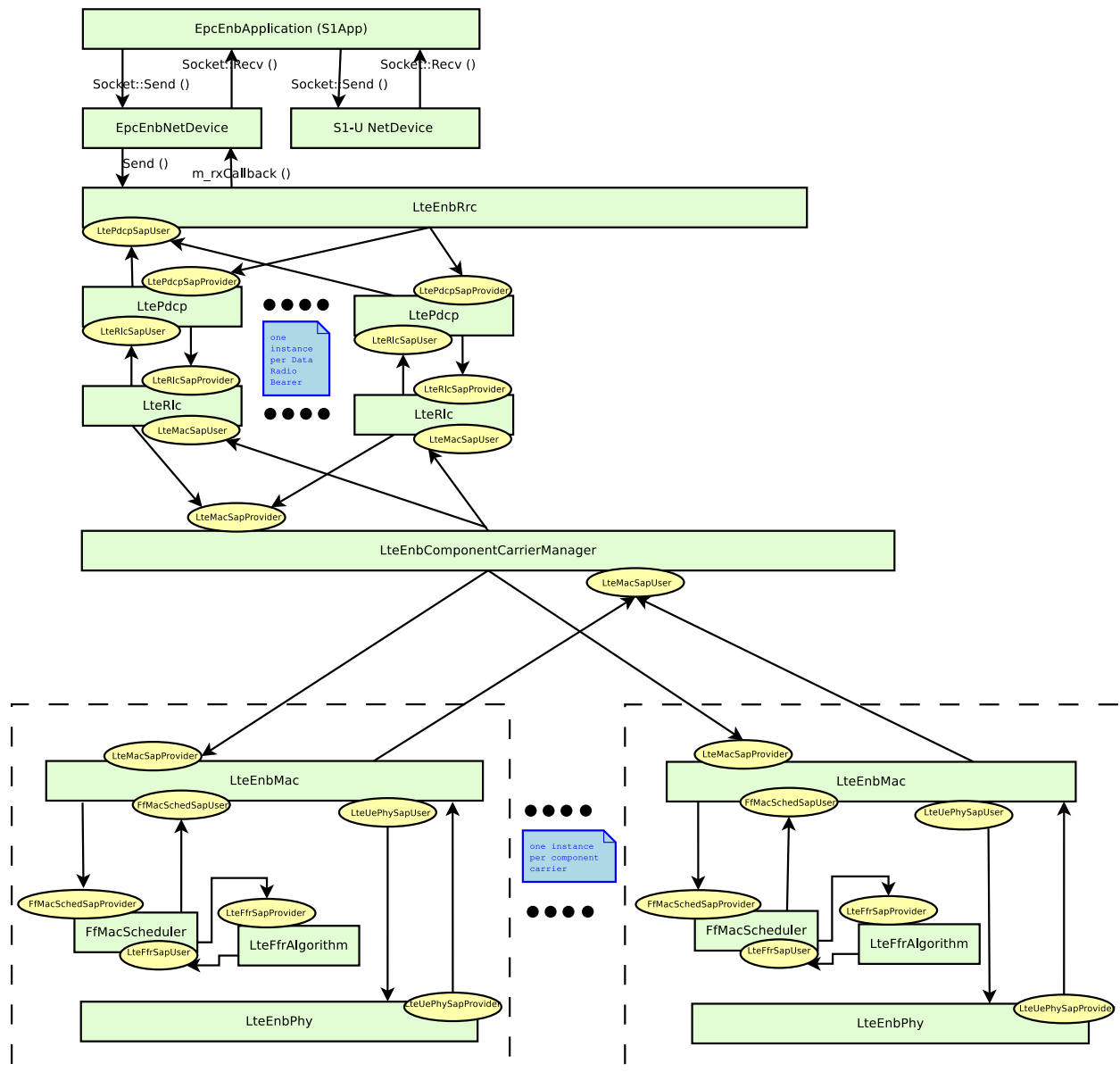


Fig. 19.63: eNB Data Plane Architecture

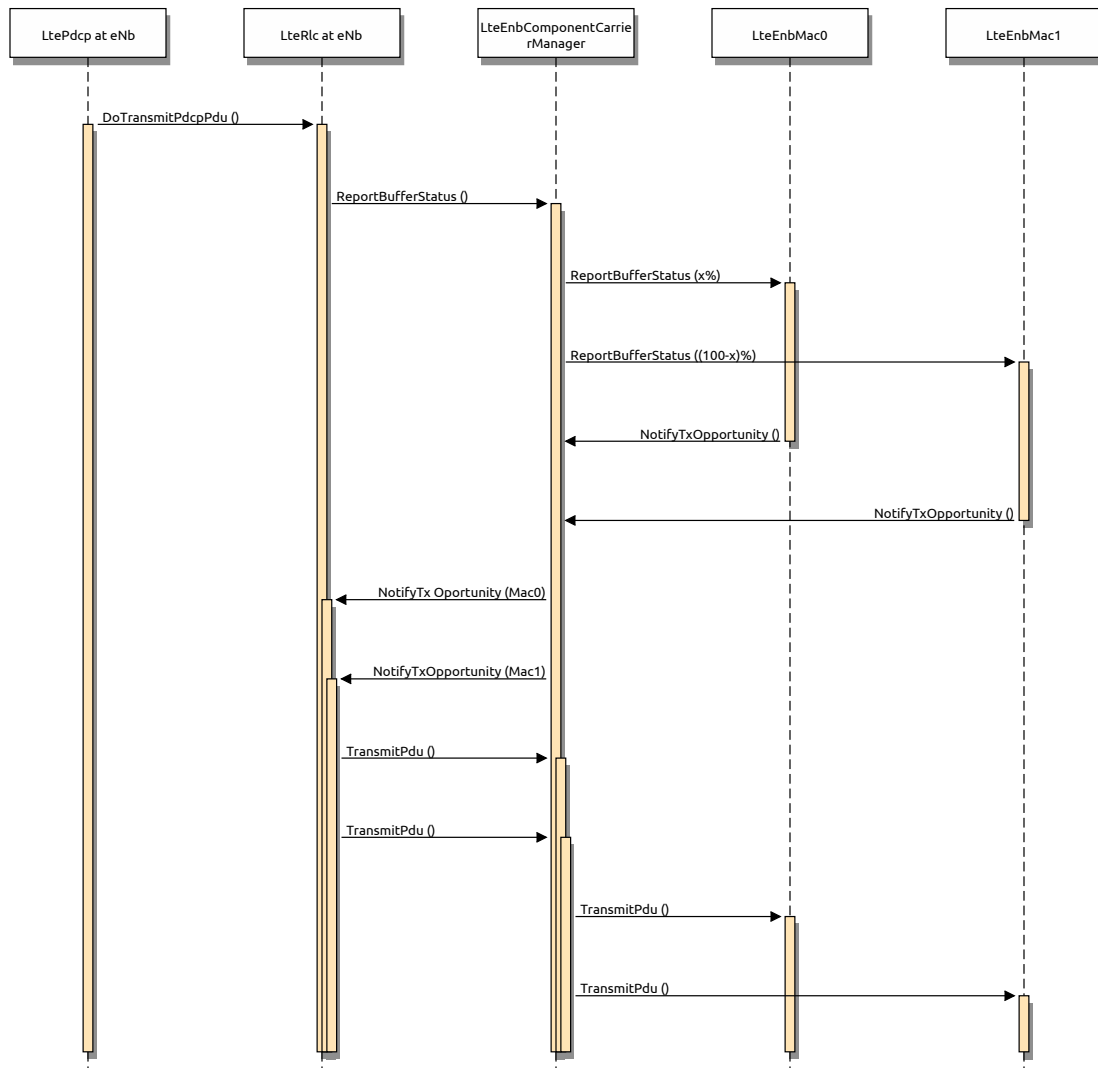


Fig. 19.64: Sequence Diagram of downlink buffer status reporting (BSR) with CA

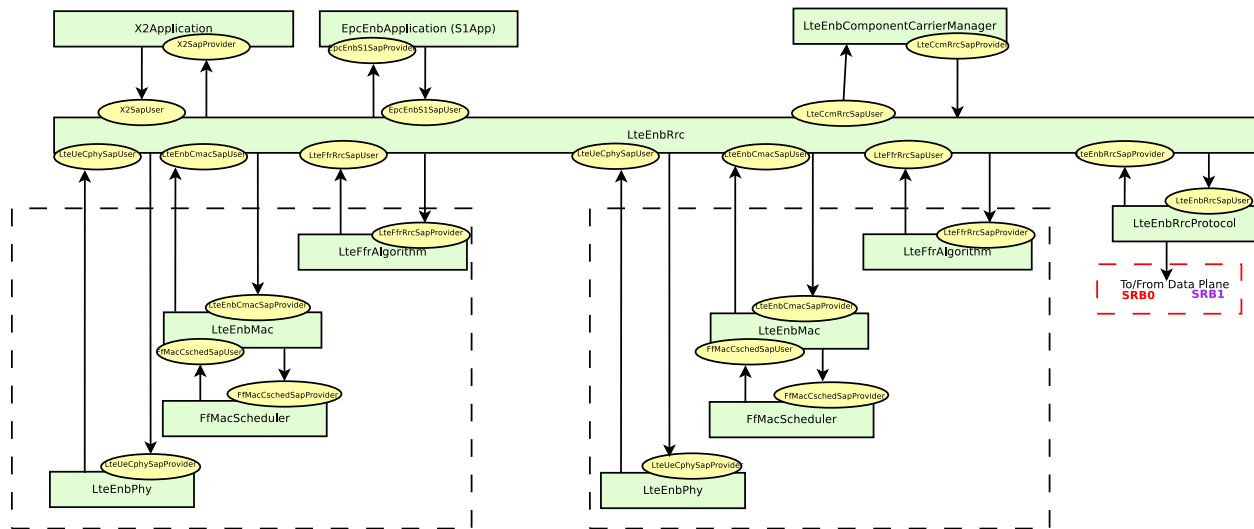


Fig. 19.65: eNB Control Plane Architecture

LteEnbComponentCarrierManager. The latter, according to algorithm dependent policies, forwards a BSR to component carriers. The communication between the LteEnbMac and the LteEnbComponentCarrierManager is done through a specific set of SAP functions which are implemented in the LteU1CcmRrcSapUser and the LteU1CcmRrcSapProvider.

CA impact on control plane of UE

Figure *UE Control Plane Architecture* shows the relation between the different classes associated to the UE control plane. The control plane implementation at the UE is basically the same as the eNB control plane implementation. Each component carrier control SAP (both for PHY and MAC layer objects) is linked in a one-to-one fashion directly to the RRC instance. The Ue RRC instance is then connected to the LteUeComponentCarrierManager in the same way as in the eNB.

CCHelper is the class that is implemented to help the configuration of the physical layer parameters, such as uplink and downlink, bandwidth and EARFCN of each carrier.

CCM RRC MAC interfaces

The Component carrier manager (CCM) is also developed by using the SAP interface design. The following SAP interfaces are implemented for CCM and MAC:

- the LteCcmMacSapUser part is provided by MAC and is used by the CCM
- the LteCcmMacSapProvider part is provided by CCM and is used by the MAC layer

When the primary component carrier receives an uplink BSR it uses the LteCcmMacSapUser to forward it to the CCM, which should decide how to split the traffic corresponding to this BSR among carriers. Once this decision is made, the CCM uses the LteCcmMacSapProvider interface to send back an uplink BSR to some of the MAC instances. Additionally, the LteCcmMacSapUser can be used by the MAC to notify about the PRB occupancy in the downlink to the CCM. This information may be used by the CCM to decide how to split the traffic and whether to use the secondary carriers.

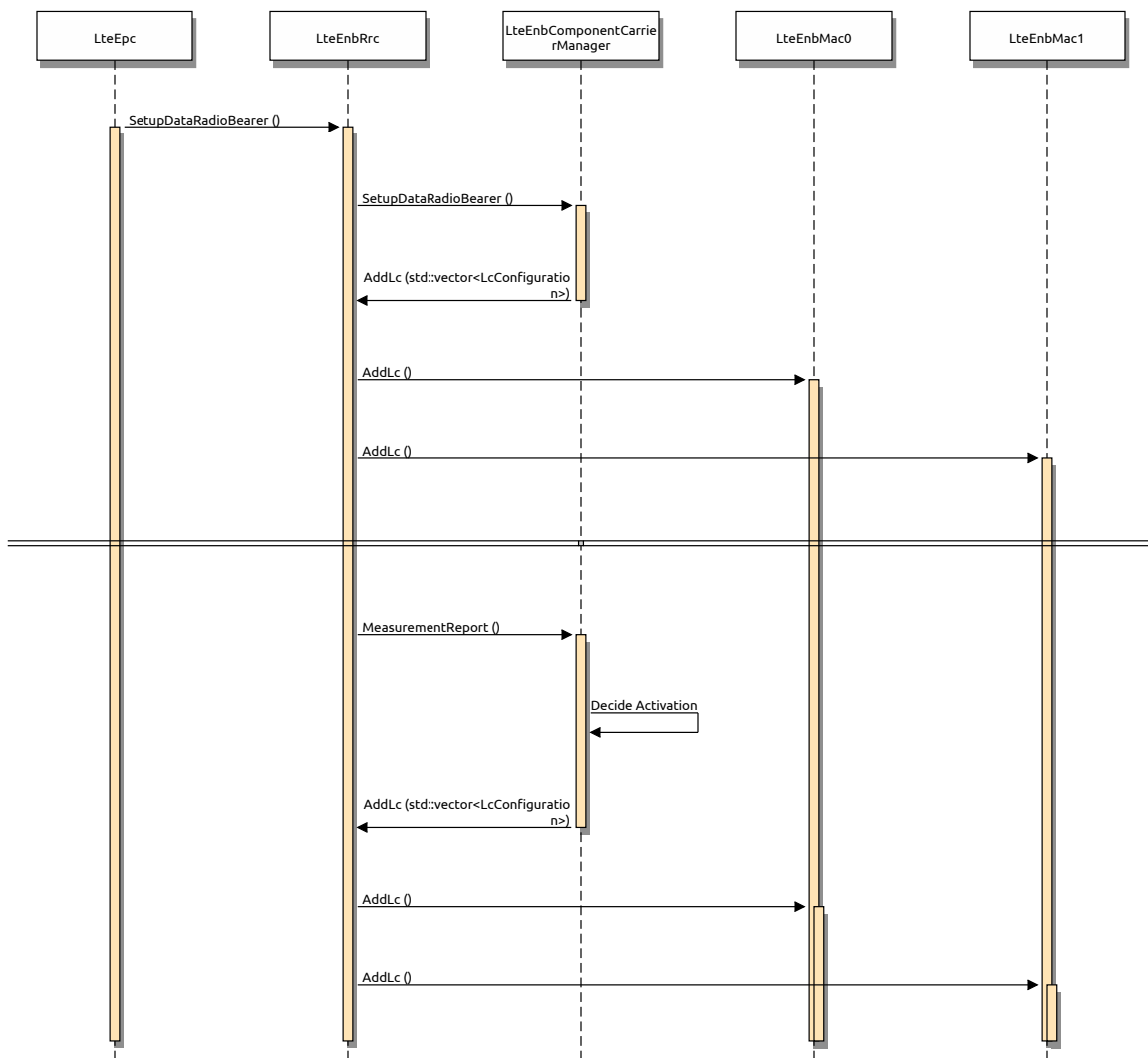


Fig. 19.66: Sequence Diagram of Data Radio Bearer Setup

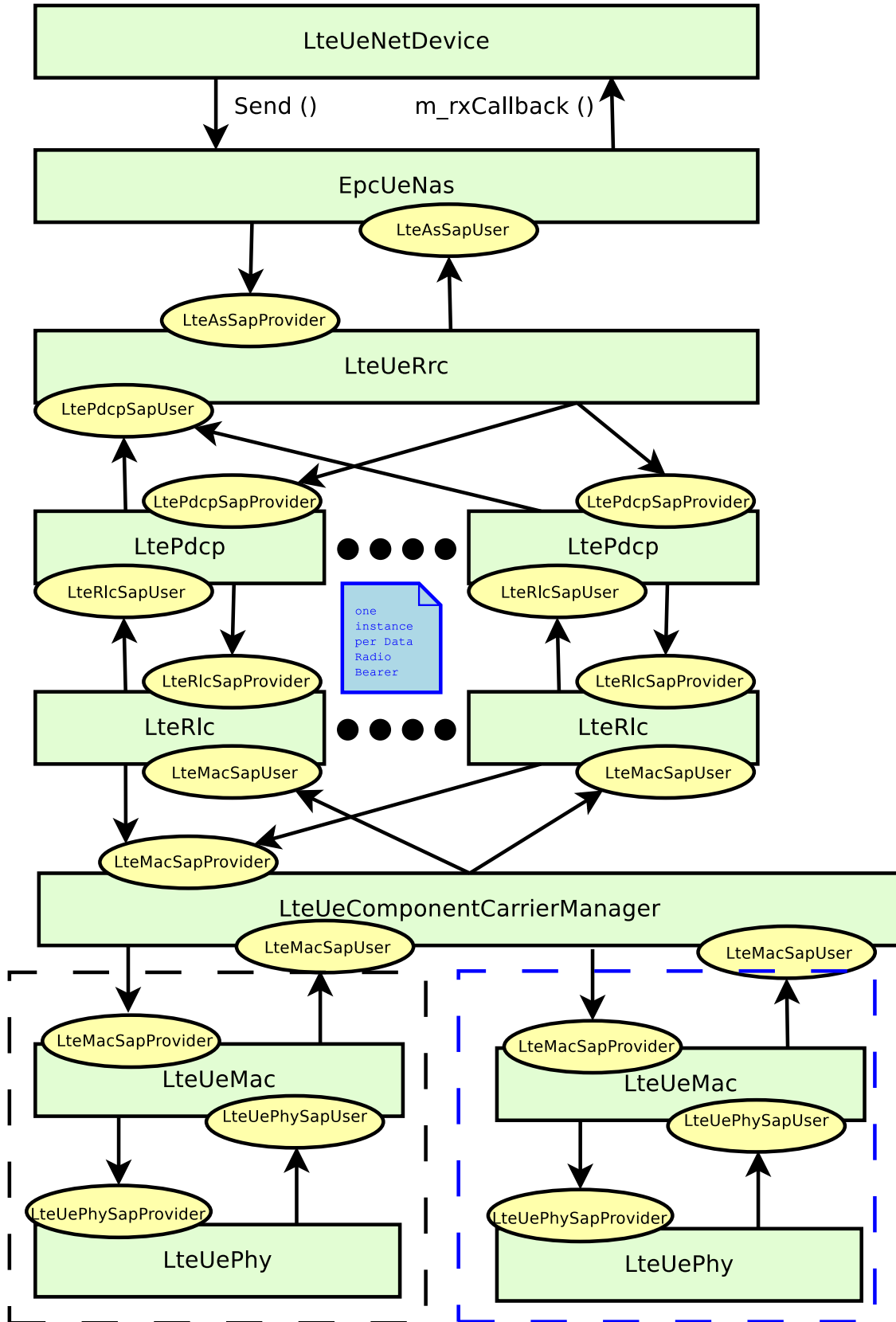


Fig. 19.67: UE Data Plane Architecture

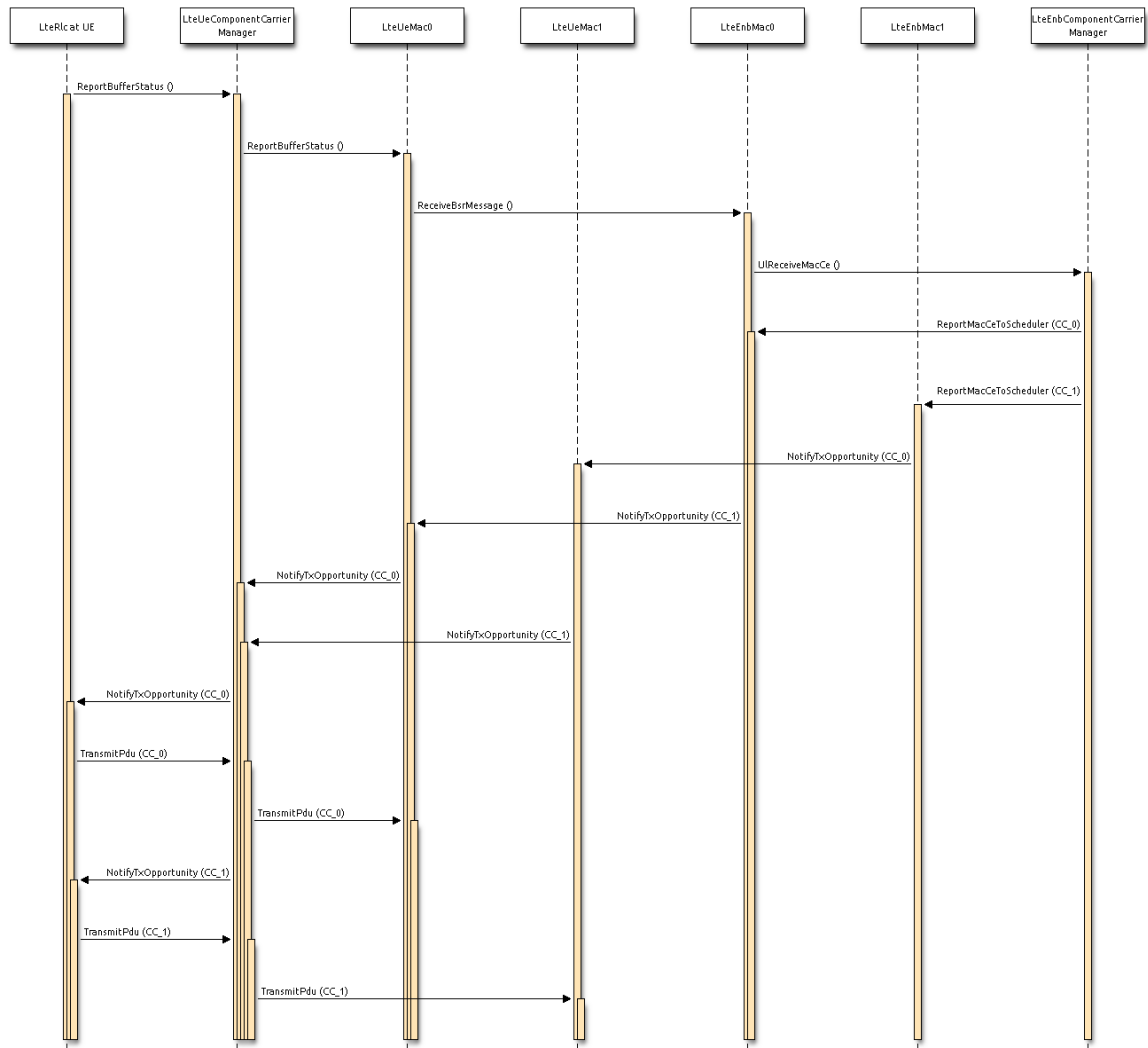


Fig. 19.68: Uplink buffer status reporting with CA

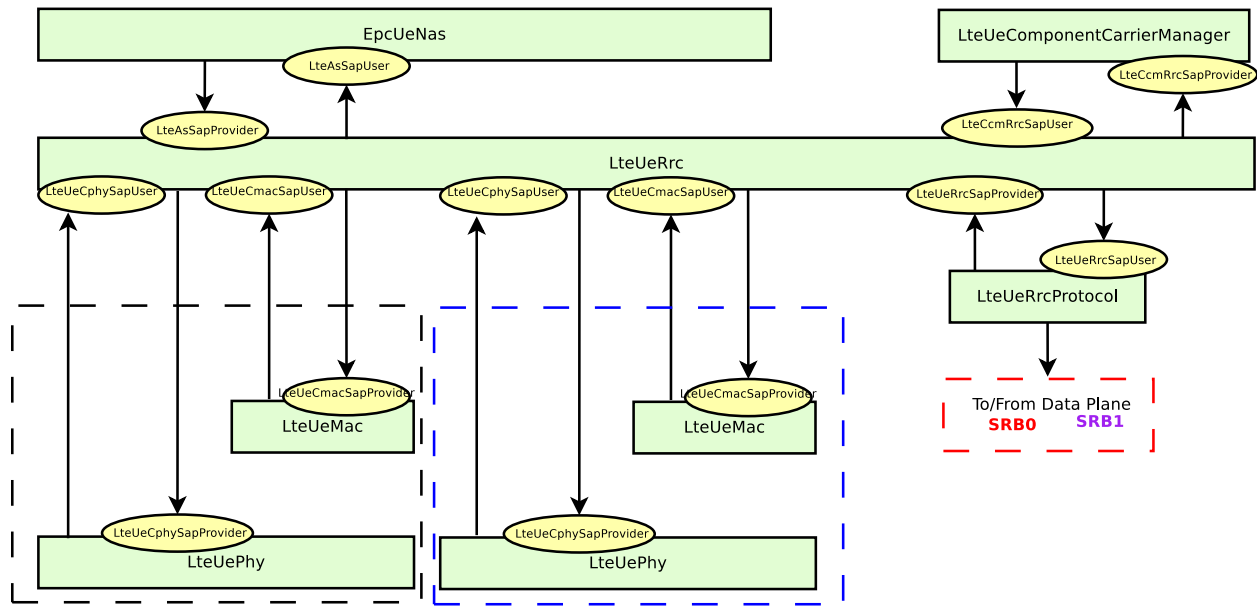


Fig. 19.69: UE Control Plane Architecture

CCM RRC SAP interfaces

The following SAP interfaces are implemented for CCM and RRC:

- the **LteCcmRrcSapProvider** is provided by the CCM and is used by the RRC layer
- the **LteCcmRrcSapUser** is provided by RRC and is used by the CCM

By using the **LteCcmRrcSapUser** the CCM may request a specific measurement reporting configuration to be fulfilled by the UEs attached to the eNB. When a UE measurement report is received, as a result of this configuration, the eNB RRC entity shall forward this report to the CCM through the **LteCcmRrcSapProvider::ReportUeMeas** SAP function. Additionally, the **LteCcmRrcSapProvider** offers different functions to the RRC that can be used to add and remove of UEs, setup or release of radio bearer, configuration of the signalling bearer, etc.

Component carrier managers

Currently, there are two component carrier manager implementations available. The first one is the **NoOpComponentCarrierManager**, which is the default CCM choice. When this CCM is used the carrier aggregation feature is disabled. This CCM forwards all traffic, the uplink and the downlink, over the primary carrier, and does not use secondary carriers. Another implementation is the **RrComponentCarrierManager**, which splits the traffic equally among carriers, by diving the buffer status report among different carriers. SRB0 and SRB1 flows will be forwarded only over primary carrier.

19.1.18 Helpers

Two helper objects are used to setup simulations and configure the various components. These objects are:

- `LteHelper`, which takes care of the configuration of the LTE radio access network, as well as of coordinating the setup and release of EPS bearers. The `LteHelper` class provides both the API definition and its implementation.
- `EpcHelper`, which takes care of the configuration of the Evolved Packet Core. The `EpcHelper` class is an abstract base class, which only provides the API definition; the implementation is delegated to the child classes in order to allow for different EPC network models.

A third helper object is used to configure the *Carrier Aggregation* functionality:

- `CcHelper`, which takes care of the configuration of the `LteEnbComponentCarrierMap`, basically, it creates a user specified number of `LteEnbComponentCarrier`. `LteUeComponentCarrierMap` is currently created starting from the `LteEnbComponentCarrierMap`. `LteHelper::InstallSingleUeDevice`, in this implementation, is needed to invoke after the `LteHelper::InstallSingleEnbDevice` to ensure that the `LteEnbComponentCarrierMap` is properly initialized.

It is possible to create a simple LTE-only simulations by using the `LteHelper` alone, or to create complete LTE-EPC simulations by using both `LteHelper` and `EpcHelper`. When both helpers are used, they interact in a master-slave fashion, with the `LteHelper` being the Master that interacts directly with the user program, and the `EpcHelper` working “under the hood” to configure the EPC upon explicit methods called by the `LteHelper`. The exact interactions are displayed in the Figure *Sequence diagram of the interaction between `LteHelper` and `EpcHelper`*.

19.2 User Documentation

19.2.1 Background

We assume the reader is already familiar with how to use the ns-3 simulator to run generic simulation programs. If this is not the case, we strongly recommend the reader to consult [ns3tutorial].

19.2.2 Usage Overview

The ns-3 LTE model is a software library that allows the simulation of LTE networks, optionally including the Evolved Packet Core (EPC). The process of performing such simulations typically involves the following steps:

1. *Define the scenario* to be simulated
2. *Write a simulation program* that recreates the desired scenario topology/architecture. This is done accessing the ns-3 LTE model library using the `ns3::LteHelper` API defined in `src/lte/helper/lte-helper.h`.
3. *Specify configuration parameters* of the objects that are being used for the simulation. This can be done using input files (via the `ns3::ConfigStore`) or directly within the simulation program.
4. *Configure the desired output* to be produced by the simulator
5. *Run* the simulation.

All these aspects will be explained in the following sections by means of practical examples.

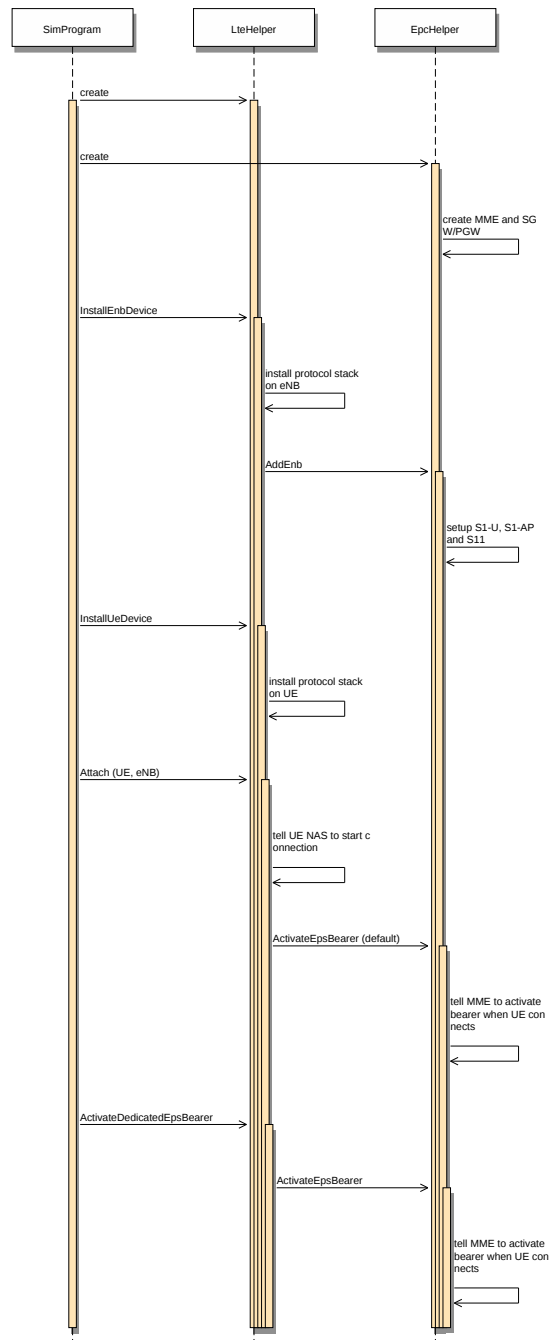


Fig. 19.70: Sequence diagram of the interaction between LteHelper and EpcHelper.

19.2.3 Basic simulation program

Here is the minimal simulation program that is needed to do an LTE-only simulation (without EPC).

1. Initial boilerplate:

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/mobility-module.h"
#include "ns3/lte-module.h"

using namespace ns3;

int main(int argc, char *argv[])
{
    // the rest of the simulation program follows
}
```

2. Create an LteHelper object:

```
Ptr<LteHelper> lteHelper = CreateObject<LteHelper>();
```

This will instantiate some common objects (e.g., the Channel object) and provide the methods to add eNBs and UEs and configure them.

3. Create Node objects for the eNB(s) and the UEs:

```
NodeContainer enbNodes;
enbNodes.Create(1);
NodeContainer ueNodes;
ueNodes.Create(2);
```

Note that the above Node instances at this point still don't have an LTE protocol stack installed; they're just empty nodes.

4. Configure the Mobility model for all the nodes:

```
MobilityHelper mobility;
mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
mobility.Install(enbNodes);
mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
mobility.Install(ueNodes);
```

The above will place all nodes at the coordinates (0,0,0). Please refer to the documentation of the ns-3 mobility model for how to set your own position or configure node movement.

5. Install an LTE protocol stack on the eNB(s):

```
NetDeviceContainer enbDevs;
enbDevs = lteHelper->InstallEnbDevice(enbNodes);
```

6. Install an LTE protocol stack on the UEs:

```
NetDeviceContainer ueDevs;
ueDevs = lteHelper->InstallUeDevice(ueNodes);
```

7. Attach the UEs to an eNB. This will configure each UE according to the eNB configuration, and create an RRC connection between them:

```
lteHelper->Attach(ueDevs, enbDevs.Get(0));
```

8. Activate a data radio bearer between each UE and the eNB it is attached to:

```
enum EpsBearer::Qci q = EpsBearer::GBR_CONV_VOICE;
EpsBearer bearer(q);
lteHelper->ActivateDataRadioBearer(ueDevs, bearer);
```

this method will also activate two saturation traffic generators for that bearer, one in uplink and one in downlink.

9. Set the stop time:

```
Simulator::Stop(Seconds(0.005));
```

This is needed otherwise the simulation will last forever, because (among others) the start-of-subframe event is scheduled repeatedly, and the ns-3 simulator scheduler will hence never run out of events.

10. Run the simulation:

```
Simulator::Run();
```

11. Cleanup and exit:

```
Simulator::Destroy();
return 0;
}
```

For how to compile and run simulation programs, please refer to [\[ns3tutorial\]](#).

19.2.4 Configuration of LTE model parameters

All the relevant LTE model parameters are managed through the ns-3 attribute system. Please refer to the [\[ns3tutorial\]](#) and [\[ns3manual\]](#) for detailed information on all the possible methods to do it (environmental variables, C++ API, GtkConfigStore...).

In the following, we just briefly summarize how to do it using input files together with the ns-3 ConfigStore. First of all, you need to put the following in your simulation program, right after `main ()` starts:

```
CommandLine cmd(__FILE__);
cmd.Parse(argc, argv);
ConfigStore inputConfig;
inputConfig.ConfigureDefaults();
// parse again so you can override default values from the command line
cmd.Parse(argc, argv);
```

for the above to work, make sure you also `#include "ns3/config-store.h"`. Now create a text file named (for example) `input-defaults.txt` specifying the new default values that you want to use for some attributes:

```
default ns3::LteHelper::Scheduler "ns3::PffMacScheduler"
default ns3::LteHelper::PathlossModel "ns3::FriisSpectrumPropagationLossModel"
default ns3::LteEnbNetDevice::UlBandwidth "25"
default ns3::LteEnbNetDevice::DlBandwidth "25"
default ns3::LteEnbNetDevice::DlEarfcn "100"
default ns3::LteEnbNetDevice::UlEarfcn "18100"
```

(continues on next page)

(continued from previous page)

```
default ns3::LteUePhy::TxPower "10"
default ns3::LteUePhy::NoiseFigure "9"
default ns3::LteEnbPhy::TxPower "30"
default ns3::LteEnbPhy::NoiseFigure "5"
```

Supposing your simulation program is called `src/lte/examples/lte-sim-with-input`, you can now pass these settings to the simulation program in the following way:

```
./ns3 run src/lte/examples/lte-sim-with-input
--command-template="%s --ns3::ConfigStore::Filename=input-defaults.txt
--ns3::ConfigStore::Mode=Load --ns3::ConfigStore::FileFormat=RawText"
```

Furthermore, you can generate a template input file with the following command:

```
./ns3 run src/lte/examples/lte-sim-with-input
--command-template="%s --ns3::ConfigStore::Filename=input-defaults.txt
--ns3::ConfigStore::Mode=Save --ns3::ConfigStore::FileFormat=RawText"
```

note that the above will put in the file `input-defaults.txt` *all* the default values that are registered in your particular build of the simulator, including lots of non-LTE attributes.

19.2.5 Configure LTE MAC Scheduler

There are several types of LTE MAC scheduler user can choose here. User can use following codes to define scheduler type:

```
Ptr<LteHelper> lteHelper = CreateObject<LteHelper>();
lteHelper->SetSchedulerType("ns3::FdMtFfMacScheduler"); // FD-MT scheduler
lteHelper->SetSchedulerType("ns3::TdMtFfMacScheduler"); // TD-MT scheduler
lteHelper->SetSchedulerType("ns3::TtaFfMacScheduler"); // TTA scheduler
lteHelper->SetSchedulerType("ns3::FdBetFfMacScheduler"); // FD-BET scheduler
lteHelper->SetSchedulerType("ns3::TdBetFfMacScheduler"); // TD-BET scheduler
lteHelper->SetSchedulerType("ns3::FdTbfqFfMacScheduler"); // FD-TBFQ scheduler
lteHelper->SetSchedulerType("ns3::TdTbfqFfMacScheduler"); // TD-TBFQ scheduler
lteHelper->SetSchedulerType("ns3::PssFfMacScheduler"); //PSS scheduler
```

TBFQ and PSS have more parameters than other schedulers. Users can define those parameters in following way:

```
* TBFQ scheduler::

Ptr<LteHelper> lteHelper = CreateObject<LteHelper>();
lteHelper->SetSchedulerAttribute("DebtLimit", IntegerValue(yourvalue)); // default
↳ value -625000 bytes(-5Mb)
lteHelper->SetSchedulerAttribute("CreditLimit", UintegerValue(yourvalue)); // default
↳ value 625000 bytes(5Mb)
lteHelper->SetSchedulerAttribute("TokenPoolSize", UintegerValue(yourvalue)); // default
↳ value 1 byte
lteHelper->SetSchedulerAttribute("CreditableThreshold", UintegerValue(yourvalue)); //
↳ default value 0

* PSS scheduler::
```

(continues on next page)

(continued from previous page)

```
Ptr<LteHelper> lteHelper = CreateObject<LteHelper>();
lteHelper->SetSchedulerAttribute("nMux", UIntegerValue(yourvalue)); // the maximum
↳ number of UE selected by TD scheduler
lteHelper->SetSchedulerAttribute("PssFdSchedulerType", StringValue("CoItA")); // PF
↳ scheduler type in PSS
```

In TBFQ, default values of debt limit and credit limit are set to -5Mb and 5Mb respectively based on paper [FABokhari2009]. Current implementation does not consider credit threshold ($C = 0$). In PSS, if user does not define nMux, PSS will set this value to half of total UE. The default FD scheduler is PFsch.

In addition, token generation rate in TBFQ and target bit rate in PSS need to be configured by Guarantee Bit Rate (GBR) or Maximum Bit Rate (MBR) in epc bearer QoS parameters. Users can use following codes to define GBR and MBR in both downlink and uplink:

```
Ptr<LteHelper> lteHelper = CreateObject<LteHelper>();
enum EpsBearer::Qci q = EpsBearer::yourvalue; // define Qci type
GbrQosInformation qos;
qos.gbrDl = yourvalue; // Downlink GBR
qos.gbrUl = yourvalue; // Uplink GBR
qos.mbrDl = yourvalue; // Downlink MBR
qos.mbrUl = yourvalue; // Uplink MBR
EpsBearer bearer(q, qos);
lteHelper->ActivateDedicatedEpsBearer(ueDevs, bearer, EpcTft::Default());
```

In PSS, TBR is obtained from GBR in bearer level QoS parameters. In TBFQ, token generation rate is obtained from the MBR setting in bearer level QoS parameters, which therefore needs to be configured consistently. For constant bit rate (CBR) traffic, it is suggested to set MBR to GBR. For variance bit rate (VBR) traffic, it is suggested to set MBR k times larger than GBR in order to cover the peak traffic rate. In current implementation, k is set to three based on paper [FABokhari2009]. In addition, current version of TBFQ does not consider RLC header and PDCP header length in MBR and GBR. Another parameter in TBFQ is packet arrival rate. This parameter is calculated within scheduler and equals to the past average throughput which is used in PF scheduler.

Many useful attributes of the LTE-EPC model will be described in the following subsections. Still, there are many attributes which are not explicitly mentioned in the design or user documentation, but which are clearly documented using the ns-3 attribute system. You can easily print a list of the attributes of a given object together with their description and default value passing `--PrintAttributes=` to a simulation program, like this:

```
./ns3 run lene-simple --command-template="%s --PrintAttributes=ns3::LteHelper"
```

You can try also with other LTE and EPC objects, like this:

```
./ns3 run lene-simple --command-template="%s --PrintAttributes=ns3::LteEnbNetDevice"
./ns3 run lene-simple --command-template="%s --PrintAttributes=ns3::LteEnbMac"
./ns3 run lene-simple --command-template="%s --PrintAttributes=ns3::LteEnbPhy"
./ns3 run lene-simple --command-template="%s --PrintAttributes=ns3::LteUePhy"
./ns3 run lene-simple --command-template="%s --PrintAttributes=ns3::PointToPointEpcHelper"
↳ "
```

19.2.6 Simulation Output

The ns-3 LTE model currently supports the output to file of PHY, MAC, RLC and PDCP level Key Performance Indicators (KPIs). You can enable it in the following way:

```
Ptr<LteHelper> lteHelper = CreateObject<LteHelper>();

// configure all the simulation scenario here...

lteHelper->EnablePhyTraces();
lteHelper->EnableMacTraces();
lteHelper->EnableRlcTraces();
lteHelper->EnablePdcPTraces();

Simulator::Run();
```

RLC and PDCP KPIs are calculated over a time interval and stored on ASCII files, two for RLC KPIs and two for PDCP KPIs, in each case one for uplink and one for downlink. The time interval duration can be controlled using the attribute `ns3::RadioBearerStatsCalculator::EpochDuration`.

The columns of the RLC KPI files is the following (the same for uplink and downlink):

1. start time of measurement interval in seconds since the start of simulation
2. end time of measurement interval in seconds since the start of simulation
3. Cell ID
4. unique UE ID (IMSI)
5. cell-specific UE ID (RNTI)
6. Logical Channel ID
7. Number of transmitted RLC PDUs
8. Total bytes transmitted.
9. Number of received RLC PDUs
10. Total bytes received
11. Average RLC PDU delay in seconds
12. Standard deviation of the RLC PDU delay
13. Minimum value of the RLC PDU delay
14. Maximum value of the RLC PDU delay
15. Average RLC PDU size, in bytes
16. Standard deviation of the RLC PDU size
17. Minimum RLC PDU size
18. Maximum RLC PDU size

Similarly, the columns of the PDCP KPI files is the following (again, the same for uplink and downlink):

1. start time of measurement interval in seconds since the start of simulation
2. end time of measurement interval in seconds since the start of simulation
3. Cell ID

4. unique UE ID (IMSI)
5. cell-specific UE ID (RNTI)
6. Logical Channel ID
7. Number of transmitted PDCP PDUs
8. Total bytes transmitted.
9. Number of received PDCP PDUs
10. Total bytes received
11. Average PDCP PDU delay in seconds
12. Standard deviation of the PDCP PDU delay
13. Minimum value of the PDCP PDU delay
14. Maximum value of the PDCP PDU delay
15. Average PDCP PDU size, in bytes
16. Standard deviation of the PDCP PDU size
17. Minimum PDCP PDU size
18. Maximum PDCP PDU size

Note: The PDCP traces for data radio bearers are not generated when *SM RLC* is used.

MAC KPIs are basically a trace of the resource allocation reported by the scheduler upon the start of every subframe. They are stored in ASCII files. For downlink MAC KPIs the format is the following:

1. Simulation time in seconds at which the allocation is indicated by the scheduler
2. Cell ID
3. unique UE ID (IMSI)
4. Frame number
5. Subframe number
6. cell-specific UE ID (RNTI)
7. MCS of TB 1
8. size of TB 1
9. MCS of TB 2 (0 if not present)
10. size of TB 2 (0 if not present)

while for uplink MAC KPIs the format is:

1. Simulation time in seconds at which the allocation is indicated by the scheduler
2. Cell ID
3. unique UE ID (IMSI)
4. Frame number
5. Subframe number
6. cell-specific UE ID (RNTI)
7. MCS of TB

8. size of TB

The names of the files used for MAC KPI output can be customized via the ns-3 attributes `ns3::MacStatsCalculator::DlOutputFilename` and `ns3::MacStatsCalculator::UlOutputFilename`.

PHY KPIs are distributed in seven different files, configurable through the attributes

1. `ns3::PhyStatsCalculator::DlRsrpSinrFilename`
2. `ns3::PhyStatsCalculator::UeSinrFilename`
3. `ns3::PhyStatsCalculator::InterferenceFilename`
4. `ns3::PhyStatsCalculator::DlTxOutputFilename`
5. `ns3::PhyStatsCalculator::UlTxOutputFilename`
6. `ns3::PhyStatsCalculator::DlRxOutputFilename`
7. `ns3::PhyStatsCalculator::UlRxOutputFilename`

In the RSRP/SINR file, the following content is available:

1. Simulation time in seconds at which the allocation is indicated by the scheduler
2. Cell ID
3. unique UE ID (IMSI)
4. RSRP
5. Linear average over all RBs of the downlink SINR in linear units

The contents in the UE SINR file are:

1. Simulation time in seconds at which the allocation is indicated by the scheduler
2. Cell ID
3. unique UE ID (IMSI)
4. uplink SINR in linear units for the UE

In the interference filename the content is:

1. Simulation time in seconds at which the allocation is indicated by the scheduler
2. Cell ID
3. List of interference values per RB

In UL and DL transmission files the parameters included are:

1. Simulation time in milliseconds
2. Cell ID
3. unique UE ID (IMSI)
4. RNTI
5. Layer of transmission
6. MCS
7. size of the TB
8. Redundancy version
9. New Data Indicator flag

And finally, in UL and DL reception files the parameters included are:

1. Simulation time in milliseconds
2. Cell ID
3. unique UE ID (IMSI)
4. RNTI
5. Transmission Mode
6. Layer of transmission
7. MCS
8. size of the TB
9. Redundancy version
10. New Data Indicator flag
11. Correctness in the reception of the TB

Note: The traces generated by simulating the scenarios involving the RLF will have a discontinuity in time from the moment of the RLF event until the UE connects again to an eNB.

19.2.7 Fading Trace Usage

In this section we will describe how to use fading traces within LTE simulations.

Fading Traces Generation

It is possible to generate fading traces by using a dedicated matlab script provided with the code (`/lte/model/fading-traces/fading-trace-generator.m`). This script already includes the typical taps configurations for three 3GPP scenarios (i.e., pedestrian, vehicular and urban as defined in Annex B.2 of [TS36104]); however users can also introduce their specific configurations. The list of the configurable parameters is provided in the following:

- `fc` : the frequency in use (it affects the computation of the doppler speed).
- `v_km_h` : the speed of the users
- `traceDuration` : the duration in seconds of the total length of the trace.
- `numRBs` : the number of the resource block to be evaluated.
- `tag` : the tag to be applied to the file generated.

The file generated contains ASCII-formatted real values organized in a matrix fashion: every row corresponds to a different RB, and every column correspond to a different temporal fading trace sample.

It has to be noted that the ns-3 LTE module is able to work with any fading trace file that complies with the above described ASCII format. Hence, other external tools can be used to generate custom fading traces, such as for example other simulators or experimental devices.

Fading Traces Usage

When using a fading trace, it is of paramount importance to specify correctly the trace parameters in the simulation, so that the fading model can load and use it correctly. The parameters to be configured are:

- **TraceFilename** : the name of the trace to be loaded (absolute path, or relative path w.r.t. the path from where the simulation program is executed);
- **TraceLength** : the trace duration in seconds;
- **SamplesNum** : the number of samples;
- **WindowSize** : the size of the fading sampling window in seconds;

It is important to highlight that the sampling interval of the fading trace has to be 1 ms or greater, and in the latter case it has to be an integer multiple of 1 ms in order to be correctly processed by the fading module.

The default configuration of the matlab script provides a trace 10 seconds long, made of 10,000 samples (i.e., 1 sample per TTI=1ms) and used with a windows size of 0.5 seconds amplitude. These are also the default values of the parameters above used in the simulator; therefore their settage can be avoided in case the fading trace respects them.

In order to activate the fading module (which is not active by default) the following code should be included in the simulation program:

```
Ptr<LteHelper> lteHelper = CreateObject<LteHelper>();
lteHelper->SetFadingModel("ns3::TraceFadingLossModel");
```

And for setting the parameters:

```
lteHelper->SetFadingModelAttribute("TraceFilename", StringValue("src/lte/model/fading-
↪traces/fading_trace_EPA_3kmph.fad"));
lteHelper->SetFadingModelAttribute("TraceLength", TimeValue(Seconds(10)));
lteHelper->SetFadingModelAttribute("SamplesNum", UIntegerValue(10000));
lteHelper->SetFadingModelAttribute("WindowSize", TimeValue(Seconds(0.5)));
lteHelper->SetFadingModelAttribute("RbNum", UIntegerValue(100));
```

It has to be noted that, **TraceFilename** does not have a default value, therefore it has to be always set explicitly.

The simulator provide natively three fading traces generated according to the configurations defined in in Annex B.2 of [TS36104]. These traces are available in the folder `src/lte/model/fading-traces/`. An excerpt from these traces is represented in the following figures.

19.2.8 Mobility Model with Buildings

We now explain by examples how to use the buildings model (in particular, the `MobilityBuildingInfo` and the `BuildingPropagationModel` classes) in an ns-3 simulation program to setup an LTE simulation scenario that includes buildings and indoor nodes.

1. Header files to be included:

```
#include "ns3/mobility-building-info.h"
#include "ns3/buildings-propagation-loss-model.h"
#include "ns3/building.h"
```

2. Pathloss model selection:

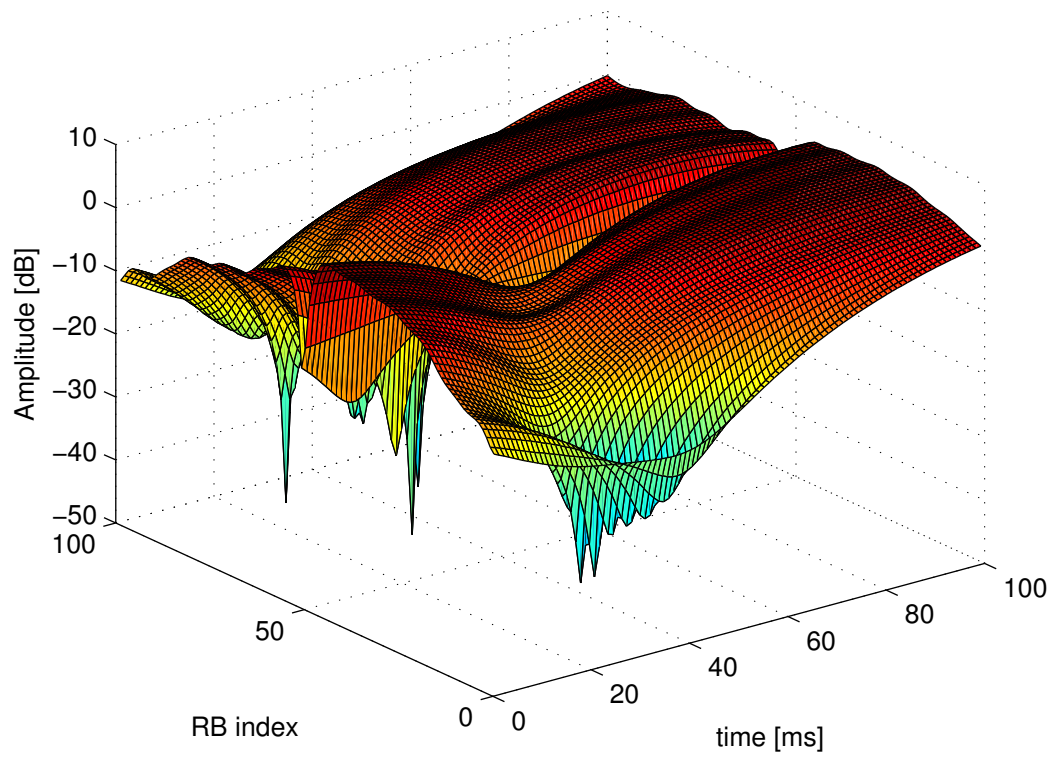


Fig. 19.71: Excerpt of the fading trace included in the simulator for a pedestrian scenario (speed of 3 kmph).

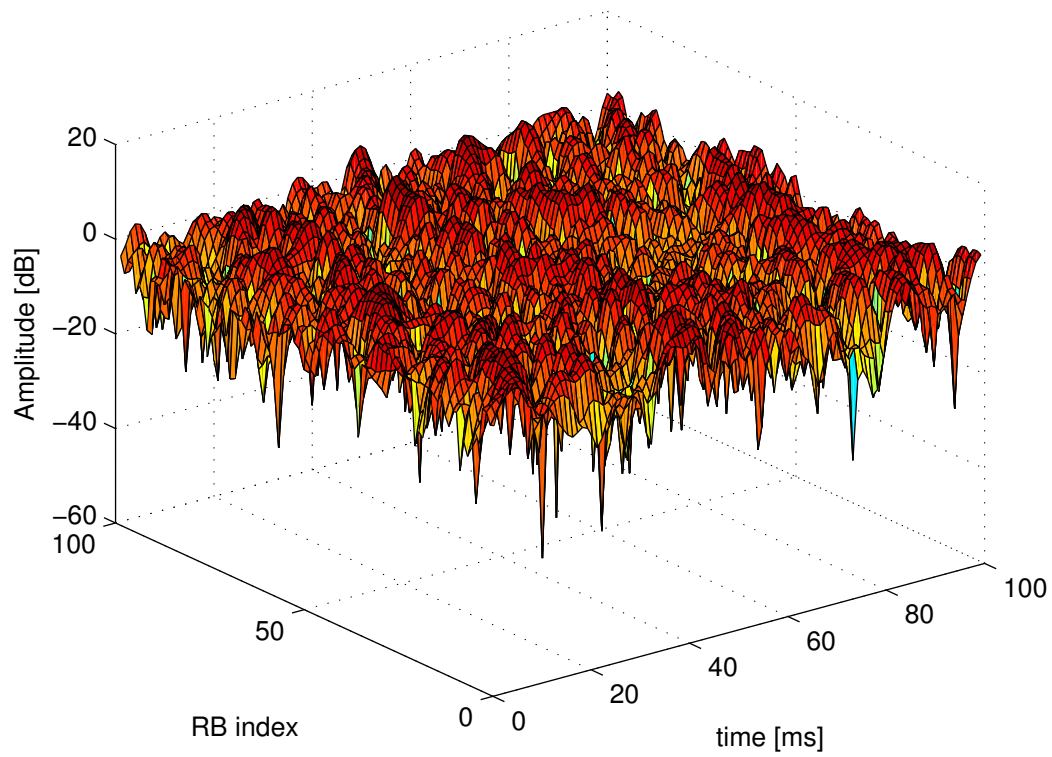


Fig. 19.72: Excerpt of the fading trace included in the simulator for a vehicular scenario (speed of 60 kmph).

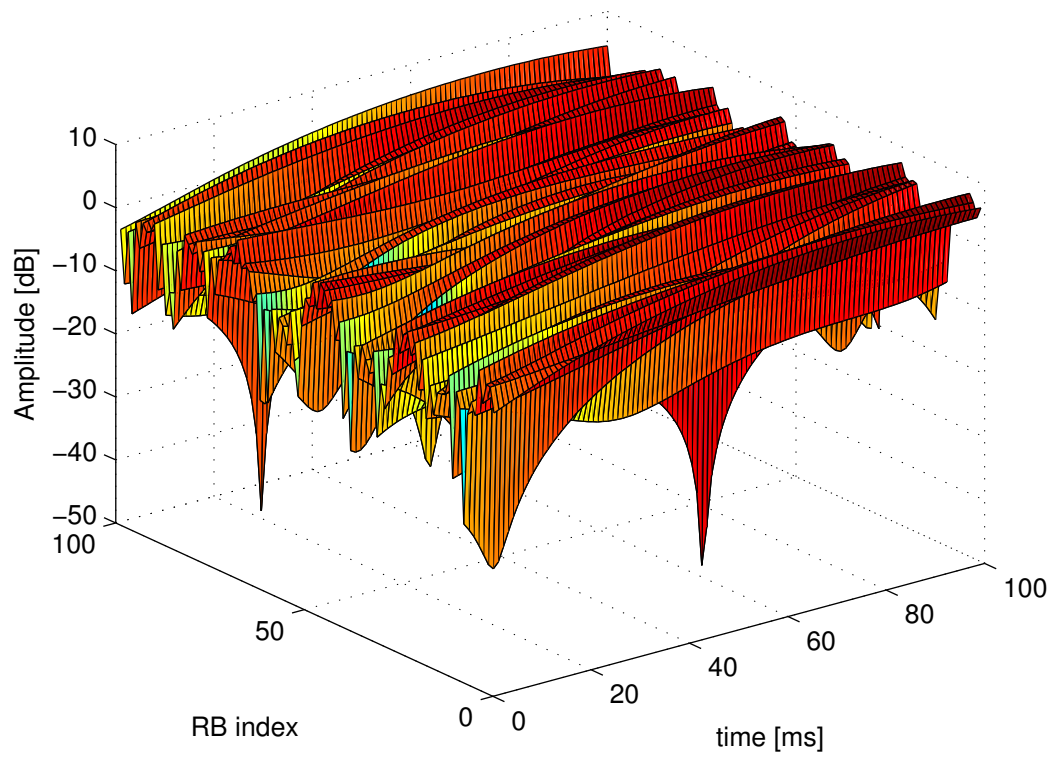


Fig. 19.73: Excerpt of the fading trace included in the simulator for an urban scenario (speed of 3 kmph).

```
Ptr<LteHelper> lteHelper = CreateObject<LteHelper>();

lteHelper->SetAttribute("PathlossModel", StringValue(
↳ "ns3::BuildingsPropagationLossModel"));
```

3. EUTRA Band Selection

The selection of the working frequency of the propagation model has to be done with the standard ns-3 attribute system as described in the correspond section (“Configuration of LTE model parameters”) by means of the DLEarfcn and ULEarfcn parameters, for instance:

```
lteHelper->SetEnbDeviceAttribute("DLEarfcn", UIntegerValue(100));
lteHelper->SetEnbDeviceAttribute("ULEarfcn", UIntegerValue(18100));
```

It is to be noted that using other means to configure the frequency used by the propagation model (i.e., configuring the corresponding BuildingsPropagationLossModel attributes directly) might generates conflicts in the frequencies definition in the modules during the simulation, and is therefore not advised.

1. Mobility model selection:

```
MobilityHelper mobility;
mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");

It is to be noted that any mobility model can be used.
```

2. Building creation:

```
double x_min = 0.0;
double x_max = 10.0;
double y_min = 0.0;
double y_max = 20.0;
double z_min = 0.0;
double z_max = 10.0;
Ptr<Building> b = CreateObject<Building>();
b->SetBoundaries(Box(x_min, x_max, y_min, y_max, z_min, z_max));
b->SetBuildingType(Building::Residential);
b->SetExtWallsType(Building::ConcreteWithWindows);
b->SetNFloors(3);
b->SetNRoomsX(3);
b->SetNRoomsY(2);
```

This will instantiate a residential building with base of 10 x 20 meters and height of 10 meters whose external walls are of concrete with windows; the building has three floors and has an internal 3 x 2 grid of rooms of equal size.

3. Node creation and positioning:

```
ueNodes.Create(2);
mobility.Install(ueNodes);
BuildingsHelper::Install(ueNodes);
NetDeviceContainer ueDevs;
ueDevs = lteHelper->InstallUeDevice(ueNodes);
Ptr<ConstantPositionMobilityModel> mm0 = enbNodes.Get(0)->GetObject
↳ <ConstantPositionMobilityModel>();
Ptr<ConstantPositionMobilityModel> mm1 = enbNodes.Get(1)->GetObject
```

(continues on next page)

(continued from previous page)

```

↪ <ConstantPositionMobilityModel>();
mm0->SetPosition(Vector(5.0, 5.0, 1.5));
mm1->SetPosition(Vector(30.0, 40.0, 1.5));

```

4. Finalize the building and mobility model configuration:

```
BuildingsHelper::MakeMobilityModelConsistent();
```

See the documentation of the *buildings* module for more detailed information.

19.2.9 PHY Error Model

The Physical error model consists of the data error model and the downlink control error model, both of them active by default. It is possible to deactivate them with the ns3 attribute system, in detail:

```

Config::SetDefault("ns3::LteSpectrumPhy::CtrlErrorModelEnabled", BooleanValue(false));
Config::SetDefault("ns3::LteSpectrumPhy::DataErrorModelEnabled", BooleanValue(false));

```

19.2.10 MIMO Model

In this subsection we illustrate how to configure the MIMO parameters. LTE defines 7 types of transmission modes:

- Transmission Mode 1: SISO.
- Transmission Mode 2: MIMO Tx Diversity.
- Transmission Mode 3: MIMO Spatial Multiplexity Open Loop.
- Transmission Mode 4: MIMO Spatial Multiplexity Closed Loop.
- Transmission Mode 5: MIMO Multi-User.
- Transmission Mode 6: Closer loop single layer precoding.
- Transmission Mode 7: Single antenna port 5.

According to model implemented, the simulator includes the first three transmission modes types. The default one is the Transmission Mode 1 (SISO). In order to change the default Transmission Mode to be used, the attribute DefaultTransmissionMode of the LteEnbRrc can be used, as shown in the following:

```

Config::SetDefault("ns3::LteEnbRrc::DefaultTransmissionMode", UintegerValue(0)); // SISO
Config::SetDefault("ns3::LteEnbRrc::DefaultTransmissionMode", UintegerValue(1)); // MIMO_
↪Tx diversity(1 layer)
Config::SetDefault("ns3::LteEnbRrc::DefaultTransmissionMode", UintegerValue(2)); // MIMO_
↪Spatial Multiplexity(2 layers)

```

For changing the transmission mode of a certain user during the simulation a specific interface has been implemented in both standard schedulers:

```
void TransmissionModeConfigurationUpdate(uint16_t rnti, uint8_t txMode);
```

This method can be used both for developing transmission mode decision engine (i.e., for optimizing the transmission mode according to channel condition and/or user's requirements) and for manual switching from simulation script. In the latter case, the switching can be done as shown in the following:

```
Ptr<LteEnbNetDevice> lteEnbDev = enbDevs.Get(0)->GetObject<LteEnbNetDevice>();
PointerValue ptrval;
enbNetDev->GetAttribute("FfMacScheduler", ptrval);
Ptr<RrFfMacScheduler> rrsched = ptrval.Get<RrFfMacScheduler>();
Simulator::Schedule(Seconds(0.2), &RrFfMacScheduler::TransmissionModeConfigurationUpdate,
↳ rrsched, rnti, 1);
```

Finally, the model implemented can be reconfigured according to different MIMO models by updating the gain values (the only constraints is that the gain has to be constant during simulation run-time and common for the layers). The gain of each Transmission Mode can be changed according to the standard ns3 attribute system, where the attributes are: TxMode1Gain, TxMode2Gain, TxMode3Gain, TxMode4Gain, TxMode5Gain, TxMode6Gain and TxMode7Gain. By default only TxMode1Gain, TxMode2Gain and TxMode3Gain have a meaningful value, that are the ones derived by `_[CatreuxMIMO]` (i.e., respectively 0.0, 4.2 and -2.8 dB).

19.2.11 Use of AntennaModel

We now show how associate a particular AntennaModel with an eNB device in order to model a sector of a macro eNB. For this purpose, it is convenient to use the CosineAntennaModel provided by the ns-3 antenna module. The configuration of the eNB is to be done via the LteHelper instance right before the creation of the EnbNetDevice, as shown in the following:

```
lteHelper->SetEnbAntennaModelType("ns3::CosineAntennaModel");
lteHelper->SetEnbAntennaModelAttribute("Orientation", DoubleValue(0));
lteHelper->SetEnbAntennaModelAttribute("Beamwidth", DoubleValue(60));
lteHelper->SetEnbAntennaModelAttribute("MaxGain", DoubleValue(0.0));
```

the above code will generate an antenna model with a 60 degrees beamwidth pointing along the X axis. The orientation is measured in degrees from the X axis, e.g., an orientation of 90 would point along the Y axis, and an orientation of -90 would point in the negative direction along the Y axis. The beamwidth is the -3 dB beamwidth, e.g, for a 60 degree beamwidth the antenna gain at an angle of ± 30 degrees from the direction of orientation is -3 dB.

To create a multi-sector site, you need to create different ns-3 nodes placed at the same position, and to configure separate EnbNetDevice with different antenna orientations to be installed on each node.

19.2.12 Radio Environment Maps

By using the class RadioEnvironmentMapHelper it is possible to output to a file a Radio Environment Map (REM), i.e., a uniform 2D grid of values that represent the Signal-to-noise ratio in the downlink with respect to the eNB that has the strongest signal at each point. It is possible to specify if REM should be generated for data or control channel. Also user can set the RbId, for which REM will be generated. Default RbId is -1, what means that REM will generated with averaged Signal-to-noise ratio from all RBs.

To do this, you just need to add the following code to your simulation program towards the end, right before the call to Simulator::Run():

```
Ptr<RadioEnvironmentMapHelper> remHelper = CreateObject<RadioEnvironmentMapHelper>();
remHelper->SetAttribute("Channel", PointerValue(lteHelper->
↳ GetDownlinkSpectrumChannel()));
remHelper->SetAttribute("OutputFile", StringValue("rem.out"));
remHelper->SetAttribute("XMin", DoubleValue(-400.0));
remHelper->SetAttribute("XMax", DoubleValue(400.0));
remHelper->SetAttribute("XRes", UIntegerValue(100));
```

(continues on next page)

(continued from previous page)

```

remHelper->SetAttribute("YMin", DoubleValue(-300.0));
remHelper->SetAttribute("YMax", DoubleValue(300.0));
remHelper->SetAttribute("YRes", UIntegerValue(75));
remHelper->SetAttribute("Z", DoubleValue(0.0));
remHelper->SetAttribute("UseDataChannel", BooleanValue(true));
remHelper->SetAttribute("RbId", IntegerValue(10));
remHelper->Install();

```

By configuring the attributes of the `RadioEnvironmentMapHelper` object as shown above, you can tune the parameters of the REM to be generated. Note that each `RadioEnvironmentMapHelper` instance can generate only one REM; if you want to generate more REMs, you need to create one separate instance for each REM.

Note that the REM generation is very demanding, in particular:

- the run-time memory consumption is approximately 5KB per pixel. For example, a REM with a resolution of 500x500 would need about 1.25 GB of memory, and a resolution of 1000x1000 would need about 5 GB (too much for a regular PC at the time of this writing). To overcome this issue, the REM is generated at successive steps, with each step evaluating at most a number of pixels determined by the value of the attribute `RadioEnvironmentMapHelper::MaxPointsPerIteration`.
- if you generate a REM at the beginning of a simulation, it will slow down the execution of the rest of the simulation. If you want to generate a REM for a program and also use the same program to get simulation result, it is recommended to add a command-line switch that allows to either generate the REM or run the complete simulation. For this purpose, note that there is an attribute `RadioEnvironmentMapHelper::StopWhenDone` (default: true) that will force the simulation to stop right after the REM has been generated.

The REM is stored in an ASCII file in the following format:

- column 1 is the x coordinate
- column 2 is the y coordinate
- column 3 is the z coordinate
- column 4 is the SINR in linear units

A minimal gnuplot script that allows you to plot the REM is given below:

```

set view map;
set xlabel "X"
set ylabel "Y"
set cblabel "SINR (dB)"
unset key
plot "rem.out" using ($1):($2):(10*log10($4)) with image

```

As an example, here is the REM that can be obtained with the example program `lena-dual-stripe`, which shows a three-sector LTE macrocell in a co-channel deployment with some residential femtocells randomly deployed in two blocks of apartments.

Note that the `lena-dual-stripe` example program also generate gnuplot-compatible output files containing information about the positions of the UE and eNB nodes as well as of the buildings, respectively in the files `ues.txt`, `enbs.txt` and `buildings.txt`. These can be easily included when using gnuplot. For example, assuming that your gnuplot script (e.g., the minimal gnuplot script described above) is saved in a file named `my_plot_script`, running the following command would plot the location of UEs, eNBs and buildings on top of the REM:

```
gnuplot -p enbs.txt ues.txt buildings.txt my_plot_script
```

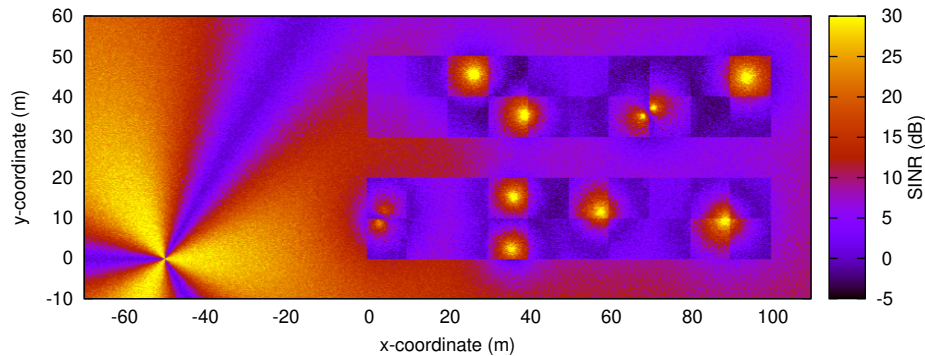


Fig. 19.74: REM obtained from the lena-dual-stripe example

19.2.13 AMC Model and CQI Calculation

The simulator provides two possible schemes for what concerns the selection of the MCSs and correspondingly the generation of the CQIs. The first one is based on the GSoC module [Piro2011] and works per RB basis. This model can be activated with the ns3 attribute system, as presented in the following:

```
Config::SetDefault("ns3::LteAmc::AmcModel", EnumValue(LteAmc::PiroEW2010));
```

While, the solution based on the physical error model can be controlled with:

```
Config::SetDefault("ns3::LteAmc::AmcModel", EnumValue(LteAmc::MiErrorModel));
```

Finally, the required efficiency of the PiroEW2010 AMC module can be tuned thanks to the Ber attribute(), for instance:

```
Config::SetDefault("ns3::LteAmc::Ber", DoubleValue(0.00005));
```

19.2.14 Evolved Packet Core (EPC)

We now explain how to write a simulation program that allows to simulate the EPC in addition to the LTE radio access network. The use of EPC allows to use IPv4 and IPv6 networking with LTE devices. In other words, you will be able to use the regular ns-3 applications and sockets over IPv4 and IPv6 over LTE, and also to connect an LTE network to any other IPv4 and IPv6 network you might have in your simulation.

First of all, in addition to LteHelper that we already introduced in *Basic simulation program*, you need to use an additional EpcHelper class, which will take care of creating the EPC entities and network topology. Note that you can't use EpcHelper directly, as it is an abstract base class; instead, you need to use one of its child classes, which provide different EPC topology implementations. In this example we will consider PointToPointEpcHelper, which implements an EPC based on point-to-point links. To use it, you need first to insert this code in your simulation program:

```
Ptr<LteHelper> lteHelper = CreateObject<LteHelper>();
Ptr<PointToPointEpcHelper> epcHelper = CreateObject<PointToPointEpcHelper>();
```

Then, you need to tell the LTE helper that the EPC will be used:

```
lteHelper->SetEpcHelper(epcHelper);
```

the above step is necessary so that the LTE helper will trigger the appropriate EPC configuration in correspondence with some important configuration, such as when a new eNB or UE is added to the simulation, or an EPS bearer is created. The EPC helper will automatically take care of the necessary setup, such as S1 link creation and S1 bearer setup. All this will be done without the intervention of the user.

Calling `lteHelper->SetEpcHelper(epcHelper)` enables the use of EPC, and has the side effect that any new `LteEnbRrc` that is created will have the `EpsBearerToRlcMapping` attribute set to `RLC_UM_ALWAYS` instead of `RLC_SM_ALWAYS` if the latter was the default; otherwise, the attribute won't be changed (e.g., if you changed the default to `RLC_AM_ALWAYS`, it won't be touched).

It is to be noted that the `EpcHelper` will also automatically create the PGW node and configure it so that it can properly handle traffic from/to the LTE radio access network. Still, you need to add some explicit code to connect the PGW to other IPv4/IPv6 networks (e.g., the internet, another EPC). Here is a very simple example about how to connect a single remote host (IPv4 type) to the PGW via a point-to-point link:

```
Ptr<Node> pgw = epcHelper->GetPgwNode();

// Create a single RemoteHost
NodeContainer remoteHostContainer;
remoteHostContainer.Create(1);
Ptr<Node> remoteHost = remoteHostContainer.Get(0);
InternetStackHelper internet;
internet.Install(remoteHostContainer);

// Create the internet
PointToPointHelper p2ph;
p2ph.SetDeviceAttribute("DataRate", DataRateValue(DataRate("100Gb/s")));
p2ph.SetDeviceAttribute("Mtu", UintegerValue(1500));
p2ph.SetChannelAttribute("Delay", TimeValue(Seconds(0.010)));
NetDeviceContainer internetDevices = p2ph.Install(pgw, remoteHost);
Ipv4AddressHelper ipv4h;
ipv4h.SetBase("1.0.0.0", "255.0.0.0");
Ipv4InterfaceContainer internetIpIfaces = ipv4h.Assign(internetDevices);
// interface 0 is localhost, 1 is the p2p device
Ipv4Address remoteHostAddr = internetIpIfaces.GetAddress(1);

Ipv4StaticRoutingHelper ipv4RoutingHelper;
Ptr<Ipv4StaticRouting> remoteHostStaticRouting;
remoteHostStaticRouting = ipv4RoutingHelper.GetStaticRouting(remoteHost->GetObject<Ipv4>
    ↪());
remoteHostStaticRouting->AddNetworkRouteTo(epcHelper->GetEpcIpv4NetworkAddress(),
    Ipv4Mask("255.255.0.0"), 1);
```

Now, you should go on and create LTE eNBs and UEs as explained in the previous sections. You can of course configure other LTE aspects such as pathloss and fading models. Right after you created the UEs, you should also configure them for IP networking. This is done as follows. We assume you have a container for UE and eNodeB nodes like this:


```
NodeContainer ueNodes;
NodeContainer enbNodes;
```

to configure an LTE-only simulation, you would then normally do something like this:

```
NetDeviceContainer ueLteDevs = lteHelper->InstallUeDevice(ueNodes);
lteHelper->Attach(ueLteDevs, enbLteDevs.Get(0));
```

in order to configure the UEs for IP networking, you just need to additionally do like this:

```
// we install the IP stack on the UEs
InternetStackHelper internet;
internet.Install(ueNodes);

// assign IP address to UEs
for (uint32_t u = 0; u < ueNodes.GetN(); ++u)
{
    Ptr<Node> ue = ueNodes.Get(u);
    Ptr<NetDevice> ueLteDevice = ueLteDevs.Get(u);
    Ipv4InterfaceContainer ueIpIface;
    ueIpIface = epcHelper->AssignUeIpv4Address(NetDeviceContainer(ueLteDevice));
    // set the default gateway for the UE
    Ptr<Ipv4StaticRouting> ueStaticRouting;
    ueStaticRouting = ipv4RoutingHelper.GetStaticRouting(ue->GetObject<Ipv4>());
    ueStaticRouting->SetDefaultRoute(epcHelper->GetUeDefaultGatewayAddress(), 1);
}
```

The activation of bearers is done in a slightly different way with respect to what done for an LTE-only simulation. First, the method `ActivateDataRadioBearer` is not to be used when the EPC is used. Second, when EPC is used, the default EPS bearer will be activated automatically when you call `LteHelper::Attach()`. Third, if you want to setup dedicated EPS bearer, you can do so using the method `LteHelper::ActivateDedicatedEpsBearer()`. This method takes as a parameter the Traffic Flow Template(TFT), which is a struct that identifies the type of traffic that will be mapped to the dedicated EPS bearer. Here is an example for how to setup a dedicated bearer for an application at the UE communicating on port 1234:

```
Ptr<EpcTft> tft = Create<EpcTft>();
EpcTft::PacketFilter pf;
pf.localPortStart = 1234;
pf.localPortEnd = 1234;
tft->Add(pf);
lteHelper->ActivateDedicatedEpsBearer(ueLteDevs,
                                     EpsBearer(EpsBearer::NGBR_VIDEO_TCP_DEFAULT),
                                     tft);
```

you can of course use custom `EpsBearer` and `EpcTft` configurations, please refer to the doxygen documentation for how to do it.

Finally, you can install applications on the LTE UE nodes that communicate with remote applications over the internet. This is done following the usual ns-3 procedures. Following our simple example with a single `remoteHost`, here is how to setup downlink communication, with an `UdpClient` application on the remote host, and a `PacketSink` on the LTE UE (using the same variable names of the previous code snippets)

```
uint16_t dlPort = 1234;
PacketSinkHelper packetSinkHelper("ns3::UdpSocketFactory",
```

(continues on next page)

(continued from previous page)

```

                                InetSocketAddress(Ipv4Address::GetAny(), dlPort));
ApplicationContainer serverApps = packetSinkHelper.Install(ue);
serverApps.Start(Seconds(0.01));
UdpClientHelper client(ueIpIface.GetAddress(0), dlPort);
ApplicationContainer clientApps = client.Install(remoteHost);
clientApps.Start(Seconds(0.01));

```

That's all! You can now start your simulation as usual:

```

Simulator::Stop(Seconds(10));
Simulator::Run();

```

19.2.15 Using the EPC with emulation mode

In the previous section we used PointToPoint links for the connection between the eNBs and the SGW (S1-U interface) and among eNBs (X2-U and X2-C interfaces). The LTE module supports using emulated links instead of PointToPoint links. This is achieved by just replacing the creation of LteHelper and EpcHelper with the following code:

```

Ptr<LteHelper> lteHelper = CreateObject<LteHelper>();
Ptr<EmuEpcHelper> epcHelper = CreateObject<EmuEpcHelper>();
lteHelper->SetEpcHelper(epcHelper);
epcHelper->Initialize();

```

The attributes `ns3::EmuEpcHelper::sgwDeviceName` and `ns3::EmuEpcHelper::enbDeviceName` are used to set the name of the devices used for transporting the S1-U, X2-U and X2-C interfaces at the SGW and eNB, respectively. We will now show how this is done in an example where we execute the example program `lena-simple-epc-emu` using two virtual ethernet interfaces.

First of all we build ns-3 appropriately:

```

# configure
./ns3 configure --enable-sudo --enable-modules=lte,fd-net-device --enable-examples

# build
./ns3

```

Then we setup two virtual ethernet interfaces, and start wireshark to look at the traffic going through:

```

# note: you need to be root

# create two paired veth devices
ip link add name veth0 type veth peer name veth1
ip link show

# enable promiscuous mode
ip link set veth0 promisc on
ip link set veth1 promisc on

# bring interfaces up
ip link set veth0 up
ip link set veth1 up

```

(continues on next page)

(continued from previous page)

```
# start wireshark and capture on veth0
wireshark &
```

We can now run the example program with the simulated clock:

```
./ns3 run lena-simple-epc-emu --command="%s --ns3::EmuEpcHelper::sgwDeviceName=veth0
--ns3::EmuEpcHelper::enbDeviceName=veth1"
```

Using wireshark, you should see ARP resolution first, then some GTP packets exchanged both in uplink and downlink.

The default setting of the example program is 1 eNB and 1 UE. You can change this via command line parameters, e.g.:

```
./ns3 run lena-simple-epc-emu --command="%s --ns3::EmuEpcHelper::sgwDeviceName=veth0
--ns3::EmuEpcHelper::enbDeviceName=veth1 --nEnbs=2 --nUesPerEnb=2"
```

To get a list of the available parameters:

```
./ns3 run lena-simple-epc-emu --command="%s --PrintHelp"
```

To run with the realtime clock: it turns out that the default debug build is too slow for realtime. Softening the real time constraints with the BestEffort mode is not a good idea: something can go wrong (e.g., ARP can fail) and, if so, you won't get any data packets out. So you need a decent hardware and the optimized build with statically linked modules:

```
./ns3 configure -d optimized --enable-static --enable-modules=lte --enable-examples
--enable-sudo
```

Then run the example program like this:

```
./ns3 run lena-simple-epc-emu --command="%s --ns3::EmuEpcHelper::sgwDeviceName=veth0
--ns3::EmuEpcHelper::enbDeviceName=veth1
--SimulatorImplementationType=ns3::RealtimeSimulatorImpl
--ns3::RealtimeSimulatorImpl::SynchronizationMode=HardLimit"
```

note the HardLimit setting, which will cause the program to terminate if it cannot keep up with real time.

The approach described in this section can be used with any type of net device. For instance, [Baldo2014] describes how it was used to run an emulated LTE-EPC network over a real multi-layer packet-optical transport network.

19.2.16 Custom Backhaul

In the previous sections, *Evolved Packet Core (EPC)*, we explained how to write a simulation program using EPC with a predefined backhaul network between the RAN and the EPC. We used the `PointToPointEpcHelper`. This `EpcHelper` creates point-to-point links between the eNBs and the SGW.

We now explain how to write a simulation program that allows the simulator user to create any kind of backhaul network in the simulation program.

First of all, in addition to `LteHelper`, you need to use the `NoBackhaulEpcHelper` class, which implements an EPC but without connecting the eNBs with the core network. It just creates the network elements of the core network:

```
Ptr<LteHelper> lteHelper = CreateObject<LteHelper>();
Ptr<NoBackhaulEpcHelper> epcHelper = CreateObject<NoBackhaulEpcHelper>();
```

Then, as usual, you need to tell the LTE helper that the EPC will be used:

```
lteHelper->SetEpcHelper(epcHelper);
```

Now, you should create the backhaul network. Here we create point-to-point links as it is done by the `PointToPointEpcHelper`. We assume you have a container for eNB nodes like this:

```
NodeContainer enbNodes;
```

We get the SGW node:

```
Ptr<Node> sgw = epcHelper->GetSgwNode();
```

And we connect every eNB from the container with the SGW with a point-to-point link. We also assign IPv4 addresses to the interfaces of eNB and SGW with `sluIpv4AddressHelper.Assign(sgwEnbDevices)` and finally we tell the `EpcHelper` that this enb has a new S1 interface with `epcHelper->AddS1Interface(enb, enbS1uAddress, sgwS1uAddress)`, where `enbS1uAddress` and `sgwS1uAddress` are the IPv4 addresses of the eNB and the SGW, respectively:

```
Ipv4AddressHelper sluIpv4AddressHelper;

// Create networks of the S1 interfaces
sluIpv4AddressHelper.SetBase("10.0.0.0", "255.255.255.252");

for (uint16_t i = 0; i < enbNodes.GetN(); ++i)
{
    Ptr<Node> enb = enbNodes.Get(i);

    // Create a point to point link between the eNB and the SGW with
    // the corresponding new NetDevices on each side
    PointToPointHelper p2ph;
    DataRate sluLinkDataRate = DataRate("10Gb/s");
    uint16_t sluLinkMtu = 2000;
    Time sluLinkDelay = Time(0);
    p2ph.SetDeviceAttribute("DataRate", DataRateValue(sluLinkDataRate));
    p2ph.SetDeviceAttribute("Mtu", UIntegerValue(sluLinkMtu));
    p2ph.SetChannelAttribute("Delay", TimeValue(sluLinkDelay));
    NetDeviceContainer sgwEnbDevices = p2ph.Install(sgw, enb);

    Ipv4InterfaceContainer sgwEnbIpIfaces = sluIpv4AddressHelper.Assign(sgwEnbDevices);
    sluIpv4AddressHelper.NewNetwork();

    Ipv4Address sgwS1uAddress = sgwEnbIpIfaces.GetAddress(0);
    Ipv4Address enbS1uAddress = sgwEnbIpIfaces.GetAddress(1);

    // Create S1 interface between the SGW and the eNB
    epcHelper->AddS1Interface(enb, enbS1uAddress, sgwS1uAddress);
}
```

This is just an example how to create a custom backhaul network. In this other example, we connect all eNBs and the SGW to the same CSMA network:

```
// Create networks of the S1 interfaces
sluIpv4AddressHelper.SetBase("10.0.0.0", "255.255.255.0");
```

(continues on next page)

(continued from previous page)

```

NodeContainer sgwEnbNodes;
sgwEnbNodes.Add(sgw);
sgwEnbNodes.Add(enbNodes);

Csmahelper csmah;
NetDeviceContainer sgwEnbDevices = csmah.Install(sgwEnbNodes);
Ptr<NetDevice> sgwDev = sgwEnbDevices.Get(0);

Ipv4InterfaceContainer sgwEnbIpIfaces = s1uIpv4AddressHelper.Assign(sgwEnbDevices);
Ipv4Address sgwS1uAddress = sgwEnbIpIfaces.GetAddress(0);

for (uint16_t i = 0; i < enbNodes.GetN(); ++i)
{
    Ptr<Node> enb = enbNodes.Get(i);
    Ipv4Address enbS1uAddress = sgwEnbIpIfaces.GetAddress(i + 1);

    // Create S1 interface between the SGW and the eNB
    epcHelper->AddS1Interface(enb, enbS1uAddress, sgwS1uAddress);
}

```

As you can see, apart from how you create the backhaul network, i.e. the point-to-point links or the CSMA network, the important point is to tell the EpcHelper that an eNB has a new S1 interface.

Now, you should continue configuring your simulation program as it is explained in *Evolved Packet Core (EPC)* subsection. This configuration includes: the internet, installing the LTE eNBs and possibly configuring other LTE aspects, installing the LTE UEs and configuring them as IP nodes, activation of the dedicated EPS bearers and installing applications on the LTE UEs and on the remote hosts.

19.2.17 Network Attachment

As shown in the basic example in section *Basic simulation program*, attaching a UE to an eNodeB is done by calling `LteHelper::Attach` function.

There are 2 possible ways of network attachment. The first method is the “*manual*” one, while the second one has a more “*automatic*” sense on it. Each of them will be covered in this section.

Manual attachment

This method uses the `LteHelper::Attach` function mentioned above. It has been the only available network attachment method in earlier versions of LTE module. It is typically invoked before the simulation begins:

```
lteHelper->Attach(ueDevs, enbDev); // attach one or more UEs to a single eNodeB
```

`LteHelper::InstallEnbDevice` and `LteHelper::InstallUeDevice` functions must have been called before attaching. In an EPC-enabled simulation, it is also required to have IPv4/IPv6 properly pre-installed in the UE.

This method is very simple, but requires you to know exactly which UE belongs to to which eNodeB before the simulation begins. This can be difficult when the UE initial position is randomly determined by the simulation script.

One may choose the distance between the UE and the eNodeB as a criterion for selecting the appropriate cell. It is quite simple (at least from the simulator’s point of view) and sometimes practical. But it is important to note that sometimes distance does not make a single correct criterion. For instance, the eNodeB antenna directivity should be considered

as well. Besides that, one should also take into account the channel condition, which might be fluctuating if there is fading or shadowing in effect. In these kind of cases, network attachment should not be based on distance alone.

In real life, UE will automatically evaluate certain criteria and select the best cell to attach to, without manual intervention from the user. Obviously this is not the case in this `LteHelper::Attach` function. The other network attachment method uses more “*automatic*” approach to network attachment, as will be described next.

Automatic attachment using Idle mode cell selection procedure

The strength of the received signal is the standard criterion used for selecting the best cell to attach to. The use of this criterion is implemented in the *initial cell selection* process, which can be invoked by calling another version of the `LteHelper::Attach` function, as shown below:

```
lteHelper->Attach(ueDevs); // attach one or more UEs to a strongest cell
```

The difference with the manual method is that the destination eNodeB is not specified. The procedure will find the best cell for the UEs, based on several criteria, including the strength of the received signal (RSRP).

After the method is called, the UE will spend some time to measure the neighbouring cells, and then attempt to attach to the best one. More details can be found in section *Initial Cell Selection* of the Design Documentation.

It is important to note that this method only works in EPC-enabled simulations. LTE-only simulations must resort to manual attachment method.

Closed Subscriber Group

An interesting use case of the initial cell selection process is to setup a simulation environment with Closed Subscriber Group (CSG).

For example, a certain eNodeB, typically a smaller version such as femtocell, might belong to a private owner (e.g. a household or business), allowing access only to some UEs which have been previously registered by the owner. The eNodeB and the registered UEs altogether form a CSG.

The access restriction can be simulated by “labeling” the CSG members with the same CSG ID. This is done through the attributes in both eNodeB and UE, for example using the following `LteHelper` functions:

```
// label the following eNodeBs with CSG identity of 1 and CSG indication enabled
lteHelper->SetEnbDeviceAttribute("CsgId", UIntegerValue(1));
lteHelper->SetEnbDeviceAttribute("CsgIndication", BooleanValue(true));

// label one or more UEs with CSG identity of 1
lteHelper->SetUeDeviceAttribute("CsgId", UIntegerValue(1));

// install the eNodeBs and UEs
NetDeviceContainer csgEnbDevs = lteHelper->InstallEnbDevice(csgEnbNodes);
NetDeviceContainer csgUeDevs = lteHelper->InstallUeDevice(csgUeNodes);
```

Then enable the initial cell selection procedure on the UEs:

```
lteHelper->Attach(csgUeDevs);
```

This is necessary because the CSG restriction only works with automatic method of network attachment, but not in the manual method.

Note that setting the CSG indication of an eNodeB as false (the default value) will disable the restriction, i.e., any UEs can connect to this eNodeB.

19.2.18 Configure UE measurements

The active UE measurement configuration in a simulation is dictated by the selected so called “consumers”, such as handover algorithm. Users may add their own configuration into action, and there are several ways to do so:

1. direct configuration in eNodeB RRC entity;
2. configuring existing handover algorithm; and
3. developing a new handover algorithm.

This section will cover the first method only. The second method is covered in *Automatic handover trigger*, while the third method is explained in length in Section *Handover algorithm* of the Design Documentation.

Direct configuration in eNodeB RRC works as follows. User begins by creating a new `LteRrcSap::ReportConfigEutra` instance and pass it to the `LteEnbRrc::AddUeMeasReportConfig` function. The function will return the `measId` (measurement identity) which is a unique reference of the configuration in the eNodeB instance. This function must be called before the simulation begins. The measurement configuration will be active in all UEs attached to the eNodeB throughout the duration of the simulation. During the simulation, user can capture the measurement reports produced by the UEs by listening to the existing `LteEnbRrc::RecvMeasurementReport` trace source.

The structure *ReportConfigEutra* is in accord with 3GPP specification. Definition of the structure and each member field can be found in Section 6.3.5 of [TS36331].

The code sample below configures Event A1 RSRP measurement to every eNodeB within the container devs:

```
LteRrcSap::ReportConfigEutra config;
config.eventId = LteRrcSap::ReportConfigEutra::EVENT_A1;
config.threshold1.choice = LteRrcSap::ThresholdEutra::THRESHOLD_RSRP;
config.threshold1.range = 41;
config.triggerQuantity = LteRrcSap::ReportConfigEutra::RSRP;
config.reportInterval = LteRrcSap::ReportConfigEutra::MS480;

std::vector<uint8_t> measIdList;

for (auto it = devs.Begin(); it != devs.End(); it++)
{
    Ptr<NetDevice> dev = *it;
    Ptr<LteEnbNetDevice> enbDev = dev->GetObject<LteEnbNetDevice>();
    Ptr<LteEnbRrc> enbRrc = enbDev->GetRrc();

    uint8_t measId = enbRrc->AddUeMeasReportConfig(config);
    measIdList.push_back(measId); // remember the measId created

    enbRrc->TraceConnect("RecvMeasurementReport",
                        "context",
                        MakeCallback(&RecvMeasurementReportCallback));
}
```

Note that thresholds are expressed as range. In the example above, the range 41 for RSRP corresponds to -100 dBm. The conversion from and to the range format is due to Section 9.1.4 and 9.1.7 of [TS36133]. The `EutraMeasurementMapping` class has several static functions that can be used for this purpose.

The corresponding callback function would have a definition similar as below:

```
void
RecvMeasurementReportCallback(std::string context,
```

(continues on next page)

(continued from previous page)

```
uint64_t imsi,
uint16_t cellId,
uint16_t rnti,
LteRrcSap::MeasurementReport measReport);
```

This method will register the callback function as a consumer of UE measurements. In the case where there are more than one consumers in the simulation (e.g. handover algorithm), the measurements intended for other consumers will also be captured by this callback function. Users may utilize the `measId` field, contained within the `LteRrcSap::MeasurementReport` argument of the callback function, to tell which measurement configuration has triggered the report.

In general, this mechanism prevents one consumer to unknowingly intervene with another consumer's reporting configuration.

Note that only the reporting configuration part (i.e. `LteRrcSap::ReportConfigEutra`) of the UE measurements parameter is open for consumers to configure, while the other parts are kept hidden. The intra-frequency limitation is the main motivation behind this API implementation decision:

- there is only one, unambiguous and definitive *measurement object*, thus there is no need to configure it;
- *measurement identities* are kept hidden because of the fact that there is one-to-one mapping between reporting configuration and measurement identity, thus a new measurement identity is set up automatically when a new reporting configuration is created;
- *quantity configuration* is configured elsewhere, see [Performing measurements](#); and
- *measurement gaps* are not supported, because it is only applicable for inter-frequency settings;

19.2.19 X2-based handover

As defined by 3GPP, handover is a procedure for changing the serving cell of a UE in CONNECTED mode. The two eNodeBs involved in the process are typically called the *source eNodeB* and the *target eNodeB*.

In order to enable the execution of X2-based handover in simulation, there are two requirements that must be met. Firstly, EPC must be enabled in the simulation (see [Evolved Packet Core \(EPC\)](#)).

Secondly, an X2 interface must be configured between the two eNodeBs, which needs to be done explicitly within the simulation program:

```
lteHelper->AddX2Interface(enbNodes);
```

where `enbNodes` is a `NodeContainer` that contains the two eNodeBs between which the X2 interface is to be configured. If the container has more than two eNodeBs, the function will create an X2 interface between every pair of eNodeBs in the container.

Lastly, the target eNodeB must be configured as “open” to X2 HANDOVER REQUEST. Every eNodeB is open by default, so no extra instruction is needed in most cases. However, users may set the eNodeB to “closed” by setting the boolean attribute `LteEnbRrc::AdmitHandoverRequest` to *false*. As an example, you can run the `lena-x2-handover` program and setting the attribute in this way:

```
NS_LOG=EpcX2:LteEnbRrc ./ns3 run lena-x2-handover --command="%s --
↪ns3::LteEnbRrc::AdmitHandoverRequest=false"
```

After the above three requirements are fulfilled, the handover procedure can be triggered manually or automatically. Each will be presented in the following subsections.

Manual handover trigger

Handover event can be triggered “manually” within the simulation program by scheduling an explicit handover event. The `LteHelper` object provides a convenient method for the scheduling of a handover event. As an example, let us assume that `ueLteDevs` is a `NetDeviceContainer` that contains the UE that is to be handed over, and that `enbLteDevs` is another `NetDeviceContainer` that contains the source and the target eNB. Then, a handover at 0.1s can be scheduled like this:

```
lteHelper->HandoverRequest(Seconds(0.100),
                           ueLteDevs.Get(0),
                           enbLteDevs.Get(0),
                           enbLteDevs.Get(1));
```

Note that the UE needs to be already connected to the source eNB, otherwise the simulation will terminate with an error message.

For an example with full source code, please refer to the `lena-x2-handover` example program.

Automatic handover trigger

Handover procedure can also be triggered “automatically” by the serving eNodeB of the UE. The logic behind the trigger depends on the handover algorithm currently active in the eNodeB RRC entity. Users may select and configure the handover algorithm that will be used in the simulation, which will be explained shortly in this section. Users may also opt to write their own implementation of handover algorithm, as described in Section [Handover algorithm](#) of the Design Documentation.

Selecting a handover algorithm is done via the `LteHelper` object and its `SetHandoverAlgorithmType` method as shown below:

```
Ptr<LteHelper> lteHelper = CreateObject<LteHelper>();
lteHelper->SetHandoverAlgorithmType("ns3::A2A4RsrqHandoverAlgorithm");
```

The selected handover algorithm may also provide several configurable attributes, which can be set as follows:

```
lteHelper->SetHandoverAlgorithmAttribute("ServingCellThreshold",
                                         UIntegerValue(30));
lteHelper->SetHandoverAlgorithmAttribute("NeighbourCellOffset",
                                         UIntegerValue(1));
```

Three options of handover algorithm are included in the LTE module. The *A2-A4-RSRQ* handover algorithm (named as `ns3::A2A4RsrqHandoverAlgorithm`) is the default option, and the usage has already been shown above.

Another option is the *strongest cell* handover algorithm (named as `ns3::A3RsrpHandoverAlgorithm`), which can be selected and configured by the following code:

```
lteHelper->SetHandoverAlgorithmType("ns3::A3RsrpHandoverAlgorithm");
lteHelper->SetHandoverAlgorithmAttribute("Hysteresis",
                                         DoubleValue(3.0));
lteHelper->SetHandoverAlgorithmAttribute("TimeToTrigger",
                                         TimeValue(MilliSeconds(256)));
```

The last option is a special one, called the *no-op* handover algorithm, which basically disables automatic handover trigger. This is useful for example in cases where manual handover trigger need an exclusive control of all handover decision. It does not have any configurable attributes. The usage is as follows:

```
lteHelper->SetHandoverAlgorithmType("ns3::NoOpHandoverAlgorithm");
```

For more information on each handover algorithm's decision policy and their attributes, please refer to their respective subsections in Section *Handover algorithm* of the Design Documentation.

Finally, the `InstallEnbDevice` function of `LteHelper` will instantiate one instance of the selected handover algorithm for each eNodeB device. In other words, make sure to select the right handover algorithm before finalizing it in the following line of code:

```
NetDeviceContainer enbLteDevs = lteHelper->InstallEnbDevice(enbNodes);
```

Example with full source code of using automatic handover trigger can be found in the `1ena-x2-handover-measures` example program.

Tuning simulation with handover

As mentioned in the Design Documentation, the current implementation of handover model may produce unpredicted behaviour when handover failure occurs. This subsection will focus on the steps that should be taken into account by users if they plan to use handover in their simulations.

The major cause of handover failure that we will tackle is the error in transmitting handover-related signaling messages during the execution of a handover procedure. As apparent from the Figure *Sequence diagram of the X2-based handover* from the Design Documentation, there are many of them and they use different interfaces and protocols. For the sake of simplicity, we can safely assume that the X2 interface (between the source eNodeB and the target eNodeB) and the S1 interface (between the target eNodeB and the SGW/PGW) are quite stable. Therefore we will focus our attention to the RRC protocol (between the UE and the eNodeBs) and the Random Access procedure, which are normally transmitted through the air and susceptible to degradation of channel condition.

A general tips to reduce transmission error is to *ensure high enough SINR* level in every UE. This can be done by a proper planning of the network topology that *minimizes network coverage hole*. If the topology has a known coverage hole, then the UE should be configured not to venture to that area.

Another approach to keep in mind is to *avoid too-late handovers*. In other words, handover should happen before the UE's SINR becomes too low, otherwise the UE may fail to receive the handover command from the source eNodeB. Handover algorithms have the means to control how early or late a handover decision is made. For example, A2-A4-RSRQ handover algorithm can be configured with a higher threshold to make it decide a handover earlier. Similarly, smaller hysteresis and/or shorter time-to-trigger in the strongest cell handover algorithm typically results in earlier handovers. In order to find the right values for these parameters, one of the factors that should be considered is the UE movement speed. Generally, a faster moving UE requires the handover to be executed earlier. Some research work have suggested recommended values, such as in [Lee2010].

The above tips should be enough in normal simulation uses, but in the case some special needs arise then an extreme measure can be taken into consideration. For instance, users may consider *disabling the channel error models*. This will ensure that all handover-related signaling messages will be transmitted successfully, regardless of distance and channel condition. However, it will also affect all other data or control packets not related to handover, which may be an unwanted side effect. Otherwise, it can be done as follows:

```
Config::SetDefault("ns3::LteSpectrumPhy::CtrlErrorModelEnabled", BooleanValue(false));  
Config::SetDefault("ns3::LteSpectrumPhy::DataErrorModelEnabled", BooleanValue(false));
```

By using the above code, we disable the error model in both control and data channels and in both directions (downlink and uplink). This is necessary because handover-related signaling messages are transmitted using these channels. An exception is when the simulation uses the ideal RRC protocol. In this case, only the Random Access procedure is left to be considered. The procedure consists of control messages, therefore we only need to disable the control channel's error model.

Handover traces

The RRC model, in particular the `LteEnbRrc` and `LteUeRrc` objects, provide some useful traces which can be hooked up to some custom functions so that they are called upon start and end of the handover execution phase at both the UE and eNB side. As an example, in your simulation program you can declare the following methods:

```
void
NotifyHandoverStartUe(std::string context,
                      uint64_t imsi,
                      uint16_t cellId,
                      uint16_t rnti,
                      uint16_t targetCellId)
{
    std::cout << Simulator::Now().GetSeconds() << " " << context
               << " UE IMSI " << imsi
               << ": previously connected to CellId " << cellId
               << " with RNTI " << rnti
               << ", doing handover to CellId " << targetCellId
               << std::endl;
}

void
NotifyHandoverEndOkUe(std::string context,
                      uint64_t imsi,
                      uint16_t cellId,
                      uint16_t rnti)
{
    std::cout << Simulator::Now().GetSeconds() << " " << context
               << " UE IMSI " << imsi
               << ": successful handover to CellId " << cellId
               << " with RNTI " << rnti
               << std::endl;
}

void
NotifyHandoverStartEnb(std::string context,
                       uint64_t imsi,
                       uint16_t cellId,
                       uint16_t rnti,
                       uint16_t targetCellId)
{
    std::cout << Simulator::Now().GetSeconds() << " " << context
               << " eNB CellId " << cellId
               << ": start handover of UE with IMSI " << imsi
               << " RNTI " << rnti
               << " to CellId " << targetCellId
               << std::endl;
}

void
NotifyHandoverEndOkEnb(std::string context,
                       uint64_t imsi,
                       uint16_t cellId,
```

(continues on next page)

(continued from previous page)

```

        uint16_t rnti)
{
    std::cout << Simulator::Now().GetSeconds() << " " << context
              << " eNB CellId " << cellId
              << ": completed handover of UE with IMSI " << imsi
              << " RNTI " << rnti
              << std::endl;
}

```

Then, you can hook up these methods to the corresponding trace sources like this:

```

Config::Connect("/NodeList/*/DeviceList/*/LteEnbRrc/HandoverStart",
               MakeCallback(&NotifyHandoverStartEnb));
Config::Connect("/NodeList/*/DeviceList/*/LteUeRrc/HandoverStart",
               MakeCallback(&NotifyHandoverStartUe));
Config::Connect("/NodeList/*/DeviceList/*/LteEnbRrc/HandoverEndOk",
               MakeCallback(&NotifyHandoverEndOkEnb));
Config::Connect("/NodeList/*/DeviceList/*/LteUeRrc/HandoverEndOk",
               MakeCallback(&NotifyHandoverEndOkUe));

```

Handover failure events can also be traced by trace sink functions with a similar signature as above (including IMSI, cell ID, and RNTI). Four different failure events are traced:

1. HandoverFailureNoPreamble: Handover failure due to non allocation of non-contention-based preamble at eNB
2. HandoverFailureMaxRach: Handover failure due to maximum RACH attempts
3. HandoverFailureLeaving: Handover leaving timeout at source eNB
4. HandoverFailureJoining: Handover joining timeout at target eNB

Similarly, one can hook up methods to the corresponding trace sources like this:

```

Config::Connect("/NodeList/*/DeviceList/*/LteEnbRrc/HandoverFailureNoPreamble",
               MakeCallback(&NotifyHandoverFailureNoPreamble));
Config::Connect("/NodeList/*/DeviceList/*/LteEnbRrc/HandoverFailureMaxRach",
               MakeCallback(&NotifyHandoverFailureMaxRach));
Config::Connect("/NodeList/*/DeviceList/*/LteEnbRrc/HandoverFailureLeaving",
               MakeCallback(&NotifyHandoverFailureLeaving));
Config::Connect("/NodeList/*/DeviceList/*/LteEnbRrc/HandoverFailureJoining",
               MakeCallback(&NotifyHandoverFailureJoining));

```

The example program `src/lte/examples/lena-x2-handover.cc` illustrates how the above instructions can be integrated in a simulation program. You can run the program like this:

```
./ns3 run lena-x2-handover
```

and it will output the messages printed by the custom handover trace hooks. In order to additionally print out some meaningful logging information, you can run the program like this:

```
NS_LOG=LteEnbRrc:LteUeRrc:EpcX2 ./ns3 run lena-x2-handover
```

19.2.20 Frequency Reuse Algorithms

In this section we will describe how to use Frequency Reuse Algorithms in eNb within LTE simulations. There are two possible ways of configuration. The first approach is the “manual” one, it requires more parameters to be configured, but allow user to configure FR algorithm as he/she needs. The second approach is more “automatic”. It is very convenient, because is the same for each FR algorithm, so user can switch FR algorithm very quickly by changing only type of FR algorithm. One drawback is that “automatic” approach uses only limited set of configurations for each algorithm, what make it less flexible, but is sufficient for most of cases.

These two approaches will be described more in following sub-section.

If user do not configure Frequency Reuse algorithm, default one (i.e. `LteFrNoOpAlgorithm`) is installed in eNb. It acts as if FR algorithm was disabled.

One thing that should be mentioned is that most of implemented FR algorithms work with cell bandwidth greater or equal than 15 RBs. This limitation is caused by requirement that at least three continuous RBs have to be assigned to UE for transmission.

Manual configuration

Frequency reuse algorithm can be configured “manually” within the simulation program by setting type of FR algorithm and all its attributes. Currently, seven FR algorithms are implemented:

- `ns3::LteFrNoOpAlgorithm`
- `ns3::LteFrHardAlgorithm`
- `ns3::LteFrStrictAlgorithm`
- `ns3::LteFrSoftAlgorithm`
- `ns3::LteFfrSoftAlgorithm`
- `ns3::LteFfrEnhancedAlgorithm`
- `ns3::LteFfrDistributedAlgorithm`

Selecting a FR algorithm is done via the `LteHelper` object and its `SetFfrAlgorithmType` method as shown below:

```
Ptr<LteHelper> lteHelper = CreateObject<LteHelper>();
lteHelper->SetFfrAlgorithmType("ns3::LteFrHardAlgorithm");
```

Each implemented FR algorithm provide several configurable attributes. Users do not have to care about UL and DL bandwidth configuration, because it is done automatically during cell configuration. To change bandwidth for FR algorithm, configure required values for `LteEnbNetDevice`:

```
uint8_t bandwidth = 100;
lteHelper->SetEnbDeviceAttribute("DLBandwidth", UintegerValue(bandwidth));
lteHelper->SetEnbDeviceAttribute("ULBandwidth", UintegerValue(bandwidth));
```

Now, each FR algorithms configuration will be described.

Hard Frequency Reuse Algorithm

As described in Section *Hard Frequency Reuse* of the Design Documentation ns3::LteFrHardAlgorithm uses one sub-band. To configure this sub-band user need to specify offset and bandwidth for DL and UL in number of RBs.

Hard Frequency Reuse Algorithm provides following attributes:

- DlSubBandOffset: Downlink Offset in number of Resource Block Groups
- DlSubBandwidth: Downlink Transmission SubBandwidth Configuration in number of Resource Block Groups
- UlSubBandOffset: Uplink Offset in number of Resource Block Groups
- UlSubBandwidth: Uplink Transmission SubBandwidth Configuration in number of Resource Block Groups

Example configuration of LteFrHardAlgorithm can be done in following way:

```
lteHelper->SetFfrAlgorithmType("ns3::LteFrHardAlgorithm");
lteHelper->SetFfrAlgorithmAttribute("DlSubBandOffset", UintegerValue(8));
lteHelper->SetFfrAlgorithmAttribute("DlSubBandwidth", UintegerValue(8));
lteHelper->SetFfrAlgorithmAttribute("UlSubBandOffset", UintegerValue(8));
lteHelper->SetFfrAlgorithmAttribute("UlSubBandwidth", UintegerValue(8));
NetDeviceContainer enbDevs = lteHelper->InstallEnbDevice(enbNodes.Get(0));
```

Above example allow eNB to use only RBs from 8 to 16 in DL and UL, while entire cell bandwidth is 25.

Strict Frequency Reuse Algorithm

Strict Frequency Reuse Algorithm uses two sub-bands: one common for each cell and one private. There is also RSRQ threshold, which is needed to decide within which sub-band UE should be served. Moreover the power transmission in these sub-bands can be different.

Strict Frequency Reuse Algorithm provides following attributes:

- UlCommonSubBandwidth: Uplink Common SubBandwidth Configuration in number of Resource Block Groups
- UlEdgeSubBandOffset: Uplink Edge SubBand Offset in number of Resource Block Groups
- UlEdgeSubBandwidth: Uplink Edge SubBandwidth Configuration in number of Resource Block Groups
- DlCommonSubBandwidth: Downlink Common SubBandwidth Configuration in number of Resource Block Groups
- DlEdgeSubBandOffset: Downlink Edge SubBand Offset in number of Resource Block Groups
- DlEdgeSubBandwidth: Downlink Edge SubBandwidth Configuration in number of Resource Block Groups
- RsrqThreshold: If the RSRQ of is worse than this threshold, UE should be served in edge sub-band
- CenterPowerOffset: PdschConfigDedicated::Pa value for center sub-band, default value dB0
- EdgePowerOffset: PdschConfigDedicated::Pa value for edge sub-band, default value dB0
- CenterAreaTpc: TPC value which will be set in DL-DCI for UEs in center area, Absolute mode is used, default value 1 is mapped to -1 according to TS36.213 Table 5.1.1.1-2
- EdgeAreaTpc: TPC value which will be set in DL-DCI for UEs in edge area, Absolute mode is used, default value 1 is mapped to -1 according to TS36.213 Table 5.1.1.1-2

Example below allow eNB to use RBs from 0 to 6 as common sub-band and from 12 to 18 as private sub-band in DL and UL, RSRQ threshold is 20 dB, power in center area equals `LteEnbPhy::TxPower - 3dB`, power in edge area equals `LteEnbPhy::TxPower + 3dB`:

```

lteHelper->SetFfrAlgorithmType("ns3::LteFrStrictAlgorithm");
lteHelper->SetFfrAlgorithmAttribute("DlCommonSubBandwidth", UIntegerValue(6));
lteHelper->SetFfrAlgorithmAttribute("UlCommonSubBandwidth", UIntegerValue(6));
lteHelper->SetFfrAlgorithmAttribute("DlEdgeSubBandOffset", UIntegerValue(6));
lteHelper->SetFfrAlgorithmAttribute("DlEdgeSubBandwidth", UIntegerValue(6));
lteHelper->SetFfrAlgorithmAttribute("UlEdgeSubBandOffset", UIntegerValue(6));
lteHelper->SetFfrAlgorithmAttribute("UlEdgeSubBandwidth", UIntegerValue(6));
lteHelper->SetFfrAlgorithmAttribute("RsrqThreshold", UIntegerValue(20));
lteHelper->SetFfrAlgorithmAttribute("CenterPowerOffset",
                                   UIntegerValue(LteRrcSap::PdschConfigDedicated::dB_
↪3));
lteHelper->SetFfrAlgorithmAttribute("EdgePowerOffset",
                                   UIntegerValue(LteRrcSap::PdschConfigDedicated::dB3));
lteHelper->SetFfrAlgorithmAttribute("CenterAreaTpc", UIntegerValue(1));
lteHelper->SetFfrAlgorithmAttribute("EdgeAreaTpc", UIntegerValue(2));
NetDeviceContainer enbDevs = lteHelper->InstallEnbDevice(enbNodes.Get(0));

```

Soft Frequency Reuse Algorithm

With Soft Frequency Reuse Algorithm, eNb uses entire cell bandwidth, but there are two sub-bands, within UEs are served with different power level.

Soft Frequency Reuse Algorithm provides following attributes:

- **UlEdgeSubBandOffset**: Uplink Edge SubBand Offset in number of Resource Block Groups
- **UlEdgeSubBandwidth**: Uplink Edge SubBandwidth Configuration in number of Resource Block Groups
- **DlEdgeSubBandOffset**: Downlink Edge SubBand Offset in number of Resource Block Groups
- **DlEdgeSubBandwidth**: Downlink Edge SubBandwidth Configuration in number of Resource Block Groups
- **AllowCenterUeUseEdgeSubBand**: If true center UEs can receive on edge sub-band RBGs, otherwise edge sub-band is allowed only for edge UEs, default value is true
- **RsrqThreshold**: If the RSRQ of is worse than this threshold, UE should be served in edge sub-band
- **CenterPowerOffset**: PdschConfigDedicated::Pa value for center sub-band, default value dB0
- **EdgePowerOffset**: PdschConfigDedicated::Pa value for edge sub-band, default value dB0
- **CenterAreaTpc**: TPC value which will be set in DL-DCI for UEs in center area, Absolute mode is used, default value 1 is mapped to -1 according to TS36.213 Table 5.1.1.1-2
- **EdgeAreaTpc**: TPC value which will be set in DL-DCI for UEs in edge area, Absolute mode is used, default value 1 is mapped to -1 according to TS36.213 Table 5.1.1.1-2

Example below configures RBs from 8 to 16 to be used by cell edge UEs and this sub-band is not available for cell center users. RSRQ threshold is 20 dB, power in center area equals `LteEnbPhy::TxPower`, power in edge area equals `LteEnbPhy::TxPower + 3dB`:

```

lteHelper->SetFfrAlgorithmType("ns3::LteFrSoftAlgorithm");
lteHelper->SetFfrAlgorithmAttribute("DlEdgeSubBandOffset", UIntegerValue(8));
lteHelper->SetFfrAlgorithmAttribute("DlEdgeSubBandwidth", UIntegerValue(8));
lteHelper->SetFfrAlgorithmAttribute("UlEdgeSubBandOffset", UIntegerValue(8));
lteHelper->SetFfrAlgorithmAttribute("UlEdgeSubBandwidth", UIntegerValue(8));
lteHelper->SetFfrAlgorithmAttribute("AllowCenterUeUseEdgeSubBand", BooleanValue(false));

```

(continues on next page)

(continued from previous page)

```

lteHelper->SetFfrAlgorithmAttribute("RsrqThreshold", UIntegerValue(20));
lteHelper->SetFfrAlgorithmAttribute("CenterPowerOffset",
                                   UIntegerValue(LteRrcSap::PdschConfigDedicated::dB0));
lteHelper->SetFfrAlgorithmAttribute("EdgePowerOffset",
                                   UIntegerValue(LteRrcSap::PdschConfigDedicated::dB3));
NetDeviceContainer enbDevs = lteHelper->InstallEnbDevice(enbNodes.Get(0));

```

Soft Fractional Frequency Reuse Algorithm

Soft Fractional Frequency Reuse (SFFR) uses three sub-bands: center, medium (common) and edge. User have to configure only two of them: common and edge. Center sub-band will be composed from the remaining bandwidth. Each sub-band can be served with different transmission power. Since there are three sub-bands, two RSRQ thresholds needs to be configured.

Soft Fractional Frequency Reuse Algorithm provides following attributes:

- **UllCommonSubBandwidth**: Uplink Common SubBandwidth Configuration in number of Resource Block Groups
- **UllEdgeSubBandOffset**: Uplink Edge SubBand Offset in number of Resource Block Groups
- **UllEdgeSubBandwidth**: Uplink Edge SubBandwidth Configuration in number of Resource Block Groups
- **DlCommonSubBandwidth**: Downlink Common SubBandwidth Configuration in number of Resource Block Groups
- **DlEdgeSubBandOffset**: Downlink Edge SubBand Offset in number of Resource Block Groups
- **DlEdgeSubBandwidth**: Downlink Edge SubBandwidth Configuration in number of Resource Block Groups
- **CenterRsrqThreshold**: If the RSRQ of is worse than this threshold, UE should be served in medium sub-band
- **EdgeRsrqThreshold**: If the RSRQ of is worse than this threshold, UE should be served in edge sub-band
- **CenterAreaPowerOffset**: PdschConfigDedicated::Pa value for center sub-band, default value dB0
- **MediumAreaPowerOffset**: PdschConfigDedicated::Pa value for medium sub-band, default value dB0
- **EdgeAreaPowerOffset**: PdschConfigDedicated::Pa value for edge sub-band, default value dB0
- **CenterAreaTpc**: TPC value which will be set in DL-DCI for UEs in center area, Absolute mode is used, default value 1 is mapped to -1 according to TS36.213 Table 5.1.1.1-2
- **MediumAreaTpc**: TPC value which will be set in DL-DCI for UEs in medium area, Absolute mode is used, default value 1 is mapped to -1 according to TS36.213 Table 5.1.1.1-2
- **EdgeAreaTpc**: TPC value which will be set in DL-DCI for UEs in edge area, Absolute mode is used, default value 1 is mapped to -1 according to TS36.213 Table 5.1.1.1-2

In example below RBs from 0 to 6 will be used as common (medium) sub-band, RBs from 6 to 12 will be used as edge sub-band and RBs from 12 to 24 will be used as center sub-band (it is composed with remaining RBs). RSRQ threshold between center and medium area is 28 dB, RSRQ threshold between medium and edge area is 18 dB. Power in center area equals $\text{LteEnbPhy::TxPower} - 3\text{dB}$, power in medium area equals $\text{LteEnbPhy::TxPower} + 3\text{dB}$, power in edge area equals $\text{LteEnbPhy::TxPower} + 3\text{dB}$:

```

lteHelper->SetFfrAlgorithmType("ns3::LteFfrSoftAlgorithm");
lteHelper->SetFfrAlgorithmAttribute("UllCommonSubBandwidth", UIntegerValue(6));
lteHelper->SetFfrAlgorithmAttribute("DlCommonSubBandwidth", UIntegerValue(6));
lteHelper->SetFfrAlgorithmAttribute("DlEdgeSubBandOffset", UIntegerValue(0));
lteHelper->SetFfrAlgorithmAttribute("DlEdgeSubBandwidth", UIntegerValue(6));
lteHelper->SetFfrAlgorithmAttribute("UllEdgeSubBandOffset", UIntegerValue(0));

```

(continues on next page)

(continued from previous page)

```

lteHelper->SetFfrAlgorithmAttribute("ULEdgeSubBandwidth", IntegerValue(6));
lteHelper->SetFfrAlgorithmAttribute("CenterRsrqThreshold", IntegerValue(28));
lteHelper->SetFfrAlgorithmAttribute("EdgeRsrqThreshold", IntegerValue(18));
lteHelper->SetFfrAlgorithmAttribute("CenterAreaPowerOffset",
    IntegerValue(LteRrcSap::PdschConfigDedicated::dB_
→3));
lteHelper->SetFfrAlgorithmAttribute("MediumAreaPowerOffset",
    IntegerValue(LteRrcSap::PdschConfigDedicated::dB0));
lteHelper->SetFfrAlgorithmAttribute("EdgeAreaPowerOffset",
    IntegerValue(LteRrcSap::PdschConfigDedicated::dB3));
NetDeviceContainer enbDevs = lteHelper->InstallEnbDevice(enbNodes.Get(0));

```

Enhanced Fractional Frequency Reuse Algorithm

Enhanced Fractional Frequency Reuse (EFFF) reserve part of system bandwidth for each cell (typically there are 3 cell types and each one gets 1/3 of system bandwidth). Then part of this subbandwidth it used as *Primary Segment* with reuse factor 3 and as *Secondary Segment* with reuse factor 1. User has to configure (for DL and UL) offset of the cell subbandwidth in number of RB, number of RB which will be used as *Primary Segment* and number of RB which will be used as *Secondary Segment*. *Primary Segment* is used by cell at will, but RBs from *Secondary Segment* can be assigned to UE only if CQI feedback from this UE have higher value than configured CQI threshold. UE is considered as edge UE when its RSRQ is lower than RsrqThreshold.

Since each eNb needs to know where are Primary and Secondary of other cell types, it will calculate them assuming configuration is the same for each cell and only subbandwidth offsets are different. So it is important to divide available system bandwidth equally to each cell and apply the same configuration of Primary and Secondary Segments to them.

Enhanced Fractional Frequency Reuse Algorithm provides following attributes:

- **ULSubBandOffset**: Uplink SubBand Offset for this cell in number of Resource Block Groups
- **ULReuse3SubBandwidth**: Uplink Reuse 3 SubBandwidth Configuration in number of Resource Block Groups
- **ULReuse1SubBandwidth**: Uplink Reuse 1 SubBandwidth Configuration in number of Resource Block Groups
- **DLSubBandOffset**: Downlink SubBand Offset for this cell in number of Resource Block Groups
- **DLReuse3SubBandwidth**: Downlink Reuse 3 SubBandwidth Configuration in number of Resource Block Groups
- **DLReuse1SubBandwidth**: Downlink Reuse 1 SubBandwidth Configuration in number of Resource Block Groups
- **RsrqThreshold**: If the RSRQ of is worse than this threshold, UE should be served in edge sub-band
- **CenterAreaPowerOffset**: PdschConfigDedicated::Pa value for center sub-band, default value dB0
- **EdgeAreaPowerOffset**: PdschConfigDedicated::Pa value for edge sub-band, default value dB0
- **DLCqiThreshold**: If the DL-CQI for RBG of is higher than this threshold, transmission on RBG is possible
- **ULCqiThreshold**: If the UL-CQI for RBG of is higher than this threshold, transmission on RBG is possible
- **CenterAreaTpc**: TPC value which will be set in DL-DCI for UEs in center area, Absolute mode is used, default value 1 is mapped to -1 according to TS36.213 Table 5.1.1.1-2
- **EdgeAreaTpc**: TPC value which will be set in DL-DCI for UEs in edge area, Absolute mode is used, default value 1 is mapped to -1 according to TS36.213 Table 5.1.1.1-2

In example below offset in DL and UL is 0 RB, 4 RB will be used in *Primary Segment* and *Secondary Segment*. RSRQ threshold between center and edge area is 25 dB. DL and UL CQI thresholds are set to value of 10. Power in center area equals $\text{LteEnbPhy::TxPower} - 6\text{dB}$, power in edge area equals $\text{LteEnbPhy::TxPower} + 0\text{dB}$:

```

lteHelper->SetFfrAlgorithmType("ns3::LteFfrEnhancedAlgorithm");
lteHelper->SetFfrAlgorithmAttribute("RsrqThreshold", IntegerValue(25));
lteHelper->SetFfrAlgorithmAttribute("DlCqiThreshold", IntegerValue(10));
lteHelper->SetFfrAlgorithmAttribute("UlCqiThreshold", IntegerValue(10));
lteHelper->SetFfrAlgorithmAttribute("CenterAreaPowerOffset",
                                   IntegerValue(LteRrcSap::PdschConfigDedicated::dB_
↪6));
lteHelper->SetFfrAlgorithmAttribute("EdgeAreaPowerOffset",
                                   IntegerValue(LteRrcSap::PdschConfigDedicated::dB0));
lteHelper->SetFfrAlgorithmAttribute("UlSubBandOffset", IntegerValue(0));
lteHelper->SetFfrAlgorithmAttribute("UlReuse3SubBandwidth", IntegerValue(4));
lteHelper->SetFfrAlgorithmAttribute("UlReuse1SubBandwidth", IntegerValue(4));
lteHelper->SetFfrAlgorithmAttribute("DlSubBandOffset", IntegerValue(0));
lteHelper->SetFfrAlgorithmAttribute("DlReuse3SubBandwidth", IntegerValue(4));
lteHelper->SetFfrAlgorithmAttribute("DlReuse1SubBandwidth", IntegerValue(4));

```

Distributed Fractional Frequency Reuse Algorithm

Distributed Fractional Frequency Reuse requires X2 interface between all eNB to be installed. X2 interfaces can be installed only when EPC is configured, so this FFR scheme can be used only with EPC scenarios.

With Distributed Fractional Frequency Reuse Algorithm, eNB uses entire cell bandwidth and there can be two sub-bands: center sub-band and edge sub-band. Within these sub-bands UEs can be served with different power level. Algorithm adaptively selects RBs for cell-edge sub-band on basis of coordination information (i.e. RNTP) from adjacent cells and notifies the base stations of the adjacent cells, which RBs it selected to use in edge sub-band. If there are no UE classified as edge UE in cell, eNB will not use any RBs as edge sub-band.

Distributed Fractional Frequency Reuse Algorithm provides following attributes:

- **CalculationInterval**: Time interval between calculation of Edge sub-band, Default value 1 second
- **RsrqThreshold**: If the RSRQ of is worse than this threshold, UE should be served in edge sub-band
- **RsrpDifferenceThreshold**: If the difference between the power of the signal received by UE from the serving cell and the power of the signal received from the adjacent cell is less than a RsrpDifferenceThreshold value, the cell weight is incremented
- **CenterPowerOffset**: PdschConfigDedicated::Pa value for edge sub-band, default value dB0
- **EdgePowerOffset**: PdschConfigDedicated::Pa value for edge sub-band, default value dB0
- **EdgeRbNum**: Number of RB that can be used in edge sub-band
- **CenterAreaTpc**: TPC value which will be set in DL-DCI for UEs in center area, Absolute mode is used, default value 1 is mapped to -1 according to TS36.213 Table 5.1.1.1-2
- **EdgeAreaTpc**: TPC value which will be set in DL-DCI for UEs in edge area, Absolute mode is used, default value 1 is mapped to -1 according to TS36.213 Table 5.1.1.1-2

In example below calculation interval is 500 ms. RSRQ threshold between center and edge area is 25. RSRP Difference Threshold is set to be 5. In DL and UL 6 RB will be used by each cell in edge sub-band. Power in center area equals $\text{LteEnbPhy::TxPower} - 0\text{dB}$, power in edge area equals $\text{LteEnbPhy::TxPower} + 3\text{dB}$:

```

lteHelper->SetFfrAlgorithmType("ns3::LteFfrDistributedAlgorithm");
lteHelper->SetFfrAlgorithmAttribute("CalculationInterval", TimeValue(MilliSeconds(500)));
lteHelper->SetFfrAlgorithmAttribute("RsrqThreshold", IntegerValue(25));
lteHelper->SetFfrAlgorithmAttribute("RsrpDifferenceThreshold", IntegerValue(5));

```

(continues on next page)

(continued from previous page)

```

lteHelper->SetFfrAlgorithmAttribute("EdgeRbNum", UIntegerValue(6));
lteHelper->SetFfrAlgorithmAttribute("CenterPowerOffset",
    ↪ UIntegerValue(LteRrcSap::PdschConfigDedicated::dB0));
lteHelper->SetFfrAlgorithmAttribute("EdgePowerOffset",
    ↪ UIntegerValue(LteRrcSap::PdschConfigDedicated::dB3));

```

Automatic configuration

Frequency Reuse algorithms can also be configured in more “automatic” way by setting only the bandwidth and FrCellTypeId. During initialization of FR instance, configuration for set bandwidth and FrCellTypeId will be taken from configuration table. It is important that only sub-bands will be configured, thresholds and transmission power will be set to default values. If one wants, he/she can change thresholds and transmission power as show in previous sub-section.

There are three FrCellTypeId : 1, 2, 3, which correspond to three different configurations for each bandwidth. Three configurations allow to have different configurations in neighbouring cells in hexagonal eNB layout. If user needs to have more different configuration for neighbouring cells, he/she need to use manual configuration.

Example below show automatic FR algorithm configuration:

```

lteHelper->SetFfrAlgorithmType("ns3::LteFfrSoftAlgorithm");
lteHelper->SetFfrAlgorithmAttribute("FrCellTypeId", UIntegerValue(1));
NetDeviceContainer enbDevs = lteHelper->InstallEnbDevice(enbNodes.Get(0));

```

19.2.21 Uplink Power Control

Uplink Power Control functionality is enabled by default. User can disable it by setting the boolean attribute ns3::LteUePhy::EnableUplinkPowerControl to true.

User can switch between Open Loop Power Control and Closed Loop Power Control mechanisms by setting the boolean attribute ns3::LteUePowerControl::ClosedLoop. By default Closed Loop Power Control with Accumulation Mode is enabled.

Path-loss is key component of Uplink Power Control. It is computed as difference between filtered RSRP and ReferenceSignalPower parameter. ReferenceSignalPower is sent with SIB2.

Attributes available in Uplink Power Control:

- ClosedLoop: if true Closed Loop Uplink Power Control mode is enabled and Open Loop Power Control otherwise, default value is false
- AccumulationEnabled: if true Accumulation Mode is enabled and Absolute mode otherwise, default value is false
- Alpha: the path loss compensation factor, default value is 1.0
- Pcmín: minimal UE TxPower, default value is -40 dBm
- Pcmáx: maximal UE TxPower, default value is 23 dBm
- PoNominalPusch: this parameter should be set by higher layers, but currently it needs to be configured by attribute system, possible values are integers in range (-126 ... 24), Default value is -80
- PoUePusch: this parameter should be set by higher layers, but currently it needs to be configured by attribute system, possible values are integers in range (-8 ... 7), Default value is 0

- `PsrsOffset`: this parameter should be set by higher layers, but currently it needs to be configured by attribute system, possible values are integers in range (0 ... 15), Default value is 7, what gives `P_Srs_Offset_Value = 0`

Traced values in Uplink Power Control:

- `ReportPuschTxPower`: Current UE TxPower for PUSCH
- `ReportPucchTxPower`: Current UE TxPower for PUCCH
- `ReportSrsTxPower`: Current UE TxPower for SRS

Example configuration is presented below:

```
Config::SetDefault("ns3::LteUePhy::EnableUplinkPowerControl", BooleanValue(true));
Config::SetDefault("ns3::LteEnbPhy::TxPower", DoubleValue(30));
Config::SetDefault("ns3::LteUePowerControl::ClosedLoop", BooleanValue(true));
Config::SetDefault("ns3::LteUePowerControl::AccumulationEnabled", BooleanValue(true));
```

As an example, user can take a look and run the `lena-uplink-power-control` program.

19.2.22 Examples Programs

The directory `src/lte/examples/` contains some example simulation programs that show how to simulate different LTE scenarios.

19.2.23 Reference scenarios

There is a vast amount of reference LTE simulation scenarios which can be found in the literature. Here we list some of them:

- The system simulation scenarios mentioned in section A.2 of [TR36814].
- The dual stripe model [R4-092042], which is partially implemented in the example program `src/lte/examples/lena-dual-stripe.cc`. This example program features a lot of configurable parameters which can be customized by changing the corresponding global variables. To get a list of all these global variables, you can run this command:

```
./ns3 run lena-dual-stripe --command-template="%s --PrintGlobals"
```

The following subsection presents an example of running a simulation campaign using this example program.

Handover simulation campaign

In this subsection, we will demonstrate an example of running a simulation campaign using the LTE module of *ns-3*. The objective of the campaign is to compare the effect of each built-in handover algorithm of the LTE module.

The campaign will use the `lena-dual-stripe` example program. First, we have to modify the example program to produce the output that we need. In this occasion, we want to produce the number of handovers, user average throughput, and average SINR.

The number of handovers can be obtained by counting the number of times the *HandoverEndOk* *Handover traces* is fired. Then the user average throughput can be obtained by enabling the RLC *Simulation Output*. Finally, SINR can be obtained by enabling the PHY simulation output. The following sample code snippet shows one possible way to obtain the above:

```

void
NotifyHandoverEndOkUe(std::string context, uint64_t imsi,
                      uint16_t cellId, uint16_t rnti)
{
    std::cout << "Handover IMSI " << imsi << std::endl;
}

int
main(int argc, char *argv[])
{
    /*** SNIP ***/

    Config::Connect("/NodeList/*/DeviceList/*/LteUeRrc/HandoverEndOk",
                   MakeCallback(&NotifyHandoverEndOkUe));

    lteHelper->EnablePhyTraces();
    lteHelper->EnableRlcTraces();
    Ptr<RadioBearerStatsCalculator> rlcStats = lteHelper->GetRlcStats();
    rlcStats->SetAttribute("StartTime", TimeValue(Seconds(0)));
    rlcStats->SetAttribute("EpochDuration", TimeValue(Seconds(simTime)));

    Simulator::Run();
    Simulator::Destroy();
    return 0;
}

```

Then we have to configure the parameters of the program to suit our simulation needs. We are looking for the following assumptions in our simulation:

- 7 sites of tri-sector macro eNodeBs (i.e. 21 macrocells) deployed in hexagonal layout with 500 m inter-site distance.
- Although *lena-dual-stripe* is originally intended for a two-tier (macrocell and femtocell) simulation, we will simplify our simulation to one-tier (macrocell) simulation only.
- UEs are randomly distributed around the sites and attach to the network automatically using Idle mode cell selection. After that, UE will roam the simulation environment with 60 kmph movement speed.
- 50 seconds simulation duration, so UEs would have traveled far enough to trigger some handovers.
- 46 dBm macrocell Tx power and 10 dBm UE Tx power.
- EPC mode will be used because the X2 handover procedure requires it to be enabled.
- Full-buffer downlink and uplink traffic, both in 5 MHz bandwidth, using TCP protocol and Proportional Fair scheduler.
- Ideal RRC protocol.

Table *lena-dual-stripe parameter configuration for handover campaign* below shows how we configure the parameters of *lena-dual-stripe* to achieve the above assumptions.

Table 19.4: lena-dual-stripe parameter configuration for handover campaign

Parameter name	Value	Description
simTime	50	50 seconds simulation duration
nBlocks	0	Disabling apartment buildings and femtocells
nMacroEnbSites	7	Number of macrocell sites (each site has 3 cells)
nMacroEnbSitesX	2	The macrocell sites will be positioned in a 2-3-2 formation
interSiteDistance	500	500 m distance between adjacent macrocell sites
macroEnbTxPowerDbm	46	46 dBm Tx power for each macrocell
epc	1	Enable EPC mode
epcDl	1	Enable full-buffer DL traffic
epcUl	1	Enable full-buffer UL traffic
useUdp	0	Disable UDP traffic and enable TCP instead
macroUeDensity	0.00002	Determines number of UEs (translates to 48 UEs in our simulation)
outdoorUeMinSpeed	16.6667	Minimum UE movement speed in m/s (60 kmph)
outdoorUeMaxSpeed	16.6667	Maximum UE movement speed in m/s (60 kmph)
macroEnbBandwidth	25	5 MHz DL and UL bandwidth
generateRem	1	(Optional) For plotting the Radio Environment Map

Some of the required assumptions are not available as parameters of `lena-dual-stripe`. In this case, we override the default attributes, as shown in Table *Overriding default attributes for handover campaign* below.

Table 19.5: Overriding default attributes for handover campaign

Default value name	Value	Description
ns3::LteHelper::HandoverAlgorithm	ns3::NoOpHandoverAlgorithm, ns3::A3RsrpHandoverAlgorithm, ns3::A2A4RsrqHandoverAlgorithm or	Choice of handover algorithm
ns3::LteHelper::Scheduler	ns3::PffMacScheduler	Proportional Fair scheduler
ns3::LteHelper::UseIdealRrc	1	Ideal RRC protocol
ns3::RadioBearerStatsCalculator::DLRlcOutputFilename	no-op-DlRlcStats.txt	File name for DL RLC trace output
ns3::RadioBearerStatsCalculator::ULRlcOutputFilename	no-op-UlRlcStats.txt	File name for UL RLC trace output
ns3::PhyStatsCalculator::DLRsrpSinrFilename	no-op-DlRsrpSinrStats.txt	File name for DL PHY RSRP/SINR trace output
ns3::PhyStatsCalculator::ULSinrFilename	no-op-UlSinrStats.txt	File name for UL PHY SINR trace output

`ns-3` provides many ways for passing configuration values into a simulation. In this example, we will use the command line arguments. It is basically done by appending the parameters and their values to the `ns3` call when starting each individual simulation. So the `ns3` calls for invoking our 3 simulations would look as below:

```
$ ./ns3 run "lena-dual-stripe
--simTime=50 --nBlocks=0 --nMacroEnbSites=7 --nMacroEnbSitesX=2
--epc=1 --useUdp=0 --outdoorUeMinSpeed=16.6667 --outdoorUeMaxSpeed=16.6667
--ns3::LteHelper::HandoverAlgorithm=ns3::NoOpHandoverAlgorithm
--ns3::RadioBearerStatsCalculator::DLRlcOutputFilename=no-op-DlRlcStats.txt
--ns3::RadioBearerStatsCalculator::ULRlcOutputFilename=no-op-UlRlcStats.txt
--ns3::PhyStatsCalculator::DLRsrpSinrFilename=no-op-DlRsrpSinrStats.txt
--ns3::PhyStatsCalculator::ULSinrFilename=no-op-UlSinrStats.txt
```

(continues on next page)

(continued from previous page)

```

--RngRun=1" > no-op.txt

$ ./ns3 run "lena-dual-stripe
--simTime=50 --nBlocks=0 --nMacroEnbSites=7 --nMacroEnbSitesX=2
--epc=1 --useUdp=0 --outdoorUeMinSpeed=16.6667 --outdoorUeMaxSpeed=16.6667
--ns3::LteHelper::HandoverAlgorithm=ns3::A3RsrpHandoverAlgorithm
--ns3::RadioBearerStatsCalculator::DlRlcOutputFilename=a3-rsrp-DlRlcStats.txt
--ns3::RadioBearerStatsCalculator::UlRlcOutputFilename=a3-rsrp-UlRlcStats.txt
--ns3::PhyStatsCalculator::DlRsrpSinrFilename=a3-rsrp-DlRsrpSinrStats.txt
--ns3::PhyStatsCalculator::UlSinrFilename=a3-rsrp-UlSinrStats.txt
--RngRun=1" > a3-rsrp.txt

$ ./ns3 run "lena-dual-stripe
--simTime=50 --nBlocks=0 --nMacroEnbSites=7 --nMacroEnbSitesX=2
--epc=1 --useUdp=0 --outdoorUeMinSpeed=16.6667 --outdoorUeMaxSpeed=16.6667
--ns3::LteHelper::HandoverAlgorithm=ns3::A2A4RsrqHandoverAlgorithm
--ns3::RadioBearerStatsCalculator::DlRlcOutputFilename=a2-a4-rsrq-DlRlcStats.txt
--ns3::RadioBearerStatsCalculator::UlRlcOutputFilename=a2-a4-rsrq-UlRlcStats.txt
--ns3::PhyStatsCalculator::DlRsrpSinrFilename=a2-a4-rsrq-DlRsrpSinrStats.txt
--ns3::PhyStatsCalculator::UlSinrFilename=a2-a4-rsrq-UlSinrStats.txt
--RngRun=1" > a2-a4-rsrq.txt

```

Some notes on the execution:

- Notice that some arguments are not specified because they are already the same as the default values. We also keep the handover algorithms on each own default settings.
- Note the file names of simulation output, e.g. RLC traces and PHY traces, because we have to make sure that they are not overwritten by the next simulation run. In this example, we specify the names one by one using the command line arguments.
- The `--RngRun=1` argument at the end is used for setting the run number used by the random number generator used in the simulation. We re-run the same simulations with different *RngRun* values, hence creating several independent replications of the same simulations. Then we average the results obtained from these replications to achieve some statistical confidence.
- We can add a `--generateRem=1` argument to generate the files necessary for generating the Radio Environment Map (REM) of the simulation. The result is Figure *REM obtained from a simulation in handover campaign* below, which can be produced by following the steps described in Section *Radio Environment Maps*. This figure also shows the position of eNodeBs and UEs at the beginning of a simulation using *RngRun* = 1. Other values of *RngRun* may produce different UE position.

After hours of running, the simulation campaign will eventually end. Next we will perform some post-processing on the produced simulation output to obtain meaningful information out of it.

In this example, we use GNU Octave to assist the processing of throughput and SINR data, as demonstrated in a sample GNU Octave script below:

```

% RxBytes is the 10th column
DlRxBytes = load ("no-op-DlRlcStats.txt") (:,10);
DlAverageThroughputKbps = sum (DlRxBytes) * 8 / 1000 / 50

% RxBytes is the 10th column
UlRxBytes = load ("no-op-UlRlcStats.txt") (:,10);
UlAverageThroughputKbps = sum (UlRxBytes) * 8 / 1000 / 50

```

(continues on next page)

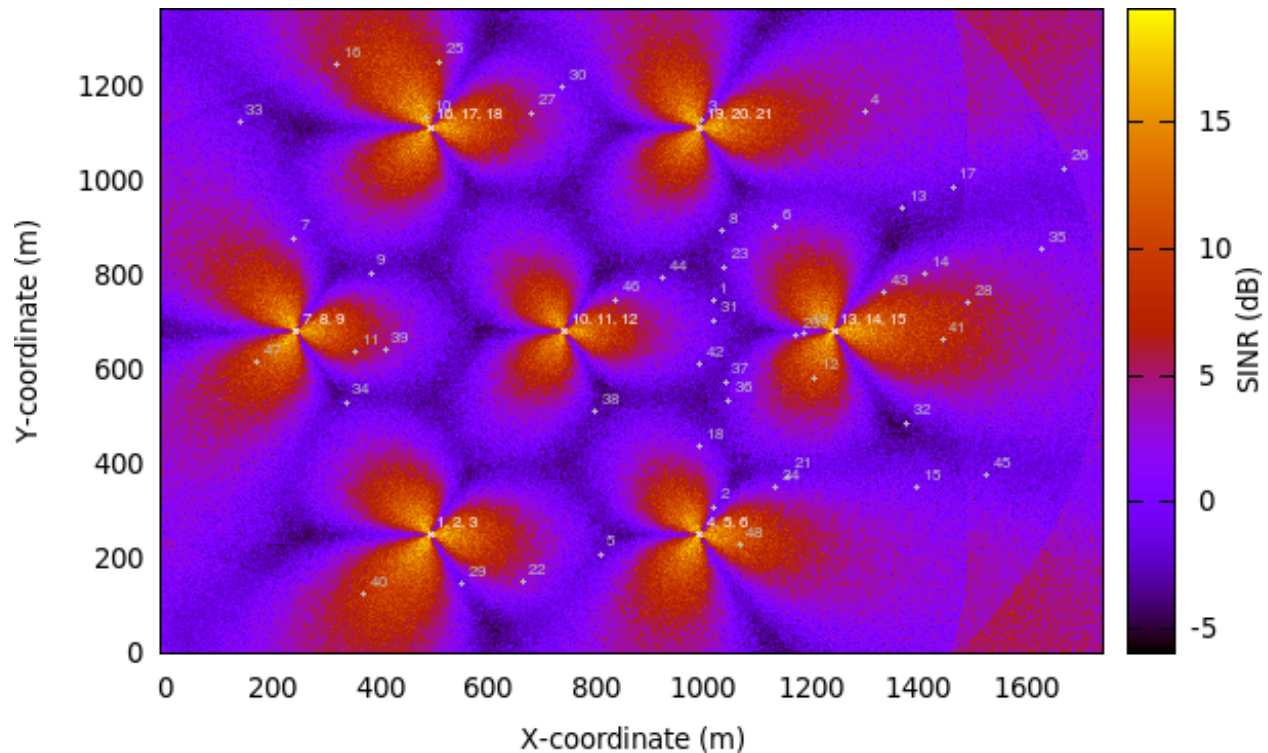


Fig. 19.75: REM obtained from a simulation in handover campaign

(continued from previous page)

```
% Sinr is the 6th column
DlSinr = load ("no-op-DlRsrpSinrStats.txt") (:,6);
% eliminate NaN values
idx = isnan (DlSinr);
DlSinr (idx) = 0;
DlAverageSinrDb = 10 * log10 (mean (DlSinr)) % convert to dB

% Sinr is the 5th column
UlSinr = load ("no-op-UlSinrStats.txt") (:,5);
% eliminate NaN values
idx = isnan (UlSinr);
UlSinr (idx) = 0;
UlAverageSinrDb = 10 * log10 (mean (UlSinr)) % convert to dB
```

As for the number of handovers, we can use simple shell scripting to count the number of occurrences of string “Handover” in the log file:

```
$ grep "Handover" no-op.txt | wc -l
```

Table [Results of handover campaign](#) below shows the complete statistics after we are done with post-processing on every individual simulation run. The values shown are the average of the results obtained from RngRun of 1, 2, 3, and 4.

Table 19.6: Results of handover campaign

Statistics	No-op	A2-A4-RSRQ	Strongest cell
Average DL system throughput	6 615 kbps	20 509 kbps	19 709 kbps
Average UL system throughput	4 095 kbps	5 705 kbps	6 627 kbps
Average DL SINR	-0.10 dB	5.19 dB	5.24 dB
Average UL SINR	9.54 dB	81.57 dB	79.65 dB
Number of handovers per UE per second	0	0.05694	0.04771

The results show that having a handover algorithm in a mobility simulation improves both user throughput and SINR significantly. There is little difference between the two handover algorithms in this campaign scenario. It would be interesting to see their performance in different scenarios, such as scenarios with home eNodeBs deployment.

Frequency Reuse examples

There are two examples showing Frequency Reuse Algorithms functionality.

lena-frequency-reuse is simple example with 3 eNBs in triangle layout. There are 3 cell edge UEs, which are located in the center of this triangle and 3 cell center UEs (one near each eNB). User can also specify the number of randomly located UEs. FR algorithm is installed in eNBs and each eNB has different FrCellTypeId, what means each eNB uses different FR configuration. User can run lena-frequency-reuse with 6 different FR algorithms: NoOp, Hard FR, Strict FR, Soft FR, Soft FFR and Enhanced FFR. To run scenario with Distributed FFR algorithm, user should use lena-distributed-ffr. These two examples are very similar, but they were split because Distributed FFR requires EPC to be used, and other algorithms do not.

To run lena-frequency-reuse with different Frequency Reuse algorithms, user needs to specify FR algorithm by overriding the default attribute ns3::LteHelper::FfrAlgorithm. Example command to run lena-frequency-reuse with Soft FR algorithm is presented below:

```
$ ./ns3 run "lena-frequency-reuse --ns3::LteHelper::FfrAlgorithm=ns3::LteFrSoftAlgorithm"
```

In these examples functionality to generate REM and spectrum analyzer trace was added. User can enable generation of it by setting generateRem and generateSpectrumTrace attributes.

Command to generate REM for RB 1 in data channel from lena-frequency-reuse scenario with Soft FR algorithm is presented below:

```
$ ./ns3 run "lena-frequency-reuse --ns3::LteHelper::FfrAlgorithm=ns3::LteFrSoftAlgorithm
--generateRem=true --remRbId=1"
```

Radio Environment Map for Soft FR is presented in Figure *REM for RB 1 obtained from lena-frequency-reuse example with Soft FR algorithm enabled*.

Command to generate spectrum trace from lena-frequency-reuse scenario with Soft FFR algorithm is presented below (Spectrum Analyzer position needs to be configured inside script):

```
$ ./ns3 run "lena-frequency-reuse --ns3::LteHelper::FfrAlgorithm=ns3::LteFfrSoftAlgorithm
--generateSpectrumTrace=true"
```

Example spectrum analyzer trace is presented in figure *Spectrum Analyzer trace obtained from lena-frequency-reuse example with Soft FFR algorithm enabled*. Spectrum Analyzer was located near eNB with FrCellTypeId 2.. As can be seen, different data channel subbands are sent with different power level (according to configuration), while control channel is transmitted with uniform power along entire system bandwidth.

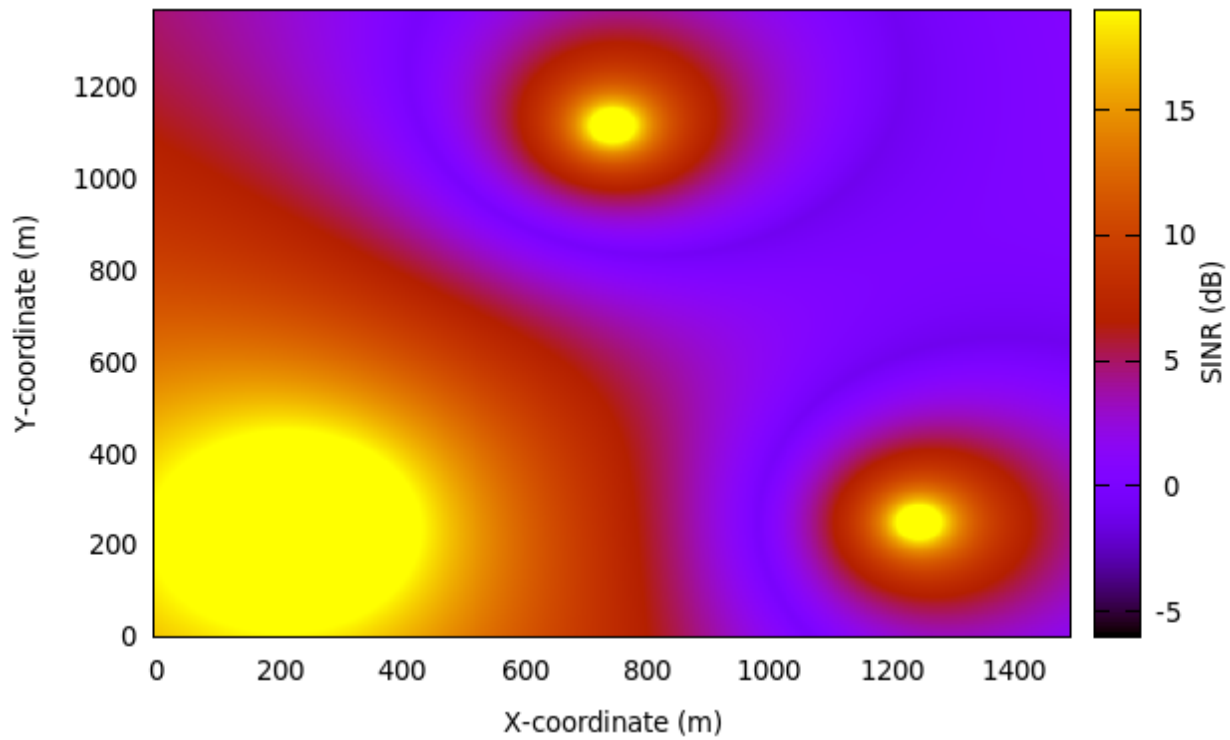


Fig. 19.76: REM for RB 1 obtained from lena-frequency-reuse example with Soft FR algorithm enabled

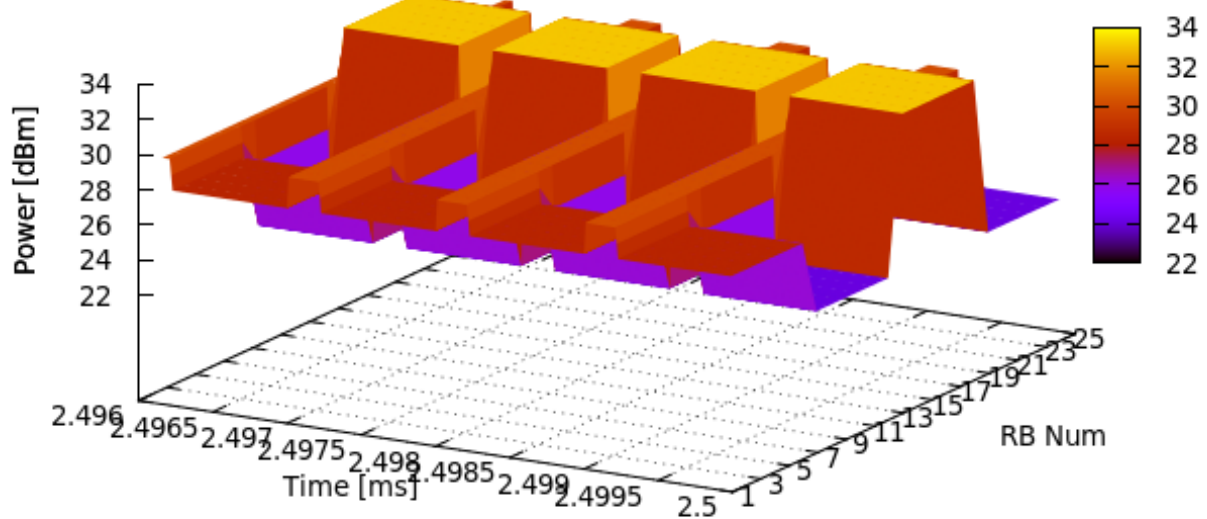


Fig. 19.77: Spectrum Analyzer trace obtained from lena-frequency-reuse example with Soft FFR algorithm enabled. Spectrum Analyzer was located near eNB with FrCellTypeId 2.

lena-dual-stripe can be also run with Frequency Reuse algorithms installed in all macro eNB. User needs to specify FR algorithm by overriding the default attribute `ns3::LteHelper::FfrAlgorithm`. Example command to run lena-dual-stripe with Hard FR algorithm is presented below:

```
$ ./ns3 run "lena-dual-stripe
--simTime=50 --nBlocks=0 --nMacroEnbSites=7 --nMacroEnbSitesX=2
--epc=1 --useUdp=0 --outdoorUeMinSpeed=16.6667 --outdoorUeMaxSpeed=16.6667
--ns3::LteHelper::HandoverAlgorithm=ns3::NoOpHandoverAlgorithm
--ns3::LteHelper::FfrAlgorithm=ns3::LteFrHardAlgorithm
--ns3::RadioBearerStatsCalculator::DlRlcOutputFilename=no-op-DlRlcStats.txt
--ns3::RadioBearerStatsCalculator::UlRlcOutputFilename=no-op-UlRlcStats.txt
--ns3::PhyStatsCalculator::DlRsrpSinrFilename=no-op-DlRsrpSinrStats.txt
--ns3::PhyStatsCalculator::UlSinrFilename=no-op-UlSinrStats.txt
--RngRun=1" > no-op.txt
```

Example command to generate REM for RB 1 in data channel from lena-dual-stripe scenario with Hard FR algorithm is presented below:

```
$ ./ns3 run "lena-dual-stripe
--simTime=50 --nBlocks=0 --nMacroEnbSites=7 --nMacroEnbSitesX=2
--epc=0 --useUdp=0 --outdoorUeMinSpeed=16.6667 --outdoorUeMaxSpeed=16.6667
--ns3::LteHelper::HandoverAlgorithm=ns3::NoOpHandoverAlgorithm
--ns3::LteHelper::FfrAlgorithm=ns3::LteFrHardAlgorithm
--ns3::RadioBearerStatsCalculator::DlRlcOutputFilename=no-op-DlRlcStats.txt
--ns3::RadioBearerStatsCalculator::UlRlcOutputFilename=no-op-UlRlcStats.txt
--ns3::PhyStatsCalculator::DlRsrpSinrFilename=no-op-DlRsrpSinrStats.txt
--ns3::PhyStatsCalculator::UlSinrFilename=no-op-UlSinrStats.txt
--RngRun=1 --generateRem=true --remRbId=1" > no-op.txt
```

Radio Environment Maps for RB 1, 10 and 20 generated from lena-dual-stripe scenario with Hard Frequency Reuse algorithm are presented in the figures below. These RB were selected because each one is used by different FR cell type.

19.2.24 Carrier aggregation examples

The carrier aggregation feature is not enabled by default. The user can enable it by setting the boolean attribute `ns3::LteHelper::UseCa` to true. The number of component carriers to be used in carrier aggregation can be configured by setting the attribute `ns3::LteHelper::NumberOfComponentCarriers`. Currently the maximum number is 5. Additionally, the component carrier manager needs to be configured. By default the `NoOpComponentCarrierManager` is selected, which means that only the primary carrier is enabled. The Component carrier manager (CCM) implementation that uses all the available carriers is `RrComponentCarrierManager`. The CCM can be configured by using the attribute `LteHelper::EnbComponentCarrierManager`.

An example configuration is presented below:

```
Config::SetDefault("ns3::LteHelper::UseCa", BooleanValue(useCa));
Config::SetDefault("ns3::LteHelper::NumberOfComponentCarriers", UIntegerValue(2));
Config::SetDefault("ns3::LteHelper::EnbComponentCarrierManager", StringValue(
↪ "ns3::RrComponentCarrierManager"));
```

As an example, the user can take a look and run the lena-simple and lena-simple-epc programs and enable LTE traces to check the performance. A new column is added to PHY and MAC traces to indicate the component carrier.

The test suite `lte-carrier-aggregation` is also a test program that can be used as an example as it can be run in a mode to write results to output files by setting the `s_writeResults` boolean static variable to true. The test can be run

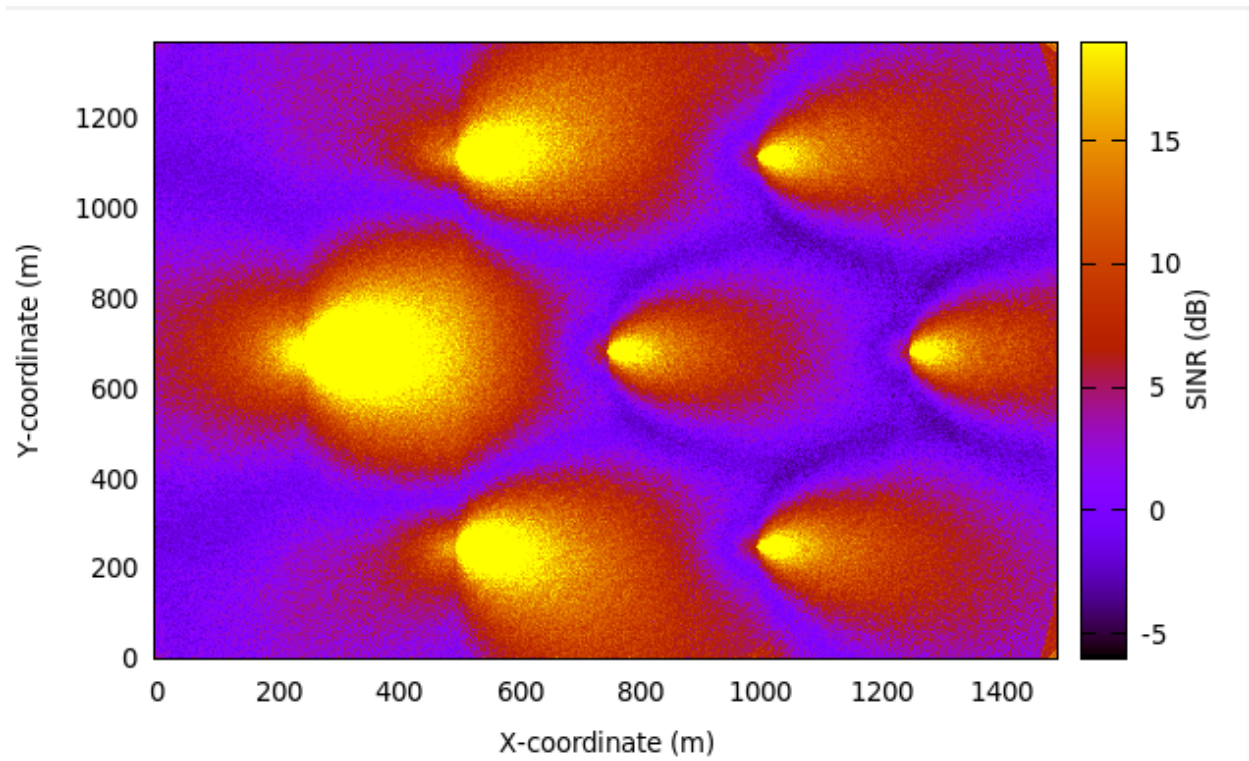


Fig. 19.78: REM for RB 1 obtained from lena-dual-stripe simulation with Hard FR algorithm enabled

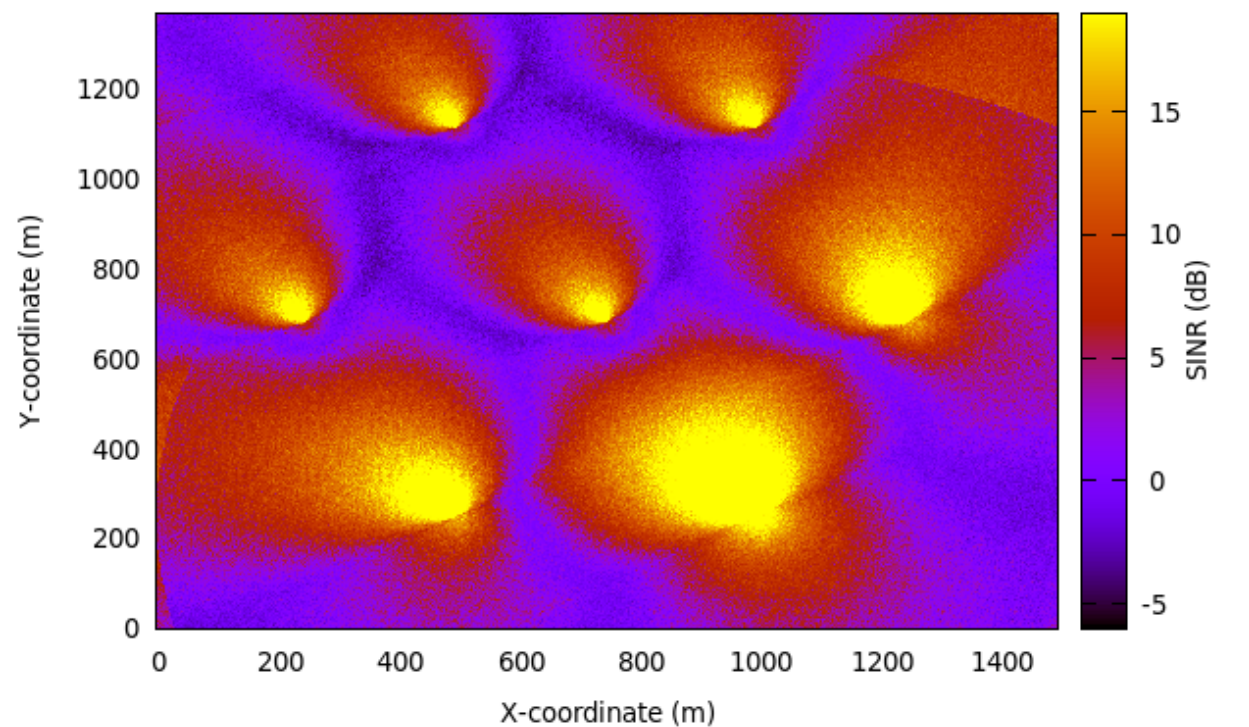


Fig. 19.79: REM for RB 10 obtained from lena-dual-stripe simulation with Hard FR algorithm enabled

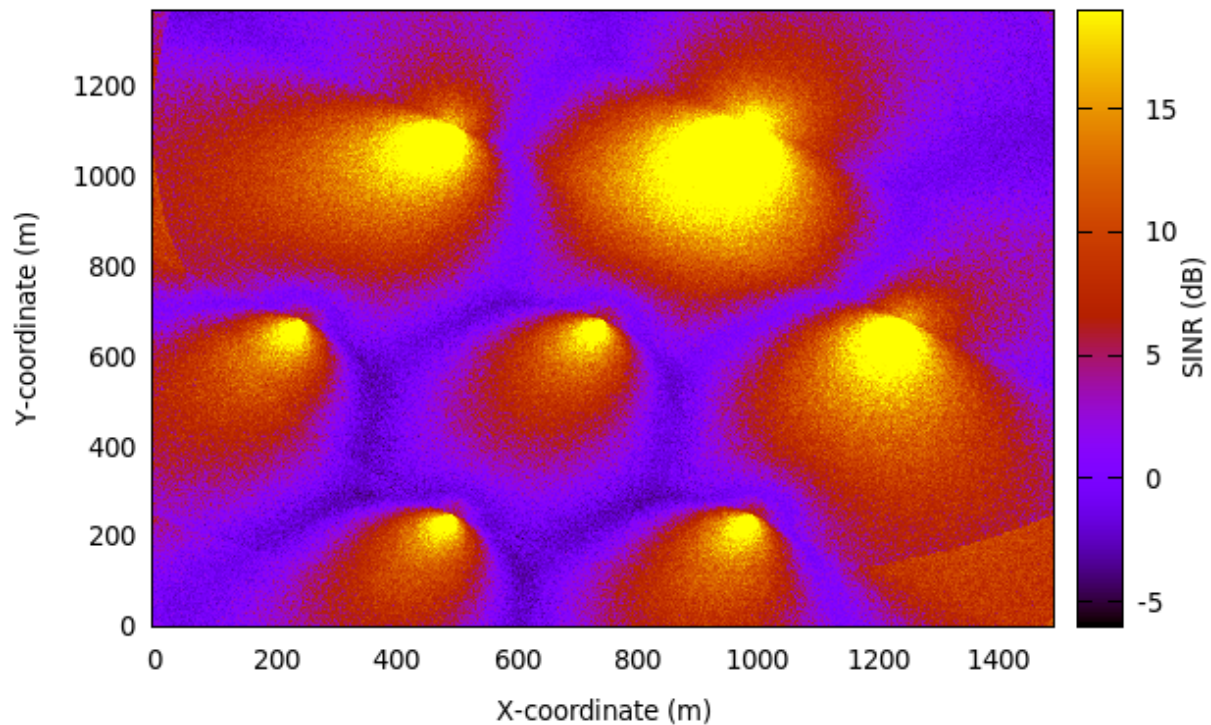


Fig. 19.80: REM for RB 20 obtained from lena-dual-stripe simulation with Hard FR algorithm enabled

by using a *test-runner*:

```
./ns3 run 'test-runner --suite=lte-carrier-aggregation'
```

To plot the test results, a file has to be created in the root folder of the ns-3 repository, and added to it with the following content :

```
set terminal png set xlabel "Number of users" set ylabel "Throughput per UE [Mbps]" set key top right
downlink_results="carrier_aggregation_results_dl.txt" uplink_results="carrier_aggregation_results_ul.txt"
set output "ca-test-example-dl.png" set title "Downlink performance"
```

plot downlink_results using 1:(\$2==1 ? \$3/1000000

```
[1/0) w lp t 'NO SDL', ] downlink_results using 1:($2==2 ? $3/1000000 : 1/0) w lp t 'RR SDL 1',
downlink_results using 1:($2==3 ? $3/1000000 : 1/0) w lp t 'RR SDL 2'
```

```
set output "ca-test-example-ul.png" set title "Uplink performance"
```

plot uplink_results using 1:(\$2==1 ? \$3/1000000

```
[1/0) w lp t 'NO SDL', ] uplink_results using 1:($2==2 ? $3/1000000 : 1/0) w lp t 'RR SDL 1',
uplink_results using 1:($2==3 ? $3/1000000 : 1/0) w lp t 'RR SDL 2'
```

gnuplot can be run by providing the file name, so that in the ns-3 root directory figures are generated. An example to run this test suite is shown in figures: *fig-ca-test-example-ul* and *fig-ca-test-example-dl*.

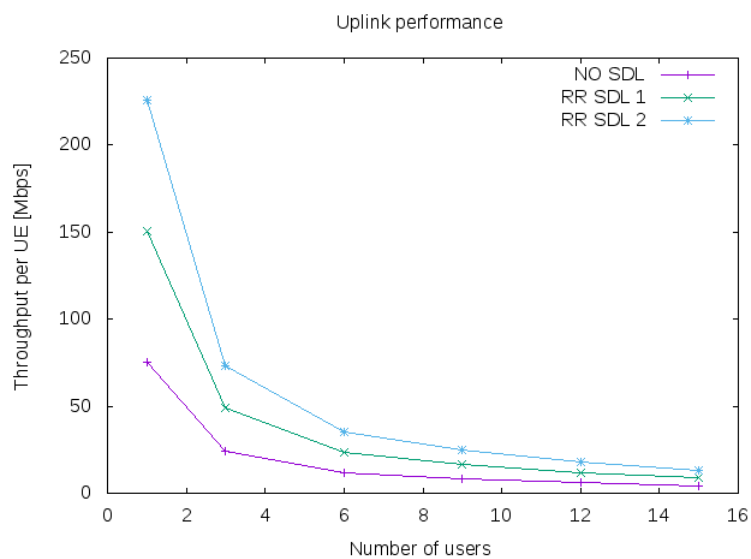


Fig. 19.81: Example of CA test performance in the uplink

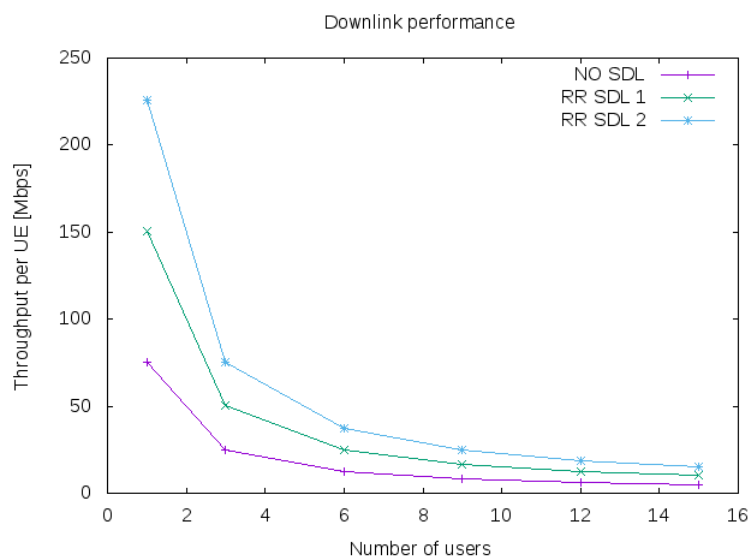


Fig. 19.82: Example of CA test performance in the downlink

19.2.25 Radio link failure example

The example *lena-radio-link-failure.cc* is an example to simulate the RLF functionality. In particular, it simulates only one moving UE using *Ideal* or *Real* RRC protocol with EPC performing downlink and uplink communication in two scenarios shown in *Scenario A: Radio link failure example with one eNB* and *Scenario B: Radio link failure example with two eNBs*

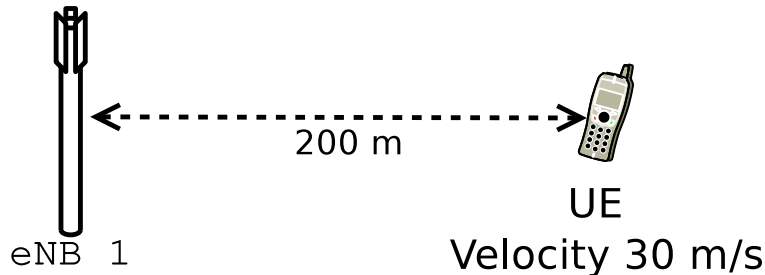


Fig. 19.83: Scenario A: Radio link failure example with one eNB

We note that, the RLF detection is enabled by default, which can be disabled by configuring the `LteUePhy::EnableRlfDetection` to false, e.g.,:

```
Config::SetDefault("ns3::LteUePhy::EnableRlfDetection", BooleanValue(false));
```

In this example, to study the impact of a RLF on the user's quality of experience, we compute an instantaneous (i.e., every 200 ms) DL throughput of the UE, and writes it into a file for plotting purposes. For example, to simulate the "Scenario A" with *Ideal* and *Real* RRC protocol a user can use the following commands:

```
Ideal RRC:
./ns3 run "lena-radio-link-failure
--numberOfEnbs=1 --useIdealRrc=1
--interSiteDistance=1200 --n310=1 --n311=1
--t310=1 --enableCtrlErrorModel=1
--enableDataErrorModel=1 --simTime=25"

Real RRC:
./ns3 run "lena-radio-link-failure
--numberOfEnbs=1 --useIdealRrc=0
--interSiteDistance=1200 --n310=1 --n311=1
--t310=1 --enableCtrlErrorModel=1
--enableDataErrorModel=1 --simTime=25"
```

After running the above two commands, we can use a simple gnuplot script to plot the throughput as shown in the Figure *Downlink instantaneous throughput of UE in scenario A*, e.g.,

```
set terminal png
set output "lena-radio-link-failure-one-enb-thrput.png"
set multiplot
set xlabel "Time [s]"
set ylabel "Instantaneous throughput UE [Mbps]"
set grid
set title "LTE RLF example 1 eNB DL instantaneous throughput"
plot "rlf_dl_thrput_1_eNB_ideal_rrc" using ($1):($2) with linespoints
title 'Ideal RRC' linestyle 1 lw 2 lc rgb 'blue', "rlf_dl_thrput_1_eNB_real_rrc"
```

(continues on next page)

(continued from previous page)

```
using ($1):($2) with linespoints title 'Real RRC' linestyle 2 lw 2 lc rgb 'red'
unset multiplot
```

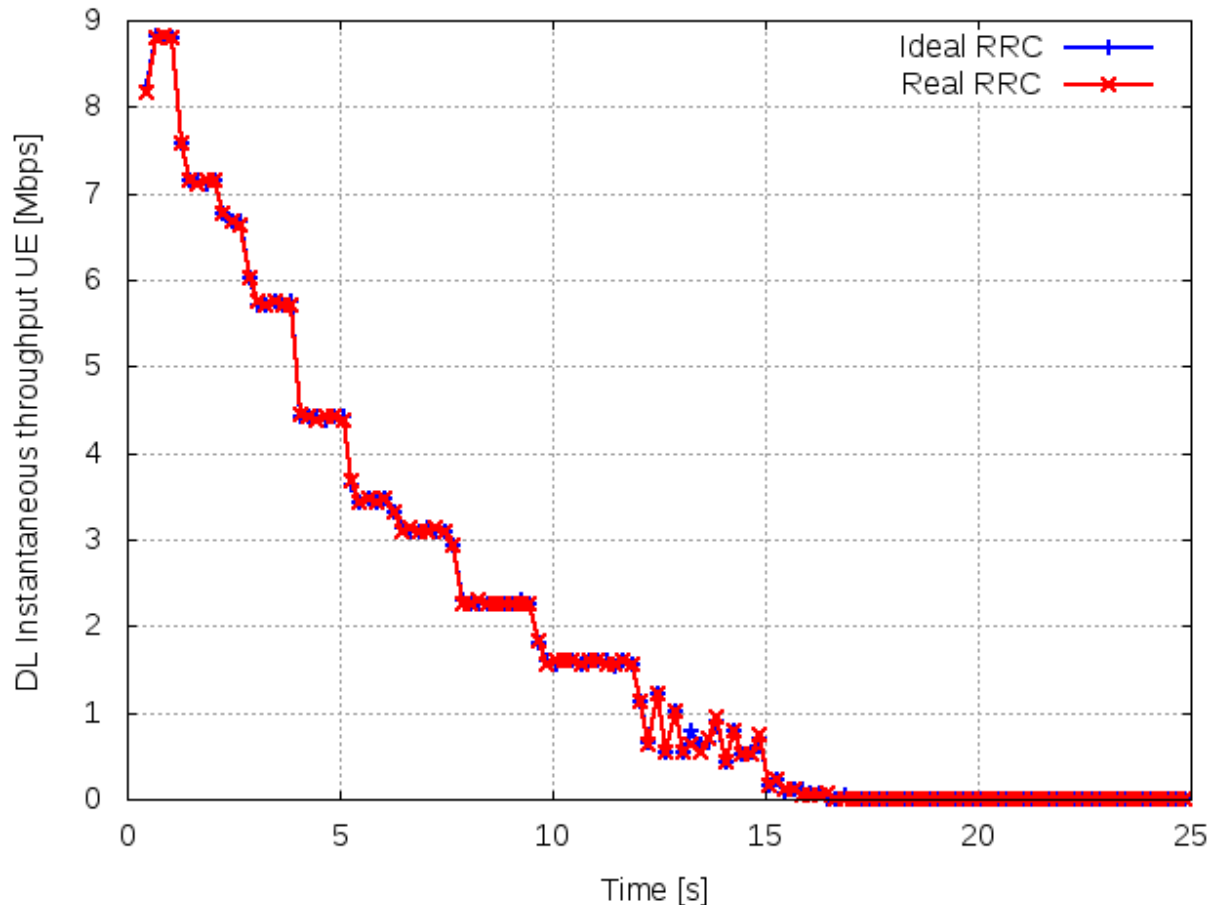


Fig. 19.84: Downlink instantaneous throughput of UE in scenario A

In the simulator, a UE can synchronize (i.e., start reading system information) with an eNB at a low RSRP level, which defaults to -140 dBm (see `QRxLevMin` attribute of eNB RRC). It enables the UE to start the random access procedure with the eNB. In this scenario, when using the *Ideal* RRC the UE after the RLF will connect and disconnect from the eNB several times. This is because in the *Ideal* RRC mode, once the UE is able to receive Random Access Response (RAR) from the eNB, it can complete the RRC connection establishment procedure (*RRC connection establishment*) without any errors, since all the RRC messages are exchanged ideally between the eNB and the UE. However, soon after the connection establishment, it ends up in RLF due to the poor channel quality. On the other hand, with the *Real* RRC the UE after the RLF will not be able to complete the RRC connection establishment procedure due to the loss of RRC messages. Thus, it will not be able to establish the connection with the eNB. Therefore, in both the cases the UE throughput drops to zero as shown in the Figure *Downlink instantaneous throughput of UE in scenario A*. It is also worthwhile to mention that towards the end of the simulation (using *Ideal* or *Real* RRC) there are occasions where RAR timer at the UE MAC would timeout due to the increased distance between the eNB and the UE, which causes errors while decoding this message at the UE (Note: the downlink control error model is enabled by default).

Similarly, to simulate the “Scenario B” with *Ideal* and *Real* RRC protocol following commands can be used:

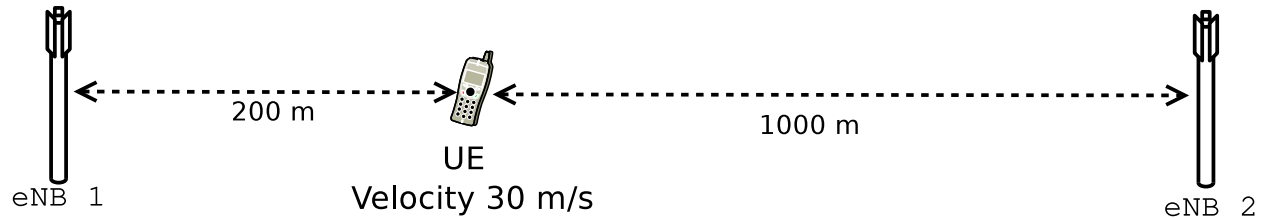


Fig. 19.85: Scenario B: Radio link failure example with two eNBs

```

Ideal RRC:
./ns3 run "lena-radio-link-failure
--numberOfEnbs=2 --useIdealRrc=1
--interSiteDistance=1200 --n310=1 --n311=1
--t310=1 --enableCtrlErrorModel=1
--enableDataErrorModel=1 --simTime=25"

Real RRC:
./ns3 run "lena-radio-link-failure
--numberOfEnbs=2 --useIdealRrc=0
--interSiteDistance=1200 --n310=1 --n311=1
--t310=1 --enableCtrlErrorModel=1
--enableDataErrorModel=1 --simTime=25"

```

Figure *Downlink instantaneous throughput of UE in scenario B*, shows the throughput in “Scenario B”. We note that in this scenario the handover algorithm is not used. As expected, with *Ideal* RRC protocol the UE after the RLF can complete the random access procedure with the second eNB. Interestingly, the DL SINR after the connection establishment is not low enough to trigger the RLF, but it is low enough to impact the DL control reception for some TBs, which in turn causes loss of data. It can be observed from the slightly unstable throughput of the UE after connecting to the second eNB. On the other hand, with *Real* RRC the UE faces problems in connection establishment phase due to the loss of RRC messages, in particular, the RRC connection request from the UE. This is the reason why the UE throughput after the RLF remains zero for a more extended period as compared to the *ideal* RRC protocol.

19.2.26 Troubleshooting and debugging tips

Many users post on the ns-3-users mailing list asking, for example, why they do not get any traffic in their simulation, or maybe only uplink but no downlink traffic is generated, etc. In most of the cases, this is a bug in the user simulation program. Here the reader can find some tips to debug the program and find out the cause of the problem.

The general approach is to selectively and incrementally enable the logging of relevant LTE module components, verifying upon each activation that the output is as expected. In detail:

- first check the control plane, in particular the RRC connection establishment procedure, by enabling the log components `LteUeRrc` and `LteEnbRrc`
- then check packet transmissions on the data plane, starting by enabling the log components `LteUeNetDevice` and the `EpcSgwApplication`, `EpcPgwApplication` and `EpcEnbApplication`, then moving down the LTE radio stack (PDCP, RLC, MAC, and finally PHY). All this until you find where packets stop being processed / forwarded.

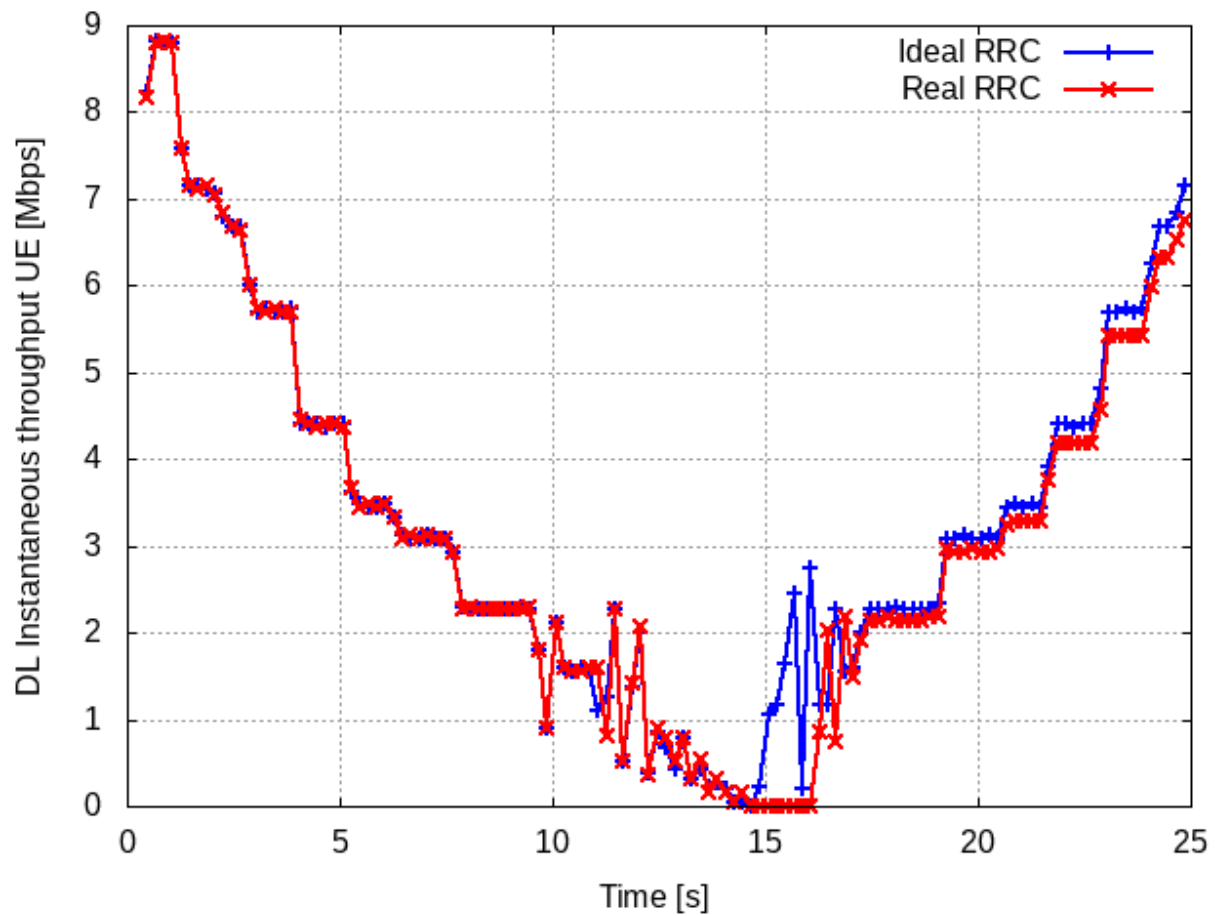


Fig. 19.86: Downlink instantaneous throughput of UE in scenario B

19.3 Testing Documentation

19.3.1 Overview

To test and validate the ns-3 LTE module, several test suites are provided which are integrated with the ns-3 test framework. To run them, you need to have configured the build of the simulator in this way:

```
$ ./ns3 configure --enable-tests --enable-modules=lte --enable-examples
$ ./test.py
```

The above will run not only the test suites belonging to the LTE module, but also those belonging to all the other ns-3 modules on which the LTE module depends. See the ns-3 manual for generic information on the testing framework.

You can get a more detailed report in HTML format in this way:

```
$ ./test.py -w results.html
```

After the above command has run, you can view the detailed result for each test by opening the file `results.html` with a web browser.

You can run each test suite separately using this command:

```
$ ./test.py -s test-suite-name
```

For more details about `test.py` and the ns-3 testing framework, please refer to the ns-3 manual.

19.3.2 Description of the test suites

Unit Tests

SINR calculation in the Downlink

The test suite `lte-downlink-sinr` checks that the SINR calculation in downlink is performed correctly. The SINR in the downlink is calculated for each RB assigned to data transmissions by dividing the power of the intended signal from the considered eNB by the sum of the noise power plus all the transmissions on the same RB coming from other eNBs (the interference signals):

$$\gamma = \frac{P_{\text{signal}}}{P_{\text{noise}} + \sum P_{\text{interference}}}$$

In general, different signals can be active during different periods of time. We define a *chunk* as the time interval between any two events of type either start or end of a waveform. In other words, a chunk identifies a time interval during which the set of active waveforms does not change. Let i be the generic chunk, T_i its duration and SINR_i its SINR, calculated with the above equation. The calculation of the average SINR $\bar{\gamma}$ to be used for CQI feedback reporting uses the following formula:

$$\bar{\gamma} = \frac{\sum_i \gamma_i T_i}{\sum_i T_i}$$

The test suite checks that the above calculation is performed correctly in the simulator. The test vectors are obtained offline by an Octave script that implements the above equation, and that recreates a number of random transmitted signals and interference signals that mimic a scenario where an UE is trying to decode a signal from an eNB while facing interference from other eNBs. The test passes if the calculated values are equal to the test vector within a tolerance of 10^{-7} . The tolerance is meant to account for the approximation errors typical of floating point arithmetic.

SINR calculation in the Uplink

The test suite `lte-uplink-sinr` checks that the SINR calculation in uplink is performed correctly. This test suite is identical to `lte-downlink-sinr` described in the previous section, with the difference that both the signal and the interference now refer to transmissions by the UEs, and reception is performed by the eNB. This test suite recreates a number of random transmitted signals and interference signals to mimic a scenario where an eNB is trying to decode the signal from several UEs simultaneously (the ones in the cell of the eNB) while facing interference from other UEs (the ones belonging to other cells).

The test vectors are obtained by a dedicated Octave script. The test passes if the calculated values are equal to the test vector within a tolerance of 10^{-7} which, as for the downlink SINR test, deals with floating point arithmetic approximation issues.

E-UTRA Absolute Radio Frequency Channel Number (EARFCN)

The test suite `lte-earfcn` checks that the carrier frequency used by the `LteSpectrumValueHelper` class (which implements the LTE spectrum model) is done in compliance with [TS36101], where the E-UTRA Absolute Radio Frequency Channel Number (EARFCN) is defined. The test vector for this test suite comprises a set of EARFCN values and the corresponding carrier frequency calculated by hand following the specification of [TS36101]. The test passes if the carrier frequency returned by `LteSpectrumValueHelper` is the same as the known value for each element in the test vector.

System Tests

Dedicated Bearer Deactivation Tests

The test suite ‘`lte-test-deactivate-bearer`’ creates test case with single ENodeB and Three UE’s. Each UE consists of one Default and one Dedicated EPS bearer with same bearer specification but with different ARP. Test Case Flow is as follows: Attach UE -> Create Default+Dedicated Bearer -> Deactivate one of the Dedicated bearer

Test case further deactivates dedicated bearer having bearer ID 2(LCID=BearerId+2) of First UE (UE_ID=1) User can schedule bearer deactivation after specific time delay using `Simulator::Schedule ()` method.

Once the test case execution ends it will create `D1R1cStats.txt` and `U1R1cStats.txt`. Key fields that need to be checked in statistics are:

Start	end	Cell ID	IMSI	RNTI	LCID	TxBytes	RxBytes	
-------	-----	---------	------	------	------	---------	---------	--

Test case executes in three epochs:

1. In first Epoch (0.04s-1.04s) All UE’s and corresponding bearers gets attached and packet flow over the dedicated bearers activated.
2. In second Epoch (1.04s-2.04s), bearer deactivation is instantiated, hence User can see relatively less number of TxBytes on UE_ID=1 and LCID=4 as compared to other bearers.
3. In third Epoch (2.04s-3.04s) since bearer deactivation of UE_ID=1 and LCID=4 is completed, user will not see any logging related to LCID=4.

Test case passes if and only if

1. IMSI=1 and LCID=4 completely removed in third epoch
2. No packets seen in TxBytes and RxBytes corresponding to IMSI=1 and LCID=4

If above criteria do not match, the test case is considered to be failed

Adaptive Modulation and Coding Tests

The test suite `lte-link-adaptation` provides system tests recreating a scenario with a single eNB and a single UE. Different test cases are created corresponding to different SNR values perceived by the UE. The aim of the test is to check that in each test case the chosen MCS corresponds to some known reference values. These reference values are obtained by re-implementing in Octave (see `src/lte/test/reference/lte_amc.m`) the model described in Section [Adaptive Modulation and Coding](#) for the calculation of the spectral efficiency, and determining the corresponding MCS index by manually looking up the tables in [R1-081483]. The resulting test vector is represented in Figure [Test vector for Adaptive Modulation and Coding](#).

The MCS which is used by the simulator is measured by obtaining the tracing output produced by the scheduler after 4ms (this is needed to account for the initial delay in CQI reporting). The SINR which is calculated by the simulator is also obtained using the `LteChunkProcessor` interface. The test passes if both the following conditions are satisfied:

1. the SINR calculated by the simulator correspond to the SNR of the test vector within an absolute tolerance of 10^{-7} ;
2. the MCS index used by the simulator exactly corresponds to the one in the test vector.

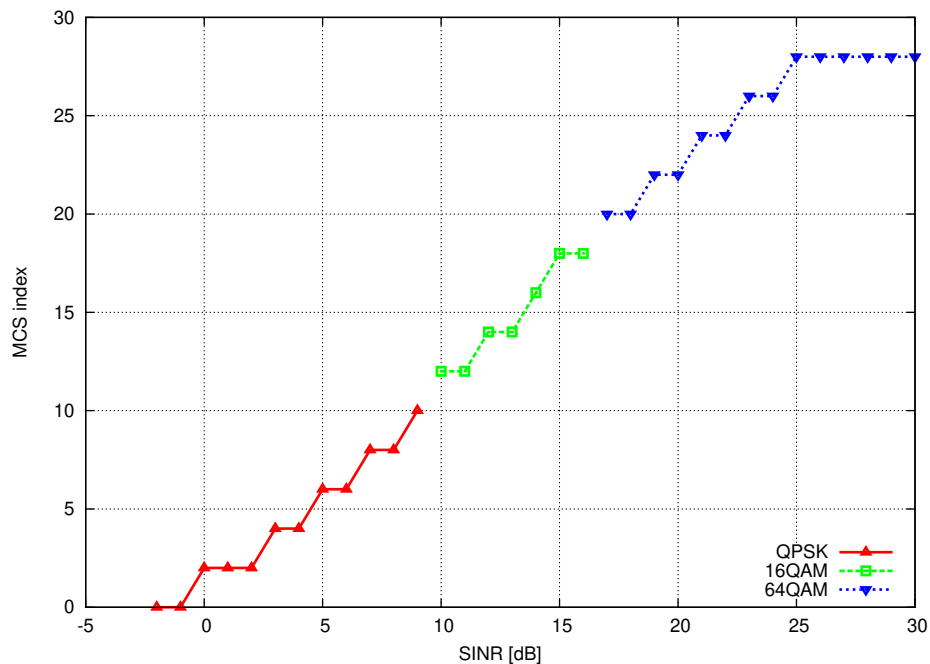


Fig. 19.87: Test vector for Adaptive Modulation and Coding

Inter-cell Interference Tests

The test suite `lte-interference` provides system tests recreating an inter-cell interference scenario with two eNBs, each having a single UE attached to it and employing Adaptive Modulation and Coding both in the downlink and in the uplink. The topology of the scenario is depicted in Figure [Topology for the inter-cell interference test](#). The d_1 parameter represents the distance of each UE to the eNB it is attached to, whereas the d_2 parameter represent the interferer distance. We note that the scenario topology is such that the interferer distance is the same for uplink and downlink; still, the actual interference power perceived will be different, because of the different propagation loss in the uplink and downlink bands. Different test cases are obtained by varying the d_1 and d_2 parameters.

The test vectors are obtained by use of a dedicated octave script (available in

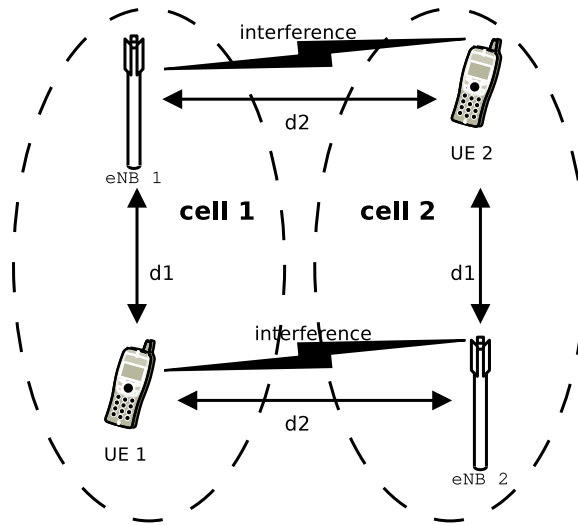


Fig. 19.88: Topology for the inter-cell interference test

src/lte/test/reference/lte_link_budget_interference.m), which does the link budget calculations (including interference) corresponding to the topology of each test case, and outputs the resulting SINR and spectral efficiency. The latter is then used to determine (using the same procedure adopted for *Adaptive Modulation and Coding Tests*). We note that the test vector contains separate values for uplink and downlink.

UE Measurements Tests

The test suite *lte-ue-measurements* provides system tests recreating an inter-cell interference scenario identical of the one defined for *lte-interference* test-suite. However, in this test the quantities to be tested are represented by RSRP and RSRQ measurements performed by the UE in two different points of the stack: the source, which is UE PHY layer, and the destination, that is the eNB RRC.

The test vectors are obtained by the use of a dedicated octave script (available in *src/lte/test/reference/lte-ue-measurements.m*), which does the link budget calculations (including interference) corresponding to the topology of each test case, and outputs the resulting RSRP and RSRQ. The obtained values are then used for checking the correctness of the UE Measurements at PHY layer. After that, they have to be converted according to 3GPP formatting for the purpose of checking their correctness at eNB RRC level.

UE measurement configuration tests

Besides the previously mentioned test suite, there are 3 other test suites for testing UE measurements: *lte-ue-measurements-piecewise-1*, *lte-ue-measurements-piecewise-2*, and *lte-ue-measurements-handover*. These test suites are more focused on the reporting trigger procedure, i.e. the correctness of the implementation of the event-based triggering criteria is verified here.

In more specific, the tests verify the *timing* and the *content* of each measurement reports received by eNodeB. Each test case is an stand-alone LTE simulation and the test case will pass if measurement report(s) only occurs at the prescribed time and shows the correct level of RSRP (RSRQ is not verified at the moment).

Piecewise configuration

The piecewise configuration aims to test a particular UE measurements configuration. The simulation script will setup the corresponding measurements configuration to the UE, which will be active throughout the simulation.

Since the reference values are precalculated by hands, several assumptions are made to simplify the simulation. Firstly, the channel is only affected by path loss model (in this case, Friis model is used). Secondly, the ideal RRC protocol is used, and layer 3 filtering is disabled. Finally, the UE moves in a predefined motion pattern between 4 distinct spots, as depicted in Figure *UE movement trace throughout the simulation in piecewise configuration* below. Therefore the fluctuation of the measured RSRP can be determined more easily.

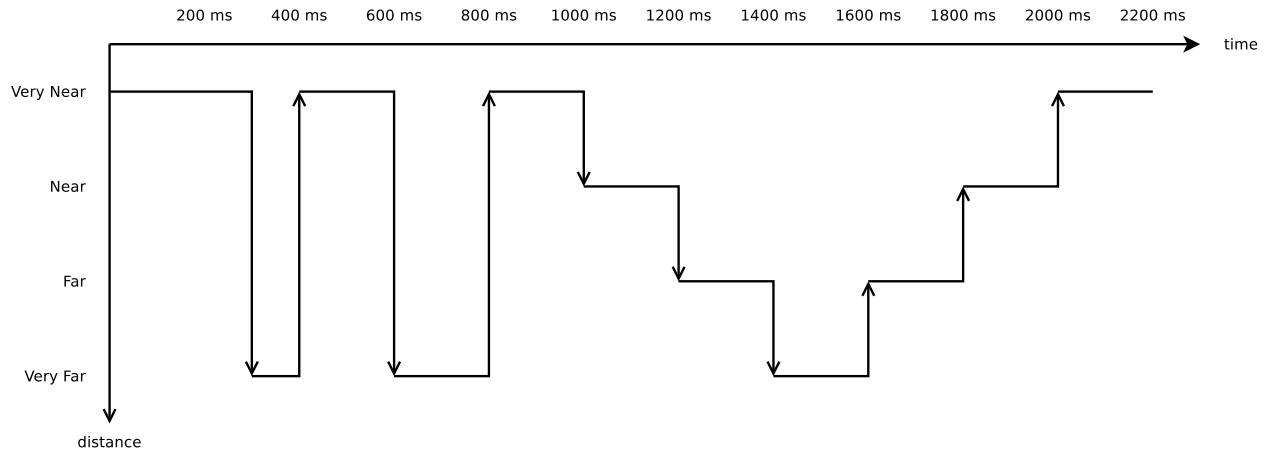


Fig. 19.89: UE movement trace throughout the simulation in piecewise configuration

The motivation behind the “teleport” between the predefined spots is to introduce drastic change of RSRP level, which will guarantee the triggering of entering or leaving condition of the tested event. By performing drastic changes, the test can be run within shorter amount of time.

Figure *Measured RSRP trace of an example Event A1 test case in piecewise configuration* below shows the measured RSRP after layer 1 filtering by the PHY layer during the simulation with a piecewise configuration. Because layer 3 filtering is disabled, these are the exact values used by the UE RRC instance to evaluate reporting trigger procedure. Notice that the values are refreshed every 200 ms, which is the default filtering period of PHY layer measurements report. The figure also shows the time when entering and leaving conditions of an example instance of Event A1 (serving cell becomes better than threshold) occur during the simulation.

Each reporting criterion is tested several times with different threshold/offset parameters. Some test scenarios also take hysteresis and time-to-trigger into account. Figure *Measured RSRP trace of an example Event A1 with hysteresis test case in piecewise configuration* depicts the effect of hysteresis in another example of Event A1 test.

Piecewise configuration is used in two test suites of UE measurements. The first one is *lte-ue-measurements-piecewise-1*, henceforth Piecewise test #1, which simulates 1 UE and 1 eNodeB. The other one is *lte-ue-measurements-piecewise-2*, which has 1 UE and 2 eNodeBs in the simulation.

Piecewise test #1 is intended to test the event-based criteria which are not dependent on the existence of a neighboring cell. These criteria include Event A1 and A2. The other events are also briefly tested to verify that they are still working correctly (albeit not reporting anything) in the absence of any neighboring cell. Table *UE measurements test scenarios using piecewise configuration #1* below lists the scenarios tested in piecewise test #1.

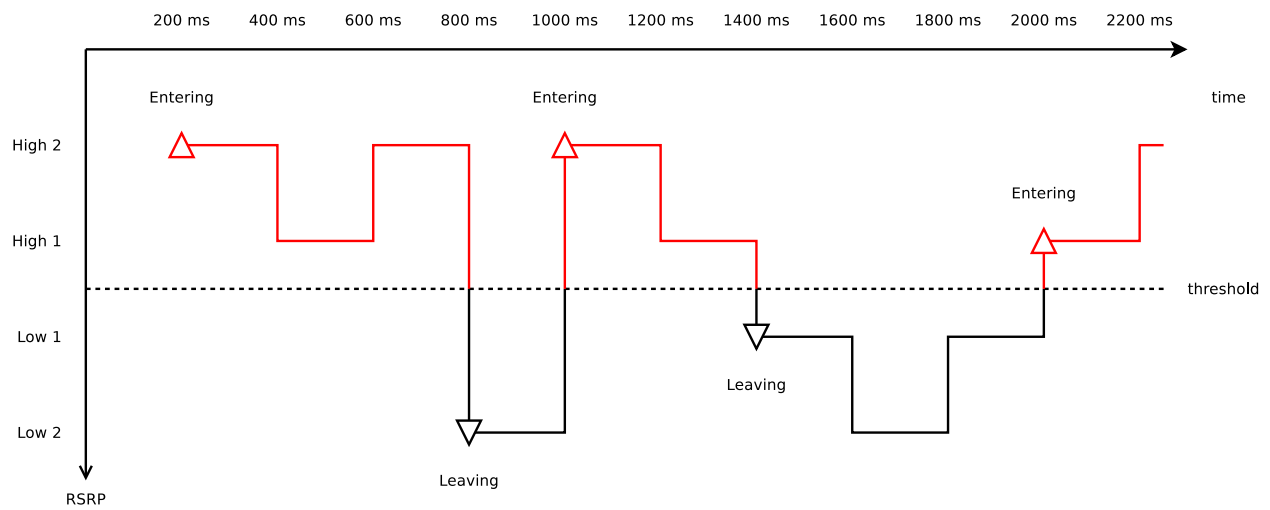


Fig. 19.90: Measured RSRP trace of an example Event A1 test case in piecewise configuration

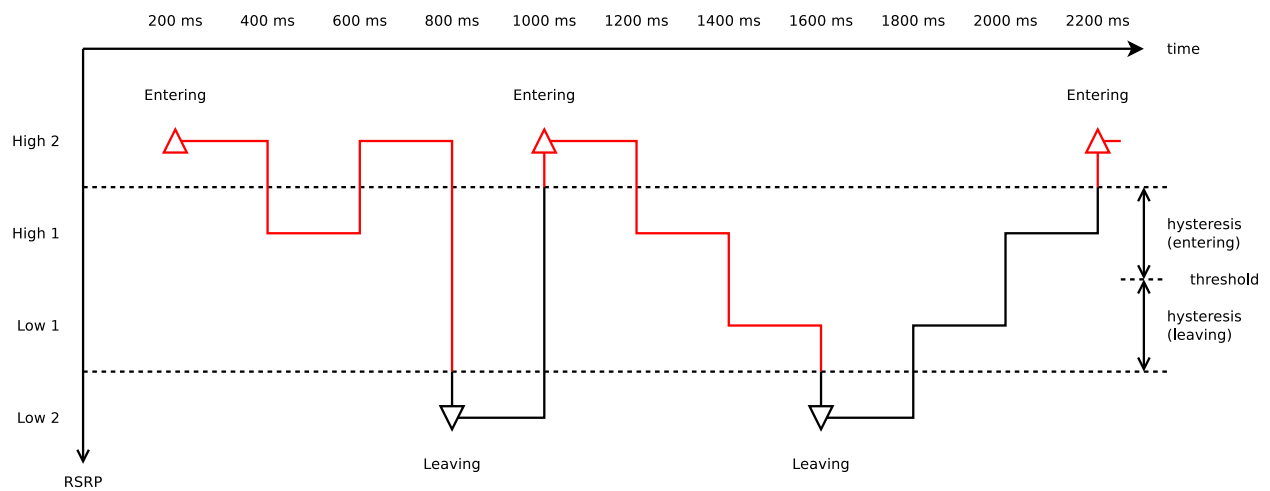


Fig. 19.91: Measured RSRP trace of an example Event A1 with hysteresis test case in piecewise configuration

Table 19.7: UE measurements test scenarios using piecewise configuration #1

Test #	Reporting Criteria	Threshold/Offset	Hysteresis	Time-to-Trigger
1	Event A1	Low	No	No
2	Event A1	Normal	No	No
3	Event A1	Normal	No	Short
4	Event A1	Normal	No	Long
5	Event A1	Normal	No	Super
6	Event A1	Normal	Yes	No
7	Event A1	High	No	No
8	Event A2	Low	No	No
9	Event A2	Normal	No	No
10	Event A2	Normal	No	Short
11	Event A2	Normal	No	Long
12	Event A2	Normal	No	Super
13	Event A2	Normal	Yes	No
14	Event A2	High	No	No
15	Event A3	Zero	No	No
16	Event A4	Normal	No	No
17	Event A5	Normal-Normal	No	No

Other events such as Event A3, A4, and A5 depend on measurements of neighbouring cell, so they are more thoroughly tested in Piecewise test #2. The simulation places the nodes on a straight line and instruct the UE to “*jump*” in a similar manner as in Piecewise test #1. Handover is disabled in the simulation, so the role of serving and neighbouring cells do not switch during the simulation. Table *UE measurements test scenarios using piecewise configuration #2* below lists the scenarios tested in Piecewise test #2.

Table 19.8: UE measurements test scenarios using piecewise configuration #2

Test #	Reporting Criteria	Threshold/Offset	Hysteresis	Time-to-Trigger
1	Event A1	Low	No	No
2	Event A1	Normal	No	No
3	Event A1	Normal	Yes	No
4	Event A1	High	No	No
5	Event A2	Low	No	No
6	Event A2	Normal	No	No
7	Event A2	Normal	Yes	No
8	Event A2	High	No	No
9	Event A3	Positive	No	No
10	Event A3	Zero	No	No
11	Event A3	Zero	No	Short
12	Event A3	Zero	No	Super
13	Event A3	Zero	Yes	No
14	Event A3	Negative	No	No
15	Event A4	Low	No	No
16	Event A4	Normal	No	No
17	Event A4	Normal	No	Short
18	Event A4	Normal	No	Super
19	Event A4	Normal	Yes	No
20	Event A4	High	No	No

continues on next page

Table 19.8 – continued from previous page

Test #	Reporting Criteria	Threshold/Offset	Hysteresis	Time-to-Trigger
21	Event A5	Low-Low	No	No
22	Event A5	Low-Normal	No	No
23	Event A5	Low-High	No	No
24	Event A5	Normal-Low	No	No
25	Event A5	Normal-Normal	No	No
26	Event A5	Normal-Normal	No	Short
27	Event A5	Normal-Normal	No	Super
28	Event A5	Normal-Normal	Yes	No
29	Event A5	Normal-High	No	No
30	Event A5	High-Low	No	No
31	Event A5	High-Normal	No	No
32	Event A5	High-High	No	No

One note about the tests with time-to-trigger, they are tested using 3 different values of time-to-trigger: *short* (shorter than report interval), *long* (shorter than the filter measurement period of 200 ms), and *super* (longer than 200 ms). The first two ensure that time-to-trigger evaluation always use the latest measurement reports received from PHY layer. While the last one is responsible for verifying time-to-trigger cancellation, for example when a measurement report from PHY shows that the entering/leaving condition is no longer true before the first trigger is fired.

Handover configuration

The purpose of the handover configuration is to verify whether UE measurement configuration is updated properly after a successful handover takes place. For this purpose, the simulation will construct 2 eNodeBs with different UE measurement configuration, and the UE will perform handover from one cell to another. The UE will be located on a straight line between the 2 eNodeBs, and the handover will be invoked manually. The duration of each simulation is 2 seconds (except the last test case) and the handover is triggered exactly at halfway of simulation.

The *lte-ue-measurements-handover* test suite covers various types of configuration differences. The first one is the difference in report interval, e.g. the first eNodeB is configured with 480 ms report interval, while the second eNodeB is configured with 240 ms report interval. Therefore, when the UE performed handover to the second cell, the new report interval must take effect. As in piecewise configuration, the timing and the content of each measurement report received by the eNodeB will be verified.

Other types of differences covered by the test suite are differences in event and differences in threshold/offset. Table *UE measurements test scenarios using handover configuration* below lists the tested scenarios.

Table 19.9: UE measurements test scenarios using handover configuration

Test #	Test Subject	Initial Configuration	Post-Handover Configuration
1	Report interval	480 ms	240 ms
2	Report interval	120 ms	640 ms
3	Event	Event A1	Event A2
4	Event	Event A2	Event A1
5	Event	Event A3	Event A4
6	Event	Event A4	Event A3
7	Event	Event A2	Event A3
8	Event	Event A3	Event A2
9	Event	Event A4	Event A5
10	Event	Event A5	Event A4
11	Threshold/offset	RSRP range 52 (Event A1)	RSRP range 56 (Event A1)
12	Threshold/offset	RSRP range 52 (Event A2)	RSRP range 56 (Event A2)
13	Threshold/offset	A3 offset -30 (Event A3)	A3 offset +30 (Event A3)
14	Threshold/offset	RSRP range 52 (Event A4)	RSRP range 56 (Event A4)
15	Threshold/offset	RSRP range 52-52 (Event A5)	RSRP range 56-56 (Event A5)
16	Time-to-trigger	1024 ms	100 ms
17	Time-to-trigger	1024 ms	640 ms

Round Robin scheduler performance

The test suite `lte-rr-ff-mac-scheduler` creates different test cases with a single eNB and several UEs, all having the same Radio Bearer specification. In each test case, the UEs see the same SINR from the eNB; different test cases are implemented by using different distance among UEs and the eNB (i.e., therefore having different SINR values) and different numbers of UEs. The test consists on checking that the obtained throughput performance is equal among users and matches a reference throughput value obtained according to the SINR perceived within a given tolerance.

The test vector is obtained according to the values of transport block size reported in table 7.1.7.2.1-1 of [TS36213], considering an equal distribution of the physical resource block among the users using Resource Allocation Type 0 as defined in Section 7.1.6.1 of [TS36213]. Let τ be the TTI duration, N be the number of UEs, B the transmission bandwidth configuration in number of RBs, G the RBG size, M the modulation and coding scheme in use at the given SINR and $S(M, B)$ be the transport block size in bits as defined by 3GPP TS 36.213. We first calculate the number L of RBGs allocated to each user as

$$L = \left\lfloor \frac{B}{NG} \right\rfloor$$

The reference throughput T in bit/s achieved by each UE is then calculated as

$$T = \frac{S(M, LG)}{8 \tau}$$

The test passes if the measured throughput matches with the reference throughput within a relative tolerance of 0.1. This tolerance is needed to account for the transient behavior at the beginning of the simulation (e.g., CQI feedback is only available after a few subframes) as well as for the accuracy of the estimator of the average throughput performance over the chosen simulation time (0.4s). This choice of the simulation time is justified by the need to follow the ns-3 guidelines of keeping the total execution time of the test suite low, in spite of the high number of test cases. In any case, we note that a lower value of the tolerance can be used when longer simulations are run.

In Figure [fig-lenaThrTestCase1](#), the curves labeled “RR” represent the test values calculated for the RR scheduler test, as a function of the number of UEs and of the MCS index being used in each test case.

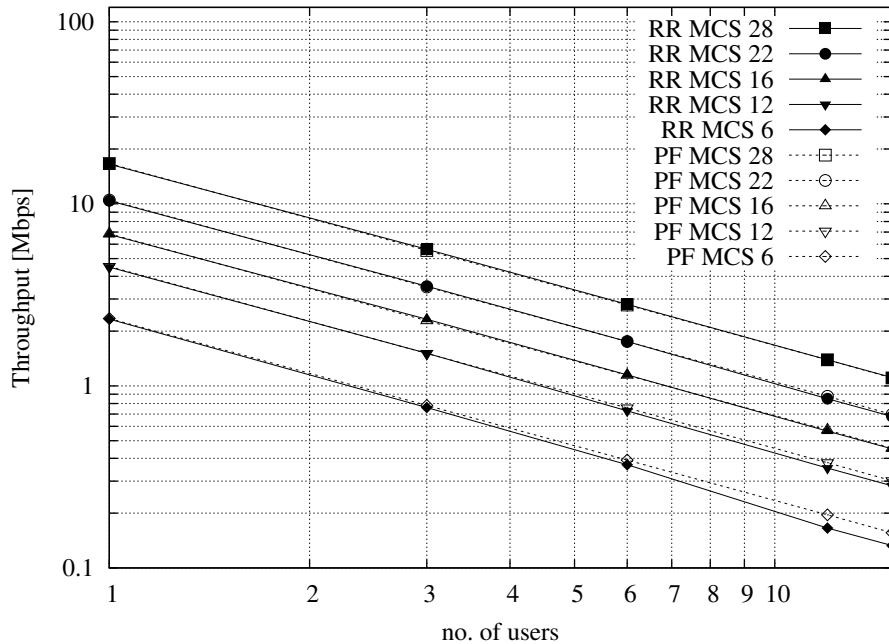


Fig. 19.92: Test vectors for the RR and PF Scheduler in the downlink in a scenario where all UEs use the same MCS.

Proportional Fair scheduler performance

The test suite `lte-pf-ff-mac-scheduler` creates different test cases with a single eNB, using the Proportional Fair (PF) scheduler, and several UEs, all having the same Radio Bearer specification. The test cases are grouped in two categories in order to evaluate the performance both in terms of the adaptation to the channel condition and from a fairness perspective.

In the first category of test cases, the UEs are all placed at the same distance from the eNB, and hence all placed in order to have the same SINR. Different test cases are implemented by using a different SINR value and a different number of UEs. The test consists on checking that the obtained throughput performance matches with the known reference throughput up to a given tolerance. The expected behavior of the PF scheduler when all UEs have the same SNR is that each UE should get an equal fraction of the throughput obtainable by a single UE when using all the resources. We calculate the reference throughput value by dividing the throughput achievable by a single UE at the given SINR by the total number of UEs. Let τ be the TTI duration, B the transmission bandwidth configuration in number of RBs, M the modulation and coding scheme in use at the given SINR and $S(M, B)$ be the transport block size as defined in [TS36213]. The reference throughput T in bit/s achieved by each UE is calculated as

$$T = \frac{S(M, B)}{\tau N}$$

The curves labeled “PF” in Figure [fig-lenaThrTestCase1](#) represent the test values calculated for the PF scheduler tests of the first category, that we just described.

The second category of tests aims at verifying the fairness of the PF scheduler in a more realistic simulation scenario where the UEs have a different SINR (constant for the whole simulation). In these conditions, the PF scheduler will give to each user a share of the system bandwidth that is proportional to the capacity achievable by a single user alone considered its SINR. In detail, let M_i be the modulation and coding scheme being used by each UE (which is a deterministic function of the SINR of the UE, and is hence known in this scenario). Based on the MCS, we determine the achievable rate R_i for each user i using the procedure described in Section-[ref{sec:pfs}](#). We then define the achievable

rate ratio $\rho_{R,i}$ of each user i as

$$\rho_{R,i} = \frac{R_i}{\sum_{j=1}^N R_j}$$

Let now T_i be the throughput actually achieved by the UE i , which is obtained as part of the simulation output. We define the obtained throughput ratio $\rho_{T,i}$ of UE i as

$$\rho_{T,i} = \frac{T_i}{\sum_{j=1}^N T_j}$$

The test consists of checking that the following condition is verified:

$$\rho_{R,i} = \rho_{T,i}$$

if so, it means that the throughput obtained by each UE over the whole simulation matches with the steady-state throughput expected by the PF scheduler according to the theory. This test can be derived from [Holtzman2000] as follows. From Section 3 of [Holtzman2000], we know that

$$\frac{T_i}{R_i} = c, \forall i$$

where c is a constant. By substituting the above into the definition of $\rho_{T,i}$ given previously, we get

$$\begin{aligned} \frac{T_i}{\sum_{j=1}^N T_j} &= \frac{cR_i}{\sum_{j=1}^N cR_j} \\ &= \frac{cR_i}{c \sum_{j=1}^N R_j} \\ &= \frac{R_i}{\sum_{j=1}^N R_j} \end{aligned}$$

which is exactly the expression being used in the test.

Figure *Throughput ratio evaluation for the PF scheduler in a scenario where the UEs have MCS index 28, 24, 16, 12, 6* presents the results obtained in a test case with UEs $i = 1, \dots, 5$ that are located at a distance from the base station such that they will use respectively the MCS index 28, 24, 16, 12, 6. From the figure, we note that, as expected, the obtained throughput is proportional to the achievable rate. In other words, the PF scheduler assign more resources to the users that use a higher MCS index.

Maximum Throughput scheduler performance

Test suites `lte-fdmt-ff-mac-scheduler` and `lte-tdmt-ff-mac-scheduler` create different test cases with a single eNB and several UEs, all having the same Radio Bearer specification, using the Frequency Domain Maximum Throughput (FDMT) scheduler and Time Domain Maximum Throughput (TDMT) scheduler respectively. In other words, UEs are all placed at the same distance from the eNB, and hence all placed in order to have the same SNR. Different test cases are implemented by using a different SNR values and a different number of UEs. The test consists on checking that the obtained throughput performance matches with the known reference throughput up to a given tolerance. The expected behavior of both FDMT and TDMT scheduler when all UEs have the same SNR is that scheduler allocates all RBGs to the first UE defined in script. This is because the current FDMT and TDMT implementation always select the first UE to serve when there are multiple UEs having the same SNR value. We calculate the reference throughput value for first UE by the throughput achievable of a single UE at the given SNR, while reference throughput value for other UEs by zero. Let τ be the TTI duration, B the transmission bandwidth configuration in number of RBs, M the modulation and coding scheme in use at the given SNR and $S(M, B)$ be the transport block size as defined in [TS36213]. The reference throughput T in bit/s achieved by each UE is calculated as

$$T = \frac{S(M, B)}{\tau}$$

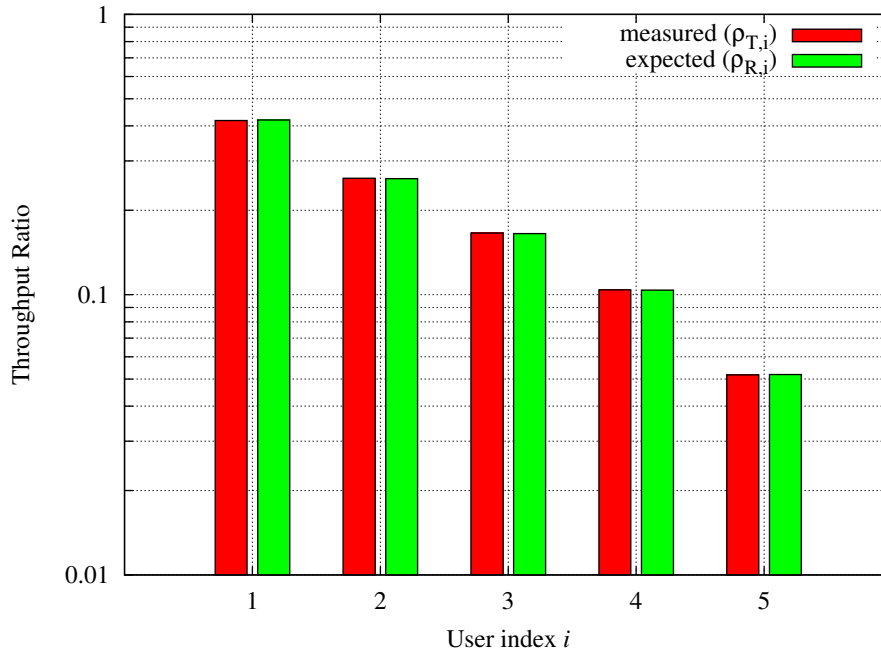


Fig. 19.93: Throughput ratio evaluation for the PF scheduler in a scenario where the UEs have MCS index 28, 24, 16, 12, 6

Throughput to Average scheduler performance

Test suites `lte-tta-ff-mac-scheduler` create different test cases with a single eNB and several UEs, all having the same Radio Bearer specification using TTA scheduler. Network topology and configurations in TTA test case are the same as the test for MT scheduler. More complex test case needs to be developed to show the fairness feature of TTA scheduler.

Blind Average Throughput scheduler performance

Test suites `lte-tdbet-ff-mac-scheduler` and `lte-fdbet-ff-mac-scheduler` create different test cases with a single eNB and several UEs, all having the same Radio Bearer specification.

In the first test case of `lte-tdbet-ff-mac-scheduler` and `lte-fdbet-ff-mac-scheduler`, the UEs are all placed at the same distance from the eNB, and hence all placed in order to have the same SNR. Different test cases are implemented by using a different SNR value and a different number of UEs. The test consists on checking that the obtained throughput performance matches with the known reference throughput up to a given tolerance. In long term, the expected behavior of both TD-BET and FD-BET when all UEs have the same SNR is that each UE should get an equal throughput. However, the exact throughput value of TD-BET and FD-BET in this test case is not the same.

When all UEs have the same SNR, TD-BET can be seen as a specific case of PF where achievable rate equals to 1. Therefore, the throughput obtained by TD-BET is equal to that of PF. On the other hand, FD-BET performs very similar to the round robin (RR) scheduler in case of that all UEs have the same SNR and the number of UE(or RBG) is an integer multiple of the number of RBG(or UE). In this case, FD-BET always allocate the same number of RBGs to each UE. For example, if eNB has 12 RBGs and there are 6 UEs, then each UE will get 2 RBGs in each TTI. Or if eNB has 12 RBGs and there are 24 UEs, then each UE will get 2 RBGs per two TTIs. When the number of UE (RBG) is not an integer multiple of the number of RBG (UE), FD-BET will not follow the RR behavior because it will assigned different number of RBGs to some UEs, while the throughput of each UE is still the same.

The second category of tests aims at verifying the fairness of the both TD-BET and FD-BET schedulers in a more realistic simulation scenario where the UEs have a different SNR (constant for the whole simulation). In this case, both scheduler should give the same amount of averaged throughput to each user.

Specifically, for TD-BET, let F_i be the fraction of time allocated to user i in total simulation time, R_i^{fb} be the full bandwidth achievable rate for user i and T_i be the achieved throughput of user i . Then we have:

$$T_i = F_i R_i^{fb}$$

In TD-BET, the sum of F_i for all user equals one. In long term, all UE has the same T_i so that we replace T_i by T . Then we have:

$$T = \frac{1}{\sum_{i=1}^N \frac{1}{R_i^{fb}}}$$

Token Band Fair Queue scheduler performance

Test suites `lte-fdtbfq-ff-mac-scheduler` and `lte-tdtbfq-ff-mac-scheduler` create different test cases for testing three key features of TBFQ scheduler: traffic policing, fairness and traffic balance. Constant Bit Rate UDP traffic is used in both downlink and uplink in all test cases. The packet interval is set to 1ms to keep the RLC buffer non-empty. Different traffic rate is achieved by setting different packet size. Specifically, two classes of flows are created in the test suites:

- Homogeneous flow: flows with the same token generation rate and packet arrival rate
- Heterogeneous flow: flows with different packet arrival rate, but with the same token generation rate

In test case 1 verifies traffic policing and fairness features for the scenario that all UEs are placed at the same distance from the eNB. In this case, all Ues have the same SNR value. Different test cases are implemented by using a different SNR value and a different number of UEs. Because each flow have the same traffic rate and token generation rate, TBFQ scheduler will guarantee the same throughput among UEs without the constraint of token generation rate. In addition, the exact value of UE throughput is depended on the total traffic rate:

- If total traffic rate $\leq$ maximum throughput, UE throughput = traffic rate
- If total traffic rate $>$ maximum throughput, UE throughput = maximum throughput / N

Here, N is the number of UE connected to eNodeB. The maximum throughput in this case equals to the rate that all RBGs are assigned to one UE(e.g., when distance equals 0, maximum throughput is 2196000 byte/sec). When the traffic rate is smaller than max bandwidth, TBFQ can police the traffic by token generation rate so that the UE throughput equals its actual traffic rate (token generation rate is set to traffic generation rate); On the other hand, when total traffic rate is bigger than the max throughput, eNodeB cannot forward all traffic to UEs. Therefore, in each TTI, TBFQ will allocate all RBGs to one UE due to the large packets buffered in RLC buffer. When a UE is scheduled in current TTI, its token counter is decreased so that it will not be scheduled in the next TTI. Because each UE has the same traffic generation rate, TBFQ will serve each UE in turn and only serve one UE in each TTI (both in TD TBFQ and FD TBFQ). Therefore, the UE throughput in the second condition equals to the evenly share of maximum throughput.

Test case 2 verifies traffic policing and fairness features for the scenario that each UE is placed at the different distance from the eNB. In this case, each UE has the different SNR value. Similar to test case 1, UE throughput in test case 2 is also depended on the total traffic rate but with a different maximum throughput. Suppose all UEs have a high traffic load. Then the traffic will saturate the RLC buffer in eNodeB. In each TTI, after selecting one UE with highest metric, TBFQ will allocate all RBGs to this UE due to the large RLC buffer size. On the other hand, once RLC buffer is saturated, the total throughput of all UEs cannot increase any more. In addition, as we discussed in test case 1, for homogeneous flows which have the same t_i and r_i , each UE will achieve the same throughput in long term. Therefore, we can use the same method in TD BET to calculate the maximum throughput:

$$T = \frac{N}{\sum_{i=1}^N \frac{1}{R_i^{fb}}}$$

Here, T is the maximum throughput. R_i^{fb} be the full bandwidth achievable rate for user i . N is the number of UE.

When the total traffic rate is bigger than T , the UE throughput equals to $\frac{T}{N}$. Otherwise, UE throughput equals to its traffic generation rate.

In test case 3, three flows with different traffic rate are created. Token generation rate for each flow is the same and equals to the average traffic rate of three flows. Because TBFQ use a shared token bank, tokens contributed by UE with lower traffic load can be utilized by UE with higher traffic load. In this way, TBFQ can guarantee the traffic rate for each flow. Although we use heterogeneous flow here, the calculation of maximum throughput is as same as that in test case 2. In calculation max throughput of test case 2, we assume that all UEs suffer high traffic load so that scheduler always assign all RBGs to one UE in each TTI. This assumes is also true in heterogeneous flow case. In other words, whether those flows have the same traffic rate and token generation rate, if their traffic rate is bigger enough, TBFQ performs as same as it in test case 2. Therefore, the maximum bandwidth in test case 3 is as same as it in test case 2.

In test case 3, in some flows, token generate rate does not equal to MBR, although all flows are CBR traffic. This is not accorded with our parameter setting rules. Actually, the traffic balance feature is used in VBR traffic. Because different UE's peak rate may occur in different time, TBFQ use shared token bank to balance the traffic among those VBR traffics. Test case 3 use CBR traffic to verify this feature. But in the real simulation, it is recommended to set token generation rate to MBR.

Priority Set scheduler performance

Test suites `lte-pss-ff-mac-scheduler` create different test cases with a single eNB and several UEs. In all test cases, we select PFsch in FD scheduler. Same testing results can also be obtained by using CoItA scheduler. In addition, all test cases do not define nMux so that TD scheduler in PSS will always select half of total UE.

In the first class test case of `lte-pss-ff-mac-scheduler`, the UEs are all placed at the same distance from the eNB, and hence all placed in order to have the same SNR. Different test cases are implemented by using a different TBR for each UEs. In each test cases, all UEs have the same Target Bit Rate configured by GBR in EPS bear setting. The expected behavior of PSS is to guarantee that each UE's throughput at least equals its TBR if the total flow rate is blow maximum throughput. Similar to TBFQ, the maximum throughput in this case equals to the rate that all RBGs are assigned to one UE. When the traffic rate is smaller than max bandwidth, the UE throughput equals its actual traffic rate; On the other hand, UE throughput equals to the evenly share of the maximum throughput.

In the first class of test cases, each UE has the same SNR. Therefore, the priority metric in PF scheduler will be determined by past average throughput $T_j(t)$ because each UE has the same achievable throughput $R_j(k, t)$ in PFsch or same $CoI[k, n]$ in CoItA. This means that PSS will performs like a TD-BET which allocates all RBGs to one UE in each TTI. Then the maximum value of UE throughput equals to the achievable rate that all RBGs are allocated to this UE.

In the second class of test case of `lte-pss-ff-mac-scheduler`, the UEs are all placed at the same distance from the eNB, and hence all placed in order to have the same SNR. Different TBR values are assigned to each UE. There also exist an maximum throughput in this case. Once total traffic rate is bigger than this threshold, there will be some UEs that cannot achieve their TBR. Because there is no fading, subband CQIs for each RBGs frequency are the same. Therefore, in FD scheduler, in each TTI, priority metrics of UE for all RBGs are the same. This means that FD scheduler will always allocate all RBGs to one user. Therefore, in the maximum throughput case, PSS performs like a TD-BET. Then we have:

$$T = \frac{N}{\sum_{i=1}^N \frac{1}{R_i^{fb}}}$$

Here, T is the maximum throughput. R_i^{fb} be the full bandwidth achievable rate for user i . N is the number of UE.

Channel and QoS aware scheduler performance

The performance of the Channel and QoS aware scheduler can be tested in the similar way to performance of Priority Set scheduler when GBR flows are not delay sensitive by measuring if the achieved throughput at RLC layer is close to the TBR. Having this in mind, the performance of the CQA scheduler is tested by using the same test cases as the `lte-pss-ff-mac-scheduler`. Additionally, in [Bbojovic2014] can be found performance evaluation of CQA scheduler when the GBR flows are delay sensitive by considering different QoE metrics.

Building Propagation Loss Model

The aim of the system test is to verify the integration of the `BuildingPathlossModel` with the `lte` module. The test exploits a set of three pre calculated losses for generating the expected SINR at the receiver counting the transmission and the noise powers. These SINR values are compared with the results obtained from a LTE simulation that uses the `BuildingPathlossModel`. The reference loss values are calculated off-line with an Octave script (`/test/reference/lte_pathloss.m`). Each test case passes if the reference loss value is equal to the value calculated by the simulator within a tolerance of 0.001 dB, which accounts for numerical errors in the calculations.

Physical Error Model

The test suite `lte-phy-error-model` generates different test cases for evaluating both data and control error models. For what concern the data, the test consists of six test cases with single eNB and a various number of UEs, all having the same Radio Bearer specification. Each test is designed for evaluating the error rate perceived by a specific TB size in order to verify that it corresponds to the expected values according to the BLER generated for CB size analog to the TB size. This means that, for instance, the test will check that the performance of a TB of N bits is analogous to the one of a CB size of N bits by collecting the performance of a user which has been forced the generation of a such TB size according to the distance to eNB. In order to significantly test the BLER at MAC level, we configured the Adaptive Modulation and Coding (AMC) module, the `LteAmc` class, for making it less robust to channel conditions by using the PiroEW2010 AMC model and configuring it to select the MCS considering a target BER of 0.03 (instead of the default value of 0.00005). We note that these values do not reflect the actual BER, since they come from an analytical bound which does not consider all the transmission chain aspects; therefore the BER and BLER actually experienced at the reception of a TB is in general different.

The parameters of the six test cases are reported in the following:

1. 4 UEs placed 1800 meters far from the eNB, which implies the use of MCS 2 (SINR of -5.51 dB) and a TB of 256 bits, that in turns produce a BLER of 0.33 (see point A in figure [BLER for tests 1, 2, 3.](#)).
2. 2 UEs placed 1800 meters far from the eNB, which implies the use of MCS 2 (SINR of -5.51 dB) and a TB of 528 bits, that in turns produce a BLER of 0.11 (see point B in figure [BLER for tests 1, 2, 3.](#)).
3. 1 UE placed 1800 meters far from the eNB, which implies the use of MCS 2 (SINR of -5.51 dB) and a TB of 1088 bits, that in turns produce a BLER of 0.02 (see point C in figure [BLER for tests 1, 2, 3.](#)).
4. 1 UE placed 600 meters far from the eNB, which implies the use of MCS 12 (SINR of 4.43 dB) and a TB of 4800 bits, that in turns produce a BLER of 0.3 (see point D in figure [BLER for tests 4, 5.](#)).
5. 3 UEs placed 600 meters far from the eNB, which implies the use of MCS 12 (SINR of 4.43 dB) and a TB of 1632 bits, that in turns produce a BLER of 0.55 (see point E in figure [BLER for tests 4, 5.](#)).
6. 1 UE placed 470 meters far from the eNB, which implies the use of MCS 16 (SINR of 8.48 dB) and a TB of 7272 bits (segmented in 2 CBs of 3648 and 3584 bits), that in turns produce a BLER of 0.14, since each CB has CBLER equal to 0.075 (see point F in figure [BLER for test 6.](#)).

The test condition verifies that in each test case the expected number of packets received correctly corresponds to a Bernoulli distribution with a confidence interval of 99%, where the probability of success in each trail is $p = 1 - BER$ and n is the total number of packets sent.

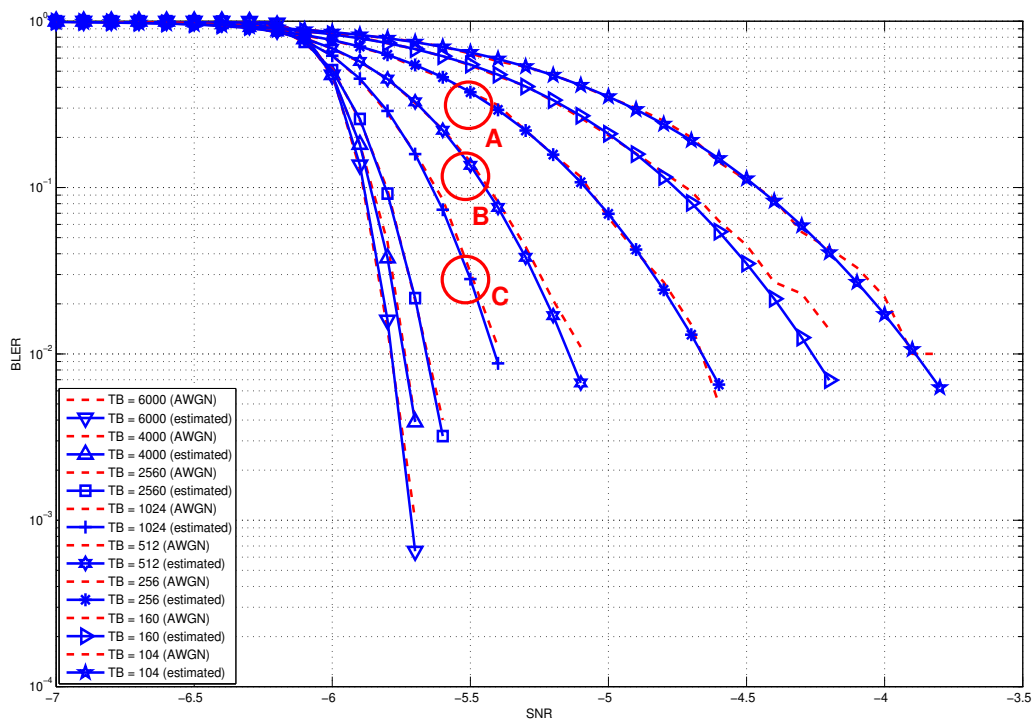


Fig. 19.94: BLER for tests 1, 2, 3.

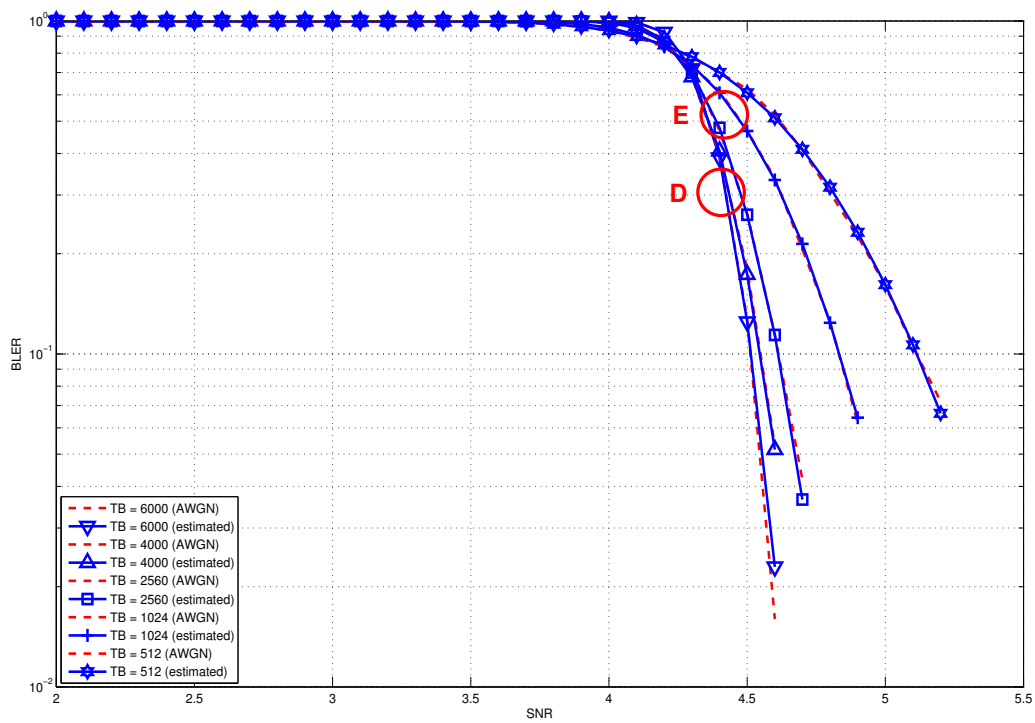


Fig. 19.95: BLER for tests 4, 5.

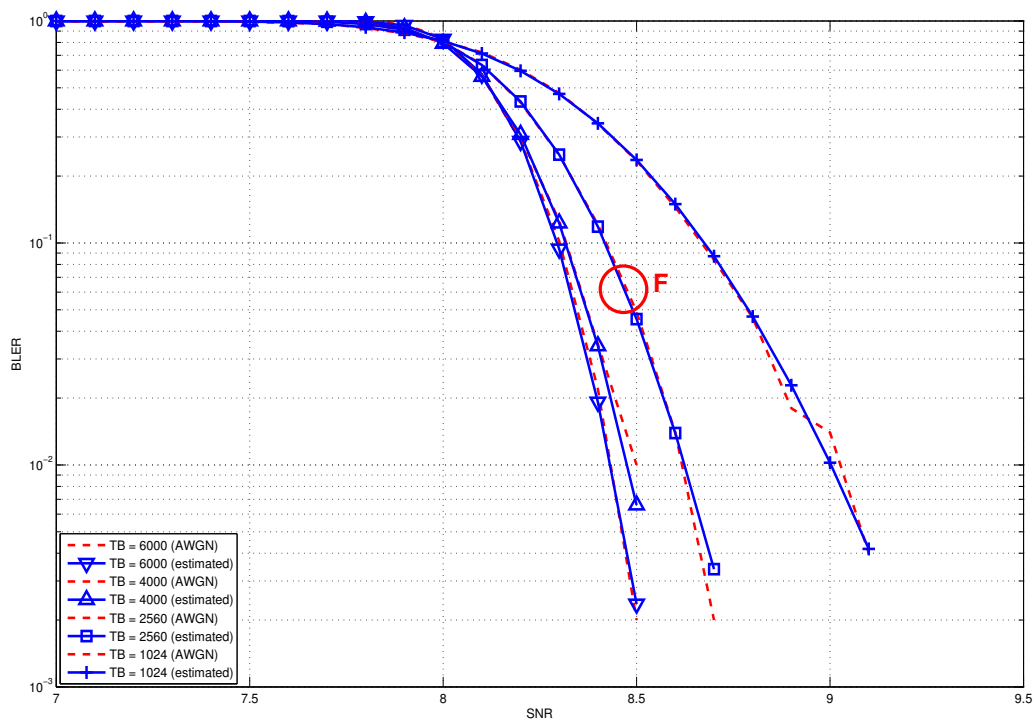


Fig. 19.96: BLER for test 6.

The error model of PCFICH-PDCCH channels consists of 4 test cases with a single UE and several eNBs, where the UE is connected to only one eNB in order to have the remaining acting as interfering ones. The errors on data are disabled in order to verify only the ones due to erroneous decodification of PCFICH-PDCCH. As before, the system has been forced on working in a less conservative fashion in the AMC module for appreciating the results in border situations. The parameters of the 4 tests cases are reported in the following:

1. 2 eNBs placed 1078 meters far from the UE, which implies a SINR of -2.00 dB and a TB of 217 bits, that in turns produce a BLER of 0.007.
2. 3 eNBs placed 1040 meters far from the UE, which implies a SINR of -4.00 dB and a TB of 217 bits, that in turns produce a BLER of 0.045.
3. 4 eNBs placed 1250 meters far from the UE, which implies a SINR of -6.00 dB and a TB of 133 bits, that in turns produce a BLER of 0.206.
4. 5 eNBs placed 1260 meters far from the UE, which implies a SINR of -7.00 dB and a TB of 81 bits, that in turns produce a BLER of 0.343.

The test condition verifies that in each test case the expected number of packets received correct corresponds to a Bernoulli distribution with a confidence interval of 99.8%, where the probability of success in each trail is $p = 1 - BER$ and n is the total number of packet sent. The larger confidence interval is due to the errors that might be produced in quantizing the MI and the error curve.

HARQ Model

The test suite `lte-harq` includes two tests for evaluating the HARQ model and the related extension in the error model. The test consists on checking whether the amount of bytes received during the simulation corresponds to the expected ones according to the values of transport block and the HARQ dynamics. In detail, the test checks whether the throughput obtained after one HARQ retransmission is the expected one. For evaluating the expected throughput the expected TB delivering time has been evaluated according to the following formula:

$$T = P_s^1 \times 1 + P_s^2 \times 2 + (1 - P_s^2) \times 3$$

where P_s^i is the probability of receiving with success the HARQ block at the attempt i (i.e., the RV with 3GPP naming). According to the scenarios, in the test we always have P_s^1 equal to 0.0, while P_s^2 varies in the two tests, in detail:

$$T_{\text{test-1}} = 0.0 \times 1 + 0.926 \times 2 + 0.074 \times 3 = 2.074$$

$$T_{\text{test-2}} = 0.0 \times 1 + 0.752 \times 2 + 0.248 \times 3 = 2.248$$

The expected throughput is calculated by counting the number of transmission slots available during the simulation (e.g., the number of TTIs) and the size of the TB in the simulation, in detail:

$$\text{Thr}_{\text{test-i}} = \frac{TTI_{\text{NUM}}}{T_{\text{test-i}}} TB_{\text{size}} = \begin{cases} \frac{1000}{2.074} 66 = 31822 \text{ bps} & \text{for test-1} \\ \frac{1000}{2.248} 472 = 209964 \text{ bps} & \text{for test-2} \end{cases}$$

where TTI_{NUM} is the total number of TTIs in 1 second. The test is performed both for Round Robin scheduler. The test passes if the measured throughput matches with the reference throughput within a relative tolerance of 0.1. This tolerance is needed to account for the transient behavior at the beginning of the simulation and the on-fly blocks at the end of the simulation.

MIMO Model

The test suite `lte-mimo` aims at verifying both the effect of the gain considered for each Transmission Mode on the system performance and the Transmission Mode switching through the scheduler interface. The test consists on checking whether the amount of bytes received during a certain window of time (0.1 seconds in our case) corresponds to the expected ones according to the values of transport block size reported in table 7.1.7.2.1-1 of [TS36213], similarly to what done for the tests of the schedulers.

The test is performed both for Round Robin and Proportional Fair schedulers. The test passes if the measured throughput matches with the reference throughput within a relative tolerance of 0.1. This tolerance is needed to account for the transient behavior at the beginning of the simulation and the transition phase between the Transmission Modes.

Antenna Model integration

The test suite `lte-antenna` checks that the `AntennaModel` integrated with the LTE model works correctly. This test suite recreates a simulation scenario with one eNB node at coordinates (0,0,0) and one UE node at coordinates (x,y,0). The eNB node is configured with an `CosineAntennaModel` having given orientation and beamwidth. The UE instead uses the default `IsotropicAntennaModel`. The test checks that the received power both in uplink and downlink account for the correct value of the antenna gain, which is determined offline; this is implemented by comparing the uplink and downlink SINR and checking that both match with the reference value up to a tolerance of 10^{-6} which accounts for numerical errors. Different test cases are provided by varying the x and y coordinates of the UE, and the beamwidth and the orientation of the antenna of the eNB.

RLC

Two test suites `lte-rlc-um-transmitter` and `lte-rlc-am-transmitter` check that the UM RLC and the AM RLC implementation work correctly. Both these suites work by testing RLC instances connected to special test entities that play the role of the MAC and of the PDCP, implementing respectively the `LteMacSapProvider` and `LteRlcSapUser` interfaces. Different test cases (i.e., input test vector consisting of series of primitive calls by the MAC and the PDCP) are provided that check the behavior in the following cases:

1. one SDU, one PDU: the MAC notifies a TX opportunity causes the creation of a PDU which exactly contains a whole SDU
2. segmentation: the MAC notifies multiple TX opportunities that are smaller than the SDU size stored in the transmission buffer, which is then to be fragmented and hence multiple PDUs are generated;
3. concatenation: the MAC notifies a TX opportunity that is bigger than the SDU, hence multiple SDUs are concatenated in the same PDU
4. buffer status report: a series of new SDUs notifications by the PDCP is interleaved with a series of TX opportunity notification in order to verify that the buffer status report procedure is correct.

In all these cases, an output test vector is determined manually from knowledge of the input test vector and knowledge of the expected behavior. These test vectors are specialized for UM RLC and AM RLC due to their different behavior. Each test case passes if the sequence of primitives triggered by the RLC instance being tested is exactly equal to the output test vector. In particular, for each PDU transmitted by the RLC instance, both the size and the content of the PDU are verified to check for an exact match with the test vector.

The AM RLC implementation features an additional test suite, `lte-rlc-am-e2e`, which tests the correct retransmission of RLC PDUs in presence of channel losses. The test instantiates an RLC AM transmitter and a receiver, and interposes a channel that randomly drops packets according to a fixed loss probability. Different test cases are instantiated using different `RngRun` values and different loss probability values. Each test case passes if at the end of the simulation all SDUs are correctly delivered to the upper layers of the receiving RLC AM entity.

RRC

The test suite `lte-rrc` tests the correct functionality of the following aspects:

1. MAC Random Access
2. RRC System Information Acquisition
3. RRC Connection Establishment
4. RRC Reconfiguration

The test suite considers a type of scenario with four eNBs aligned in a square layout with 100-meter edges. Multiple UEs are located at a specific spot on the diagonal of the square and are instructed to connect to the first eNB. Each test case implements an instance of this scenario with specific values of the following parameters:

- number of UEs
- number of Data Radio Bearers to be activated for each UE
- time t_0^c at which the first UE is instructed to start connecting to the eNB
- time interval d^i between the start of connection of UE n and UE $n + 1$; the time at which user n connects is thus determined as $t_n^c = t_0^c + nd^i$ sdf
- the relative position of the UEs on the diagonal of the square, where higher values indicate larger distance from the serving eNodeB, i.e., higher interference from the other eNodeBs
- a boolean flag indicating whether the ideal or the real RRC protocol model is used

Each test cases passes if a number of test conditions are positively evaluated for each UE after a delay d^e from the time it started connecting to the eNB. The delay d^e is determined as

$$d^e = d^{si} + d^{ra} + d^{ce} + d^{cr}$$

where:

- d^{si} is the max delay necessary for the acquisition of System Information. We set it to 90ms accounting for 10ms for the MIB acquisition and 80ms for the subsequent SIB2 acquisition
- d^{ra} is the delay for the MAC Random Access (RA) procedure. This depends on preamble collisions as well as on the availability of resources for the UL grant allocation. The total amount of necessary RA attempts depends on preamble collisions and failures to allocate the UL grant because of lack of resources. The number of collisions depends on the number of UEs that try to access simultaneously; we estimated that for a 0.99 RA success probability, 5 attempts are sufficient for up to 20 UEs, and 10 attempts for up to 50 UEs. For the UL grant, considered the system bandwidth and the default MCS used for the UL grant (MCS 0), at most 4 UL grants can be assigned in a TTI; so for n UEs trying to do RA simultaneously the max number of attempts due to the UL grant issue is $\lceil n/4 \rceil$. The time for a RA attempt is determined by 3ms + the value of `LteEnbMac::RaResponseWindowSize`, which defaults to 3ms, plus 1ms for the scheduling of the new transmission.
- d^{ce} is the delay required for the transmission of RRC CONNECTION SETUP + RRC CONNECTION SETUP COMPLETED. We consider a round trip delay of 10ms plus $\lceil 2n/4 \rceil$ considering that 2 RRC packets have to be transmitted and that at most 4 such packets can be transmitted per TTI. In cases where interference is high, we accommodate one retry attempt by the UE, so we double the d^{ce} value and then add d^{si} on top of it (because the timeout has reset the previously received SIB2).
- d^{cr} is the delay required for eventually needed RRC CONNECTION RECONFIGURATION transactions. The number of transactions needed is 1 for each bearer activation. Similarly to what done for d^{ce} , for each transaction we consider a round trip delay of 10ms plus $\lceil 2n/4 \rceil$. delay of 20ms.

The base version of the test `LteRrcConnectionEstablishmentTestCase` tests for correct RRC connection establishment in absence of channel errors. The conditions that are evaluated for this test case to pass are, for each UE:

- the RRC state at the UE is `CONNECTED_NORMALLY`
- the UE is configured with the `CellId`, `DLBandwidth`, `ULBandwidth`, `DLEarfcn` and `ULEarfcn` of the eNB
- the IMSI of the UE stored at the eNB is correct
- the number of active Data Radio Bearers is the expected one, both at the eNB and at the UE
- for each Data Radio Bearer, the following identifiers match between the UE and the eNB: EPS bearer id, DRB id, LCID

The test variant `LteRrcConnectionEstablishmentErrorTestCase` is similar except for the presence of errors in the transmission of a particular RRC message of choice during the first connection attempt. The error is obtained by temporarily moving the UE to a far away location; the time of movement has been determined empirically for each instance of the test case based on the message that it was desired to be in error. The test case checks that at least one of the following conditions is false at the time right before the UE is moved back to the original location:

- the RRC state at the UE is `CONNECTED_NORMALLY`
- the UE context at the eNB is present
- the RRC state of the UE Context at the eNB is `CONNECTED_NORMALLY`

Initial cell selection

The test suite *lte-cell-selection* is responsible for verifying the *Initial Cell Selection* procedure. The test is a simulation of a small network of 2 non-CSG cells and 2 non-CSG cells. Several static UEs are then placed at predefined locations. The UEs enter the simulation without being attached to any cell. Initial cell selection is enabled for these UEs, so each UE will find the best cell and attach to it by themselves.

At predefined check points time during the simulation, the test verifies that every UE is attached to the right cell. Moreover, the test also ensures that the UE is properly connected, i.e., its final state is `CONNECTED_NORMALLY`. Figure *Sample result of cell selection test* depicts the network layout and the expected result. When a UE is depicted as having 2 successful cell selections (e.g., UE #3 and #4), any of them is accepted by the test case.

The figure shows that CSG members may attach to either CSG or non-CSG cells, and simply choose the stronger one. On the other hand, non-members can only attach to non-CSG cells, even when they are actually receiving stronger signal from a CSG cell.

For reference purpose, Table *UE error rate in Initial Cell Selection test* shows the error rate of each UE when receiving transmission from the control channel. Based on this information, the check point time for UE #3 is done at a later time than the others to compensate for its higher risk of failure.

Table 19.10: UE error rate in Initial Cell Selection test

UE #	Error rate
1	0.00%
2	1.44%
3	12.39%
4	0.33%
5	0.00%
6	0.00%

The test uses the default Friis path loss model and without any channel fading model.

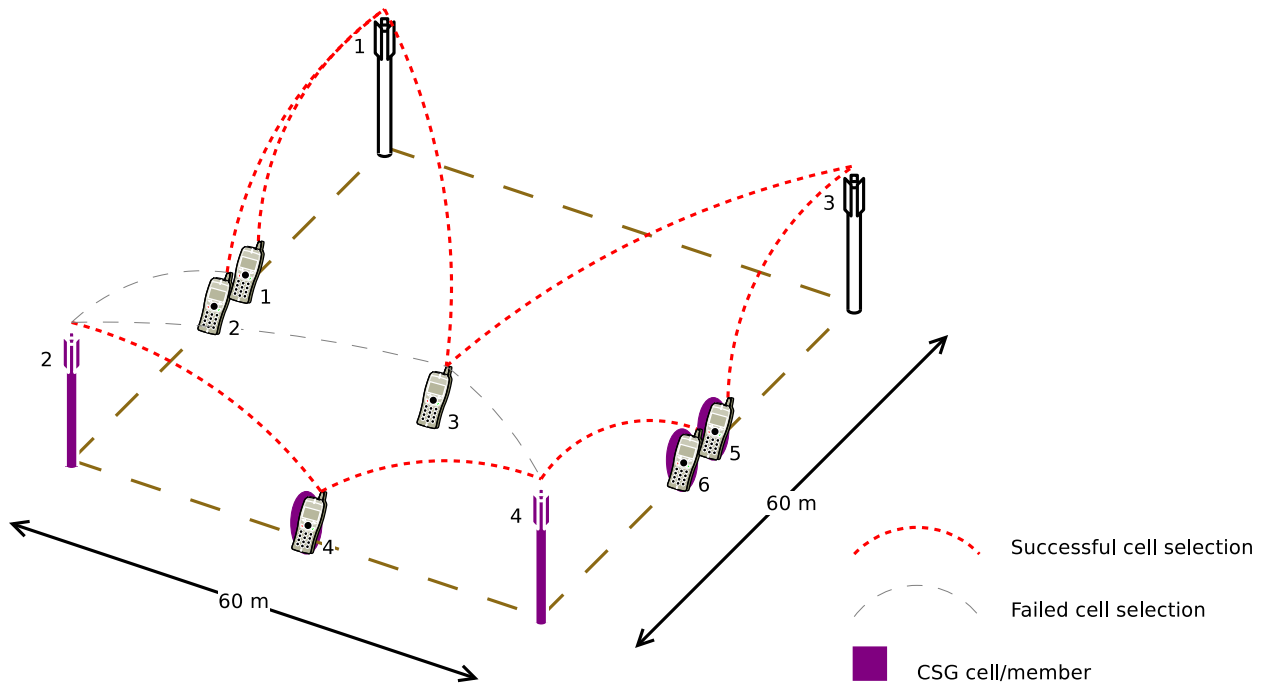


Fig. 19.97: Sample result of cell selection test

Secondary cell selection

The unit test `lte-secondary-cell-selection`` tests that a UE can attach to any component carrier (not just the 0-th one) during initial cell selection.

Handover with multiple component carriers

The unit test suite `lte-primary-cell-change` tests a number of handover cases between different component carriers, for both the Ideal RRC and the Real RRC. The following handover cases are tested:

- (inter-eNB) eNB to eNB with one component carrier
- (inter-eNB) handover between the first carrier of two eNBs
- (inter-eNB) handover between the second carrier of one eNB to the first carrier of another eNB
- (inter-eNB) handover between the second carrier of one eNB to the second carrier of another eNB
- (intra-eNB) three handover cases between component carriers of the same eNB

GTP-U protocol

The unit test suite `epc-gtpu` checks that the encoding and decoding of the GTP-U header is done correctly. The test fills in a header with a set of known values, adds the header to a packet, and then removes the header from the packet. The test fails if, upon removing, any of the fields in the GTP-U header is not decoded correctly. This is detected by comparing the decoded value from the known value.

S1-U interface

Two test suites (`epc-s1u-uplink` and `epc-s1u-downlink`) make sure that the S1-U interface implementation works correctly in isolation. This is achieved by creating a set of simulation scenarios where the EPC model alone is used, without the LTE model (i.e., without the LTE radio protocol stack, which is replaced by simple CSMA devices). This checks that the interoperation between multiple `EpcEnbApplication` instances in multiple eNBs and the `EpcSgwPg-wApplication` instance in the SGW/PGW node works correctly in a variety of scenarios, with varying numbers of end users (nodes with a CSMA device installed), eNBs, and different traffic patterns (packet sizes and number of total packets). Each test case works by injecting the chosen traffic pattern in the network (at the considered UE or at the remote host for in the uplink or the downlink test suite respectively) and checking that at the receiver (the remote host or each considered UE, respectively) that exactly the same traffic patterns is received. If any mismatch in the transmitted and received traffic pattern is detected for any UE, the test fails.

TFT classifier

The test suite `epc-tft-classifier` checks in isolation that the behavior of the `EpcTftClassifier` class is correct. This is performed by creating different classifier instances where different TFT instances are activated, and testing for each classifier that an heterogeneous set of packets (including IP and TCP/UDP headers) is classified correctly. Several test cases are provided that check the different matching aspects of a TFT (e.g. local/remote IP address, local/remote port) both for uplink and downlink traffic. Each test case corresponds to a specific packet and a specific classifier instance with a given set of TFTs. The test case passes if the bearer identifier returned by the classifier exactly matches with the one that is expected for the considered packet.

End-to-end LTE-EPC data plane functionality

The test suite `lte-epc-e2e-data` ensures the correct end-to-end functionality of the LTE-EPC data plane. For each test case in this suite, a complete LTE-EPC simulation scenario is created with the following characteristics:

- a given number of eNBs
- for each eNB, a given number of UEs
- for each UE, a given number of active EPS bearers
- for each active EPS bearer, a given traffic pattern (number of UDP packets to be transmitted and packet size)

Each test is executed by transmitting the given traffic pattern both in the uplink and in the downlink, at subsequent time intervals. The test passes if all the following conditions are satisfied:

- for each active EPS bearer, the transmitted and received traffic pattern (respectively at the UE and the remote host for uplink, and vice versa for downlink) is exactly the same
- for each active EPS bearer and each direction (uplink or downlink), exactly the expected number of packet flows over the corresponding `RadioBearer` instance

X2 handover

The test suite `lte-x2-handover` checks the correct functionality of the X2 handover procedure. The scenario being tested is a topology with two eNBs connected by an X2 interface. Each test case is a particular instance of this scenario defined by the following parameters:

- the number of UEs that are initially attached to the first eNB
- the number of EPS bearers activated for each UE
- a list of handover events to be triggered, where each event is defined by: + the start time of the handover trigger + the index of the UE doing the handover + the index of the source eNB + the index of the target eNB
- a boolean flag indicating whether the target eNB admits the handover or not
- a boolean flag indicating whether the ideal RRC protocol is to be used instead of the real RRC protocol
- the type of scheduler to be used (RR or PF)

Each test case passes if the following conditions are true:

- at time 0.06s, the test `CheckConnected` verifies that each UE is connected to the first eNB
- for each event in the handover list:
 - at the indicated event start time, the indicated UE is connected to the indicated source eNB
 - 0.1s after the start time, the indicated UE is connected to the indicated target eNB
 - 0.6s after the start time, for each active EPS bearer, the uplink and downlink sink applications of the indicated UE have achieved a number of bytes which is at least half the number of bytes transmitted by the corresponding source applications

The condition “UE is connected to eNB” is evaluated positively if and only if all the following conditions are met:

- the eNB has the context of the UE (identified by the RNTI value retrieved from the UE RRC)
- the RRC state of the UE at the eNB is `CONNECTED_NORMALLY`
- the RRC state at the UE is `CONNECTED_NORMALLY`
- the UE is configured with the `CellId`, `DlBandwidth`, `UplinkBandwidth`, `DlEarfcn` and `UplinkEarfcn` of the eNB
- the IMSI of the UE stored at the eNB is correct
- the number of active Data Radio Bearers is the expected one, both at the eNB and at the UE
- for each Data Radio Bearer, the following identifiers match between the UE and the eNB: EPS bearer id, DRB id, LCID

Automatic X2 handover

The test suite `lte-x2-handover-measures` checks the correct functionality of the handover algorithm. The scenario being tested is a topology with two, three or four eNBs connected by an X2 interface. The eNBs are located in a straight line in the X-axis. A UE moves along the X-axis going from the neighborhood of one eNB to the next eNB. Each test case is a particular instance of this scenario defined by the following parameters:

- the number of eNBs in the X-axis
- the number of UEs
- the number of EPS bearers activated for the UE

- a list of check point events to be triggered, where each event is defined by: + the time of the first check point event + the time of the last check point event + interval time between two check point events + the index of the UE doing the handover + the index of the eNB where the UE must be connected
- a boolean flag indicating whether UDP traffic is to be used instead of TCP traffic
- the type of scheduler to be used
- the type of handover algorithm to be used
- a boolean flag indicating whether handover is admitted by default
- a boolean flag indicating whether the ideal RRC protocol is to be used instead of the real RRC protocol

The test suite consists of many test cases. In fact, it has been one of the most time-consuming test suite in ns-3. The test cases run with *some* combination of the following variable parameters:

- number of eNBs: 2, 3, 4;
- number of EPS bearers: 0, 1, 2;
- RRC: ideal, real (see [RRC protocol models](#));
- MAC scheduler: round robin, proportional fair (see [The FemtoForum MAC Scheduler Interface](#)); and
- handover algorithm: A2-A4-RSRQ, strongest cell (see [Handover algorithm](#)).

Each test case passes if the following conditions are true:

- at time 0.08s, the test CheckConnected verifies that each UE is connected to the first eNB
- for each event in the check point list:
 - at the indicated check point time, the indicated UE is connected to the indicated eNB
 - 0.5s after the check point, for each active EPS bearer, the uplink and downlink sink applications of the UE have achieved a number of bytes which is at least half the number of bytes transmitted by the corresponding source applications

The condition “UE is connected to eNB” is evaluated positively if and only if all the following conditions are met:

- the eNB has the context of the UE (identified by the RNTI value retrieved from the UE RRC)
- the RRC state of the UE at the eNB is CONNECTED_NORMALLY
- the RRC state at the UE is CONNECTED_NORMALLY
- the UE is configured with the CellId, DIBandwidth, UIBandwidth, DLEarfcn and ULEarfcn of the eNB
- the IMSI of the UE stored at the eNB is correct
- the number of active Data Radio Bearers is the expected one, both at the eNB and at the UE
- for each Data Radio Bearer, the following identifiers match between the UE and the eNB: EPS bearer id, DRB id, LCID

Handover delays

Handover procedure consists of several message exchanges between UE, source eNodeB, and target eNodeB over both RRC protocol and X2 interface. Test suite `lte-handover-delay` verifies that this procedure consistently spends the same amount of time.

The test suite will run several handover test cases. Each test case is an individual simulation featuring a handover at a specified time in simulation. For example, the handover in the first test case is invoked at time +0.100s, while in the second test case it is at +0.101s. There are 10 test cases, each testing a different subframe in LTE. Thus the last test case has the handover at +0.109s.

The simulation scenario in the test cases is as follow:

- EPC is enabled
- 2 eNodeBs with circular (isotropic) antenna, separated by 1000 meters
- 1 static UE positioned exactly in the center between the eNodeBs
- no application installed
- no channel fading
- default path loss model (Friis)
- 0.300s simulation duration

The test case runs as follow. At the beginning of the simulation, the UE is attached to the first eNodeB. Then at the time specified by the test case input argument, a handover request will be explicitly issued to the second eNodeB. The test case will then record the starting time, wait until the handover is completed, and then record the completion time. If the difference between the completion time and starting time is less than a predefined threshold, then the test passes.

A typical handover in the current ns-3 implementation takes 4.2141 ms when using Ideal RRC protocol model, and 19.9283 ms when using Real RRC protocol model. Accordingly, the test cases use 5 ms and 20 ms as the maximum threshold values. The test suite runs 10 test cases with Ideal RRC protocol model and 10 test cases with Real RRC protocol model. More information regarding these models can be found in Section [RRC protocol models](#).

The motivation behind using subframes as the main test parameters is the fact that subframe index is one of the factors for calculating RA-RNTI, which is used by Random Access during the handover procedure. The test cases verify this computation, utilizing the fact that the handover will be delayed when this computation is broken. In the default simulation configuration, the handover delay observed because of a broken RA-RNTI computation is typically 6 ms.

Handover failure

The test suite `lte-handover-failure` tests the proper operation of a number of handover failure cases by inducing the conditions leading to the following eight failure modes:

1. Maximum number of RACH transmissions exceeded from UE to target eNB
2. Non-allocation of non-contention-based preamble at the target eNB, due to the maximum number reached
3. HANDOVER JOINING timeout before reception of RRC CONNECTION RECONFIGURATION at source eNB
4. HANDOVER JOINING timeout before completion of non-contention RACH process to target eNB
5. HANDOVER JOINING timeout before reception of RRC CONNECTION RECONFIGURATION COMPLETE at target eNB
6. HANDOVER LEAVING timeout before reception of RRC CONNECTION RECONFIGURATION at source eNB

7. HANDOVER LEAVING timeout before completion of non-contention RACH process to target eNB
8. HANDOVER LEAVING timeout before reception of RRC CONNECTION RECONFIGURATION COMPLETE attarget eNB

Both Ideal and Real RRC models are checked by this test suite.

Selection of target cell in handover algorithm

eNodeB may utilize *Handover algorithm* to automatically create handover decisions during simulation. The decision includes the UE which should do the handover and the target cell where the UE should perform handover to.

The test suite `lte-handover-target` verifies that the handover algorithm is making the right decision, in particular, in choosing the right target cell. It consists of several short test cases for different network topology (2×2 grid and 3×2 grid) and types of handover algorithm (the A2-A4-RSRQ handover algorithm and the strongest cell handover algorithm).

Each test case is a simulation of a micro-cell environment with the following parameter:

- EPC is enabled
- several circular (isotropic antenna) micro-cell eNodeBs in a rectangular grid layout, with 130 m distance between each adjacent point
- 1 static UE, positioned close to and attached to the source cell
- no control channel error model
- no application installed
- no channel fading
- default path loss model (Friis)
- 1s simulation duration

To trigger a handover, the test case “shutdowns” the source cell at +0.5s simulation time. Figure *lte-handover-target test scenario in a 2×2 grid* below illustrates the process. This is done by setting the source cell’s Tx power to a very low value. As a result, the handover algorithm notices that the UE deserves a handover and several neighboring cells become candidates of target cell at the same time.

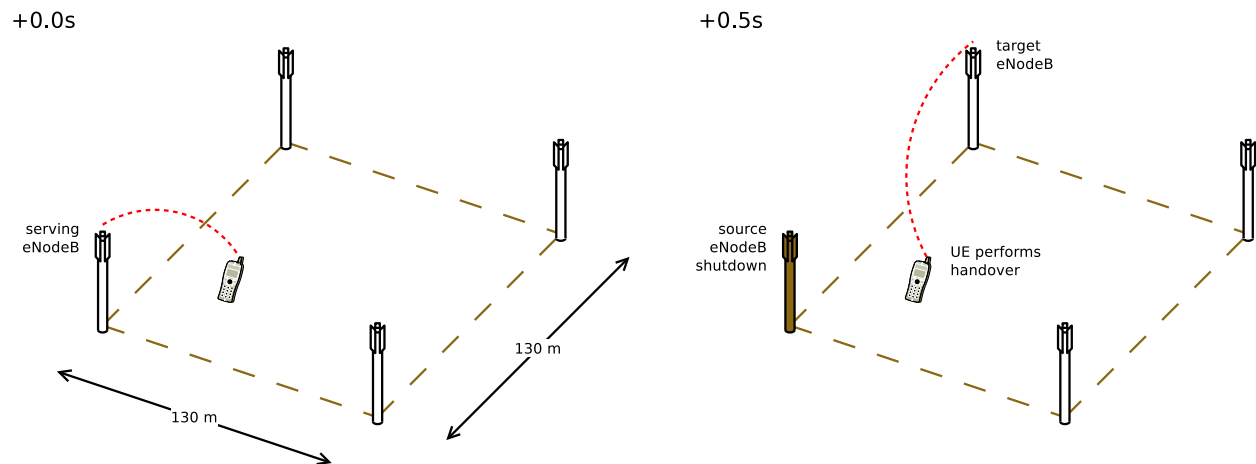


Fig. 19.98: lte-handover-target test scenario in a 2×2 grid

The test case then verifies that the handover algorithm, when faced with more than one options of target cells, is able to choose the right one.

Downlink Power Control

The test suite `lte-downlink-power-control` checks correctness of Downlink Power Control in three different ways:

- `LteDownlinkPowerControlSpectrumValue` test case check if `LteSpectrumValueHelper::CreateTxPowerSpectralDensity` is creating correct spectrum value for PSD for downlink transmission. The test vector contain EARFCN, system bandwidth, TX power, TX power for each RB, active RBs, and expected TxPSD. The test passes if TxPDS generated by `LteSpectrumValueHelper::CreateTxPowerSpectralDensity` is equal to expected TxPSD.
- `LteDownlinkPowerControlTestCase` test case check if TX power difference between data and control channel is equal to configured `PdschConfigDedicated::P_A` value. TX power of control channel is measured by `LteTestSinrChunkProcessor` added to `RsPowerChunkProcessor` list in UE `DownlinkSpectrumPhy`. Tx power of data channel is measured in similar way, but it had to be implemented. Now `LteTestSinrChunkProcessor` is added to `DataPowerChunkProcessor` list in UE `DownlinkSpectrumPhy`. Test vector contain a set of all available `P_A` values. Test pass if power difference equals `P_A` value.
- `LteDownlinkPowerControlRrcConnectionReconfiguration` test case check if `RrcConnectionReconfiguration` is performed correctly. When FR entity gets UE measurements, it immediately calls function to change `P_A` value for this UE and also triggers callback connected with this event. Then, test check if UE gets `RrcConnectionReconfiguration` message (it trigger callback). Finally, it checks if eNB receive `RrcConnectionReconfigurationCompleted` message, what also trigger callback. The test passes if all event have occurred. The test is performed two times, with `IdealRrcProtocol` and with `RealRrcProtocol`.

Uplink Power Control Tests

UE uses Uplink Power Control to automatically change Tx Power level for Uplink Physical Channels. Tx Power is computed based on path-loss, number of RB used for transmission, some configurable parameters and TPC command from eNB.

The test suite `lte-uplink-power-control` verifies if Tx Power is computed correctly. There are three different test cases:

- `LteUplinkOpenLoopPowerControlTestCase` test case checks Uplink Power Control functionality in Open Loop mechanism. UE is attached to eNB and is transmitting data in Downlink and Uplink. Uplink Power Control with Open Loop mechanism is enabled and UE changes position each 100 ms. In each position Uplink Power Control entity is calculating new Tx Power level for all uplink channels. These values are traced and test passes if Uplink Tx Power for PUSCH, PUCCH and SRS in each UE position are equal to expected values.
- `LteUplinkClosedLoopPowerControlAbsoluteModeTestCase` test case checks Uplink Power Control functionality with Closed Loop mechanism and Absolute Mode enabled. UE is attached to eNB and is transmitting data in Downlink and Uplink. Uplink Power Control with Closed Loop mechanism and Absolute Mode is enabled. UE is located 100 m from eNB and is not changing its position. `LteFfrSimple` algorithm is used on eNB side to set TPC values in DL-DCI messages. TPC configuration in eNB is changed every 100 ms, so every 100 ms Uplink Power Control entity in UE should calculate different Tx Power level for all uplink channels. These values are traced and test passes if Uplink Tx Power for PUSCH, PUCCH and SRS computed with all TCP values are equal to expected values.
- `LteUplinkClosedLoopPowerControlAccumulatedModeTestCase` test case checks Closed Loop Uplink Power Control functionality with Closed Loop mechanism and Accumulative Mode enabled. UE is attached to eNB and is transmitting data in Downlink and Uplink. Uplink Power Control with Closed Loop mechanism and Accumulative Mode is enabled. UE is located 100 m from eNB and is not changing its position. As in above test case, `LteFfrSimple` algorithm is used on eNB side to set TPC values in DL-DCI messages, but in this case TPC

command are set in DL-DCI only configured number of times, and after that TPC is set to be 1, what is mapped to value of 0 in Accumulative Mode (TS36.213 Table 5.1.1.1-2). TPC configuration in eNB is changed every 100 ms. UE is accumulating these values and calculates Tx Power levels for all uplink channels based on accumulated value. If computed Tx Power level is lower than minimal UE Tx Power, UE should transmit with its minimal Tx Power. If computed Tx Power level is higher than maximal UE Tx Power, UE should transmit with its maximal Tx Power. Tx Power levels for PUSCH, PUCCH and SRS are traced and test passes if they are equal to expected values.

Frequency Reuse Algorithms

The test suite `lte-frequency-reuse` contain two types of test cases.

First type of test cases check if RBGs are used correctly according to FR algorithm policy. We are testing if scheduler use only RBGs allowed by FR configuration. To check which RBGs are used `LteSimpleSpectrumPhy` is attached to Downlink Channel. It notifies when data downlink channel transmission has occurred and pass signal `TxPsd` spectrum value to check which RBs were used for transmission. The test vector comprise a set of configuration for Hard and Strict FR algorithms (there is no point to check other FR algorithms in this way because they use entire cell bandwidth). Test passes if none of not allowed RBGs are used.

Second type of test cases check if UE is served within proper sub-band and with proper transmission power. In this test scenario, there are two eNBs. There are also two UEs and each eNB is serving one. One uses Frequency Reuse algorithm and second one does not. Second eNB is responsible for generating interferences in whole system bandwidth. UE served by first eNB is changing position each few second (rather slow because time is needed to report new UE Measurements). To check which RBGs are used for this UE `LteSimpleSpectrumPhy` is attached to Downlink Channel. It notifies when data downlink channel transmission in cell 1 has occurred and pass signal `TxPsd` spectrum value to check which RBs were used for transmission and their power level. The same approach is applied in Uplink direction and second `LteSimpleSpectrumPhy` is attached to Uplink Channel. Test passes if UE served by eNB with FR algorithm is served in DL and UL with expected RBs and with expected power level. Test vector comprise a configuration for Strict FR, Soft FR, Soft FFR, Enhanced FFR. Each FR algorithm is tested with all schedulers, which support FR (i.e. PF, PSS, CQA, TD-TBFQ, FD-TBFQ). (Hard FR do not use UE measurements, so there is no point to perform this type of test for Hard FR).

Test case for Distributed FFR algorithm is quite similar to above one, but since eNBs need to exchange some information, scenario with EPC enabled and X2 interfaces is considered. Moreover, both eNB are using Distributed FFR algorithm. There are 2 UE in first cell, and 1 in second cell. Position of each UE is changed (rather slow because time is needed to report new UE Measurements), to obtain different result from calculation in Distributed FFR algorithm entities. To check which RBGs are used for UE transmission `LteSimpleSpectrumPhy` is attached to Downlink Channel. It notifies when data downlink channel transmission has occurred and pass signal `TxPsd` spectrum value to check which RBs were used for transmission and their power level. The same approach is applied in Uplink direction and second `LteSimpleSpectrumPhy` is attached to Uplink Channel. Test passes if UE served by eNB in cell 2, is served in DL and UL with expected RBs and with expected power level. Test vector comprise a configuration for Distributed FFR. Test is performed with all schedulers, which support FR (i.e. PF, PSS, CQA, TD-TBFQ, FD-TBFQ).

Inter-cell Interference with FR algorithms Tests

The test suite `lte-interference-fr` is very similar to `lte-interference`. Topology (Figure *Topology for the inter-cell interference test*) is the same and test checks interference level. The difference is that, in this test case Frequency Reuse algorithms are enabled and we are checking interference level on different RBGs (not only on one). For example, when we install Hard FR algorithm in eNBs, and first half of system bandwidth is assigned to one eNB, and second half to second eNB, interference level should be much lower compared to legacy scenario. The test vector comprise a set of configuration for all available Frequency Reuse Algorithms. Test passes if calculated SINR on specific RBs is equal to these obtained by Octave script.

Carrier aggregation test

The test suite `lte-carrier-aggregation` is a system test program that creates different test cases with a single eNB and several UEs, all having the same radio bearer specification. Different test cases are implemented by using different SINR values and different numbers of UEs. eNBs and UEs are configured to use the secondary carrier and the component carrier manager is configured to split the data uniformly between primary and secondary carrier. The test consists of checking that the throughput obtained over the different carriers are equal considering a given tolerance. For more details about this test, see the section Carrier aggregation usage example.

Carrier aggregation test for eNB and UE configuration

The test suite `carrier-aggregation-config-test` is a system test program, which verifies the following two cases:

- When carrier aggregation is enabled and UE carriers configuration is different than the default configuration done in `LteHelper`, we check that the UE(s) is configured properly once it receives RRC Connection Reconfiguration message from eNB.
- A user can configure 2 or more eNBs and UEs with different configuration parameters, i.e., each eNB and UE can have different EARFCN and Bandwidths and a UE connects to an eNB with similar DL EARFCN. In this test, we check with CA enabled but the end results will be the same if carrier aggregation is not enabled and we have more than one eNBs and UEs with different configurations.

Since, we do not need EPC to test the configuration, this test only simulates the LTE radio access with RLC SM. There are two test cases, Test 1 tests that the UE is configured properly after receiving RRC Connection Reconfiguration message from the eNB, which will overwrite UE default configuration done in `LteHelper` for the sake of creating PHY and MAC instances equal to the number of component carriers. Test 2 tests that every eNB or UE in a simulation scenario can be configured with different EARFCNs and Bandwidths. For both test cases, it also counts the number of times the hooked trace source `SCarrierConfigured` get triggered. As, it reflects how many UEs got attached to their respective eNB. If the count is not equal to the number of UEs in the scenario, the test fails, which could be the result of improper UE configuration.

Radio link failure Test

The test suite `lte-radio-link-failure` is a system test, which tests the radio link failure functionality using Ideal and Real RRC protocols. In particular, it tests the following to verify the Radio link Failure (RLF) implementation.

1. The state and the configuration of the UE while it is connected to the eNB.
2. The state of the UE while T310 timer is running at the UE.
3. The number of out-of-sync and in-synch indications received.
4. The state of the UE before the simulation end.
5. The UE context existence at the eNB before the simulation end.

This test simulates only one static UE with EPC performing downlink and uplink communication in the following two scenarios:

One eNB using Ideal and Real RRC

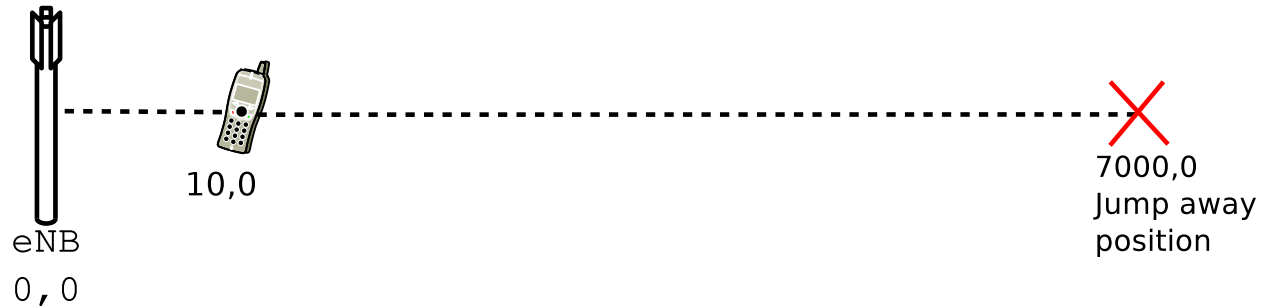


Fig. 19.99: RLF scenario with one eNB

In this scenario, the UE is initially placed near to the eNB, and on the following instances above conditions are verified against the expected outcome.

At 0.3 sec: It verifies that the UE is well connected, i.e., it is in “CONNECTED_NORMALLY” state, and is attached to the eNB with cell id 1. It also checks for the match between the configuration of the UE and the UE context at the eNB, e.g., IMSI, bandwidth, D/UL EARFCN, number of bearers and the bearer IDs. The miss match would result in the test suite failure.

At 0.4 sec: The UE jumps far away from the eNB, which causes the DL SINR at the UE to fall below -5 dB. In result, the UE PHY after monitoring the SINR for 20 consecutive frames will send a notification to the UE RRC. In this test, the N310 counter is set to 1; thus, the UE RRC will start the T310 (set to 1 sec) timer upon the first notification from the PHY layer.

At 1 sec: At this stage, it is expected that the T310 timer is still running, and the UE is connected to the eNB.

Upon RLF: It is expected that the UE RRC will start the T310 timer upon reaching the configured, i.e., N310 = 1 number of notification from the eNB. The RRC will receive no in-sync indication since the UE stays at far away position.

Before the end of simulation: The expected behavior is that the UE state will be in “IDLE_CELL_SEARCH” since there is no eNB available where it has jumped. Moreover, the deletion of the UE context from the eNB is also verified.

Two eNBs using Ideal and Real RRC

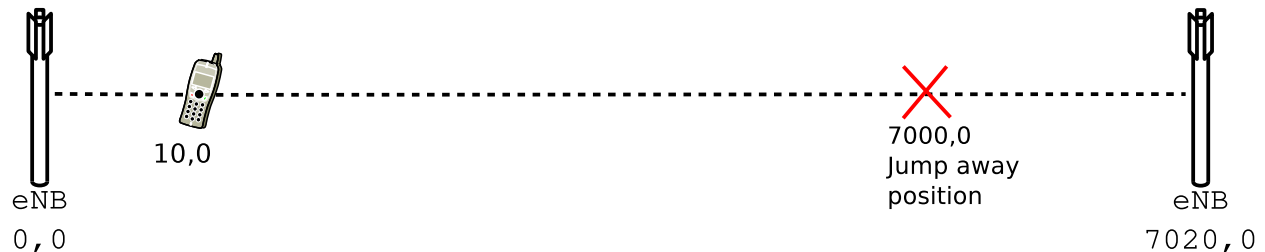


Fig. 19.100: RLF scenario with two eNBs

In this scenario, the only difference is the addition of a second eNB near the position where the UE jumps away. Therefore, except the outcome before the end of the simulation, all the outcomes are similar to that we expected in the first scenario.

Before the end of simulation: It is expected that the UE after the RLF will connect to the second eNB, i.e., it will be in “CONNECTED_NORMALLY” state, and its context exists in the second eNB.

19.4 Profiling Documentation

19.4.1 Overview and objectives

The main objective of the profiling carried out is to assess the simulator performance on a broad set of scenarios. This evaluation provides reference values for simulation running times and memory consumption figures. It also helps to identify potential performance improvements and to check for scalability problems when increasing the number of eNodeB and UEs attached to those.

In the following sections, a detailed description of the general profiling framework employed to perform the study is introduced. It also includes details on the main performed tests and its results evaluation.

19.4.2 Framework description

Simulation scripts

The simulation script used for all the E-UTRAN results showed in this documentation is located at `src/lte/examples/lena-profiling.cc`. It uses the complete PHY and MAC UE/eNodeB implementation with a simplified RLC implementation on top. This script generates a squared grid topology, placing a eNodeB at the centre of each square. UEs attached to this node are scattered randomly across the square (using a random uniform distribution along X and Y axis). If *BuildingPropagationModel* is used, the squares are replaced by rooms. To generate the UL and DL traffic, the RLC implementation always report data to be transferred.

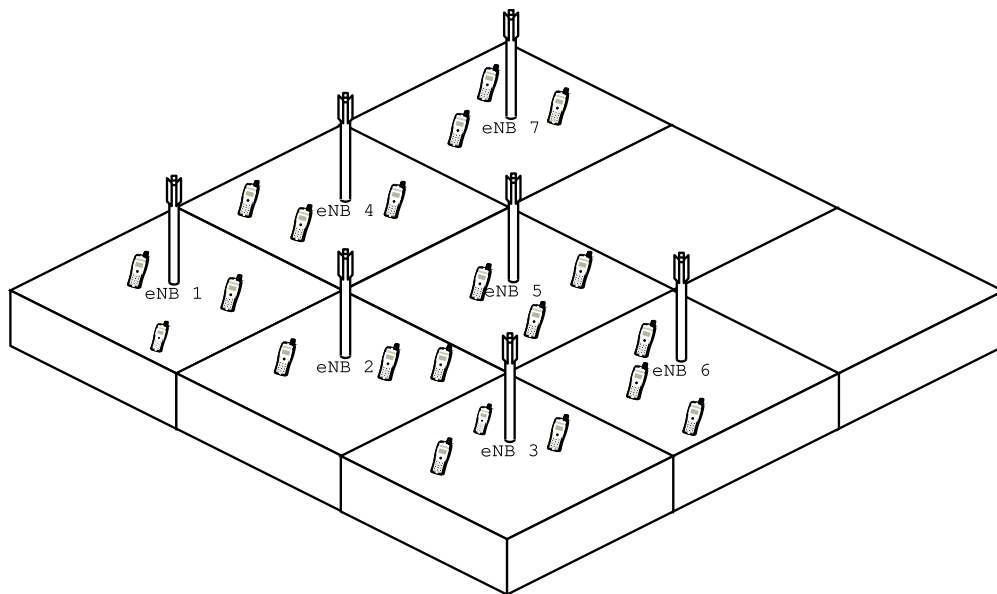


Fig. 19.101: E-UTRAN

For the EPC results, the script is `src/lte/examples/lena-simple-epc.cc`. It uses a complete E-UTRAN implementation (PHY+MAC+RLC/UM+PDCP) and the most relevant EPC user plane entities the PGW and SGW, including GTP-U tunneling. This script generates a given number of eNodeBs, distributed across a line and attaches a single UE to every eNodeB. It also creates an EPC network and an external host connected to it through the Internet. Each UE

sends and receives data to and from the remote host. In addition, each UE is also sending data to the UE camped in the adjacent eNodeB.

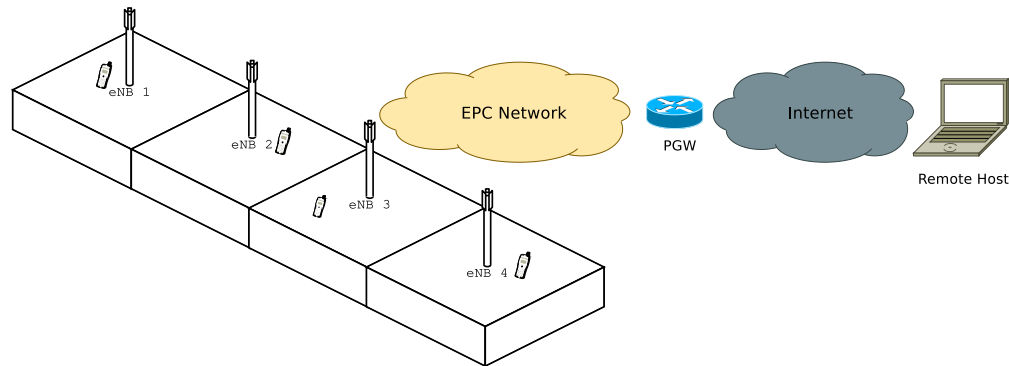


Fig. 19.102: Propagation Model

RLC and MAC traces are enabled for all UEs and all eNodeBs and those traces are written to disk directly. The MAC scheduler used is *round robin*.

Simulation input parameters

The *lena-profiling* simulation script accepts the following input parameters:

- **simTime**: time to simulate (in seconds)
- **nUe**: number of UEs attached to each eNodeB
- **nEnb**: number of eNodeB composing the grid per floor
- **nFloors**: number of floors, 0 for *Friis propagation model* (no walls), 1 or greater for *Building propagation model* generating a nFloors-storey building.
- **traceDirectory**: destination directory where simulation traces will be stored

The *lena-simple-epc* script accepts those other parameters:

- **simTime**: time to simulate (in seconds)
- **numberOfNodes**: number of eNodeB + UE pairs created

Time measurement

Running time is measured using default Linux shell command **time**. This command counts how much user time the execution of a program takes.

Perl script

To simplify the process of running the profiling script for a wide range of values and collecting its timing data, a simple Perl script to automate the complete process is provided. It is placed in `src/lte/test/lte-test-run-time.pl` for *lena-profiling* and in `src/lte/epc-test-run-time.pl` for *lena-simple-epc*. It simply runs a batch of simulations with a range of parameters and stores the timing results in a CSV file called *lteTimes.csv* and *epcTimes.csv* respectively. The range of values each parameter sweeps can be modified editing the corresponding script.

Requirements

The following Perl modules are required to use the provided script, all of them available from CPAN:

- IO::CaptureOutput
- Statistics::Descriptive

For installing the modules, simply use the following command:

```
perl -MCPAN -e 'install moduleName'
```

Plotting results

To plot the results obtained from running the Perl scripts, two gnuplot scripts are provided, in `src/lte/test/lte-test-run-plot` and `src/lte/test/epc-test-run-plot`. Most of the plots available in this documentation can be reproduced with those, typing the commands `gnuplot < src/lte/test/lte-test-run-plot` and `gnuplot < src/lte/test/epc-test-run-plot`.

Reference software and equipment

All timing tests had been run in a Intel Pentium IV 3.00 GHz machine with 512 Mb of RAM memory running Fedora Core 10 with a 2.6.27.41-170.2.117 kernel, storing the traces directly to the hard disk.

Also, as a reference configuration, the build has been configured static and optimized. The exact ns3 command issued is:

```
CXXFLAGS="-O3 -w" ./ns3 configure -d optimized --enable-static --enable-examples  
--enable-modules=lte
```

19.4.3 Results

E-UTRAN

The following results and figures had been obtained with LENA **changeset 2c5b0d697717**.

Running time

This scenario, evaluates the running time for a fixed simulation time (10s) and Friis propagation mode increasing the number of UEs attached to each eNodeB and the number of planted eNodeBs in the scenario.

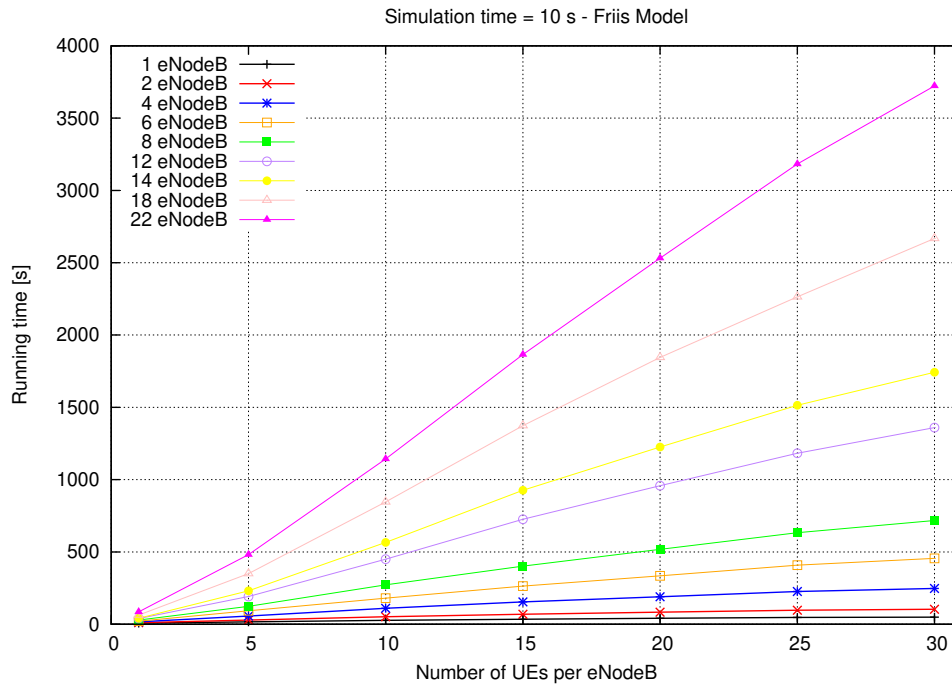


Fig. 19.103: Running time

The figure shows the expected behaviour, since it increases linearly respect the number of UEs per eNodeB and quadratically respect the total number of eNodeBs.

Propagation model

The objective of this scenario is to evaluate the impact of the propagation model complexity in the overall run time figures. Therefore, the same scenario is simulated twice: once using the more simple Friis model, once with the more complex Building model. The rest of the parameters (e.g. number of eNodeB and of UE attached per eNodeB) were maintained. The timing results for both models are compared in the following figure.

In this situation, results are also coherent with what is expected. The more complex the model, the higher the running time. Moreover, as the number of computed path losses increases (i.e. more UEs per eNodeB or more eNodeBs) the extra complexity of the propagation model drives the running time figures further apart.

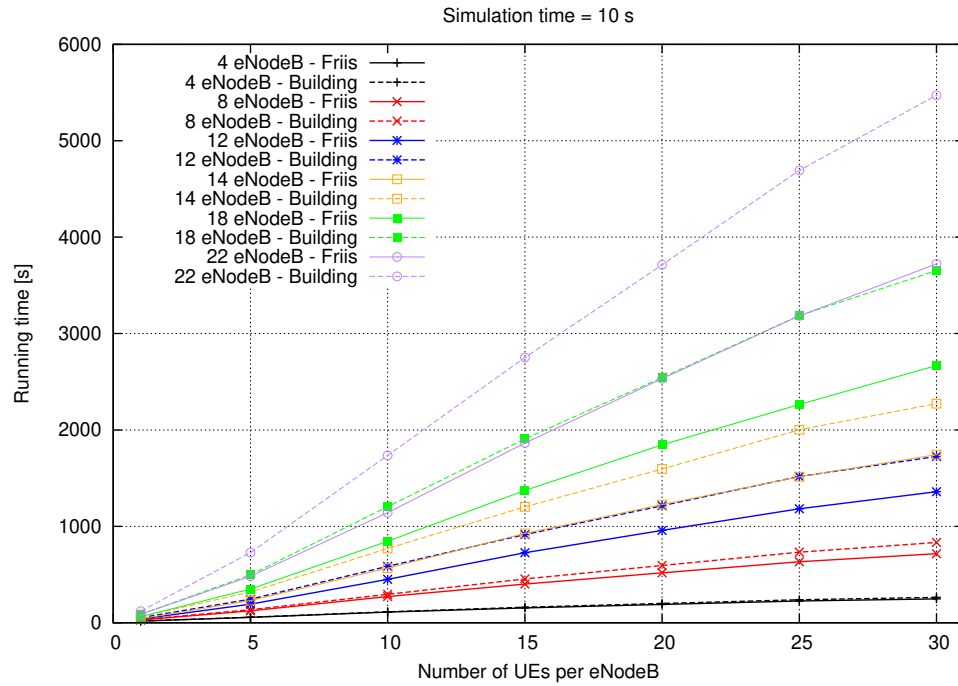


Fig. 19.104: Propagation Model

Simulation time

In this scenario, for a fixed set of UEs per eNodeB, different simulation times had been run. As the simulation time increases, running time should also increase linearly, i.e. for a given scenario, simulate four seconds should take twice times what it takes to simulate two seconds. The slope of this line is a function of the complexity of the scenario: the more eNodeB / UEs placed, the higher the slope of the line.

Memory usage

Massif tool to profile memory consumption

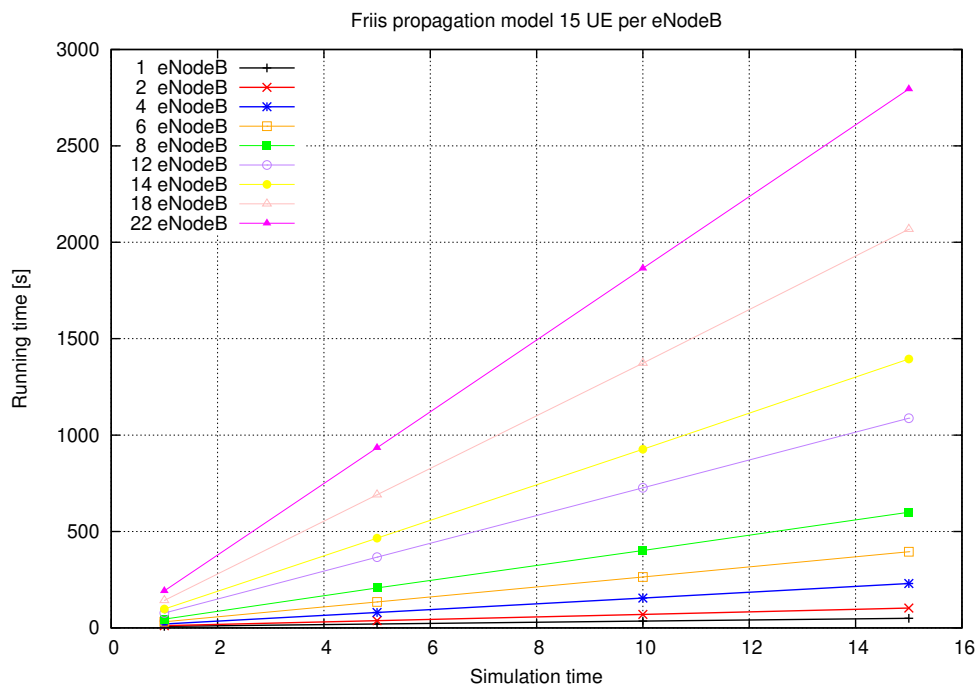


Fig. 19.105: Simulation time

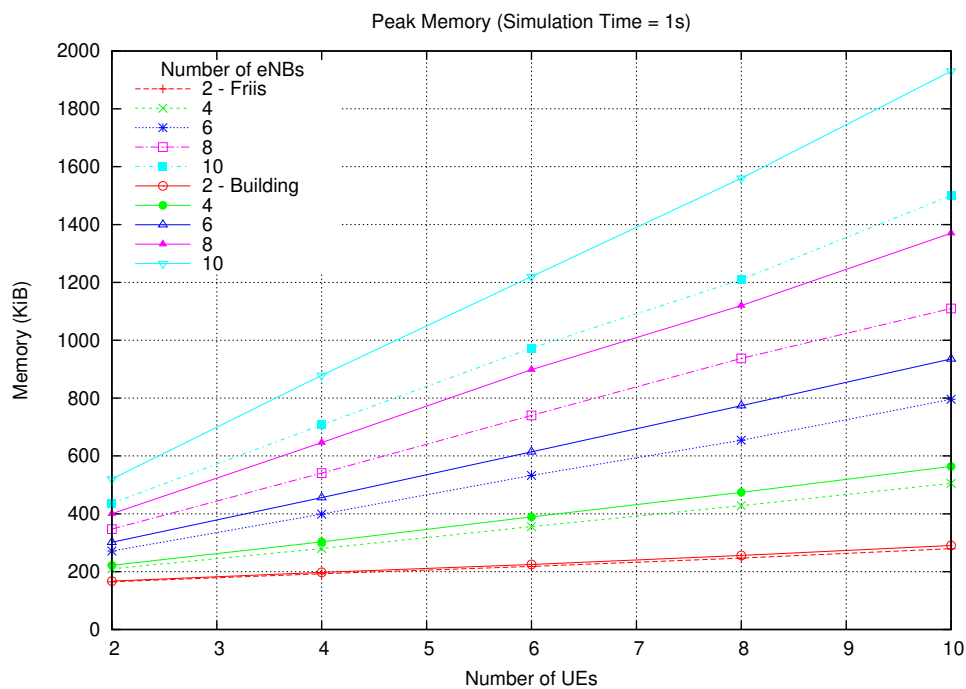


Fig. 19.106: Memory profile

EPC

The following results and figures had been obtained with LENA **changeset e8b3ccdf6673**. The rationale behind the two scenarios profiled on this section is the same than for the E-UTRA part.

Running time

Running time evolution is quadratic since we increase at the same time the number of eNodeB and the number of UEs.

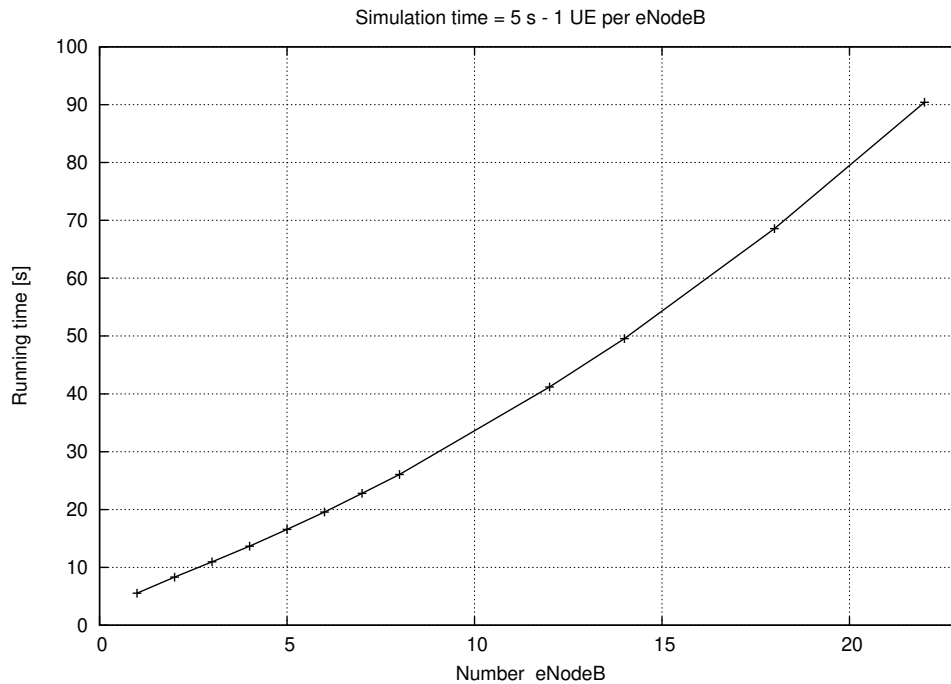


Fig. 19.107: Running time

To estimate the additional complexity of the upper LTE Radio Protocol Stack model and the EPC model, we compare two scenarios using the simplified E-UTRAN version (using only PHY, MAC and the simplified RLC/SM, with no EPC and no ns-3 applications) against the complete E-UTRAN + EPC (with UM RLC, PDCP, end-to-end IP networking and regular ns-3 applications). Both configuration have been tested with the same number of UEs per eNodeB, the same number of eNodeBs, and approximately the same volume of transmitted data (an exact match was not possible due to the different ways in which packets are generated in the two configurations).

From the figure, it is evident that the additional complexity of using the upper LTE stack plus the EPC model translates approximately into a doubling of the execution time of the simulations. We believe that, considered all the new features that have been added, this figure is acceptable.

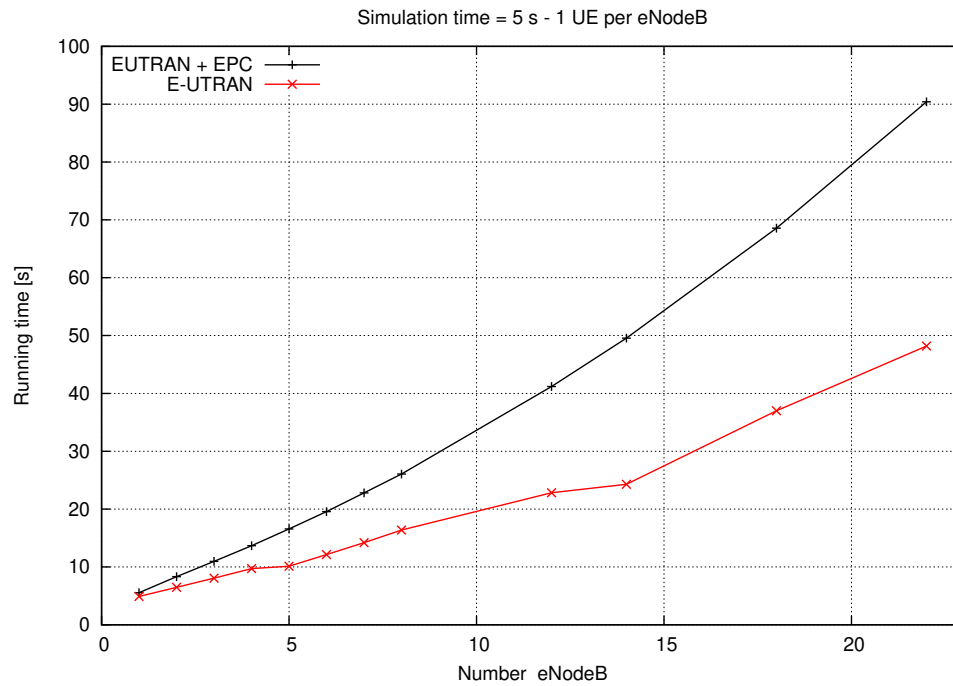


Fig. 19.108: EPC E-UTRAN running time

Simulation time

Finally, again the linearity of the running time as the simulation time increases gets validated through a set of experiments, as the following figure shows.

19.5 References

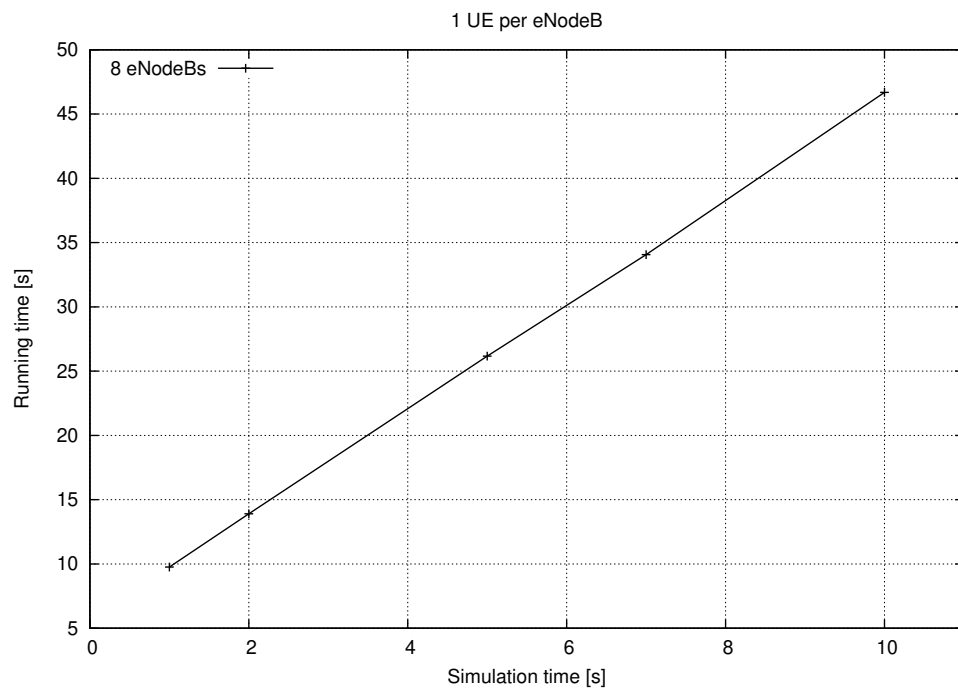


Fig. 19.109: Simulation time

WI-FI MESH MODULE DOCUMENTATION

20.1 Design Documentation

20.1.1 Overview

The *ns-3 mesh* module extends the *ns-3 wifi* module to provide mesh networking capabilities according to the IEEE 802.11s standard [ieee80211s].

The basic purpose of IEEE 802.11s is to define a mode of operation for Wi-Fi that permits frames to be forwarded over multiple radio hops transparent to higher layer protocols such as IP. To accomplish this, mesh-capable stations form a *Mesh Basic Service Set* (MBSS) by running a pair-wise peering protocol to establish forwarding associations, and by running a routing protocol to find paths through the network. A special gateway device called a *mesh gate* allows a MBSS to interconnect with a Distribution System (DS).

The basic enhancements defined by IEEE 802.11s include:

- discovery services
- peering management
- security
- beaconing and synchronization
- the Mesh Coordination Function (MCF)
- power management
- channel switching
- extended frame formats
- path selection and forwarding
- interworking (proxy mesh gateways)
- intra-mesh congestion control, and
- emergency service support.

The *ns-3* models implement only a subset of the above service extensions, focusing mainly on those items related to peering and routing/forwarding of data frames through the mesh.

The Mesh NetDevice based on 802.11s D3.0 draft standard was added in *ns-3.6* and includes the Mesh Peering Management Protocol and HWMP (routing) protocol implementations. An overview presentation by Kirill Andreev was published at the Workshop on ns-3 in 2009 [And09]. An overview paper is available at [And10].

As of ns-3.23 release, the model has been updated to the 802.11s-2012 standard [ieee80211s] with regard to packet formats, based on the contribution in [Hep15].

These changes include:

- Category codes and the categories compliant to IEEE-802.11-2012 Table 8-38—Category values.
- Information Elements (An adjustment of the element ID values was needed according to Table 8-54 of IEEE-802.11-2012).
- Mesh Peering Management element format changed according to IEEE-802.11-2012 Figure 8-370.
- Mesh Configuration element format changed according to IEEE-802.11-2012 Figure 8-363.
- PERR element format changed according to IEEE-802.11-2012 Figure 8-394.

With these changes the messages of the Peering Management Protocol and Hybrid Wireless Mesh Protocol will be transmitted compliant to IEEE802.11-2012 and the resulting pcap trace files can be analyzed by Wireshark.

The multi-interface mesh points are supported as an extension of IEEE draft version 3.0. Note that corresponding *ns-3* mesh device helper creates a single interface station by default.

Overview of IEEE 802.11s

The implementation of the 802.11s extension consists of two main parts: the Peer Management Protocol (PMP) and Hybrid Wireless Mesh Protocol (HWMP).

The tasks of the peer management protocol are the following:

- opening links, detecting beacons, and starting peer link finite state machine, and
- closing peer links due to transmission failures or beacon loss.

If a peer link between the sender and receiver does not exist, a frame will be dropped. So, the plug-in to the peer management protocol (PMP) is the first in the list of `ns3::MeshWifiInterfaceMacPlugins` to be used.

Peer management protocol

The peer management protocol consists of three main parts:

- the protocol itself, `ns3::dot11s::PeerManagementProtocol`, which keeps all active peer links on interfaces, handles all changes of their states and notifies the routing protocol about link failures.
- the MAC plug-in, `ns3::dot11s::PeerManagementProtocolMac`, which drops frames if there is no peer link, and peeks all needed information from management frames and information elements from beacons.
- the peer link, `ns3::dot11s::PeerLink`, which keeps finite state machine of each peer link, keeps beacon loss counter and counter of successive transmission failures.

The procedure of closing a peer link is not described in detail in the standard, so in the model the link may be closed by:

- beacon loss (see an appropriate attribute of `ns3::dot11s::PeerLink` class)
- transmission failure – when a predefined number of successive packets have failed to transmit, the link will be closed.

The peer management protocol is also responsible for beacon collision avoidance, because it keeps beacon timing elements from all neighbours. Note that the `PeerManagementProtocol` is not attached to the `MeshPointDevice` as a routing protocol, but the structure is similar: the upper tier of the protocol is `ns3::dot11s::PeerManagementProtocol` and its plug-in is `ns3::dot11s::PeerManagementProtocolMac`.

Hybrid Wireless Mesh Protocol

HWMP is implemented in both modes, reactive and proactive, although path maintenance is not implemented (so active routes may time out and need to be rebuilt, causing packet loss). Also the model implements an ability to transmit broadcast data and management frames as unicasts (see appropriate attributes). This feature is disabled at a station when the number of neighbors of the station is more than a threshold value.

Forwarding delay

Previous versions of this model have had issues with collisions due to the lack of a model for forwarding delay, and lack of clarity about whether backoff should be invoked when forwarding a broadcast frame. These issues become problematic for mesh, because in a topology in which multiple nodes within radio range of the next hop node decide to forward a received frame at the same time, they will repeatedly collide (if no backoff is triggered). Past contributors have argued for the triggering of backoff when forwarding a frame (e.g., [Hep16]), but current wifi module maintainers concluded that the standard does not call for backoff in this case. This was also privately confirmed with a Wi-Fi vendor. [Issue 478](#) in the GitLab.com tracker has more discussion on this point.

In practice, it is assumed that collisions can be avoided due to the fact that each mesh node will take slightly different times to process and forward the frame, and one will go first and trigger a channel busy detection on the other nodes. To accomplish this in ns-3, we must include a model for forwarding delay that includes some randomness.

The class `ns3::MeshPointDevice` is responsible for forwarding unicast frames, and the class `ns3::dot11s::HwmpProtocol` is responsible for forwarding management frames when HWMP is used. `ns3::MeshPointDevice` has an attribute called *ForwardingDelay* that configures a random variable (units of microseconds) from which a forwarding delay value is drawn for each frame forwarding event. The default configuration of this attribute is a uniform random variable between 300 and 400 microseconds. The mean value was chosen based on the measurement results reported in [Hep16] which measured and derived an average 350 microsecond delay in forwarding frames on real mesh devices. The range of this variable is somewhat arbitrary and determined by some simulation testing to provide a generally low probability of collision; the 100 microsecond range is roughly 11 slot times. The HWMP protocol can also access this random variable to create forwarding delay for the forwarding of management frames. Users may substitute other random variable configurations as desired.

20.1.2 Scope and Limitations

Supported features

- Peering Management Protocol (PMP), including link close heuristics and beacon collision avoidance.
- Hybrid Wireless Mesh Protocol (HWMP), including proactive and reactive modes, unicast/broadcast propagation of management traffic, multi-radio extensions.
- 802.11e compatible airtime link metric.

Verification

- Comes with the custom Wireshark dissector.
- Linux kernel mac80211 layer compatible message formats.

Unsupported features

- Mesh Coordinated Channel Access (MCCA).
- Internetworking: mesh access point and mesh portal.
- Security.
- Power save.
- Path maintenance (sending PREQ proactively before a path expires)
- Though multi-radio operation is supported, no channel assignment protocol is proposed for now. (Correct channel switching is not implemented)

Models yet to be created

- Mesh access point (QoS + non-QoS?)
- Mesh portal (QoS + non-QoS?)

Open issues

Users should be aware that the mesh module has not been actively maintained for several years and that there may be some performance and standards-alignment issues with the current code. Below is a listing of possible (confirmed and unconfirmed) issues.

A bug was previously reported in the Wi-Fi module that manifests itself as performance degradation in large mesh networks, due to incorrect duplicate frame detection for QoS data frames (https://www.nsnam.org/bugzilla/show_bug.cgi?id=2326).

Mesh does not work for 802.11n/ac/ax stations (<https://gitlab.com/nsnam/ns-3-dev/-/issues/176>).

Mesh PCAP is not decoded properly by Wireshark (https://www.nsnam.org/bugzilla/show_bug.cgi?id=2880).

Energy module can not be used on mesh devices (https://www.nsnam.org/bugzilla/show_bug.cgi?id=2265).

IE11S_MESH_PEERING_PROTOCOL_VERSION should be removed as per standard. Protocol ID should actually be part of the Mesh Peering Management IE (https://www.nsnam.org/bugzilla/show_bug.cgi?id=2600).

MeshInformationElementVector printing error (https://www.nsnam.org/bugzilla/show_bug.cgi?id=2728).

Mesh is not compatible with IPv6 (https://www.nsnam.org/bugzilla/show_bug.cgi?id=2881).

Mesh is forwarding multicast frames as unicast rather than as group-addressed frames (<https://gitlab.com/nsnam/ns-3-dev/-/issues/485>).

Mesh group addresses are not being set correctly for multicast frames (<https://gitlab.com/nsnam/ns-3-dev/-/issues/476>).

20.2 User Documentation

20.2.1 Using the MeshNetDevice

20.3 Testing Documentation

20.4 References

MPI FOR DISTRIBUTED SIMULATION

Parallel and distributed discrete event simulation allows the execution of a single simulation program on multiple processors. By splitting up the simulation into logical processes, LPs, each LP can be executed by a different processor. This simulation methodology enables very large-scale simulations by leveraging increased processing power and memory availability. In order to ensure proper execution of a distributed simulation, message passing between LPs is required. To support distributed simulation in *ns-3*, the standard Message Passing Interface (MPI) is used, along with a new distributed simulator class. Currently, dividing a simulation for distributed purposes in *ns-3* can only occur across point-to-point links.

21.1 Current Implementation Details

During the course of a distributed simulation, many packets must cross simulator boundaries. In other words, a packet that originated on one LP is destined for a different LP, and in order to make this transition, a message containing the packet contents must be sent to the remote LP. Upon receiving this message, the remote LP can rebuild the packet and proceed as normal. The process of sending and receiving messages between LPs is handled easily by the new MPI interface in *ns-3*.

Along with simple message passing between LPs, a distributed simulator is used on each LP to determine which events to process. It is important to process events in time-stamped order to ensure proper simulation execution. If a LP receives a message containing an event from the past, clearly this is an issue, since this event could change other events which have already been executed. To address this problem, two conservative synchronization algorithms with lookahead are used in *ns-3*. For more information on different synchronization approaches and parallel and distributed simulation in general, please refer to “Parallel and Distributed Simulation Systems” by Richard Fujimoto.

The default parallel synchronization strategy implemented in the `DistributedSimulatorImpl` class is based on a globally synchronized algorithm using an MPI collective operation to synchronize simulation time across all LPs. A second synchronization strategy based on local communication and null messages is implemented in the `NullMessageSimulatorImpl` class. For the null message strategy the global all-to-all gather is not required; LPs only need to communicate with LPs that have shared point-to-point links. The algorithm to use is controlled by which the *ns-3* global value `SimulatorImplementationType`.

The best algorithm to use is dependent on the communication and event scheduling pattern for the application. In general, null message synchronization algorithms will scale better due to local communication scaling better than a global all-to-all gather that is required by `DistributedSimulatorImpl`. There are two known cases where the global synchronization performs better. The first is when most LPs have point-to-point link with most other LPs, in other words the LPs are nearly fully connected. In this case the null message algorithm will generate more message passing traffic than the all-to-all gather. A second case where the global all-to-all gather is more efficient is when there are long periods of simulation time when no events are occurring. The all-to-all gather algorithm is able to quickly determine the next event time globally. The nearest neighbor behavior of the null message algorithm will require more communications to propagate that knowledge; each LP is only aware of neighbor next event times.

21.1.1 Remote point-to-point links

As described in the introduction, dividing a simulation for distributed purposes in *ns-3* currently can only occur across point-to-point links; therefore, the idea of remote point-to-point links is very important for distributed simulation in *ns-3*. When a point-to-point link is installed, connecting two nodes, the point-to-point helper checks the system id, or rank, of both nodes. The rank should be assigned during node creation for distributed simulation and is intended to signify on which LP a node belongs. If the two nodes are on the same rank, a regular point-to-point link is created. If, however, the two nodes are on different ranks, then these nodes are intended for different LPs, and a remote point-to-point link is used. If a packet is to be sent across a remote point-to-point link, MPI is used to send the message to the remote LP.

21.1.2 Distributing the topology

Currently, the full topology is created on each rank, regardless of the individual node system ids. Only the applications are specific to a rank. For example, consider node 1 on LP 1 and node 2 on LP 2, with a traffic generator on node 1. Both node 1 and node 2 will be created on both LP1 and LP2; however, the traffic generator will only be installed on LP1. While this is not optimal for memory efficiency, it does simplify routing, since all current routing implementations in *ns-3* will work with distributed simulation.

21.2 Running Distributed Simulations

21.2.1 Prerequisites

Ensure that MPI is installed, as well as `mpic++`. In Ubuntu repositories, these are `openmpi-bin`, `openmpi-common`, `openmpi-doc`, `libopenmpi-dev`. In Fedora, these are `openmpi` and `openmpi-devel`.

Note:

There is a conflict on some Fedora systems between `libotf` and `openmpi`. A possible “quick-fix” is to `yum remove libotf` before installing `openmpi`. This will remove conflict, but it will also remove `emacs`. Alternatively, these steps could be followed to resolve the conflict:

- 1) Rename the tiny `otfdump` which `emacs` says it needs:

```
$ mv /usr/bin/otfdump /usr/bin/otfdump.emacs-version
```

- 2) Manually resolve `openmpi` dependencies:

```
$ sudo yum install libgfortran libtorque numactl
```

- 3) Download rpm packages:

```
openmpi-1.3.1-1.fc11.i586.rpm  
openmpi-devel-1.3.1-1.fc11.i586.rpm  
openmpi-libs-1.3.1-1.fc11.i586.rpm  
openmpi-vt-1.3.1-1.fc11.i586.rpm
```

from <http://mirrors.kernel.org/fedora/releases/11/Everything/i386/os/Packages/>

- 4) Force the packages in:

```
$ sudo rpm -ivh --force \
openmpi-1.3.1-1.fc11.i586.rpm \
openmpi-libs-1.3.1-1.fc11.i586.rpm \
openmpi-devel-1.3.1-1.fc11.i586.rpm \
openmpi-vt-1.3.1-1.fc11.i586.rpm
```

Also, it may be necessary to add the openmpi bin directory to PATH in order to execute mpic++ and mpirun from the command line. Alternatively, the full path to these executables can be used. Finally, if openmpi complains about the inability to open shared libraries, such as libmpi_cxx.so.0, it may be necessary to add the openmpi lib directory to LD_LIBRARY_PATH.

Here is an example of setting up PATH and LD_LIBRARY_PATH using a bash shell:

- For a 32-bit Linux distribution:

```
$ export PATH=$PATH:/usr/lib/openmpi/bin
$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/lib/openmpi/lib
```

For a 64-bit Linux distribution:

```
$ export PATH=$PATH:/usr/lib64/openmpi/bin
$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/lib64/openmpi/lib
```

These lines can be added into ~/.bash_profile or ~/.bashrc to avoid having to retype them when a new shell is opened.

Note 2: There is a separate issue on recent Fedora distributions, which is that the libraries are built with AVX instructions. On older machines or some virtual machines, this results in an illegal instruction being thrown. This is not an ns-3 issue, a simple MPI test case will also fail. The AVX instructions are being called during initialization.

The symptom of this is that attempts to run an ns-3 MPI program will fail with the error: *terminated with signal SIGILL*. To check if this is the problem, run:

```
$ grep avx /proc/cpuinfo
```

and it will not return anything if AVX is not present.

If AVX is not supported, it is recommended to switch to a different MPI implementation such as MPICH:

```
$ dnf remove openmpi openmpi-devel
$ dnf install mpich mpich-devel environment-modules
$ module load mpi/mpich-x86_64
```

21.2.2 Building and Running Examples

If you already built ns-3 without MPI enabled, you must re-build:

```
$ ./ns3 distclean
```

Configure ns-3 with the `--enable-mpi` option:

```
$ ./ns3 configure -d debug --enable-examples --enable-tests --enable-mpi
```

Ensure that MPI is enabled by checking the optional features shown from the output of configure.

Next, build ns-3:

```
$ ./ns3
```

After building *ns-3* with *mpi* enabled, the example programs are now ready to run with *mpiexec*. It is advised to avoid running *ns3* directly with *mpiexec*; two options that should be more robust are to either use the *--command-template* way of running the *mpiexec* program, or to use *./ns3 shell* and run the executables directly on the command line. Here are a few examples (from the root *ns-3* directory):

```
$ ./ns3 run simple-distributed --command-template="mpiexec -np 2 %s"
$ ./ns3 run nms-p2p-nix-distributed --command-template="mpiexec -np 2 -machinefile_
↪mpihosts %s --nix=0"
```

An example using the null message synchronization algorithm:

```
$ ./ns3 run simple-distributed --command-template="mpiexec -np 2 %s --nullmsg"
```

The *np* switch is the number of logical processors to use. The *machinefile* switch is which machines to use. In order to use *machinefile*, the target file must exist (in this case *mpihosts*). This can simply contain something like:

```
localhost
localhost
localhost
...
```

Or if you have a cluster of machines, you can name them.

The other alternative to *command-template* is to use *./ns3 shell*. Here are the equivalent examples to the above (assuming optimized build profile):

```
$ ./ns3 shell
$ cd build/src/mpi/examples
$ mpiexec -np 2 ns3-dev-simple-distributed-optimized
$ mpiexec -np 2 -machinefile mpihosts ns3-dev-nms-p2p-nix-distributed-optimized --nix=0
$ mpiexec -np 2 ns3-dev-simple-distributed-optimized --nullmsg
```

21.2.3 Setting synchronization algorithm to use

The global value *SimulatorImplementationType* is used to set the synchronization algorithm to use. This value must be set before the *MpiInterface::Enable* method is invoked if the default *DistributedSimulatorImpl* is not used. Here is an example code snippet showing how to add a command line argument to control the synchronization algorithm choice::

```
cmd.AddValue("nullmsg", "Enable the use of null-message synchronization", nullmsg);
if (nullmsg)
{
    GlobalValue::Bind("SimulatorImplementationType",
        StringValue("ns3::NullMessageSimulatorImpl"));
}
else
{
    GlobalValue::Bind("SimulatorImplementationType",
        StringValue("ns3::DistributedSimulatorImpl"));
}
```

(continues on next page)

(continued from previous page)

```
// Enable parallel simulator with the command line arguments
MpiInterface::Enable(&argc, &argv);
```

21.2.4 Creating custom topologies

The example programs in `src/mpi/examples` give a good idea of how to create different topologies for distributed simulation. The main points are assigning system ids to individual nodes, creating point-to-point links where the simulation should be divided, and installing applications only on the LP associated with the target node.

Assigning system ids to nodes is simple and can be handled two different ways. First, a `NodeContainer` can be used to create the nodes and assign system ids:

```
NodeContainer nodes;
nodes.Create(5, 1); // Creates 5 nodes with system id 1.
```

Alternatively, nodes can be created individually, assigned system ids, and added to a `NodeContainer`. This is useful if a `NodeContainer` holds nodes with different system ids:

```
NodeContainer nodes;
Ptr<Node> node1 = CreateObject<Node>(0); // Create node1 with system id 0
Ptr<Node> node2 = CreateObject<Node>(1); // Create node2 with system id 1
nodes.Add(node1);
nodes.Add(node2);
```

Next, where the simulation is divided is determined by the placement of point-to-point links. If a point-to-point link is created between two nodes with different system ids, a remote point-to-point link is created, as described in [Current Implementation Details](#).

Finally, installing applications only on the LP associated with the target node is very important. For example, if a traffic generator is to be placed on node 0, which is on LP0, only LP0 should install this application. This is easily accomplished by first checking the simulator system id, and ensuring that it matches the system id of the target node before installing the application.

21.3 Tracing During Distributed Simulations

Depending on the system id (rank) of the simulator, the information traced will be different, since traffic originating on one simulator is not seen by another simulator until it reaches nodes specific to that simulator. The easiest way to keep track of different traces is to just name the trace files or pcaps differently, based on the system id of the simulator. For example, something like this should work well, assuming all of these local variables were previously defined:

```
if (MpiInterface::GetSystemId() == 0)
{
    pointToPoint.EnablePcapAll("distributed-rank0");
    phy.EnablePcap("distributed-rank0", apDevices.Get(0));
    csma.EnablePcap("distributed-rank0", csmaDevices.Get(0), true);
}
else if (MpiInterface::GetSystemId() == 1)
{
    pointToPoint.EnablePcapAll("distributed-rank1");
    phy.EnablePcap("distributed-rank1", apDevices.Get(0));
```

(continues on next page)

(continued from previous page)

```
    csma.EnablePcap("distributed-rank1", csmaDevices.Get(0), true);  
}
```


MOBILITY

The mobility framework in *ns-3* provides support for modeling movement and position in network simulations. This system enables simulation of mobile networks, wireless protocols, and location-dependent phenomena.

The *ns-3* mobility system consists of three main components:

- **Mobility Models:** Track and report on the current position, velocity, and movement patterns of network nodes
- **Position Allocators:** Determine initial node placement
- **Helper Classes:** Simplify configuration and provide parsers for external mobility trace formats

Most mobility-related source code is located in the `src/mobility` directory. For building-related mobility models, see `src/buildings`.

The design includes mobility models, position allocators, and helper functions.

In *ns-3*, *MobilityModel`* objects track the evolution of position with respect to a (cartesian) coordinate system. The mobility model is typically aggregated to an `ns3::Node` object and queried using `GetObject<MobilityModel>()`. The base class `ns3::MobilityModel` is subclassed for different motion behaviors.

The initial position of objects is typically set with a `PositionAllocator`. These types of objects will lay out the position on a notional canvas. Once the simulation starts, the position allocator may no longer be used, or it may be used to pick future mobility “waypoints” for such mobility models.

Most users interact with the mobility system using mobility helper classes. The `MobilityHelper` combines a mobility model and position allocator, and can be used with a node container to install a similar mobility capability on a set of nodes.

We first describe the coordinate system and issues surrounding multiple coordinate systems.

22.1 Scope and Limitations

Current Support:

- Cartesian, geocentric cartesian, and geographic coordinates are presently supported
- Twelve (12) open-field mobility models and one (1) buildings aware mobility model
- Integration with external mobility trace generators

Current Limitations:

- Z-axis movement support varies by model
- Limited built-in support for obstacle-aware movement
- No native support for indoor mobility patterns

22.2 Coordinate Systems

ns-3 supports multiple coordinate systems to accommodate different simulation scenarios and real-world applications.

Cartesian Coordinates (Default):

- **Format:** (x, y, z) coordinates in 3D space
- **Use case:** Most suitable for simulation scenarios spanning a few kilometers
- **Implementation:** `ns3::Vector` class represents both positions and velocities

Topocentric Coordinates:

- **Format:** Cartesian coordinates with a specific Earth surface reference point
- **Use case:** Converting between local and global coordinate systems
- **Implementation:** Extends Cartesian with geographic origin reference
(`GeographicPositions::TopocentricToGeographicCoordinates` in `src/mobility/model/geographic-position.cc`)

Geocentric Cartesian Coordinates:

- **Format:** Earth-centered Cartesian system with origin at Earth's center of mass
- **Use case:** Satellite communications and global-scale simulations
- **Implementation:** `GeocentricConstantPositionMobilityModel`
- **Reference:** Follows 3GPP TR 38.811, Section 6.3 [2]

Geographic Coordinates:

- **Format:** Latitude, longitude, and altitude relative to Earth's surface
- **Use case:** GPS-based applications and real-world location mapping
- **Implementation:** `GeographicPosition` class with conversion utilities

At the moment, the geocentric Cartesian coordinates are adopted by the `GeocentricConstantPositionMobilityModel` only. This class implements the `Get/SetPosition` methods, which leverage the `GeographicPosition` class to offer conversions to and from Cartesian coordinates. Additionally, users can set the position of a node by its geographical coordinates via the methods `Get/SetGeographicPosition`.

22.3 Available Mobility Models

All mobility models inherit from the base class `ns3::MobilityModel`. Child classes implement specific movement behaviors. Below is an overview of the key mobility models available in *ns-3*.

22.3.1 Core Classes

MobilityModel Base Class

The `ns3::MobilityModel` class is the foundation of the mobility system:

```
// Key methods
Vector GetPosition() const;           // Current position
Vector GetVelocity() const;          // Current velocity
double GetDistanceFrom(Ptr<const MobilityModel> other) const;
```

(continues on next page)

(continued from previous page)

```
// Attributes
Vector Position;           // Current position
Vector Velocity;          // Current velocity

// Trace sources
TracedCallback<Ptr<const MobilityModel>> CourseChange;
```

Key Features:

- **Position Tracking**
- **Velocity Tracking**
- **Distance Calculation:** Calculates distance to another mobility model object
- **Course Change Notifications:** Trace source for movement events
- **Node Aggregation:** Typically aggregated to ns3::Node objects

22.3.2 Child Classes**Static Models:**

- **ConstantPositionMobilityModel:** Nodes remain at fixed positions throughout the simulation. The position is set once during initialization and only changes if explicitly set by the user during runtime. This model is ideal for stationary infrastructure nodes, base stations, or scenarios where mobility is not required. It has near zero computational overhead during simulation runtime.
- **GeocentricConstantPositionMobilityModel:** Similar to ConstantPositionMobilityModel but with geographic coordinate support. Positions can be specified using latitude, longitude, and altitude coordinates, which are internally converted to the simulation's Cartesian coordinate system. Useful for real-world GPS-based scenarios and satellite communication simulations.

Linear Motion Models:

- **ConstantVelocityMobilityModel:** Nodes move in a straight line with constant speed. The velocity vector (speed and direction) is set once and remains unchanged unless explicitly modified by external events. This model is commonly used with ns-2 mobility traces where setdest commands update the velocity.
- **ConstantAccelerationMobilityModel:** Implements uniformly accelerated motion following kinematic equations. The node starts with an initial velocity and acceleration vector, with position updated according to:

$$\vec{p}(t) = \vec{p}_0 + \vec{v}_0 \cdot t + \frac{1}{2} \vec{a} \cdot t^2$$

This model is useful for scenarios involving vehicle acceleration/deceleration or projectile motion.

Random Motion Models:

- **RandomWalk2dMobilityModel:** Implements a two-dimensional random walk where nodes change direction and speed at regular time intervals. At each step, a new direction is randomly selected from a uniform distribution [0, 2pi], and speed is drawn from a configurable random variable. Movement is constrained within specified rectangular bounds, with reflection or wrapping behavior at boundaries. The model includes configurable parameters for step time, speed distribution, and boundary behavior.
- **RandomWalk2dOutdoorMobilityModel:** Enhanced version of RandomWalk2dMobilityModel that incorporates building awareness. Nodes avoid moving through building obstacles by checking for intersections with building polygons before committing to movement. When a building collision is detected, the node selects an alternative

direction. This model requires integration with building models and is particularly useful for urban mobility scenarios.

- **RandomDirection2dMobilityModel**: Nodes move in straight lines for random durations, then pause and select new random directions. Unlike **RandomWalk2d**, movement occurs in longer straight-line segments rather than frequent small steps. The model alternates between movement and pause periods, with both durations drawn from configurable random variables. Direction changes occur uniformly over $[0, 2\pi]$, and speed is constant during each movement phase.
- **RandomWaypointMobilityModel**: Classic random waypoint model where nodes move from their current position to randomly selected destination waypoints. Upon reaching a waypoint, the node pauses for a random duration, then selects a new random destination. Movement between waypoints follows straight-line paths at constant speeds drawn from a configurable distribution. This model exhibits well-known characteristics including non-uniform spatial distribution and speed decay over time.
- **SteadyStateRandomWaypointMobilityModel**: Addresses the initial transient behavior of the standard **RandomWaypointMobilityModel** by starting nodes with positions and velocities drawn from the model's steady-state distribution. This eliminates the artificial clustering and speed artifacts present during the initial simulation phase of the standard random waypoint model, providing more realistic results from simulation start.

Advanced Models:

- **GaussMarkovMobilityModel**: Implements a Gaussian-Markov stochastic process where velocity components are correlated over time. The model balances between random movement and momentum conservation using a tunable randomness parameter α in the range $[0, 1]$. When $\alpha=0$, movement is completely random; when $\alpha=1$, movement maintains constant velocity. The velocity update equation is:

$$v_n = \alpha \cdot v_{n-1} + (1 - \alpha) \cdot \bar{v} + \sqrt{1 - \alpha^2} \cdot v_{\text{random}}$$

This model produces more realistic mobility patterns with temporal correlation, suitable for human pedestrian movement.

- **WaypointMobilityModel**: Follows user-defined sequences of waypoints with precise timing control. Each waypoint specifies a position and arrival time, allowing deterministic or scripted mobility patterns. The model supports complex trajectories, synchronized movement scenarios, and replay of real-world mobility traces. Waypoints can be added dynamically during simulation, enabling adaptive mobility patterns based on simulation events.
- **HierarchicalMobilityModel**: Supports group mobility scenarios using parent-child relationships. Child nodes move relative to a parent mobility model, with the final position being the vector sum of parent and child positions. The parent model defines the group's overall movement pattern, while child models define individual member movements relative to the group. This architecture enables scenarios like vehicular convoys, pedestrian groups, or mobile sensor clusters where individual nodes maintain local mobility within a moving group context.

Position Allocators:

- **ListPositionAllocator**: Uses a predefined list of positions
- **GridPositionAllocator**: Arranges nodes in regular grid patterns
- **RandomRectanglePositionAllocator**: Uniform distribution within rectangular areas
- **RandomBoxPositionAllocator**: Uniform distribution within 3D box regions
- **RandomDiscPositionAllocator**: Uniform distribution within circular areas
- **UniformDiscPositionAllocator**: Even distribution on disc circumference

Other class types are also present in the *building module*. They describe how positions are assigned to buildings.

A position allocator is not always required, as some mobility models generate initial positions during initialization. Among the built-in ns-3 models, **SteadyStateRandomWaypointMobilityModel** is the only one with this capability.

22.4 Usage

Most *ns-3* program authors typically interact with the mobility system only at configuration time. However, various *ns-3* objects interact with mobility objects repeatedly during runtime, such as a propagation model trying to determine the path loss between two mobile nodes.

22.4.1 Basic Configuration

Random Walk Setup

```
// Create nodes
NodeContainer nodes;
nodes.Create(10);

// Configure mobility
MobilityHelper mobility;

// Grid initial placement
mobility.SetPositionAllocator("ns3::GridPositionAllocator",
                             "MinX", DoubleValue(0.0),
                             "MinY", DoubleValue(0.0),
                             "DeltaX", DoubleValue(10.0),
                             "DeltaY", DoubleValue(10.0),
                             "GridWidth", UIntegerValue(5),
                             "LayoutType", StringValue("RowFirst"));

// Random walk mobility
mobility.SetMobilityModel("ns3::RandomWalk2dMobilityModel",
                          "Bounds", RectangleValue(Rectangle(-100, 100, -100, 100)),
                          "Speed", StringValue("ns3::ConstantRandomVariable[Constant=5.0]"),
                          "Direction", StringValue(
                              "ns3::UniformRandomVariable[Min=0|Max=6.28]"));

mobility.Install(nodes);
```

Group Mobility Configuration

```
// Reference point group mobility
GroupMobilityHelper groupMobility;

// Parent mobility (reference point)
groupMobility.SetReferencePointMobilityModel("ns3::WaypointMobilityModel");

// Child mobility (group members)
groupMobility.SetMemberMobilityModel("ns3::RandomWalk2dMobilityModel",
                                     "Bounds", RectangleValue(Rectangle(-10, 10, -10, 10)));

// Set reference point waypoints
```

(continues on next page)

(continued from previous page)

```

Ptr<WaypointMobilityModel> reference = groupMobility.GetReferencePointMobilityModel();
reference->AddWaypoint(Waypoint(Seconds(0), Vector(0, 0, 0)));
reference->AddWaypoint(Waypoint(Seconds(100), Vector(100, 0, 0)));
reference->AddWaypoint(Waypoint(Seconds(200), Vector(100, 100, 0)));

groupMobility.Install(groupNodes);

```

Geographic Positioning

```

// Use geocentric coordinates
MobilityHelper mobility;
mobility.SetMobilityModel("ns3::GeocentricConstantPositionMobilityModel");

// Set geographic position
Ptr<GeocentricConstantPositionMobilityModel> model =
    node->GetObject<GeocentricConstantPositionMobilityModel>();
model->SetGeographicPosition(Vector(latitude, longitude, altitude));

```

22.4.2 Advanced Usage

Random Number Stream Assignment

To avoid the possibility that configuration or parameter changes in a program affect its mobility behavior, it is necessary to assign fixed stream numbers to any random variables associated with mobility models. It is strongly advised to read the section on how to perform independent replications (see “Random Variables”, the chapter 3.1 of ns-3 Manual).

```

// Assign specific random streams
int64_t streamIndex = 100;
MobilityHelper mobility;
// ... configure mobility ...
mobility.Install(nodes);

// Assign streams after installation
int64_t streamsUsed = mobility.AssignStreams(nodes, streamIndex);

```

Class `MobilityModel` and class `PositionAllocator` both have public API to assign streams to underlying random variables:

```

/**
 * Assign a fixed random variable stream number to the random variables
 * used by this model. Return the number of streams (possibly zero) that
 * have been assigned.
 *
 * \param stream first stream index to use
 * \return the number of stream indices assigned by this model
 */
int64_t AssignStreams(int64_t stream);

```

External Tool Integration

BonnMotion

BonnMotion is a Java software which creates and analyses mobility scenarios [1]. It is developed within the Communication Systems group at the Institute of Computer Science 4 of the University of Bonn, Germany, where it serves as a tool for the investigation of mobile ad hoc network characteristics. BonnMotion is being jointly developed by the Communication Systems group at the University of Bonn, Germany, the Toilers group at the Colorado School of Mines, Golden, CO, USA, and the Distributed Systems group at the University of Osnabrück, Germany.

- **Installation:** [Installation instructions](#) for using BonnMotion with *ns-3*
- **Documentation:** [Documentation](#) available
- **Output Format:** *ns-2* compatible traces

SUMO (Simulation of Urban Mobility)

Literature on the subject of urban wireless network often use SUMO to model the mobility in a given scenario, either based on a fictional setup or on real urban environment extracted through tools like OpenStreetMap.

- **Purpose:** Realistic vehicular mobility simulation
- **Website:** [SUMO](#)
- **Integration:** Exports *ns-2* format traces
- **Use Case:** Urban traffic scenarios

TraNS

- **Purpose:** Traffic and network simulation environment
- **Website:** [TraNS](#)
- **Features:** Combines SUMO with network simulation
- **Output:** Compatible mobility traces

[ns2] setdest Utility

The *ns-2* [setdest](#) utility can generate basic mobility patterns.

22.4.3 Helpers

A special mobility helper is provided that is mainly aimed at supporting the installation of mobility to a Node container (when using containers at the helper API level). The MobilityHelper class encapsulates a MobilityModel factory object and a PositionAllocator used for initial node layout.

Group mobility is also configurable via a GroupMobilityHelper object. Group mobility reuses the HierarchicalMobilityModel allowing one to define a reference (parent) mobility model and child (member) mobility models, with the position being the vector sum of the two mobility model positions (i.e., the child position is defined as an offset to the parent position). In the GroupMobilityHelper, the parent mobility model is not associated with any node, and is used as the parent mobility model for all (distinct) child mobility models. The reference point group mobility model [1] is the basis for this *ns-3* model.

MobilityHelper

The MobilityHelper class simplifies mobility configuration:

```
// Basic configuration
MobilityHelper mobility;
mobility.SetPositionAllocator("ns3::GridPositionAllocator",
                             "MinX", DoubleValue(0.0),
                             "MinY", DoubleValue(0.0),
                             "DeltaX", DoubleValue(5.0),
                             "DeltaY", DoubleValue(10.0));

mobility.SetMobilityModel("ns3::RandomWalk2dMobilityModel",
                          "Bounds", RectangleValue(Rectangle(-50, 50, -50, 50)));

// Installation
mobility.Install(nodeContainer);
```

GroupMobilityHelper

Supports group mobility scenarios using the reference point group mobility model [1]:

```
GroupMobilityHelper groupMobility;
groupMobility.SetReferencePointMobilityModel("ns3::RandomWaypointMobilityModel");
groupMobility.SetMemberMobilityModel("ns3::RandomWalk2dMobilityModel");
groupMobility.Install(groupNodes);
```

Ns2MobilityHelper

Parses ns-2 format mobility traces for compatibility with existing tools:

```
Ns2MobilityHelper ns2mobility("mobility-trace.ns_movements");
ns2mobility.Install();
```

The ns-2 mobility format is a widely used mobility trace format. Valid trace files use the following ns-2 statements:

```
$node set X_ x1
$node set Y_ y1
$node set Z_ z1
$ns at $time $node setdest x2 y2 speed
```

Supported ns-2 Commands

- \$node set X_ x1: Set initial X position
- \$node set Y_ y1: Set initial Y position
- \$node set Z_ z1: Set initial Z position
- \$ns at \$time \$node setdest x2 y2 speed: Move to destination at specified time

Note that in ns-3, movement along the Z dimension is not supported by all mobility models.

22.4.4 Tracing and Visualization

Tracing provides access to mobility models whenever the mobility model declares a course change (a change in position or velocity). Mobility models can also be polled at any time to find the current position of the model.

Course Change Tracing

An example of this tracing method is the Waypoint Mobility Model. When a node selects a new waypoint, the `NotifyCourseChange()` function of the `MobilityModel` class (parent to the `WaypointMobilityModel` class) is called, triggering the course change trace. This allow the user to log when an object has reached certain points.

```
// Enable course change logging
void CourseChangeCallback(Ptr<const MobilityModel> model) {
    Vector pos = model->GetPosition();
    Vector vel = model->GetVelocity();
    std::cout << "Node moved to: " << pos << " with velocity: " << vel << std::endl;
}

// Connect trace source
Config::Connect("/NodeList/*/ns3::MobilityModel/CourseChange",
                MakeCallback(&CourseChangeCallback));
```

Mobility Model Polling

If client code needs access to a mobility model's position or other state information outside of course change events, it may directly query the mobility model at any time. Mobility models are often aggregated (using ns-3 Object aggregation) to ns-3 nodes, so the mobility model pointer can usually be easily obtained from a node pointer using `GetObject()`.

```
// Periodic position printing
void PrintPositions(NodeContainer nodes) {
    for (auto i = nodes.Begin(); i != nodes.End(); ++i) {
        Ptr<MobilityModel> mobility = (*i)->GetObject<MobilityModel>();
        Vector pos = mobility->GetPosition();
        std::cout << "Node " << (*i)->GetId() << ": " << pos << std::endl;
    }
    Simulator::Schedule(Seconds(1.0), &LogPositions, nodes);
}
```

22.4.5 Common Use Cases

Wireless Network Evaluation

- **Scenario:** Mobile ad-hoc networks (MANETs)
- **Models:** `RandomWaypoint`, `RandomWalk2d`
- **Considerations:** Realistic speed distributions, pause times

Vehicular Networks

- **Scenario:** Vehicle-to-vehicle (V2V) communication
- **Tools:** SUMO integration
- **Models:** Trace-based mobility from traffic simulators

Sensor Networks

- **Scenario:** Environmental monitoring

- **Models:** ConstantPosition with occasional RandomWalk
- **Considerations:** Energy-efficient movement patterns

Satellite Communications

- **Scenario:** Low Earth Orbit (LEO) constellations
- **Models:** GeocentricConstantPosition or custom orbital models
- **Coordinates:** Geocentric Cartesian system

22.5 Examples and Tests

The following example programs demonstrate mobility usage:

- `main-random-topology.cc` - Random initial placement
- `main-random-walk.cc` - Random walk mobility
- `main-grid-topology.cc` - Grid-based positioning
- `ns2-mobility-trace.cc` - *ns-2* trace file parsing
- `reference-point-group-mobility-example.cc` - Group mobility demonstration

22.5.1 ns2-mobility-trace Example

The `ns2-mobility-trace.cc` program is an example of loading an *ns-2* trace file that specifies the movements of two nodes over 100 seconds of simulation time. It is paired with the file `default.ns_movements`.

The program behaves as follows:

- A `Ns2MobilityHelper` object is created, with the specified trace file.
- A log file is created, using the log file name argument.
- A node container is created with the number of nodes specified in the command line. For this particular trace file, specify the value 2 for this argument.
- The `Install()` method of `Ns2MobilityHelper` to set mobility to nodes. At this moment, the file is read line by line, and the movement is scheduled in the simulator.
- A callback is configured, so each time a node changes its course a log message is printed.

Example usage:

```
$ ./ns3 run "ns2-mobility-trace \  
--traceFile=src/mobility/examples/default.ns_movements \  
--nodeNum=2 \  
--duration=100.0 \  
--logFile=ns2-mob.log"
```

Sample log file output:

```
+0.0ns POS: x=150, y=93.986, z=0; VEL:0, y=50.4038, z=0  
+0.0ns POS: x=195.418, y=150, z=0; VEL:50.1186, y=0, z=0  
+104727357.0ns POS: x=200.667, y=150, z=0; VEL:50.1239, y=0, z=0  
+204480076.0ns POS: x=205.667, y=150, z=0; VEL:0, y=0, z=0
```

22.5.2 bonnmotion-ns2-example

The `bonnmotion-ns2-example.cc` program, which models the movement of a single mobile node for 1000 seconds of simulation time, has a few associated files:

- `bonnmotion.ns_movements` is the *ns-2*-formatted mobility trace
- `bonnmotion.params` is a BonnMotion-generated file with some metadata about the mobility trace
- `bonnmotion.ns_params` is another BonnMotion-generated file with *ns-2*-related metadata.

Neither of the latter two files is used by *ns-3*, although they are generated as part of the BonnMotion process to output *ns-2*-compatible traces.

The program `bonnmotion-ns2-example.cc` will output the following to stdout:

```
At 0.00 node 0: Position(329.82, 66.06, 0.00);   Speed(0.53, -0.22, 0.00)
At 100.00 node 0: Position(378.38, 45.59, 0.00);   Speed(0.00, 0.00, 0.00)
At 200.00 node 0: Position(304.52, 123.66, 0.00);   Speed(-0.92, 0.97, 0.00)
At 300.00 node 0: Position(274.16, 131.67, 0.00);   Speed(-0.53, -0.46, 0.00)
At 400.00 node 0: Position(202.11, 123.60, 0.00);   Speed(-0.98, 0.35, 0.00)
At 500.00 node 0: Position(104.60, 158.95, 0.00);   Speed(-0.98, 0.35, 0.00)
At 600.00 node 0: Position(31.92, 183.87, 0.00);   Speed(0.76, -0.51, 0.00)
At 700.00 node 0: Position(107.99, 132.43, 0.00);   Speed(0.76, -0.51, 0.00)
At 800.00 node 0: Position(184.06, 80.98, 0.00);   Speed(0.76, -0.51, 0.00)
At 900.00 node 0: Position(250.08, 41.76, 0.00);   Speed(0.60, -0.05, 0.00)
```

The motion of the mobile node is sampled every 100 seconds, and its position and speed are printed out. This output may be compared to the output of a similar *ns-2* program (found in the *ns-2* `tc1/ex/` directory of *ns-2*) running from the same mobility trace.

The next file is generated from *ns-2* (users will have to download and install *ns-2* and run this Tcl program to see this output). The output of the *ns-2* `bonnmotion-example.tcl` program is shown below for comparison (file `bonnmotion-example.tr`):

```
M 0.000000 0 (329.82, 66.06, 0.00), (378.38, 45.59), 0.57
M 100.000000 0 (378.38, 45.59, 0.00), (378.38, 45.59), 0.57
M 119.37150 0 (378.38, 45.59, 0.00), (286.69, 142.52), 1.33
M 200.000000 0 (304.52, 123.66, 0.00), (286.69, 142.52), 1.33
M 276.35353 0 (286.69, 142.52, 0.00), (246.32, 107.57), 0.70
M 300.000000 0 (274.16, 131.67, 0.00), (246.32, 107.57), 0.70
M 354.65589 0 (246.32, 107.57, 0.00), (27.38, 186.94), 1.04
M 400.000000 0 (202.11, 123.60, 0.00), (27.38, 186.94), 1.04
M 500.000000 0 (104.60, 158.95, 0.00), (27.38, 186.94), 1.04
M 594.03719 0 (27.38, 186.94, 0.00), (241.02, 42.45), 0.92
M 600.000000 0 (31.92, 183.87, 0.00), (241.02, 42.45), 0.92
M 700.000000 0 (107.99, 132.43, 0.00), (241.02, 42.45), 0.92
M 800.000000 0 (184.06, 80.98, 0.00), (241.02, 42.45), 0.92
M 884.77399 0 (241.02, 42.45, 0.00), (309.59, 37.22), 0.60
M 900.000000 0 (250.08, 41.76, 0.00), (309.59, 37.22), 0.60
```

The output formatting is slightly different, and the course change times are additionally plotted, but it can be seen that the position vectors are the same between the two traces at intervals of 100 seconds.

The mobility computations performed on the *ns-2* trace file are slightly different in *ns-2* and *ns-3*, and floating-point arithmetic is used, so there is a chance that the position in *ns-2* may be slightly different than the respective position when using the trace file in *ns-3*.

22.5.3 reference-point-group-mobility-example

The reference point group mobility model [1] is demonstrated in the example program `reference-point-group-mobility-example.cc`. This example runs a short simulation that illustrates a parent `WaypointMobilityModel` traversing a rectangular course within a bounding box, and three member nodes independently execute a two-dimensional random walk around the parent position, within a small bounding box.

The example illustrates configuration using the `GroupMobilityHelper` and manual configuration without a helper; the configuration option is selectable by command-line argument.

The example outputs two mobility trace files, a course change trace and a time-series trace of node position. The latter trace file can be parsed by a Bash script (`reference-point-group-mobility-animate.sh`) to create PNG images at one-second intervals, which can then be combined using an image processing program such as `ImageMagick` to form a basic animated gif of the mobility.

22.5.4 Tests

Specific validation results and test cases are documented in the *ns-3* test suite under `src/mobility/test/`.

22.6 Validation

No formal validation has been done.

22.7 Extra models

22.7.1 Orbital Mobility

This mobility model was originally developed by Tim Schubert as part of *ns-3-leo* [4]. It was then ported, refactored and optimized by Thiago Myazaki as part of Google Summer of Code (GSoC) 2025.

It allows for the configuration of satellite scenarios, with the objective to create an example integrating LEO satellites using beamforming, channel and propagation models from the NR Module and *ns-3*, leveraging a helper model to aid the instantiation of LEO satellites.

It models simple circular orbits with constant orbital velocity.

In addition to the mobility model, we have also ported an example from *ns-3-leo* [4] that outputs position traces as ECEF coordinates. Using a Python script developed during the project, `plot-orbital-traces.py`, we produced animated plots that helped validate the nodes' initial positions and their movement over time, under different configurations of numbers of nodes, inclination, and altitude.

We also have a unified example (*leo-satellite-example.cc*) that combines three use cases into a single program controlled by the `--mode` switch: `orbit` for mobility tracing, `antenna` for antenna pointing toward a ground station, and `channel` for full 3GPP NTN propagation loss estimation.

Characteristics & Assumptions

The LEO mobility model relies on classical two-body orbital mechanics. The following references provide the theoretical background:

- [Orbital mechanics](#) — general overview of orbital mechanics and the two-body problem.
- [Orbital elements](#) — parameters defining an orbit (semi-major axis, eccentricity, inclination, RAAN, argument of perigee, true anomaly).
- [Kepler's laws of planetary motion](#) — the third law is used to derive the orbital angular velocity from the satellite altitude.
- [Low Earth orbit](#) — definition and typical characteristics of LEO.

The model has the following assumptions:

- **Orbit type:** only models ideal circular orbits. There is no support for elliptical orbits.
- **Fixed speed:** Since the orbit is circular and not elliptical, we assume the satellite nodes are moving at constant speed along their orbits, given a certain altitude.
- **Static Earth:** While the model takes into consideration Earth's rotation, it does not consider Earth's orbit around the sun; i.e., the Earth is static.
- In the plots, Earth is modeled as a perfect sphere.

The model computes satellite positions in three steps:

- First step: Compute the satellite's angular position along its orbit from the initial orbital offset and the angular velocity (derived from altitude via Keplerian mechanics, see [Orbital elements](#)) multiplied by the elapsed simulation time.
- Second step: Apply orbital plane inclination relative to the equator and rotation by the RAAN.
- Third step: Apply the Rodrigues rotation formula (see [Rodrigues' rotation formula](#)) to rotate the satellite around the orbital plane normal by the computed angular position.

The figures below illustrate how satellites are distributed along their orbits at different angular spacings. They can be plotted with the help of the `plot-orbital-traces.py` script.

These examples illustrate the trade-off between the resolution time step and the path accuracy. With a coarser resolution ([Fig. 22.1](#)), the satellite's movement appears as larger discrete jumps, while a finer resolution ([Fig. 22.3](#)) provides a smoother and more precise representation of the orbital trajectory at the cost of more frequent mobility updates. The intermediate case ([Fig. 22.2](#)) shows the effect of a 60 s resolution.

After precomputing these, we apply the plane inclination with respect to the equator, as shown in [Fig. 22.4](#), where we add a 30 degree inclination to the orbital plane with respect to the equator.

When we have multiple orbital planes per constellation, we divide 360 degrees by the number of planes, and apply a Z-axis rotation relative to the RAAN, starting from the prime meridian. This step effectively spaces the orbital planes out around the constellation.

For example, with two orbital planes, we would have them separated by 180 degrees, as shown in [Fig. 22.4](#) and [Fig. 22.5](#).

The third and final step is to apply a rotation around the orbital plane normal vector, like adjusting a volume dial, to emulate the orbital movement of the satellite in its orbit. For example, compare [Fig. 22.4](#) and [Fig. 22.6](#).

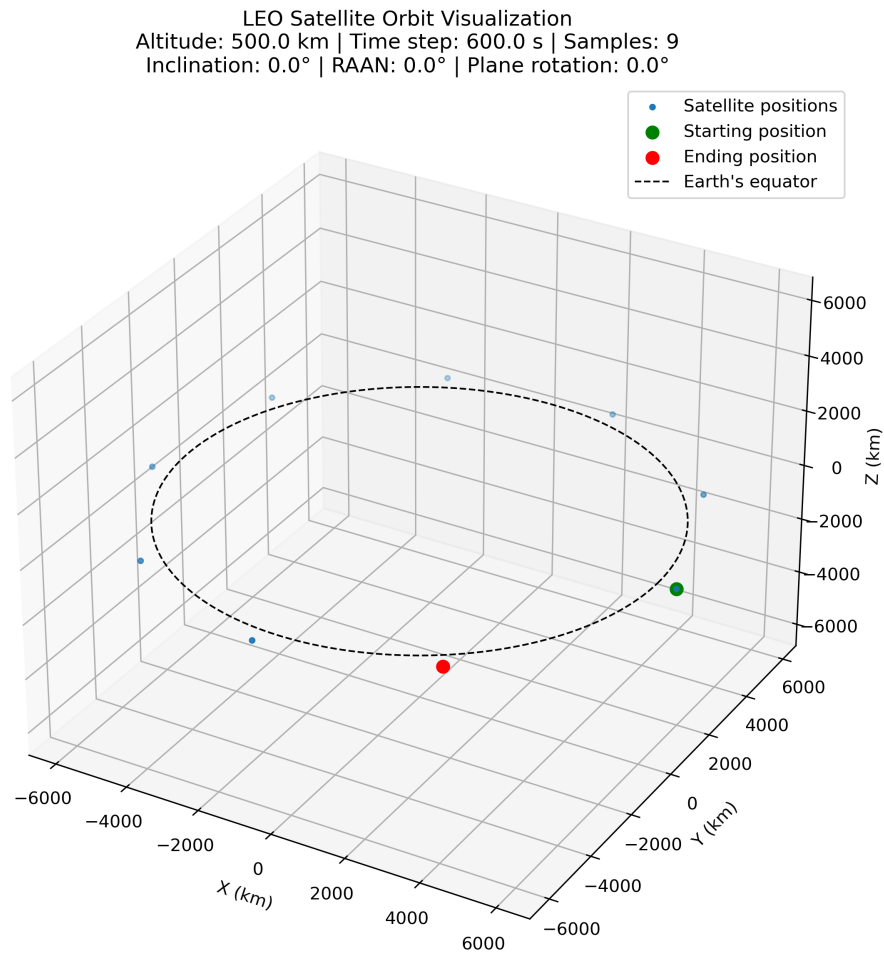


Fig. 22.1: LEO orbit with 600 s resolution, settable via the `LeoCircularOrbitMobilityModel::Resolution` attribute

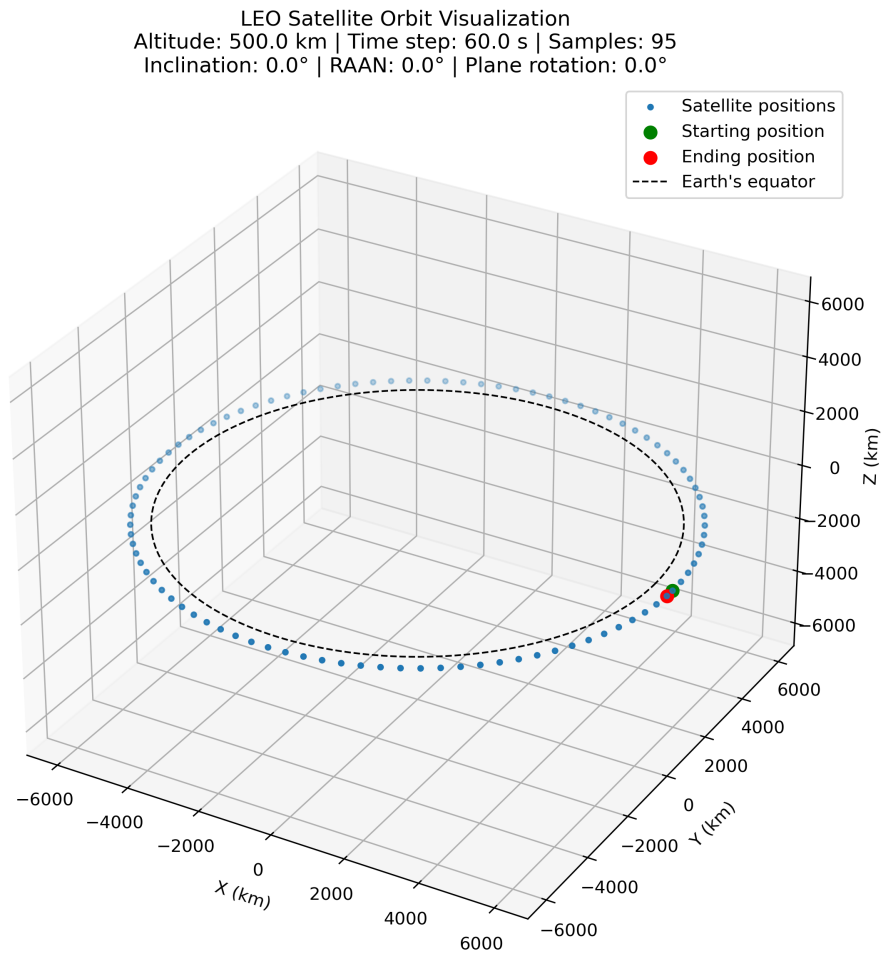


Fig. 22.2: LEO orbit with 60 s resolution

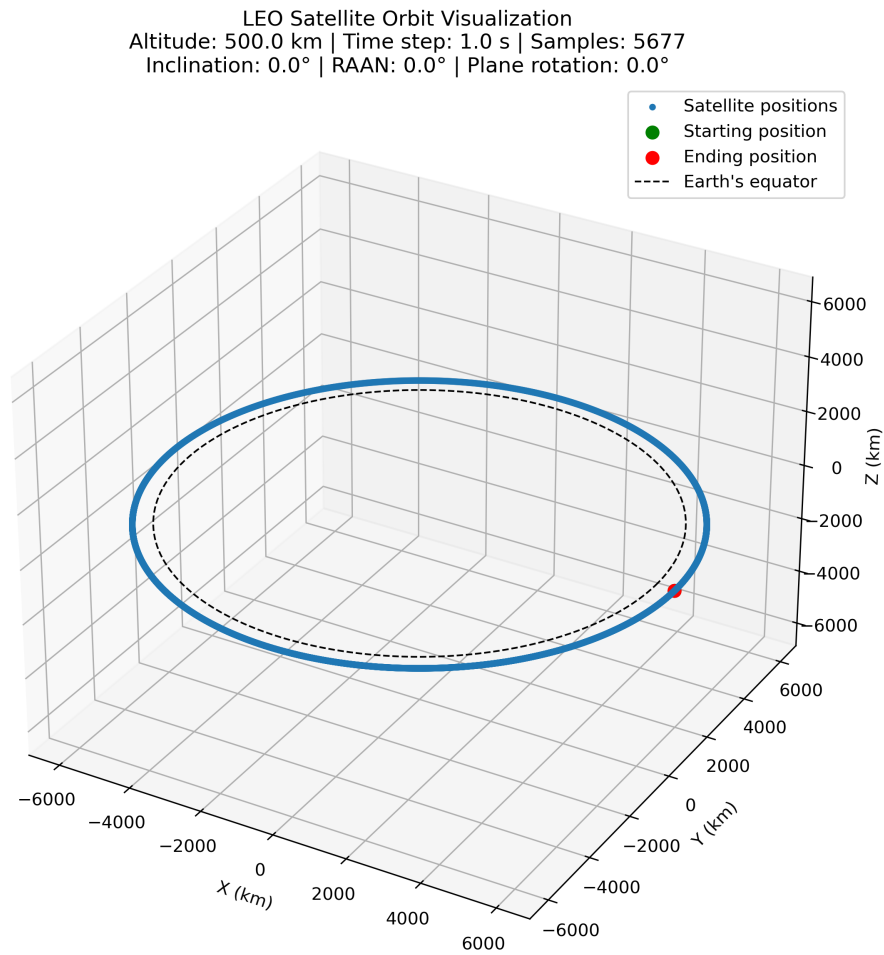


Fig. 22.3: LEO orbit with 1 s resolution

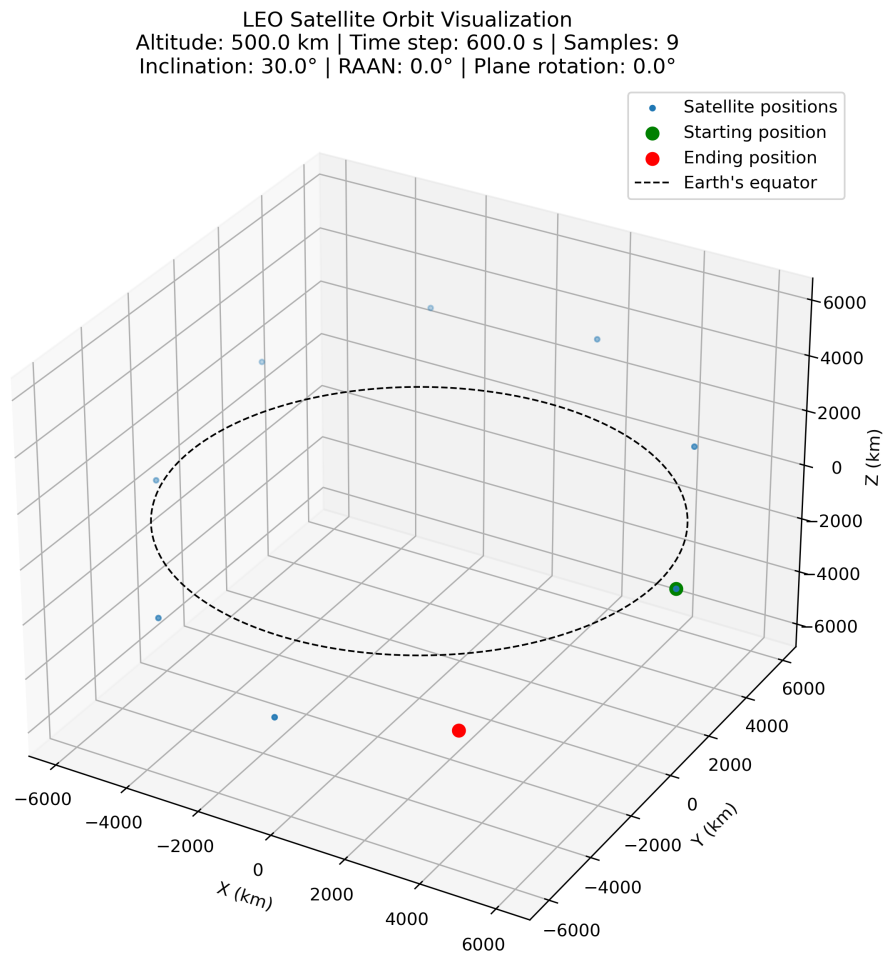


Fig. 22.4: LEO orbit with 30 degrees of inclination, settable via the `LeoCircularOrbitMobilityModel::Inclination` attribute

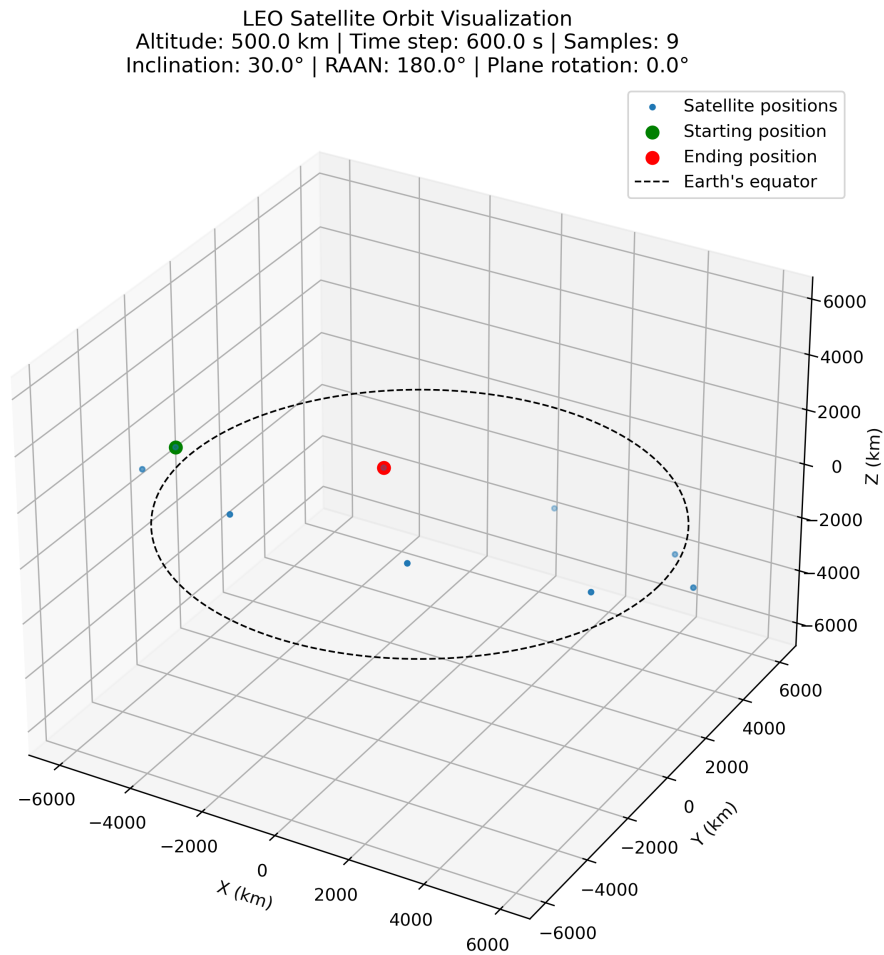


Fig. 22.5: LEO orbit with 30 degrees of inclination and 180 degrees of RAAN rotation

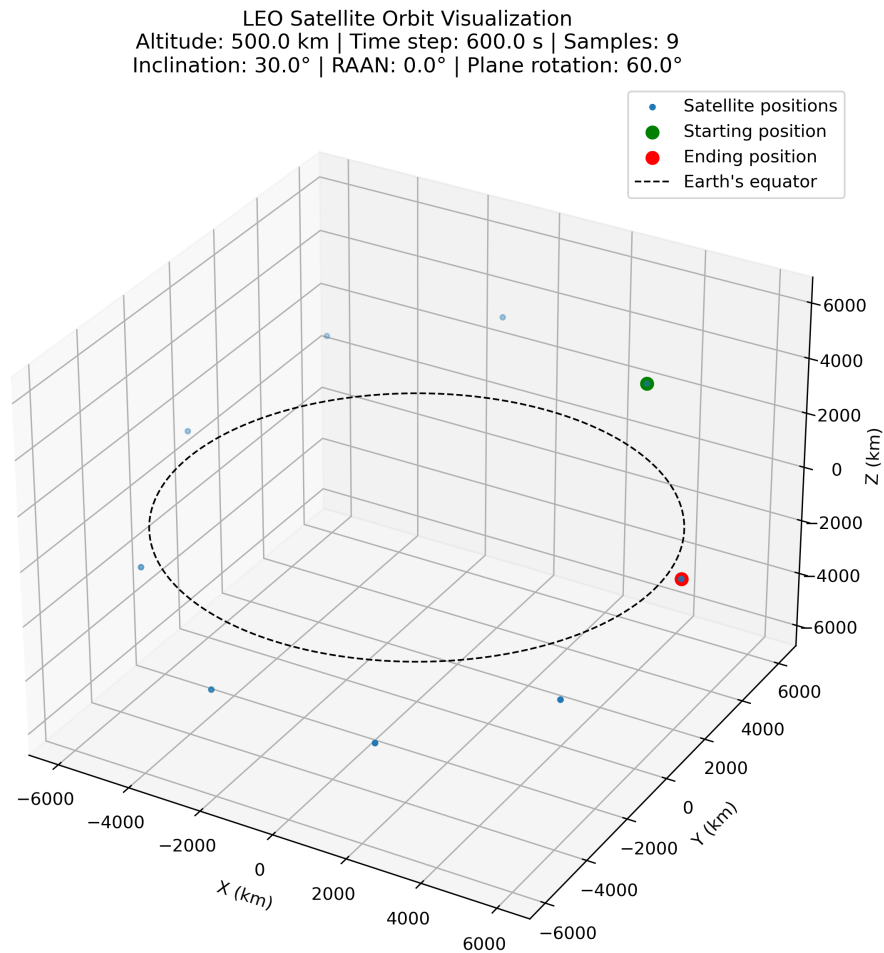


Fig. 22.6: LEO orbit with 30 degrees of inclination, 0 degrees of RAAN rotation, and 60 degrees of plane rotation

Usage

There are three approaches to setting up a LEO satellite constellation:

- **Approach 1:** Use a CSV file with constellation parameters, leveraging the `LeoOrbitNodeHelper` to create satellite nodes and install orbital mobility in a single step.
- **Approach 2:** Use `LeoOrbitalShell` objects with the `LeoOrbitNodeHelper` to configure constellations programmatically and install orbital mobility in a single step.
- **Approach 3:** Use the standard ns-3 `MobilityHelper` and `PositionAllocator` pattern directly, which allows for more fine-grained control over node creation and mobility.

Users will typically want to use the first two approaches.

Approach 1: CSV file with constellation parameters

A CSV file can define one or more orbital shells, each with different altitude, inclination, number of planes, and number of satellites per plane. For example, a file representing part of the Starlink constellation (`examples/leo/starlink.csv`):

```
# altitudeKm,inclinationDegrees,numberOfPlanes,numberOfSatellitesPerPlane
1150.0,53.0,32,50
1110.0,53.8,32,50
1130.0,74.0,8,50
(...)
```

The helper reads the file, creates the appropriate number of nodes for all shells, and installs orbital mobility on each:

```
std::string orbitFile("examples/leo/starlink.csv");
LeoOrbitNodeHelper orbit(resolution);
NodeContainer satellites = orbit.CreateNodesAndInstallMobility(orbitFile);
```

The resolution parameter is a Time value that controls how frequently the mobility model fires `CourseChange` trace notifications. It does not affect the accuracy of the orbital position model itself; satellite positions are always computed exactly on demand when `GetPosition()` is called. The resolution is handled separately from the orbital parameters (altitude, inclination, planes, satellites) because it is not a property of the orbit but rather a simulation-level control over how often trace callbacks are invoked.

Approach 2: `LeoOrbitalShell` objects

The `LeoOrbitalShell` class bundles the following orbital parameters:

- **Altitude** in kilometers
- **Inclination angle** in degrees
- **Number of planes**
- **Number of satellites per plane**
- **Optional** Walker Delta phasing factor
- **Optional** RAAN span in degrees (for Walker Star constellations)

One or more `LeoOrbitalShell` objects can be passed directly to the helper:

```
LeoOrbitNodeHelper orbit(resolution);

// Single shell
NodeContainer satellites = orbit.CreateNodesAndInstallMobility(LeoOrbitalShell(1200, 30, 1, 2));
```

(continues on next page)

(continued from previous page)

```
// Multiple shells
NodeContainer satellites = orbit.CreateNodesAndInstallMobility(
    {LeoOrbitalShell(1200, 30, 32, 16), LeoOrbitalShell(1180, 45, 12, 10)});

// Walker Delta 53: 1600/32/1 constellation with phasing factor F = 1
NodeContainer satellites = orbit.CreateNodesAndInstallMobility(LeoOrbitalShell(1200, 53, 32, 50, 1));
```

Walker Delta constellations are specified in the literature using the notation $i: T/P/F$, where i is the inclination in degrees, T is the total number of satellites, P is the number of equally spaced orbital planes, and F is the phasing factor. The `LeoOrbitalShell` constructor takes the number of satellites per plane S rather than the total (with $T = P * S$); the optional fifth parameter sets the phasing factor F . F staggers satellites in adjacent planes by $F * 360 / T$ degrees. Without phasing ($F = 0$, the default), all planes have their first satellite at the same orbital offset.

For Walker Star constellations (e.g., Iridium), orbital planes are distributed over 180 degrees instead of 360 degrees. Set the `raanSpanDeg` member on the `LeoOrbitalShell` object:

```
// Walker Star: Iridium-like 86.4: 66/6/1 constellation
LeoOrbitalShell orbit(780, 86.4, 6, 11, 1);
orbit.raanSpanDeg = 180.0;
NodeContainer satellites = helper.CreateNodesAndInstallMobility(orbit);
```

The `RaanspanDeg` attribute on `LeoCircularOrbitPositionAllocator` can also be set directly when using approach 3.

In both approaches 1 and 2, `CreateNodesAndInstallMobility()` returns a `NodeContainer` with all satellite nodes already placed at their initial orbital positions.

Approach 3: Standard MobilityHelper pattern

For a single orbital shell, the standard ns-3 `MobilityHelper` and `PositionAllocator` pattern can be used directly:

```
uint16_t numOrbits = 1;
int32_t numSatellites = 16;
NodeContainer satellites(numOrbits * numSatellites);

MobilityHelper mobility;
mobility.SetPositionAllocator("ns3::LeoCircularOrbitPositionAllocator",
    "NumOrbits", IntegerValue(numOrbits),
    "NumSatellites", IntegerValue(numSatellites));
mobility.SetMobilityModel("ns3::LeoCircularOrbitMobilityModel",
    "Altitude", DoubleValue(1200),
    "Inclination", DoubleValue(30));
mobility.Install(satellites);
```

When using this approach, note the following caveats:

- The number of nodes in the `NodeContainer` must equal `NumOrbits * NumSatellites`. If these are inconsistent, the position allocator will wrap around or leave nodes at default positions.
- The `Altitude` and `Inclination` attributes on the mobility model apply uniformly to all nodes in a single `Install()` call. For multi-shell constellations (e.g., Starlink with shells at different altitudes), a separate `MobilityHelper` and `Install()` call is required for each shell.

Resolution

As described above, the `Resolution` attribute controls how often `CourseChange` trace notifications are fired. A value of zero disables periodic notifications entirely.

leo-satellite-example

This unified example combines three use cases into a single program, controlled by the `--mode` switch. It sets up a LEO constellation (from a CSV file or a default orbit), places a ground station, and optionally steers each satellite's antenna toward the ground station. At each `CourseChange` event, it outputs CSV trace data.

Modes

The `--mode` switch selects one of three modes:

- `orbit` Mobility tracing only (position + speed).
- `antenna` Antenna pointing toward a ground station (no propagation).
- `channel` Full 3GPP NTN propagation loss estimation.

Common command-line options

- `orbitFile`: path to a CSV containing orbit parameters for satellites
- `traceFile`: path to a CSV file to store trace data; if omitted or empty, traces are written to stdout
- `resolution`: time interval between `CourseChange` notifications (seconds)
- `duration`: total simulation time in seconds

Mode-specific options

- `groundStation`: ground station location as "lat,lon,alt" or "auto" to place it beneath the first satellite (default: "41.275,1.988,14")
- `scenario`: NTN propagation scenario (channel mode only); NTN-DenseUrban, NTN-Urban, NTN-Suburban, NTN-Rural (default: NTN-Rural)
- `frequency`: carrier frequency in Hz (channel mode only, default: 2e9)
- `antennaGain`: satellite antenna gain in dBi (antenna/channel mode, default: 34.6)
- `antennaUpdatePeriod`: antenna orientation update interval (default: 500 ms)

Running in orbit mode (mobility tracing only):

```
$ ./ns3 run "leo-satellite-example --mode=orbit --duration=10s \  
--resolution=500ms \  
--orbitFile=src/mobility/examples/telesat.csv \  
--traceFile=telesat-trace.csv"
```

Running in antenna mode (antenna pointing toward ground station):

```
$ ./ns3 run "leo-satellite-example --mode=antenna --duration=10s \  
--resolution=500ms \  
--orbitFile=src/mobility/examples/telesat.csv \  
--traceFile=telesat-trace.csv"
```

In antenna mode, the `CourseChange` callback outputs two ECEF coordinates per event: the satellite position and a second point along the antenna pointing direction toward the ground station. These can be visualized with `plot-orbital-traces.py` to generate an animated plot showing each satellite's antenna pointing vector.

Running in channel mode (full 3GPP NTN propagation):

```
$ ./ns3 run "leo-satellite-example --mode=channel --duration=60s \
--resolution=1000ms \
--scenario=NTN-Rural \
--frequency=2e9 \
--orbitFile=/path/to/orbit-parameters.csv \
--traceFile=/path/to/leo-ntn-trace.csv"
```

In channel mode, the example outputs a colon-separated trace containing the simulation time, satellite node id, satellite ECEF position, the endpoint of the antenna pointing vector, path loss in dB, slant distance in meters, and elevation angle in degrees.

Sample constellation CSV files are provided in `examples/leo/`:

- `starlink.csv` (Walker Delta)
- `one-web.csv` (Walker Star)
- `telesat.csv` (Walker Delta)

To use the Starlink constellation:

```
~/ns-3-dev/$ ./ns3 run "leo-satellite-example --mode=antenna --duration=1000s \
--resolution=500ms \
--orbitFile=src/mobility/examples/starlink.csv \
--traceFile=starlink-orbit-trace"
```

plot-orbital-traces.py

With this script we are able to generate animated plots which we can use to validate whether the implementations and programs are correctly placing satellite nodes in their initial positions, and whether their positions are being correctly updated, as we can see the nodes orbiting around Earth in the plot. To generate this plot, the user must pass two arguments via command line, specifying file paths to the input file, where traces will be taken for plotting, and the output file, which is the output video file itself. Additionally, if the user passes the flag `-a`, then the script will also plot a vector from each satellite toward the ground station, showing the antenna pointing direction. This requires input data produced by the `leo-satellite-example` with `--mode=antenna`.

To execute it, run the following:

```
$ plot-orbital-traces.py -i <path-to-traceFile> -o <path-to-output-file>.mp4 -a
```

The `-a` option plots the antenna pointing vector for each satellite, but requires trace data from the `leo-satellite-example` with `--mode=antenna`. For example, using the Starlink orbit traces:

```
~/ns-3-dev/$ ./src/mobility/examples/plot-orbital-traces.py -i starlink-orbit-trace -o
↳ starlink-animation.mp4
```

To also plot the antenna pointing vector toward the ground station, try:

```
~/ns-3-dev/$ ./ns3 run "leo-satellite-example --mode=antenna --duration=100"
~/ns-3-dev/$ ./src/mobility/examples/plot-orbital-traces.py -i leo-satellite-example-
↳ trace.csv -o satellite-antenna-trace.mp4 -a
```

By default, the trace is written to `leo-satellite-example-trace.csv`. Use `--traceFile=<path>` to change the output file path, or `--writeTrace=false` to write to stdout instead.

References

[4] ns-3-leo

22.8 References

- [1] **T. Camp, J. Boleng, V. Davies.** “A survey of mobility models for ad hoc network research”, in *Wireless Communications and Mobile Computing*, 2002: vol. 2, pp. 2483-2502.
- [2] 3GPP. 2018. TR 38.811, Study on New Radio (NR) to support non-terrestrial networks, V15.4.0. (2020-09).
- [3] BonnMotion documentation

NETWORK MODULE

23.1 Packets

The design of the Packet framework of *ns* was heavily guided by a few important use-cases:

- avoid changing the core of the simulator to introduce new types of packet headers or trailers
- maximize the ease of integration with real-world code and systems
- make it easy to support fragmentation, defragmentation, and, concatenation which are important, especially in wireless systems.
- make memory management of this object efficient
- allow actual application data or dummy application bytes for emulated applications

Each network packet contains a byte buffer, a set of byte tags, a set of packet tags, and metadata.

The byte buffer stores the serialized content of the headers and trailers added to a packet. The serialized representation of these headers is expected to match that of real network packets bit for bit (although nothing forces you to do this) which means that the content of a packet buffer is expected to be that of a real packet.

Fragmentation and defragmentation are quite natural to implement within this context: since we have a buffer of real bytes, we can split it in multiple fragments and re-assemble these fragments. We expect that this choice will make it really easy to wrap our Packet data structure within Linux-style skb or BSD-style mbuf to integrate real-world kernel code in the simulator. We also expect that performing a real-time plug of the simulator to a real-world network will be easy.

One problem that this design choice raises is that it is difficult to pretty-print the packet headers without context. The packet metadata describes the type of the headers and trailers which were serialized in the byte buffer. The maintenance of metadata is optional and disabled by default. To enable it, you must call `Packet::EnablePrinting()` and this will allow you to get non-empty output from `Packet::Print()` and `Packet::Print()`.

Also, developers often want to store data in packet objects that is not found in the real packets (such as timestamps or flow-ids). The Packet class deals with this requirement by storing a set of tags (class Tag). We have found two classes of use cases for these tags, which leads to two different types of tags. So-called ‘byte’ tags are used to tag a subset of the bytes in the packet byte buffer while ‘packet’ tags are used to tag the packet itself. The main difference between these two kinds of tags is what happens when packets are copied, fragmented, and reassembled: ‘byte’ tags follow bytes while ‘packet’ tags follow packets. Another important difference between these two kinds of tags is that byte tags cannot be removed and are expected to be written once, and read many times, while packet tags are expected to be written once, read many times, and removed exactly once. An example of a ‘byte’ tag is a `FlowIdTag` which contains a flow id and is set by the application generating traffic. An example of a ‘packet’ tag is a cross-layer QoS class id set by an application and processed by a lower-level MAC layer.

Memory management of Packet objects is entirely automatic and extremely efficient: memory for the application-level payload can be modeled by a virtual buffer of zero-filled bytes for which memory is never allocated unless explicitly requested by the user or unless the packet is fragmented or serialized out to a real network device. Furthermore, copying,

adding, and, removing headers or trailers to a packet has been optimized to be virtually free through a technique known as Copy On Write.

Packets (messages) are fundamental objects in the simulator and their design is important from a performance and resource management perspective. There are various ways to design the simulation packet, and tradeoffs among the different approaches. In particular, there is a tension between ease-of-use, performance, and safe interface design.

23.1.1 Packet design overview

Unlike *ns-2*, in which Packet objects contain a buffer of C++ structures corresponding to protocol headers, each network packet in *ns-3* contains a byte Buffer, a list of byte Tags, a list of packet Tags, and a PacketMetadata object:

- The byte buffer stores the serialized content of the chunks added to a packet. The serialized representation of these chunks is expected to match that of real network packets bit for bit (although nothing forces you to do this) which means that the content of a packet buffer is expected to be that of a real packet. Packets can also be created with an arbitrary zero-filled payload for which no real memory is allocated.
- Each list of tags stores an arbitrarily large set of arbitrary user-provided data structures in the packet. Each Tag is uniquely identified by its type; only one instance of each type of data structure is allowed in a list of tags. These tags typically contain per-packet cross-layer information or flow identifiers (i.e., things that you wouldn't find in the bits on the wire).

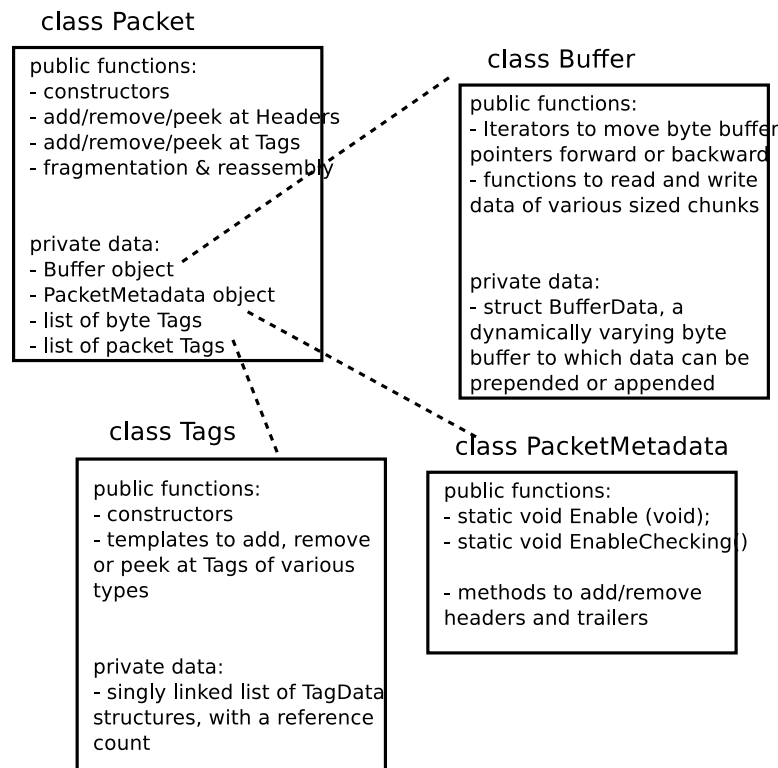


Fig. 23.1: Implementation overview of Packet class.

Figure *Implementation overview of Packet class* is a high-level overview of the Packet implementation; more detail on the byte Buffer implementation is provided later in Figure *Implementation overview of a packet's byte Buffer*. In *ns-3*, the Packet byte buffer is analogous to a Linux skbuff or BSD mbuf; it is a serialized representation of the actual data in the packet. The tag lists are containers for extra items useful for simulation convenience; if a Packet is converted to an emulated packet and put over an actual network, the tags are stripped off and the byte buffer is copied directly into a real packet.

Packets are reference counted objects. They are handled with smart pointer (Ptr) objects like many of the objects in the ns-3 system. One small difference you will see is that class Packet does not inherit from class Object or class RefCountBase, and implements the Ref() and Unref() methods directly. This was designed to avoid the overhead of a vtable in class Packet.

The Packet class is designed to be copied cheaply; the overall design is based on Copy on Write (COW). When there are multiple references to a packet object, and there is an operation on one of them, only so-called “dirty” operations will trigger a deep copy of the packet:

- ns3::Packet::AddHeader()
- ns3::Packet::AddTrailer()
- both versions of ns3::Packet::AddAtEnd()
- Packet::RemovePacketTag()

The fundamental classes for adding to and removing from the byte buffer are class Header and class Trailer. Headers are more common but the below discussion also largely applies to protocols using trailers. Every protocol header that needs to be inserted and removed from a Packet instance should derive from the abstract Header base class and implement the private pure virtual methods listed below:

- ns3::Header::SerializeTo()
- ns3::Header::DeserializeFrom()
- ns3::Header::GetSerializedSize()
- ns3::Header::PrintTo()

Basically, the first three functions are used to serialize and deserialize protocol control information to/from a Buffer. For example, one may define class TCPHeader : public Header. The TCPHeader object will typically consist of some private data (like a sequence number) and public interface access functions (such as checking the bounds of an input). But the underlying representation of the TCPHeader in a Packet Buffer is 20 serialized bytes (plus TCP options). The TCPHeader::SerializeTo() function would therefore be designed to write these 20 bytes properly into the packet, in network byte order. The last function is used to define how the Header object prints itself onto an output stream.

Similarly, user-defined Tags can be appended to the packet. Unlike Headers, Tags are not serialized into a contiguous buffer but are stored in lists. Tags can be flexibly defined to be any type, but there can only be one instance of any particular object type in the Tags buffer at any time.

23.1.2 Using the packet interface

This section describes how to create and use the ns3::Packet object.

Creating a new packet

The following command will create a new packet with a new unique Id.:

```
Ptr<Packet> pkt = Create<Packet>();
```

What is the Uid (unique Id)? It is an internal id that the system uses to identify packets. It can be fetched via the following method:

```
uint32_t uid = pkt->GetUid();
```

But please note the following. This uid is an internal uid and cannot be counted on to provide an accurate counter of how many “simulated packets” of a particular protocol are in the system. It is not trivial to make this uid into such a counter, because of questions such as what should the uid be when the packet is sent over broadcast media, or when

fragmentation occurs. If a user wants to trace actual packet counts, he or she should look at e.g. the IP ID field or transport sequence numbers, or other packet or frame counters at other protocol layers.

We mentioned above that it is possible to create packets with zero-filled payloads that do not actually require a memory allocation (i.e., the packet may behave, when delays such as serialization or transmission delays are computed, to have a certain number of payload bytes, but the bytes will only be allocated on-demand when needed). The command to do this is, when the packet is created:

```
Ptr<Packet> pkt = Create<Packet>(N);
```

where N is a positive integer.

The packet now has a size of N bytes, which can be verified by the `GetSize()` method:

```
/**
 * \returns the size in bytes of the packet (including the zero-filled
 *          initial payload)
 */
uint32_t GetSize() const;
```

You can also initialize a packet with a character buffer. The input data is copied and the input buffer is untouched. The constructor applied is:

```
Packet(uint8_t const *buffer, uint32_t size);
```

Here is an example:

```
Ptr<Packet> pkt1 = Create<Packet>(reinterpret_cast<const uint8_t*>("hello"), 5);
```

Packets are freed when there are no more references to them, as with all *ns-3* objects referenced by the `Ptr` class.

Adding and removing Buffer data

After the initial packet creation (which may possibly create some fake initial bytes of payload), all subsequent buffer data is added by adding objects of class `Header` or class `Trailer`. Note that, even if you are in the application layer, handling packets, and want to write application data, you write it as an `ns3::Header` or `ns3::Trailer`. If you add a `Header`, it is prepended to the packet, and if you add a `Trailer`, it is added to the end of the packet. If you have no data in the packet, then it makes no difference whether you add a `Header` or `Trailer`. Since the APIs and classes for header and trailer are pretty much identical, we'll just look at class `Header` here.

The first step is to create a new header class. All new `Header` classes must inherit from class `Header`, and implement the following methods:

- `Serialize ()`
- `Deserialize ()`
- `GetSerializedSize ()`
- `Print ()`

To see a simple example of how these are done, look at the `UdpHeader` class headers `src/internet/model/udp-header.cc`. There are many other examples within the source code.

Once you have a header (or you have a preexisting header), the following `Packet` API can be used to add or remove such headers.:

```

/**
 * Add header to this packet. This method invokes the
 * Header::GetSerializedSize and Header::Serialize
 * methods to reserve space in the buffer and request the
 * header to serialize itself in the packet buffer.
 *
 * \param header a reference to the header to add to this packet.
 */
void AddHeader(const Header & header);

/**
 * Deserialize and remove the header from the internal buffer.
 *
 * This method invokes Header::Deserialize(begin) and should be used for
 * fixed-length headers.
 *
 * \param header a reference to the header to remove from the internal buffer.
 * \returns the number of bytes removed from the packet.
 */
uint32_t RemoveHeader(Header &header);

/**
 * Deserialize but does _not_ remove the header from the internal buffer.
 * This method invokes Header::Deserialize.
 *
 * \param header a reference to the header to read from the internal buffer.
 * \returns the number of bytes read from the packet.
 */
uint32_t PeekHeader(Header &header) const;

```

For instance, here are the typical operations to add and remove a UDP header.:

```

// add header
Ptr<Packet> packet = Create<Packet>();
UdpHeader udpHeader;
// Fill out udpHeader fields appropriately
packet->AddHeader(udpHeader);
...
// remove header
UdpHeader udpHeader;
packet->RemoveHeader(udpHeader);
// Read udpHeader fields as needed

```

If the header is variable-length, then another variant of RemoveHeader() is needed:

```

/**
 * \brief Deserialize and remove the header from the internal buffer.
 *
 * This method invokes Header::Deserialize(begin, end) and should be
 * used for variable-length headers (where the size is determined somehow
 * by the caller).
 *
 * \param header a reference to the header to remove from the internal buffer.
 * \param size number of bytes to deserialize
 * \returns the number of bytes removed from the packet.

```

(continues on next page)

(continued from previous page)

```
*/
uint32_t RemoveHeader(Header &header, uint32_t size);
```

In this case, the caller must figure out and provide the right ‘size’ as an argument (the Deserialization routine may not know when to stop). An example of this type of header would be a series of Type-Length-Value (TLV) information elements, where the ending point of the series of TLVs can be deduced from the packet length.

Adding and removing Tags

There is a single base class of Tag that all packet tags must derive from. They are used in two different tag lists in the packet; the lists have different semantics and different expected use cases.

As the names imply, ByteTags follow bytes and PacketTags follow packets. What this means is that when operations are done on packets, such as fragmentation, concatenation, and appending or removing headers, the byte tags keep track of which packet bytes they cover. For instance, if a user creates a TCP segment, and applies a ByteTag to the segment, each byte of the TCP segment will be tagged. However, if the next layer down inserts an IPv4 header, this ByteTag will not cover those bytes. The converse is true for the PacketTag; it covers a packet despite the operations on it.

Each tag type must subclass ns3::Tag, and only one instance of each Tag type may be in each tag list. Here are a few differences in the behavior of packet tags and byte tags.

- **Fragmentation:** As mentioned above, when a packet is fragmented, each packet fragment (which is a new packet) will get a copy of all packet tags, and byte tags will follow the new packet boundaries (i.e. if the fragmented packets fragment across a buffer region covered by the byte tag, both packet fragments will still have the appropriate buffer regions byte tagged).
- **Concatenation:** When packets are combined, two different buffer regions will become one. For byte tags, the byte tags simply follow the respective buffer regions. For packet tags, only the tags on the first packet survive the merge.
- **Finding and Printing:** Both classes allow you to iterate over all of the tags and print them.
- **Removal:** Users can add and remove the same packet tag multiple times on a single packet (AddPacketTag() and RemovePacketTag()). The packet However, once a byte tag is added, it can only be removed by stripping all byte tags from the packet. Removing one of possibly multiple byte tags is not supported by the current API.

If a user wants to take an existing packet object and reuse it as a new packet, he or she should remove all byte tags and packet tags before doing so. An example is the UdpEchoServer class, which takes the received packet and “turns it around” to send back to the echo client.

The Packet API for byte tags is given below.:

```
/**
 * \param tag the new tag to add to this packet
 *
 * Tag each byte included in this packet with the
 * new tag.
 *
 * Note that adding a tag is a const operation which is pretty
 * un-intuitive. The rationale is that the content and behavior of
 * a packet is not changed when a tag is added to a packet: any
 * code which was not aware of the new tag is going to work just
 * the same if the new tag is added. The real reason why adding a
 * tag was made a const operation is to allow a trace sink which gets
 * a packet to tag the packet, even if the packet is const (and most
```

(continues on next page)

(continued from previous page)

```

* trace sources should use const packets because it would be
* totally evil to allow a trace sink to modify the content of a
* packet).
*/
void AddByteTag(const Tag &tag) const;
/**
 * \returns an iterator over the set of byte tags included in this packet.
 */
ByteTagIterator GetByteTagIterator() const;
/**
 * \param tag the tag to search in this packet
 * \returns true if the requested tag type was found, false otherwise.
 *
 * If the requested tag type is found, it is copied in the user's
 * provided tag instance.
 */
bool FindFirstMatchingByteTag(Tag &tag) const;

/**
 * Remove all the tags stored in this packet.
 */
void RemoveAllByteTags();

/**
 * \param os output stream in which the data should be printed.
 *
 * Iterate over the tags present in this packet, and
 * invoke the Print method of each tag stored in the packet.
 */
void PrintByteTags(std::ostream &os) const;

```

The Packet API for packet tags is given below.:

```

/**
 * \param tag the tag to store in this packet
 *
 * Add a tag to this packet. This method calls the
 * Tag::GetSerializedSize and, then, Tag::Serialize.
 *
 * Note that this method is const, that is, it does not
 * modify the state of this packet, which is fairly
 * un-intuitive.
 */
void AddPacketTag(const Tag &tag) const;
/**
 * \param tag the tag to remove from this packet
 * \returns true if the requested tag is found, false
 * otherwise.
 *
 * Remove a tag from this packet. This method calls
 * Tag::Deserialize if the tag is found.
 */

```

(continues on next page)

(continued from previous page)

```

bool RemovePacketTag(Tag &tag);
/**
 * \param tag the tag to search in this packet
 * \returns true if the requested tag is found, false
 *         otherwise.
 *
 * Search a matching tag and call Tag::Deserialize if it is found.
 */
bool PeekPacketTag(Tag &tag) const;
/**
 * Remove all packet tags.
 */
void RemoveAllPacketTags();

/**
 * \param os the stream in which we want to print data.
 *
 * Print the list of 'packet' tags.
 *
 * \sa Packet::AddPacketTag, Packet::RemovePacketTag, Packet::PeekPacketTag,
 *     Packet::RemoveAllPacketTags
 */
void PrintPacketTags(std::ostream &os) const;

/**
 * \returns an object which can be used to iterate over the list of
 * packet tags.
 */
PacketTagIterator GetPacketTagIterator() const;

```

Here is a simple example illustrating the use of tags from the code in `src/internet/model/udp-socket-impl.cc`:

```

Ptr<Packet> p; // pointer to a pre-existing packet
SocketIpTtlTag tag
tag.SetTtl(m_ipMulticastTtl); // Convey the TTL from UDP layer to IP layer
p->AddPacketTag(tag);

```

This tag is read at the IP layer, then stripped (`src/internet/model/ipv4-l3-protocol.cc`):

```

uint8_t ttl = m_defaultTtl;
SocketIpTtlTag tag;
bool found = packet->RemovePacketTag(tag);
if (found)
{
    ttl = tag.GetTtl();
}

```


Fragmentation and concatenation

Packets may be fragmented or merged together. For example, to fragment a packet `p` of 90 bytes into two packets, one containing the first 10 bytes and the other containing the remaining 80, one may call the following code:

```
Ptr<Packet> frag0 = p->CreateFragment(0, 10);
Ptr<Packet> frag1 = p->CreateFragment(10, 90);
```

As discussed above, the packet tags from `p` will follow to both packet fragments, and the byte tags will follow the byte ranges as needed.

Now, to put them back together:

```
frag0->AddAtEnd(frag1);
```

Now `frag0` should be equivalent to the original packet `p`. If, however, there were operations on the fragments before being reassembled (such as tag operations or header operations), the new packet will not be the same.

Enabling metadata

We mentioned above that packets, being on-the-wire representations of byte buffers, present a problem to print out in a structured way unless the printing function has access to the context of the header. For instance, consider a `tcpdump`-like printer that wants to pretty-print the contents of a packet.

To enable this usage, packets may have metadata enabled (disabled by default for performance reasons). This class is used by the `Packet` class to record every operation performed on the packet's buffer, and provides an implementation of `Packet::Print()` method that uses the metadata to analyze the content of the packet's buffer.

The metadata is also used to perform extensive sanity checks at runtime when performing operations on a `Packet`. For example, this metadata is used to verify that when you remove a header from a packet, this same header was actually present at the front of the packet. These errors will be detected and will abort the program.

To enable this operation, users will typically insert one or both of these statements at the beginning of their programs:

```
Packet::EnablePrinting();
Packet::EnableChecking();
```

23.1.3 Sample programs

See `src/network/examples/main-packet-header.cc` and `src/network/examples/main-packet-tag.cc`.

23.1.4 Implementation details

Private member variables

A `Packet` object's interface provides access to some private data:

```
Buffer m_buffer;
ByteTagList m_byteTagList;
PacketTagList m_packetTagList;
PacketMetadata m_metadata;
mutable uint32_t m_refCount;
static uint32_t m_globalUid;
```

Each Packet has a Buffer and two Tags lists, a PacketMetadata object, and a ref count. A static member variable keeps track of the UIDs allocated. The actual uid of the packet is stored in the PacketMetadata.

Note: that real network packets do not have a UID; the UID is therefore an instance of data that normally would be stored as a Tag in the packet. However, it was felt that a UID is a special case that is so often used in simulations that it would be more convenient to store it in a member variable.

Buffer implementation

Class Buffer represents a buffer of bytes. Its size is automatically adjusted to hold any data prepended or appended by the user. Its implementation is optimized to ensure that the number of buffer resizes is minimized, by creating new Buffers of the maximum size ever used. The correct maximum size is learned at runtime during use by recording the maximum size of each packet.

Authors of new Header or Trailer classes need to know the public API of the Buffer class. (add summary here)

The byte buffer is implemented as follows:

```
struct BufferData {
    uint32_t m_count;
    uint32_t m_size;
    uint32_t m_initialStart;
    uint32_t m_dirtyStart;
    uint32_t m_dirtySize;
    uint8_t m_data[1];
};
struct BufferData *m_data;
uint32_t m_zeroAreaSize;
uint32_t m_start;
uint32_t m_size;
```

- BufferData::m_count: reference count for BufferData structure
- BufferData::m_size: size of data buffer stored in BufferData structure
- BufferData::m_initialStart: offset from start of data buffer where data was first inserted
- BufferData::m_dirtyStart: offset from start of buffer where every Buffer which holds a reference to this BufferData instance have written data so far
- BufferData::m_dirtySize: size of area where data has been written so far
- BufferData::m_data: pointer to data buffer
- Buffer::m_zeroAreaSize: size of zero area which extends before m_initialStart
- Buffer::m_start: offset from start of buffer to area used by this buffer
- Buffer::m_size: size of area used by this Buffer in its BufferData structure

This data structure is summarized in Figure *Implementation overview of a packet's byte Buffer*. Each Buffer holds a pointer to an instance of a BufferData. Most Buffers should be able to share the same underlying BufferData and thus simply increase the BufferData's reference count. If they have to change the content of a BufferData inside the Dirty Area, and if the reference count is not one, they first create a copy of the BufferData and then complete their state-changing operation.

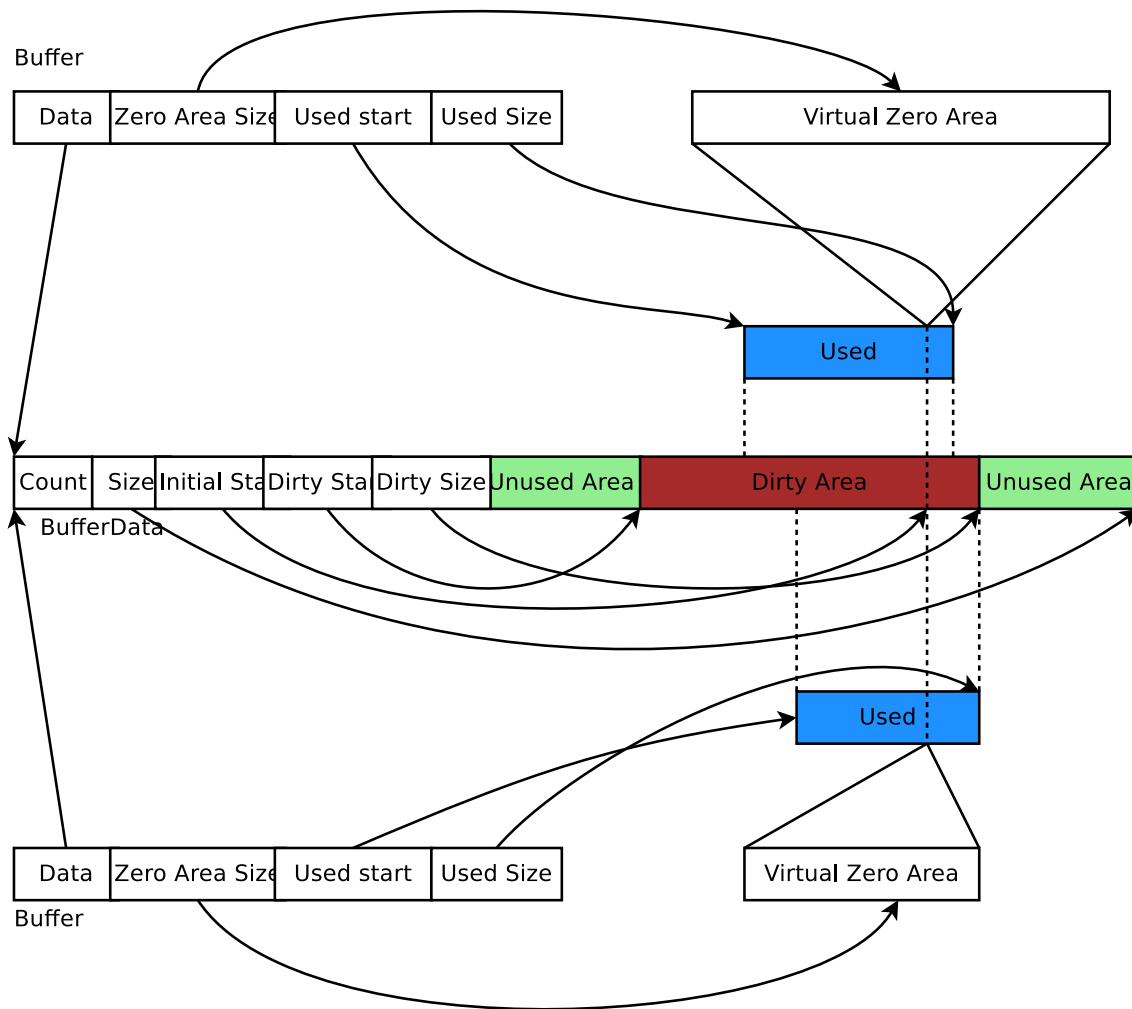


Fig. 23.2: Implementation overview of a packet's byte Buffer.

Tags implementation

(XXX revise me)

Tags are implemented by a single pointer which points to the start of a linked list of TagData data structures. Each TagData structure points to the next TagData in the list (its next pointer contains zero to indicate the end of the linked list). Each TagData contains an integer unique id which identifies the type of the tag stored in the TagData.:

```
struct TagData {
    struct TagData *m_next;
    uint32_t m_id;
    uint32_t m_count;
    uint8_t m_data[Tags::SIZE];
};
class Tags {
    struct TagData *m_next;
};
```

Adding a tag is a matter of inserting a new TagData at the head of the linked list. Looking at a tag requires you to find the relevant TagData in the linked list and copy its data into the user data structure. Removing a tag and updating the content of a tag requires a deep copy of the linked list before performing this operation. On the other hand, copying a Packet and its tags is a matter of copying the TagData head pointer and incrementing its reference count.

Tags are found by the unique mapping between the Tag type and its underlying id. This is why at most one instance of any Tag can be stored in a packet. The mapping between Tag type and underlying id is performed by a registration as follows:

```
/* A sample Tag implementation
 */
struct MyTag {
    uint16_t m_streamId;
};
```

Memory management

Describe dataless vs. data-full packets.

Copy-on-write semantics

The current implementation of the byte buffers and tag list is based on COW (Copy On Write). An introduction to COW can be found in Scott Meyer's "More Effective C++", items 17 and 29). This design feature and aspects of the public interface borrows from the packet design of the Georgia Tech Network Simulator. This implementation of COW uses a customized reference counting smart pointer class.

What COW means is that copying packets without modifying them is very cheap (in terms of CPU and memory usage) and modifying them can be also very cheap. What is key for proper COW implementations is being able to detect when a given modification of the state of a packet triggers a full copy of the data prior to the modification: COW systems need to detect when an operation is "dirty" and must therefore invoke a true copy.

Dirty operations:

- ns3::Packet::AddHeader
- ns3::Packet::AddTrailer
- both versions of ns3::Packet::AddAtEnd

- `ns3::Packet::RemovePacketTag`

Non-dirty operations:

- `ns3::Packet::AddPacketTag`
- `ns3::Packet::PeekPacketTag`
- `ns3::Packet::RemoveAllPacketTags`
- `ns3::Packet::AddByteTag`
- `ns3::Packet::FindFirstMatchingByteTag`
- `ns3::Packet::RemoveAllByteTags`
- `ns3::Packet::RemoveHeader`
- `ns3::Packet::RemoveTrailer`
- `ns3::Packet::CreateFragment`
- `ns3::Packet::RemoveAtStart`
- `ns3::Packet::RemoveAtEnd`
- `ns3::Packet::CopyData`

Dirty operations will always be slower than non-dirty operations, sometimes by several orders of magnitude. However, even the dirty operations have been optimized for common use-cases which means that most of the time, these operations will not trigger data copies and will thus be still very fast.

23.2 Error Model

This section documents a few error model objects, typically associated with `NetDevice` models, that are maintained as part of the network module:

- `RateErrorModel`
- `ListErrorModel`
- `ReceiveListErrorModel`
- `BurstErrorModel`
- `BinaryErrorModel`

Error models are used to indicate that a packet should be considered to be errored, according to the underlying (possibly stochastic or empirical) error model.

23.2.1 Model Description

The source code for error models live in the directory `src/network/Utils`.

Two types of error models are generally provided. The first are stochastic models. In this case, packets are errored according to underlying random variable distributions. An example of this is the `RateErrorModel`.

The `ns3::RateErrorModel` errors packets according to an underlying random variable distribution, which is by default a `UniformRandomVariable` distributed between 0.0 and 1.0. The error rate and error units (bit, byte, or packet) are set by the user. For instance, by setting `ErrorRate` to 0.1 and `ErrorUnit` to “Packet”, in the long run, around 10% of the packets will be lost.

Another stochastic error model is the `BurstErrorModel` which allows configuration of (random) burst error drop patterns.

The other type of model is a deterministic model, in which packets are errored according to a particular prescribed pattern. An example is the `ListErrorModel` that allows users to specify the list of packets to be errored, by listing the specific packet UIDs. A variant of the `ListErrorModel`, the `ReceiveListErrorModel`, allows users to list packets to be errored by the order of their arrivals (without regard to their UIDs). The `BinaryErrorModel` alternates between errored and error-free packets.

Design

Error models are *ns-3* objects and can be created using the typical pattern of `CreateObject<>()`. They have configuration attributes.

An `ErrorModel` can be applied anywhere, but are commonly deployed on `NetDevice` models so that artificial losses (mimicking channel losses) can be induced.

Scope and Limitations

No known limitations. There are no existing models that try to modify the packet contents (e.g. apply bit or byte errors to the byte buffers). This type of operation will likely be performance-expensive, and existing Packet APIs may not easily support it.

Most *ns-3* wireless models have their own technology-specific error models and do not typically use these packet error models, although some (e.g., Wi-Fi) support the addition of `ErrorModel` as an optional model.

References

The initial *ns-3* error models were ported from ns-2 (`queue/errmodel.{cc,h}`)

23.2.2 Usage

The base class API is as follows:

- `bool ErrorModel::IsCorrupt (Ptr<Packet> pkt)`: Evaluate the packet and return true or false whether the packet should be considered errored or not. Some models could potentially alter the contents of the packet bit buffer.
- `void ErrorModel::Reset ()`: Reset any state.
- `void ErrorModel::Enable ()`: Enable the model
- `void ErrorModel::Disable ()`: Disable the model; `IsCorrupt()` will always return false.
- `bool ErrorModel::IsEnabled () const`: Return the enabled state

Several *ns-3* `NetDevices` contain attributes holding pointers to error models. The error model is applied in the notional physical layer processing chain of the device, and drops should show up on the `PhyRxDrop` trace source of the device. The following are known to include an attribute with a pointer available to hold this type of error model:

- `SimpleNetDevice`
- `PointToPointNetDevice`
- `CsmaNetDevice`
- `VirtualNetDevice`

However, the `ErrorModel` could be used anywhere where packets are used

Helpers

This model is typically not used with helpers.

Examples

Error models are used in the tutorial fifth and sixth programs.

The directory `examples/error-model/` contains an example `simple-error-model.cc` that exercises the Rate and List error models.

Troubleshooting

No known issues.

23.2.3 Validation

The `error-model` unit test suite provides a single test case of a particular combination of `ErrorRate` and `ErrorUnit` for the `RateErrorModel` applied to a `SimpleNetDevice`.

23.2.4 Acknowledgements

The basic `ErrorModel`, `RateErrorModel`, and `ListErrorModel` classes were ported from *ns-2* to *ns-3* in 2007. The `ReceiveListErrorModel` was added at that time.

The burst error model is due to Truc Anh N. Nguyen at the University of Kansas (James P.G. Sterbenz <jpgs@ittc.ku.edu>, director, ResiliNets Research Group (<https://resilinetns.org/>), Information and Telecommunication Technology Center (ITTC) and Department of Electrical Engineering and Computer Science, The University of Kansas Lawrence, KS USA). Work supported in part by NSF FIND (Future Internet Design) Program under grant CNS-0626918 (Postmodern Internet Architecture), NSF grant CNS-1050226 (Multilayer Network Resilience Analysis and Experimentation on GENI), US Department of Defense (DoD), and ITTC at The University of Kansas.

23.3 Node and NetDevices Overview

This chapter describes how *ns-3* nodes are put together, and provides a walk-through of how packets traverse an internet-based Node.

In *ns-3*, nodes are instances of `ns3::Node`. This class may be subclassed, but instead, the conceptual model is that we *aggregate* or insert objects to it rather than define subclasses.

One might think of a bare *ns-3* node as a shell of a computer, to which one may add `NetDevices` (cards) and other innards including the protocols and applications. *High-level node architecture* illustrates that `ns3::Node` objects contain a list of `ns3::Application` instances (initially, the list is empty), a list of `ns3::NetDevice` instances (initially, the list is empty), a list of `ns3::Node::ProtocolHandler` instances, a unique integer ID, and a system ID (for distributed simulation).

The design tries to avoid putting too many dependencies on the class `ns3::Node`, `ns3::Application`, or `ns3::NetDevice` for the following:

- IP version, or whether IP is at all even used in the `ns3::Node`.

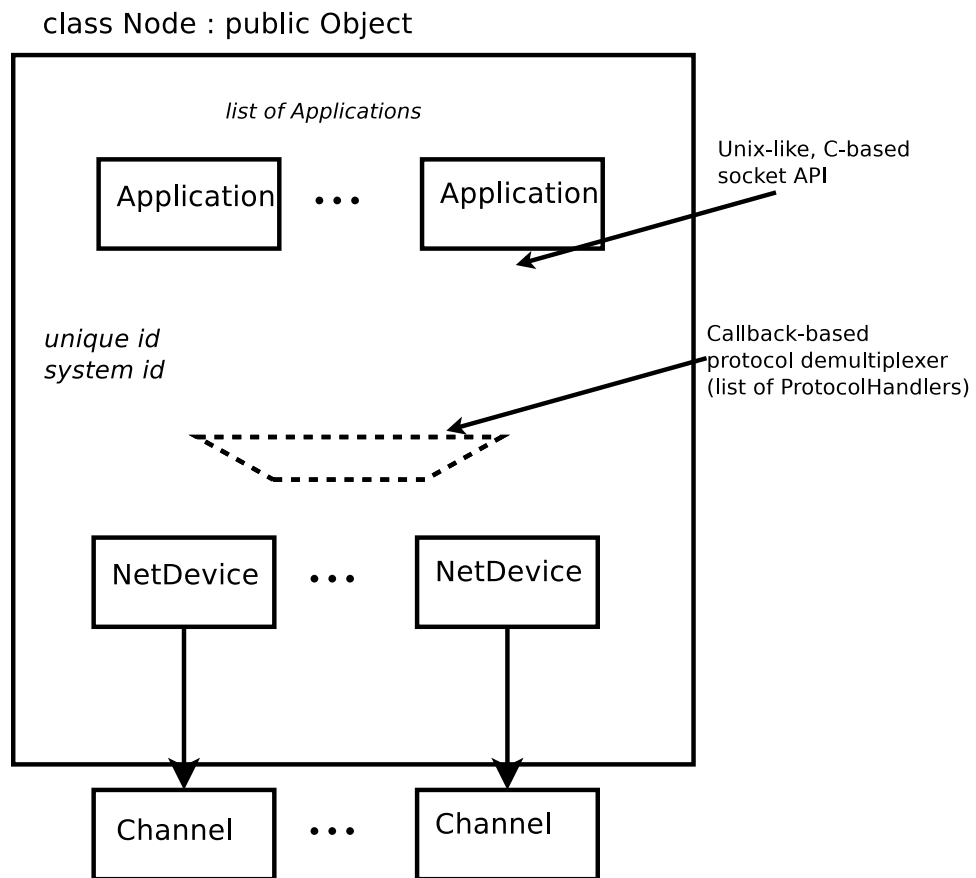


Fig. 23.3: High-level node architecture

- implementation details of the IP stack.

From a software perspective, the lower interface of applications corresponds to the C-based sockets API. The upper interface of `ns3::NetDevice` objects corresponds to the device independent sublayer of the Linux stack. Everything in between can be aggregated and plumbed together as needed.

Let's look more closely at the protocol demultiplexer. We want incoming frames at layer-2 to be delivered to the right layer-3 protocol such as IPv4. The function of this demultiplexer is to register callbacks for receiving packets. The callbacks are indexed based on the `EtherType` in the layer-2 frame.

Many different types of higher-layer protocols may be connected to the `NetDevice`, such as IPv4, IPv6, ARP, MPLS, IEEE 802.1x, and packet sockets. Therefore, the use of a callback-based demultiplexer avoids the need to use a common base class for all of these protocols, which is problematic because of the different types of objects (including packet sockets) expected to be registered there.

23.4 Sockets APIs

The `sockets API` is a long-standing API used by user-space applications to access network services in the kernel. A *socket* is an abstraction, like a Unix file handle, that allows applications to connect to other Internet hosts and exchange reliable byte streams and unreliable datagrams, among other services.

ns-3 provides two types of sockets APIs, and it is important to understand the differences between them. The first is a *native ns-3* API, while the second uses the services of the native API to provide a `POSIX-like` API as part of an overall application process. Both APIs strive to be close to the typical sockets API that application writers on Unix systems are accustomed to, but the POSIX variant is much closer to a real system's sockets API.

23.4.1 ns-3 sockets API

The native sockets API for *ns-3* provides an interface to various types of transport protocols (TCP, UDP) as well as to packet sockets and, in the future, Netlink-like sockets. However, users are cautioned to understand that the semantics are *not* the exact same as one finds in a real system (for an API which is very much aligned to real systems, see the next section).

`ns3::Socket` is defined in `src/network/model/socket.h`. Readers will note that many public member functions are aligned with real sockets function calls, and all other things being equal, we have tried to align with a Posix sockets API. However, note that:

- *ns-3* applications handle a smart pointer to a `Socket` object, not a file descriptor;
- there is no notion of synchronous API or a *blocking* API; in fact, the model for interaction between application and socket is one of asynchronous I/O, which is not typically found in real systems (more on this below);
- the C-style socket address structures are not used;
- the API is not a complete sockets API, such as supporting all socket options or all function variants;
- many calls use `ns3::Packet` class to transfer data between application and socket. This may seem peculiar to pass *Packets* across a stream socket API, but think of these packets as just fancy byte buffers at this level (more on this also below).

Basic operation and calls

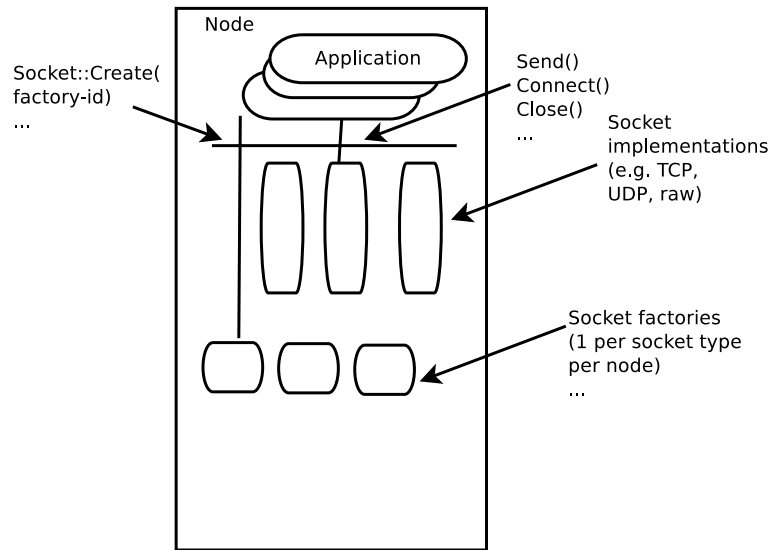


Fig. 23.4: Implementation overview of native sockets API

Creating sockets

An application that wants to use sockets must first create one. On real systems using a C-based API, this is accomplished by calling `socket()`

```
int socket(int domain, int type, int protocol);
```

which creates a socket in the system and returns an integer descriptor.

In ns-3, we have no equivalent of a system call at the lower layers, so we adopt the following model. There are certain *factory* objects that can create sockets. Each factory is capable of creating one type of socket, and if sockets of a particular type are able to be created on a given node, then a factory that can create such sockets must be aggregated to the Node:

```
static Ptr<Socket> CreateSocket(Ptr<Node> node, TypeId tid);
```

Examples of TypeIds to pass to this method are `ns3::TcpSocketFactory`, `ns3::PacketSocketFactory`, and `ns3::UdpSocketFactory`.

This method returns a smart pointer to a Socket object. Here is an example:

```
Ptr<Node> n0;
// Do some stuff to build up the Node's internet stack
Ptr<Socket> localSocket =
    Socket::CreateSocket(n0, TcpSocketFactory::GetTypeId());
```

In some ns-3 code, sockets will not be explicitly created by user's main programs, if an ns-3 application does it. For instance, for `ns3::OnOffApplication`, the function `ns3::OnOffApplication::StartApplication()` performs the socket creation, and the application holds the socket pointer.

Using sockets

Below is a typical sequence of socket calls for a TCP client in a real implementation:

```
sock = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
bind(sock, ...);
connect(sock, ...);
send(sock, ...);
recv(sock, ...);
close(sock);
```

There are analogs to all of these calls in ns-3, but we will focus on two aspects here. First, most usage of sockets in real systems requires a way to manage I/O between the application and kernel. These models include *blocking sockets*, *signal-based I/O*, and *non-blocking sockets* with polling. In ns-3, we make use of the callback mechanisms to support a fourth mode, which is analogous to POSIX *asynchronous I/O*.

In this model, on the sending side, if the `send()` call were to fail because of insufficient buffers, the application suspends the sending of more data until a function registered at the `ns3::Socket::SetSendCallback()` callback is invoked. An application can also ask the socket how much space is available by calling `ns3::Socket::GetTxAvailable()`. A typical sequence of events for sending data (ignoring connection setup) might be:

```
SetSendCallback(MakeCallback(&HandleSendCallback));
Send();
Send();
...
// Send fails because buffer is full
// Wait until HandleSendCallback is called
// HandleSendCallback is called by socket, since space now available
Send(); // Start sending again
```

Similarly, on the receive side, the socket user does not block on a call to `recv()`. Instead, the application sets a callback with `ns3::Socket::SetRecvCallback()` in which the socket will notify the application when (and how much) there is data to be read, and the application then calls `ns3::Socket::Recv()` to read the data until no more can be read.

23.4.2 Packet vs. buffer variants

There are two basic variants of `Send()` and `Recv()` supported:

```
virtual int Send(Ptr<Packet> p) = 0;
int Send(const uint8_t* buf, uint32_t size);

Ptr<Packet> Recv();
int Recv(uint8_t* buf, uint32_t size);
```

The non-Packet variants are provided for legacy API reasons. When calling the raw buffer variant of `ns3::Socket::Send()`, the buffer is immediately written into a Packet and the packet variant is invoked.

Users may find it semantically odd to pass a Packet to a stream socket such as TCP. However, do not let the name bother you; think of `ns3::Packet` to be a fancy byte buffer. There are a few reasons why the Packet variants are more likely to be preferred in ns-3:

- Users can use the Tags facility of packets to, for example, encode a flow ID or other helper data at the application layer.
- Users can exploit the copy-on-write implementation to avoid memory copies (on the receive side, the conversion back to a `uint8_t* buf` may sometimes incur an additional copy).

- Use of Packet is more aligned with the rest of the ns-3 API

23.4.3 Sending dummy data

Sometimes, users want the simulator to just pretend that there is an actual data payload in the packet (e.g. to calculate transmission delay) but do not want to actually produce or consume the data. This is straightforward to support in ns-3; have applications call `Ptr<Packet> Create<Packet>(uint32_t)` instead of `Ptr<Packet> Create<Packet>(const uint32_t*, uint32_t)`. Similarly, passing in a zero to the pointer argument in the raw buffer variants has the same effect. Note that, if some subsequent code tries to read the Packet data buffer, the fake buffer will be converted to a real (zeroed) buffer on the spot, and the efficiency will be lost there.

23.4.4 Use of Send() vs. SendTo()

There are two variants of methods used to send data to the socket:

```
virtual int Send(Ptr<Packet> p, uint32_t flags) = 0;

virtual int SendTo(Ptr<Packet> p, uint32_t flags,
                  const Address &toAddress) = 0;
```

The first method is used if the socket has already been connected (`Socket::Connect()`) to a peer address. In the case of stream-based sockets like TCP, the connect call is required to bind the socket to a peer address, and thereafter, `Send()` is typically used. In the case of datagram-based sockets like UDP, the socket is not required to be connected to a peer address before sending, and the socket may be used to send data to different destination addresses; in this case, the `SendTo()` method is used to specify the destination address for the datagram.

23.4.5 Socket options

ToS (Type of Service)

The native sockets API for ns-3 provides two public methods (of the Socket base class):

```
void SetIpTos(uint8_t ipTos);
uint8_t GetIpTos() const;
```

to set and get, respectively, the type of service associated with the socket. These methods are equivalent to using the `IP_TOS` option of BSD sockets. Clearly, setting the type of service only applies to sockets using the IPv4 protocol. However, users typically do not set the type of service associated with a socket through `ns3::Socket::SetIpTos()` because sockets are normally created by application helpers and users cannot get a pointer to the sockets.

Applications have a `Tos` attribute to simplify the ToS setup:

```
InetSocketAddress destAddress(ipv4Address, udpPort);
OnOffHelper onoff("ns3::UdpSocketFactory", destAddress);
onoff.SetAttribute("Tos", IntegerValue(tos));
```

For this to work, the application must eventually call the `ns3::Socket::Connect()` method to connect to the provided `destAddress` and the `Connect` method of the particular socket type must support setting the type of service associated with a socket (by using the `ns3::Socket::SetIpTos()` method). Currently, the socket types that support setting the type of service in such a way are `ns3::UdpSocketImpl` and `ns3::TcpSocketBase`.

The type of service associated with a socket is then used to determine the value of the Type of Service field (renamed as Differentiated Services field by RFC 2474) of the IPv4 header of the packets sent through that socket, as detailed in the next sections.

Setting the ToS with UDP sockets

For IPv4 packets, the ToS field is set according to the following rules:

- If the socket is connected, the ToS field is set to the ToS value associated with the socket.
- If the socket is not connected, the ToS field is set to the value specified in the destination address (of type `ns3::InetSocketAddress`) passed to `ns3::Socket::SendTo()`, and the ToS value associated with the socket is ignored.

Setting the ToS with TCP sockets

For IPv4 packets, the ToS field is set to the ToS value associated with the socket.

Priority

The native sockets API for ns-3 provides two public methods (of the `Socket` base class):

```
void SetPriority(uint8_t priority);
uint8_t GetPriority() const;
```

to set and get, respectively, the priority associated with the socket. These methods are equivalent to using the `SO_PRIORITY` option of BSD sockets. Only values in the range 0..6 can be set through the above method.

Note that setting the type of service associated with a socket (by calling `ns3::Socket::SetIpTos()`) also sets the priority for the socket to the value that the `ns3::Socket::IpTos2Priority()` function returns when it is passed the type of service value. This function is implemented after the Linux `rt_tos2priority` function, which takes an 8-bit value as input and returns a value which is a function of bits 3-6 (where bit 0 is the most significant bit) of the input value:

Bits 3-6	Priority
0 to 3	0 (Best Effort)
4 to 7	2 (Bulk)
8 to 11	6 (Interactive)
12 to 15	4 (Interactive Bulk)

The rationale is that bits 3-6 of the Type of Service field were interpreted as the TOS subfield by (the obsolete) RFC 1349. Readers can refer to the doxygen documentation of `ns3::Socket::IpTos2Priority()` for more information, including how DSCP values map onto priority values.

The priority set for a socket (as described above) is then used to determine the priority of the packets sent through that socket, as detailed in the next sections. Currently, the socket types that support setting the packet priority are `ns3::UdpSocketImpl`, `ns3::TcpSocketBase` and `ns3::PacketSocket`. The packet priority is used, e.g., by queuing disciplines such as the default `PfifoFastQueueDisc` to classify packets into distinct queues.

Setting the priority with UDP sockets

If the packet is an IPv4 packet and the value to be inserted in the ToS field is not null, then the packet is assigned a priority based on such ToS value (according to the `ns3::Socket::IpTos2Priority()` function). Otherwise, the priority associated with the socket is assigned to the packet.

Setting the priority with TCP sockets

Every packet is assigned a priority equal to the priority associated with the socket.

Setting the priority with packet sockets

Every packet is assigned a priority equal to the priority associated with the socket.

23.4.6 Socket errno

to be completed

23.4.7 Example programs

to be completed

23.4.8 POSIX-like sockets API

23.5 Simple NetDevice

Placeholder chapter

23.6 Queues

This section documents the queue object, which is typically used by NetDevices and QueueDiscs to store packets.

Packets stored in a queue can be managed according to different policies. Currently, only the DropTail policy is available.

23.6.1 Model Description

The source code for the new module lives in the directory `src/network/utils`.

The Queue class has been redesigned as a template class object to allow us to instantiate queues storing different types of items. The unique template type parameter specifies the type of items stored in the queue. The only requirement on the item type is that it must provide a `GetSize()` method which returns the size of the packet included in the item. Currently, queue items can be objects of the following classes:

- Packet
- QueueItem and subclasses (e.g., QueueDiscItem)

- `WifiMacQueueItem`

The internal queues of the queue discs are of type `Queue<QueueDiscItem>` (an alias of which being `InternalQueue`). A number of network devices (`SimpleNetDevice`, `PointToPointNetDevice`, `CsmaNetDevice`) use a `Queue<Packet>` to store packets to be transmitted. `WifiNetDevices` use instead queues of type `WifiMacQueue`, which is a subclass of `Queue` storing objects of type `WifiMacQueueItem`. Other devices, such as `WiMax` and `LTE`, use specialized queues.

Design

The `Queue` class derives from the `QueueBase` class, which is a non-template class providing all the methods that are independent of the type of the items stored in the queue. The `Queue` class provides instead all the operations that depend on the item type, such as `enqueue`, `dequeue`, `peek` and `remove`. The `Queue` class also provides the ability to trace certain queue operations such as enqueueing, dequeuing, and dropping.

`Queue` is an abstract base class and is subclassed for specific scheduling and drop policies. Subclasses need to define the following public methods:

- `bool Enqueue (Ptr<Item> item):` Enqueue a packet
- `Ptr<Item> Dequeue ():` Dequeue a packet
- `Ptr<Item> Remove ():` Remove a packet
- `Ptr<const Item> Peek ():` Peek a packet

The `Enqueue` method does not allow to store a packet if the queue capacity is exceeded. Subclasses may also define specialized public methods. For instance, the `WifiMacQueue` class provides a method to dequeue a packet based on its tid and MAC address.

There are five trace sources that may be hooked:

- `Enqueue`
- `Dequeue`
- `Drop`
- `DropBeforeEnqueue`
- `DropAfterDequeue`

Also, the `QueueBase` class defines two additional trace sources:

- `PacketsInQueue`
- `BytesInQueue`

DropTail

This is a basic first-in-first-out (FIFO) queue that performs a tail drop when the queue is full.

The `DropTailQueue` class defines one attribute:

- `MaxSize:` the maximum queue size

23.6.2 Usage

Helpers

A typical usage pattern is to create a device helper and to configure the queue type and attributes from the helper, such as this example:

```
PointToPointHelper p2p;

p2p.SetQueue("ns3::DropTailQueue");
p2p.SetDeviceAttribute("DataRate", StringValue("10Mbps"));
p2p.SetChannelAttribute("Delay", StringValue("2ms"));
NetDeviceContainer devn0n2 = p2p.Install(n0n2);

p2p.SetQueue("ns3::DropTailQueue");
p2p.SetDeviceAttribute("DataRate", StringValue("10Mbps"));
p2p.SetChannelAttribute("Delay", StringValue("3ms"));
NetDeviceContainer devn1n2 = p2p.Install(n1n2);

p2p.SetQueue("ns3::DropTailQueue",
             "MaxSize", StringValue("50p"));
p2p.SetDeviceAttribute("DataRate", StringValue(linkDataRate));
p2p.SetChannelAttribute("Delay", StringValue(linkDelay));
NetDeviceContainer devn2n3 = p2p.Install(n2n3);
```

Please note that the SetQueue method of the PointToPointHelper class allows to specify “ns3::DropTailQueue” instead of “ns3::DropTailQueue<Packet>”. The same holds for CsmaHelper, SimpleNetDeviceHelper and TrafficControlHelper.

Output

The ns-3 ascii trace helpers used by many of the NetDevices will hook the Enqueue, Dequeue, and Drop traces of these queues and print out trace statements, such as the following from examples/udp/udp-echo.cc:

```
+ 2 /NodeList/0/DeviceList/1/$ns3::CsmaNetDevice/TxQueue/Enqueue ns3::EthernetHeader
( length/type=0x806, source=00:00:00:00:00:01, destination=ff:ff:ff:ff:ff:ff)
ns3::ArpHeader (request source mac: 00-06-00:00:00:00:00:01 source ipv4: 10.1.1.1
dest ipv4: 10.1.1.2) Payload (size=18) ns3::EthernetTrailer (fcs=0)
- 2 /NodeList/0/DeviceList/1/$ns3::CsmaNetDevice/TxQueue/Dequeue ns3::EthernetHeader
( length/type=0x806, source=00:00:00:00:00:01, destination=ff:ff:ff:ff:ff:ff)
ns3::ArpHeader (request source mac: 00-06-00:00:00:00:00:01 source ipv4: 10.1.1.1
dest ipv4: 10.1.1.2) Payload (size=18) ns3::EthernetTrailer (fcs=0)
```

which shows an enqueue “+” and dequeue “-” event at time 2 seconds.

Users are, of course, free to define and hook their own trace sinks to these trace sources.

Examples

The drop-tail queue is used in several examples, such as `examples/udp/udp-echo.cc`.

23.7 Queue limits

This section documents the queue limits model, which is used by the traffic control to limit the NetDevices queueing delay. It operates on the transmission path of the network node.

The reduction of the NetDevices queueing delay is essential to improve the effectiveness of Active Queue Management (AQM) algorithms. Careful assessment of the queueing delay includes a byte-based measure of the NetDevices queue length. In this design, traffic control can use different byte-based schemes to limit the queueing delay. Currently the only available scheme is DynamicQueueLimits, which is modelled after the dynamic queue limit library of Linux.

23.7.1 Model Description

The source code for the model lives in the directory `src/network/utils`.

The model allows a byte-based measure of the netdevice queue. The byte-based measure more accurately approximates the time required to empty the queue than a packet-based measure.

To inform the upper layers about the transmission of packets, NetDevices can call a couple of functions:

- `void NotifyQueuedBytes (uint32_t bytes)`: Report the number of bytes queued to the device queue
- `void NotifyTransmittedBytes (uint32_t bytes)`: Report the number of bytes transmitted by device

Based on this information, the QueueLimits object can stop the transmission queue.

In case of multiqueue NetDevices this mechanism is available for each queue.

The QueueLimits model can be used on any NetDevice modelled in ns-3.

Design

An abstract base class, class QueueLimits, is subclassed for specific byte-based limiting strategies.

Common operations provided by the base class QueueLimits include:

- `void Reset ()`: Reset queue limits state
- `void Completed (uint32_t count)`: Record the number of completed bytes and recalculate the limit
- `int32_t Available () const`: Return how many bytes can be queued
- `void Queued (uint32_t count)`: Record number of bytes queued

DynamicQueueLimits

Dynamic queue limits (DQL) is a basic library implemented in the Linux kernel to limit the Ethernet queueing delay. DQL is a general purpose queue length controller. The goal of DQL is to calculate the limit as the minimum number of bytes needed to prevent starvation.

Three attributes are defined in the DynamicQueueLimits class:

- **HoldTime**: The DQL algorithm hold time
- **MaxLimit**: Maximum limit
- **MinLimit**: Minimum limit

The DQL algorithm hold time is 1 s. Reducing the HoldTime increases the responsiveness of DQL with consequent greater number of limit variation events. Conversely, increasing the HoldTime decreases the responsiveness of DQL with a minor number of limit variation events. The limit calculated by DQL is in the range from MinLimit to MaxLimit. The default values are respectively 0 and DQL_MAX_LIMIT. Increasing the MinLimit is recommended in case of higher NetDevice transmission rate (e.g. 1 Gbps) while reducing the MaxLimit is recommended in case of lower NetDevice transmission rate (e.g. 500 Kbps).

There is one trace source in DynamicQueueLimits class that may be hooked:

- **Limit**: Limit value calculated by DQL

23.7.2 Usage

Helpers

A typical usage pattern is to create a traffic control helper and configure the queue limits type and attributes from the helper, such as this example:

```
TrafficControlHelper tch;  
uint32_t handle = tch.SetRootQueueDisc("ns3::PfifoFastQueueDisc", "Limit",  
↳ UintegerValue(1000));  
  
tch.SetQueueLimits("ns3::DynamicQueueLimits", "HoldTime", StringValue("4ms"));
```

then install the configuration on a NetDevices container

```
tch.Install (devices);
```

NIX-VECTOR ROUTING DOCUMENTATION

Nix-vector routing is a simulation specific routing protocol and is intended for large network topologies. The on-demand nature of this protocol as well as the low-memory footprint of the nix-vector provides improved performance in terms of memory usage and simulation run time when dealing with a large number of nodes.

24.1 Model Description

The source code for the NixVectorRouting module lives in the directory `src/nix-vector-routing`.

ns-3 nix-vector-routing performs on-demand route computation using a breadth-first search and an efficient route-storage data structure known as a nix-vector.

When a packet is generated at a node for transmission, the route is calculated, and the nix-vector is built.

How is the Nix-Vector calculated? The nix-vector stores an index for each hop along the path, which corresponds to the neighbor-index. This index is used to determine which net-device and gateway should be used.

How does the routing take place? To route a packet, the nix-vector must be transmitted with the packet. At each hop, the current node extracts the appropriate neighbor-index from the nix-vector and transmits the packet through the corresponding net-device. This continues until the packet reaches the destination.

Note: Nix-Vector routing does not use any routing metrics (interface metrics) during the calculation of nix-vector. It is only based on the shortest path calculated according to BFS.

How does Nix decide between two equally short path from source to destination? It depends on how the topology is constructed i.e., the order in which the net-devices are added on a node and net-devices added on the channels associated with current node's net-devices. Please check the `nix-simple.cc` example below to understand how nix-vectors are calculated.

How does Nix reacts to topology changes? Routes in Nix are specific to a given network topology, and are cached by the sender node. Nix monitors the following events: Interface up/down, Route add/removal, Address add/removal to understand if the cached routes are valid or if they have to be purged.

If the topology changes while the packet is “in flight”, the associated NixVector is invalid, and have to be rebuilt by an intermediate node. This is possible because the NixVecor carries an “Epoch”, i.e., a counter indicating when the NixVector has been created. If the topology changes, the Epoch is globally updated, and any outdated NixVector is rebuilt.

ns-3 supports IPv4 as well as IPv6 Nix-Vector routing.

24.1.1 Scope and Limitations

Currently, the *ns-3* model of nix-vector routing supports IPv4 and IPv6 p2p links, CSMA links and multiple WiFi networks with the same channel object. It does not (yet) provide support for efficient adaptation to link failures. It simply flushes all nix-vector routing caches.

NixVectorRouting performs a subnet matching check, but it does **not** check entirely if the addresses have been appropriately assigned. In other terms, using Nix-Vector routing, it is possible to have a working network that violates some good practices in IP address assignments.

In case of IPv6, Nix assumes the link-local addresses assigned are **unique**. When using the IPv6 stack, the link-local address allocation is unique by default over the entire topology. However, if the link-local addresses are assigned manually, the user must ensure uniqueness of link-local addresses.

NixVectorRouting supports routes to IPv4 and IPv6 loopback addresses on localhost. Although it is not really intended to route to these addresses, it can do so.

24.2 Usage

The usage pattern is the one of all the Internet routing protocols. Since NixVectorRouting is not installed by default in the Internet stack, it is necessary to set it in the Internet Stack helper by using `InternetStackHelper::SetRoutingHelper`.

Remember to include the header file `ns3/nix-vector-routing-module.h` to use IPv4 or IPv6 Nix-Vector routing.

Note: The previous header files `ns3/ipv4-nix-vector-helper.h` and `ns3/ipv4-nix-vector-routing.h` are deprecated and will be removed in the future. These files are replaced with more generic (having IPv6 capabilities) `ns3/nix-vector-helper.h` and `ns3/nix-vector-routing.h` respectively.

- Using IPv4 Nix-Vector Routing:

```
Ipv4NixVectorHelper nixRouting;  
InternetStackHelper stack;  
stack.SetRoutingHelper(nixRouting); // has effect on the next Install()  
stack.Install(allNodes);           // allNodes is the NodeContainer
```

- Using IPv6 Nix-Vector Routing:

```
Ipv6NixVectorHelper nixRouting;  
InternetStackHelper stack;  
stack.SetRoutingHelper(nixRouting); // has effect on the next Install()  
stack.Install(allNodes);           // allNodes is the NodeContainer
```

Note: The NixVectorHelper helper class helps to use NixVectorRouting functionality. The NixVectorRouting model class can also be used directly to use Nix-Vector routing. `ns3/nix-vector-routing-module.h` contains the header files for both the classes.

24.2.1 Examples

The examples for the NixVectorRouting module lives in the directory `src/nix-vector-routing/examples`.

There are examples which use both IPv4 and IPv6 networking.

1. nix-simple.cc

```
/*
 *
 *      /-----\
 *    /           \
 *  * n0 -- n1 -- n2 -- n3
 *
 *
 *  * n0 IP: 10.1.1.1, 10.1.4.1
 *  * n1 IP: 10.1.1.2, 10.1.2.1
 *  * n2 IP: 10.1.2.2, 10.1.3.1, 10.1.4.2
 *  * n3 IP: 10.1.3.2
 */
```

In this topology, we install Nix-Vector routing between source `n0` and destination `n3`. The shortest possible route will be `n0 -> n2 -> n3`.

Let's see how the nix-vector will be generated for this path:

`n0` has 2 neighbors i.e. `n1` and `n3`. `n0` is connected to both using separate net-devices. But the net-device for `n0 - n1` p2p link was created before the netdevice for `n0 - n2` p2p link. Thus, `n2` has neighbor-index of 1 (`n1` has 0) with respect to `n0`.

`n2` has 3 neighbors i.e. `n1`, `n3` and `n0`. The `n2` net-device for `n1 - n2` p2p link was created before the `n2` net-device for `n2 - n3` p2p link which was before the `n2` netdevice for `n0 - n2` p2p link. This, `n3` has neighbor-index of 01 (`n1` has 00 and `n0` has 10) with respect to `n2`.

Thus, the nix-vector for the path from `n0` to `n3` is 101.

Note: This neighbor-index or nix-index has total number of bits equal to minimum number of bits required to represent all the neighbors in their binary form.

Note: If there are multiple netdevices connected to the current netdevice on the channel then it depends on which order netdevices were added to the channel.

a. Using IPv4:

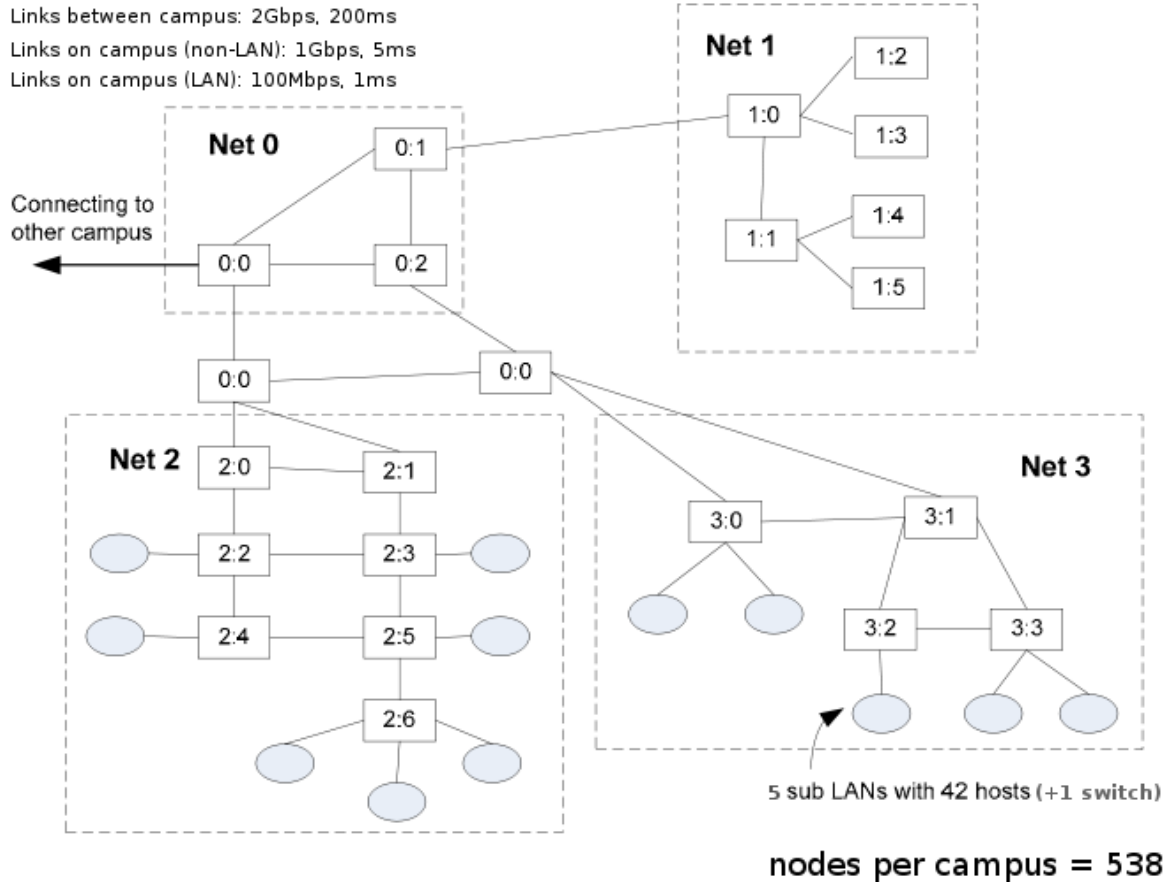
```
# By default IPv4 network is selected
./ns3 run nix-simple
```

b. Using IPv6:

```
# Use the --useIPv6 flag
./ns3 run "nix-simple --useIPv6"
```

2. nms-p2p-nix.cc

This example demonstrates the advantage of Nix-Vector routing as Nix performs source-based routing (BFS) to have faster routing.



a. Using IPv4:

```
# By default IPv4 network is selected
./ns3 run nms-p2p-nix
```

b. Using IPv6:

```
# Use the --useIPv6 flag
./ns3 run "nms-p2p-nix --useIPv6"
```

3. nix-simple-multi-address.cc

This is an IPv4 example demonstrating multiple interface addresses. This example also shows how address assignment in between the simulation causes the all the route caches and Nix caches to flush.

```
# By default IPv4 network is selected
./ns3 run nix-simple-multi-address
```

4. nix-double-wifi.cc

This example demonstrates the working of Nix with two Wifi networks operating on the same Wifi channel object. The example uses `ns3::YansWifiChannel` for both the wifi networks.

a. Using IPv4:

```
# By default IPv4 network is selected
./ns3 run nix-double-wifi
```

(continues on next page)

(continued from previous page)

```
# Use the --enableNixLog to enable NixVectorRouting logging.  
./ns3 run "nix-double-wifi --enableNixLog"
```

b. Using IPv6:

```
# Use the --useIPv6 flag  
./ns3 run "nix-double-wifi --useIPv6"  
# Use the --enableNixLog to enable NixVectorRouting logging.  
./ns3 run "nix-double-wifi --useIPv6 --enableNixLog"
```


OPTIMIZED LINK STATE ROUTING (OLSR)

This model implements the base specification of the Optimized Link State Routing (OLSR) protocol, which is a dynamic mobile ad hoc unicast routing protocol. It has been developed at the University of Murcia (Spain) by Francisco J. Ros for NS-2, and was ported to NS-3 by Gustavo Carneiro at INESC Porto (Portugal).

The implementation is based on OLSR Version 1 ([RFC 3626](#) [[rfc3626](#)]) and it is *not* compliant with OLSR Version 2 ([RFC 7181](#) [[rfc7181](#)]) or any of the Version 2 extensions.

25.1 Model Description

The source code for the OLSR model lives in the directory *src/olsr*. As stated before, the model is based on [RFC 3626](#) ([\[rfc3626\]](#)). Moreover, many design choices are based on the previous ns2 model.

25.1.1 Scope and Limitations

The model is for IPv4 only.

- Mostly compliant with OLSR as documented in [RFC 3626](#) ([\[rfc3626\]](#)),
- The use of multiple interfaces was not supported by the NS-2 version, but is supported in NS-3;
- OLSR does not respond to the routing event notifications corresponding to dynamic interface up and down (`ns3::RoutingProtocol::NotifyInterfaceUp` and `ns3::RoutingProtocol::NotifyInterfaceDown`) or address insertion/removal (`ns3::RoutingProtocol::NotifyAddAddress` and `ns3::RoutingProtocol::NotifyRemoveAddress`).
- Unlike the NS-2 version, does not yet support MAC layer feedback as described in [RFC 3626](#) ([\[rfc3626\]](#));

Host Network Association (HNA) is supported in this implementation of OLSR. Refer to `examples/olsr-hna.cc` to see how the API is used.

25.1.2 References

25.2 Usage

The usage pattern is the one of all the Internet routing protocols. Since OLSR is not installed by default in the Internet stack, it is necessary to set it in the Internet Stack helper by using `InternetStackHelper::SetRoutingHelper`

Typically, OLSR is enabled in a main program by use of an `OlsrHelper` class that installs OLSR into an `Ipv4ListRoutingProtocol` object. The following sample commands will enable OLSR in a simulation using this helper

class along with some other routing helper objects. The setting of priority value 10, ahead of the staticRouting priority of 0, means that OLSR will be consulted for a route before the node's static routing table.:

```
NodeContainer c:
...
// Enable OLSR
NS_LOG_INFO("Enabling OLSR Routing.");
OlsrHelper olsr;

Ipv4StaticRoutingHelper staticRouting;

Ipv4ListRoutingHelper list;
list.Add(staticRouting, 0);
list.Add(olsr, 10);

InternetStackHelper internet;
internet.SetRoutingHelper(list);
internet.Install(c);
```

Once installed, the OLSR “main interface” can be set with the `SetMainInterface()` command. If the user does not specify a main address, the protocol will select the first primary IP address that it finds, starting first the loopback interface and then the next non-loopback interface found, in order of Ipv4 interface index. The loopback address of 127.0.0.1 is not selected. In addition, a number of protocol constants are defined in `olsr-routing-protocol.cc`.

Olsr is automatically started at time zero of the simulation, based on the ns-3 Node initialization sequence that eventually leads to `olsr::RoutingProtocol::DoInitialize()`. Note: a patch to allow the user to start and stop the protocol at other times would be welcome.

25.2.1 Examples

The examples are in the `src/olsr/examples/` directory. However, many other examples exist in the general examples directory, e.g., `examples/routing/manet-routing-compare.cc`.

For a specific example of the HNA feature, see the program `src/olsr/examples/olsr-hna.cc`.

25.2.2 Helpers

A helper class for OLSR has been written. After an IPv4 topology has been created and unique IP addresses assigned to each node, the simulation script writer can call one of three overloaded functions with different scope to enable OLSR: `ns3::OlsrHelper::Install (NodeContainer container);` `ns3::OlsrHelper::Install (Ptr<Node> node);` or `ns3::OlsrHelper::InstallAll ()`

25.2.3 Attributes

In addition, the behavior of OLSR can be modified by changing certain attributes. The method `ns3::OlsrHelper::Set ()` can be used to set OLSR attributes. These include `HelloInterval`, `TcInterval`, `MidInterval`, `Willingness`. Other parameters are defined as macros in `olsr-routing-protocol.cc`.

The list of configurable attributes is:

- `HelloInterval` (time, default 2s), HELLO messages emission interval.
- `TcInterval` (time, default 5s), TC messages emission interval.
- `MidInterval` (time, default 5s), MID messages emission interval.

- HnaInterval (time, default 5s), HNA messages emission interval.
- Willingness (enum, default olsr::Willingness::DEFAULT), Willingness of a node to carry and forward traffic for other nodes.

25.2.4 Tracing

The available traces are:

- Rx: Receive OLSR packet.
- Tx: Send OLSR packet.
- RoutingTableChanged: The OLSR routing table has changed.

25.2.5 Caveats

Presently, OLSR is limited to use with an Ipv4ListRouting object, and does not respond to dynamic changes to a device's IP address or link up/down notifications; i.e. the topology changes are due to loss/gain of connectivity over a wireless channel.

The code does not present any known issue.

25.3 Validation

The code validation has been done through Wireshark message compliance and unit testings.

OPENFLOW SWITCH SUPPORT

ns-3 simulations can use OpenFlow switches (McKeown et al.¹), widely used in research. OpenFlow switches are configurable via the OpenFlow API, and also have an MPLS extension for quality-of-service and service-level-agreement support. By extending these capabilities to ns-3 for a simulated OpenFlow switch that is both configurable and can use the MPLS extension, ns-3 simulations can accurately simulate many different switches.

The OpenFlow software implementation distribution is hereby referred to as the OFSID. This is a demonstration of running OpenFlow in software that the OpenFlow research group has made available. There is also an OFSID that Ericsson researchers created to add MPLS capabilities; this is the OFSID currently used with ns-3. The design will allow the users to, with minimal effort, switch in a different OFSID that may include more efficient code than a previous OFSID.

26.1 Model Description

The model relies on building an external OpenFlow switch library (OFSID), and then building some ns-3 wrappers that call out to the library. The source code for the ns-3 wrappers lives in the directory `src/openflow/model`.

26.1.1 Design

The OpenFlow module presents a `OpenFlowSwitchNetDevice` and a `OpenFlowSwitchHelper` for installing it on nodes. Like the `Bridge` module, it takes a collection of `NetDevices` to set up as ports, and it acts as the intermediary between them, receiving a packet on one port and forwarding it on another, or all but the received port when flooding. Like an OpenFlow switch, it maintains a configurable flow table that can match packets by their headers and do different actions with the packet based on how it matches. The module's understanding of OpenFlow configuration messages are kept the same format as a real OpenFlow-compatible switch, so users testing Controllers via ns-3 won't have to rewrite their Controller to work on real OpenFlow-compatible switches.

The ns-3 OpenFlow switch device models an OpenFlow-enabled switch. It is designed to express basic use of the OpenFlow protocol, with the maintaining of a virtual Flow Table and TCAM to provide OpenFlow-like results.

The functionality comes down to the Controllers, which send messages to the switch that configure its flows, producing different effects. Controllers can be added by the user, under the `ofi` namespace extending `ofi::Controller`. To demonstrate this, a `DropController`, which creates flows for ignoring every single packet, and `LearningController`, which effectively makes the switch a more complicated `BridgeNetDevice`. A user versed in a standard OFSID, and/or OF protocol, can write virtual controllers to create switches of all kinds of types.

¹ McKeown, N.; Anderson, T.; Balakrishnan, H.; Parulkar, G.; Peterson, L.; Rexford, J.; Shenker, S.; Turner, J.; OpenFlow: enabling innovation in campus networks, ACM SIGCOMM Computer Communication Review, Vol. 38, Issue 2, April 2008.

OpenFlow switch Model

The OpenFlow switch device behaves somewhat according to the diagram setup as a classical OFSID switch, with a few modifications made for a proper simulation environment.

Normal OF-enabled Switch:

Secure Channel	<--OF Protocol-->	Controller is external
Hardware or Software Flow Table		

ns-3 OF-enabled Switch (module):

m_controller->ReceiveFromSwitch()	<--OF Protocol-->	Controller is internal
Software Flow Table, virtual TCAM		

In essence, there are two differences:

1) No SSL, Embedded Controller: Instead of a secure channel and connecting to an outside location for the Controller program/machine, we currently only allow a Controller extended from `ofi::Controller`, an extension of an `ns3::Object`. This means ns-3 programmers cannot model the SSL part of the interface or possibility of network failure. The connection to the OpenFlowSwitch is local and there aren't any reasons for the channel/connection to break down. <<This difference may be an option in the future. Using EmuNetDevices, it should be possible to engage an external Controller program/machine, and thus work with controllers designed outside of the ns-3 environment, that simply use the proper OF protocol when communicating messages to the switch through a tap device.>>

2) Virtual Flow Table, TCAM: Typical OF-enabled switches are implemented on a hardware TCAM. The OFSID we turn into a library includes a modelled software TCAM, that produces the same results as a hardware TCAM. We include an attribute `FlowTableLookupDelay`, which allows a simple delay of using the TCAM to be modelled. We don't endeavor to make this delay more complicated, based on the tasks we are running on the TCAM, that is a possible future improvement.

The OpenFlowSwitch network device is aimed to model an OpenFlow switch, with a TCAM and a connection to a controller program. With some tweaking, it can model every switch type, per OpenFlow's extensibility. It outsources the complexity of the switch ports to NetDevices of the user's choosing. It should be noted that these NetDevices must behave like practical switch ports, i.e. a Mac Address is assigned, and nothing more. It also must support a `SendFrom` function so that the OpenFlowSwitch can forward across that port.

26.1.2 Scope and Limitations

All MPLS capabilities are implemented on the OFSID side in the OpenFlowSwitchNetDevice, but ns-3-mpls hasn't been integrated, so ns-3 has no way to pass in proper MPLS packets to the OpenFlowSwitch. If it did, one would only need to make `BufferFromPacket` pick up the `MplsLabelStack` or whatever the MPLS header is called on the Packet, and build the MPLS header into the `ofpbuf`.

26.1.3 Future Work

26.1.4 References

26.2 Usage

The OFSID requires `libxml2` (for MPLS FIB xml file parsing), and `libdl` (for address fault checking).

26.2.1 Building OFSID

In order to use the OpenFlowSwitch module, you must create and link the OFSID (OpenFlow Software Implementation Distribution) to ns-3. To do this:

1. Obtain the OFSID code. An ns-3 specific OFSID branch is provided to ensure operation with ns-3. Use Git to download this branch and CMake to build the library:

```
$ git clone https://gitlab.com/nsnam/openflow
$ cd openflow
```

From the “openflow” directory, run:

```
$ cmake -B build -D CMAKE_INSTALL_PREFIX=./out .
$ cmake --build build
$ cmake --install build
```

2. Your OFSID is now built into a libopenflow.a library! To link to an ns-3 build with this OpenFlow switch module, run from the ns-3-dev (or whatever you have named your distribution):

```
$ ./ns3 configure --enable-examples --enable-tests --with-openflow=path/to/openflow/
↳ out
```

3. Under ---- Summary of optional NS-3 features: you should see:

```
"NS-3 OpenFlow Integration      : enabled"
```

indicating the library has been linked to ns-3. Run:

```
$ ./ns3 build
```

to build ns-3 and activate the OpenFlowSwitch module in ns-3.

26.2.2 Examples

For an example demonstrating its use in a simple learning controller/switch, run:

```
$ ./ns3 run openflow-switch
```

To see it in detailed logging, run:

```
$ ./ns3 run "openflow-switch -v"
```

26.2.3 Helpers

26.2.4 Attributes

The SwitchNetDevice provides following Attributes:

- FlowTableLookUpDelay: This time gets run off the clock when making a lookup in our Flow Table.
- **Flags: OpenFlow specific configuration flags. They are defined in the ofp_config_flags enum. Choices include:**
 - OFPC_SEND_FLOW_EXP (Switch notifies controller when a flow has expired),
 - OFPC_FRAG_NORMAL (Match fragment against Flow table), OFPC_FRAG_DROP (Drop fragments),

OFPC_FRAG_REASM (Reassemble only if OFPC_IP_REASM set, which is currently impossible, because switch implementation does not support IP reassembly) OFPC_FRAG_MASK (Mask Fragments)

- **FlowTableMissSendLength:** When the packet doesn't match in our Flow Table, and we forward to the controller, this sets # of bytes forwarded (packet is not forwarded in its entirety, unless specified).

Note: TODO

26.2.5 Tracing

Note: TODO

26.2.6 Logging

Note: TODO

26.2.7 Caveats

Note: TODO

26.3 Validation

This model has one test suite which can be run as follows:

```
$ ./test.py --suite=openflow
```


POINTTOPOINT NETDEVICE

This is the introduction to PointToPoint NetDevice chapter, to complement the PointToPoint model doxygen.

27.1 Overview of the PointToPoint model

The *ns-3* point-to-point model is of a very simple point to point data link connecting exactly two PointToPointNetDevice devices over an PointToPointChannel. This can be viewed as equivalent to a full duplex RS-232 or RS-422 link with null modem and no handshaking.

Data is encapsulated in the Point-to-Point Protocol (PPP – RFC 1661), however the Link Control Protocol (LCP) and associated state machine is not implemented. The PPP link is assumed to be established and authenticated at all times.

Data is not framed, therefore Address and Control fields will not be found. Since the data is not framed, there is no need to provide Flag Sequence and Control Escape octets, nor is a Frame Check Sequence appended. All that is required to implement non-framed PPP is to prepend the PPP protocol number for IP Version 4 which is the sixteen-bit number 0x21 (see <http://www.iana.org/assignments/ppp-numbers>).

The PointToPointNetDevice provides following Attributes:

- Address: The ns3::Mac48Address of the device (if desired);
- DataRate: The data rate (ns3::DataRate) of the device;
- TxQueue: The transmit queue (ns3::Queue) used by the device;
- InterframeGap: The optional ns3::Time to wait between “frames”;
- Rx: A trace source for received packets;
- Drop: A trace source for dropped packets.

The PointToPointNetDevice models a transmitter section that puts bits on a corresponding channel “wire.” The DataRate attribute specifies the number of bits per second that the device will simulate sending over the channel. In reality no bits are sent, but an event is scheduled for an elapsed time consistent with the number of bits in each packet and the specified DataRate. The implication here is that the receiving device models a receiver section that can receive any any data rate. Therefore there is no need, nor way to set a receive data rate in this model. By setting the DataRate on the transmitter of both devices connected to a given PointToPointChannel one can model a symmetric channel; or by setting different DataRates one can model an asymmetric channel (e.g., ADSL).

The PointToPointNetDevice supports the assignment of a “receive error model.” This is an ErrorModel object that is used to simulate data corruption on the link.

27.2 Point-to-Point Channel Model

The point to point net devices are connected via an `PointToPointChannel`. This channel models two wires transmitting bits at the data rate specified by the source net device. There is no overhead beyond the eight bits per byte of the packet sent. That is, we do not model Flag Sequences, Frame Check Sequences nor do we “escape” any data.

The `PointToPointChannel` provides following Attributes:

- Delay: An `ns3::Time` specifying the propagation delay for the channel.

27.3 Using the `PointToPointNetDevice`

The `PointToPoint` net devices and channels are typically created and configured using the associated `PointToPointHelper` object. The various ns3 device helpers generally work in a similar way, and their use is seen in many of our example programs and is also covered in the *ns-3* tutorial.

The conceptual model of interest is that of a bare computer “husk” into which you plug net devices. The bare computers are created using a `NodeContainer` helper. You just ask this helper to create as many computers (we call them `Nodes`) as you need on your network:

```
NodeContainer nodes;  
nodes.Create(2);
```

Once you have your nodes, you need to instantiate a `PointToPointHelper` and set any attributes you may want to change. Note that since this is a point-to-point(as compared to a point-to-multipoint) there may only be two nodes with associated net devices connected by a `PointToPointChannel`.

```
PointToPointHelper pointToPoint;  
pointToPoint.SetDeviceAttribute("DataRate", StringValue("5Mbps"));  
pointToPoint.SetChannelAttribute("Delay", StringValue("2ms"));
```

Once the attributes are set, all that remains is to create the devices and install them on the required nodes, and to connect the devices together using a `PointToPoint` channel. When we create the net devices, we add them to a container to allow you to use them in the future. This all takes just one line of code.:

```
NetDeviceContainer devices = pointToPoint.Install(nodes);
```

27.4 `PointToPoint` Tracing

Like all *ns-3* devices, the Point-to-Point Model provides a number of trace sources. These trace sources can be hooked using your own custom trace code, or you can use our helper functions to arrange for tracing to be enabled on devices you specify.

27.4.1 Upper-Level (MAC) Hooks

From the point of view of tracing in the net device, there are several interesting points to insert trace hooks. A convention inherited from other simulators is that packets destined for transmission onto attached networks pass through a single “transmit queue” in the net device. We provide trace hooks at this point in packet flow, which corresponds (abstractly) only to a transition from the network to data link layer, and call them collectively the device MAC hooks.

When a packet is sent to the Point-to-Point net device for transmission it always passes through the transmit queue. The transmit queue in the `PointToPointNetDevice` inherits from `Queue`, and therefore inherits three trace sources:

- An Enqueue operation source (see `ns3::Queue::m_traceEnqueue`);
- A Dequeue operation source (see `ns3::Queue::m_traceDequeue`);
- A Drop operation source (see `ns3::Queue::m_traceDrop`).

These three trace sources on the single transmit queue of the device can be used to observe MAC-level queuing behavior.

In addition, the `PointToPointNetDevice` provides the standard set of MAC-level `NetDevice` trace sources:

- `MacTx`: Trace source for packets handed down from upper layers;
- `MacTxDrop`: Trace source for packets dropped by the device before transmission;
- `MacRx`: Trace source for packets successfully received in non-promiscuous mode;
- `MacPromiscRx`: Trace source for packets received in promiscuous mode and fired only when a promiscuous receive callback is installed via `SetPromiscReceiveCallback`;
- `Sniffer / PromiscSniffer`: Trace sources simulating non-promiscuous and promiscuous packet sniffers.

The `m_traceEnqueue` event is triggered when a packet is placed on the transmit queue. This happens at the time that `ns3::PointToPointNetDevice::Send` or `ns3::PointToPointNetDevice::SendFrom` is called by a higher layer to queue a packet for transmission. An Enqueue trace event firing should be interpreted as only indicating that a higher level protocol has sent a packet to the device.

The `m_traceDequeue` event is triggered when a packet is removed from the transmit queue. Dequeues from the transmit queue can happen in two situations: 1) If the underlying channel is idle when `PointToPointNetDevice::Send` is called, a packet is dequeued from the transmit queue and immediately transmitted; 2) a packet may be dequeued and immediately transmitted in an internal `TransmitCompleteEvent` that functions much like a transmit complete interrupt service routine. An Dequeue trace event firing may be viewed as indicating that the `PointToPointNetDevice` has begun transmitting a packet.

27.4.2 Lower-Level (PHY) Hooks

Similar to the upper level trace hooks, there are trace hooks available at the lower levels of the net device. We call these the PHY hooks. These events fire from the device methods that talk directly to the `PointToPointChannel`.

The trace source `m_dropTrace` is called to indicate a packet that is dropped by the device. This happens when a packet is discarded as corrupt due to a receive error model indication (see `ns3::ErrorModel` and the associated attribute “`ReceiveErrorModel`”).

The other low-level trace source fires on reception of a packet (see `ns3::PointToPointNetDevice::m_rxTrace`) from the `PointToPointChannel`.

PROPAGATION

The *ns-3* propagation module defines two generic interfaces, namely `PropagationLossModel` and `PropagationDelayModel`, to model respectively the propagation loss and the propagation delay.

28.1 Scope and Limitations

Current Support

- A wide range of propagation loss models (Friis, Two-Ray Ground, Log-Distance, Nakagami, etc.) covering free-space, empirical, and stochastic channel conditions
- Propagation delay modeling including constant speed and random delay models
- Frequency-dependent propagation loss models for different simulation scenarios
- Support for spectrum-aware propagation loss calculations
- Compatibility with both PHY and channel modules in *ns-3*

Current Limitations

- Some empirical models may have limited validity outside their intended environments
- Computational cost can increase significantly when using complex models with large node counts
- Limited built-in support for site-specific or ray-tracing-based propagation models
- Integration with terrain and obstacle-aware models requires external tools or extensions

28.2 PropagationLossModel

Propagation loss models calculate the Rx signal power considering the Tx signal power and the mutual Rx and Tx antennas positions.

A propagation loss model can be “chained” to another one, making a list. The final Rx power takes into account all the chained models. In this way one can use a slow fading and a fast fading model (for example), or model separately different fading effects.

The following propagation loss models are implemented:

Simple Range Models

- `FriisPropagationLossModel`
- `TwoRayGroundPropagationLossModel`
- `LogDistancePropagationLossModel`

- ThreeLogDistancePropagationLossModel
- RangePropagationLossModel

Fading Models

- NakagamiPropagationLossModel
- JakesPropagationLossModel

Abstract Models

- FixedRssLossModel
- RandomPropagationLossModel
- MatrixPropagationLossModel

Macro and Standard Models

- Cost231PropagationLossModel
- OkumuraHataPropagationLossModel
- Kun2600MhzPropagationLossModel
- ItuR1411LosPropagationLossModel
- ItuR1411NlosOverRooftopPropagationLossModel
- **ThreeGppPropagationLossModel**
 - ThreeGppRMaPropagationLossModel
 - ThreeGppUMaPropagationLossModel
 - ThreeGppUmiStreetCanyonPropagationLossModel
 - ThreeGppIndoorOfficePropagationLossModel
 - ThreeGppNTNDenseUrbanPropagationLossModel
 - ThreeGppNTNUrbanPropagationLossModel
 - ThreeGppNTNSuburbanPropagationLossModel
 - ThreeGppNTNRuralPropagationLossModel

Other models could be available thanks to other modules, e.g., the building module.

Each of the available propagation loss models of ns-3 is explained in one of the following subsections.

28.2.1 FriisPropagationLossModel

This model implements the Friis propagation loss model. This model was first described in the work of H.T.¹. The original equation was described as:

$$\frac{P_r}{P_t} = \frac{A_r A_t}{d^2 \lambda^2}$$

with the following equation for the case of an isotropic antenna with no heat loss:

$$A_{isotr.} = \frac{\lambda^2}{4\pi}$$

¹ Friis, H.T., "A Note on a Simple Transmission Formula," Proceedings of the IRE , vol.34, no.5, pp.254,256, May 1946

The final equation becomes:

$$\frac{P_r}{P_t} = \frac{\lambda^2}{(4\pi d)^2}$$

Modern extensions to this original equation are:

$$P_r = \frac{P_t G_t G_r \lambda^2}{(4\pi d)^2 L}$$

With:

P_t : transmission power (W)

P_r : reception power (W)

G_t : transmission gain (unit-less)

G_r : reception gain (unit-less)

λ : wavelength (m)

d : distance (m)

L : system loss (unit-less)

In the implementation, λ is calculated as $\frac{C}{f}$, where $C = 299792458$ m/s is the speed of light in vacuum, and f is the frequency in Hz which can be configured by the user via the Frequency attribute.

The Friis model is valid only for propagation in free space within the so-called far field region, which can be considered approximately as the region for $d > 3\lambda$. The model will still return a value for $d < 3\lambda$, as doing so (rather than triggering a fatal error) is practical for many simulation scenarios. However, we stress that the values obtained in such conditions shall not be considered realistic.

Related with this issue, we note that the Friis formula is undefined for $d = 0$, and results in $P_r > P_t$ for $d < \lambda/2\sqrt{\pi}$.

Both these conditions occur outside of the far field region, so in principle the Friis model shall not be used in these conditions. In practice, however, Friis is often used in scenarios where accurate propagation modeling is not deemed important, and values of $d = 0$ can occur.

To allow practical use of the model in such scenarios, we have to 1) return some value for $d = 0$, and 2) avoid large discontinuities in propagation loss values (which could lead to artifacts such as bogus capture effects which are much worse than inaccurate propagation loss values). The two issues are conflicting, as, according to the Friis formula, $\lim_{d \rightarrow 0} P_r = +\infty$; so if, for $d = 0$, we use a fixed loss value, we end up with an infinitely large discontinuity, which as we discussed can cause undesirable simulation artifacts.

To avoid these artifact, this implementation of the Friis model provides an attribute called MinLoss which allows to specify the minimum total loss (in dB) returned by the model. This is used in such a way that P_r continuously increases for $d \rightarrow 0$, until MinLoss is reached, and then stay constant; this allow to return a value for $d = 0$ and at the same time avoid discontinuities. The model won't be much realistic, but at least the simulation artifacts discussed before are avoided. The default value of MinLoss is 0 dB, which means that by default the model will return $P_r = P_t$ for $d \leq \lambda/2\sqrt{\pi}$. We note that this value of d is outside of the far field region, hence the validity of the model in the far field region is not affected.

28.2.2 TwoRayGroundPropagationLossModel

This model implements a Two-Ray Ground propagation loss model ported from NS2

The Two-ray ground reflection model uses the formula

$$P_r = \frac{P_t * G_t * G_r * (H_t^2 * H_r^2)}{d^4 * L}$$

The original equation in Rappaport's book assumes $L = 1$. To be consistent with the free space equation, L is added here.

H_t and H_r are set at the respective nodes z coordinate plus a model parameter set via `SetHeightAboveZ`.

The two-ray model does not give a good result for short distances, due to the oscillation caused by constructive and destructive combination of the two rays. Instead the Friis free-space model is used for small distances.

The crossover distance, below which Friis is used, is calculated as follows:

$$d_{Cross} = \frac{(4 * \pi * H_t * H_r)}{\lambda}$$

In the implementation, λ is calculated as $\frac{C}{f}$, where $C = 299792458$ m/s is the speed of light in vacuum, and f is the frequency in Hz which can be configured by the user via the `Frequency` attribute.

28.2.3 LogDistancePropagationLossModel

This model implements a log distance propagation model.

The reception power is calculated with a so-called log-distance propagation model:

$$L = L_0 + 10n \log\left(\frac{d}{d_0}\right)$$

where:

n : the path loss distance exponent

d_0 : reference distance (m)

L_0 : path loss at reference distance (dB)

d : - distance (m)

L : path loss (dB)

When the path loss is requested at a distance smaller than the reference distance, the tx power is returned.

28.2.4 ThreeLogDistancePropagationLossModel

This model implements a log distance path loss propagation model with three distance fields. This model is the same as `ns3::LogDistancePropagationLossModel` except that it has three distance fields: near, middle and far with different exponents.

Within each field the reception power is calculated using the log-distance propagation equation:

$$L = L_0 + 10 \cdot n_0 \log_{10}\left(\frac{d}{d_0}\right)$$

Each field begins where the previous ends and all together form a continuous function.

There are three valid distance fields: near, middle, far. Actually four: the first from 0 to the reference distance is invalid and returns txPowerDbm.

$$\underbrace{0 \dots d_0}_{=0} \underbrace{\dots d_1}_{n_0} \underbrace{\dots d_2}_{n_1} \underbrace{\dots \infty}_{n_2}$$

Complete formula for the path loss in dB:

$$L = \begin{cases} 0 & d < d_0 \\ L_0 + 10 \cdot n_0 \log_{10}\left(\frac{d}{d_0}\right) & d_0 \leq d < d_1 \\ L_0 + 10 \cdot n_0 \log_{10}\left(\frac{d_1}{d_0}\right) + 10 \cdot n_1 \log_{10}\left(\frac{d}{d_1}\right) & d_1 \leq d < d_2 \\ L_0 + 10 \cdot n_0 \log_{10}\left(\frac{d_1}{d_0}\right) + 10 \cdot n_1 \log_{10}\left(\frac{d_2}{d_1}\right) + 10 \cdot n_2 \log_{10}\left(\frac{d}{d_2}\right) & d_2 \leq d \end{cases}$$

where:

d_0, d_1, d_2 : three distance fields (m)

n_0, n_1, n_2 : path loss distance exponent for each field (unitless)

L_0 : path loss at reference distance (dB)

d : - distance (m)

L : path loss (dB)

When the path loss is requested at a distance smaller than the reference distance d_0 , the tx power (with no path loss) is returned. The reference distance defaults to 1m and reference loss defaults to FriisPropagationLossModel with 5.15 GHz and is thus $L_0 = 46.67$ dB.

28.2.5 RangePropagationLossModel

This propagation loss depends only on the distance (range) between transmitter and receiver.

The single MaxRange attribute (units of meters) determines path loss. Receivers at or within MaxRange meters receive the transmission at the transmit power level. Receivers beyond MaxRange receive at power -1000 dBm (effectively zero).

28.2.6 NakagamiPropagationLossModel

This propagation loss model implements the Nakagami-m fast fading model, which accounts for the variations in signal strength due to multipath fading. The model does not account for the path loss due to the distance traveled by the signal, hence for typical simulation usage it is recommended to consider using it in combination with other models that take into account this aspect.

The Nakagami-m distribution is applied to the power level. The probability density function is defined as

$$p(x; m, \omega) = \frac{2m^m}{\Gamma(m)\omega^m} x^{2m-1} e^{-\frac{m}{\omega} x^2}$$

with m the fading depth parameter and ω the average received power.

It is implemented by either a GammaRandomVariable or a ErlangRandomVariable random variable.

The implementation of the model allows to specify different values of the m parameter (and hence different fast fading profiles) for three different distance ranges:

$$\underbrace{0 \dots d_1}_{m_0} \underbrace{\dots d_2}_{m_1} \underbrace{\dots \infty}_{m_2}$$

For $m = 1$ the Nakagami-m distribution equals the Rayleigh distribution. Thus this model also implements Rayleigh distribution based fast fading.

28.2.7 JakesPropagationLossModel

This model implements a single path stationary Jakes propagation loss model for simulating Rayleigh fading channels. This model was first described in Microwave Mobile Communications², but the implementation available in ns-3 is based on a later document by Zheng et al.³ (Section II.A). The Jakes method is used to model effective Rayleigh fading channels through a deterministic approach, providing a computational simplification at the cost of slightly decreased accuracy compared to statistical methods. This model considers one transmitter-receiver pair and calculates the complex channel coefficients that represent the fading envelope.

The model generates complex channel coefficients using the sum-of-sinusoids approach:

$$X(t) = X_c(t) + jX_s(t)$$

where the in-phase and quadrature components are given by:

$$X_c(t) = \frac{2}{\sqrt{M}} \sum_{n=1}^M \cos(\psi_n) \cos(\omega_d t \cos(\alpha_n) + \phi_n)$$

$$X_s(t) = \frac{2}{\sqrt{M}} \sum_{n=1}^M \sin(\psi_n) \cos(\omega_d t \cos(\alpha_n) + \phi_n)$$

with the angle of arrival for the n-th sinusoid:

$$\alpha_n = \frac{2\pi n - \pi + \theta}{4M}, \quad n = 1, 2, \dots, M$$

where:

M : number of sinusoids (determines accuracy vs. computational complexity) ω_d : maximum Doppler frequency (rad/s) θ : uniformly distributed random variable over $[-\pi, \pi]$ ϕ_n : uniformly distributed random phase over $[-\pi, \pi]$ for each sinusoid ψ_n : uniformly distributed random variable over $[-\pi, \pi]$ for each sinusoid t : time variable

The parameters θ , ϕ_n , and ψ_n are statistically independent and uniformly distributed over $[-\pi, \pi]$ for all n . The resulting complex coefficient $X(t)$ has a magnitude that follows a Rayleigh distribution, making it suitable for modeling non-line-of-sight propagation scenarios.

The channel gain in dB is calculated from the complex coefficient as:

$$G_{dB}(t) = 10 \log_{10} \left(\frac{|X(t)|^2}{2} \right)$$

where $|X(t)|^2 = X_c(t)^2 + X_s(t)^2$ is the squared magnitude of the complex coefficient. The implementation uses oscillators where each oscillator contributes:

$$\text{oscillator}_n(t) = A_n e^{j(\omega_n t + \phi)}$$

with complex amplitude:

$$A_n = \frac{2}{\sqrt{M}} e^{j\psi_n} = \frac{2}{\sqrt{M}} (\cos(\psi_n) + j \sin(\psi_n))$$

and angular frequency:

$$\omega_n = \omega_d \cos(\alpha_n)$$

The total complex gain is the sum of all oscillator contributions.

²

W. C. Jakes, Microwave Mobile Communications. Piscataway, NJ:IEEE Press, 1994.

³ ZHENG, Yahong Rosa et XIAO, Chengshan. Simulation models with correct statistical properties for Rayleigh fading channels. IEEE Transactions on communications, 2003, vol. 51, no 6, p. 920-928.

28.2.8 FixedRssLossModel

This model sets a constant received power level independent of the transmit power.

The received power is constant independent of the transmit power; the user must set received power level. Note that if this loss model is chained to other loss models, it should be the first loss model in the chain. Else it will disregard the losses computed by loss models that precede it in the chain.

28.2.9 RandomPropagationLossModel

This model implements a completely random propagation loss mechanism where the attenuation varies randomly for each transmission, regardless of the distance or environmental conditions between nodes. The propagation loss changes every time the model is called, meaning that all packets (even those transmitted between two fixed nodes) experience independent random propagation losses.

Purpose and Use Cases:

The Random Propagation Loss Model is primarily designed for software testing purposes rather than for modeling a real-world channel. It serves several specific use cases:

- Show how to randomize results and work with the random stream module
- Fuzzing Testing: Provides random inputs to test the robustness of networking protocols and applications under unpredictable channel conditions
- Development Testing: Helps developers identify edge cases and validate that their code handles a wide range of signal strength variations
- Statistical Analysis: Enables Monte Carlo simulations where propagation effects need to be decoupled from spatial relationships

This model could also be used to generate something akin to small scale fading effects when paired with a path loss model, if the user wanted to create some new model from scratch using a specially-designed random variable.

Mathematical Formulation:

The received power is calculated as:

$$P_r = P_t - X$$

where:

P_r : received power in dBm P_t : transmitted power in dBm X : random attenuation value in dB, drawn from the configured random variable

The attenuation value X is generated independently for each call to the model, ensuring complete randomness across all transmissions. Configuration: The model uses a configurable random variable to determine the attenuation values:

Default: ns3::ConstantRandomVariable[Constant=1.0] (constant 1 dB attenuation)

Configurable: Any random variable stream can be used (uniform, exponential, normal, etc.)

Example Configurations:

```
// Uniform random attenuation between 0 and 10 dB
randomLoss->SetAttribute("Variable", StringValue("ns3::UniformRandomVariable[Min=0.
↪0|Max=10.0]"));

// Normal distribution with mean 5 dB, standard deviation 2 dB
```

(continues on next page)

(continued from previous page)

```
randomLoss->SetAttribute("Variable", StringValue("ns3::NormalRandomVariable[Mean=5.
↪0|Variance=4.0]"));

// Exponential distribution with mean 3 dB
randomLoss->SetAttribute("Variable", StringValue("ns3::ExponentialRandomVariable[Mean=3.
↪0]"));
```

Important Characteristics:

- No Spatial Correlation: The loss is completely independent of distance between transmitter and receiver
- No Environmental Modeling: Physical obstacles, terrain, or atmospheric conditions are ignored
- Temporal Independence: Each packet transmission experiences an independent random loss
- Deterministic Reproducibility: When using the same random seed, results are reproducible for testing

Stream Management:

The model supports proper random stream assignment for reproducible results across simulation runs. The random variable stream can be configured to ensure deterministic behavior when needed for regression testing.

28.2.10 MatrixPropagationLossModel

The propagation loss is fixed for each pair of nodes and doesn't depend on their actual positions. This model should be useful for synthetic tests. Note that by default the propagation loss is assumed to be symmetric.

28.2.11 Cost231PropagationLossModel

This model implements the COST 231⁵ propagation loss model, also known as the Hata Model PCS Extension. It is a radio propagation model that extends the Hata Model (which in turn is based on the Okumura Model) to cover a more elaborate range of frequencies.

COST (COopération européenne dans le domaine de la recherche Scientifique et Technique) is a European Union Forum for cooperative scientific research which developed this model based on various experiments and research studies. This model is specifically applicable to urban areas and can be extended to evaluate path loss in suburban or rural quasi-open/open areas.

Model Applicability:

- Frequency Range: 1500 MHz to 2000 MHz
- Mobile Station Antenna Height: 1 to 10 meters
- Base Station Antenna Height: 30 to 200 meters
- Link Distance: up to 20 kilometers
- Environment: Urban areas (primary), extensible to suburban/rural

Mathematical Formulation:

The path loss in dB is calculated using the COST 231 formula:

$$L = 46.3 + 33.9 \log_{10}(f) - 13.82 \log_{10}(h_b) - C_H + (44.9 - 6.55 \log_{10}(h_b)) \log_{10}(d) + S$$

⁵ "Digital Mobile Radio: COST 231 View on the Evolution Towards 3rd Generation Systems", Commission of the European Communities, L-2920, Luxembourg, 1989

where the mobile antenna height correction factor C_H is given by:

$$C_H = 0.8 + (1.11 \log_{10}(f) - 0.7)h_m - 1.56 \log_{10}(f)$$

Parameters:

f : frequency in MHz h_b : base station antenna height in meters h_m : mobile station antenna height in meters d : distance between transmitter and receiver in kilometers S : shadowing factor in dB (default: 10 dB) L : path loss in dB

Model Behavior:

For distances smaller than the minimum distance threshold (default 0.5m), the model returns 0 dB path loss. The model includes a configurable shadowing factor to account for environmental variations. Default parameters are set for 2.3 GHz frequency with base station height of 50m and mobile station height of 3m.

Implementation Notes:

The model uses logarithmic calculations in base 10, with frequency converted to MHz, distance to kilometers, and all height measurements in meters. The final result represents the path loss that should be subtracted from the transmitted power to obtain the received power level.

28.2.12 OkumuraHataPropagationLossModel

This model is used to model open area pathloss for long distance (i.e., > 1 Km). In order to include all the possible frequencies usable by LTE we need to consider several variants of the well known Okumura Hata model. In fact, the original Okumura Hata model⁴ is designed for frequencies ranging from 150 MHz to 1500 MHz, the COST231 [Page 532, 5](#) extends it for the frequency range from 1500 MHz to 2000 MHz. Another important aspect is the scenarios considered by the models, in fact the all models are originally designed for urban scenario and then only the standard one and the COST231 are extended to suburban, while only the standard one has been extended to open areas. Therefore, the model cannot cover all scenarios at all frequencies. In the following we detail the models adopted.

The pathloss expression of the COST231 OH is:

$$L = 46.3 + 33.9 \log f - 13.82 \log h_b + (44.9 - 6.55 \log h_b) \log d - F(h_M) + C$$

where

$$F(h_M) = \begin{cases} (1.1 \log(f)) - 0.7 \times h_M - (1.56 \times \log(f) - 0.8) & \text{for medium and small size cities} \\ 3.2 \times (\log(11.75 \times h_M))^2 & \text{for large cities} \end{cases}$$

$$C = \begin{cases} 0dB & \text{for medium-size cities and suburban areas} \\ 3dB & \text{for large cities} \end{cases}$$

and

f : frequency [MHz]

h_b : eNB height above the ground [m]

h_M : UE height above the ground [m]

d : distance [km]

$\log$: is a logarithm in base 10 (this for the whole document)

⁴ M.Hata, "Empirical formula for propagation loss in land mobile radio services", IEEE Trans. on Vehicular Technology, vol. 29, pp. 317-325, 1980

This model is only for urban scenarios.

The pathloss expression of the standard OH in urban area is:

$$L = 69.55 + 26.16 \log f - 13.82 \log h_b + (44.9 - 6.55 \log h_b) \log d - C_H$$

where for small or medium sized city

$$C_H = 0.8 + (1.1 \log f - 0.7)h_M - 1.56 \log f$$

and for large cities

$$C_H = \begin{cases} 8.29(\log(1.54h_M))^2 - 1.1 & \text{if } 150 \leq f \leq 200 \\ 3.2(\log(11.75h_M))^2 - 4.97 & \text{if } 200 < f \leq 1500 \end{cases}$$

There extension for the standard OH in suburban is

$$L_{SU} = L_U - 2 \left(\log \frac{f}{28} \right)^2 - 5.4$$

where

L_U : pathloss in urban areas

The extension for the standard OH in open area is

$$L_O = L_U - 4.78(\log f)^2 + 18.33 \log f - 40.94$$

The literature lacks of extensions of the COST231 to open area (for suburban it seems that we can just impose $C = 0$); therefore we consider it a special case of the suburban one.

28.2.13 Kun2600MhzPropagationLossModel

This is the empirical model for the pathloss at 2600 MHz for urban areas which is described in Kun et al.⁸. The model is as follows. Let d be the distance between the transmitter and the receiver in meters; the pathloss L in dB is calculated as:

$$L = 36 + 26 \log d$$

28.2.14 ItuR1411LosPropagationLossModel

This model is designed for Line-of-Sight (LoS) short range outdoor communication in the frequency range 300 MHz to 100 GHz. This model provides an upper and lower bound respectively according to the following formulas

$$L_{LoS,l} = L_{bp} + \begin{cases} 20 \log \frac{d}{R_{bp}} & \text{for } d \leq R_{bp} \\ 40 \log \frac{d}{R_{bp}} & \text{for } d > R_{bp} \end{cases}$$

$$L_{LoS,u} = L_{bp} + 20 + \begin{cases} 25 \log \frac{d}{R_{bp}} & \text{for } d \leq R_{bp} \\ 40 \log \frac{d}{R_{bp}} & \text{for } d > R_{bp} \end{cases}$$

where the breakpoint distance is given by

$$R_{bp} \approx \frac{4h_b h_m}{\lambda}$$

and the above parameters are

⁸ Sun Kun, Wang Ping, Li Yingze, "Path loss models for suburban scenario at 2.3GHz, 2.6GHz and 3.5GHz", in Proc. of the 8th International Symposium on Antennas, Propagation and EM Theory (ISAPE), Kunming, China, Nov 2008.

λ : wavelength [m]

h_b : eNB height above the ground [m]

h_m : UE height above the ground [m]

d : distance [m]

and L_{bp} is the value for the basic transmission loss at the break point, defined as:

$$L_{bp} = \left| 20 \log \left(\frac{\lambda^2}{8\pi h_b h_m} \right) \right|$$

The value used by the simulator is the average one for modeling the median pathloss.

28.2.15 ItuR1411NlosOverRooftopPropagationLossModel

This model is designed for Non-Line-of-Sight (LoS) short range outdoor communication over rooftops in the frequency range 300 MHz to 100 GHz. This model includes several scenario-dependent parameters, such as average street width, orientation, etc. It is advised to set the values of these parameters manually (using the ns-3 attribute system) according to the desired scenario.

In detail, the model is based on work by Walfisch et al.⁶ and Ikegami et al.⁷, where the loss is expressed as the sum of free-space loss (L_{bf}), the diffraction loss from rooftop to street (L_{rts}) and the reduction due to multiple screen diffraction past rows of building (L_{msd}). The formula is:

$$L_{NLOS1} = \begin{cases} L_{bf} + L_{rts} + L_{msd} & \text{for } L_{rts} + L_{msd} > 0 \\ L_{bf} & \text{for } L_{rts} + L_{msd} \leq 0 \end{cases}$$

The free-space loss is given by:

$$L_{bf} = 32.4 + 20 \log(d/1000) + 20 \log(f)$$

where:

f : frequency [MHz]

d : distance (where $d > 1$) [m]

The term L_{rts} takes into account the width of the street and its orientation, according to the formulas

$$L_{rts} = -8.2 - 10 \log(w) + 10 \log(f) + 20 \log(\Delta h_m) + L_{ori}$$

$$L_{ori} = \begin{cases} -10 + 0.354\varphi & \text{for } 0^\circ \leq \varphi < 35^\circ \\ 2.5 + 0.075(\varphi - 35) & \text{for } 35^\circ \leq \varphi < 55^\circ \\ 4.0 - 0.114(\varphi - 55) & \text{for } 55^\circ \leq \varphi \leq 90^\circ \end{cases}$$

$$\Delta h_m = h_r - h_m$$

where:

h_r : is the height of the rooftop [m]

h_m : is the height of the mobile [m]

φ : is the street orientation with respect to the direct path (degrees)

⁶ J.Walfisch and H.L. Bertoni, "A Theoretical model of UHF propagation in urban environments," in IEEE Trans. Antennas Propagat., vol.36, 1988, pp.1788- 1796

⁷ F.Ikegami, T.Takeuchi, and S.Yoshida, "Theoretical prediction of mean field strength for Urban Mobile Radio", in IEEE Trans. Antennas Propagat., Vol.39, No.3, 1991

The multiple screen diffraction loss depends on the BS antenna height relative to the building height and on the incidence angle. The former is selected as the higher antenna in the communication link. Regarding the latter, the “settled field distance” is used for select the proper model; its value is given by

$$d_s = \frac{\lambda d^2}{\Delta h_b^2}$$

with

$$\Delta h_b = h_b - h_m$$

Therefore, in case of $l > d_s$ (where l is the distance over which the building extend), it can be evaluated according to

$$L_{msd} = L_{bsh} + k_a + k_d \log(d/1000) + k_f \log(f) - 9 \log(b)$$

$$L_{bsh} = \begin{cases} -18 \log(1 + \Delta h_b) & \text{for } h_b > h_r \\ 0 & \text{for } h_b \leq h_r \end{cases}$$

$$k_a = \begin{cases} 71.4 & \text{for } h_b > h_r \text{ and } f > 2000 \text{ MHz} \\ 54 & \text{for } h_b > h_r \text{ and } f \leq 2000 \text{ MHz} \\ 54 - 0.8\Delta h_b & \text{for } h_b \leq h_r \text{ and } d \geq 500 \text{ m} \\ 54 - 1.6\Delta h_b & \text{for } h_b \leq h_r \text{ and } d < 500 \text{ m} \end{cases}$$

$$k_d = \begin{cases} 18 & \text{for } h_b > h_r \\ 18 - 15 \frac{\Delta h_b}{h_r} & \text{for } h_b \leq h_r \end{cases}$$

$$k_f = \begin{cases} -8 & \text{for } f > 2000 \text{ MHz} \\ -4 + 0.7(f/925 - 1) & \text{for medium city and suburban centres and } f \leq 2000 \text{ MHz} \\ -4 + 1.5(f/925 - 1) & \text{for metropolitan centres and } f \leq 2000 \text{ MHz} \end{cases}$$

Alternatively, in case of $l < d_s$, the formula is:

$$L_{msd} = -10 \log(Q_M^2)$$

where

$$Q_M = \begin{cases} 2.35 \left(\frac{\Delta h_b}{d} \sqrt{\frac{b}{\lambda}} \right)^{0.9} & \text{for } h_b > h_r \\ \frac{b}{d} & \text{for } h_b \approx h_r \\ \frac{b}{2\pi d} \sqrt{\frac{\lambda}{\rho}} \left(\frac{1}{\theta} - \frac{1}{2\pi + \theta} \right) & \text{for } h_b < h_r \end{cases}$$

where:

$$\theta = \arctan\left(\frac{\Delta h_b}{b}\right)$$

$$\rho = \sqrt{\Delta h_b^2 + b^2}$$

28.2.16 ThreeGppPropagationLossModel

The base class `ThreeGppPropagationLossModel` and its derived classes implement the path loss and shadow fading models described in 3GPP TR 38.901⁹. 3GPP TR 38.901 includes multiple scenarios modeling different propagation environments, i.e., indoor, outdoor urban and rural, for frequencies between 0.5 and 100 GHz.

Implemented features:

- Path loss and shadowing models (3GPP TR 38.901, Sec. 7.4.1)
- Autocorrelation of shadow fading (3GPP TR 38.901, Sec. 7.4.4)

⁹ 3GPP. 2018. TR 38.901, Study on channel model for frequencies from 0.5 to 100 GHz, V15.0.0. (2018-06).

- **Channel condition models** (3GPP TR 38.901, Sec. 7.4.2)
- **O2I Low/High Building penetration losses** (3GPP TR 38.901, Sec. 7.4.3.1). The Low/High Building penetration losses can be enabled for UEs that are in O2I channel condition state. For determining the O2O/O2I state in ns-3 there are two possible scenario setups: 1) ns-3 buildings and BuildingsChannelConditionModel are used. Then the O2I condition is calculated based on the buildings in the scenario. In this case losses are considered by default, however there is the option to disable them by setting the ThreeGppPropagationLossModel attribute BuildingPenetrationLossesEnabled to false. 2) ns-3 buildings are not used, instead one of the 3GPP stochastic channel condition models is used, such as: ThreeGppRmaChannelConditionModel, ThreeGppUmaChannelConditionModel, ThreeGppUmiStreetCanyonChannelConditionModel. These models are extended to calculate the O2O/O2I state probabilistically. Additionally, it is possible to configure to calculate O2I condition deterministically based on the UE height. In both cases, the O2O/O2I is updated at the same time as LOS/nLOS, i.e. with the same periodicity. For other 3GPP channel condition models in ns-3 there are no O2I losses because both TX and RX, are either indoor or outdoor, such as in the case of indoor or V2V scenarios. For this case, to consider O2I Low/High Losses, the attribute of the ChannelCondition O2iThreshold (that indicates the ratio between O2O and O2I states) must be set to a value different from 0. Possible values are from 0 to 1, with 1 corresponding to 100% O2I conditions.

To be implemented:

- O2I Car penetration losses (3GPP TR 38.901, Sec. 7.4.3.2).
- Spatial consistent update of the channel states (3GPP TR 38.901 Sec. 7.6.3.3)

Configuration

The ThreeGppPropagationLossModel instance is paired with a ChannelConditionModel instance used to retrieve the LOS/NLOS channel condition. By default, a 3GPP channel condition model related to the same scenario is set (e.g., by default, ThreeGppRmaPropagationLossModel is paired with ThreeGppRmaChannelConditionModel), but it can be configured using the method SetChannelConditionModel. The channel condition models are stored inside the propagation module, for a limitation of the current spectrum API and to avoid a circular dependency between the spectrum and the propagation modules. Please note that it is necessary to install at least one ChannelConditionModel when using any ThreeGppPropagationLossModel subclass. Please look below for more information about the Channel Condition models.

The operating frequency has to be set using the attribute “Frequency”, otherwise an assert is raised. The addition of the shadow fading component can be enabled/disabled through the attribute “ShadowingEnabled”. Other scenario-related parameters can be configured through attributes of the derived classes.

One other attribute, “EnforceParameterRanges”, can be set to true if the model is configured with parameters that fall outside of the applicability range specified in TR38.901. By default, the simulator will only warn (if logging is enabled) if a value is outside of the applicability range. If this parameter is set to true, the simulation will abort with an error message (instead of just logging a warning) if the applicability range is exceeded.

Implementation details

The method DoCalcRxPower computes the propagation loss considering the path loss and the shadow fading (if enabled). The path loss is computed by the method GetLossLos or GetLossNlos depending on the LOS/NLOS channel condition, and their implementation is left to the derived classes. The shadow fading is computed by the method GetShadowing, which generates an additional random loss component characterized by Gaussian distribution with zero mean and scenario-specific standard deviation. Subsequent shadowing components of each BS-UT link are correlated as described in 3GPP TR 38.901, Sec. 7.4.4^{Page 536, 9}.

Note 1: The TR defines height ranges for UTs and BSs, depending on the chosen propagation model (for the exact values, please see below in the specific model documentation). If the user does not set correct values, the model will emit a warning but perform the calculation anyway.

Note 2: The 3GPP model is originally intended to be used to represent BS-UT links. However, in ns-3, we may need to compute the pathloss between two BSs or UTs to evaluate the interference. We have decided to support this case by

considering the tallest node as a BS and the smallest as a UT. As a consequence, the height values may be outside the validity range of the chosen class: therefore, an inaccuracy warning may be printed, but it can be ignored.

There are four derived class, each one implementing the propagation model for a different scenario:

ThreeGppRMaPropagationLossModel

This class implements the LOS/NLOS path loss and shadow fading models described in 3GPP TR 38.901^{Page 536, 9}, Table 7.4.1-1 for the RMa scenario. It supports frequencies between 0.5 and 30 GHz. It is possible to configure some scenario-related parameters through the attributes AvgBuildingHeight and AvgStreetWidth.

As specified in the TR, the 2D distance between the transmitter and the receiver should be between 10 m and 10 km for the LOS case, or between 10 m and 5 km for the NLOS case, otherwise the model may not be accurate (a warning message is printed if the user has enabled logging on the model, or the simulation aborts, depending on whether “EnforceParameterRanges” is set to true). Also, the height of the base station (hBS) should be between 10 m and 150 m, while the height of the user terminal (hUT) should be between 1 m and 10 m.

ThreeGppUMaPropagationLossModel

This implements the LOS/NLOS path loss and shadow fading models described in 3GPP TR 38.901^{Page 536, 9}, Table 7.4.1-1 for the UMa scenario. It supports frequencies between 0.5 and 100 GHz.

As specified in the TR, the 2D distance between the transmitter and the receiver should be between 10 m and 5 km both for the LOS and NLOS cases, otherwise the model may not be accurate (a warning message is printed if the user has enabled logging on the model, or the simulation aborts, depending on whether “EnforceParameterRanges” is set to true). Also, the height of the base station (hBS) should be 25 m and the height of the user terminal (hUT) should be between 1.5 m and 22.5 m.

ThreeGppUmiStreetCanyonPropagationLossModel

This implements the LOS/NLOS path loss and shadow fading models described in 3GPP TR 38.901^{Page 536, 9}, Table 7.4.1-1 for the UMi-Street Canyon scenario. It supports frequencies between 0.5 and 100 GHz.

As specified in the TR, the 2D distance between the transmitter and the receiver should be between 10 m and 5 km both for the LOS and NLOS cases, otherwise the model may not be accurate (a warning message is printed if the user has enabled logging on the model, or the simulation aborts, depending on whether “EnforceParameterRanges” is set to true). Also, the height of the base station (hBS) should be 10 m and the height of the user terminal (hUT) should be between 1.5 m and 10 m (the validity range is reduced because we assume that the height of the UT nodes is always lower than the height of the BS nodes).

ThreeGppIndoorOfficePropagationLossModel

This implements the LOS/NLOS path loss and shadow fading models described in 3GPP TR 38.901^{Page 536, 9}, Table 7.4.1-1 for the Indoor-Office scenario. It supports frequencies between 0.5 and 100 GHz.

As specified in the TR, the 3D distance between the transmitter and the receiver should be between 1 m and 150 m both for the LOS and NLOS cases, otherwise the model may not be accurate (a warning log message is printed if the user has enabled logging on the model, or the simulation aborts, depending on whether “EnforceParameterRanges” is set to true).

Testing

The test suite `ThreeGppPropagationLossModelsTestSuite` provides test cases for the classes implementing the 3GPP propagation loss models. The test cases `ThreeGppRmaPropagationLossModelTestCase`, `ThreeGppUmaPropagationLossModelTestCase`, `ThreeGppUmiPropagationLossModelTestCase` and `ThreeGppIndoorOfficePropagationLossModelTestCase` compute the path loss between two nodes and compares it with the value obtained using the formulas in 3GPP TR 38.901^{Page 536, 9}, Table 7.4.1-1. The test case `ThreeGppShadowingTestCase` checks if the shadowing is correctly computed by testing the deviation of the overall propagation loss from the path loss. The test is carried out for all the scenarios, both in LOS and NLOS condition.

28.2.17 ThreeGppNTNPropagationLossModel

Four different classes have been derived from the base class `ThreeGppPropagationLossModel` to support Non-Terrestrial Networks, one for each scenario presented in 3GPP TR 38.811¹², i.e., dense urban, urban, suburban and rural.

Implemented features:

- Line of Sight and Not Line of Sight (LOS/NLOS) probability models (3GPP TR 38.811, Sec. 6.6.1)
- Path loss, shadowing models and clutter loss (3GPP TR 38.811, Sec. 6.6.2)
- Atmospheric absorption model (3GPP TR 38.811, Sec. 6.6.4)
- Ionospheric and tropospheric scintillation (3GPP TR 38.811, Sec. 6.6.6)
- All the features already implemented in `ThreeGppPropagationLossModel`

ThreeGppNTNDenseUrbanPropagationLossModel

This class implements the LOS/NLOS path loss and shadow fading models described in 3GPP TR 38.811¹², Table 6.6.2-1 for the Dense Urban scenario. It supports frequencies between 0.5 and 100 GHz.

ThreeGppNTNUrbanPropagationLossModel

This class implements the LOS/NLOS path loss and shadow fading models described in 3GPP TR 38.811¹², Table 6.6.2-2 for the Urban scenario. It supports frequencies between 0.5 and 100 GHz.

ThreeGppNTNSuburbanPropagationLossModel

This class implements the LOS/NLOS path loss and shadow fading models described in 3GPP TR 38.811¹², Table 6.6.2-3 for the Suburban scenario. It supports frequencies between 0.5 and 100 GHz.

ThreeGppNTNRuralPropagationLossModel

This class implements the LOS/NLOS path loss and shadow fading models described in 3GPP TR 38.811¹², Table 6.6.2-3 for the Rural scenario. It supports frequencies between 0.5 and 100 GHz.

¹² 3GPP. 2018. TR 38.811, Study on New Radio (NR) to support non-terrestrial networks, V15.4.0. (2020-09).

28.3 Usage

This section describes how to effectively use propagation models in ns-3 simulations, covering basic configuration, advanced techniques, performance optimization, and common applications.

28.3.1 Basic Configuration

Setting Up a Propagation Model

The most fundamental step in using propagation models is configuring them properly in your ns-3 simulation. Here's the basic workflow:

1. Include Required Headers

```
#include "ns3/propagation-loss-model.h"
#include "ns3/propagation-delay-model.h"
#include "ns3/yans-wifi-helper.h"
```

2. Create and Configure the Channel

```
// Create the wireless channel
YansWifiChannelHelper channel = YansWifiChannelHelper::Default();

// Set the propagation loss model
channel.AddPropagationLoss("ns3::FriisPropagationLossModel");

// Set the propagation delay model
channel.SetPropagationDelay("ns3::ConstantSpeedPropagationDelayModel");
```

3. Apply to PHY Helper

```
YansWifiPhyHelper phy;
phy.SetChannel(channel.Create());
```

Setting Model Parameters

Most propagation models accept parameters that can be configured using attributes:

```
// Configure Friis model parameters
channel.AddPropagationLoss("ns3::FriisPropagationLossModel",
                          "Frequency", DoubleValue(2.4e9),
                          "SystemLoss", DoubleValue(1.0));
```

```
// Configure Log Distance model
channel.AddPropagationLoss("ns3::LogDistancePropagationLossModel",
                          "Exponent", DoubleValue(3.0),
                          "ReferenceDistance", DoubleValue(1.0),
                          "ReferenceLoss", DoubleValue(46.6777));
```

Using Different Models

ns-3 provides several built-in propagation models:

```
// Free space model (ideal conditions)
channel.AddPropagationLoss("ns3::FriisPropagationLossModel");
```

```
// Log distance model (general purpose)
channel.AddPropagationLoss("ns3::LogDistancePropagationLossModel");
```

```
// Two-ray ground reflection model
channel.AddPropagationLoss("ns3::TwoRayGroundPropagationLossModel");
```

```
// Okumura-Hata model (urban environments)
channel.AddPropagationLoss("ns3::OkumuraHataPropagationLossModel");
```

28.3.2 Model Stacking

One must be aware of the stackability of propagation models in ns-3. This allows you to combine different propagation effects for more realistic simulations but if not used properly, it may lead to flawed results. The order matters! The Base model should be set first, then additional effects

Propagation loss models can be chained together, where each model in the chain applies its loss calculation to the signal:

```
YansWifiChannelHelper channel = YansWifiChannelHelper::Default();
// Free space model (ideal conditions)
channel.AddPropagationLoss("ns3::FriisPropagationLossModel");
// Add a random propagation loss model for fading
channel.AddPropagationLoss("ns3::RandomPropagationLossModel");
```

As the above example shows, the stacking of models is very easy and does not require any additional tool. This also means that the user must be careful not to stack models by inadvertence.

28.3.3 Classification for stacking

If we take the list of available loss models and create two categories, base loss model and stacking models, here is the result:

Base Models (Primary Path Loss - Use ONE (1) as foundation):

- FriisPropagationLossModel
- TwoRayGroundPropagationLossModel
- LogDistancePropagationLossModel
- ThreeLogDistancePropagationLossModel
- RangePropagationLossModel
- OkumuraHataPropagationLossModel
- Cost231PropagationLossModel
- ItuR1411LosPropagationLossModel
- ItuR1411NlosOverRooftopPropagationLossModel
- Kun2600MhzPropagationLossModel

Stacking Models (Additional Effects - Stack on base models):

- JakesPropagationLossModel

- RandomPropagationLossModel
- NakagamiPropagationLossModel
- FixedRssLossModel
- MatrixPropagationLossModel

Complete 3GPP Models (Self-contained):

3GPP models are comprehensive, do not stack other models with them as they already include multiple propagation effects.

- ThreeGppPropagationLossModel
 - ThreeGppRMaPropagationLossModel
 - ThreeGppUMaPropagationLossModel
 - ThreeGppUmiStreetCanyonPropagationLossModel
 - ThreeGppIndoorOfficePropagationLossModel
 - ThreeGppNTNDenseUrbanPropagationLossModel
 - ThreeGppNTNUrbanPropagationLossModel
 - ThreeGppNTNSuburbanPropagationLossModel
 - ThreeGppNTNRuralPropagationLossModel

28.3.4 Typical Stacking Patterns

Pattern 1: Base + Shadowing

```
// Base path loss
channel.AddPropagationLoss("ns3::LogDistancePropagationLossModel");
// Add log-normal shadowing
channel.AddPropagationLoss("ns3::RandomPropagationLossModel",
                           "Variable", StringValue("ns3::LogNormalRandomVariable[Mu=0.
↪0|Sigma=4.0]"));
```

Pattern 2: Base + Fading

```
// Base path loss
channel.AddPropagationLoss("ns3::FriisPropagationLossModel");
// Add Nakagami fading
channel.AddPropagationLoss("ns3::NakagamiPropagationLossModel");
```

Pattern 3: Base + Shadowing + Fading

```
// Base path loss
channel.AddPropagationLoss("ns3::TwoRayGroundPropagationLossModel");
// Add shadowing
channel.AddPropagationLoss("ns3::RandomPropagationLossModel",
                           "Variable", StringValue("ns3::LogNormalRandomVariable[Mu=0.
↪0|Sigma=6.0]"));
// Add fading
channel.AddPropagationLoss("ns3::JakesPropagationLossModel");
```

28.4 ChannelConditionModel

The loss models require to know if two nodes are in Line-of-Sight (LoS) or if they are not. The interface for that is represented by this class. The main method is `GetChannelCondition(a, b)`, which returns a `ChannelCondition` object containing the information about the channel state.

We modeled the LoS condition in two ways: (i) by using a probabilistic model specified by the 3GPP (), and (ii) by using an ns-3 specific building-aware model, which checks the space position of the BSs and the UTs. For what regards the first option, the probability is independent of the node location: in other words, following the 3GPP model, two UT spatially separated by an epsilon may have different LoS conditions. To take into account mobility, we have inserted a parameter called “UpdatePeriod,” which indicates how often a 3GPP-based channel condition has to be updated. By default, this attribute is set to 0, meaning that after the channel condition is generated, it is never updated. With this default value, we encourage the users to run multiple simulations with different seeds to get statistical significance from the data. For the users interested in using mobile nodes, we suggest changing this parameter to a value that takes into account the node speed and the desired accuracy. For example, lower-speed node conditions may be updated in terms of seconds, while high-speed UT or BS may be updated more often.

The two approach are coded, respectively, in the classes:

- `ThreeGppChannelConditionModel`
- `BuildingsChannelConditionModel` (see the building module documentation for further details)

28.4.1 ThreeGppChannelConditionModel

This is the base class for the 3GPP channel condition models. It provides the possibility to update the condition of each channel periodically, after a given time period which can be configured through the attribute “UpdatePeriod”. If “UpdatePeriod” is set to 0, the channel condition is never updated. It has five derived classes implementing the channel condition models described in 3GPP TR 38.901^{Page 536, 9} for different propagation scenarios.

ThreeGppRmaChannelConditionModel

This class implements the statistical channel condition model described in 3GPP TR 38.901^{Page 536, 9}, Table 7.4.2-1, for the RMa scenario.

ThreeGppUmaChannelConditionModel

This class implements the statistical channel condition model described in 3GPP TR 38.901^{Page 536, 9}, Table 7.4.2-1, for the UMa scenario.

ThreeGppUmiStreetCanyonChannelConditionModel

This class implements the statistical channel condition model described in 3GPP TR 38.901^{Page 536, 9}, Table 7.4.2-1, for the UMi-Street Canyon scenario.

ThreeGppIndoorMixedOfficeChannelConditionModel

This class implements the statistical channel condition model described in 3GPP TR 38.901^{Page 536, 9}, Table 7.4.2-1, for the Indoor-Mixed office scenario.

ThreeGppIndoorOpenOfficeChannelConditionModel

This class implements the statistical channel condition model described in 3GPP TR 38.901^{Page 536, 9}, Table 7.4.2-1, for the Indoor-Open office scenario.

28.4.2 ThreeGppNTNChannelConditionModel

This is the base class for the 3GPP NTN channel condition models. It provides the possibility to update the condition of each channel periodically, after a given time period which can be configured through the attribute “UpdatePeriod”. If “UpdatePeriod” is set to 0, the channel condition is never updated. It has four derived classes implementing the channel condition models described in 3GPP TR 38.811^{Page 539, 12} for the different propagation scenarios. i.e., dense urban, urban, suburban and rural.

ThreeGppDenseUrbanChannelConditionModel

This class implements the statistical channel condition model described in 3GPP TR 38.811^{Page 539, 12}, Table 6.6.1-1, for the Dense Urban scenario.

ThreeGppUrbanChannelConditionModel

This class implements the statistical channel condition model described in 3GPP TR 38.811^{Page 539, 12}, Table 6.6.1-1, for the Urban scenario.

ThreeGppSuburbanStreetCanyonChannelConditionModel

This class implements the statistical channel condition model described in 3GPP TR 38.811^{Page 539, 12}, Table 6.6.1-1, for the Suburban scenario.

ThreeGppRuralChannelConditionModel

This class implements the statistical channel condition model described in 3GPP TR 38.811^{Page 539, 12}, Table 6.6.1-1, for the Rural office scenario.

28.4.3 Testing

The test suite ChannelConditionModelsTestSuite contains a single test case:

- ThreeGppChannelConditionModelTestCase, which tests all the 3GPP channel condition models. It determines the channel condition between two nodes multiple times, estimates the LOS probability, and compares it with the value given by the formulas in 3GPP TR 38.901^{Page 536, 9}, Table 7.4.2-1

28.5 PropagationDelayModel

The following propagation delay models are implemented:

- ConstantSpeedPropagationDelayModel
- RandomPropagationDelayModel

28.5.1 ConstantSpeedPropagationDelayModel

In this model, the signal travels with constant speed. The delay is calculated according with the transmitter and receiver positions. The Euclidean distance between the Tx and Rx antennas is used. Beware that, according to this model, the Earth is flat.

28.5.2 RandomPropagationDelayModel

The propagation delay is totally random, and it changes each time the model is called. All the packets (even those between two fixed nodes) experience a random delay. As a consequence, the packets order is not preserved.

28.6 Models for vehicular environments

The 3GPP TR 37.885¹⁰ specifications extends the channel modeling framework described in TR 38.901^{Page 536, 9} to simulate wireless channels in vehicular environments. The extended framework supports frequencies between 0.5 to 100 GHz and provides the possibility to simulate urban and highway propagation environments. To do so, new propagation loss and channel condition models, as well as new parameters for the fast fading model, are provided.

28.6.1 Vehicular channel condition models

To properly capture channel dynamics in vehicular environments, three different channel conditions have been identified:

- LOS (Line Of Sight): represents the case in which the direct path between the transmitter and the receiver is not blocked
- NLOSv (Non Line Of Sight vehicle): when the direct path between the transmitter and the receiver is blocked by a vehicle
- NLOS (Non Line Of Sight): when the direct path is blocked by a building

TR 37.885 includes two models that can be used to determine the condition of the wireless channel between a pair of nodes, the first for urban and the second for highway environments. Each model includes both a deterministic and a stochastic part, and works as follows:

1. The model determines the presence of buildings obstructing the direct path between the communicating nodes. This is done in a deterministic way, looking at the possible interceptions between the direct path and the buildings. If the path is obstructed, the channel condition is set to NLOS.
2. If not, the model determines the presence of vehicles obstructing the direct path. This is done using a probabilistic model, which is specific for the scenario of interest. If the path is obstructed, the channel condition is set to NLOSv, otherwise is set to LOS.

¹⁰ 3GPP. 2019. TR 37.885, Study on evaluation methodology of new Vehicle-to-Everything (V2X) use cases for LTE and NR, V15.3.0. (2019-06).

These models have been implemented by extending the interface `ChannelConditionModel` with the following classes. They have been included in the building module, because they make use of `Buildings` objects to determine the presence of obstructions caused by buildings.

- `ThreeGppV2vUrbanChannelConditionModel`: implements the model described in Table 6.2-1 of TR 37.885 for the urban scenario.
- `ThreeGppV2vHighwayChannelConditionModel`: implements the model described in Table 6.2-1 of TR 37.885 for the highway scenario.

These models rely on `Buildings` objects to determine the presence of obstructing buildings. When considering large scenarios with a large number of buildings, this process may become computationally demanding and dramatically increase the simulation time. To solve this problem, we implemented two fully-probabilistic models that can be used as an alternative to the ones included in TR 37.885. These models are based on the work carried out by M. Boban et al.¹¹, which derived a statistical representation of the three channel conditions. With the fully-probabilistic models there is no need to determine the presence of blocking buildings in a deterministic way, and therefore the computational effort is reduced. To determine the channel condition, these models account for the propagation environment, i.e., urban or highway, as well as for the density of vehicles in the scenario, which can be high, medium, or low.

The classes implementing the fully-probabilistic models are:

- `ProbabilisticV2vUrbanChannelConditionModel`: implements the model described in M. Boban et al. [Page 546, 11](#) for the urban scenario.
- `ProbabilisticV2vHighwayChannelConditionModel`: implements the model described in M. Boban et al.¹¹ for the highway scenario.

Both the classes own the attribute “Density”, which can be used to select the proper value depending on the scenario that have to be simulated. Differently from the hybrid models described above, these classes have been included in the propagation module, since they do not have any dependency on the building module.

NOTE: Both the hybrid and the fully-probabilistic models supports the modeling of outdoor scenarios, no support is provided for the modeling of indoor scenarios.

28.6.2 Vehicular propagation loss models

The propagation models described in TR 37.885 determines the attenuation caused by path loss and shadowing by considering the propagation environment and the channel condition.

These models have been implemented by extending the interface `ThreeGppPropagationLossModel` with the following classes, which are part of the propagation module:

- `ThreeGppV2vUrbanPropagationLossModel`: implements the models defined in Table 6.2.1-1 of TR 37.885 for the urban scenario.
- `ThreeGppV2vHighwayPropagationLossModel`: implements the models defined in Table 6.2.1-1 of TR 37.885 for the highway scenario.

As for all the classes extending the interface `ThreeGppPropagationLossModel`, they have to be paired with an instance of the class `ChannelConditionModel` which is used to determine the channel condition. This is done by setting the attribute `ChannelConditionModel`. To build the channel modeling framework described in TR 37.885, `ThreeGppV2vUrbanChannelConditionModel` or `ThreeGppV2vHighwayChannelConditionModel` should be used, but users are allowed to test any other combination.

¹¹ M. Boban, X. Gong, and W. Xu, “Modeling the evolution of line-of-sight blockage for V2V channels,” in IEEE 84th Vehicular Technology Conference (VTC-Fall), 2016.

28.6.3 Vehicular fast fading model

The fast fading model described in Sec. 6.2.3 of TR 37.885 is based on the one specified in TR 38.901, whose implementation is provided in the `spectrum` module (see the [spectrum module documentation](#)). This model is general and includes different parameters which can be tuned to simulate multiple propagation environments. To better model the channel dynamics in vehicular environments, TR 37.885 provides new sets of values for these parameters, specific for vehicle-to-vehicle transmissions in urban and highway scenarios. To select the parameters for vehicular scenarios, it is necessary to set the attribute “Scenario” of the class `ThreeGppChannelModel` using the value “V2V-Urban” or “V2V-Highway”.

additionally, TR 37.885 specifies a new equation to compute the Doppler component, which accounts for the mobility of both nodes, as well as scattering from the environment. In particular, the scattering effect is considered by deviating the Doppler frequency by a random value, whose distribution depends on the parameter v_{scatt} . TR 37.885 specifies that v_{scatt} should be set to the maximum speed of the vehicles in the layout and, if $v_{scatt} = 0$, the scattering effect is not considered. The Doppler equation is implemented in the class `ThreeGppSpectrumPropagationLossModel`. By means of the attribute “vScatt”, it is possible to adjust the value of $v_{scatt} = 0$ (by default, the value is set to 0).

28.6.4 Example

We implemented the example `three-gpp-v2v-channel-example.cc` which shows how to configure the different classes to simulate wireless propagation in vehicular scenarios, it can be found in the folder `examples/channel-models`.

We considered two communicating vehicles moving within the scenario, and computed the SNR experienced during the entire simulation, with a time resolution of 10 ms. The vehicles are equipped with 2x2 antenna arrays modeled using the [3GPP antenna model](#). The bearing and the downtilt angles are properly configured and the optimal beamforming vectors are computed at the beginning of the simulation.

The simulation script accepts the following command line parameters:

- *frequency*: the operating frequency in Hz
- *txPow*: the transmission power in dBm
- *noiseFigure*: the noise figure in dB
- *scenario*: the simulation scenario, “V2V-Urban” or “V2V-Highway”

The “V2V-Urban” scenario simulates urban environment with a rectangular grid of buildings. The vehicles moves with a waypoint mobility model. They start from the same position and travel in the same direction, along the main street. The first vehicle moves at 60 km/h and the second at 30 km/h. At a certain point, the first vehicle turns left while the second continues on the main street.

The “V2V-Highway” scenario simulates an highway environment in which the two vehicles travel on the same lane, in the same direction, and keep a safety distance of 20 m. They maintain a constant speed of 140 km/h.

The example generates the output file `example-output.txt`. Each row of the file is organized as follows:

```
Time[s] TxPosX[m] TxPosY[m] RxPosX[m] RxPosY[m] ChannelState SNR[dB] Pathloss[dB]
```

We also provide the bash script `three-gpp-v2v-channel-example.sh` which reads the output file and generates two figures:

1. `map.gif`, a GIF representing the simulation scenario and vehicle mobility;
2. `snr.png`, which represents the behavior of the SNR.

28.7 References

SPECTRUM MODULE

The Spectrum module aims at providing support for modeling the frequency-dependent aspects of communications in *ns-3*. The model was first introduced in [Baldo2009Spectrum], and has been enhanced and refined over the years.

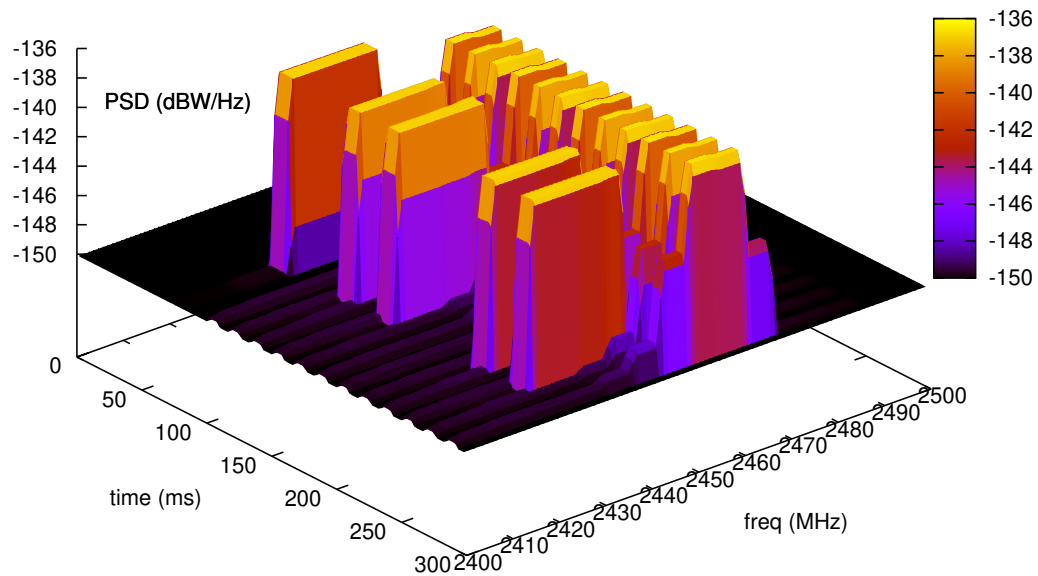


Fig. 29.1: Spectrogram produced by a spectrum analyzer in a scenario involving wifi signals interfered by a microwave oven, as simulated by the example `adhoc-aloah-ideal-phy-with-microwave-oven`.

29.1 Model Description

The module provides:

- a set of classes for modeling signals and
- a Channel/PHY interface based on a power spectral density signal representation that is technology-independent
- two technology-independent Channel implementations based on the Channel/PHY interface
- a set of basic PHY model implementations based on the Channel/PHY interface

The source code for the spectrum module is located at `src/spectrum`.

29.1.1 Design

Signal model

The signal model is implemented by the `SpectrumSignalParameters` class. This class provides the following information for a signal being transmitted/received by PHY devices:

- a reference to the transmitting PHY device
- a reference to the transmitting mobility model, which should be used in `StartRx` instead of retrieving it from PHY in order to wraparound models to work
- a reference to the antenna model used by the transmitting PHY device to transmit this signal
- the duration of the signal
- the Power Spectral Density (PSD) of the signal, which is assumed to be constant for the duration of the signal
- the frequency domain 3D spectrum channel matrix that is needed for MIMO computations when multiple streams are transmitted
- the 3D precoding matrix that is needed for MIMO computations when multiple streams are transmitted.

The PSD is represented as a set of discrete scalar values each corresponding to a certain subband in frequency. The set of frequency subbands to which the PSD refers to is defined by an instance of the `SpectrumModel` class. The PSD itself is implemented as an instance of the `SpectrumValue` class which contains a reference to the associated `SpectrumModel` class instance. The `SpectrumValue` class provides several arithmetic operators to allow to perform calculations with PSD instances. Additionally, the `SpectrumConverter` class provides means for the conversion of `SpectrumValue` instances from one `SpectrumModel` to another.

The frequency domain 3D channel matrix is needed in MIMO systems in which multiple transmit and receive antenna ports can exist, hence the PSD is multidimensional. The dimensions are: the number of receive antenna ports, the number of transmit antenna ports, and the number of subbands in frequency (or resource blocks).

The 3D precoding matrix is also needed in MIMO systems to be able to correctly perform the calculations of the received signal and interference. The dimensions are: the number of transmit antenna ports, the number of transmit streams, and the number of subbands (or resource blocks).

For a more formal mathematical description of the signal model just described, the reader is referred to [[Baldo2009Spectrum](#)].

The `SpectrumSignalParameters` class is meant to include only information that is valid for all signals; as such, it is not meant to be modified to add technology-specific information (such as type of modulation and coding schemes used, info on preambles and reference signals, etc). Instead, such information shall be put in a new class that inherits from `SpectrumSignalParameters` and extends it with any technology-specific information that is needed. This design is intended to model the fact that in the real world we have signals of different technologies being simultaneously transmitted and received over the air.

Channel/PHY interface

The spectrum Channel/PHY interface is defined by the base classes `SpectrumChannel` and `SpectrumPhy`. Their interaction simulates the transmission and reception of signals over the medium. The way this interaction works is depicted in *Sequence diagram showing the interaction between `SpectrumPhy` and `SpectrumChannel`*:

Spectrum Channel implementations

The module provides two `SpectrumChannel` implementations: `SingleModelSpectrumChannel` and `MultiModelSpectrumChannel`. They both provide this functionality:

- If a wraparound model is aggregated to the channel, automatically apply it to transmitting nodes.
- Propagation loss modeling, in three forms:
 - you can plug models based on `PropagationLossModel` on these channels. Only linear models (where the loss value does not depend on the transmission power) can be used. These models are single-frequency in the sense that the loss value is applied equally to all components of the power spectral density.
 - you can plug models based on `SpectrumPropagationLossModel` on these channels. These models can have frequency-dependent loss, i.e., a separate loss value is calculated and applied to each component of the power spectral density.
 - you can plug models based on `PhasedArraySpectrumPropagationLossModel` on these channels. These models can have frequency-dependent loss, i.e., a separate loss value is calculated and applied to each component of the power spectral density. Additionally, these models support the phased antenna array at the transmitter and the receiver, i.e., ns-3 antenna type `PhasedArrayModel`.
- Propagation delay modeling, by plugging a model based on `PropagationDelayModel`. The delay is independent of frequency and applied to the signal as a whole. Delay modeling is implemented by scheduling the `StartRx` event with a delay respect to the `StartTx` event.

`SingleModelSpectrumChannel` and `MultiModelSpectrumChannel` are quite similar, the main difference is that `MultiModelSpectrumChannel` allows to use different `SpectrumModel` instances with the same channel instance, by automatically taking care of the conversion of PSDs among the different models.

Example model implementations

The spectrum module provides some basic implementation of several components that are mainly intended as a proof-of-concept and as an example for building custom models with the spectrum module. Here is a brief list of the available implementations:

- `SpectrumModel300Khz300GhzLog` and `SpectrumModelIsm2400MhzRes1Mhz` are two example `SpectrumModel` implementations
- `HalfDuplexIdealPhy`: a basic PHY model using a gaussian interference model (implemented in `SpectrumInterference`) together with an error model based on Shannon capacity (described in [\[Baldo2009Spectrum\]](#) and implemented in `SpectrumErrorModel`). This PHY uses the `GenericPhy` interface. Its additional custom signal parameters are defined in `HalfDuplexIdealPhySignalParameters`.
- `WifiSpectrumValueHelper` is an helper object that makes it easy to create `SpectrumValues` representing PSDs and RF filters for the wifi technology.
- `AlohaNoackNetDevice`: a minimal `NetDevice` that allows to send packets over `HalfDuplexIdealPhy` (or other PHY model based on the `GenericPhy` interface). It provides `NetDevice` trace sources `MacTx`, `MacTxDrop`, `MacRx`, and `MacPromiscRx`.
- `SpectrumAnalyzer`, `WaveformGenerator` and `MicrowaveOven` are examples of PHY models other than communication devices - the names should be self-explaining.

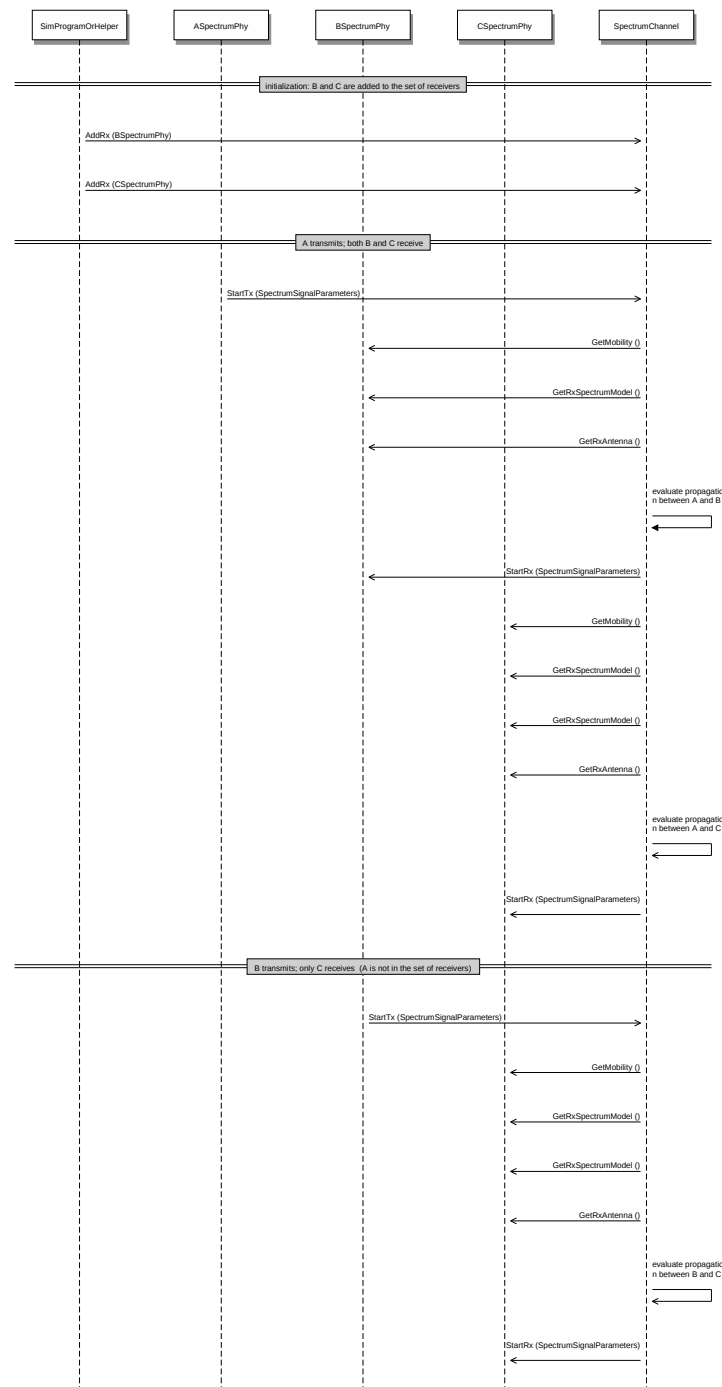


Fig. 29.2: Sequence diagram showing the interaction between SpectrumPhy and SpectrumChannel

Sionna RT channel model

Overview

The Sionna RT channel model as alternative to stochastic 3GPP channel models (e.g., TR 38.901) with a deterministic, geometry-based ray tracer. At each channel update event, ns-3 calls Sionna RT in-process through pybind11. The resulting per-path delays, Doppler shifts, and angles-of-arrival/departure are converted directly into ns-3's internal *ChannelMatrix* format.

Key properties: - In-process communication — no sockets, RPC calls, or temporary files - Compatible with all modules that use *MatrixBasedChannelModel* (5G-LENA, LTE, etc.) - GPU-accelerated (NVIDIA CUDA) with CPU fallback via LLVM/Dr.Jit - Scenes defined in Mitsuba 3 XML format (exportable from Blender)

The spectrum module also exposes an integration with the sionna.rt ray-tracing library through two classes: *SionnaRtChannelModel* and *SionnaRtSpectrumPropagationLossModel*.

SionnaRtChannelModel is a *MatrixBasedChannelModel* implementation that creates frequency-selective, phase-coherent MIMO channel matrices from Sionna RT scenes. It loads either built-in Sionna RT scenarios or custom XML scene files, runs the Sionna RT path solver, extracts impulse responses, delays, Doppler, and angular metadata, and caches channel matrices per link.

Features of *SionnaRtChannelModel* include:

- scene selection via the *Scenario* attribute, supporting both built-in scenario names and XML scene paths;
- a configurable operating frequency through the *Frequency* attribute;
- an update mechanism driven by *UpdatePeriod* to refresh cached channel matrices when the channel coherence time is exceeded;
- path-count control using *MaxNumberOfPaths* and recursive path solver depth reduction;
- optional path delay normalization via *NormalizeDelays*;
- scene image rendering support through *IsImageRenderedEnabled*, *OutputImageDirectory* and *OutputImageName*;
- shape merging for scene loading via *IsMergeShapeEnabled*.

The model also exposes an *RtPathSolverConfig* structure with parameters such as *maxDepth*, *los*, *specularReflection*, *diffuseReflection*, *refraction*, *diffraction*, *edgeDiffraction*, *syntheticArray* and *seed* which is decoupled from ns-3 *RngRun/RngSeed* for fine-grained control over the ray-tracing path solver behavior.

SionnaRtSpectrumPropagationLossModel is a *PhasedArraySpectrumPropagationLossModel* implementation that uses a *SionnaRtChannelModel* instance to obtain the underlying channel matrix and path metadata. It computes the received power spectral density by applying 3GPP-style fast fading and beamforming gain to the channel matrix, accounting for path delays and Doppler shifts.

Key behaviors of *SionnaRtSpectrumPropagationLossModel*:

- it is intended for use with *SingleModelSpectrumChannel* or *MultiModelSpectrumChannel*;
- it accepts a *MatrixBasedChannelModel* via *SetChannelModel()*;
- it can forward ray-tracing solver settings to the attached channel model through *SetRtPathSolverConfig()*;
- it caches and reuses long-term beamforming components per node pair and recomputes them only when the channel realization or beamforming vectors change;
- it is suitable for phased-array MIMO simulations in which frequency-dependent channel effects, beamforming, and Doppler are required.

The example program `src/spectrum/examples/sionna-rt-channel-example.cc` illustrates how to configure the Sionna RT channel model, attach the spectrum propagation loss model to a channel, and compute average SNR over a simple two-node scenario.

29.1.2 References

29.2 Usage

The main use case of the spectrum model is for developers who want to develop a new model for the PHY layer of some wireless technology to be used within ns-3. Here are some notes on how the spectrum module is expected to be used.

- `SpectrumPhy` and `SpectrumChannel` are abstract base classes. Real code will use classes that inherit from these classes.
- If you are implementing a new model for some wireless technology of your interest, and want to use the spectrum module, you'll typically create your own module and make it depend on the spectrum module. Then you typically have to implement:
 - a child class of `SpectrumModel` which defines the (sets of) frequency subbands used by the considered wireless technology. **Note:** instances of `SpectrumModel` are typically statically allocated, in order to allow several `SpectrumValue` instances to reference the same `SpectrumModel` instance.
 - a child class of `SpectrumPhy` which will handle transmission and reception of signals (including, if appropriate, interference and error modeling).
 - a child class of `SpectrumSignalParameters` which will contain all the information needed to model the signals for the wireless technology being considered that is not already provided by the base `SpectrumSignalParameters` class. Examples of such information are the type of modulation and coding schemes used, the PHY preamble format, info on the pilot/reference signals, etc.
- The available `SpectrumChannel` implementations (`SingleModelSpectrumChannel` and `MultiModelSpectrumChannel`, are quite generic. Chances are you can use them as-is. Whether you prefer one or the other it is just a matter of whether you will have a single `SpectrumModel` or multiple ones in your simulations.
- Typically, there will be a single `SpectrumChannel` instance to which several `SpectrumPhy` instances are plugged. The rule of thumb is that all PHYs that are interfering with each other shall be plugged on the same channel. Multiple `SpectrumChannel` instances are expected to be used mainly when simulating completely orthogonal channels; for example, when simulating the uplink and downlink of a Frequency Division Duplex system, it is a good choice to use two `SpectrumChannel` instances in order to reduce computational complexity.
- Different types of `SpectrumPhy` (i.e., instances of different child classes) can be plugged on the same `SpectrumChannel` instance. This is one of the main features of the spectrum module, to support inter-technology interference. For example, if you implement a `WifiSpectrumPhy` and a `BluetoothSpectrumPhy`, and plug both on a `SpectrumChannel`, then you'll be able to simulate interference between wifi and bluetooth and vice versa.
- Different child classes of `SpectrumSignalParameters` can coexist in the same simulation, and be transmitted over the same channel object. Again, this is part of the support for inter-technology interference. A PHY device model is expected to use the `DynamicCast<>` operator to determine if a signal is of a certain type it can attempt to receive. If not, the signal is normally expected to be considered as interference.
- Many propagation loss and delay models can be added to these channels. The base class `SpectrumChannel` provides an `AssignStreams()` method to allow the deterministic configuration of random variable stream numbers with a single API call. See the ns-3 Manual chapter on random variables for more information.

29.2.1 Helpers

The helpers provided in `src/spectrum/helpers` are mainly intended for the example implementations described in *Example model implementations*. If you are developing your custom model based on the spectrum framework, you will probably prefer to define your own helpers.

29.2.2 Attributes

- Both `SingleModelSpectrumChannel` and `MultiModelSpectrumChannel` have an attribute `MaxLossDb` which can use to avoid propagating signals affected by very high propagation loss. You can use this to reduce the complexity of interference calculations. Just be careful to choose a value that does not make the interference calculations inaccurate.
- The example implementations described in *Example model implementations* also have several attributes.

29.2.3 Output

- Both `SingleModelSpectrumChannel` and `MultiModelSpectrumChannel` provide a trace source called `PathLoss` which is fired whenever a new path loss value is calculated. **Note:** only single-frequency path loss is accounted for, see the attribute description.
- The example implementations described in *Example model implementations* also provide some trace sources.
- The helper class `SpectrumAnalyzerHelper` can be conveniently used to generate an output text file containing the spectrogram produced by a `SpectrumAnalyzer` instance. The format is designed to be easily plotted with `gnuplot`. For example, if you run the example `adhoc-aloha-ideal-phy-with-microwave-oven` you will get an output file called `spectrum-analyzer-output-3-0.tr`. From this output file, you can generate a figure similar to *Spectrogram produced by a spectrum analyzer in a scenario involving wifi signals interfered by a microwave oven, as simulated by the example adhoc-aloha-ideal-phy-with-microwave-oven*. by executing the following `gnuplot` commands:

```
unset surface
set pm3d at s
set palette
set key off
set view 50,50
set xlabel "time (ms)"
set ylabel "freq (MHz)"
set zlabel "PSD (dBW/Hz)" offset 15,0,0
splot "./spectrum-analyzer-output-3-0.tr" using ($1*1000.0):($2/1e6):(10*log10($3))
```

29.2.4 Examples

The example programs in `src/spectrum/examples/` allow to see the example implementations described in *Example model implementations* in action.

29.2.5 Troubleshooting

- **Disclaimer on inter-technology interference:** the spectrum model makes it very easy to implement an inter-technology interference model, but this does not guarantee that the resulting model is accurate. For example, the gaussian interference model implemented in the `SpectrumInterference` class can be used to calculate inter-technology interference, however the results might not be valid in some scenarios, depending on the actual waveforms involved, the number of interferers, etc. Moreover, it is very important to use error models that are consistent with the interference model. The responsibility of ensuring that the models being used are correct is left to the user.

29.3 Testing

In this section we describe the test suites that are provided within the spectrum module.

29.3.1 SpectrumValue test

The test suite `spectrum-value` verifies the correct functionality of the arithmetic operators implemented by the `SpectrumValue` class. Each test case corresponds to a different operator. The test passes if the result provided by the operator implementation is equal to the reference values which were calculated offline by hand. Equality is verified within a tolerance of 10^{-6} which is to account for numerical errors.

29.3.2 SpectrumConverter test

The test suite `spectrum-converter` verifies the correct functionality of the `SpectrumConverter` class. Different test cases correspond to the conversion of different `SpectrumValue` instances to different `SpectrumModel` instances. Each test passes if the `SpectrumValue` instance resulting from the conversion is equal to the reference values which were calculated offline by hand. Equality is verified within a tolerance of 10^{-6} which is to account for numerical errors.

Describe how the model has been tested/validated. What tests run in the test suite? How much API and code is covered by the tests? Again, references to outside published work may help here.

29.3.3 Interference test

The test suite `spectrum-interference` verifies the correct functionality of the `SpectrumInterference` and `ShannonSpectrumErrorModel` in a scenario involving four signals (an intended signal plus three interferers). Different test cases are created corresponding to different PSDs of the intended signal and different amount of transmitted bytes. The test passes if the output of the error model (successful or failed) coincides with the expected one which was determined offline by manually calculating the achievable rate using Shannon's formula.

29.3.4 IdealPhy test

The test verifies that `AlohaNoackNetDevice` and `HalfDuplexIdealPhy` work properly when installed in a node. The test recreates a scenario with two nodes (a TX and a RX) affected by a path loss such that a certain SNR is obtained. The TX node transmits with a pre-determined PHY rate and with an application layer rate which is larger than the PHY rate, so as to saturate the channel. `PacketSocket` is used in order to avoid protocol overhead. Different test cases correspond to different PHY rate and SNR values. For each test case, we calculated offline (using Shannon's formula) whether the PHY rate is achievable or not. Each test case passes if the following conditions are satisfied:

- if the PHY rate is achievable, the application throughput shall be within 1% of the PHY rate;

- if the PHY rate is not achievable, the application throughput shall be zero.

29.4 Additional Models

29.4.1 TV Transmitter Model

A TV Transmitter model is implemented by the `TvSpectrumTransmitter` class. This model enables transmission of realistic TV signals to be simulated and can be used for interference modeling. It provides a customizable power spectral density (PSD) model, with configurable attributes including the type of modulation (with models for analog, 8-VSB, and COFDM), signal bandwidth, power spectral density level, frequency, and transmission duration. A helper class, `TvSpectrumTransmitterHelper`, is also provided to assist users in setting up simulations.

Main Model Class

The main TV Transmitter model class, `TvSpectrumTransmitter`, provides a user-configurable PSD model that can be transmitted on the `SpectrumChannel`. It inherits from `SpectrumPhy` and is comprised of attributes and methods to create and transmit the signal on the channel.

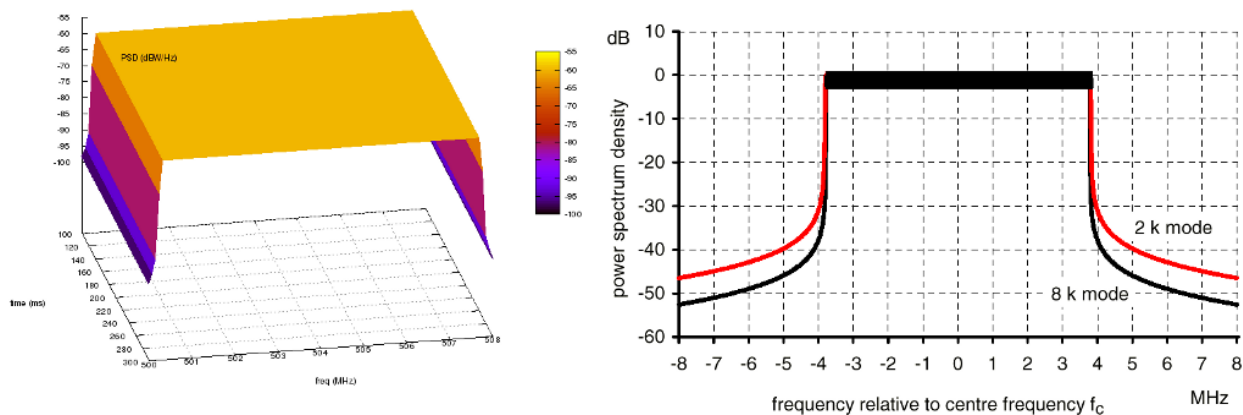


Fig. 29.3: 8K COFDM signal spectrum generated from `TvSpectrumTransmitter` (Left) and theoretical COFDM signal spectrum [[KoppCOFDM](#)] (Right)

One of the user-configurable attributes is the type of modulation for the TV transmitter to use. The options are 8-VSB (Eight-Level Vestigial Sideband Modulation) which is notably used in the North America ATSC digital television standard, COFDM (Coded Orthogonal Frequency Division Multiplexing) which is notably used in the DVB-T and ISDB-T digital television standards adopted by various countries around the world, and analog modulation which is a legacy technology but is still being used by some countries today. To accomplish realistic PSD models for these modulation types, the signals' PSDs were approximated from real standards and developed into models that are scalable by frequency and power. The COFDM PSD is approximated from Figure 12 (8k mode) of [[KoppCOFDM](#)], the 8-VSB PSD is approximated from Figure 3 of [[Baron8VSB](#)], and the analog PSD is approximated from Figure 4 of [[QualcommAnalog](#)]. Note that the analog model is approximated from the NTSC standard, but other analog modulation standards such as PAL have similar signals. The approximated COFDM PSD model is in 8K mode. The other configurable attributes are the start frequency, signal/channel bandwidth, base PSD, antenna type, starting time, and transmit duration.

`TvSpectrumTransmitter` uses `IsotropicAntennaModel` as its antenna model by default, but any model that inherits from `AntennaModel` is selectable, so directional antenna models can also be used. The propagation loss models used in simulation are configured in the `SpectrumChannel` that the user chooses to use. Terrain and spherical Earth/horizon effects may be supported in future ns-3 propagation loss models.

After the attributes are set, along with the `SpectrumChannel`, `MobilityModel`, and node locations, the PSD of the TV transmitter signal can be created and transmitted on the channel.

Helper Class

The helper class, `TvSpectrumTransmitterHelper`, consists of features to assist users in setting up TV transmitters for their simulations. Functionality is also provided to easily simulate real-world scenarios.

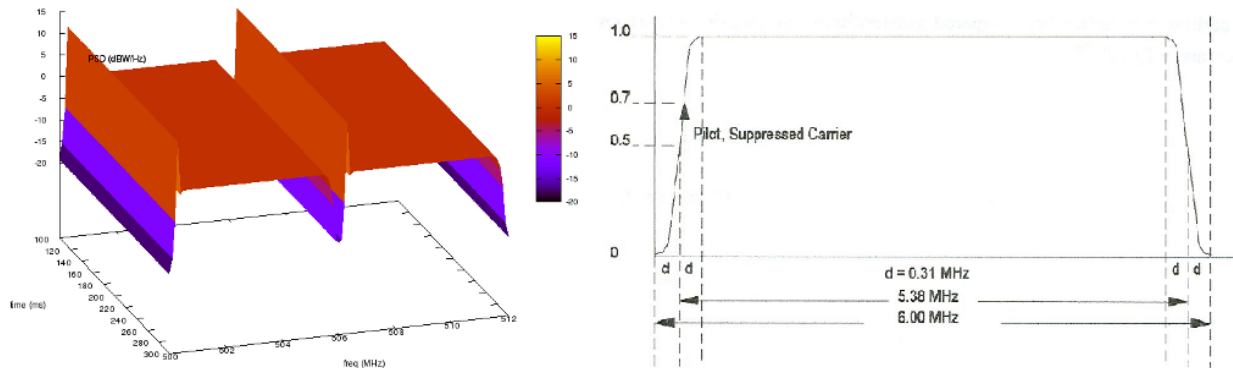


Fig. 29.4: North America ATSC channel 19 & 20 signals generated using `TvSpectrumTransmitterHelper` (Left) and theoretical 8-VSB signal [Baron8VSB] (Right). Note that the theoretical signal is not shown in dB while the ns-3 generated signals are.

Using this helper class, users can easily set up TV transmitters right after configuring attributes. Multiple transmitters can be created at a time. Also included are real characteristics of specific geographic regions that can be used to run realistic simulations. The regions currently included are North America, Europe, and Japan. The frequencies and bandwidth of each TV channel for each these regions are provided.

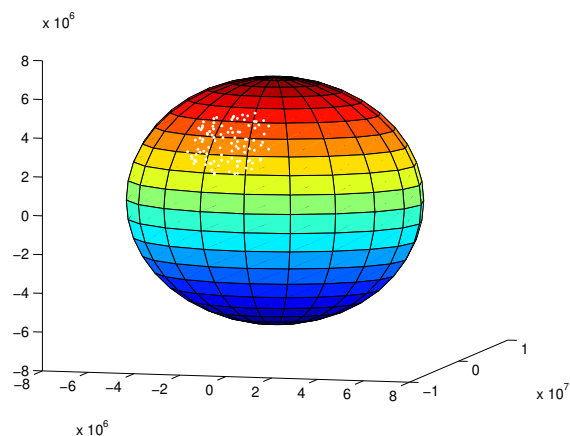


Fig. 29.5: Plot from MATLAB implementation of `CreateRegionalTvTransmitters` method in `TvSpectrumTransmitterHelper`. Shows 100 random points on Earth's surface (with altitude 0) corresponding to TV transmitter locations within a 2000 km radius of 35° latitude and -100° longitude.

A method (`CreateRegionalTvTransmitters`) is provided that enables users to randomly generate multiple TV transmitters from a specified region with a given density within a chosen radius around a point on Earth's surface. The region, which determines the channel frequencies of the generated TV transmitters, can be specified to be one of the three provided, while the density determines the amount of transmitters generated. The TV transmitters' antenna heights

(altitude) above Earth's surface can also be randomly generated to be within a given maximum altitude. This method models Earth as a perfect sphere, and generated location points are referenced accordingly in Earth-Centered Earth-Fixed Cartesian coordinates. Note that bodies of water on Earth are not considered in location point generation—TV transmitters can be generated anywhere on Earth around the origin point within the chosen maximum radius.

Examples

Two example simulations are provided that demonstrate the functionality of the TV transmitter model. `tv-trans-example` simulates two 8-VSB TV transmitters with adjacent channel frequencies. `tv-trans-regional-example` simulates randomly generated COFDM TV transmitters (modeling the DVB-T standard) located around the Paris, France area with channel frequencies and bandwidths corresponding to the European television channel allocations.

Testing

The `tv-spectrum-transmitter` test suite verifies the accuracy of the spectrum/PSD model in `TvSpectrumTransmitter` by testing if the maximum power spectral density, start frequency, and end frequency comply with expected values for various test cases.

The `tv-helper-distribution` test suite verifies the functionality of the method in `TvSpectrumTransmitterHelper` that generates a random number of TV transmitters based on the given density (low, medium, or high) and maximum number of TV channels. It verifies that the number of TV transmitters generated does not exceed the expected bounds.

The `CreateRegionalTvTransmitters` method in `TvSpectrumTransmitterHelper` described in *Helper Class* uses two methods from the `GeographicPositions` class in the Mobility module to generate the random Cartesian points on or above earth's surface around an origin point which correspond to TV transmitter positions. The first method converts Earth geographic coordinates to Earth-Centered Earth-Fixed (ECEF) Cartesian coordinates, and is tested in the `geo-to-cartesian` test suite by comparing (with 10 meter tolerance) its output with the output of the geographic to ECEF conversion function [[MatlabGeo](#)] of the MATLAB Mapping Toolbox for numerous test cases. The other used method generates random ECEF Cartesian points around the given geographic origin point, and is tested in the `rand-cart-around-geo` test suite by verifying that the generated points do not exceed the given maximum distance radius from the origin point.

29.4.2 3GPP TR 38.901 fast fading model

The framework described by TR 38.901 [[TR38901](#)] is a 3D statistical Spatial Channel Model supporting different propagation environments (e.g., urban, rural, indoor), multi-antenna operations and the modeling of wireless channels between 0.5 and 100 GHz. The overall channel is represented by the matrix $H(t, \tau)$, in which each entry $H_{u,s}(t, \tau)$ corresponds to the impulse response of the channel between the s -th element of the transmitting antenna and the u -th element of the receiving antenna. $H_{u,s}(t, \tau)$ is generated by the superposition of N different multi-path components, called clusters, each of which composed of M different rays. The channel matrix generation procedure accounts for large and small scale propagation phenomena. The classes `ThreeGppSpectrumPropagationLossModel` and `ThreeGppChannelModel` included in the spectrum module take care of the generation of the channel coefficients and the computation of the frequency-dependent propagation loss.

Implementation

Our implementation is described in [Zugno]. It is based on the model described in [Zhang], but the code has been refactored, extended, and aligned to TR 38.901 [TR38901]. The fundamental assumption behind this model is the channel reciprocity, i.e., the impulse response of the channel between node a and node b is the same as between node b and node a. To deal with the equivalence of the channel between a and b, no matter who is the transmitter and who is the receiver, the model considers the pair of nodes to be composed by one “s” and one “u” node. The channel matrix, as well as other parameters, are saved and used under the assumption that, within a pair, the definition of the “s” and “u” node will always be the same. For more details, please have a look at the documentation of the classes `ThreeGppChannelModel` and `ThreeGppSpectrumPropagationLossModel`.

Note:

- Since ns-3.47, the model includes an update technique aligned with the 3GPP TR 38.901 spatial-consistency procedure (see [TR38901], Sec. 7.6.3), based on Procedure A (Sec. 7.6.3.2). The channel-parameter evolution follows the Procedure A update equations to preserve correlation across consecutive channel evaluations, while the criteria for *when* to apply an update are adapted for ns-3’s discrete-event operation (i.e., updates are triggered when the channel is evaluated and the configured `UpdatePeriod` has elapsed; if the maximum distance the BS and/or UT traveled between evaluations exceeds the 1 m step-size constraint, the model falls back to re-generating channel parameters and thus the channel matrix). Drop-based spatial consistency across multiple initial locations (Sec. 7.6.3.1) and Procedure B are not implemented.
- Issue regarding the blockage model: according to 3GPP TR 38.901 v15.0.0 (2018-06) section 7.6.4.1, the blocking region for self-blocking is provided in LCS. However, here, `clusterAOA` and `clusterZOA` are in GCS and blocking check is performed for self-blocking similar to non-self blocking, that is in GCS. One would expect the angles to be transposed to LCS before checking self-blockage.
- No error model is provided in this module; a link-to-system campaign may be needed to incorporate it in existing modules.

ThreeGppSpectrumPropagationLossModel

The class `ThreeGppSpectrumPropagationLossModel` implements the `PhasedArraySpectrumPropagationLossModel` interface and enables the modeling of frequency dependent propagation phenomena while taking into account the specific pair of the phased antenna array at the transmitter and the receiver. The main method is `DoCalcRxPowerSpectralDensity`, which takes as input the `SpectrumSignalParameters` structure, which contains, among others, the power spectral density (PSD) of the transmitted signal, and the precoding matrix of the transmitter which is needed for MIMO computations. Other input parameters are the mobility models of the transmitting and receiving node, and the phased antenna array of the transmitting and receiving node. `DoCalcRxPowerSpectralDensity` computes the PSD of the received signal (used in case of single stream transmission), and the frequency domain 3D channel matrix between receive and transmit antenna ports (needed for MIMO computations when multiple streams are transmitted). Finally, it returns the `SpectrumSignalParameters` which contains the received PSD and the frequency domain 3D spectrum channel matrix.

Procedure used to compute the PSD of the received signal and the frequency domain 3D spectrum channel matrix:

1. Retrieve the beamforming vectors To account for the beamforming, `ThreeGppSpectrumPropagationLossModel` has to retrieve the beamforming vectors of the transmitting and receiving antennas. The method `DoCalcRxPowerSpectralDensity` uses the antenna objects that are passed as parameters for both the transmitting and receiving devices, and calls the method `GetBeamformingVector` to retrieve the beamforming vectors of these antenna objects.
2. Retrieve the channel matrix and the channel params The `ThreeGppSpectrumPropagationLossModel` relies on the `ThreeGppChannelModel` class to obtain the channel matrix and channel parameters. In particular, it makes use of the method `GetChannel`, which returns a `ChannelMatrix` object containing the channel matrix, the generation time, the node pair, and the phased antenna array pair among which is created this channel matrix. Additionally, it makes use of the method `GetParams` which returns a `ChannelParams` object containing the channel parameters. Notice that the

channel information is split into these two structures (ChannelMatrix and ChannelParams) to support multiple collocated phased antenna arrays at TX/RX node. ChannelParams (also its specialization ThreeGppChannelParams structure) contains parameters which are common for all the channels among the same RX/TX node pair, while ChannelMatrix contains the channel matrix for the specific pair of the phased antenna arrays of TX/RX nodes. For example, if the TX and the RX node have multiple collocated antenna arrays, then there will be multiple channel matrices among this pair of nodes for different pairs of antenna arrays of the TX and the RX node. These channel matrices that are among the same pair of nodes have common channel parameters, i.e., they share the same channel condition, cluster powers, cluster delays, AoD, AoA, ZoD, ZoA, K_factor, delay spread, etc. On the other hand, each pair of TX and RX antenna arrays has a specific channel matrix and fading, which depends on the actual antenna element positions and field patterns of each pair of antenna array subpartitions. The ThreeGppChannelModel instance is automatically created in the ThreeGppSpectrumPropagationLossModel constructor and it can be configured by using the method SetChannelModelAttribute.

Notice that in MultiModelSpectrumChannel it is checked whether the TX/RX SpectrumPhy instances belong to different TX/RX nodes. This is needed to avoid pathloss models calculations among the phased antenna arrays of the same node, because there are no models yet in ns-3 that support the calculation of this kind of interference.

4. Compute the long term component The method GetLongTerm returns the long term component obtained by multiplying the channel matrix and the beamforming vectors. The function CalculateLongTermComponent calculates the long term component per RX and TX port pair. Finally, GetLongTerm returns a 3D long term channel matrix whose dimensions are the number of the receive antenna ports, the number transmit antenna ports, and the number of clusters. When multiple ports are being configured note that the sub-array partition model is adopted for TXRU virtualization, as described in Section 5.2.2 of 3GPP TR 36.897 [TR36897], and so equal beam weights are used for all the ports. Support of the full-connection model for TXRU virtualization would need extensions. To reduce the computational load, the long term components associated to the different channels are stored in the m_longTermMap and recomputed only if the associated channel matrix is updated or if the transmitting and/or receiving beamforming vectors have changed. Given the channel reciprocity assumption, for each node pair a single long term component is saved in the map.

5. Apply the small scale fading, calculate the channel gain, generate the frequency domain 3D spectrum channel matrix, and finally compute the received PSD The method CalcBeamformingGain computes the channel gain in each sub-band and applies it to the PSD of the transmitted signal to obtain the received PSD. To compute the sub-band gain, it accounts for the Doppler phenomenon and the time dispersion effect on each cluster. In order to reduce the computational load, the Doppler component of each cluster is computed considering only the central ray. Also, as specified [here](#), it is possible to account for the effect of environmental scattering following the model described in Sec. 6.2.3 of 3GPP TR 37.885. This is done by deviating the Doppler frequency by a random value, whose distribution depends on the parameter v_{scatt} . The value of v_{scatt} can be configured using the attribute “vScatt” (by default it is set to 0, so that the scattering effect is not considered). Function GenSpectrumChannelMatrix generates the received PSD for each pair of the transmit and receive antenna ports. It creates a frequency domain 3D spectrum channel matrix whose dimensions are the number of receive antenna ports, the number of transmit antenna ports, and the number of resource blocks. Finally, the frequency domain 3D spectrum channel matrix is used to obtain the received PSD. In case of multiple ports at the transmitter the PSD is calculated by summing per each RB the real parts of the diagonal elements of the $(H^*P)^H * (H^*P)$, where H is the frequency domain spectrum channel matrix and P is the precoding matrix.

ThreeGppChannelModel

The class ThreeGppChannelModel implements the channel matrix generation procedure described in 3GPP TR 38.901 [TR38901] and an update technique aligned with 3GPP TR 38.901 spatial-consistency feature (Procedure A), with ns-3-specific update triggering criteria suited to discrete-event simulation.

The main method is GetChannel, which takes as input the mobility models of the transmitter and receiver nodes, the associated antenna objects, and returns a ChannelMatrix object containing:

- the channel matrix of size $U \times S \times N$, where U is the number of receiving antenna elements, S is the number of transmitting antenna elements and N is the number of clusters
- the clusters delays, as an array of size N

- the clusters arrival and departure angles, as a 2D array in which each row corresponds to a direction (AOA, ZOA, AOD, ZOD) and each column corresponds to a different cluster
- a time stamp indicating the time at which the channel matrix was generated
- the node IDs
- other channel parameters

The ChannelMatrix objects are cached per link in the map `m_channelMatrixMap`. A **new channel realization** (i.e., a new ChannelMatrix generated from newly generated channel parameters) is created when:

- the **channel condition** for the link changes (e.g., LOS/NLOS and/or O2I transitions), as reported by the configured channel-condition model; or
- the **antenna-array configuration** changes (e.g., number of antenna elements/ports), since this changes the required matrix dimensions.
- spatial-consistency updates are enabled (UpdatePeriod is nonzero) and an update attempt observes a maximum endpoint displacement > 1 m (exceeding the Procedure A step-size constraint), the model re-generates channel parameters (and thus the channel matrix).

If spatial-consistency updates are enabled and the 1 m step-size constraint is satisfied, the cached channel parameters are **updated** (Procedure A), and the ChannelMatrix is recomputed from the updated parameters (without drawing a new independent realization).

The channel condition is provided by the configured channel-condition model, which may be time-varying (e.g., stochastic blockage) and may be evaluated/updated according to its own internal logic (potentially with its own periodicity), independent of the small-scale fading evolution modeled in the ThreeGppChannelModel. The channel condition may change regardless of whether the nodes are static or mobile.

Antenna-parameter changes are expected to be rare; one example is NR initial access/attachment, where a simplified receiver configuration may be assumed during attachment and then restored to the user-configured antenna array, triggering a matrix regeneration.

The ChannelMatrix itself does **not** depend on beamforming. Beamforming vectors (and the precoding matrix) are applied later in `ThreeGppSpectrumPropagationLossModel::DoCalcRxPowerSpectralDensity` when computing the long-term component and the received PSD (see the procedure in `ThreeGppSpectrumPropagationLossModel` above). Beamforming weights may change at each reception/transmission without forcing regeneration of the cached ChannelMatrix.

The propagation scenario and the operating frequency can be configured through the Scenario and Frequency attributes, respectively. Spatial-consistency evolution aligned with **3GPP TR 38.901 Procedure A update equations** is enabled when the UpdatePeriod attribute is set to a **non-zero** value (see below).

Spatial consistency procedure: The spatial consistency update technique is aligned with 3GPP TR 38.901 Sec. 7.6.3.2 (**Procedure A**). Procedure A aims to preserve spatial correlation of cluster-specific random terms across consecutive channel evaluations as nodes move (e.g., cluster delays from Fig. 7.5-1, Step 5; the per-cluster shadowing term used in the cluster-power computation; and the random offsets/signs used for AOD/AOA/ZOD/ZOA generation in Step 7).

In ns-3, the **channel-parameter evolution** follows the Procedure A update equations, while the **update-triggering criteria** are adapted to a discrete-event simulation environment. In TR 38.901, Procedure A is described for updates at about 1 m granularity, without prescribing a discrete evaluation schedule, and there is no specified time or distance at which a new channel realization is drawn; in ns-3, updates are attempted when the channel is evaluated and the configured UpdatePeriod has elapsed, and the model enforces the 1 m step-size constraint (falling back to re-generation of channel parameters when it is exceeded).

With Procedure A, large-scale parameters evolve as a first-order, spatially correlated stochastic process. Continuity is obtained by evolving a cached channel state over time/distance; no position-indexed channel field is defined. As a consequence, the channel is trajectory-dependent, and revisiting the same location does not necessarily reproduce the exact same channel realization. This behavior is consistent with the stochastic modeling approach defined in TR 38.901 [TR38901]. Users requiring strict position-based repeatability (including under LOS/NLOS state changes driven by the

channel-condition model) would have to implement a map/field-based approach (e.g., spatially correlated LSP maps in WINNER-derived frameworks such as QuaDRiGa) [WIN2D112] [QUADRIGA] or an explicitly position-seeded regeneration policy.

Drop-based spatial consistency across multiple initial locations (TR 38.901 Sec. 7.6.3.1) and Procedure B (TR 38.901 Sec. 7.6.3.2) are not implemented in this model.

The initial channel realization for a link is generated using the standard 3GPP cluster/ray generation procedure (Fig. 7.5-1). Spatial consistency is then introduced through subsequent Procedure A updates of the same link. During each update attempt, Procedure A updates the channel parameters in a cluster-wise manner. First, cluster delays are updated according to Eq. 7.6-9 in [TR38901]. Second, cluster powers are updated using the updated cluster delays; during this step, the per-cluster shadowing term is evolved (not re-generated) based on the correlation distance of the configured scenario. Third, cluster departure and arrival angles are updated using the UT/BS motion information, as defined in Eqs. 7.6-10b and 7.6-10c and the subsequent angle-update equations in Eqs. 7.6-11 to 7.6-14.

In ns-3, the update period is defined by `UpdatePeriod` and is analogous to t_{Δ} in TR 38.901 Sec. 7.6.3.2. When the channel is evaluated and `UpdatePeriod` has elapsed since the last parameter generation/update, the model proceeds as follows:

- If the maximum endpoint displacement since the last parameter generation/update is **within 1 m**, the model applies **Procedure A updates** to obtain a spatially consistent channel evolution.
- If the maximum endpoint displacement is **greater than 1 m**, the model **re-generates** the channel parameters (and thus the channel matrix) instead of applying Procedure A for that update step.

The small update distance is consistent with the intent of Procedure A to evolve small-scale channel parameters smoothly as the terminal moves; larger displacements between evaluations are handled by re-generation rather than attempting to extrapolate multiple intermediate updates.

Note on the 1 m update distance vs. correlation distances: The 1 m rule is a step-size constraint for applying Procedure A updates (i.e., the model updates the channel in small spatial increments). It does not imply that the channel is only correlated over 1 m. The scenario correlation distances provided in TR 38.901 Table 7.6.3.1-2 parameterize the *rate* at which cluster-level random variables (e.g., per-cluster shadowing) decorrelate with displacement across successive updates, via an exponential correlation function (TR 38.901, Eq. 7.4-5) of the form $R(\Delta x) = \exp(-\Delta x/d_{cor})$. Consequently, when Procedure A is applied repeatedly over many small steps, variables can remain correlated over tens of meters when d_{cor} is large, even though each individual update step is limited to 1 m.

Design note (displacement > 1 m and update triggering): TR 38.901 recommends choosing t_{Δ} such that the displacement between successive updates remains within 1 m (often expressed as $v \cdot t_{\Delta} < 1$ m). In link-/physical-level channel simulators, the channel is commonly evolved at a fixed update distance/period along a user trajectory (e.g., NYUSIM) [NYUSIM]. In a system-level simulator, however, forcing periodic channel updates even when the link is not used (no traffic, no transmissions) may be computationally expensive in large-scale simulations.

Since ns-3 is a discrete-event simulator, channel evaluations may occur at irregular times (e.g., sparse or bursty traffic), and an update attempt may observe a displacement larger than 1 m. In that situation, several behaviors are possible: (i) keep the previous channel state without updating, (ii) re-generate a new realization, (iii) approximate the motion by splitting the interval into smaller steps and applying multiple updates, or (iv) force periodic channel updates independent of traffic (i.e., drive the evolution from a dedicated timer and update internal state even when the channel is not queried).

Option (i) avoids discontinuities but may keep the channel unchanged over large displacements. Option (ii) is conservative and simple, but introduces a new realization when the update-distance constraint is exceeded. Option (iii) would require assumptions about the intermediate trajectory (only start/end positions are known) and increases computational cost. Option (iv) may also be computationally expensive in large-scale simulations, since it updates channels even when the channel is not evaluated.

ns-3 chooses option (ii) as a fallback: if the displacement exceeds 1 m at an update instant, the channel parameters are re-generated.

How to choose ``UpdatePeriod``. To obtain Procedure A updates (rather than frequent re-generation), `UpdatePeriod` should be selected so that the expected maximum endpoint displacement stays below 1 m between updates, i.e.,

$\text{UpdatePeriod} \lesssim 1 \text{ m} / v_{\text{end,max}}$.

Example: if the maximum expected endpoint speed is 5 m/s, choosing $\text{UpdatePeriod} \leq 0.2 \text{ s}$ keeps the maximum endpoint displacement below 1 m.

Important: UpdatePeriod sets a **minimum interval** between successive channel updates; the channel state is updated only when the channel is evaluated, so the effective update instants are traffic-driven (e.g., receptions/transmissions or other events that require channel evaluation). For mobile scenarios, users should ensure that the channel is evaluated at least as often as UpdatePeriod (e.g., by periodic transmissions/receptions, control signaling, or application traffic). Otherwise, spatial-consistency updates may be skipped and the model may fall back to re-generation after larger displacements.

If the channel is not evaluated for longer than UpdatePeriod , then the next evaluation triggers an update attempt based on the current displacement since the last generation/update (Procedure A if within 1 m, otherwise re-generation).

The following rule-of-thumb values illustrate a recommended upper bound for UpdatePeriod to keep the maximum endpoint displacement within 1 m. Users should also ensure that the channel is evaluated at least that often (e.g., via periodic signaling or traffic), otherwise updates may be skipped.

Speed (km/h)	Speed (m/s)	Max UpdatePeriod (ms)	Recommended channel-evaluation / signaling period (ms)
3	0.833	1200	≤ 1200
30	8.33	120	≤ 120
60	16.67	60	≤ 60

For links where both endpoints move (e.g., V2V), two different “displacements” are relevant:

- The **maximum endpoint displacement** (max motion of either endpoint since the last generation/update) is used to enforce the **1 m step-size constraint** for applying Procedure A.
- The **relative (geometry) displacement** (change in the Tx–Rx relative position vector) is used as the displacement input for **correlation updates of cluster-level random terms** (e.g., the per-cluster shadowing term used in the cluster power update).

For Procedure A, several cluster-level random terms are evolved as functions of displacement between consecutive updates, using exponential spatial correlation models (e.g., [TR38901], Eq. 7.4-5). In TR 38.901 the procedure is described from the perspective of a moving terminal; when both endpoints move (e.g., V2V), ns-3 generalizes the triggering constraint by using the maximum displacement of either endpoint since the last generation/update (maximum endpoint displacement), while still using the change of the Tx–Rx relative position vector (relative/geometry displacement) as the displacement input for correlation updates of cluster-level random terms (e.g., the per-cluster shadowing term used in the cluster power update).

Thus, vehicles traveling in the same direction at similar speeds may have a small relative (geometry) displacement (slow geometry evolution), even though the maximum endpoint displacement can be large and may force a smaller UpdatePeriod to avoid exceeding the 1 m step-size constraint.

Note that **relative (geometry) displacement close to zero does not necessarily mean that no update occurs**. Procedure A includes update components that are driven by **endpoint motion over t_{Δ}** (in ns-3, UpdatePeriod is the analogue of t_{Δ}), in addition to components that use displacement-dependent spatial correlation. For example, the per-cluster delay evolution uses endpoint velocity terms over t_{Δ} (see [TR38901], Eq. 7.6-9). Cluster-angle updates also use the endpoint motion through the rotation/velocity terms (see [TR38901], Eqs. 7.6-10b and 7.6-10c) and the subsequent angle-update equations (Eqs. 7.6-11 to 7.6-14). Therefore, even in scenarios where the Tx–Rx geometry changes slowly (e.g., near-parallel motion), ns-3 may still perform a Procedure A update when UpdatePeriod expires (subject to the 1 m step-size constraint), and **motion-/time-dependent components** (e.g., angle evolution and related Doppler/phase terms) may still evolve. In contrast, updates that explicitly take the relative/geometry displacement as input (e.g., per-cluster shadowing correlation via the exponential model in [TR38901], Eq. 7.4-5) will exhibit little or no change when the relative/geometry displacement is near zero.

Example (V2V, same direction): $v_1 = 120$ km/h, $v_2 = 100$ km/h, so the relevant speed for the 1 m step-size constraint is the maximum endpoint speed, $v_{\text{end,max}} = \max(v_1, v_2) = 120$ km/h = 33.3 m/s. To keep the maximum endpoint displacement below 1 m between updates, choose `UpdatePeriod` ≤ 30 ms and ensure the channel is evaluated (e.g., via periodic signaling/traffic) with a period ≤ 30 ms.

Channel consistency applicability: The correlation distances used by the Procedure A updates depend on the configured scenario. For terrestrial cellular scenarios such as UMa, UMi, RMa, and Indoor, the spatial correlation distances used for per-cluster shadowing updates are specified in 3GPP TR 38.901 (see Table 7.6.3.1-2, “Correlation distance for spatial consistency”) and are reflected in the corresponding 3GPP parameter tables of the `ThreeGppChannelModel`.

For other scenarios, such as V2V and NTN, the relevant 3GPP technical reports (e.g., TR 37.885 for V2X and TR 38.811 for NTN) describe channel models and mobility assumptions but do not provide standardized tables for cluster correlation distances comparable to those in TR 38.901. Consequently, while the spatial consistency procedure can be used when these scenarios are configured, users should be aware that appropriate correlation-distance values must be selected and configured in the `ThreeGppChannelModel` parameter tables based on the intended deployment and propagation environment. In the absence of standardized 3GPP values, the current implementation uses a short correlation distance (1 m) for per-cluster shadowing, resulting in weak spatial correlation unless configured otherwise.

Blockage model: 3GPP TR 38.901 also provides an optional feature that can be used to model the blockage effect due to the presence of obstacles, such as trees, cars or humans, at the level of a single cluster. This differs from a complete blockage, which would result in an LOS to NLOS transition. Therefore, when this feature is enabled, an additional attenuation is added to certain clusters, depending on their angle of arrival. There are two possible methods for the computation of the additional attenuation, i.e., stochastic (Model A) and geometric (Model B). In this work, we used the implementation provided by [Zhang], which uses the stochastic method. In particular, the model is implemented by the method `CalcAttenuationOfBlockage`, which computes the additional attenuation. The blockage feature can be disabled through the attribute “Blockage”. Also, the attributes “NumNonselfBlocking”, “PortraitMode” and “BlockerSpeed” can be used to configure the model.

Testing

The test suite `ThreeGppChannelTestSuite` includes five test cases:

- `ThreeGppChannelMatrixComputationTest` checks if the channel matrix has the correct dimensions and if it correctly normalized
- `ThreeGppChannelMatrixUpdateTest`, which checks if the channel matrix is correctly updated when the coherence time exceeds
- `ThreeGppSpectrumPropagationLossModelTest`, which tests the functionalities of the class `ThreeGppSpectrumPropagationLossModel`. It builds a simple network composed of two nodes, computes the power spectral density received by the receiving node, and
 1. Checks if the long term components for the direct and the reverse link are the same,
 2. Checks if the long term component is updated when changing the beamforming vectors,
 3. Checks if the long term is updated when changing the channel matrix
- `ThreeGppCalcLongTermMultiPortTest`, which tests that the channel matrices are correctly generated when multiple transmit and receive antenna ports are used.
- `ThreeGppMimoPolarizationTest`, which tests that the channel matrices are correctly generated when dual-polarized antennas are being used.
- `ThreeGppChannelConsistencyTest` is designed to verify that consecutive channel realizations remain spatially consistent while the user is moving, which is the scenario addressed by the implemented channel consistency Procedure A (3GPP TR 38.901, Sec. 7.6.3).

The test considers both LOS and NLOS conditions, thereby exercising the different channel update mechanisms defined for each case, in particular the cluster delay update (Eq. 7.6-9) and the cluster departure and arrival angle

updates (Eqs. 7.6-10b and 7.6-10c). Multiple test cases are defined for different carrier frequencies and user speeds. For each test case, the channel coherence time is computed and the channel update period (configured through the UpdatePeriod attribute) is set accordingly.

The test verifies that spatially consistent channel updates produce significantly smoother channel evolution (approximately five times smaller variations) compared to generating independent channel realizations under the same LOS/NLOS conditions. To evaluate channel consistency, several metrics are tracked, including per-cluster parameters such as cluster delay, cluster power, and cluster angles (AOA and ZOA), as well as a global channel metric, the Frobenius norm of the channel matrix. For the per-cluster analysis, the third cluster, as defined in Table 7.8.5 for channel consistency calibration, is selected.

Note: TR 38.901 includes a calibration procedure that can be used to validate the model, but it requires some additional features which are not currently implemented, thus is left as future work.

References

29.4.3 Two-Ray fading model

The model aims to provide a performance-oriented alternative to the 3GPP TR 38.901 framework [TR38901] which is implemented in the `ThreeGppSpectrumPropagationLossModel` and `ThreeGppChannelModel` classes and whose implementation is described in [Zugno2020]. The overall design described in [Pagin2023] follows the general approach of [Polese2018], with aim of providing the means for computing a 3GPP TR 38.901-like end-to-end channel gain by combining several statistical terms. The frequency range of applicability is the same as that of [TR38901], i.e., 0.5 - 100 GHz.

Use-cases

The use-cases for this channel model comprise large-scale MIMO simulations involving a high number of nodes (100+), such as multi-cell LTE and 5G deployments in dense urban areas, for which the full 3GPP TR 38.901 does not represent a viable option.

Implementation - `TwoRaySpectrumPropagationLossModel`

The computation of the channel gain is taken care of by the `TwoRaySpectrumPropagationLossModel` class. In particular, the latter samples a statistical term which combines:

- The array and beamforming gain, computed as outlined in [Rebato2018] using the `CalcBeamformingGain` function. This term supports the presence of multiple antenna elements both at the transmitter and at the receiver and arbitrary antenna radiation patterns. Specifically, the array gain is compute as:

$$G_{AA}(\theta, \varphi) = |\mathbf{a}^T(\theta, \varphi) \mathbf{w}(\theta_0, \varphi_0)|^2 = |\text{AF}_v(\theta, \varphi)|^2 |\text{AF}_h(\theta, \varphi)|^2 G(\theta, \varphi),$$

where:

$$\text{AF}_v(\theta, \varphi) = \frac{1}{\sqrt{N_v}} \sum_{m=0}^{N_v-1} e^{jkd_v m (\cos \theta - \cos \theta_0)}$$

and:

$$\text{AF}_h(\theta, \varphi) = \frac{1}{\sqrt{N_h}} \sum_{n=0}^{N_h-1} e^{jkd_h n (\sin \theta \sin \varphi - \sin \theta_0 \sin \varphi_0)}$$

In turn, N_h , N_v are the number of horizontal and vertical antenna elements respectively, d_h , d_v are the element spacing in the horizontal and vertical direction respectively. The figures below depict the resulting array radiation pattern versus

the relative azimuth of transmitter and receiver, for antenna arrays featuring 3GPP TR 38.901 (`ThreeGppAntennaModel`, top) and isotropic (`IsotropicAntennaModel`, bottom) antenna elements, respectively. These figures match the corresponding plots of [Asplund].

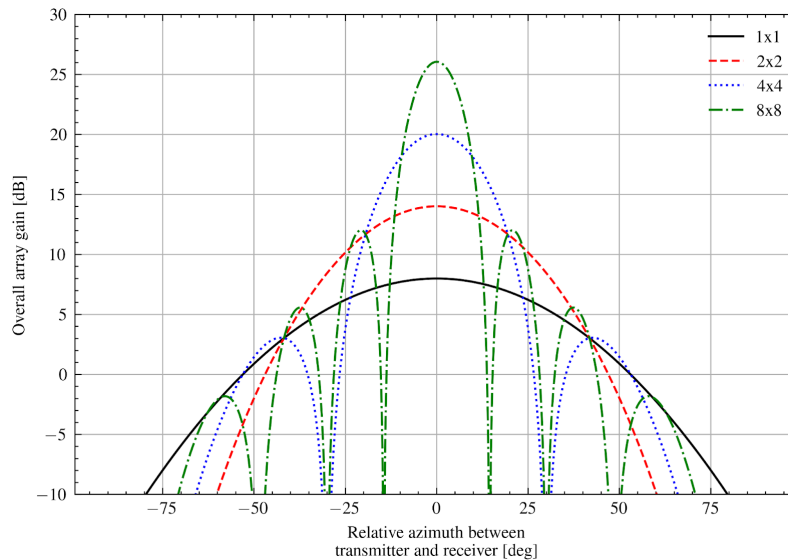


Fig. 29.6: Radiation pattern produced by the `CalcBeamformingGain` method when using antenna arrays featuring `ThreeGppAntennaModel` antenna elements, for various Uniform Planar Array (UPA) configurations.

Whenever the link is in NLOS, a penalty factor is introduced, to account for beam misalignment due to the lack of a dominant multipath component [Kulkarni].

- A fast fading term, sampled using the Fluctuating Two Ray (FTR) model distribution [Romero]. The latter is a fading model which is more general than typical ones, taking into account two dominant specular components and a mixture of scattered paths. As, a consequence it has been shown to provide a better fit to fading phenomena at mmWaves. The model parameters are automatically picked once the simulation scenario is set, using a lookup table which associates the simulation parameters (such as carrier frequency and LOS condition) to the FTR parameters providing the best fit to the corresponding TR 38.901 channel statistics. As a consequence, this channel model can be used for all the frequencies which are supported by the 38.901 model, i.e., 0.5-100 GHz. The calibration has been done by first obtaining the statistics of the channel gain due to the small-scale fading in the 3GPP model, using an ad hoc simulation script (`src/spectrum/examples/three-gpp-two-ray-channel-calibration.cc`). Then, this information has been used as a reference to estimate the FTR parameters yielding the closest (in a goodness-of-fit sense) fading realizations, using a custom Python script (`src/spectrum/utis/two-ray-to-three-gpp-ch-calibration.py`).

Note:

- To then obtain a full channel model characterization, the model is intended to be used in conjunction of the path loss and shadowing capability provided by the `ThreeGppPropagationLossModel` class. Indeed, the goal of this model is to provide channel realizations which are as close as possible to ones of [TR38901], but at a fraction of the complexity. Since the path loss and shadowing terms are not computationally demanding anyway, the ones of [Zugno2020] have been kept;
- Currently, the value of NLoS beamforming factor penalty factor is taken from the preliminary work of [Kulkarni] and it is scenario-independent; As future work, the possibility of using scenario-dependent penalty factors will be investigated.

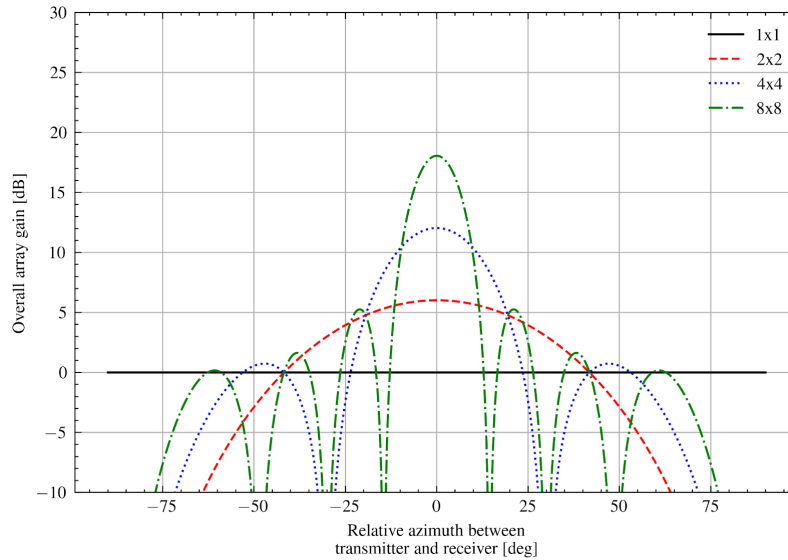


Fig. 29.7: Radiation pattern produced by the `CalcBeamformingGain` method when using antenna arrays featuring `IsotropicAntennaModel` antenna elements, for various Uniform Planar Array (UPA) configurations.

Calibration

The purpose of the calibration procedures is to compute offline a look-up table which associates the FTR fading model parameters with the simulation parameters. In particular, the [TR38901] fading distributions depend on:

- The scenario (RMA, UMA, UMi-StreetCanyon, InH-OfficeOpen, InH-OfficeMixed);
- The LOS condition (LoS/NLoS); and
- The carrier frequency.

As a consequence, the calibration output is a map which associates LoS condition and scenario to a list of carrier frequency-FTR parameters values. The latter represent the FTR parameters yielding channel realizations which exhibit the closest statistics to [TR38901].

The actual calibration is a two-step procedure which:

1. First generates reference channel gain curves using the `src/spectrum/examples/three-gpp-two-ray-channel-calibration.cc` simulation script. Specifically, the script samples `numRealizations` channel realizations and computes for each of them the end-to-end channel gain by setting the speed of the TX and RX pair to 0, disabling the shadowing and fixing the LOS condition. In such a way, any variation around the mean is due to the small-scale fading only. The channel gain samples are produced, and returned on output conditioned on the value of `enableOutput`, for each combination of LoS condition, channel model scenario and carrier frequency. The latter cover the whole [TR38901] frequency range of 0.5 - 100 GHz with a relatively sparse resolution (500 MHz), since the dependency of the fading distribution with respect to the carrier frequency is actually relatively weak.

2. Then, the output of the above script is parsed by the `two-ray-to-three-gpp-ch-calibration.py` Python companion script. In particular, reference ECDFs are obtained from the channel gains sampled using the model of [TR38901]. In turn, the reference ECDFs (one for each LoS condition, channel model scenario and carrier frequency combination) are compared to FTR distributions ECDFs obtained using different values of the parameters. Finally, the parameters which provide the best fit (in a goodness-of-fit sense) for the specific scenario, LOS condition and carrier frequency are found. The parameters to test are picked initially by performing an exhaustive search within a discrete grid of possible values, and then by iteratively refining the previous search runs by scanning the neighborhood of the most recent identified values. In such regard, the Anderson-Darling statistical test is used to rank the various FTR distributions and eventually pick the one providing the closest approximation to the reference statistics.

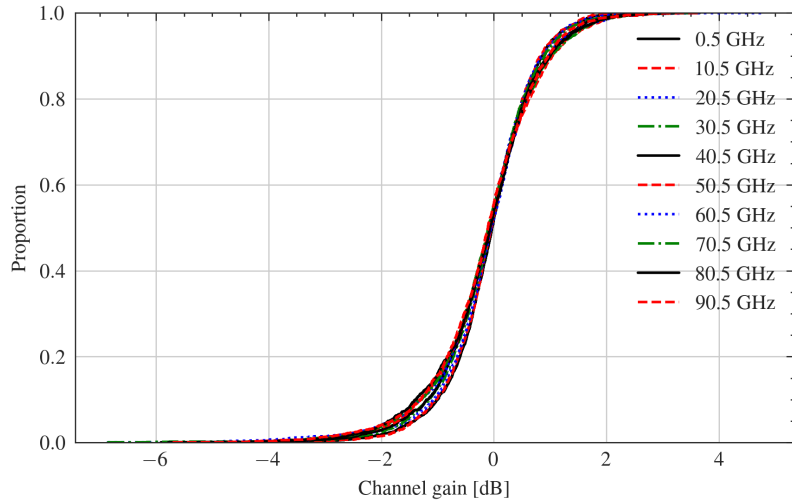


Fig. 29.8: Empirical CDF of the reference channel gains obtained using the `three-gpp-two-ray-channel-calibration` simulation script when keeping a fixed LoS condition and channel scenario and varying the carrier frequency only.

Testing

The test suite `TwoRaySplmTestSuite` includes three test cases:

- `FtrFadingModelAverageTest`, which checks that the average of the Fluctuating Two Ray (FTR) fading model realizations is consistent with the theoretical value provided in [\[Romero\]](#).
- `ArrayResponseTest`, which checks that the overall array response at boresight computed by the `CalcBeamformingGain` function coincides with the expected theoretical values.
- `OverallGainAverageTest`, which checks that the average overall channel gain obtained using the `DoCalcRxPowerSpectralDensity` method of the `TwoRaySpectrumPropagationLossModel` class is close (it is, after all, a simplified and performance-oriented model) to the one obtained using the `ThreeGppSpectrumPropagationLossModel` and `ThreeGppChannelModel` classes.

References

29.4.4 3GPP TR 38.811 Non-Terrestrial Networks

3GPP with [\[3GPPTR38811\]](#) has extended the channel model presented in [\[3GPPTR38901\]](#) to support the so called Non-Terrestrial Networks (NTN), i.e. communication scenarios where the propagation of the signal travels through the atmosphere. The channel spectrum estimation procedure is the same as the one described in [\[3GPPTR38901\]](#), with a new set of parameters.

Use-cases

The use-cases for this channel model include simulations in 3D/vertical environments with communicating nodes placed in different types of orbit, in the atmosphere and/or on the ground.

Implementation

The channel spectrum estimation procedure is already implemented (as described in [ns3Zugno]) into the classes `ThreeGppChannelModel` and `ThreeGppSpectrumPropagationLossModel`, that have been extended to support NTN through the introduction of the appropriate parameters. 3GPP considers two frequencies of interest: S-band (2GHz) and Ka-band (20/30GHz). Many channel parameters are dependent on the frequency band in use, but no specific range of frequencies of these bands has been given by 3GPP. Hence, this implementation considers S-band anything below 13GHz, and Ka-band anything above it. Four propagation scenarios are identified, in decreasing order of building height and density: Dense Urban, Urban, Suburban and Rural. Channel estimation parameters are scenario-dependent. **Note:** For satellite, the parameters that describe the departure angle spread are 0. Thus, in the logarithmic scale at which these parameters are represented is $-\infty$.

References

29.4.5 Wraparound Models

The wrap around mechanism is a simulation technique used in cellular network modeling to eliminate edge effects and create a more realistic interference environment. The most common setup used by 3GPP is the hexagonal deployment wraparound, which transforms a finite hexagonal cellular cluster into what appears to be an infinite network by creating virtual copies of the cluster around the original one.

This also can significantly reduce memory and computational requirements of simulations to achieve similar results in respect to interference, avoiding the simulation of additional rings and then filtering only central rings with equivalent interference.

WraparoundModel Implementation

When a wraparound model is aggregated to the `SingleModelSpectrumChannel` or `MultiModelSpectrumChannel`, the base class `WraparoundModel` creates a virtual `MobilityModel` for the transmitter, in respect to each receiver.

The virtual `MobilityModel` is carried via the `SpectrumSignalParameters` to the receiver, which must use that model to retrieve the transmitter position, `NodeId` and buildings related information during `StartRx`.

The base `WraparoundModel` only reuses the existing transmitter mobility model, while its children classes may implement different wraparound techniques.

HexagonalWraparoundModel Implementation

`HexagonalWraparoundModel` implements wraparound for the standard cellular network setups using hexagonal clusters, and is based on [Panwar]. Without wraparound, only the central cells experience symmetric interference from all directions. Edge cells receive unrealistic interference patterns because they lack neighboring cells on certain sides, making their performance data statistically invalid for real-world analysis. The wrap around mechanism addresses this “edge effect” problem by ensuring all cells in the simulation experience equivalent interference conditions. For this reason, wraparound is mandatory for 5G NR calibration of selected scenarios, as defined in [TR38901].

Interference with ring 1 and wraparound (orange) is higher than even ring 3 (green). This, in spite of only simulating 7 tri-cell clusters of ring 1 versus the 19 clusters of ring 3. The interference of both ring 1 with wraparound and

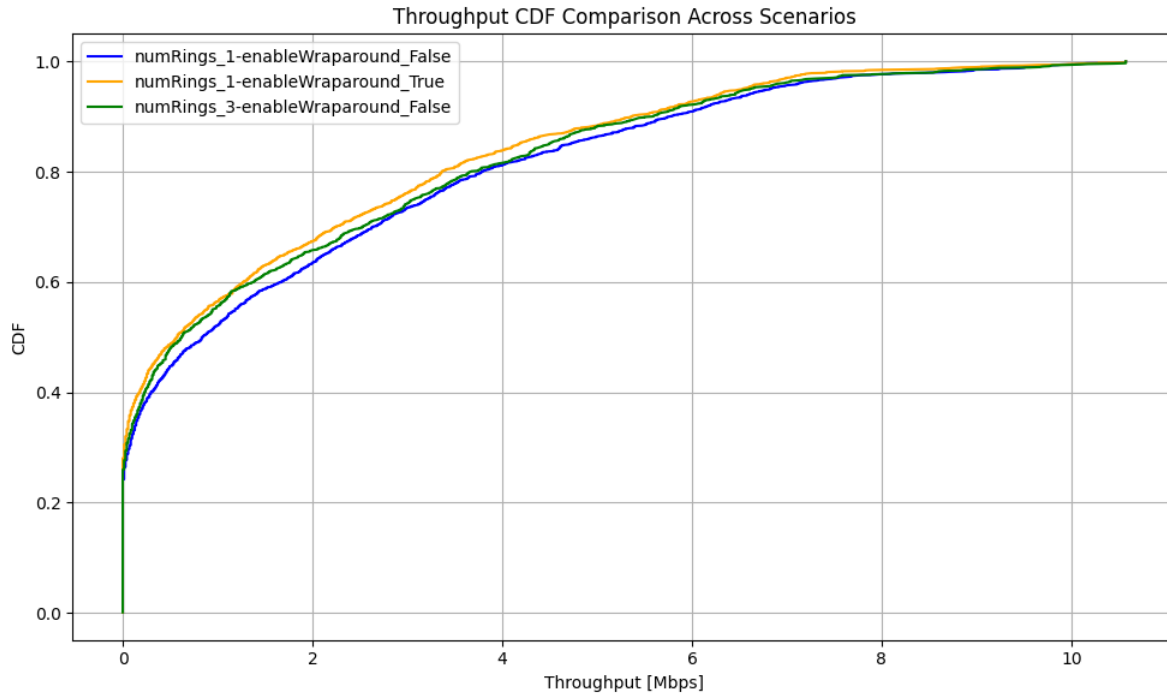


Fig. 29.9: Ring 1 setup with 7 tri-sector cell clusters, with (orange) and without wraparound (blue), plus ring 3 setup with 19 tri-sector cell clusters without wraparound (green).

ring 3 have much much higher than ring 1 without wraparound (blue). As a result, throughput without wraparound is unrealistically high (indicated by lower curves shifted to the right). This is confirmed in the following right figure.

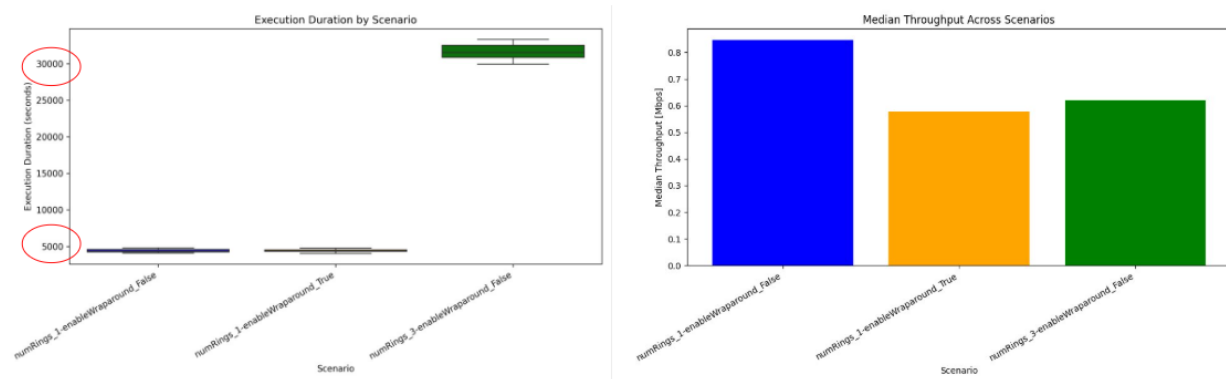


Fig. 29.10: Left: Execution times of ring 1 setup with and without wraparound, plus ring 3 setup without wraparound in an AMD Ryzen Threadripper PRO 7965WX. Right: Median throughput of the different configurations.

Since the computational cost of wireless simulations scales quadratically with the number of nodes in the channel, we significantly reduce computational costs by cutting the number of simulated clusters. For this particular 5G-NR outdoor calibration example with different number of rings and wraparound, the speedup of ring 1 with wraparound against ring 3 without wraparound is of 6x, for comparable results.

Note: ring 3 simulations without wraparound require the user to manually filtering results to only cells and UEs that are part of the ring 1 cell clusters. This is because they suffer first and second tier interference. The remaining results for 12 out of 19 cells, and their respective UEs, would be discarded.

The wrap around mechanism operates by creating six additional virtual copies of the original hexagonal cluster, positioned symmetrically around the central cluster. It usually follows these steps:

- **Virtual Cluster Creation:** Six copies of the original cluster are mathematically placed around the central cluster to simulate an infinite grid
- **Distance Calculation:** For each signal transmission, the system calculates seven different distances - one to the actual transmitter and six to the virtual transmitter positions
- **Minimum Distance Selection:** The shortest distance is used for path loss and signal strength calculations, ensuring realistic propagation effects
- **UE Mobility Handling:** When a user device moves to connect with a virtual cell, it's automatically repositioned within the central cluster while maintaining the same relative position to its serving cell (moving UE to central cluster is not currently implemented)

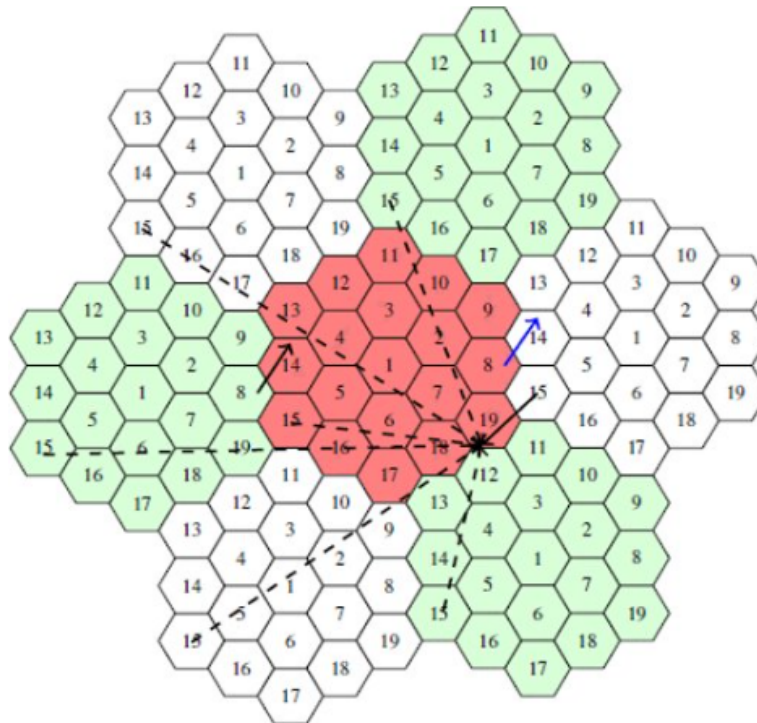


Fig. 29.11: Virtual cell clusters (white and green) around real cell cluster (red). Virtual distances are shown as dashed arrows. The virtual transmitter position that results in the smallest distance to receiver is used in the virtual mobility model. [Panwar]

The technique is implemented using geometric distance calculations with specific equations for different cluster sizes:

- ring 3: 19-site clusters: Standard configuration with 2-tier interference (57 cells for tri-sector antennas)
- ring 1: 7-site clusters: Smaller configuration with 1-tier interference (21 cells for tri-sector antennas)
- ring 0: 3-site clusters: Minimal configuration for quick testing (9 cells for tri-sector antennas)

Before start transmitting, each site position needs to be registered to the `HexagonalWraparoundModel` via `AddSitePosition(Vector3D position)`. The number of sites should also be set via `SetNumSites(uint8_t sites)`. Finally, the inter-site distance (ISD) must be set via `SetSiteDistance(double isd_in_meters)`.

This can be automatically done by hexagonal deployment helpers of LTE and NR, by setting the `EnableWraparound` attribute to true. After setting up the hexagonal deployment, the user should call `GetWraparoundModel` method of the hexagonal deployment helper, and then aggregate that model to the spectrum channel model.

Caveats and Limitations

This implementation does not simply model edge interference in a localized or realistic way. Instead, because all signals are routed through higher-layer protocols, the effect resembles a “wormhole” model where transmissions can unrealistically appear at distant points in the topology. The transmitter itself stays fixed, but its signals may propagate as if emerging from arbitrary locations, depending on geometry and inter-site distances.

A side-effect is that mobility and handover do not strictly reflect physical adjacency. Simulations may misleadingly suggest a larger or denser deployment, allowing handover not just to neighboring cells but also to virtually adjacent ones (including cells that are physically on opposite sides of the layout). Plots can therefore be confusing: devices that look widely separated in absolute position might be treated as if they were close in virtual space.

The use of virtual mobility inside the spectrum channel models further complicates interpretation. Transmitters are treated as if they relocate, and any additional attenuation (such as building shadowing modeled through ns-3 propagation loss and buildings modules) is applied even though the device itself never actually moved. The buildings themselves are not virtualized, but buildings deployed into the non-virtualized topology still affect virtual transmissions.

[[Panwar](#)] discusses the use of wraparound for mobility models also, such that a node that left the physical site boundaries would wormhole around and reappear within the site at the mirror location. This means replacing the physical position with the virtual position when going out of physical cell areas, so the node stays within the physical topology throughout the simulation. This is not currently implemented. As such, nodes may appear outside of physical cells, but still are considered as transmitting from within the virtual cells.

References

6LOWPAN: IPV6 OVER LOW-POWER WIRELESS PERSONAL AREA NETWORKS

This describes the implementation of *ns-3* model for the compression of IPv6 packets over low-power wireless networks, as originally specified by [RFC 4944](#) (“Transmission of IPv6 Packets over IEEE 802.15.4 Networks”) and [RFC 6282](#) (“Compression Format for IPv6 Datagrams over IEEE 802.15.4-Based Networks”).

The 6LoWPAN optimized neighbour discovery (6LoWPAN-ND) is also implemented in this module, as specified by [RFC 8505](#) (“Registration Extensions for IPv6 over Low-Power Wireless Personal Area Network (6LoWPAN) Neighbor Discovery”) and [RFC 6775](#) (“Neighbor Discovery Optimization for IPv6 over Low-Power Wireless Personal Area Networks (6LoWPANs)”).

6LoWPAN was originally proposed for IEEE 802.15.4 (Lr-WPAN), but it has been also proposed for other constrained resource networks, such as Sub-1 GHz low-power RF, Bluetooth LE, etc. To reflect this, the *ns-3* module design allows for 6LoWPAN to be used on any network, even non-constrained ones like CSMA or WiFi. The module strictly follows [RFC 4944](#) and [RFC 6282](#), with the exception of header compression 2 (HC2) encoding which is not supported, as it has been superseded by IP Header Compression (IPHC) and Next Header Compression (NHC) types ([RFC 6282](#)).

SixLowPanNetDevice

The whole module is developed as a transparent NetDevice, which can act as a proxy between IPv6 and any NetDevice (the module has been successfully tested with `PointToPointNetDevice`, `CsmaNetDevice` and `LrWpanNetDevice`). For this reason, the module implements a virtual NetDevice, and all the calls are passed without modifications to the underlying NetDevice. The only important difference is in the `GetMtu` behaviour, which will always return *at least* 1280 bytes, as mandated by the minimum IPv6 MTU:

- If the underlying NetDevice has an MTU smaller than 1280 octets (common for IEEE 802.15.4), 6LoWPAN silently raises the reported MTU to 1280 and handles fragmentation and reassembly internally at the 6LoWPAN level.
- If the underlying NetDevice has an MTU of 1280 octets or larger, the MTU is left unchanged and 6LoWPAN fragmentation is applied only when necessary (uncommon, but foreseen by the standard).

In both cases this is completely transparent to the upper layers.

The source code for the `sixlowpan` module lives in the directory `src/sixlowpan`.

30.1 Scope and Limitations

The following is a list of known limitations of the *ns-3* 6LowPAN implementation:

- 6lowPAN requires a preset MAC address before the start of the simulation. In other words, it cannot dynamically extract the underlying MAC addresses once the simulation has started.
- When used along side IEEE 802.15.4, only short addresses (16-bit) are supported.
- HC2 is not included but it is deprecated in the RFCs.

The following is a list of limitations for 6LowPAN-ND:

- Lack of support for multi-hop DAD exchanges
- Limited NA (EARO) status errors supported from [RFC 6775](#) (Currently only supports “Duplicate Address” error)
- Currently supports only single-hop, mesh-under routing topologies between 6LBR and 6LN
- Missing Transaction ID validation, which is part of the [RFC 8505](#) specification

30.2 Compression

IPHC stateful (context-based) and HC1 compressions are supported. The IPv6/MAC addressing schemes defined in [RFC 6282](#) and [RFC 4944](#) are different. One adds the PanId in the pseudo-MAC address (4944) and the other doesn't (6282).

The expected use cases (confirmed by the RFC editor) is to *never* have a mixed environment where part of the nodes are using HC1 and part IPHC because this would lead to confusion on what the IPv6 address of a node is. Due to this, the nodes configured to use IPHC will drop the packets compressed with HC1 and vice-versa. The drop is logged in the drop trace as `DROP_DISALLOWED_COMPRESSION`.

When using the IPHC stateful compression, nodes need to be aware of the context. To manually set the context, it is possible to use the `SixLowPanHelper::AddContext` function. Please be aware that installing different contexts for different nodes will lead to decompression failures.

30.3 Routing Handling

6lowPAN provides two different approaches for routing IPv6 packets within a 6lowPAN network:

1. Mesh-under routing
2. Route over routing

Mesh-under routing

A mesh-under routing approach indicates that a routing system is implemented below IP and 6lowPAN will make the packet routing decisions based on layer 2 addresses. The 6lowPAN *ns-3* module provides a very simple mesh-under routing, implemented as a flooding

At node level, each packet is re-broadcasted if its BC0 sequence number is not in the cache of the recently seen packets. The cache length (by default 10) can be changed through the `MeshCacheLength` attribute. This functionality can be activated through the `UseMeshUnder` attribute and fine-tuned using the `MeshUnderRadius` and `MeshUnderJitter` attributes.

Note: Flooding in a PAN generates a lot of overhead, which is often not wanted. Moreover, when using the mesh-under facility, ALL the packets are sent without acknowledgment because, at lower level, they are sent to a broadcast address.

The current mesh-under flooding routing mechanism could be improved by providing the following:

- Adaptive hop-limit calculation,
- Adaptive forwarding jitter,
- Use of direct (non mesh) transmission for packets directed to 1-hop neighbors.

Route-over routing

The routing decisions are made at the network layer (over IP) and use IPv6 addresses for routing like traditional IP networks. The usage of route-over routing requires more processing at each hop as the IPV6 and other headers needs to be processed. However, it is more flexible than the mesh-under approach as you can use one or more routing mechanism to reach the destination.

Examples of routing protocols used with 6lowPAN route-over include protocols such as the Routing protocol for Low-Power and Lossy Networks (RPL) and the Ad-hoc On-Demand Distance Vector for IPV6 (AODVv6).

30.4 6lowPAN Optimized Neighbor Discovery (6lowPAN-ND)

IPv6 Neighbor Discovery (ND) as defined in [rfc`4861`](#) assumes always on, multicast friendly links, which are not possible for low-power and lossy networks (LLNs) such as those using IEEE 802.15.4.

[RFC 6775](#) and [RFC 8505](#) define 6LoWPAN Neighbor Discovery (6LoWPAN-ND) to address these limitations. Some key optimizations include:

- Host-initiated interactions (sleep-aware RAs): routers need no longer send periodic or unsolicited RAs, instead relying on hosts to initiate interactions for address registration.
- Use of unicast communication for address resolution and neighbor discovery.
- Registration of addresses with a 6LoWPAN Router (6LR) or 6LoWPAN Border Router (6LBR) to reduce broadcast traffic and energy usage.
- 6LoWPAN Context Option (6CO) distributes header-compression contexts to hosts using 6LoWPAN, enabling efficient compression of IPv6 headers.

In ns-3, 6LoWPAN-ND subclasses `Icmpv6L4Protocol`, taking over the functions of conventional Ipv6 Neighbour Discovery as defined in [RFC 4861](#).

It does this by introducing a registration mechanism for 6LoWPAN nodes (6LNs) to register their addresses with a 6LoWPAN router (6LR) or border router (6LBR). This allows 6LNs to use the 6LR/LBR as a proxy for address resolution and neighbor discovery, reducing the need for broadcast messages.

Every node that implements 6LoWPAN-ND will typically have a protocol stack that resembles the following:

L4 SixLowPanNdProtocol (Subclasses Icmpv6L4Protocol)
L3 Ipv6L3Protocol
L2.5 SixLowPanNetDevice (shim)
L2 LrWpanNetDevice

30.5 6LoWPAN-ND Network Topologies:

Mesh-Under Routing

The current implementation of 6LoWPAN-ND in *ns-3* supports mesh-under routing with single-hop star topologies, where one or more 6LoWPAN Nodes (6LNs) register their IPv6 addresses directly with a single 6LoWPAN Router (6LR), which may also act as the 6LoWPAN Border Router (6LBR).

In this setup:

- All address registration (using NS(EARO)) is performed over the same local link.
- Router Advertisements (RAs) and Neighbor Solicitations (NS) are exchanged directly between 6LNs and the 6LR/LBR.
- The registration cache is maintained at the 6LBR.

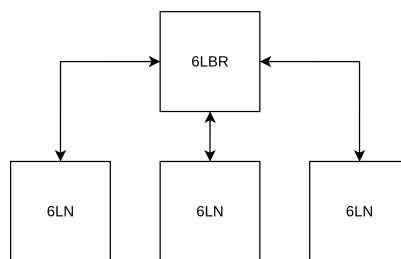


Fig. 30.1: Example topology of a 6LoWPAN-ND network using mesh-under routing

Route-Over Topology

The multi-hop route-over topology is defined in [RFC 8505](#), and is currently in active development in *ns-3*. In this topology, 6LNs register with a single 6LR, and the 6LR can forward packets to the 6LBR, allowing for multi-hop DAD and global address registration across the LLN.

In this setup:

- All link-local address registration (using NS(EARO)) is performed over the same local link.
- Router Advertisements (RAs) and Neighbor Solicitations (NS) are exchanged directly between 6LNs and the 6LR/LBR, or between 6LRs and the 6LBR.
- The global address registration cache is maintained at the 6LBR.

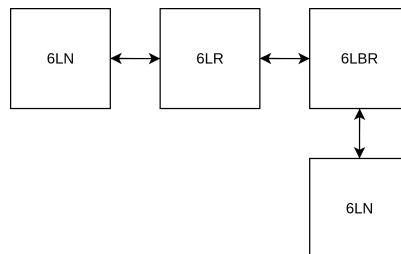


Fig. 30.2: Example topology of a 6LoWPAN-ND network using route-over topology

30.6 6LoWPAN-ND Node Roles

6LoWPAN-ND defines the following node roles:

- **6LBR (SixLowPanBorderRouter)**: Acts as the gateway between the 6LoWPAN network and the wider Internet. It maintains the Globally Unique Address registration cache for all nodes in the LLN, and Link-local addresses for nodes that are within 1 hop to the 6LBR.
- **6LR (SixLowPanRouter)**: Forwards packets between 6LNs and the 6LBR. It may also perform address registration for Link-local addresses for nodes within its network.
- **6LN (SixLowPanNodeOnly)**: A device that participates in the 6LoWPAN network, capable of sending and receiving IPv6 packets.

An additional node role, `SixLowPanNode`, is used for nodes that start out as 6LNs, but can be upgraded to 6LR later in the simulation. It is used in Route-Over topologies, where a 6LN can become a 6LR to forward packets for other 6LNs as part of multi-hop DAD.

30.7 Usage

As the name implies, 6LoWPAN can handle only IPv6 packets. Any other protocol will be discarded.

6LoWPAN can be used alongside other L3 protocols in networks supporting an `EtherType` (e.g., Ethernet, WiFi, etc.). If the network does not have an `EtherType` in the frame header (like in the case of 802.15.4), then the network must be uniform, as is all the devices connected by the same channel must use 6LoWPAN.

The reason is simple: if the L2 frame doesn't have a "EtherType" field, then there is no demultiplexing at MAC layer and the protocol carried by L2 frames must be known in advance.

Note: By default, 6LoWPAN will use an `EtherType` equal to 0xA0ED, as mandated by [RFC 7973](#). If the device does not support `EtherTypes` (e.g., 802.15.4), this value is discarded.

IPHC stateful (context-based) compression is supported. Contexts map a short numeric ID to an IPv6 prefix, allowing addresses that share that prefix to be compressed more aggressively.

Context configuration is only required when using IPHC stateful (context-based) compression. When no contexts are configured, nodes automatically fall back to stateless IPHC compression and no context setup is needed. If contexts are configured, all nodes in the network must share an identical set of contexts, or decompression will fail.

When stateful compression is used, there are two ways to distribute contexts to nodes:

Automatic Context Distribution (Recommended)

If 6LoWPAN-ND is enabled, the 6LBR automatically advertises contexts to 6LNs via the 6LoWPAN Context Option (6CO) carried in Router Advertisements. No manual context configuration is required on the 6LNs:

```
// Configure 6LBR to advertise prefix and context automatically
SixLowPanHelper sixlowpan;
sixlowpan.InstallSixLowPanNdBorderRouter(borderRouterDevice, "2001::");
sixlowpan.SetAdvertisedPrefix(borderRouterDevice, Ipv6Prefix("2001::", 64));
sixlowpan.AddAdvertisedContext(borderRouterDevice, Ipv6Prefix("2001::", 64));

// 6LNs learn prefix and contexts automatically from Router Advertisements
sixlowpan.InstallSixLowPanNdNode(nodeDevice);
```

Manual Context Configuration

If 6LoWPAN-ND is not used (i.e., standard IPv6 Neighbor Discovery is used instead), contexts can be configured manually and identically on every node in the network:

```
// Manual context setup - must be identical on all nodes
sixlowpan.AddContext(allDevices, 0, Ipv6Prefix("2001::", 64), Time(Minutes(60)));
```

Note that when using manual context configuration, all nodes in the network must have identical context configurations to avoid decompression failures

30.7.1 Helpers

The 6lowPAN helpers are patterned after other device helpers.

The main helper is `SixLowPanHelper`, which provides methods to install 6LoWPAN on a `NetDevice` and configure the attributes of the 6LoWPAN protocol. The typical use case is over `LrWpanNetDevice`, as shown below (taken from `src/sixlowpan/examples/example-sixlowpan-nd-basic.cc`):

```
NodeContainer nodes;
nodes.Create(1 + numLn); // node 0 = 6LBR, nodes 1-n = 6LNs

MobilityHelper mobility;
mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
mobility.Install(nodes);

LrWpanHelper lrWpanHelper;
NetDeviceContainer lrWpanDevices = lrWpanHelper.Install(nodes);
lrWpanHelper.CreateAssociatedPan(lrWpanDevices, 0);

InternetStackHelper internetv6;
internetv6.Install(nodes);

SixLowPanHelper sixlowpan;
NetDeviceContainer sixlowpanNetDevices = sixlowpan.Install(lrWpanDevices);
```

Note: 6LoWPAN can be used on any `NetDevice`, not just `LrWpanNetDevice`. For an example using `CsmaNetDevice`, see `src/sixlowpan/examples/example-sixlowpan.cc`.

Users have two options for neighbor discovery in their 6LoWPAN networks:

1. **Use standard IPv6 Neighbor Discovery:** If 6LoWPAN-ND is not installed, the simulation will use the standard IPv6 Neighbor Discovery protocol ([RFC 4861](#)) with manual IPv6 address assignment. This approach works but lacks the optimizations for low-power networks.
2. **Use 6LoWPAN-ND (Recommended for realistic scenarios):** Installing 6LoWPAN-ND provides optimized neighbor discovery with automatic address configuration, energy-efficient unicast communication, and context distribution for header compression.

To install 6LoWPAN-ND, two helper methods are used:

- `SixLowPanHelper::InstallSixLowPanNdBorderRouter`: called on the **single** net device that will act as the 6LoWPAN Border Router (6LBR). Only one 6LBR is supported per network.
- `SixLowPanHelper::InstallSixLowPanNdNode`: called on each net device that will act as a 6LoWPAN Node (6LN). This must **not** be called on the 6LBR device.

The helper also provides methods to set the advertised prefix and other attributes related to 6LoWPAN-ND.:

```
// Configure 6LoWPAN ND
// Node 0 = 6LBR, Node 1 = 6LN
sixlowpan.InstallSixLowPanNdBorderRouter(sixlowpanNetDevices.Get(0), "2001::");
sixlowpan.SetAdvertisedPrefix(sixlowpanNetDevices.Get(0), Ipv6Prefix("2001::", 64));

sixlowpan.InstallSixLowPanNdNode(sixlowpanNetDevices.Get(1));
```

30.7.2 Attributes

The 6lowPAN provide some attributes:

- **CompressionType:** (enum, default IPHC), selects the header compression scheme: HC1 (RFC4944) or IPHC (RFC6282).
- **OmitUdpChecksum:** (boolean, default true), Omit the UDP checksum in IPHC compression.
- **FragmentReassemblyListSize:** (integer, default 0), indicating the number of packets that can be reassembled at the same time. If the limit is reached, the oldest packet is discarded. Zero means infinite.
- **FragmentExpirationTimeout:** (Time, default 60 seconds), being the timeout to wait for further fragments before discarding a partial packet.
- **CompressionThreshold:** (unsigned 32 bits integer, default 0), minimum compressed payload size.
- **UseMeshUnder:** (boolean, default false), it enables mesh-under flood routing.
- **MeshUnderRadius:** (unsigned 8 bits integer, default 10), the maximum number of hops that a packet will be forwarded.
- **MeshCacheLength:** (unsigned 16 bits integer, default 10), the length of the cache for each source.
- **MeshUnderJitter:** (ns3::UniformRandomVariable[Min=0.0|Max=10.0]), the jitter in ms a node uses to forward mesh-under packets - used to prevent collisions.

The CompressionThreshold attribute is similar to Contiki's SICSLOWPAN_CONF_MIN_MAC_PAYLOAD option. If a compressed packet size is less than the threshold, the uncompressed version is used (plus one byte for the correct dispatch header). This option is useful when a MAC requires a minimum frame size (e.g., ContikiMAC) and the compression would violate the requirement.

The following is a list of attributes connected to the 6lowPAN-ND implementation:

- **AddressRegistrationJitter:** The amount of jitter (in milliseconds) applied before sending an Address Registration. This jitter, sampled uniformly between 0 and the maximum value, helps avoid packet collisions among nodes registering simultaneously.
- **RegistrationLifetime:** Specifies the lifetime of a registered address in the neighbor cache of a router. The value is in units of 60 seconds (i.e., a value of 65535 represents the maximum of ~45 days).
- **AdvanceTime:** Time (in seconds) before expiration that the protocol proactively maintains or refreshes Router Advertisement and registration state. Useful for avoiding expiry during active operation.
- **DefaultRouterLifetime:** Lifetime assigned to a default router entry. After this period, the router will no longer be considered valid unless refreshed. Default is 60 minutes.
- **DefaultPrefixInformationPreferredLifetime:** Preferred lifetime for prefix information sent in Router Advertisements. Affects address autoconfiguration preferences. Default is 10 minutes.
- **DefaultPrefixInformationValidLifetime:** Valid lifetime for prefixes advertised. Beyond this period, the prefix is no longer valid for use in address formation. Default is 10 minutes.

- **DefaultContextValidLifetime:** Lifetime of 6LoWPAN Context Information Options (CIOs). Determines how long compression context information remains valid. Default is 10 minutes.
- **DefaultAbroValidLifetime:** Lifetime of Authoritative Border Router Option (ABRO) information. Default is 10 minutes. This is relevant in multihop 6LoWPAN ND deployments with multiple border routers.
- **MaxRtrSolicitationInterval:** The maximum interval between sending Router Solicitations when retrying with backoff. This controls how long a node waits between RS attempts. Default is 60 seconds.

30.7.3 Traces

The supported 6LoWPAN trace sources are:

- **Tx:** Exposing packet (including 6LoWPAN header), `SixLoWPanNetDevice` Ptr, interface index.
- **Rx:** Exposing packet (including 6LoWPAN header), `SixLoWPanNetDevice` Ptr, interface index.
- **Drop:** Exposing `DropReason`, packet (including 6LoWPAN header), `SixLoWPanNetDevice` Ptr, interface index.

The Tx and Rx traces are called as soon as a packet is received or sent. The Drop trace is invoked when a packet (or a fragment) is discarded.

The supported 6LoWPAN-ND trace sources are:

- **AddressRegistrationResult:** Exposing the IPv6 address associated with the registration, and a boolean indicating whether it was successful or not.
- **MulticastRS:** Exposing the IPv6 address associated with the multicast Router Solicitation that was sent.

The `AddressRegistrationResult` trace is called when a 6LoWPAN node successfully registers its address with a 6LR or 6LBR, or when the registration fails, which is a core function of the 6LoWPAN-ND protocol. The `MulticastRS` trace is called when a multicast Router Solicitation is sent by a 6LoWPAN node. This is useful for monitoring the behaviour of nodes on startup as they attempt to discover routers in the network.

30.8 Examples and Tests

Example can be found in `src/sixlowpan/examples/` and test under `src/sixlowpan/test/`.

The following shows 6lowPAN usage under *ns-3*:

- `example-sixlowpan.cc`: A simple example showing end-to-end data transfer, with a CSMA network forced to carry 6LoWPAN compressed packets.

The following examples have been created to show the 6lowPAN-ND usage:

- `example-sixlowpan-nd-basic.cc`: An example showing how to instantiate the 6LBR and 6LN in 6lowpan-nd with the helper methods in a typical 6lowpan network, enabling features like automatic address configuration.

The following tests have been created to verify the correct behavior of 6lowPAN-ND:

- `sixlowpan-nd-packet-test.cc`: Contains unit tests that verify helper methods for parsing and validating 6LoWPAN-ND packets, including NS(EARO) and NA(EARO) formats.
- `sixlowpan-nd-reg-test.cc`: Verifies basic address registration flows between 1-20 6LNs and a 6LBR, including successful and failed registration scenarios, as well as multicast RS binary backoff behaviour.
- `sixlowpan-nd-rovr-test.cc`: Runs a scenario where an address registration for an already registered address is made, under a different ROVR, to test that the 6LBR behaviour, and NA (EARO) response is correct.

30.9 Validation

The 6LoWPAN and 6LoWPAN-ND models have been verified against Wireshark, checking that packets are correctly interpreted and verified.

6LoWPAN Header Compression Validation

The 6LoWPAN header compression implementation has been verified by comparing the compressed packets generated by ns-3 with reference implementations and ensuring proper decompression by standard tools.

6LoWPAN-ND Protocol Validation

The 6LoWPAN-ND implementation has been verified through analysis of PCAP traces generated in the examples and tests. The validation process includes:

- **Packet Format Verification:** PCAP traces from `example-sixlowpan-nd-basic.cc` are analyzed using Wireshark to verify that:
 - Router Solicitation (RS) messages contain proper 6LoWPAN-ND options
 - Router Advertisement (RA) messages include correct 6LoWPAN Context Options (6CO) and Authoritative Border Router Options (ABRO)
 - Neighbor Solicitation with Extended Address Registration Option (NS-EARO) messages are properly formatted
 - Neighbor Advertisement with Extended Address Registration Option (NA-EARO) responses contain correct status codes
- **Protocol Flow Validation:** The timing and sequence of 6LoWPAN-ND message exchanges are verified to ensure compliance with RFC 6775 and RFC 8505 specifications:
 - Initial Router Solicitation upon node startup
 - Address registration sequences using NS(EARO)/NA(EARO) exchanges
 - Context distribution via 6CO options in Router Advertisements

30.10 References

- [1] Z. Shelby and C. Bormann, 6LoWPAN: The Wireless Embedded Internet. Wiley, 2011.
- [2] RFC 8505: Registration Extensions for 6LoWPAN Neighbor Discovery, E. Thubert, November 2018.
- [3] RFC 6775: Neighbor Discovery Optimization for IPv6 over Low-Power Wireless Personal Area Networks (6LoWPANs), Z. Shelby, S. Chakrabarti, E. Nordmark, C. Bormann, November 2012.

TOPOLOGY INPUT READERS

The topology modules aim at reading a topology file generated by an automatic topology generator.

The process is divided in two steps:

- running a topology generator to build a topology file
- reading the topology file and build a ns-3 simulation

Hence, model is focused on being able to read correctly the various topology formats.

Currently there are three models:

- `ns3::OrbisTopologyReader` for [Orbis 0.7](#) traces
- `ns3::InetTopologyReader` for [Inet 3.0](#) traces
- `ns3::RocketfuelTopologyReader` for [Rocketfuel](#) traces

An helper `ns3::TopologyReaderHelper` is provided to assist on trivial tasks.

In some cases it might not be simple to identify which *ns-3* node corresponds to a given node in the topology file. To simplify this task, each node created by `ns3::TopologyReaderHelper` has a name. The name format is “<TopologyReader>/NodeName/<label>” where “<TopologyReader>” is either “InetTopology”, “OrbisTopology”, or “RocketFuelTopology”, and “<label>” is the identifier of the node in the topology file (can be either a number or a string, depending on the file type). Assuming that there are 10 nodes, labeled with number starting from 0, the code could be:

```
for (uint32_t nodeNumber = 0; nodeNumber < 10; nodeNumber++)
{
    Ptr<Node> node = Names::Find<Node>("InetTopology/NodeName/" + std::to_
↪string(nodeNumber));
    if (node)
    {
        // Do something
    }
}
```

A good source for topology data is also [Archipelago](#).

The current Archipelago [Measurements](#), monthly updated, are stored in the CAIDA website using a complete notation and triple data source, one for each working group.

A different and more compact notation reporting only the AS-relationships (a sort of more Orbis-like format) is here: [as-relationships](#).

The compact notation can be easily stripped down to a pure Orbis format, just removing the double relationships (the compact format use one-way links, while Orbis use two-way links) and pruning the 3rd parameter. Note that with the compact data Orbis can then be used create a rescaled version of the topology, thus being the most effective way (to my best knowledge) to make an internet-like topology.

Examples can be found in the directory `src/topology-read/examples/`

TRAFFIC CONTROL LAYER

32.1 Traffic Control Layer

The Traffic Control layer aims at introducing an equivalent of the Linux Traffic Control infrastructure into ns-3. The Traffic Control layer sits in between the NetDevices (L2) and any network protocol (e.g. IP). It is in charge of processing packets and performing actions on them: scheduling, dropping, marking, policing, etc.

32.1.1 Introducing the Traffic Control Layer

The Traffic Control layer intercepts both outgoing packets flowing downwards from the network layer to the network device and incoming packets flowing in the opposite direction. Currently, only outgoing packets are processed by the Traffic Control layer. In particular, outgoing packets are enqueued in a queuing discipline, which can perform multiple actions on them.

In the following, more details are given about how the Traffic Control layer intercepts outgoing and incoming packets and, more in general, about how the packets traverse the network stack.

Transmitting packets

The IPv{4,6} interfaces uses the aggregated object TrafficControlLayer to send down packets, instead of calling NetDevice::Send() directly. After the analysis and the process of the packet, when the backpressure mechanism allows it, TrafficControlLayer will call the Send() method on the right NetDevice.

Receiving packets

The callback chain that (in the past) involved IPv{4,6}L3Protocol and NetDevices, through ReceiveCallback, is extended to involve TrafficControlLayer. When an IPv{4,6}Interface is added in the IPv{4,6}L3Protocol, the callback chain is configured to have the following packet exchange:

NetDevice → Node → TrafficControlLayer → IPv{4,6}L3Protocol

32.1.2 Brief description of old node/device/protocol interactions

The main question that we would like to answer in the following paragraphs is: how a ns-3 node can send/receive packets?

If we analyze any example out there, the ability of the node to receive/transmit packets derives from the interaction of two helper:

- L2 Helper (something derived from NetDevice)
- L3 Helper (usually from Internet module)

L2 Helper main operations

Any good L2 Helper will do the following operations:

- Create n netdevices (n>1)
- Attach a channel between these devices
- Call Node::AddDevice ()

Obviously the last point is the most important.

Node::AddDevice (network/model/node.cc:128) assigns an interface index to the device, calls NetDevice::SetNode, sets the receive callback of the device to Node::NonPromiscReceiveFromDevice. Then, it schedules NetDevice::Initialize() method at Seconds(0), then notify the registered DeviceAdditionListener handlers (not used BY ANY-ONE).

Node::NonPromiscReceiveFromDevice calls Node::ReceiveFromDevice.

Node::ReceiveFromDevice iterates through ProtocolHandlers, which are callbacks which accept as signature:

ProtocolHandler (Ptr<NetDevice>, Ptr<const Packet>, protocol, from_addr, to_addr, packetType).

If device, protocol number and promiscuous flag corresponds, the handler is invoked.

Who is responsible to set ProtocolHandler ? We will analyze that in the next section.

L3 Helper

We have only internet which provides network protocol (IP). That module splits the operations between two helpers: InternetStackHelper and Ipv{4,6}AddressHelper.

InternetStackHelper::Install (internet/helper/internet-stack-helper.cc:423) creates and aggregates protocols {ArpL3,Ipv4L3,Icmpv4}Protocol. It creates the routing protocol, and if Ipv6 is enabled it adds {Ipv6L3,Icmpv6L4}Protocol. In any case, it instantiates and aggregates an UdpL4Protocol object, along with a PacketSocketFactory. Ultimately, it creates the required objects and aggregates them to the node.

Let's assume an Ipv4 environment (things are the same for Ipv6).

Ipv4AddressHelper::Assign (src/internet/helper/ipv4-address-helper.cc:131) registers the handlers. The process is a bit long. The method is called with a list of NetDevice. For each of them, the node and Ipv4L3Protocol pointers are retrieved; if an Ipv4Interface is already registered for the device, on that the address is set. Otherwise, the method Ipv4L3Protocol::AddInterface is called, before adding the address.

IP interfaces

In `Ipv4L3Protocol::AddInterface` (`src/internet/model/ipv4-l3-protocol.cc:300`) two protocol handlers are installed: one that react to `ipv4` protocol number, and one that react to `arp` protocol number (`Ipv4L3Protocol::Receive` and `ArpL3Protocol::Receive`, respectively). The interface is then created, initialized, and returned.

`Ipv4L3Protocol::Receive` (`src/internet/model/ipv4-l3-protocol.cc:472`) iterates through the interface. Once it finds the `Ipv4Interface` which has the same device as the one passed as argument, invokes the `rxTrace` callback. If the interface is down, the packet is dropped. Then, it removes the header and trim any residual frame padding. If checksum is not OK, it drops the packet. Otherwise, forward the packet to the raw sockets (not used). Then, it ask the routing protocol what is the destiny of that packet. The choices are: `Ipv4L3Protocol::{IpForward, IpMulticastForward, LocalDeliver, RouteInputError}`.

32.2 Queue disciplines

32.2.1 Model Description

Packets received by the Traffic Control layer for transmission to a netdevice can be passed to a queueing discipline (queue disc) to perform scheduling and policing. The *ns-3* term “queue disc” corresponds to what Linux calls a “qdisc”. A netdevice can have a single (root) queue disc installed on it. Installing a queue disc on a netdevice is not mandatory. If a netdevice does not have a queue disc installed on it, the traffic control layer sends the packets directly to the netdevice. This is the case, for instance, of the loopback netdevice.

As in Linux, queue discs may be simple queues or may be complicated hierarchical structures. A queue disc may contain distinct elements:

- queues, which actually store the packets waiting for transmission
- classes, which permit the definition of different treatments for different subdivisions of traffic
- filters, which determine the queue or class which a packet is destined to

Linux uses the terminology “classful qdiscs” or “classless qdiscs” to describe how packets are handled. This use of the term “class” should be distinguished from the C++ language “class”. In general, the below discussion uses “class” in the Linux, not C++, sense, but there are some uses of the C++ term, so please keep in mind the dual use of this term in the below text.

Notice that a child queue disc must be attached to every class and a packet filter is only able to classify packets of a single protocol. Also, while in Linux some queue discs (e.g., `fq-codel`) use an internal classifier and do not make use of packet filters, in *ns-3* every queue disc including multiple queues or multiple classes needs an external filter to classify packets (this is to avoid having the traffic-control module depend on other modules such as `internet`).

Queue disc configuration vary from queue disc to queue disc. A typical taxonomy divides queue discs in classful (i.e., support classes) and classless (i.e., do not support classes). More recently, after the appearance of multi-queue devices (such as Wi-Fi), some multi-queue aware queue discs have been introduced. Multi-queue aware queue discs handle as many queues (or queue discs – without using classes) as the number of transmission queues used by the device on which the queue disc is installed. An attempt is made, also, to classify each packet similarly in the queue disc and within the device (i.e., to keep the packet classification consistent across layers).

The traffic control layer interacts with a queue disc in a simple manner: after requesting to enqueue a packet, the traffic control layer requests the qdisc to “run”, i.e., to dequeue a set of packets, until a predefined number (“quota”) of packets is dequeued or the netdevice stops the queue disc. A netdevice shall stop the queue disc when its transmission queue does not have room for another packet. Also, a netdevice shall wake the queue disc when it detects that there is room for another packet in its transmission queue, but the transmission queue is stopped. Waking a queue disc is equivalent to make it run.

Every queue disc collects statistics about the total number of packets/bytes received from the upper layers (in case of root queue disc) or from the parent queue disc (in case of child queue disc), enqueued, dequeued, requeued, dropped, dropped before enqueue, dropped after dequeue, marked, and stored in the queue disc and sent to the netdevice or to the parent queue disc. Note that packets that are dequeued may be requeued, i.e., retained by the traffic control infrastructure, if the netdevice is not ready to receive them. Requeued packets are not part of the queue disc. The following identities hold:

- $\text{dropped} = \text{dropped before enqueue} + \text{dropped after dequeue}$
- $\text{received} = \text{dropped before enqueue} + \text{enqueued}$
- $\text{queued} = \text{enqueued} - \text{dequeued}$
- $\text{sent} = \text{dequeued} - \text{dropped after dequeue} (- 1 \text{ if there is a requeued packet})$

Separate counters are also kept for each possible reason to drop a packet. When a packet is dropped by an internal queue, e.g., because the queue is full, the reason is “Dropped by internal queue”. When a packet is dropped by a child queue disc, the reason is “(Dropped by child queue disc)” followed by the reason why the child queue disc dropped the packet.

The QueueDisc base class provides the SojournTime trace source, which provides the sojourn time of every packet dequeued from a queue disc, including packets that are dropped or requeued after being dequeued. The sojourn time is taken when the packet is dequeued from the queue disc, hence it does not account for the additional time the packet is retained within the queue disc in case it is requeued.

Design

A C++ abstract base class, class QueueDisc, is subclassed to implement a specific queue disc. A subclass is required to implement the following methods:

- `bool DoEnqueue (Ptr<QueueDiscItem> item):` Enqueue a packet
- `Ptr<QueueDiscItem> DoDequeue ():` Dequeue a packet
- `bool CheckConfig () const:` Check if the configuration is correct
- `void InitializeParams ():` Initialize queue disc parameters

and may optionally override the default implementation of the following method:

- `Ptr<const QueueDiscItem> DoPeek () const:` Peek the next packet to extract

The default implementation of the DoPeek method is based on the `qdisc_peek_dequeued` function of the Linux kernel, which dequeues a packet and retains it in the queue disc as a requeued packet. This approach is recommended especially for queue discs for which it is not obvious what is the next packet that will be dequeued (e.g., queue discs having multiple internal queues or child queue discs or queue discs that drop packets after dequeue). Therefore, unless the subclass redefines the DoPeek method, calling Peek causes the next packet to be dequeued from the queue disc, though the packet is still considered to be part of the queue disc and the dequeue trace is fired when Dequeue is called and the packet is actually extracted from the queue disc.

The C++ base class QueueDisc implements:

- methods to add/get a single queue, class or filter and methods to get the number of installed queues, classes or filters
- a `Classify` method which classifies a packet by processing the list of filters until a filter able to classify the packet is found
- methods to extract multiple packets from the queue disc, while handling transmission (to the device) failures by requeuing packets

The base class QueueDisc provides many trace sources:

- Enqueue
- Dequeue
- Requeue
- Drop
- Mark
- PacketsInQueue
- BytesInQueue

The C++ base class `QueueDisc` holds the list of attached queues, classes and filter by means of three vectors accessible through attributes (`InternalQueueList`, `QueueDiscClassList` and `PacketFilterList`).

Internal queues are implemented as (subclasses of) `Queue` objects. A `Queue` stores `QueueItem` objects, which consist of just a `Ptr<Packet>`. Since a queue disc has to store at least the destination address and the protocol number for each enqueued packet, a new C++ class, `QueueDiscItem`, is derived from `QueueItem` to store such additional information for each packet. Thus, internal queues are implemented as `Queue` objects storing `QueueDiscItem` objects. Also, there could be the need to store further information depending on the network layer protocol of the packet. For instance, for IPv4 and IPv6 packets it is needed to separately store the header and the payload, so that header fields can be manipulated, e.g., to support Explicit Congestion Notification as defined in RFC 3168. To this end, subclasses `Ipv4QueueDiscItem` and `Ipv6QueueDiscItem` are derived from `QueueDiscItem` to additionally store the IP header and provide protocol specific operations such as ECN marking.

Classes (in the Linux sense of the term) are implemented via the `QueueDiscClass` class, which consists of a pointer to the attached queue disc. Such a pointer is accessible through the `QueueDisc` attribute. Classful queue discs needing to set parameters for their classes can subclass `QueueDiscClass` and add the required parameters as attributes.

An abstract base class, `PacketFilter`, is subclassed to implement specific filters. Subclasses are required to implement two virtual private pure methods:

- `bool CheckProtocol (Ptr<QueueDiscItem> item) const`: check whether the filter is able to classify packets of the same protocol as the given packet
- `int32_t DoClassify (Ptr<QueueDiscItem> item) const`: actually classify the packet

`PacketFilter` provides a public method, `Classify`, which first calls `CheckProtocol` to check that the protocol of the packet matches the protocol of the filter and then calls `DoClassify`. Specific filters subclassed from `PacketFilter` should not be placed in the traffic-control module but in the module corresponding to the protocol of the classified packets.

32.2.2 Usage

The traffic control layer is automatically created and inserted on an `ns3::Node` object when typical device and internet module helpers are used. By default, the `InternetStackHelper::Install()` method aggregates a `TrafficControlLayer` object to every node. When invoked to assign an IPv{4,6} address to a device, the `Ipv{4,6}AddressHelper`, besides creating an `Ipv{4,6}Interface`, also installs the default qdisc on the device, unless a queue disc has been already installed. For single-queue `NetDevices` (such as `PointToPoint`, `Csma` and `Simple`), the default root qdisc is `FqCoDel`. For multi-queue `NetDevices` (such as `Wifi`), the default root qdisc is `Mq` with as many `FqCoDel` child qdiscs as the number of device queues.

To install a queue disc other than the default one, it is necessary to install such queue disc before an IP address is assigned to the device. Alternatively, the default queue disc can be removed from the device after assigning an IP address, by using the `Uninstall` method of the `TrafficControlHelper` C++ class, and then installing a different queue disc on the device. By uninstalling without adding a new queue disc, it is also possible to have no queue disc installed on a device.

Note that if no queue disc is installed on an underlying device, the traffic control layer will still respect flow control signals provided by the device, if any. Specifically, if no queue disc is installed on a device, and the device is stopped,

then any packet for that device will be dropped in the traffic control layer, and the device's drop trace will not record the drop – instead, the TcDrop drop trace in the traffic control layer will record the drop.

Flow control can be disabled for the devices that support it by using the `DisableFlowControl` method of their helpers. If there is no queue disc installed on the device, and the device is not performing flow control, then packets will immediately transit the traffic control layer and be sent to the device, regardless or not of whether the device's internal queue can accept it, and the traffic control layer's TcDrop trace will not be called.

Helpers

A typical usage pattern is to create a traffic control helper and to configure type and attributes of queue discs, queues, classes and filters from the helper. For example, the `pfifo_fast` can be configured as follows:

```
TrafficControlHelper tch;
uint16_t handle = tch.SetRootQueueDisc("ns3::PfifoFastQueueDisc");
tch.AddInternalQueues(handle, 3, "ns3::DropTailQueue", "MaxSize", StringValue("1000p"));
QueueDiscContainer qdiscs = tch.Install(devices);
```

The above code adds three internal queues to the root queue disc of type `PfifoFast`. With the above configuration, the config path of the root queue disc installed on the *j*-th device of the *i*-th node (the index of a device is the same as in `DeviceList`) is:

```
/NodeList/[i]/$ns3::TrafficControlLayer/RootQueueDiscList/[j]
```

and the config path of the second internal queue is:

```
/NodeList/[i]/$ns3::TrafficControlLayer/RootQueueDiscList/[j]/InternalQueueList/1
```

For this helper's configuration to take effect, it should be added to the ns-3 program after `InternetStackHelper::Install()` is called, but before IP addresses are configured using `Ipv{4,6}AddressHelper`. For an example program, see `examples/traffic-control/traffic-control.cc`.

If it is desired to install no queue disc on a device, it is necessary to use the `Uninstall` method of the `TrafficControlHelper`:

```
TrafficControlHelper tch;
tch.Uninstall(device);
```

Note that the `Uninstall` method must be called after `InternetStackHelper::Install()` is called and after that IP addresses are configured using `Ipv{4,6}AddressHelper`. For an example program, see `src/test/ns3tcp/ns3tcp-cwnd-test-suite.cc` (look at the `Ns3TcpCwndTestCase2::DoRun` method). Note also that this method does not uninstall the traffic control layer but instead removes the root queue disc on the device but keeps the traffic control layer present. Also, note that removing the root queue disc on a device supporting flow control does not disable the flow control. As mentioned above, this requires to call the `DisableFlowControl` method of the device helper, so that the device is created without support for the flow control.

32.2.3 Implementation details

In Linux, the struct `netdev_queue` is used to store information about a single transmission queue of a device: status (i.e., whether it has been stopped or not), data used by techniques such as Byte Queue Limits and a `qdisc` pointer field that is mainly used to solve the following problems:

- if a device transmission queue is (almost) empty, identify the queue disc to wake
- if a packet will be enqueued in a given device transmission queue, identify the queue disc which the packet must be enqueued into

The latter problem arises because Linux attempts to determine the device transmission queue which a packet will be enqueued into before passing the packet to a queue disc. This is done by calling a specific function of the device driver, if implemented, or by employing fallback mechanisms (such as hashing of the addresses) otherwise. The identifier of the selected device transmission queue is stored in the `queue_mapping` field of the struct `sk_buff`, so that both the queue disc and the device driver can get the same information. In ns-3, such identifier is stored in a member of the `QueueDiscItem` class.

The `NetDeviceQueue` class in ns-3 is the equivalent of the Linux struct `netdev_queue`. The `qdisc` field of the Linux struct `netdev_queue`, however, cannot be similarly stored in a `NetDeviceQueue` object, because it would make the network module depend on the traffic-control module. Instead, this information is stored in the `TrafficControlLayer` object aggregated to each node. In particular, a `TrafficControlLayer` object holds a struct `NetDeviceInfo` which stores, for each `NetDevice`, a pointer to the root queue disc installed on the device, a pointer to the netdevice queue interface (see below) aggregated to the device, and a vector of pointers (one for each device transmission queue) to the queue discs to activate when the above problems occur. The traffic control layer takes care of configuring such a vector at initialization time, based on the “wake mode” of the root queue disc. If the wake mode of the root queue disc is `WAKE_ROOT`, then all the elements of the vector are pointers to the root queue disc. If the wake mode of the root queue disc is `WAKE_CHILD`, then each element of the vector is a pointer to a distinct child queue disc. This requires that the number of child queue discs matches the number of netdevice queues. It follows that the wake mode of a classless queue disc must necessarily be `WAKE_ROOT`. These two configurations are illustrated by the figures below.

Setup of a queue disc (wake mode: `WAKE_ROOT`) below shows how the `TrafficControlLayer` map looks like in case of a classful root queue disc whose wake mode is `WAKE_ROOT`.

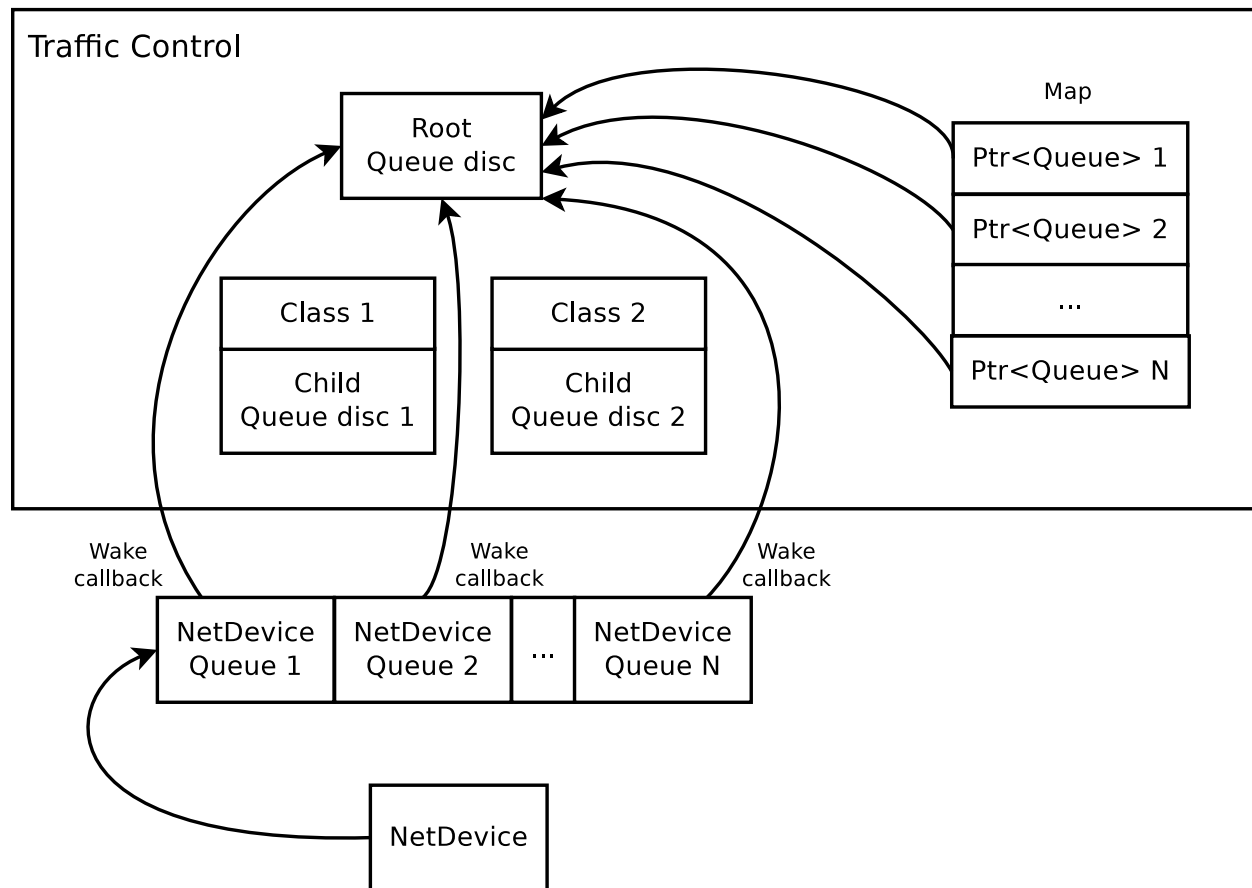


Fig. 32.1: Setup of a queue disc (wake mode: `WAKE_ROOT`)

Setup of a multi-queue aware queue disc below shows instead how the `TrafficControlLayer` map looks like in case of a

classful root queue disc whose wake mode is WAKE_CHILD.

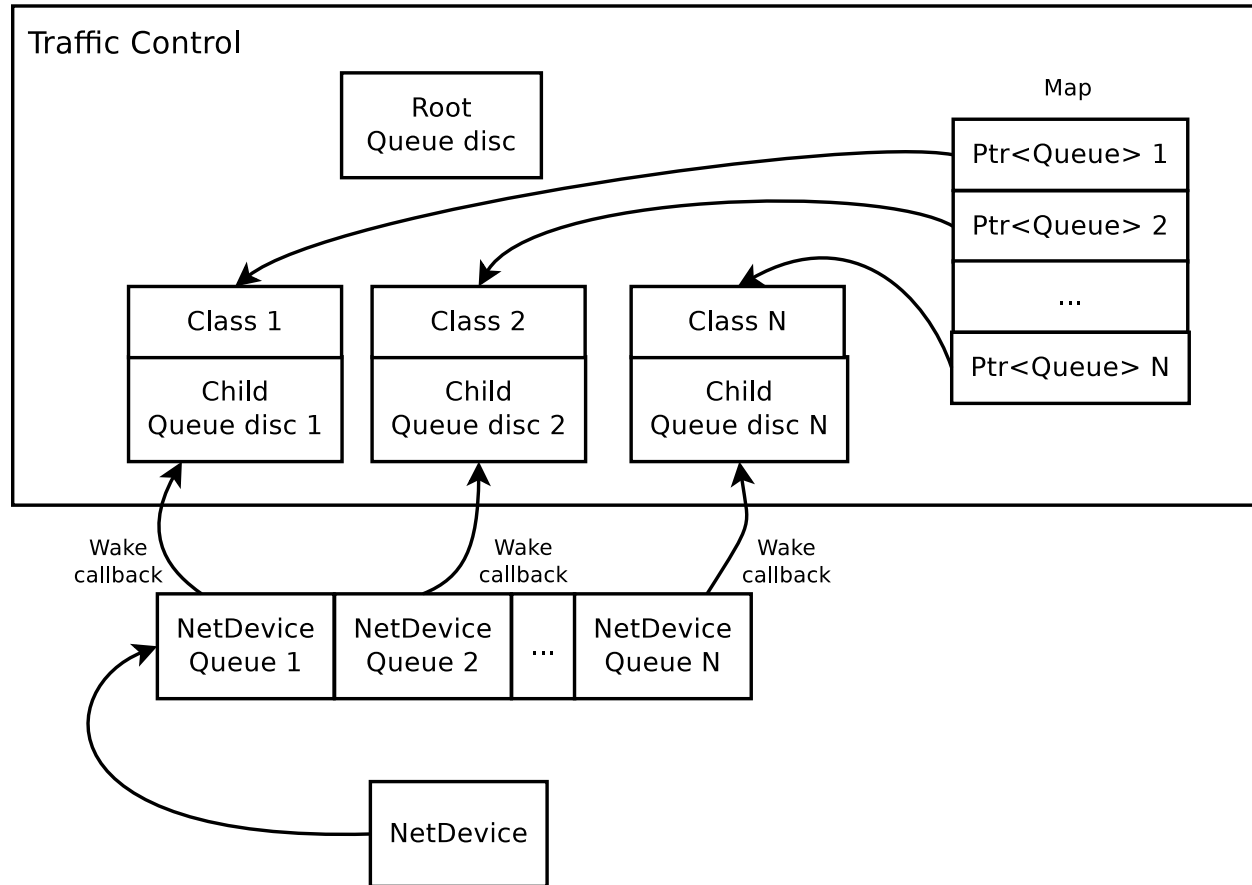


Fig. 32.2: Setup of a multi-queue aware queue disc

A `NetDeviceQueueInterface` object is used by the traffic control layer to access the information stored in the `NetDeviceQueue` objects, retrieve the number of transmission queues of the device and get the transmission queue selected for the transmission of a given packet. A `NetDeviceQueueInterface` object must be therefore aggregated to all the devices having an interface supporting the traffic control layer (i.e., an IPv4 or IPv6 interface). In particular:

- a `NetDeviceQueueInterface` object is aggregated to all the devices by the `NetDevice` helper classes, at `Install` time. See, for example, the implementation in the method `CsmaHelper::InstallPriv()`.
- when notified that a netdevice queue interface has been aggregated, traffic control aware devices can cache the pointer to the netdevice queue interface created by the traffic control layer into a member variable. Also, multi-queue devices can set the number of device transmission queues and set the select queue callback through the netdevice queue interface
- at initialization time, the traffic control (after calling `device->Initialize()` to ensure that the netdevice has set the number of device transmission queues, if it has to do so) completes the installation of the queue discs by setting the wake callbacks on the device transmission queues (through the netdevice queue interface). Also, the traffic control calls the `Initialize` method of the root queue discs. This initialization of queue discs triggers calls to the `CheckConfig` and `InitializeParams` methods of the queue disc.

Requeue

In Linux, a packet dequeued from a queue disc can be requeued (i.e., stored somewhere and sent to the device at a later time) in some circumstances. Firstly, the function used to dequeue a packet (`dequeue_skb`) actually dequeues a packet only if the device is multi-queue or the (unique) device queue is not stopped. If a packet has been dequeued from the queue disc, it is passed to the `sch_direct_xmit` function for transmission to the device. This function checks whether the device queue the packet is destined to is stopped, in which case the packet is requeued. Otherwise, the packet is sent to the device. If the device returns `NETDEV_TX_BUSY`, the packet is requeued. However, it is advised that the function called to send a packet to the device (`ndo_start_xmit`) should always return `NETDEV_TX_OK`, which means that the packet is consumed by the device driver and thus needs not to be requeued. However, the `ndo_start_xmit` function of the device driver is allowed to return `NETDEV_TX_BUSY` (and hence the packet is requeued) when there is no room for the received packet in the device queue, despite the queue is not stopped. This case is considered as a corner case or an hard error, and should be avoided.

ns-3 implements the requeue mechanism in a similar manner, the only difference being that packets are not requeued when such corner cases occur. Basically, the method used to dequeue a packet (`QueueDisc::DequeuePacket`) actually dequeues a packet only if the device is multi-queue or the (unique) device queue is not stopped. If a packet has been dequeued from the queue disc, it is passed to the `QueueDisc::Transmit` method for transmission to the device. This method checks whether the device queue the packet is destined to is stopped, in which case the packet is requeued. Otherwise, the packet is sent to the device. We request netdevices to stop a device queue when it is not able to store another packet, so as to avoid the situation in which a packet is received that cannot be enqueued while the device queue is not stopped. Should such a corner case occur, the netdevice drops the packet but, unlike Linux, the value returned by `NetDevice::Send` is ignored and the packet is not requeued.

The way the requeue mechanism is implemented in ns-3 has the following implications:

- if the underlying device has a single queue, no packet will ever be requeued. Indeed, if the device queue is not stopped when `QueueDisc::DequeuePacket` is called, it will not be stopped also when `QueueDisc::Transmit` is called, hence the packet is not requeued (recall that a packet is not requeued after being sent to the device, as the value returned by `NetDevice::Send` is ignored).
- if the underlying device does not implement flow control, i.e., it does not stop its queue(s), no packet will ever be requeued (recall that a packet is only requeued by `QueueDisc::Transmit` when the device queue the packet is destined to is stopped)

It turns out that packets may only be requeued when the underlying device is multi-queue and supports flow control.

32.3 Fifo queue disc

32.3.1 Model Description

`FifoQueueDisc` implements the FIFO (First-In First-Out) policy. Packets are enqueued in the unique internal queue, which is implemented as a `DropTail` queue. The queue disc capacity can be specified in terms of either packets or bytes, depending on the value of the `Mode` attribute.

User is allowed to provide an internal queue before the queue disc is initialized. If no internal queue is provided, one `DropTail` queue having the same capacity of the queue disc is created by default. No packet filter can be added to a `FifoQueueDisc`.

Attributes

The `FifoQueueDisc` class holds the following attribute:

- **MaxSize**: The maximum number of packets/bytes the queue disc can hold. The default value is 1000 packets.

32.3.2 Validation

The `fifo` model is tested using `FifoQueueDiscTestSuite` class defined in `src/traffic-control/test/fifo-queue-disc-test-suite.cc`. The test aims to check that the capacity of the queue disc is not exceeded and packets are dequeued in the correct order.

32.4 pfifo_fast queue disc

32.4.1 Model Description

`PfifofastQueueDisc` behaves like `pfifo_fast`, which is the default queue disc enabled on Linux systems (init systems such as `systemd` may override such default setting). Packets are enqueued in three priority bands (implemented as FIFO droptail queues) based on their priority (users can read *Use of `Send()` vs. `SendTo()`* for details on how to set packet priority). The four least significant bits of the priority are used to determine the selected band according to the following table:

Priority & 0xf	Band
0	1
1	2
2	2
3	2
4	1
5	2
6	0
7	0
8	1
9	1
10	1
11	1
12	1
13	1
14	1
15	1

The system behaves similar to three `ns3::DropTail` queues operating together, in which packets from higher priority bands are always dequeued before a packet from a lower priority band is dequeued.

The queue disc capacity, i.e., the maximum number of packets that can be enqueued in the queue disc, is set through the `MaxSize` attribute, which plays the same role as `txqueuelen` in Linux. If no internal queue is provided, three `DropTail` queues having each a capacity equal to `MaxSize` are created by default. User is allowed to provide queues, but they must be three, operate in packet mode and each have a capacity not less than `MaxSize`. No packet filter can be added to a `PfifofastQueueDisc`.

Attributes

The `PfifoFastQueueDisc` class holds a single attribute:

- **MaxSize:** The maximum number of packets accepted by the queue disc. The default value is 1000.

Examples

Various examples located in `src/traffic-control/examples` (e.g., `code1-vs-pfifo-asymmetric.cc`) shows how to configure and install a `PfifoFastQueueDisc` on internet nodes.

32.4.2 Validation

The `pfifo_fast` model is tested using `PfifoFastQueueDiscTestSuite` class defined in `src/test/ns3tc/pfifo-fast-queue-disc-test-suite.cc`. The suite includes 4 test cases:

- Test 1: The first test checks whether IPv4 packets are enqueued in the correct band based on the TOS byte
- Test 2: The second test checks whether IPv4 packets are enqueued in the correct band based on the TOS byte
- Test 3: The third test checks that the queue disc cannot enqueue more packets than its limit
- Test 4: The fourth test checks that packets that the filters have not been able to classify are enqueued into the default band of 1

32.5 Prio queue disc

32.5.1 Model Description

`PrioQueueDisc` implements a strict priority policy, where packets are dequeued from a band only if higher priority bands are all empty. `PrioQueueDisc` is a classful queue disc and can have an arbitrary number of bands, each of which is handled by a queue disc of any kind. The capacity of `PrioQueueDisc` is not limited; packets can only be dropped by child queue discs (which may have a limited capacity). If no packet filter is installed or able to classify a packet, then the packet is enqueued into a priority band based on its priority (modulo 16), which is used as an index into an array called `priomap`. Users can read *Use of `Send()` vs. `SendTo()`* for details on how to set the packet priority. If a packet is classified by an installed packet filter and the returned value `i` is non-negative and less than the number of priority bands, then the packet is enqueued into the `i`-th priority band. Otherwise, the packet is enqueued into the priority band specified by the first element of the `priomap` array.

If no queue disc class is added by the user before the queue disc is initialized, three child queue discs of type `FifoQueueDisc` are automatically added. It has to be noted that `PrioQueueDisc` needs at least two child queue discs.

Attributes

The `PrioQueueDisc` class holds the following attribute:

- **Priomap:** The priority to band mapping. The default value is the same mapping as the (fixed) one used by `PfifoFastQueueDisc`.

Examples

An example of how to configure PrioQueueDisc with custom child queue discs and priomap is provided by *queue-discs-benchmark.cc* located in `examples/traffic-control`:

```
TrafficControlHelper tch;
uint16_t handle = tch.SetRootQueueDisc("ns3::PrioQueueDisc", "Priomap", StringValue("0 1_
↪ 0 1 0 1 0 1 0 1 0 1 0 1"));
TrafficControlHelper::ClassIdList cid = tch.AddQueueDiscClasses(handle, 2,
↪ "ns3::QueueDiscClass");
tch.AddChildQueueDisc(handle, cid[0], "ns3::FifoQueueDisc");
tch.AddChildQueueDisc(handle, cid[1], "ns3::RedQueueDisc");
```

The code above adds two classes (bands) to a PrioQueueDisc. The highest priority one is a FifoQueueDisc, the other one is a RedQueueDisc. The attribute Priomap is set to an array containing only 0 and 1 (since PrioQueueDisc only has two bands).

32.5.2 Validation

PrioQueueDisc is tested using PrioQueueDiscTestSuite class defined in `src/traffic-control/test/prio-queue-disc-test-suite.cc`. The test aims to check that: i) packets are enqueued in the correct band based on their priority and the priomap or according to the value returned by the installed packet filter; ii) packets are dequeued in the correct order.

The test suite can be run using the following commands:

```
$ ./ns3 configure --enable-examples --enable-tests
$ ./ns3 build
$ ./test.py -s prio-queue-disc
```

or

```
$ NS_LOG="PrioQueueDisc" ./ns3 run "test-runner --suite=prio-queue-disc"
```

32.6 TBF queue disc

This chapter describes the TBF ([Ref1]) queue disc implementation in *ns-3*. The TBF model in *ns-3* is ported based on Linux kernel code implemented by A. Kuznetsov and D. Torokhov.

TBF is a qdisc that allows controlling the bandwidth of the output according to a set rate with the possibility of managing burst conditions also. The TBF implementation consists of a bucket (buffer) having a limited capacity into which tokens (normally representing a unit of bytes or a single packet of predetermined size) are added at a fixed rate 'r' called the token rate. Whenever a packet arrives into the tx queue (fifo by default), the bucket is checked to see if there are appropriate number of tokens that is equivalent to the length of the packet in bytes. If yes, then the tokens are removed and the packet is passed for transmission. If no, then packets will have to wait until there are sufficient tokens in the bucket. This data conformance can be thus put into three possible scenarios [Ref3]:

1. Data rate = Token rate : Packets pass without delay.
2. Data rate < Token rate : The tokens might accumulate and the bucket might become full. Then, the next packets to enter TBF will be transmitted right away without having any limit applied to them, until the bucket is empty. This is called a burst condition and in TBF the burst parameter defines the size of the bucket. In order to overcome this problem and provide better control over the bursts, TBF implements a second bucket which is smaller and

generally the same size as the MTU. This second bucket cannot store large amount of tokens, but its replenishing rate will be a lot faster than the one of the big bucket. This second rate is called 'peakRate' and it will determine the maximum rate of a burst.

3. Data rate > Token rate : This causes the TBF algorithm to throttle itself for a while as soon as the bucket gets empty. This is called an 'overlimit situation' [Ref2]. In this situation, some of the packets will be blocked until enough tokens are available at which time a schedule for the waking of the queue will be done. If packets keep coming in, at a larger rate, then the packets will start to get dropped when the total number of bytes exceeds the QueueLimit.

32.6.1 Model Description

The TBF queue disc does not require packet filters, does not admit internal queues and uses a single child queue disc. If the user does not provide a child queue disc, a Fifo queue disc operating in the same mode (packet or byte) as the TBF queue disc and having a size equal to the TBF QueueLimit attribute is created. Otherwise, the capacity of the TBF queue disc is determined by the capacity of the child queue disc.

There are two token buckets: first bucket and second bucket. The size of the first bucket called 'Burst' should always be greater than the size of the second bucket called the Mtu (which is usually the size of a single packet). But the 'PeakRate' which is the second bucket's token rate should be always greater than the 'Rate' which is the first bucket's token rate.

If the PeakRate is zero, then the second bucket does not exist. In order to activate the second bucket, both the Mtu and PeakRate values have to be greater than zero. If the Mtu value is zero at initialization time, then if a NetDevice exists, the Mtu's value will be equal to the Mtu of the NetDevice. But if no NetDevice exists, then the QueueDisc will complain thus prompting the user to set the Mtu value.

The source code for the TBF model is located in the directory `src/traffic-control/model` and consists of 2 files `tbq-queue-disc.h` and `tbq-queue-disc.cc` defining a `TbfQueueDisc` class.

- `class TbfQueueDisc`: This class implements the main TBF algorithm:
 - `TbfQueueDisc::DoEnqueue()`: This routine enqueue's the incoming packet if the queue is not full and drops the packet otherwise.
 - `TbfQueueDisc::DoPeek()`: This routine peeks for the top item in the queue and if the queue is not empty, it returns the topmost item.
 - `TbfQueueDisc::DoDequeue()`: This routine performs the dequeuing of packets according to the following logic:
 - * It calls `TbfQueueDisc::Peek()` and calculates the size of the packet to be dequeued in bytes.
 - * Then it calculates the time difference 'delta', which is the time elapsed since the last update of tokens in the buckets.
 - * If the second bucket exists, the number of tokens are updated according to the 'PeakRate' and 'delta'.
 - * From this second bucket a number of tokens equal to the size of the packet to be dequeued is subtracted.
 - * Now the number of tokens in the first bucket are updated according to 'Rate' and 'delta'.
 - * From this first bucket a number of tokens equal to the size of the packet to be dequeued is subtracted.
 - * If after this, both the first and second buckets have tokens greater than zero, then the packet is dequeued.
 - * Else, an event to `QueueDisc::Run()` is scheduled after a time period when enough tokens will be present for the dequeue operation.

References

Attributes

The key attributes that the TbfQueueDisc class holds include the following:

- **MaxSize**: The maximum number of packets/bytes the queue disc can hold. The default value is 1000 packets.
- **Burst**: Size of the first bucket, in bytes. The default value is 125000 bytes.
- **Mtu**: Size of second bucket defaults to the MTU of the attached NetDevice, if any, or 0 otherwise.
- **Rate**: Rate at which tokens enter the first bucket. The default value is 125KB/s.
- **PeakRate**: Rate at which tokens enter the second bucket. The default value is 0KB/s, which means that there is no second bucket.

TraceSources

The TbfQueueDisc class provides the following trace sources:

- **TokensInFirstBucket**: Number of First Bucket Tokens in bytes
- **TokensInSecondBucket**: Number of Second Bucket Tokens in bytes

Examples

The example for TBF is *tbx-example.cc* located in `examples/traffic-control/`. The command to run the file (the invocation below shows the available command-line options) is:

```
.. sourcecode:: bash
```

```
$ ./ns3 run "tbx-example -PrintHelp" $ ./ns3 run "tbx-example -burst=125000 -rate=1Mbps  
-peakRate=1.5Mbps"
```

The expected output from the previous commands are traced value changes in the number of tokens in the first and second buckets.

32.6.2 Validation

The TBF model is tested using TbfQueueDiscTestSuite class defined in `src/traffic-control/test/tbf-queue-disc-test-suite.cc`. The suite includes 4 test cases:

- Test 1: Simple Enqueue/Dequeue with verification of attribute setting and subtraction of tokens from the buckets.
- Test 2: When DataRate == FirstBucketTokenRate; packets should pass smoothly.
- Test 3: When DataRate >>> FirstBucketTokenRate; some packets should get blocked and waking of queue should get scheduled.
- Test 4: When DataRate < FirstBucketTokenRate; burst condition, peakRate is set so that bursts are controlled.

The test suite can be run using the following commands:

```
.. sourcecode:: bash
```

```
$ ./ns3 configure --enable-examples --enable-tests $ ./ns3 build $ ./test.py -s tbf-queue-disc
```

or


```
.. sourcecode:: bash
```

```
$ NS_LOG="TbfQueueDisc" ./ns3 run "test-runner --suite=tbq-queue-disc"
```

32.7 RED queue disc

32.7.1 Model Description

Random Early Detection (RED) is a queue discipline that aims to provide early signals to transport protocol congestion control (e.g. TCP) that congestion is imminent, so that they back off their rate gracefully rather than with a bunch of tail-drop losses (possibly incurring TCP timeout). The model in ns-3 is a port of Sally Floyd's ns-2 RED model.

Note that, starting from ns-3.25, RED is no longer a queue variant and cannot be installed as a NetDevice queue. Instead, RED is a queue disc and must be installed in the context of the traffic control (see the examples mentioned below).

The RED queue disc does not require packet filters, does not admit child queue discs and uses a single internal queue. If not provided by the user, a DropTail queue operating in the same mode (packet or byte) as the queue disc and having a size equal to the RED MaxSize attribute is created. Otherwise, the capacity of the queue disc is determined by the capacity of the internal queue provided by the user.

Adaptive Random Early Detection (ARED)

ARED is a variant of RED with two main features: (i) automatically sets Queue weight, MinTh and MaxTh and (ii) adapts maximum drop probability. The model in ns-3 contains implementation of both the features, and is a port of Sally Floyd's ns-2 ARED model. Note that the user is allowed to choose and explicitly configure the simulation by selecting feature (i) or feature (ii), or both.

Feng's Adaptive RED

Feng's Adaptive RED is a variant of RED that adapts the maximum drop probability. The model in ns-3 contains implementation of this feature, and is a port of ns-2 Feng's Adaptive RED model.

Nonlinear Random Early Detection (NLRED)

NLRED is a variant of RED in which the linear packet dropping function of RED is replaced by a nonlinear quadratic function. This approach makes packet dropping gentler for light traffic load and aggressive for heavy traffic load.

Explicit Congestion Notification (ECN)

This RED model supports an ECN mode of operation to notify endpoints of congestion that may be developing in a bottleneck queue, without resorting to packet drops. Such a mode is enabled by setting the UseEcn attribute to true (it is false by default) and only affects incoming packets with the ECT bit set in their header. When the average queue length is between the minimum and maximum thresholds, an incoming packet is marked instead of being dropped. When the average queue length is above the maximum threshold, an incoming packet is marked (instead of being dropped) only if the UseHardDrop attribute is set to false (it is true by default).

The implementation of support for ECN marking is done in such a way as to not impose an internet module dependency on the traffic control module. The RED model does not directly set ECN bits on the header, but delegates that job to

the QueueDiscItem class. As a result, it is possible to use RED queues for other non-IP QueueDiscItems that may or may not support the Mark () method.

References

The RED queue disc aims to be close to the results cited in: S.Floyd, K.Fall <http://icir.org/floyd/papers/redsims.ps>

ARED queue implementation is based on the algorithm provided in: S. Floyd et al, <http://www.icir.org/floyd/papers/adaptiveRed.pdf>

Feng's Adaptive RED queue implementation is based on the algorithm provided in: W. C. Feng et al, <http://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=752150>

NLRED queue implementation is based on the algorithm provided in: Kaiyu Zhou et al, <http://www.sciencedirect.com/science/article/pii/S1389128606000879>

The addition of explicit congestion notification (ECN) to IP: K. K. Ramakrishnan et al, <https://tools.ietf.org/html/rfc3168>

Attributes

The RED queue contains a number of attributes that control the RED policies:

- MaxSize
- MeanPktSize
- IdlePktSize
- Wait (time)
- Gentle mode
- MinTh, MaxTh
- Queue weight
- LInterm
- LinkBandwidth
- LinkDelay
- UseEcn
- UseHardDrop

In addition to RED attributes, ARED queue requires following attributes:

- ARED (Boolean attribute. Default: false)
- AdaptMaxP (Boolean attribute to adapt m_curMaxP. Default: false)
- Target Delay (time)
- Interval (time)
- LastSet (time)
- Top (upper limit of m_curMaxP)
- Bottom (lower limit of m_curMaxP)
- Alpha (increment parameter for m_curMaxP)
- Beta (decrement parameter for m_curMaxP)

- RTT

In addition to RED attributes, Feng's Adaptive RED queue requires following attributes:

- FengAdaptive (Boolean attribute, Default: false)
- Status (status of current queue length, Default: Above)
- FengAlpha (increment parameter for m_curMaxP, Default: 3)
- FengBeta (decrement parameter for m_curMaxP, Default: 2)

The following attribute should be turned on to simulate NLRED queue disc:

- NLRED (Boolean attribute. Default: false)

Consult the ns-3 documentation for explanation of these attributes.

Simulating ARED

To switch on ARED algorithm, the attribute ARED must be set to true, as done in `src/traffic-control/examples/adaptive-red-tests.cc`:

```
Config::SetDefault("ns3::RedQueueDisc::ARED", BooleanValue(true));
```

Setting ARED to true implicitly configures both: (i) automatic setting of Queue weight, MinTh and MaxTh and (ii) adapting m_curMaxP.

NOTE: To explicitly configure (i) or (ii), set ARED attribute to false and follow the procedure described next:

To configure (i); Queue weight, MinTh and MaxTh, all must be set to 0, as done in `src/traffic-control/examples/adaptive-red-tests.cc`:

```
Config::SetDefault("ns3::RedQueueDisc::QW", DoubleValue(0.0));
Config::SetDefault("ns3::RedQueueDisc::MinTh", DoubleValue(0));
Config::SetDefault("ns3::RedQueueDisc::MaxTh", DoubleValue(0));
```

To configure (ii); AdaptMaxP must be set to true, as done in `src/traffic-control/examples/adaptive-red-tests.cc`:

```
Config::SetDefault("ns3::RedQueueDisc::AdaptMaxP", BooleanValue(true));
```

Simulating Feng's Adaptive RED

To switch on Feng's Adaptive RED algorithm, the attribute FengAdaptive must be set to true, as done in `examples/traffic-control/red-vs-fengadaptive.cc`:

```
Config::SetDefault("ns3::RedQueueDisc::FengAdaptive", BooleanValue(true));
```

Simulating NLRED

To switch on NLRED algorithm, the attribute NLRED must be set to true, as shown below:

```
Config::SetDefault("ns3::RedQueueDisc::NLRED", BooleanValue(true));
```

Examples

The RED queue example is found at `src/traffic-control/examples/red-tests.cc`.

ARED queue examples can be found at: `src/traffic-control/examples/adaptive-red-tests.cc` and `src/traffic-control/examples/red-vs-ared.cc`

Feng's Adaptive RED example can be found at: `examples/traffic-control/red-vs-fengadaptive.cc`

NLRED queue example can be found at: `examples/traffic-control/red-vs-nlred.cc`

32.7.2 Validation

The RED model has been validated and the report is currently stored at: <https://gitlab.com/nsnam/ns-3-dev/uploads/6937dac2431a0b61209728ecbfd442ac/report-red-ns3.pdf>

32.8 CoDel queue disc

This chapter describes the CoDel ([Nic12], [Nic14]) queue disc implementation in *ns-3*.

Developed by Kathleen Nichols and Van Jacobson as a solution to the bufferbloat [Buf14] problem, CoDel (Controlled Delay Management) is a queuing discipline that uses a packet's sojourn time (time in queue) to make decisions on packet drops.

Note that, starting from ns-3.25, CoDel is no longer a queue variant and cannot be installed as a NetDevice queue. Instead, CoDel is a queue disc and must be installed in the context of the traffic control (see the examples mentioned below).

32.8.1 Model Description

The source code for the CoDel model is located in the directory `src/traffic-control/model` and consists of 2 files *codel-queue-disc.h* and *codel-queue-disc.cc* defining a `CoDelQueueDisc` class and a helper `CoDelTimestampTag` class. The code was ported to *ns-3* by Andrew McGregor based on Linux kernel code implemented by Dave Täht and Eric Dumazet.

- class `CoDelQueueDisc`: This class implements the main CoDel algorithm:
 - `CoDelQueueDisc::DoEnqueue()`: This routine tags a packet with the current time before pushing it into the queue. The timestamp tag is used by `CoDelQueue::DoDequeue()` to compute the packet's sojourn time. If the queue is full upon the packet arrival, this routine will drop the packet and record the number of drops due to queue overflow, which is stored in *m_dropOverLimit*.
 - `CoDelQueueDisc::ShouldDrop()`: This routine is `CoDelQueueDisc::DoDequeue()`'s helper routine that determines whether a packet should be dropped or not based on its sojourn time. If the sojourn time goes above *m_target* and remains above continuously for at least *m_interval*, the routine returns `true` indicating that it is OK to drop the packet. Otherwise, it returns `false`.

- `CoDelQueueDisc::DoDequeue()`: This routine performs the actual packet drop based on `CoDelQueueDisc::ShouldDrop()`'s return value and schedules the next drop/mark.
- `class CoDelTimestampTag`: This class implements the timestamp tagging for a packet. This tag is used to compute the packet's sojourn time (the difference between the time the packet is dequeued and the time it is pushed into the queue).

There are 2 branches to `CoDelQueueDisc::DoDequeue()`:

1. If the queue is currently in the dropping state, which means the sojourn time has remained above m_target for more than $m_interval$, the routine determines if it's OK to leave the dropping state or it's time for the next drop/mark. When `CoDelQueueDisc::ShouldDrop()` returns false, the queue can move out of the dropping state (set $m_dropping$ to false). Otherwise, the queue continuously drops/marks packets and updates the time for next drop ($m_dropNext$) until one of the following conditions is met:
 1. The queue is empty, upon which the queue leaves the dropping state and exits `CoDelQueueDisc::ShouldDrop()` routine;
 2. `CoDelQueueDisc::ShouldDrop()` returns false (meaning the sojourn time goes below m_target) upon which the queue leaves the dropping state;
 3. It is not yet time for next drop/mark ($m_dropNext$ is less than current time) upon which the queue waits for the next packet dequeue to check the condition again.
2. If the queue is not in the dropping state, the routine enters the dropping state and drop/mark the first packet if `CoDelQueueDisc::ShouldDrop()` returns true (meaning the sojourn time has gone above m_target for at least $m_interval$ for the first time or it has gone above again after the queue leaves the dropping state).

The CoDel queue disc does not require packet filters, does not admit child queue discs and uses a single internal queue. If not provided by the user, a DropTail queue operating in the same mode (packet or byte) as the queue disc and having a size equal to the CoDel MaxSize attribute is created. Otherwise, the capacity of the queue disc is determined by the capacity of the internal queue provided by the user.

References

Attributes

The key attributes that the `CoDelQueue` class holds include the following:

- **MaxSize**: The maximum number of packets/bytes the queue can hold. The default value is `1500 * DEFAULT_CODEL_LIMIT`, which is `1500 * 1000` bytes.
- **MinBytes**: The CoDel algorithm minbytes parameter. The default value is 1500 bytes.
- **Interval**: The sliding-minimum window. The default value is 100 ms.
- **Target**: The CoDel algorithm target queue delay. The default value is 5 ms.
- **UseEcn**: True to use ECN (packets are marked instead of being dropped). The default value is false.
- **CeThreshold**: The CoDel CE threshold for marking packets. Disabled by default.

Examples

The first example is *codel-vs-pfifo-basic-test.cc* located in *src/traffic-control/examples*. To run the file (the first invocation below shows the available command-line options):

```
$ ./ns3 run "codel-vs-pfifo-basic-test --PrintHelp"
$ ./ns3 run "codel-vs-pfifo-basic-test --queueType=CoDel --pcapFileName=codel.pcap --
↪ cwndTrFileName=cwndCoDel.tr"
```

The expected output from the previous commands are two files: *codel.pcap* file and *cwndCoDel.tr* (ASCII trace) file. The .pcap file can be analyzed using Wireshark or tcptrace:

```
$ tcptrace -l -r -n -W codel.pcap
```

The second example is *codel-vs-pfifo-asymmetric.cc* located in *src/traffic-control/examples*. This example is intended to model a typical cable modem deployment scenario. To run the file:

```
$ ./ns3 run "codel-vs-pfifo-asymmetric --PrintHelp"
$ ./ns3 run codel-vs-pfifo-asymmetric
```

The expected output from the previous commands is six pcap files:

- codel-vs-pfifo-asymmetric-CoDel-server-lan.pcap
- codel-vs-pfifo-asymmetric-CoDel-router-wan.pcap
- codel-vs-pfifo-asymmetric-CoDel-router-lan.pcap
- codel-vs-pfifo-asymmetric-CoDel-cmts-wan.pcap
- codel-vs-pfifo-asymmetric-CoDel-cmts-lan.pcap
- codel-vs-pfifo-asymmetric-CoDel-host-lan.pcap

One attribute file:

- codel-vs-pfifo-asymmetric-CoDel.attr

Five ASCII trace files:

- codel-vs-pfifo-asymmetric-CoDel-drop.tr
- codel-vs-pfifo-asymmetric-CoDel-drop-state.tr
- codel-vs-pfifo-asymmetric-CoDel-sojourn.tr
- codel-vs-pfifo-asymmetric-CoDel-length.tr
- codel-vs-pfifo-asymmetric-CoDel-cwnd.tr

32.8.2 Validation

The CoDel model is tested using *CoDelQueueDiscTestSuite* class defined in *src/traffic-control/test/codel-queue-test-suite.cc*. The suite includes 5 test cases:

- Test 1: The first test checks the enqueue/dequeue with no drops and makes sure that CoDel attributes can be set correctly.
- Test 2: The second test checks the enqueue with drops due to queue overflow.
- Test 3: The third test checks the *NewtonStep()* arithmetic against explicit port of Linux implementation
- Test 4: The fourth test checks the *ControlLaw()* against explicit port of Linux implementation

- Test 5: The fifth test checks the enqueue/dequeue with drops according to CoDel algorithm
- Test 6: The sixth test checks the enqueue/dequeue with marks according to CoDel algorithm

The test suite can be run using the following commands:

```
$ ./ns3 configure --enable-examples --enable-tests
$ ./ns3 build
$ ./test.py -s codel-queue-disc
```

or

```
$ NS_LOG="CoDelQueueDisc" ./ns3 run "test-runner --suite=codel-queue-disc"
```

32.9 FqCoDel queue disc

This chapter describes the FqCoDel ([Hoe16]) queue disc implementation in *ns-3*.

The FlowQueue-CoDel (FQ-CoDel) algorithm is a combined packet scheduler and Active Queue Management (AQM) algorithm developed as part of the bufferbloat-fighting community effort ([Buf16]). FqCoDel classifies incoming packets into different queues (by default, 1024 queues are created), which are served according to a modified Deficit Round Robin (DRR) queue scheduler. Each queue is managed by the CoDel AQM algorithm. FqCoDel distinguishes between “new” queues (which don’t build up a standing queue) and “old” queues, that have queued enough data to be around for more than one iteration of the round-robin scheduler.

FqCoDel is installed by default on single-queue NetDevices (such as PointToPoint, Csma and Simple). Also, on multi-queue devices (such as Wifi), the default root qdisc is Mq with as many FqCoDel child queue discs as the number of device queues.

32.9.1 Model Description

The source code for the FqCoDel queue disc is located in the directory `src/traffic-control/model` and consists of 2 files *fq-codel-queue-disc.h* and *fq-codel-queue-disc.cc* defining a `FqCoDelQueueDisc` class and a helper `FqCoDelFlow` class. The code was ported to *ns-3* based on Linux kernel code implemented by Eric Dumazet. Set associative hashing is also based on the Linux kernel [CAKE](#) queue management code. Set associative hashing is used to reduce the number of hash collisions in comparison to choosing queues normally with a simple hash. For a given number of queues, set associative hashing has fewer collisions than a traditional hash, as long as the number of flows is lesser than the number of queues. Essentially, it makes the queue management system more efficient. Set associative hashing is a vital component of CAKE, which is another popular flow management algorithm that is implemented in Linux and is being tested for FqCoDel. Furthermore, this module can be directly used with CAKE when its other components are implemented in *ns-3*. The only changes needed to incorporate this new hashing scheme are in the `SetAssociativeHash` and `DoEnqueue` methods, as described below.

- class `FqCoDelQueueDisc`: This class implements the main FqCoDel algorithm:
 - `FqCoDelQueueDisc::DoEnqueue()`: If no packet filter has been configured, this routine calls the `QueueDiscItem::Hash()` method to classify the given packet into an appropriate queue. Otherwise, the configured filters are used to classify the packet. If the filters are unable to classify the packet, the packet is dropped. Otherwise, an option is provided if set associative hashing is to be used. The packet is now handed over to the CoDel algorithm for timestamping. Then, if the queue is not currently active (i.e., if it is not in either the list of new or the list of old queues), it is added to the end of the list of new queues, and its deficit is initiated to the configured quantum. Otherwise, the queue is left in its current queue list. Finally, the total number of enqueued packets is compared with the configured limit, and if it is above this value (which can happen since a packet was just enqueued), packets are dropped from the head of the queue with

the largest current byte count until the number of dropped packets reaches the configured drop batch size or the backlog of the queue has been halved. Note that this in most cases means that the packet that was just enqueued is not among the packets that get dropped, which may even be from a different queue.

- `FqCoDelQueueDisc::SetAssociativeHash()`: An outer hash is identified for the given packet. This corresponds to the set into which the packet is to be enqueued. A set consists of a group of queues. The set determined by outer hash is enumerated; if a queue corresponding to this packet's flow is found (we use per-queue tags to achieve this), or in case of an inactive queue, or if a new queue can be created for this set without exceeding the maximum limit, the index of this queue is returned. Otherwise, all queues of this full set are active and correspond to flows different from the current packet's flow. In such cases, the index of first queue of this set is returned. We don't consider creating new queues for the packet in these cases, since this approach may waste resources in the long run. The situation highlighted is a guaranteed collision and cannot be avoided without increasing the overall number of queues.
 - `FqCoDelQueueDisc::DoDequeue()`: The first task performed by this routine is selecting a queue from which to dequeue a packet. To this end, the scheduler first looks at the list of new queues; for the queue at the head of that list, if that queue has a negative deficit (i.e., it has already dequeued at least a quantum of bytes), it is given an additional amount of deficit, the queue is put onto the end of the list of old queues, and the routine selects the next queue and starts again. Otherwise, that queue is selected for dequeue. If the list of new queues is empty, the scheduler proceeds down the list of old queues in the same fashion (checking the deficit, and either selecting the queue for dequeuing, or increasing deficit and putting the queue back at the end of the list). After having selected a queue from which to dequeue a packet, the CoDel algorithm is invoked on that queue. As a result of this, one or more packets may be discarded from the head of the selected queue, before the packet that should be dequeued is returned (or nothing is returned if the queue is or becomes empty while being handled by the CoDel algorithm). Finally, if the CoDel algorithm does not return a packet, then the queue must be empty, and the scheduler does one of two things: if the queue selected for dequeue came from the list of new queues, it is moved to the end of the list of old queues. If instead it came from the list of old queues, that queue is removed from the list, to be added back (as a new queue) the next time a packet for that queue arrives. Then (since no packet was available for dequeue), the whole dequeue process is restarted from the beginning. If, instead, the scheduler did get a packet back from the CoDel algorithm, it subtracts the size of the packet from the byte deficit for the selected queue and returns the packet as the result of the dequeue operation.
 - `FqCoDelQueueDisc::FqCoDelDrop()`: This routine is invoked by `FqCoDelQueueDisc::DoEnqueue()` to drop packets from the head of the queue with the largest current byte count. This routine keeps dropping packets until the number of dropped packets reaches the configured drop batch size or the backlog of the queue has been halved.
- `class FqCoDelFlow`: This class implements a flow queue, by keeping its current status (whether it is in the list of new queues, in the list of old queues or inactive) and its current deficit.

In Linux, by default, packet classification is done by hashing (using a Jenkins hash function) the 5-tuple of IP protocol, source and destination IP addresses and port numbers (if they exist). This value modulo the number of queues is salted by a random value selected at initialization time, to prevent possible DoS attacks if the hash is predictable ahead of time. Alternatively, any other packet filter can be configured. In *ns-3*, packet classification is performed in the same way as in Linux. Neither internal queues nor classes can be configured for an FqCoDel queue disc.

Possible next steps

- what to do if ECT(1) and either/both ECT(0) and NotECT are in the same flow queue (hash collisions or tunnels)—our L4S traffic flows will avoid this situation by supporting AccECN and ECN++ (and if it happens in practice, the CoDel logic will just apply two separate thresholds)
- adding a ramp marking response instead of step threshold
- adding a floor value (to suppress marks if the queue length is below a certain number of bytes or packets)
- adding a heuristic such as in PIE to avoid marking a packet if it arrived to an empty flow queue (check on ingress, remember at egress time)

References

Attributes

The key attributes that the `FqCoDelQueue` class holds include the following:

- `UseEcn`: True to use ECN (packets are marked instead of being dropped)
- `Interval`: The interval parameter to be used on the CoDel queues. The default value is 100 ms.
- `Target`: The target parameter to be used on the CoDel queues. The default value is 5 ms.
- `MaxSize`: The limit on the maximum number of packets stored by FqCoDel.
- `Flows`: The number of flow queues managed by FqCoDel.
- `DropBatchSize`: The maximum number of packets dropped from the fat flow.
- `Perturbation`: The salt used as an additional input to the hash function used to classify packets.
- `CeThreshold`: The FqCoDel CE threshold for marking packets
- `UseL4s`: True to use L4S (only ECT1 packets are marked at CE threshold). This corresponds to the Linux `ce_threshold_selector` setting of 0x01/0x01 (https://man7.org/linux/man-pages/man8/tc-fq_codel.8.html).
- `EnableSetAssociativeHash`: The parameter used to enable set associative hash.

Perturbation is an optional configuration attribute and can be used to generate different hash outcomes for different inputs. For instance, the tuples used as input to the hash may cause hash collisions (mapping to the same bucket) for a given set of inputs, but by changing the perturbation value, the same hash inputs now map to distinct buckets.

Note that the quantum, i.e., the number of bytes each queue gets to dequeue on each round of the scheduling algorithm, is set by default to the MTU size of the device (at initialisation time). The `FqCoDelQueueDisc::SetQuantum()` method can be used (at any time) to configure a different value.

Examples

A typical usage pattern is to create a traffic control helper and to configure type and attributes of queue disc and filters from the helper. For example, FqCodel can be configured as follows:

```
TrafficControlHelper tch;
tch.SetRootQueueDisc("ns3::FqCoDelQueueDisc", "DropBatchSize", UIntegerValue(1)
                    "Perturbation", UIntegerValue(256));
QueueDiscContainer qdiscs = tch.Install(devices);
```

The example for FqCoDel's L4S mode is *FqCoDel-L4S-example.cc* located in `src/traffic-control/examples`. To run the file (the first invocation below shows the available command-line options):

```
$ ./ns3 run "FqCoDel-L4S-example --PrintHelp"  
$ ./ns3 run "FqCoDel-L4S-example --scenarioNum=5"
```

The expected output from the previous command are .dat files.

32.9.2 Validation

The FqCoDel model is tested using `FqCoDelQueueDiscTestSuite` class defined in `src/test/ns3tc/codel-queue-test-suite.cc`. The suite includes 5 test cases:

- Test 1: The first test checks that packets that cannot be classified by any available filter are dropped.
- Test 2: The second test checks that IPv4 packets having distinct destination addresses are enqueued into different flow queues. Also, it checks that packets are dropped from the fat flow in case the queue disc capacity is exceeded.
- Test 3: The third test checks the dequeue operation and the deficit round robin-based scheduler.
- Test 4: The fourth test checks that TCP packets with distinct port numbers are enqueued into different flow queues.
- Test 5: The fifth test checks that UDP packets with distinct port numbers are enqueued into different flow queues.
- Test 6: The sixth test checks that the packets are marked correctly.
- Test 7: The seventh test checks the working of set associative hashing and its linear probing capabilities by using TCP packets with different hashes enqueued into different sets and queues.
- Test 8: The eighth test checks the L4S mode of FqCoDel where ECT1 packets are marked at CE threshold (target delay does not matter) while ECT0 packets continue to be marked at target delay (CE threshold does not matter).

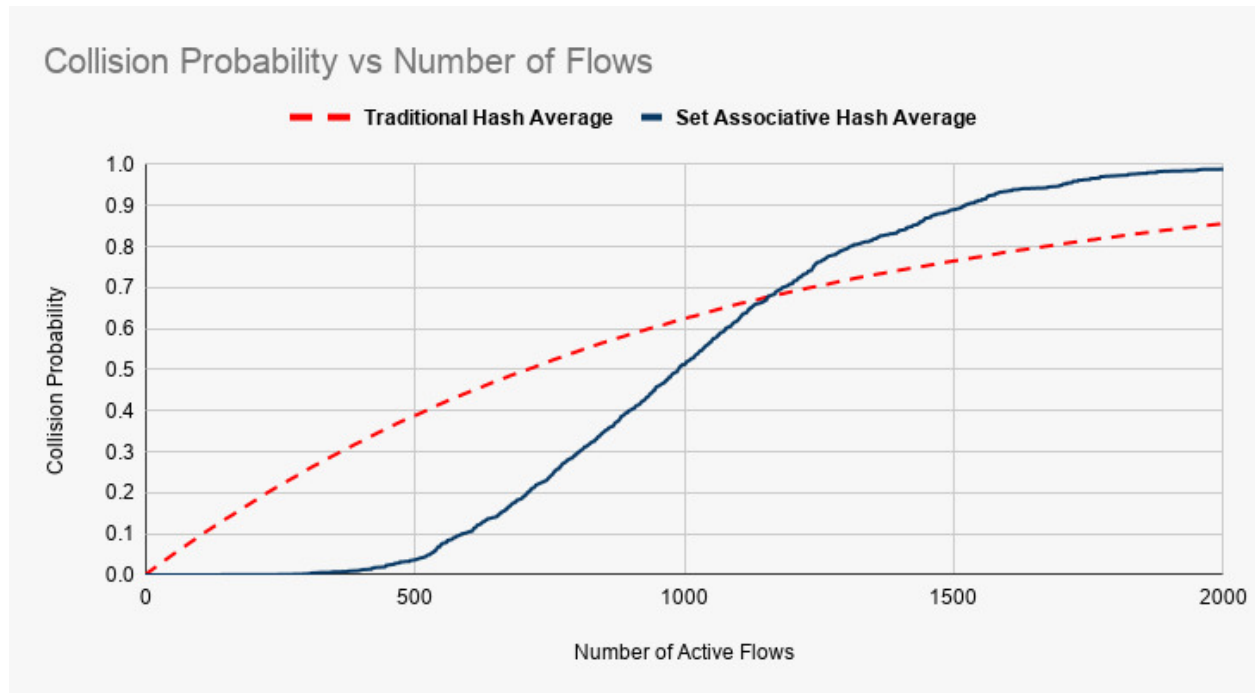
The test suite can be run using the following commands:

```
$ ./ns3 configure --enable-examples --enable-tests  
$ ./ns3 build  
$ ./test.py -s fq-codel-queue-disc
```

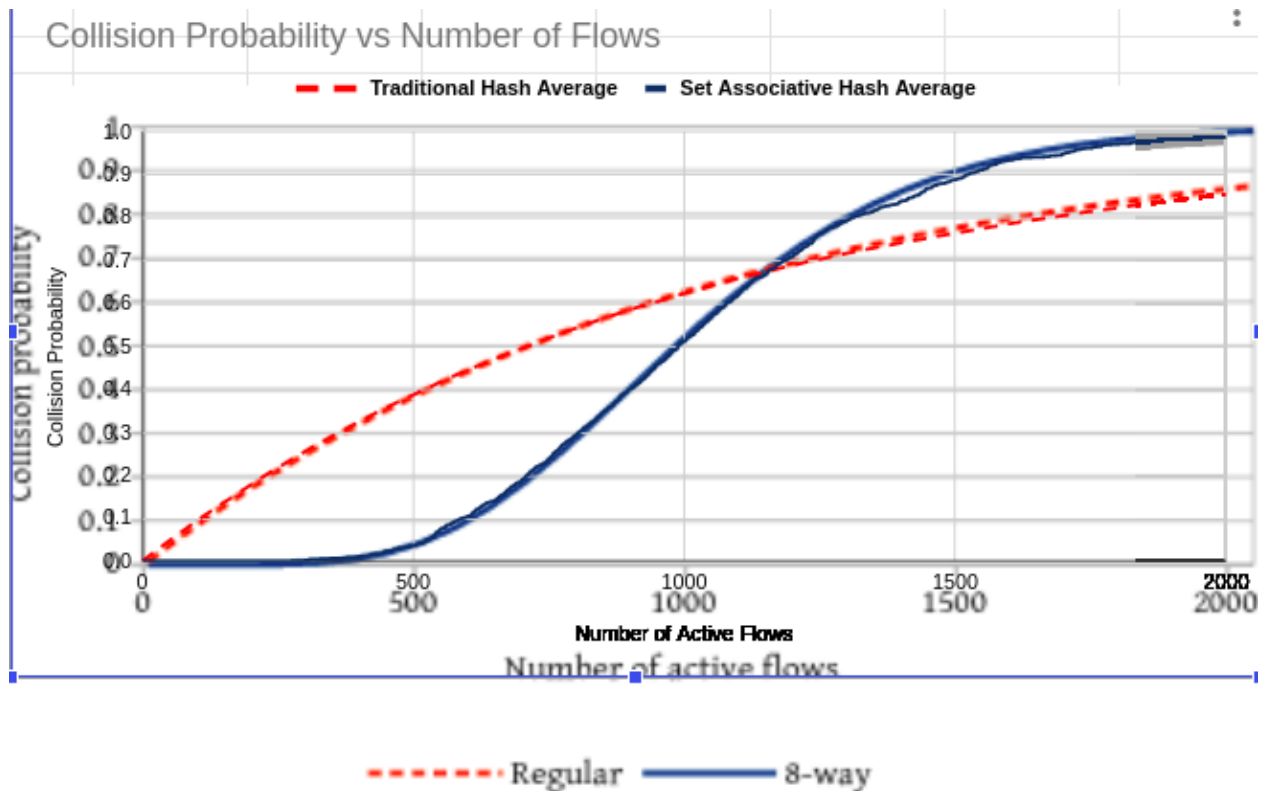
or:

```
$ NS_LOG="FqCoDelQueueDisc" ./ns3 run "test-runner --suite=fq-codel-queue-disc"
```

Set associative hashing is tested by generating a probability collision graph. This graph is then overlapped with the theoretical graph provided in the original CAKE paper (refer to Figure 1 from [CAKE](#)). The generated graph is linked below:



The overlapped graph is also linked below:



The steps to replicate this graph are available on this [link](#).

32.10 Cobalt queue disc

This chapter describes the COBALT (CoDel BLUE Alternate) ([Cake16]) queue disc implementation in *ns-3*.

COBALT queue disc is an integral component of CAKE smart queue management system. It is a combination of the CoDel ([Kath17]) and BLUE ([BLUE02]) Active Queue Management algorithms.

32.10.1 Model Description

The source code for the COBALT model is located in the directory `src/traffic-control/model` and consists of 2 files: *cobalt-queue-disc.h* and *cobalt-queue-disc.cc* defining a `CobaltQueueDisc` class and a helper `CobaltTimestampTag` class. The code was ported to *ns-3* by Vignesh Kanan, Harsh Lara, Shefali Gupta, Jendaipou Palmei and Mohit P. Tahiliani based on the Linux kernel code.

Stefano Avallone and Pasquale Imputato helped in verifying the correctness of COBALT model in *ns-3* by comparing the results obtained from it to those obtained from the Linux model of COBALT. A detailed comparison of *ns-3* model of COBALT with Linux model of COBALT is provided in ([Cobalt19]).

- `class CobaltQueueDisc`: This class implements the main Cobalt algorithm:
- `CobaltQueueDisc::DoEnqueue ()`: This routine tags a packet with the current time before pushing it into the queue. The timestamp tag is used by `CobaltQueue::DoDequeue()` to compute the packet's sojourn time. If the queue is full upon the packet arrival, this routine will drop the packet and record the number of drops due to queue overflow, which is stored in `m_stats.qLimDrop`.
- `CobaltQueueDisc::ShouldDrop ()`: This routine is `CobaltQueueDisc::DoDequeue()`'s helper routine that determines whether a packet should be dropped or not based on its sojourn time. If L4S mode is enabled then if the packet is ECT1 is checked and if delay is greater than CE threshold then the packet is marked and returns false. If the sojourn time goes above `m_target` and remains above continuously for at least `m_interval`, the routine returns true indicating that it is OK to drop the packet. Otherwise, it returns false. If L4S mode is turned off and CE threshold marking is enabled, then if the delay is greater than CE threshold, packet is marked. This routine decides if a packet should be dropped based on the dropping state of CoDel and drop probability of BLUE. The idea is to have both algorithms running in parallel and their effectiveness is decided by their respective parameters (Pdrop of BLUE and dropping state of CoDel). If either of them decide to drop the packet, the packet is dropped.
- `CobaltQueueDisc::DoDequeue ()`: This routine performs the actual packet drop based on `CobaltQueueDisc::ShouldDrop()`'s return value and schedules the next drop. Cobalt will decrease BLUE's drop probability if the queue is empty. This will ensure that the queue does not underflow. Otherwise Cobalt will take the next packet from the queue and calculate its drop state by running CoDel and BLUE in parallel till there are none left to drop.

References

Attributes

The key attributes that the `CobaltQueue Disc` class holds include the following:

- `MaxSize`: The maximum number of packets/bytes accepted by this queue disc.
- `Interval`: The sliding-minimum window. The default value is 100 ms.
- `Target`: The Cobalt algorithm target queue delay. The default value is 5 ms.
- `Pdrop`: Value of drop probability.
- `Increment`: Increment value of drop probability. Default value is 1./256 .

- **Decrement**: Decrement value of drop probability. Default value is 1./4096 .
- **CeThreshold**: The CoDel CE threshold for marking packets.
- **UseL4s**: True to use L4S (only ECT1 packets are marked at CE threshold).
- **Count**: Cobalt count.
- **DropState**: Dropping state of Cobalt. Default value is false.
- **Sojourn**: Per packet time spent in the queue.
- **DropNext**: Time until next packet drop.

Examples

An example program named *cobalt-vs-codel.cc* is located in `src/traffic-control/examples`. Use the following command to run the program.

```
$ ./ns3 run cobalt-vs-codel
```

32.10.2 Validation

The COBALT model is tested using `CobaltQueueDiscTestSuite` class defined in *src/traffic-control/test/cobalt-queue-test-suite.cc*. The suite includes 2 test cases:

- Test 1: Simple enqueue/dequeue with no drops.
- Test 2: Change of BLUE's drop probability upon queue full (Activation of Blue).
- Test 3: This test verifies ECN marking.
- Test 4: CE threshold marking test.

The test suite can be run using the following commands:

```
$ ./ns3 configure --enable-examples --enable-tests
$ ./ns3 build
$ ./test.py -s cobalt-queue-disc
```

or

```
$ NS_LOG="CobaltQueueDisc" ./ns3 run "test-runner --suite=cobalt-queue-disc"
```

32.11 FqCobalt queue disc

This chapter describes the FqCobalt ([Pal19]) queue disc implementation in *ns-3*.

The FlowQueue-Cobalt (FQ-Cobalt) algorithm is similar to FlowQueue-CoDel (FQ-CoDel) algorithm available in *FqCoDel queue disc*. The documentation for Cobalt is available in `ns-3-dev/src/traffic-control/doc/cobalt.rst`.

FqCobalt is one of the key components of the CAKE smart queue management framework ([Hoe18]). The COBALT AQM is preferred to the CoDel AQM for CAKE because it adds a heuristic called BLUE to cover cases in which the CoDel control law is too sluggish to respond to queue growth.

32.11.1 Model Description

The source code for the FqCobalt queue disc is located in the directory `src/traffic-control/model` and consists of 2 files `fq-cobalt-queue-disc.h` and `fq-cobalt-queue-disc.cc` defining a `FqCobaltQueueDisc` class and a helper `FqCobaltFlow` class. The code was ported to *ns-3* based on Linux kernel code implemented by Jonathan Morton (https://github.com/torvalds/linux/blob/master/net/sched/sch_cake.c).

The Model Description is similar to the FqCoDel documentation mentioned above.

References

Attributes

Most of the key attributes are similar to the FqCoDel implementation mentioned above. One difference is the absence of the `MinBytes` parameter.

Some additional parameters implemented as attributes are:

- `Pdrop`: Value of drop probability.
- `Increment`: Increment value of drop probability. Default value is `1./256`.
- `Decrement`: Decrement value of drop probability. Default value is `1./4096`.
- `BlueThreshold`: The threshold after which Blue is enabled. Default value is `400ms`.

Note that if the user wants to disable Blue Enhancement then the user can set it to a large value; for example, to `Time::Max()`.

Examples

A typical usage pattern is to create a traffic control helper and to configure the type and attributes of the queue disc and filters from the helper. For example, *FqCobalt* can be configured as follows:

```
TrafficControlHelper tch;
tch.SetRootQueueDisc("ns3::FqCobaltQueueDisc",
                    "DropBatchSize", UintegerValue(1),
                    "Perturbation", UintegerValue(256));
QueueDiscContainer qdiscs = tch.Install(devices);
```

32.11.2 Validation

The FqCobalt model is tested using `FqCobaltQueueDiscTestSuite` class defined in `src/test/ns3tc/codel-queue-test-suite.cc`.

The tests are similar to the ones for FqCoDel queue disc mentioned in first section of this document. The test suite can be run using the following commands:

```
$ ./ns3 configure --enable-examples --enable-tests
$ ./ns3 build
$ ./test.py -s fq-cobalt-queue-disc
```

or:

```
$ NS_LOG="FqCobaltQueueDisc" ./ns3 run "test-runner --suite=fq-cobalt-queue-disc"
```

32.12 PIE queue disc

This chapter describes the PIE ([Pan13], [Pan16]) queue disc implementation in *ns-3*.

Proportional Integral controller Enhanced (PIE) is a queuing discipline that aims to solve the bufferbloat [Buf14] problem. The model in *ns-3* is a port of Preethi Natarajan's *ns-2* PIE model.

32.12.1 Model Description

The source code for the PIE model is located in the directory `src/traffic-control/model` and consists of 2 files *pie-queue-disc.h* and *pie-queue-disc.cc* defining a `PieQueueDisc` class. The code was ported to *ns-3* by Mohit P. Tahiliani, Shravya K. S. and Smriti Murali based on *ns-2* code implemented by Preethi Natarajan, Rong Pan, Chiara Piglion, Greg White and Takashi Hayakawa. The implementation was aligned with RFC 8033 by Vivek Jain and Mohit P. Tahiliani for the *ns-3.32* release, with additional unit test cases contributed by Bhaskar Kataria.

- `class PieQueueDisc`: This class implements the main PIE algorithm:
 - `PieQueueDisc::DoEnqueue()`: This routine checks whether the queue is full, and if so, drops the packets and records the number of drops due to queue overflow. If queue is not full then if `ActiveThreshold` is set then it checks if queue delay is higher than `ActiveThreshold` and if it is then, this routine calls `PieQueueDisc::DropEarly()`, and depending on the value returned, the incoming packet is either enqueued or dropped.
 - `PieQueueDisc::DropEarly()`: The decision to enqueue or drop the packet is taken by invoking this routine, which returns a boolean value; false indicates enqueue and true indicates drop.
 - `PieQueueDisc::CalculateP()`: This routine is called at a regular interval of *m_tUpdate* and updates the drop probability, which is required by `PieQueueDisc::DropEarly()`
 - `PieQueueDisc::DoDequeue()`: This routine calculates queue delay using timestamps (by default) or, optionally with the *UseDequeueRateEstimator* attribute enabled, calculates the average departure rate to estimate queue delay. A queue delay estimate required for updating the drop probability in `PieQueueDisc::CalculateP()`. Starting with the *ns-3.32* release, the default approach to calculate queue delay has been changed to use timestamps.

References

Attributes

The key attributes that the `PieQueue` class holds include the following:

- `MaxSize`: The maximum number of bytes or packets the queue can hold.
- `MeanPktSize`: Mean packet size in bytes. The default value is 1000 bytes.
- `Tupdate`: Time period to calculate drop probability. The default value is 30 ms.
- `Supdate`: Start time of the update timer. The default value is 0 ms.
- `DequeueThreshold`: Minimum queue size in bytes before dequeue rate is measured. The default value is 10000 bytes.
- `QueueDelayReference`: Desired queue delay. The default value is 20 ms.
- `MaxBurstAllowance`: Current max burst allowance in seconds before random drop. The default value is 0.1 seconds.
- `A`: Value of alpha. The default value is 0.125.

- B: Value of beta. The default value is 1.25.
- UseDequeueRateEstimator: Enable/Disable usage of Dequeue Rate Estimator (Default: false).
- UseEcn: True to use ECN. Packets are marked instead of being dropped (Default: false).
- MarkEcnThreshold: ECN marking threshold (Default: 10% as suggested in RFC 8033).
- UseDerandomization: Enable/Disable Derandomization feature mentioned in RFC 8033 (Default: false).
- UseCapDropAdjustment: Enable/Disable Cap Drop Adjustment feature mentioned in RFC 8033 (Default: true).
- ActiveThreshold: Threshold for activating PIE (disabled by default).

Examples

The example for PIE is *pie-example.cc* located in *src/traffic-control/examples*. To run the file (the first invocation below shows the available command-line options):

```
$ ./ns3 run "pie-example --PrintHelp"  
$ ./ns3 run "pie-example --writePcap=1"
```

The expected output from the previous commands are ten .pcap files.

32.12.2 Validation

The PIE model is tested using *PieQueueDiscTestSuite* class defined in *src/traffic-control/test/pie-queue-test-suite.cc*. The suite includes the following test cases:

- Test 1: simple enqueue/dequeue with defaults, no drops
- Test 2: more data with defaults, unforced drops but no forced drops
- Test 3: same as test 2, but with higher *QueueDelayReference*
- Test 4: same as test 2, but with reduced dequeue rate
- Test 5: same dequeue rate as test 4, but with higher *Tupdate*
- Test 6: same as test 2, but with *UseDequeueRateEstimator* enabled
- Test 7: test with *CapDropAdjustment* disabled
- Test 8: test with *CapDropAdjustment* enabled
- Test 9: PIE queue disc is ECN enabled, but packets are not ECN capable
- Test 10: Packets are ECN capable, but PIE queue disc is not ECN enabled
- Test 11: Packets and PIE queue disc both are ECN capable
- Test 12: test with *Derandomization* enabled
- Test 13: same as test 11 but with accumulated drop probability set below the low threshold
- Test 14: same as test 12 but with accumulated drop probability set above the high threshold
- Test 15: Tests Active/Inactive feature, *ActiveThreshold* set to a high value so PIE never starts.
- Test 16: Tests Active/Inactive feature, *ActiveThreshold* set to a low value so PIE starts early.

The test suite can be run using the following commands:


```
$ ./ns3 configure --enable-examples --enable-tests
$ ./ns3 build
$ ./test.py -s pie-queue-disc
```

or alternatively (to see logging statements in a debug build):

```
$ NS_LOG="PieQueueDisc" ./ns3 run "test-runner --suite=pie-queue-disc"
```

32.13 FQ-PIE queue disc

This chapter describes the FQ-PIE ([Ram19]) queue disc implementation in *ns-3*.

The FQ-PIE queue disc combines the Proportional Integral Controller Enhanced (PIE) AQM algorithm with the FlowQueue scheduler that is part of FQ-CoDel (also available in *ns-3*). FQ-PIE was introduced to Linux kernel version 5.6.

32.13.1 Model Description

The source code for the `FqPieQueueDisc` is located in the directory `src/traffic-control/model` and consists of 2 files `fq-pie-queue-disc.h` and `fq-pie-queue-disc.cc` defining a `FqPieQueueDisc` class and a helper `FqPieFlow` class. The code was ported to *ns-3* based on Linux kernel code implemented by Mohit P. Tahlilani.

This model calculates drop probability independently in each flow queue. One difficulty, as pointed out by [CableLabs14], is that PIE calculates drop probability based on the departure rate of a (flow) queue, which may be more highly variable than the aggregate queue. An alternative, which CableLabs has called SFQ-PIE, is to calculate an overall drop probability for the entire queue structure, and then scale this drop probability based on the ratio of the queue depth of each flow queue compared with the depth of the current largest queue. This *ns-3* model does not implement the SFQ-PIE variant described by CableLabs.

References

Attributes

The key attributes that the `FqPieQueue` class holds include the following. First, there are PIE-specific attributes that are copied into the individual PIE flow queues:

- `UseEcn`: Whether to use ECN marking
- `MarkEcnThreshold`: ECN marking threshold (RFC 8033 suggests 0.1 (i.e., 10%) default).
- `UseL4s`: Whether to use L4S (only mark ECT1 packets at CE threshold)
- `MeanPktSize`: Constant used to roughly convert bytes to packets
- `A`: Alpha value in PIE algorithm drop probability calculation
- `B`: Beta value in PIE algorithm drop probability calculation
- `Tupdate`: Time period to calculate drop probability
- `Supdate`: Start time of the update timer
- `DequeueThreshold`: Minimum queue size in bytes before dequeue rate is measured
- `QueueDelayReference`: AQM latency target

- `MaxBurstAllowance`: AQM max burst allowance before random drop
- `UseDequeueRateEstimator`: Enable/Disable usage of Dequeue Rate Estimator
- `UseCapDropAdjustment`: Enable/Disable Cap Drop Adjustment feature mentioned in RFC 8033
- `UseDerandomization`: Enable/Disable Derandomization feature mentioned in RFC 8033

Second, there are QueueDisc level, or FQ-specific attributes:: * `MaxSize`: Maximum number of packets in the queue disc * `Flows`: Maximum number of flow queues * `DropBatchSize`: Maximum number of packets dropped from the fat flow * `Perturbation`: Salt value used as hash input when classifying flows * `EnableSetAssociativeHash`: Enable or disable set associative hash * `SetWays`: Size of a set of queues in set associative hash

Examples

A typical usage pattern is to create a traffic control helper and to configure the type and attributes of queue disc and filters from the helper. For example, FqPIE can be configured as follows:

```
TrafficControlHelper tch;
tch.SetRootQueueDisc("ns3::FqPieQueueDisc",
                    "DropBatchSize", UintegerValue(1)
                    "Perturbation", UintegerValue(256));
QueueDiscContainer qdiscs = tch.Install(devices);
```

32.13.2 Validation

The FqPie model is tested using `FqPieQueueDiscTestSuite` class defined in `src/test/ns3tc/fq-pie-queue-test-suite.cc`.

The tests are similar to the ones for FqCoDel queue disc mentioned in first section of this document. The test suite can be run using the following commands:

```
$ ./ns3 configure --enable-examples --enable-tests
$ ./ns3 build
$ ./test.py -s fq-pie-queue-disc
```

or:

```
$ NS_LOG="FqPieQueueDisc" ./ns3 run "test-runner --suite=fq-pie-queue-disc"
```

32.14 Mq queue disc

This chapter describes the mq queue disc implementation in *ns-3*.

mq is a classful multiqueue dummy scheduler developed to best fit the multiqueue traffic control API in Linux. The mq scheduler presents device transmission queues as classes, allowing to attach different queue discs to them, which are grafted to the device transmission queues.

Mq is installed by default on multi-queue devices (such as Wifi) with as many FqCodel child queue discs as the number of device queues.

32.14.1 Model Description

mq is a multi-queue aware queue disc, meaning that it has as many child queue discs as the number of device transmission queues. Each child queue disc maps to a distinct device transmission queue. Every packet is enqueued into the child queue disc which maps to the device transmission queue in which the device will enqueue the packet.

In *ns-3*, `MqQueueDisc` has a wake mode of `WAKE_CHILD`, which means that the traffic control layer enqueues packets directly into one of the child queue discs (multi-queue devices can provide a callback to inform the traffic control layer of the device transmission queue that will be selected for a given packet). Therefore, `MqQueueDisc::DoEnqueue ()` shall never be called (in fact, it raises a fatal error). Given that dequeuing packets is triggered by enqueueing a packet in the queue disc or by the device invoking the wake callback, it turns out that `MqQueueDisc::DoDequeue ()` is never called as well (in fact, it raises a fatal error, too).

The mq queue disc does not require packet filters, does not admit internal queues and must have as many child queue discs as the number of device transmission queues.

Examples

A typical usage pattern is to create a traffic control helper used to add the required number of queue disc classes, attach child queue discs to the classes and (if needed) add packet filters to the child queue discs. The following code shows how to install an mq queue disc having `FqCoDel` child queue discs:

```
TrafficControlHelper tch;
uint16_t handle = tch.SetRootQueueDisc("ns3::MqQueueDisc");
TrafficControlHelper::ClassIdList cls = tch.AddQueueDiscClasses(handle, numTxQueues,
    ↪ "ns3::QueueDiscClass");
tch.AddChildQueueDiscs(handle, cls, "ns3::FqCoDelQueueDisc");
QueueDiscContainer qdiscs = tch.Install(devices);
```

Note that the child queue discs attached to the classes do not necessarily have to be of the same type.

32.14.2 Validation

The mq model is tested using `WifiAcMappingTestSuite` class defined in `src/test/wifi-ac-mapping-test-suite.cc`. The suite considers a node with a QoS-enabled wifi device (which has 4 transmission queues) and includes 4 test cases:

- Test 1: EF-marked packets are enqueued in the queue disc which maps to the `AC_VI` queue
- Test 2: AF11-marked packets are enqueued in the queue disc which maps to the `AC_BK` queue
- Test 3: AF32-marked packets are enqueued in the queue disc which maps to the `AC_BE` queue
- Test 4: CS7-marked packets are enqueued in the queue disc which maps to the `AC_VO` queue

The test suite can be run using the following commands:

```
$ ./ns3 configure --enable-examples --enable-tests
$ ./ns3 build
$ ./test.py -s ns3-wifi-ac-mapping
```

or

```
$ NS_LOG="WifiAcMappingTest" ./ns3 run "test-runner --suite=ns3-wifi-ac-mapping"
```


UAN FRAMEWORK

The main goal of the UAN Framework is to enable researchers to model a variety of underwater network scenarios. The UAN model is broken into four main parts: The channel, PHY, MAC and Autonomous Underwater Vehicle (AUV) models.

The need for underwater wireless communications exists in applications such as remote control in offshore oil industry¹, pollution monitoring in environmental systems, speech transmission between divers, mapping of the ocean floor, mine counter measures^{2,4}, seismic monitoring of ocean faults as well as climate changes monitoring. Unfortunately, making on-field measurements is very expensive and there are no commonly accepted standard to base on. Hence, the priority to make research work going on, it is to realize a complete simulation framework that researchers can use to experiment, make tests and make performance evaluation and comparison.

The NS-3 UAN module is a first step in this direction, trying to offer a reliable and realistic tool. In fact, the UAN module offers accurate modelling of the underwater acoustic channel, a model of the WHOI acoustic modem (one of the widely used acoustic modems)⁶ and its communications performance, and some MAC protocols.

33.1 Model Description

The source code for the UAN Framework lives in the directory `src/uan` and in `src/energy` for the contribution on the li-ion battery model.

The UAN Framework is composed of two main parts:

- the AUV mobility models, including Electric motor propelled AUV (REMUS class³Page 621, 4) and Seaglider⁵ models
- the energy models, including AUV energy models, AUV energy sources (batteries) and an acoustic modem energy model

As enabling component for the energy models, a Li-Ion batteries energy source has been implemented basing on^{7,8}.

¹ BINGHAM, D.; DRAKE, T.; HILL, A.; LOTT, R.; The Application of Autonomous Underwater Vehicle (AUV) Technology in the Oil Industry – Vision and Experiences, URL: http://www.fig.net/pub/fig_2002/Ts4-4/TS4_4_bingham_etal.pdf

² AUVfest2008: Underwater mines; URL: <http://oceanexplorer.noaa.gov/explorations/08auvfest/background/mines/mines.html>

⁴ WHOI, Autonomous Underwater Vehicle, REMUS; URL: <http://www.whoi.edu/page.do?pid=29856>

⁶ L. Freitag, M. Grund, I. Singh, J. Partan, P. Koski, K. Ball, and W. Hole, The whoi micro-modem: an acoustic communications and navigation system for multiple platforms, In Proc. IEEE OCEANS05 Conf, 2005. URL: <http://ieeexplore.ieee.org/iel5/10918/34367/01639901.pdf>

³ Hydroinc (acquired by Huntington Ingalls Industries) Products; URL: <https://tsd.huntingtoningalls.com/what-we-do/unmanned-systems/unmanned-underwater-vehicles/>

⁵ Eriksen, C.C., T.J. Osse, R.D. Light, T. Wen, T.W. Lehman, P.L. Sabin, J.W. Ballard, and A.M. Chiodi. Seaglider: A Long-Range Autonomous Underwater Vehicle for Oceanographic Research, IEEE Journal of Oceanic Engineering, 26, 4, October 2001. URL: <http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=972073&userType=inst>

⁷ C. M. Shepherd, “Design of Primary and Secondary Cells - Part 3. Battery discharge equation,” U.S. Naval Research Laboratory, 1963

⁸ Tremblay, O.; Dessaint, L.-A.; Dekkiche, A.-I., “A Generic Battery Model for the Dynamic Simulation of Hybrid Electric Vehicles,” Ecole de Technologie Supérieure, Université du Québec, 2007 URL: <http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4544139>

33.1.1 Design

UAN Propagation Models

Modelling of the underwater acoustic channel has been an active area of research for quite some time. Given the complications involved, surface and bottom interactions, varying speed of sound, etc. . . , the detailed models in use for ocean acoustics research are much too complex (in terms of runtime) for use in network level simulations. We have attempted to provide the often used models as well as make an attempt to bridge, in part, the gap between complicated ocean acoustic models and network level simulation. The three propagation models included are the ideal channel model, the Thorp propagation model and the Bellhop propagation model (Available as an addition).

All of the Propagation Models follow the same simple interface in `ns3::UanPropModel`. The propagation models provide a power delay profile (PDP) and pathloss information. The PDP is retrieved using the `GetPdp` method which returns type `UanPdp`. `ns3::UanPdp` utilises a tapped delay line model for the acoustic channel. The `UanPdp` class is a container class for Taps, each tap has a delay and amplitude member corresponding to the time of arrival (relative to the first tap arrival time) and amplitude. The propagation model also provides pathloss between the source and receiver in dB re 1uPa. The PDP and pathloss can then be used to find the received signal power over a duration of time (i.e. received signal power in a symbol duration and ISI which interferes with neighbouring signals). Both `UanPropModelIdeal` and `UanPropModelThorp` return a single impulse for a PDP.

a) Ideal Channel Model `ns3::UanPropModelIdeal`

The ideal channel model assumes 0 pathloss inside a cylindrical area with bounds set by attribute. The ideal channel model also assumes an impulse PDP.

b) Thorp Propagation Model `ns3::UanPropModelThorp`

The Thorp Propagation Model calculates pathloss using the well-known Thorp approximation. This model is similar to the underwater channel model implemented in ns2 as described here:

Harris, A. F. and Zorzi, M. 2007. Modeling the underwater acoustic channel in ns2. In Proceedings of the 2nd international Conference on Performance Evaluation Methodologies and Tools (Nantes, France, October 22 - 27, 2007). ValueTools, vol. 321. ICST (Institute for Computer Sciences Social-Informatics and Telecommunications Engineering), ICST, Brussels, Belgium, 1-8.

The frequency used in calculation however, is the center frequency of the modulation as found from `ns3::UanTxMode`. The Thorp Propagation Model also assumes an impulse channel response.

c) Bellhop Propagation Model `ns3::UanPropModelBh` (Available as an addition)

The Bellhop propagation model reads propagation information from a database. A configuration file describing the location, and resolution of the archived information must be supplied via attributes. We have included a utility, `create-dat`, which can create these data files using the Bellhop Acoustic Ray Tracing software (<http://oalib.hlsresearch.com/>).

The `create-dat` utility requires a Bellhop installation to run. Bellhop takes environment information about the channel, such as sound speed profile, surface height bottom type, water depth, and uses a Gaussian ray tracing algorithm to determine propagation information. Arrivals from Bellhop are grouped together into equal length taps (the arrivals in a tap duration are coherently summed). The maximum taps are then aligned to take the same position in the PDP. The `create-dat` utility averages together several runs and then normalizes the average such that the sum of all taps is 1. The same configuration file used to create the data files using `create-dat` should be passed via attribute to the Bellhop Propagation Model.

The Bellhop propagation model is available as a patch. The link address will be made available here when it is posted online. Otherwise email lentracy@gmail.com for more information.

UAN PHY Model Overview

The PHY has been designed to allow for relatively easy extension to new networking scenarios. We feel this is important as, to date, there has been no commonly accepted network level simulation model for underwater networks. The lack of commonly accepted network simulation tools has resulted in a wide array of simulators and models used to report results in literature. The lack of standardization makes comparing results nearly impossible.

The main component of the PHY Model is the generic PHY class, `ns3::UanPhyGen`. The PHY class's general responsibility is to handle packet acquisition, error determination, and forwarding of successful packets up to the MAC layer. The Generic PHY uses two models for determination of signal to noise ratio (SINR) and packet error rate (PER). The combination of the PER and SINR models determine successful reception of packets. The PHY model connects to the channel via a Transducer class. The Transducer class is responsible for tracking all arriving packets and departing packets over the duration of the events. How the PHY class and the PER and SINR models respond to packets is based on the "Mode" of the transmission as described by the `ns3::UanTxMode` class.

When a MAC layer sends down a packet to the PHY for transmission it specifies a "mode number" to be used for the transmission. The PHY class accepts, as an attribute, a list of supported modes. The mode number corresponds to an index in the supported modes. The `UanTxMode` contains simple modulation information and a unique string id. The generic PHY class will only acquire arriving packets which use a mode which is in the supported modes list of the PHY. The mode along with received signal power, and other pertinent attributes (e.g. possibly interfering packets and their modes) are passed to the SINR and PER models for calculation of SINR and probability of error.

Several simple example PER and SINR models have been created. a) The PER models - Default (simple) PER model (`ns3::UanPhyPerGenDefault`): The Default PER model tests the packet against a threshold and assumes error (with prob. 1) if the SINR is below the threshold or success if the SINR is above the threshold - Micromodem FH-FSK PER (`ns3::UanPhyPerUmodem`). The FH-FSK PER model calculates probability of error assuming a rate 1/2 convolutional code with constraint length 9 and a CRC check capable of correcting up to 1 bit error. This is similar to what is used in the receiver of the WHOI Micromodem.

b) SINR models - Default Model (`ns3::UanPhyCalcSinrDefault`), The default SINR model assumes that all transmitted energy is captured at the receiver and that there is no ISI. Any received signal power from interferes acts as additional ambient noise. - FH-FSK SINR Model (`ns3::UanPhyCalcSinrFhFsk`), The WHOI Micromodem operating in FH-FSK mode uses a predetermined hopping pattern that is shared by all nodes in the network. We model this by only including signal energy receiving within one symbol time (as given by `ns3::UanTxMode`) in calculating the received signal power. A channel clearing time is given to the FH-FSK SINR model via attribute. Any signal energy arriving in adjacent signals (after a symbol time and the clearing time) is considered ISI and is treated as additional ambient noise. Interfering signal arrivals inside a symbol time (any symbol time) is also counted as additional ambient noise. - Frequency filtered SINR (`ns3::UanPhyCalcSinrDual`). This SINR model calculates SINR in the same manner as the default model. This model however only considers interference if there is an overlap in frequency of the arriving packets as determined by `UanTxMode`.

In addition to the generic PHY a dual phy layer is also included (`ns3::UanPhyDual`). This wraps two generic phy layers together to model a net device which includes two receivers. This was primarily developed for `UanMacRc`, described in the next section.

UAN MAC Model Overview

Over the last several years there have been a myriad of underwater MAC proposals in the literature. We have included three MAC protocols with this distribution: a) CW-MAC, a MAC protocol which uses a slotted contention window similar in nature to the IEEE 802.11 DCF. Nodes have a constant contention window measured in slot times (configured via attribute). If the channel is sensed busy, then nodes backoff by randomly (uniform distribution) choose a slot to transmit in. The slot time durations are also configured via attribute. This MAC was described in

Parrish N.; Tracy L.; Roy S. Arabshahi P.; and Fox, W., System Design Considerations for Undersea Networks: Link and Multiple Access Protocols, IEEE Journal on Selected Areas in Communications (JSAC), Special Issue on Underwater Wireless Communications and Networks, Dec. 2008.

b) RC-MAC (`ns3::UanMacRc` `ns3::UanMacRcGw`) a reservation channel protocol which dynamically divides the available bandwidth into a data channel and a control channel. This MAC protocol assumes there is a gateway node which all network traffic is destined for. The current implementation assumes a single gateway and a single network neighborhood (a single hop network). RTS/CTS handshaking is used and time is divided into cycles. Non-gateway nodes transmit RTS packets on the control channel in parallel to data packet transmissions which were scheduled in the previous cycle at the start of a new cycle, the gateway responds on the data channel with a CTS packet which includes packet transmission times of data packets for received RTS packets in the previous cycle as well as bandwidth allocation information. At the end of a cycle ACK packets are transmitted for received data packets.

When a publication is available it will be cited here.

c) Simple ALOHA (`ns3::UanMacAloha`) Nodes transmit at will.

AUV mobility models

The AUV mobility models have been designed as in the follows.

Use cases

The user will be able to:

- program the AUV to navigate over a path of waypoints
- control the velocity of the AUV
- control the depth of the AUV
- control the direction of the AUV
- control the pitch of the AUV
- tell the AUV to emerge or submerge to a specified depth

AUV mobility models design

Implement a model of the navigation of AUV. This involves implementing two classes modelling the two major categories of AUVs: electric motor propelled (like REMUS class^{3Page 621, 4)} and “sea gliders”^{Page 621, 5}. The classic AUVs are submarine-like devices, propelled by an electric motor linked with a propeller. Instead, the “sea glider” class exploits small changes in its buoyancy that, in conjunction with wings, can convert vertical motion to horizontal. So, a glider will reach a point into the water by describing a “saw-tooth” movement. Modelling the AUV navigation, involves in considering a real-world AUV class thus, taking into account maximum speed, directional capabilities, emerging and submerging times. Regarding the sea gliders, it is modelled the characteristic saw-tooth movement, with AUV’s speed driven by buoyancy and glide angle.

An `ns3::AuvMobilityModel` interface has been designed to give users a generic interface to access AUV’s navigation functions. The `AuvMobilityModel` interface is implemented by the `RemusMobilityModel` and the `GliderMobilityModel` classes. The AUV’s mobility models organization it is shown in *AUV’s mobility model classes overview*. Both models use a constant velocity movement, thus the `AuvMobilityModel` interface derives from the `ConstantVelocityMobilityModel`. The two classes hold the navigation parameters for the two different AUVs, like maximum pitch angles, maximum operating depth, maximum and minimum speed values. The `Glider` model holds also some extra parameters like maximum buoyancy values, and maximum and minimum glide slopes. Both classes, `RemusMobilityModel` and `GliderMobilityModel`, handle also the AUV power consumption, utilizing the relative power models. Has been modified the `WaypointMobilityModel` to let it use a generic underlying `ConstantVelocityModel` to validate the waypoints and, to keep trace of the node’s position. The default model is the classic `ConstantVelocityModel` but, for example in case of REMUS mobility model, the user can install the AUV mobility model into the waypoint model and then validating the waypoints against REMUS navigation constraints.

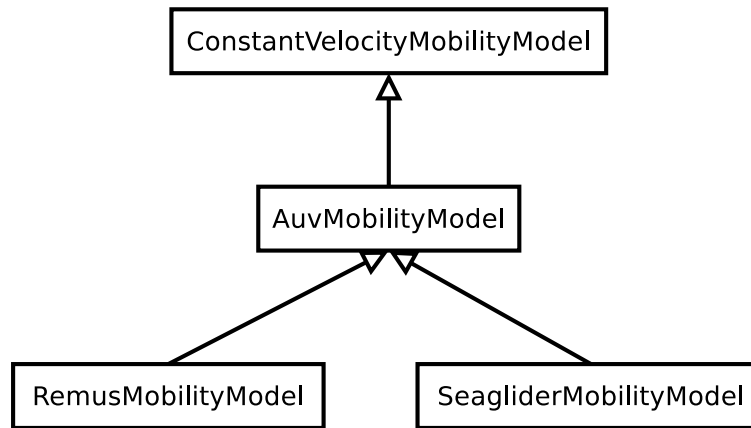


Fig. 33.1: AUV's mobility model classes overview

Energy models

The energy models have been designed as in the follows.

Use cases

The user will be able to:

- use a specific power profile for the acoustic modem
- use a specific energy model for the AUV
- trace the power consumption of AUV navigation, through AUV's energy model
- trace the power consumption underwater acoustic communications, through acoustic modem power profile

We have integrated the Energy Model with the UAN module, to implement energy handling. We have implemented a specific energy model for the two AUV classes and, an energy source for Lithium batteries. This will be really useful for researchers to keep trace of the AUV operational life. We have implemented also an acoustic modem power profile, to keep trace of its power consumption. This can be used to compare protocols specific power performance. In order to use such power profile, the acoustic transducer physical layer has been modified to use the modem power profile. We have decoupled the physical layer from the transducer specific energy model, to let the users change the different energy models without changing the physical layer.

AUV energy models

Basing on the Device Energy Model interface, it has been implemented a specific energy model for the two AUV classes (REMUS and Seaglider). This models reproduce the AUV's specific power consumption to give users accurate information. This model can be naturally used to evaluates the AUV operating life, as well as mission-related power consumption, etc. Have been developed two AUV energy models:

- GliderEnergyModel, computes the power consumption of the vehicle based on the current buoyancy value and vertical speed^{Page 621, 5}
- RemusEnergyModel, computes the power consumption of the vehicle based on the current speed, as it is propelled by a brush-less electric motor

Note: TODO extend a little bit

AUV energy sources

Note: [TODO]

Acoustic modem energy model

Basing on the Device Energy Model interface, has been implemented a generic energy model for acoustic modem. The model allows to trace four modem's power-states: Sleep, Idle, Receiving, Transmitting. The default parameters for the energy model are set to fit those of the WHOI μ -modem. The class follows pretty closely the RadioEnergyModel class as the transducer behaviour is pretty close to that of a Wi-Fi radio.

The default power consumption values implemented into the model are as follows^{Page 621, 6}:

Modem State	Power Consumption
TX	50 W
RX	158 mW
Idle	158 mW
Sleep	5.8 mW

UAN module energy modifications

The UAN module has been modified in order to utilize the implemented energy classes. Specifically, it has been modified the physical layer of the UAN module. It Has been implemented an UpdatePowerConsumption method that takes the modem's state as parameter. It checks if an energy source is installed into the node and, in case, it then use the AcousticModemEnergyModel to update the power consumption with the current modem's state. The modem power consumption's update takes place whenever the modem changes its state.

A user should take into account that, if the power consumption handling is enabled (if the node has an energy source installed), all the communications processes will terminate whether the node depletes all the energy source.

Li-Ion batteries model

A generic Li-Ion battery model has been implemented based on^{Page 621, 7Page 621, 8}. The model can be fitted to any type of Li-Ion battery simply changing the model's parameters. The default values are fitted for the Panasonic CGR18650DA Li-Ion Battery⁹. [TODO insert figure] As shown in figure the model approximates very well the Li-Ion cells. Regarding Seagliders, the batteries used into the AUV are Electrochem 3B36 Lithium / Sulfuryl Chloride cells¹⁰. Also with this cell type, the model seems to approximates the different discharge curves pretty well, as shown in the figure.

Note: should I insert the li-ion model details here? I think it is better to put them into an Energy-related chapter..

⁹ Panasonic CGR18650DA Datasheet, URL: http://www.panasonic.com/industrial/includes/pdf/Panasonic_LiIon_CGR18650DA.pdf

¹⁰ Electrochem 3B36 Datasheet, URL: <http://www.electrochem.com.cn/products/Primary/HighRate/CSC/3B36.pdf>

33.1.2 Scope and Limitations

The framework is designed to simulate AUV's behaviour. We have modeled the navigation and power consumption behaviour of REMUS class and Seaglider AUVs. The communications stack, associated with the AUV, can be modified depending on simulation needs. Usually, the default underwater stack is being used, composed of an half duplex acoustic modem, an Aloha MAC protocol and a generic physical layer.

Regarding the AUV energy consumption, the user should be aware that the level of accuracy differs for the two classes:

- Seaglider, high level of accuracy, thanks to the availability of detailed information on AUV's components and behaviour^{Page 621, 5}^{Page 626, 10}. Have been modeled both the navigation power consumption and the Li battery packs (according to^{Page 621, 5}).
- REMUS, medium level of accuracy, due to the lack of publicly available information on AUV's components. We have approximated the power consumption of the AUV's motor with a linear behaviour and, the energy source uses an ideal model (BasicEnergySource) with a power capacity equal to that specified in^{Page 621, 4}.

33.1.3 Future Work

Some ideas could be :

- insert a data logging capability
- modify the framework to use sockets (enabling the possibility to use applications)
- introduce some more MAC protocols
- modify the physical layer to let it consider the Doppler spread (problematic in underwater environments)
- introduce OFDM modulations

33.1.4 References

33.2 Usage

The main way that users who write simulation scripts will typically interact with the UAN Framework is through the helper API and through the publicly visible attributes of the model.

The helper API is defined in `src/uan/helper/acoustic-modem-energy-model-helper.{cc,h}` and in `/src/uan/helper/...{cc,h}`.

The example folder `src/uan/examples/` contain some basic code that shows how to set up and use the models. further examples can be found into the Unit tests in `src/uan/test/...cc`

33.2.1 Examples

Examples of the Framework's usage can be found into the examples folder. There are mobility related examples and UAN related ones.

Mobility Model Examples

- **auv-energy-model:**

In this example we show the basic usage of an AUV energy model. Specifically, we show how to create a generic node, adding to it a basic energy source and consuming energy from the energy source. In this example we show the basic usage of an AUV energy model.

The Seaglider AUV power consumption depends on buoyancy and vertical speed values, so we simulate a 20 seconds movement at 0.3 m/s of vertical speed and 138g of buoyancy. Then a 20 seconds movement at 0.2 m/s of vertical speed and 138g of buoyancy and then a stop of 5 seconds.

The required energy will be drained by the model basing on the given buoyancy/speed values, from the energy source installed onto the node. We finally register a callback to the TotalEnergyConsumption traced value.

- **auv-mobility:**

In this example we show how to use the AuvMobilityHelper to install an AUV mobility model into a (set of) node. Then we make the AUV to submerge to a depth of 1000 meters. We then set a callback function called on reaching of the target depth. The callback then makes the AUV to emerge to water surface (0 meters). We set also a callback function called on reaching of the target depth. The emerge callback then, stops the AUV.

During the whole navigation process, the AUV's position is tracked by the TracePos function and plotted into a Gnuplot graph.

- **waypoint-mobility:**

We show how to use the WaypointMobilityModel with a non-standard ConstantVelocityMobilityModel. We first create a waypoint model with an underlying RemusMobilityModel setting the mobility trace with two waypoints. We then create a waypoint model with an underlying GliderMobilityModel setting the waypoints separately with the AddWaypoint method. The AUV's position is printed out every seconds.

UAN Examples

- **li-ion-energy-source**

In this simple example, we show how to create and drain energy from a LiIonEnergySource. We make a series of discharge calls to the energy source class, with different current drain and durations, until all the energy is depleted from the cell (i.e. the voltage of the cell goes below the threshold level). Every 20 seconds we print out the actual cell voltage to verify that it follows the discharge curve^{Page 626, 9}. At the end of the example it is verified that after the energy depletion call, the cell voltage is below the threshold voltage.

- **uan-energy-auv**

This is a comprehensive example where all the project's components are used. We setup two nodes, one fixed surface gateway equipped with an acoustic modem and a moving Seaglider AUV with an acoustic modem too. Using the waypoint mobility model with an underlying GliderMobilityModel, we make the glider descend to -1000 meters and then emerge to the water surface. The AUV sends a generic 17-bytes packet every 10 seconds during the navigation process. The gateway receives the packets and stores the total bytes amount. At the end of the simulation are shown the energy consumptions of the two nodes and the networking stats.

33.2.2 Helpers

In this section we give an overview of the available helpers and their behaviour.

AcousticModemEnergyModelHelper

This helper installs AcousticModemEnergyModel into UanNetDevice objects only. It requires an UanNetDevice and an EnergySource as input objects.

The helper creates an AcousticModemEnergyModel with default parameters and associate it with the given energy source. It configures an EnergyModelCallback and an EnergyDepletionCallback. The depletion callback can be configured as a parameter.

AuvGliderHelper

Installs into a node (or set of nodes) the Seaglider's features:

- waypoint model with underlying glider mobility model
- glider energy model
- glider energy source
- micro modem energy model

The glider mobility model is the GliderMobilityModel with default parameters. The glider energy model is the GliderEnergyModel with default parameters.

Regarding the energy source, the Seaglider features two battery packs, one for motor power and one for digital-analog power. Each pack is composed of 12 (10V) and 42 (24V) lithium chloride DD-cell batteries, respectively^{Page 621, 5}. The total power capacity is around 17.5 MJ (3.9 MJ + 13.6 MJ). In the original version of the Seaglider there was 18 + 63 D-cell with a total power capacity of 10MJ.

The packs design is as follows:

- 10V - 3 in-series string x 4 strings = 12 cells - typical capacity ~100 Ah
- 24V - 7 in-series-strings x 6 strings = 42 cells - typical capacity ~150 Ah

Battery cells are Electrochem 3B36, with 3.6 V nominal voltage and 30.0 Ah nominal capacity. The 10V battery pack is associated with the electronic devices, while the 24V one is associated with the pump motor.

The micro modem energy model is the MicroModemEnergyModel with default parameters.

AuvRemusHelper

Install into a node (or set of nodes) the REMUS features:

- waypoint model with REMUS mobility model validation
- REMUS energy model
- REMUS energy source
- micro modem energy model

The REMUS mobility model is the RemusMobilityModel with default parameters. The REMUS energy model is the RemusEnergyModel with default parameters.

Regarding the energy source, the REMUS features a rechargeable lithium ion battery pack rated 1.1 kWh @ 27 V (40 Ah) in operating conditions (specifications from [Page 621, 3](#) and Hydroinc European salesman). Since more detailed information about battery pack were not publicly available, the energy source used is a BasicEnergySource.

The micro modem energy model is the MicroModemEnergyModel with default parameters.

33.2.3 Attributes

Note: TODO

33.2.4 Tracing

Note: TODO

33.2.5 Logging

Note: TODO

33.2.6 Caveats

Note: TODO

33.3 Validation

This model has been tested with three UNIT test:

- auv-energy-model
- auv-mobility
- li-ion-energy-source

33.3.1 Auv Energy Model

Includes test cases for single packet energy consumption, energy depletion, Glider and REMUS energy consumption. The unit test can be found in `src/uan/test/auv-energy-model-test.cc`.

The single packet energy consumption test do the following:

- creates a two node network, one surface gateway and one fixed node at -500 m of depth
- install the acoustic communication stack with energy consumption support into the nodes
- a packet is sent from the underwater node to the gateway

- it is verified that both, the gateway and the fixed node, have consumed the expected amount of energy from their sources

The energy depletion test do the following steps:

- create a node with an empty energy source
- try to send a packet
- verify that the energy depletion callback has been invoked

The Glider energy consumption test do the following:

- create a node with glider capabilities
- make the vehicle to move to a predetermined waypoint
- verify that the energy consumed for the navigation is correct, according to the glider specifications

The REMUS energy consumption test do the following:

- create a node with REMUS capabilities
- make the vehicle to move to a predetermined waypoint
- verify that the energy consumed for the navigation is correct, according to the REMUS specifications

33.3.2 Auv Mobility

Includes test cases for glider and REMUS mobility models. The unit test can be found in `src/uan/test/auv-mobility-test.cc`.

- create a node with glider capabilities
- set a specified velocity vector and verify if the resulting buoyancy is the one that is supposed to be
- make the vehicle to submerge to a specified depth and verify if, at the end of the process the position is the one that is supposed to be
- make the vehicle to emerge to a specified depth and verify if, at the end of the process the position is the one that is supposed to be
- make the vehicle to navigate to a specified point, using direction, pitch and speed settings and, verify if at the end of the process the position is the one that is supposed to be
- make the vehicle to navigate to a specified point, using a velocity vector and, verify if at the end of the process the position is the one that is supposed to be

The REMUS mobility model test do the following: * create a node with glider capabilities * make the vehicle to submerge to a specified depth and verify if, at the end of the process the position is the one that is supposed to be * make the vehicle to emerge to a specified depth and verify if, at the end of the process the position is the one that is supposed to be * make the vehicle to navigate to a specified point, using direction, pitch and speed settings and, verify if at the end of the process the position is the one that is supposed to be * make the vehicle to navigate to a specified point, using a velocity vector and, verify if at the end of the process the position is the one that is supposed to be

33.3.3 Li-Ion Energy Source

Includes test case for Li-Ion energy source. The unit test can be found in `src/energy/test/li-ion-energy-source-test.cc`.

The test case verify that after a well-known discharge time with constant current drain, the cell voltage has followed the datasheet discharge curve^{Page 626, 9}.

VIRTUALNETDEVICE

This is the introduction to VirtualNetDevice, to complement the VirtualNetDevice model doxygen.

34.1 Overview of the VirtualNetDevice

The VirtualNetDevice is a “virtual” NetDevice implementation, similar to Linux TUN/TAP interfaces. Instead of sending packets over a simulated channel, it delegates the actual transmission of L2 frames to user code via a callback configured with SetSendCallback().

The user code may also inject frames into the device using the VirtualNetDevice::Receive() method. Together, these features allow one to build tunnels (for example IP-over-UDP-over-IP, or IP-over-IP) without needing to implement a full NetDevice subclass.

34.2 Model Description

The source code for this module is written in the directory src/virtual-net-device.

The device exposes a standard NetDevice interface and can be attached to a node and configured with an address and MTU. By default, the device reports itself as a point-to-point device, does not require ARP, and supports SendFrom; these behaviors can be configured via:

- SetIsPointToPoint()
- SetNeedsArp()
- SetSupportsSendFrom()

34.3 Usage

Typical usage is to create a device, install it on a node, configure a transmit callback, and then forward packets to and from some external mechanism (e.g., a socket or a tunnel endpoint) in user code.

A minimal sketch is as follows:

```
Ptr<VirtualNetDevice> dev = CreateObject<VirtualNetDevice>();
dev->SetAddress(Mac48Address::Allocate());
dev->SetSendCallback(MakeCallback(&MyTunnelEndpoint::Send, endpoint));
node->AddDevice(dev);
```

To inject a frame into the node's protocol stack, call `VirtualNetDevice::Receive()` with the appropriate protocol number and source and destination addresses.

34.4 Traces

The standard set of MAC-level `NetDevice` trace sources is provided:

- `MacTx`: Trace source indicating a packet has arrived for transmission by this device
- `MacRx`: Trace source for packets successfully received in non-promiscuous mode
- `MacPromiscRx`: Trace source for packets received in promiscuous mode and fired only when a promiscuous receive callback is installed via `SetPromiscReceiveCallback`
- `Sniffer / PromiscSniffer`: Trace sources simulating non-promiscuous and promiscuous packet sniffers

WI-FI MODULE

35.1 Design Documentation

ns-3 nodes can contain a collection of NetDevice objects, much like an actual computer contains separate interface cards for Ethernet, Wifi, Bluetooth, etc. This chapter describes the *ns-3* WifiNetDevice and related models. By adding WifiNetDevice objects to *ns-3* nodes, one can create models of 802.11-based infrastructure and ad hoc networks.

35.1.1 Overview of the model

The WifiNetDevice models a wireless network interface controller based on the IEEE 802.11 standard [ieee80211]. We will go into more detail below but in brief, *ns-3* provides models for these aspects of 802.11:

- basic 802.11 DCF with **infrastructure** and **adhoc** modes
- **802.11a**, **802.11b**, **802.11g**, **802.11n** (both 2.4 and 5 GHz bands), **802.11ac**, **802.11ax** (2.4, 5 and 6 GHz bands) and **802.11be** physical layers
- Power Save mode (TIM element in Beacon frames, transmission of PS-Poll frames, setting PHYs of STAs to sleep to save energy and resuming them to listen to Beacon frames)
- **MSDU aggregation** and **MPDU aggregation** extensions of 802.11n, and both can be combined together (two-level aggregation)
- 802.11ax **DL OFDMA** and **UL OFDMA** (including support for the MU EDCA Parameter Set)
- 802.11be **Multi-link** discovery and setup, **Multi-Link Operations (MLO)** in **STR** (Simultaneous Transmit Receive) mode and **EMLSR** (Enhanced Multi-Link Single-Radio) mode
- QoS-based EDCA and queueing extensions of **802.11e**
- the ability to use different propagation loss models and propagation delay models, please see the chapter on *Propagation* for more detail
- packet error models and frame detection models that have been validated against link simulations and other references
- various rate control algorithms including **Aarf**, **Arf**, **Cara**, **Onoe**, **Rraa**, **ConstantRate**, **Minstrel** and **Minstrel-HT**
- 802.11s (mesh), described in another chapter

The set of 802.11 models provided in *ns-3* attempts to provide an accurate MAC-level implementation of the 802.11 specification and to provide a packet-level abstraction of the PHY-level for different PHYs, corresponding to 802.11a/b/e/g/n/ac/ax/be specifications.

In *ns-3*, nodes can have multiple *WifiNetDevices* on separate channels, and the *WifiNetDevice* can coexist with other device types. With the use of the **SpectrumWifiPhy** framework, one can also build scenarios involving cross-channel interference or multiple wireless technologies on a single channel.

The source code for the *WifiNetDevice* and its models lives in the directory `src/wifi`.

The implementation is modular and provides roughly three sublayers of models:

- the **PHY layer models**: they model amendment-specific and common PHY layer operations and functions.
- **lower MAC models**: they model functions such as medium access (DCF and EDCA), frame protection (RTS/CTS) and acknowledgment (ACK/BlockAck). In *ns-3*, the lower-level MAC is comprised of a **Frame Exchange Manager** hierarchy, a **Channel Access Manager**, **Txop** objects, **BlockAckManager** objects, and a **MAC middle** entity.
- **upper MAC models**: they implement non-time-critical processes in Wifi such as the MAC-level beacon generation, probing, and association state machines, and a set of **Rate control algorithms**. In the literature, this sublayer is sometimes called the **upper MAC** and consists of more software-oriented implementations vs. time-critical hardware implementations.

Next, we provide a design overview of each layer, shown in Figure *WifiNetDevice architecture*. For 802.11be Multi-Link Devices (MLDs), there as many instances of *WifiPhy*, *FrameExchangeManager* and *ChannelAccessManager* as the number of links.

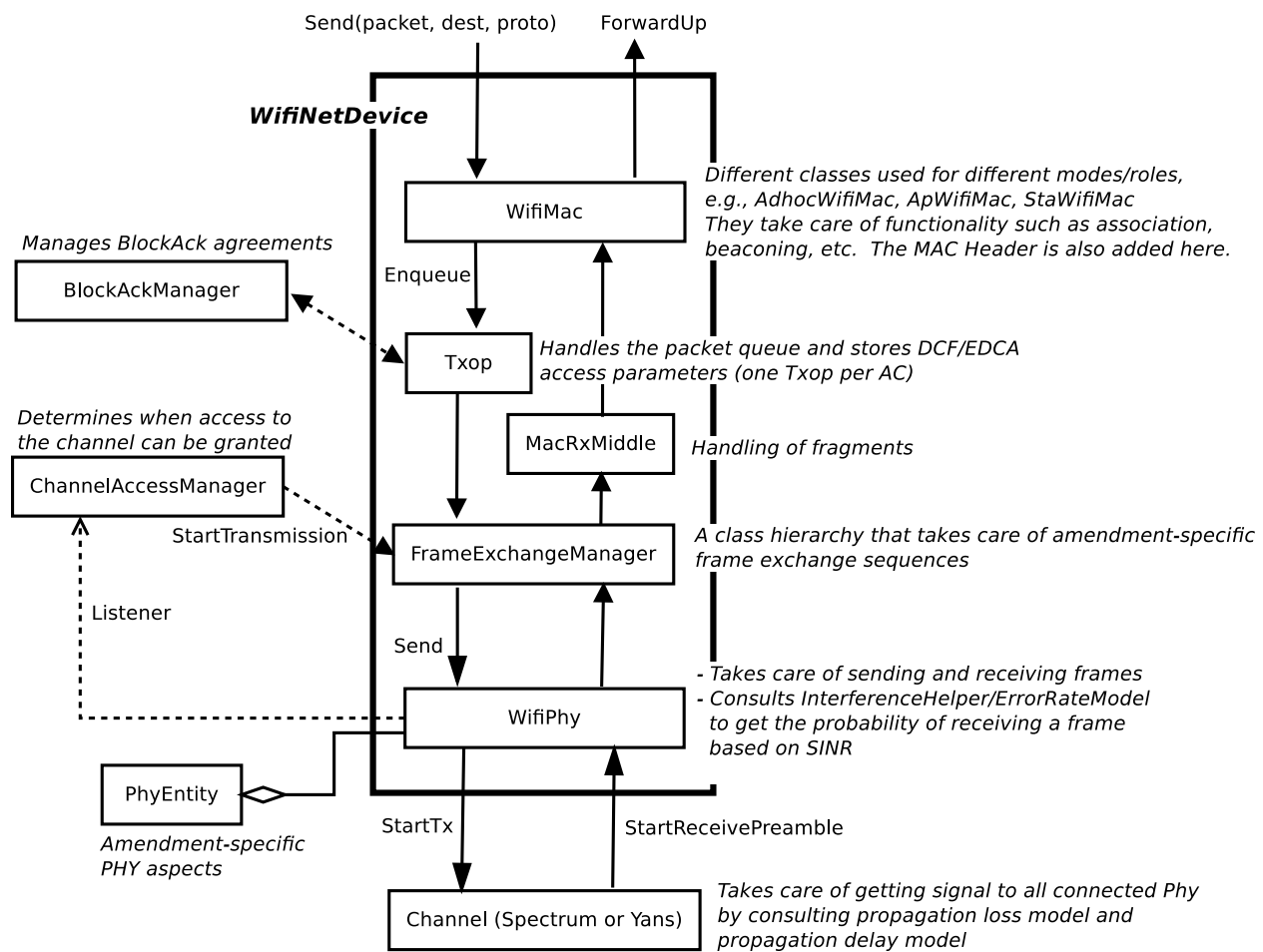


Fig. 35.1: *WifiNetDevice* architecture

MAC high models

There are presently three variants of parent class `ns3::WifiMac`: Access Point (AP) (`ns3::ApWifiMac`), non-AP Station (STA) (`ns3::StaWifiMac`), and STA in an Independent Basic Service Set (IBSS) - also commonly referred to as an ad hoc network (`ns3::AdhocWifiMac`).

The simplest of these is `ns3::AdhocWifiMac`, which implements a Wi-Fi MAC that does not perform any kind of beacon generation, probing, or association. The `ns3::StaWifiMac` class implements an active probing and association state machine that handles automatic re-association whenever too many beacons are missed. Finally, `ns3::ApWifiMac` implements an AP that generates periodic beacons, and that accepts every attempt to associate.

These three MAC high models share a common parent in `ns3::WifiMac`, which exposes, among other MAC configuration, an attribute `QoSSupported` that allows configuration of 802.11e/WMM-style QoS support.

There are also several **rate control algorithms** that can be used by the MAC low layer. These are all implemented as subclasses of `ns3::WifiRemoteStationManager`. A complete list of available rate control algorithms is provided in a separate section.

MAC low layer

The lower MAC is split into three main components:

1. `ns3::FrameExchangeManager` a class hierarchy which implement the frame exchange sequences introduced by the supported IEEE 802.11 amendments. It also handles frame aggregation, frame retransmissions, protection and acknowledgment.
2. `ns3::ChannelAccessManager` which implements the DCF and EDCAF functions.
3. `ns3::Txop` and `ns3::QoSTxop` which handle the packet queue. The `ns3::Txop` object is used by high MACs that are not QoS-enabled, and for transmission of frames (e.g., of type Management) that the standard says should access the medium using the DCF. `ns3::QoSTxop` is used by QoS-enabled high MACs.

PHY layer models

In short, the physical layer models are mainly responsible for modeling the reception of packets and for tracking energy consumption. There are typically three main components to packet reception:

- each packet received is probabilistically evaluated for successful or failed reception. The probability depends on the modulation, on the signal to noise (and interference) ratio for the packet, and on the state of the physical layer (e.g. reception is not possible while transmission or sleeping is taking place);
- an object exists to track (bookkeeping) all received signals so that the correct interference power for each packet can be computed when a reception decision has to be made; and
- one or more error models corresponding to the modulation and standard are used to look up probability of successful reception.

ns-3 offers users a choice between two physical layer models, with a base interface defined in the `ns3::WifiPhy` class. The `YansWifiPhy` class implements a simple physical layer model, which is described in a paper entitled [Yet Another Network Simulator](#). The acronym *Yans* derives from this paper title. The `SpectrumWifiPhy` class is a more advanced implementation based on the Spectrum framework used for other *ns-3* wireless models. Spectrum allows a fine-grained frequency decomposition of the signal, and permits scenarios to include multiple technologies coexisting on the same channel.

35.1.2 Scope and Limitations

The IEEE 802.11 standard [ieee80211] is a large specification, and not all aspects are covered by *ns-3*; the documentation of *ns-3*'s conformance by itself would lead to a very long document. This section attempts to summarize compliance with the standard and with behavior found in practice.

The physical layer and channel models operate on a per-packet basis, with no frequency-selective propagation nor interference effects when using the default YansWifiPhy model. Directional antennas are also not supported at this time. For additive white Gaussian noise (AWGN) scenarios, or wideband interference scenarios, performance is governed by the application of analytical models (based on modulation and factors such as channel width) to the received signal-to-noise ratio, where noise combines the effect of thermal noise and of interference from other Wi-Fi packets. Interference from other wireless technologies is only modeled when the SpectrumWifiPhy is used. The following details pertain to the physical layer and channel models:

- 802.11n/ac/ax/be beamforming is not supported
- 802.11n RIFS is not supported
- 802.11 PCF/HCF/HCCA are not implemented
- Channel Switch Announcement is not supported
- Authentication and encryption are missing
- Processing delays are not modeled
- Cases where RTS/CTS and ACK are transmitted using HT/VHT/HE/EHT formats are not supported
- Energy consumption model does not consider MIMO
- 802.11ax preamble puncturing is supported by the PHY but is currently not exploited by the MAC
- Only minimal MU-MIMO is supported (ideal PHY assumed, no MAC layer yet)

At the MAC layer, most of the main functions found in deployed Wi-Fi equipment for 802.11a/b/e/g/n/ac/ax/be are implemented, but there are scattered instances where some limitations in the models exist. Support for 802.11n, ac, ax and be is evolving.

Some implementation choices that are not imposed by the standard are listed below:

- BSSBasicRateSet for 802.11b has been assumed to be 1-2 Mbit/s
- BSSBasicRateSet for 802.11a/g has been assumed to be 6-12-24 Mbit/s
- OperationalRateSet is assumed to contain all mandatory rates (see [issue 183](#))
- The wifi manager always selects the lowest basic rate for management frames.
- If a STA (AP or non-AP) supports VHT, a Block Ack agreement is always setup once a first packet is enqueued regardless of whether it will be transmitted in an A-MPDU.
- Once an A-MSDU is created, it is no longer modified, even before it is actually transmitted for the first time. This means that this A-MSDU cannot be aggregated to other MSDUs using A-MSDU aggregation.

The following open issues are classified as unfixed bugs in the ns-3 tracker, and users should be aware of them:

- The default TXOP limits do not conform to the standard (e.g., 2.528 ms for AC_BE is not default); see *Issue 289* <<https://gitlab.com/nsnam/ns-3-dev/-/issues/289>>_ and *Merge Request 1976* <https://gitlab.com/nsnam/ns-3-dev/-/merge_requests/1976>_.
- CCA issues with use of ED and PD thresholds have been reported; see *Issue 1247* <<https://gitlab.com/nsnam/ns-3-dev/-/issues/1247>>_ and *Issue 1056* <<https://gitlab.com/nsnam/ns-3-dev/-/issues/1056>>_ and *Issue 900* <<https://gitlab.com/nsnam/ns-3-dev/-/issues/900>>_.

- Use of MLO possibly causes the WifiPhy::MonitorSniffRx trace to miss some packets; see *Issue 1179* <<https://gitlab.com/nsnam/ns-3-dev/-/issues/1179>>_.
- MinstrelHt rate control has reported bugs; see *Issue 886* <<https://gitlab.com/nsnam/ns-3-dev/-/issues/886>>_ and *Issue 51* <<https://gitlab.com/nsnam/ns-3-dev/-/issues/51>>_ and *Merge Request 2344* <https://gitlab.com/nsnam/ns-3-dev/-/merge_requests/2344>_.
- The 1600ns guard interval configuration performance for 802.11ax seems to be incorrect; see *Issue 1010* <<https://gitlab.com/nsnam/ns-3-dev/-/issues/1010>>_.
- Support for 22 MHz channels for the HE PHY is limited; see *Issue 402* <<https://gitlab.com/nsnam/ns-3-dev/-/issues/402>>_.
- HT simulations cannot use SingleModelSpectrumChannel in 2.4 GHz; see *Issue 1090* <<https://gitlab.com/nsnam/ns-3-dev/-/issues/1090>>_.
- The spatial-reuse-example.cc produces results inconsistent with its commented expected results; see *Issue 566* <<https://gitlab.com/nsnam/ns-3-dev/-/issues/566>>_.
- There may be lingering issues with the Wi-Fi frame capture model; see *Issue 746* <<https://gitlab.com/nsnam/ns-3-dev/-/issues/746>>_.

The following hyperlink links to *all open wifi module issues* <https://gitlab.com/nsnam/ns-3-dev/-/issues?label_name%5B%5D=module%3A%3Awifi>_, not all of which are bugs (some are related to feature requests).

35.1.3 Design Details

The remainder of this section is devoted to more in-depth design descriptions of some of the Wi-Fi models. Users interested in skipping to the section on usage of the wifi module (*User Documentation*) may do so at this point. We organize these more detailed sections from the bottom-up, in terms of layering, by describing the channel and PHY models first, followed by the MAC models.

We focus first on the choice between physical layer frameworks. *ns-3* contains support for a Wi-Fi-only physical layer model called YansWifiPhy that offers no frequency-level decomposition of the signal. For simulations that involve only Wi-Fi signals on the Wi-Fi channel, and that do not involve frequency-dependent propagation loss or fading models, the default YansWifiPhy framework is a suitable choice. For simulations involving mixed technologies on the same channel, or frequency dependent effects, the SpectrumWifiPhy is more appropriate. The two frameworks are very similarly configured.

The SpectrumWifiPhy framework uses the *Spectrum Module* channel framework.

The YansWifiChannel is the only concrete channel model class in the *ns-3* wifi module. The `ns3::YansWifiChannel` implementation uses the propagation loss and delay models provided within the *ns-3 Propagation* module. In particular, a number of propagation models can be added (chained together, if multiple loss models are added) to the channel object, and a propagation delay model also added. Packets sent from a `ns3::YansWifiPhy` object onto the channel with a particular signal power, are copied to all of the other `ns3::YansWifiPhy` objects after the signal power is reduced due to the propagation loss model(s), and after a delay corresponding to transmission (serialization) delay and propagation delay due to any channel propagation delay model (typically due to speed-of-light delay between the positions of the devices).

Only objects of `ns3::YansWifiPhy` may be attached to a `ns3::YansWifiChannel`; therefore, objects modeling other (interfering) technologies such as LTE are not allowed. Furthermore, packets from different channels do not interact; if a channel is logically configured for e.g. channels 5 and 6, the packets do not cause adjacent channel interference (even if their channel numbers overlap).

WifiPhy and related models

The `ns3::WifiPhy` is an abstract base class representing the 802.11 physical layer functions. Packets passed to this object (via a `Send()` method) are sent over a channel object, and upon reception, the receiving PHY object decides (based on signal power and interference) whether the packet was successful or not. This class also provides a number of callbacks for notifications of physical layer events, exposes a notion of a state machine that can be monitored for MAC-level processes such as carrier sense, and handles sleep/wake/off models and energy consumption. The `ns3::WifiPhy` hooks to the `ns3::FrameExchangeManager` object in the `WifiNetDevice`.

There are currently two implementations of the `WifiPhy`: the `ns3::YansWifiPhy` and the `ns3::SpectrumWifiPhy`. They each work in conjunction with five other objects:

- **PhyEntity**: Contains the amendment-specific part of the PHY processing
- **WifiPpdu**: Models the amendment-specific PHY protocol data unit (PPDU)
- **WifiPhyStateHelper**: Maintains the PHY state machine
- **InterferenceHelper**: Tracks all packets observed on the channel
- **ErrorModel**: Computes a probability of error for a given SNR

PhyEntity

A bit of background

Some restructuring of `ns3::WifiPhy` and `ns3::WifiMode` (among others) was necessary considering the size and complexity of the corresponding files. In addition, adding and maintaining new PHY amendments had become a complex task (especially those implemented inside other modules, e.g. DMG). The adopted solution was to have `PhyEntity` classes that contain the “clause” specific (i.e. HT/VHT/HE/EHT etc) parts of the PHY process.

The notion of “PHY entity” is in the standard at the beginning of each PHY layer description clause, e.g. section 21.1.1 of IEEE 802.11-2016:

:: Clause 21 specifies the **PHY entity** for a very high throughput (VHT) orthogonal frequency division multiplexing (OFDM) system.

Note that there is already such a name inside the wave module (e.g. `WaveNetDevice::AddPhy`) to designate the `WifiPhy`s on each 11p channel, but the wording is only used within the classes and there is no file using that name, so no ambiguity in using the name for 802.11 amendments.

Architecture

The abstract base class `ns3::PhyEntity` enables to have a unique set of APIs to be used by each PHY entity, corresponding to the different amendments of the IEEE 802.11 standard. The currently implemented PHY entities are:

- `ns3::DsssPhy`: PHY entity for DSSS and HR/DSSS (11b)
- `ns3::OfdmPhy`: PHY entity for OFDM (11a and 11p)
- `ns3::ErpOfdmPhy`: PHY entity for ERP-OFDM (11g)
- `ns3::HtPhy`: PHY entity for HT (11n)
- `ns3::VhtPhy`: PHY entity for VHT (11ac)
- `ns3::HePhy`: PHY entity for HE (11ax)
- `ns3::EhtPhy`: PHY entity for EHT (11be)

Their inheritance diagram is given in Figure *PhyEntity hierarchy* and closely follows the standard's logic, e.g. section 21.1.1 of IEEE 802.11-2016:

:: The VHT PHY is **based** on the HT PHY defined in Clause 19, which **in turn** is **based** on the OFDM PHY defined in Clause 17.

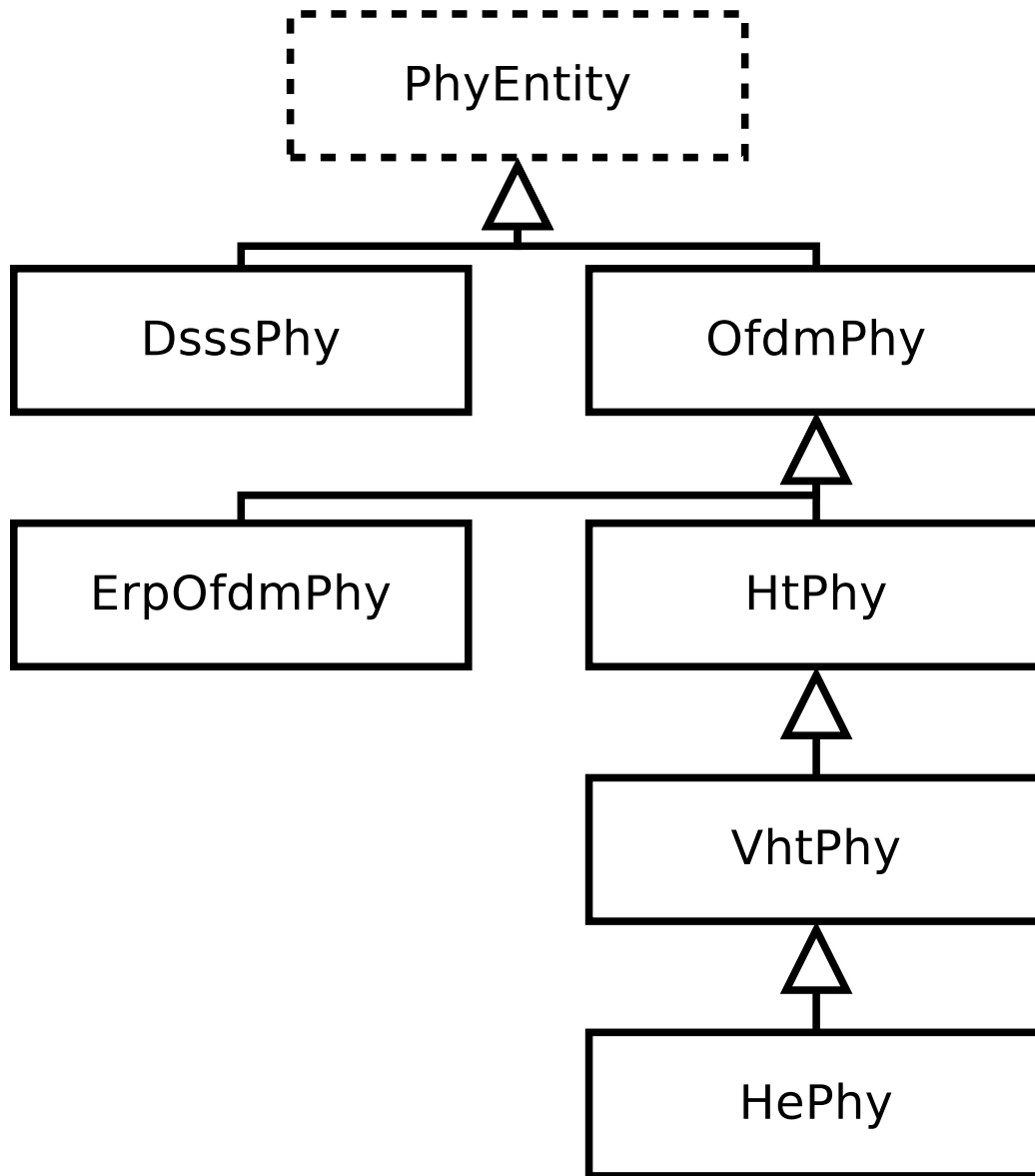


Fig. 35.2: *PhyEntity hierarchy*

Such an architecture enables to handle the following operations in an amendment- specific manner:

- WifiMode handling and data/PHY rate computation,
- PPDU field size and duration computation, and
- Transmit and receive paths.

WifiPdu

In the same vein as `PhyEntity`, the `ns3::WifiPdu` base class has been specialized into the following amendment-specific PPDU:

- `ns3::DsssPdu`: PPDU for DSSS and HR/DSSS (11b)
- `ns3::OfdmPdu`: PPDU for OFDM (11a and 11p)
- `ns3::ErpOfdmPdu`: PPDU for ERP-OFDM (11g)
- `ns3::HtPdu`: PPDU for HT (11n)
- `ns3::VhtPdu`: PPDU for VHT (11ac)
- `ns3::HePdu`: PPDU for HE (11ax)
- `ns3::EhtPdu`: PPDU for EHT (11be)

Their inheritance diagram is given in Figure [WifiPdu hierarchy](#) and closely follows the standard's logic, e.g. section 21.3.8.1 of IEEE 802.11-2016:

:: To maintain compatibility with non-VHT STAs, specific non-VHT fields are defined that can be received by non-VHT STAs compliant with **Clause 17** [OFDM] or **Clause 19** [HT].

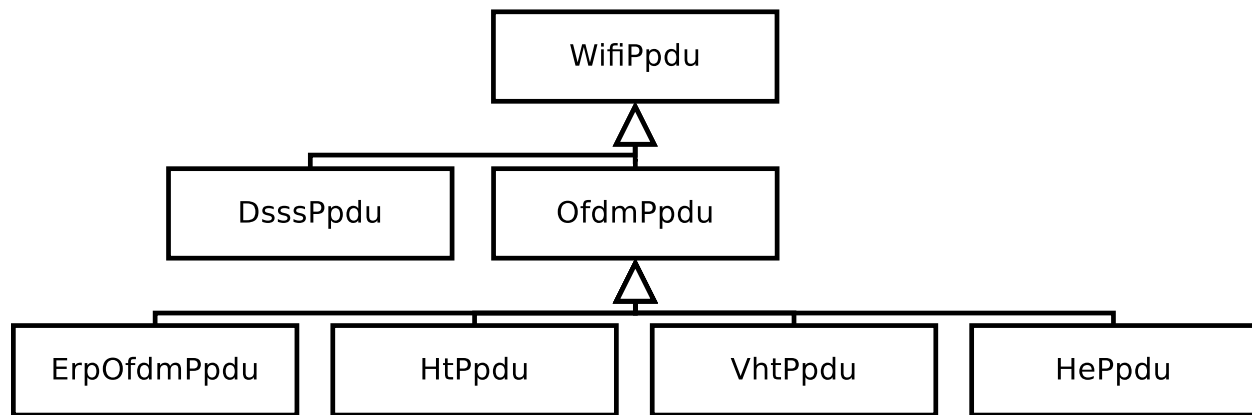


Fig. 35.3: *WifiPdu hierarchy*

YansWifiPhy and WifiPhyStateHelper

Class `ns3::YansWifiPhy` is responsible for taking packets passed to it from the MAC (the `ns3::FrameExchangeManager` object) and sending them onto the `ns3::YansWifiChannel` to which it is attached. It is also responsible to receive packets from that channel, and, if reception is deemed to have been successful, to pass them up to the MAC.

The energy of the signal intended to be received is calculated from the transmission power and adjusted based on the Tx gain of the transmitter, Rx gain of the receiver, and any path loss propagation model in effect.

Class `ns3::WifiPhyStateHelper` manages the state machine of the PHY layer, and allows other objects to hook as *listeners* to monitor PHY state. The main use of listeners is for the MAC layer to know when the PHY is busy or not (for transmission and collision avoidance).

The PHY layer can be in one of these states:

1. TX: the PHY is currently transmitting a signal on behalf of its associated MAC
2. RX: the PHY is synchronized on a signal and is waiting until it has received its last bit to forward it to the MAC.

3. CCA_BUSY: the PHY is issuing a PHY-CCA.indication(BUSY) indication for the primary channel.
4. IDLE: the PHY is not in the TX, RX, or CCA_BUSY states.
5. SWITCHING: the PHY is switching channels.
6. SLEEP: the PHY is in a power save mode and cannot send nor receive frames.
7. OFF: the PHY is powered off and cannot send nor receive frames.

Packet reception works as follows. For YansWifiPhy, most of the logic is implemented in the WifiPhy base class. The YansWifiChannel calls `WifiPhy::StartReceivePreamble()`. The latter calls `PhyEntity::StartReceivePreamble()` of the appropriate PHY entity to start packet reception, but first there is a check of the packet's notional signal power level against a threshold value stored in the attribute `WifiPhy::RxSensitivity`. Any packet with a power lower than `RxSensitivity` will be dropped with no further processing. The default value is -101 dBm, which is the thermal noise floor for 20 MHz signal at room temperature. The purpose of this attribute is two-fold: 1) very weak signals that will not affect the outcome will otherwise consume simulation memory and event processing, so they are discarded, and 2) this value can be adjusted upwards to function as a basic carrier sense threshold limitation for experiments involving spatial reuse considerations. Users are cautioned about the behavior of raising this threshold; namely, that all packets with power below this threshold will be discarded upon reception.

In `StartReceivePreamble()`, the packet is immediately added to the interference helper for signal-to-noise tracking, and then further reception steps are decided upon the state of the PHY. In the case that the PHY is transmitting, for instance, the packet will be dropped. If the PHY is IDLE, or if the PHY is receiving and an optional `FrameCaptureModel` is being used (and the packet is within the capture window), then `PhyEntity::StartPreambleDetectionPeriod()` is called next.

The `PhyEntity::StartPreambleDetectionPeriod()` will typically schedule an event, `PhyEntity::EndPreambleDetectionPeriod()`, to occur at the notional end of the first OFDM symbol, to check whether the preamble has been detected. As of revisions to the model in ns-3.30, any state machine transitions from IDLE state are suppressed until after the preamble detection event.

The `PhyEntity::EndPreambleDetectionPeriod()` method will check, with a preamble detection model, whether the signal is strong enough to be received, and if so, an event `PhyEntity::EndReceiveField()` is scheduled for the end of the preamble and the PHY is put into the CCA_BUSY state. Currently, there is only a simple threshold-based preamble detection model in ns-3, called `ThresholdPreambleDetectionModel`. If there is no preamble detection model, the preamble is assumed to have been detected. It is important to note that, starting with the ns-3.30 release, the default in the `WifiPhyHelper` is to add the `ThresholdPreambleDetectionModel` with a threshold RSSI of -82 dBm, and a threshold SNR of 4 dB. Both the RSSI and SNR must be above these respective values for the preamble to be successfully detected. The default sensitivity has been reduced in ns-3.30 compared with that of previous releases, so some packet receptions that were previously successful will now fail on this check. More details on the modeling behind this change are provided in [lanante2019].

The `PhyEntity::EndReceiveField()` method will check the correct reception of the current preamble and header field and, if so, calls `PhyEntity::StartReceiveField()` for the next field, otherwise the reception is aborted and PHY is put either in IDLE state or in CCA_BUSY state, depending on whether a PHY-CCA.indication(BUSY) is being issued on not for the primary channel.

The next event at `PhyEntity::StartReceiveField()` checks, using the interference helper and error model, whether the header was successfully decoded, and if so, a `PhyRxPayloadBegin` callback (equivalent to the PHY-RXSTART primitive) is triggered. The PHY header is often transmitted at a lower modulation rate than is the payload. The portion of the packet corresponding to the PHY header is evaluated for probability of error based on the observed SNR. The `InterferenceHelper` object returns a value for "probability of error (PER)" for this header based on the SNR that has been tracked by the `InterferenceHelper`. The `PhyEntity` then draws a random number from a uniform distribution and compares it against the PER and decides success or failure.

This is iteratively performed up to the beginning of the data field upon which `PhyEntity::StartReceivePayload()` is called.

Even if packet objects received by the PHY are not part of the reception process, they are tracked by the InterferenceHelper object for purposes of SINR computation and making clear channel assessment decisions. If, in the course of reception, a packet is errored or dropped due to the PHY being in a state in which it cannot receive a packet, the packet is added to the interference helper, and the aggregate of the energy of all such signals is compared against an energy detection threshold to determine whether the PHY should enter a CCA_BUSY state.

A PHY-CCA.indication(BUSY) is issued if a signal occupying the primary channel with a received power above `WifiPhy::CcaSensitivity` (defaulted to -82 dBm) has been received by the PHY or if the measured energy on the primary channel is higher than the energy detection threshold `WifiPhy::CcaEdThreshold` (defaulted to -62 dBm).

When channel bonding is used, CCA indication for signals not occupying the primary channel is also reported. Since 802.11ac and above needs to sense CCA sensitivity for secondary channels larger than 20 MHz, CCA sensitivity thresholds can be adjusted per secondary channel width using `VhtConfiguration::SecondaryCcaSensitivityThresholds` attribute.

For 802.11ax and above, and if the operational bandwidth is equal or larger than 40 MHz, each 20 MHz subchannel of the operational bandwidth is being sensed and PHY-CCA.indication also reports a CCA_BUSY duration indication for each of these 20 MHz subchannel. A zero duration for a given 20 MHz subchannel indicates the 20 MHz subchannel is IDLE.

The above describes the case in which the packet is a single MPDU. For more recent Wi-Fi standards using MPDU aggregation, `StartReceivePayload` schedules an event for reception of each individual MPDU (`ScheduleEndOfMpdus`), which then forwards each MPDU as they arrive up to `FrameExchangeManager`, if the reception of the MPDU has been successful. Once the A-MPDU reception is finished, `FrameExchangeManager` is also notified about the amount of successfully received MPDUs.

InterferenceHelper

The `InterferenceHelper` is an object that tracks all incoming packets and calculates probability of error values for packets being received, and also evaluates whether and for how long energy on the channel rises above a given threshold.

The basic operation of probability of error calculations is shown in Figure *SNIR function over time*. Packets are represented as bits (not symbols) in the *ns-3* model, and the `InterferenceHelper` breaks the packet into one or more “chunks”, each with a different signal to noise (and interference) ratio (SNIR). Each chunk is separately evaluated by asking for the probability of error for a given number of bits from the error model in use. The `InterferenceHelper` builds an aggregate “probability of error” value based on these chunks and their duration, and returns this back to the `WifiPhy` for a reception decision.

From the SNIR function we can derive the Bit Error Rate (BER) and Packet Error Rate (PER) for the modulation and coding scheme being used for the transmission.

If MIMO is used and the number of spatial streams is lower than the number of active antennas at the receiver, then a gain is applied to the calculated SNIR as follows (since STBC is not used):

$$gain(dB) = 10 \log\left(\frac{RX \text{ antennas}}{spatial \text{ streams}}\right)$$

Having more TX antennas can be safely ignored for AWGN. The resulting gain is:

antennas	NSS	gain
2 x 1	1	0 dB
1 x 2	1	3 dB
2 x 2	1	3 dB
3 x 3	1	4.8 dB
3 x 3	2	1.8 dB
3 x 3	3	0 dB
4 x 4	1	6 dB

(continues on next page)

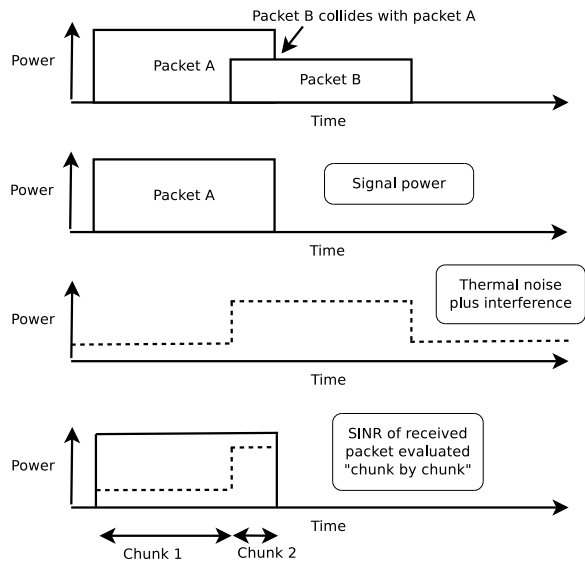


Fig. 35.4: SNIR function over time

(continued from previous page)

4	x	4	2	3 dB
4	x	4	3	1.2 dB
4	x	4	4	0 dB
...				

ErrorRateModel

ns-3 makes a packet error or success decision based on the input received SNR of a frame and based on any possible interfering frames that may overlap in time; i.e. based on the signal-to-noise (plus interference) ratio, or SINR. The relationship between packet error ratio (PER) and SINR in ns-3 is defined by the ns3::ErrorRateModel, of which there are several. The PER is a function of the frame’s modulation and coding (MCS), its SINR, and the specific ErrorRateModel configured for the MCS.

ns-3 has updated its default ErrorRateModel over time. The current (as of ns-3.33 release) model for recent OFDM-based standards (i.e., 802.11n/ac/ax), is the ns3::TableBasedErrorRateModel. The default for 802.11a/g is the ns3::YansErrorRateModel, and the default for 802.11b is the ns3::DsssErrorRateModel. The error rate model for recent standards was updated during the ns-3.33 release cycle (previously, it was the ns3::NistErrorRateModel).

The error models are described in more detail in outside references. The current OFDM model is based on work published in [patidar2017], using link simulations results from the MATLAB WLAN Toolbox, and validated against IEEE TGn results [erceg2004]. For publications related to other error models, please refer to [pei80211ofdm], [pei80211b], [lacage2006yans], [Haccoun], [hepner2015] and [Frenger] for a detailed description of the legacy PER models.

The current ns-3 error rate models are for additive white gaussian noise channels (AWGN) only; any potential frequency-selective fading effects are not modeled.

In summary, there are four error models:

- 1. ns3::TableBasedErrorRateModel: for OFDM modes and reuses ns3::DsssErrorRateModel for 802.11b modes. This is the default for 802.11n/ac/ax.
- 2. ns3::YansErrorRateModel: for OFDM modes and reuses ns3::DsssErrorRateModel for 802.11b modes. This is the default for 802.11a/g.

3. `ns3::DsssErrorRateModel`: contains models for 802.11b modes. The 802.11b 1 Mbps and 2 Mbps error models are based on classical modulation analysis. If GNU Scientific Library (GSL) is installed, the 5.5 Mbps and 11 Mbps from [pursley2009] are used for CCK modulation; otherwise, results from a backup MATLAB-based CCK model are used.
4. `ns3::NistErrorRateModel`: for OFDM modes and reuses `ns3::DsssErrorRateModel` for 802.11b modes.

Users may select either NIST, YANS or Table-based models for OFDM, and DSSS will be used in either case for 802.11b. The NIST model was a long-standing default in ns-3 (through release 3.32).

TableBasedErrorRateModel

The `ns3::TableBasedErrorRateModel` has been recently added and is now the *ns-3* default for 802.11n/ac/ax, while `ns3::YansErrorRateModel` is the *ns-3* default for 802.11a/g.

Unlike analytical error models based on error bounds, `ns3::TableBasedErrorRateModel` contains end-to-end link simulation tables (PER vs SNR) for AWGN channels. Since it is infeasible to generate such look-up tables for all desired packet sizes and input SNRs, we adopt the recommendation of IEEE P802.11 TGax [porat2016] that proposed estimating PER for any desired packet length using BCC FEC encoding by extrapolating the results from two reference lengths: 32 (all lengths less than 400) bytes and 1458 (all lengths greater or equal to 400) bytes respectively. In case of LDPC FEC encoding, IEEE P802.11 TGax recommends the use of a single reference length. Hence, we provide two tables for BCC and one table for LDPC that are generated using a reliable and publicly available commercial link simulator (MATLAB WLAN Toolbox) for each modulation and coding scheme. Note that BCC tables are limited to MCS 9. For higher MCSs, the models fall back to the use of the YANS analytical model.

The validation scenario is set as follows:

1. Ideal channel and perfect channel estimation.
2. Perfect packet synchronization and detection.
3. Phase tracking, phase correction, phase noise, carrier frequency offset, power amplifier non-linearities etc. are not considered.

Several packets are simulated across the link to obtain PER, the number of packets needed to reliably estimate a PER value is computed using the consideration that the ratio of the estimation error to the true value should be within 10 % with probability 0.95. For each SNR value, simulations were run until a total of 40000 packets were simulated.

The obtained results are very close to TGax curves as shown in Figure *Comparison of table-based OFDM Error Model with TGax results*.

Legacy ErrorRateModels

The original error rate model was called the `ns3::YansErrorRateModel` and was based on analytical results. For 802.11b modulations, the 1 Mbps mode is based on DBPSK. BER is from equation 5.2-69 from [proakis2001]. The 2 Mbps model is based on DQPSK. Equation 8 of [ferrari2004]. More details are provided in [lacage2006yans].

The `ns3::NistErrorRateModel` was later added. The model was largely aligned with the previous `ns3::YansErrorRateModel` for DSSS modulations 1 Mbps and 2 Mbps, but the 5.5 Mbps and 11 Mbps models were re-based on equations (17) and (18) from [pursley2009]. For OFDM modulations, newer results were obtained based on work previously done at NIST [miller2003]. The results were also compared against the CMU wireless network emulator, and details of the validation are provided in [pei80211ofdm]. Since OFDM modes use hard-decision on punctured codes, the coded BER is calculated using Chernoff bounds [hepner2015].

The 802.11b model was split from the OFDM model when the NIST error rate model was added, into a new model called `DsssErrorRateModel`.

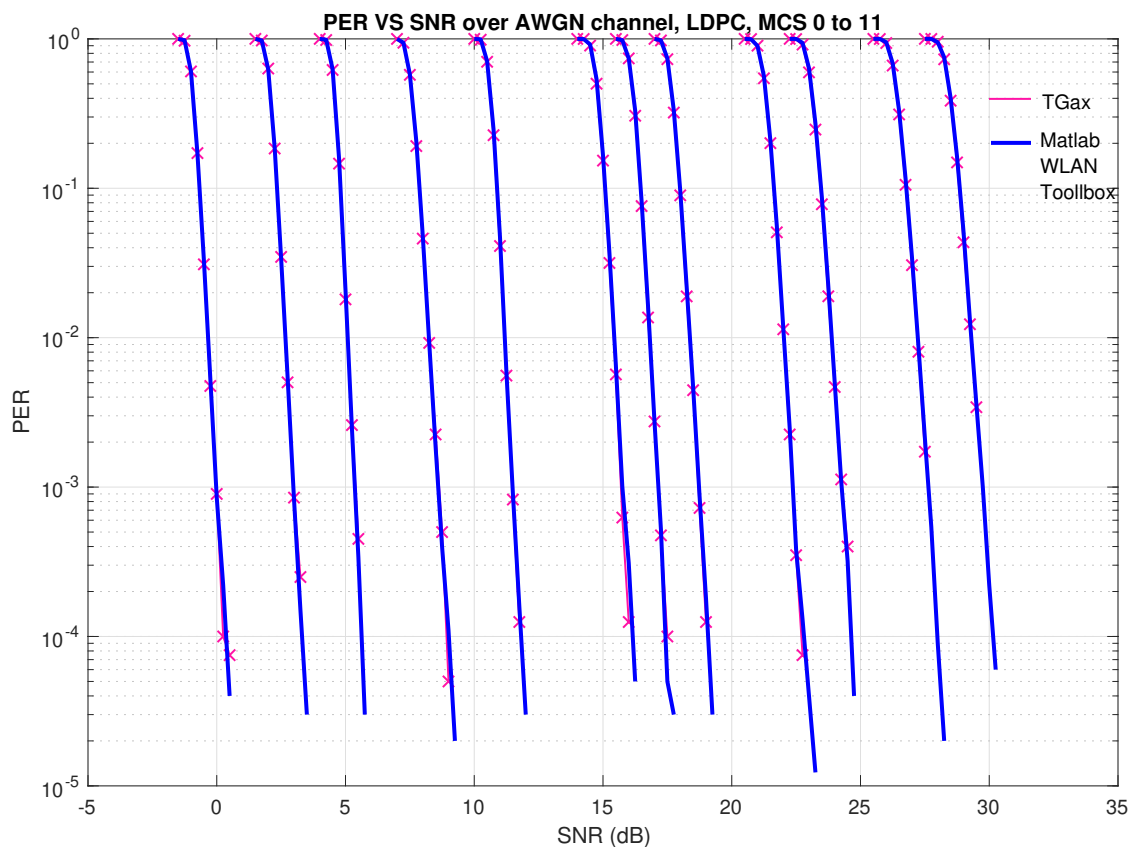


Fig. 35.5: Comparison of table-based OFDM Error Model with TGax results.

Furthermore, the 5.5 Mbps and 11 Mbps models for 802.11b rely on library methods implemented in the GNU Scientific Library (GSL). The ns3 build system tries to detect whether the host platform has GSL installed; if so, it compiles in the newer models from [pursley2009] for 5.5 Mbps and 11 Mbps; if not, it uses a backup model derived from MATLAB simulations.

The error curves for analytical models are shown to diverge from link simulation results for higher MCS in Figure *YANS and NIST error model comparison with TGN results*. This prompted the move to a new error model based on link simulations (the default TableBasedErrorRateModel, which provides curves close to those depicted by the TGN dashed line).

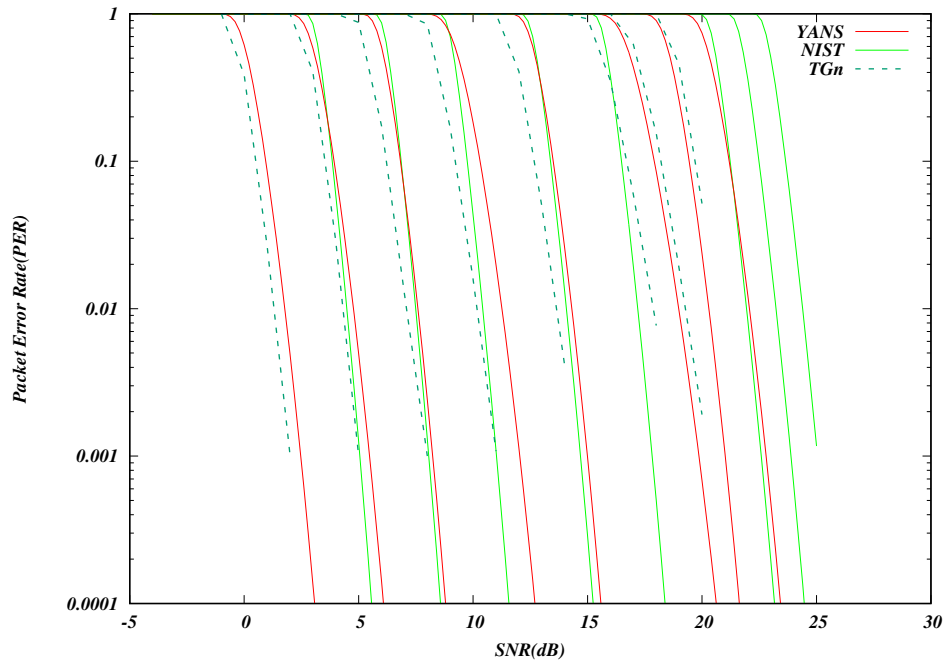


Fig. 35.6: *YANS and NIST error model comparison with TGN results*

SpectrumWifiPhy

This section describes the implementation of the SpectrumWifiPhy class that can be found in `src/wifi/model/spectrum-wifi-phy.{cc,h}`.

The implementation also makes use of additional classes found in the same directory:

- `wifi-spectrum-phy-interface.{cc,h}`
- `wifi-spectrum-signal-parameters.{cc,h}`

and classes found in the spectrum module:

- `wifi-spectrum-value-helper.{cc,h}`

The current SpectrumWifiPhy class reuses the existing interference manager and error rate models originally built for YansWifiPhy, but allows, as a first step, foreign (non Wi-Fi) signals to be treated as additive noise.

Two main changes were needed to adapt the Spectrum framework to Wi-Fi. First, the physical layer must send signals compatible with the Spectrum channel framework, and in particular, the MultiModelSpectrumChannel that allows signals from different technologies to coexist (SingleModelSpectrumChannel may also work for pure Wi-Fi simulations in 5 GHz and 6 GHz bands (but not 2.4 GHz); if you get an error using SingleModelSpectrumChannel, switch to MultiModelSpectrumChannel). Second, the InterferenceHelper must be extended to support the insertion of non-Wi-Fi

signals and to add their received power to the noise, in the same way that unintended Wi-Fi signals (perhaps from a different SSID or arriving late from a hidden node) are added to the noise.

Unlike `YansWifiPhy`, where there are no foreign signals, `CCA_BUSY` state will be raised for foreign signals that are higher than `CcaEdThreshold` (see section 16.4.8.5 in the 802.11-2012 standard for definition of CCA Mode 1). The attribute `WifiPhy::CcaEdThreshold` therefore potentially plays a larger role in this model than in the `YansWifiPhy` model.

To support the Spectrum channel, the `YansWifiPhy` transmit and receive methods were adapted to use the Spectrum channel API. This required developing a few `SpectrumModel`-related classes. The class `WifiSpectrumValueHelper` is used to create Wi-Fi signals with the spectrum framework and spread their energy across the bands. The spectrum is sub-divided into sub-bands (the width of an OFDM subcarrier, which depends on the technology). The power allocated to a particular channel is spread across the sub-bands roughly according to how power would be allocated to sub-carriers. Adjacent channels are modeled by the use of OFDM transmit spectrum masks as defined in the standards.

The class `WifiBandwidthFilter` is used to discard signals early in the transmission process by ignoring any Wi-Fi PPDU whose TX band (including guard bands) does not overlap the current operating channel. Therefore, it bypasses the signal propagation/loss calculations reducing the computational load and increasing the simulation performance. To enable the `WifiBandwidthFilter`, the user can use object aggregation as follows: .. sourcecode:: cpp

```
Ptr<WifiBandwidthFilter>      wifiFilter      =      CreateObject<WifiBandwidthFilter>      ();
Ptr<MultiModelSpectrumChannel> spectrumChannel = CreateObject<MultiModelSpectrumChannel>
(); spectrumChannel->AddSpectrumTransmitFilter(wifiFilter);
```

To support an easier user configuration experience, the existing `YansWifi` helper classes (in `src/wifi/helper`) were copied and adapted to provide equivalent `SpectrumWifi` helper classes.

Finally, for reasons related to avoiding C++ multiple inheritance issues, a small forwarding class called `WifiSpectrumPhyInterface` was inserted as a shim between the `SpectrumWifiPhy` and the Spectrum channel. The `WifiSpectrumPhyInterface` calls a different `SpectrumWifiPhy::StartRx ()` method to start the reception process. This method performs the check of the signal power against the `WifiPhy::RxSensitivity` attribute and discards weak signals, and also checks if the signal is a Wi-Fi signal; non-Wi-Fi signals are added to the `InterferenceHelper` and can raise `CCA_BUSY` but are not further processed in the reception chain. After this point, valid Wi-Fi signals cause `WifiPhy::StartReceivePreamble` to be called, and the processing continues as described above.

Furthermore, in order to support more flexible channel switching, the `SpectrumWifiPhy` can hold multiple instances of `WifiSpectrumPhyInterface` (*Multiple RF interfaces concept*). Each of these instances handles a given frequency range of the spectrum, identified by a start and a stop frequency expressed in MHz, and there can be no overlap in spectrum between them. Only one of these `WifiSpectrumPhyInterface` instances corresponds to the active RF interface of the `SpectrumWifiPhy`, the other ones are referred to as inactive RF interfaces and might be disconnected from the spectrum channel.

If the `SpectrumWifiPhy::TrackSignalsFromInactiveInterfaces` attribute is set to true (default), inactive RF interfaces are connected to their respective spectrum channels and the `SpectrumWifiPhy` also receive signals from these inactive RF interfaces when they belong to a configured portion of the frequency range covered by the interface. The portion of the spectrum being monitored by an inactive interface is specified by a center frequency and a channel width, and is seamlessly set to equivalent of the operating channel of the spectrum PHY that is actively using that frequency range. The `SpectrumWifiPhy` forwards these received signals from inactive interfaces to the `InterferenceHelper` without further processing them. The benefit of the latter is that more accurate PHY-CCA indication can be generated upon channel switching if one or more signals started to be transmitted on the new channel before the switch occurs, which would be ignored otherwise. This is illustrated in Figure *Illustration of signals tracking upon channel switching*, where the parts in red are only generated when `SpectrumWifiPhy::TrackSignalsFromInactiveInterfaces` is set to true.

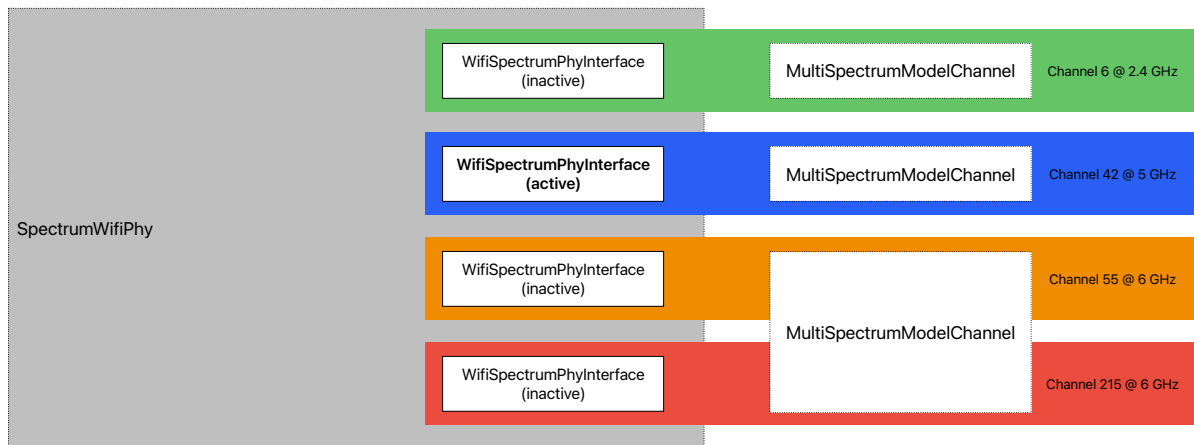


Fig. 35.7: Multiple RF interfaces concept

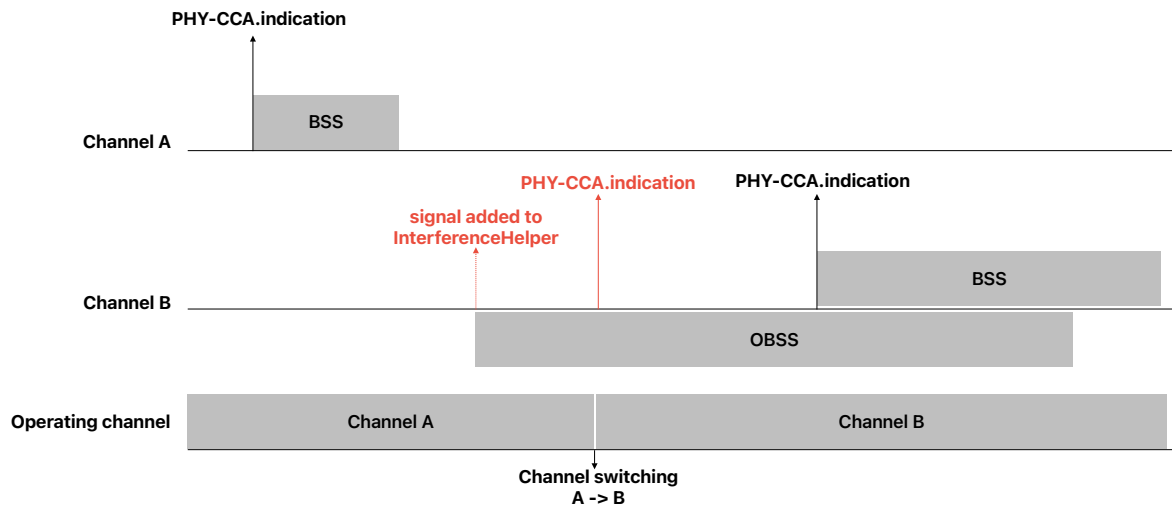


Fig. 35.8: Illustration of signals tracking upon channel switching

TX power

The transmission power level used by a PHY is configured via 3 attributes:

- `WifiPhy::TxPowerLevels` configures the number of power levels supported by the PHY (default 1)
- `WifiPhy::TxPowerStart` configures the minimum Tx power level supported by the PHY (default 16.0206 dBm, equivalent to 40 mW)
- `WifiPhy::TxPowerEnd` configures the maximum Tx power level supported by the PHY (default 16.0206 dBm, equivalent to 40 mW)

Even though the standard allows up to 8 different power levels, the mapping of these integers to power levels is implementation dependent. The ns-3 model allows for up to 255 (1..255) power levels to be supported. If only one power level is configured (the default), the transmission power is equal to `WifiPhy::TxPowerStart` (or `WifiPhy::TxPowerEnd`, since they are equal by default). If more than one power level is configured, the transmission power at a given power level is linearly interpolated in decibels between `WifiPhy::TxPowerStart` and `WifiPhy::TxPowerEnd` according to the following formula:

$$txPower = WifiPhy::TxPowerStart + (powerLevel - 1) \times \frac{WifiPhy::TxPowerEnd - WifiPhy::TxPowerStart}{WifiPhy::TxPowerLevels - 1}$$

where `powerLevel` is the requested power level and is an integer between 1 and `WifiPhy::TxPowerLevels`.

MU-MIMO PHY support

There is a basic support for MU-MIMO at the PHY layer that has been introduced in release ns-3.40. The current model can be used for both downlink and uplink MU-MIMO transmissions.

The use of OFDMA and MU-MIMO for Multi-User transmissions depends on how the TXVECTOR is filled in by the MAC layer (not implemented yet). Since mixed OFDMA and MU-MIMO configurations are not supported, the TXVECTOR determines it carries information for MU-MIMO if all users are assigned the same RU, otherwise it corresponds to an OFDMA transmission.

At the PHY layer, OFDMA and MU-MIMO transmissions are handled in a similar way, the main difference lies in MU-MIMO having the same spectrum shared simultaneously with multiple transmitters (for the uplink direction). The current PHY abstraction model assumes perfect conditions where the interference helper is able to detect signals belonging to the same MU-MIMO transmission and make sure they do not interfere with each others. Interference with other signals, including other MU-MIMO transmissions, is still supported by the model.

The MAC model

Infrastructure association

Association in infrastructure mode is a high-level MAC function performed by the Association Manager, which is implemented through a base class (`WifiAssocManager`) and a default subclass (`WifiDefaultAssocManager`), and controlled by the `AssocType` attribute of the `StaWifiMac` class. This attribute controls whether the non-AP STA/MLD performs a legacy association or an ML setup with the AP device (the latter is only available for EHT devices associating with a multi-link AP; if this option is selected when this condition is not met, it falls back to legacy association automatically). The interaction between the station MAC, the Association Manager base class and subclass is illustrated in Figure [Scanning procedure](#).

The STA wifi MAC requests the Association Manager to start a scanning procedure with specified parameters, including the type of scanning (active or passive), the desired SSID, the list of channels to scan, etc. The STA wifi MAC then expects to be notified of the best AP to associate with at the end of the scanning procedure. Every Beacon or Probe Response frame received during scanning is forwarded to the Association Manager, which keeps a list of candidate

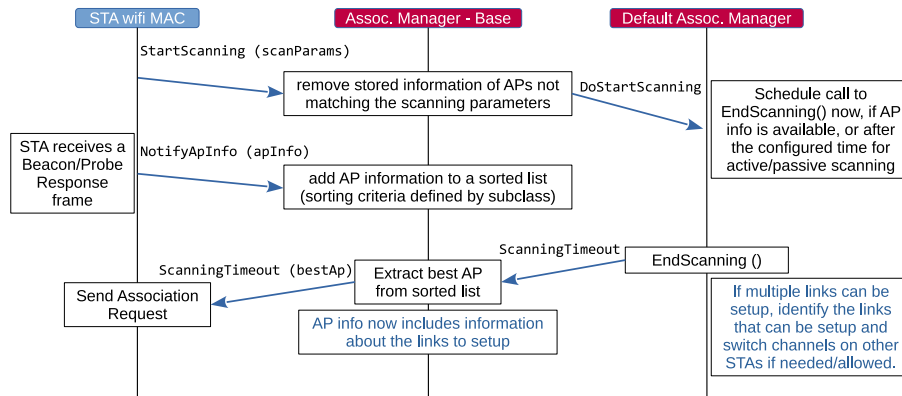


Fig. 35.9: Scanning procedure

APs that match the scanning parameters. The sorting criterium for such a list is defined by the Association Manager subclass. The default Association Manager sorts APs in decreasing order of the SNR of the received Beacon/Probe Response frame.

When notified of the start of a scanning procedure, the default Association Manager schedules a call to a method that processes the information included in the frames received up to the time such a method is called. When both the AP and the STA have multiple links (i.e., they are 802.11be MLDs), the default Association Manager attempts to setup as many links as possible. This involves switching operating channel on some of the STA's links to match those on which the APs affiliated with the AP MLD are operating.

If association is rejected by the AP for some reason, the STA will try to associate to the next best AP until the candidate list is exhausted which then sends STA to 'REFUSED' state. If this occurs, the simulation user will need to force reassociation retry in some way, perhaps by changing configuration (i.e. the STA will not persistently try to associate upon a refusal).

When associated, if the configuration is changed by the simulation user, the STA will try to reassociate with the existing AP.

If the number of missed beacons exceeds the threshold, the STA will notify the rest of the device that the link is down (association is lost) and restart the scanning process. Note that this can also happen when an association request fails without explicit refusal (i.e., the AP fails to respond to association request). In case of non-AP MLDs, in order for losing association, it is necessary that no beacon is received on any link for an interval of duration equal to the maximum number of missed beacons times the interval between two consecutive Beacon frames.

Roaming

Roaming at layer-2 (i.e. a STA migrates its association from one AP to another) is not presently supported. Because of that, the Min/Max channel dwelling time implementation as described by the IEEE 802.11 standard [ieee80211] is also omitted, since it is only meaningful on the context of channel roaming.

Channel access

The 802.11 Distributed Coordination Function is used to calculate when to grant access to the transmission medium. While implementing the DCF would have been particularly easy if we had used a recurring timer that expired every slot, we chose to use the method described in [ji2004sslwn] where the backoff timer duration is lazily calculated whenever needed since it is claimed to have much better performance than the simpler recurring timer solution.

The DCF basic access is described in section 10.3.4.2 of [ieee80211-2016].

- “A STA may transmit an MPDU when it is operating under the DCF access method [...] when the STA determines that the medium is idle when a frame is queued for transmission, and remains idle for a period of a DIFS, or an EIFS (10.3.2.3.7) from the end of the immediately preceding medium-busy event, whichever is the greater, and the backoff timer is zero. Otherwise the random backoff procedure described in 10.3.4.3 shall be followed.”

Thus, a station is allowed not to invoke the backoff procedure if all of the following conditions are met:

- the medium is idle when a frame is queued for transmission
- the medium remains idle until the most recent of these two events: a DIFS from the time when the frame is queued for transmission; an EIFS from the end of the immediately preceding medium-busy event (associated with the reception of an erroneous frame)
- the backoff timer is zero

The backoff procedure of DCF is described in section 10.3.4.3 of [ieee80211-2016].

- “A STA shall invoke the backoff procedure to transfer a frame when finding the medium busy as indicated by either the physical or virtual CS mechanism.”
- “A backoff procedure shall be performed immediately after the end of every transmission with the More Fragments bit set to 0 of an MPDU of type Data, Management, or Control with subtype PS-Poll, even if no additional transmissions are currently queued.”

The EDCA backoff procedure is slightly different than the DCF backoff procedure and is described in section 10.22.2.2 of [ieee80211-2016]. The backoff procedure shall be invoked by an EDCAF when any of the following events occur:

- a frame is “queued for transmission such that one of the transmit queues associated with that AC has now become non-empty and any other transmit queues associated with that AC are empty; the medium is busy on the primary channel”
- “The transmission of the MPDU in the final PPDU transmitted by the TXOP holder during the TXOP for that AC has completed and the TXNAV timer has expired, and the AC was a primary AC”
- “The transmission of an MPDU in the initial PPDU of a TXOP fails [...] and the AC was a primary AC”
- “The transmission attempt collides internally with another EDCAF of an AC that has higher priority”
- (optionally) “The transmission by the TXOP holder of an MPDU in a non-initial PPDU of a TXOP fails”

Additionally, section 10.22.2.4 of [ieee80211-2016] introduces the notion of slot boundary, which basically occurs following $SIFS + AIFSN * slotTime$ of idle medium after the last busy medium that was the result of a reception of a frame with a correct FCS or following $EIFS - DIFS + AIFSN * slotTime + SIFS$ of idle medium after the last indicated busy medium that was the result of a frame reception that has resulted in FCS error, or following a slotTime of idle medium occurring immediately after any of these conditions.

On these specific slot boundaries, each EDCAF shall make a determination to perform one and only one of the following functions:

- Decrement the backoff timer.
- Initiate the transmission of a frame exchange sequence.
- Invoke the backoff procedure due to choosing not to transmit (to be included in the version of the standard following IEEE 802.11-2020)

- Invoke the backoff procedure due to an internal collision.
- Do nothing.

Thus, if an EDCAF decrements its backoff timer on a given slot boundary and, as a result, the backoff timer has a zero value, the EDCAF cannot immediately transmit, but it has to wait for another slotTime of idle medium before transmission can start.

When the Channel Access Manager determines that channel access can be granted, it determines the largest primary channel that is considered idle based on the CCA-BUSY indication provided by the PHY. Such an information is passed to the Frame Exchange Manager, which in turn informs the Multi-User Scheduler (if any) and the Wifi Remote Station Manager. As a result, PPDU's are transmitted on the largest idle primary channel. For example, if a STA is operating on a 40 MHz channel and the secondary20 channel is indicated to be busy, transmissions will occur on the primary20 channel.

In ns-3, by default, beacons for both QoS and non-QoS APs access the channel after a PIFS interval (SIFS plus one slot time), and with zero backoff (CWmin and CWmax both set to zero), giving them higher priority than other access categories. Beacons are given a separate Txop from data frames. The DCF parameters described above are non-standard, but appear to be what some vendors have implemented in their products. To change the DCF parameters for ns-3 access points, the `ApWifiMac::DoCompleteConfig()` method must be manually edited.

The higher-level MAC functions are implemented in a set of other C++ classes and deal with:

- packet fragmentation and defragmentation,
- use of the RTS/CTS protocol,
- rate control algorithm,
- connection and disconnection to and from an Access Point,
- the MAC transmission queue,
- beacon generation,
- MSDU aggregation,
- etc.

Frame Exchange Managers

As the IEEE 802.11 standard evolves, more and more features are added and it is more and more difficult to have a single component handling all of the allowed frame exchange sequences. A hierarchy of `FrameExchangeManager` classes has been introduced to make the code clean and scalable, while avoiding code duplication. Each `FrameExchangeManager` class handles the frame exchange sequences introduced by a given amendment. The `FrameExchangeManager` hierarchy is depicted in Figure [*FrameExchangeManager hierarchy*](#).

The features supported by every `FrameExchangeManager` class are as follows:

- `FrameExchangeManager` is the base class. It handles the basic sequences for non-QoS stations: MPDU followed by Normal Ack, RTS/CTS and CTS-to-self, NAV setting and resetting, MPDU fragmentation
- `QosFrameExchangeManager` adds TXOP support: multiple protection setting, TXOP truncation via CF-End, TXOP recovery, ignore NAV when responding to an RTS sent by the TXOP holder
- `HtFrameExchangeManager` adds support for Block Ack (compressed variant), A-MSDU and A-MPDU aggregation, Implicit Block Ack Request policy
- `VhtFrameExchangeManager` adds support for S-MPDUs
- `HeFrameExchangeManager` adds support for the transmission and reception of multi-user frames via DL OFDMA and UL OFDMA, as detailed below.

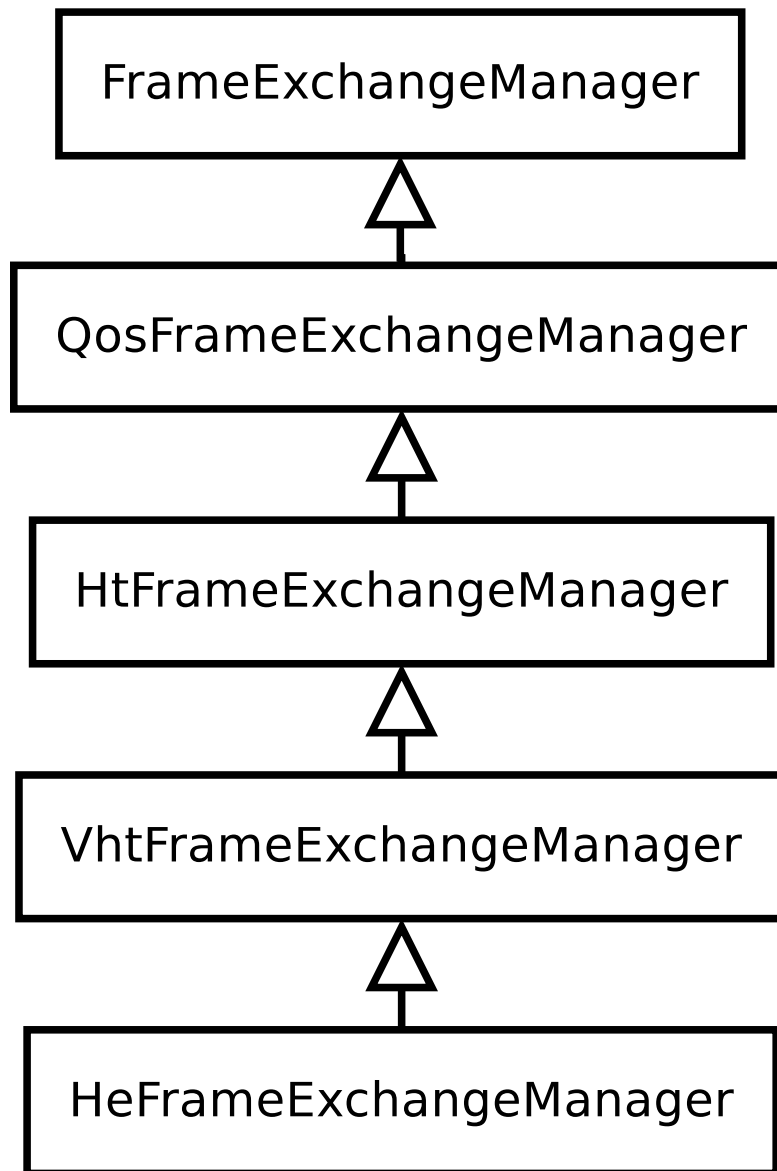


Fig. 35.10: *FrameExchangeManager* hierarchy

FrameExchangeManager classes may have attributes controlling the frame exchange sequences they handle. For instance, the FrameExchangeManager base class has a **ProtectedIfResponded** attribute to enable/disable RTS/CTS protection for stations that have already responded to a frame requiring acknowledgment in the same TXOP, even if such frame had not been protected by RTS/CTS.

MAC queues

Each EDCA function (on QoS stations) and the DCF (on non-QoS stations) have their own MAC queue (an instance of the WifiMacQueue class) to store packets received from the upper layer and waiting for transmission. On QoS stations, each received packet is assigned a User Priority based on the socket priority (see, e.g., the wifi-multi-tos or the wifi-mac-ofdma examples), which determines the Access Category that handles the packet. By default, wifi MAC queues support flow control, hence upper layers do not forward a packet down if there is no room for it in the corresponding MAC queue. Wifi MAC queues do not support dynamic queue limits (byte queue limits); therefore, there is no backpressure into the traffic control layer until the WifiMacQueue for an access category is completely full (i.e., when the queue depth reaches the value of the MaxSize attribute, which defaults to 500 packets). TCP small queues (TSQ) [corbet2012] is a Linux feature that provides feedback from the Wi-Fi device to the socket layer, to control how much data is queued at the Wi-Fi level. ns-3 TCP does not implement TSQ, nor does the WifiNetDevice provide that specific feedback (although some use of the existing trace sources may be enough to support it). Regardless, experimental tests have demonstrated that TSQ interferes with Wi-Fi aggregation on uplink transfers [grazia2022]. Packets stay in the wifi MAC queue until they are acknowledged or discarded. A packet may be discarded because, e.g., its lifetime expired (i.e., it stayed in the queue for too long) or the maximum number of retries was reached. The maximum lifetime for a packet can be configured via the MaxDelay attribute of WifiMacQueue. There are a number of traces that can be used to track the outcome of a packet transmission (see the corresponding doxygen documentation):

- WifiMac trace sources: AckedMpdu, NackedMpdu, DroppedMpdu, MpduResponseTimeout, PsduResponseTimeout, PsduMapResponseTimeout
- WifiMacQueue trace source: Expired

The WifiMac class also provides packet-level trace sources:

- MacTx: A packet has been received by the WifiNetDevice and is about to be enqueued
- MacTxDrop: A packet has been dropped in the MAC layer before being queued for transmission
- MacPromiscRx: A packet has been received and is being forwarded up the local protocol stack (promiscuous trace)
- MacRx: A packet has been received and is being forwarded up the local protocol stack (non-promiscuous trace)
- MacRxDrop: A packet has been dropped in the MAC layer after it has been passed up from the physical layer

Internally, a wifi MAC queue is made of multiple *container queues*, each storing frames of a given type (data, management, or control) addressed to a given receiver and, for QoS data, a given TID. For single-user transmissions, the next container queue to serve among those pending for an Access Category is determined by a *wifi MAC queue scheduler*, held by the WifiMac instance. (For multi-user transmissions, see the Multi-User Scheduler section below; a Multi-User scheduler may or may not consult the wifi MAC queue scheduler to identify the stations to serve with a Multi-User DL or UL transmission.)

The wifi MAC queue scheduler is pluggable. It is modeled by the abstract base class WifiMacQueueScheduler and a templated implementation class WifiMacQueueSchedulerImpl<Priority>, which maintains, per Access Category, a set of container queues sorted by a priority value of type Priority. Concrete subclasses define how that priority is assigned and updated. The WifiSchedPrecedence<Priority> helper wraps a user-supplied priority type and enforces that control frames always precede management frames, which in turn always precede data frames; within the same type, the user priority value breaks ties.

The base class exposes a DropPolicy attribute (DropOldest or DropNewest, default DropNewest) that applies to every subclass and controls what happens when a data frame is enqueued into a full container queue. DropNewest drops the

incoming frame; DropOldest drops the oldest non-in-flight, non-retry data frame across the queues of that AC. Control and management frames are never dropped by the scheduler.

Two concrete schedulers are provided:

- `FcfsWifiQueueScheduler` (the default) assigns, as the priority of each container queue, the enqueue time of the head-of-queue frame. Data frames are therefore served in first-come-first-serve order across all receivers (and TIDs) of an AC.
- `RrWifiQueueScheduler` assigns, as the priority of each container queue, the last time a frame from that queue was transmitted inside a TXOP of which this device is the holder. Container queues of an AC are therefore sorted in increasing order of last-served time, yielding round-robin service among receivers (and, for QoS data, among TIDs of the same receiver).

Power Save mode

The Power Save mode feature requires support from both the APs and the non-AP STAs. The *ns-3* implementation closely follows the 802.11 specifications (IEEE 802.11-2024, section 11.2.3 “Power management in a non-DMG infrastructure network”), which can be summarized as follows:

- As soon as one of the associated STAs switches to powersave mode, the AP buffers all the group addressed frames (by blocking the corresponding queues) and transmits them immediately after a Beacon frame containing a DTIM indicating the presence of buffered group addressed frames (by temporarily unblocking the corresponding queues for the time required to transmit them all).
- The AP buffers all unicast frames addressed to a non-AP STA in powersave mode or to a non-AP MLD having no affiliated non-AP STA in active mode and operating on a link on which the unicast frame can be sent (this is achieved by blocking the queues storing packets destined to such non-AP STA for all the time during which the non-AP STA is in powersave mode). The presence of buffered units for non-AP STAs in powersave mode is indicated in the TIM element of Beacon frames
- When a non-AP STA in powersave mode receives a Beacon frame containing a TIM element indicating the presence of buffered units at the AP, a PS-Poll frame is enqueued and channel access is requested to transmit such a frame.
- An AP replies to a PS-Poll frame received from a non-AP STA in powersave mode by sending a buffered unit addressed to that non-AP STA.
- When a non-AP STA in powersave mode receives a buffered unit from the AP in response to a PS-Poll frame, it acknowledges its reception and enqueues another PS-Poll frame if the received buffered unit indicates the presence of more buffered units at the AP.

Non-AP STAs (possibly affiliated with a non-AP MLD) can be configured to switch between active and powersave mode by using the `PowerSaveMode` attribute of the `PowerSaveManager` class. `PowerSaveManager` is an abstract base class and a default subclass (`DefaultPowerSaveManager`) is provided. The purpose of a Power Save Manager is to switch the state of a non-AP STA in powersave mode between active and doze (i.e., sleep), based on the notifications received from the other MAC components. The behavior of the default Power Save Manager can be summarized as follows:

- A non-AP STA in powersave mode is allowed to sleep (i.e., transition to the doze state) if the last Beacon frame received from the associated AP does not indicate the presence of buffered group addressed frames nor the presence of buffered unicast frames addressed to the non-AP STA, and the non-AP STA has no frames to transmit. Whether a non-AP STA in powersave mode can transition to the doze state is checked in many situations:
 - after receiving a Beacon frame
 - upon completion of the association procedure
 - after receiving a frame from the AP in response to a PS-Poll frame

- after receiving a group addressed frame from the AP
- when releasing the channel after completing a TXOP
- after dropping a PS-Poll frame (e.g., due to reaching the max retry limit)
- When a non-AP STA in powersave mode is put to sleep, an event is scheduled to wake up the non-AP STA right before the expected transmission time of the Beacon frame that is sent a number of Beacon intervals equal to the value of the ListenInterval attribute of the PowerSaveManager class after the last received Beacon frame.
- When an event (such as the enqueue of a data unit from the upper layer or the unblocking of a queue) that triggers a channel access request occurs, the non-AP STA in powersave mode is awakened (if in sleep mode).

The wifi-power-save test suite (src/wifi/test/power-save-test.cc) contains an illustration of the reference frame exchanges.

Multi-user transmissions

Since the introduction of the IEEE 802.11ax amendment, multi-user (MU) transmissions are possible, both in downlink (DL) and uplink (UL), by using OFDMA and/or MU-MIMO. Currently, ns-3 only supports multi-user transmissions via OFDMA. Three acknowledgment sequences are implemented for DL OFDMA.

The first acknowledgment sequence is made of multiple BlockAckRequest/BlockAck frames sent as single-user frames, as shown in Figure *Acknowledgment of DL MU frames in single-user format*.

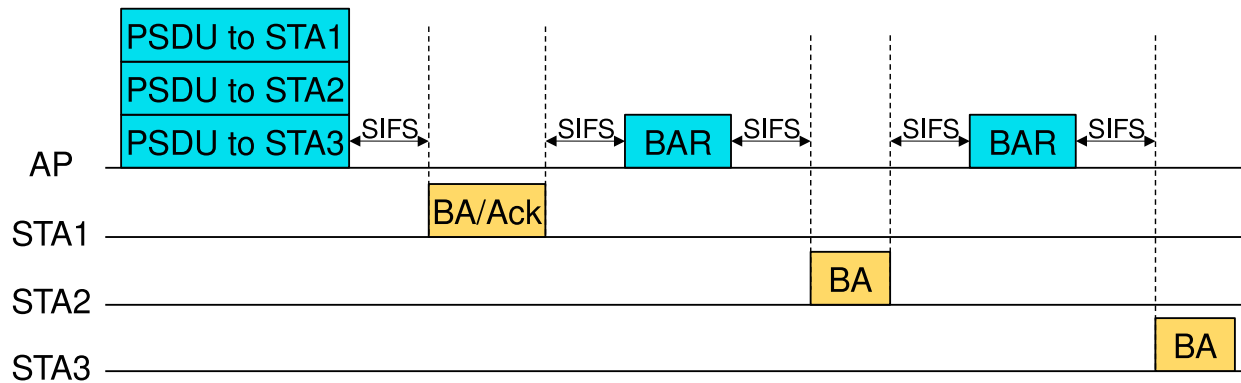


Fig. 35.11: Acknowledgment of DL MU frames in single-user format

For the second acknowledgment sequence, an MU-BAR Trigger Frame is sent (as a single-user frame) to solicit BlockAck responses sent in TB PPDU, as shown in Figure *Acknowledgment of DL MU frames via MU-BAR Trigger Frame sent as single-user frame*.

For the third acknowledgment sequence, an MU-BAR Trigger Frame is aggregated to every PSDU included in the DL MU PPDU and the BlockAck responses are sent in TB PPDU, as shown in Figure *Acknowledgment of DL MU frames via aggregated MU-BAR Trigger Frames*.

For UL OFDMA, both BSRP Trigger Frames and Basic Trigger Frames are supported, as shown in Figure *Frame exchange sequences using UL OFDMA*. A BSRP Trigger Frame is sent by an AP to solicit stations to send QoS Null frames containing Buffer Status Reports. A Basic Trigger Frame is sent by an AP to solicit stations to send data frames in TB PPDU, which are acknowledged by the AP via a Multi-STA BlockAck frame. Note that, in order for the two frame exchange sequences to be separated by a SIFS (as shown in Figure *Frame exchange sequences using UL OFDMA*), it is necessary that the transmitting Access Category has a non-zero TXOP Limit, there is enough remaining time in the TXOP to perform the frame exchange sequence initiated by the Basic Trigger Frame and the Multi-User scheduler (described next) chooses to send a Basic Trigger Frame after a BSRP Trigger Frame.

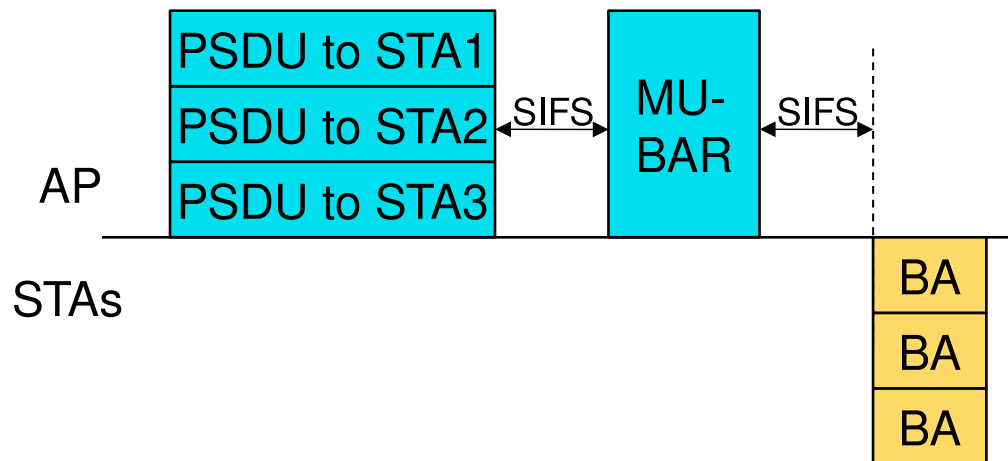


Fig. 35.12: Acknowledgment of DL MU frames via MU-BAR Trigger Frame sent as single-user frame

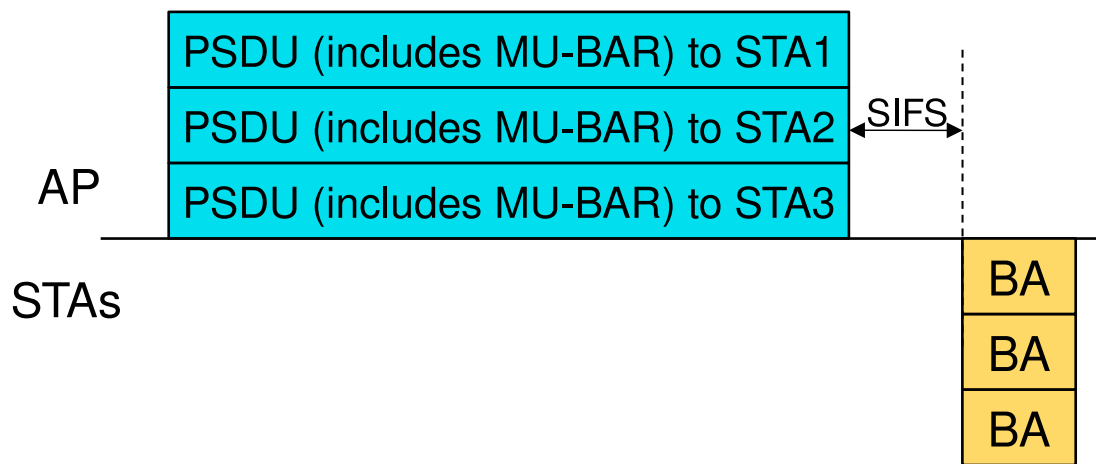


Fig. 35.13: Acknowledgment of DL MU frames via aggregated MU-BAR Trigger Frames

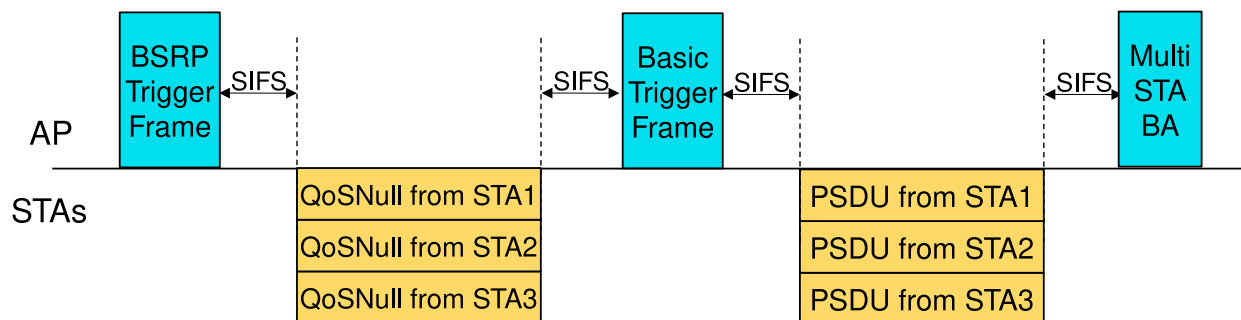


Fig. 35.14: Frame exchange sequences using UL OFDMA

Multi-User Scheduler

A new component, named **MultiUserScheduler**, is in charge of determining what frame exchange sequence the aggregated AP has to perform when gaining a TXOP (DL OFDMA, UL OFDMA or BSRP Trigger Frame), along with the information needed to perform the selected frame exchange sequence (e.g., the set of PSDUs to send in case of DL OFDMA). A TXOP is gained (some time) after requesting channel access, which is normally done by DCF/EDCA (Txop/QosTxop) if the device has frames to transmit. In order for an AP to coordinate UL MU transmissions even without DL traffic, the duration of the access request interval can be set to a non-zero value through the `AccessReqInterval` attribute. The access request interval is the interval between two consecutive requests for channel access made by the **MultiUserScheduler**; such requests are made independently of the presence of frames in the queues of the AP. It is also possible to set the Access Category for which the **MultiUserScheduler** makes requests for channel access (via the `AccessReqAc` attribute) and to choose whether the access request interval is measured starting from the last time the **MultiUserScheduler** made a request for channel access or from the last time channel access was obtained by DCF/EDCA (via the `DelayAccessReqUponAccess` attribute).

MultiUserScheduler is an abstract base class. Currently, the only available subclass is **RrMultiUserScheduler**. By default, no multi-user scheduler is aggregated to an AP (hence, OFDMA is not enabled).

Round-robin Multi-User Scheduler

The Round-robin Multi-User Scheduler dynamically assigns a priority to each station to ensure airtime fairness in the selection of stations for DL multi-user transmissions. The `NStations` attribute enables to set the maximum number of stations that can be the recipients of a DL multi-user frame. Therefore, every time an HE AP accesses the channel to transmit a DL multi-user frame, the scheduler determines the number of stations the AP has frames to send to (capped at the value specified through the mentioned attribute) and attempts to allocate equal sized RUs to as many such stations as possible without leaving RUs of the same size unused. For instance, if the channel bandwidth is 40 MHz and the determined number of stations is 5, the first 4 stations (in order of priority) are allocated a 106-tone RU each (if 52-tone RUs were allocated, we would have three 52-tone RUs unused). If central 26-tone RUs can be allocated (as determined by the `UseCentral26TonesRus` attribute), possible stations that have not been allocated an RU are assigned one of such 26-tone RU. In the previous example, the fifth station would have been allocated one of the two available central 26-tone RUs.

When UL OFDMA is enabled (via the `EnableUlofdma` attribute), every DL OFDMA frame exchange is followed by an UL OFDMA frame exchange involving the same set of stations and the same RU allocation as the preceding DL multi-user frame. The transmission of a BSRP Trigger Frame can optionally (depending on the value of the `EnableBsrp` attribute) precede the transmission of a Basic Trigger Frame in order for the AP to collect information about the buffer status of the stations.

Enhanced multi-link single radio operation (EMLSR)

The IEEE 802.11be amendment introduced EMLSR operating mode to allow a non-AP MLD to alternate frame exchanges over a subset of setup links identified as EMLSR links (see section 35.3.17 of IEEE 802.11be D4.1). *ns-3* supports EMLSR operations as described in the following.

Architecture of a non-AP MLD supporting EMLSR operating mode

The architecture of a non-AP MLD supporting EMLSR operating mode is based on the assumption that only one PHY instance (referred to as the *main PHY*) has full TX/RX capabilities, while the other PHY instances (referred to as *auxiliary PHYs* or *aux PHYs* for brevity) have limited TX/RX capabilities. As a consequence, only the main PHY is able to carry out frame exchanges with the AP MLD. Given that frame exchanges can occur on any of the EMLSR links, the link on which the main PHY operates is dynamically switched during a simulation, as detailed below.

Enabling/disabling EMLSR mode

EMLSR mode can be enabled on the link(s) of a non-AP EHT device that supports the EMLSR operating mode and performs ML setup with an AP MLD that supports the EMLSR operating mode. The `EmlsrActivated` attribute of the EHT configuration of an EHT device determines whether the EMLSR operating mode is supported by the device. When the `EmlsrActivated` attribute is set to true for a non-AP EHT device and the `ns3::StaWifiMac::AssocType` attribute is set to `ML_SETUP`, the `WifiMacHelper` will install an EMLSR Manager by using the type and attribute values configured through the `SetEmlsrManager` method.

EMLSR mode on the links of a non-AP MLD can be enabled or disabled by using the `EmlsrLinkSet` attribute of the EMLSR Manager base class (after multi-link setup, EMLSR mode is disabled by default). Setting the `EmlsrLinkSet` attribute triggers the transmission of an EML Operating Mode Notification frame to the AP to communicate the new set of EMLSR links, if ML setup has been completed. Otherwise, the set of EMLSR links is stored and the EML Operating Mode Notification frame is sent as soon as the ML setup is completed. Therefore, users can choose to enable EMLSR mode on some links upon the completion of ML setup by setting the `EmlsrLinkSet` attribute at (or before) initialization time; alternatively, they can leave the `EmlsrLinkSet` attribute empty at initialization time and set it at runtime to enable EMLSR mode on some links at a specific simulation time (after ML setup). The selection of the link used by the non-AP MLD to transmit the EML Operating Mode Notification frame is done by the EMLSR Manager subclass. The default EMLSR Manager subclass, `DefaultEmlsrManager`, selects the link on which the main PHY is operating. When the non-AP MLD receives the acknowledgment for the EML Operating Mode Notification frame, it starts a timer whose duration is the transition timeout advertised by the AP MLD. When the timer expires, or the non-AP MLD receives an EML Operating Mode Notification frame from the AP MLD, the EMLSR mode is assumed to be enabled on the requested set of links (referred to as EMLSR links), if this set is not empty, or disabled, otherwise. The set of links on which it is requested to enable EMLSR mode must include the link on which the main PHY is operating; the PHY instances operating on the other links on which EMLSR mode is enabled are considered aux PHYs.

The PHY instance acting as main PHY is configured through the `MainPhyId` attribute of the EMLSR Manager base class. Such a class also enables to define the TX/RX capabilities of the aux PHYs:

- the `AuxPhyMaxModClass` attribute indicates the maximum modulation class supported by aux PHYs
- the `AuxPhyChannelWidth` attribute indicates the maximum channel width (MHz) supported by aux PHYs. The value of this attribute may be automatically capped based on the maximum supported modulation class.
- the `AuxPhyTxCapable` attribute indicates whether aux PHYs are capable of transmitting frames

The EMLSR Manager base class also provides the `PutAuxPhyToSleep` attribute to control whether aux PHYs opportunistically switch to the sleep state. Specifically, when this attribute is set to true, the behavior is as follows. For DL TXOPs, aux PHYs are put to sleep after an ICF is received; for UL TXOPs, aux PHYs are put to sleep when the CTS frame is received, if RTS/CTS is used, or when the transmission of the data frame starts, otherwise. Aux PHYs are resumed from sleep when the TXOP ends.

EMLSR operations, as detailed below, may lead to situations in which no PHY operates on a link for a certain time interval. Whether to freeze or reset the backoff counters for that link during such intervals is controlled by the `ResetBackoffThreshold` attribute of the `ChannelAccessManager`: if the duration of the interval during which no PHY operates on a link is longer than the value of this attribute, the backoff counters are reset; otherwise, they are frozen until a PHY is connected to the link.

Downlink TXOP

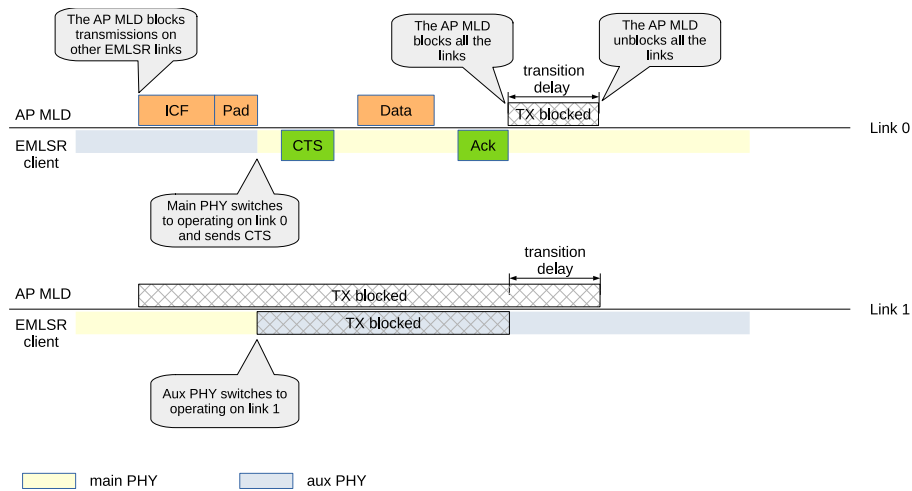


Fig. 35.15: EMLSR operations: Downlink TXOP

When an AP MLD that supports EMLSR operating mode has to initiate a frame exchange on a link with a non-AP MLD that is operating in EMLSR mode on that link, it sends an MU-RTS Trigger Frame soliciting a response from the non-AP MLD (and possibly others) as the initial Control frame (ICF) for that exchange (see Figure *EMLSR operations: Downlink TXOP*). The MU-RTS Trigger Frame is carried in a non-HT duplicate PPDU transmitted at a rate of 6 Mbps, 12 Mbps or 24 Mbps. When the transmission of an initial Control frame starts, the AP MLD blocks transmissions to the solicited EMLSR clients on the EMLSR links other than the link used to transmit the initial Control frame, so that the AP MLD does not initiate another frame exchange on such links. The MU-RTS Trigger Frame includes a Padding field whose transmission duration is the maximum among the padding delays advertised by all the EMLSR clients solicited by the MU-RTS Trigger Frame. The padding delay should be long enough to allow the EMLSR client to get ready to transmit the CTS response (e.g., make the main PHY switch to operate on the link where the DL TXOP is starting). In *ns-3*, if the ICF is received by the main PHY, no link switch occurs. If the ICF is received by an aux PHY (as shown in Fig. *EMLSR operations: Downlink TXOP*), the main PHY switches to operate on the link where the TXOP is starting at the end of the reception of the ICF (including the Padding field) and the main PHY sends the CTS response. In *ns-3*, the behavior of the aux PHY that received the ICF is determined by the EMLSR Manager subclass. The `DefaultEmlsrManager` class provides the `SwitchAuxPhy` attribute for this purpose. If this attribute is true, the aux PHY switches to operate on the link on which the main PHY was operating and stays on such a link even after the completion of the TXOP (as shown in Fig. *EMLSR operations: Downlink TXOP*); if this attribute is false, the aux PHY does not perform any switch, but the main PHY switches back to operate on its original link once the TXOP is completed (as shown in Fig. *EMLSR operations: Uplink TXOP*).

The frame exchange with an EMLSR client is assumed to terminate when the AP MLD does not start a frame transmission a SIFS after the response to the last frame transmitted by the AP MLD or the AP MLD transmits a frame that is not addressed to the EMLSR client. When a frame exchange with an EMLSR client terminates, the AP MLD blocks transmissions on all the EMLSR links and starts a timer whose duration is the transition delay advertised by the EMLSR client. When the timer expires, the EMLSR client is assumed to be back to the listening operations and transmissions on all the EMLSR links are unblocked.

The padding delay and the transition delay for an EMLSR client can be set through the `EmlsrPaddingDelay` attribute and the `EmlsrTransitionDelay` attribute of the `EmlsrManager` base class, respectively.

Uplink TXOP

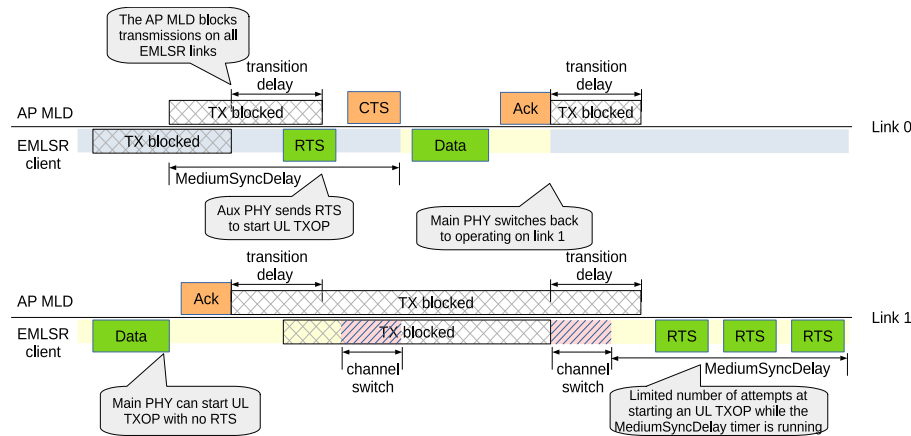


Fig. 35.16: EMLSR operations: Uplink TXOP

An EMLSR client can normally start an UL TXOP on any of the EMLSR links, provided that no DL or UL TXOP is ongoing on another EMLSR link. When channel access is obtained on a link on which the main PHY is operating, the EMLSR client can directly transmit a Data frame, provided that the protection manager does not request to use a protection mechanism and no `MediumSyncDelay` timer is running on that link (see below). Figure *EMLSR operations: Uplink TXOP* shows that the EMLSR client obtains a TXOP on link 1, which the main PHY is operating on, and a Data frame is transmitted without protection. Clearly, the EMLSR client blocks transmissions on the other EMLSR links as soon as it starts transmitting the Data frame, while the AP MLD blocks transmissions to the EMLSR client on the other EMLSR links as soon as it receives an MPDU from the EMLSR client. When the UL TXOP terminates, the AP MLD starts a transition delay timer, during which it does not attempt to start a frame exchange with the EMLSR client on any EMLSR link.

When channel access is obtained on a link on which an aux PHY is operating, the behavior depends on whether the aux PHY is TX capable or not, as controlled via the `AuxPhyTxCapable` attribute of the EMLSR Manager base class.

Let us consider first the case in which the aux PHY is capable of transmitting PPDUs. As shown in Fig. *EMLSR operations: Uplink TXOP*, the EMLSR client obtains a TXOP on link 0, which an aux PHY is operating on. The aux PHY has to transmit an RTS frame (independently of whether the protection manager requests to use a protection mechanism or not) and receive the CTS response from the AP. The RTS frame is actually transmitted by the aux PHY if the main PHY is able to switch to that link and be ready to take over the UL TXOP right after the reception of the CTS frame. The Default EMLSR Manager allows the transmission of the RTS frame if the channel switch delay (plus the remaining switching time if the main PHY is currently switching) does not exceed the remaining time until the end of the reception of the CTS frame.

In the example shown in Fig. *EMLSR operations: Uplink TXOP*, the `SwitchAuxPhy` attribute of the `DefaultEmlsrManager` class is set to false. This means that the aux PHY stays on link 0 and, therefore, no PHY is operating on link 1 while the main PHY is carrying out the UL TXOP on link 0. Once such UL TXOP terminates, the main PHY is switched back to operate on link 1.

If the aux PHY is not TX capable, the behavior is determined by the specific EMLSR Manager subclass. The Default EMLSR Manager simply does nothing, i.e., the transmit opportunity gained by the aux PHY is just dropped.

MediumSyncDelay timer

An EMLSR client may lose medium sync on a link for several reasons:

- the EMLSR client transmits a PPDU on another link, the TX duration exceeds a predefined threshold of 72 microseconds (as defined in the 802.11be amendment) and the `InDeviceInterference` attribute of the EMLSR Manager base class is set to true (which implies that, when the PHY of a device transmits, an interference is generated such that the other PHYs of the same device are not able to decode anything and cannot decrease the backoff counter)
- No PHY operates on the link for at least 72 microseconds
- The aux PHY operating on the link awakes after sleeping for at least 72 microseconds

When the EMLSR client loses medium sync on a link for a period of time longer than 72 microseconds, it has to start a `MediumSyncDelay` timer for that link. While the `MediumSyncDelay` timer is running for a link, the EMLSR client can still access the medium on that link, but it has to mandatorily use RTS/CTS protection for the frame exchange. Additionally, the EMLSR client can perform at most a predefined number of attempts to start an UL TXOP on a link for which a `MediumSyncDelay` timer is running. The `MediumSyncDelay` timer for a link is cancelled if an MPDU is received on that link. The duration of the `MediumSyncDelay` timer, the maximum number of TXOP attempts and the threshold to be used instead of the normal CCA sensitivity for the primary 20 MHz channel are all advertised by the AP MLD and can be configured through the attributes of the `EmlsrManager` class: `MediumSyncDuration`, `MsdMaxNTxops` and `MsdOfdmEdThreshold`, respectively.

Figure *EMLSR operations: Uplink TXOP* shows that the transmission of the Data frame on link 1 causes a `MediumSyncDelay` timer to start on link 0; this timer is cancelled when the CTS frame is subsequently received. Similarly, a `MediumSyncDelay` timer is started on link 1 after that no PHY operated on that link (for at least 72 microseconds). While the `MediumSyncDelay` timer is running on link 1, the EMLSR client obtains a TXOP on link 1 (on which the main PHY is operating) and, as mandated by the 802.11be specs, sends an RTS frame. If there is no response to the RTS frame, the EMLSR client can attempt to obtain the TXOP again and transmit another RTS frame. However, the number of attempts performed while the `MediumSyncDelay` timer is running is limited by the value advertised by the AP MLD and configured through the `MsdMaxNTxops` attribute of the `EmlsrManager` class.

Advanced EMLSR Manager

The Advanced EMLSR Manager explores a few opportunities to improve the performance of EMLSR clients. In case aux PHYs are not TX capable, the Advanced EMLSR Manager implements mechanisms for the main PHY to switch to an auxiliary link and start an UL TXOP. More specifically, when the backoff counter reaches zero on an auxiliary link, it is checked whether a PPDU that might be an ICF is being received on any other EMLSR link. A PPDU being received might be an ICF if any of the following conditions holds:

- the PHY header of the PPDU is being received and the modulation class is non-HT
- the MAC header of an MPDU in the PPDU has been received, the `UseNotifiedMacHdr` attribute of the EMLSR Manager base class is true and the MAC header indicates this MPDU is a Trigger Frame addressed to the EMLSR client or to the broadcast address
- the payload of the PPDU is being received, the MAC header of an MPDU in the PPDU has not been received yet or the `UseNotifiedMacHdr` attribute of the EMLSR Manager base class is false, and the modulation class is non-HT

If a PPDU being received on another EMLSR link might be an ICF and the `AllowULTxopInRx` attribute of the Advanced EMLSR Manager is set to false, then the UL transmit opportunity is dropped. Otherwise, it is checked whether it is possible and convenient for the main PHY to switch to the auxiliary link. The following conditions must all hold in order to request the main PHY to switch to the auxiliary link:

- the main PHY is not in TX state

- the main PHY is not switching (unless switching can be interrupted, see below) nor is it trying to get channel access on another auxiliary link
- no MediumSyncDelay timer is running on the auxiliary link or the maximum number of TXOP attempts has not yet been reached
- the main PHY is expected to get channel access on the auxiliary link more quickly. More precisely, let AC X be the AC that is about to gain channel access on the aux PHY link, the main PHY is requested to switch if we do not expect any AC, with priority higher than or equal to AC X and with frames to send on the main PHY link, to gain channel access on the main PHY link before AC X is able to start transmitting on the aux PHY link.

Note that the last condition is not checked if the `CheckAccessOnMainPhyLink` attribute is set to false and AC X has a priority higher than or equal to the AC specified by the `MinAcToSkipCheckAccess` attribute.

If the main PHY switches to the auxiliary link, it cannot transmit as soon as it completes the channel switch, but it has to verify that the medium has been virtually and physically idle during a PIFS period by means of a NAV and CCA check. If the `UseAuxPhyCca` attribute of the Advanced EMLSR Manager is set to true or the maximum channel width supported by the aux PHY is at least equal to the maximum channel width supported by the main PHY, then the NAV and CCA check is performed by the aux PHY in the PIFS period preceding the main PHY channel switch end. Otherwise, the NAV and CCA check is performed by the main PHY in the PIFS period following the main PHY channel switch end.

If the NAV and CCA check indicates that the medium has not been idle in the PIFS period, the main PHY has to contend for channel access on the auxiliary link. However, in case the aux PHYs do not switch link, the main PHY will attempt to get channel access on the auxiliary link for a limited amount of time, indicated by the `SwitchMainPhyBackDelay` attribute, and then it will switch back to the preferred link, in order not to leave the preferred link without a PHY operating on it for a too long time. Specifically, at the end of the PIFS period during which the medium has been determined to be busy, if channel access is not expected to be gained on the auxiliary link for any AC that has traffic to send on that link within a `SwitchMainPhyBackDelay` plus a channel switch delay (due to, e.g., NAV reservation), the main PHY switches back to the preferred link (unless there is a reason to postpone the switch, see below). Otherwise, a `SwitchMainPhyBack` timer is started having a duration of `SwitchMainPhyBackDelay`. The Advanced EMLSR Manager also connects a PHY listener to the main PHY, so that it is notified of events such as RX start, RX end, TX start and CCA busy start. If the Advanced EMLSR Manager is notified of such events, or is notified of the reception of the MAC header of an MPDU by the main PHY, while the `SwitchMainPhyBack` timer is running and channel access is not expected to be gained on the auxiliary link for any AC that has traffic to send on that link within the remaining time until the `SwitchMainPhyBack` timer expires plus a channel switch delay, then the main PHY switches back to the preferred link unless there is a reason to postpone the switch. The switch back to the preferred link is postponed if:

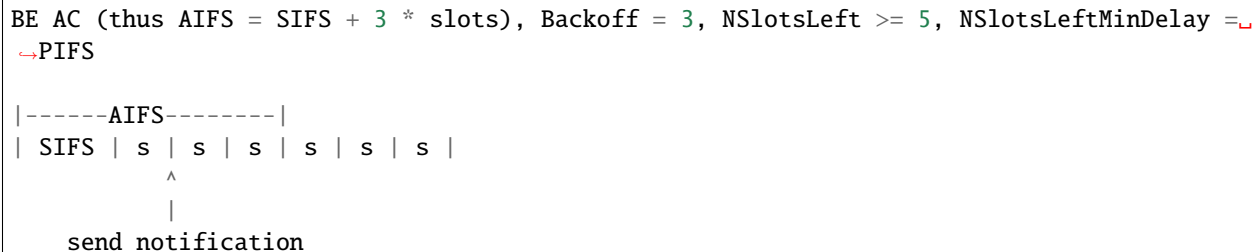
- a PPDU that might be an ICF is being received on an EMLSR link on which the NAV is not set or the TXOP holder is the associated AP MLD, OR
- on the link on which the main PHY is currently operating the NAV is not set or the TXOP holder is the associated AP MLD, the medium is idle and channel access is expected to be gained within a channel switch delay

A similar check is applied to possibly postpone the switch back to the preferred link when the `SwitchMainPhyBack` timer expires.

The Advanced EMLSR Manager provides the `KeepMainPhyAfterDLTxop` attribute to control the behavior of the main PHY at the end of a DL TXOP carried out on an aux PHY link, in case aux PHYs are not TX capable and do not switch link. If such attribute is set to false (default value), the main PHY immediately switches back to the preferred link. If such attribute is set to true, it is checked whether channel access on the aux PHY link is expected to be gained within a switch main PHY back delay plus a channel switch delay: if it is, the main PHY stays on the aux PHY link and a switch main PHY back timer is started; otherwise, the main PHY switches back to the preferred link.

The Advanced EMLSR Manager also connects a callback to the `NSlotsLeftAlert` trace source of the Channel Access Manager, which sends notifications when at most a configurable number of slots remain until the backoff of an AC expires. It must be noted that this notification is only sent if channel access has been requested by the AC for which the number of remaining backoff slots has reached the given threshold. Thus, if an AC terminates a TXOP and generates a new backoff value, but it has no packets in the queue, then channel access is not requested and the notification is

not scheduled; if a new packet arrives after that the backoff counter has reached zero, channel access is requested and notification is sent immediately (while channel access is gained at the next slot boundary, which may be a few microseconds later). If channel access is requested while the backoff counter is non-zero, the notification is sent when the number of slots remaining until the backoff of an AC expires reaches the configured value, if the backoff counter starts at a value greater than or equal to the configured value, or as soon as the notification can be sent based on the value of the `NSlotsLeftMinDelay` attribute of the Channel Access Manager. This attribute indicates the minimum amount of time that must elapse since the start of the AIFS to enable the dispatching of the notification. Example:



When the Advanced EMLSR Manager receives such a notification, it evaluates the opportunity of switching the main PHY to the auxiliary link on which the notification has been received. Specifically, the Advanced EMLSR Manager performs the check described above to determine whether the potential UL transmit opportunity shall be dropped (it is dropped if a PPDU being received on another EMLSR link might be an ICF and the `AllowULTxopInRx` attribute is set to false) and then it checks whether it is convenient for the main PHY to switch to the auxiliary link as described above (with the exception that the expected delay until backoff end is also taken into account). If the main PHY is requested to switch to the auxiliary link and the backoff on the auxiliary link counts down to zero while the main PHY is switching, a NAV and CCA check is performed as described above (by the aux PHY in the PIFS period preceding the main PHY channel switch end or by the main PHY in the PIFS period following the main PHY channel switch end). If the remaining backoff time when the main PHY completes the switch to the aux PHY link is greater than or equal to a PIFS, no NAV and CCA check is scheduled, as the main PHY has enough time to perform CCA on the aux PHY link. If the remaining backoff time when the main PHY completes the switch to the aux PHY link is less than a PIFS, the UL TXOP starts as soon as the backoff counter reaches zero, if aux PHY CCA can be used, or a PIFS after the end of the main PHY switch (provided that the NAV and CCA check indicates medium idle), if main PHY CCA is requested.

The Advanced EMLSR Manager has the `InterruptSwitch` attribute that can be set to true to interrupt a main PHY switch when it is determined that the main PHY shall switch to a different link while still completing the previous switch; in such cases, the previous switch is interrupted and the new channel switch starts. This opportunity is exploited in some situations:

- If the main PHY is switching while a TX capable aux PHY transmits an RTS to start an UL TXOP, the main PHY switch can be interrupted and the main PHY can start switching to the link on which the RTS has been sent, in order to meet the constraint that the main PHY shall be operating on that link right after the reception of the CTS frame.
- The main PHY is requested to switch to a link on which a TX capable aux PHY has sent an RTS, but a CTS timeout occurs while the main PHY is switching. If the aux PHYs do not switch link, the main PHY switch is interrupted and the main PHY returns to the preferred link; if the aux PHYs switch link, the main PHY switch is interrupted and the main PHY returns to the link it just left.
- If the main PHY is switching while an aux PHY is receiving an ICF, the switch can be interrupted so that the main PHY can start switching to the aux PHY link and be ready to operate on that link upon reception of the ICF.
- If channel access is gained, or is about to be gained, on a link on which a non-TX capable aux PHY is operating and the main PHY is switching, the main PHY switch can be interrupted and the main PHY can start switching to the aux PHY link, provided that the main PHY was not switching to start a (DL or UL) TXOP and the other conditions to request the main PHY to switch are satisfied.

AP EMLSR Manager

A manager which takes EMLSR specific decisions can also be installed on the AP MLD side. The base class is named `ApEmlsrManager`, which is inherited by the `DefaultApEmlsrManager` and the `AdvancedApEmlsrManager`. One of the choices that the AP MLD has to take is how to behave in case the transmission of an ICF fails due to a cross link collision, i.e., an ICF that is addressed to EMLSR client(s) fails because the EMLSR client(s) are sending or have sent an ICF on another EMLSR link. By default, an ICF failure is treated just like any other failure: the contention window is updated and the failure is notified to the remote station manager. However, the advanced AP EMLSR manager provides two attributes, `UpdateCwAfterFailedIcf` and `ReportFailedIcf`, that can be used, respectively, to control whether to update the contention window and report the failure to the remote station manager in case of ICF failure due to cross link collision.

EMLSR traces

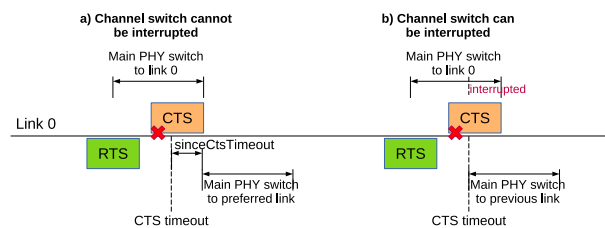


Fig. 35.17: Illustration of EMLSR TXOP ended trace

The EMLSR implementation provides the following traces to further analyze the performance of EMLSR clients:

- The `WifiMac::IcfDropReason` trace is fired every time an ICF is dropped by an EMLSR client and provides the reason for dropping the ICF and the link on which the ICF was transmitted. The reasons for dropping an ICF are:
 - `WifiIcfDrop::USING_OTHER_LINK`: the ICF is received right after starting an UL transmission on another link; the ICF is dropped and the UL TXOP continues.
 - `WifiIcfDrop::NOT_ENOUGH_TIME_TX`: the ICF is received while the main PHY is completing a TX and there is not enough time for the main PHY to complete the TX, start switching and be operating on the link the ICF is received on at the end of the ICF reception
 - `WifiIcfDrop::NOT_ENOUGH_TIME_SWITCH`: the ICF is received while the main PHY is completing a switch and there is not enough time for the main PHY to complete the channel switch, start switching and be operating on the link the ICF is received on at the end of the ICF reception
- The `StaWifiMac::EmlsrLinkSwitch` trace provides information about start/end of EMLSR link switch events. This trace is fired: (i) when a PHY operating on a link starts switching to another link, thus the PHY is disconnected from the previous link; (ii) when a PHY is connected to a new link after performing a channel switch. This trace provides: the ID of the previous link, in case the PHY is disconnected, or the ID of the new link, in case the PHY is connected; a pointer to the PHY that switches link; a boolean value indicating if the PHY is connected to (true) or disconnected from (false) the given link
- The `EmlsrManager::MainPhySwitch` trace is fired when the main PHY switches channel to operate on another link. Information associated with the main PHY switch is provided through a struct that is inherited from `struct EmlsrMainPhySwitchTrace`. Different inherited structs are defined for the different reasons for the main PHY to switch link. All the inherited structs provide a `GetName` method returning a string identifying the particular inherited struct. The available inherited structs are:
 - `EmlsrDLTxopIcfReceivedByAuxPhyTrace`: main PHY is starting switching to another link to carry out a DL TXOP after that an aux PHY received an ICF on that link

- `EmlsrULTxopRtsSentByAuxPhyTrace`: main PHY is starting switching to another link to take over an UL TXOP started by a TX capable aux PHY that transmitted an RTS frame on that link
- `EmlsrTxopEndedTrace`: main PHY is starting switching because a (DL or UL) TXOP ended. This trace is called when aux PHYs do not switch link and the main PHY switches back to the preferred link when a TXOP carried out on another link ends
- `EmlsrRtsSentByAuxPhyCtsTimeoutTrace`: main PHY is starting switching after a CTS timeout occurred on the link on which an RTS was transmitted to start an UL TXOP. This trace has a parameter, `sinceCtsTimeout`, that provides the time elapsed since the CTS timeout occurred. Normally, this trace is called when aux PHYs do not switch links, because the main PHY has to return to the preferred link upon CTS timeout, because a TXOP did not start. In some cases, the main PHY may be switching when CTS timeout occurs; this happens when an aux PHY that is TX capable transmits an RTS and the main PHY starts switching to the aux PHY link. In such a case, the main PHY completes the current link switch and then it starts switching to return back to the preferred link (see case (a) in Fig. *Illustration of EMLSR TXOP ended trace*). If the main PHY switch can be interrupted (see case (b) in Fig. *Illustration of EMLSR TXOP ended trace*), which is the case with the Advanced EMLSR Manager when the `InterruptSwitch` attribute is false, the previous switch is interrupted and the main PHY starts switching to the previous link (in this case, the time elapsed since the CTS timeout occurred is zero). This holds true for both the case aux PHYs do not switch link and the case aux PHYs switch link.
- `EmlsrSwitchMainPhyBackTrace`: main PHY is switching back to the preferred link after that it did not manage to gain channel access on an aux PHY link before the expiration of the switch main PHY back timer, whose duration is set to the value of the `ns3::AdvancedEmlsrManager::SwitchMainPhyBackDelay` attribute. The main PHY can switch back before the timer expiration in case it is determined that channel access is not expected to be gained within the timer expiration plus a channel switch delay. This trace has three parameters: the time elapsed since the switch main PHY back timer started, the reason for switching back before the timer expiration (if that is the case) and a boolean value indicating whether the main PHY is switching when it is requested to switch back. This trace is provided by the `AdvancedEmlsrManager` and is only fired when aux PHYs are not TX capable and do not switch link.

Ack manager

Since the introduction of the IEEE 802.11e amendment, multiple acknowledgment policies are available, which are coded in the Ack Policy subfield in the QoS Control field of QoS Data frames (see Section 9.2.4.5.4 of the IEEE 802.11-2016 standard). For instance, an A-MPDU can be sent with the *Normal Ack or Implicit Block Ack Request* policy, in which case the receiver replies with a Normal Ack or a Block Ack depending on whether the A-MPDU contains a single MPDU or multiple MPDUs, or with the *Block Ack* policy, in which case the receiver waits to receive a Block Ack Request in the future to which it replies with a Block Ack.

`WifiAckManager` is the abstract base class introduced to provide an interface for multiple ack managers. Currently, the default ack manager is the `WifiDefaultAckManager`.

WifiDefaultAckManager

The `WifiDefaultAckManager` allows to determine which acknowledgment policy to use depending on the value of its attributes:

- `UseExplicitBar`: used to determine the ack policy to use when a response is needed from the recipient and the current transmission includes multiple frames (A-MPDU) or there are frames transmitted previously for which an acknowledgment is needed. If this attribute is true, the *Block Ack* policy is used. Otherwise, the *Implicit Block Ack Request* policy is used.
- `BaThreshold`: used to determine when the originator of a Block Ack agreement needs to request a response from the recipient. A value of zero means that a response is requested at every frame transmission. Otherwise,

a non-zero value (less than or equal to 1) means that a response is requested upon transmission of a frame whose sequence number is distant at least `BaThreshold` multiplied by the transmit window size from the starting sequence number of the transmit window.

- `DLMuAckSequenceType`: used to select the acknowledgment sequence for DL MU frames (acknowledgment in single-user format, acknowledgment via MU-BAR Trigger Frame sent as single-user frame, or acknowledgment via MU-BAR Trigger Frames aggregated to the data frames).

Protection manager

The protection manager is in charge of determining the protection mechanism to use, if any, when sending a frame.

`WifiProtectionManager` is the abstract base class introduced to provide an interface for multiple protection managers. Currently, the default protection manager is the `WifiDefaultProtectionManager`.

WifiDefaultProtectionManager

The `WifiDefaultProtectionManager` selects a protection mechanism based on the information provided by the remote station manager.

Rate control algorithms

Multiple rate control algorithms are available in *ns-3*. Some rate control algorithms are modeled after real algorithms used in real devices; others are found in literature. The following rate control algorithms can be used by the MAC low layer:

Algorithms found in real devices:

- `ArfWifiManager`
- `OnoeWifiManager`
- `ConstantRateWifiManager`
- `MinstrelWifiManager`
- `MinstrelHtWifiManager`

Algorithms in literature:

- `IdealWifiManager` (default for `WifiHelper`)
- `AarfWifiManager` [[lacage2004aarfamrr](#)]
- `AmrrWifiManager` [[lacage2004aarfamrr](#)]
- `CaraWifiManager` [[kim2006cara](#)]
- `RraaWifiManager` [[wong2006rraa](#)]
- `AarfcdfWifiManager` [[maguolo2008aarfcd](#)]
- `ParfWifiManager` [[akella2007parf](#)]
- `AparfWifiManager` [[chevillat2005aparf](#)]
- `ThompsonSamplingWifiManager` [[krotoev2020rate](#)]

ConstantRateWifiManager

The constant rate control algorithm always uses the same transmission mode for every packet. Users can set a desired ‘DataMode’ for all ‘unicast’ packets and ‘ControlMode’ for all ‘request’ control packets (e.g. RTS).

To specify different data mode for non-unicast packets, users must set the ‘NonUnicastMode’ attribute of the `WifiRemoteStationManager`. Otherwise, `WifiRemoteStationManager` will use a mode with the lowest rate for non-unicast packets.

The 802.11 standard is quite clear on the rules for selection of transmission parameters for control response frames (e.g. CTS and ACK). *ns-3* follows the standard and selects the rate of control response frames from the set of basic rates or mandatory rates. This means that control response frames may be sent using different rate even though the `ConstantRateWifiManager` is used. The `ControlMode` attribute of the `ConstantRateWifiManager` is used for RTS frames only. The rate of CTS and ACK frames are selected according to the 802.11 standard. However, users can still manually add `WifiMode` to the basic rate set that will allow control response frames to be sent at other rates. Please consult the [project wiki](#) on how to do this.

Available attributes:

- `DataMode` (default `WifiMode::OfdmRate6Mbps`): specify a mode for all non-unicast packets
- `ControlMode` (default `WifiMode::OfdmRate6Mbps`): specify a mode for all ‘request’ control packets

IdealWifiManager

The ideal rate control algorithm selects the best mode according to the SNR of the previous packet sent. Consider node *A* sending a unicast packet to node *B*. When *B* successfully receives the packet sent from *A*, *B* records the SNR of the received packet into a `ns3::SnrTag` and adds the tag to an ACK back to *A*. By doing this, *A* is able to learn the SNR of the packet sent to *B* using an out-of-band mechanism (thus the name ‘ideal’). *A* then uses the SNR to select a transmission mode based on a set of SNR thresholds, which was built from a target BER and mode-specific SNR/BER curves.

Available attribute:

- `BerThreshold` (default 1e-6): The maximum Bit Error Rate that is used to calculate the SNR threshold for each mode.

Note that the `BerThreshold` has to be low enough to select a robust enough MCS (or mode) for a given SNR value, without being too restrictive on the target BER. Indeed we had noticed that the previous default value (i.e. 1e-5) led to the selection of HE MCS-11 which resulted in high PER. With this new default value (i.e. 1e-6), a HE STA moving away from a HE AP has smooth throughput decrease (whereas with 1e-5, better performance was seen further away, which is not “ideal”).

ThompsonSamplingWifiManager

Thompson Sampling (TS) is a classical solution to the Multi-Armed Bandit problem. *ThompsonSamplingWifiManager* implements a rate control algorithm based on TS with the goal of providing a simple statistics-based algorithm with a low number of parameters.

The algorithm maintains the number of successful transmissions α_i and the number of unsuccessful transmissions β_i for each MCS *i*, both of which are initially set to zero.

To select MCS for a data frame, the algorithm draws a sample frame success rate q_i from the beta distribution with shape parameters $(1 + \alpha_i, 1 + \beta_i)$ for each MCS and then selects MCS with the highest expected throughput calculated as the sample frame success rate multiplied by MCS rate.

To account for changing channel conditions, exponential decay is applied to α_i and β_i . The rate of exponential decay is controlled with the *Decay* attribute which is the inverse of the time constant. Default value of 1 Hz results in using exponential window with the time constant of 1 second. Setting this value to zero effectively disables exponential decay and can be used in static scenarios.

Control frames are always transmitted using the most robust MCS, except when the standard specifies otherwise, such as for ACK frames.

As the main goal of this algorithm is to provide a stable baseline, it does not take into account backoff overhead, inter-frame spaces and aggregation for MCS rate calculation. For an example of a more complex statistics-based rate control algorithm used in real devices, consider Minstrel-HT described below.

MinstrelWifiManager

The minstrel rate control algorithm is a rate control algorithm originated from madwifi project. It is currently the default rate control algorithm of the Linux kernel.

Minstrel keeps track of the probability of successfully sending a frame of each available rate. Minstrel then calculates the expected throughput by multiplying the probability with the rate. This approach is chosen to make sure that lower rates are not selected in favor of the higher rates (since lower rates are more likely to have higher probability).

In minstrel, roughly 10 percent of transmissions are sent at the so-called lookaround rate. The goal of the lookaround rate is to force minstrel to try higher rate than the currently used rate.

For a more detailed information about minstrel, see [\[linuxminstrel\]](#).

MinstrelHtWifiManager

This is the extension of minstrel for 802.11n/ac/ax.

802.11ax OBSS PD spatial reuse

802.11ax mode supports OBSS PD spatial reuse feature. OBSS PD stands for Overlapping Basic Service Set Preamble-Detection. OBSS PD is an 802.11ax specific feature that allows a STA, under specific conditions, to ignore an inter-BSS PPDU.

OBSS PD Algorithm

`ObssPdAlgorithm` is the base class of OBSS PD algorithms. It implements the common functionalities. First, it makes sure the necessary callbacks are setup. Second, when a PHY reset is requested by the algorithm, it performs the computation to determine the TX power restrictions and informs the PHY object.

The PHY keeps tracks of incoming requests from the MAC to get access to the channel. If a request is received and if PHY reset(s) indicating TX power limitations occurred before a packet was transmitted, the next packet to be transmitted will be sent with a reduced power. Otherwise, no TX power restrictions will be applied.

Constant OBSS PD Algorithm

Constant OBSS PD algorithm is a simple OBSS PD algorithm implemented in the `ConstantObssPdAlgorithm` class.

Once a HE preamble and its header have been received by the PHY, `ConstantObssPdAlgorithm::ReceiveHeSig` is triggered. The algorithm then checks whether this is an OBSS frame by comparing its own BSS color with the BSS color of the received preamble. If this is an OBSS frame, it compares the received RSSI with its configured OBSS PD level value. The PHY then gets reset to IDLE state in case the received RSSI is lower than that constant OBSS PD level value, and is informed about a TX power restrictions.

Note: since our model is based on a single threshold, the PHY only supports one restricted power level.

Groupcast with retries (GCR)

The IEEE 802.11aa amendment introduced GCR functionality to provide reliable multicast transmissions. The standard proposes two retransmission policies: unsolicited retry (GCR-UR) and block acknowledgement (GCR-BA). *ns-3* supports both GCR-UR and GCR-BA operations as described in the following. Only one retransmission policy can be selected and it cannot be changed at run-time.

GCR service activation

For any HT-capable AP, the `WifiMacHelper` will install an GCR Manager by using the type and attribute values configured through the `SetGcrManager` method.

GCR-UR service is used by an AP if the `RetransmissionPolicy` attribute of the GCR manager base class is set to `GCR_UNSOLICITED_RETRY` and if there is at least one HT-capable STA associated to the AP. Similarly, GCR-BA service is used by an AP if the `RetransmissionPolicy` attribute of the GCR manager base class is set to `GCR_BLOCK_ACK` and if there is at least one HT-capable STA associated to the AP.

The current implementation does not support hybrid scenarios made of GCR capable and non-GCR capable STAs.

Groupcast transmissions using GCR-UR

When a multicast data packet is ready for transmission, it is first transmitted similarly to the legacy No-Ack/No-retry. Afterwards, the GCR-UR service retransmits the packet multiple times. The number of retransmission per multicast data packet is controlled through the `UnsolicitedRetryLimit` attribute of the GCR manager base class (default is 7), unless it gets dropped because of lifetime expiry.

The GCR manager holds a counter that gets incremented at each transmission of a groupcast packet, which is used to determine if a groupcast packet shall be retransmitted. The frame exchange manager polls the GCR manager to know whether a given groupcast packet shall be kept for retransmission. As long as a packet needs to be retransmitted, the groupcast packet is not removed from the queue and the retry flag is set in the MAC header.

When GCR-UR service is used, A-MPDU can be enabled for groupcast frames the same way as for unicast ones. In that case, the AP attempts to establish a Block Ack agreement prior to a groupcast transmission with each member STA of the group corresponding to that transmission. As long as the Block Ack agreement is not successfully established with all members of the group, MPDU aggregation cannot be used.

the establishment of the Block Ack agreements is done by sending ADDBA Request frames containing a GCR Group Address element, which indicates to the receiving STAs the group address for which the agreements are being established.

The AP retransmits the same A-MPDU multiple times and, once the last retransmission is done, it advances its scoreboard boundaries since it does not get any feedback from the receivers.

Upon reception of an A-MPDU, GCR-capable STAs flush any previous pending MPDUs by assuming an implicit BAR has been received. This ensures the receiver window follows the transmit window. In order to prevent MPDUs to stay pending and never get forwarded up, an event is also scheduled to ensure the receiver window gets flushed once the maximum MSDU lifetime has elapsed. This event is canceled upon reception of a new A-MPDU belonging to the same Block Ack agreement.

Groupcast transmissions using GCR-BA

The use of GCR-BA for a given group address requires the establishment of a block ack agreement with all members of that group address as described in the previous section.

When a TXOP is granted to the AP and at least one groupcast is ready for transmission, the AP sends one or more groupcast MPDUs and schedules transmissions of a Block Ack Request packets to each member of the group. The first request is sent a SIFS after transmission of the last MPDU, unless the remaining TXOP duration does not allow it. The Block Ack Request packets may be transmitted in following TXOPs. If a STA does not answer to a Block Ack Request, it gets retransmitted until the Block Ack response is received.

Once Block Ack responses have been received from all members of the group, the AP determines the groupcast MPDUs that have been successfully received by all members of the group. These MPDUs are removed from the queues and the AP advances its transmit window accordingly. In a following TXOP, the AP retransmits the groupcast MPDUs that have not been successfully received by all members of the group, and eventually adds other queued groupcast MPDU to the same group, if the TXOP limit allows it.

TXVECTOR for groupcast transmissions using GCR service

When GCR is used, the lowest MCS and modulation class that can be decoded by all STAs that are members of the group address is selected for the transmission. Furthermore, it selects the minimum channel width and number of spatial streams supported over all members of the group. Similarly, depending on the selected modulation class, a guard interval duration that is supported by all members of the group address is picked. The current implementation assumes all GCR-capable STAs are members of all groups.

Protection for groupcast transmissions

Protection used for groupcast transmission can be controlled through the `GcrProtectionMode` attribute of the GCR manager base class. If RTS/CTS protection mode is selected and if the groupcast data frames have a size larger than the value of the `WifiRemoteStationManager::RtsCtsThreshold` attribute, a RTS frame is directed to one of the STA that is part of the group, prior to groupcast MPDUs transmitted using the GCR service. If CTS-to-self protection mode is selected, a CTS frame is transmitted prior to groupcast MPDUs.

The protection manager is responsible for selecting the protection frames to send prior to data transmissions. It relies on `WifiRemoteStationManager::NeedRts` and `WifiRemoteStationManager::NeedCtsToSelf`, which check the configured protection mode in GCR manager when data transmissions involve groupcast MPDUs using the GCR service.

When RTS/CTS is used for protection, the default implementation picks the STA with the lowest AID as individual address to set as recipient of the RTS frame.

Concealment of GCR transmissions

MSDUs transmitted via the GCR service are sent in an A-MSDU made of a single A-MSDU subframe, regardless of current A-MSDU size settings. The Address 1 field of the A-MSDU is set to the concealment address, which can be configured through the `GcrManager::GcrConcealmentAddress` attribute.

Modifying Wifi model

Modifying the default wifi model is one of the common tasks when performing research. We provide an overview of how to make changes to the default wifi model in this section. Depending on your goal, the common tasks are (in no particular order):

- Creating or modifying the default Wi-Fi frames/headers by making changes to `wifi-mac-header.*`.
- MAC low modification. For example, handling new/modified control frames (think RTS/CTS/ACK/Block ACK), making changes to two-way transaction/four-way transaction. Users usually make changes to `frame-exchange-manager.*` or its subclasses to accomplish this. Handling of control frames is performed in `FrameExchangeManager::ReceiveMpdu`.
- MAC high modification. For example, handling new management frames (think beacon/probe), beacon/probe generation. Users usually make changes to `wifi-mac.*`, `sta-wifi-mac.*`, `ap-wifi-mac.*`, or `adhoc-wifi-mac.*` to accomplish this.
- Wi-Fi queue management. The files `txop.*` and `qos-txop.*` are of interest for this task.
- Channel access management. Users should modify the files `channel-access-manager.*`, which grant access to `Txop` and `QosTxop`.
- Fragmentation and RTS thresholds are handled by Wi-Fi remote station manager. Note that Wi-Fi remote station manager simply indicates if fragmentation and RTS are needed. Fragmentation is handled by `Txop` or `QosTxop` while RTS/CTS transaction is handled by `FrameExchangeManager`.
- Modifying or creating new rate control algorithms can be done by creating a new child class of Wi-Fi remote station manager or modifying the existing ones.

35.2 User Documentation

35.2.1 Using the WifiNetDevice

The modularity provided by the implementation makes low-level configuration of the `WifiNetDevice` powerful but complex. For this reason, we provide some helper classes to perform common operations in a simple matter, and leverage the *ns-3* attribute system to allow users to control the parameterization of the underlying models.

Users who use the low-level *ns-3* API and who wish to add a `WifiNetDevice` to their node must create an instance of a `WifiNetDevice`, plus a number of constituent objects, and bind them together appropriately (the `WifiNetDevice` is very modular in this regard, for future extensibility). At the low-level API, this can be done with about 20 lines of code (see `ns3::WifiHelper::Install`, and `ns3::YansWifiPhyHelper::Create`). They also must create, at some point, a `Channel`, which also contains a number of constituent objects (see `ns3::YansWifiChannelHelper::Create`).

However, a few helpers are available for users to add these devices and channels with only a few lines of code, if they are willing to use defaults, and the helpers provide additional API to allow the passing of attribute values to change default values. Commonly used attribute values are listed in the `Attributes` section. The scripts in `examples/wireless` can be browsed to see how this is done. Next, we describe the common steps to create a `WifiNetDevice` from the bottom layer (`Channel`) up to the device layer (`WifiNetDevice`).

To create a `WifiNetDevice`, users need to follow these steps:

- Decide on which physical layer framework, the SpectrumWifiPhy or YansWifiPhy, to use. This will affect which Channel and Phy type to use.
- Configure the Channel: Channel takes care of getting signal from one device to other devices on the same Wi-Fi channel. The main configurations of WifiChannel are propagation loss model and propagation delay model.
- Configure the WifiPhy: WifiPhy takes care of actually sending and receiving wireless signal from Channel. Here, WifiPhy decides whether each frame will be successfully decoded or not depending on the received signal strength and noise. Thus, the main configuration of WifiPhy is the error rate model, which is the one that actually calculates the probability of successfully decoding the frame based on the signal.
- Configure WifiMac: this step is more related to the architecture and device level. The users configure the wifi architecture (i.e. ad-hoc or ap-sta) and whether QoS (802.11e), HT (802.11n) and/or VHT (802.11ac) and/or HE (802.11ax) features are supported or not.
- Create WifiDevice: at this step, users configure the desired wifi standard (e.g. **802.11b**, **802.11g**, **802.11a**, **802.11n**, **802.11ac** or **802.11ax**) and rate control algorithm.
- Configure mobility: finally, a mobility model is (usually) required before WifiNetDevice can be used; even if the devices are stationary, their relative positions are needed for propagation loss calculations.

The following sample code illustrates a typical configuration using mostly default values in the simulator, and infrastructure mode:

```
NodeContainer wifiStaNode;
wifiStaNode.Create(10); // Create 10 station node objects
NodeContainer wifiApNode;
wifiApNode.Create(1); // Create 1 access point node object

// Create a channel helper and phy helper, and then create the channel
YansWifiChannelHelper channel = YansWifiChannelHelper::Default();
YansWifiPhyHelper phy = YansWifiPhyHelper::Default();
phy.SetChannel(channel.Create());

// Create a WifiMacHelper, which is reused across STA and AP configurations
WifiMacHelper mac;

// Create a WifiHelper, which will use the above helpers to create
// and install Wifi devices. Configure a Wifi standard to use, which
// will align various parameters in the Phy and Mac to standard defaults.
WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211n);
// Declare NetDeviceContainers to hold the container returned by the helper
NetDeviceContainer wifiStaDevices;
NetDeviceContainer wifiApDevice;

// Perform the installation
mac.SetType("ns3::StaWifiMac");
wifiStaDevices = wifi.Install(phy, mac, wifiStaNodes);
mac.SetType("ns3::ApWifiMac");
wifiApDevice = wifi.Install(phy, mac, wifiApNode);
```

At this point, the 11 nodes have Wi-Fi devices configured, attached to a common channel. The rest of this section describes how additional configuration may be performed.

YansWifiChannelHelper

The YansWifiChannelHelper has an unusual name. Readers may wonder why it is named this way. The reference is to the [yans simulator](#) from which this model is taken. The helper can be used to create a YansWifiChannel with a default PropagationLoss and PropagationDelay model.

Users will typically type code such as:

```
YansWifiChannelHelper wifiChannelHelper = YansWifiChannelHelper::Default();  
Ptr<Channel> wifiChannel = wifiChannelHelper.Create();
```

to get the defaults. Specifically, the default is a channel model with a propagation delay equal to a constant, the speed of light (`ns3::ConstantSpeedPropagationDelayModel`), and a propagation loss based on a default log distance model (`ns3::LogDistancePropagationLossModel`), using a default exponent of 3. Please note that the default log distance model is configured with a reference loss of 46.6777 dB at reference distance of 1m. The reference loss of 46.6777 dB was calculated using Friis propagation loss model at 5.15 GHz. The reference loss must be changed if **802.11b**, **802.11g**, **802.11n** (at 2.4 GHz) or **802.11ax** (at 2.4 GHz) are used since they operate at 2.4 GHz.

Note the distinction above in creating a helper object vs. an actual simulation object. In *ns-3*, helper objects (used at the helper API only) are created on the stack (they could also be created with operator new and later deleted). However, the actual *ns-3* objects typically inherit from `class ns3::Object` and are assigned to a smart pointer. See the chapter in the *ns-3* manual for a discussion of the *ns-3* object model, if you are not familiar with it.

The following two methods are useful when configuring YansWifiChannelHelper:

- `YansWifiChannelHelper::AddPropagationLoss` adds a `PropagationLossModel`; if one or more `PropagationLossModels` already exist, the new model is chained to the end
- `YansWifiChannelHelper::SetPropagationDelay` sets a `PropagationDelayModel` (not chainable)

YansWifiPhyHelper

Physical devices (base class `ns3::WifiPhy`) connect to `ns3::YansWifiChannel` models in *ns-3*. We need to create `WifiPhy` objects appropriate for the `YansWifiChannel`; here the `YansWifiPhyHelper` will do the work.

The `YansWifiPhyHelper` class configures an object factory to create instances of a `YansWifiPhy` and adds some other objects to it, including possibly a supplemental `ErrorRateModel` and a pointer to a `MobilityModel`. The user code is typically:

```
YansWifiPhyHelper wifiPhyHelper;  
wifiPhyHelper.SetChannel(wifiChannel);
```

The default `YansWifiPhyHelper` is configured with `TableBasedErrorRateModel` (`ns3::TableBasedErrorRateModel`). You can change the error rate model by calling the `YansWifiPhyHelper::SetErrorRateModel` method.

Optionally, if pcap tracing is needed, a user may use the following command to enable pcap tracing:

```
WifiPhyHelper::SetPcapDataLinkType(enum SupportedPcapDataLinkTypes dlt)
```

ns-3 supports `RadioTap` and `Prism` tracing extensions for 802.11.

For MLD devices, it is also possible to select one of these PCAP capture type:

```
WifiPhyHelper::SetPcapCaptureType(PcapCaptureType type)
```

ns-3 supports three PCAP capture types for MLD devices: * a single PCAP file for the device, regardless of PHYs and links

```
phyHelper.SetPcapCaptureType(WifiPhyHelper::PcapCaptureType::PCAP_PER_DEVICE);
```

- a PCAP file generated per PHY (default behavior)

```
phyHelper.SetPcapCaptureType(WifiPhyHelper::PcapCaptureType::PCAP_PER_PHY);
```

- a PCAP file generated per link

```
phyHelper.SetPcapCaptureType(WifiPhyHelper::PcapCaptureType::PCAP_PER_LINK);
```

The PCAP files will always be generated with the suffix of `nodeId-deviceId.pcap`, unless the user has enabled object names on either the Node or the NetDevice, in which case the string name will be used. When PCAP files are generated per PHY, that suffix is `nodeId-deviceId-linkId.pcap`. When PCAP files are generated per link, the suffix is `nodeId-deviceId-phyId.pcap`.

When PCAP files are generated per link, if there is no generated PCAP for a given link, it means no packet has been transmitted nor received on that link. Since PHYs of non-AP MLDs may swap their links during setup, the link ID appended to the PCAP file might be higher than the highest PHY ID.

In case of SLD devices, the configuration of the capture type has no impact since a single PCAP file will always be generated per device.

Note that we haven't actually created any WifiPhy objects yet; we've just prepared the YansWifiPhyHelper by telling it which channel it is connected to. The Phy objects are created in the next step.

In order to enable 802.11n/ac/ax/be MIMO, the number of antennas as well as the number of supported spatial streams need to be configured. For example, this code enables MIMO with 2 antennas and 2 spatial streams:

```
wifiPhyHelper.Set("Antennas", UintegerValue(2));
wifiPhyHelper.Set("MaxSupportedTxSpatialStreams", UintegerValue(2));
wifiPhyHelper.Set("MaxSupportedRxSpatialStreams", UintegerValue(2));
```

It is also possible to configure less streams than the number of antennas in order to benefit from diversity gain, and to define different MIMO capabilities for downlink and uplink. For example, this code configures a node with 3 antennas that supports 2 spatial streams in downstream and 1 spatial stream in upstream:

```
wifiPhyHelper.Set("Antennas", UintegerValue(3));
wifiPhyHelper.Set("MaxSupportedTxSpatialStreams", UintegerValue(2));
wifiPhyHelper.Set("MaxSupportedRxSpatialStreams", UintegerValue(1));
```

802.11n PHY layer can support both 20 (default) or 40 MHz channel width, and 802.11ac/ax PHY layer can use either 20, 40, 80 (default), 160 or 320 MHz channel width. See below for further documentation on setting the frequency, channel width, and channel number.

```
WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211ac);
wifi.SetRemoteStationManager("ns3::ConstantRateWifiManager",
                             "DataMode", StringValue("VhtMcs9"),
                             "ControlMode", StringValue("VhtMcs0"));

//Install PHY and MAC
Ssid ssid = Ssid("ns3-wifi");
WifiMacHelper mac;

mac.SetType("ns3::StaWifiMac",
            "Ssid", SsidValue(ssid),
```

(continues on next page)

(continued from previous page)

```

        "ActiveProbing", BooleanValue(false));

NetDeviceContainer staDevice;
staDevice = wifi.Install(phy, mac, wifiStaNode);

mac.SetType("ns3::ApWifiMac",
            "Ssid", SsidValue(ssid));

NetDeviceContainer apDevice;
apDevice = wifi.Install(phy, mac, wifiApNode);

```

Channel, frequency, channel width, and band configuration

There is a unique `ns3::WifiPhy` attribute, named `ChannelSettings`, that enables to set channel number, channel width, frequency band and primary20 index for each segment all together, in order to eliminate the possibility of inconsistent settings. The `ChannelSettings` attribute can be set in a number of ways (see below) by providing either a `StringValue` object or an `AttributeContainerValue` object:

- Defining a `StringValue` object to set the `ChannelSettings` attribute

```
StringValue value("{38, 40, BAND_5GHZ, 0}");
```

- Defining an `AttributeContainerValue` object to set the `ChannelSettings` attribute

```
WifiPhy::ChannelSettingsValue value;
value.Set(WifiPhy::ChannelSegments{{38, 40, WIFI_PHY_BAND_5GHZ, 0}});
```

In both cases, the operating channel will be channel 38 in the 5 GHz band, which has a width of 40 MHz, and the primary20 channel will be the 20 MHz subchannel with the lowest center frequency (index 0).

The operating channel settings can then be configured in a number of ways:

- by setting global configuration default; e.g.

```
Config::SetDefault("ns3::WifiPhy::ChannelSettings", StringValue("{38, 40, BAND_5GHZ, 0}
↪"));
```

- by setting an attribute value in the helper; e.g.

```
WifiPhy::ChannelSettingsValue value;
value.Set(WifiPhy::ChannelSegments{{38, 40, WIFI_PHY_BAND_5GHZ, 0}});
YansWifiPhyHelper wifiPhyHelper = YansWifiPhyHelper::Default();
wifiPhyHelper.Set("ChannelSettings", value);
```

- by setting the `WifiHelper::SetStandard(enum WifiStandard)` method; and
- by performing post-installation configuration of the option, either via a `Ptr` to the `WifiPhy` object, or through the `Config` namespace; e.g.:

```
Config::Set("/NodeList/0/DeviceList/*/ns3::WifiNetDevice/Phy/ns3::WifiPhy/
↪ChannelSettings",
            StringValue("{38, 40, BAND_5GHZ, 0}"));
```

This section provides guidance on how to properly configure these settings.

WifiHelper::SetStandard()

WifiHelper::SetStandard() is a method required to set various parameters in the Mac and Phy to standard values, but also to check that the channel settings as described above are allowed. For instance, a channel in the 2.4 GHz band cannot be configured if the standard is 802.11ac, or a channel in the 6 GHz band can only be configured if the standard is 802.11ax (or beyond).

WifiHelper::SetStandard() can also be called by passing a string with the standard name. For example any of the strings “802.11ax”, “11ax” or “HE” can be passed in order to set the standard as WIFI_STANDARD_80211ax.

The following values for WifiStandard are defined in src/wifi/model/wifi-standards.h:

```
WIFI_STANDARD_80211a,
WIFI_STANDARD_80211b,
WIFI_STANDARD_80211g,
WIFI_STANDARD_80211p,
WIFI_STANDARD_80211n,
WIFI_STANDARD_80211ac,
WIFI_STANDARD_80211ax,
WIFI_STANDARD_80211be
```

By default, the WifiHelper (the typical use case for WifiPhy creation) will configure the WIFI_STANDARD_80211ax standard by default. Other values for standards should be passed explicitly to the WifiHelper object.

If user has not already configured ChannelSettings when SetStandard is called, the user obtains default values, as described next.

Default settings for the operating channel

Not all the parameters in the channel settings have to be set to a valid value, but they can be left unspecified, in which case default values are substituted as soon as the WifiStandard is set. Here are the rules (applied in the given order):

- If the band is unspecified (i.e., it is set to WIFI_PHY_BAND_UNSPECIFIED or “BAND_UNSPECIFIED”), the default band for the configured standard is set (5 GHz band for 802.11{a, ac, ax, be, p} and 2.4 GHz band for all the others).
- If both the channel width and the channel number are unspecified (i.e., they are set to zero), the default channel width for the configured standard and band is set (22 MHz for 802.11b, 10 MHz for 802.11p, 80 MHz for 802.11ac, for 802.11ax and for 802.11be if the band is 5 GHz, and 20 MHz for all other cases).
- If the channel width is unspecified but the channel number is valid, the settings are valid only if there exists a unique channel with the given number for the configured standard and band, in which case the channel width is set to the width of such unique channel. Otherwise, the simulation aborts.
- If the channel number is unspecified (i.e., it is set to zero), the default channel number for the configured standard, band and channel width is used (the default channel number is the first one in the list of channels that can be used with the configured standard, band and channel width)

Following are a few examples to clarify these rules:

```
WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211ac);
YansWifiPhyHelper phyHelper;
phyHelper.Set("ChannelSettings", StringValue("{58, 0, BAND_5GHZ, 0}"));
// channel width unspecified
// -> it is set to 80 MHz (width of channel 58)
```



```
WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211n);
YansWifiPhyHelper phyHelper;
phyHelper.Set("ChannelSettings", StringValue("{0, 40, BAND_5GHZ, 0}"));
// channel number unspecified
// -> it is set to channel 38 (first 40 MHz channel in the 5GHz band)
```

```
WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211ax);
YansWifiPhyHelper phyHelper;
phyHelper.Set("ChannelSettings", StringValue("{0, 0, BAND_2_4GHZ, 0}"));
// both channel number and width unspecified
// -> width set to 20 MHz (default width for 802.11ax in the 2.4 GHz band)
// -> channel number set to 1 (first 20 MHz channel in the 2.4 GHz band)
```

```
WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211a);
YansWifiPhyHelper phyHelper;
phyHelper.Set("ChannelSettings", StringValue("{0, 0, BAND_UNSPECIFIED, 0}"));
// band, channel number and width unspecified
// -> band is set to WIFI_PHY_BAND_5GHZ (default band for 802.11a)
// -> width set to 20 MHz (default width for 802.11a in the 5 GHz band)
// -> channel number set to 36 (first 20 MHz channel in the 5 GHz band)
```

The default value for the ChannelSettings attribute leaves all the parameters unspecified, except for the primary20 index, which is equal to zero.

WifiPhy::Frequency

The configured WifiPhy channel center frequency can be got via the attribute Frequency in the class WifiPhy. It is expressed in units of MHz.

Note that this is a change in definition from ns-3.25 and earlier releases, where this attribute referred to the start of the overall frequency band on which the channel resides, not the specific channel center frequency.

WifiPhy::ChannelWidth

The configured WifiPhy channel width can be got via the attribute ChannelWidth in the class WifiPhy. It is expressed in units of MHz.

WifiPhy::ChannelNumber

Several channel numbers are defined and well-known in practice. However, valid channel numbers vary by geographical region around the world, and there is some overlap between the different standards.

The configured WifiPhy channel number can be got via the attribute ChannelNumber in the class WifiPhy.

In ns-3, a ChannelNumber may be defined or unknown. These terms are not found in the code; they are just used to describe behavior herein.

If a ChannelNumber is defined, it means that WifiPhy has stored a map of ChannelNumber to the center frequency and channel width commonly known for that channel in practice. For example:

- Channel 1, when IEEE 802.11b is configured, corresponds to a channel width of 22 MHz and a center frequency of 2412 MHz.
- Channel 36, when IEEE 802.11n is configured at 5 GHz, corresponds to a channel width of 20 MHz and a center frequency of 5180 MHz.

The following channel numbers are well-defined for 2.4 GHz standards:

- channels 1-14 with ChannelWidth of 22 MHz for 802.11b
- channels 1-14 with ChannelWidth of 20 MHz for 802.11n-2.4GHz and 802.11g

The following channel numbers are well-defined for 5 GHz standards:

Table 35.1: 5 GHz channel numbers

ChannelWidth	ChannelNumber
20 MHz	36, 40, 44, 48, 52, 56, 60, 64, 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140, 144, 149, 153, 161, 165, 169
40 MHz	38, 46, 54, 62, 102, 110, 118, 126, 134, 142, 151, 159
80 MHz	42, 58, 106, 122, 138, 155
160 MHz	50, 114
10 MHz (802.11p)	172, 174, 176, 178, 180, 182, 184
5 MHz (802.11p)	171, 173, 175, 177, 179, 181, 183

The following channel numbers are well-defined for 6 GHz standards (802.11ax and later):

Table 35.2: 6 GHz channel numbers

ChannelWidth	ChannelNumber
20 MHz	1, 5, 9, 13, 17, 21, 25, 29, 33, 37, 41, 45, 49, 53, 57, 61, 65, 69, 73, 77, 81, 85, 89, 93, 97, 101, 105, 109, 113, 117, 121, 125, 129, 133, 137, 141, 145, 149, 153, 157, 161, 165, 169, 173, 177, 181, 185, 189, 193, 197, 201, 205, 209, 213, 217, 221, 225, 229, 233
40 MHz	3, 11, 19, 27, 35, 43, 51, 59, 67, 75, 83, 91, 99, 107, 115, 123, 131, 139, 147, 155, 163, 171, 179, 187, 195, 203, 211, 219, 227
80 MHz	7, 23, 39, 55, 71, 87, 103, 119, 135, 151, 167, 183, 199, 215
160 MHz	15, 47, 79, 111, 143, 175, 207
320 MHz (802.11be and later)	31, 95, 159 (320 MHz-1) 63, 127, 191 (320 MHz-2)

The channel number may be set either before or after creation of the WifiPhy object.

If an unknown channel number (other than zero) is configured, the simulator will exit with an error; for instance, such as:

```
Ptr<WifiPhy> wifiPhy = ...;
wifiPhy->SetAttribute("ChannelSettings", StringValue("{1321, 20, BAND_5GHZ, 0}"));
```

The known channel numbers are defined in the implementation file `src/wifi/model/wifi-phy-operating-channel.cc`. Of course, this file may be edited by users to extend to additional channel numbers.

If a known channel number is configured against an incorrect value of the WifiPhyStandard, the simulator will exit with an error; for instance, such as:

```
WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211n);
...
```

(continues on next page)

(continued from previous page)

```
Ptr<WifiPhy> wifiPhy = ...;
wifiPhy->SetAttribute("ChannelSettings", StringValue("{14, 20, BAND_5GHZ, 0}"));
```

In the above, while channel number 14 is well-defined in practice for 802.11b only, it is for 2.4 GHz band, not 5 GHz band.

If a limitation is configured for the supported bandwidth of the PHY radio through the MaxRadioBw attribute, the simulator will exit with an error if the configured bandwidth exceeds the maximum allowed bandwidth.

WifiPhy::Primary20MHzIndex

The configured WifiPhy primary 20MHz channel index can be got via the attribute Primary20MHzIndex in the class WifiPhy.

Non-contiguous 160 MHz operating channel configuration (80+80 MHz)

An 80+80 MHz operating channel can be configured by setting each 80 MHz channel for each segment. For example, the following example code snippet configures a non-contiguous 160 MHz channel where the first segment operates on channel 42 and the second segment operates on channel 106:

```
Ptr<WifiPhy> wifiPhy = ...;
wifiPhy->SetAttribute("ChannelSettings", StringValue("{42, 80, BAND_5GHZ, 0};{106, 80, BAND_5GHZ, 0}"));
```

It is still possible to have channel numbers set to zero in order to select the default 80+80 MHz channel:

```
Ptr<WifiPhy> wifiPhy = ...;
wifiPhy->SetAttribute("ChannelSettings", StringValue("{0, 80, BAND_UNSPECIFIED, 0};{0, 80, BAND_UNSPECIFIED, 0}"));
```

which results in the use of channels 42 and 106.

Note that only the primary 20 MHz channel index of the first specified segment is considered. 80+80MHz requires the use of SpectrumWifiPhy.

Order of operation issues

Channel settings can be configured either before or after the wifi standard. If the channel settings are configured before the wifi standard, the channel settings are stored and applied when the wifi standard is configured. Otherwise, they are applied immediately.

The wifi standard can be configured only once, i.e., it is not possible to change standard during a simulation. It is instead possible to change the channel settings at any time.

Advertisement of the channel width

STAs are advertising their channel width in capabilities and operations (for APs) information elements contained in management frames.

The channel width that is being advertised is the value that is returned by `WifiPhy::ChannelWidth`, which corresponds to the value that is set in the `ChannelSettings` attribute (or the total width in case of non-contiguous channel operation).

Not all the channel widths can be advertised for non-AP STAs, depending on the standard and the band. This is not a problem as long as the AP the non-AP STA is associated with is operating on the same channel width or on a narrower one. Otherwise, the AP might select a larger channel width for the transmission of downlink PPDU's, which would then be dropped by the PHY of the non-AP STA.

For example, since VHT Capabilities Information Elements only allow the advertisement of 80 and 160 MHz channel widths, a VHT non-AP STA with its `ChannelSettings` set to use a 40 MHz channel width won't be able to receive any downlink PPDU's if the VHT AP has its `ChannelSettings` set to operate on an 80 MHz channel width.

It is possible to configure the behavior of the association manager to either abort the simulation when such a problematic situation is detected (default) or to allow the association anyway, depending on the configured value for the `ns3::WifiMac::AllowAssociationWithDifferentChannelWidth` attribute.

If the default association manager is used, it is also possible to skip candidate APs that are operating on a larger channel width than the non-AP STA and which would result in the problematic situation described above. This is done by setting the `ns3::WifiMac::SkipCandidateAPsWithLargerChannelWidth` attribute to true.

SpectrumWifiPhyHelper

The API for this helper closely tracks the API of the `YansWifiPhyHelper`, with the exception that a channel of type `ns3::SpectrumChannel` instead of type `ns3::YansWifiChannel` must be used with it.

Its API has been extended for 802.11be multi-link and EMLSR in order to attach multiple spectrum channels to a same PHY. For that purpose, a user may use the following command to attach a spectrum channel to the PHY objects that will be created upon a call to `ns3::WifiHelper::Install`:

```
SpectrumWifiPhyHelper::AddChannel(const Ptr<SpectrumChannel> channel,
                                   const FrequencyRange& freqRange)
```

where `FrequencyRange` is a structure that contains the start and stop frequencies expressed in MHz which corresponds to the spectrum portion that is covered by the channel.

WifiMacHelper

The next step is to configure the MAC model. We use `WifiMacHelper` to accomplish this. `WifiMacHelper` takes care of both the MAC low model and MAC high model, and configures an object factory to create instances of a `ns3::WifiMac`. It is used to configure MAC parameters like type of MAC, and to select whether 802.11/WMM-style QoS and/or 802.11n-style High Throughput (HT) and/or 802.11ac-style Very High Throughput (VHT) support and/or 802.11ax-style High Efficiency (HE) support are/is required.

By default, it creates an ad-hoc MAC instance that does not have 802.11e/WMM-style QoS nor 802.11n-style High Throughput (HT) nor 802.11ac-style Very High Throughput (VHT) nor 802.11ax-style High Efficiency (HE) support enabled.

For example the following user code configures a non-QoS and non-HT/non-VHT/non-HE MAC that will be a non-AP STA in an infrastructure network where the AP has SSID `ns-3-ssid`:

```
WifiMacHelper wifiMacHelper;  
Ssid ssid = Ssid("ns-3-ssid");  
wifiMacHelper.SetType("ns3::StaWifiMac",  
    "Ssid", SsidValue(ssid),  
    "ActiveProbing", BooleanValue(false));
```

The following code shows how to create an AP with QoS enabled:

```
WifiMacHelper wifiMacHelper;  
wifiMacHelper.SetType("ns3::ApWifiMac",  
    "Ssid", SsidValue(ssid),  
    "QosSupported", BooleanValue(true),  
    "BeaconGeneration", BooleanValue(true),  
    "BeaconInterval", TimeValue(Seconds(2.5)));
```

To create ad-hoc MAC instances, simply use `ns3::AdhocWifiMac` instead of `ns3::StaWifiMac` or `ns3::ApWifiMac`.

With QoS-enabled MAC models it is possible to work with traffic belonging to four different Access Categories (ACs): **AC_VO** for voice traffic, **AC_VI** for video traffic, **AC_BE** for best-effort traffic and **AC_BK** for background traffic.

When selecting **802.11n** as the desired wifi standard, both 802.11e/WMM-style QoS and 802.11n-style High Throughput (HT) support gets enabled. Similarly when selecting **802.11ac** as the desired wifi standard, 802.11e/WMM-style QoS, 802.11n-style High Throughput (HT) and 802.11ac-style Very High Throughput (VHT) support gets enabled. And when selecting **802.11ax** as the desired wifi standard, 802.11e/WMM-style QoS, 802.11n-style High Throughput (HT), 802.11ac-style Very High Throughput (VHT) and 802.11ax-style High Efficiency (HE) support gets enabled.

For MAC instances that have QoS support enabled, the `ns3::WifiMacHelper` can be also used to set:

- block ack threshold (number of packets for which block ack mechanism should be used);
- block ack inactivity timeout.

For example the following user code configures a MAC that will be a non-AP STA with QoS enabled and a block ack threshold for AC_BE set to 2 packets, in an infrastructure network where the AP has SSID `ns-3-ssid`:

```
WifiMacHelper wifiMacHelper;  
Ssid ssid = Ssid("ns-3-ssid");  
wifiMacHelper.SetType("ns3::StaWifiMac",  
    "Ssid", SsidValue(ssid),  
    "QosSupported", BooleanValue(true),  
    "BE_BlockAckThreshold", UIntegerValue(2),  
    "ActiveProbing", BooleanValue(false));
```

For MAC instances that have 802.11n-style High Throughput (HT) and/or 802.11ac-style Very High Throughput (VHT) and/or 802.11ax-style High Efficiency (HE) support enabled, the `ns3::WifiMacHelper` can be also used to set:

- MSDU aggregation parameters for a particular Access Category (AC) in order to use 802.11n/ac A-MSDU feature;
- MPDU aggregation parameters for a particular Access Category (AC) in order to use 802.11n/ac A-MPDU feature.

By default, MSDU aggregation feature is disabled for all ACs and MPDU aggregation is enabled for AC_VI and AC_BE, with a maximum aggregation size of 65535 bytes.

For example the following user code configures a MAC that will be a non-AP STA with HT and QoS enabled, MPDU aggregation enabled for AC_VO with a maximum aggregation size of 65535 bytes, and MSDU aggregation enabled for AC_BE with a maximum aggregation size of 7935 bytes, in an infrastructure network where the AP has SSID `ns-3-ssid`:

```
WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211n);

WifiMacHelper wifiMacHelper;
Ssid ssid = Ssid("ns-3-ssid");
wifiMacHelper.SetType("ns3::StaWifiMac",
    "Ssid", SsidValue(ssid),
    "VO_MaxAmpduSize", UIntegerValue(65535),
    "BE_MaxAmsduSize", UIntegerValue(7935),
    "ActiveProbing", BooleanValue(false));
```

802.11ax APs support sending multi-user frames via DL OFDMA and UL OFDMA if a Multi-User Scheduler is aggregated to the wifi MAC (by default no scheduler is aggregated). WifiMacHelper enables to aggregate a Multi-User Scheduler to an AP and set its parameters:

```
WifiMacHelper wifiMacHelper;
wifiMacHelper.SetMultiUserScheduler("ns3::RrMultiUserScheduler",
    "EnableUlofdma", BooleanValue(true),
    "EnableBsrp", BooleanValue(false));
```

The Ack Manager is in charge of selecting the acknowledgment method among the three available methods (see section *MAC queues*). The default ack manager enables to select the acknowledgment method, e.g.:

```
Config::SetDefault("ns3::WifiDefaultAckManager::DlMuAckSequenceType",
    EnumValue(WifiAcknowledgment::DL_MU_AGGREGATE_TF));
```

MAC queue scheduler

By default, a wifi device uses the FcfsWifiQueueScheduler (first-come, first-served). A different scheduler can be selected on the MAC helper before installing the devices. For example, to serve receivers in a round-robin fashion at the AP:

```
WifiMacHelper wifiMacHelper;
wifiMacHelper.SetMacQueueScheduler("ns3::RrWifiQueueScheduler");
// ... remaining SetType / install calls as usual
```

Any attributes declared on the scheduler base class (ns3::WifiMacQueueSchedulerImpl) can be passed as additional arguments to SetMacQueueScheduler. The main one is DropPolicy, which selects whether an enqueue into a full data queue drops the newest (default) or the oldest in-queue data frame:

```
wifiMacHelper.SetMacQueueScheduler("ns3::RrWifiQueueScheduler",
    "DropPolicy", StringValue("DropOldest"));
```

Notes:

- The scheduler is not tied to any particular 802.11 amendment; it works with any QoS-capable MAC. On a non-QoS MAC, RrWifiQueueScheduler produces the same serving order as FcfsWifiQueueScheduler.
- The scheduler is per-device. The most visible effect of RrWifiQueueScheduler is on downlink from an AP to several STAs on the same Access Category: it prevents head-of-line blocking of one STA by a large backlog to another.
- Control and management frames always preempt data frames regardless of the scheduler in use.

- The wifi MAC queue scheduler is consulted for single-user transmissions. DL OFDMA / UL OFDMA scheduling is performed by the separate Multi-User Scheduler (see the `SetMultiUserScheduler` snippet above).

Selection of the Access Category (AC)

Since ns-3.26, the `QosTag` is no longer used to assign a user priority to an MSDU. Instead, the selection of the Access Category (AC) for an MSDU is based on the value of the DS field in the IP header of the packet (ToS field in case of IPv4, Traffic Class field in case of IPv6). Details on how to set the ToS field of IPv4 packets are given in the *ToS (Type of Service)* section of the documentation. In summary, users can create an address of type `ns3::InetSocketAddress` with the desired type of service value and pass it to the application helpers:

```
InetSocketAddress destAddress(ipv4Address, udpPort);
OnOffHelper onoff("ns3::UdpSocketFactory", destAddress);
onoff.SetAttribute("Tos", UintegerValue(tos));
```

Mapping the values of the DS field onto user priorities is performed similarly to the Linux `mac80211` subsystem. Basically, the `ns3::WifiNetDevice::SelectQueue()` method sets the user priority (UP) of an MSDU to the three most significant bits of the DS field. The Access Category is then determined based on the user priority according to the following table:

UP	Access Category
7	AC_VO
6	AC_VO
5	AC_VI
4	AC_VI
3	AC_BE
0	AC_BE
2	AC_BK
1	AC_BK

ToS and DSCP values map onto user priorities and access categories according to the following table.

DiffServ PHB	ToS (binary)	UP	Access Category
EF	101110xx	5	AC_VI
AF11	001010xx	1	AC_BK
AF21	010010xx	2	AC_BK
AF31	011010xx	3	AC_BE
AF41	100010xx	4	AC_VI
AF12	001100xx	1	AC_BK
AF22	010100xx	2	AC_BK
AF32	011100xx	3	AC_BE
AF42	100100xx	4	AC_VI
AF13	001110xx	1	AC_BK
AF23	010110xx	2	AC_BK
AF33	011110xx	3	AC_BE
AF43	100110xx	4	AC_VI
CS0	000000xx	0	AC_BE
CS1	001000xx	1	AC_BK
CS2	010000xx	2	AC_BK
CS3	011000xx	3	AC_BE
CS4	100000xx	4	AC_VI
CS5	101000xx	5	AC_VI
CS6	110000xx	6	AC_VO
CS7	111000xx	7	AC_VO

So, for example, a ToS equal to 0xc0 (binary 11000000) will map to CS6, User Priority 6, and Access Category AC_VO. Also, the ns3-wifi-ac-mapping test suite (defined in `src/test/ns3wifi/wifi-ac-mapping-test-suite.cc`) can provide additional useful information.

Note that `ns3::WifiNetDevice::SelectQueue()` also sets the packet priority to the user priority, thus overwriting the value determined by the socket priority (users can read *Use of Send() vs. SendTo()* for details on how to set the packet priority). Also, given that the Traffic Control layer calls `ns3::WifiNetDevice::SelectQueue()` before enqueueing the packet into a queue disc, it turns out that queuing disciplines (such as `PfifoFastQueueDisc`) that classifies packets based on their priority will use the user priority instead of the socket priority.

WifiHelper

We're now ready to create `WifiNetDevices`. First, let's create a `WifiHelper` with default settings:

```
WifiHelper wifiHelper;
```

What does this do? It sets the default wifi standard to **802.11a** and sets the `RemoteStationManager` to `ns3::ArfWifiManager`. You can change the `RemoteStationManager` by calling the `WifiHelper::SetRemoteStationManager` method. To change the wifi standard, call the `WifiHelper::SetStandard` method with the desired standard.

Now, let's use the `wifiPhyHelper` and `wifiMacHelper` created above to install `WifiNetDevices` on a set of nodes in a `NodeContainer` "c":

```
NetDeviceContainer wifiContainer = WifiHelper::Install(wifiPhyHelper, wifiMacHelper, c);
```

This creates the `WifiNetDevice` which includes also a `WifiRemoteStationManager`, a `WifiMac`, and a `WifiPhy` (connected to the matching `Channel`).

The `WifiHelper::SetStandard` method sets various default timing parameters as defined in the selected standard version, overwriting values that may exist or have been previously configured. In order to change parameters that are

overwritten by `WifiHelper::SetStandard`, this should be done post-install using `Config::Set`:

```
WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211n);
wifi.SetRemoteStationManager("ns3::ConstantRateWifiManager", "DataMode", StringValue(
    ↪ "HtMcs7"), "ControlMode", StringValue("HtMcs0"));

//Install PHY and MAC
Ssid ssid = Ssid("ns3-wifi");

WifiMacHelper mac;
mac.SetType("ns3::StaWifiMac",
    "Ssid", SsidValue(ssid),
    "ActiveProbing", BooleanValue(false));

NetDeviceContainer staDevice;
staDevice = wifi.Install(phy, mac, wifiStaNode);

mac.SetType("ns3::ApWifiMac",
    "Ssid", SsidValue(ssid));

NetDeviceContainer apDevice;
apDevice = wifi.Install(phy, mac, wifiApNode);

//Once install is done, we overwrite the standard timing values
Config::Set("/NodeList/*/DeviceList/*/ns3::WifiNetDevice/Phy/Slot",
    ↪ TimeValue(MicroSeconds(slot)));
Config::Set("/NodeList/*/DeviceList/*/ns3::WifiNetDevice/Phy/Sifs",
    ↪ TimeValue(MicroSeconds(sifs)));
Config::Set("/NodeList/*/DeviceList/*/ns3::WifiNetDevice/Phy/Pifs",
    ↪ TimeValue(MicroSeconds(pifs)));
```

The `WifiHelper` can be used to set the attributes of the default ack policy selector (`ConstantWifiAckPolicySelector`) or to select a different (user provided) ack policy selector, for each of the available Access Categories. As an example, the following code can be used to set the `BaThreshold` attribute of the default ack policy selector associated with BE AC to 0.5:

```
WifiHelper wifi;
wifi.SetAckPolicySelectorForAc(AC_BE, "ns3::ConstantWifiAckPolicySelector",
    "BaThreshold", DoubleValue(0.5));
```

The `WifiHelper` is also used to configure OBSS PD spatial reuse for 802.11ax. The following lines configure a `WifiHelper` to support OBSS PD spatial reuse using the `ConstantObssPdAlgorithm` with a threshold set to -72 dBm:

```
WifiHelper wifi;
wifi.SetObssPdAlgorithm("ns3::ConstantObssPdAlgorithm",
    "ObssPdLevel", DoubleValue(-72.0));
```

There are many other *ns-3* attributes that can be set on the above helpers to deviate from the default behavior; the example scripts show how to do some of this reconfiguration.

WifiPhyRxTraceHelper

The `WifiPhyRxTraceHelper` can be used to collect statistics and records of Wi-Fi reception events at the physical layer, such as whether a PPDU was received or was dropped for some reason, and whether a PPDU overlapped in time (collided) with one or more other PPDU's on the channel. Two possible types of studies that may be helped by this feature are:

- 1) Wi-Fi PPDU and MPDU reception outcomes to assess the performance of spatial reuse mechanisms
- 2) Calculation of airtime fairness between contending stations on a network

The trace helper works by hooking PHY-level traces on the applicable nodes, devices, and links and maintaining per-receiver records of all of the Wi-Fi PPDU's that are received, from each receiver's vantage point. These records include a listing of all other PPDU's that may have overlapped in time and/or frequency for a given PPDU, and the received signal power of each PPDU involved, and the outcome of the attempt at reception (whether the PPDU was dropped entirely, or whether the PPDU was partially or fully decoded).

This trace helper collects records of all Wi-Fi PPDU's received on every PHY that is enabled for the helper. As an example, consider the scenario in Figure *WifiPhyRxTraceHelper scenario of interest*, depicting the arrival of three PPDU's that overlap in time. The trace helper keeps a record of each PPDU arrival, on each enabled PHY, and also keeps track of the PPDU's that collided with it, if any. In the figure, the first PPDU overlaps in time with both the second and third PPDU. Likewise, the second PPDU overlaps in time with the first and third PPDU's, and the third PPDU overlaps in time with the first and second. All of these relationships (each frame arrival, the frames that overlapped in time with it), from the perspective of each PHY, are stored in internal data structures. Additionally, the reception outcome of each PPDU, and its constituent MPDU's, is tracked. This database of reception events can be exported by the trace helper for fine-grained tabulation of spatial reuse events of interest, or can be summarized by some built-in statistics printing methods.

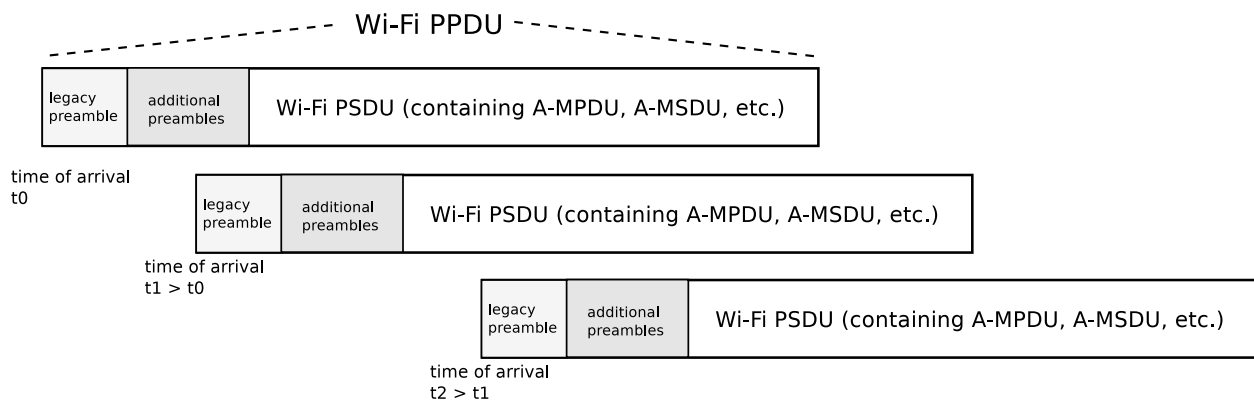


Fig. 35.18: *WifiPhyRxTraceHelper scenario of interest*

The output of this trace helper is provided in three formats:

- 1) Built-in print methods to allow printing of key statistics with a single C++ statement,
- 2) Export of a trace statistics object allowing the user to post-process or format the data according to the user's formatting choice, and
- 3) Export of the complete reception records, allowing the user to perform their own custom processing of the data.

A sample usage of this trace helper is as follows, as demonstrated in the example program `src/wifi/examples/wifi-phy-rx-trace-helper-example.cc`. First, declare an instance of it as one might do with other Wi-Fi helpers:

```
WifiPhyRxTraceHelper rxTraceHelper;
```

You may need to include the trace helper's header if it is not otherwise included by the wifi module header:

```
#include "ns3/wifi-phy-rx-trace-helper.h"
```

Next, enable the trace helper on some subset of the Wi-Fi nodes in the simulation. This step will hook traces on all of the nodes enabled. It is important to include all of the nodes within reception range of the nodes of interest, because PPDU's need to be traced at both transmission and reception times. If you are only interested in statistics or records on a given receiver, you can later (as explained below) request for only the statistics of interest. For example, if you are interested in a particular AP, in a particular BSS, and there is an interfering BSS nearby, you will want to enable all of the nodes within interference range of the AP of interest (even if not in the same BSS). So, in general, you will want to Enable() the trace helper on all Wi-Fi nodes in the simulation, unless you want to explore pruning this set for runtime scaling reasons.

In this example, we will enable on all of the Wi-Fi nodes:

```
NodeContainer c;  
c.Create(2);  
  
...  
  
NetDeviceContainer apDevice = wifi.Install(wifiPhy, wifiMac, c.Get(1));  
NetDeviceContainer staDevice = wifi.Install(wifiPhy, wifiMac, c.Get(0));  
  
...  
  
rxTraceHelper.Enable(c);
```

The important thing to note in the above is that Enable() must be called after Wi-Fi devices are installed, because it will hook traces on what has been installed up to that point.

The next configuration aspect is to establish a start and stop time for PPDU reception collection. In the example program, application data is not sent before 1 second, and we want to capture all data PPDU's, so the Start() and Stop() times are set as follows:

```
rxTraceHelper.Start(Seconds(1));  
// The last packet will be sent at time 1 sec. + (numPackets - 1) * interval  
// Configure the stop time to be 1 sec. later than this.  
Time stopTime = Seconds(1) + (numPackets - 1) * interval + Seconds(1);  
rxTraceHelper.Stop(stopTime);
```

The start and stop times can be tuned to whatever collection window is desired. Furthermore, there is a Reset() method that will clear all of the PPDU records and restart collection; this can be used if one wants to periodically sample the statistics in different time intervals.

Next, run the simulation, and either during the run or after the simulation has completed, harvest the statistics. As noted above, there are three main ways to output data. Furthermore, there are C++ method overloads that allow users to narrow the scope of output data.

The first option is to use PrintStatistics(), which will dump some statistics with built-in formatting, such as follows:

```
traceHelper.PrintStatistics();
```

This will result in a number of statistics being printed out. The statistics will be aggregated for all nodes that are enabled. The following is some of the default output of the sample program:

```
*** Print statistics for all nodes using built-in print method:  
Total PPDU's Received: 1
```

(continues on next page)

(continued from previous page)

```
Total Non-Overlapping PPDUs Received: 1
Total Overlapping PPDUs Received: 0
```

```
Successful PPDUs: 1
Failed PPDUs: 0
```

```
Total MPDUs: 1
Total Successful MPDUs: 1
Total Failed MPDUs: 0
```

In the above simple case, there is one PPDU successfully received, and within this PPDU, there was one MPDU which was successfully decoded. No other PPDUs overlapped in time with this PPDU.

Although this is a PHY trace helper, there is one important aspect from the MAC layer that is used to classify PPDUs. When reporting statistics for a given node, a PPDU is counted only if there was at least one MPDU intended for the node. If the MPDU has a MAC address that is either broadcast or belongs to the device (unicast), that PPDU will be counted. If, instead, a receiver locks onto and overhears a PPDU that ultimately will be discarded because the receiver was not an intended receiver, that PPDU reception or failure will be excluded from the statistics.

The sample program next shows how `PrintStatistics` can take an argument:

```
rxTraceHelper.PrintStatistics(c.Get(0)->GetId());
rxTraceHelper.PrintStatistics(c.Get(1));
```

In the above, the statistics can be limited to a particular node (either passed in by Node ID or by Node pointer). Furthermore, although not shown in the example, the method can be further downscoped to also take a device index argument and (in the case of multi-link operation (MLO)) a specific link ID. In the above example, all devices and all MLO links will be included in the output.

Users can write their own printing methods or further process the statistics by calling `GetStatistics()` as the following example demonstrates:

```
auto stats = rxTraceHelper.GetStatistics();
std::cout << "  numOverlappingPpdu: " << stats.m_numOverlappingPpdu << std::endl;
std::cout << "  numNonOverlappingPpdu: " << stats.m_numNonOverlappingPpdu << std::endl;
std::cout << "  numPpduFailed: " << stats.m_numPpduFailed << std::endl;
std::cout << "  numPpduSuccess: " << stats.m_numPpduSuccess << std::endl;
std::cout << "  numMpduSuccess: " << stats.m_numMpduSuccess << std::endl;
std::cout << "  numMpduFailed: " << stats.m_numMpduFailed << std::endl;
```

Similar to `PrintStatistics()`, `GetStatistics()` with no arguments will collect a statistics output structure covering all nodes, while additional arguments of Node ID, device index, and link ID can be used to constrain the statistics report.

Finally, the output of the internal data structure that tracks PPDU receptions can be exported, with the `GetPpduRecords()` method, which can be called without any arguments, and with arguments of `nodeId`, `deviceId`, and `linkId`:

```
auto optionalRecords = rxTraceHelper.GetPpduRecords(1);
```

The above statements asks for records from receiving node 1 (the AP). The example iterates the returned vector, printing out the size of this structure, as well as some data fields:

```
*** Records vector has size of 3
First record:
```

(continues on next page)

(continued from previous page)

```

first PPDU's RSSI (dBm): -30.6633
first PPDU's receiver ID: 1
first PPDU's sender ID: 0
first PPDU's start time: 1
first PPDU's end time: 1.00008
first PPDU's number of MPDUs: 1
first PPDU's sender device ID: 0
Second record:
second PPDU's RSSI (dBm): -30.6633
second PPDU's receiver ID: 1
second PPDU's sender ID: 0
second PPDU's start time: 1.00027
second PPDU's end time: 1.00032
second PPDU's number of MPDUs: 1
second PPDU's sender device ID: 0
Third record:
third PPDU's RSSI (dBm): -30.6633
third PPDU's receiver ID: 1
third PPDU's sender ID: 0
third PPDU's start time: 1.00039
third PPDU's end time: 1.00072
third PPDU's number of MPDUs: 1
third PPDU's sender device ID: 0

```

That the size of the vector is 3 may be surprising, considering that we have observed above in this example that only one PPDU is being counted in the statistics. However, in this example, there is only one data PPDU but also additional management and control frames. Specifically, the first PPDU is a Block ACK ADDBA_REQUEST frame, the second PPDU is an ACK of the AP's ADDBA_RESPONSE frame, and the third PPDU record corresponds to the actual data PPDU, starting at time 1.00039 and ending at 1.00072. The management and control frames (which also include beacons) are filtered out above by the `GetStatistics()` methods, but when the raw PPDU records are retrieved, all PPDUs received are available and the user is responsible for further filtering as they see fit.

WifiTxStatsHelper

The `WifiTxStatsHelper` is complementary to the `WifiPhyRxTraceHelper`. Where the latter is used to collect statistics of Wi-Fi reception events at the physical layer, the former is used for collecting statistics on the transmit side at the MAC layer, including the number of transmissions and retransmissions of an MPDU, and the total number of successful, re-transmitted, and failed MPDUs. In addition, the helper tracks the duration of time that MPDUs are enqueued, including the durations that they reside within the MAC queue. For failed MPDUs, their timestamps of drop and reasons for being dropped are collected.

The trace helper works by hooking MAC- and PHY-level traces on the applicable nodes, devices, and links (in multi-link operation), and maintaining per-MPDU records of every transmission. We explain the operation by way of first reviewing how a simulation is configured to use the helper, and then reviewing the public API.

A sample usage of this trace helper is as follows, as can be explored by extending an existing Wi-fi example program. First, declare an instance of the helper, as one might do with other Wi-Fi helpers, prior to calling `Simulation::Run()`:

```
WifiTxStatsHelper txStatsHelper;
```

You may need to include the trace helper's header if it is not otherwise included by the wifi module header:

```
#include "ns3/wifi-tx-stats-helper.h"
```

Next, enable the trace helper on some subset of the Wi-Fi nodes in the simulation. This step will hook traces on all of the nodes or devices enabled. Two methods are provided, depending on whether a NodeContainer or NetDeviceContainer is enabled:

```
NodeContainer n;  
txStatsHelper.Enable(n);
```

or

```
NetDeviceContainer d;  
txStatsHelper.Enable(d);
```

By default, statistics will be collected for MPDUs enqueued between the start and stop times, which default to collecting statistics for the whole simulation. To limit the duration, the following methods can be used, as an example:

```
txStatsHelper.Start(Seconds(1));  
txStatsHelper.Stop(Seconds(10));
```

Alternatively, the constructor is overloaded to allow these arguments to be passed there instead:

```
WifiTxStatsHelper txStatsHelper(Seconds(1), Seconds(10));
```

Finally, a Reset() method is provided, to clear the success and failure records. If this method is called, any in-progress MPDUs (i.e., those that have been queued but have neither been positively acked nor reached the maximum number of retries) are kept, but any completed records are deleted.

The following main methods provide the output:

```
CountPerNodeDevice_t GetSuccessesByNodeDevice() const;  
CountPerNodeDevice_t GetFailuresByNodeDevice() const;  
CountPerNodeDevice_t GetRetransmissionsByNodeDevice() const;  
uint64_t GetSuccesses() const;  
uint64_t GetFailures() const;  
uint64_t GetRetransmissions() const;  
Time GetDuration() const;  
const SuccessRecords& GetSuccessRecords(  
    WifiTxStatsHelper::MultiLinkSuccessType type = FIRST_LINK_IN_SET) const;  
const FailureRecords& GetFailureRecords() const;
```

For Multi-Link Operation (MLO) configurations, the records of successful MPDU transmissions are also available on a more granular, per-link basis:

```
const SuccessRecords& GetSuccessRecords(  
    WifiTxStatsHelper::MultiLinkSuccessType type = FIRST_LINK_IN_SET) const;
```

MPDU transmission failures can also be filtered on the drop reason type with the following methods:

```
CountPerNodeDevice_t GetFailuresByNodeDevice(WifiMacDropReason reason) const;  
uint64_t GetFailures(WifiMacDropReason reason) const;
```

The first three methods return unordered maps with counts of successful MPDU transmissions, failed MPDU transmissions, and retransmitted MPDUs (for eventually successful MPDUs).

1. `GetSuccessesByNodeDevice()`: The returned data structure counts the number of successful MPDU transmissions that were observed, indexed by a {node ID, device ID} tuple. If Multi-Link Operation is in use on the device, an acknowledged MPDU will be counted as one success regardless of how many links it was transmitted on.
 - a. `GetSuccessesByNodeDeviceLink()`: The returned data structure counts the number of successful MPDU transmissions that were observed, indexed by {node ID, device ID, link ID} tuple. The link ID corresponds to the link for which the MPDU was successfully acknowledged. If *type* is set to `FIRST_LINK_IN_SET` (default), then the success is counted on the first link in the MPDU's in-flight link ID set. Otherwise, the success is counted for all links in the MPDU's in-flight link ID set.
2. `GetFailuresByNodeDevice()`: This returned data structure counts the number of failures (maximum retries reached) that were observed, indexed by {node ID, device ID} tuple. Per-link statistics are not available for failures (which may have experienced transmissions on different links).
 - a. `GetFailuresByNodeDevice(WifiMacDropReason reason)`: This overloaded method filters for failure counts that match the provided drop reason.
3. `GetRetransmissionsByNodeDevice()`: This returned data structure counts the number of retransmissions that were observed, indexed by {node ID, device ID} tuple. Per-link statistics are not available for retransmissions. Additionally, retransmissions of MPDUs that were not ultimately successful are not counted.

The second three methods provide aggregated counts across all nodes and devices enabled for the helper. These counts should be the same as if the hashed maps of the first three methods were iterated, and the counts for each entry summed. The `GetFailures()` method has an overloaded version (`uint64_t GetFailures(WifiMacDropReason reason) const;`) to get the count of failed MPDU transmissions due to a given reason. If *type* is set to `FIRST_LINK_IN_SET` (default) for the `GetSuccessRecords` method, then the successful MPDU's record is saved in the `std::list<MpduRecord>` of the first link in the MPDU's in-flight link ID set. Otherwise, the record is saved in the lists of all links in the MPDU's in-flight link ID set.

The `GetDuration()` method will return the duration since the helper was started or last reset.

More detailed statistics can be obtained by the methods `GetSuccessRecords()` and `GetFailureRecords()`. These methods export unordered maps of success and failure records, in the form of `struct MpduRecord`, and organized by node, and device, and also by link-id in the case of success records (link-id is not available for failure records). Each `MpduRecord` tracks the time of enqueue, the time of first transmission, the ack time, and the dequeue time (a dequeue occurs when the MPDU is either positively acked or is discarded due to too many retransmissions). The node ID, device ID, MPDU sequence number, and TID are recorded, and the number of retransmissions (if any) are counted. Finally, for successful MPDUs, the `m_successLinkId` field tracks the link ID on which the MPDU was ultimately acknowledged. To obtain a complete accounting of all MPDUs tracked by the helper, both `GetSuccessRecords()` and `GetFailureRecords()` must be queried; all MPDU results will end up in one of the two data structures.

The example program `src/wifi/examples/wifi-bianchi.cc` provides an example use of this helper, by setting the program option `--useTxHelper` to true.

WifiCoTraceHelper

The `WifiCoTraceHelper` (channel occupancy trace helper) can be used to collect statistics of Wi-Fi channel occupancy, as observed from the perspective of the `WifiPhy` state of devices or links (in the case of multi-link devices). The `WifiPhyStateHelper` tracks the state of the `WifiPhy` corresponding to whether it is in idle, transmitting, receiving, or some other state. This helper object can be added to ns-3 Wi-Fi simulations to track the duration and percentage of time spent in each state.

Wi-Fi channel access relies also on additional busy states that conceptually reside in the MAC layer, corresponding to the Network Allocation Vector (NAV). The idle states reported herein correspond to the PHY (physical) carrier sense state and not the MAC (virtual) carrier sense state.

In case of an EMLSR device, PHY objects keep switching among links. This helper aggregates a PHY duration to the link it is operating on at that instant. Be aware that switching PHY objects for an EMLSR device might result in unexpected statistics. For example, if auxiliary PHY is configured to not switch in EMLSR configurations then a link could have no PHY object operating on it intermittently. As a result, the total recorded duration on all links would not be same which does not occur for non-EMLSR devices.

This helper can be added to a simulation program with statements such as the following:

```
WifiCoTraceHelper coHelper{Seconds(1.0), Seconds(11.0)};
```

The above statement, if placed just prior to the call to `Simulator::Run()`, will declare a helper object and configure it to collect statistics between one and eleven seconds. However, `WifiNetDevices` must still be added, such as the following sample code:

```
coHelper.Enable(nodeOrDeviceContainer);
```

Then finally, print the results after `Simulator::Run()` returns:

```
coHelper.PrintStatistics(std::cout);
```

This will print output such as:

```
---- COT for AP:0 ----
Showing duration by states:
IDLE:      +5.69s  (56.93%)
CCA_BUSY:  +1.18s  (11.80%)
TX:        +301.90ms (3.02%)
RX:        +2.83s  (28.25%)

---- COT for STA0:0 ----
Showing duration by states:
IDLE:      +5.71s  (57.06%)
CCA_BUSY: +377.52ms (3.78%)
TX:        +1.63s  (16.31%)
RX:        +2.29s  (22.85%)
```

You can export statistics from this helper for use cases such as printing in your own formatted statements:

```
auto records = coHelper.GetDeviceRecords();
```

You can refer an example program in `src/wifi/examples/wifi-co-trace-example.cc` on how to use these APIs.

WifiStaticSetupHelper

Wi-Fi devices normally exchange management frames over the air to perform association or setup various features. In many simulations, users are not interested in the actual exchange of such management frames because they want to evaluate the performance of the network when the requested features are already established. In such cases, actually exchanging management frames not only constitutes a waste of computing resources, but it could also lead to unexpected results due to collisions among the required management frames.

The purpose of the `WifiStaticSetupHelper` is to statically setup wifi devices at the exact time the simulation starts without actually exchanging management frames over the air. Currently, the following operations can be statically setup:

- legacy association/ML setup (note that channel scanning is automatically disabled by the helper)

- establishment of block ack agreements
- enabling EMLSR mode on EMLSR client links

The `WifiStaticSetupHelper` is made of various static functions to perform the operations above, which can be called once the installation and configuration of the wifi is completed. For example, the following line of code:

```
WifiStaticSetupHelper::SetStaticAssociation(apDev, staDevices);
```

performs association or ML setup (depending on whether involved devices are single-link or multi-link) between the AP `apDev` and the non-AP STAs included in the `staDevices` container of devices. Additionally, the following line of code:

```
WifiStaticSetupHelper::SetStaticBlockAck(apDev, staDev, 0);
```

establishes a Block Ack agreement for TID 0 between the AP `apDev` and the non-AP STA `staDev`, with the AP being the originator and the non-AP STA the recipient.

To enable EMLSR mode on the links specified via the `EmlsrManager::EmlsrLinkSet` attribute, the following line of code can be used:

```
WifiStaticSetupHelper::SetStaticEmlsr(apDev, staDev);
```

where `apDev` is the AP device and `staDev` is the EMLSR client device.

In case wifi devices exchange IPv4 or IPv6 traffic, it is also convenient to statically setup their ARP or NDISC cache, so that ARP Request/Response frames do not need to be actually exchanged. This can be achieved by calling:

```
NeighborCacheHelper nbCache;  
nbCache.PopulateNeighborCache();
```

after assigning IP addresses to the devices in the network.

For an example usage of these static helpers, you can refer to the `/examples/wireless/wifi-{ht,vht,he,eh}-network.cc` example programs.

HT configuration

HT is an acronym for High Throughput, a term synonymous with the IEEE 802.11n standard. Once the `ns3::WifiHelper::Install` has been called and the user sets the standard to a variant that supports HT capabilities (802.11n, 802.11ac, or 802.11ax), an HT configuration object will automatically be created for the device. The configuration object is used to store and manage HT-specific attributes.

802.11n/ac PHY layer can use either long (800 ns) or short (400 ns) OFDM guard intervals. To configure this parameter for a given device, the following lines of code could be used (in this example, it enables the support of a short guard interval for the first station):

```
Ptr<NetDevice> nd = wifiStaDevices.Get(0);  
Ptr<WifiNetDevice> wnd = nd->GetObject<WifiNetDevice>();  
Ptr<HtConfiguration> htConfiguration = wnd->GetHtConfiguration();  
htConfiguration->SetShortGuardIntervalSupported(true);
```

It is also possible to configure HT-specific attributes using `Config::Set`. The following line of code enables the support of a short guard interval for all stations:

```
Config::Set("/NodeList/*/DeviceList/*/$ns3::WifiNetDevice/HtConfiguration/  
↪ShortGuardIntervalSupported", BooleanValue(true));
```


VHT configuration

IEEE 802.11ac devices are also known as supporting Very High Throughput (VHT). Once the `ns3::WifiHelper::Install` has been called and either the 802.11ac or 802.11ax 5 GHz standards are configured, a VHT configuration object will be automatically created to manage VHT-specific attributes.

As of ns-3.29, however, there are no VHT-specific configuration items to manage; therefore, this object is a placeholder for future growth.

HE configuration

IEEE 802.11ax is also known as High Efficiency (HE). Once the `ns3::WifiHelper::Install` has been called and IEEE 802.11ax configured as the standard, an HE configuration object will automatically be created to manage HE-specific attributes for 802.11ax devices.

802.11ax PHY layer can use either 3200 ns, 1600 ns or 800 ns OFDM guard intervals. To configure this parameter, the following lines of code could be used (in this example, it enables the support of 1600 ns guard interval), such as in this example code snippet:

```
Ptr<NetDevice> nd = wifiStaDevices.Get(0);
Ptr<WifiNetDevice> wnd = nd->GetObject<WifiNetDevice>();
Ptr<HeConfiguration> heConfiguration = wnd->GetHeConfiguration();
heConfiguration->SetGuardInterval(NanoSeconds(1600));
```

802.11ax allows compressed Block ACKs containing a 256-bits bitmap, making possible transmissions of A-MPDUs containing up to 256 MPDUs, depending on the negotiated buffer size. In order to configure the buffer size of an 802.11ax device, the following line of code could be used:

```
WifiMacHelper mac;
mac.SetType("ns3::StaWifiMac", "MpduBufferSize", UIntegerValue(256));
```

For transmitting large MPDUs, it might also be needed to increase the maximum aggregation size (see above).

When using UL MU transmissions, solicited TB PPDU can arrive at the AP with a different delay, due to the different propagation delay from the various stations. In real systems, late TB PPDU cause a variable amount of interference depending on the receiver's sensitivity. This phenomenon can be modeled through the `ns3::HeConfiguration::MaxTbPpduDelay` attribute, which defines the maximum delay with which a TB PPDU can arrive with respect to the first TB PPDU in order to be decoded properly. TB PPDU arriving after more than `MaxTbPpduDelay` since the first TB PPDU are discarded and considered as interference.

Mobility configuration

Finally, a mobility model must be configured on each node with Wi-Fi device. Mobility model is used for calculating propagation loss and propagation delay. Two examples are provided in the next section. Users are referred to the chapter on [Mobility](#) module for detailed information.

Example configuration

We provide two typical examples of how a user might configure a Wi-Fi network – one example with an ad-hoc network and one example with an infrastructure network. The two examples were modified from the two examples in the `examples/wireless` folder (`wifi-simple-adhoc.cc` and `wifi-simple-infra.cc`). Users are encouraged to see examples in the `examples/wireless` folder.

AdHoc WifiNetDevice configuration

In this example, we create two ad-hoc nodes equipped with 802.11a Wi-Fi devices. We use the `ns3::ConstantSpeedPropagationDelayModel` as the propagation delay model and `ns3::LogDistancePropagationLossModel` with the exponent of 3.0 as the propagation loss model. Both devices are configured with `ConstantRateWifiManager` at the fixed rate of 12Mbps. Finally, we manually place them by using the `ns3::ListPositionAllocator`:

```
std::string phyMode("OfdmRate12Mbps");

NodeContainer c;
c.Create(2);

WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211a);

YansWifiPhyHelper wifiPhy = YansWifiPhyHelper::Default();
// ns-3 supports RadioTap and Prism tracing extensions for 802.11
wifiPhy.SetPcapDataLinkType(WifiPhyHelper::DLT_IEEE802_11_RADIO);

YansWifiChannelHelper wifiChannel;
wifiChannel.SetPropagationDelay("ns3::ConstantSpeedPropagationDelayModel");
wifiChannel.AddPropagationLoss("ns3::LogDistancePropagationLossModel",
                              "Exponent", DoubleValue(3.0));
wifiPhy.SetChannel(wifiChannel.Create());

// Add a non-QoS upper mac, and disable rate control (i.e. ConstantRateWifiManager)
WifiMacHelper wifiMac;
wifi.SetRemoteStationManager("ns3::ConstantRateWifiManager",
                             "DataMode",StringValue(phyMode),
                             "ControlMode",StringValue(phyMode));

// Set it to adhoc mode
wifiMac.SetType("ns3::AdhocWifiMac");
NetDeviceContainer devices = wifi.Install(wifiPhy, wifiMac, c);

// Configure mobility
MobilityHelper mobility;
Ptr<ListPositionAllocator> positionAlloc = CreateObject<ListPositionAllocator>();
positionAlloc->Add(Vector(0.0, 0.0, 0.0));
positionAlloc->Add(Vector(5.0, 0.0, 0.0));
mobility.SetPositionAllocator(positionAlloc);
mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
mobility.Install(c);

// other set up (e.g. InternetStack, Application)
```

Infrastructure (access point and clients) WifiNetDevice configuration

This is a typical example of how a user might configure an access point and a set of clients. In this example, we create one access point and two clients. Each node is equipped with 802.11b Wi-Fi device:

```
std::string phyMode("DsssRate1Mbps");

NodeContainer ap;
ap.Create(1);
NodeContainer stas;
stas.Create(2);

WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211b);

YansWifiPhyHelper wifiPhy = YansWifiPhyHelper::Default();
// ns-3 supports RadioTap and Prism tracing extensions for 802.11
wifiPhy.SetPcapDataLinkType(WifiPhyHelper::DLT_IEEE802_11_RADIO);

YansWifiChannelHelper wifiChannel;
// reference loss must be changed since 802.11b is operating at 2.4GHz
wifiChannel.SetPropagationDelay("ns3::ConstantSpeedPropagationDelayModel");
wifiChannel.AddPropagationLoss("ns3::LogDistancePropagationLossModel",
                              "Exponent", DoubleValue(3.0),
                              "ReferenceLoss", DoubleValue(40.0459));
wifiPhy.SetChannel(wifiChannel.Create());

// Add a non-QoS upper mac, and disable rate control
WifiMacHelper wifiMac;
wifi.SetRemoteStationManager("ns3::ConstantRateWifiManager",
                             "DataMode",StringValue(phyMode),
                             "ControlMode",StringValue(phyMode));

// Setup the rest of the upper mac
Ssid ssid = Ssid("wifi-default");

// setup AP.
wifiMac.SetType("ns3::ApWifiMac",
               "Ssid", SsidValue(ssid));
NetDeviceContainer apDevice = wifi.Install(wifiPhy, wifiMac, ap);
NetDeviceContainer devices = apDevice;

// setup STAs.
wifiMac.SetType("ns3::StaWifiMac",
               "Ssid", SsidValue(ssid),
               "ActiveProbing", BooleanValue(false));
NetDeviceContainer staDevices = wifi.Install(wifiPhy, wifiMac, stas);
devices.Add(staDevices);

// Configure mobility
MobilityHelper mobility;
Ptr<ListPositionAllocator> positionAlloc = CreateObject<ListPositionAllocator>();
positionAlloc->Add(Vector(0.0, 0.0, 0.0));
```

(continues on next page)

(continued from previous page)

```

positionAlloc->Add(Vector(5.0, 0.0, 0.0));
positionAlloc->Add(Vector(0.0, 5.0, 0.0));
mobility.SetPositionAllocator(positionAlloc);
mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
mobility.Install(ap);
mobility.Install(sta);

// other set up (e.g. InternetStack, Application)

```

Multiple RF interfaces configuration

```

NodeContainer ap;
ap.Create(1);
NodeContainer sta;
sta.Create(1);

WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211be);

// Create multiple spectrum channels
Ptr<MultiModelSpectrumChannel> spectrumChannel2_4Ghz =
    CreateObject<MultiModelSpectrumChannel>();
Ptr<MultiModelSpectrumChannel> spectrumChannel5Ghz =
    CreateObject<MultiModelSpectrumChannel>();
Ptr<MultiModelSpectrumChannel> spectrumChannel6Ghz =
    CreateObject<MultiModelSpectrumChannel>();

// optional: set up propagation loss model separately for each spectrum channel

// SpectrumWifiPhyHelper (3 links)
SpectrumWifiPhyHelper phy(3);
phy.SetPcapDataLinkType(WifiPhyHelper::DLT_IEEE802_11_RADIO);
phy.SetPcapCaptureType(WifiPhyHelper::PcapCaptureType::PCAP_PER_LINK);
phy.AddChannel(spectrumChannel2_4Ghz, WIFI_SPECTRUM_2_4_GHZ);
phy.AddChannel(spectrumChannel5Ghz, WIFI_SPECTRUM_5_GHZ);
phy.AddChannel(spectrumChannel6Ghz, WIFI_SPECTRUM_6_GHZ);

// configure operating channel for each link
phy.Set(0, "ChannelSettings", StringValue("{42, 0, BAND_2_4GHZ, 0}"));
phy.Set(1, "ChannelSettings", StringValue("{42, 0, BAND_5GHZ, 0}"));
phy.Set(2, "ChannelSettings", StringValue("{215, 0, BAND_6GHZ, 0}"));

// configure rate manager for each link
wifi.SetRemoteStationManager(0,
    "ns3::ConstantRateWifiManager",
    "DataMode", StringValue("EhtMcs11"),
    "ControlMode", StringValue("ErpOfdmRate24Mbps"));
wifi.SetRemoteStationManager(1,
    "ns3::ConstantRateWifiManager",
    "DataMode", StringValue("EhtMcs9"),

```

(continues on next page)

(continued from previous page)

```

        "ControlMode", StringValue("OfdmRate24Mbps"));
wifi.SetRemoteStationManager(2,
    "ns3::ConstantRateWifiManager",
    "DataMode", StringValue("EhtMcs7"),
    "ControlMode", StringValue("HeMcs4"));

// setup AP.
wifiMac.SetType("ns3::ApWifiMac",
    "Ssid", SsidValue(ssid));
NetDeviceContainer apDevice = wifi.Install(wifiPhy, wifiMac, ap);
NetDeviceContainer devices = apDevice;

// setup STA.
wifiMac.SetType("ns3::StaWifiMac",
    "Ssid", SsidValue(ssid),
    "ActiveProbing", BooleanValue(false));
NetDeviceContainer staDevice = wifi.Install(wifiPhy, wifiMac, sta);
devices.Add(staDevice);

// Configure mobility
MobilityHelper mobility;
Ptr<ListPositionAllocator> positionAlloc = CreateObject<ListPositionAllocator>();
positionAlloc->Add(Vector(0.0, 0.0, 0.0));
positionAlloc->Add(Vector(5.0, 0.0, 0.0));
mobility.SetPositionAllocator(positionAlloc);
mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
mobility.Install(c);

// other set up (e.g. InternetStack, Application)

```

EMLSR configuration

DISCLAIMER EMLSR support is still experimental. Simulations with EMLSR activated may crash unexpectedly.

Proper EMLSR operations require the configuration of multiple RF interfaces, as illustrated above. Therefore, in order to configure EMLSR operations, we can start from the configuration shown above and apply the changes described in the following. Firstly, we need to activate EMLSR on both the AP MLD and the EMLSR client. This can be done by configuring the appropriate EhtConfiguration attribute through the wifi helper:

```

WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211be);
wifi.ConfigEhtOptions("EmlsrActivated", BooleanValue(true));

```

The EhtConfiguration class also provides attributes for the various EMLSR parameters that an AP MLD that supports EMLSR has to advertise. Given that these attributes are simply ignored by non-AP MLDs, we can use the same wifi helper to configure such parameters:

```

WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211be);
wifi.ConfigEhtOptions("EmlsrActivated", BooleanValue(true));
wifi.ConfigEhtOptions("TransitionTimeout", TimeValue(MicroSeconds(1024)));

```

(continues on next page)

(continued from previous page)

```
wifi.ConfigEhtOptions("MediumSyncDuration", TimeValue(MicroSeconds(3200)));
wifi.ConfigEhtOptions("MsdOfdmEdThreshold", IntegerValue(-72);
wifi.ConfigEhtOptions("MsdMaxNTxops", UIntegerValue(0)); // unlimited attempts
```

EMLSR parameters that are client specific can be configured through the attributes of the EMLSR Manager class and subclass. The WifiMacHelper class provides the SetEmlsrManager method to conveniently set the type of EMLSR Manager and its attributes:

```
// setup STA.
wifiMac.SetType("ns3::StaWifiMac",
               "Ssid", SsidValue(ssid),
               "ActiveProbing", BooleanValue(false));
wifiMac.SetEmlsrManager("ns3::DefaultEmlsrManager",
                       "EmlsrLinkSet",
                       StringValue("0,1,2"), // enable EMLSR on all EMLSR links
                       "MainPhyId",
                       UIntegerValue(1),
                       "EmlsrPaddingDelay",
                       TimeValue(MicroSeconds(32)),
                       "EmlsrTransitionDelay",
                       TimeValue(MicroSeconds(128)),
                       "SwitchAuxPhy",
                       BooleanValue(false),
                       "AuxPhyChannelWidth",
                       UIntegerValue(40));
NetDeviceContainer staDevice = wifi.Install(wifiPhy, wifiMac, sta);
devices.Add(staDevice);
```

Note that the channel switch delay should be less than the transition delay:

```
// SpectrumWifiPhyHelper (3 links)
SpectrumWifiPhyHelper phy(3);
phy.SetPcapDataLinkType(WifiPhyHelper::DLT_IEEE802_11_RADIO);
phy.SetPcapCaptureType(WifiPhyHelper::PcapCaptureType::PCAP_PER_LINK);
phy.AddChannel(spectrumChannel2_4Ghz, WIFI_SPECTRUM_2_4_GHZ);
phy.AddChannel(spectrumChannel5Ghz, WIFI_SPECTRUM_5_GHZ);
phy.AddChannel(spectrumChannel6Ghz, WIFI_SPECTRUM_6_GHZ);
phy.Set("ChannelSwitchDelay", TimeValue(MicroSeconds(100)));
```

35.3 Testing Documentation

At present, most of the available documentation about testing and validation exists in publications, some of which are referenced below.

35.3.1 Error model

Validation results for the 802.11b error model are available in this [technical report](#)

Two clarifications on the results should be noted. First, Figure 1-4 of the above reference corresponds to the *ns-3* NIST BER model. In the program in the Appendix of the paper (80211b.c), there are two constants used to generate the data. The first, packet size, is set to 1024 bytes. The second, “noise”, is set to a value of 7 dB; this was empirically picked to align the curves the best with the reported data from the CMU testbed. Although a value of 1.55 dB would correspond to the reported -99 dBm noise floor from the CMU paper, a noise figure of 7 dB results in the best fit with the CMU experimental data. This default of 7 dB is the `RxNoiseFigure` in the `ns3::YansWifiPhy` model. Other values for noise figure will shift the curves leftward or rightward but not change the slope.

The curves can be reproduced by running the `wifi-clear-channel-cmu.cc` example program in the `examples/wireless` directory, and the figure produced (when GNU Scientific Library (GSL) is enabled) is reproduced below in Figure *Clear channel (AWGN) error model for 802.11b*.

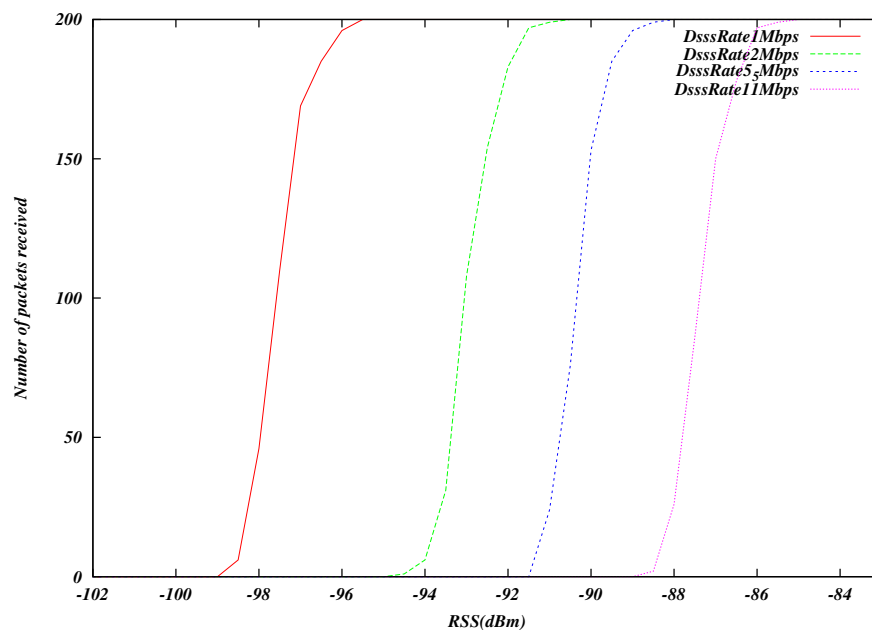


Fig. 35.19: Clear channel (AWGN) error model for 802.11b

Validation results for the 802.11a/g OFDM error model are available in this [technical report](#). The curves can be reproduced by running the `wifi-ofdm-validation.cc` example program in the `examples/wireless` directory, and the figure is reproduced below in Figure *Frame error rate (NIST model) for 802.11a/g (OFDM) Wi-Fi*.

Similar curves for 802.11n/ac/ax can be obtained by running the `wifi-ofdm-ht-validation.cc`, `wifi-ofdm-vht-validation.cc` and `wifi-ofdm-he-validation.cc` example programs in the `examples/wireless` directory, and the figures are reproduced below in Figure *Frame error rate (NIST model) for 802.11n (HT OFDM) Wi-Fi*, Figure *Frame error rate (NIST model) for 802.11ac (VHT OFDM) Wi-Fi* and Figure *Frame error rate (NIST model) for 802.11ax (HE OFDM) Wi-Fi*, respectively. There is no validation for those curves yet.

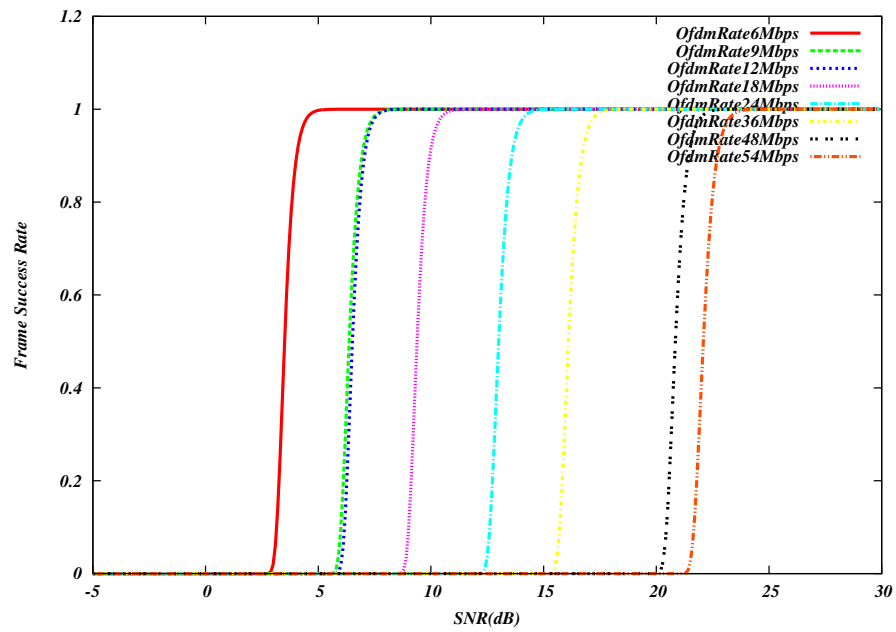


Fig. 35.20: Frame error rate (NIST model) for 802.11a/g (OFDM) Wi-Fi

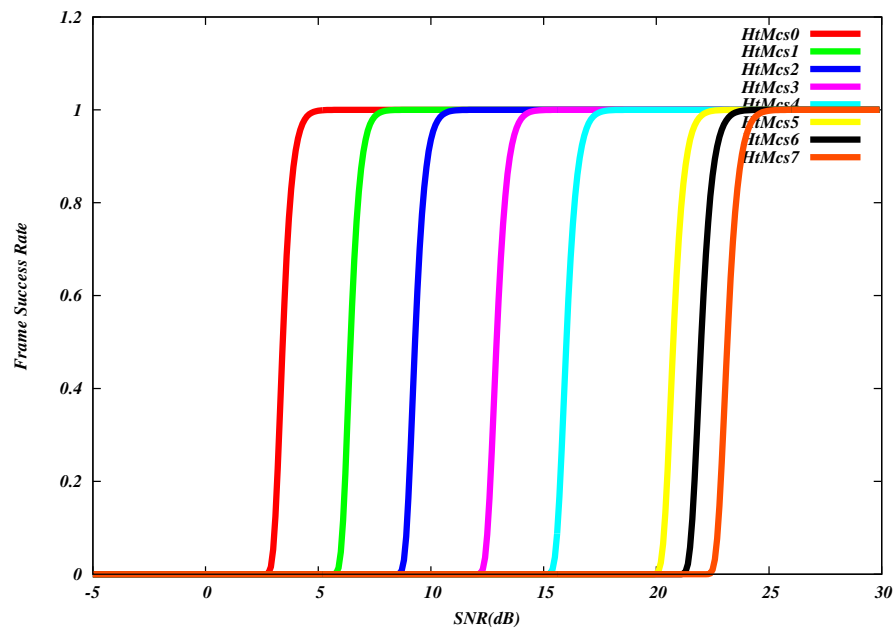


Fig. 35.21: Frame error rate (NIST model) for 802.11n (HT OFDM) Wi-Fi

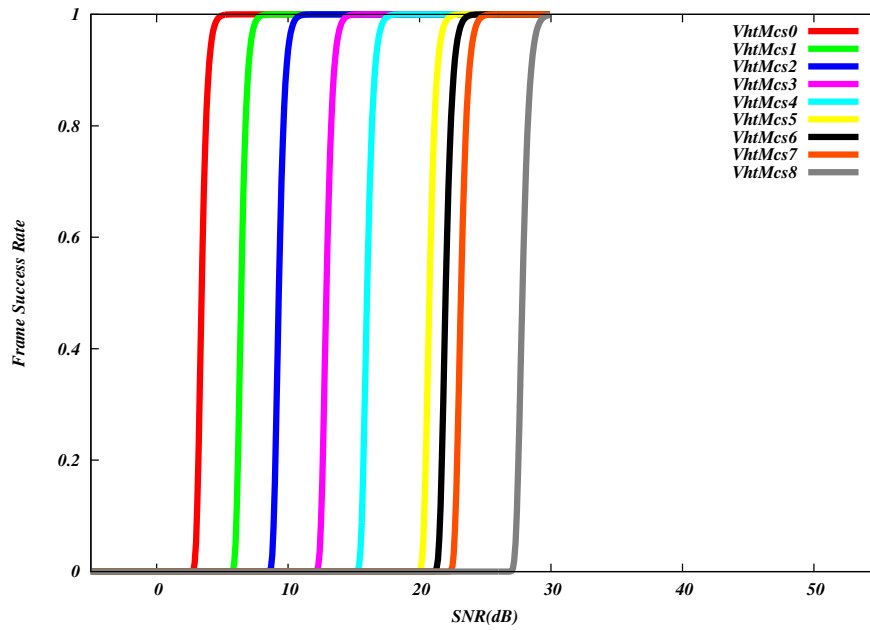


Fig. 35.22: Frame error rate (NIST model) for 802.11ac (VHT OFDM) Wi-Fi

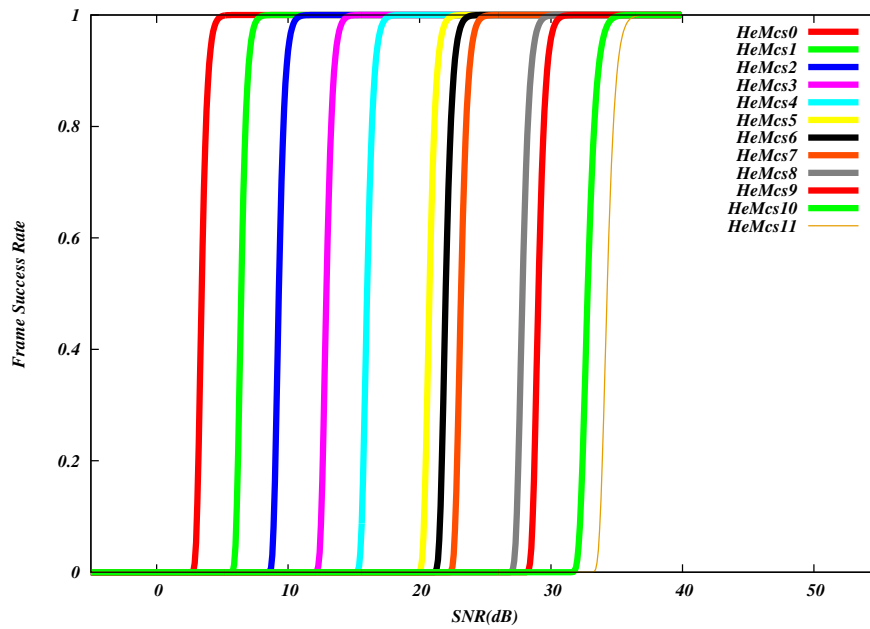


Fig. 35.23: Frame error rate (NIST model) for 802.11ax (HE OFDM) Wi-Fi

35.3.2 MAC validation

Validation of the 802.11 DCF MAC layer has been performed in [baldo2010].

802.11 PCF operation has been verified by running 'wifi-pcf' example with PCAP files generation enabled, and observing the frame exchange using Wireshark.

35.3.3 SpectrumWifiPhy

The SpectrumWifiPhy implementation has been verified to produce equivalent results to the legacy YansWifiPhy by using the saturation and packet error rate programs (described below) and toggling the implementation between the two physical layers.

A basic unit test is provided using injection of hand-crafted packets to a receiving Phy object, controlling the timing and receive power of each packet arrival and checking the reception results. However, most of the testing of this Phy implementation has been performed using example programs described below, and during the course of a (separate) LTE/Wi-Fi coexistence study not documented herein.

Saturation performance

The program `examples/wireless/wifi-spectrum-saturation-example.cc` allows user to select either the *SpectrumWifiPhy* or *YansWifiPhy* for saturation tests. The wifiType can be toggled by the argument '`--wifiType=ns3::YansWifiPhy`' or '`--wifiType=ns3::SpectrumWifiPhy`'

There isn't any difference in the output, which is to be expected because this test is more of a test of the DCF than the physical layer.

By default, the program will use the *SpectrumWifiPhy* and will run for 10 seconds of saturating UDP data, with 802.11n features enabled. It produces this output for the main 802.11n rates (with short and long guard intervals):

```
wifiType: ns3::SpectrumWifiPhy distance: 1m
index  MCS   width Rate (Mb/s) Tput (Mb/s) Received
   0     0    20      6.5    5.81381   4937
   1     1    20      13    11.8266   10043
   2     2    20     19.5   17.7935   15110
   3     3    20      26    23.7958   20207
   4     4    20      39    35.7331   30344
   5     5    20      52    47.6174   40436
   6     6    20     58.5   53.6102   45525
   7     7    20      65    59.5501   50569
   ...
  63    15    40     300   254.902  216459
```

The above output shows the first 8 (of 32) modes, and last mode, that will be output from the program. The first 8 modes correspond to short guard interval disabled and channel bonding disabled. The subsequent 24 modes run by this program are variations with short guard interval enabled (cases 9-16), and then with channel bonding enabled and short guard first disabled then enabled (cases 17-32). Cases 33-64 repeat the same configurations but for two spatial streams (MIMO abstraction).

When run with the legacy *YansWifiPhy*, as in `./ns3 run "wifi-spectrum-saturation-example --wifiType=ns3::YansWifiPhy"`, the same output is observed:

```
wifiType: ns3::YansWifiPhy distance: 1m
index  MCS   width Rate (Mb/s) Tput (Mb/s) Received
```

(continues on next page)

(continued from previous page)

0	0	20	6.5	5.81381	4937
1	1	20	13	11.8266	10043
2	2	20	19.5	17.7935	15110
3	3	20	26	23.7958	20207
...					

This is to be expected since YansWifiPhy and SpectrumWifiPhy use the same error rate model in this case.

Packet error rate performance

The program `examples/wireless/wifi-spectrum-per-example.cc` allows users to select either *SpectrumWifiPhy* or *YansWifiPhy*, as above, and select the distance between the nodes, and to log the reception statistics and received SNR (as observed by the `WifiPhy::MonitorSnifferRx` trace source), using a Friis propagation loss model. The transmit power is lowered from the default of 40 mW (16 dBm) to 1 dBm to lower the baseline SNR; the distance between the nodes can be changed to further change the SNR. By default, it steps through the same index values as in the saturation example (0 through 31) for a 50m distance, for 10 seconds of simulation time, producing output such as:

```
wifiType: ns3::SpectrumWifiPhy distance: 50m; time: 10; TxPower: 1 dBm (1.3 mW)
index  MCS  Rate (Mb/s) Tput (Mb/s) Received Signal (dBm) Noise (dBm) SNR (dB)
  0      0    6.50      5.77    7414      -79.71    -93.97    14.25
  1      1   13.00     11.58   14892      -79.71    -93.97    14.25
  2      2   19.50     17.39   22358      -79.71    -93.97    14.25
  3      3   26.00     22.96   29521      -79.71    -93.97    14.25
  4      4   39.00      0.00      0         N/A       N/A       N/A
  5      5   52.00      0.00      0         N/A       N/A       N/A
  6      6   58.50      0.00      0         N/A       N/A       N/A
  7      7   65.00      0.00      0         N/A       N/A       N/A
```

As in the above saturation example, running this program with *YansWifiPhy* will yield identical output.

Interference performance

The program `examples/wireless/wifi-spectrum-per-interference.cc` is based on the previous packet error rate example, but copies over the `WaveformGenerator` from the unlicensed LTE interferer test, to allow users to inject a non-Wi-Fi signal (using the `--waveformPower` argument) from the command line. Another difference with respect to the packet error rate example program is that the transmit power is set back to the default of 40 mW (16 dBm). By default, the interference generator is off, and the program should behave similarly to the other packet error rate example, but by adding small amounts of power (e.g. `--waveformPower=0.001`), one will start to observe SNR degradation and frame loss.

Some sample output with default arguments (no interference) is:

```
./ns3 run "wifi-spectrum-per-interference"

wifiType: ns3::SpectrumWifiPhy distance: 50m; time: 10; TxPower: 16 dBm (40 mW)
index  MCS  Rate (Mb/s) Tput (Mb/s) Received Signal (dBm) Noi+Inf(dBm) SNR (dB)
  0      0    6.50      5.77    7414      -64.69    -93.97    29.27
  1      1   13.00     11.58   14892      -64.69    -93.97    29.27
  2      2   19.50     17.39   22358      -64.69    -93.97    29.27
  3      3   26.00     23.23   29875      -64.69    -93.97    29.27
  4      4   39.00     34.90   44877      -64.69    -93.97    29.27
```

(continues on next page)

(continued from previous page)

5	5	52.00	46.51	59813	-64.69	-93.97	29.27
6	6	58.50	52.39	67374	-64.69	-93.97	29.27
7	7	65.00	58.18	74819	-64.69	-93.97	29.27
...							

while a small amount of waveform power will cause frame losses to occur at higher order modulations, due to lower SNR:

```
./ns3 run "wifi-spectrum-per-interference --waveformPower=0.001"
```

wifiType: ns3::SpectrumWifiPhy distance: 50m; sent: 1000 TxPower: 16 dBm (40 mW)

index	MCS	Rate (Mb/s)	Tput (Mb/s)	Received Signal (dBm)	Noi+Inf(dBm)	SNR (dB)	
0	0	6.50	5.77	7414	-64.69	-80.08	15.38
1	1	13.00	11.58	14892	-64.69	-80.08	15.38
2	2	19.50	17.39	22358	-64.69	-80.08	15.38
3	3	26.00	23.23	29873	-64.69	-80.08	15.38
4	4	39.00	0.41	531	-64.69	-80.08	15.38
5	5	52.00	0.00	0	N/A	N/A	N/A
6	6	58.50	0.00	0	N/A	N/A	N/A
7	7	65.00	0.00	0	N/A	N/A	N/A
...							

If ns3::YansWifiPhy is selected as the wifiType, the waveform generator will not be enabled because only transmitters of type YansWifiPhy may be connected to a YansWifiChannel.

The interference signal as received by the sending node is typically below the default -62 dBm CCA Mode 1 threshold in this example. If it raises above, the sending node will suppress all transmissions.

35.3.4 Bianchi validation

The program `src/wifi/examples/wifi-bianchi.cc` allows user to compare ns-3 simulation results against the Bianchi model presented in [bianchi2000] and [bianchi2005].

The MATLAB code used to generate the Bianchi model, as well as the generated outputs, are provided in the folder `src/wifi/examples/reference`. User can regenerate Bianchi results by running `generate_bianchi.m` in MATLAB.

By default, the program `src/wifi/examples/wifi-bianchi.cc` simulates an 802.11a adhoc ring scenario, with a PHY rate set to 54 Mbit/s, and loop from 5 stations to 50 stations, by a step of 5 stations. It generates a plt file, which allows user to quickly generate an eps file using gnuplot and visualize the graph.

```
./ns3 run "wifi-bianchi"
```

The user has the possibility to select the standard (only 11a, 11b or 11g currently supported), to select the PHY rate (in Mbit/s), as well as to choose between an adhoc or an infrastructure configuration.

When run for 802.11g 6 Mbit/s in infrastructure mode, the output is:

```
./ns3 run "wifi-bianchi --standard=11g --phyRate=6 --duration=500 --infra"
```

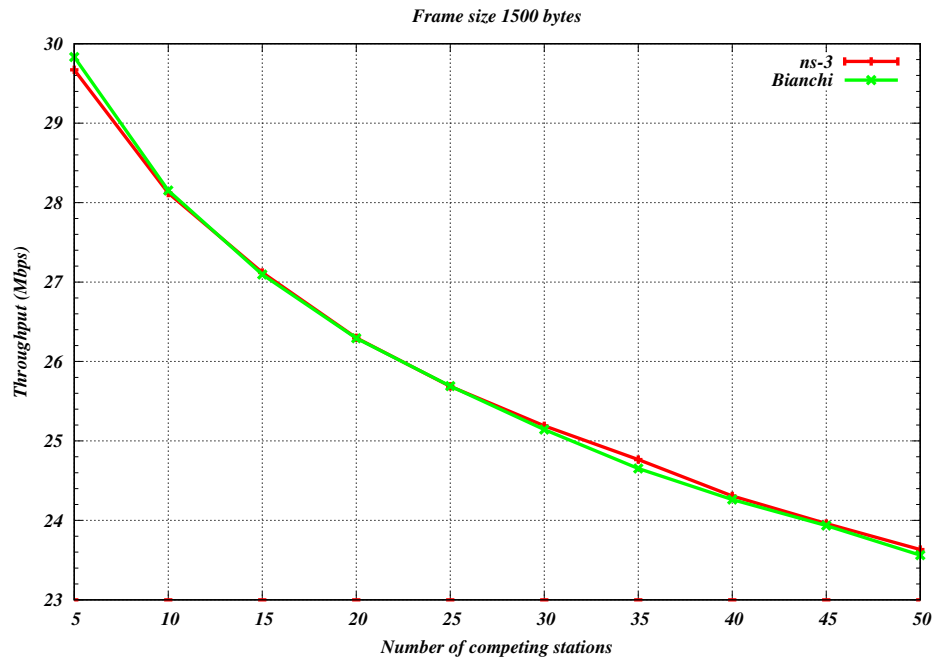


Fig. 35.24: Bianchi throughput validation results for 802.11a 54 Mbps in adhoc configuration

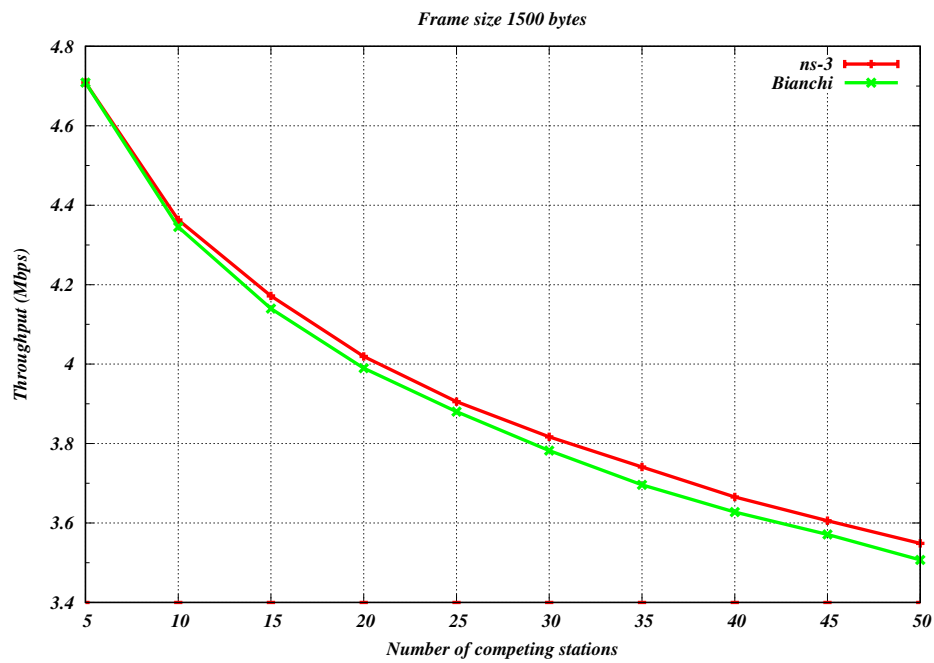


Fig. 35.25: Bianchi throughput validation results for 802.11g 6 Mbps in infrastructure configuration

35.3.5 Multi-user transmissions validation

The implementation of the OFDMA support has been validated against a theoretical model [magrin2021mu] .

A preliminary evaluation of the usage of OFDMA in 802.11ax, in terms of latency in non-saturated conditions, throughput in saturated conditions and transmission range with UL OFDMA, is provided in [avallone2021wcm] .

35.3.6 Channel Occupancy Helper Testing

The Channel Occupancy helper (WifiCoTraceHelper class) has been tested by comparing the occupancy results for single packet transmissions of various sizes (one, two, or three symbols) with the results predicted by offline calculation of the expected values. Additionally, the helper has been validated in saturated traffic conditions as described in the research publication “Use of Channel Occupancy for Multi Link WiFi 7 Scheduler Design in ns-3” to be presented at COMSNETS 25 [kumar2025comsnets] .

35.4 API and behavior changes across releases

This page tracks the most relevant changes to the API and behavior of the wifi module occurred across the various releases of ns-3.

35.4.1 Wifi MAC queue scheduler refactor

As of ns-3.48, the WifiMacQueueSchedulerImpl template base class has absorbed the DropPolicy attribute and the HasToDropBeforeEnqueue implementation that previously lived in FcfsWifiQueueScheduler, so that every subclass (including the new RrWifiQueueScheduler) inherits the same drop behavior. As part of this change:

- The nested enum ns3::FcfsWifiQueueScheduler::DropPolicy has been removed; use ns3::WifiMacQueueSchedulerImpl::DropPolicy instead. The attribute path (.../FcfsWifiQueueScheduler/DropPolicy) still resolves via inheritance and is unchanged for users configuring via Config::Set or the attribute string.
- The free struct ns3::FcfsPrio (and its comparison operators) has been removed. FcfsWifiQueueScheduler now derives from WifiMacQueueSchedulerImpl<WifiSchedPrecedence<Time>>; new subclasses should similarly parameterize the base with WifiSchedPrecedence<...>.

35.4.2 ChannelSettings attribute

Prior to ns-3.36, channels, channel widths, and operating bands were set separately. As of ns-3.36, a new tuple object that we call ChannelSettings has consolidated all of these settings. Users should specify the channel number, channel width, frequency band, and primary channel index as a tuple (and continue to set the Wi-Fi standard separately). Since ns-3.43, this attribute can contain multiple tuples to support 80+80MHz operating channels.

For instance, where pre-ns-3.36 code may have said:

```
WifiPhyHelper phy;  
phy.Set("ChannelNumber", UIntegerValue(36));
```

the equivalent new code is:

```
WifiPhyHelper phy;  
phy.Set("ChannelSettings", StringValue("{36, 20, BAND_5GHZ, 0}"));
```

which denotes that channel 36 is used on a 20 MHz channel in the 5GHz band, and because a larger channel width greater than 20 MHz is not being used, there is no need to indicate the primary 20 MHz channel so it is set to zero in the last argument. Users can read *Channel, frequency, channel width, and band configuration* for more details.

35.5 References

ZIGBEE

This chapter describes the implementation of *ns-3* models for Zigbee Pro stack (also known as Zigbee 3.x) as specified by the Connectivity Standards Alliance (CSA), Zigbee Specification Revision 22 1.0 (2017).

The model described in the present document, represents the uncertified simulated models of Zigbee (TM) technology to be used in *ns-3* for research and educational purposes. The objective is to provide *ns-3* with a non-IP dependent routing alternative to be used with the *ns-3* 1r-wpan (IEEE 802.15.4-2011) module. This module is presented here with no guarantees of any kind and it has no association whatsoever with the CSA or Zigbee (TM) products.

The source code for the new module lives in the directory `src/zigbee`.

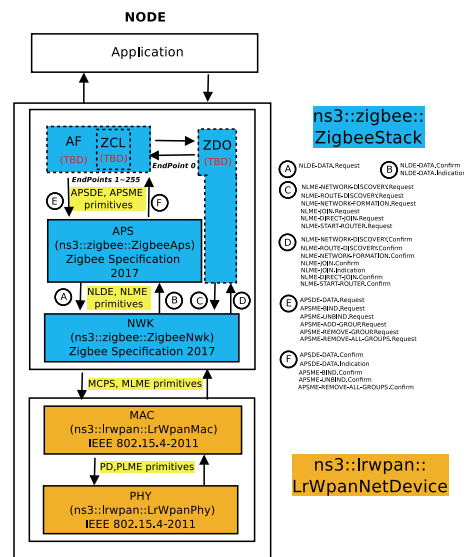


Fig. 36.1: Zigbee Stack Architecture in *ns-3*

The Zigbee stack implementation is designed to operate on top of an existing *ns-3* NetDevice that aggregates an object derived from LrWpanMacBase (which is compliant with IEEE 802.15.4-2011 MAC standards). This is the case of NetDevices such as the LrWpanNetDevice.

While other technologies like 6LoWPAN can interact with the underlying MAC layer through general-purpose NetDevice interfaces, Zigbee has specific requirements that necessitate certain features from a 1r-wpan MAC. Consequently, the *ns-3* Zigbee implementation directly accesses the aggregated LrWpanMacBase and interfaces with it accordingly.

The current scope of the project includes **only the NWK layer and the APS layer in the Zigbee stack**. However, the project can be later extended to support higher layers like the application framework (AF) or services like the Zigbee Cluster Library (ZCL).

36.1 Scope and Limitations

The NWK has the following limitations:

- The NWK MacInterfaceTable NIB is not supported (Multiple MAC interfaces support).
- Security handling is not supported.
- Source routing and Tree routing are not implemented.
- Zigbee Stack profile 0x01 (used for tree routing and distributed address assignment) is not supported.
- A few NIB attributes setting and NWK constants are not supported by the *ns-3* attribute system.
- Trace sources are not implemented yet.
- Data broadcast do not support retries or passive acknowledgment.
- Data broadcast to low power routers is not supported as the underlying lr-wpan netdevice has no concept of energy consumption.
- Address duplication detection is not supported.
- Beacon mode is not throughly tested.
- 16-bit address resolution from a IEEE address at NWK layer is not supported.
- Route Maintenance is not supported.

The APS has the following limitations:

- No binding table support
- No security handling
- No fragmentation support
- No duplicates detection
- No acknowledged transmissions
- No extenended address (64-bit) destinations supported
- No trace sources

The following Zigbee layers or services are not supported yet:

- Zigbee Cluster Library (ZCL)
- Zigbee Device Object (ZDO)
- Application Framework (AF)

To see a list of *ns-3* Zigbee undergoing development efforts check issue [#1165](#)

36.2 The network layer (NWK)

Similar to the implementation of IEEE 802.15.4 (lr-wpan) layers, communication between layers is done via “primitives”. These primitives are implemented in *ns-3* with a combination of functions and callbacks.

The following is a list of NWK primitives supported:

- NLME-NETWORK-DISCOVERY (Request, Confirm)
- NLME-ROUTE-DISCOVERY (Request, Confirm)

- NLME-NETWORK-FORMATION (Request, Confirm)
- NLME-JOIN (Request, Confirm, Indication)
- NLME-DIRECT-JOIN (Request, Confirm)
- NLME-START-ROUTER (Request, Confirm)
- NLDE-DATA (Request, Confirm, Indication)

For details on how to use these primitives, please consult the Zigbee Pro specification.

Typically, users do not have to interact with the Zigbee network layer (NWK) directly. Most commonly, the NWK is used by the Application Support Sub-Layer (APS) instead. In the current implementation of Zigbee in *ns-3* we only support the NWK, for this reason, users wishing to use the routing, network joining and scanning capabilities of Zigbee must interact directly with the NWK.

The following is a brief explanation of how users can interact with the NWK and what are the supported capabilities.

36.2.1 Network Joining

In a Zigbee network, before devices attempt to exchange data with one another, they must first be organized and related to one another based on their specific roles. There are three primary roles in a Zigbee network:

- **Coordinator (ZC):** This is the initiating device in the network, and there can only be one coordinator at a time.
- **Router (ZR):** This device is capable of relaying data and commands on behalf of other devices.
- **End Device (ZE):** These are devices with limited capabilities (for example, they cannot route messages) that serve as endpoints within the network.

Devices must first join a Zigbee network before they can upgrade their roles to either routers or remain as end devices. There are two ways for devices to join a Zigbee network: they can either go through the MAC association process or directly join the network.

MAC Association Join

This method is the default approach used by Zigbee. As the name suggests, it utilizes the underlying MAC layer association mechanism to connect to the Zigbee network. For a comprehensive explanation of the association mechanism occurring in the MAC, please refer to the *lr-wpan* model documentation.

When employing the association mechanism, devices communicate with each other to join an existing network via the network coordinator or a designated router within the network. As a result of this joining process, devices are assigned a short address (16-bit address) that they use for routing and data exchange operations. Below is a summary of the MAC association join process:

1. At least one coordinator must be operational and capable of receiving join requests (i.e. A device successfully completed a NLME-NETWORK-FORMATION.request primitive).
2. Devices must issue a NLME-NETWORK-DISCOVERY.request to look for candidate coordinators/routers to join. The parameters channel numbers (represented by a bitmap for channels 11-26) and scan duration (ranging from 0 to 14) must also be specified to define the channel in which the device will search for a coordinator or router and the duration of that search.
3. If a coordinator (or a capable router) is found on the designated channel and within communication range, it will respond to the device's request with a beacon that contains both a PAN descriptor and a beacon payload describing the capabilities of the coordinator or router.
4. Upon receiving these beacons, the device will select "the most ideal coordinator or router candidate" to join.

5. After selecting the candidate, devices must issue a NLME-JOIN.request primitive with the network parameter set to ASSOCIATION. This will initiate the join process.
6. If the association is successful, the device will receive a confirmation from the coordinator or router, along with a short address that the device will use for its operations within the Zigbee network.
7. Short addresses are assigned by the coordinator randomly, which means there could be instances of address duplication. Although the Zigbee specification includes a mechanism for detecting address duplication, this feature is not currently supported in this implementation.
8. If the association request fails, the device will receive a confirmation with a status indicating failure, rather than SUCCESSFUL, and the short address FF:FF will be received (indicating that the device is not associated).

Note: The process described above outlines the steps for joining the network using a MAC association join. However, devices that are required to act as routers must also issue an additional NLME-START-ROUTER.request primitive after joining the network in order to begin functioning as routers.

In ns-3, Zigbee NWK, coordinators or routers can be found using the following primitive:

```
// zstack is an instance of a ZigbeeStack object installed in the node looking
// for coordinator or routers
NlmeNetworkDiscoveryRequestParams netDiscParams;
netDiscParams.m_scanChannelList.channelPageCount = 1; // only one page structure is
↳ supported
netDiscParams.m_scanChannelList.channelsField[0] = 0x7800; // Bitmap representing
↳ channels 11-14
netDiscParams.m_scanDuration = 14; // (Ranging 0-14) See lr-wpan documentation for time
↳ equivalency
zstack->GetNwk()->NlmeNetworkDiscoveryRequest(netDiscParams);
```

In ns-3 a Zigbee NWK join request (using MAC association) is used as follows:

```
// zstack is an instance of a ZigbeeStack object installed in the node sending
// the join request. The extendedPANId value is typically obtained from the network
↳ descriptor
// received from a previous network discovery request step.
zigbee::CapabilityInformation capaInfo; // define the capabilities and
capaInfo.SetDeviceType(ROUTER); // requirements of the current device
capaInfo.SetAllocateAddrOn(true);

NlmeJoinRequestParams joinParams;
joinParams.m_rejoinNetwork = zigbee::JoiningMethod::ASSOCIATION; // Must be set to
↳ ASSOCIATION
joinParams.m_capabilityInfo = capaInfo.GetCapability(); // Set the capabilities (bitmap)
joinParams.m_extendedPanId = 0xCAFE0000BEEF0000; // The 64 bits representing the
↳ extended PAN ID
zstack->GetNwk()->NlmeJoinRequest(joinParams);
```

See zigbee/examples for detailed examples using network joining.

Direct Join (a.k.a. Orphaning process)

Direct Join is employed when the number of nodes in the network and their relationships are known in advance. As the name suggests, this method involves registering devices directly with a coordinator or router. Subsequently, these devices confirm their registration with the coordinator to receive a network address.

This process is simpler than the association method of joining a network, but it is less flexible since it requires manual intervention. Below is a summary of the direct join process:

1. If a coordinator device or router is already operational, devices that can communicate with this coordinator must be manually registered using the primitive `NLME-DIRECT-JOIN.request`.
2. After this registration, the devices registered in step 1 will send a `NLME-JOIN.request` with the rejoin network parameter set to `DIRECT_OR_REJOIN`. This action triggers a MAC orphaning request message, which is used to confirm the device's existence with the coordinator.
3. The coordinator or router will respond to the orphaning message by providing an assigned short address.
4. The device accepts this short address and successfully joins the network.

Similar to the MAC association mechanism, devices that want to function as routers must issue an additional `NLME-START-ROUTER.request` once the joining process is completed.

In *ns-3*, a direct join primitive is used as follows:

```
CapabilityInformation capaInfo;
capaInfo.SetDeviceType(zigbee::MacDeviceType::ROUTER); // The joining device capability.
↳ is defined here
capaInfo.SetAllocateAddrOn(true); // If false, an extended address mode will be used.
↳ instead
// zstack is an instance of a ZigbeeStack object installed in node that is the
// coordinator or router where the joining device is being registered
NlmeDirectJoinRequestParams directParams;
directParams.m_capabilityInfo = capaInfo.GetCapability();
directParams.m_deviceAddr = Mac64Address("00:00:00:00:00:00:00:01"); // The device IEEE.
↳ address (Ext address)
zstack->GetNwk()->NlmeDirectJoinRequest(directParams);
```

The device joining the network must issue a primitive similar to this one:

```
// zstack is an instance of a ZigbeeStack object installed in the node joining the
↳ network
// The orphaning message will be sent to every channel and interface specified.
// The rejoin network parameter must be DIRECT_OR_REJOIN
NlmeJoinRequestParams joinParams;
joinParams.m_rejoinNetwork = zigbee::JoiningMethod::DIRECT_OR_REJOIN;
joinParams.m_scanChannelList.channelPageCount = 1;
joinParams.m_scanChannelList.channelsField[0] = zigbee::ALL_CHANNELS; // Bitmap.
↳ representing Ch. 11~26
joinParams.m_capabilityInfo = capaInfo.GetCapability();
joinParams.m_extendedPanId = Mac64Address("00:00:00:00:00:00:CA:FE").ConvertToInt();
zstack->GetNwk()->NlmeJoinRequest(joinParams);
```

See `zigbee/examples` for detailed examples using network joining.

36.2.2 Routing

Despite the common belief that the routing protocol contained in Zigbee is AODV, Zigbee only borrows a few basic ideas from this protocol. In fact, Zigbee supports not one but 4 different methods of routing: Mesh routing, Many-To-One routing, Source routing, and Tree routing. From these routing protocols, the current *ns-3* implementation only supports the first two.

Many-To-One Routing

In Zigbee networks, it's common for multiple nodes to need to communicate with a single node, often referred to as a concentrator. If all nodes perform a Mesh route discovery for this single point in the network, it can lead to an overwhelming number of route request messages, which may degrade network performance. Under these circumstances, Many-to-One routing is typically more efficient than Mesh routing.

In Many-to-One routing, one route discovery operation is used to establish a route from all devices to the single concentrator (or collector) node. The general procedure for Many-to-One routing is as follows:

1. The concentrator node broadcast a Many-to-One route request (RREQ) with the concentrator node address set as the target.
2. Devices receiving this broadcast store a reverse route into their routing table pointing to the path towards the concentrator.
3. If we are receiving the RREQ for the first time or the accumulated pathcost is better to the route stored in the routing table, the RREQ is re-broadcasted.

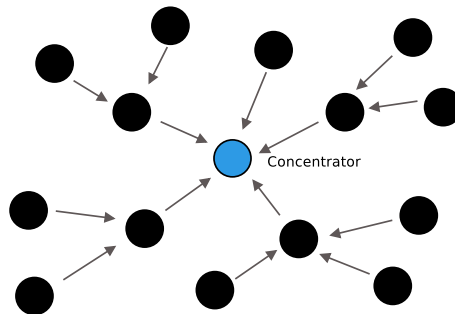


Fig. 36.2: Zigbee Many-To-One routing

There can be multiple concentrator nodes in the network and it is possible to run Many-To-One routing along Mesh routing. In *ns-3*, Many-To-One routing is achieved by using the `NLME-ROUTE-DISCOVERY.request` primitive:

```
// zstack is an instance of a ZigbeeStack object installed in node that will  
// become the concentrator node.  
// The destination address mode must be set to NO_ADDRESS to automatically  
// trigger a Many-To-One route discovery.
```

```
NlmeRouteDiscoveryRequestParams routeDiscParams;  
routeDiscParams.m_dstAddrMode = NO_ADDRESS;  
zstack->GetNwk()->NlmeRouteDiscoveryRequest(routeDiscParams);
```

Note: The process described above assumes that devices have already joined the network. A route discovery request issued before a device is part of the network (join process) will result in failure.

Mesh Routing

Mesh routing in Zigbee is often attributed to the mechanisms used by the AODV routing protocol (:rfc3561). Although Zigbee mesh routing and the AODV protocol share some similarities, there are significant differences between them that directly influence performance.

Firstly, AODV was designed for IP-based networks, whereas Zigbee operates without the concept of IP addresses, thus eliminating their associated overhead. Additionally, unlike AODV, Zigbee employs several distinct tables (such as the RREQ table, routing table, neighbor table, and discovery table) to manage route discovery and storage. The use of these tables allows Zigbee to implement various storage and update policies based on the type of information they hold. For instance, entries for neighboring nodes are typically updated frequently and have a short lifespan. In contrast, entries in routing tables are usually long-lived or do not expire but may be replaced by new entries when space is limited. The RREQ and discovery tables, on the other hand, are used exclusively during the route discovery process to optimize this process and facilitate the early detection of errors and loops.

In addition to these differences, Zigbee incorporates detailed policies, such as packet retransmissions, which come with defined default values to enhance consistency across various Zigbee implementations. For more information on the implementation details and policies that govern Zigbee, please refer to the Zigbee specification.

The primary goal of mesh routing is to establish the shortest path to a specific destination within the network. While hop count is the most commonly used metric for calculating path cost, Zigbee can also utilize the link quality indicator (LQI) to determine the shortest route to the destination (the default option). The general procedure for mesh routing is as follows:

1. The source node broadcasts a route request (RREQ) to its neighboring nodes, specifying the desired destination.
2. Upon receiving the RREQ, the device calculates the link cost and adds it to the path cost already present in the RREQ.
3. The device then searches for a valid route to the destination, first in its neighbor table, followed by its discovery table and routing table.
4. If the destination is found, the device issues a route reply (RREP). If not, the RREQ is re-broadcasted only if it is the first time that a RREQ for that destination is sent from this location, or if a better path cost has been obtained.
5. When the RREQ reaches the destination or a neighboring node that knows the destination, an RREP is sent back to the source, using the information contained in the discovery table.
6. If there is no existing entry in the routing table, a new entry is created with the details of the most optimal path.

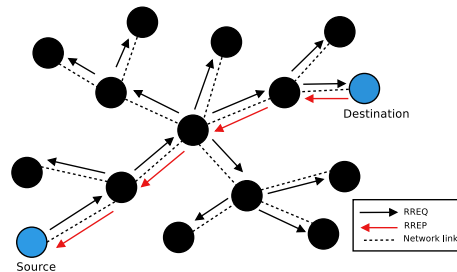


Fig. 36.3: Zigbee Mesh routing

Routing table entries are created only when an RREP is received, and only for the destination specified in the route discovery request. A separate route discovery request must be issued for each destination that needs to be recorded in the source device.

In *ns-3*, Mesh routing is achieved by using the `NLME-ROUTE-DISCOVERY.request` primitive:

```
// zstack is an instance of a ZigbeeStack object installed in node that is the
// source of the route discovery request.
// Only a destination address parameter is necessary to be specified.
// Parameters destination address mode = UCST_BCAST and radius = 0 are the default values.
// and
// do not need be explicitly written.
// Note: As specified by Zigbee, when a radius is set to 0, the radius used is equal to
// nwkMaxDepth(5) * 2.
NlmeRouteDiscoveryRequestParams routeDiscParams;
routeDiscParams.m_dstAddr = Mac16Address("BE:EF");
routeDiscParams.m_dstAddrMode = UCST_BCAST;
routeDiscParams.m_radius = 0;
zstack->GetNwk()->NlmeRouteDiscoveryRequest(routeDiscParams);
```

Alternatively, a Mesh route discovery can be performed along a data transmission request by using the NLDE-DATA request primitive with the ENABLE_ROUTE_DISCOVERY option:

```
// zstack is an instance of a ZigbeeStack object installed in node that is the
// source of the data transmission request.
// The data transmission can be sent with either the ENABLE_ROUTE_DISCOVERY option
// (default) or the
// SUPPRESS_ROUTE_DISCOVERY option.
NldeDataRequestParams dataReqParams;
Ptr<Packet> p = Create<Packet>(5);
dataReqParams.m_dstAddrMode = UCST_BCAST;
dataReqParams.m_dstAddr = Mac16Address("BE:EF");
dataReqParams.m_discoverRoute = ENABLE_ROUTE_DISCOVERY;
zstack->GetNwk()->NldeDataRequest(dataReqParams, p);
```

Note: The process described above assumes that devices have already joined the network. A route discovery request issued before a device is part of the network (join process) will result in failure.

36.3 The application support sub-layer (APS)

As its name suggests, the APS (Application Support) layer acts as an intermediate layer between the Network (NWK) layer and the Application Framework (AF). It provides services to both the Zigbee Device Object (ZDO) and the AF.

The primary functions of the APS layer include:

- Handling security and data integrity through encryption and decryption of data.
- Managing the binding table, which establishes relationships between devices.
- Managing the group table, which facilitates group destinations similar to multicast groups.
- Offering various data transmission modes, such as unicast, broadcast, and groupcast.
- Detecting and filtering duplicate data transmissions.
- Managing the fragmentation and reassembly of data packets.
- Providing a mechanism for data acknowledgment and retransmission.

The APS layer introduces abstract concepts such as **Endpoints**, **Clusters**, and **Profiles**. However, it does not assign specific meanings to these concepts; instead, it simply adds ID information to the transmitted frames.

Effective interpretation and management of **Endpoints**, **Clusters**, and **Profile IDs** require the use of the Zigbee Cluster Library (ZCL) and the ZDO, which are not currently supported in this implementation. Similarly, the APS layer can transmit data that includes information about source and destination endpoints, but it cannot initiate network formation or joining procedures. Typically, these processes are managed by the ZDO; however, since the ZDO is currently unavailable, users must interact directly with the NWK layer's network formation and joining primitives to initiate these options.

This implementation provides only the most basic functions of the APS, enabling data transmission using 16-bit or group address destinations. Features such as transmission to IEEE addresses (64-bit address destinations), fragmentation, duplication detection, and security are not currently supported.

By default, the APS layer is integrated and active within the Zigbee stack. However, users can choose to work solely with the NWK layer. To do this, they must disable the APS by using the `SetNwkLayerOnly()` function in the Zigbee helper.

The following is a list of APS primitives are included:

- APSDE-DATA (Request, Confirm, Indication) : Only Address mode 0x01 and 0x02 (Regular unicast and groupcast) supported.
- APSME-BIND (Request, Confirm)
- APSME-UNBIND (Request, Confirm)
- APSME-ADD-GROUP (Request, Confirm)
- APSME-REMOVE-GROUP (Request, Confirm)
- APSME-REMOVE-ALL-GROUPS (Request, Confirm)

36.3.1 Data Transmission

The APS layer must transmit data using the `APSD-DATA.request` primitive. This primitive supports 4 types of destination address modes:

- **0x00**: No address (Use Binding table to establish destinations) <Not supported>
- **0x01**: Group address (Groupcasting)
- **0x02**: 16-bit address with destination endpoint present (Regular Unicast or Broadcast)
- **0x03**: 64-bit address with destination endpoint not present (Regular Unicast) <Not supported>

GroupCasting (Address Mode: 0x01)

Group casting (Multicasting) is a method of transmitting data to multiple devices endpoints within a specific group. This is particularly useful in scenarios where a device needs to send the same data to multiple endpoints simultaneously, such as in lighting control systems or sensor networks. In Zigbee, group addresses are used to facilitate this type of communication. The APS layer manages group addresses and ensures that data is delivered to all devices in the group. To use group casting, devices must first be added to a group. This is done using the `APSME-ADD-GROUP` primitive, which allows a device to join a group by specifying the group ID.:

```
// Multiple groups can be added to the same device.
// Multiple endpoints can be added to the same group.
// The group address is a 16-bit address.
// In this example, group [BE:EF] and endpoint 5 is added to a device.
```

(continues on next page)

(continued from previous page)

```

ApsmeGroupRequestParams groupParams;
groupParams.m_groupAddress = Mac16Address("BE:EF");
groupParams.m_endPoint = 5;
zstack->GetNwk()->GetAps()->ApsmeAddGroupRequest(groupParams);

```

Once a device is part of a group, any device can send data to ALL the endpoints in the group using the APSDE-DATA.request primitive with the destination address mode GROUP_ADDR_DST_ENDPOINT_NOT_PRESENT (i.e., 0x01 address mode) and the appropriate group address.:

```

// The message will be delivered to ALL the endpoints in any device in the network
// that is part of the group [BE:EF].
ApsdeDataRequestParams dataReqParams;
ZigbeeApsTxOptions txOptions;
dataReqParams.m_useAlias = false;
// Default, use 16 bit address destination (No option), equivalent to 0x00
dataReqParams.m_txOptions = txOptions.GetTxOptions();
dataReqParams.m_srcEndPoint = 4; // Arbitrary value, must not be 0
dataReqParams.m_clusterId = 5; // Arbitrary value (depends on the application)
dataReqParams.m_profileId = 2; // Arbitrary value (depends on the application)
dataReqParams.m_dstAddrMode = ApsDstAddressMode::GROUP_ADDR_DST_ENDPOINT_NOT_PRESENT; //
↳dstAddrMode 0x01
dataReqParams.m_dstAddr16 = Mac16Address("BE:EF"); // The destination group address
zstack->GetNwk()->GetAps()->ApsdeDataRequest(dataReqParams, p);

```

Prior Zigbee Specification Revision 22 1.0 (2017), groupcasting used a form of controlled broadcast to propagate data to all group members in the network. However, the current specification has changed this behavior to a more efficient group addressing scheme. In the new scheme, data transmissions, propagate differently depending on whether or not the receiving or initiating device is member of the group. If the devices are members of the group, they broadcast the data to the nearby devices (like in the old approach), however, if device are not a members of the group, group addresses are treated as regular unicast addresses, and a route discovery is performed to find the closest device which is part of the group. These route are stored in the routing table, and subsequent transmissions to the same group address will use the stored route. In this way, the group address is treated as a unicast address, and the APS layer will not broadcast the data to all devices in the network. This change improves the efficiency of groupcasting by reducing unnecessary broadcasts.

UniCasting (Address Mode: 0x02)

Unicasting is a method of transmitting data to a single specific device endpoint. In Zigbee, this is achieved by using the APSDE-DATA.request primitive with the appropriate destination address mode and address information. To use unicasting, the sender must know the 16-bit address of the destination device and the endpoint it wants to communicate with. The APSDE-DATA.request primitive is used to send data to a specific device endpoint using the address mode 0x02 (Regular Unicast).

The following example demonstrates how to send data to a specific device endpoint using the APSDE-DATA.request primitive with address mode 0x02:

```

// The message will be delivered to the endpoint 5 of the device with address AB:1C
ApsdeDataRequestParams dataReqParams;
ZigbeeApsTxOptions txOptions;
dataReqParams.m_useAlias = false;
// Default, use 16 bit address destination (No option), equivalent to 0x00
dataReqParams.m_txOptions = txOptions.GetTxOptions();

```

(continues on next page)

(continued from previous page)

```

dataReqParams.m_srcEndPoint = 4; // Arbitrary value, must not be 0
dataReqParams.m_clusterId = 5;  // Arbitrary value (depends on the application)
dataReqParams.m_profileId = 2;  // Arbitrary value (depends on the application)
dataReqParams.m_dstAddrMode = ApsDstAddressMode::DST_ADDR16_DST_ENDPOINT_PRESENT; //
↳dstAddrMode 0x02
dataReqParams.m_dstAddr16 = Mac16Address("AB:1C"); // The destination device address
dataReqParams.m_dstEndPoint = 5; // The destination endpoint, must not be 0
zstack->GetNwk()->GetAps()->ApsDataRequest(dataReqParams, p);

```

36.4 Usage

Zigbee is implemented as a *ns-3* module, which means it can be used in the same way as other *ns-3* modules. It requires the *lr-wpan* module to be installed and operational, as it relies on the *lr-wpan* MAC layer to function. Zigbee networks can be created by using the *ZigbeeHelper* class, which simplifies the process of configuring and installing the Zigbee stack but it is also possible to create and configure Zigbee stacks manually.

36.4.1 Helpers

The model includes a *ZigbeeHelper* used to quickly configure and install the NWK layer on top of an existing *Lr-wpan* MAC layer. In essence, *ZigbeeHelper* creates a *ZigbeeNwk* object wraps it in a *ZigbeeStack* and connects this stack to another existing *Lr-wpanNetDevice*. All the necessary callback hooks are taken care of to ensure communication between the Zigbee NWK layer and the *Lr-wpan* MAC layer is possible. The following demonstrates a simple scenario in which a *Lr-wpanNetDevice* devices are set and a *ZigbeeHelper* is used to establish a Zigbee stack on top of these devices:

```

NodeContainer nodes;
nodes.Create(2);

// Create the Lr-wpan (IEEE 802.15.4) Netdevices and install them in each node
LrWpanHelper lrWpanHelper;
NetDeviceContainer lrwpanDevices = lrWpanHelper.Install(nodes);

// Important: NetDevices MAC extended addresses (EUI-64) must always be set
// either manually or using other methods. Short address assignment is not
// required as this is later done by the Zigbee join mechanisms.
Ptr<LrWpanNetDevice> dev0 = lrwpanDevices.Get(0)->GetObject<LrWpanNetDevice>();
Ptr<LrWpanNetDevice> dev1 = lrwpanDevices.Get(1)->GetObject<LrWpanNetDevice>();
dev0->GetMac()->SetExtendedAddress("00:00:00:00:00:00:CA:FE");
dev1->GetMac()->SetExtendedAddress("00:00:00:00:00:00:00:01");

// Install a zigbee stack on a set of devices using the helper
ZigbeeHelper zigbee;
zigbee.SetNwkLayerOnly(); // Activate if you wish to have only the NWK layer in the stack
ZigbeeStackContainer zigbeeStackContainer = zigbee.Install(lrwpanDevices);
// ...
// Call to NWK primitives contained in the NWK layer in the Zigbee stack.

```

36.4.2 Attributes

It is possible to adjust the behavior of the Zigbee module by modifying the values of the available attributes. Some of these attributes are constants and NIB (Network Information Base) attributes as defined in the Zigbee network layer (NWK) specification.

For easy reference, the attributes are named the same as those found in the specification. Constants are values that remain unchanged during the execution of the protocol but can be specified in the configuration files of actual Zigbee devices. In ns-3, both Zigbee constants and NIB attributes are represented as simple attributes.

Below is a list of the currently supported attributes:

- `NwkcCoordinatorCapable`: [Constant] Indicates whether the device is capable of becoming a Zigbee coordinator.
- `NwkcProtocolVersion`: [Constant] The version of the Zigbee NWK protocol in the device (placeholder).
- `NwkcRouteDiscoveryTime`: [Constant] The duration until a route discovery expires.
- `NwkcInitialRREQRetries`: [Constant] The number of times the first broadcast transmission of a RREQ command frame is retried.
- `NwkcRREQRetries`: [Constant] The number of times the broadcast transmission of a RREQ command frame is retried on relay by intermediate router or coordinator.
- `NwkcRREQRetryInterval`: [Constant] The duration between retries of a broadcast RREQ command frame.
- `NwkcMinRREQJitter`: [Constant] The minimum jitter for broadcast retransmission of a RREQ (msec).
- `NwkcMaxRREQJitter`: [Constant] The duration between retries of a broadcast RREQ (msec).
- `MaxPendingTxQueueSize`: The maximum size of the table storing pending packets awaiting to be transmitted after discovering a route to the destination.

36.4.3 Traces

The following trace sources have been implemented:

- `RreqRetriesExhausted`: Trace source indicating when a node has reached the maximum allowed number of RREQ retries during a route discovery request.

36.5 Examples and Tests

The prent capabilities of the Zigbee module implements only the NWK layer. Generally, Zigbee users are not required (or in some situations even allowed) to access the NWK layer API directly. This is because the NWK API is essentially created to provide services to higher Zigbee layers instead (typically the Zigbee APS layer). Nevertheless, direct usage is possible and described by the Zigbee specification. The following examples were created to highlight how a Zigbee NWK layer should be used to achieve network joining, discovery, routing, and data transmission capabilities.

All the examples listed here shows scenarios in which a quasi-layer implementation is used to handle incoming or outgoing events from the NWK layer.

- `zigbee-direct-join.cc`: An example showing the NWK layer join process of devices using the orphaning procedure (Direct join).
- `zigbee-association-join.cc`: An example showing the NWK layer join process of 3 devices in a zigbee network (MAC association).
- `zigbee-nwk-routing.cc`: Shows a simple topology of 5 router devices sequentially joining a network. Data transmission and route discovery (MESH routing) are also shown in this example

- `zigbee-nwk-routing-grid.cc`: Shows a complex grid topology of 50 router devices sequentially joining a network. Route discovery (MANY-TO-ONE routing) is also shown in this example.
- `zigbee-aps-data.cc`: Demonstrates the usage of the APS layer to transmit data (Unicast and Groupcast).)

The following unit test have been developed to ensure the correct behavior of the module:

- `zigbee-rreq-test`: Test some situations in which RREQ messages should be retried during a route discovery process.
- `zigbee-aps-data-test`: Test the APS data transmission

36.6 Validation

No formal validation has been done.

36.7 References

- [1] Zigbee Pro Specification 2017 (R22 1.0)
- [2] DSR Z-boss stack 1.0
- [3] XBee/XBee-PRO® S2C Zigbee® RF Module User Guide
- [4] Farahani, Shahin. ZigBee wireless networks and transceivers. Newnes, 2011.
- [5] Gislason, Drew. Zigbee wireless networking. Newnes, 2008.

BIBLIOGRAPHY

- [TGAX_EVAL] IEEE 802.11-14/0571r12, “11ax Evaluation Methodology,” IEEE 802.11 TGAX.
- [RTA-TIG] IEEE 802.11-18/2009r6, “Real Time Applications TIG Report,” IEEE 802.11 RTA TIG, Section 4.1: Real-time Mobile Gaming.
- [turkmani] Turkmani A.M.D., J.D. Parson and D.G. Lewis, “Radio propagation into buildings at 441, 900 and 1400 MHz”, in Proc. of 4th Int. Conference on Land Mobile Radio, 1987.
- [Rizzo2012] Luigi Rizzo, “netmap: A Novel Framework for Fast Packet I/O”, Proceedings of 2012 USENIX Annual Technical Conference, June 2012.
- [Imputato2019] Pasquale Imputato, Stefano Avallone, Enhancing the fidelity of network emulation through direct access to device buffers, Journal of Network and Computer Applications, Volume 130, 2019, Pages 63-75, (<http://www.sciencedirect.com/science/article/pii/S1084804519300220>)
- [Patel2019] Harsh Patel, Hrishikesh Hiraskar, Mohit P. Tahiliani, “Extending Network Emulation Support in ns-3 using DPDK”, Proceedings of the 2019 Workshop on ns-3, ACM, Pages 17-24, (<https://dl.acm.org/doi/abs/10.1145/3321349.3321358>)
- [TS29274] 3GPP TS 29.274 “GPRS Tunnelling Protocol for Control plane (GTPv2-C)”
- [TS36101] 3GPP TS 36.101 “E-UTRA User Equipment (UE) radio transmission and reception”
- [TS36104] 3GPP TS 36.104 “E-UTRA Base Station (BS) radio transmission and reception”
- [TS36133] 3GPP TS 36.133 “E-UTRA Requirements for support of radio resource management”
- [TS36211] 3GPP TS 36.211 “E-UTRA Physical Channels and Modulation”
- [TS36212] 3GPP TS 36.212 “E-UTRA Multiplexing and channel coding”
- [TS36213] 3GPP TS 36.213 “E-UTRA Physical layer procedures”
- [TS36214] 3GPP TS 36.214 “E-UTRA Physical layer – Measurements”
- [TS36300] 3GPP TS 36.300 “E-UTRA and E-UTRAN; Overall description; Stage 2”
- [TS36304] 3GPP TS 36.304 “E-UTRA User Equipment (UE) procedures in idle mode”
- [TS36321] 3GPP TS 36.321 “E-UTRA Medium Access Control (MAC) protocol specification”
- [TS36322] 3GPP TS 36.322 “E-UTRA Radio Link Control (RLC) protocol specification”
- [TS36323] 3GPP TS 36.323 “E-UTRA Packet Data Convergence Protocol (PDCP) specification”
- [TS36331] 3GPP TS 36.331 “E-UTRA Radio Resource Control (RRC) protocol specification”
- [TS36413] 3GPP TS 36.413 “E-UTRAN S1 application protocol (S1AP)”
- [TS36420] 3GPP TS 36.420 “E-UTRAN X2 general aspects and principles”

- [TS36423] 3GPP TS 36.423 “E-UTRAN X2 application protocol (X2AP)”
- [TR36814] 3GPP TR 36.814 “E-UTRA Further advancements for E-UTRA physical layer aspects”
- [R1-081483] 3GPP R1-081483 “Conveying MCS and TB size via PDCCH”
- [R4-092042] 3GPP R4-092042 “Simulation assumptions and parameters for FDD HeNB RF requirements”
- [FFAPI] FemtoForum “LTE MAC Scheduler Interface Specification v1.11”
- [ns3tutorial] “The ns-3 Tutorial”
- [ns3manual] “The ns-3 Manual”
- [Sesia2009] S. Sesia, I. Toufik and M. Baker, “LTE - The UMTS Long Term Evolution - from theory to practice”, Wiley, 2009
- [Baldo2009] N. Baldo and M. Miozzo, “Spectrum-aware Channel and PHY layer modeling for ns3”, Proceedings of ICST NSTools 2009, Pisa, Italy
- [Holtzman2000] J.M. Holtzman, “CDMA forward link waterfilling power control”, in Proc. of IEEE VTC Spring, 2000.
- [Piro2011] G. Piro, N. Baldo, M. Miozzo, “An LTE module for the ns-3 network simulator”, in Proc. of Wns3 2011 (in conjunction with SimuTOOLS 2011), March 2011, Barcelona (Spain)
- [Seo2004] H. Seo, B. G. Lee. “A proportional-fair power allocation scheme for fair and efficient multiuser OFDM systems”, in Proc. of IEEE GLOBECOM, December 2004, Dallas (USA)
- [Ofcom2600MHz] Ofcom, “Consultation on assessment of future mobile competition and proposals for the award of 800 MHz and 2.6 GHz spectrum and related issues”, March 2011
- [RealWireless] RealWireless, “Low-power shared access to spectrum for mobile broadband”, Final Report, Ofcom Project MC/073, 18th March 2011
- [PaduaPEM] “Ns-developers - LTE error model contribution”
- [ViennaLteSim] “The Vienna LTE Simulators”
- [LozanoCost] Joan Olmos, Silvia Ruiz, Mario García-Lozano and David Martín-Sacristán, “Link Abstraction Models Based on Mutual Information for LTE Downlink”, COST 2100 TD(10)11052 Report
- [wimaxEmd] WiMAX Forum White Paper, “WiMAX System Evaluation Methodology”, July 2008.
- [mathworks] Matlab R2011b Documentation Communications System Toolbox, “Methodology for Simulating Multi-path Fading Channels”
- [CatreuxMIMO] S. Catreux, L.J. Greenstein, V. Erceg, “Some results and insights on the performance gains of MIMO systems,” Selected Areas in Communications, IEEE Journal on , vol.21, no.5, pp. 839- 847, June 2003
- [FujitsuWhitePaper] “Enhancing LTE Cell-Edge Performance via PDCCH ICIC”.
- [Bharucha2011] Z. Bharucha, G. Auer, T. Abe, N. Miki, “Femto-to-Macro Control Channel Interference Mitigation via Cell ID Manipulation in LTE,” Vehicular Technology Conference (VTC Fall), 2011 IEEE , vol., no., pp.1-6, 5-8 Sept. 2011
- [R4-081920] 3GPP R4-081920 “LTE PDCCH/PCFICH Demodulation Performance Results with Implementation Margin”
- [FCapo2012] F.Capozi, G.Piro, L.A. Grieco, G.Boggia, P.Camarda, “Downlink Packet Scheduling in LTE Cellular Networks: Key Design Issues and a Survey”, IEEE Comm. Surveys and Tutorials, vol.2012, no.99, pp.1-23, Jun. 2012
- [FABokhari2009] F.A. Bokhari, H. Yanikomeroglu, W.K. Wong, M. Rahman, “Cross-Layer Resource Scheduling for Video Traffic in the Downlink of OFDMA-Based Wireless 4G Networks”, EURASIP J. Wirel. Commun. Netw., vol.2009, no.3, pp. 1-10, Jan. 2009.

- [WKWong2004] W.K. Wong, H.Y. Tang, V.C.M. Leung, "Token bank fair queuing: a new scheduling algorithm for wireless multimedia services", *Int. J. Commun. Syst.*, vol.17, no.6, pp.591-614, Aug.2004.
- [GMonghal2008] G. Mongha, K.I. Pedersen, I.Z. Kovacs, P.E. Mogensen, "QoS Oriented Time and Frequency Domain Packet Schedulers for The UTRAN Long Term Evolution", In *Proc. IEEE VTC*, 2008.
- [Dimou2009] K. Dimou, M. Wang, Y. Yang, M. Kazmi, A. Larmo, J. Pettersson, W. Muller, Y. Timner, "Handover within 3GPP LTE: Design Principles and Performance", *Vehicular Technology Conference Fall (VTC 2009-Fall)*, 2009 IEEE 70th, pp.1-5, 20-23 Sept. 2009
- [Lee2010] Y.J. Lee, B.J. Shin, J.C. Lim, D.H. Hong, "Effects of time-to-trigger parameter on handover performance in SON-based LTE systems", *Communications (APCC)*, 2010 16th Asia-Pacific Conference on, pp.492-496, Oct. 31 2010–Nov. 3 2010
- [Bbojovic2014] B. Bojovic, N. Baldo, "A new Channel and QoS Aware Scheduler to enhance the capacity of Voice over LTE systems", in *Proceedings of 11th International Multi-Conference on Systems, Signals & Devices (SSD'14)*, Castelldefels, 11-14 February 2014, Castelldefels (Spain).
- [Baldo2014] N. Baldo, R. Martínez, P. Dini, R. Vilalta, M. Miozzo, R. Casellas, R. Muñoz, "A Testbed for Fixed Mobile Convergence Experimentation: ADRENALINE-LENA Integration", in *Proceedings of European Wireless 2014*, 14-16 May 2014, Barcelona (Spain).
- [ASHamza2013] Abdelbaset S. Hamza, Shady S. Khalifa, Haitham S. Hamza, and Khaled Elsayed, "A Survey on Inter-Cell Interference Coordination Techniques in OFDMA-based Cellular Networks", *IEEE Communications Surveys & Tutorials*, March 19, 2013
- [ZXie2009] Zheng Xie, Bernhard Walke, "Enhanced Fractional Frequency Reuse to Increase Capacity of OFDMA Systems", *Proceedings of the 3rd international conference on New technologies, mobility and security, NTMS 2009*
- [DKimura2012] D. Kimura, H. Seki, "Inter-Cell Interference Coordination (ICIC) Technology", *FUJITSU Sci. Tech. J.*, Vol. 48, No. 1 (January 2012)
- [And09] K. Andreev, *Realization of IEEE802.11s draft standard in NS-3*.
- [And10] K. Andreev and P. Boyko, *IEEE 802.11s Mesh Networking NS-3 Model*.
- [Hep15] C. Hepner and A. Witt and R. Muenzner, *Validation of the ns-3 802.11s model and proposed changes compliant to IEEE 802.11-2012*, Poster at 2015 Workshop on ns-3, May 2015.
- [Hep16] C. Hepner and S. Moll and R. Muenzner, *Influence of Processing Delays on the VoIP Performance for IEEE 802.11s Multihop Wireless Mesh Networks: Comparison of ns-3 Network Simulations with Hardware Measurements*, *Proceedings of SIMUTOOLS 16*, August, 2016.
- [ieee80211s] IEEE Standard for Information Technology, Telecommunications and information exchange between systems, Local and metropolitan area networks, Specific requirements, Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) specifications, Amendment 10: Mesh Networking, 10 September 2011.
- [rfc3626] **RFC 3626** *Optimized Link State Routing*
- [rfc7181] **RFC 7181** *The Optimized Link State Routing Protocol Version 2*
- [Baldo2009Spectrum] N. Baldo and M. Miozzo, "Spectrum-aware Channel and PHY layer modeling for ns3", *Proceedings of ICST NSTools 2009*, Pisa, Italy
- [Baron8VSB] Baron, Stanley. "First-Hand:Digital Television: The Digital Terrestrial Television Broadcasting (DTTB) Standard." IEEE Global History Network. <[http://www.ieeeahn.org/wiki/index.php/First-Hand:Digital_Television:_The_Digital_Terrestrial_Television_Broadcasting_\(DTTB\)_Standard](http://www.ieeeahn.org/wiki/index.php/First-Hand:Digital_Television:_The_Digital_Terrestrial_Television_Broadcasting_(DTTB)_Standard)>.
- [KoppCOFDM] Kopp, Carlo. "High Definition Television." High Definition Television. Air Power Australia. <<http://www.ausairpower.net/AC-1100.html>>.

- [MatlabGeo] “Geodetic2ecef.” Convert Geodetic to Geocentric (ECEF) Coordinates. The MathWorks, Inc. <<http://www.mathworks.com/help/map/ref/geodetic2ecef.html>>.
- [QualcommAnalog] Stephen Shellhammer, Ahmed Sadek, and Wenyi Zhang. “Technical Challenges for Cognitive Radio in the TV White Space Spectrum.” Qualcomm Incorporated.
- [TR38901] 3GPP. 2018. TR 38.901. Study on channel for frequencies from 0.5 to 100 GHz. V.15.0.0. (2018-06).
- [NYUSIM] Shihao Ju and Theodore S. Rappaport. “Simulating Motion - Incorporating Spatial Consistency into the NYUSIM Channel Model.” NYU WIRELESS.
- [WIN2D112] IST-4-027756 WINNER II, Deliverable D1.1.2, “WINNER II channel models: Part I - Channel models”, 2007. Available at: <https://signserv.signal.uu.se/Publications/WINNER/WIN2D112.pdf>
- [QUADRIGA] S. Jaeckel, L. Raschkowski, K. Börner, and L. Thiele, “QuaDRiGa: Quasi Deterministic Radio Channel Generator - User Manual and Documentation”, Fraunhofer HHI. Available at: https://quadriga-channel-model.de/wp-content/uploads/2015/02/quadriga_documentation_v1.2.3.pdf
- [Zhang] Menglei Zhang, Michele Polese, Marco Mezzavilla, Sundeep Rangan, Michele Zorzi. “ns-3 Implementation of the 3GPP MIMO Channel Model for Frequency Spectrum above 6 GHz”. In Proceedings of the Workshop on ns-3 (WNS3 ‘17). 2017.
- [Zugno] Tommaso Zugno, Michele Polese, Natale Patriciello, Biljana Bojovic, Sandra Lagen, Michele Zorzi. “Implementation of a Spatial Channel Model for ns-3”. Submitted to the Workshop on ns-3 (WNS3 ‘20). 2020. Available: <https://arxiv.org/abs/2002.09341>
- [TR36897] 3GPP. 2015. TR 36.897. Study on elevation beamforming / Full-Dimension (FD) Multiple Input Multiple Output (MIMO) for LTE. V13.0.0. (2015-06)
- [Pagin2023] Pagin, Matteo, Sandra Lagen, Biljana Bojovic, Michele Polese, Michele Zorzi. 2023. “Improving the Efficiency of MIMO Simulations in ns-3” In Proceedings of the 2023 Workshop on ns-3, pp. 1–9. 2023
- [Zugno2020] Zugno, Tommaso, Michele Polese, Natale Patriciello, Biljana Bojović, Sandra Lagen, Michele Zorzi. “Implementation of a spatial channel model for ns-3.” In Proceedings of the 2020 Workshop on ns-3, pp. 49-56. 2020.
- [Polese2018] Michele Polese, Michele Zorzi. “Impact of channel models on the end-to-end performance of mmwave cellular networks”. In: 2018 IEEE 19th International Workshop on Signal Processing Advances in Wireless Communications (SPAWC).
- [Rebato2018] Rebato, Mattia, Laura Resteghini, Christian Mazzucco, Michele Zorzi. “Study of realistic antenna patterns in 5G mmWave cellular scenarios”. In: 2018 IEEE International Conference on Communications (ICC).
- [Romero] Romero-Jerez, Juan M., F. Javier Lopez-Martinez, José F. Paris, Andrea J. Goldsmith. “The fluctuating two-ray fading model: Statistical characterization and performance analysis”. In: IEEE Transactions on Wireless Communications 16.7 (2017).
- [Kulkarni] Kulkarni, Mandar N., Eugene Visotsky, Jeffrey G. Andrews. “Correction factor for analysis of MIMO wireless networks with highly directional beamforming.”, IEEE Wireless Communications Letters, 2018
- [Asplund] Asplund, Henrik, David Astely, Peter von Butovitsch, Thomas Chapman, Mattias Frenne, Farshid Ghasemzadeh, Måns Hagström et al. Advanced Antenna Systems for 5G Network Deployments: Bridging the Gap Between Theory and Practice. Academic Press, 2020.
- [ns3Zugno] Zugno Tommaso, Michele Polese, Natale Patriciello, Biljana Bojović, Sandra Lagen, Michele Zorzi. “Implementation of a spatial channel model for ns-3.” In Proceedings of the 2020 Workshop on ns-3, pp. 49-56. 2020.
- [3GPPTR38901] 3GPP. 2018. TR 38.901. Study on channel for frequencies from 0.5 to 100 GHz. V.15.0.0. (2018-06).
- [3GPPTR38811] 3GPP. 2018. TR 38.811, Study on New Radio (NR) to support non-terrestrial networks, V15.4.0. (2020-09).

- [Panwar] R. S. Panwar and K. M. Sivalingam, "Implementation of wrap around mechanism for system level simulation of LTE cellular networks in NS3," 2017 IEEE 18th International Symposium on A World of Wireless, Mobile and Multimedia Networks (WoWMoM), Macau, 2017, pp. 1-9, doi: 10.1109/WoW-MoM.2017.7974289
- [Ref1] A. Kuznetsov and D. Torokhov; Linux Cross Reference Source Code; Available online at https://raw.githubusercontent.com/torvalds/linux/8efd0d9c316af470377894a6a0f9ff63ce18c177/net/sched/sch_tbf.c.
- [Ref2] J. Vehent; Journey to the Center of the Linux Kernel: Traffic Control, Shaping and QoS; Available online at https://web.archive.org/web/20190930031243/http://wiki.linuxwall.info/doku.php/en:ressources:dossiers:networking:traffic_control#tbf_-_token_bucket_filter.
- [Ref3] Practical IP Network QoS: TBF queuing discipline; Available online at <https://web.archive.org/web/20200516025221/http://web.opalsoft.net/qos/default.php>.
- [Nic12] K. Nichols and V. Jacobson, Controlling Queue Delay, ACM Queue, Vol. 10 No. 5, May 2012. Available online at <http://queue.acm.org/detail.cfm?id=2209336>.
- [Nic14] K. Nichols and V. Jacobson, Internet-Draft: Controlled Delay Active Queue Management, March 2014. Available online at <https://datatracker.ietf.org/doc/html/draft-nichols-tsvwg-codel-02>.
- [Buf14] Bufferbloat.net. Available online at <http://www.bufferbloat.net/>.
- [Hoe16] T. Hoeiland-Joergensen, P. McKenney, D. Taht, J. Gettys and E. Dumazet, The FlowQueue-CoDel Packet Scheduler and Active Queue Management Algorithm, IETF draft. Available online at <https://tools.ietf.org/html/draft-ietf-aqm-fq-codel>
- [Buf16] Bufferbloat.net. Available online at <http://www.bufferbloat.net/>.
- [Cake16] Linux implementation of Cobalt as a part of the cake framework. Available online at https://github.com/dtaht/sch_cake/blob/master/sch_cake.c.
- [Kath17] Controlled Delay Active Queue Management (draft-ietf-aqm-fq-codel-07) Available online at <https://tools.ietf.org/html/draft-ietf-aqm-codel-07>.
- [BLUE02] Feng, W. C., Shin, K. G., Kandlur, D. D., & Saha, D. (2002). The BLUE Active Queue Management Algorithms. IEEE/ACM Transactions on Networking (ToN), 10(4), 513-528.
- [Cobalt19] Jendaipou Palmei, Shefali Gupta, Pasquale Imputato, Jonathan Morton, Mohit P. Tahiliani, Stefano Avallone and Dave Taht (2019). Design and Evaluation of COBALT Queue Discipline. IEEE International Symposium on Local and Metropolitan Area Networks (LANMAN), July 2019.
- [Pal19] J. Palmei, S. Gupta, P. Imputato, J. Morton, M. Tahiliani, S. Avallone, and D. Taht, Design and Evaluation of COBALT Queue Discipline, 2019 IEEE International Symposium on Local and Metropolitan Area Networks (LANMAN), Paris, France, 2019.
- [Hoe18] T. Hoiland-Jørgensen, D. Taht and J. Morton, "Piece of CAKE: A Comprehensive Queue Management Solution for Home Gateways," 2018 IEEE International Symposium on Local and Metropolitan Area Networks (LANMAN), Washington, DC, USA, 2018.
- [Pan13] Pan, R., Natarajan, P., Piglione, C., Prabhu, M. S., Subramanian, V., Baker, F., & VerSteeg, B. (2013, July). PIE: A lightweight control scheme to address the bufferbloat problem. In High Performance Switching and Routing (HPSR), 2013 IEEE 14th International Conference on (pp. 148-155). IEEE. Available online at <https://www.ietf.org/mail-archive/web/iccr/current/pdfB57AZSheOH.pdf>.
- [Pan16] R. Pan, P. Natarajan, F. Baker, G. White, B. VerSteeg, M.S. Prabhu, C. Piglione, V. Subramanian, Internet-Draft: PIE: A lightweight control scheme to address the bufferbloat problem, April 2016. Available online at <https://tools.ietf.org/html/draft-ietf-aqm-pie-07>.
- [Ram19] G. Ramakrishnan, M. Bhasi, V. Saicharan, L. Monis, S. D. Patil and M. P. Tahiliani, "FQ-PIE Queue Discipline in the Linux Kernel: Design, Implementation and Challenges," 2019 IEEE 44th LCN

- Symposium on Emerging Topics in Networking (LCN Symposium), Osnabrueck, Germany, 2019, pp. 117-124,
- [CableLabs14] G. White, Active Queue Management in DOCSIS 3.X Cable Modems, CableLabs white paper, May 2014.
- [ieee80211] IEEE Std 802.11-2012, *Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*
- [ieee80211-2016] IEEE Std 802.11-2016, *Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*
- [pei80211b] G. Pei and Tom Henderson, [Validation of ns-3 802.11b PHY model](#)
- [pei80211ofdm] G. Pei and Tom Henderson, [Validation of OFDM error rate model in ns-3](#)
- [lacage2006yans] M. Lacage and T. Henderson, [Yet another Network Simulator](#)
- [Haccoun] D. Haccoun and G. Begin, *High-Rate Punctured Convolutional Codes for Viterbi Sequential Decoding*, IEEE Transactions on Communications, Vol. 32, Issue 3, pp.315-319.
- [Frenger] Pål Frenger et al., “Multi-rate Convolutional Codes”.
- [ji2004sslswn] Z. Ji, J. Zhou, M. Takai and R. Bagrodia, *Scalable simulation of large-scale wireless networks with bounded inaccuracies*, in Proc. of the Seventh ACM Symposium on Modeling, Analysis and Simulation of Wireless and Mobile Systems, October 2004.
- [linuxminstrel] [minstrel linux wireless](#)
- [lacage2004aarfamrr] M. Lacage, H. Manshaei, and T. Turletti, *IEEE 802.11 rate adaptation: a practical approach*, in Proc. 7th ACM International Symposium on Modeling, Analysis and Simulation of Wireless and Mobile Systems, 2004.
- [kim2006cara] J. Kim, S. Kim, S. Choi, and D. Qiao, *CARA: Collision-Aware Rate Adaptation for IEEE 802.11 WLANs*, in Proc. 25th IEEE International Conference on Computer Communications, 2006
- [wong2006rraa] S. Wong, H. Yang, S. Lu, and V. Bharghavan, *Robust Rate Adaptation for 802.11 Wireless Networks*, in Proc. 12th Annual International Conference on Mobile Computing and Networking, 2006
- [maguolo2008aarfcd] F. Maguolo, M. Lacage, and T. Turletti, *Efficient collision detection for auto rate fallback algorithm*, in IEEE Symposium on Computers and Communications, 2008
- [proakis2001] J. Proakis, Digital Communications, Wiley, 2001.
- [miller2003] L. E. Miller, “Validation of 802.11a/UWB Coexistence Simulation.” Technical Report, October 2003. Available [online](#)
- [ferrari2004] G. Ferrari and G. Corazza, “Tight bounds and accurate approximations for DQPSK transmission bit error rate”, Electronics Letters, 40(20):1284-85, September 2004.
- [pursley2009] M. Pursley and T. Royster, “Properties and performance of the IEEE 802.11b complementary code key signal sets,” IEEE Transactions on Communications, 57(2):440-449, February 2009.
- [akella2007parf] A. Akella, G. Judd, S. Seshan, and P. Steenkiste, ‘Self-management in chaotic wireless deployments’, in Wireless Networks, Kluwer Academic Publishers, 2007, 13, 737-755. <https://web.archive.org/web/20200709172505/https://www.cs.odu.edu/~nadeem/classes/cs795-WNS-S13/papers/enter-006.pdf>
- [chevillat2005aparf] Chevillat, P.; Jelitto, J., and Truong, H. L., ‘Dynamic data rate and transmit power adjustment in IEEE 802.11 wireless LANs’, in International Journal of Wireless Information Networks, Springer, 2005, 12, 123-145. https://web.archive.org/web/20170810111231/http://www.cs.mun.ca/~yzchen/papers/papers/rate_adaptation/80211_dynamic_rate_power_adjustment_chevillat_j2005.pdf

- [hepner2015] C. Hepner, A. Witt, and R. Muenzner, “In depth analysis of the ns-3 physical layer abstraction for WLAN systems and evaluation of its influences on network simulation results”, BW-CAR Symposium on Information and Communication Systems (SInCom) 2015. <https://core.ac.uk/download/pdf/75487102.pdf#page=50>
- [baldo2010] N. Baldo et al., “Validation of the ns-3 IEEE 802.11 model using the EXTREME testbed”, Proceedings of SIMUTools Conference, March 2010.
- [lanante2019] L. Lanante Jr. et al., “Improved Abstraction for Clear Channel Assessment in ns-3 802.11 WLAN Model”, Proceedings of the 2019 Workshop on ns-3, June 2019.
- [bianchi2000] G. Bianchi, “Performance analysis of the IEEE 802.11 distributed coordination function”, IEEE Communications Letters, 18(3):535–547, 2000.
- [bianchi2005] G. Bianchi and I. Tinnirello. “Remarks on IEEE 802.11 DCF performance analysis”, IEEE Communications Letters, 9(8):765–767, 2005.
- [patidar2017] R. Patidar et al., “Link-to-System Mapping for ns-3 Wi-Fi OFDM Error Models”, Proceedings of the Workshop on ns-3, June 2017. <https://dl.acm.org/doi/10.1145/3067665.3067671>
- [erceg2004] V. Erceg and L. Schumacher and P. Kyritsi, “Tgn channel models”, IEEE 802.11-03/940r4, 2004.
- [porat2016] R. Porat et al., “11ax Evaluation Methodology”, IEE P802.11 Wireless LANs, 11-14-0571r3, 2016.
- [krotov2020rate] A. Krotov, A. Kiryanov, E. Khorov., [Rate Control With Spatial Reuse for Wi-Fi 6 Dense Deployments](#), IEEE Access, September 2020
- [magrin2021mu] D. Magrin, S. Avallone, S. Roy, and M. Zorzi, ‘Validation of the ns-3 802.11ax OFDMA implementation’, in Proceedings of WNS3 2021.
- [avallone2021wcm] S. Avallone, P. Imputato, G. Redieteb, C. Ghosh and S. Roy, “Will OFDMA Improve the Performance of 802.11 WiFi Networks?”, in IEEE Wireless Communications Magazine, DOI: 10.1109/MWC.001.2000332, to appear.
- [corbet2012] J. Corbet, “TCP Small Queues”, [LWN.net](#), July 17, 2012
- [grazia2022] C. Grazia, N. Patriciello, T. Hoiland-Jorgensen, M. Klapez and M. Casoni, “Aggregating Without Bloating: Hard Times for TCP on Wi-Fi”, IEEE/ACM Transactions on Networking, Vol. 30, No.5, October 2022.
- [kumar2025comsnets] P. Kumar, J. Kulshrestha, M. Maity and S. Roy, “Use of Channel Occupancy for Multi Link WiFi 7 Scheduler Design in ns-3” 2025 17th International Conference on COMMunication Systems & NETWORKS (COMSNETS), Bengaluru, India, 2025, to appear.

R

RFC

- RFC 1058, 153, 154
- RFC 1723, 153
- RFC 1918, 136
- RFC 2080, 151, 153, 154
- RFC 2131, 213
- RFC 2132, 213
- RFC 2328, 156, 160
- RFC 2453, 151, 153
- RFC 3393, 126
- RFC 3561, 19, 151
- RFC 3626, 151, 513, 729
- RFC 4861, 577, 580
- RFC 4944, 575, 576
- RFC 6282, 575, 576
- RFC 6621, 140
- RFC 6775, 575–577
- RFC 6928, 162
- RFC 7181, 513, 729
- RFC 7973, 579
- RFC 8415, 214, 220
- RFC 8505, 575–578
- RFC 9438, 167